MACHINE LEARNING

AI · ML · DS / REVISION NOTES

COURSERA · DEEPLEARNING.AI & STANFORD ONLINE · ANDREW NG

Supervised Machine Learning: Regression and Classification

Machine Learning Specialization, Course 1 — linear & logistic regression, gradient descent, regularization

42 SECTIONS

463 FIGURES

414 PAGES

Compiled October 2026

github.com/ankitgupta1729/AI-ML-DS

Contents

LECTURE 1 Welcome — Machine Learning 4
LECTURE 2 Machine Learning 5
LECTURE 3 Machine Learning 7
LECTURE 4 Supervised Learning 13
LECTURE 5 Classification: Breast Cancer Detection 20
LECTURE 6 Supervised Learning Unsupervised Learning 28
LECTURE 7 Unsupervised Learning 37
LECTURE 8 Jupyter Notebook 39
LECTURE 9 House Sizes and Prices 41
LECTURE 10 Prediction Y-hat and Linear Function 60
LECTURE 11 Cost Function: Squared Error Cost Function 69
LECTURE 12 Cost Function 76
LECTURE 13 3D Surface Plot 88
LECTURE 14 Sum of Squared and Pair of Values 103
LECTURE 15 Gradient Descent 112
LECTURE 16 Learning Rate 127
LECTURE 17 Gradient Descent Algorithm 135
LECTURE 18 Gradient Descent May Be Slow 148
LECTURE 19 Linear Regression Model Cost Function 169
LECTURE 20 Gradient Descent 179
LECTURE 21 Multiple Features (Variables) 189
LECTURE 22 Parameters and Features Without Vectorization 203
LECTURE 23 Without Vectorization 210
LECTURE 24 Gradient Descent 219
LECTURE 25 Feature and Parameter Values 226
LECTURE 26 Feature Scaling 232
LECTURE 27 Gradient Descent 241
LECTURE 28 Learning Rate 250

LECTURE 29 Feature Engineering 262
LECTURE 30 Polynomial Regression 267
LECTURE 31 Linear Regression and Spam 276
LECTURE 32 Interpretation of Logistic Regression Output 289
LECTURE 33 Decision Boundary 299
LECTURE 34 Squared Error Cost 318
LECTURE 35 Simplified Cost Function 337
LECTURE 36 Gradient Descent 343
LECTURE 37 Regression Example 350
LECTURE 38 Select Features to Include/exclude 363
LECTURE 39 Regularization Simpler Model W320 371
LECTURE 40 Regularized Linear Regression 382
LECTURE 41 Regularized Logistic Regression 398
LECTURE 42 Computer Vision and Walks of Life 406

LECTURE 1

Welcome — Machine Learning

Welcome to Machine learning. What is machine learning? You probably use it many times a day without even knowing it. Anytime you want to find out something like how do I make a sushi roll? You can do a web search on Google, Bing or Baidu to find out. And that works so well because their machine learning software has figured out how to rank web pages. Or when you upload pictures to Instagram or Snapchat and think to yourself, I want to tag my friends so they can see their pictures. Well these apps can recognize your friends in your pictures and label them as well. That's also machine learning. Or if you've just finished watching a Star Wars movie on the video streaming service and you think what other similar movies can I watch?

Well the streaming service will likely use machine learning to recommend something that you might like. Each time you use voice to text on your phone to write a text message. >> Hey Andrew, how's it going? >> Or tell your phone. Hey Siri play a song by Rihanna, or ask your other phone okay Google show me Indian restaurants near me. That's also machine learning. Each time you receive an email titled, Congratulations! You've won a million dollars. Well maybe you're rich, congratulations. Or more likely your email service will probably flag it as spam. That too is an application of machine learning. Beyond consumer applications that you might use, AI is also rapidly making its way into big companies and into industrial applications.

For example, I'm deeply concerned about climate change, and I'm glad to see that machine learning is already hoping to optimize wind turbine power generation. Or in healthcare, is starting to make its way into hospitals to help doctors make accurate diagnosis. Or recently at Landing AI have been doing a lot of work, putting computer vision into factories to help inspect if something coming off the assembly line has any defects. That's machine learning, it's the science of getting computers to learn without being explicitly programmed. In this course, you learn about machine learning and get to implement machine learning and code yourself. Millions of others have taken the earlier version of this course, which is of course, that led to the founding of Coursera.

And many learners ended up building exciting machine learning systems or even pursuing very successful careers in AI. I'm excited that you're on this journey with me. Welcome and let's get started.

LECTURE 2

Machine Learning

In this class, you'll learn about the state of the art and also practice implementing machine learning algorithms yourself. You'll learn about the most important machine learning algorithms, some of which are exactly what's being used in large AI or large tech companies today and you get a sense of what is the state of the art in AI. Beyond learning the algorithms though, in this class, you'll also learn all the important practical tips and tricks for making them perform well. You get to implement them and see how they work for yourself. Why is machine learning so widely used today? Machine Learning had grown up as a sub-field of AI or artificial intelligence.

We wanted to build intelligent machines. It turns out that there are a few basic things that we could program a machine to do, such as how to find the shortest path from a to b, like in your GPS. But for the most part, we just did not know how to write an explicit program to do many of the more interesting things, such as perform web search, recognize human speech, diagnose diseases from X-rays or build a self-driving car. The only way we knew how to do these things was to have a machine learn to do it by itself. For me, when I founded and was leading the Google Brain Team, I worked on problems like speech recognition, computer vision for Google Maps, Street View images and advertising, or leading AI Baidu, I worked on everything from AI for augmented reality to combating payment fraud to leading a self-driving car team.

Most recently, at landing.AI, AI Fund and Stanford University, I'm beginning to work on AI applications in the factory, large-scale agriculture, health care, e-commerce, and other problems. Today, there are hundreds of thousands, perhaps millions of people working on machine learning applications who could tell you similar stories about their work with machine learning. When you've learned these skills, I hope that you too will find the great fun to dabble in exciting different applications and maybe even different industries. In fact, I find it hard to think of any industry that machine learning is unlikely to touch in a significant way now or in the near future.

Looking even further into the future, many people, including me, are excited about the AI dream of someday building machines as intelligent as you or me. This is sometimes called Artificial General Intelligence or AGI. I think AGI has been overhyped and we're still a long way away from that goal. I don't know. It'll take 50 years or 500 years or longer to get there. But mostly AI researchers believe that the best way to get closer toward that goal is by using learning algorithms. Maybe ones that take some inspiration from how the human brain works. You also hear a little more about this Quest for AGI later in this course. According to a study by McKinsey, AI and machine learning is estimated to create an additional 13 trillion US dollars of value annually by the year 2030.

Even though machine learning is already creating tremendous amounts of value in the software industry, I think there could be even vastly greater value that has yet to be created outside the software industry in sectors such as retail, travel, transportation, automotive, materials manufacturing, and so on. Because of the massive untapped opportunities across so many different

sectors, today there is a vast unfulfilled demand for this skill set. That's why this is such a great time to be learning about machine learning. If you find machine learning applications exciting, I hope you stick with me through this class. I can almost guarantee that you'll find mastering these skills worthwhile.

In the next video, we'll look at a more formal definition of what is machine learning. And we'll begin to talk about the main types of machine learning problems and algorithms. You pick up some of the main machine learning terminology and start to get a sense of what are the different algorithms and when each one might be appropriate. So let's go on to the next video.

LECTURE 3

Machine Learning

highlighted text

What is machine learning? In this video, you'll learn the definition of what it is and also get a sense of when you might want to apply it. Let's take a look together. Here's a definition of what is machine learning that is attributed to Arthur Samuel.

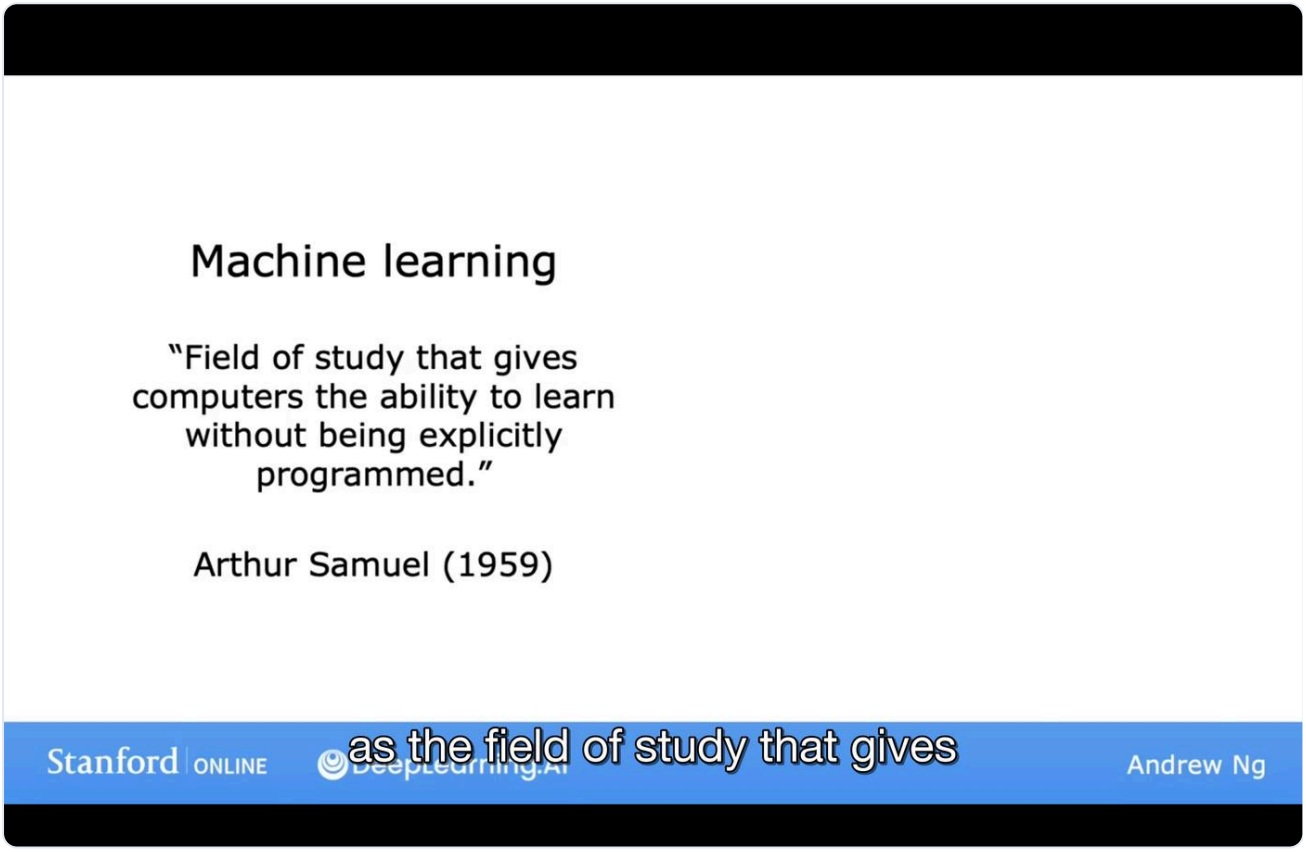


FIGURE 3.1 | Machine Learning

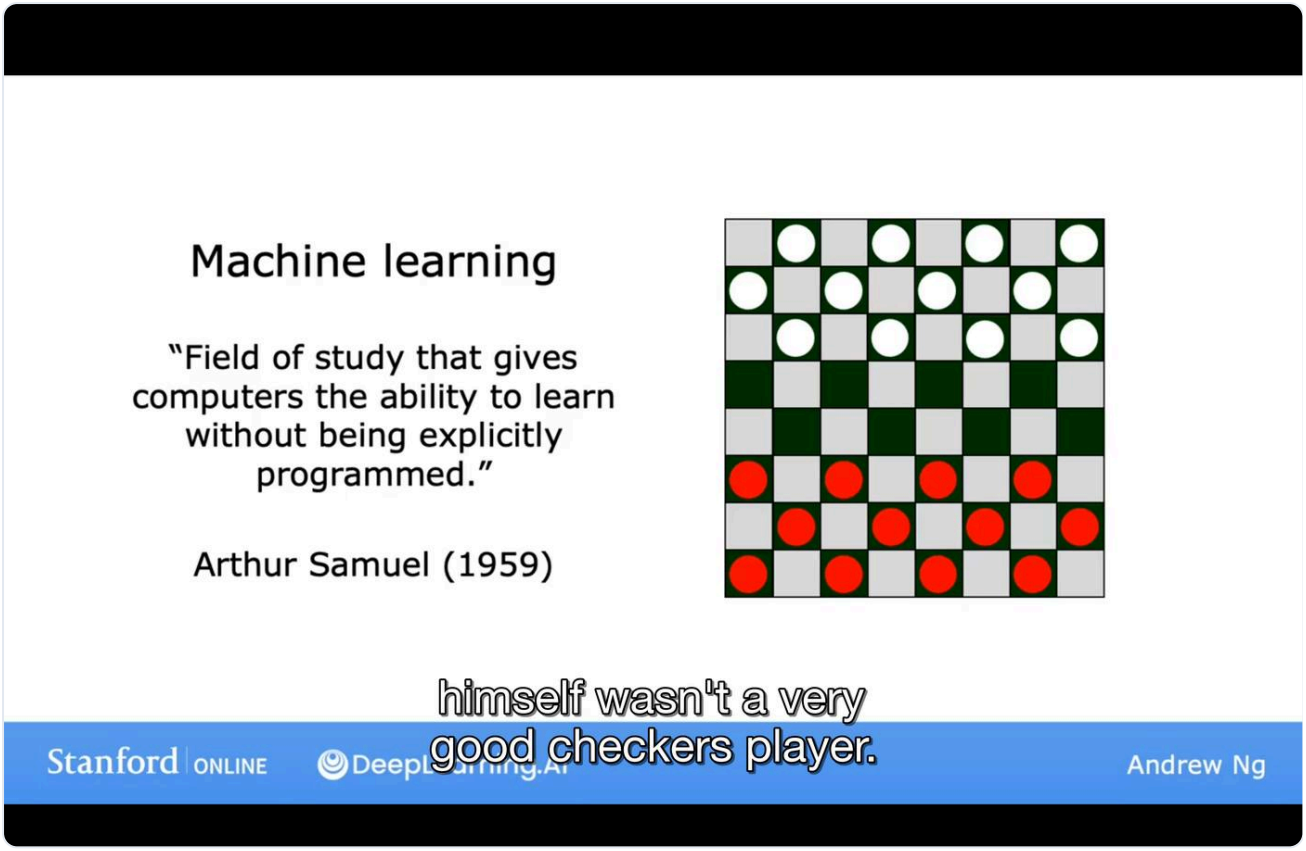


FIGURE 3.2 | Machine Learning

He defined machine learning as the field of study that gives computers the ability to learn without being explicitly programmed. Samuel's claim to fame was that back in the 1950s, he wrote a checkers playing program. The amazing thing about this program was that Arthur Samuel himself wasn't a very good checkers player. What he did was he had programmed the computer to play maybe tens of thousands of games against itself. By watching what social support positions tend to lead to wins and what positions tend to lead to losses the checkers plane program learned over time what are good or bad support positions by trying to get a good and avoid bad positions, this program learned to get better and better at playing checkers because the computer had the patience to play tens of thousands of games against itself.

It was able to get so much checkers playing experience that eventually it became a better checkers player than also, Samuel himself. Now throughout these videos, besides me trying to talk about stuff, I occasionally ask you a question to help make sure you understand the content. Here's one about what happens if the computer had played far fewer games. Please take a look and pick whichever you think is the better answer. Thanks for looking at the quiz.

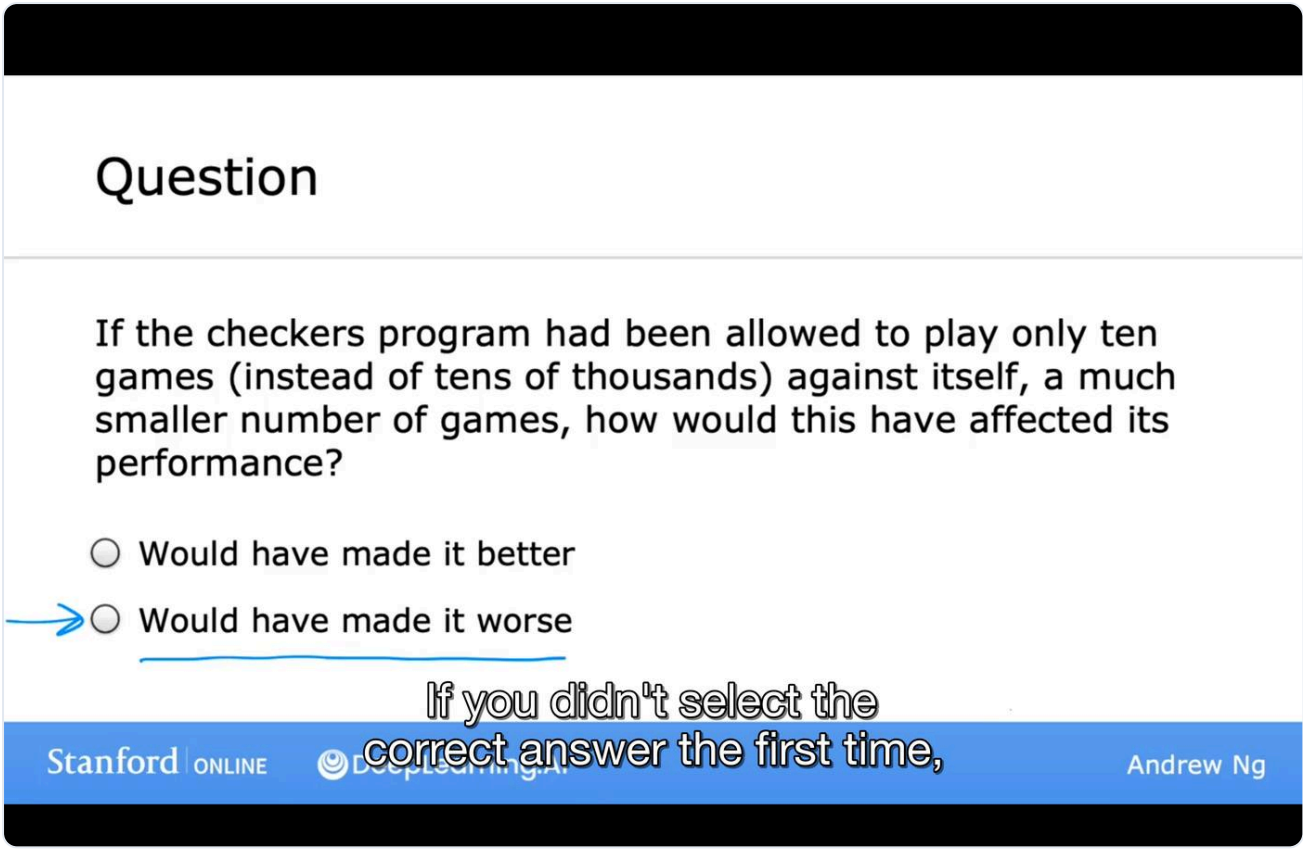


FIGURE 3.3

If you had selected this answer would have made it worse then you got the right. In general, the more opportunities you give a learning algorithm to learn, the better it will perform. If you didn't select the correct answer the first time, that's totally okay too. The point of these questions isn't to see if you can get them all correctly on the first try. These questions are here just to help you practice the concepts you are learning. Arthur Samuel's definition was a rather informal one but in the next two videos, we'll dive deeper together into what are the major types of machine learning algorithms?

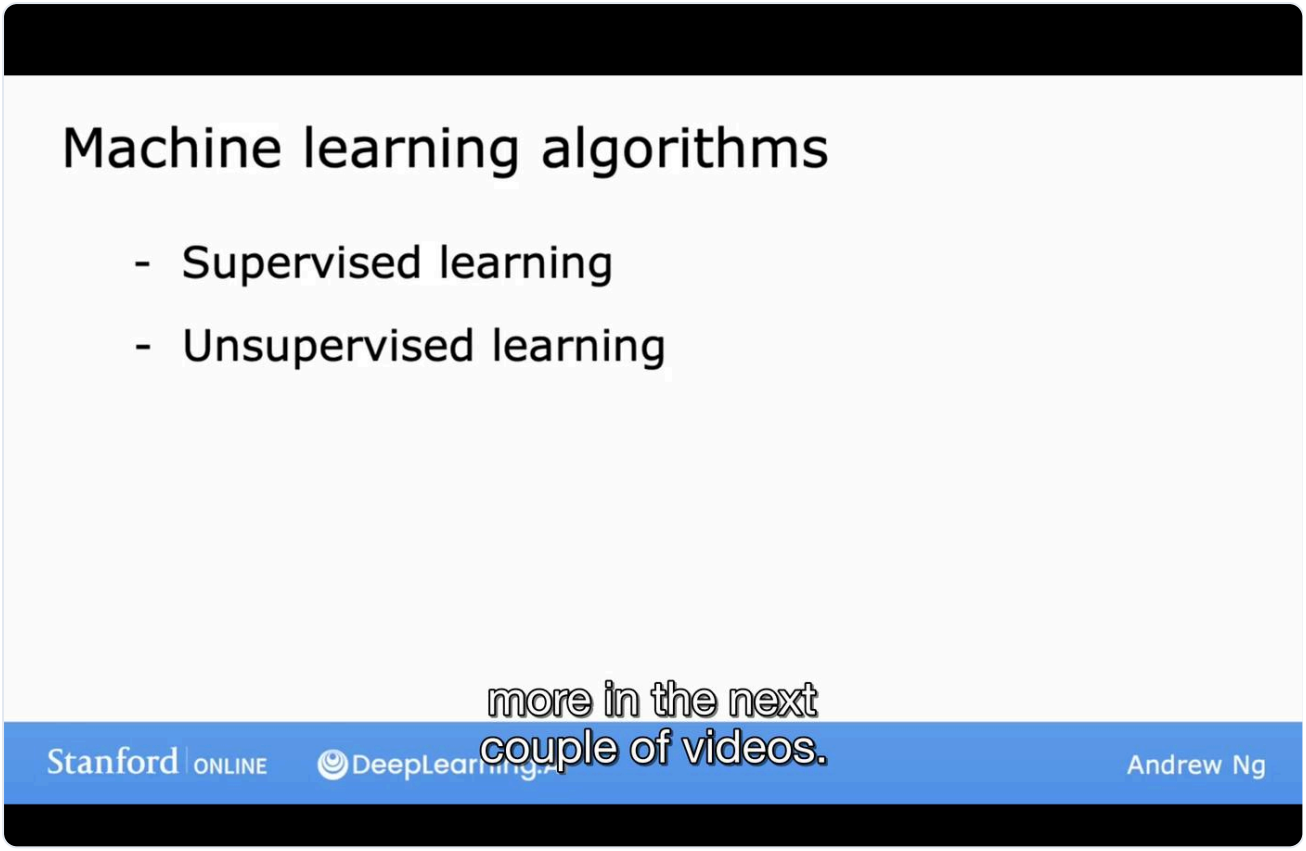


FIGURE 3.4 | Machine Learning Algorithms

In this course, you learn about many different learning algorithms. The two main types of machine learning are supervised learning and unsupervised learning. We'll define what these terms mean more in the next couple of videos. Of these two, supervised learning is the type of machine learning that is used most in many real-world applications and has seen the most rapid advancements and innovation. In this specialization, which has three courses in total, the first and second courses will focus on supervised learning, and the third will focus on unsupervised learning, recommender systems, and reinforcement learning.

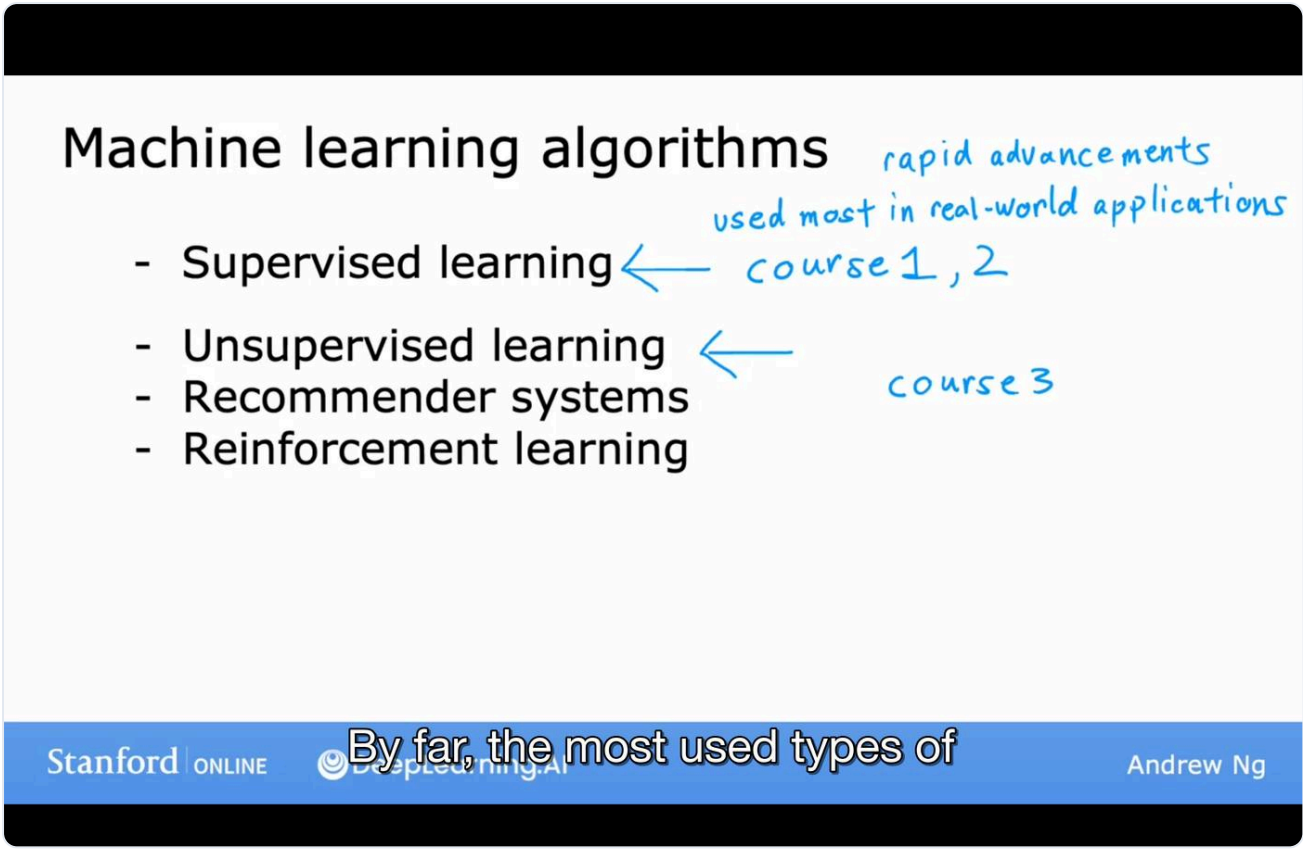


FIGURE 3.5

By far, the most used types of learning algorithms today are supervised learning, unsupervised learning, and recommender systems. The other thing we're going to spend a lot of time on in this specialization is practical advice for applying learning algorithms. This is something I feel pretty strongly about. Teaching about learning algorithms is like giving someone a set of tools and equally important, so even more important to making sure you have great tools is making sure you know how to apply them because like is it is somewhere where it gives you a state-of-the-art hammer or a state-of-the-art hand drill and say good luck. Now you have all the tools you need to build a three-story house.

It doesn't really work like that and so too, in machine learning, making sure you have the tools is really important and so is making sure that you know how to apply the tools of machine learning effectively. That's what you get in this class, the tools as well as the skills to apply them effectively. I regularly visit with friends and teams in some of the top tech companies, and even today I see experienced machine learning teams apply machine learning algorithms to some problems, and sometimes they've been going at it for six months without much success. When I look at what they're doing, I sometimes feel like I could have told them six months ago that the current approach won't work and there's a different way of using these tools that will give them a much better chance of success.

In this class, one of the relatively unique things you learn is you learn a lot about the best practices for how to actually develop a practical, valuable machine learning system. This way, you're less likely to end up in one of those teams that end up losing six months going in the wrong direction. In this class, you gain a sense of how the most skilled machine learning engineers build systems. I hope you finish this class as one of those very rare people in today's world that

know how to design and build serious machine learning systems. That's machine learning. In the next video, let's look more deeply at what is supervised learning and also what is unsupervised learning.

In addition, you'll learn when you might want to use each of them, supervised and unsupervised learning. I'll see you in the next video.

LECTURE 4

Supervised Learning

Machine learning is creating tremendous economic value today. I think 99 percent of the economic value created by machine learning today is through one type of machine learning, which is called supervised learning. Let's take a look at what that means. Supervised machine learning or more commonly, supervised learning, refers to algorithms that learn x to y or input to output mappings.

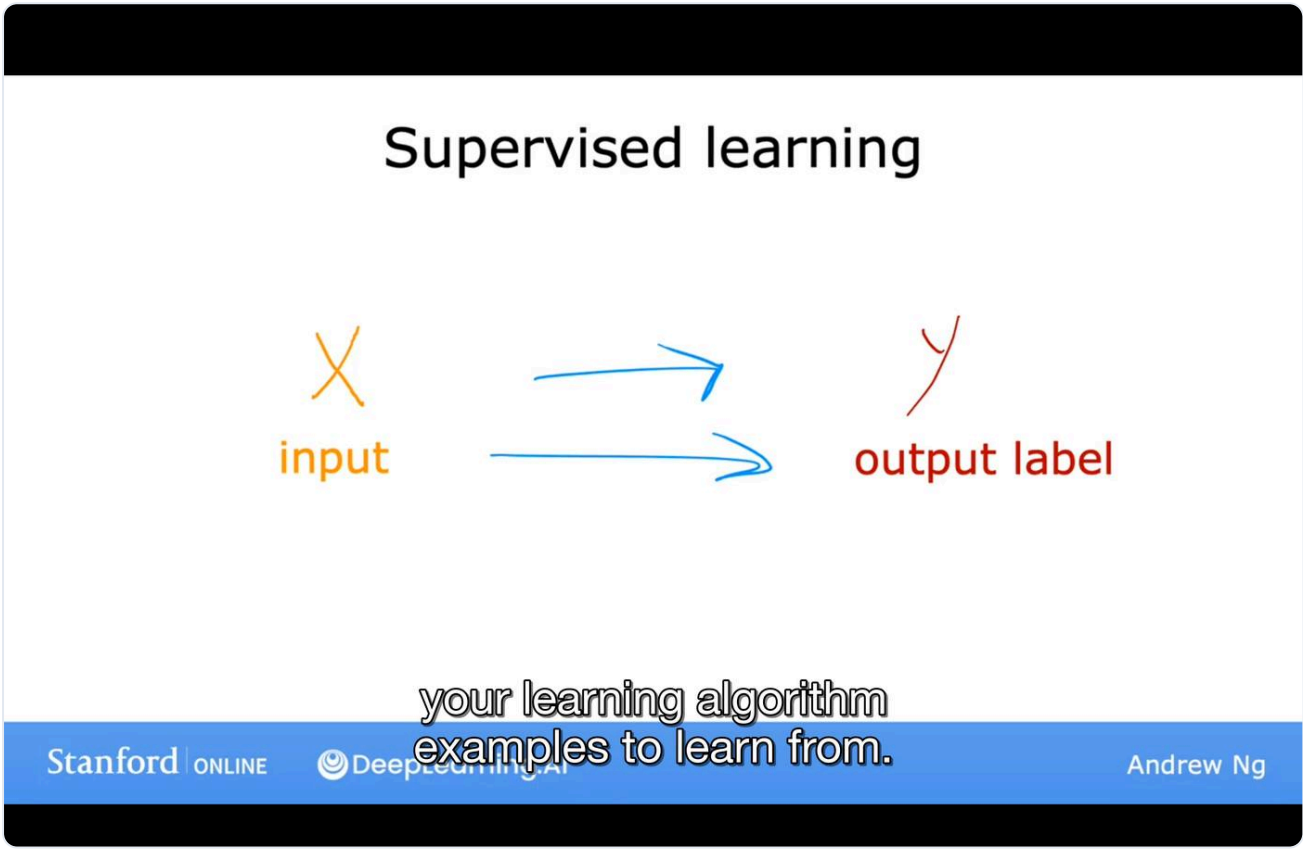


FIGURE 4.1 | Supervised Learning

The key characteristic of supervised learning is that you give your learning algorithm examples to learn from.

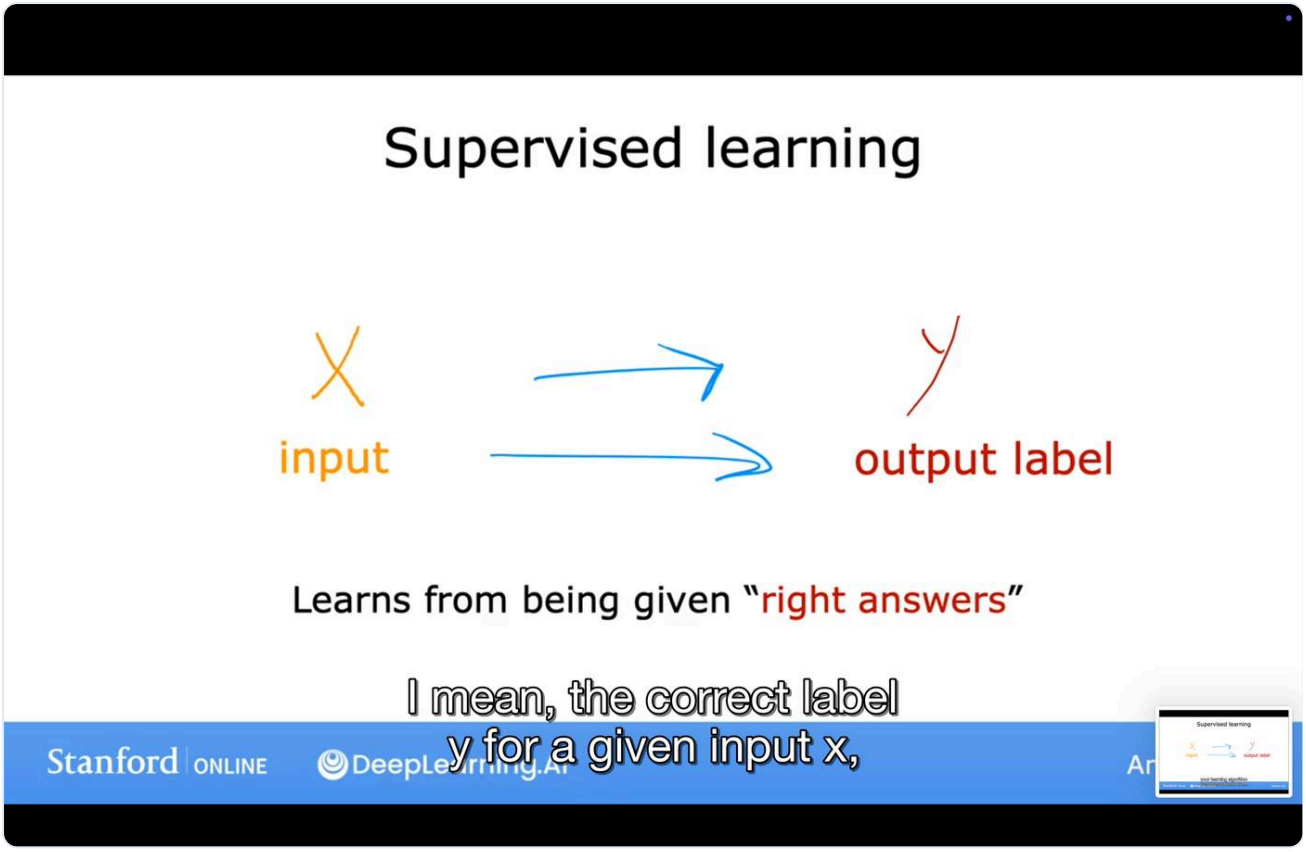


FIGURE 4.2 | Supervised Learning

That includes the right answers, whereby right answer, I mean, the correct label y for a given input x, and is by seeing correct pairs of input x and desired output label y that the learning algorithm eventually learns to take just the input alone without the output label and gives a reasonably accurate prediction or guess of the output. Let's look at some examples. If the input x is an email and the output y is this email, spam or not spam, this gives you your spam filter. Or if the input is an audio clip and the algorithm's job is output the text transcript, then this is speech recognition. Or if you want to input English and have it output to corresponding Spanish, Arabic, Hindi, Chinese, Japanese, or something else translation, then that's machine translation.

Or the most lucrative form of supervised learning today is probably used in online advertising. Nearly all the large online ad platforms have a learning algorithm that inputs some information about an ad and some information about you and then tries to figure out if you will click on that ad or not.

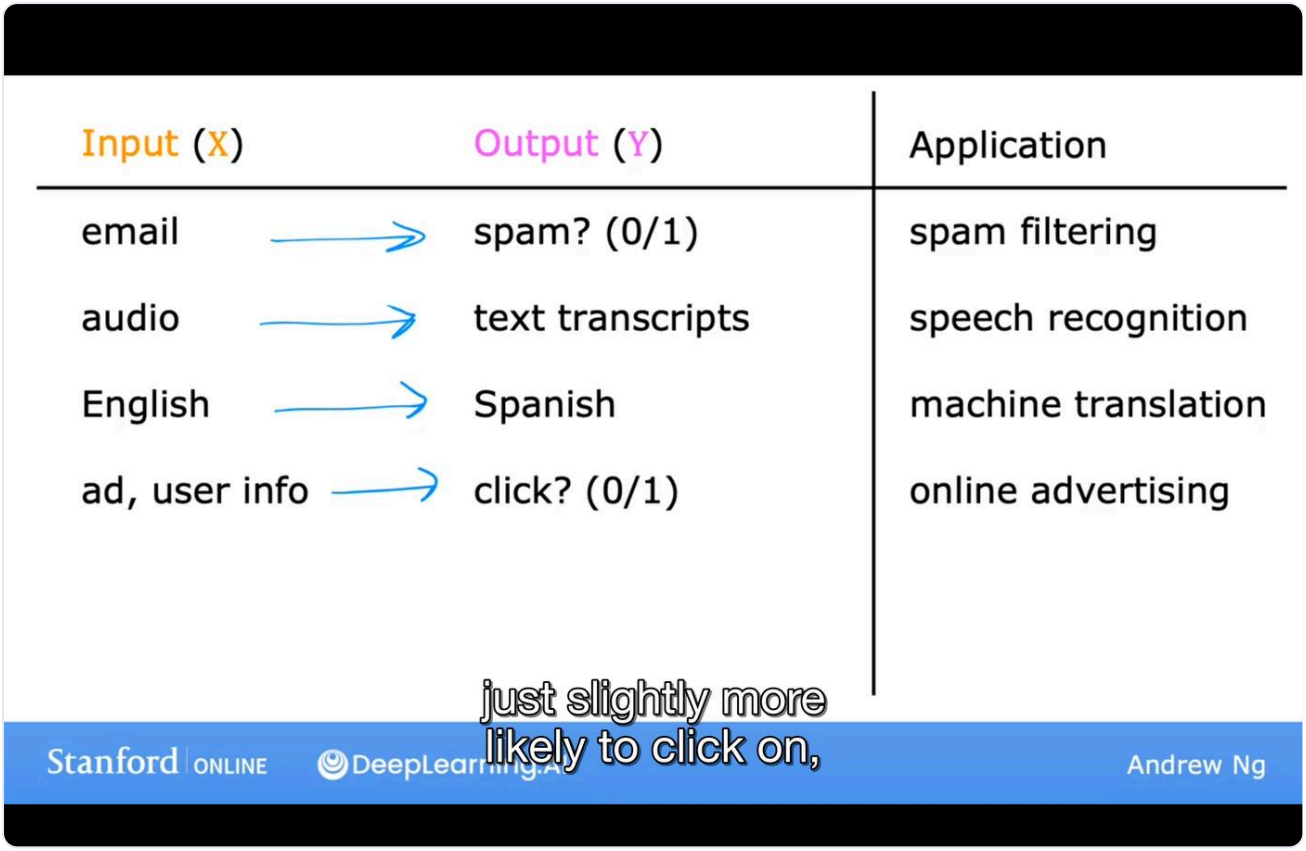


FIGURE 4.3 | Just Slightly

Because by showing you ads they're just slightly more likely to click on, for these large online ad platforms, every click is revenue, this actually drives a lot of revenue for these companies. This is something I once done a lot of work on, maybe not the most inspiring application, but it certainly has a significant economic impact in some countries today.

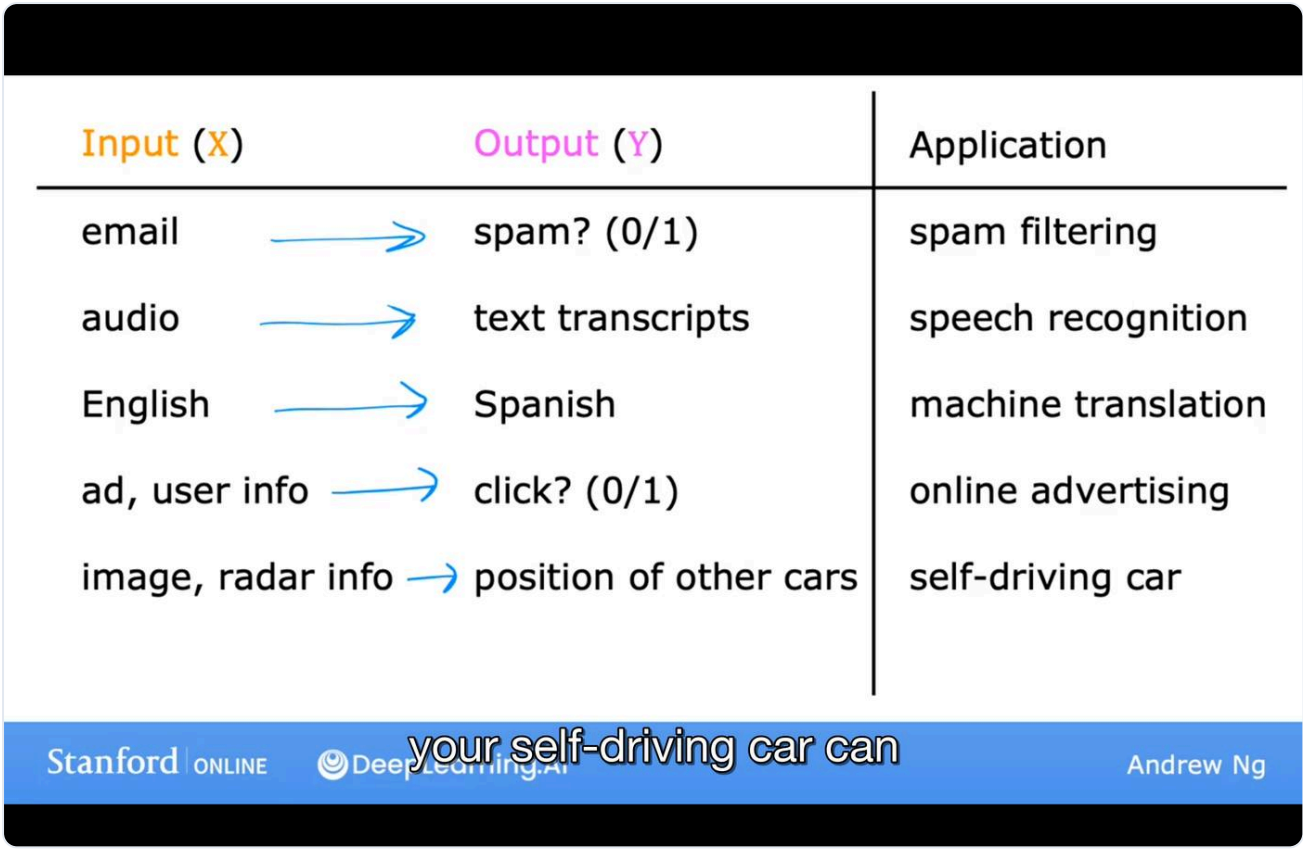


FIGURE 4.4

Or if you want to build a self-driving car, the learning algorithm would take as input an image and some information from other sensors such as a radar or other things and then try to output the position of, say, other cars so that your self-driving car can safely drive around the other cars. Or take manufacturing. I've actually done a lot of work in this sector at learning AI. You can have a learning algorithm takes as input a picture of a manufactured product, say a cell phone that just rolled off the production line and have the learning algorithm output whether or not there is a scratch, dent, or other defect in the product.

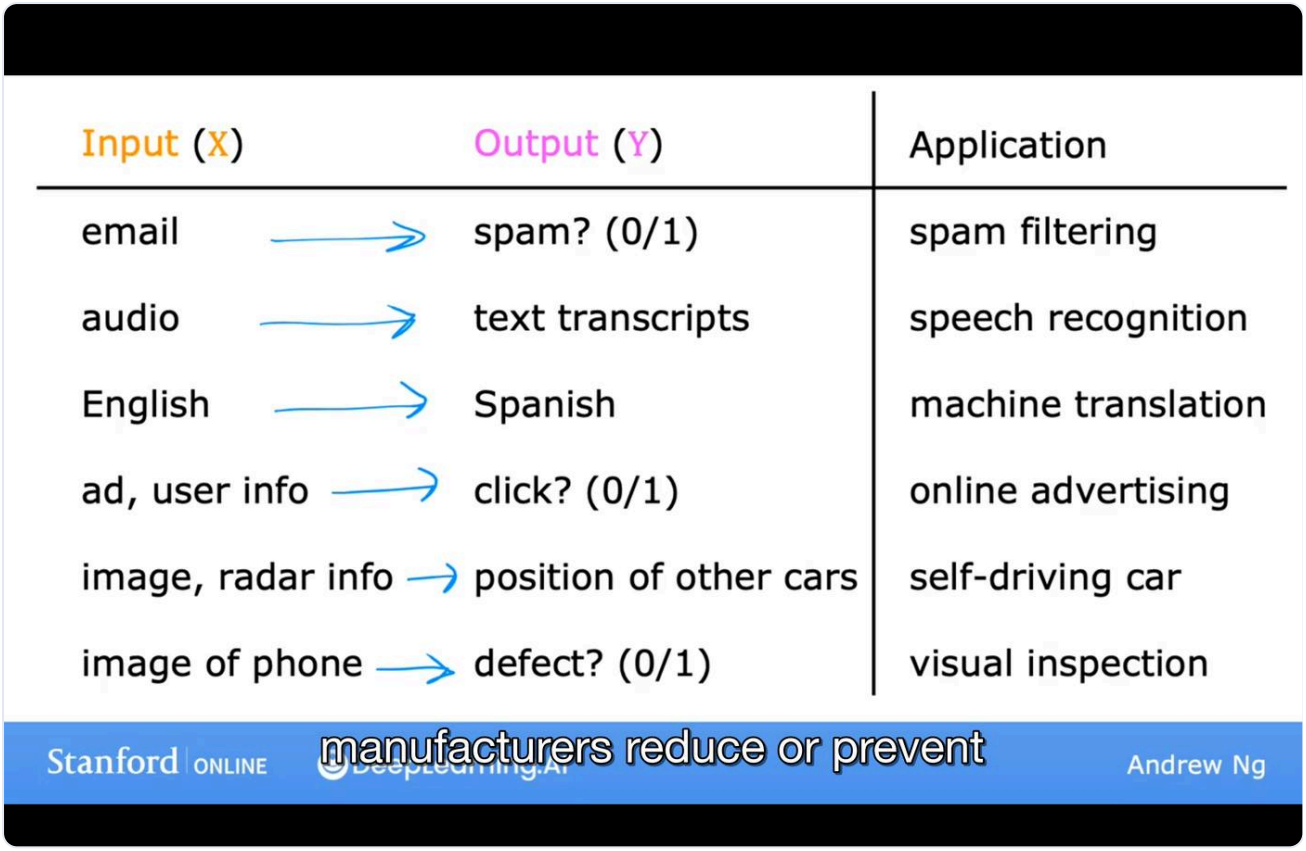


FIGURE 4.5

This is called visual inspection and it's helping manufacturers reduce or prevent defects in their products. In all of these applications, you will first train your model with examples of inputs x and the right answers, that is the labels y. After the model has learned from these input, output, or x and y pairs, they can then take a brand new input x, something it has never seen before, and try to produce the appropriate corresponding output y. Let's dive more deeply into one specific example. Say you want to predict housing prices based on the size of the house. You've collected some data and say you plot the data and it looks like this. Here on the horizontal axis is the size of the house in square feet.

Yes, I live in the United States where we still use square feet. I know most of the world uses square meters.

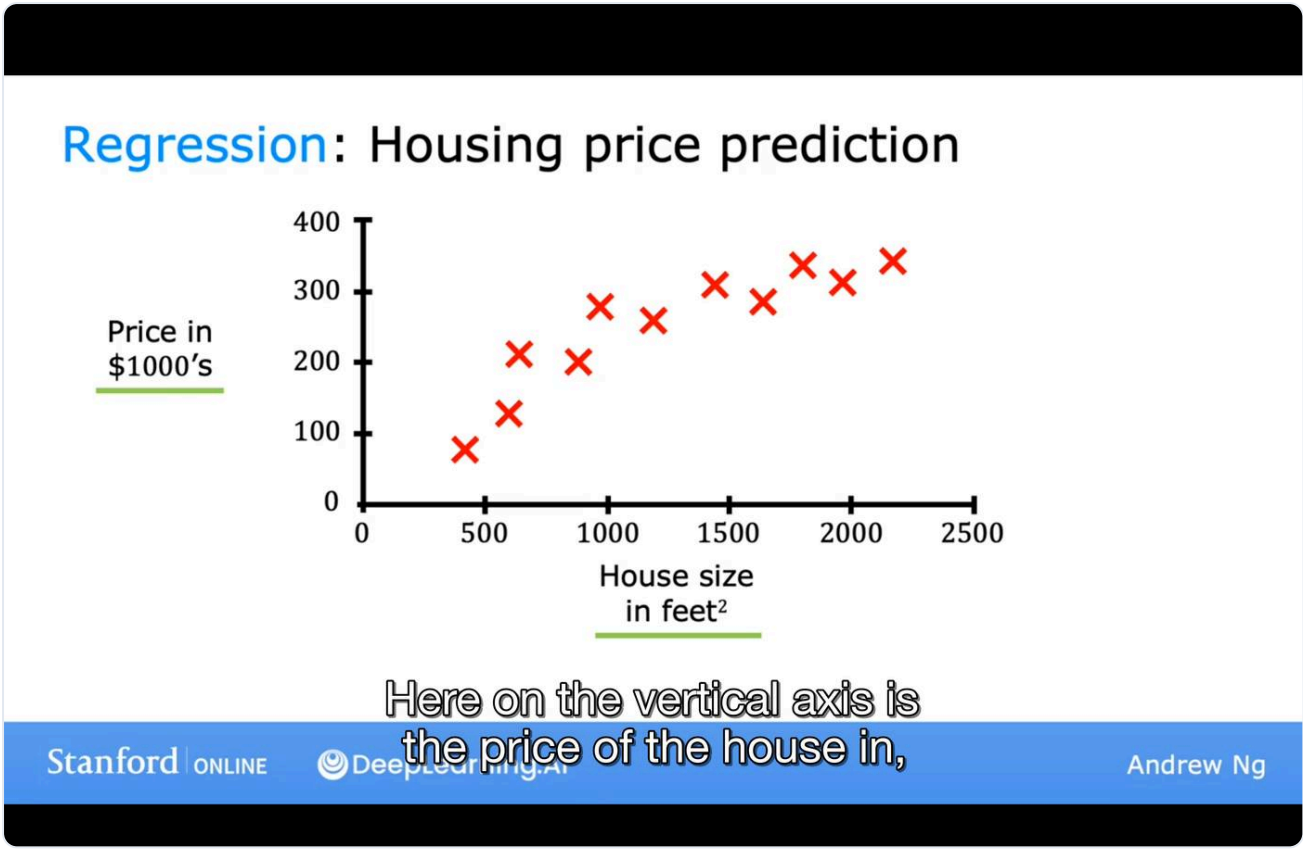


FIGURE 4.6 | Regression: Housing Price Prediction

Here on the vertical axis is the price of the house in, say, thousands of dollars. With this data, let's say a friend wants to know what's the price for their 750 square foot house. How can the learning algorithm help you? One thing a learning algorithm might be able to do is say, for the straight line to the data and reading off the straight line, it looks like your friend's house could be sold for maybe about, I don't know, $150,000. But fitting a straight line isn't the only learning algorithm you can use. There are others that could work better for this application. For example, routed and fitting a straight line, you might decide that it's better to fit a curve, a function that's slightly more complicated or more complex than a straight line.

If you do that and make a prediction here, then it looks like, well, your friend's house could be sold for closer to $200,000. One of the things you see later in this class is how you can decide whether to fit a straight line, a curve, or another function that is even more complex to the data. Now, it doesn't seem appropriate to pick the one that gives your friend the best price, but one thing you see is how to get an algorithm to systematically choose the most appropriate line or curve or other thing to fit to this data. What you've seen in this slide is an example of supervised learning. Because we gave the algorithm a dataset in which the so-called right answer, that is the label or the correct price y is given for every house on the plot.

The task of the learning algorithm is to produce more of these right answers, specifically predicting what is the likely price for other houses like your friend's house. That's why this is supervised learning. To define a little bit more terminology, this housing price prediction is the particular type of supervised learning called regression.

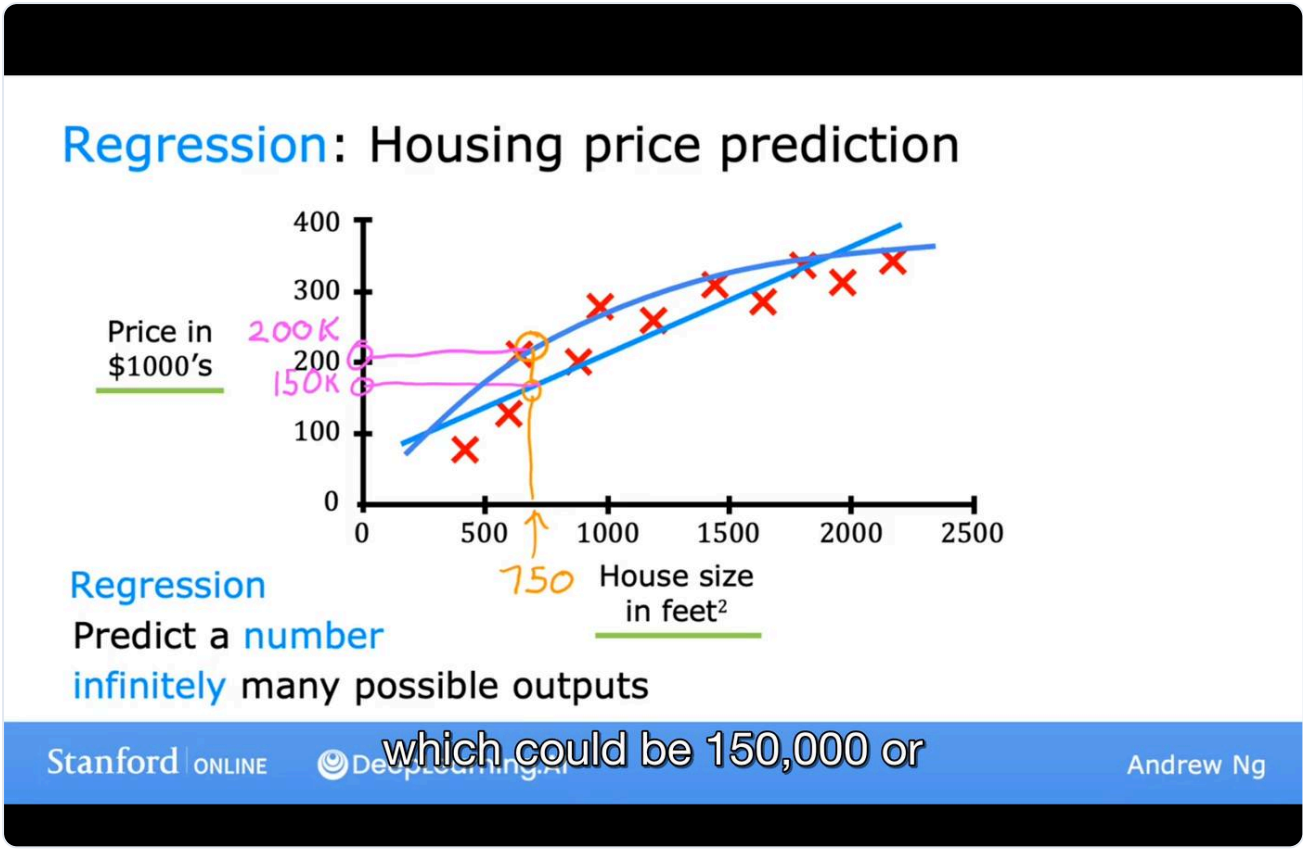


FIGURE 4.7 | Regression: Housing Price Prediction

By regression, I mean we're trying to predict a number from infinitely many possible numbers such as the house prices in our example, which could be 150,000 or 70,000 or 183,000 or any other number in between. That's supervised learning, learning input, output, or x to y mappings. You saw in this video an example of regression where the task is to predict number. But there's also a second major type of supervised learning problem called classification. Let's take a look at what that means in the next video.

LECTURE 5

Classification: Breast Cancer Detection

So supervised learning algorithms learn to predict input, output or X to Y mapping. And in the last video you saw that regression algorithms, which is a type of supervised learning algorithm learns to predict numbers out of infinitely many possible numbers. There's a second major type of supervised learning algorithm called a classification algorithm. Let's take a look at what this means. Take breast cancer detection as an example of a classification problem. Say you're building a machine learning system so that doctors can have a diagnostic tool to detect breast cancer. This is important because early detection could potentially save a patient's life.

Using a patient's medical records your machine learning system tries to figure out if a tumor that is a lump is malignant meaning cancerous or dangerous. Or if that tumor, that lump is benign, meaning that it's just a lump that isn't cancerous and isn't that dangerous? Some of my friends have actually been working on this specific problem. So maybe your dataset has tumors of various sizes. And these tumors are labeled as either benign, which I will designate in this example with a 0 or malignant, which will designate in this example with a 1.

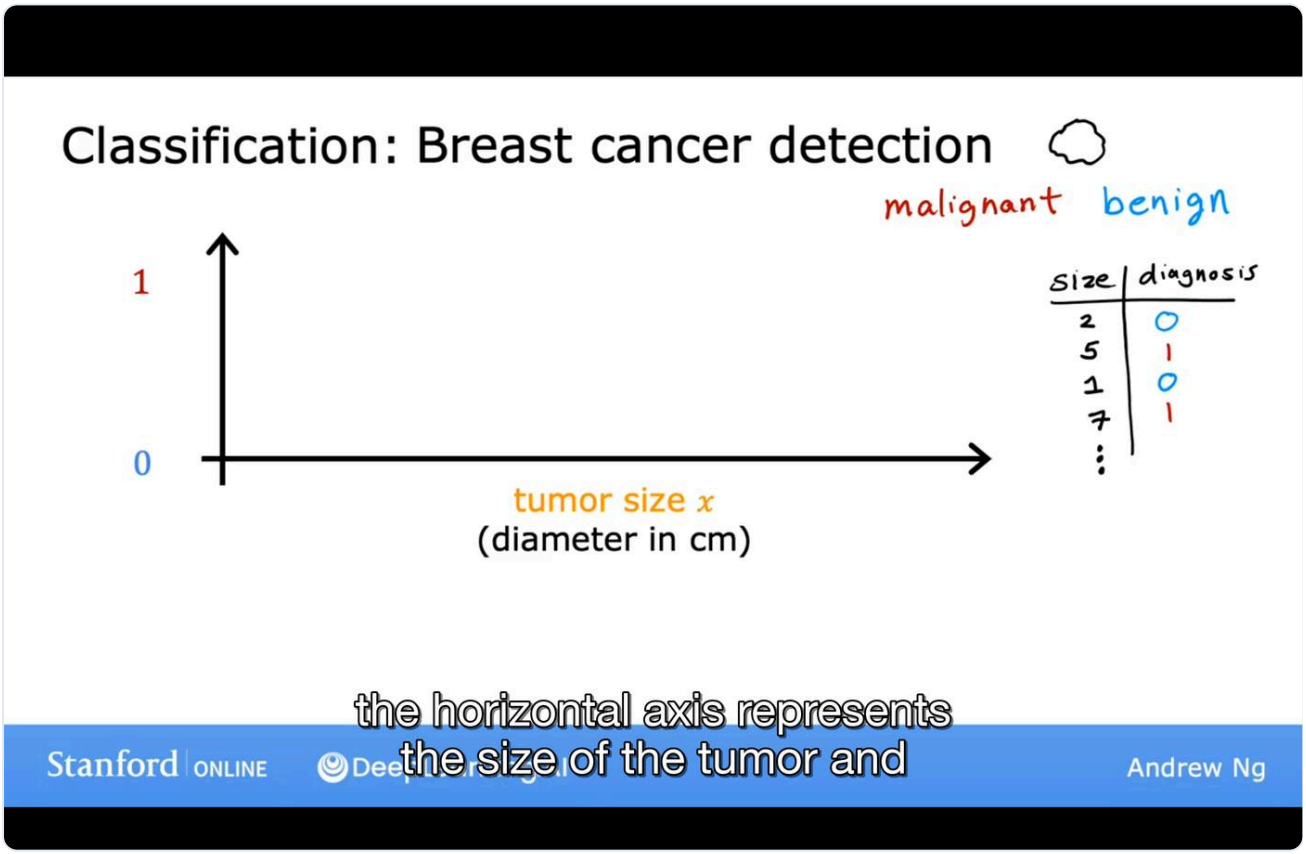


FIGURE 5.1

You can then plot your data on a graph like this where the horizontal axis represents the size of the tumor and the vertical axis takes on only two values 0 or 1 depending on whether the tumor is benign, 0 or malignant 1. One reason that this is different from regression is that we're trying

to predict only a small number of possible outputs or categories. In this case two possible outputs 0 or 1, benign or malignant. This is different from regression which tries to predict any number, all of the infinitely many number of possible numbers.

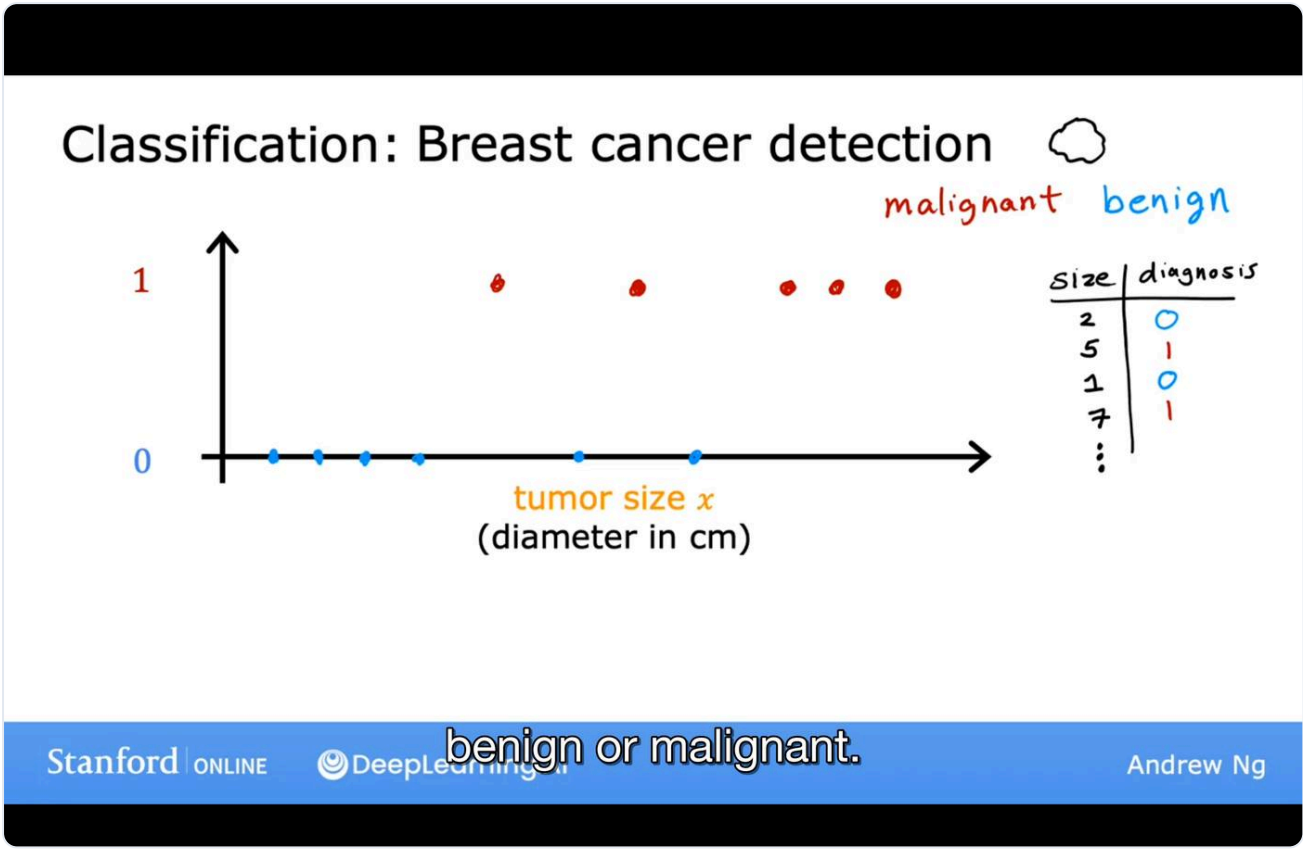


FIGURE 5.2

And so the fact that there are only two possible outputs is what makes this classification. Because there are only two possible outputs or two possible categories in this example, you can also plot this data set on a line like this. Right now, I'm going to use two different symbols to denote the category using a circle an O to denote the benign examples and a cross to denote the malignant examples.

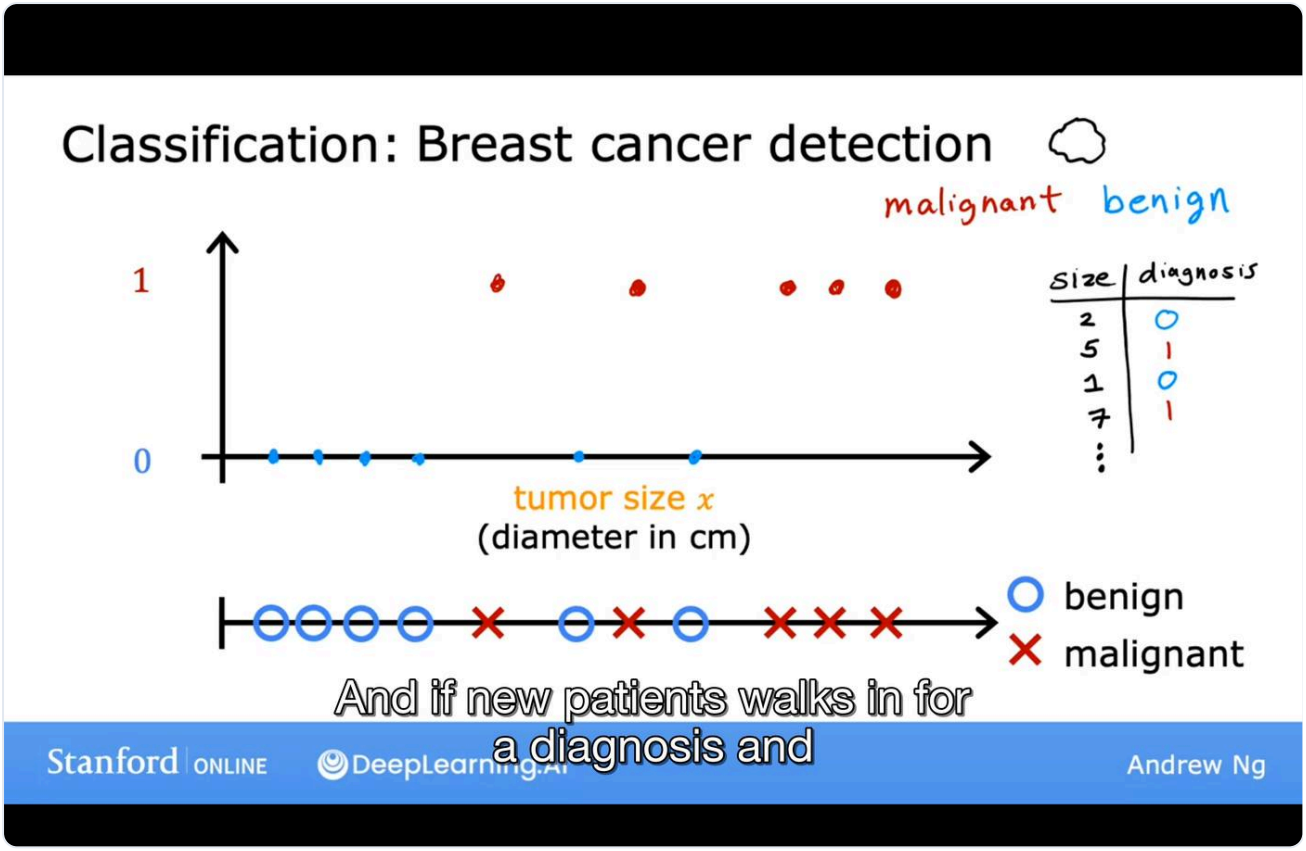


FIGURE 5.3 | Classification: Breast Cancer Detection

And if new patients walks in for a diagnosis and they have a lump that is this size, then the question is, will your system classify this tumor as benign or malignant? It turns out that in classification problems you can also have more than two possible output categories. Maybe you're learning algorithm can output multiple types of cancer diagnosis if it turns out to be malignant. So let's call two different types of cancer type 1 and type 2. In this case the average would have three possible output categories it could predict.

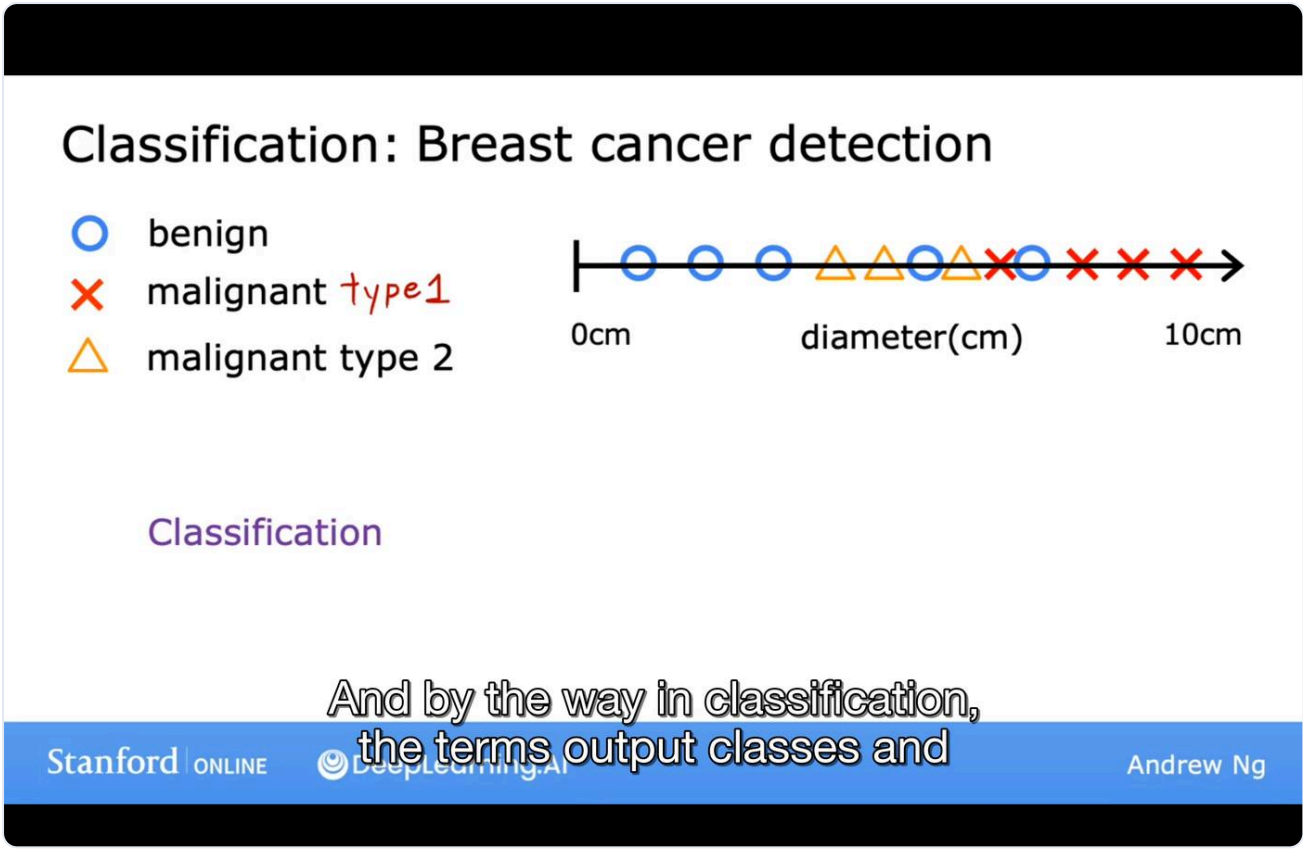


FIGURE 5.4 | Classification: Breast Cancer Detection

And by the way in classification, the terms output classes and output categories are often used interchangeably.

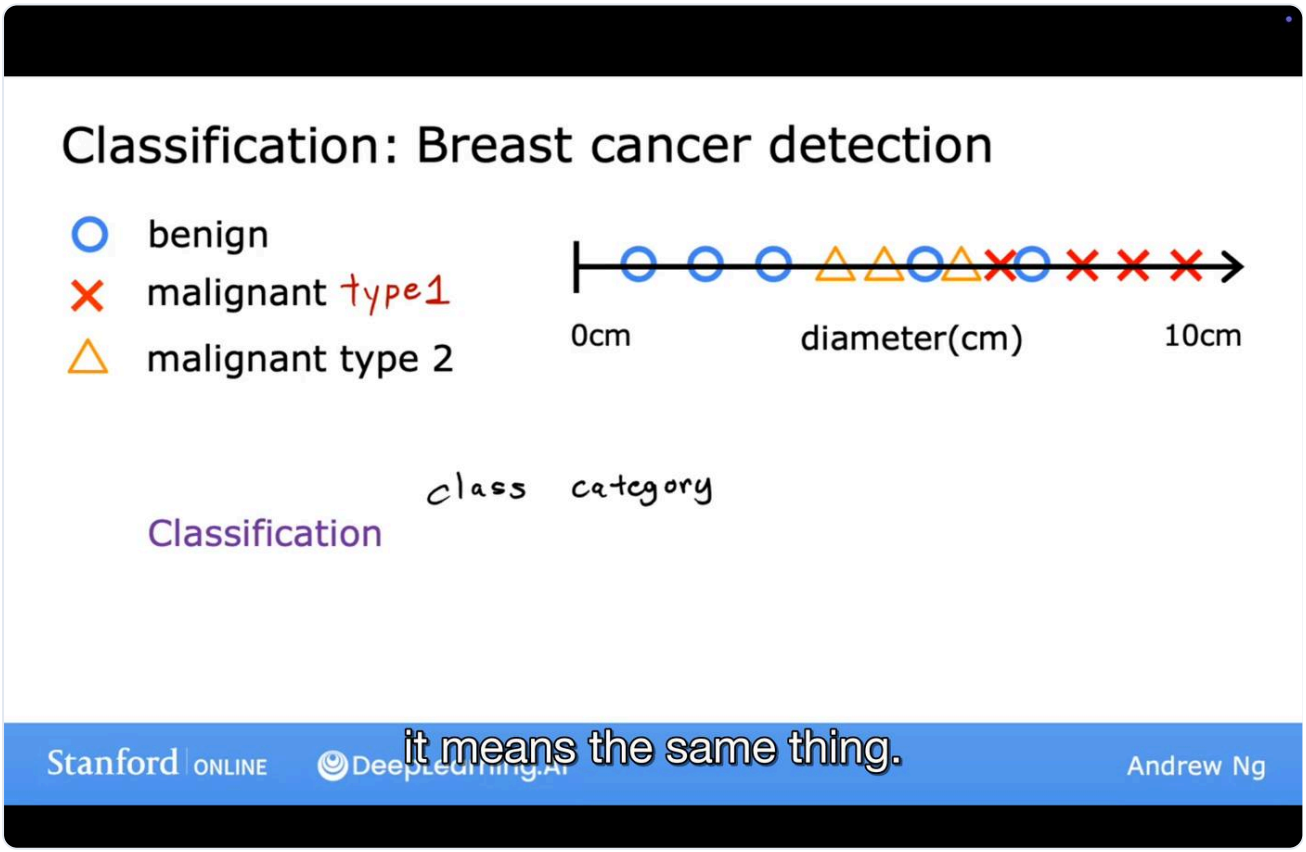


FIGURE 5.5 | Classification: Breast Cancer Detection

So what I say class or category when referring to the output, it means the same thing. So to summarize classification algorithms predict categories. Categories don't have to be numbers. It could be non numeric for example, it can predict whether a picture is that of a cat or a dog.

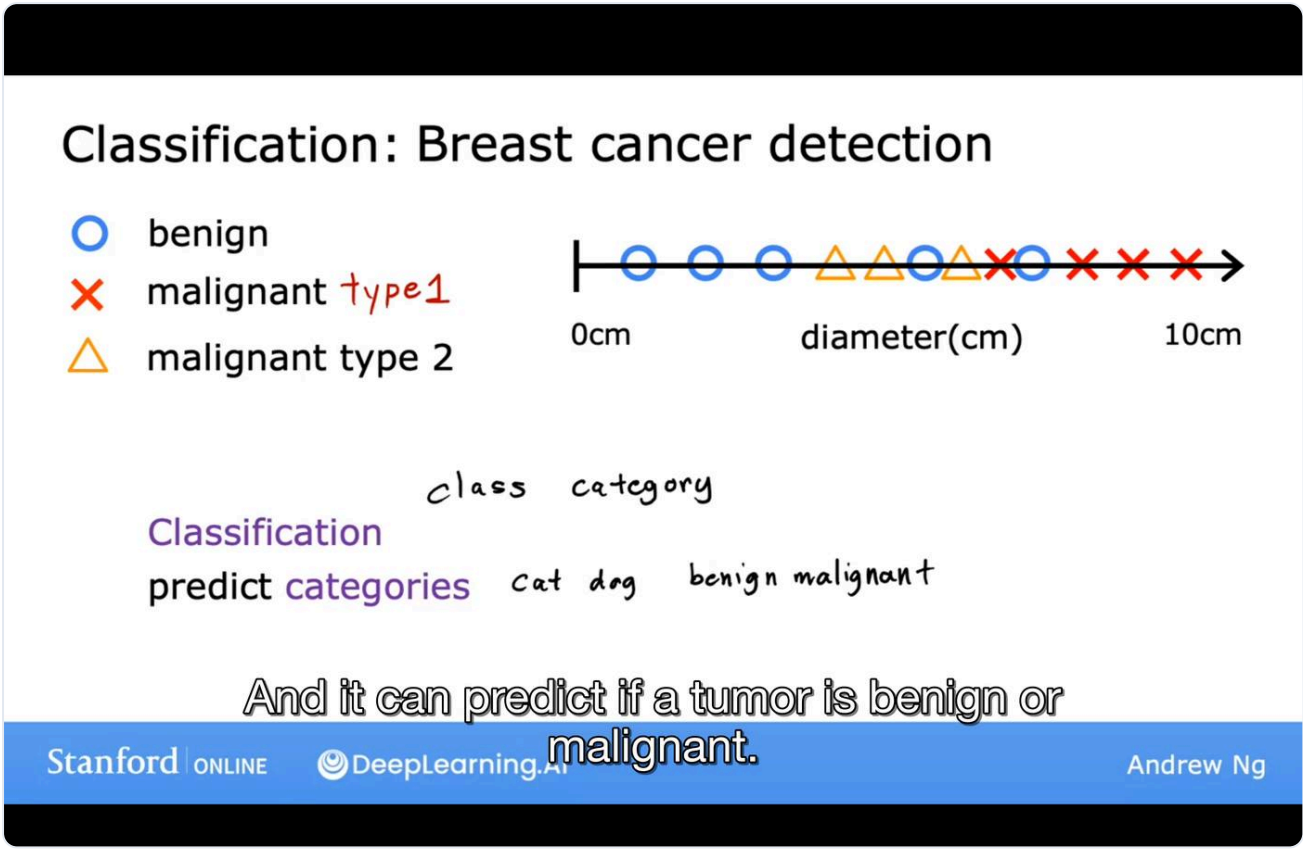


FIGURE 5.6 | Classification: Breast Cancer Detection

And it can predict if a tumor is benign or malignant. Categories can also be numbers like 0, 1 or 0, 1, 2.

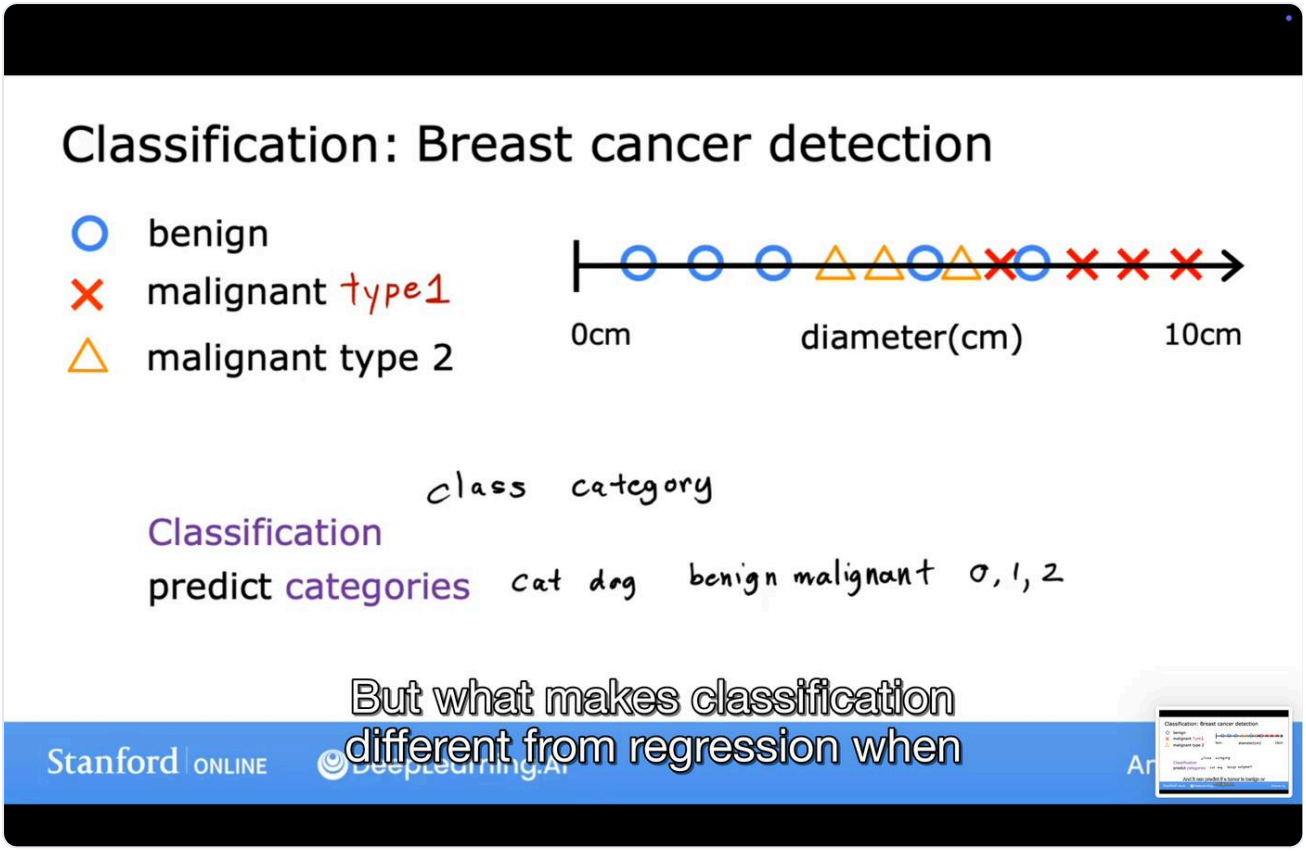


FIGURE 5.7 | Classification: Breast Cancer Detection

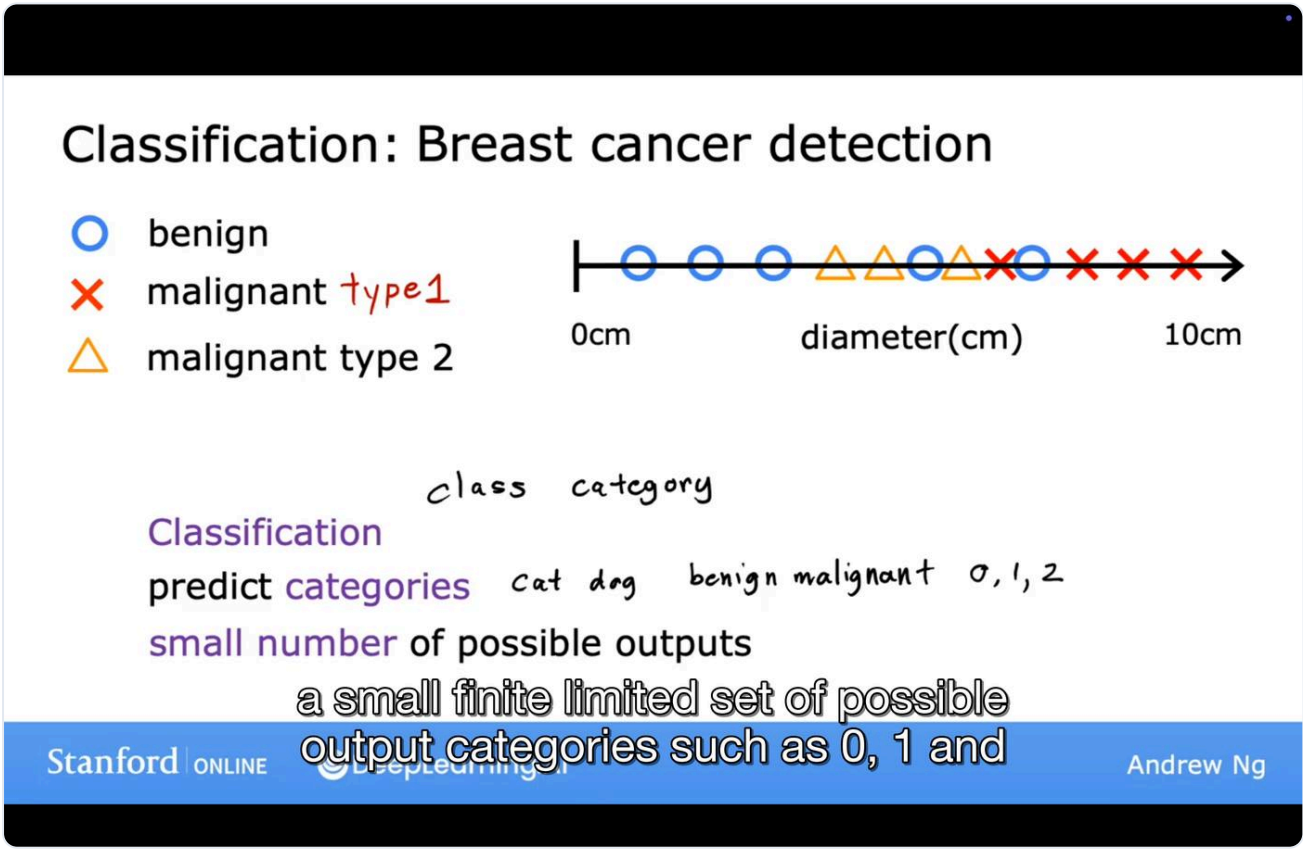


FIGURE 5.8 | Classification: Breast Cancer Detection

But what makes classification different from regression when you're interpreting the numbers is that classification predicts a small finite limited set of possible output categories such as 0, 1 and 2 but not all possible numbers in between like 0.5 or 1.7. In the example of supervised learning that we've been looking at, we had only one input value the size of the tumor. But you can also use more than one input value to predict an output. Here's an example, instead of just knowing the tumor size, say you also have each patient's age in years. Your new data set now has two inputs, age and tumor size. What in this new dataset we're going to use circles to show patients whose tumors are benign and crosses to show the patients with a tumor that was malignant.

So when a new patient comes in, the doctor can measure the patient's tumor size and also record the patient's age. And so given this, how can we predict if this patient's tumor is benign or malignant? Well, given the day said like this, what the learning algorithm might do is find some boundary that separates out the malignant tumors from the benign ones.

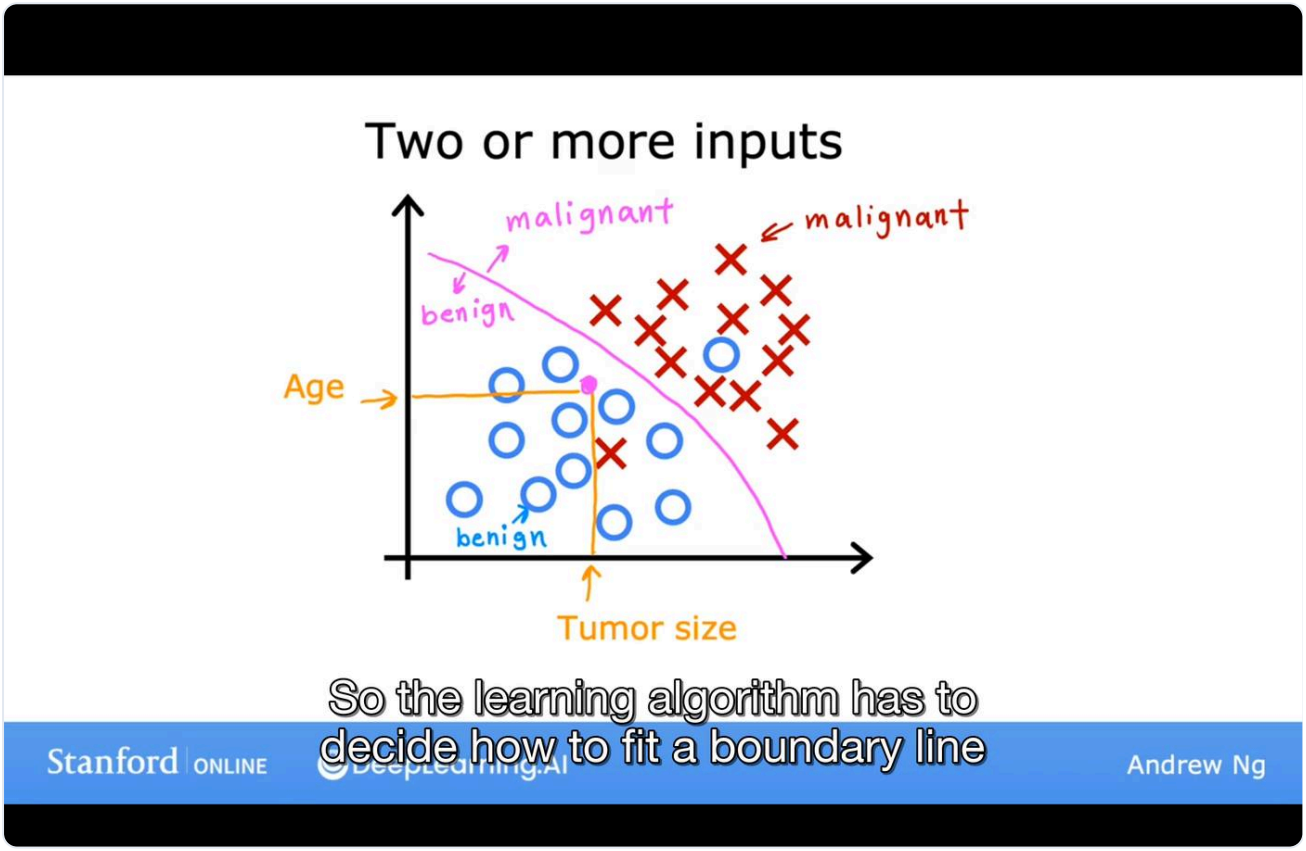


FIGURE 5.9 | Two or More Inputs

So the learning algorithm has to decide how to fit a boundary line through this data. The boundary line found by the learning algorithm would help the doctor with the diagnosis. In this case the tumor is more likely to be benign. From this example we have seen how to inputs the patient's age and tumor size can be used. In other machine learning problems often many more input values are required. My friends who worked on breast cancer detection use many additional inputs, like the thickness of the tumor clump, uniformity of the cell size, uniformity of the cell shape and so on. So to recap supervised learning maps input x to output y, where the learning algorithm learns from the quote right answers.

The two major types of supervised learning our regression and classification. In a regression application like predicting prices of houses, the learning algorithm has to predict numbers from infinitely many possible output numbers.

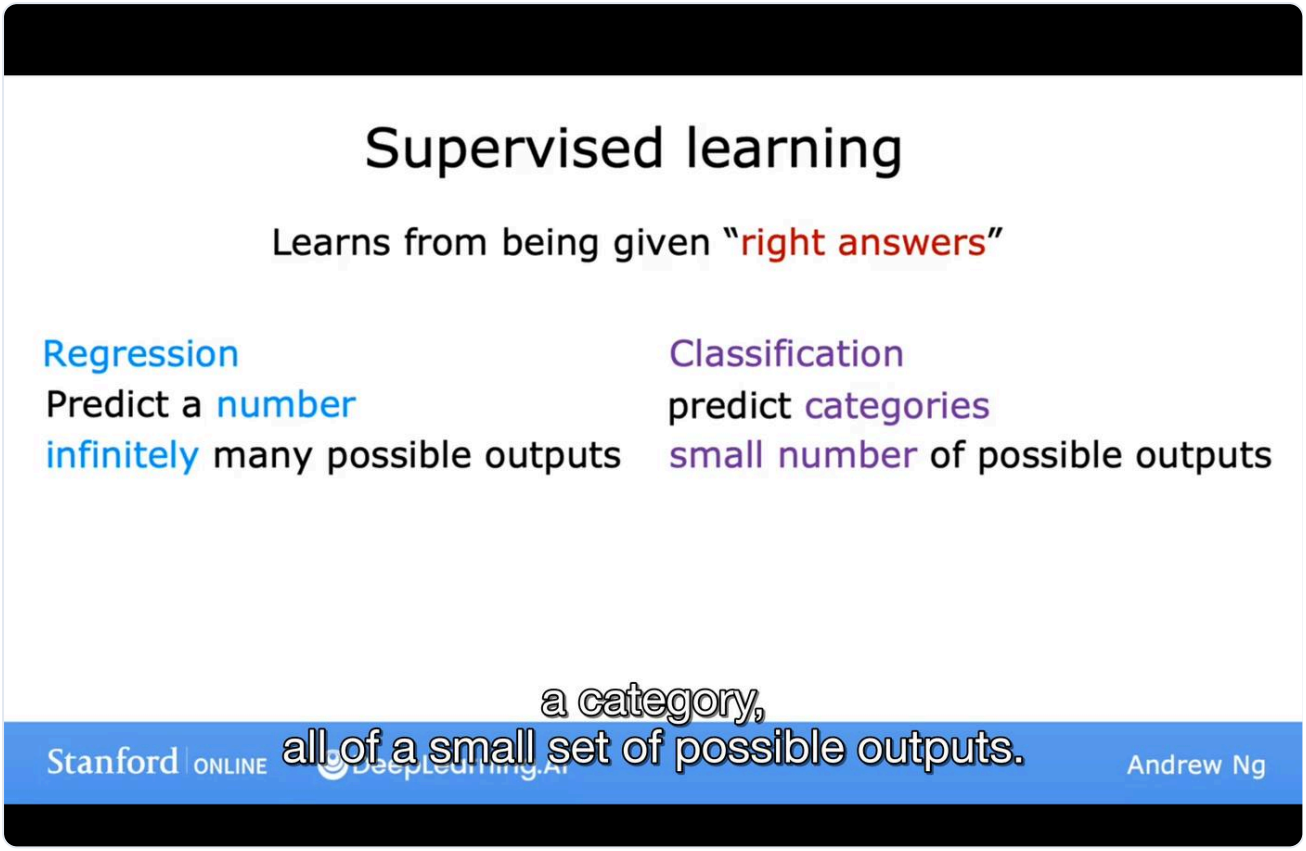


FIGURE 5.10 | Supervised Learning

Whereas in classification the learning algorithm has to make a prediction of a category, all of a small set of possible outputs. So you now know what is supervised learning, including both regression and classification. I hope you're having fun. Next there's a second major type of machine learning called unsupervised learning. Let's go on to the next video to see what that is

LECTURE 6

Supervised Learning Unsupervised Learning

After supervised learning, the most widely used form of machine learning is unsupervised learning. Let's take a look at what that means, we've talked about supervised learning and this video is about unsupervised learning. But don't let the name uncivilized for you, unsupervised learning is I think just as super as supervised learning. When we're looking at supervised learning in the last video recalled, it looks something like this in the case of a classification problem. Each example, was associated with an output label y such as benign or malignant, designated by the poles and crosses in unsupervised learning.

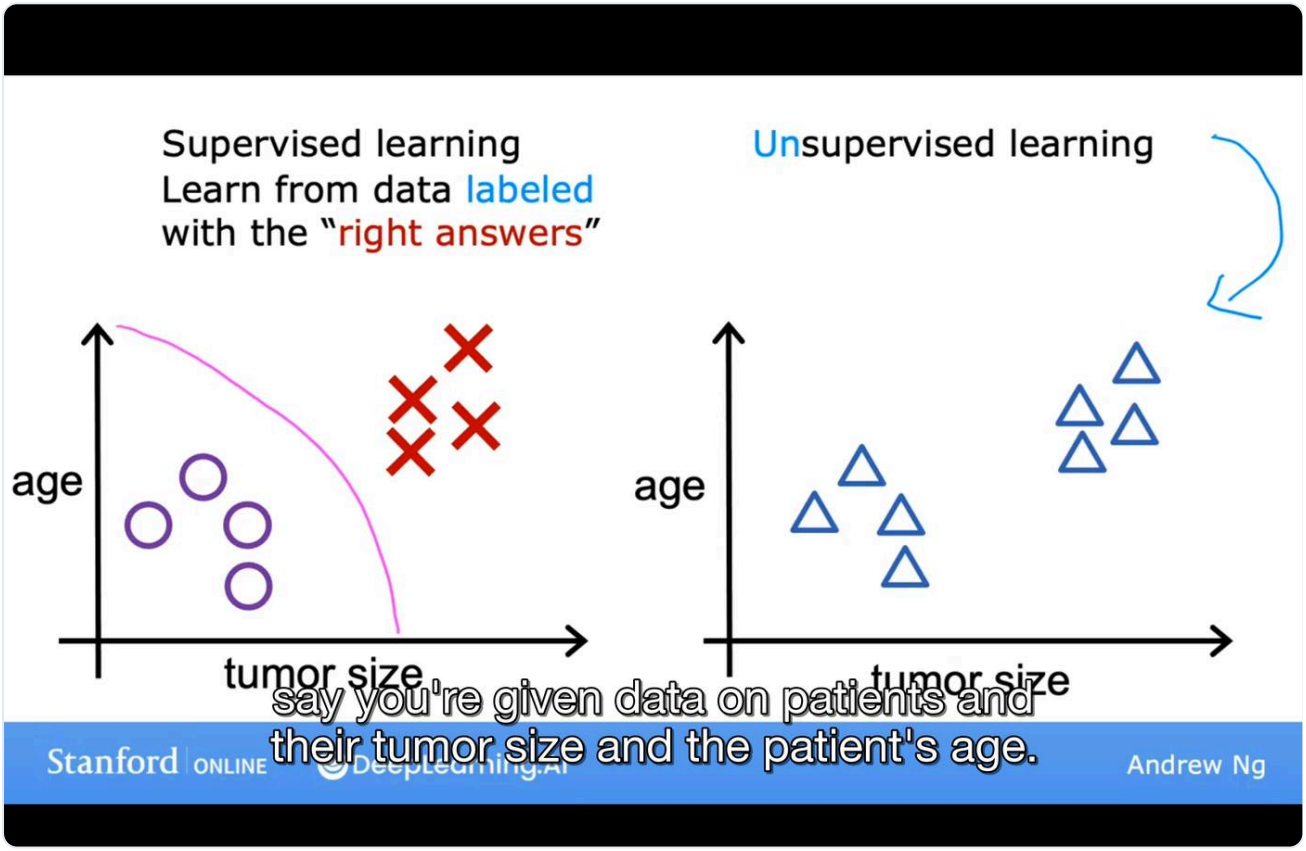


FIGURE 6.1 | Supervised Learning Unsupervised Learning

Were given data that isn't associated with any output labels y, say you're given data on patients and their tumor size and the patient's age.

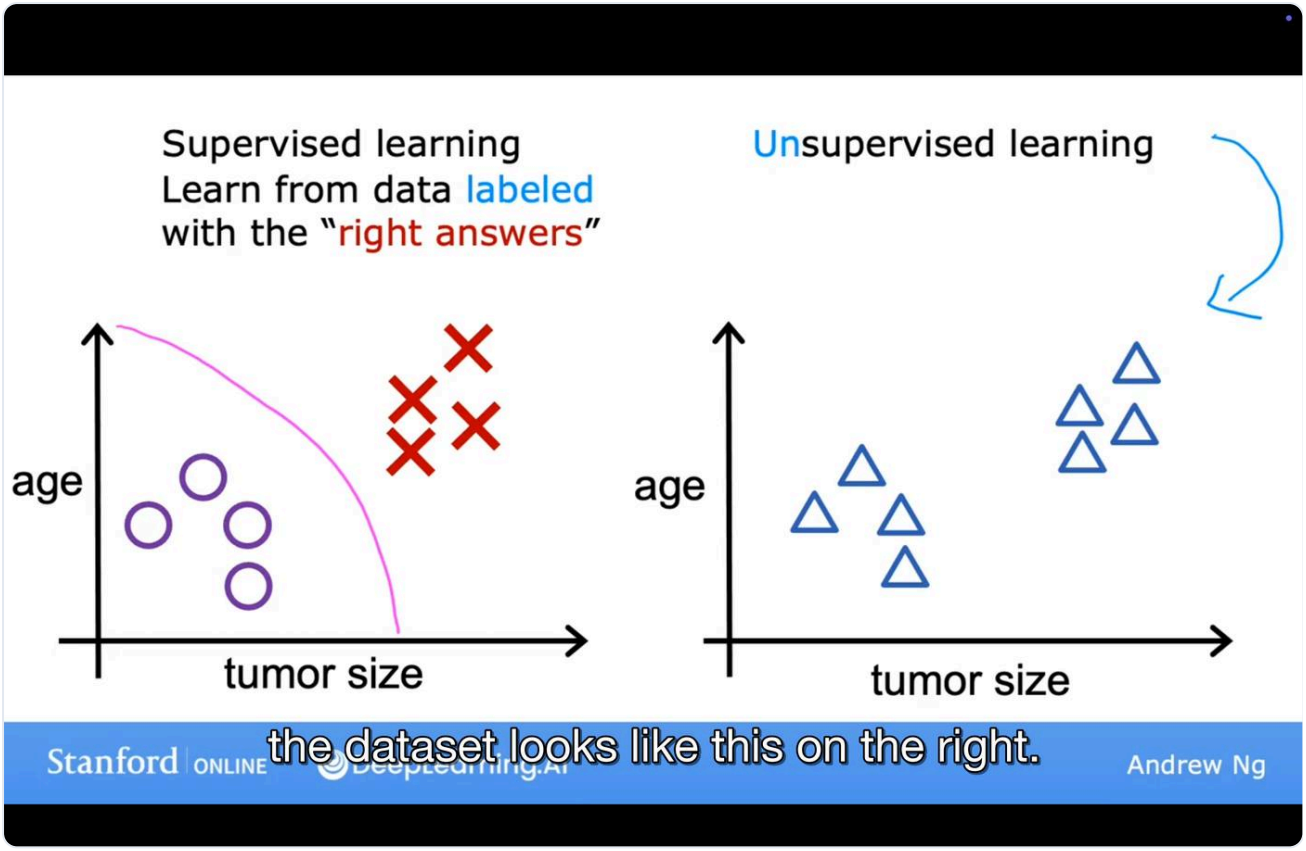


FIGURE 6.2 | Supervised Learning Unsupervised Learning

But not whether the tumor was benign or malignant, so the dataset looks like this on the right. We're not asked to diagnose whether the tumor is benign or malignant, because we're not given any labels. Why in the dataset, instead, our job is to find some structure or some pattern or just find something interesting in the data. This is unsupervised learning, we call it unsupervised because we're not trying to supervise the algorithm. To give some quote right answer for every input, instead, we asked the our room to figure out all by yourself what's interesting. Or what patterns or structures that might be in this data, with this particular data set.

An unsupervised learning algorithm, might decide that the data can be assigned to two different groups or two different clusters.

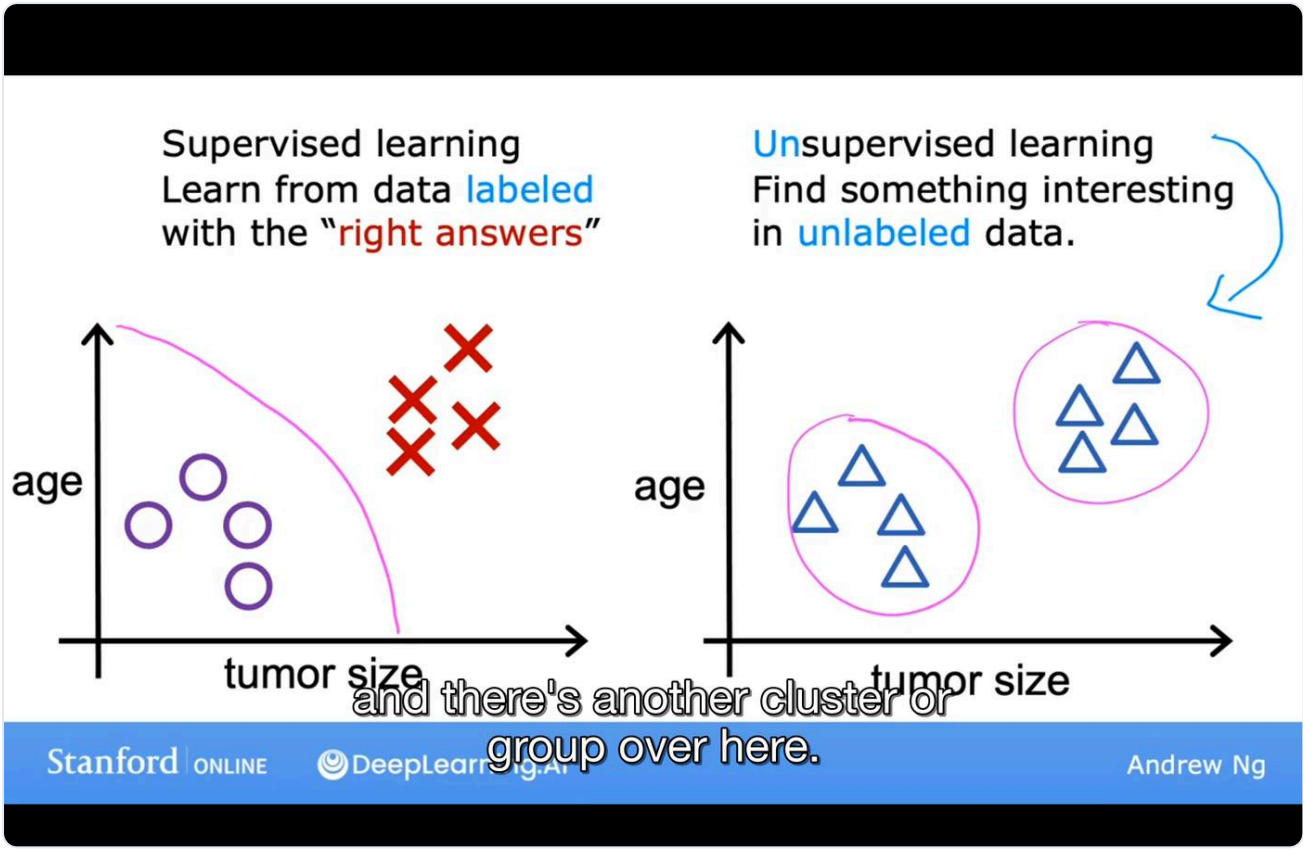


FIGURE 6.3 | Supervised Learning Unsupervised Learning

And so it might decide, that there's one cluster what group over here, and there's another cluster or group over here. This is a particular type of unsupervised learning, called a clustering algorithm.

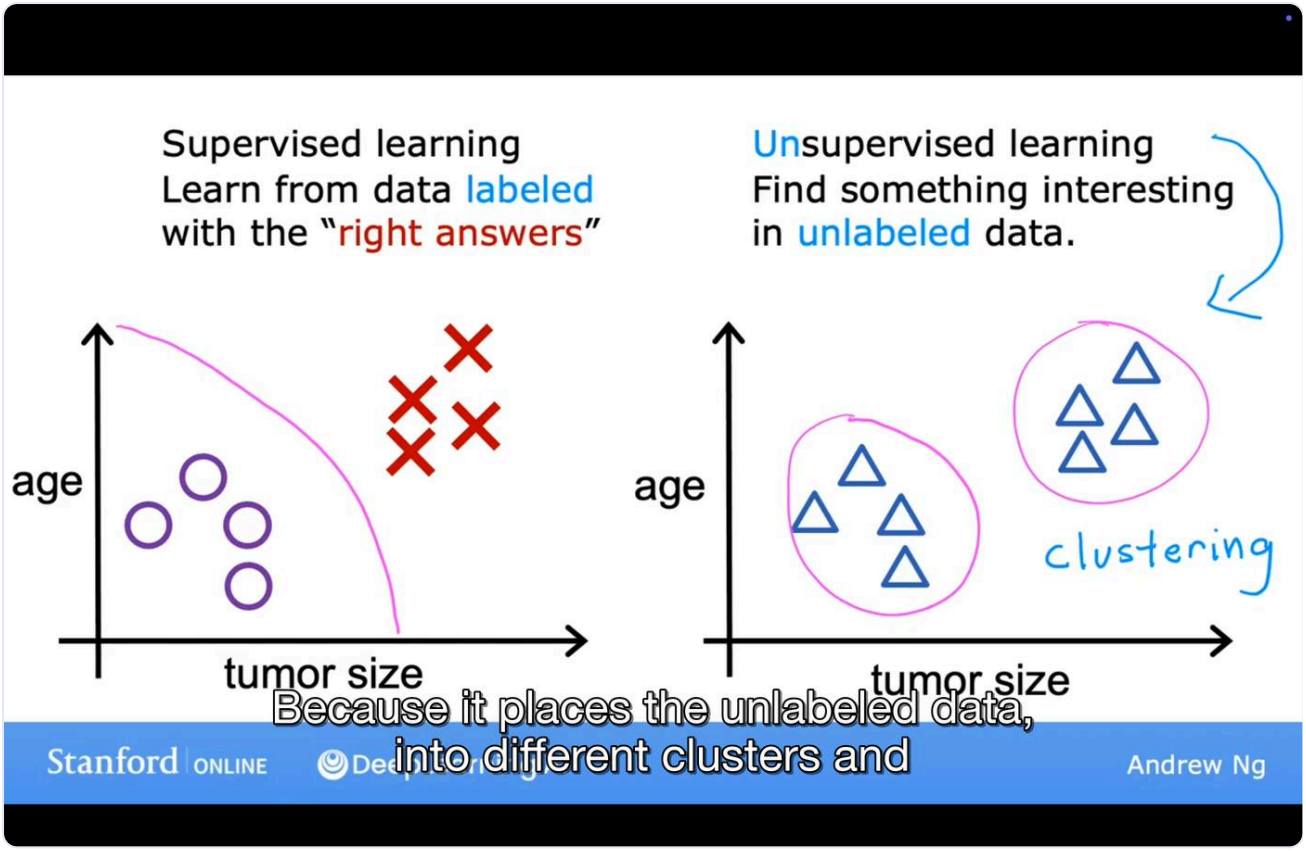


FIGURE 6.4 | Supervised Learning Unsupervised Learning

Because it places the unlabeled data, into different clusters and this turns out to be used in many applications. For example, clustering is used in google news, what google news does is every day it goes. And looks at hundreds of thousands of news articles on the internet, and groups related stories together.

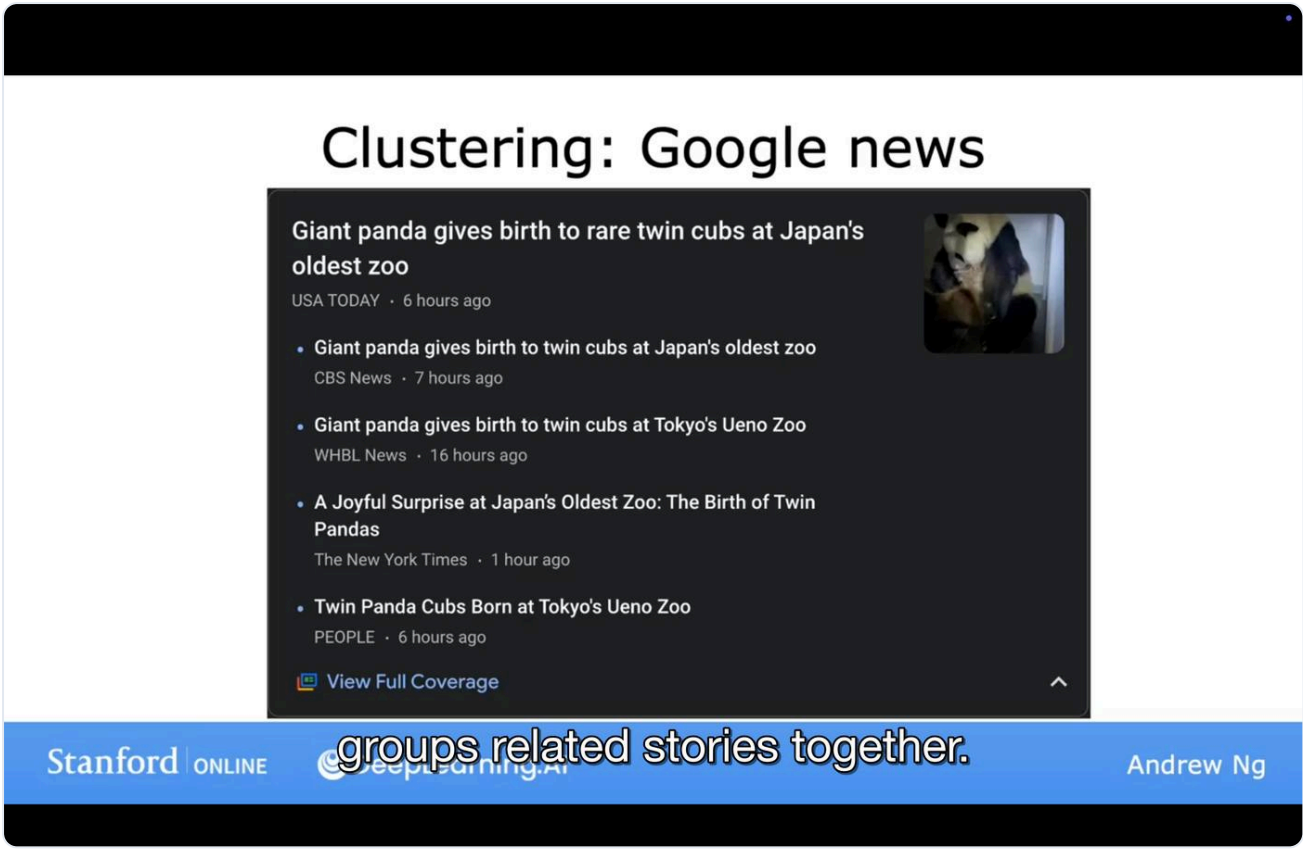


FIGURE 6.5 | Clustering: Google News

For example, here is a sample from Google News, where the headline of the top article, is giant panda gives birth to rear twin cubs at Japan's oldest zoo. This article has actually caught my eye, because my daughter loves pandas and so there are a lot of stuff panda toys. And watching panda videos in my house, and looking at this, you might notice that below this are other related articles. Maybe from the headlines alone, you can start to guess what clustering might be doing. Notice that the word panda appears here here, here, here and here and notice that the word twin also appears in all five articles. And the word Zoo also appears in all of these articles, so the clustering algorithm is finding articles.

All of all the hundreds of thousands of news articles on the internet that day, finding the articles that mention similar words and grouping them into clusters. Now, what's cool is that this clustering algorithm figures out on his own which words suggest, that certain articles are in the same group. What I mean is there isn't an employee at google news who's telling the algorithm to find articles that the word panda. And twins and zoo to put them into the same cluster, the news topics change every day.

FIGURE 6.6 | Clustering: Google News

And there are so many news stories, it just isn't feasible to people doing this every single day for all the topics that use covers. Instead the algorithm has to figure out on his own without supervision, what are the clusters of news articles today. So that's why this clustering algorithm, is a type of unsupervised learning algorithm. Let's look at the second example of unsupervised learning applied to clustering genetic or DNA data. This image shows a picture of DNA micro array data, these look like tiny grids of a spreadsheet. And each tiny column represents the genetic or DNA activity of one person, So for example, this entire Column here is from one person's DNA.

And this other column is of another person, each row represents a particular gene. So just as an example, perhaps this role here might represent a gene that affects eye color, or this role here is a gene that affects how tall someone is.

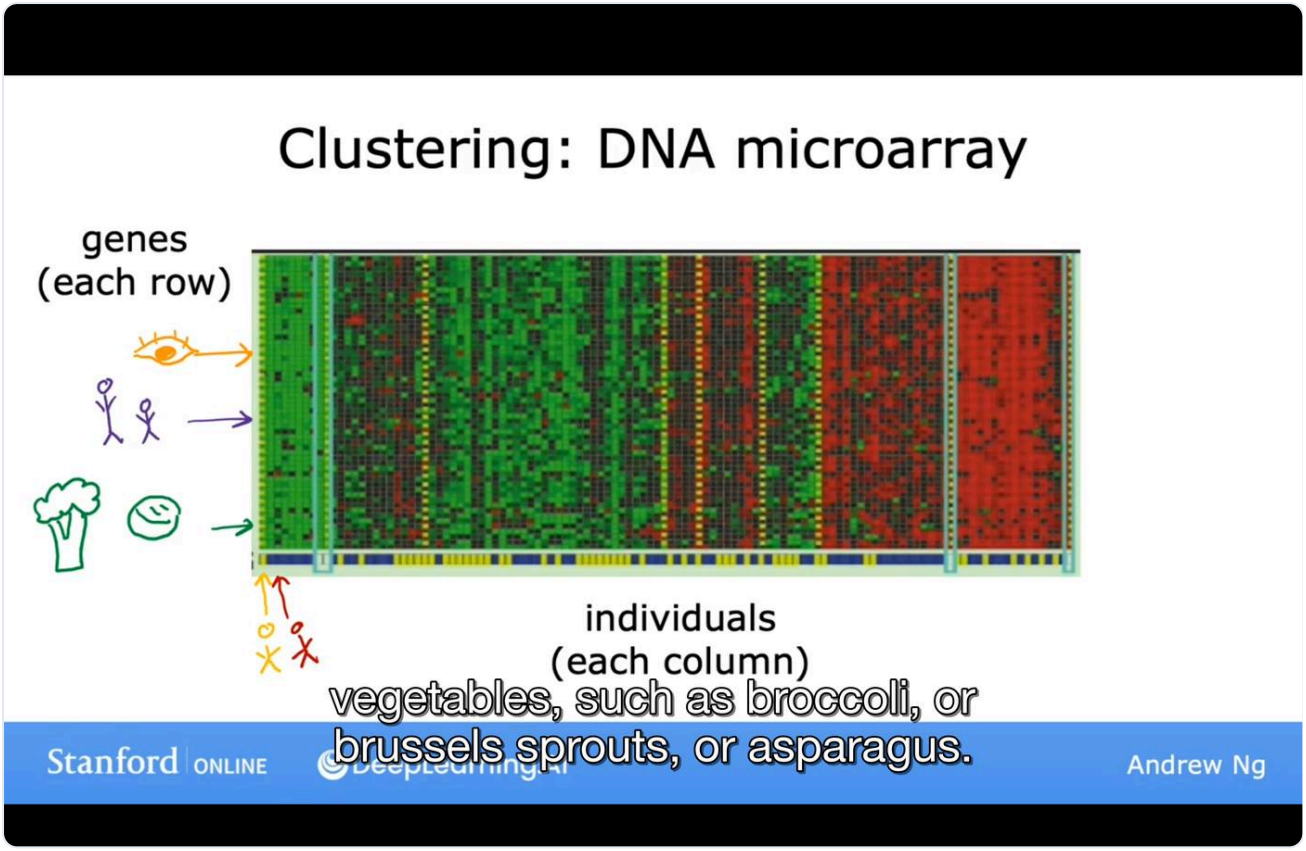


FIGURE 6.7

Researchers have even found a genetic link to whether someone dislikes certain vegetables, such as broccoli, or brussels sprouts, or asparagus. So next time someone asks you why didn't you finish your salad, you can tell them, maybe it's genetic for DNA micro race. The idea is to measure how much certain genes, are expressed for each individual person. So these colors red, green, gray, and so on, show the degree to which different individuals do, or do not have a specific gene active. And what you can do is then run a clustering algorithm to group individuals into different categories. Or different types of people like maybe these individuals that group together, and let's just call this type one.

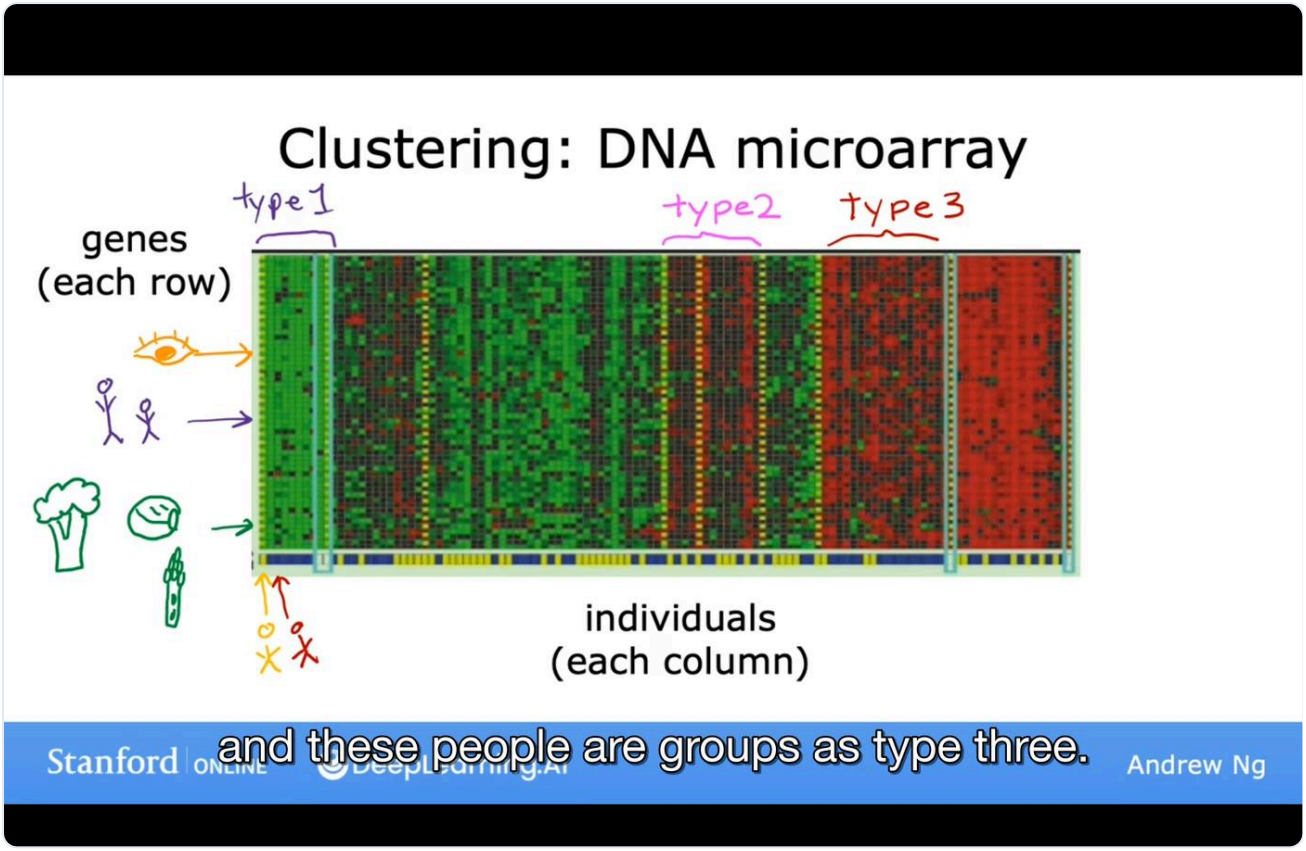


FIGURE 6.8

And these people are grouped into type two, and these people are groups as type three. This is unsupervised learning, because we're not telling the algorithm in advance, that there is a type one person with certain characteristics. Or a type two person with certain characteristics, instead what we're saying is here's a bunch of data. I don't know what the different types of people are but can you automatically find structure into data. And automatically figure out whether the major types of individuals, since we're not giving the algorithm the right answer for the examples in advance. This is unsupervised learning, here's the third example, many companies have huge databases of customer information given this data.

Can you automatically group your customers, into different market segments so that you can more efficiently serve your customers. Concretely the deep learning dot AI team did some research to better understand the deep learning dot AI community. And why different individuals take these classes, subscribed to the batch weekly newsletter, or attend our AI events. Let's visualize the deep learning dot AI community, as this collection of people running clustering. That is market segmentation found a few distinct groups of individuals, one group's primary motivation is seeking knowledge to grow their skills. Perhaps this is you, and so that's great, a second group's primary motivation is looking for a way to develop their career.

Maybe you want to get a promotion or a new job, or make some career progression if this describes you, that's great too.

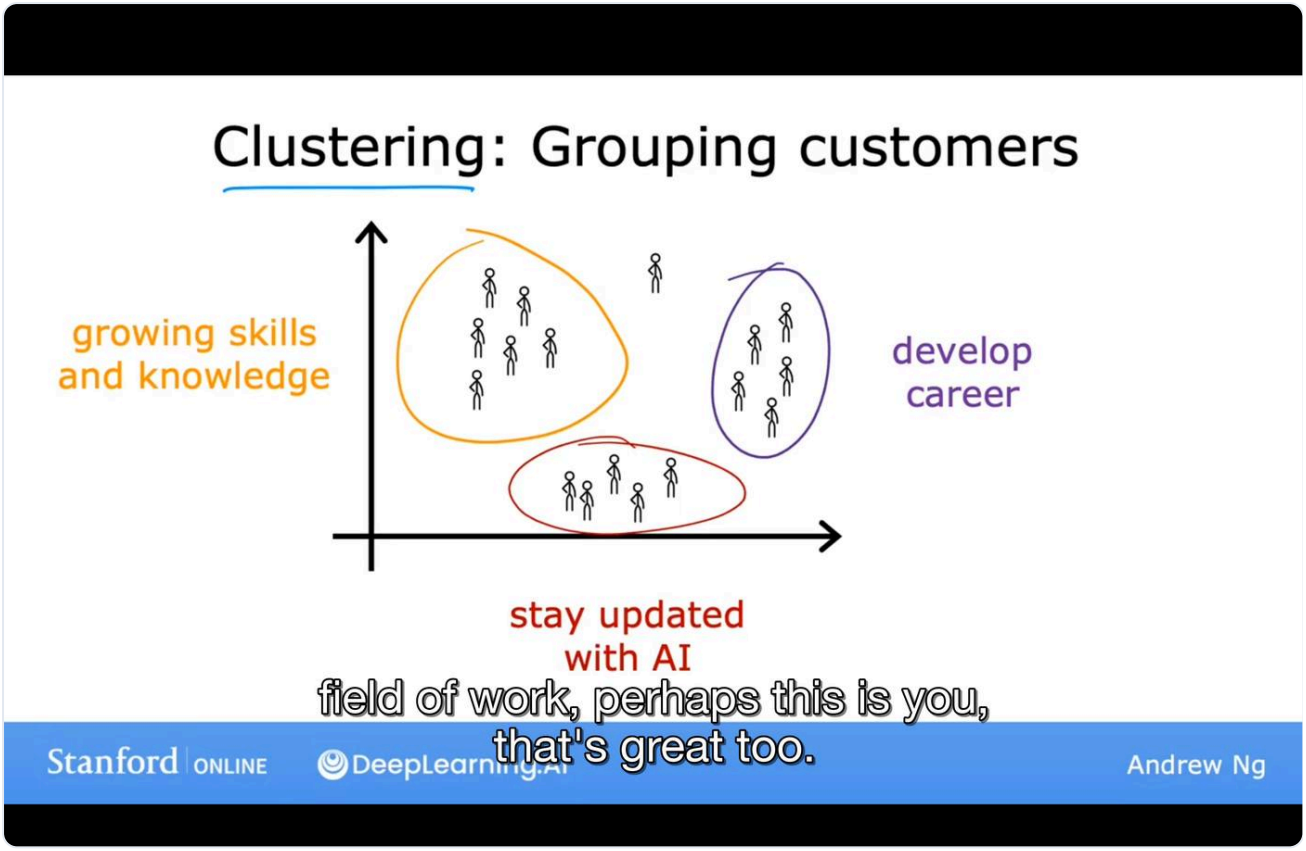


FIGURE 6.9 | Clustering: Grouping Customers

And yet another group wants to stay updated on how AI impacts their field of work, perhaps this is you, that's great too. This is a clustering that our team used to try to better serve our community as we're trying to figure out. Whether the major categories of learners in the deeper and community, So if any of these is your top motivation for learning, that's great. And I hope I'll be able to help you on your journey, or in case this is you, and you want something totally different than the other three categories. That's fine too, and I want you to know, I love you all the same, so to summarize a clustering algorithm. Which is a type of unsupervised learning algorithm, takes data without labels and tries to automatically group them into clusters.

And so maybe the next time you see or think of a panda, maybe you think of clustering as well. And besides clustering, there are other types of unsupervised learning as well. Let's go on to the next video, to take a look at some other types of unsupervised learning algorithms.

LECTURE 7

Unsupervised Learning

In the last video, you saw what is unsupervised learning, and one type of unsupervised learning called clustering. Let's give a slightly more formal definition of unsupervised learning and take a quick look at some other types of unsupervised learning other than clustering. Whereas in supervised learning, the data comes with both inputs x and input labels y, in unsupervised learning, the data comes only with inputs x but not output labels y, and the algorithm has to find some structure or some pattern or something interesting in the data. We're seeing just one example of unsupervised learning called a clustering algorithm, which groups similar data points together.

In this specialization, you'll learn about clustering as well as two other types of unsupervised learning. One is called anomaly detection, which is used to detect unusual events. This turns out to be really important for fraud detection in the financial system, where unusual events, unusual transactions could be signs of fraud and for many other applications. You also learn about dimensionality reduction.

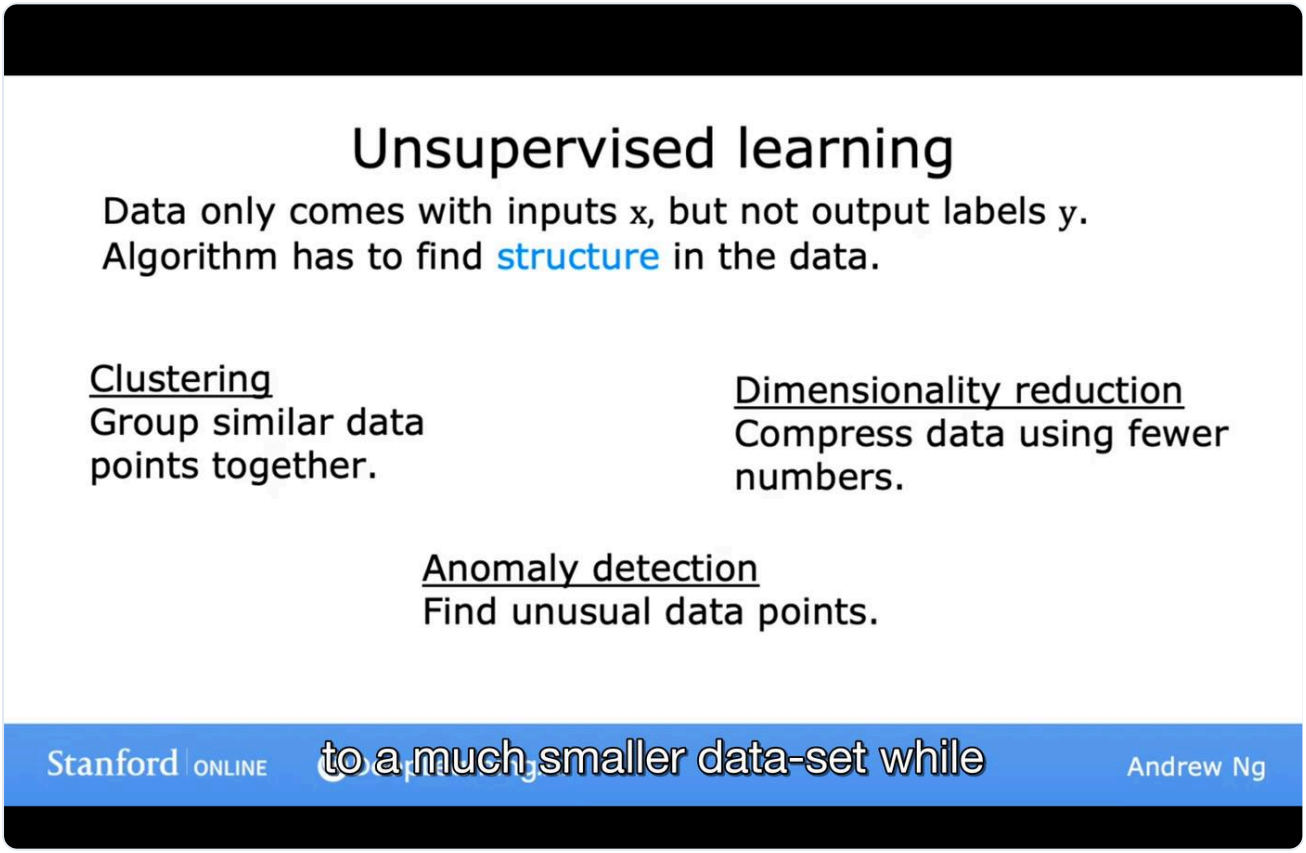


FIGURE 7.1 | Unsupervised Learning

This lets you take a big data-set and almost magically compress it to a much smaller data-set while losing as little information as possible. In case anomaly detection and dimensionality reduction don't seem to make too much sense to you yet. Don't worry about it. We'll get to this later in the specialization. Now, I'd like to ask you another question to help you check your understanding, and no pressure, if you don't get it right on the first try, is totally fine. Please select any

of the following that you think are examples of unsupervised learning. Two are unsupervised examples and two are supervised learning examples. Please take a look. Maybe you remember the spam filtering problem.

If you have labeled data you now label as spam or non-spam e-mail, you can treat this as a supervised learning problem. The second example, the news story example. That's exactly the Google News and tangible example that you saw in the last video. You can approach that using a clustering algorithm to group news articles together. That we'll use unsupervised learning. The market segmentation example that I talked about a little bit earlier. You can do that as an unsupervised learning problem as well because you can give your algorithm some data and ask it to discover market segments automatically. The final example on diagnosing diabetes. Well, actually that's a lot like our breast cancer example from the supervised learning videos.

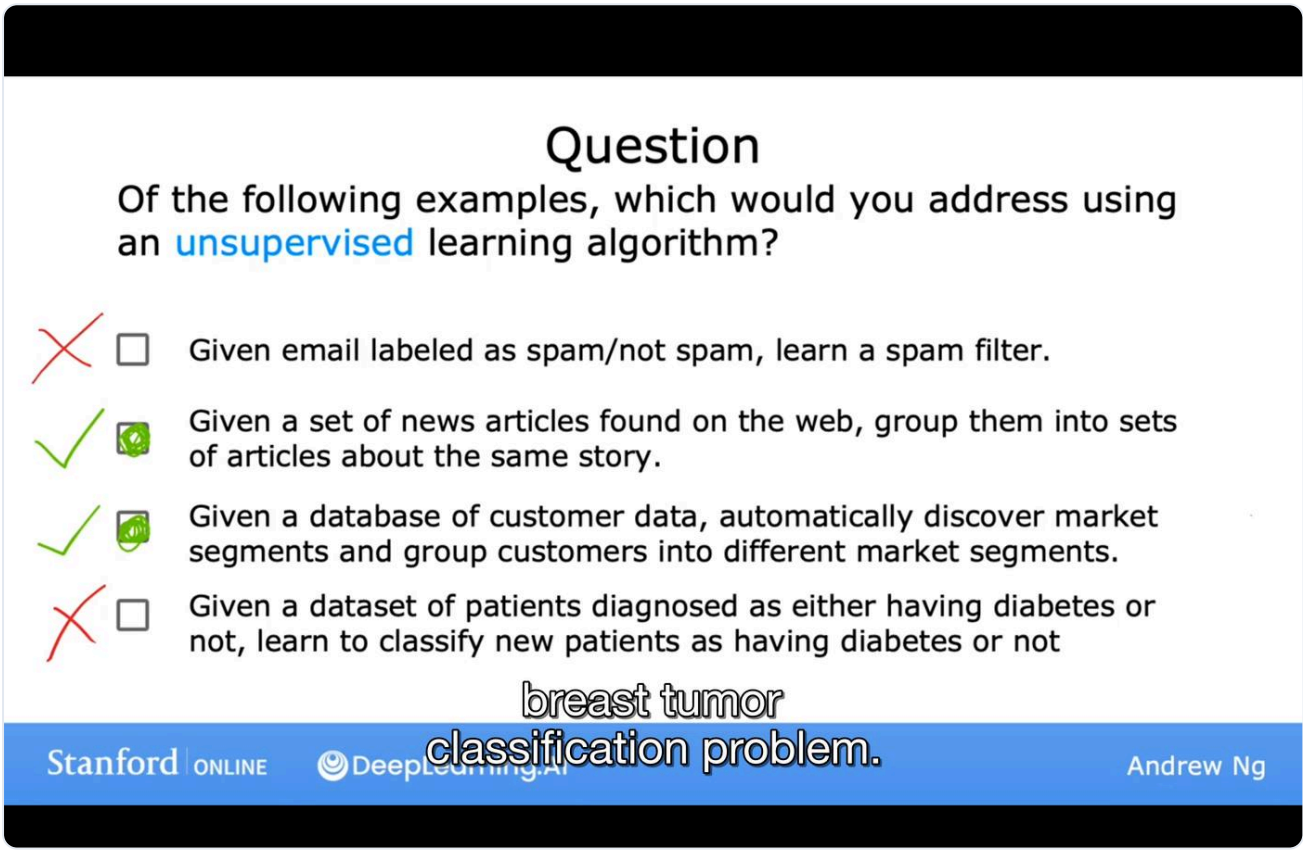


FIGURE 7.2 | Of the Following Examples, Which Would You Address Using

Only instead of benign or malignant tumors, we instead have diabetes or not diabetes. You can approach this as a supervised learning problem, just like we did for the breast tumor classification problem. Even though in the last video, we've talked mainly about clustering, in later videos, in this specialization, we'll dive much more deeply into anomaly detection and dimensionality reduction as well. That's unsupervised learning. Before we wrap up this section, I want to share with you something that I find really exciting, and useful, which is the use of Jupyter Notebooks in machine learning. Let's take a look at that in the next video.

LECTURE 8

Jupyter Notebook

From the videos, you've seen supervised learning and unsupervised learning and also examples of both. For you to more deeply understand these concepts, I'll like to invite you in this class to see, learn and maybe later write codes yourself to implement these concepts. The most widely used tool by machine learning and data science practitioners today is the Jupyter Notebook. This is the default environments that a lot of us use to code up and experiment and try things out. In this class, right here in your web browser, you build a user Jupyter Notebook environment to test out some of these ideas for yourself as well. This is not some made up simplified environment, this is the exact same environments, the exact same tool, the Jupyter Notebook that developers are using in many large countries right now.

One type of lab that you see throughout this class are optional labs, which are ones you can open and run one line at a time with usually no need to write any code yourself. Optional labs are designed to be very easy and I can guarantee you will get full marks, every single one of them because there are no marks. All you need to do is open it up and just run the code we've provided. By reading through and running the code in the optional labs, you see how machine learning code runs. You should complete them relatively quickly just by running it one line at a time from top to bottom. Optional labs are completely optional so you don't have to do them at all if you don't want to, but I hope you will take a look because running through them will give you a deeper feel, give you a little bit more experience with what machine learning algorithms, what machine learning code actually looks like.

Starting next week, there'll also be some practice labs which would give you an opportunity to write some of that code yourself but we'll get to that next week. Don't worry about it for now and I hope you just go through the next optional lab and get through the rest of the content for this week. Let's take a look at an example of a notebook. Here's what you see when you go to the first optional lab. Feel free to scroll up and down and browse and mouseover the different menus and take a look at the different options here. You might notice that there are two types of these blocks, also called cells in the notebook and there are two types of cells. One is what's called a Markdown cell, which means a bunch of tax.

Here you can actually edit the text if you don't like the text that we wrote, but this is text that describes the code. Then there's a second type of block or cell which looks like this, which has a code cell. Here, we've already provided the code and if you want to run this code cell, hitting Shift Enter will run the code in this code cell, and by the way, if you click on a markdown cell, so this showing all this formatting, go ahead and hit Shift Enter on your keyboard as well and that will also convert back to this nicely formatted text. This optional lab shows some common Python code, so you can go ahead and run this afterwards in your own Jupyter notebook.

When you jump into this notebook yourself, what I'd like you to do is select the cells and hit Shift Enter. Read through the code, see if it makes sense, try to make a prediction about what you think this code would do and then hit Shift Enter and then see what the code actually does, and if

you like it, feel free to go in and edit the code, change the code, and then run it and see what happens. If you haven't played in the Jupyter Notebook environment for, I hope you become more familiar with Python in a Jupyter Notebook. I spend a lot of hours playing around in Jupyter notebooks and so I hope you have fun with them too. After that, I look forward to seeing you in the next video where we'll take the supervised learning problem as start to flesh out our first supervised learning algorithm.

I hope that will be fun to you and look forward to seeing you there.

LECTURE 9

House Sizes and Prices

In this video, we'll look at what the overall process of supervised learning is like. Specifically, you see the first model of this course, Linear Regression Model. That just means fitting a straight line to your data. It's probably the most widely used learning algorithm in the world today.

FIGURE 9.1 | House Sizes and Prices

As you get familiar with linear regression, many of the concepts you see here will also apply to other machine learning models that you'll see later in this specialization. Let's start with a problem that you can address using linear regression. Say you want to predict the price of a house based on the size of the house. This is the example we've seen earlier this week. We're going to use a dataset on house sizes and prices from Portland, a city in the United States.

FIGURE 9.2 | House Sizes and Prices

Here we have a graph where the horizontal axis is the size of the house in square feet, and the vertical axis is the price of a house in thousands of dollars. Let's go ahead and plot the data points for various houses in the dataset.

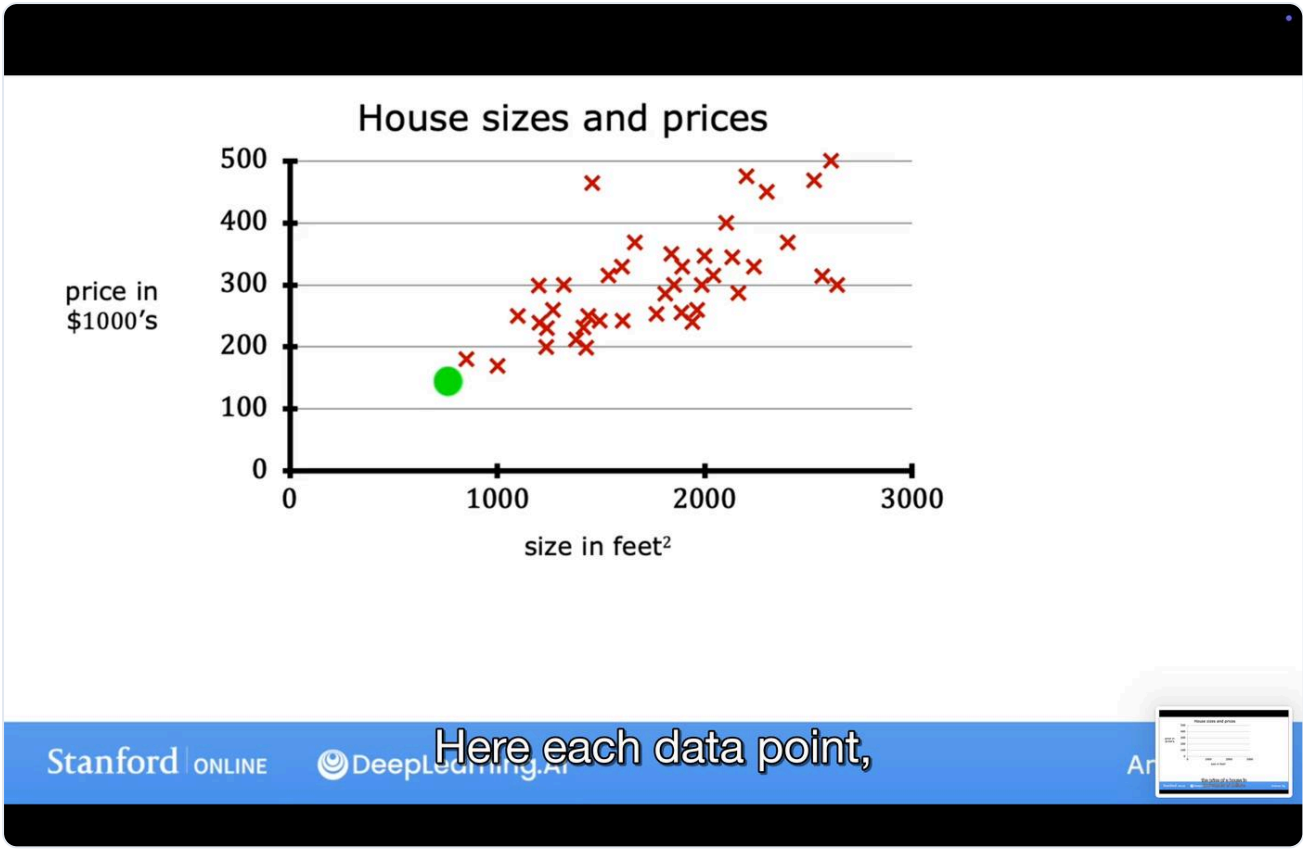


FIGURE 9.3 | House Sizes and Prices

Here each data point, each of these little crosses is a house with the size and the price that it most recently was sold for. Now, let's say you're a real estate agent in Portland and you're helping a client to sell her house. She is asking you, how much do you think I can get for this house? This dataset might help you estimate the price she could get for it. You start by measuring the size of the house, and it turns out that the house is 1250 square feet. How much do you think this house could sell for? One thing you could do this, you can build a linear regression model from this dataset. Your model will fit a straight line to the data, which might look like this.

FIGURE 9.4 | House Sizes and Prices

Based on this straight line fit to the data, you can see that the house is 1250 square feet, it will intersect the best fit line over here, and if you trace that to the vertical axis on the left, you can see the price is maybe around here, say about $220,000.

FIGURE 9.5 | House Sizes and Prices

This is an example of what's called a supervised learning model.

FIGURE 9.6 | House Sizes and Prices

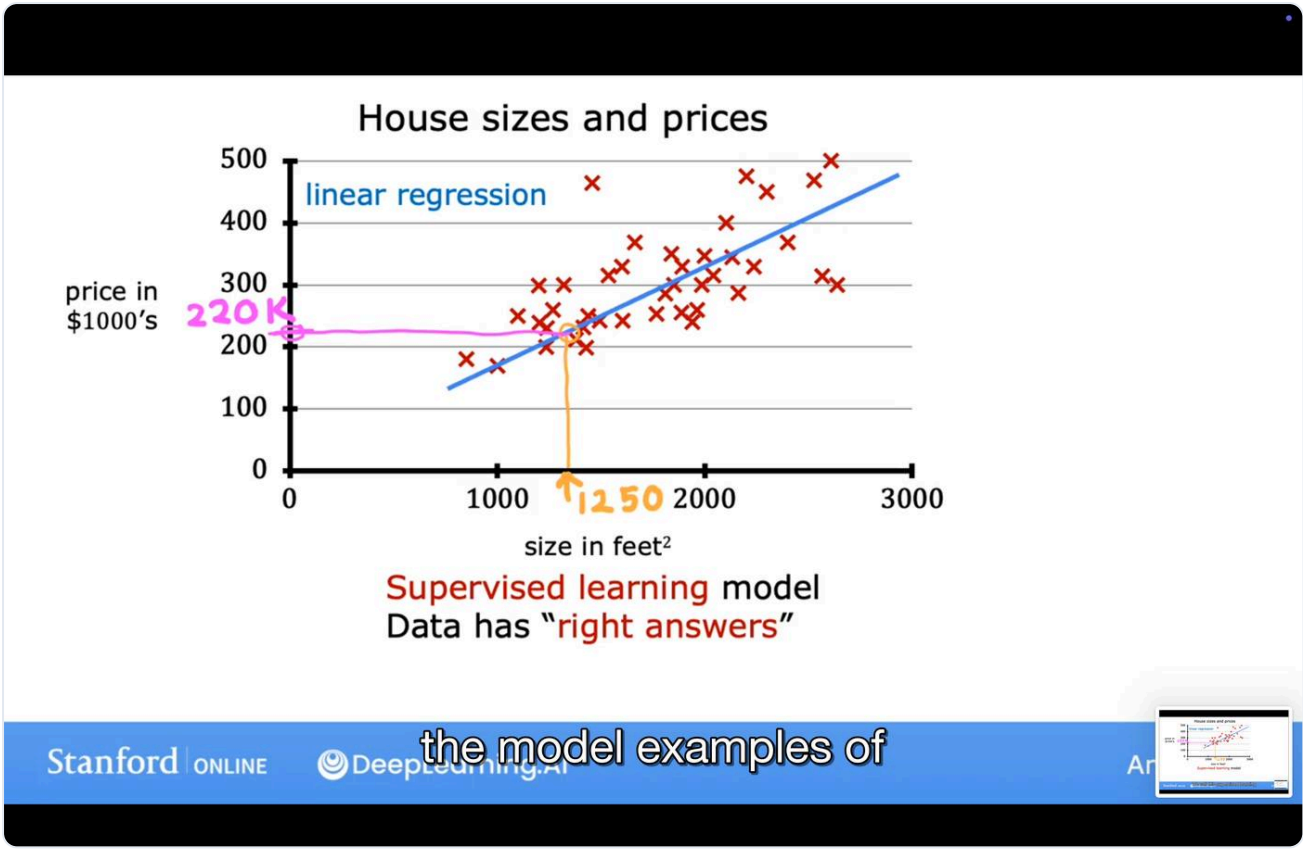


FIGURE 9.7 | House Sizes and Prices

FIGURE 9.8 | House Sizes and Prices

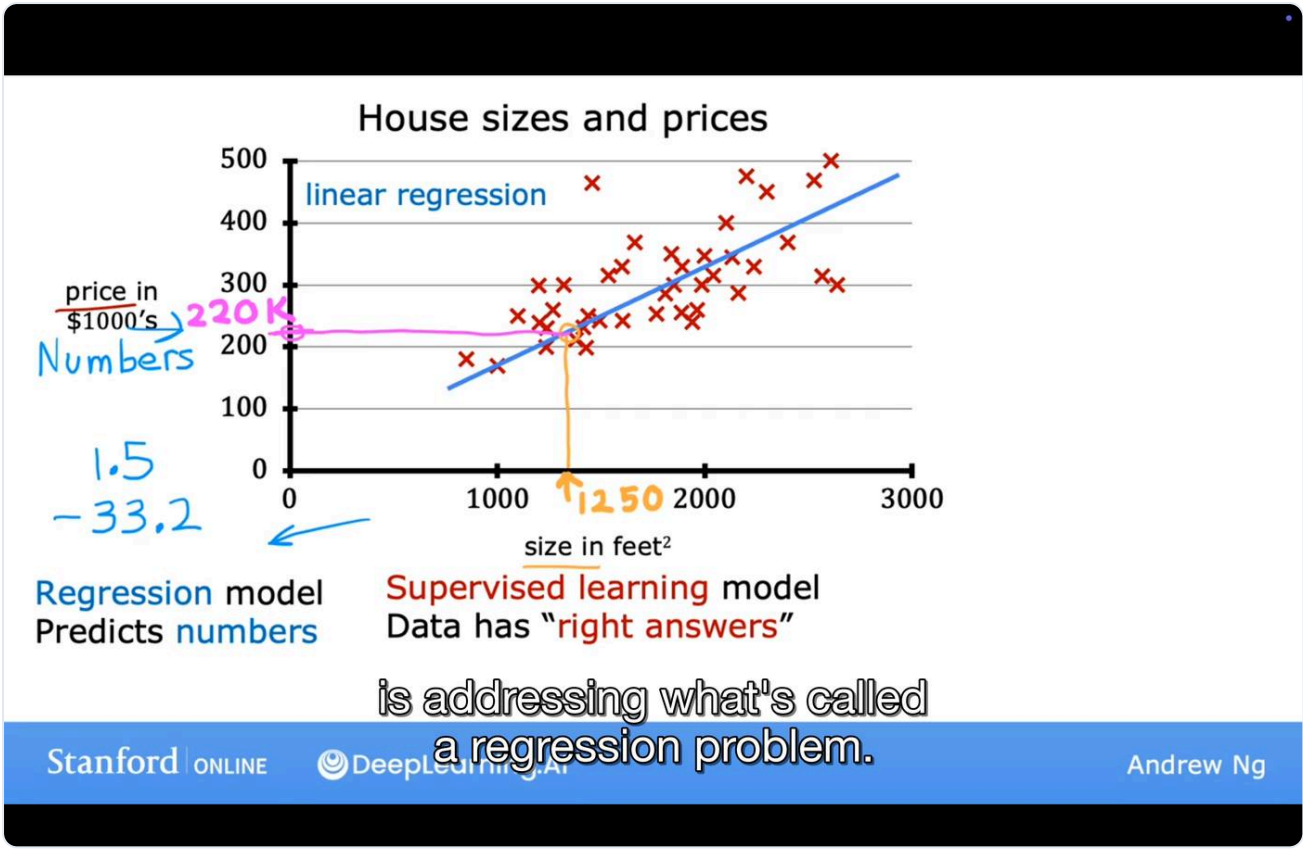


FIGURE 9.9 | House Sizes and Prices

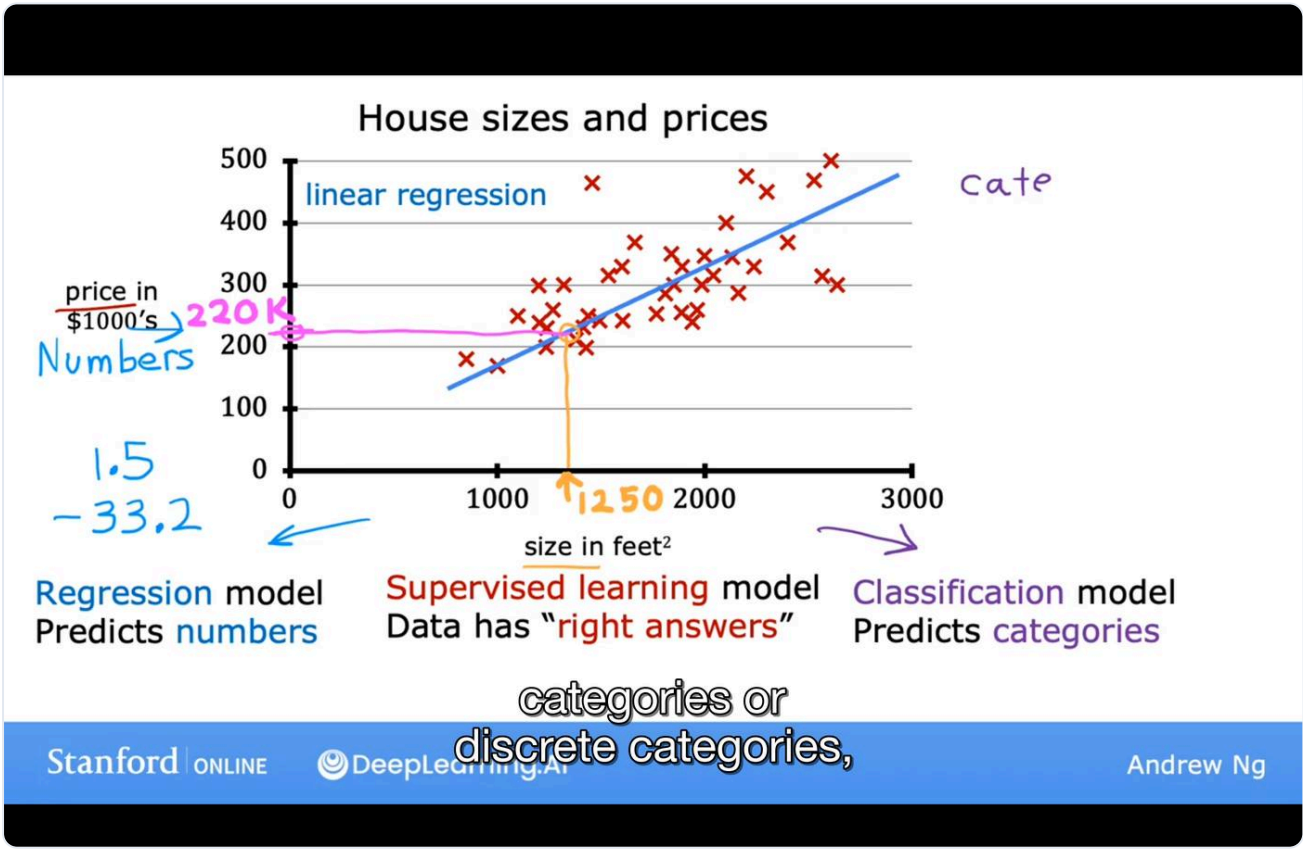


FIGURE 9.10 | House Sizes and Prices

We call this supervised learning because you are first training a model by giving a data that has right answers because you get the model examples of houses with both the size of the house, as well as the price that the model should predict for each house. Well, here are the prices, that is, the right answers are given for every house in the dataset. This linear regression model is a particular type of supervised learning model. It's called regression model because it predicts numbers as the output like prices in dollars. Any supervised learning model that predicts a number such as 220,000 or 1.5 or negative 33.2 is addressing what's called a regression problem.

Linear regression is one example of a regression model. But there are other models for addressing regression problems too. We'll see some of those later in Course 2 of this specialization. Just to remind you, in contrast with the regression model, the other most common type of supervised learning model is called a classification model. Classification model predicts categories or discrete categories, such as predicting if a picture is of a cat, meow or a dog, woof, or if given medical record, it has to predict if a patient has a particular disease. You'll see more about classification models later in this course as well.

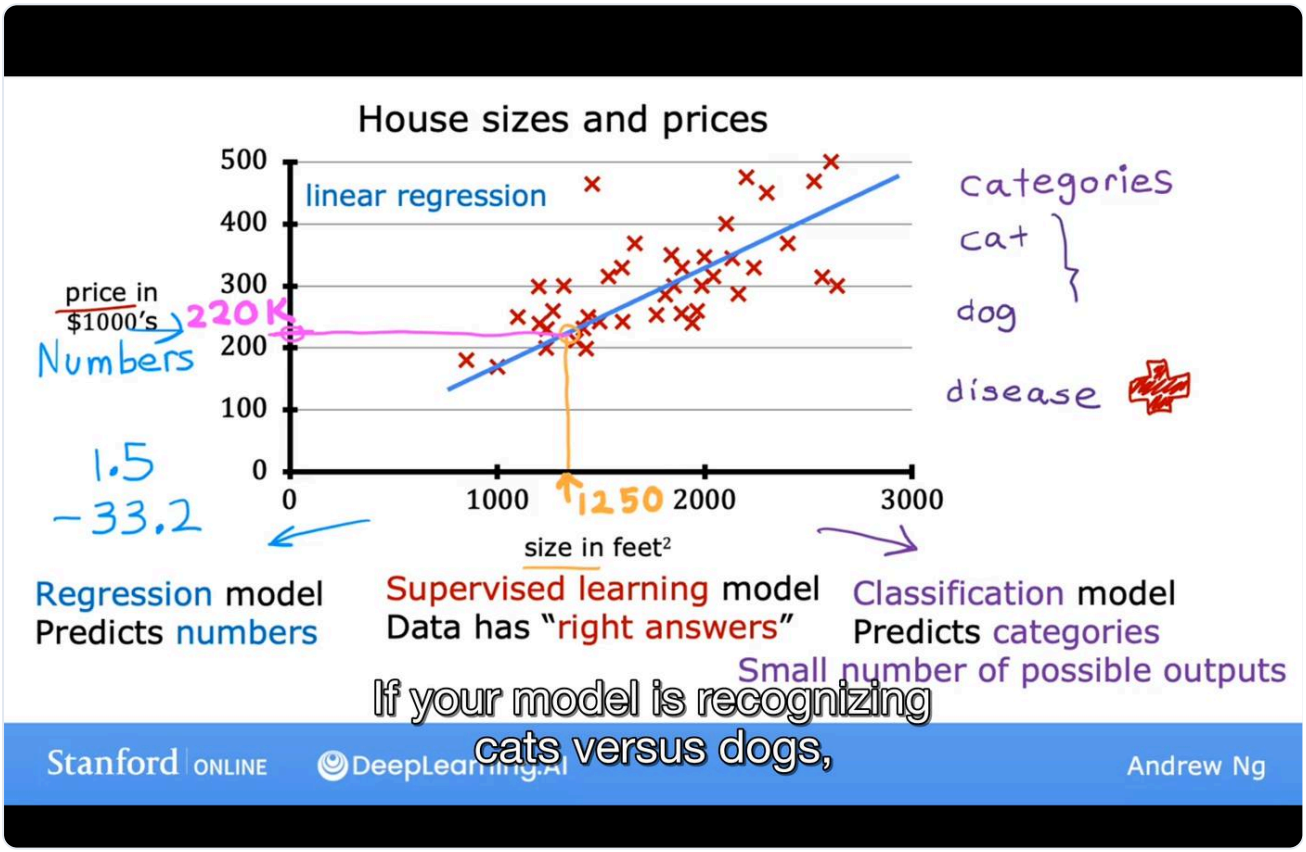


FIGURE 9.11 | House Sizes and Prices

As a reminder about the difference between classification and regression, in classification, there are only a small number of possible outputs. If your model is recognizing cats versus dogs, that's two possible outputs. Or maybe you're trying to recognize any of 10 possible medical conditions in a patient, so there's a discrete, finite set of possible outputs.

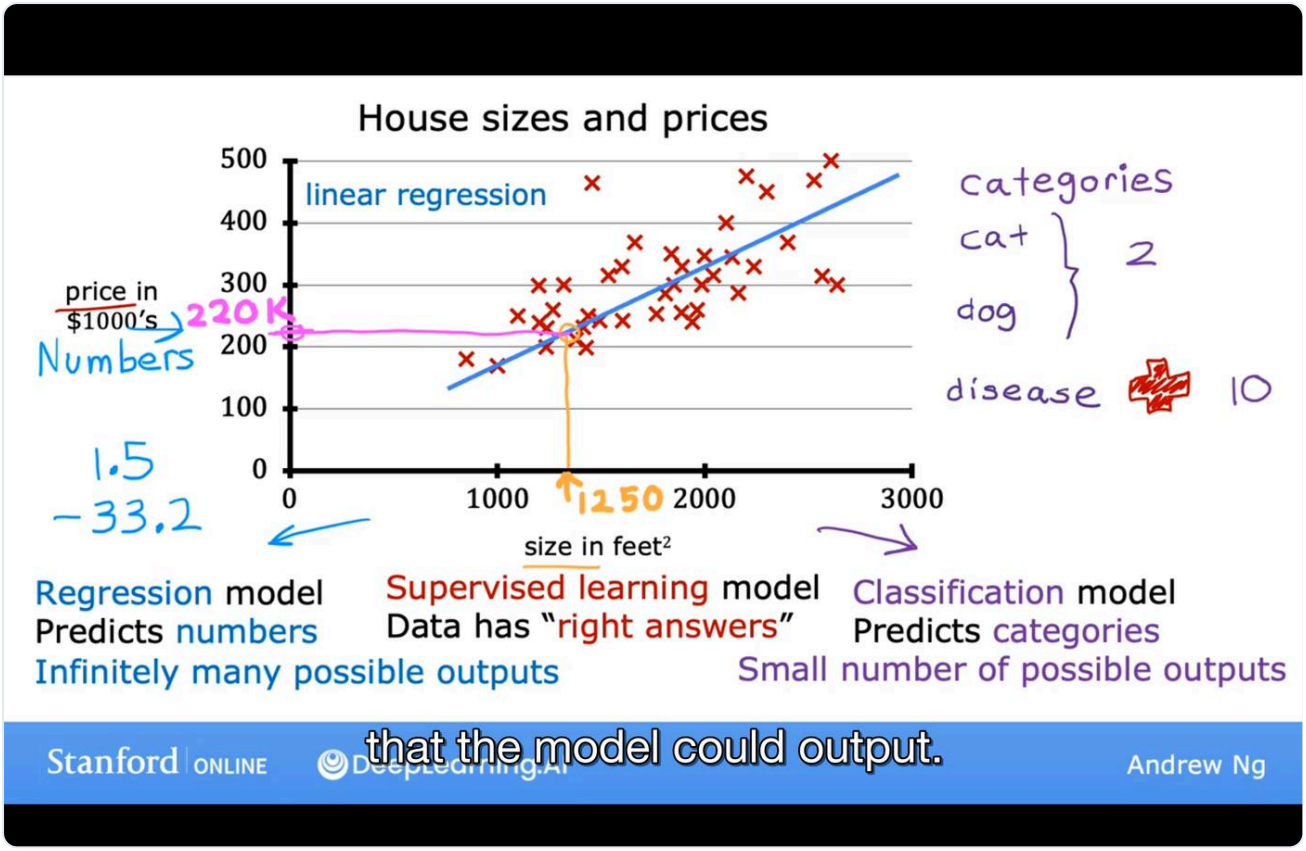


FIGURE 9.12 | House Sizes and Prices

We call it classification problem, whereas in regression, there are infinitely many possible numbers that the model could output. In addition to visualizing this data as a plot here on the left, there's one other way of looking at the data that would be useful, and that's a data table here on the right. The data comprises a set of inputs. This would be the size of the house, which is this column here. It also has outputs.

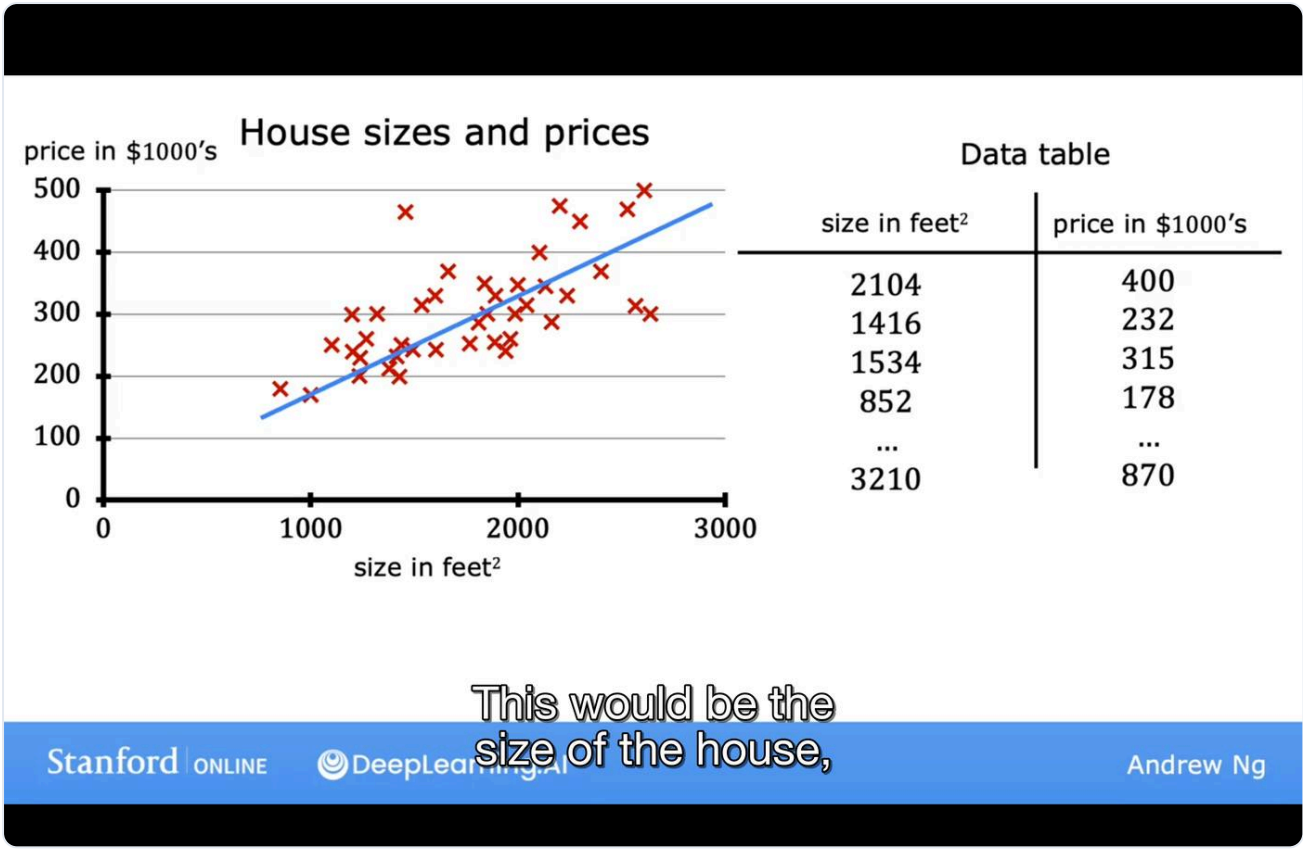


FIGURE 9.13

You're trying to predict the price, which is this column here. Notice that the horizontal and vertical axes correspond to these two columns, the size and the price. If you have, say, 47 rows in this data table, then there are 47 of these little crosses on the plot of the left, each cross corresponding to one row of the table. For example, the first row of the table is a house with size, 2,104 square feet, so that's around here, and this house is sold for $400,000 which is around here. This first row of the table is plotted as this data point over here.

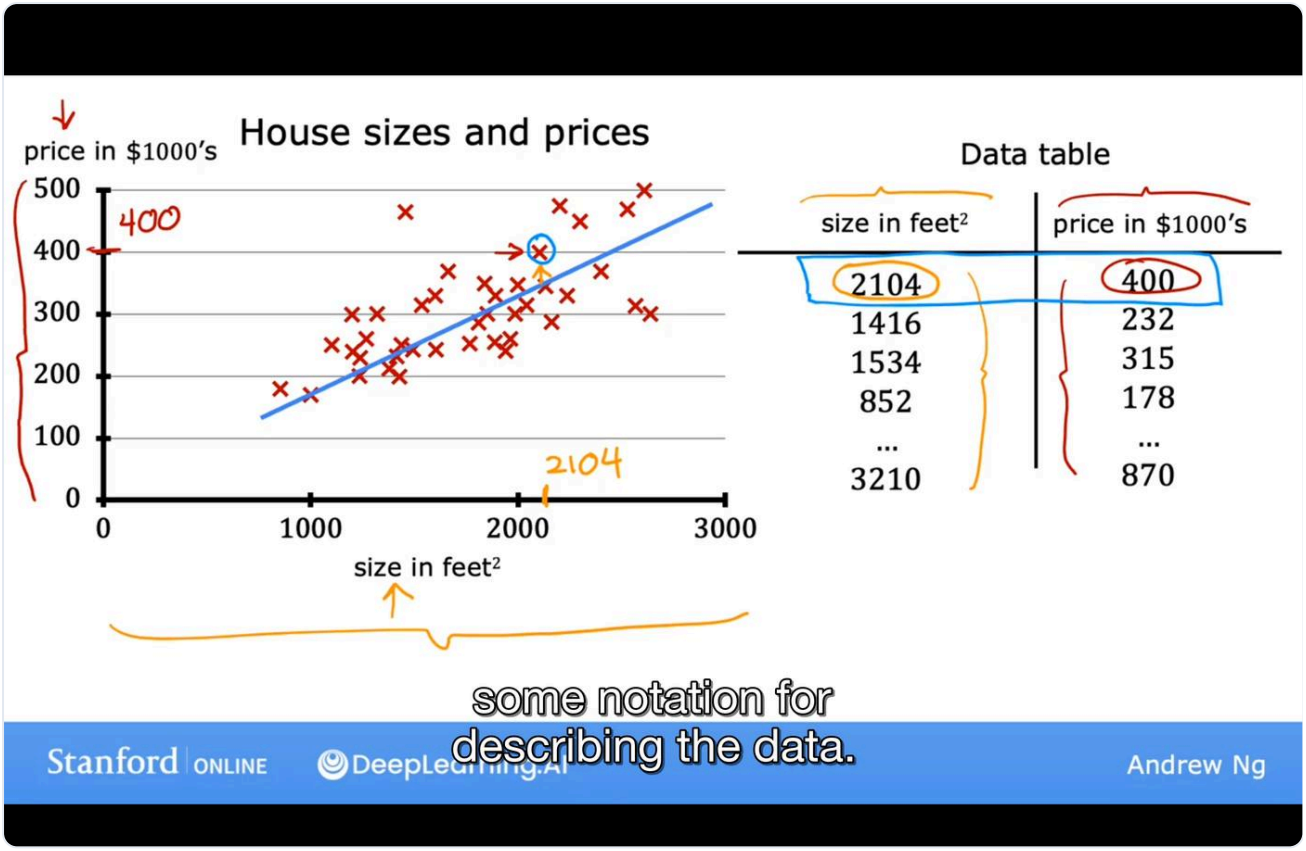


FIGURE 9.14

Now, let's look at some notation for describing the data. This is notation that you find useful throughout your journey in machine learning. As you increasingly get familiar with machine learning terminology, this would be terminology they can use to talk about machine learning concepts with others as well since a lot of this is quite standard across AI, you'll be seeing this notation multiple times in this specialization, so it's okay if you don't remember everything for assign through, it will naturally become more familiar overtime.

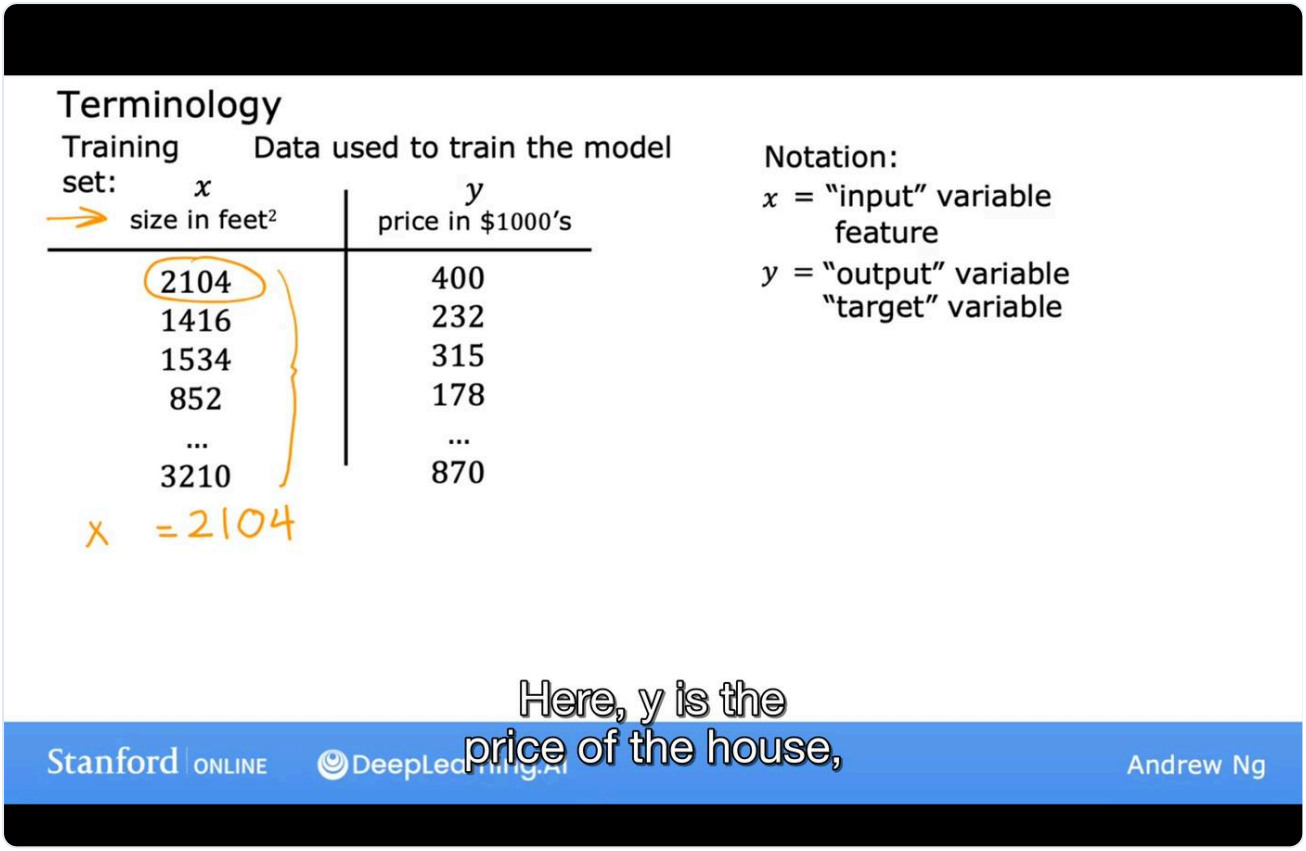


FIGURE 9.15 | Training Data Used to Train the Model Notation:

The dataset that you just saw and that is used to train the model is called a training set. Note that your client's house is not in this dataset because it's not yet sold, so no one knows what the price is. To predict the price of your client's house, you first train your model to learn from the training set and that model can then predict your client's houses price. In Machine Learning, the standard notation to denote the input here is lowercase x, and we call this the input variable, is also called a feature or an input feature. For example, for the first house in your training set, x is the size of the house, so x equals 2,104. The standard notation to denote the output variable which you're trying to predict, which is also sometimes called the target variable, is lowercase y.

Here, y is the price of the house, and for the first training example, this is equal to 400, so y equals 400.

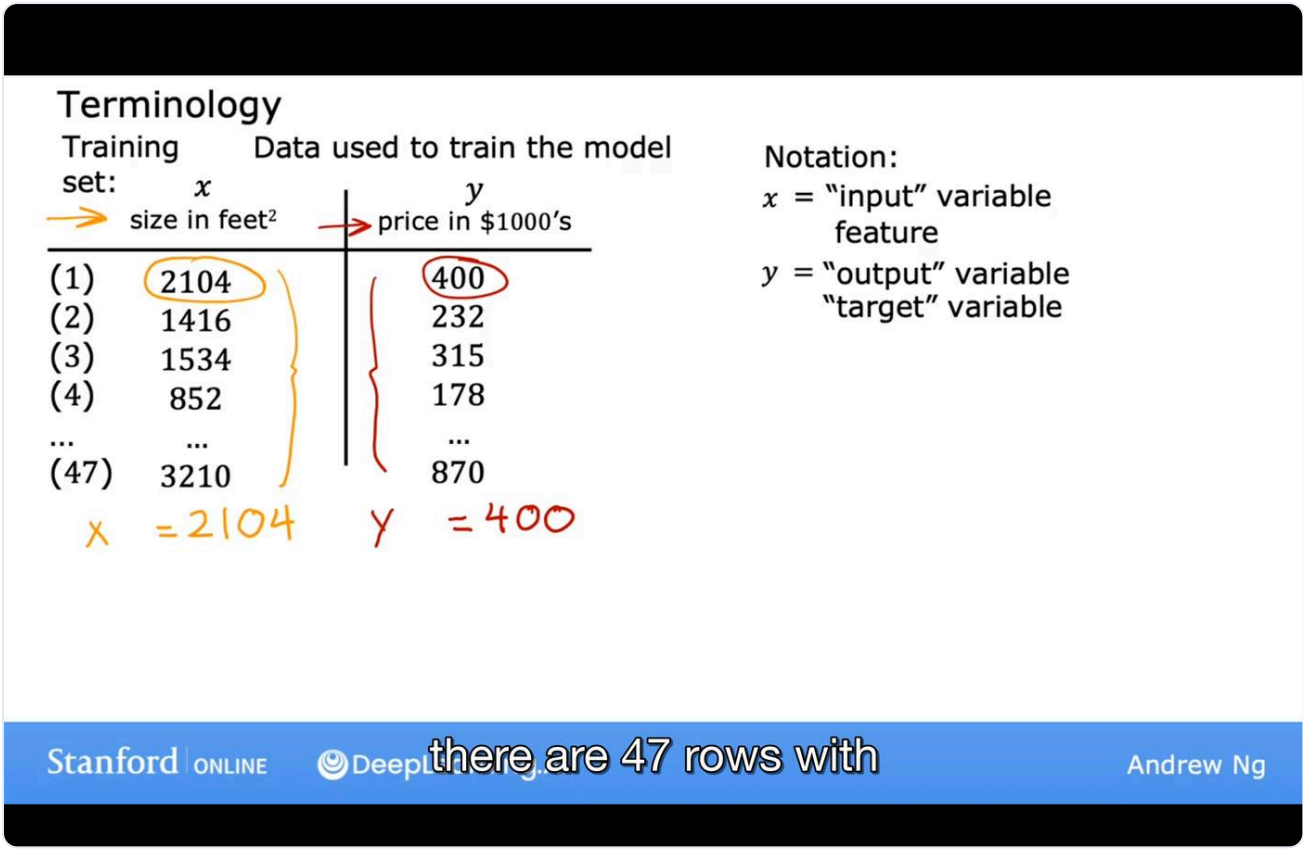


FIGURE 9.16 | Training Data Used to Train the Model Notation:

The dataset has one row for each house and in this training set, there are 47 rows with each row representing a different training example.

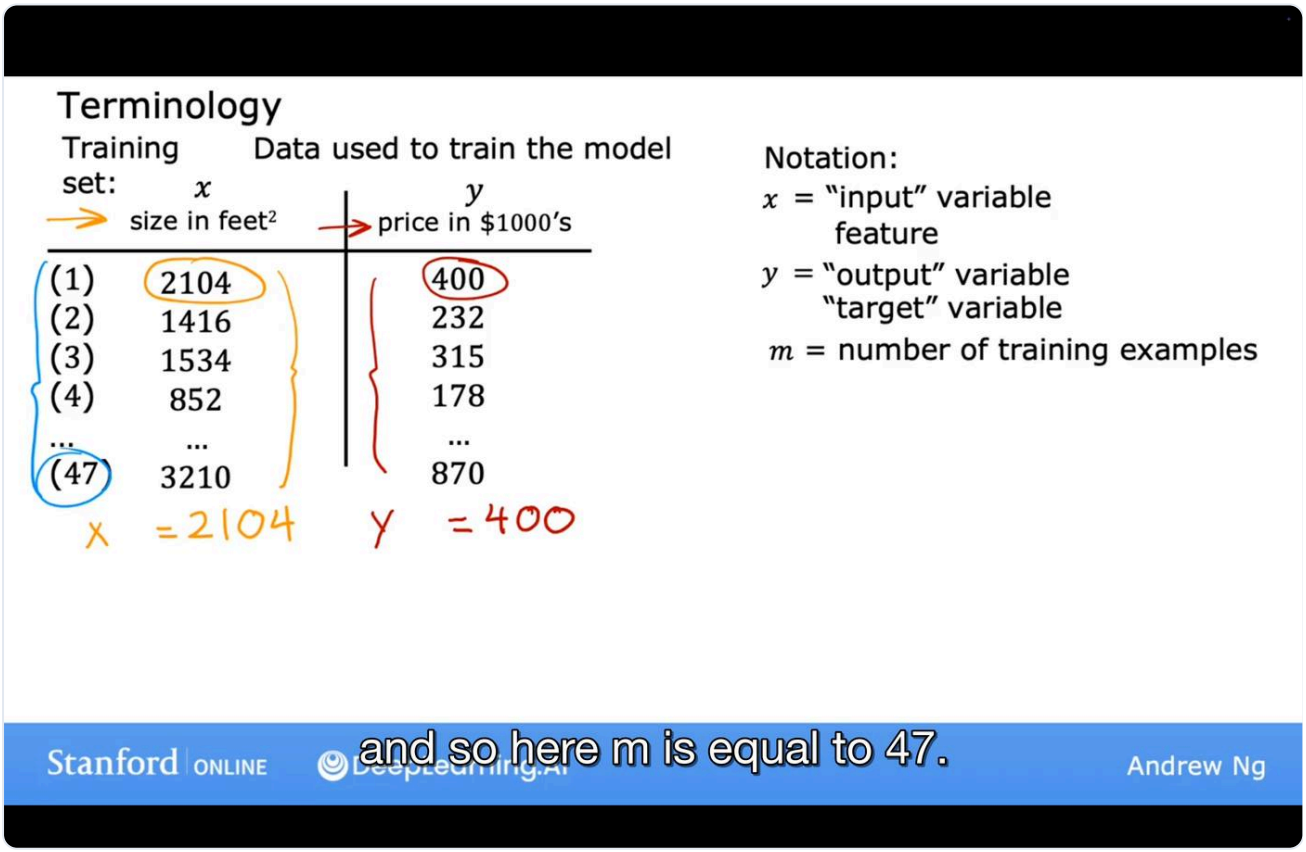


FIGURE 9.17 | Training Data Used to Train the Model Notation:

We're going to use lowercase m to refer it to the total number of training examples, and so here m is equal to 47.

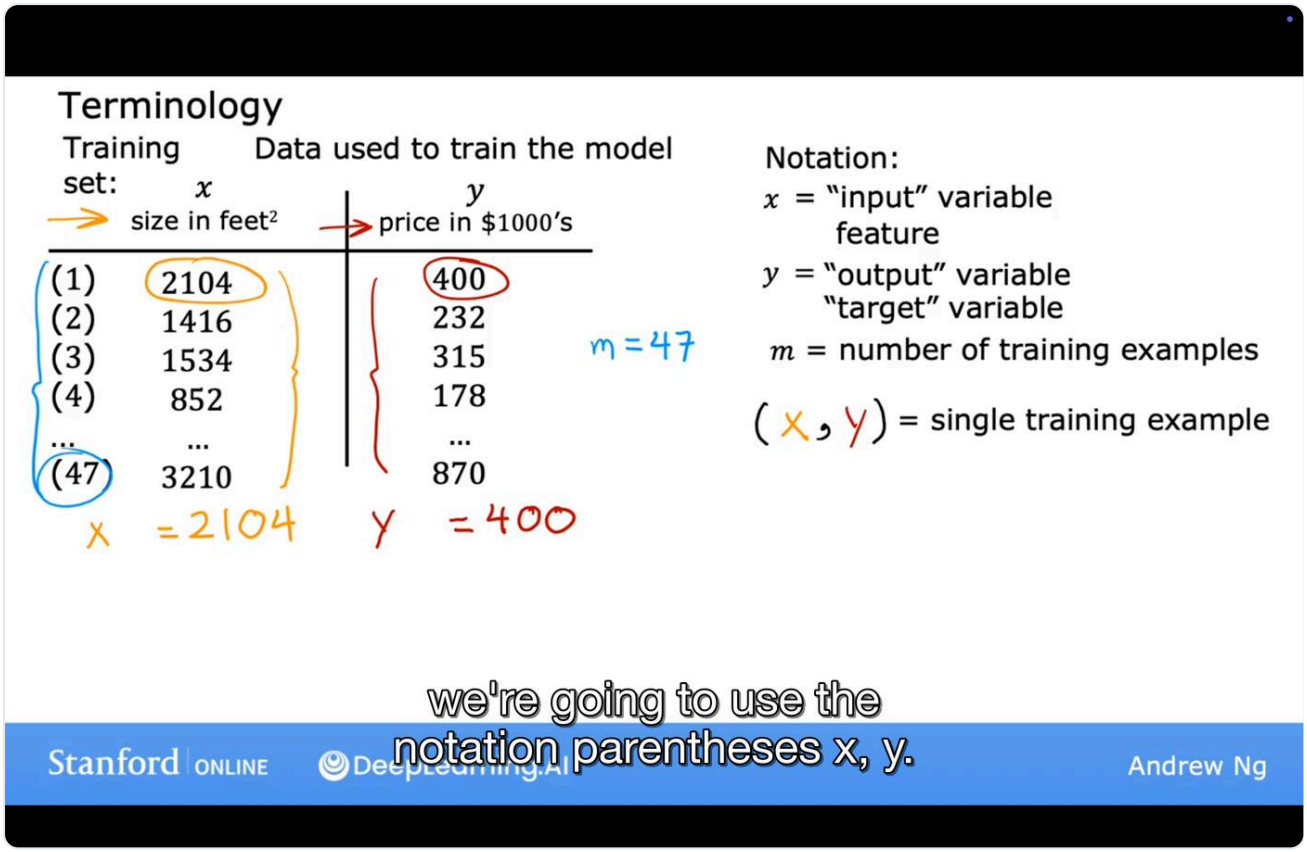


FIGURE 9.18 | Training Data Used to Train the Model Notation:

To indicate the single training example, we're going to use the notation parentheses x, y. For the first training example, (x, y), this pair of numbers is (2104, 400).

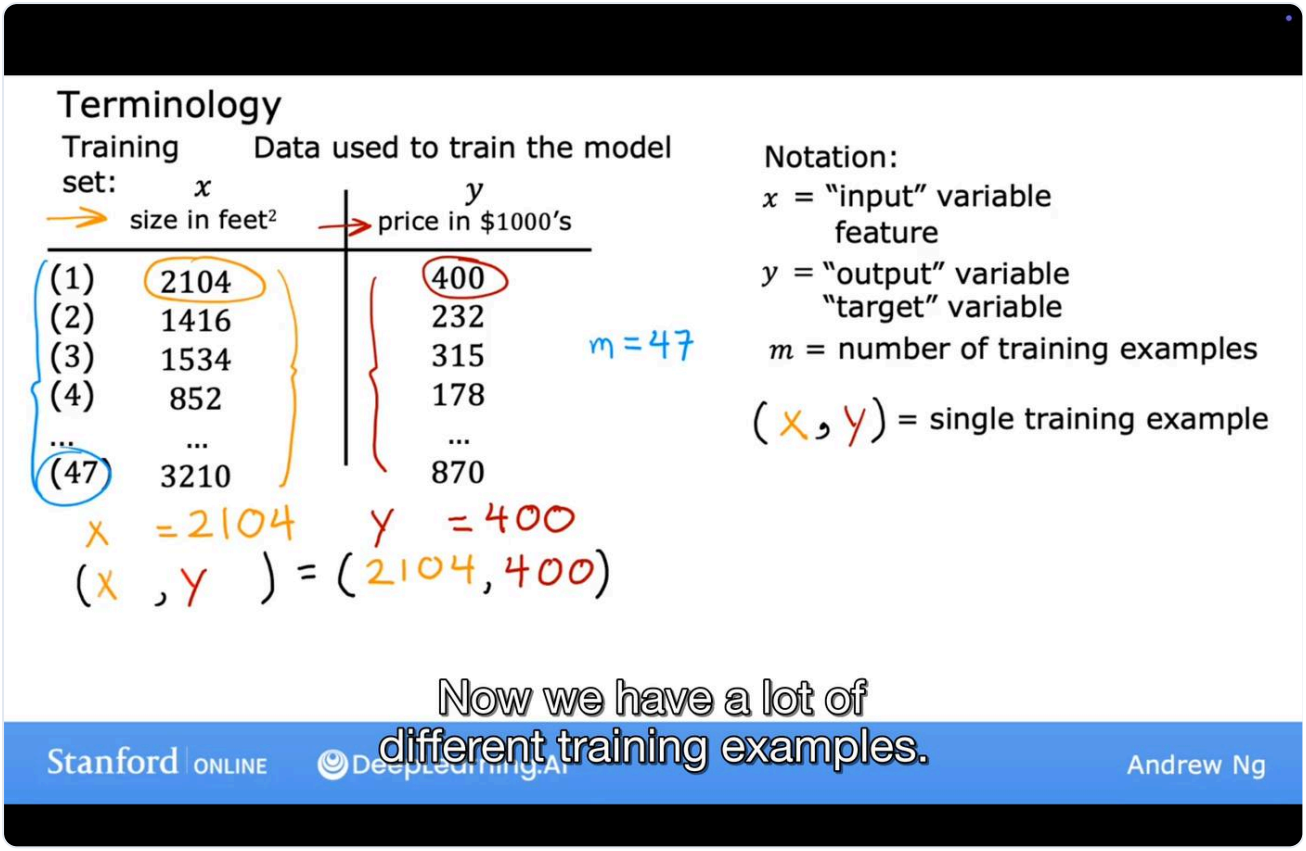


FIGURE 9.19 | Training Data Used to Train the Model Notation:

Now we have a lot of different training examples. We have 47 of them in fact. To refer to a specific training example, this will correspond to a specific row in this table on the left, I'm going to use the notation x superscript in parenthesis, i, y superscript in parentheses i.

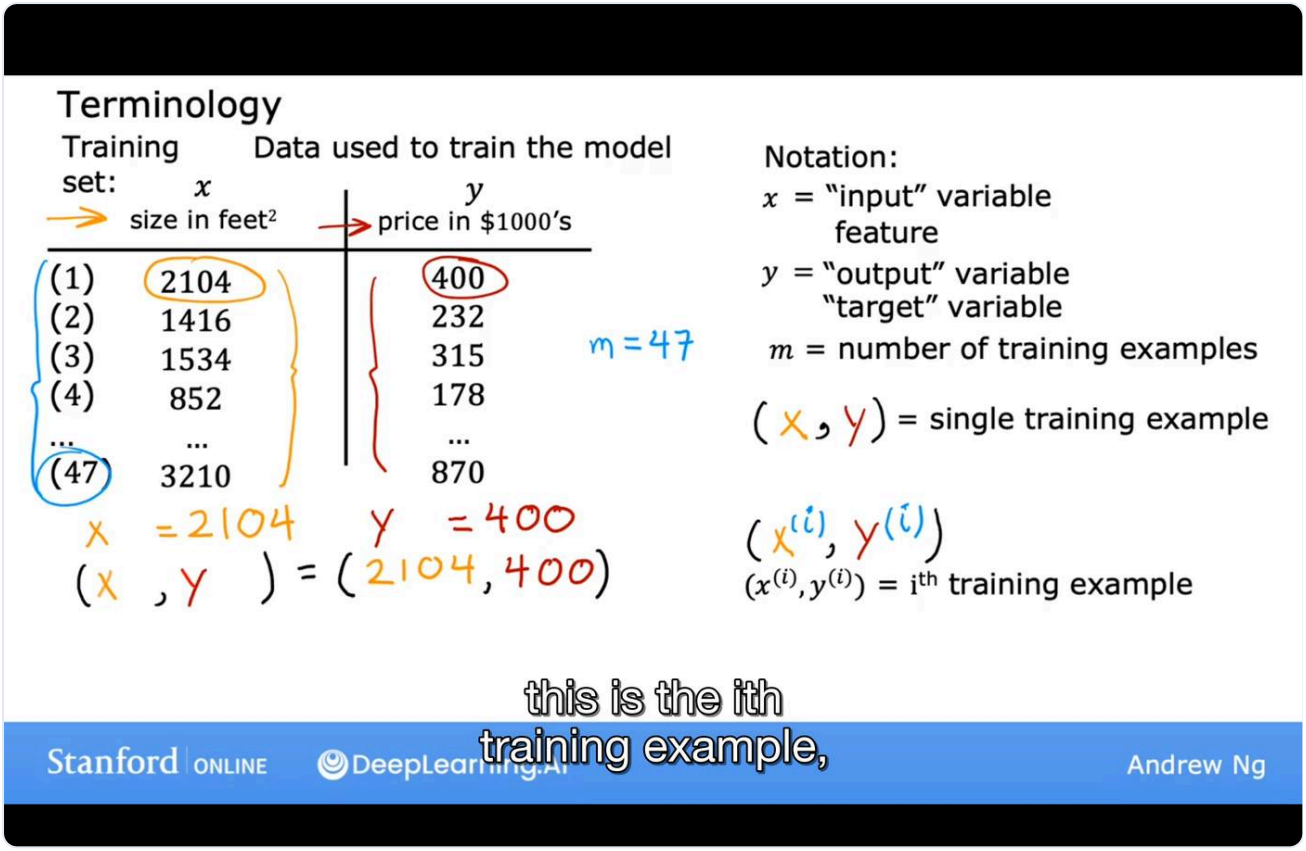


FIGURE 9.20 | Training Data Used to Train the Model Notation:

The superscript tells us that this is the ith training example, such as the first, second, or third up to the 47th training example. I here, refers to a specific row in the table.

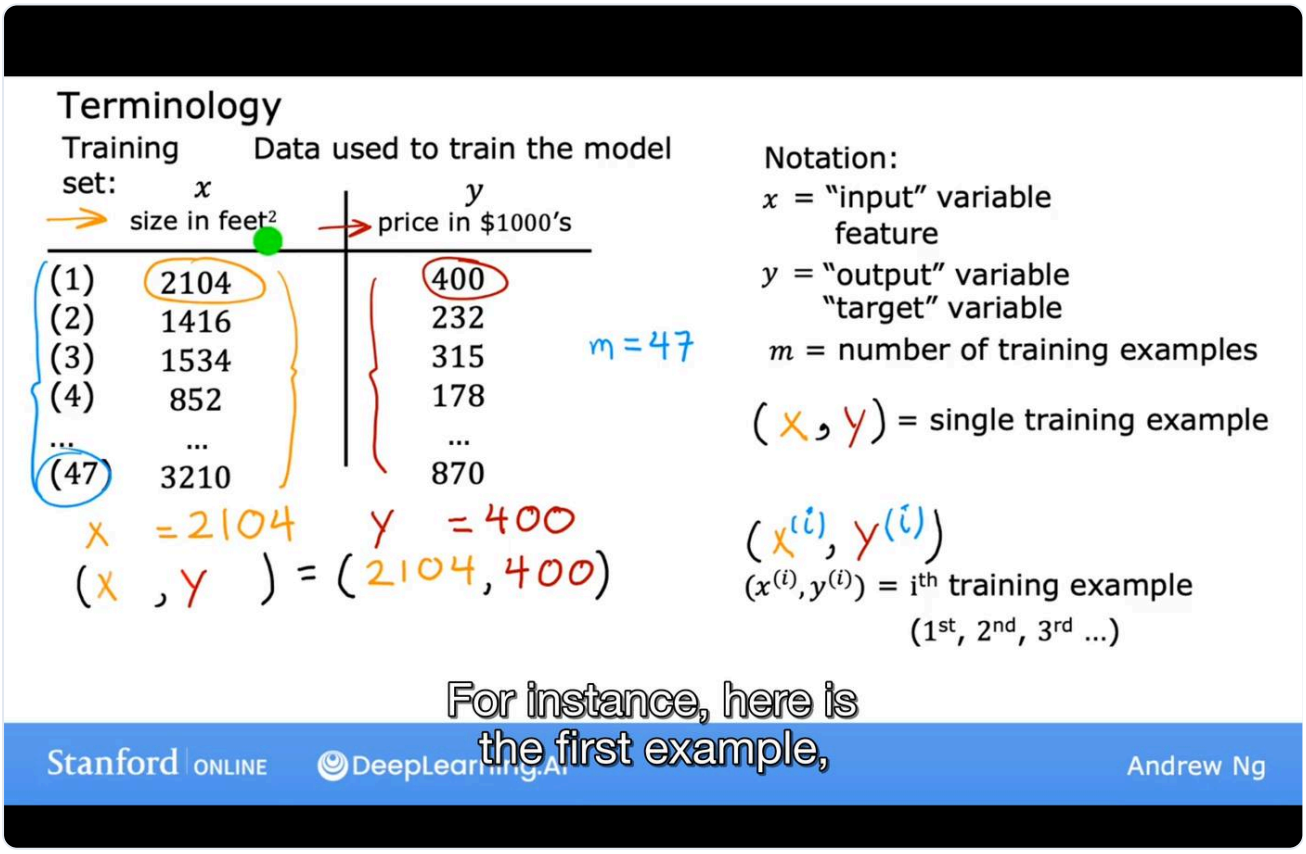


FIGURE 9.21 | Training Data Used to Train the Model Notation:

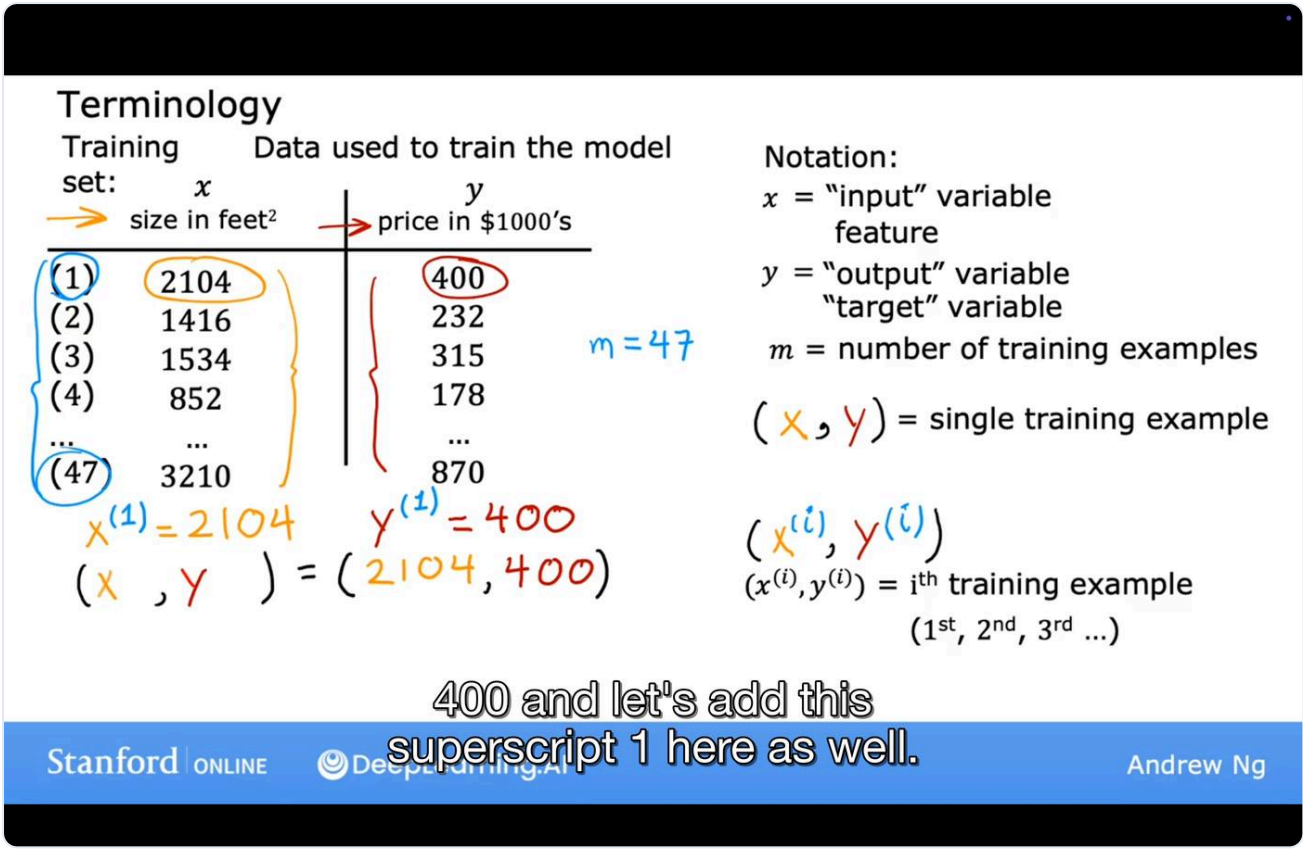


FIGURE 9.22 | Training Data Used to Train the Model Notation:

For instance, here is the first example, when i equals 1 in the training set, and so x superscript 1 is equal to 2,104 and y superscript 1 is equal to 400 and let's add this superscript 1 here as well. Just to note, this superscript i in parentheses is not exponentiation.

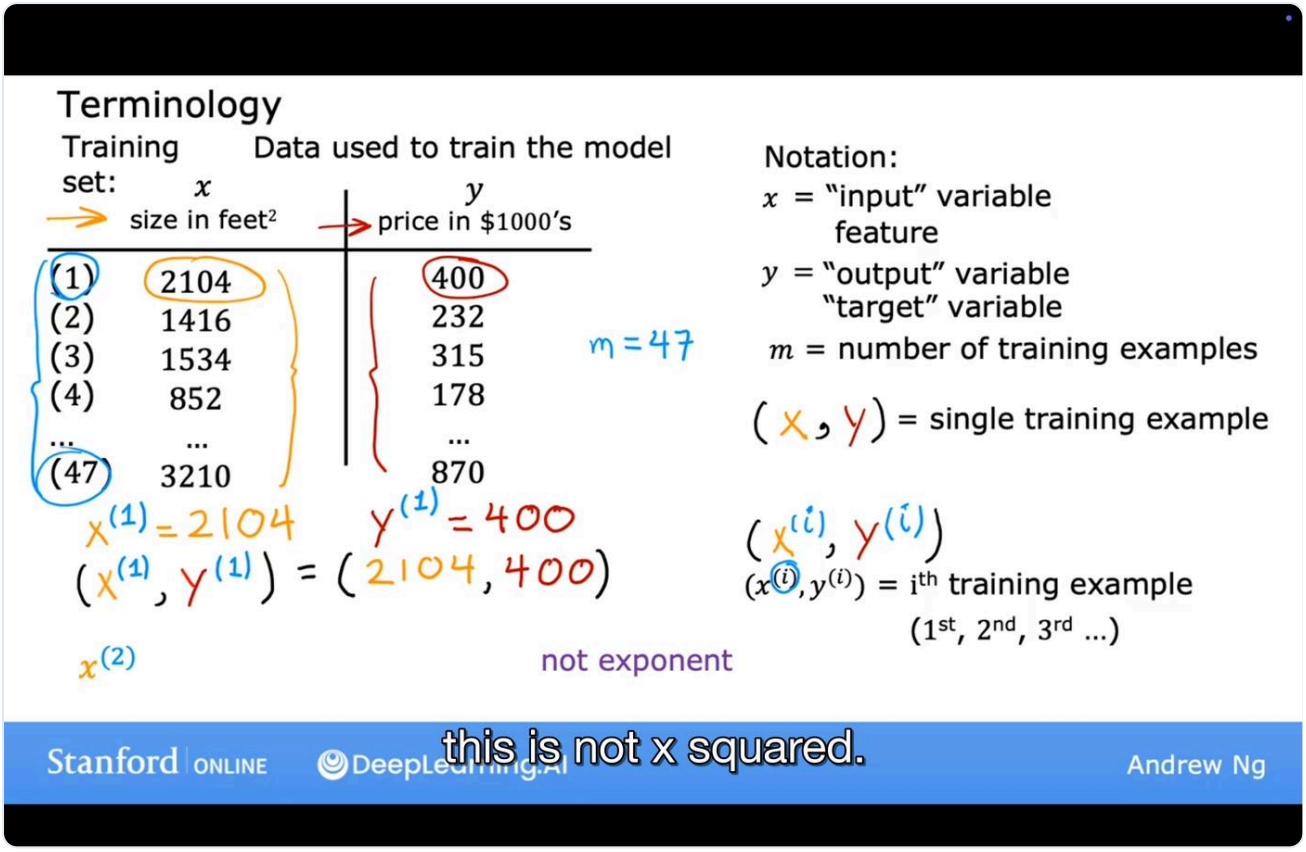


FIGURE 9.23 | Training Data Used to Train the Model Notation:

When I write this, this is not x squared. This is not x to the power 2.

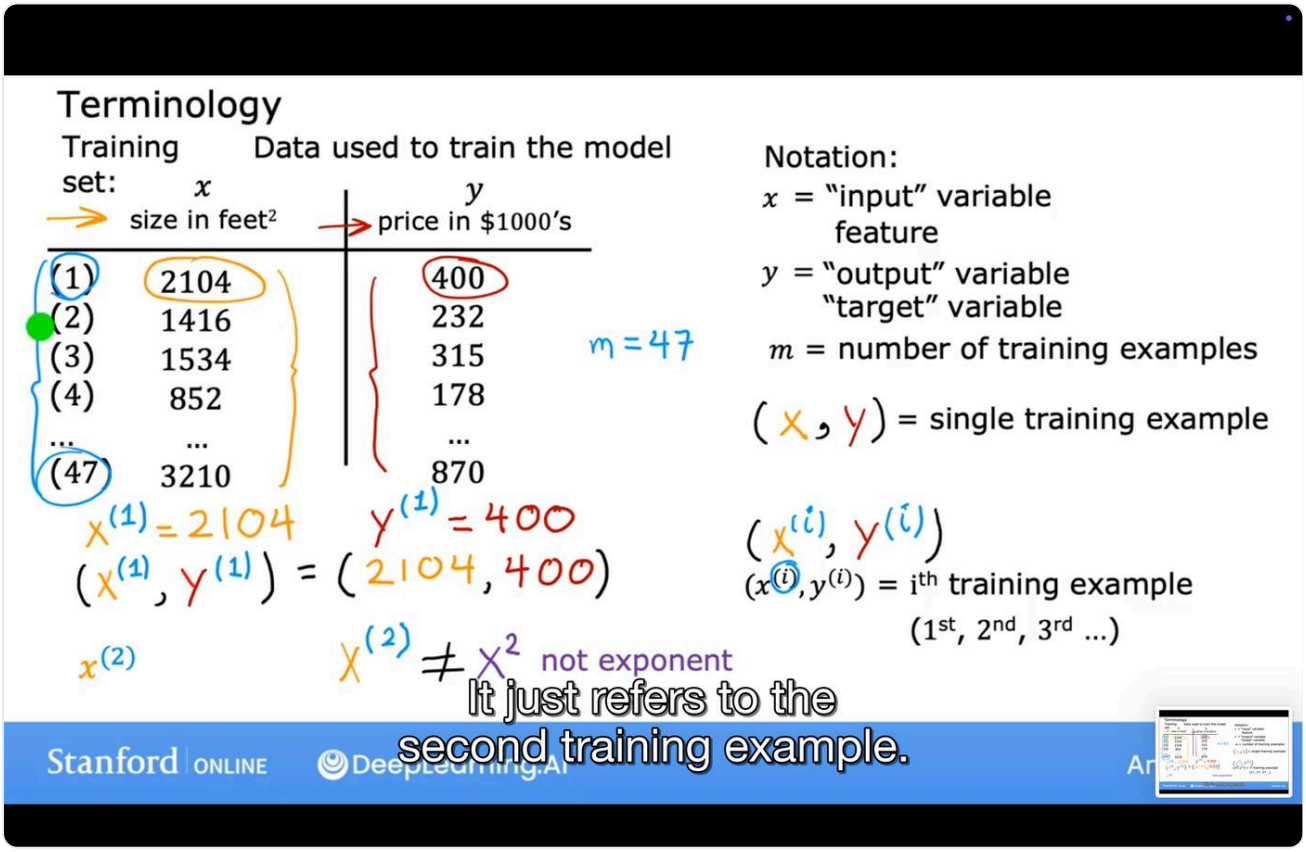


FIGURE 9.24 | Training Data Used to Train the Model Notation:

It just refers to the second training example. This i, is just an index into the training set and refers to row i in the table. In this video, you saw what a training set is like, as well as a standard notation for describing this training set. In the next video, let's look at what rotate to take this training set that you just saw and feed it to learning algorithm so that the algorithm can learn from this data. Let's see that in the next video.

LECTURE 10

Prediction Y-hat and Linear Function

Let's look in this video at the process of how supervised learning works. Supervised learning algorithm will input a dataset and then what exactly does it do and what does it output? Let's find out in this video. Recall that a training set in supervised learning includes both the input features, such as the size of the house and also the output targets, such as the price of the house. The output targets are the right answers to the model we'll learn from. To train the model, you feed the training set, both the input features and the output targets to your learning algorithm. Then your supervised learning algorithm will produce some function. We'll write this function as lowercase f, where f stands for function.

Historically, this function used to be called a hypothesis, but I'm just going to call it a function f in this class. The job with f is to take a new input x and output and estimate or a prediction, which I'm going to call y-hat, and it's written like the variable y with this little hat symbol on top.

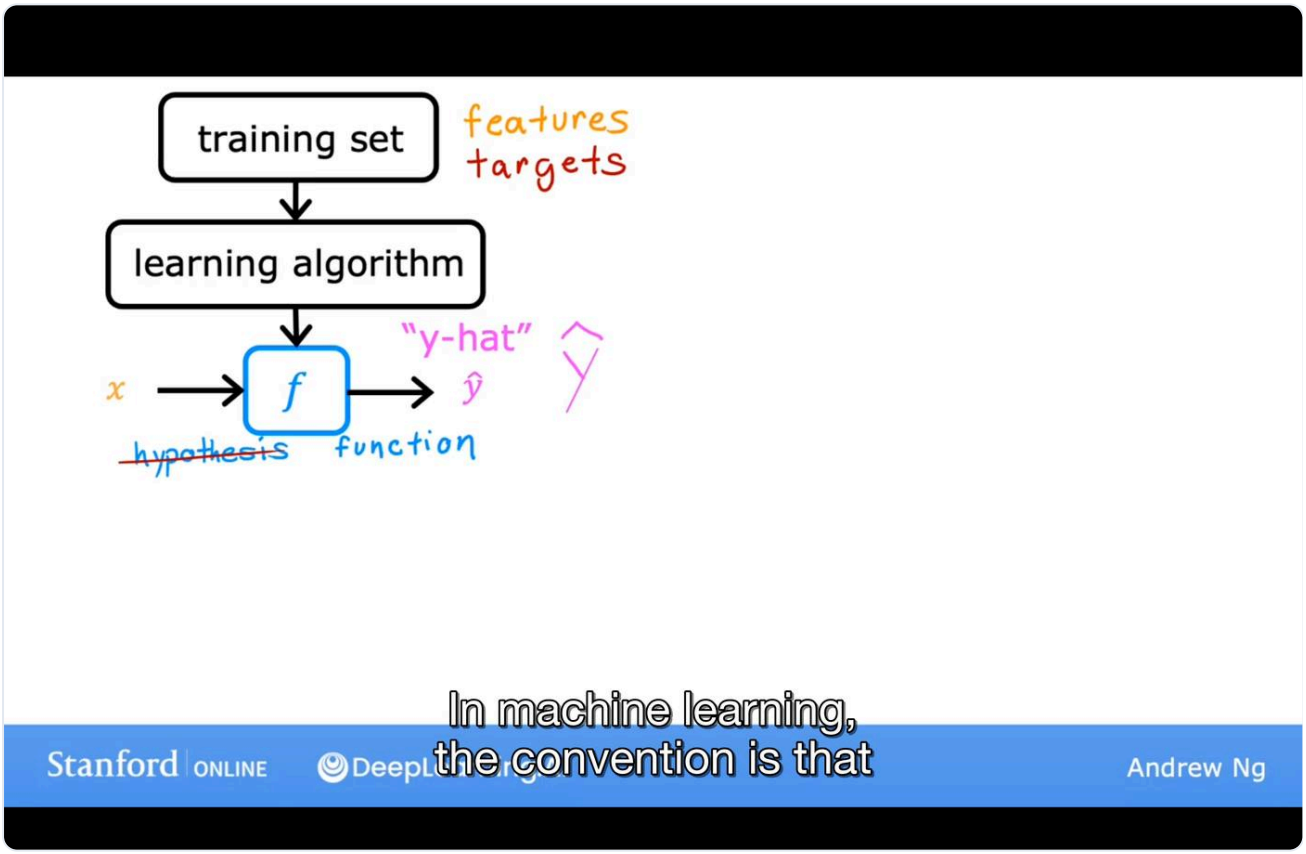


FIGURE 10.1

In machine learning, the convention is that y-hat is the estimate or the prediction for y. The function f is called the model.

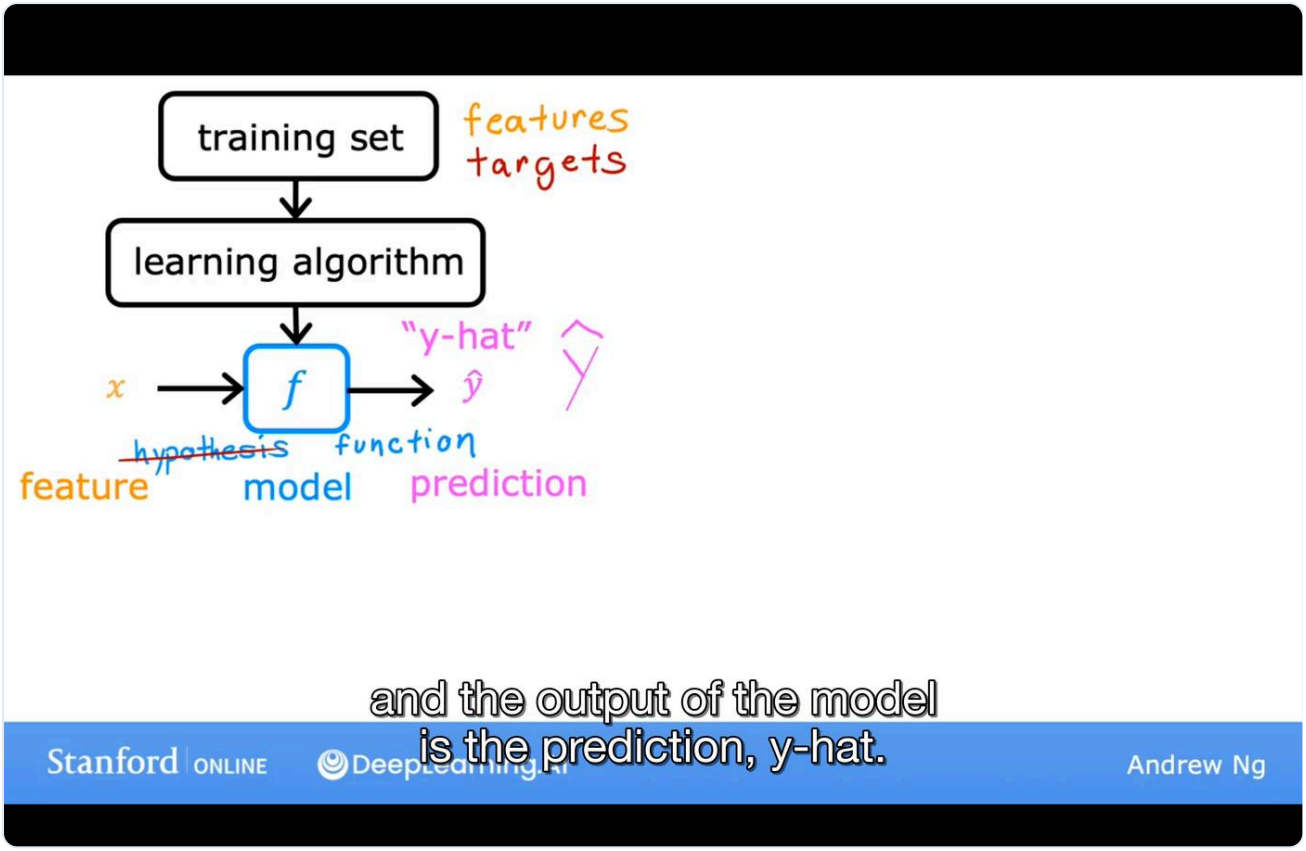


FIGURE 10.2

X is called the input or the input feature, and the output of the model is the prediction, y-hat. The model's prediction is the estimated value of y.

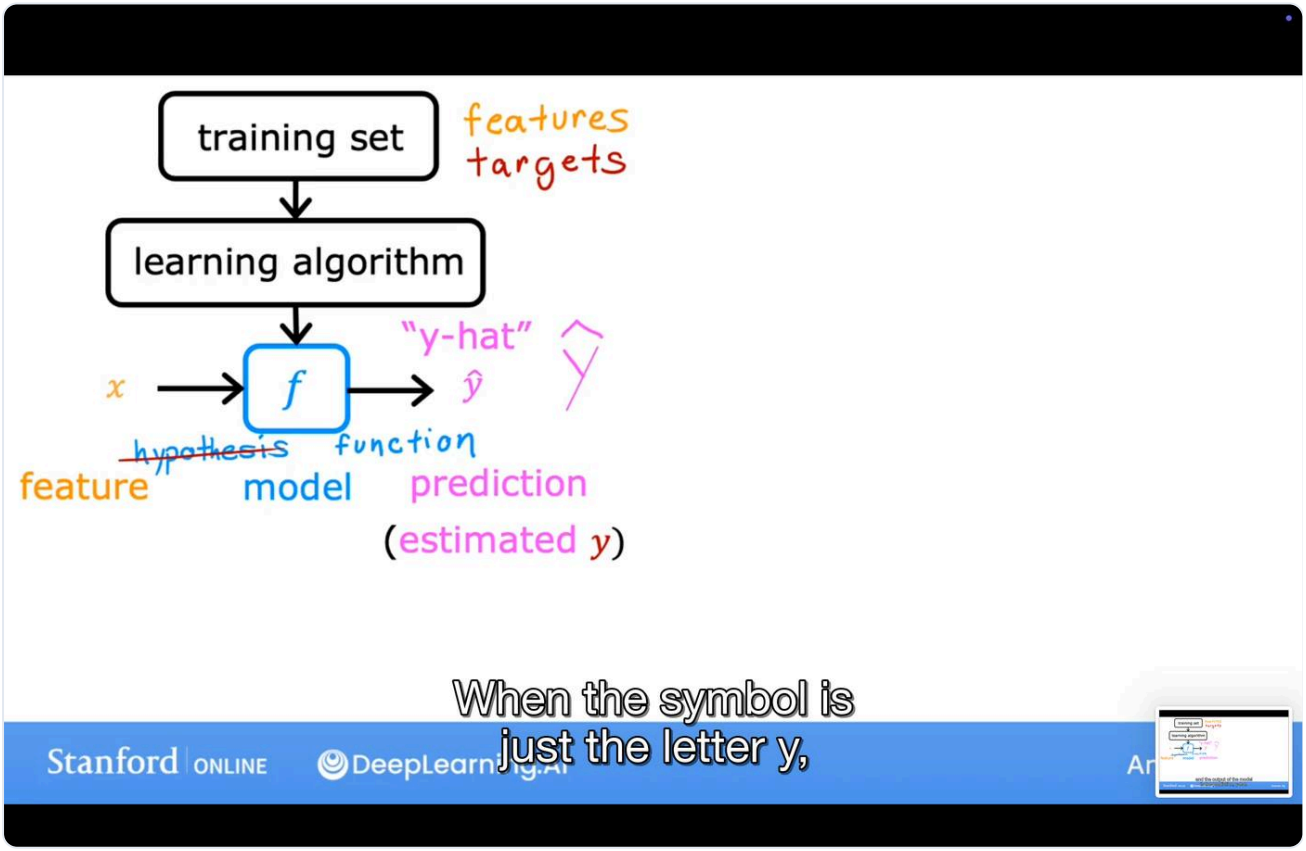


FIGURE 10.3

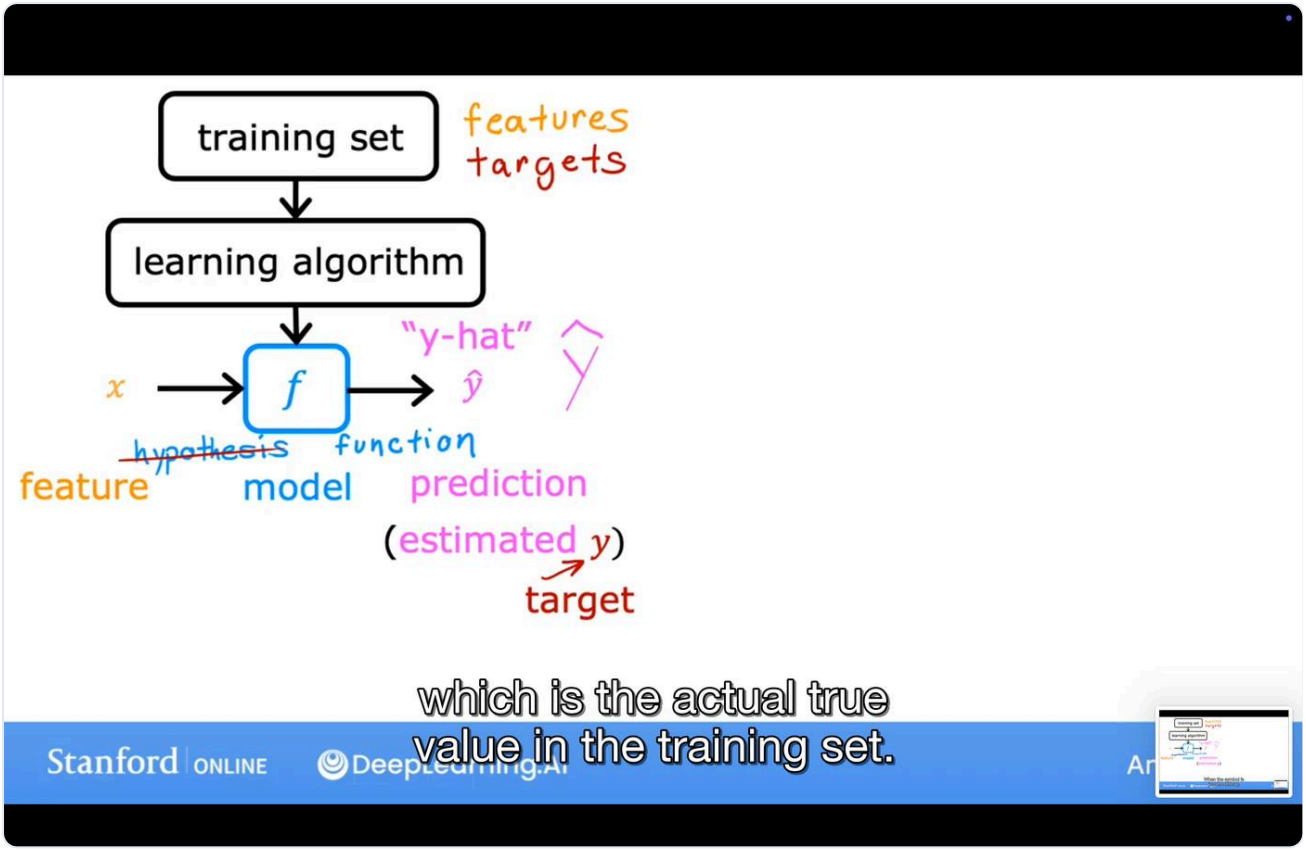


FIGURE 10.4

When the symbol is just the letter y, then that refers to the target, which is the actual true value in the training set. In contrast, y-hat is an estimate. It may or may not be the actual true value. Well, if you're helping your client to sell the house, well, the true price of the house is unknown until they sell it.

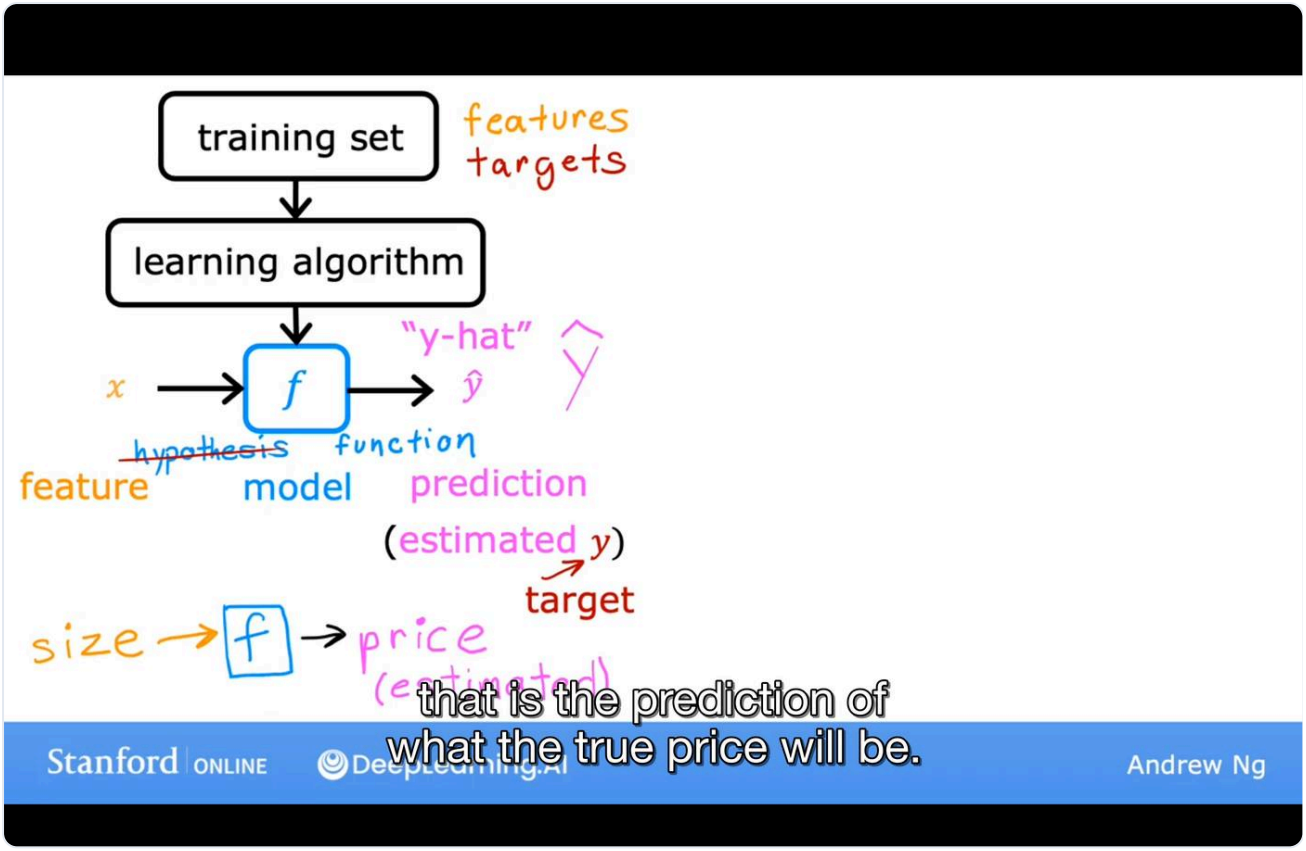


FIGURE 10.5

Your model f, given the size, outputs the price which is the estimator, that is the prediction of what the true price will be.

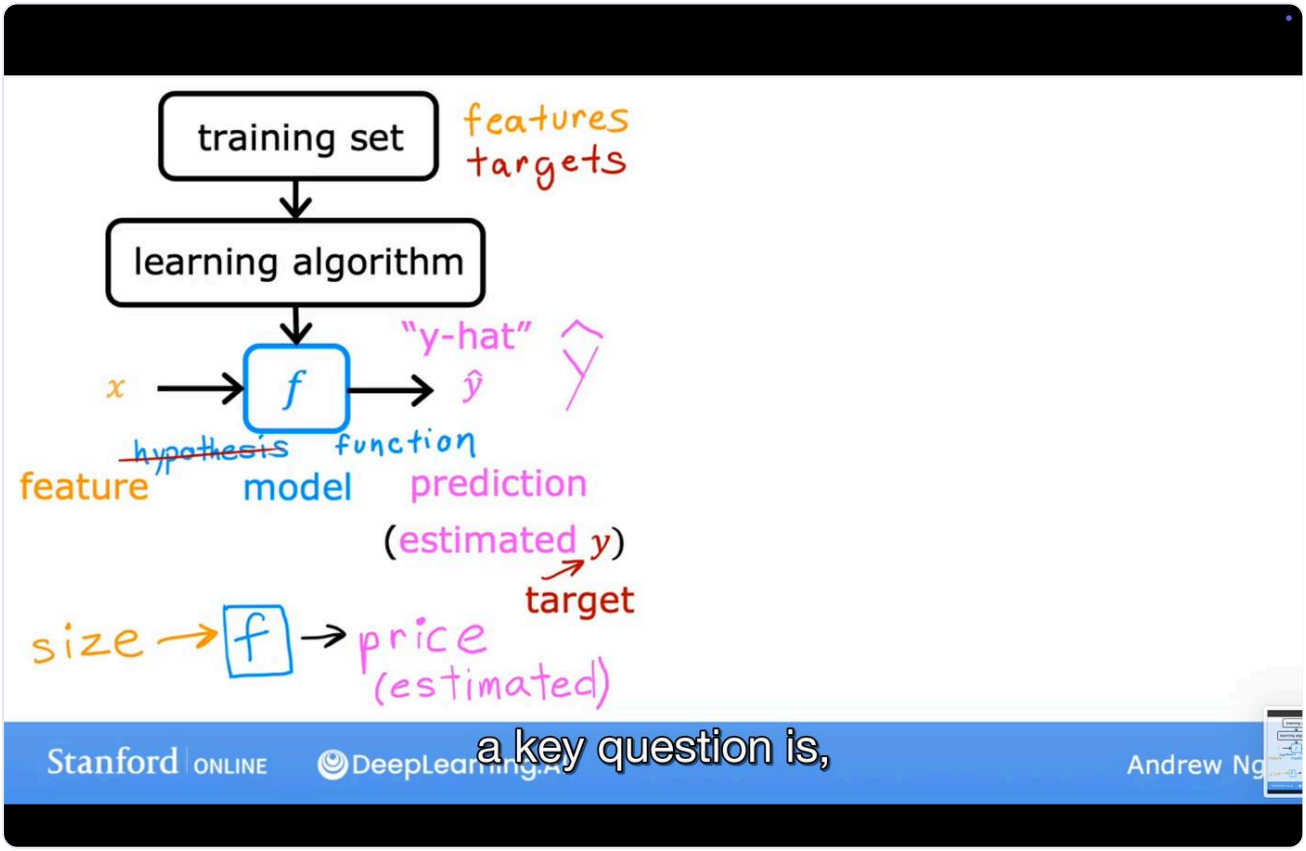


FIGURE 10.6

Now, when we design a learning algorithm, a key question is, how are we going to represent the function f? Or in other words, what is the math formula we're going to use to compute f? For now, let's stick with f being a straight line. You're function can be written as f_w, b of x equals, I'm going to use w times x plus b.

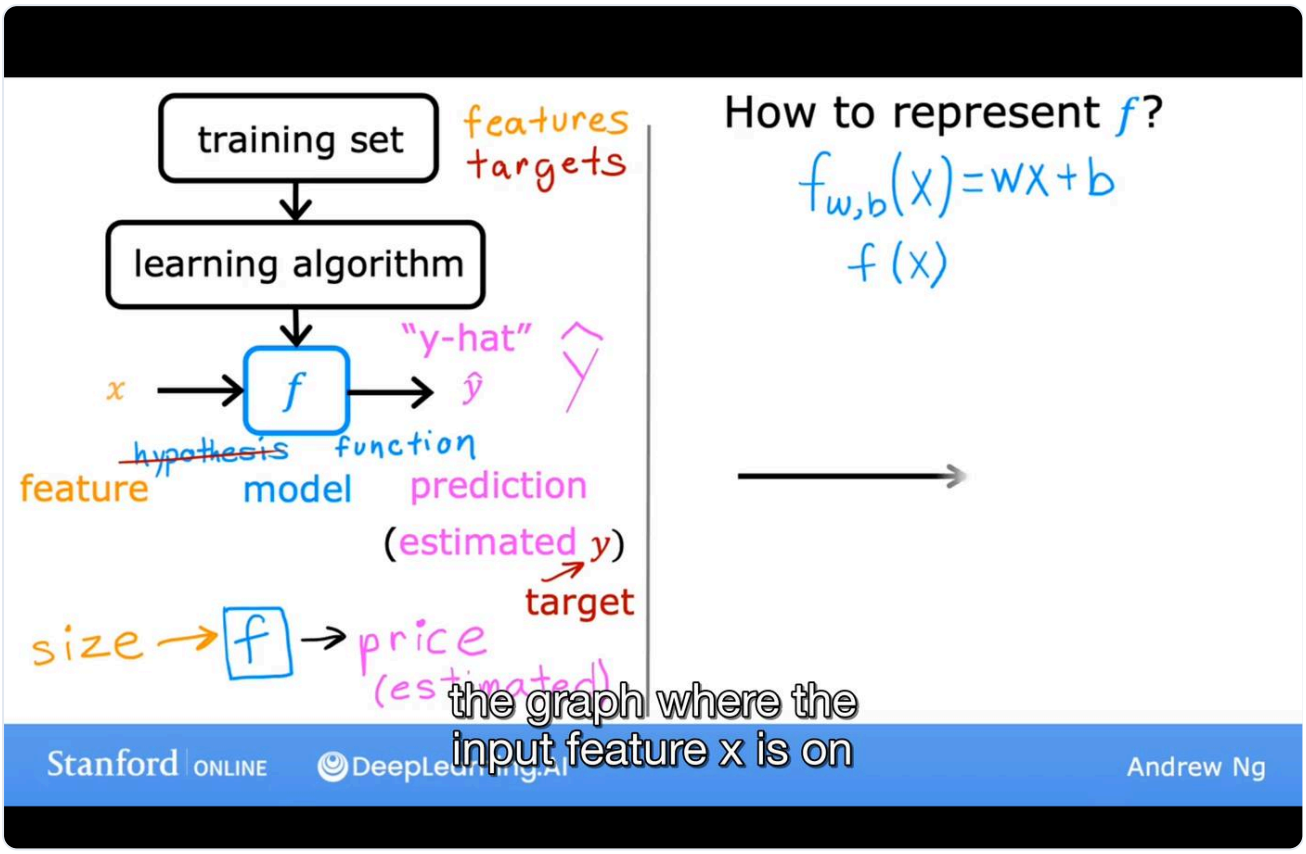


FIGURE 10.7 | How to Represent F?

I'll define w and b soon. But for now, just know that w and b are numbers, and the values chosen for w and b will determine the prediction y-hat based on the input feature x. This f_w b of x means f is a function that takes x as input, and depending on the values of w and b, f will output some value of a prediction y-hat. As an alternative to writing this, f_w, b of x, I'll sometimes just write f of x without explicitly including w and b into subscript.

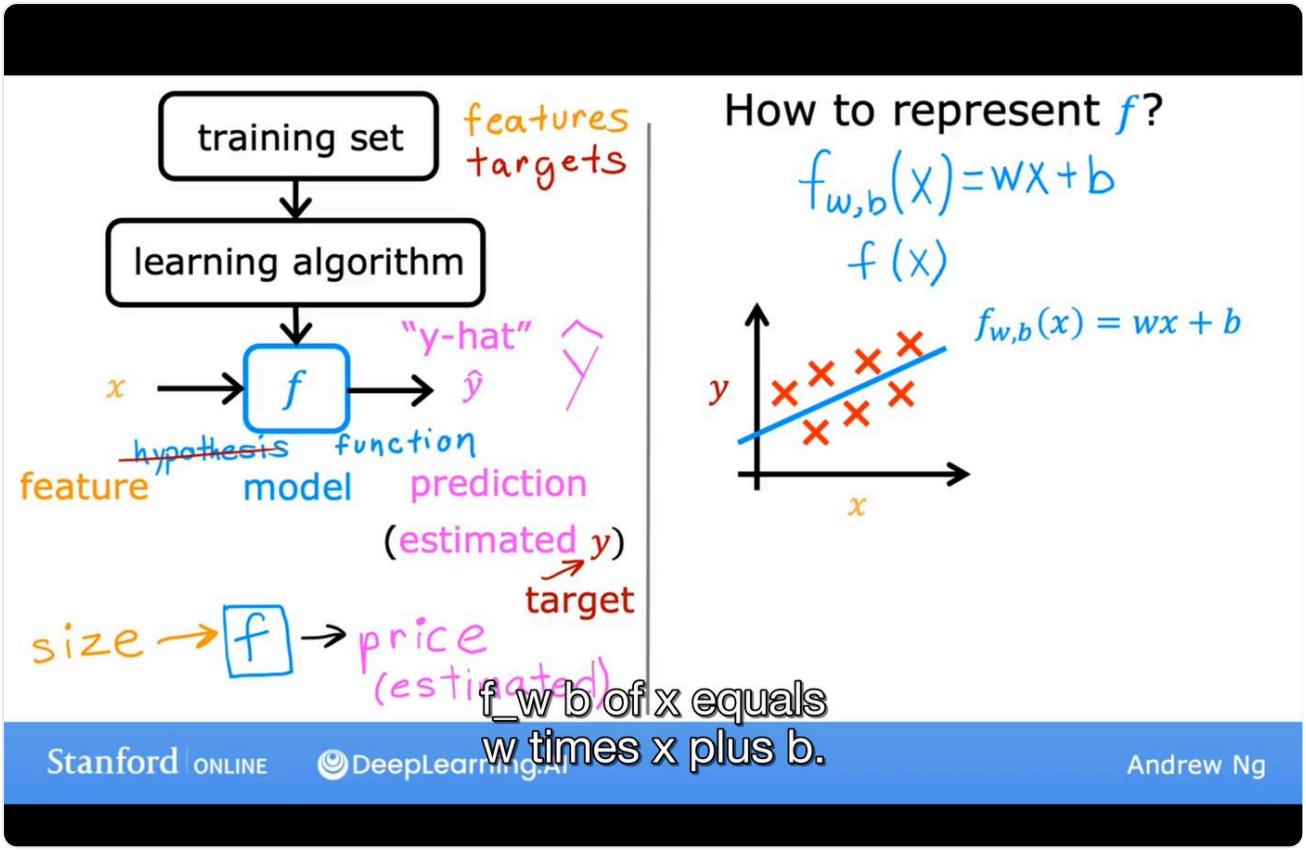


FIGURE 10.8 | How to Represent F?

Is just a simpler notation that means exactly the same thing as f_w b of x. Let's plot the training set on the graph where the input feature x is on the horizontal axis and the output target y is on the vertical axis. Remember, the algorithm learns from this data and generates the best-fit line like maybe this one here. This straight line is the linear function f_w b of x equals w times x plus b.

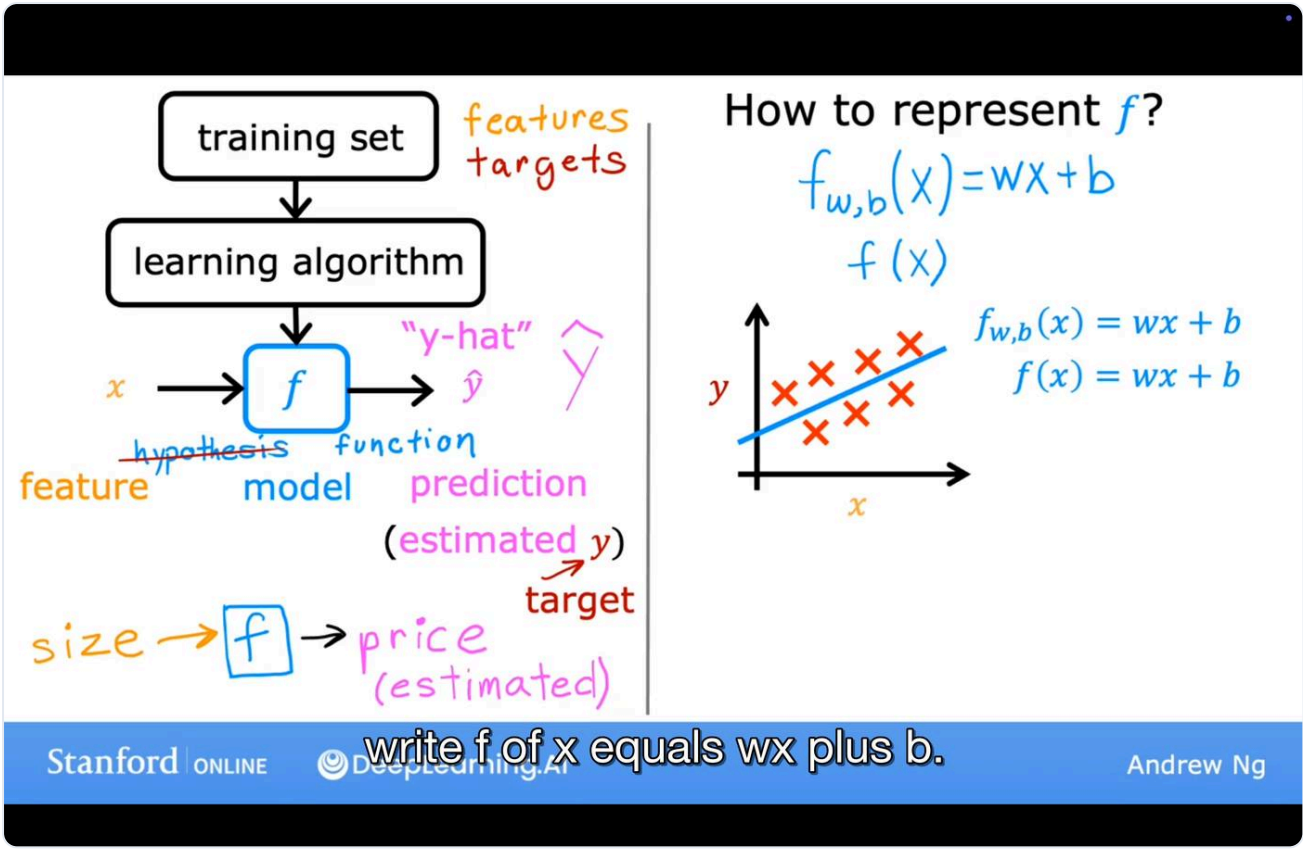


FIGURE 10.9 | How to Represent F?

Or more simply, we can drop w and b and just write f of x equals wx plus b. Here's what this function is doing, it's making predictions for the value of y using a streamline function of x. You may ask, why are we choosing a linear function, where linear function is just a fancy term for a straight line instead of some non-linear function like a curve or a parabola?

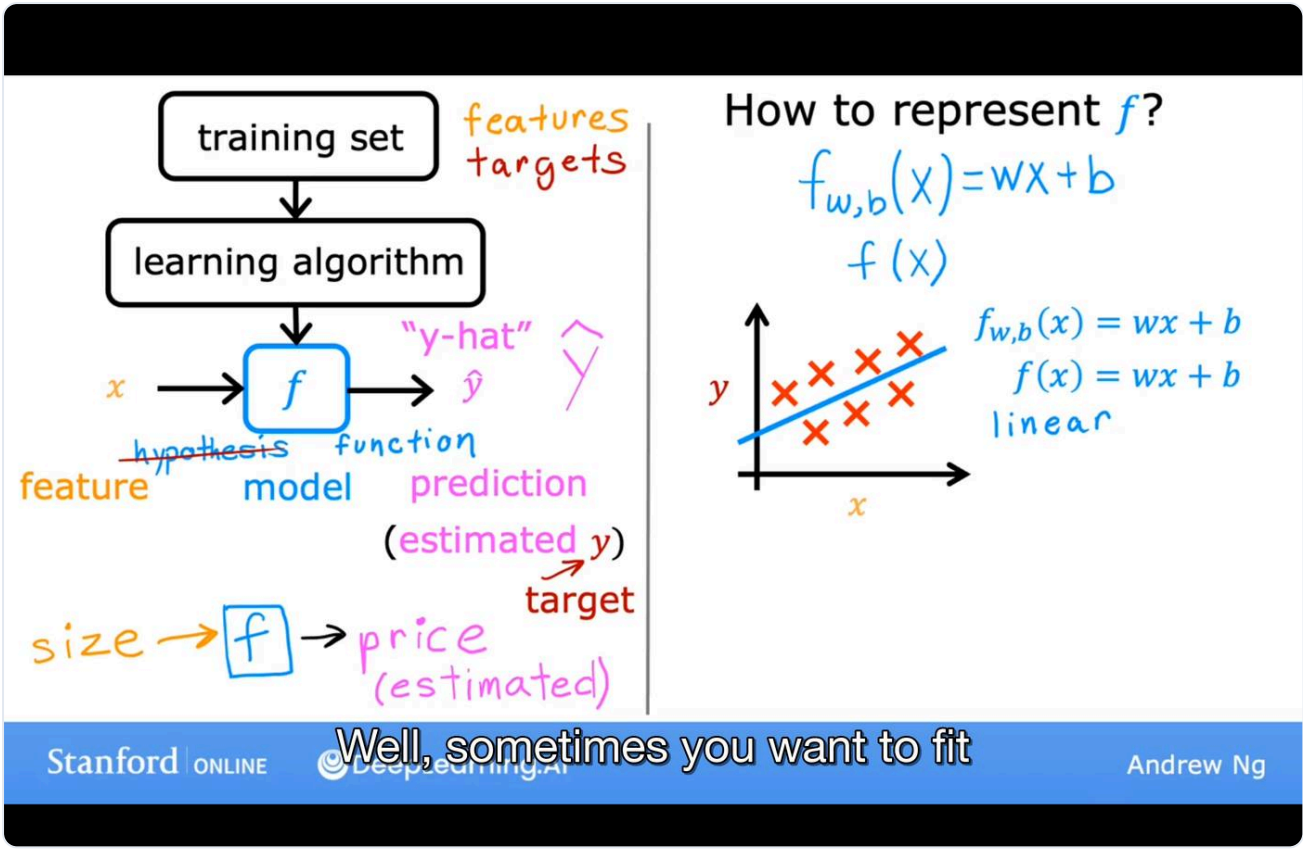


FIGURE 10.10 | How to Represent /?

Well, sometimes you want to fit more complex non-linear functions as well, like a curve like this. But since this linear function is relatively simple and easy to work with, let's use a line as a foundation that will eventually help you to get to more complex models that are non-linear. This particular model has a name, it's called linear regression. More specifically, this is linear regression with one variable, where the phrase one variable means that there's a single input variable or feature x, namely the size of the house. Another name for a linear model with one input variable is univariate linear regression, where uni means one in Latin, and where variate means variable.

Univariate is just a fancy way of saying one variable.

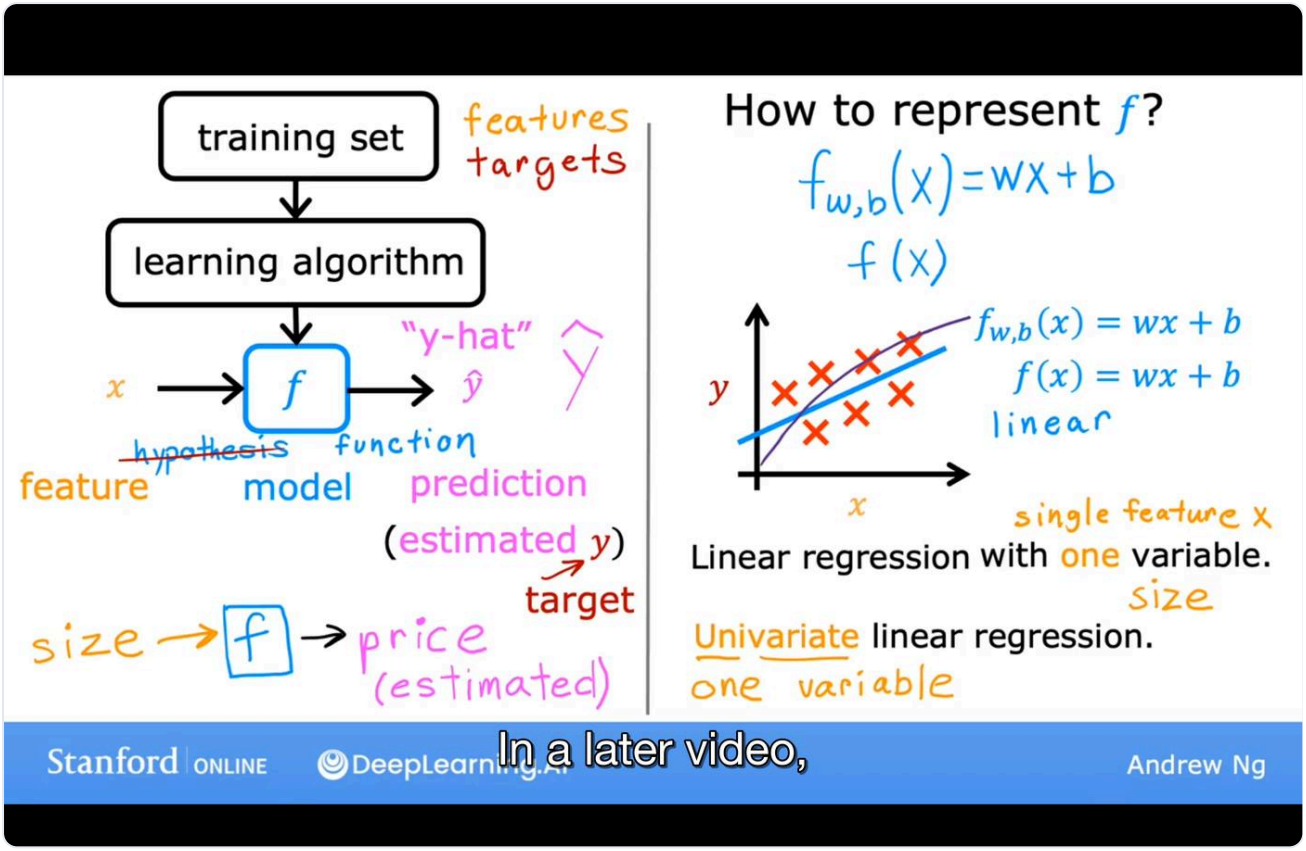


FIGURE 10.11 | How to Represent /?

In a later video, you'll also see a variation of regression where you'll want to make a prediction based not just on the size of a house, but on a bunch of other things that you may know about the house such as number of bedrooms and other features. By the way, when you're done with this video, there is another optional lab. You don't need to write any code. Just review it, run the code and see what it does. That will show you how to define in Python a straight line function. The lab will let you choose the values of w and b to try to fit the training data. You don't have to do the lab if you don't want to, but I hope you play with it when you're done watching this video.

That's linear regression. In order for you to make this work, one of the most important things you have to do is construct a cost function. The idea of a cost function is one of the most universal and important ideas in machine learning, and is used in both linear regression and in training many of the most advanced AI models in the world. Let's go on to the next video and take a look at how you can construct a cost function.

LECTURE 11

Cost Function: Squared Error Cost Function

In order to implement linear regression the first key step is first to define something called a cost function. This is something we'll build in this video, and the cost function will tell us how well the model is doing so that we can try to get it to do better. Let's look at what this means. Recall that you have a training set that contains input features x and output targets y. The model you're going to use to fit this training set is this linear function f_w, b of x equals to w times x plus b. To introduce a little bit more terminology the w and b are called the parameters of the model. In machine learning parameters of the model are the variables you can adjust during training in order to improve the model.

Sometimes you also hear the parameters w and b referred to as coefficients or as weights.

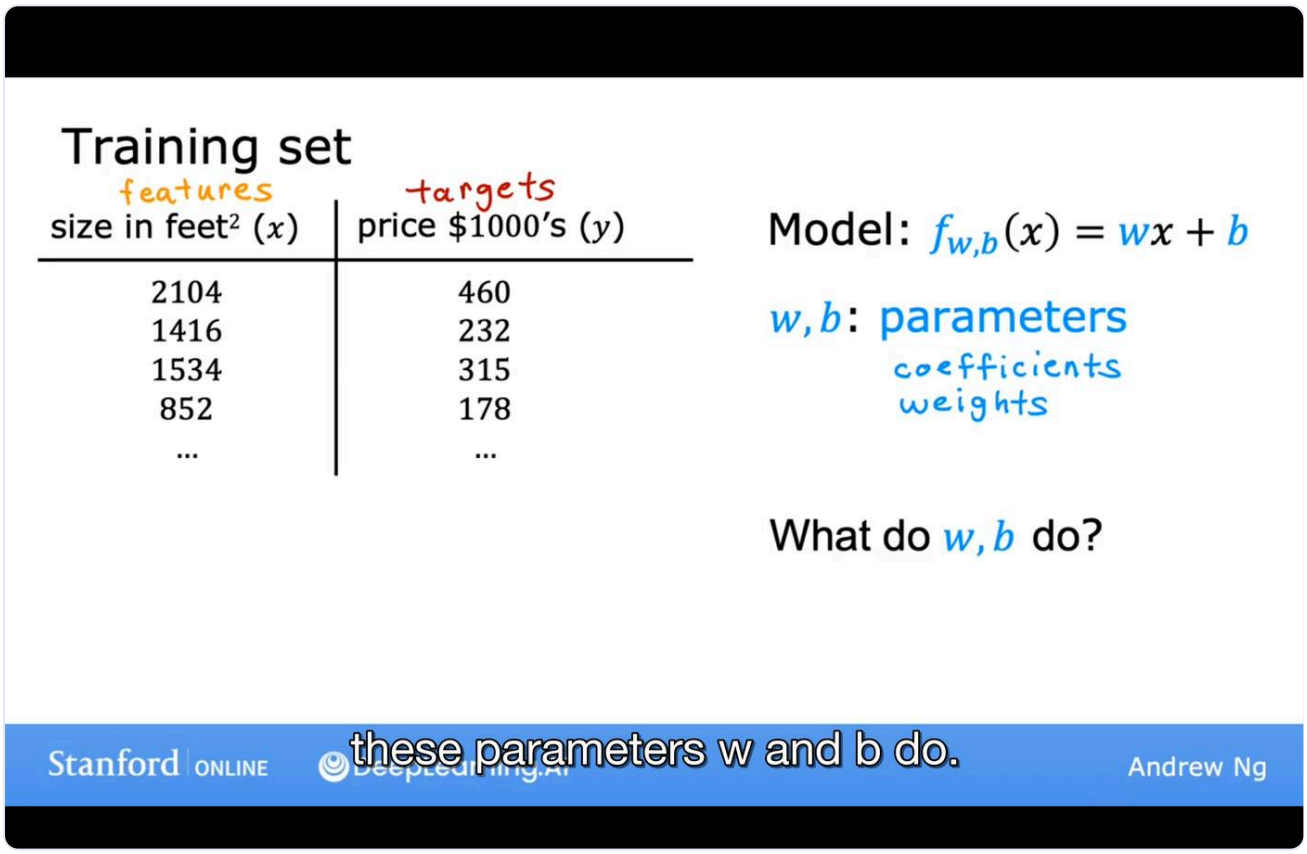


FIGURE 11.1 | Training Set

Now let's take a look at what these parameters w and b do. Depending on the values you've chosen for w and b you get a different function f of x, which generates a different line on the graph. Remember that we can write f of x as a shorthand for f_w, b of x. We're going to take a look at some plots of f of x on a chart. Maybe you're already familiar with drawing lines on charts, but even if this is a review for you, I hope this will help you build intuition on how w and b the parameters determine f. When w is equal to 0 and b is equal to 1.5, then f looks like this horizontal line. In this case, the function f of x is 0 times x plus 1.5 so f is always a constant value.

It always predicts 1.5 for the estimated value of y. Y hat is always equal to b and here b is also called the y intercept because that's where it crosses the vertical axis or the y axis on this graph. As a second example, if w is 0.5 and b is equal 0, then f of x is 0.5 times x. When x is 0, the prediction is also 0, and when x is 2, then the prediction is 0.5 times 2, which is 1. You get a line that looks like this and notice that the slope is 0.5 divided by 1.

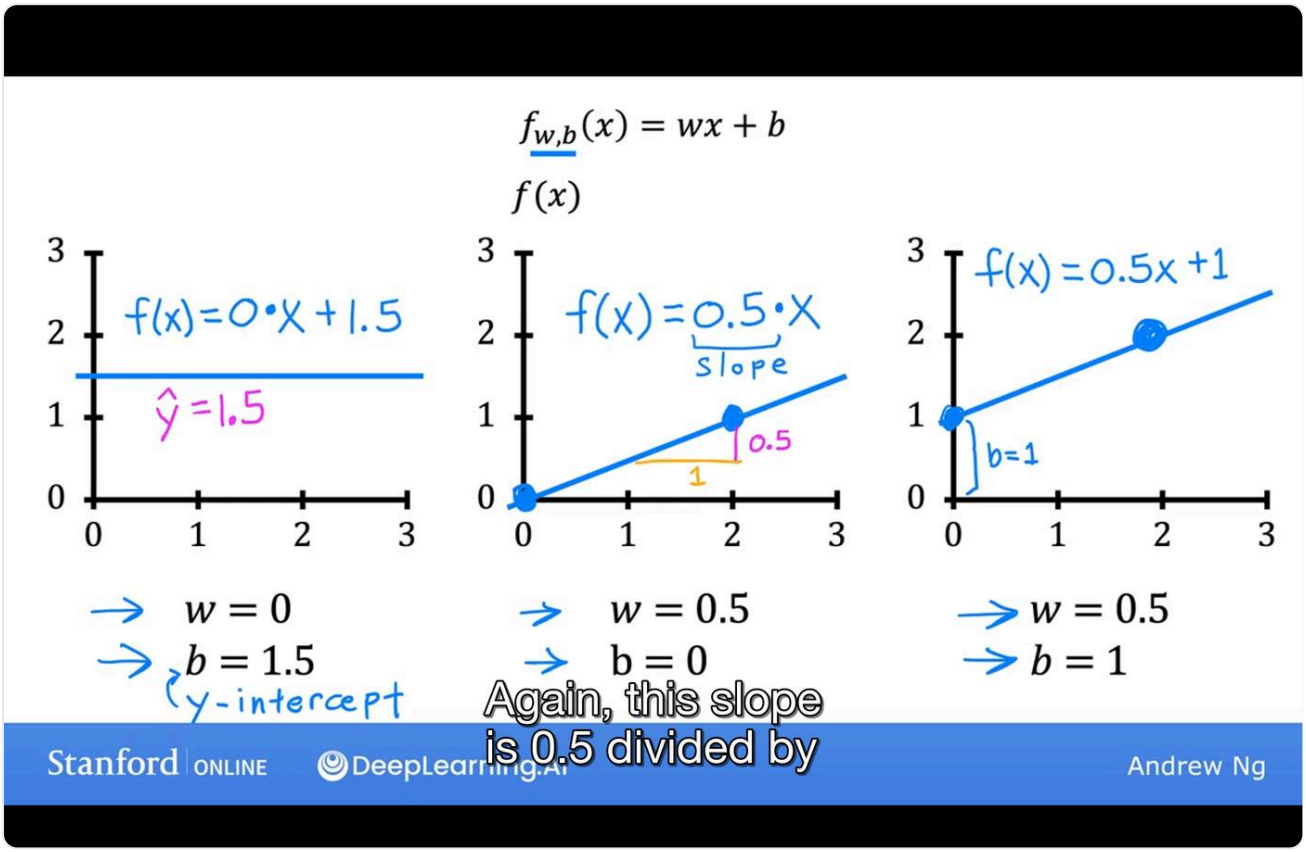


FIGURE 11.2

The value of w gives you the slope of the line, which is 0.5. Finally, if w equals 0.5 and b equals 1, then f of x is 0.5 times x plus 1 and when x is 0, then f of x equals b, which is 1 so the line intersects the vertical axis at b, the y intercept. Also when x is 2, then f of x is 2, so the line looks like this. Again, this slope is 0.5 divided by 1 so the value of w gives you the slope which is 0.5. Recall that you have a training set like the one shown here. With linear regression, what you want to do is to choose values for the parameters w and b so that the straight line you get from the function f somehow fits the data well. Like maybe this line shown here.

When I see that the line fits the data visually, you can think of this to mean that the line defined by f is roughly passing through or somewhere close to the training examples as compared to other possible lines that are not as close to these points. Just to remind you of some notation, a training example like this point here is defined by x superscript i, y superscript i where y is the target. For a given input x^i, the function f also makes a predictive value for y and a value that it predicts to y is y hat i shown here. For our choice of a model f of x^i is w times x^i plus b. Stated differently, the prediction y hat i is f of wb of x^i where for the model we're using f of x^i is equal to wx^i plus b.

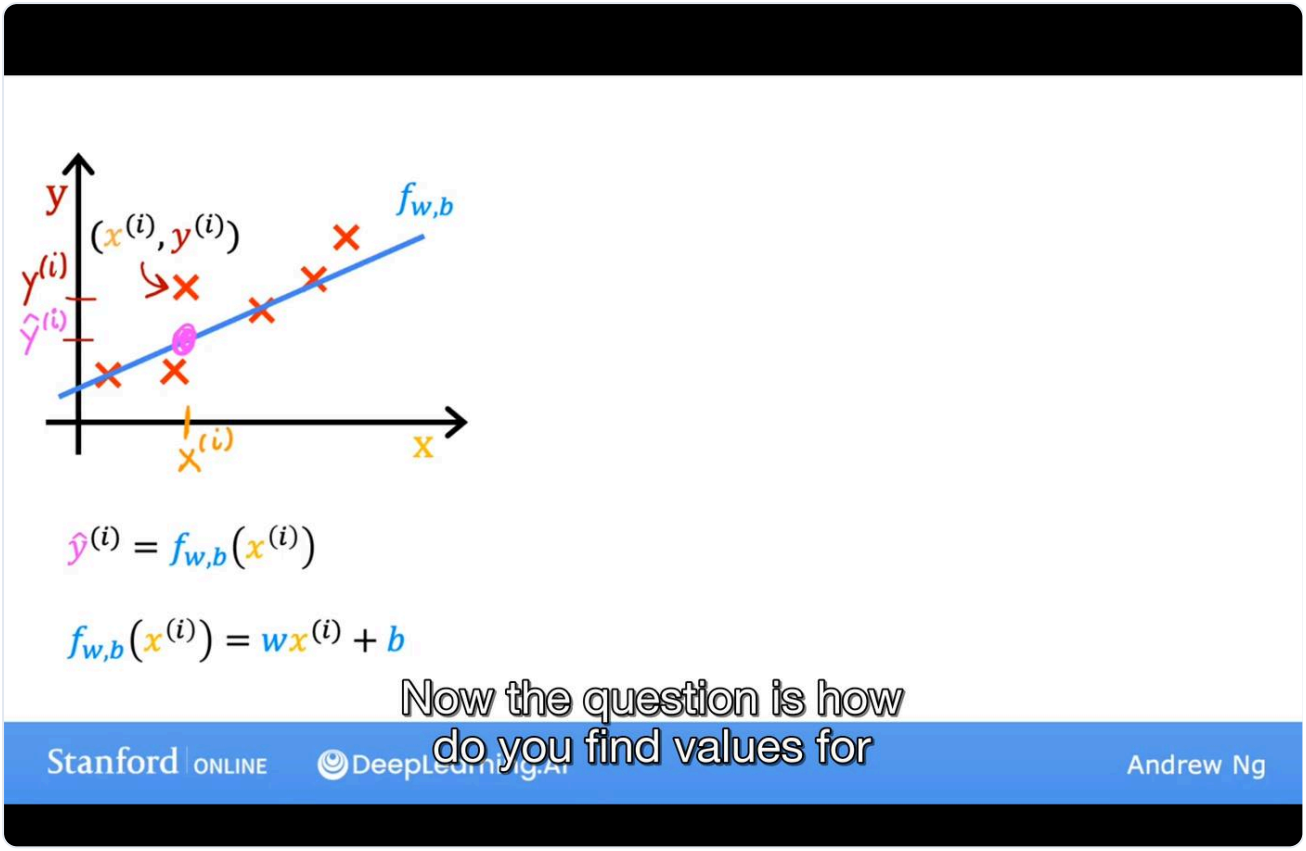


FIGURE 11.3 | Now Tine Question Is How

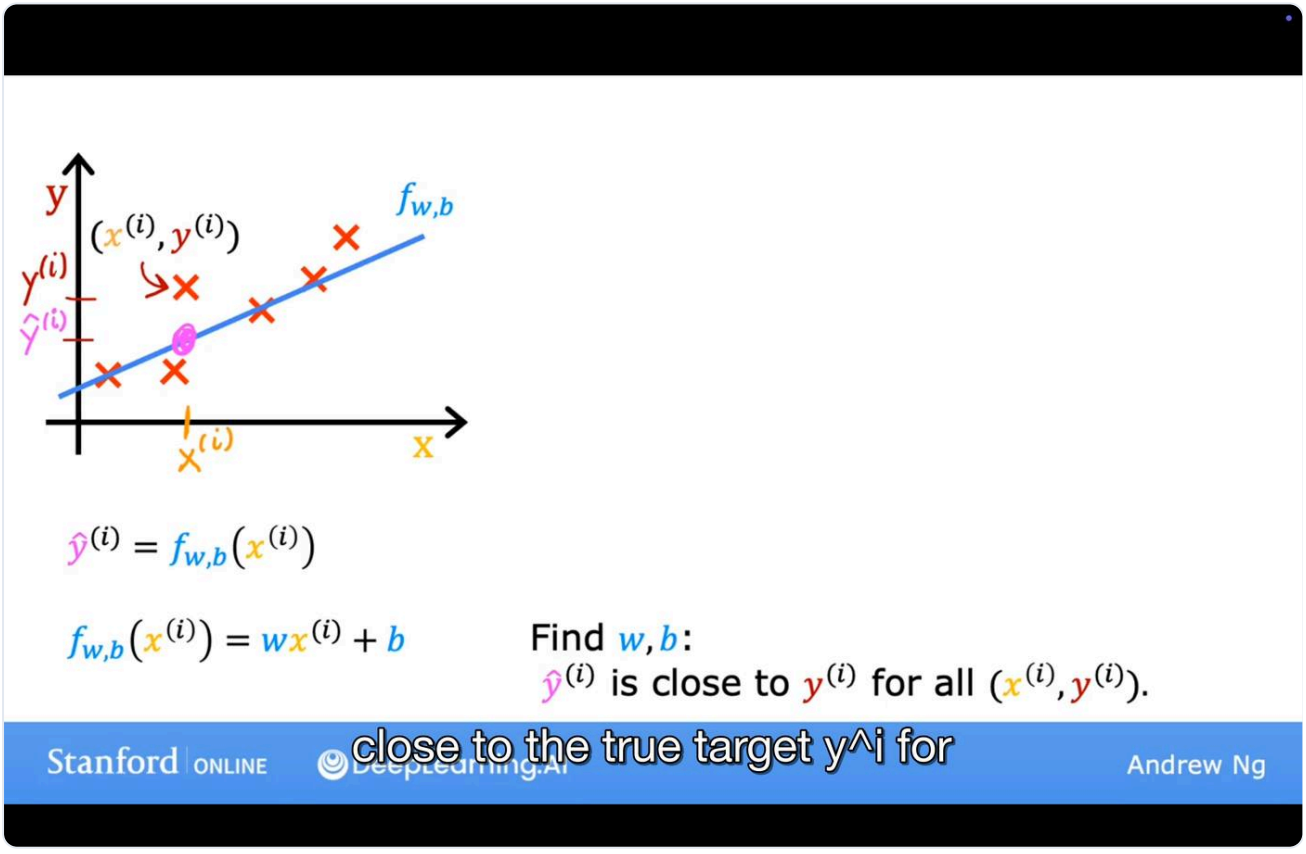


FIGURE 11.4

Now the question is how do you find values for w and b so that the prediction y hat i is close to the true target y^i for many or maybe all training examples x^i, y^i. To answer that question, let's first take a look at how to measure how well a line fits the training data. To do that, we're going to construct a cost function. The cost function takes the prediction y hat and compares it to the target y by taking y hat minus y.

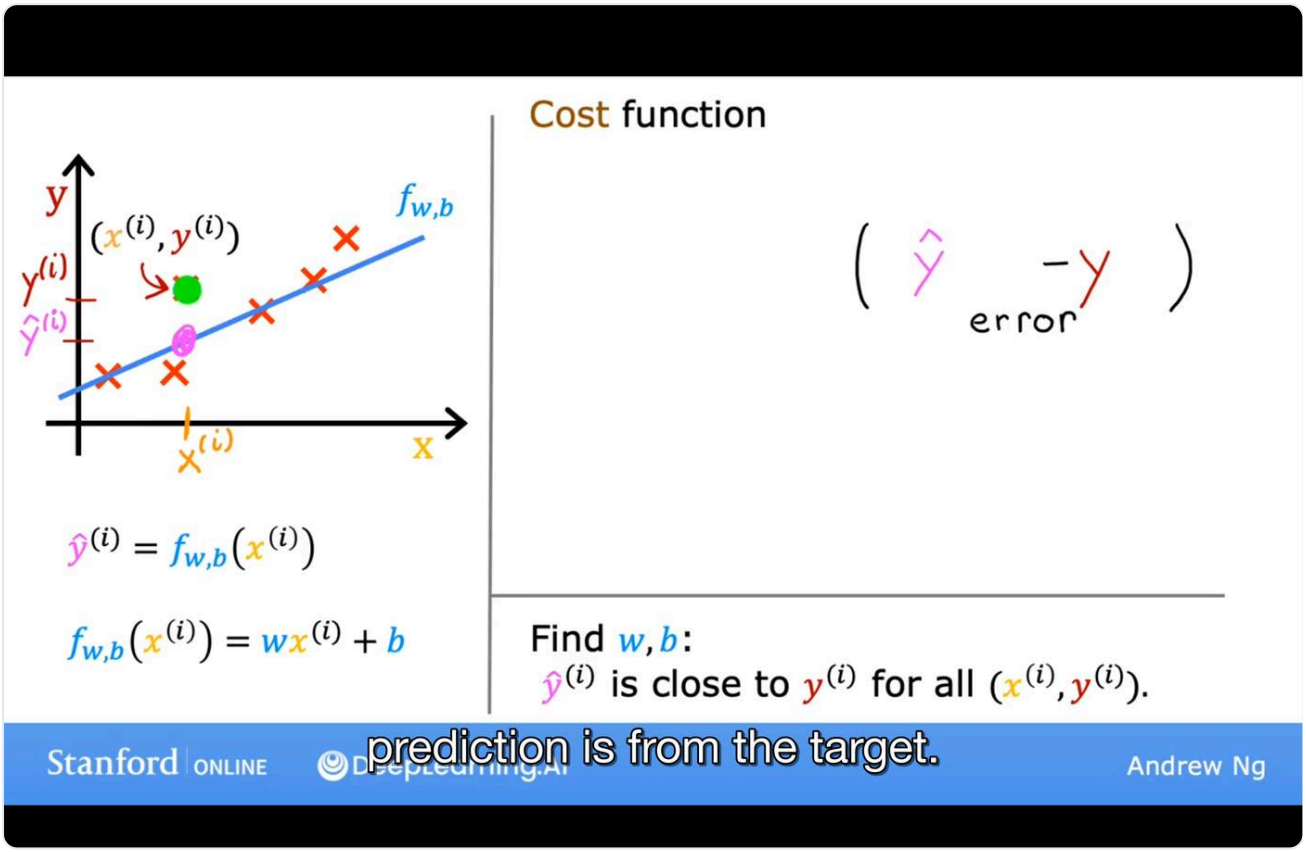


FIGURE 11.5 | Cost Function

This difference is called the error, we're measuring how far off to prediction is from the target. Next, let's computes the square of this error. Also, we're going to want to compute this term for different training examples i in the training set. When measuring the error, for example i, we'll compute this squared error term. Finally, we want to measure the error across the entire training set. In particular, let's sum up the squared errors like this. We'll sum from i equals 1,2, 3 all the way up to m and remember that m is the number of training examples, which is 47 for this dataset. Notice that if we have more training examples m is larger and your cost function will calculate a bigger number.

This is summing over more examples. To build a cost function that doesn't automatically get bigger as the training set size gets larger by convention, we will compute the average squared error instead of the total squared error and we do that by dividing by m like this. We're nearly there. Just one last thing. By convention, the cost function that machine learning people use actually divides by 2 times m. The extra division by 2 is just meant to make some of our later calculations look neater, but the cost function still works whether you include this division by 2 or not. This expression right here is the cost function and we're going to write J of wb to refer to the cost function.

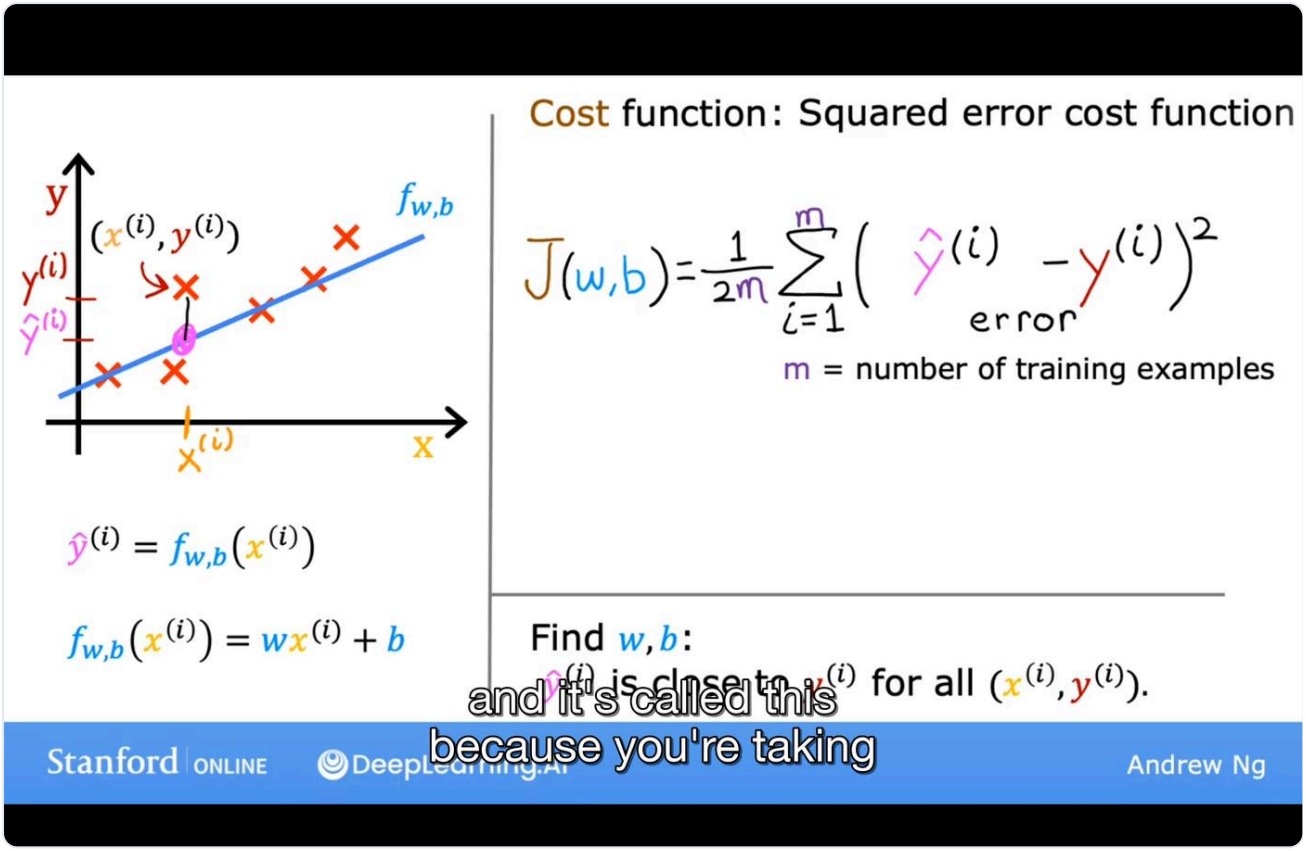


FIGURE 11.6 | Cost Function: Squared Error Cost Function

This is also called the squared error cost function, and it's called this because you're taking the square of these error terms. In machine learning different people will use different cost functions for different applications, but the squared error cost function is by far the most commonly used one for linear regression and for that matter, for all regression problems where it seems to give good results for many applications. Just as a reminder, the prediction y hat is equal to the outputs of the model f at x.

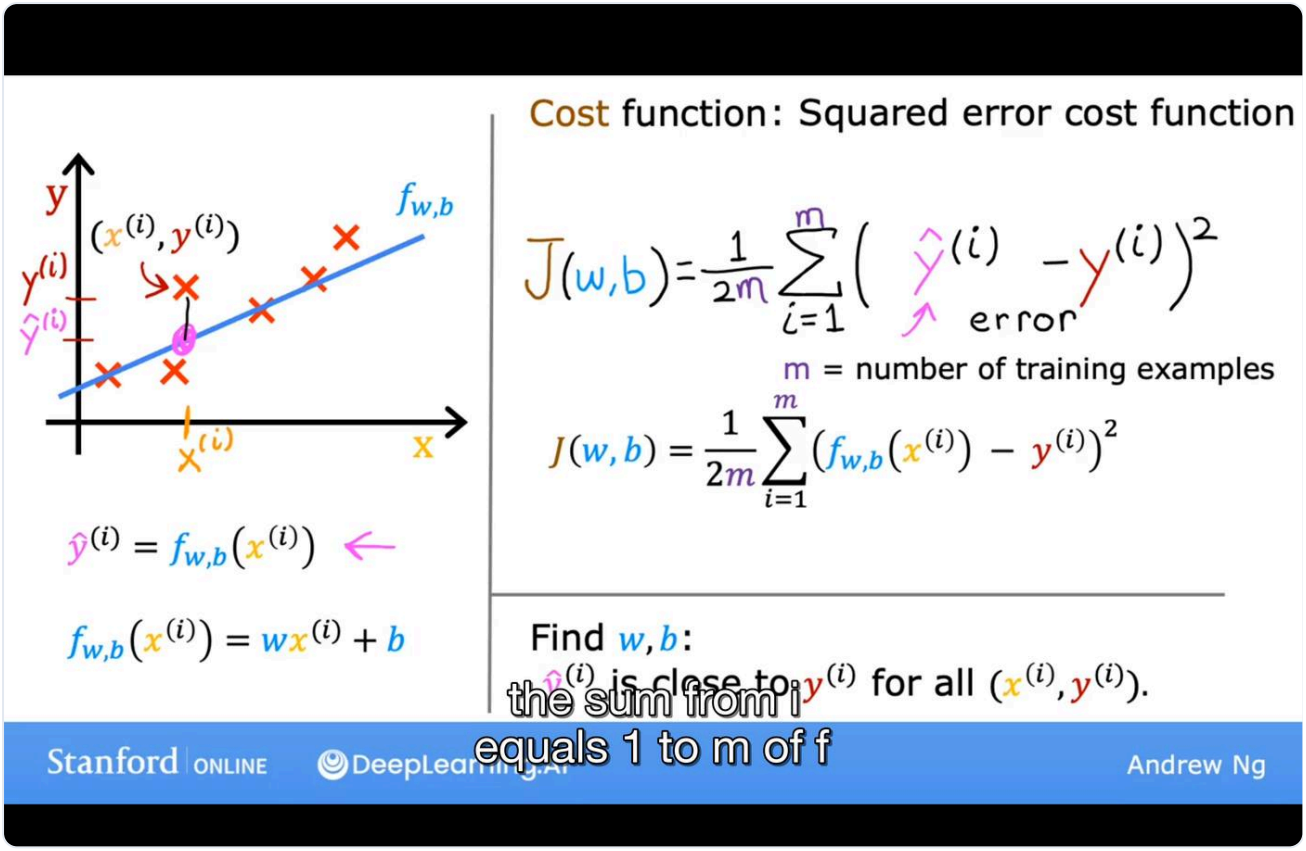


FIGURE 11.7 | Cost Function: Squared Error Cost Function

We can rewrite the cost function J of wb as 1 over 2m times the sum from i equals 1 to m of f of x^i minus y^i the quantity squared.

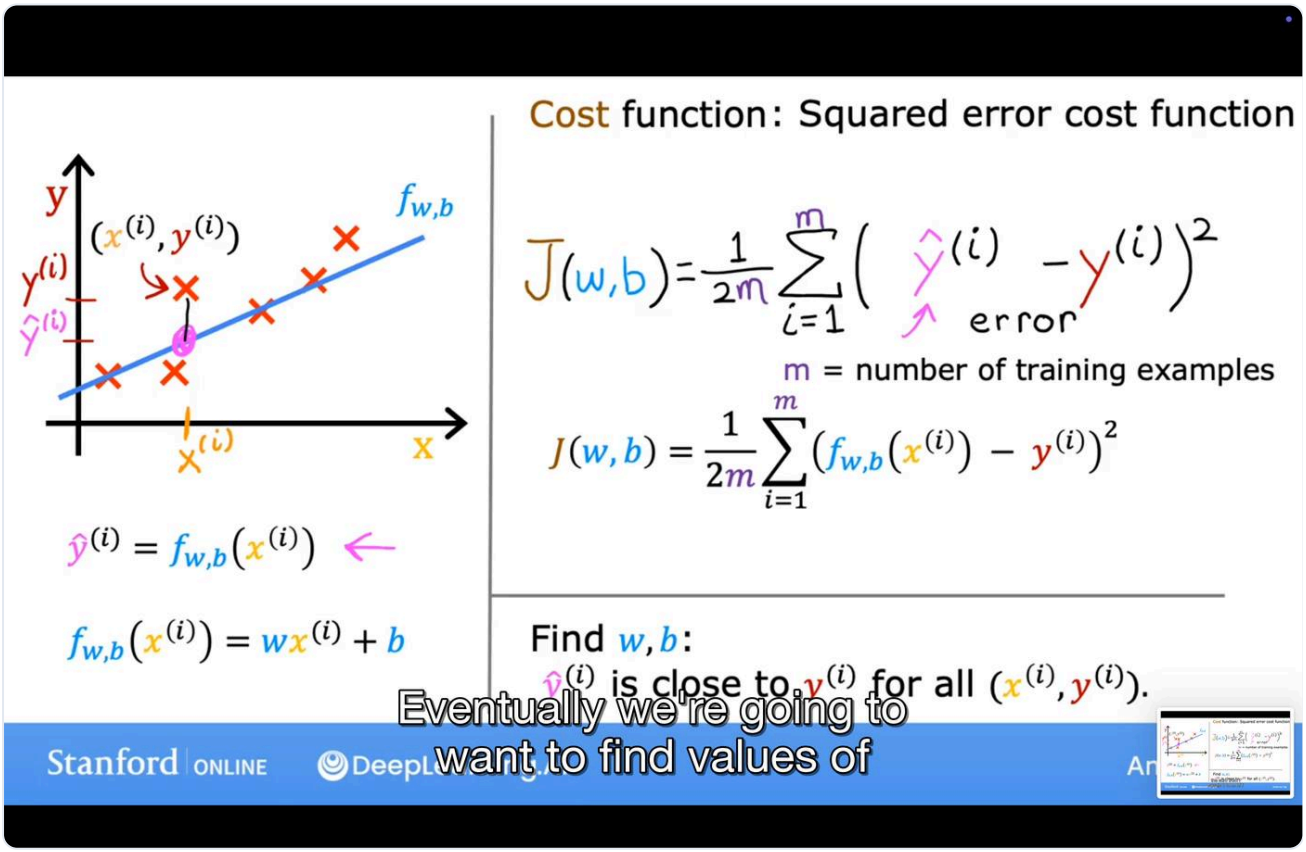


FIGURE 11.8 | Cost Function: Squared Error Cost Function

Eventually we're going to want to find values of w and b that make the cost function small. But before going there, let's first gain more intuition about what J of wb is really computing. At this point you might be thinking we've done a whole lot of math to define the cost function. But what exactly is it doing? Let's go on to the next video where we'll step through one example of what the cost function is really computing that I hope will help you build intuition about what it means if J of wb is large versus if the cost j is small. Let's go on to the next video.

LECTURE 12

Cost Function

We're seeing the mathematical definition of the cost function. Now, let's build some intuition about what the cost function is really doing. In this video, we'll walk through one example to see how the cost function can be used to find the best parameters for your model. I know this video's little bit longer than the others, but bear with me, I think it'll be worth it. To recap, here's what we've seen about the cost function so far. You want to fit a straight line to the training data, so you have this model, fw, b of x is w times x, plus b. Here, the model's parameters are w, and b. Now, depending on the values chosen for these parameters, you get different straight lines like this.

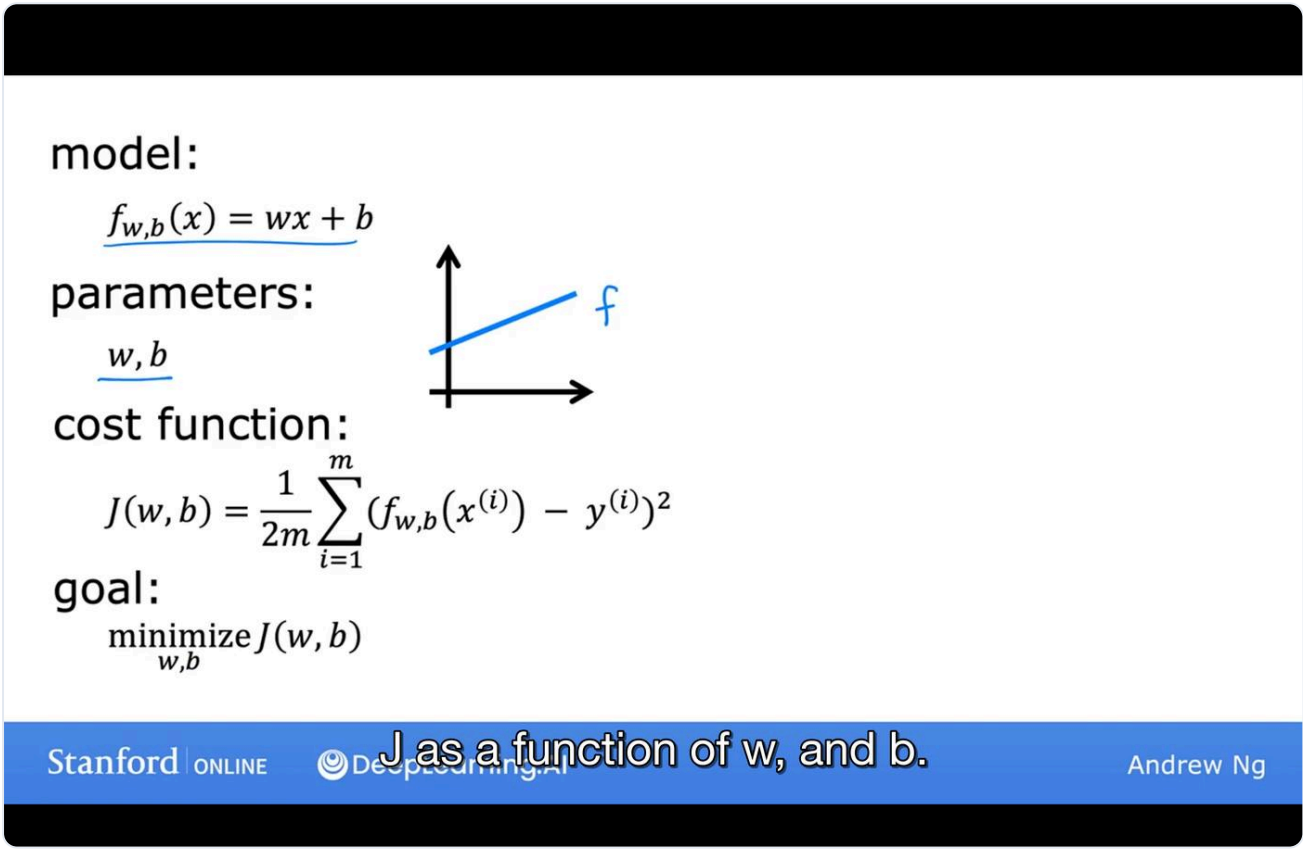


FIGURE 12.1

You want to find values for w, and b, so that the straight line fits the training data well. To measure how well a choice of w, and b fits the training data, you have a cost function J. What the cost function J does is, it measures the difference between the model's predictions, and the actual true values for y. What you see later, is that linear regression would try to find values for w, and b, then make a J of w be as small as possible. In math, we write it like this. We want to minimize, J as a function of w, and b. Now, in order for us to better visualize the cost function J, this work of a simplified version of the linear regression model.

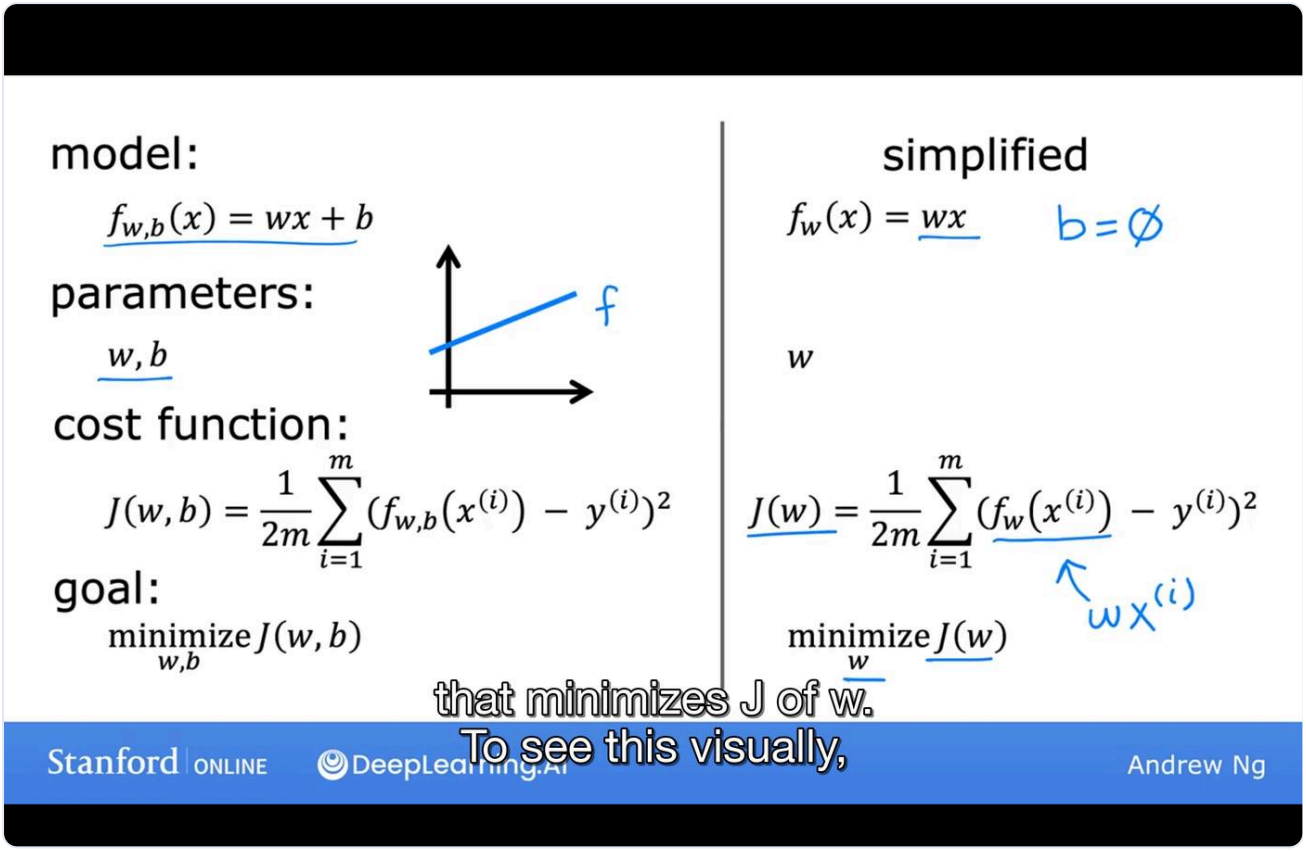


FIGURE 12.2

We're going to use the model fw of x, is w times x. You can think of this as taking the original model on the left, and getting rid of the parameter b, or setting the parameter b equal to 0. It just goes away from the equation, so f is now just w times x. You now have just one parameter w, and your cost function J, looks similar to what it was before. Taking the difference, and squaring it, except now, f is equal to w times xi, and J is now a function of just w. The goal becomes a little bit different as well, because you have just one parameter, w, not w and b. With this simplified model, the goal is to find the value for w, that minimizes J of w. To see this visually, what this means is that if b is set to 0, then f defines a line that looks like this.

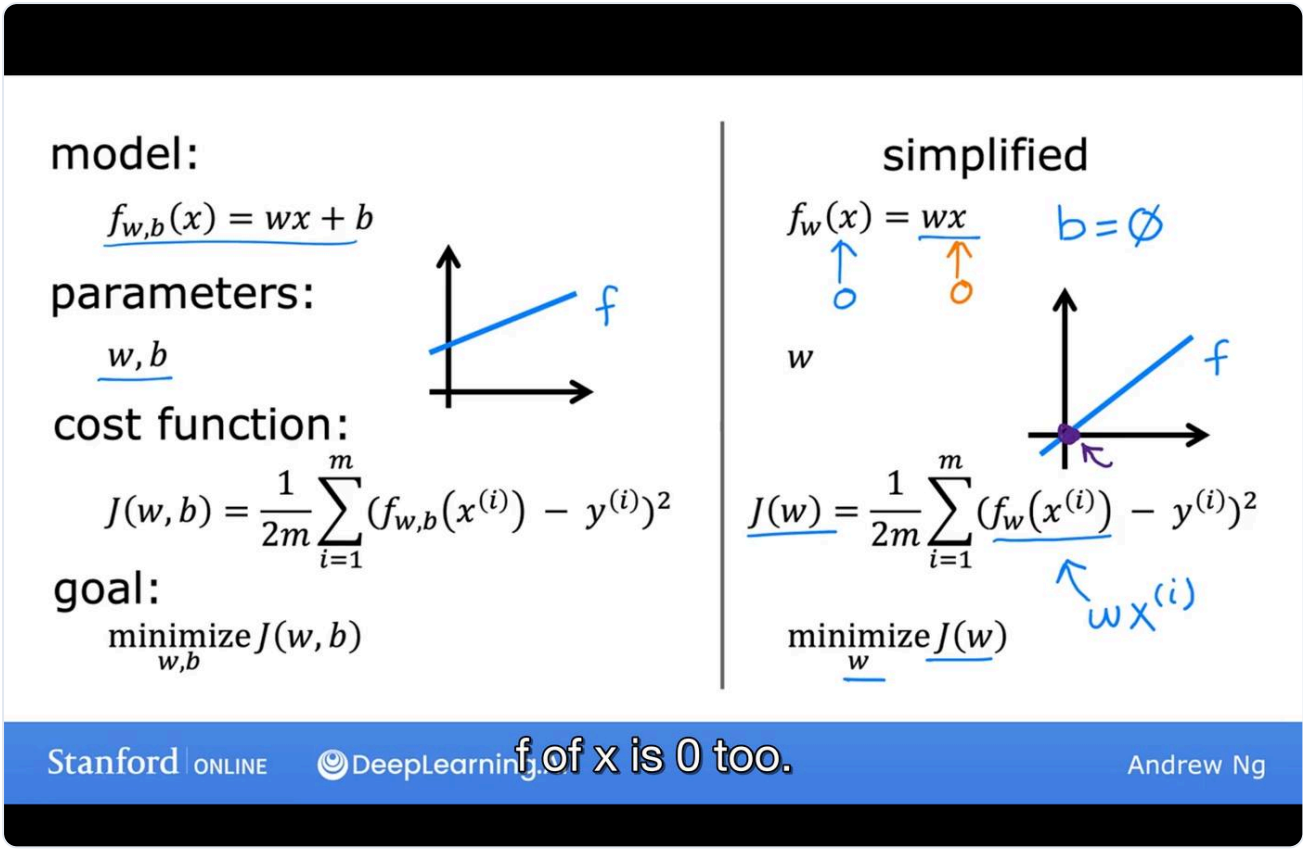


FIGURE 12.3

You see that the line passes through the origin here, because when x is 0, f of x is 0 too. Now, using this simplified model, let's see how the cost function changes as you choose different values for the parameter w. In particular, let's look at graphs of the model f of x, and the cost function J. I'm going to plot these side-by-side, and you'll be able to see how the two are related. First, notice that for f subscript w, when the parameter w is fixed, that is, is always a constant value, then fw is only a function of x, which means that the estimated value of y depends on the value of the input x. In contrast, looking to the right, the cost function J, is a function of w, where w controls the slope of the line defined by f w.

The cost defined by J, depends on a parameter, in this case, the parameter w. Let's go ahead, and plot these functions, fw of x, and J of w side-by-side so you can see how they are related. We'll start with the model, that is the function fw of x on the left. Here are the input feature x is on the horizontal axis, and the output value y is on the vertical axis. Here's the plots of three points representing the training set at positions 1, 1, 2, 2, and 3,3. Let's pick a value for w. Say w is 1. For this choice of w, the function fw, they'll say this straight line with a slope of 1.

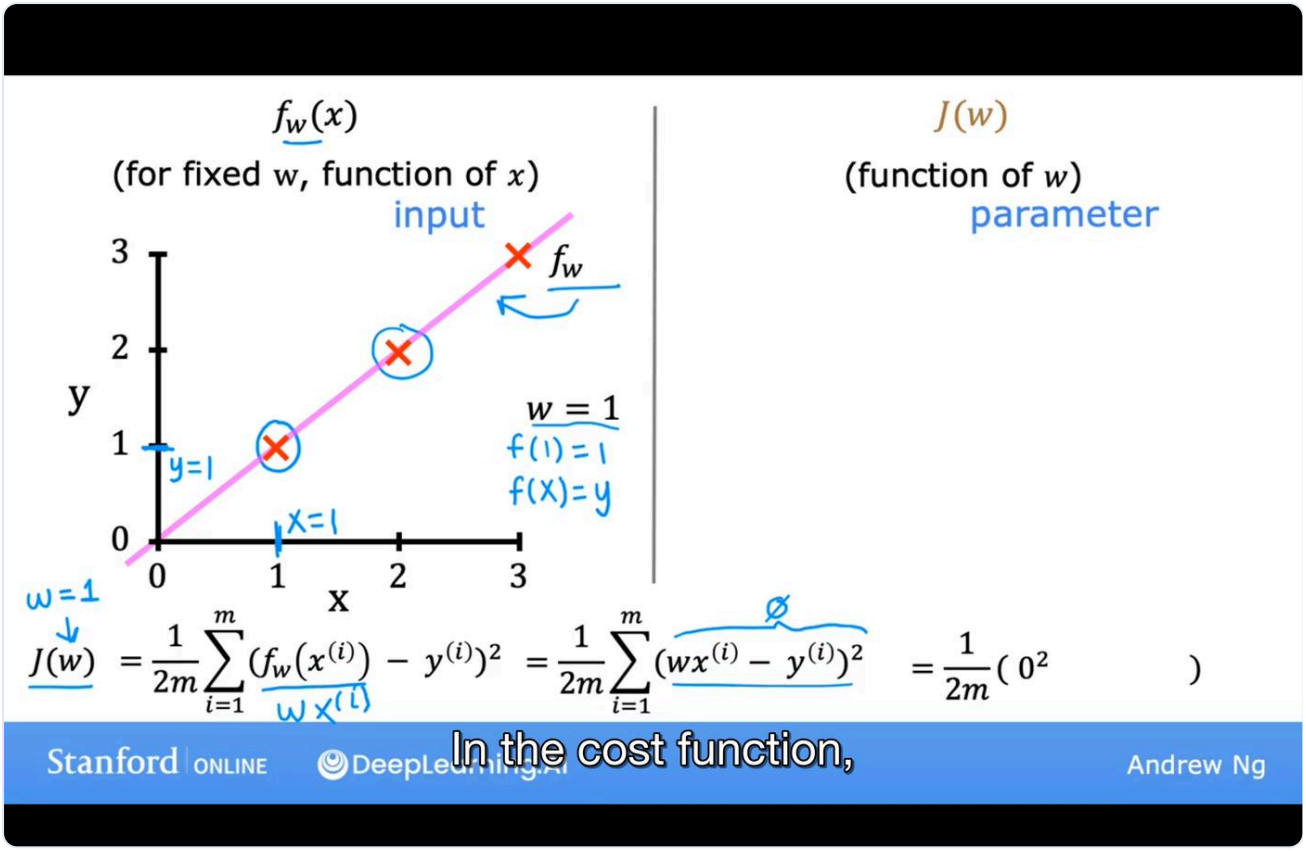


FIGURE 12.4

Now, what you can do next is calculate the cost J when w equals 1. You may recall that the cost function is defined as follows, is the squared error cost function. If you substitute fw(X^i) with w times X^i, the cost function looks like this. Where this expression is now w times X^i minus Y^i. For this value of w, it turns out that the error term inside the cost function, this w times X^i minus Y^i is equal to 0 for each of the three data points. Because for this data-set, when x is 1, then y is 1. When w is also 1, then f(x) equals 1, so f(x) equals y for this first training example, and the difference is 0. Plugging this into the cost function J, you get 0 squared.

Similarly, when x is 2, then y is 2, and f(x) is also 2. Again, f(x) equals y, for the second training example. In the cost function, the squared error for the second example is also 0 squared. Finally, when x is 3, then y is 3 and f(3) is also 3. In a cost function the third squared error term is also 0 squared.

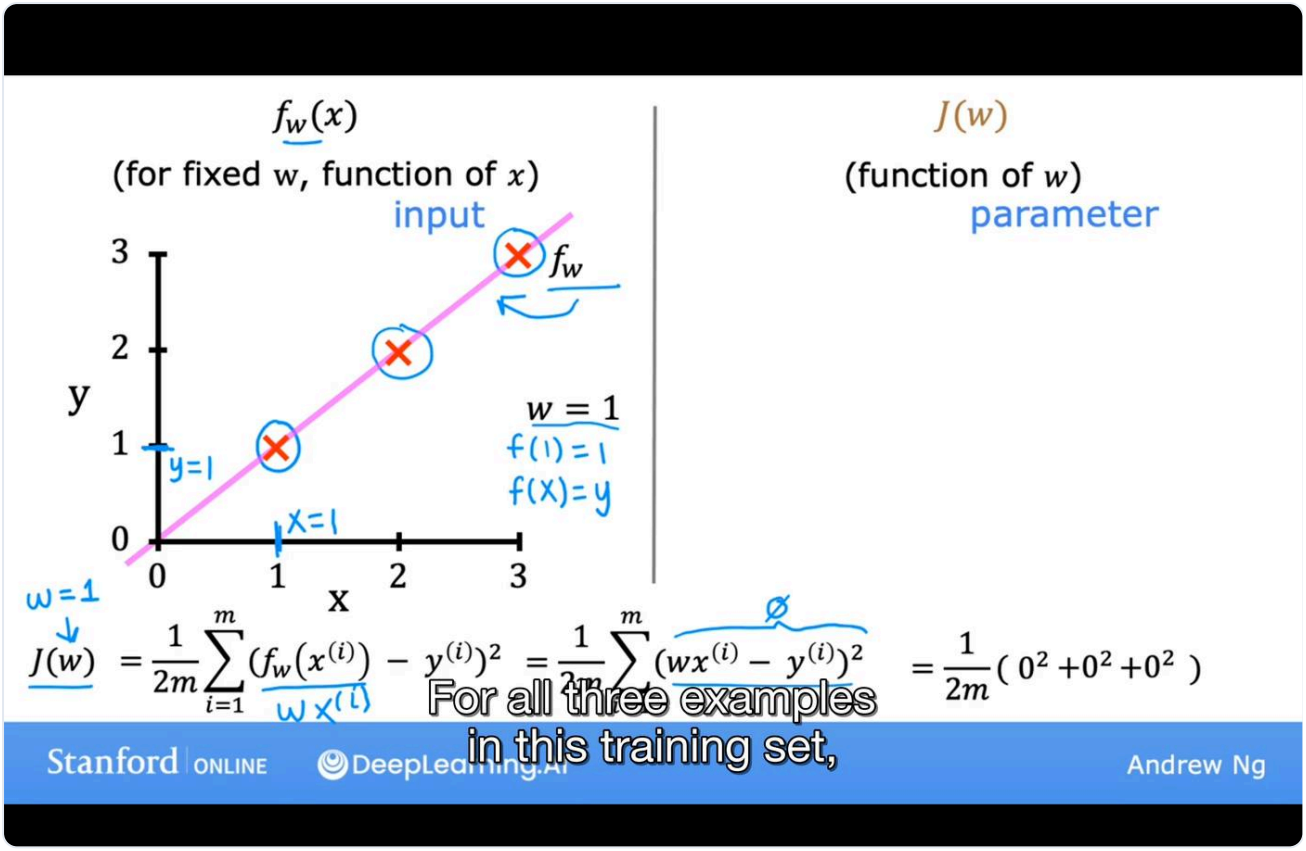


FIGURE 12.5

For all three examples in this training set, f(X^i) equals Y^i for each training example i, so f(X^i) minus Y^i is 0. For this particular data-set, when w is 1, then the cost J is equal to 0. Now, what you can do on the right is plot the cost function J. Notice that because the cost function is a function of the parameter w, the horizontal axis is now labeled w and not x, and the vertical axis is now J and not y.

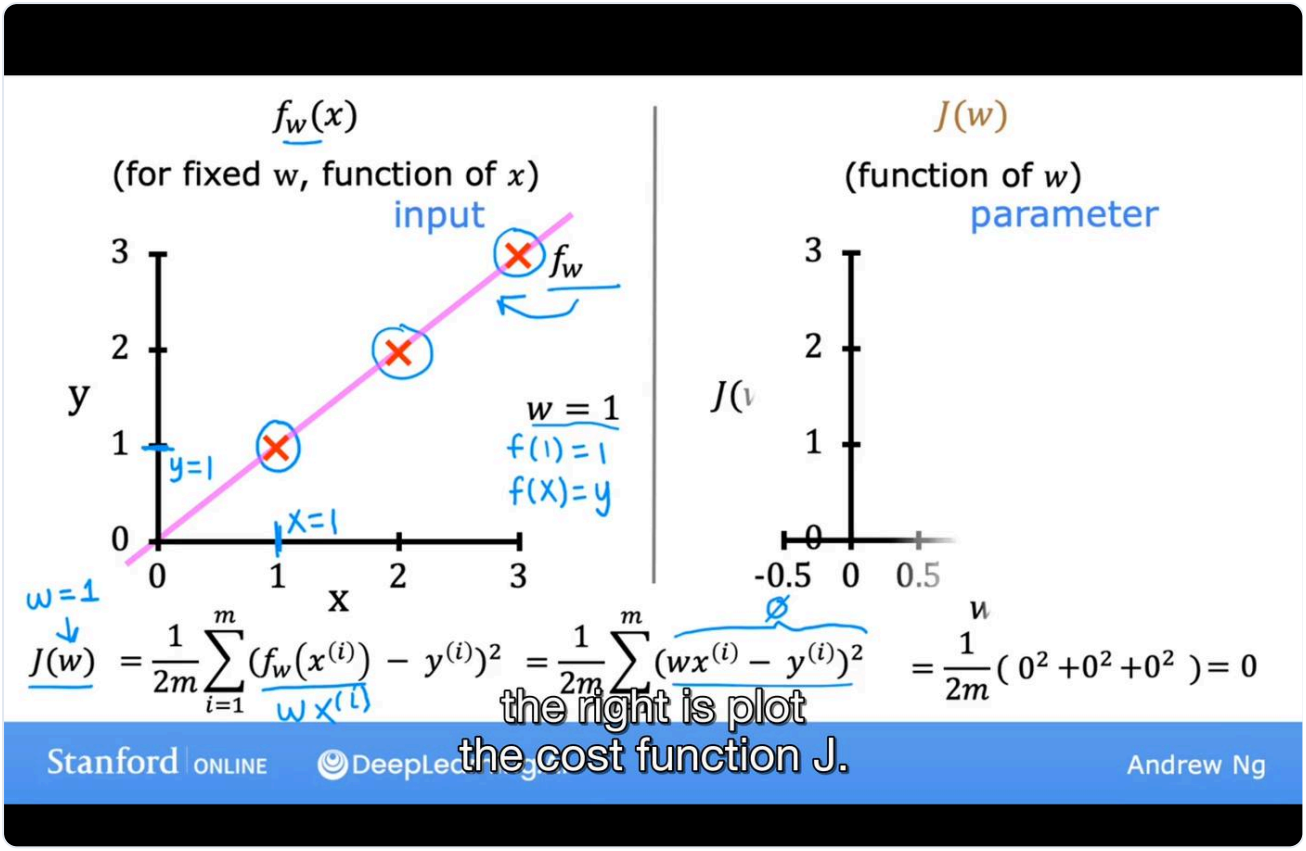


FIGURE 12.6

You have J(1) equals to 0. In other words, when w equals 1, J(w) is 0, so let me go ahead and plot that. Now, let's look at how F and J change for different values of w.

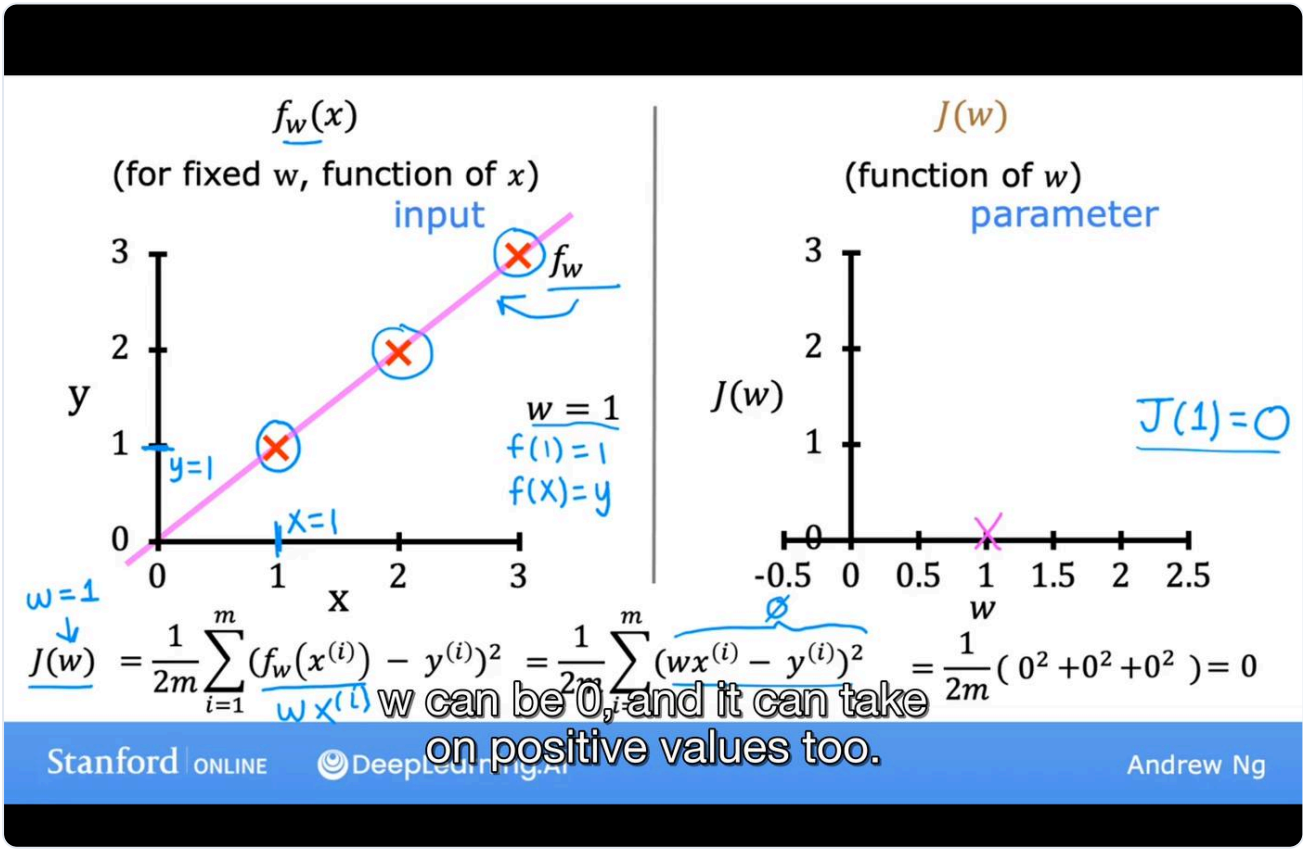


FIGURE 12.7

W can take on a range of values, so w can take on negative values, w can be 0, and it can take on positive values too. What if w is equal to 0.5 instead of 1, what would these graphs look like then? Let's go ahead and plot that. Let's set w to be equal to 0.5, and in this case, the function f(x) now looks like this, is a line with a slope equal to 0.5. Let's also compute the cost J, when w is 0.5. Recall that the cost function is measuring the squared error or difference between the estimator value, that is y hat I, which is F(X^i), and the true value, that is Y^i for each example i. Visually you can see that the error or difference is equal to the height of this vertical line here when x is equal to 1.

Because this lower line is the gap between the actual value of y and the value that the function f predicted, which is a bit further down here. For this first example, when x is 1, f(x) is 0.5. The squared error on the first example is 0.5 minus 1 squared. Remember the cost function, we'll sum over all the training examples in the training set. Let's go on to the second training example. When x is 2, the model is predicting f(x) is 1 and the actual value of y is 2. The error for the second example is equal to the height of this little line segment here, and the squared error is the square of the length of this line segment, so you get 1 minus 2 squared.

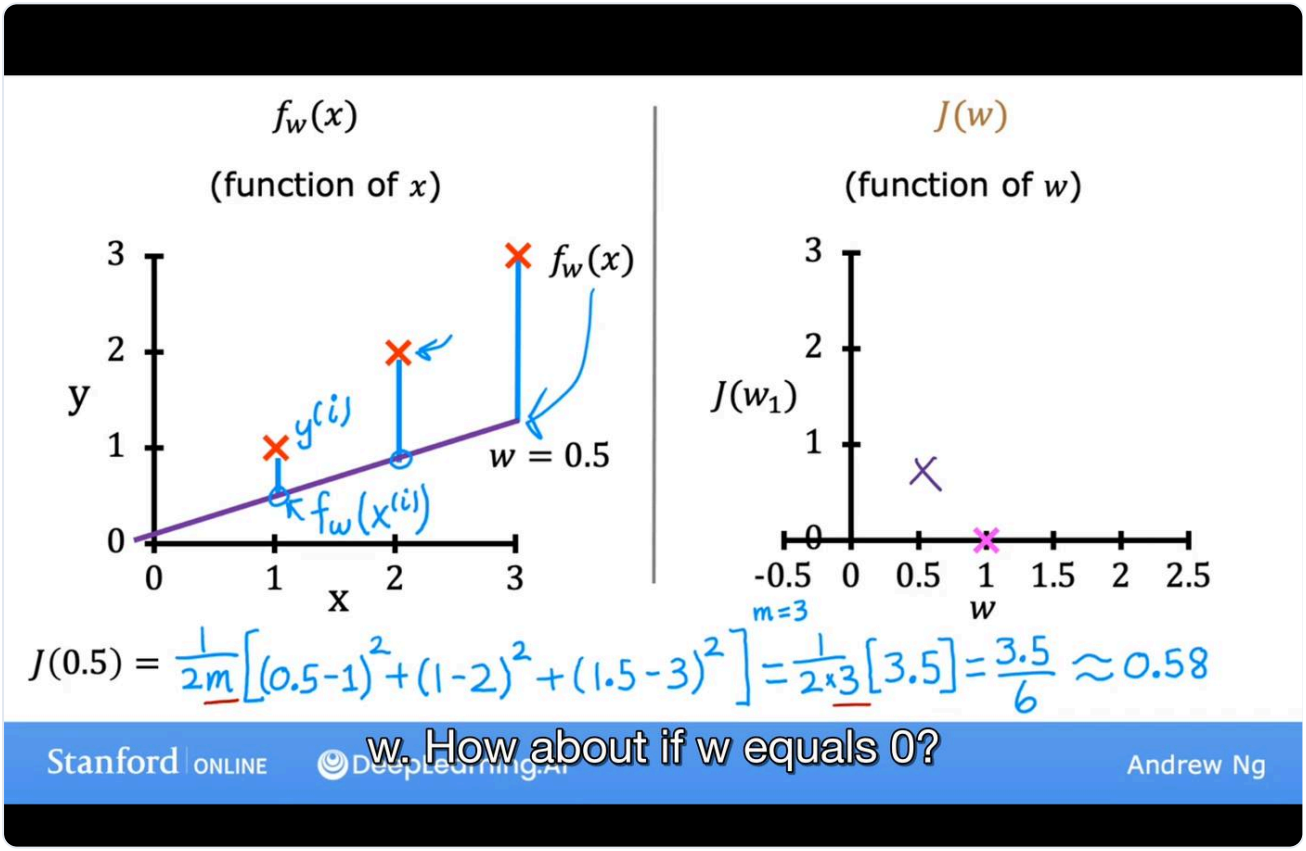


FIGURE 12.8

Let's do the third example. Repeating this process, the error here, also shown by this line segment, is 1.5 minus 3 squared. Next, we sum up all of these terms, which turns out to be equal to 3.5. Then we multiply this term by 1 over 2m, where m is the number of training examples. Since there are three training examples m equals 3, so this is equal to 1 over 2 times 3, where this m here is 3. If we work out the math, this turns out to be 3.5 divided by 6. The cost J is about 0.58. Let's go ahead and plot that over there on the right. Now, let's try one more value for w. How about if w equals 0? What do the graphs for f and J look like when w is equal to 0?

It turns out that if w is equal to 0, then f of x is just this horizontal line that is exactly on the x-axis. The error for each example is a line that goes from each point down to the horizontal line that represents f of x equals 0. The cost J when w equals 0 is 1 over 2m times the quantity, 1^2 plus 2^2 plus 3^2, and that's equal to 1 over 6 times 14, which is about 2.33.

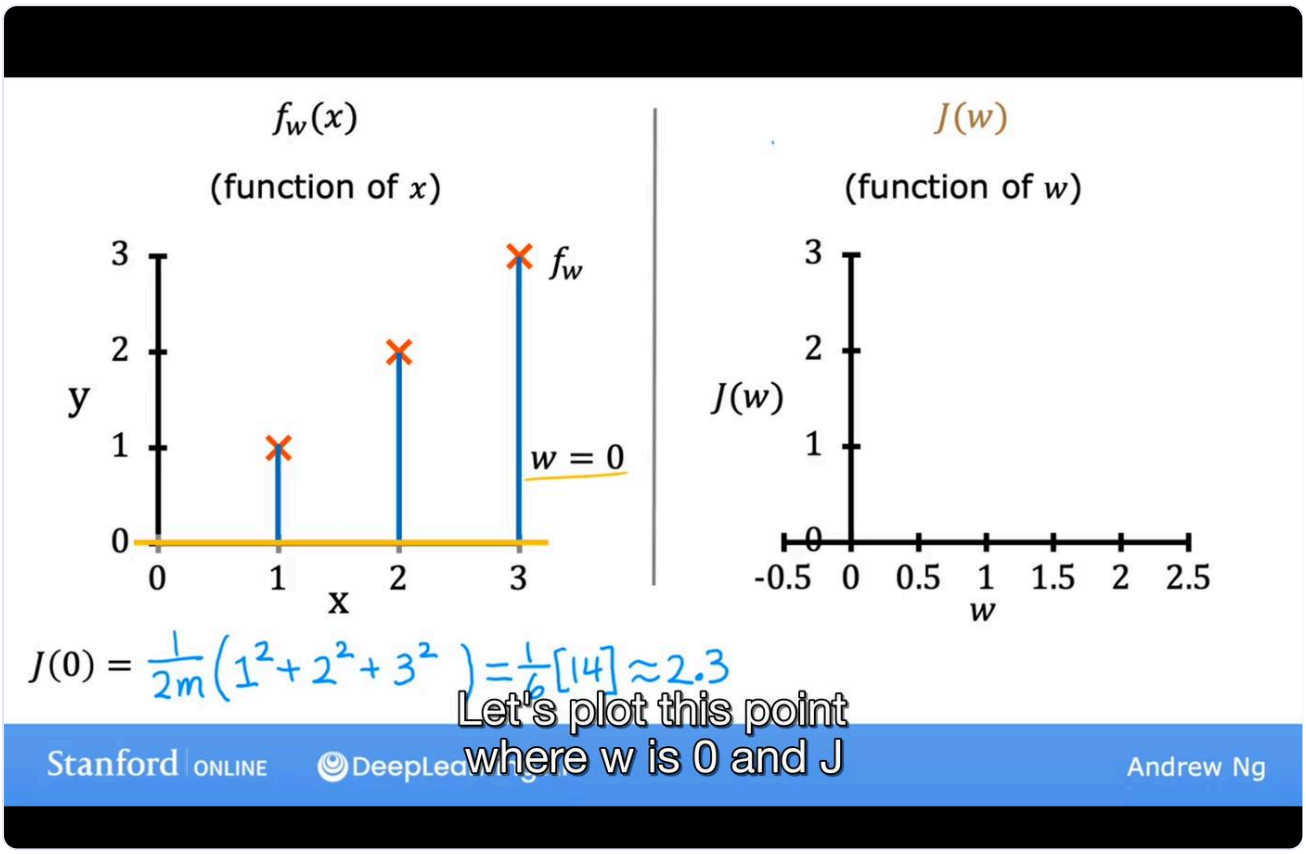


FIGURE 12.9

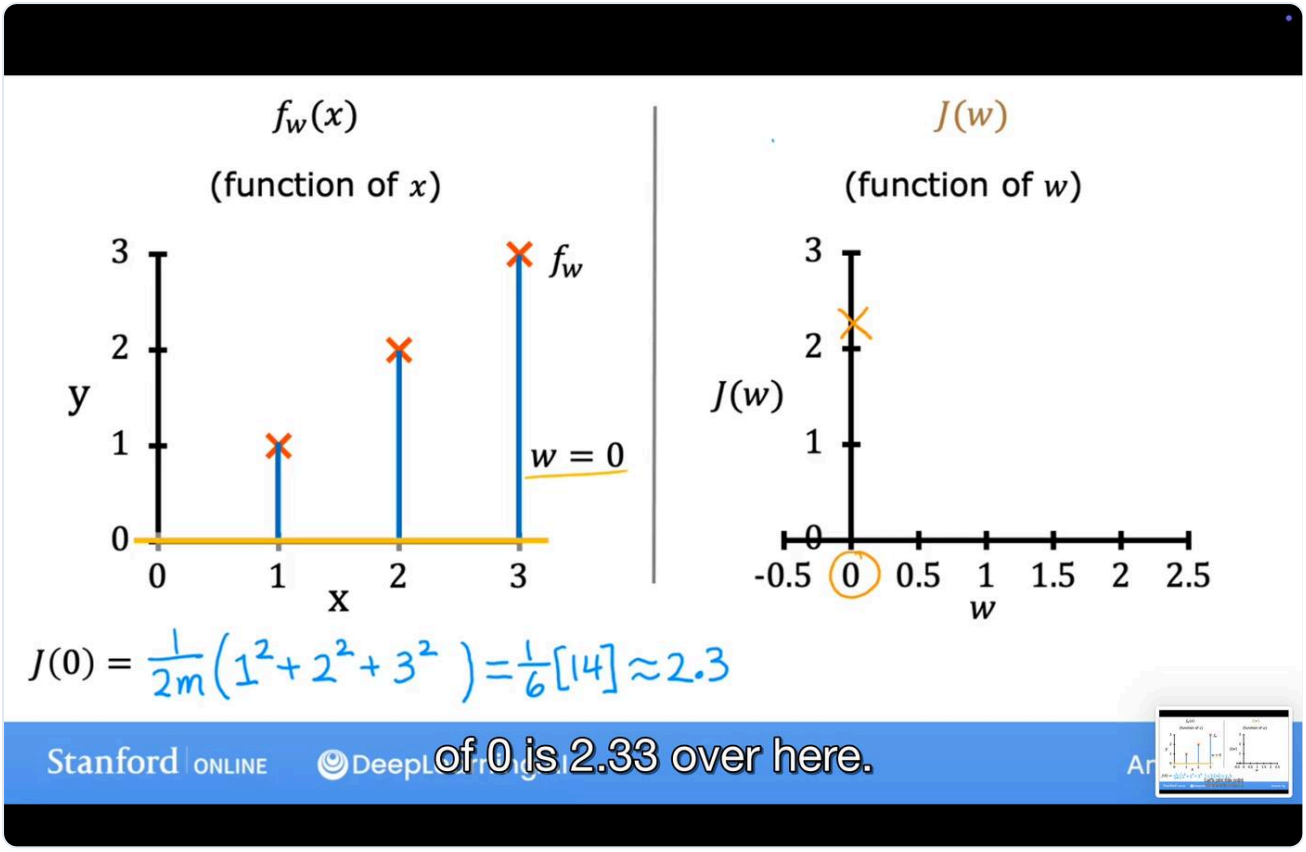


FIGURE 12.10

Let's plot this point where w is 0 and J of 0 is 2.33 over here. You can keep doing this for other values of w. Since w can be any number, it can also be a negative value. If w is negative 0.5, then the line f is a downward-sloping line like this. It turns out that when w is negative 0.5 then you end up with an even higher cost, around 5.25, which is this point up here. You can continue computing the cost function for different values of w and so on and plot these.

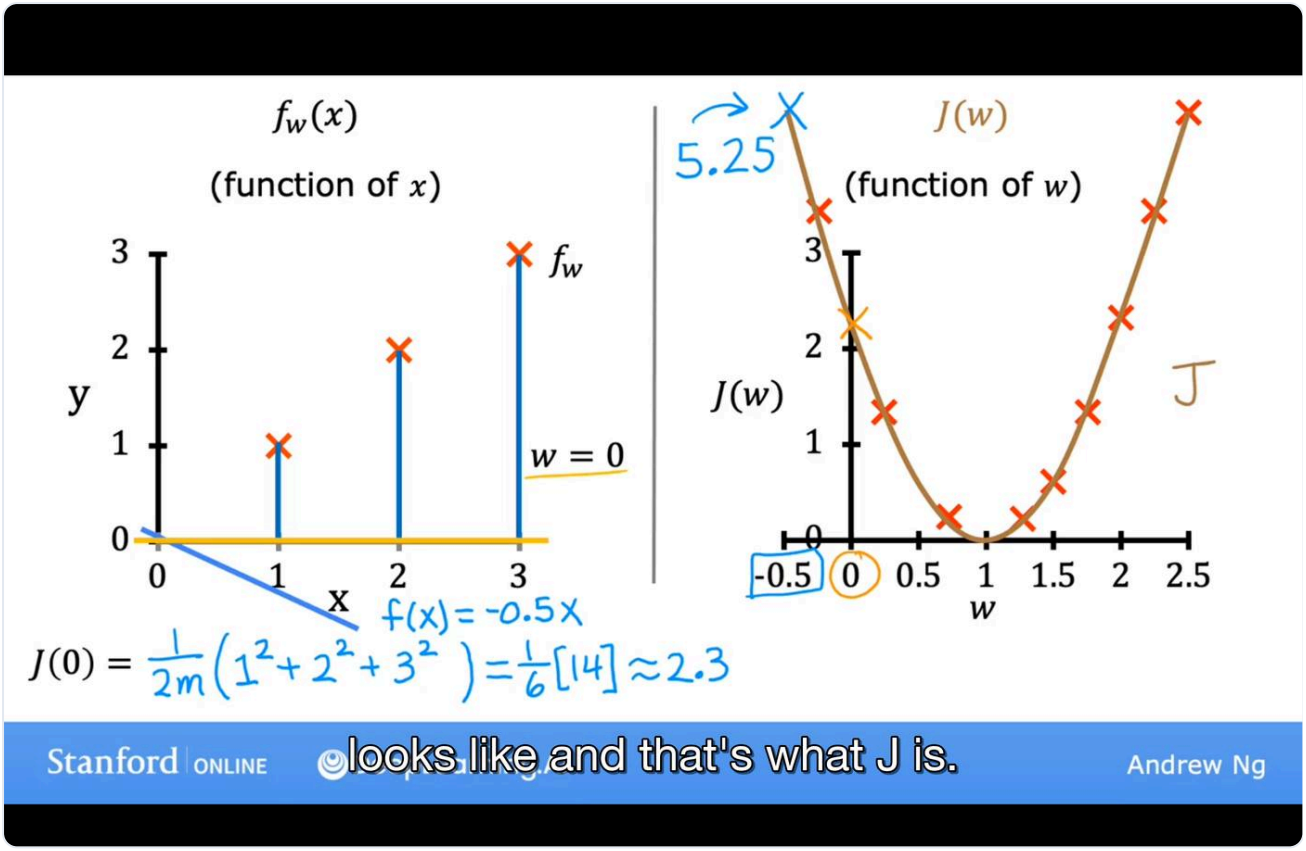


FIGURE 12.11

It turns out that by computing a range of values, you can slowly trace out what the cost function J looks like and that's what J is. To recap, each value of parameter w corresponds to different straight line fit, f of x, on the graph to the left.

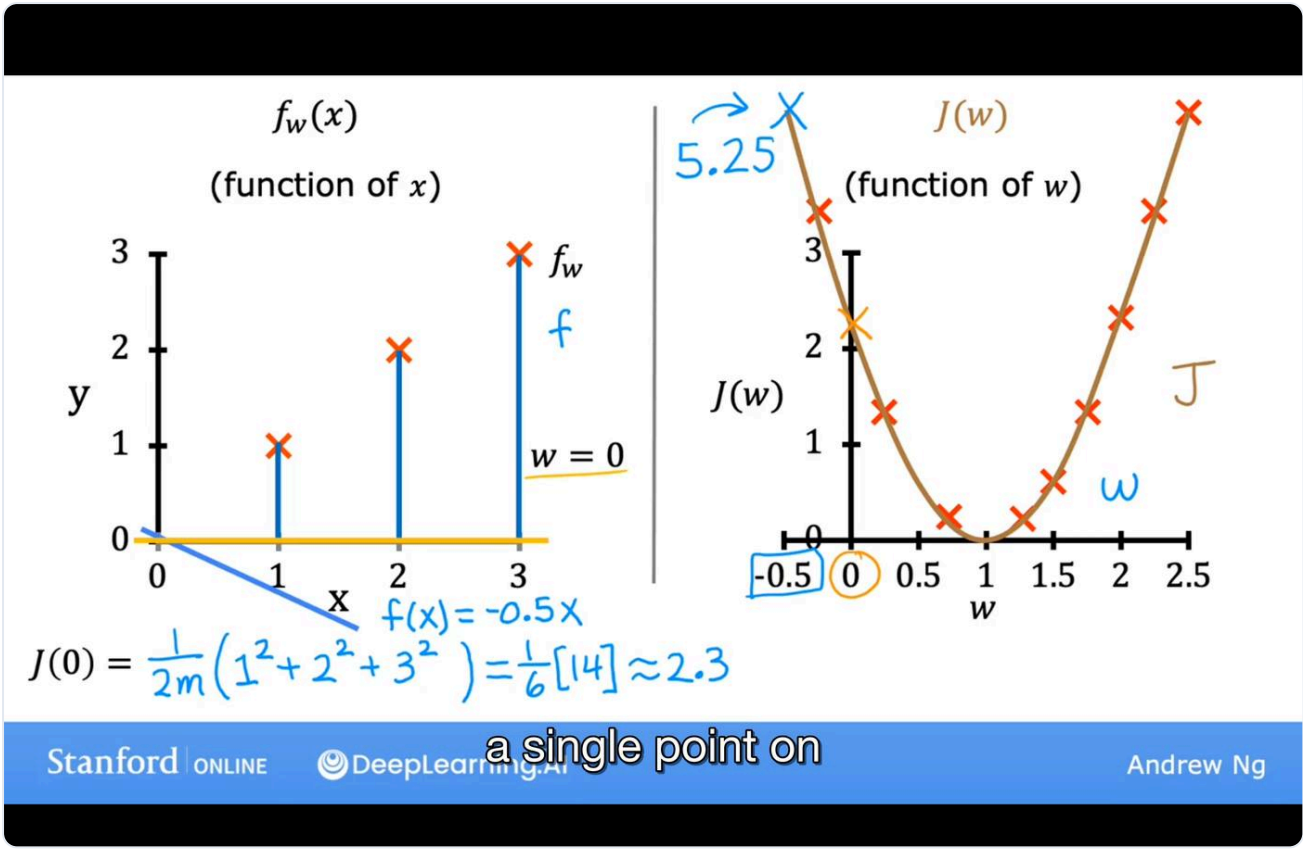


FIGURE 12.12

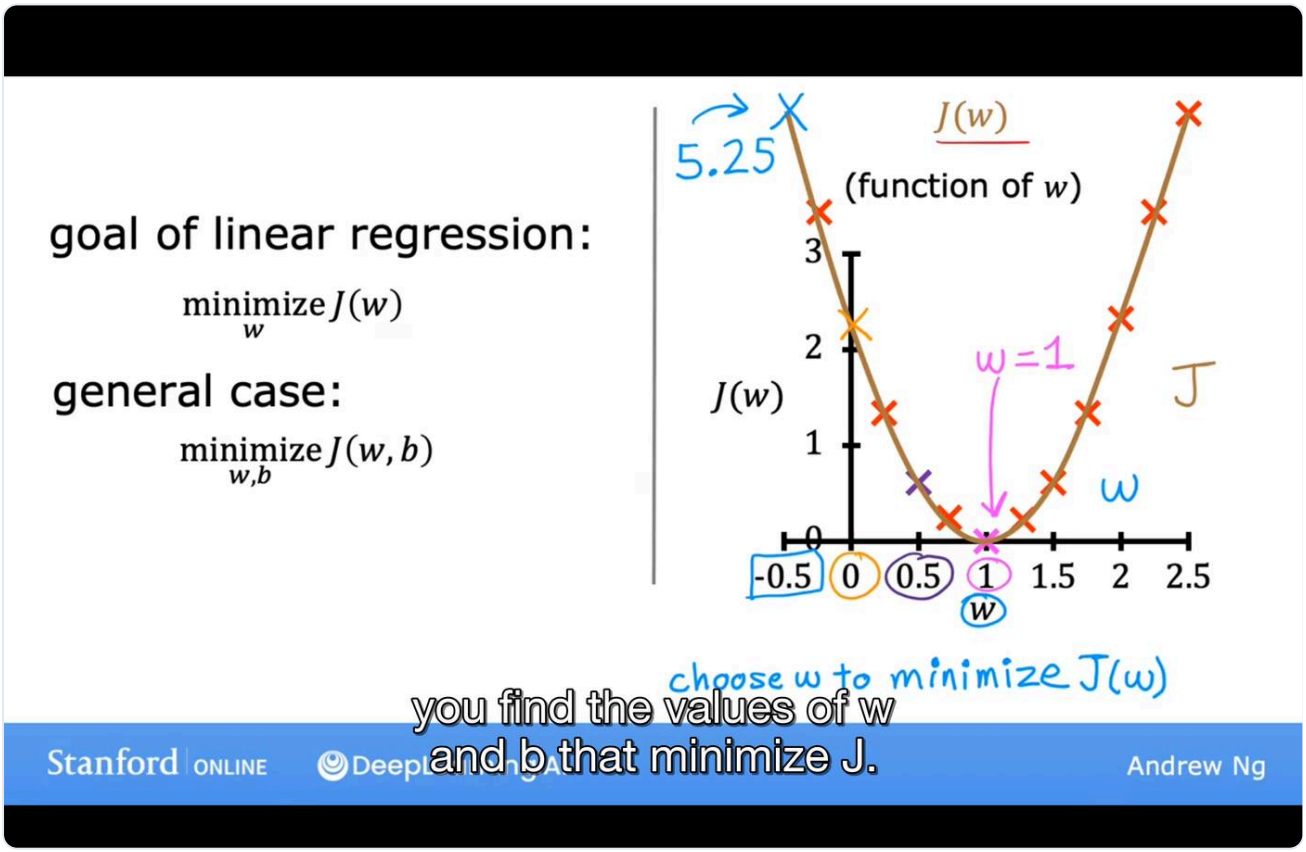


FIGURE 12.13

For the given training set, that choice for a value of w corresponds to a single point on the graph on the right because for each value of w, you can calculate the cost J of w. For example, when w equals 1, this corresponds to this straight line fit through the data and it also corresponds to this point on the graph of J, where w equals 1 and the cost J of 1 equals 0. Whereas when w equals 0.5, this gives you this line which has a smaller slope. This line in combination with the training set corresponds to this point on the cost function graph at w equals 0.5. For each value of w you wind up with a different line and its corresponding costs, J of w, and you can use these points to trace out this plot on the right.

Given this, how can you choose the value of w that results in the function f, fitting the data well? Well, as you can imagine, choosing a value of w that causes J of w to be as small as possible seems like a good bet. J is the cost function that measures how big the squared errors are, so choosing w that minimizes these squared errors, makes them as small as possible, will give us a good model. In this example, if you were to choose the value of w that results in the smallest possible value of J of w you'd end up picking w equals 1. As you can see, that's actually a pretty good choice. This results in the line that fits the training data very well.

That's how in linear regression you use the cost function to find the value of w that minimizes J. In the more general case where we had parameters w and b rather than just w, you find the values of w and b that minimize J. To summarize, you saw plots of both f and J and worked through how the two are related. As you vary w or vary w and b you end up with different straight lines and when that straight line passes across the data, the cause J is small. The goal of linear regression is to find the parameters w or w and b that results in the smallest possible value for the cost function J. Now in this video, we worked through our example with a simplified problem using only w.

In the next video, let's visualize what the cost function looks like for the full version of linear regression using both w and b. You see some cool 3D plots. Let's go to the next video.

LECTURE 13

3D Surface Plot

In the last video, you saw one visualization of the cost function J of w or J of w, b. Let's look at some further richer visualizations so that you can get an even better intuition about what the cost function is doing.

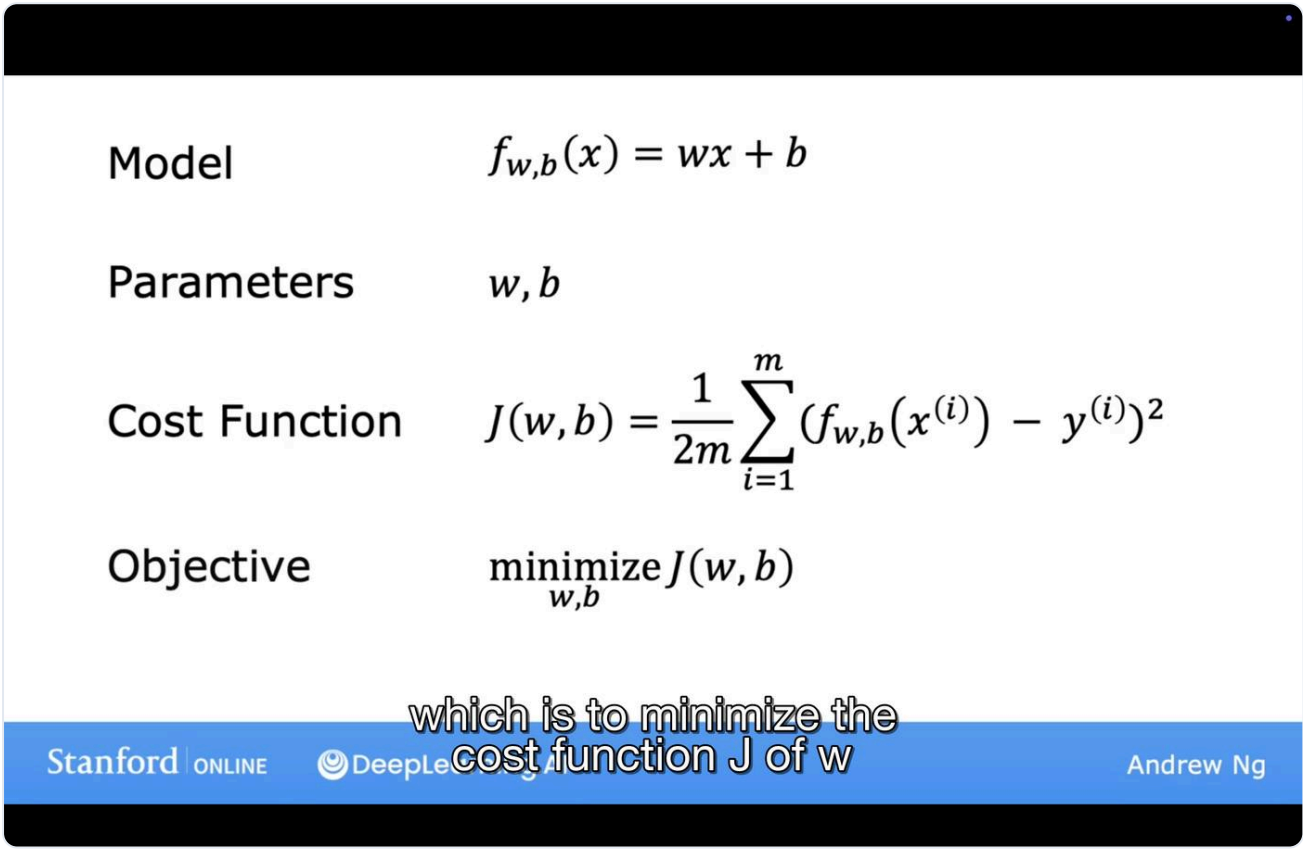


FIGURE 13.1 | Parameters W,b

Here is what we've seen so far. There's the model, the model's parameters w and b, the cost function J of w and b, as well as the goal of linear regression, which is to minimize the cost function J of w and b over parameters w and b. In the last video, we had temporarily set b to zero in order to simplify the visualizations.

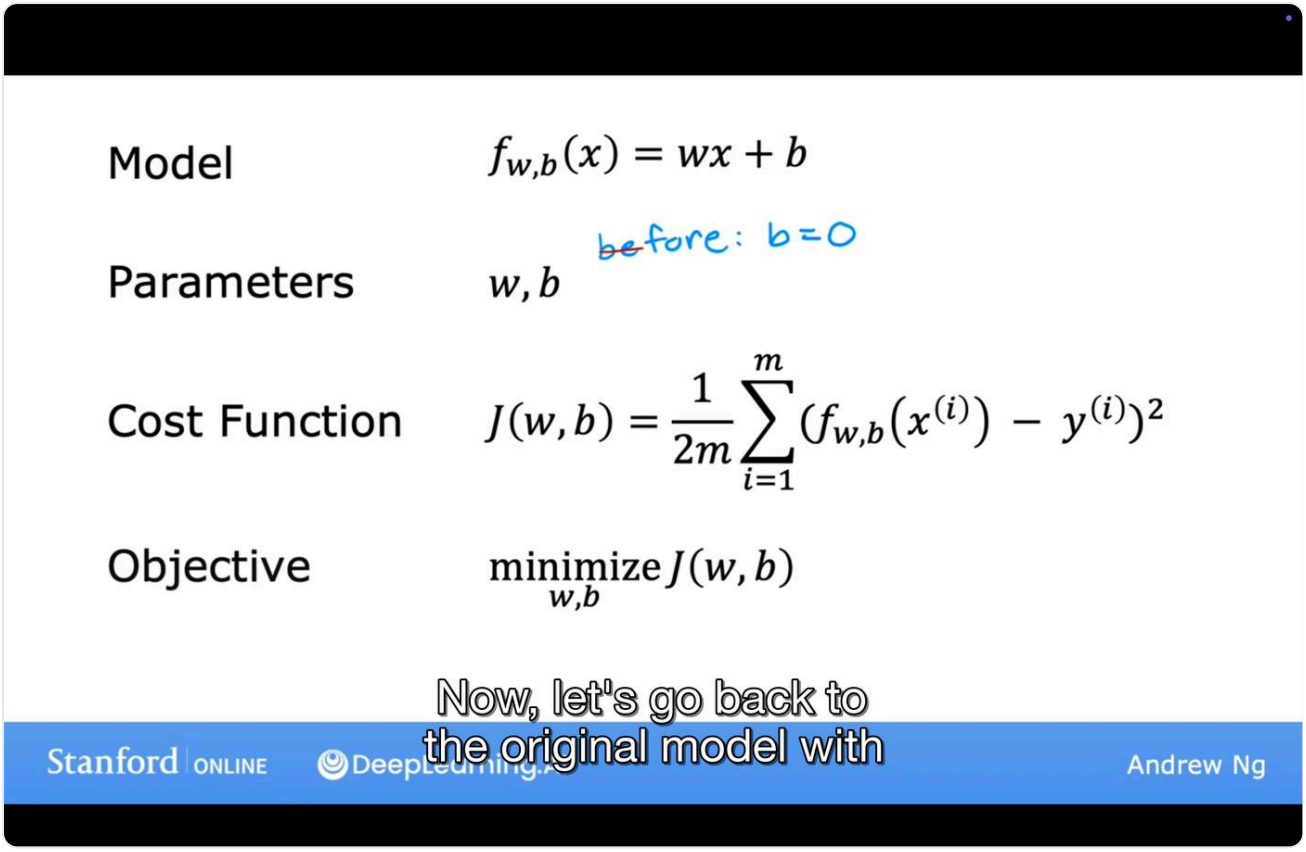


FIGURE 13.2 | Parameters W,b

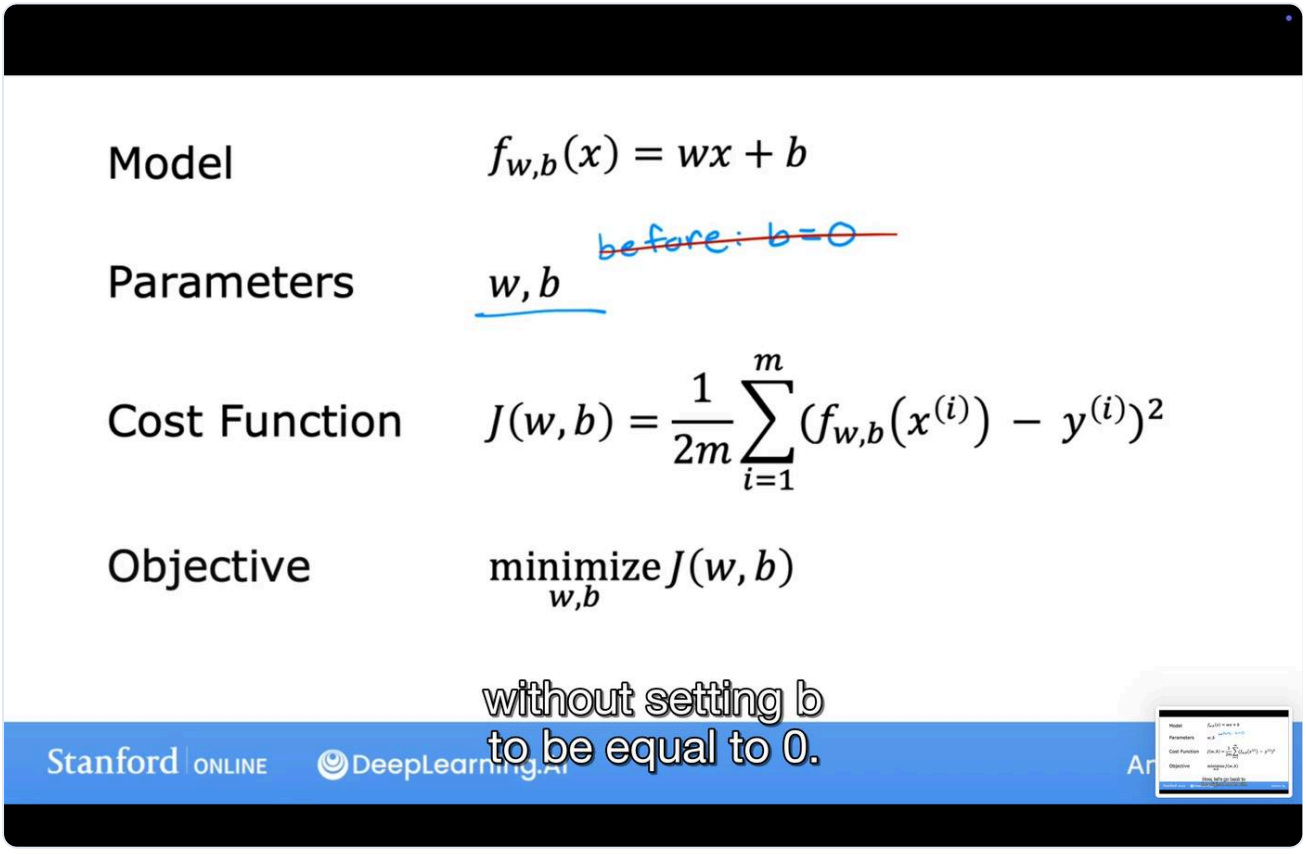


FIGURE 13.3 | Parameters W,b

Now, let's go back to the original model with both parameters w and b without setting b to be equal to 0. Same as last time, we want to get a visual understanding of the model function, f of x, shown here on the left, and how it relates to the cost function J of w, b, shown here on the right. Here's a training set of house sizes and prices. Let's say you pick one possible function of x, like this one.

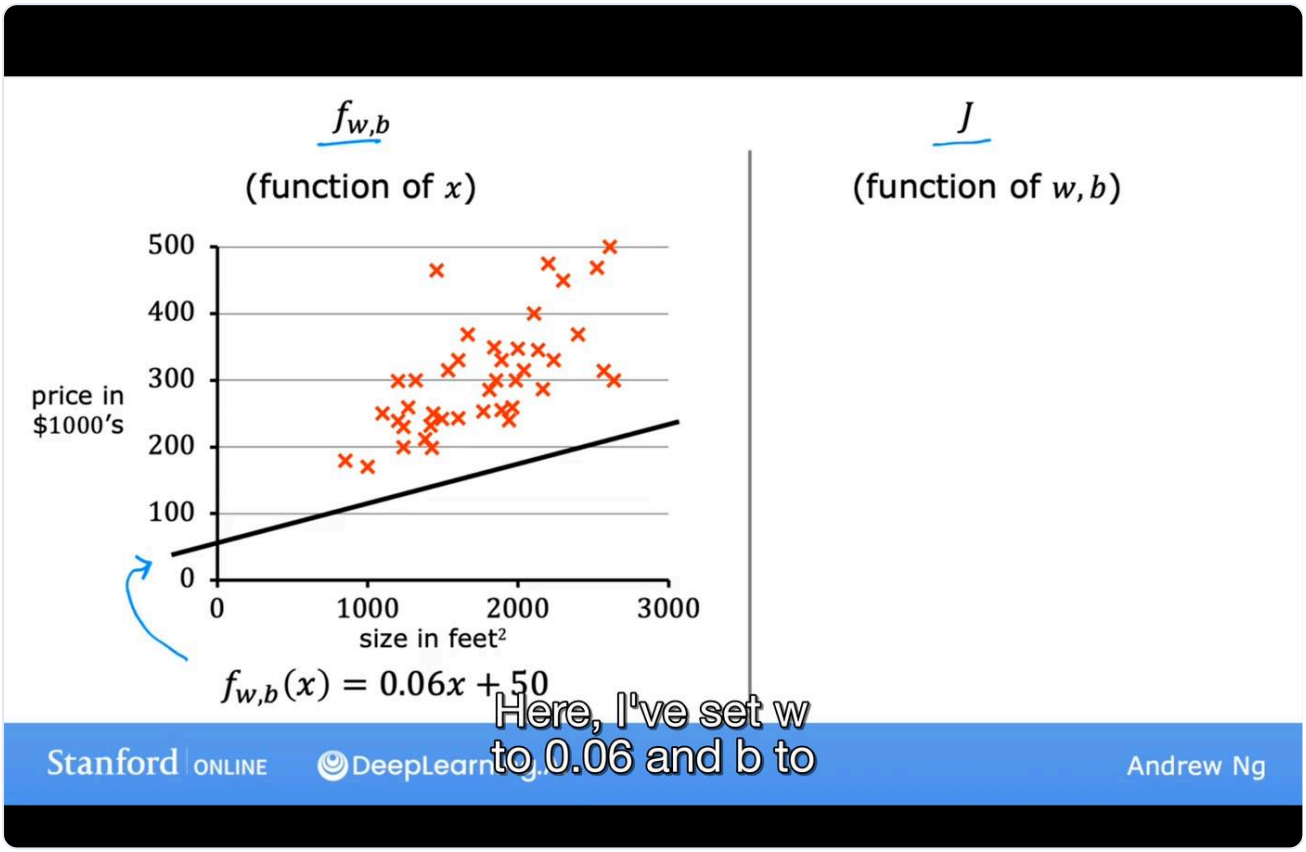


FIGURE 13.4

Here, I've set w to 0.06 and b to 50. f of x is 0.06 times x plus 50.

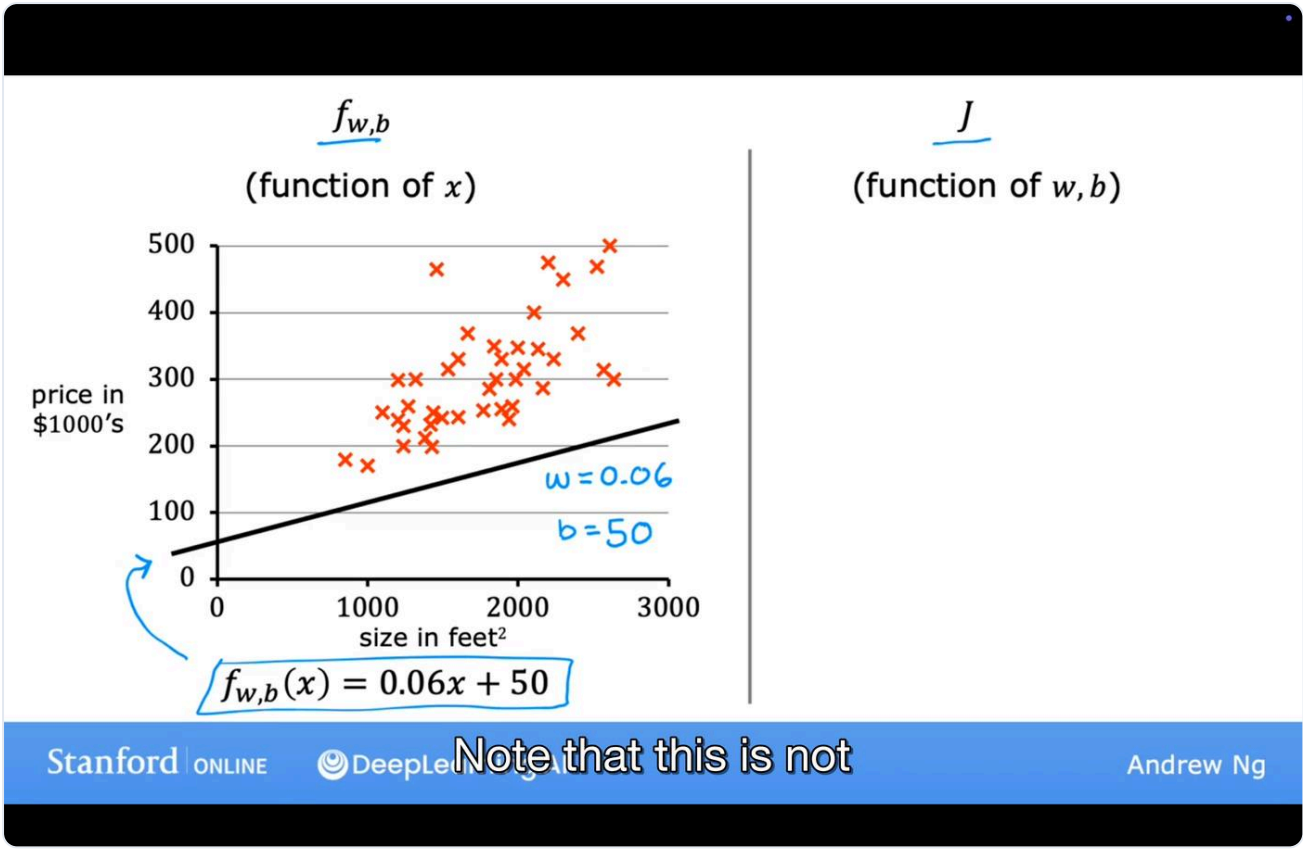


FIGURE 13.5

Note that this is not a particularly good model for this training set, is actually a pretty bad model. It seems to consistently underestimate housing prices. Given these values for w and b let's look at what the cost function J of w and b may look like. Recall what we saw last time was when you had only w, because we temporarily set b to zero to simplify things, but then we had come up with a plot of the cost function that look like this as a function of w only. When we had only one parameter, w, the cost function had this U-shaped curve, shaped a bit like a soup bowl.

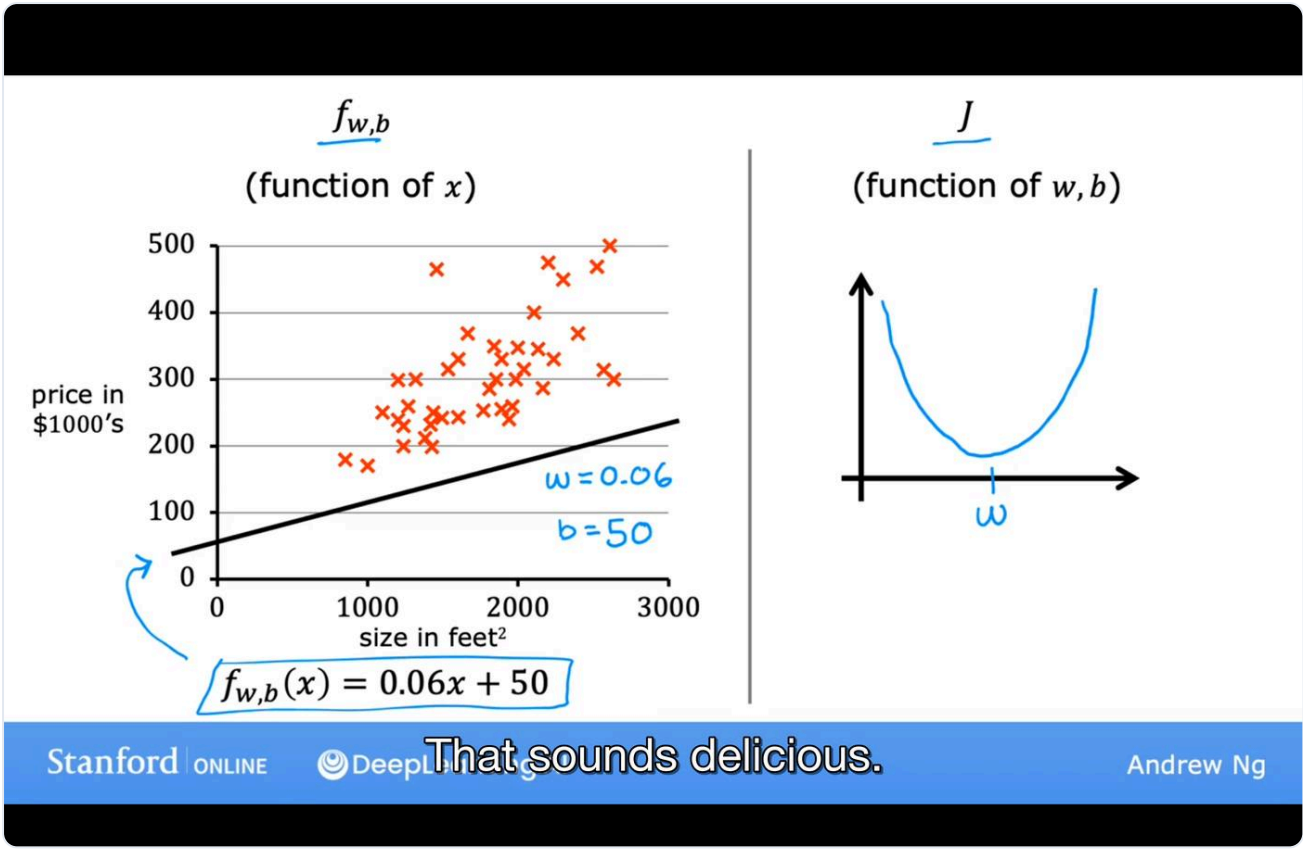


FIGURE 13.6

That sounds delicious. Now, in this housing price example that we have on this slide, we have two parameters, w and b.

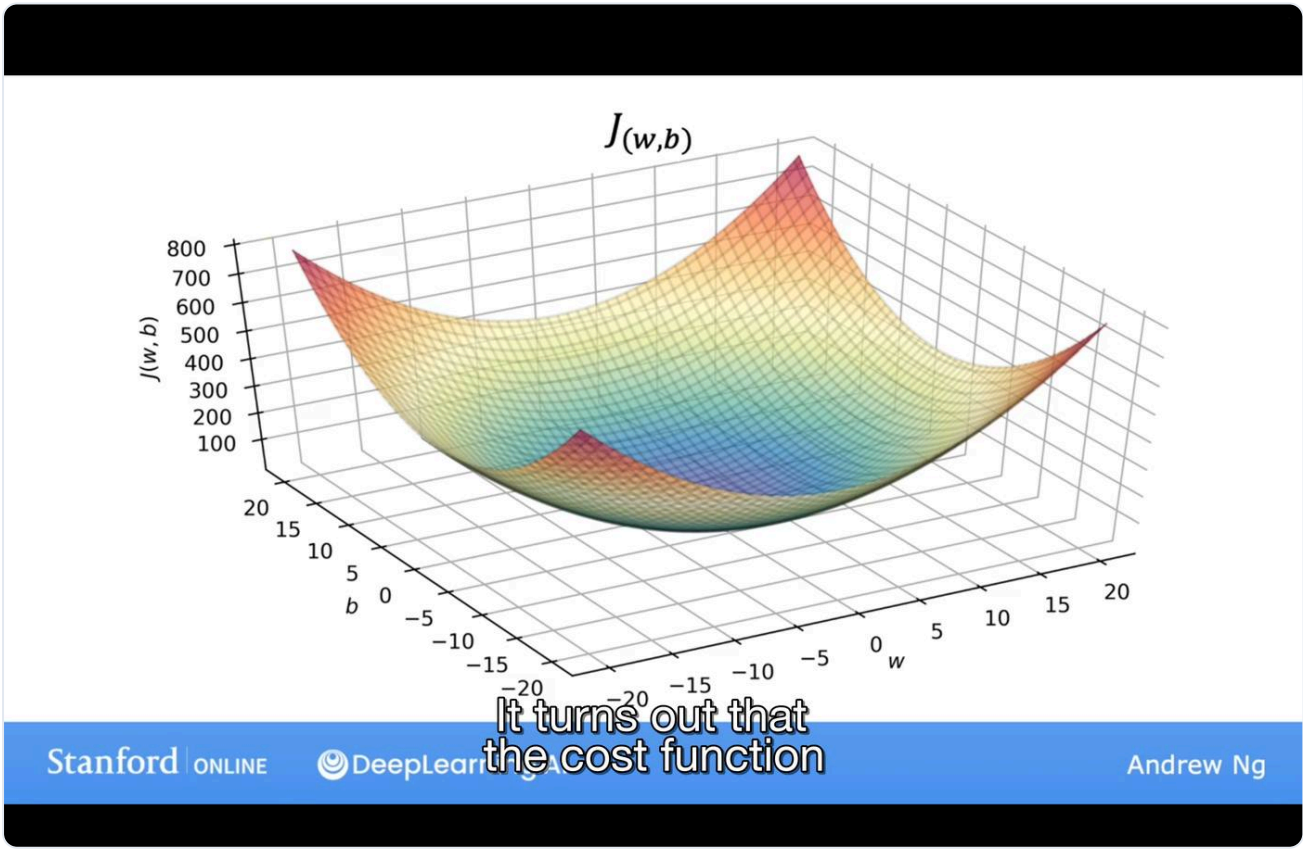


FIGURE 13.7

The plots becomes a little more complex. It turns out that the cost function also has a similar shape like a soup bowl, except in three dimensions instead of two. In fact, depending on your training set, the cost function will look something like this.

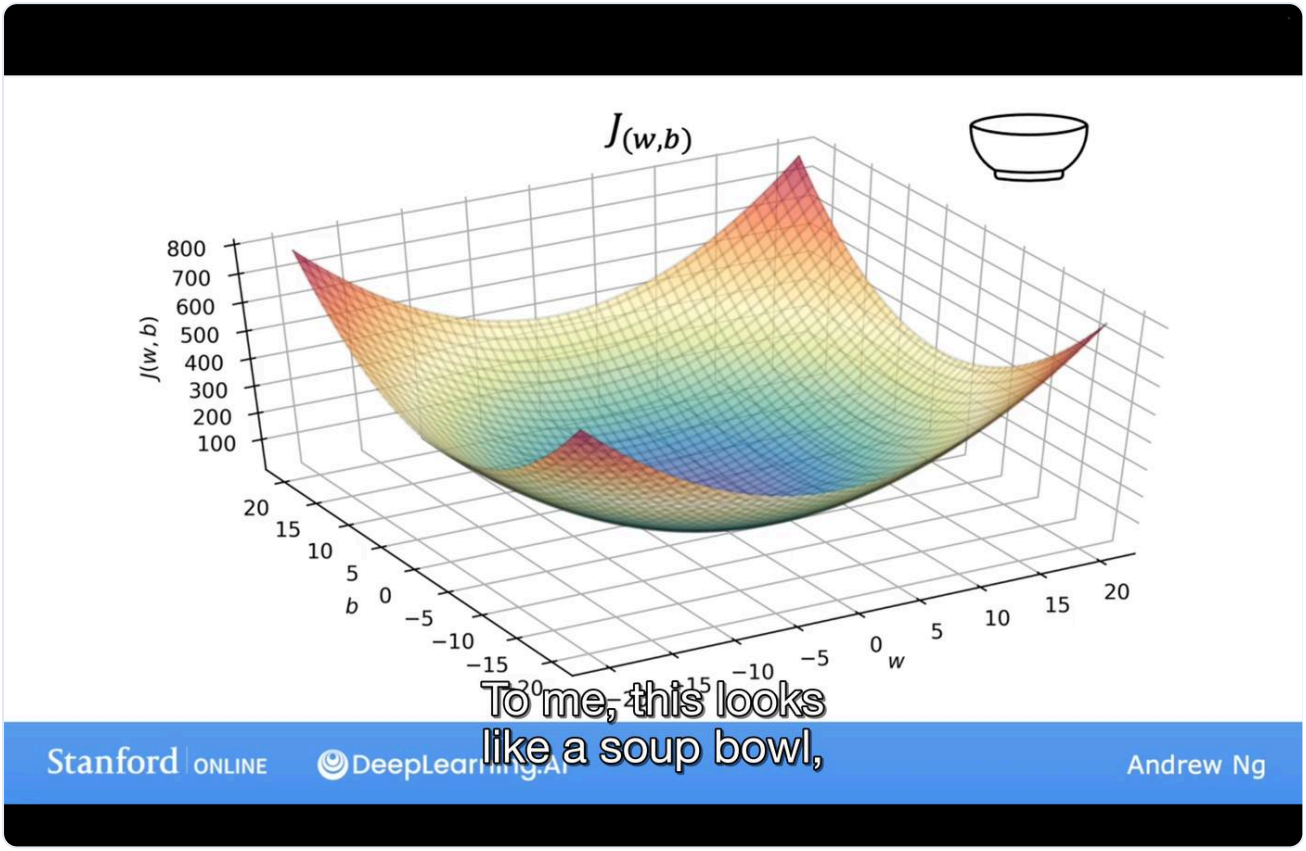


FIGURE 13.8

To me, this looks like a soup bowl, maybe because I'm a little bit hungry, or maybe to you it looks like a curved dinner plate or a hammock. Actually that sounds relaxing too, and there's your coconut drink.

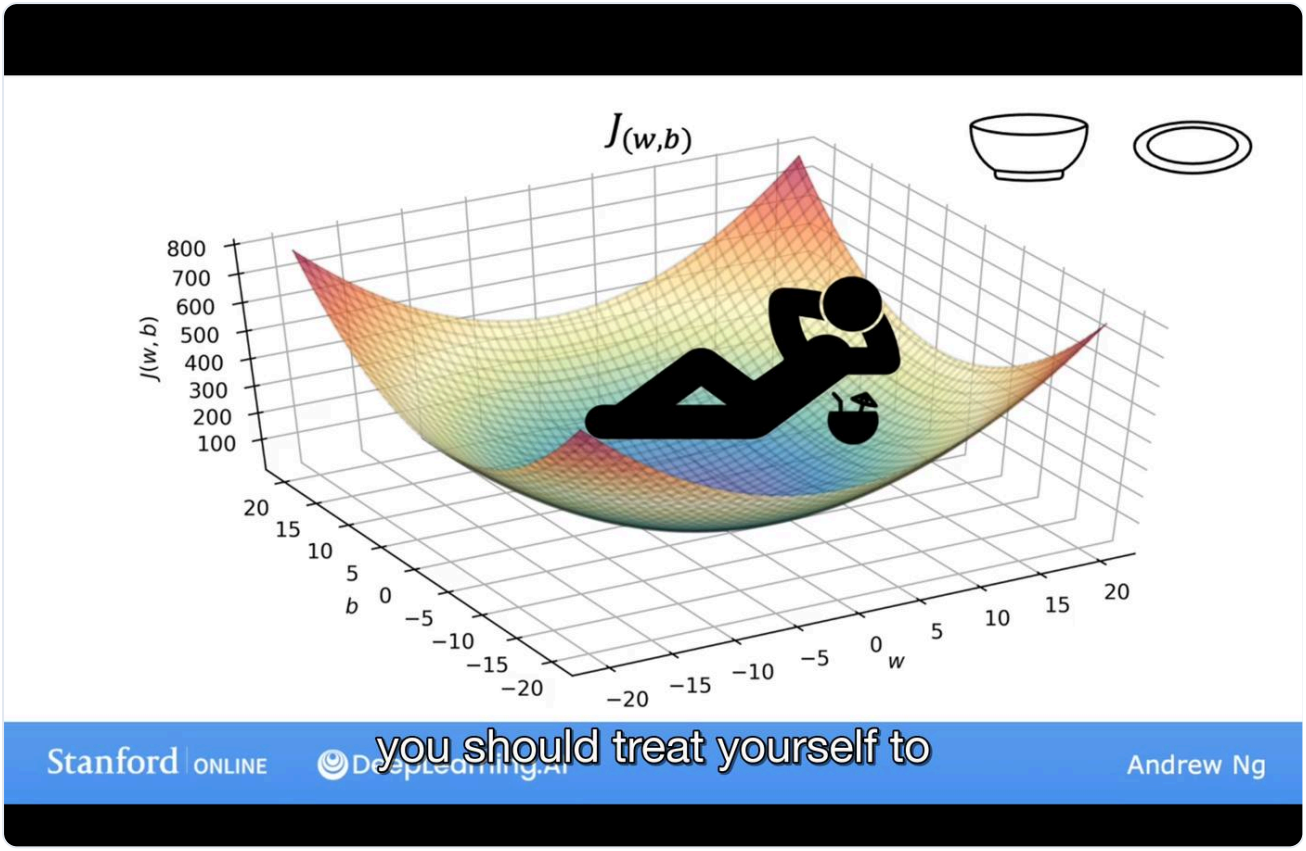


FIGURE 13.9

Maybe when you're done with this course, you should treat yourself to vacation and relax in a hammock like this. What you see here is a 3D-surface plot where the axes are labeled w and b. As you vary w and b, which are the two parameters of the model, you get different values for the cost function J of w, and b. This is a lot like the U-shaped curve you saw in the last video, except instead of having one parameter w as input for the j, you now have two parameters, w and b as inputs into this soup bowl or this hammock-shaped function J. I just want to point out that any single point on this surface represents some particular choice of w and b. For example, if w was minus 10 and b was minus 15, then the height of the surface above this point is the value of j when w is minus 10 and b is minus 15.

Now, in order to look even more closely at specific points, there's another way of plotting the cost function J that would be useful for visualization, which is, rather than using these 3D-surface plots, I like to take this exact same function J.

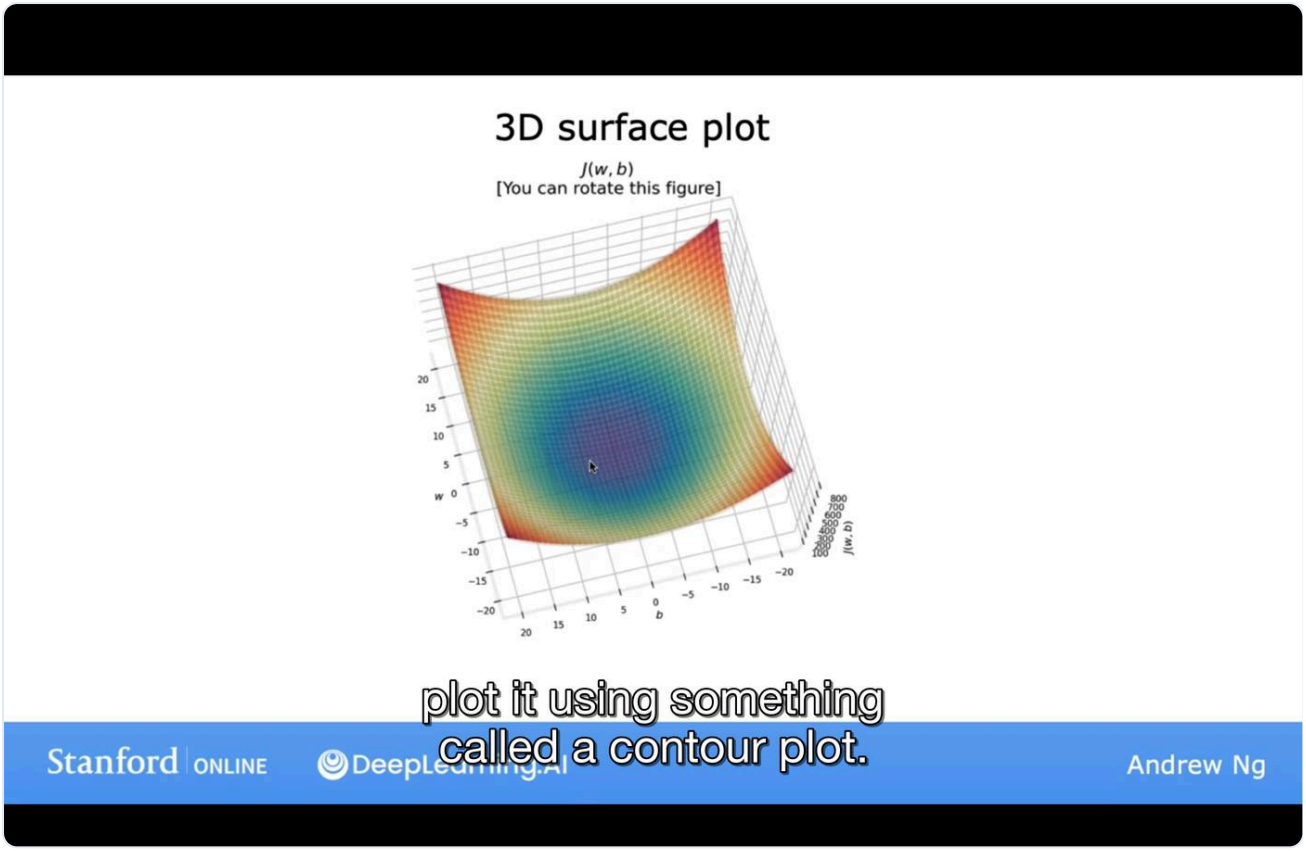


FIGURE 13.10 | 3D Surface Plot

I'm not changing the function J at all and plot it using something called a contour plot.

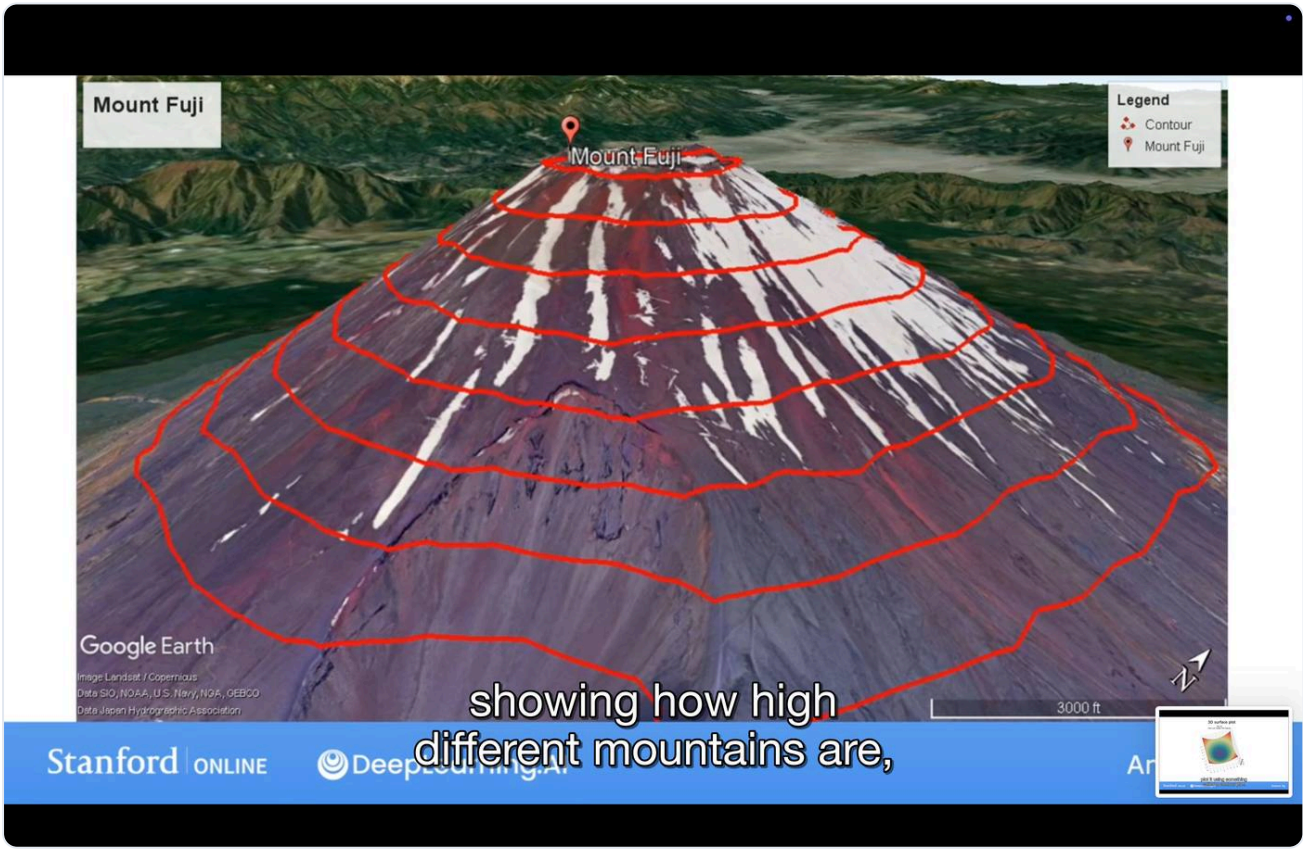


FIGURE 13.11

If you've ever seen a topographical map showing how high different mountains are, the contours in a topographical map are basically horizontal slices of the landscape of say, a mountain. This image is of Mount Fuji in Japan. I still remember my family visiting Mount Fuji when I was a teenager. It's beautiful sights.

FIGURE 13.12

If you fly directly above the mountain, that's what this contour map looks like. It shows all the points, they're at the same height for different heights. At the bottom of this slide is a 3D-surface plot of the cost function J. I know it doesn't look very bowl-shaped, but it is actually a bowl just very stretched out, which is why it looks like that. In an optional lab, that is shortly to follow, you will be able to see this in 3D and spin around the surface yourself and it'll look more obviously bowl-shaped there.

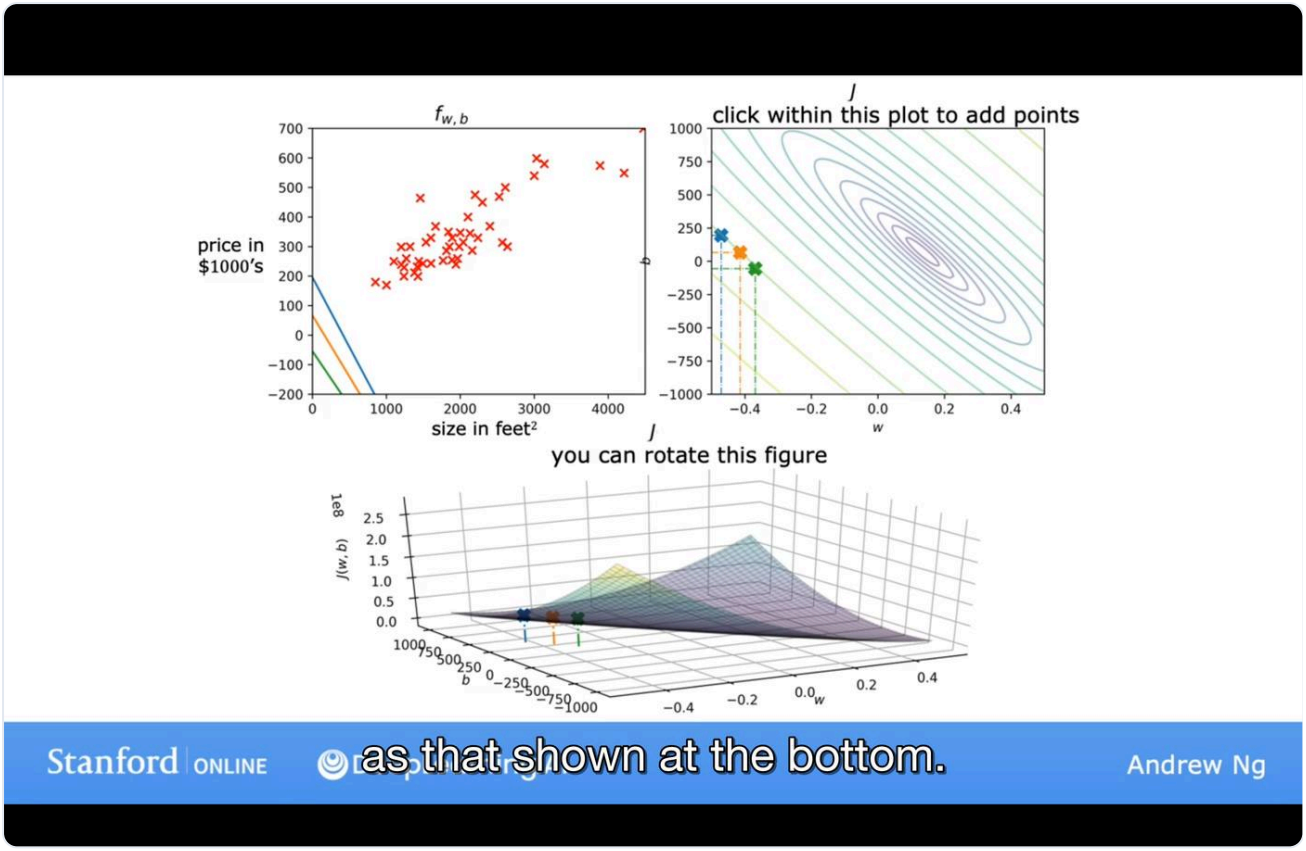


FIGURE 13.13

Next, here on the upper right is a contour plot of this exact same cost function as that shown at the bottom. The two axes on this contour plots are b, on the vertical axis, and w on the horizontal axis. What each of these ovals, also called ellipses, shows, is the center points on the 3D surface which are at the exact same height. In other words, the set of points which have the same value for the cost function J.

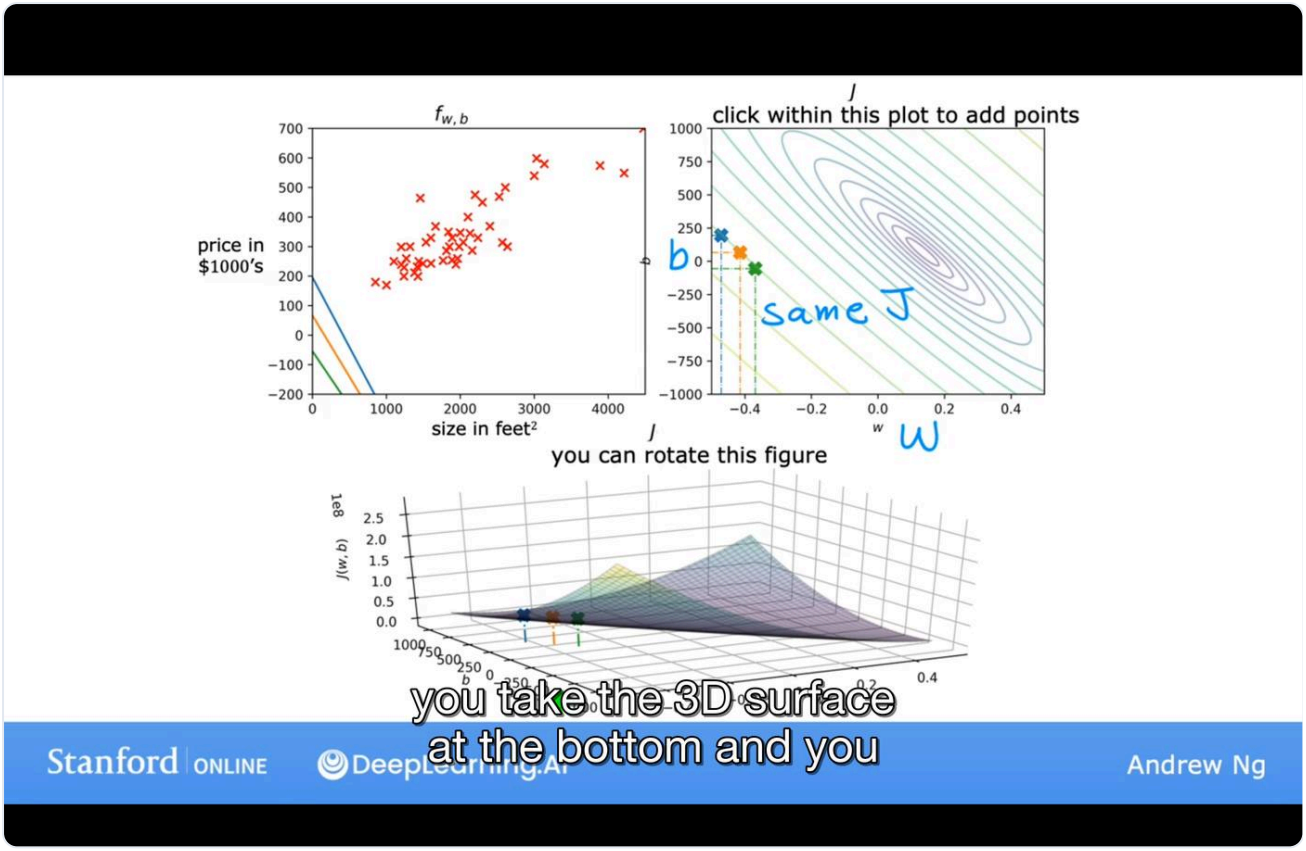


FIGURE 13.14

To get the contour plots, you take the 3D surface at the bottom and you use a knife to slice it horizontally. You take horizontal slices of that 3D surface and get all the points, they're at the same height. Therefore, each horizontal slice ends up being shown as one of these ellipses or one of these ovals.

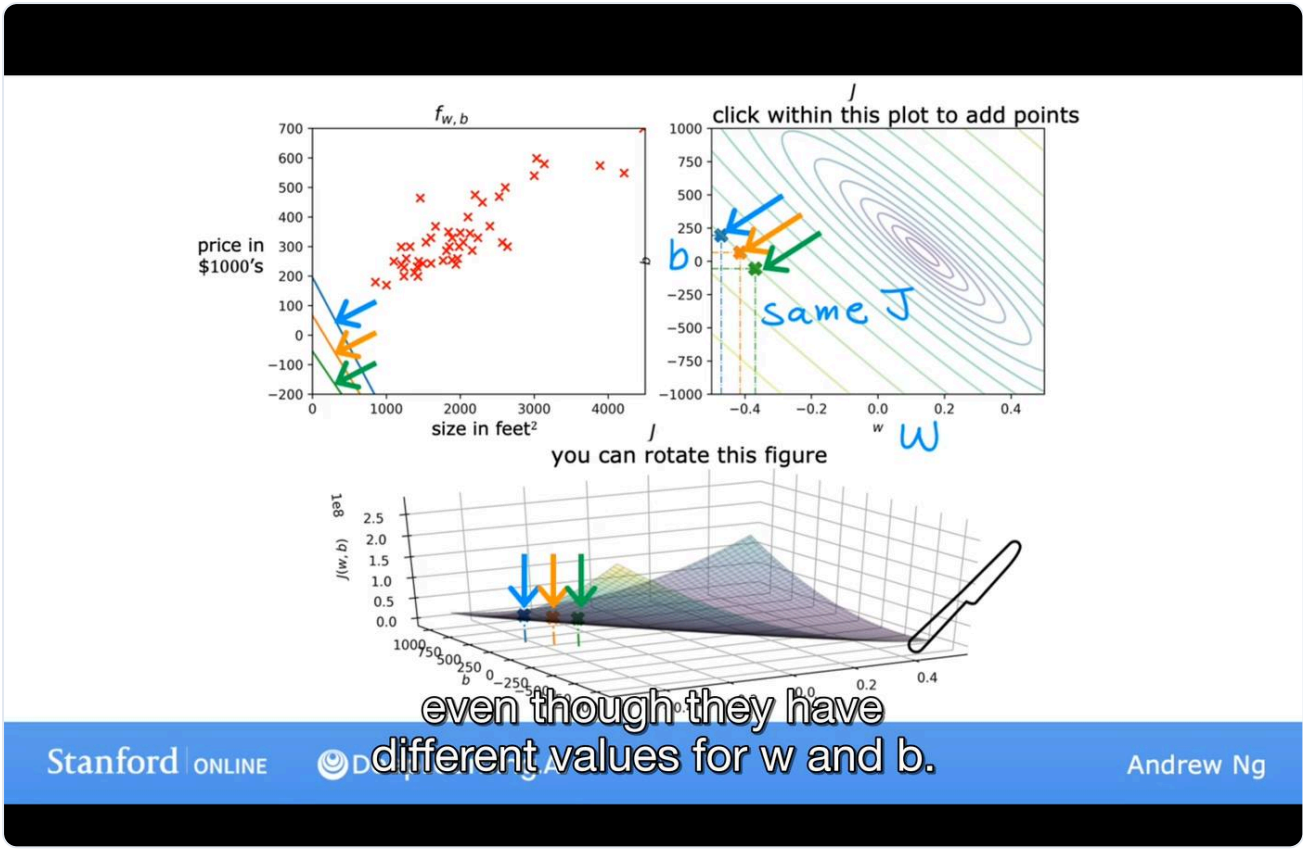


FIGURE 13.15

Concretely, if you take that point, and that point, and that point, all of these three points have the same value for the cost function J, even though they have different values for w and b.

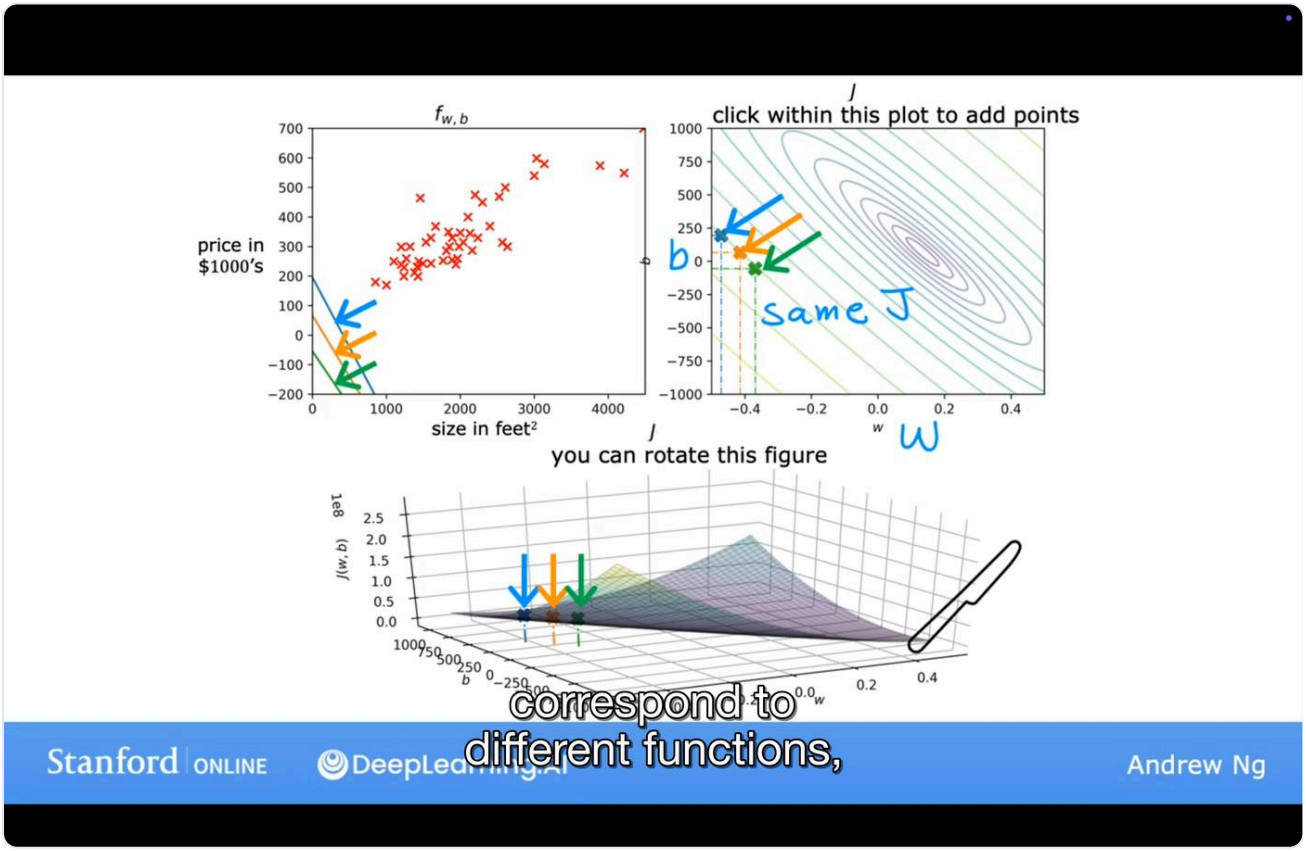


FIGURE 13.16

In the figure on the upper left, you see also that these three points correspond to different functions, f, all three of which are actually pretty bad for predicting housing prices in this case. Now, the bottom of the bowl, where the cost function J is at a minimum, is this point right here, at the center of this concentric ovals.

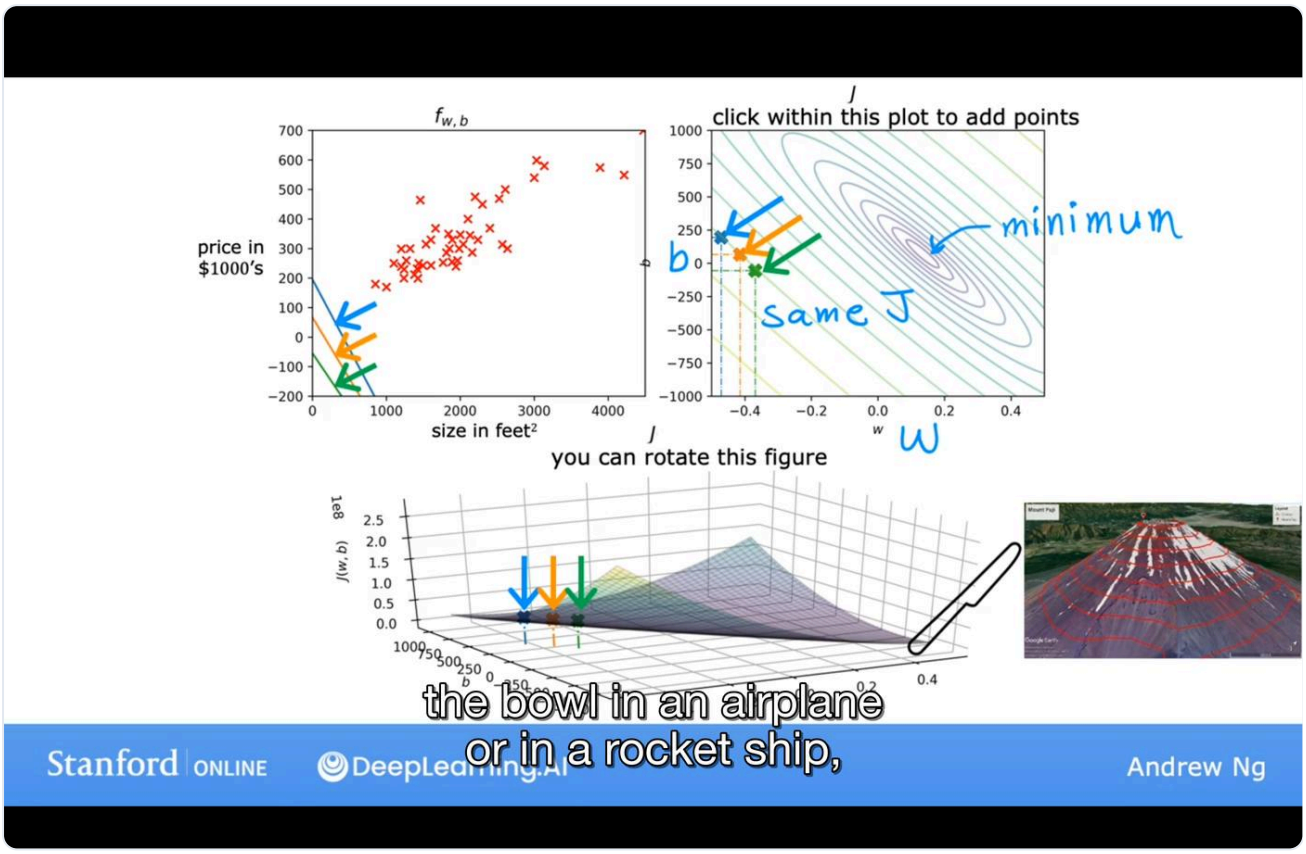


FIGURE 13.17

If you haven't seen contour plots much before, I'd like you to imagine, if you will, that you are flying high up above the bowl in an airplane or in a rocket ship, and you're looking straight down at it.

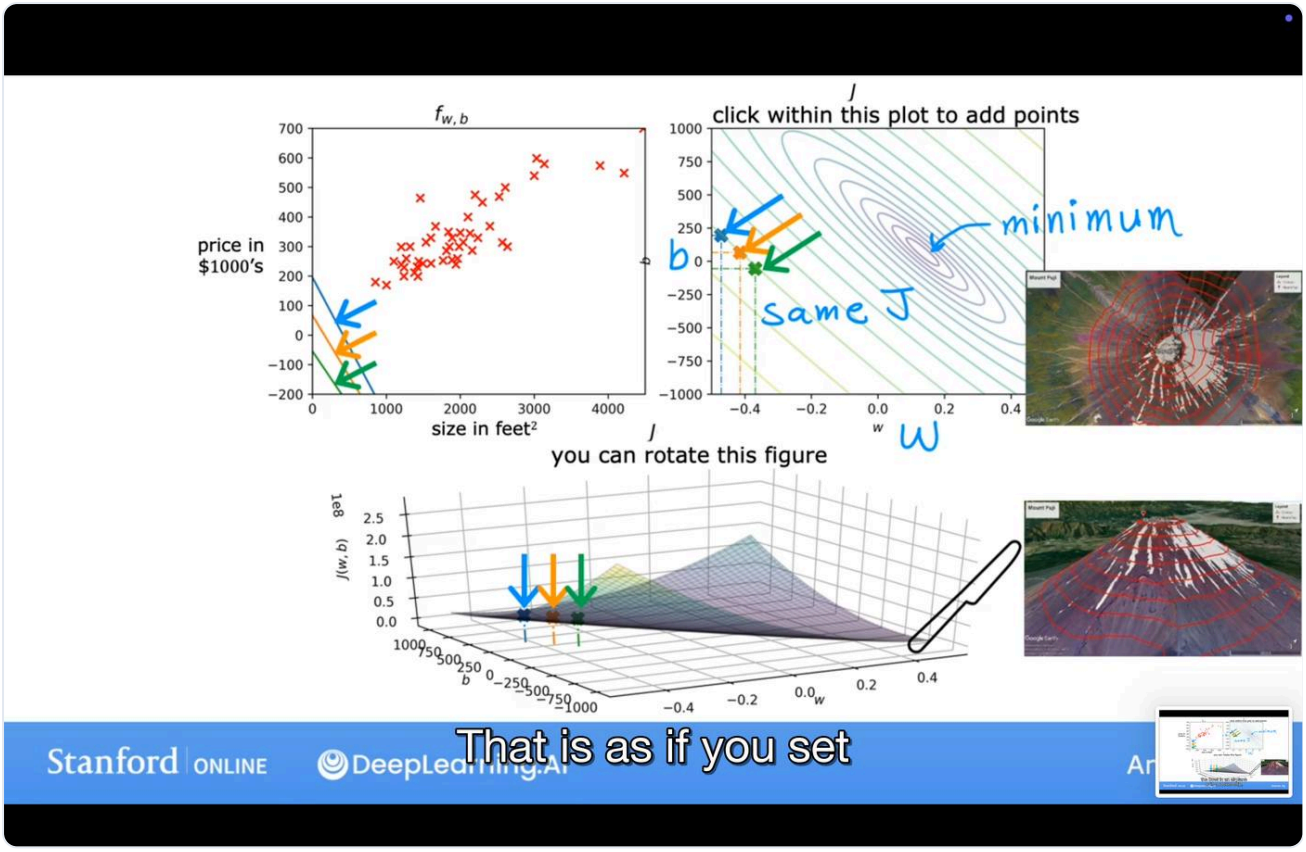


FIGURE 13.18

That is as if you set your computer monitor flat on your desk facing up and the bowl shape is coming directly out of your screen, rising above you desk.

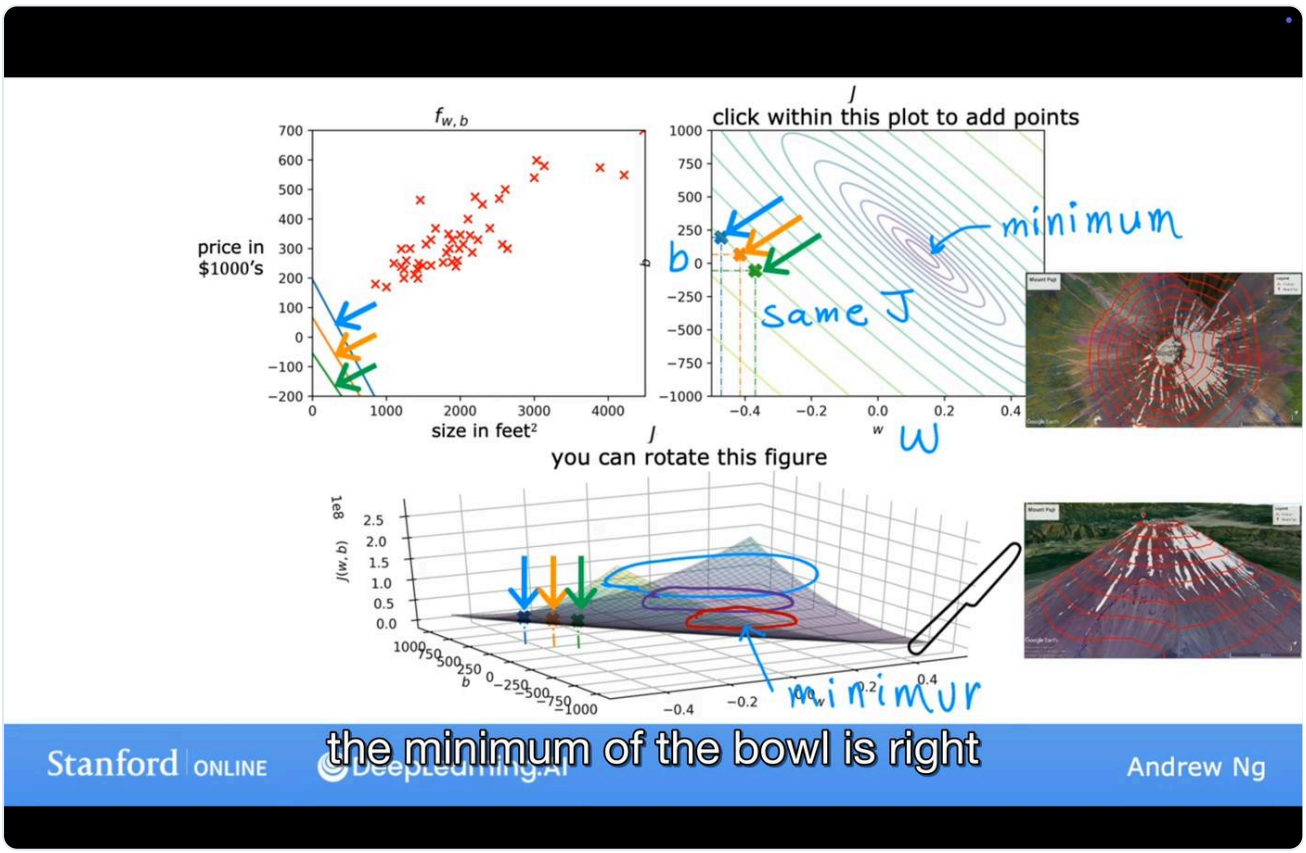


FIGURE 13.19

Imagine that the bowl shape grows out of your computer screen lying flat like that, so that each of these ovals have the same height above your screen and the minimum of the bowl is right down there in the center of the smallest oval. It turns out that the contour plots are a convenient way to visualize the 3D cost function J, but in a way, there's plotted in just 2D. In this video, you saw how the 3D bowl-shaped surface plot can also be visualized as a contour plot. Using this visualization too, in the next video, let's visualize some specific choices of w and b in the linear regression model so that you can see how these different choices affect the straight line you're fitting to the data.

Let's go on to the next video.

LECTURE 14

Sum of Squared and Pair of Values

Let's look at some more visualizations of w and b.

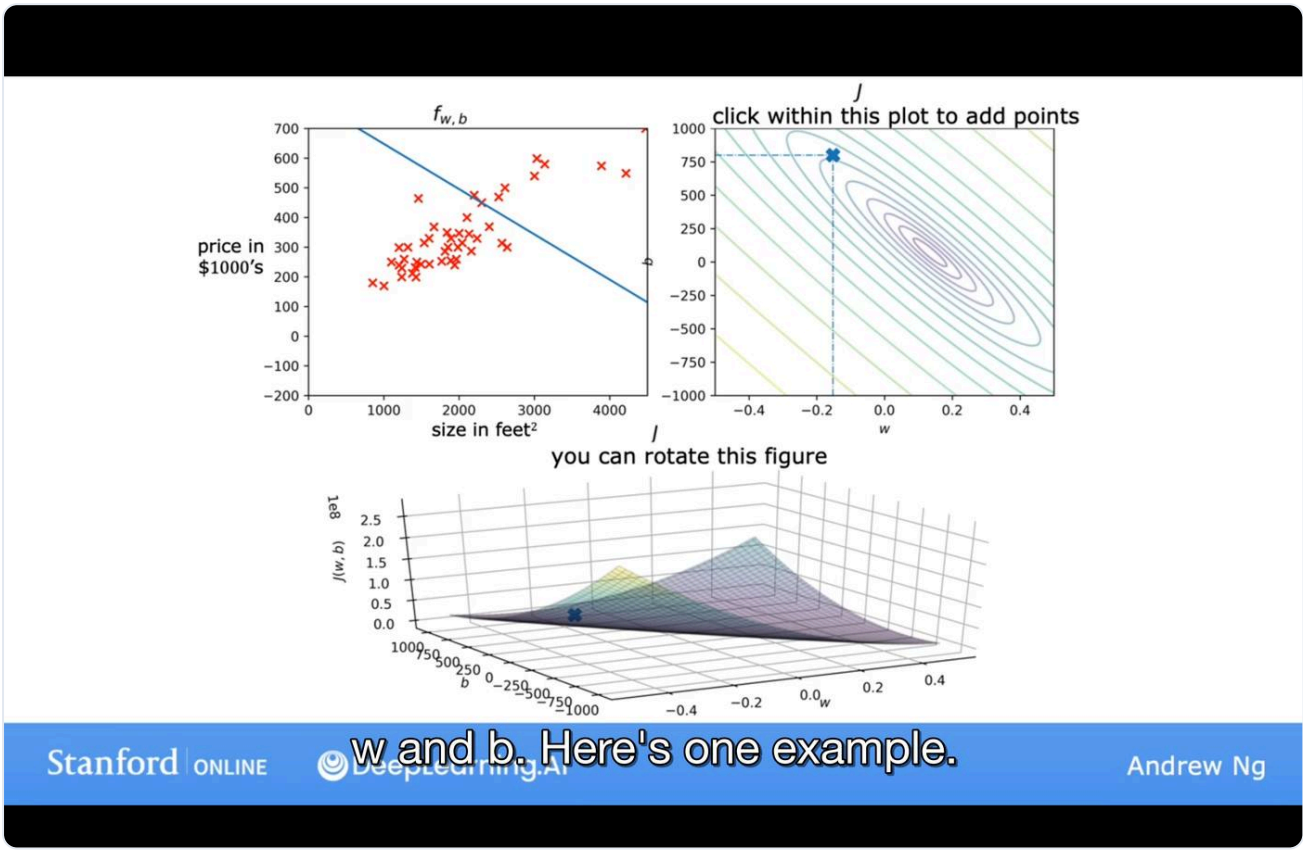


FIGURE 14.1

Here's one example. Over here, you have a particular point on the graph j. For this point, w equals about negative 0.15 and b equals about 800. This point corresponds to one pair of values for w and b that use a particular cost j.

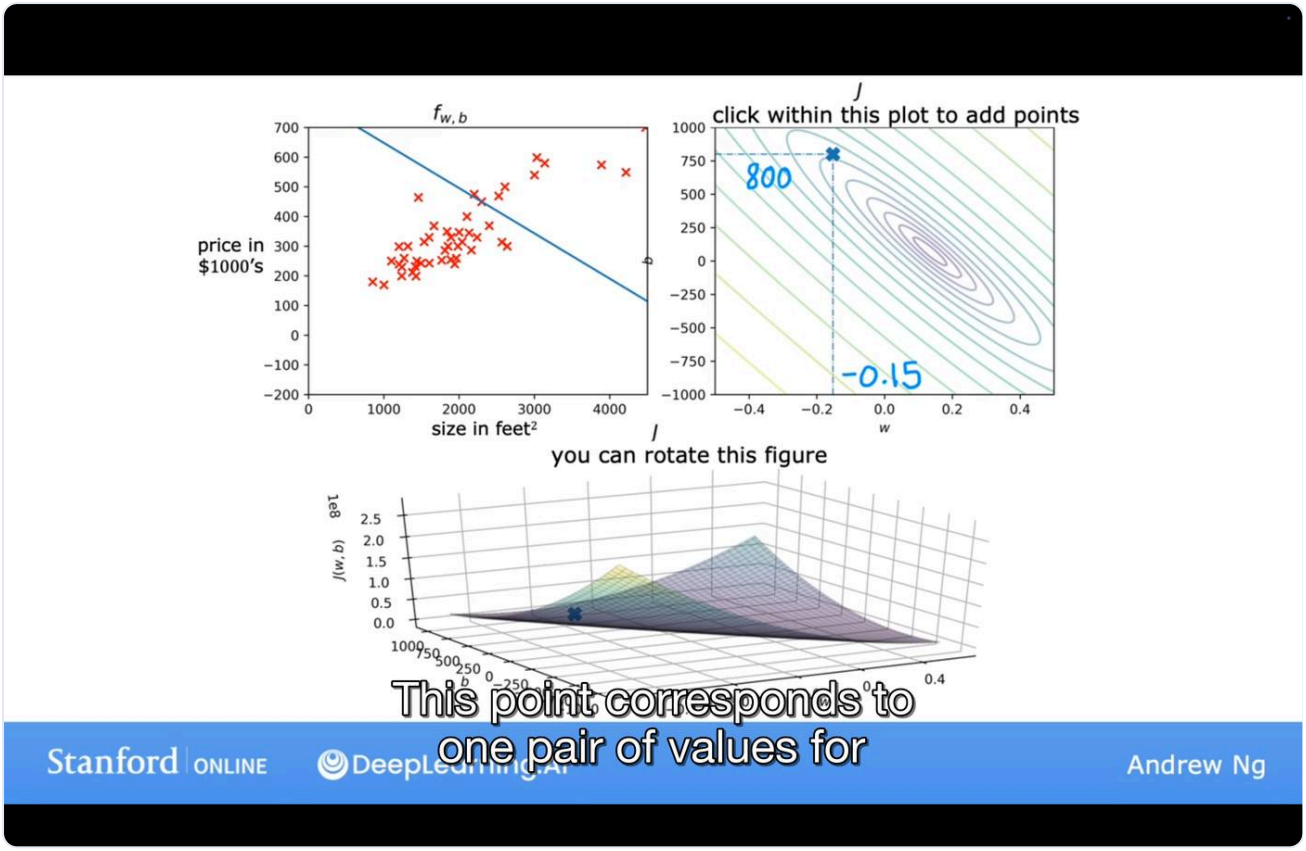


FIGURE 14.2

In fact, this booklet pair of values for w and b corresponds to this function f of x, which is this line you can see on the left. This line intersects the vertical axis at 800 because b equals 800 and the slope of the line is negative 0.15, because w equals negative 0.15. Now, if you look at the data points in the training set, you may notice that this line is not a good fit to the data.

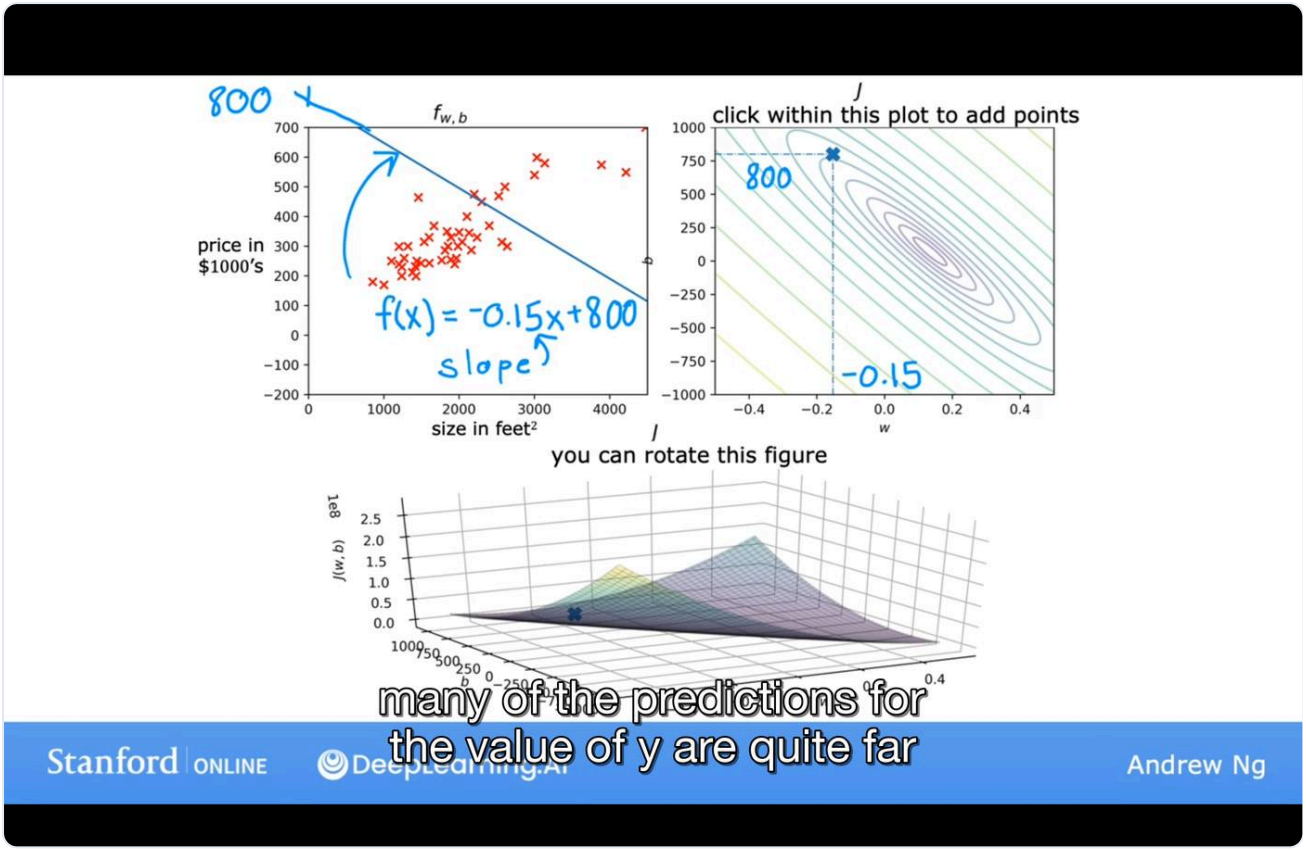


FIGURE 14.3

For this function f of x, with these values of w and b, many of the predictions for the value of y are quite far from the actual target value of y that is in the training data.

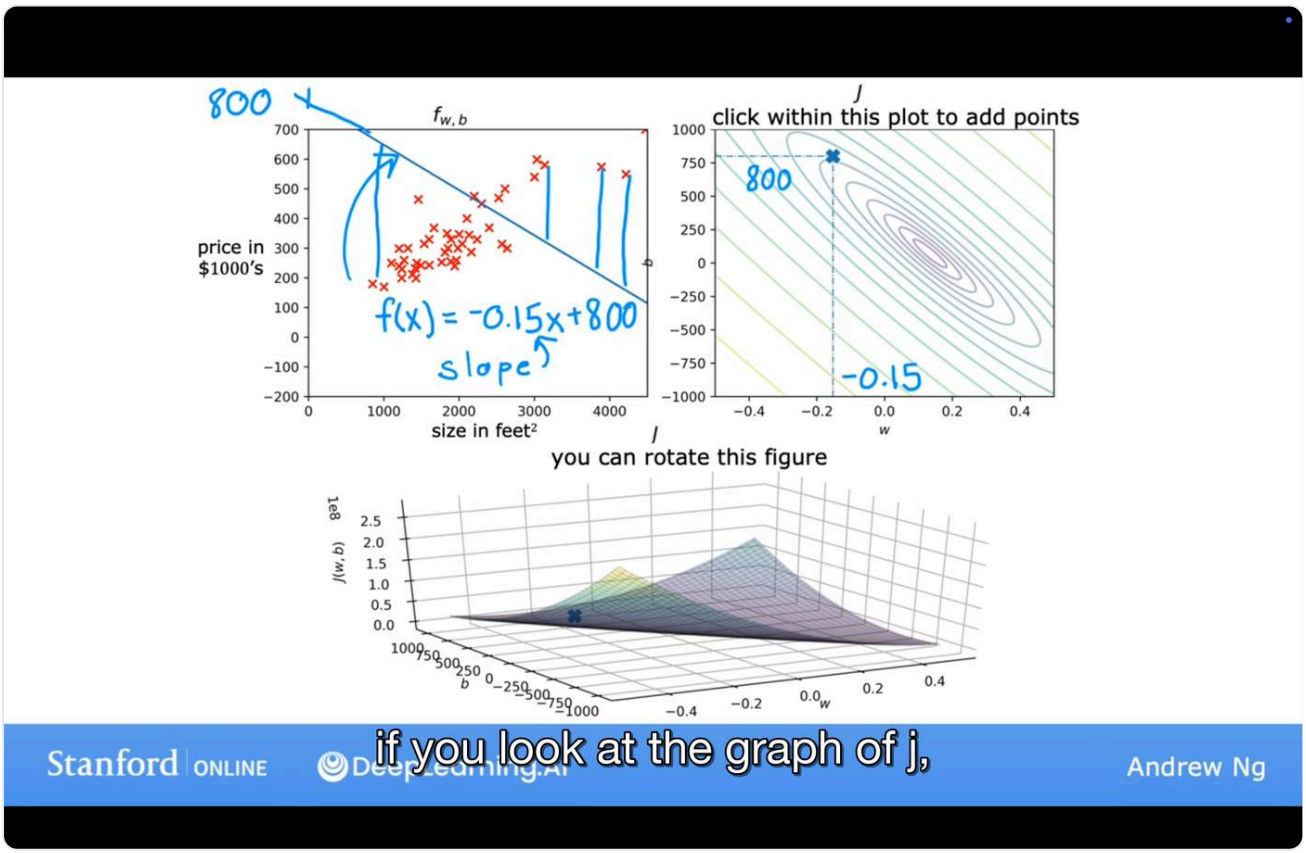


FIGURE 14.4

Because this line is not a good fit, if you look at the graph of j, the cost of this line is out here, which is pretty far from the minimum. There's a pretty high cost because this choice of w and b is just not that good a fit to the training set.

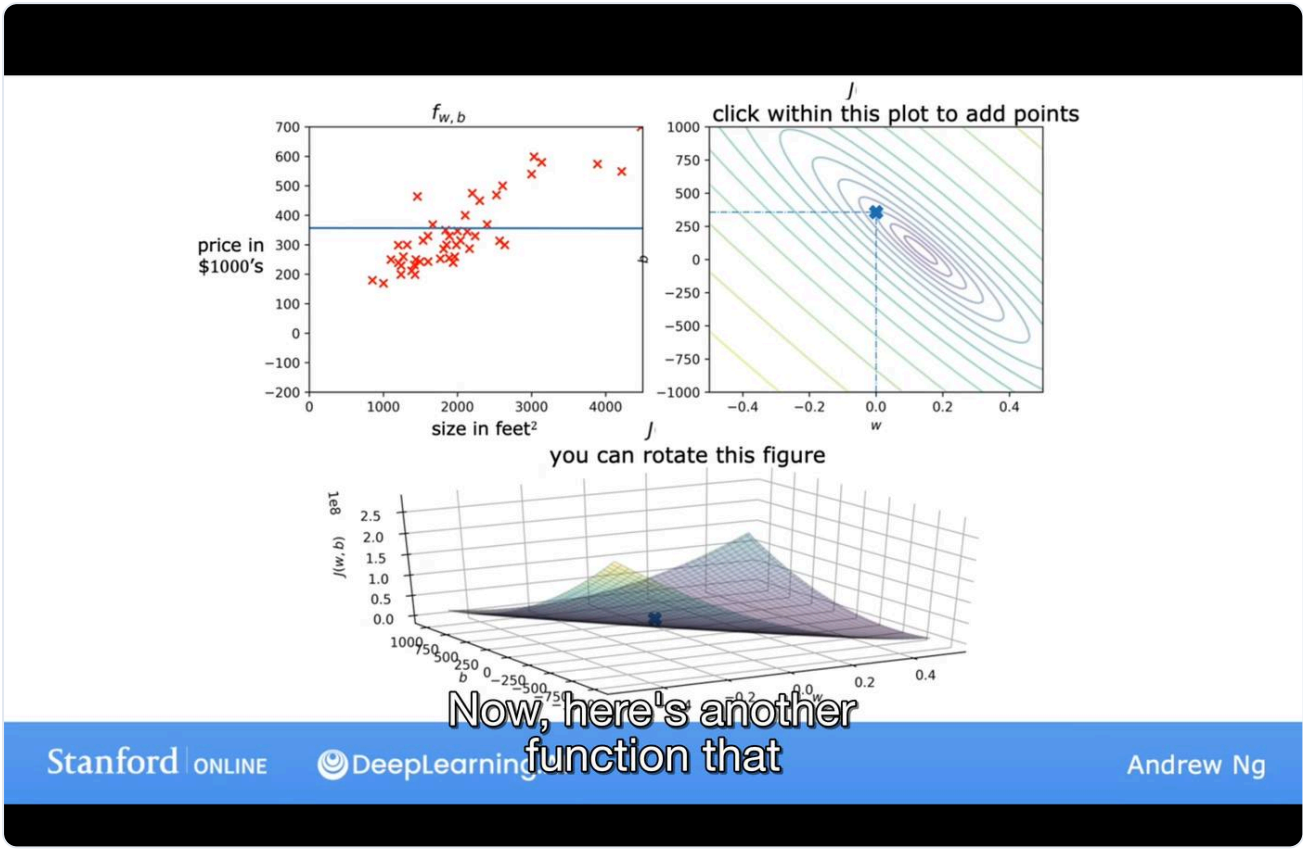


FIGURE 14.5

Now, let's look at another example with a different choice of w and b. Now, here's another function that is still not a great fit for the data, but maybe slightly less bad. This points here represents the cost for this booklet pair of w and b that creates that line.

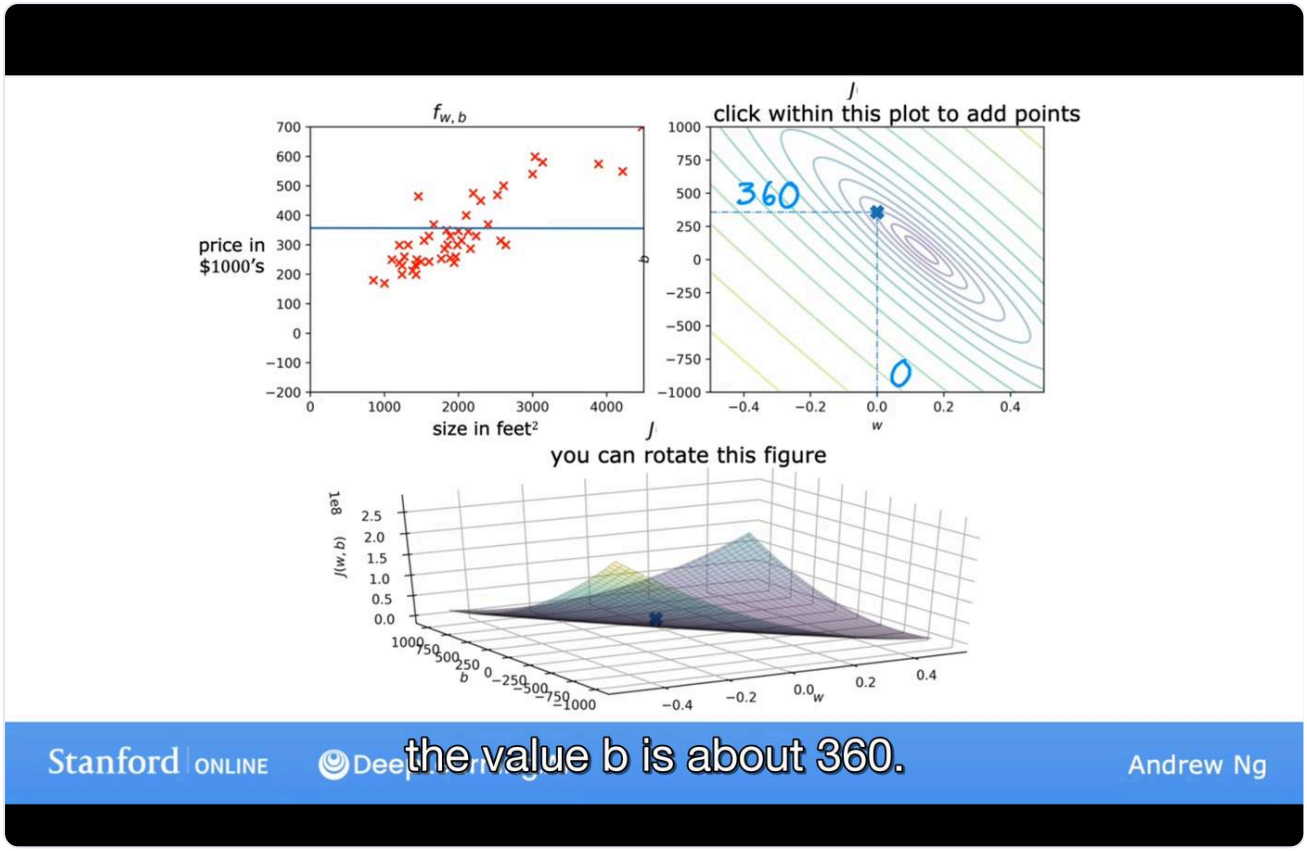


FIGURE 14.6

The value of w is equal to 0 and the value b is about 360.

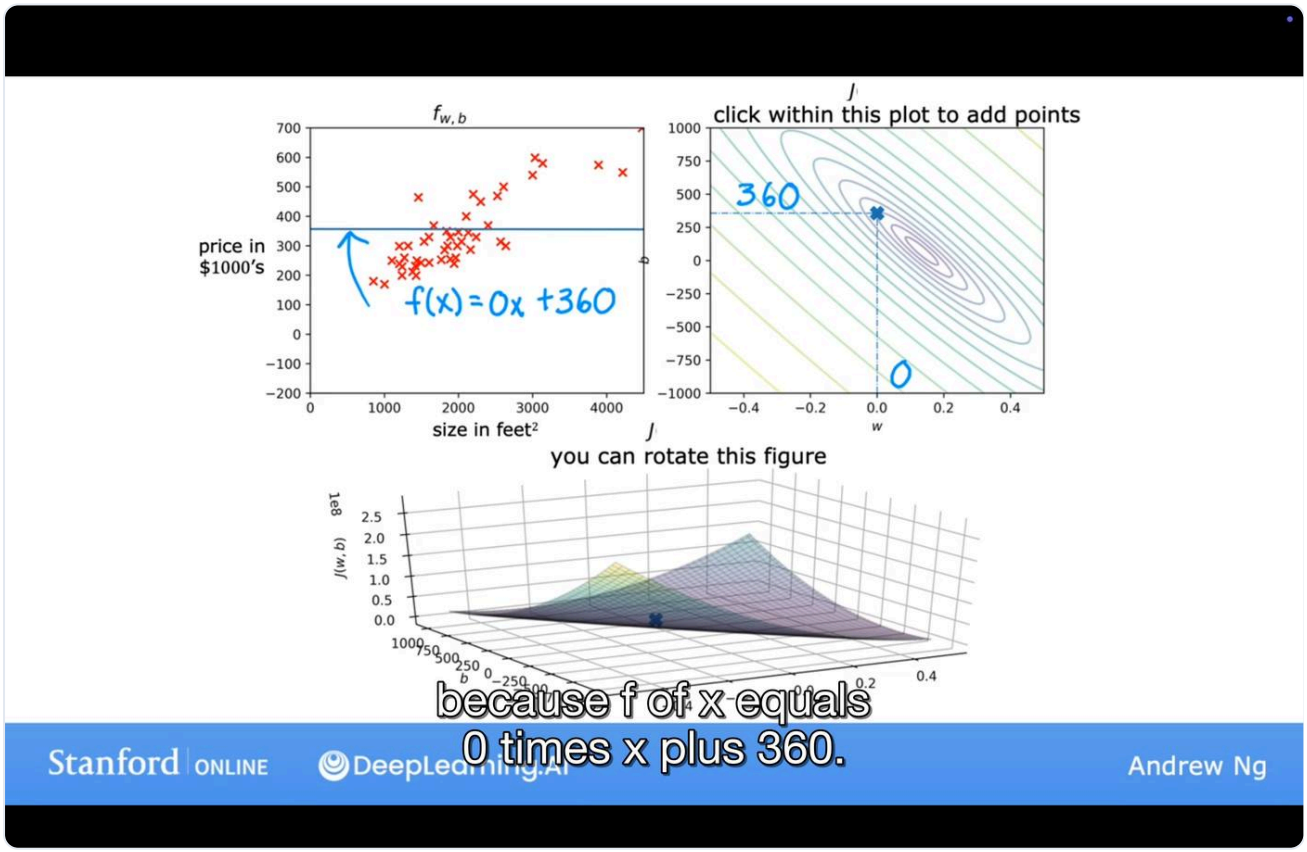


FIGURE 14.7

This pair of parameters corresponds to this function, which is a flat line, because f of x equals 0 times x plus 360. I hope that makes sense. Let's look at yet another example. Here's one more choice for w and b, and with these values, you end up with this line f of x.

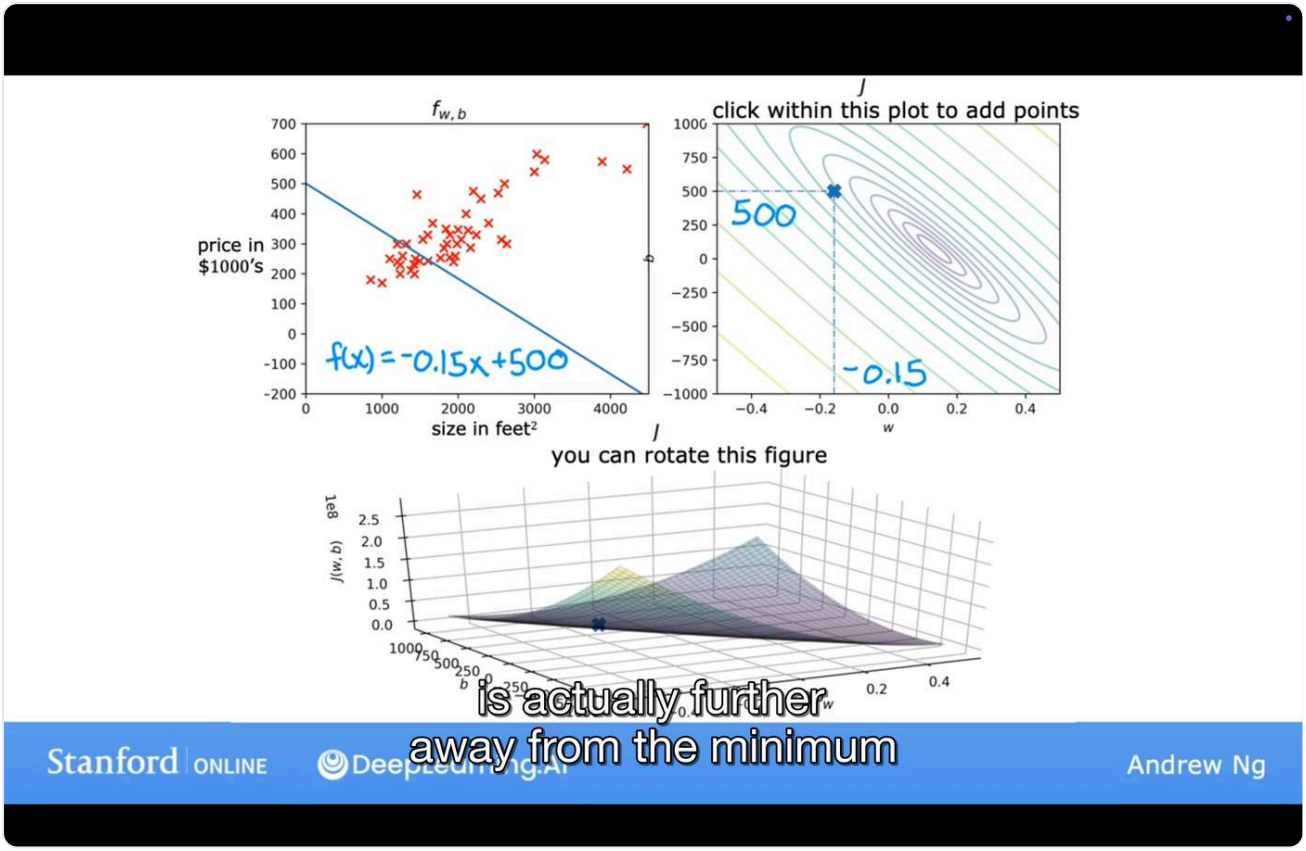


FIGURE 14.8

Again, not a great fit to the data, is actually further away from the minimum compared to the previous example. Remember that the minimum is at the center of that smallest ellipse.

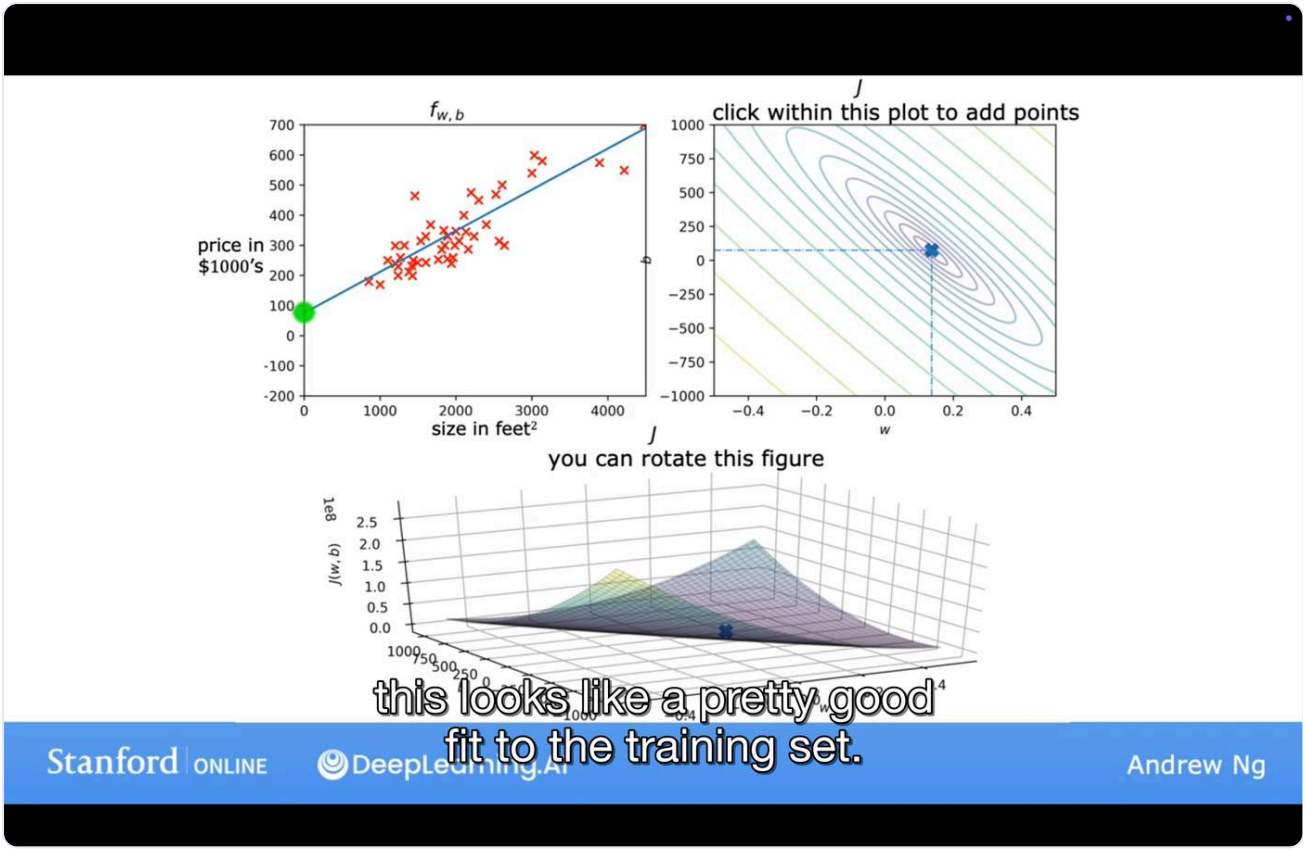


FIGURE 14.9

Last example, if you look at f of x on the left, this looks like a pretty good fit to the training set.

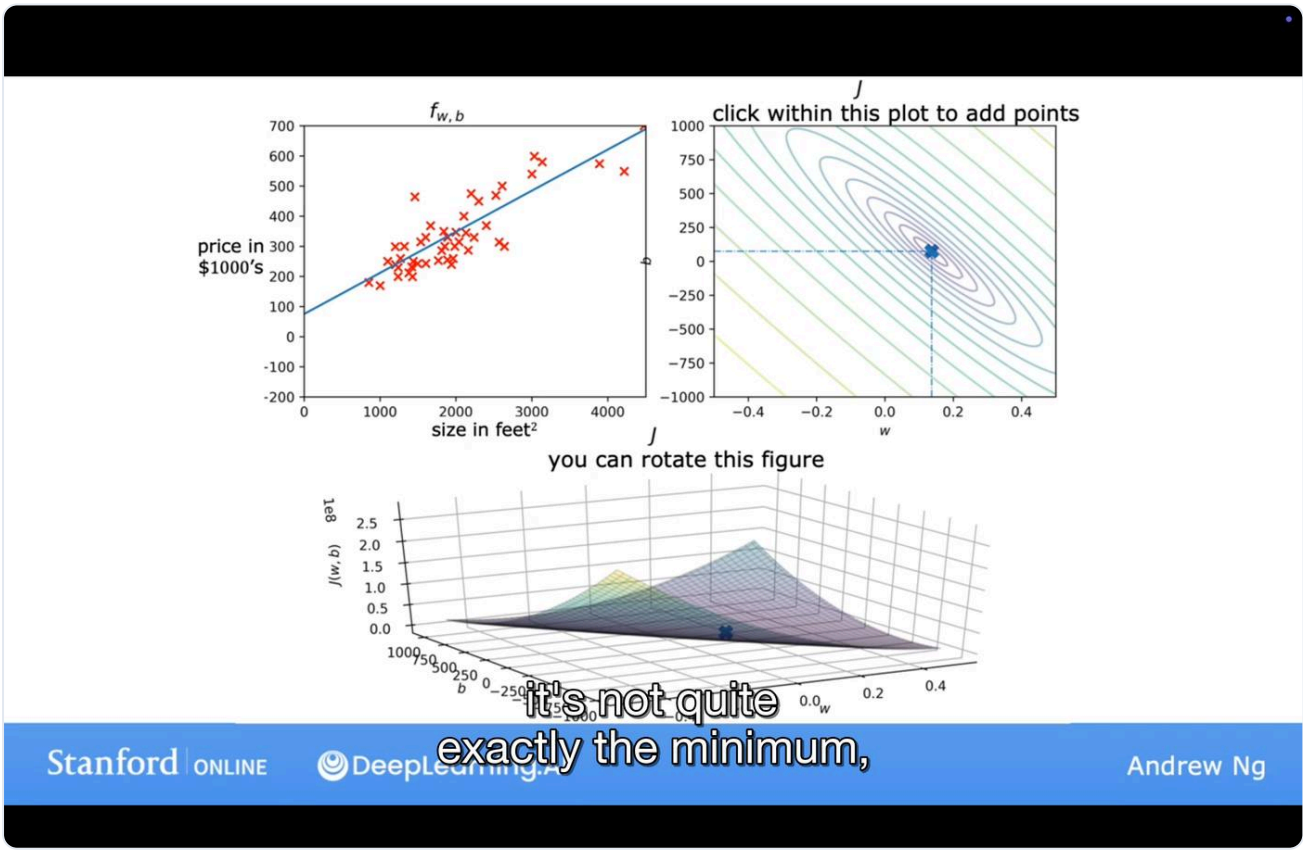


FIGURE 14.10

You can see on the right, this point representing the cost is very close to the center of the smaller ellipse, it's not quite exactly the minimum, but it's pretty close.

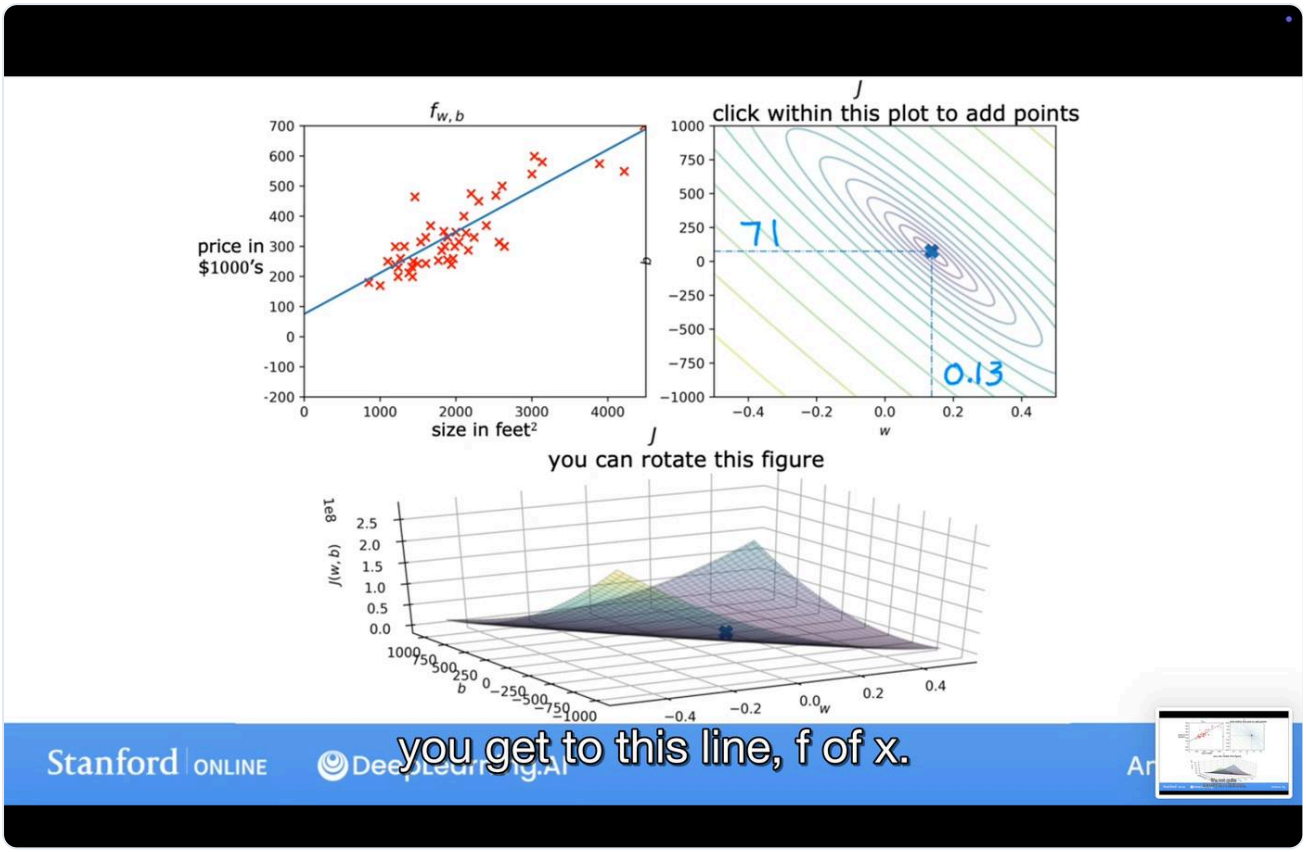


FIGURE 14.11

For this value of w and b, you get to this line, f of x. You can see that if you measure the vertical distances between the data points and the predicted values on the straight line, you'd get the error for each data point. The sum of squared errors for all of these data points is pretty close to the minimum possible sum of squared errors among all possible straight line fits. I hope that by looking at these figures, you can get a better sense of how different choices of the parameters affect the line f of x and how this corresponds to different values for the cost j, and hopefully you can see how the better fit lines correspond to points on the graph of j that are closer to the minimum possible cost for this cost function j of w and b.

In the optional lab that follows this video, you'll get to run some codes and remember all the code is given, so you just need to hit Shift Enter to run it and take a look at it and the lab will show you how the cost function is implemented in code. Given a small training set and different choices for the parameters, you'll be able to see how the cost varies depending on how well the model fits the data. In the optional lab, you also can play with in interactive console plot. Check this out. You can use your mouse cursor to click anywhere on the contour plot and you will see the straight line defined by the values you chose for the parameters w and b.

You'll see a dot up here also on the 3D surface plot showing the cost. Finally, the optional lab also has a 3D surface plot that you can manually rotate and spin around using your mouse cursor to take a better look at what the cost function looks like. I hope you'll enjoy playing with the optional lab. Now in linear regression, rather than having to manually try to read a contour plot for

the best value for w and b, which isn't really a good procedure and also won't work once we get to more complex machine learning models. What you really want is an efficient algorithm that you can write in code for automatically finding the values of parameters w and b they give you the best fit line.

That minimizes the cost function j. There is an algorithm for doing this called gradient descent. This algorithm is one of the most important algorithms in machine learning. Gradient descent and variations on gradient descent are used to train, not just linear regression, but some of the biggest and most complex models in all of AI. Let's go to the next video to dive into this really important algorithm called gradient descent.

LECTURE 15

Gradient Descent

Welcome back. In the last video, we saw visualizations of the cost function j and how you can try different choices of the parameters w and b and see what cost value they get you. It would be nice if we had a more systematic way to find the values of w and b, that results in the smallest possible cost, j of w, b. It turns out there's an algorithm called gradient descent that you can use to do that. Gradient descent is used all over the place in machine learning, not just for linear regression, but for training for example some of the most advanced neural network models, also called deep learning models. Deep learning models are something you learned about in the second course.

Learning these two of gradient descent will set you up with one of the most important building blocks in machine learning. Here's an overview of what we'll do with gradient descent. You have the cost function j of w, b right here that you want to minimize. In the example we've seen so far, this is a cost function for linear regression, but it turns out that gradient descent is an algorithm that you can use to try to minimize any function, not just a cost function for linear regression. Just to make this discussion on gradient descent more general, it turns out that gradient descent applies to more general functions, including other cost functions that work with models that have more than two parameters.

For instance, if you have a cost function J as a function of w_1, w_2 up to w_n and b, your objective is to minimize j over the parameters w_1 to w_n and b.

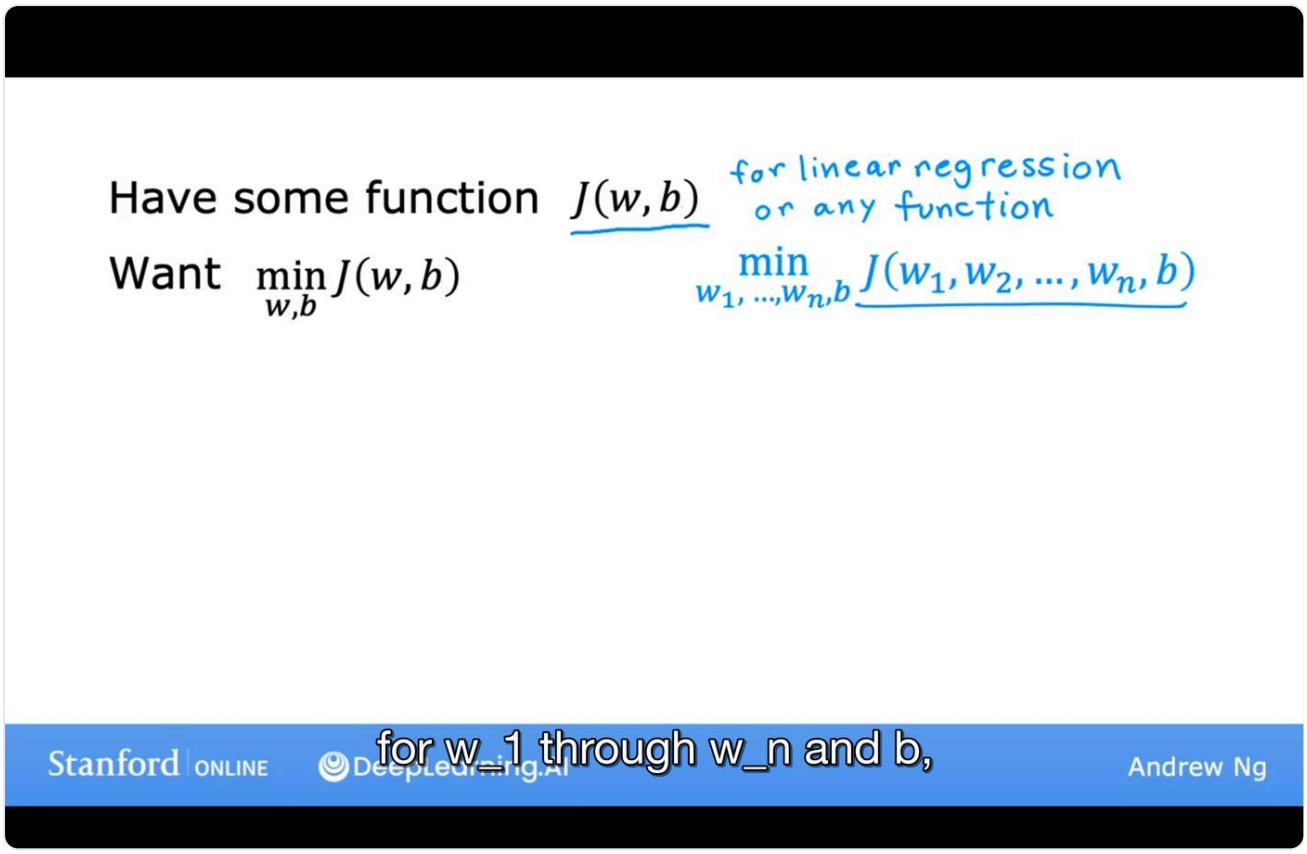


FIGURE 15.1 | Have Some Function J(w,b) Oe Any Function

In other words, you want to pick values for w_1 through w_n and b, that gives you the smallest possible value of j. It turns out that gradient descent is an algorithm that you can apply to try to minimize this cost function j as well. What you're going to do is just to start off with some initial guesses for w and b. In linear regression, it won't matter too much what the initial value are, so a common choice is to set them both to 0. For example, you can set w to 0 and b to 0 as the initial guess.

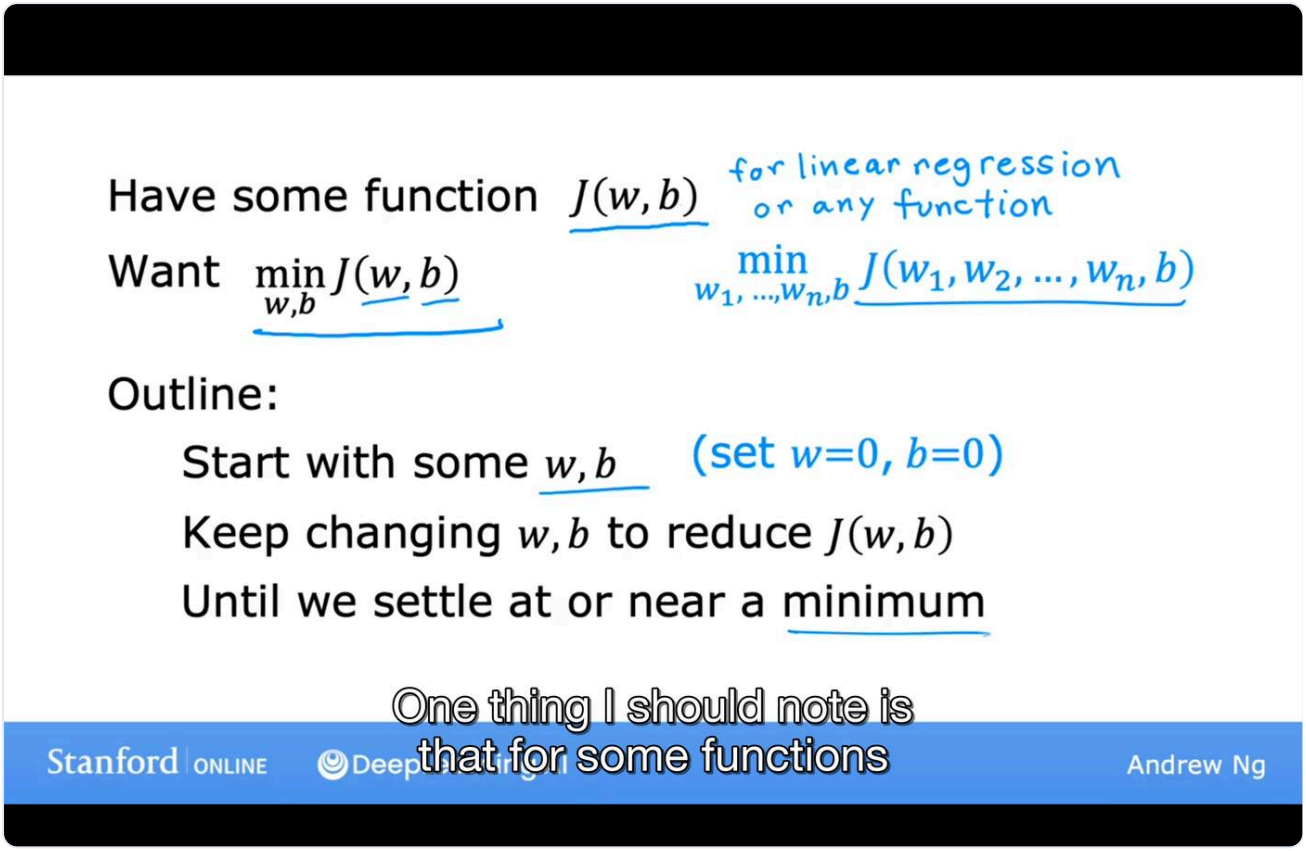


FIGURE 15.2 | Have Some Function J(w,b) o6 Any Function

With the gradient descent algorithm, what you're going to do is, you'll keep on changing the parameters w and b a bit every time to try to reduce the cost j of w, b until hopefully j settles at or near a minimum. One thing I should note is that for some functions j that may not be a bow shape or a hammock shape, it is possible for there to be more than one possible minimum. Let's take a look at an example of a more complex surface plot j to see what gradient is doing. This function is not a squared error cost function.

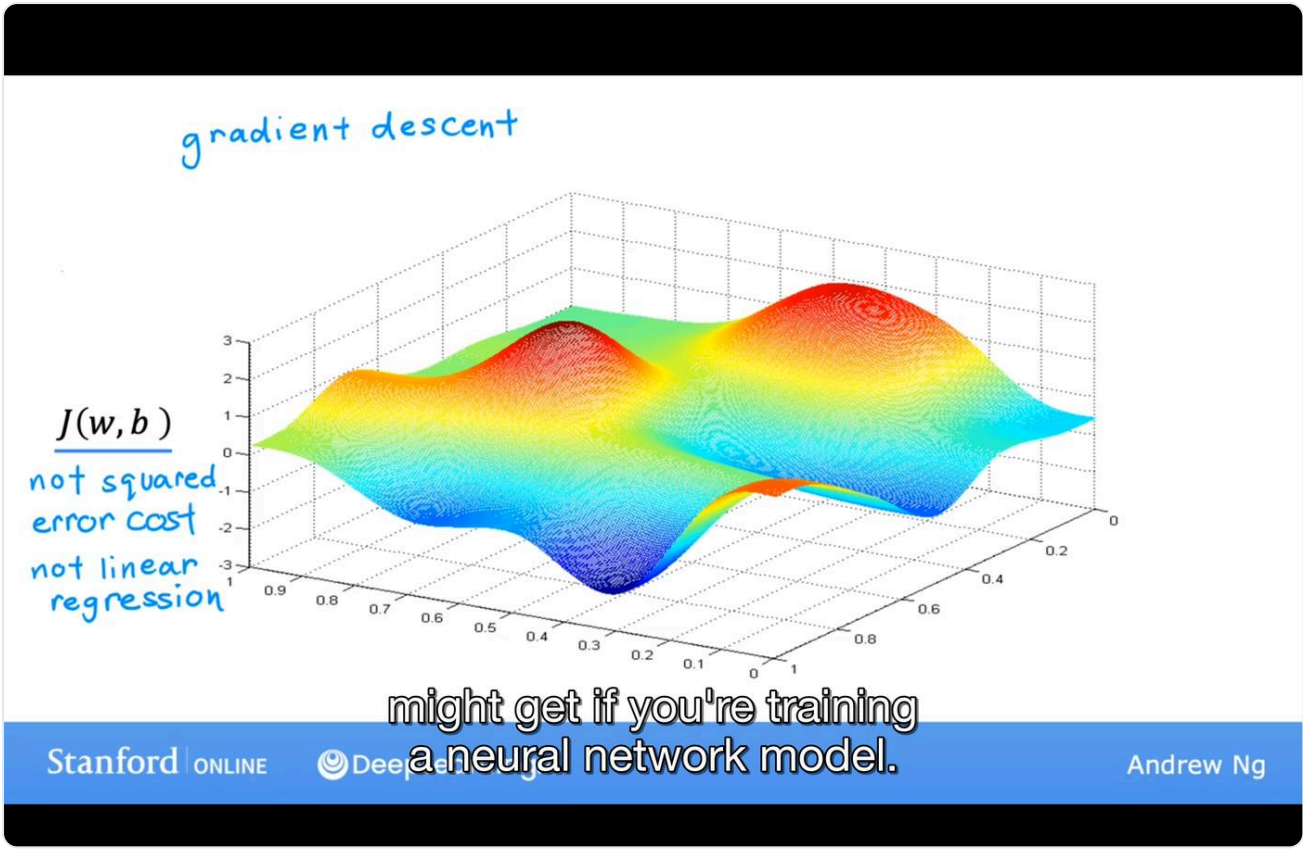


FIGURE 15.3

For linear regression with the squared error cost function, you always end up with a bow shape or a hammock shape. But this is a type of cost function you might get if you're training a neural network model. Notice the axes, that is w and b on the bottom axis. For different values of w and b, you get different points on this surface, j of w, b, where the height of the surface at some point is the value of the cost function.

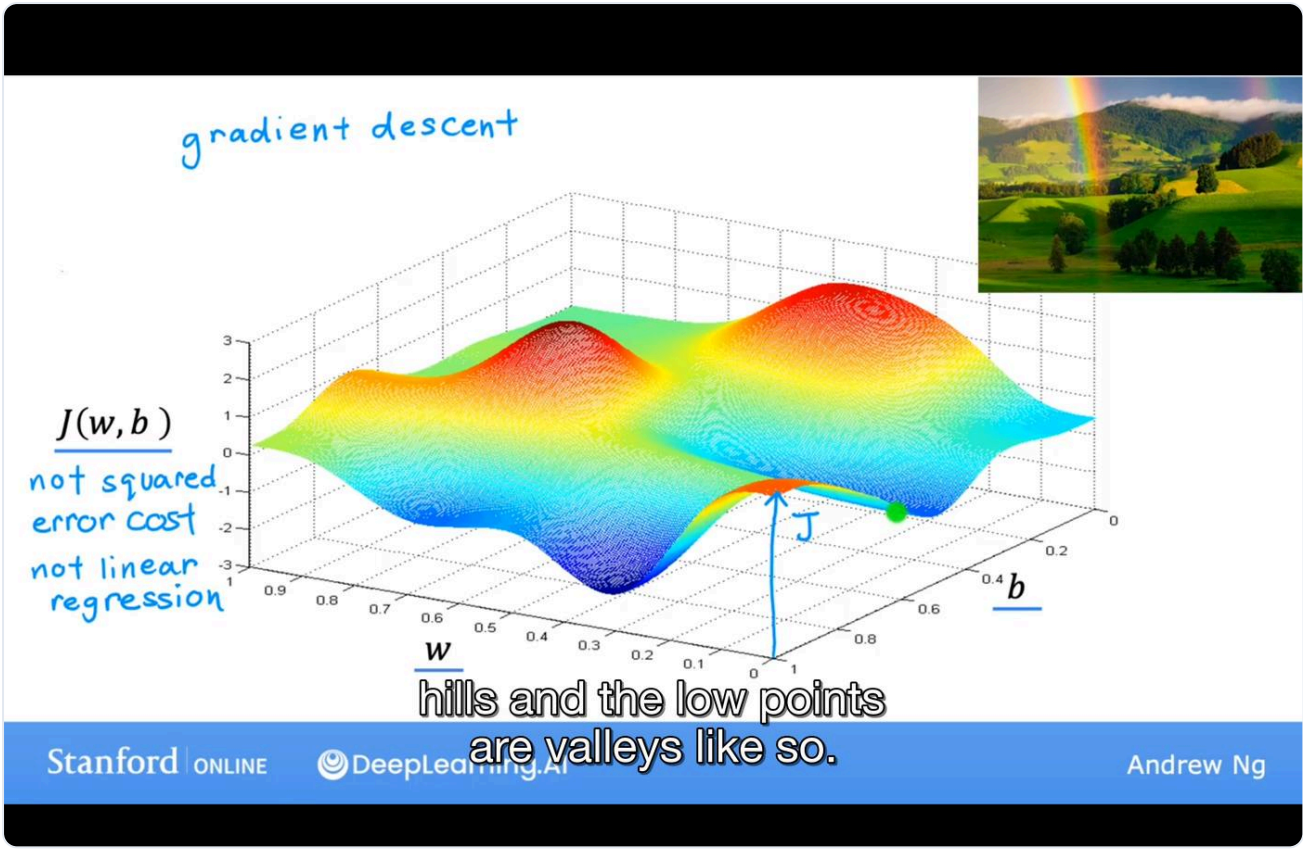


FIGURE 15.4

Now, let's imagine that this surface plot is actually a view of a slightly hilly outdoor park or a golf course where the high points are hills and the low points are valleys like so. I'd like you to imagine if you will, that you are physically standing at this point on the hill. If it helps you to relax, imagine that there's lots of really nice green grass and butterflies and flowers is a really nice hill.

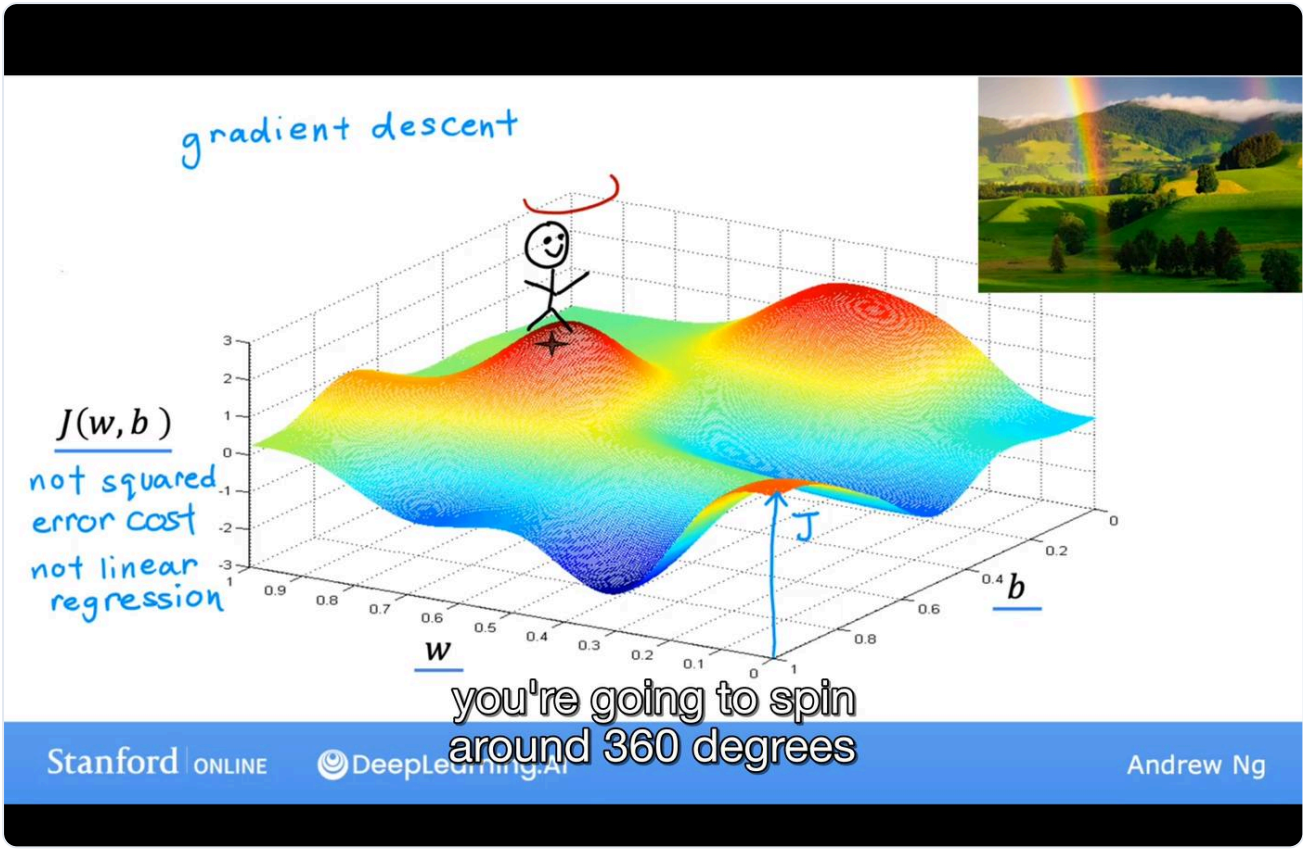


FIGURE 15.5

Your goal is to start up here and get to the bottom of one of these valleys as efficiently as possible. What the gradient descent algorithm does is, you're going to spin around 360 degrees and look around and ask yourself, if I were to take a tiny little baby step in one direction, and I want to go downhill as quickly as possible to or one of these valleys. What direction do I choose to take that baby step? Well, if you want to walk down this hill as efficiently as possible, it turns out that if you're standing at this point in the hill and you look around, you will notice that the best direction to take your next step downhill is roughly that direction.

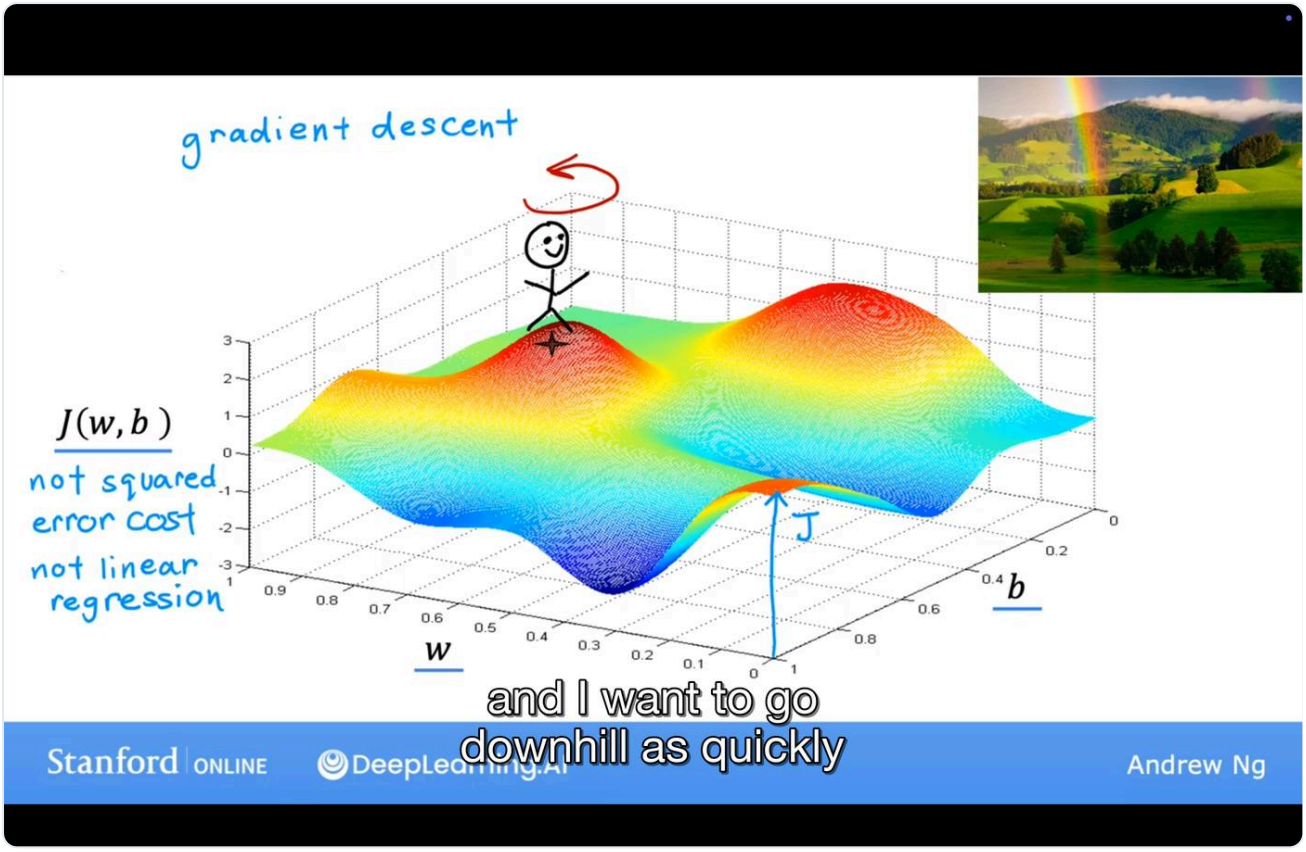


FIGURE 15.6

Mathematically, this is the direction of steepest descent. It means that when you take a tiny baby little step, this takes you downhill faster than a tiny little baby step you could have taken in any other direction. After taking this first step, you're now at this point on the hill over here. Now let's repeat the process.

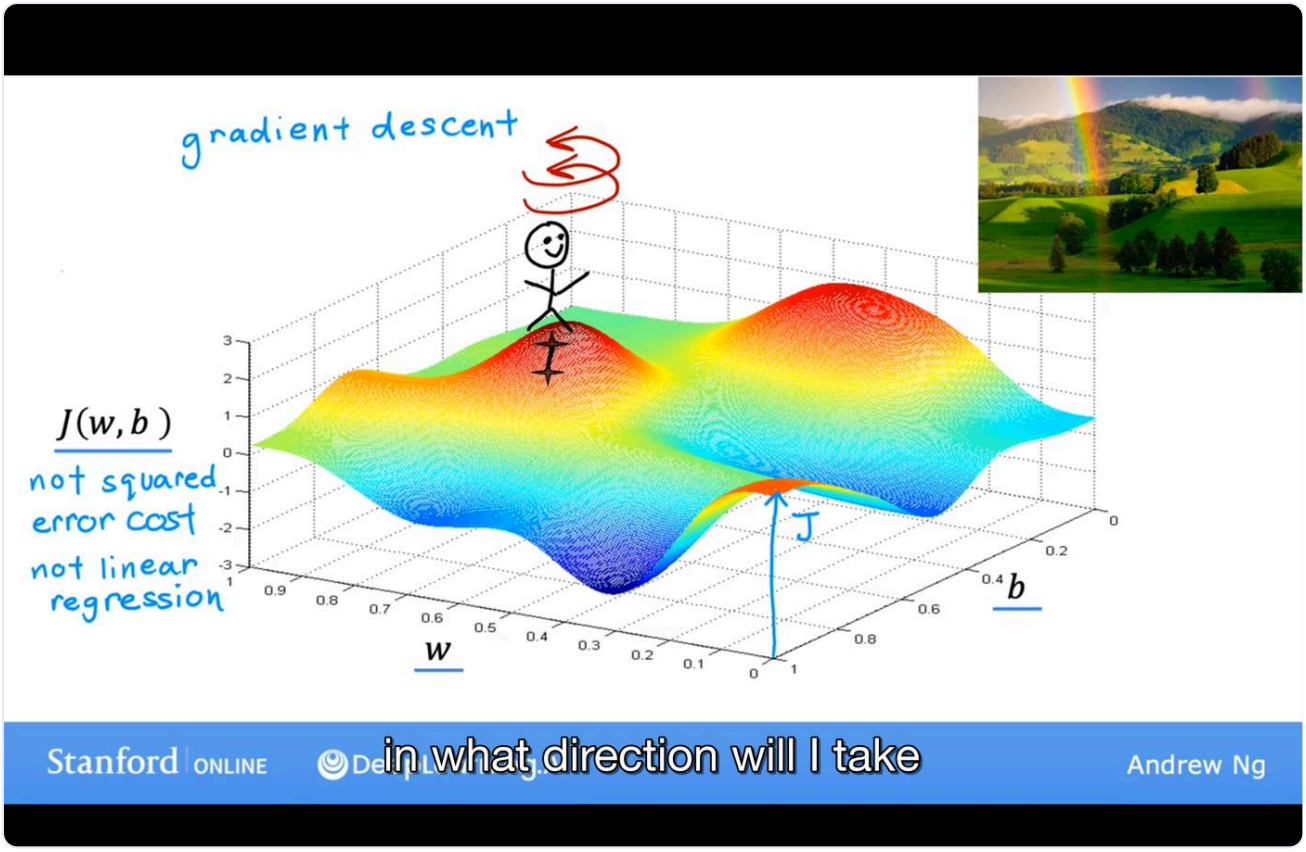


FIGURE 15.7

Standing at this new point, you're going to again spin around 360 degrees and ask yourself, in what direction will I take the next little baby step in order to move downhill?

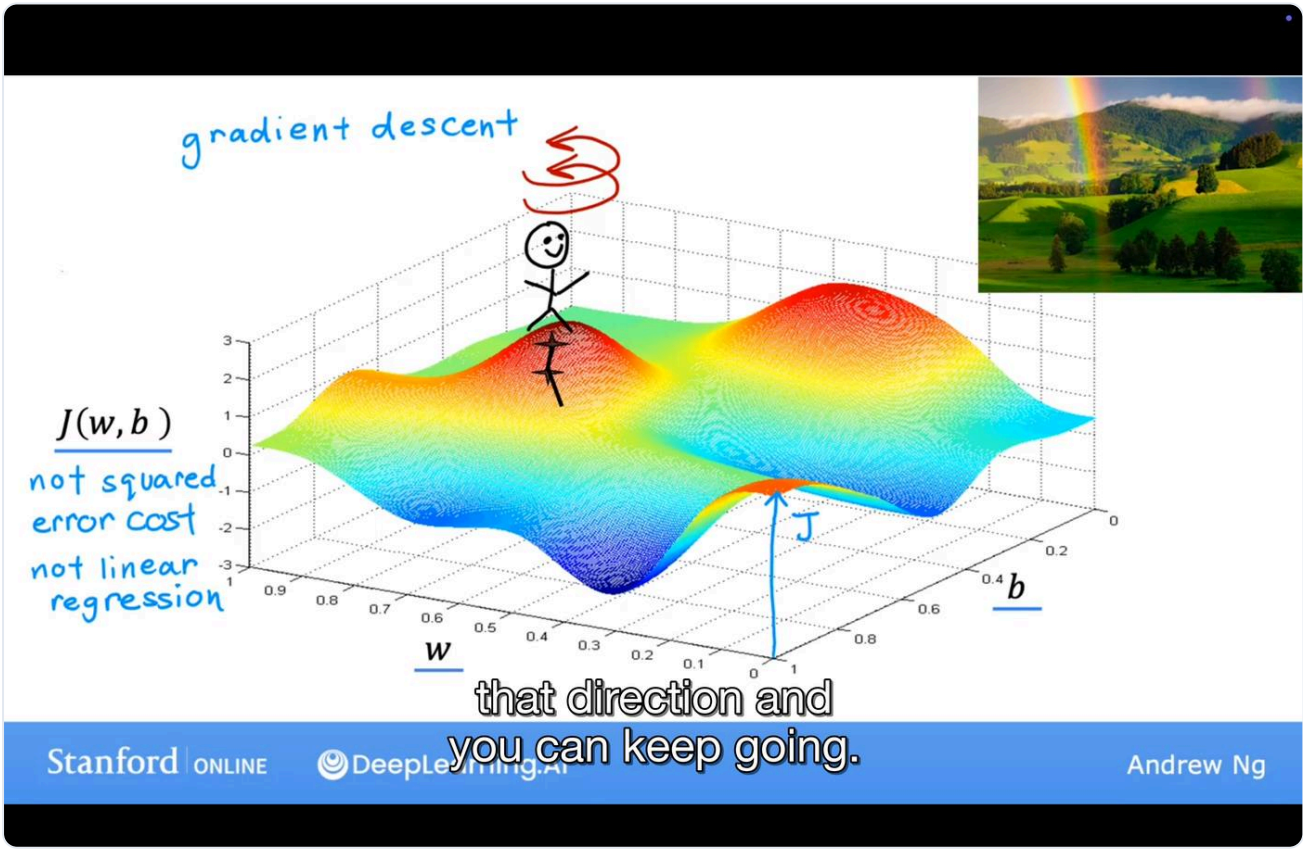


FIGURE 15.8

If you do that and take another step, you end up moving a bit in that direction and you can keep going.

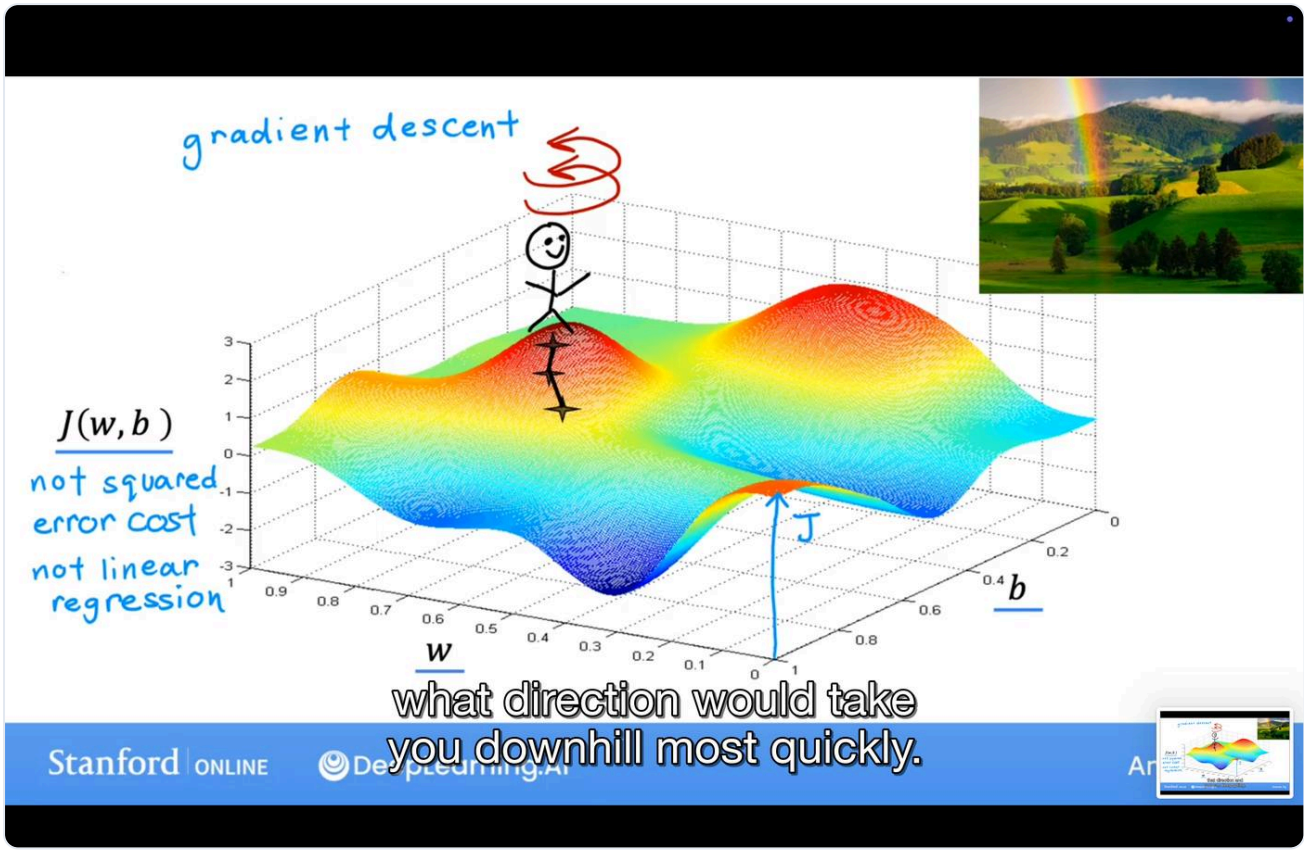


FIGURE 15.9

From this new point, you can again look around and decide what direction would take you downhill most quickly.

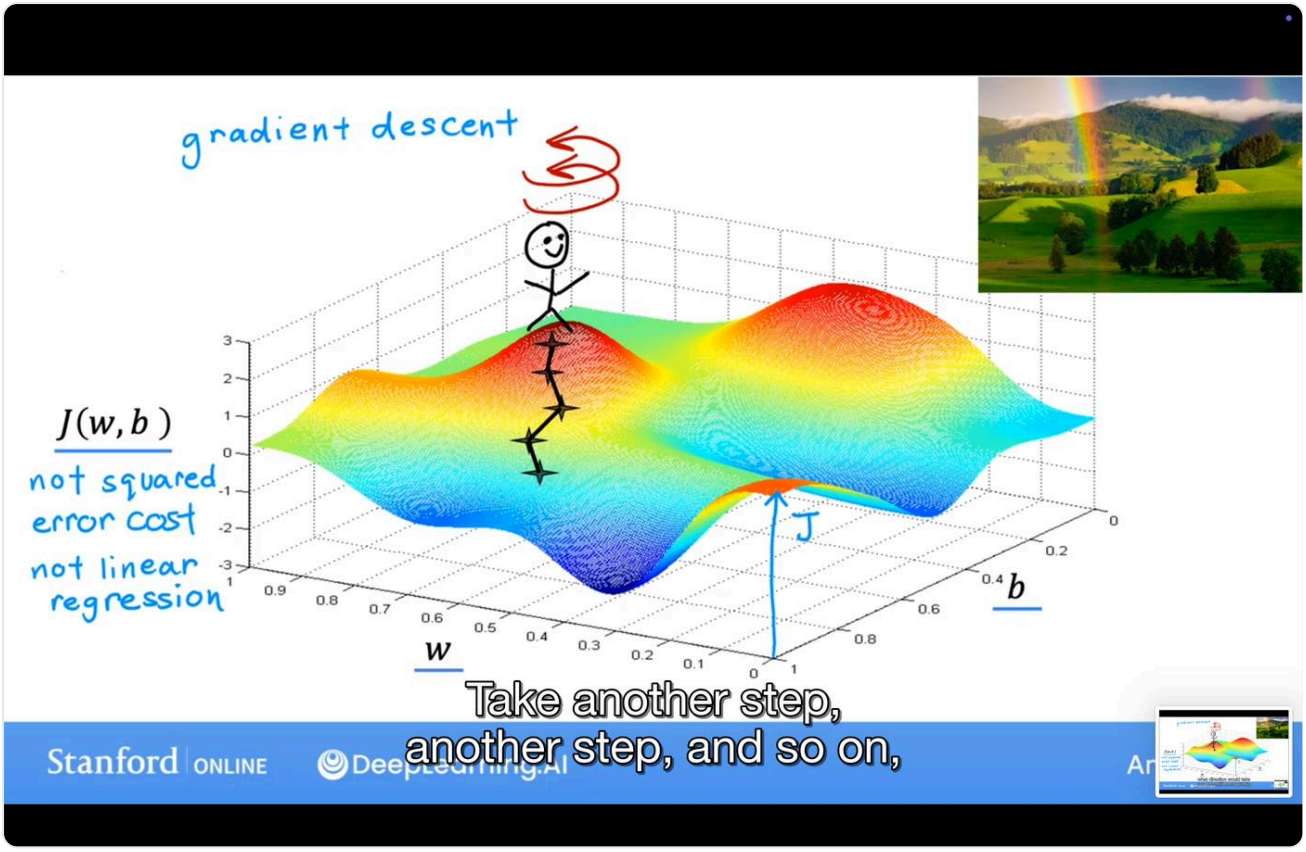


FIGURE 15.10

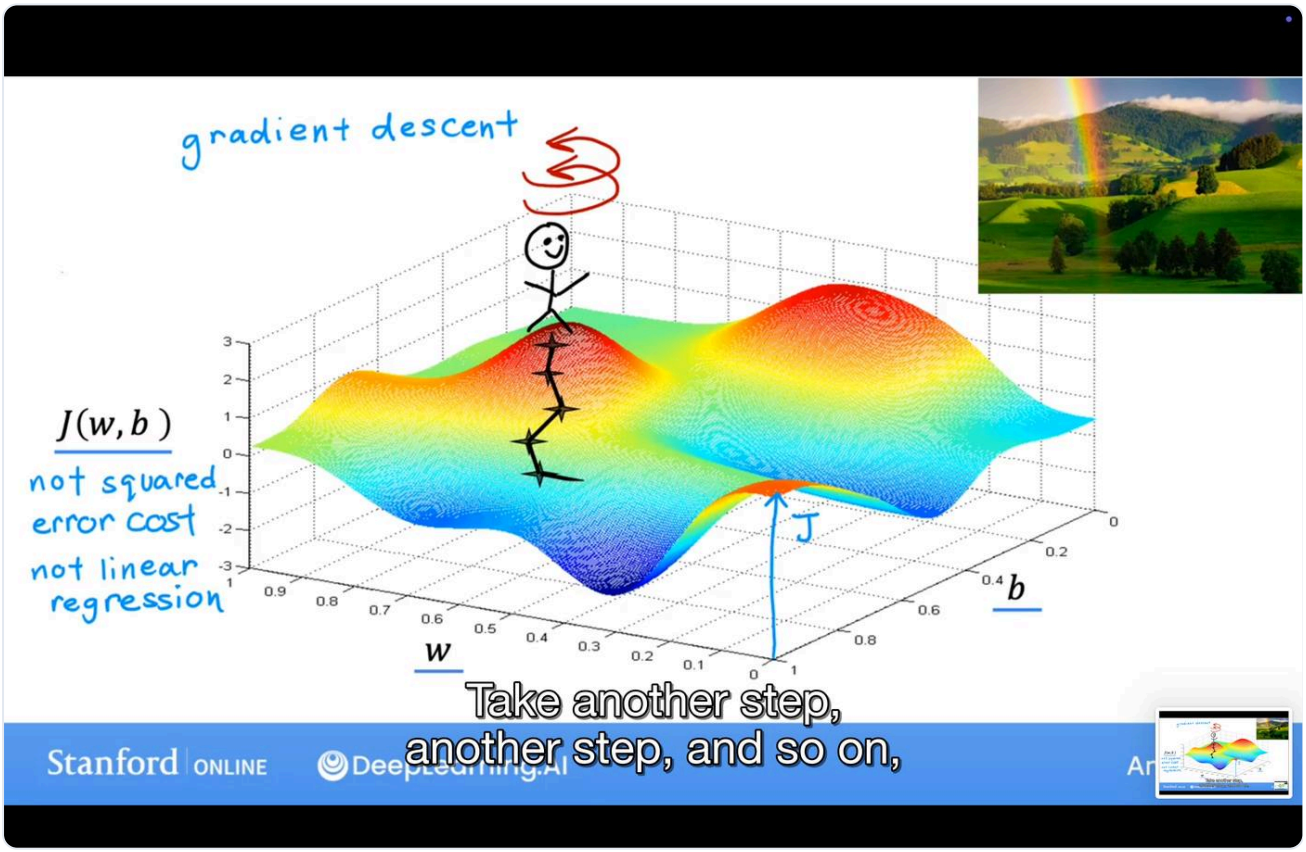


FIGURE 15.11

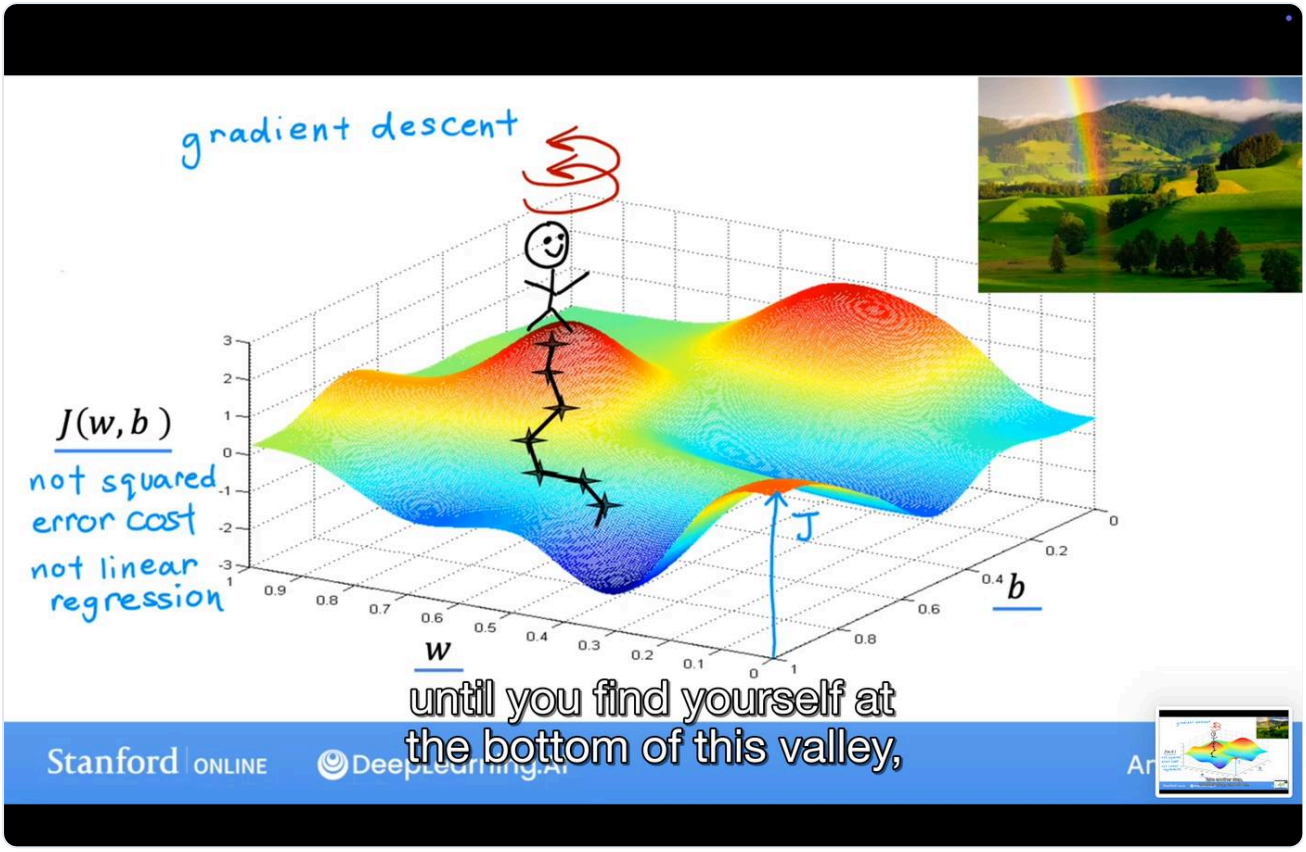


FIGURE 15.12

Take another step, another step, and so on, until you find yourself at the bottom of this valley, at this local minimum, right here.

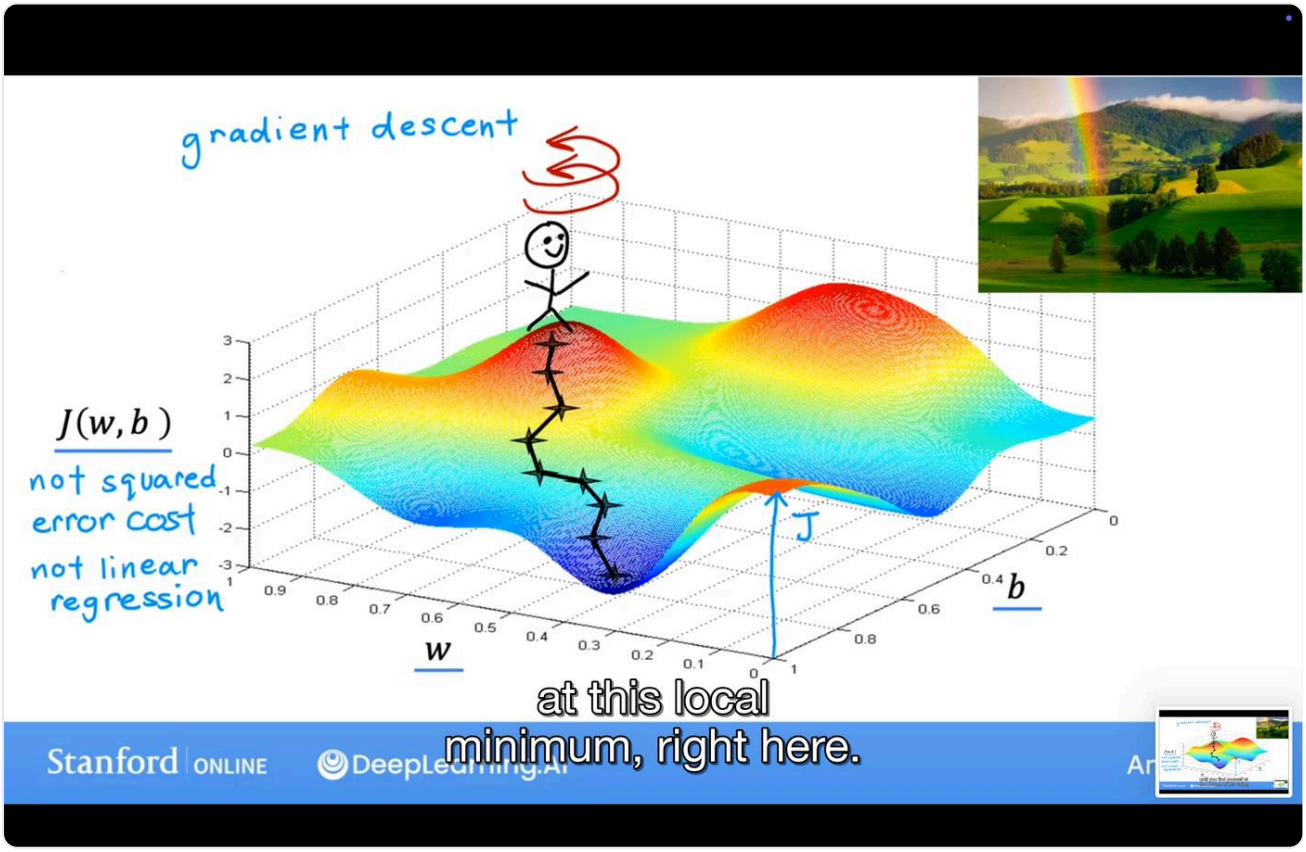


FIGURE 15.13

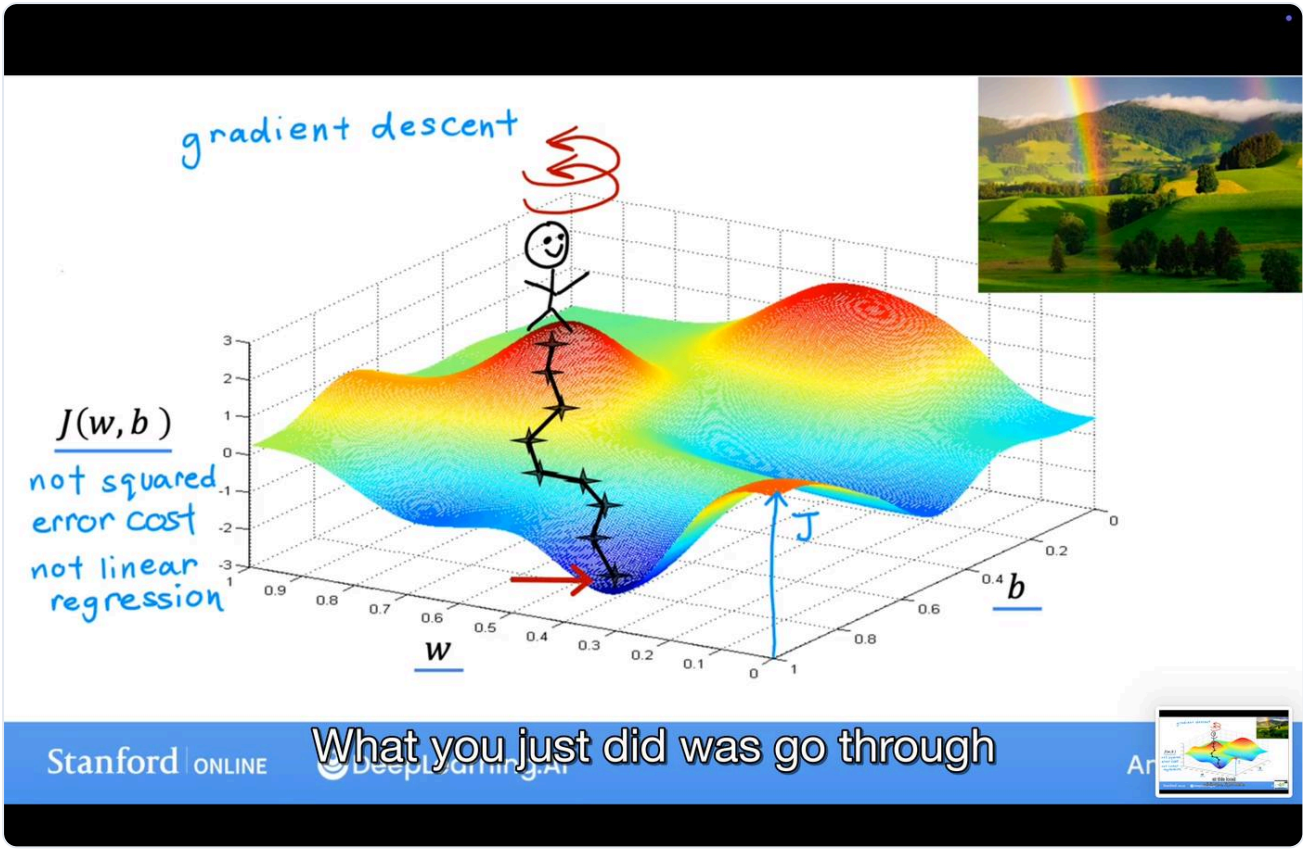


FIGURE 15.14

What you just did was go through multiple steps of gradient descent. It turns out, gradient descent has an interesting property. Remember that you can choose a starting point at the surface by choosing starting values for the parameters w and b. When you perform gradient descent a moment ago, you had started at this point over here.

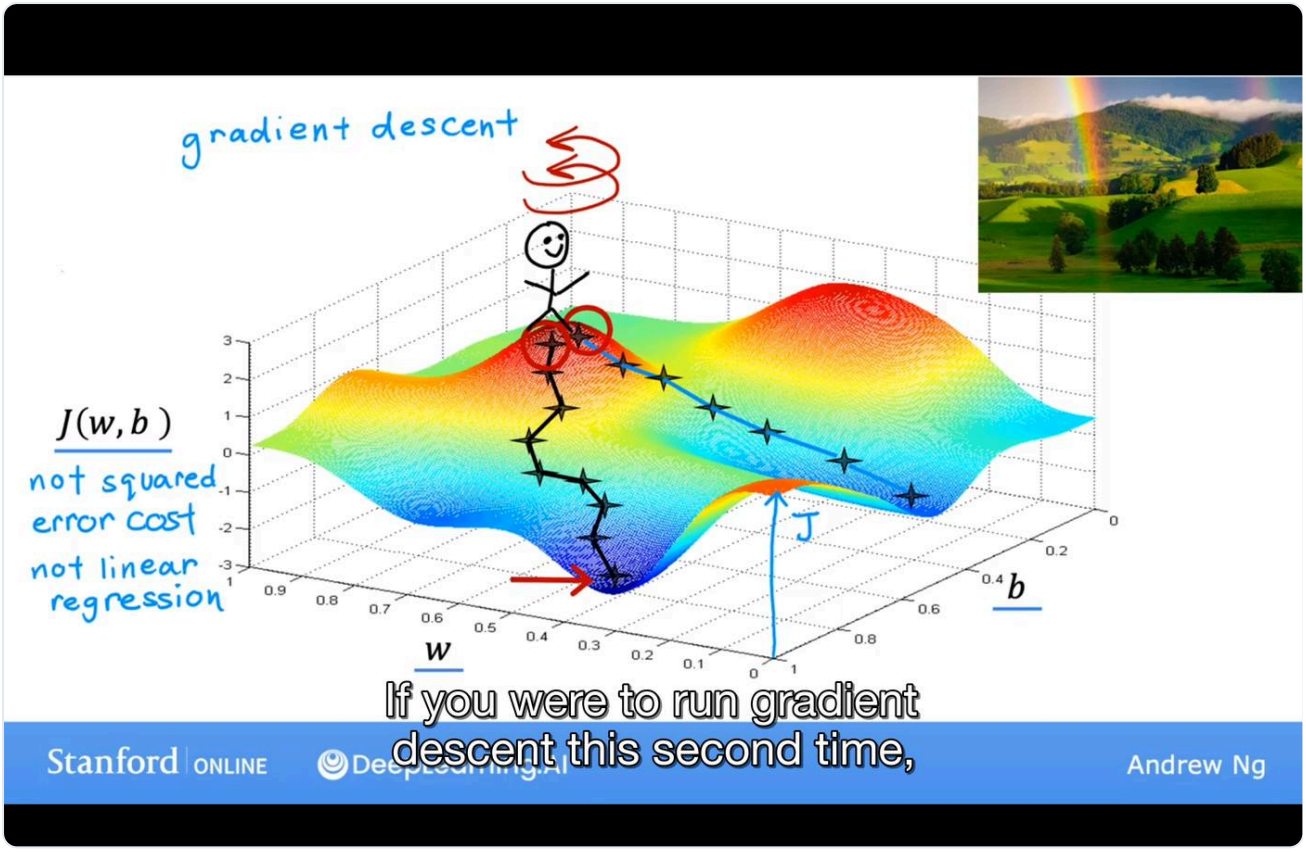


FIGURE 15.15

Now, imagine if you try gradient descent again, but this time you choose a different starting point by choosing parameters that place your starting point just a couple of steps to the right over here. If you then repeat the gradient descent process, which means you look around, take a little step in the direction of steepest ascent so you end up here. Then you again look around, take another step, and so on. If you were to run gradient descent this second time, starting just a couple steps in the right of where we did it the first time, then you end up in a totally different valley.

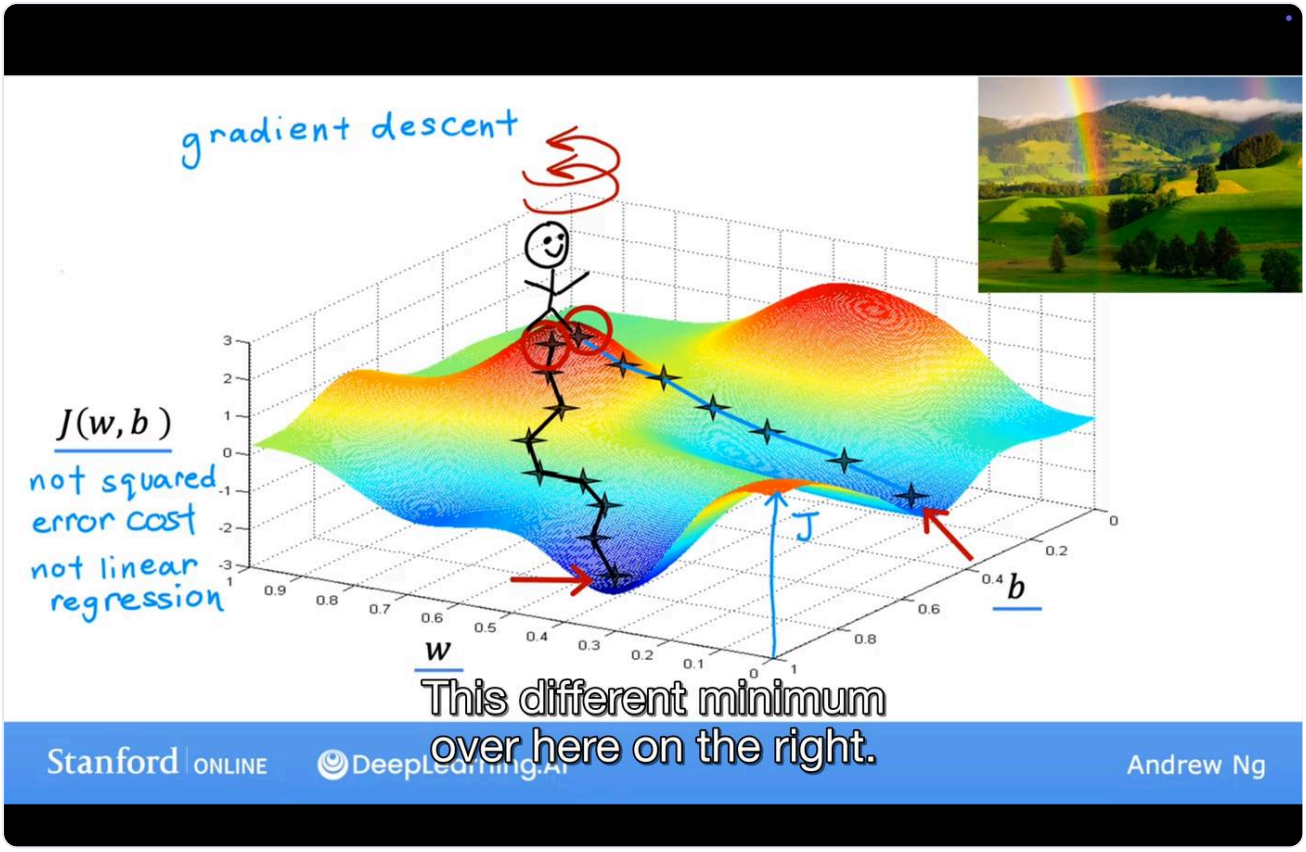


FIGURE 15.16

This different minimum over here on the right. The bottoms of both the first and the second valleys are called local minima.

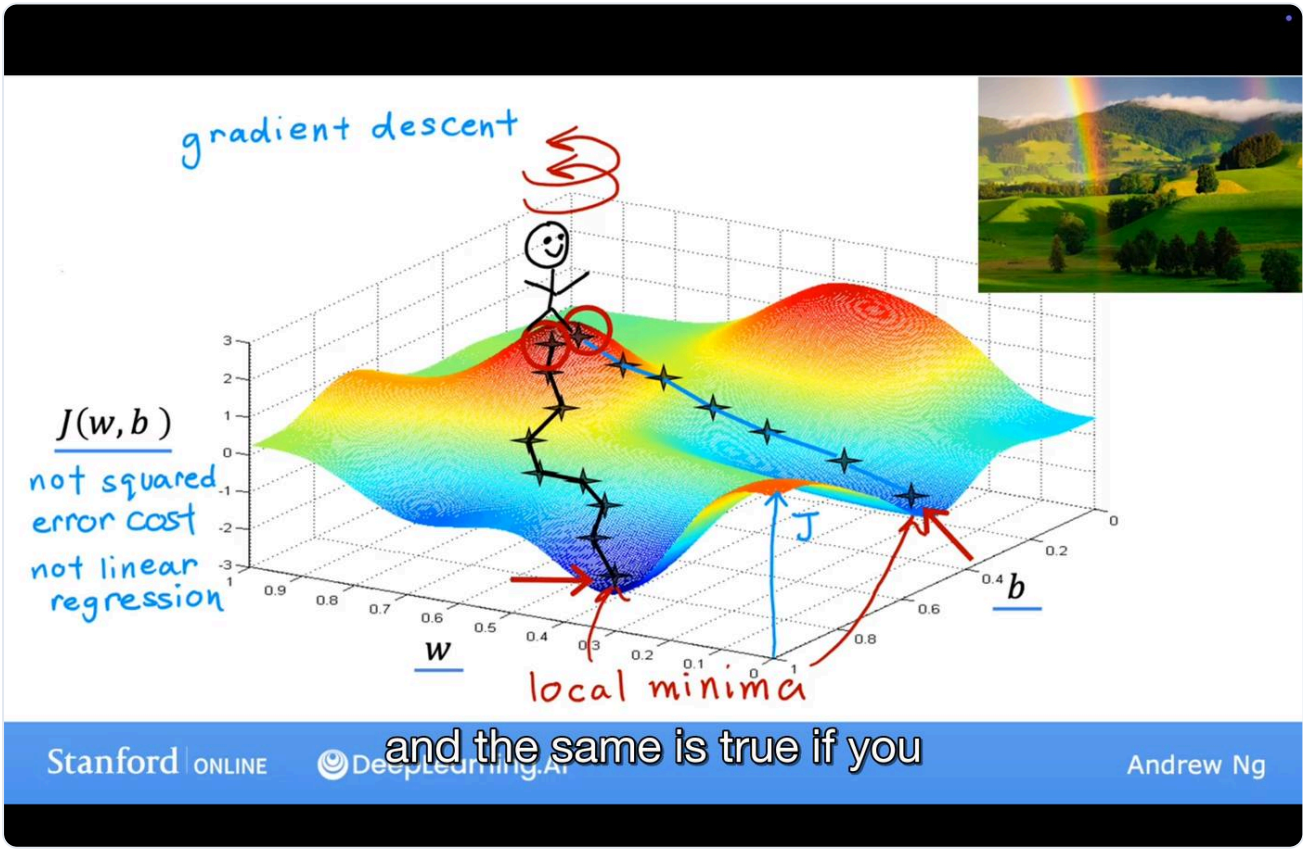


FIGURE 15.17

Because if you start going down the first valley, gradient descent won't lead you to the second valley, and the same is true if you started going down the second valley, you stay in that second minimum and not find your way into the first local minimum. This is an interesting property of the gradient descent algorithm, and you see more about this later. In this video, you saw how gradient descent helps you go downhill. In the next video, let's look at the mathematical expressions that you can implement to make gradient descent work. Let's go on to the next video.

LECTURE 16

Learning Rate

Let's take a look at how you can actually implement the gradient descent algorithm. Let me write down the gradient descent algorithm. Here it is. On each step, w, the parameter, is updated to the old value of w minus Alpha times this term d/dw of the cos function J of wb. What this expression is saying is, after your parameter w by taking the current value of w and adjusting it a small amount, which is this expression on the right, minus Alpha times this term over here. If you feel like there's a lot going on in this equation, it's okay, don't worry about it. We'll unpack it together. First, this equal notation here. Now, since I said we're assigning w a value using this equal sign, so in this context, this equal sign is the assignment operator.

Specifically, in this context, if you write code that says a equals c, it means take the value c and store it in your computer, in the variable a. Or if you write a equals a plus 1, it means set the value of a to be equal to a plus 1, or increments the value of a by one. The assignment operator encoding is different than truth assertions in mathematics. Where if I write a equals c, I'm asserting, that is, I'm claiming that the values of a and c are equal to each other. Hopefully, I will never write a truth assertion a equals a plus 1 because that just can't possibly be true. In Python and in other programming languages, truth assertions are sometimes written as equals equals, so you may see oh, that says a equals equals c if you're testing whether a is equal to c.

But in math notation, as we conventionally use it, like in these videos, the equal sign can be used for either assignments or for truth assertion. I try to make sure I was clear when I write an equal sign, whether we're assigning a value to a variable, or whether we're asserting the truth of the equality of two values. Now, this dive more deeply into what the symbols in this equation means. The symbol here is the Greek alphabet Alpha. In this equation, Alpha is also called the learning rate.

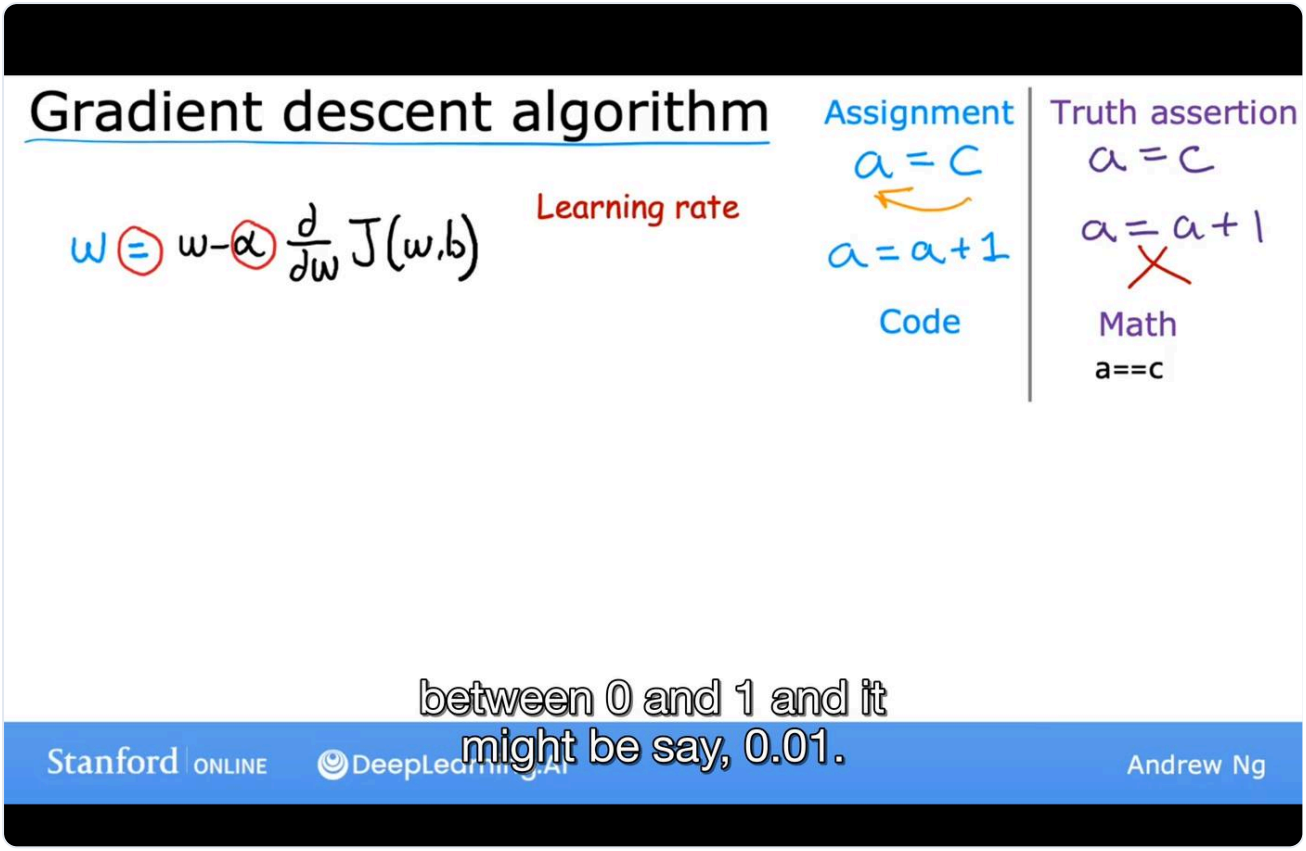


FIGURE 16.1 | Learning Rate

The learning rate is usually a small positive number between 0 and 1 and it might be say, 0.01. What Alpha does is, it basically controls how big of a step you take downhill. If Alpha is very large, then that corresponds to a very aggressive gradient descent procedure where you're trying to take huge steps downhill. If Alpha is very small, then you'd be taking small baby steps downhill. We'll come back later to dive more deeply into how to choose a good learning rate Alpha. Finally, this term here, that's the derivative term of the cost function J. Let's not worry about the details of this derivative right now. But later on, you'll get to see more about the derivative term.

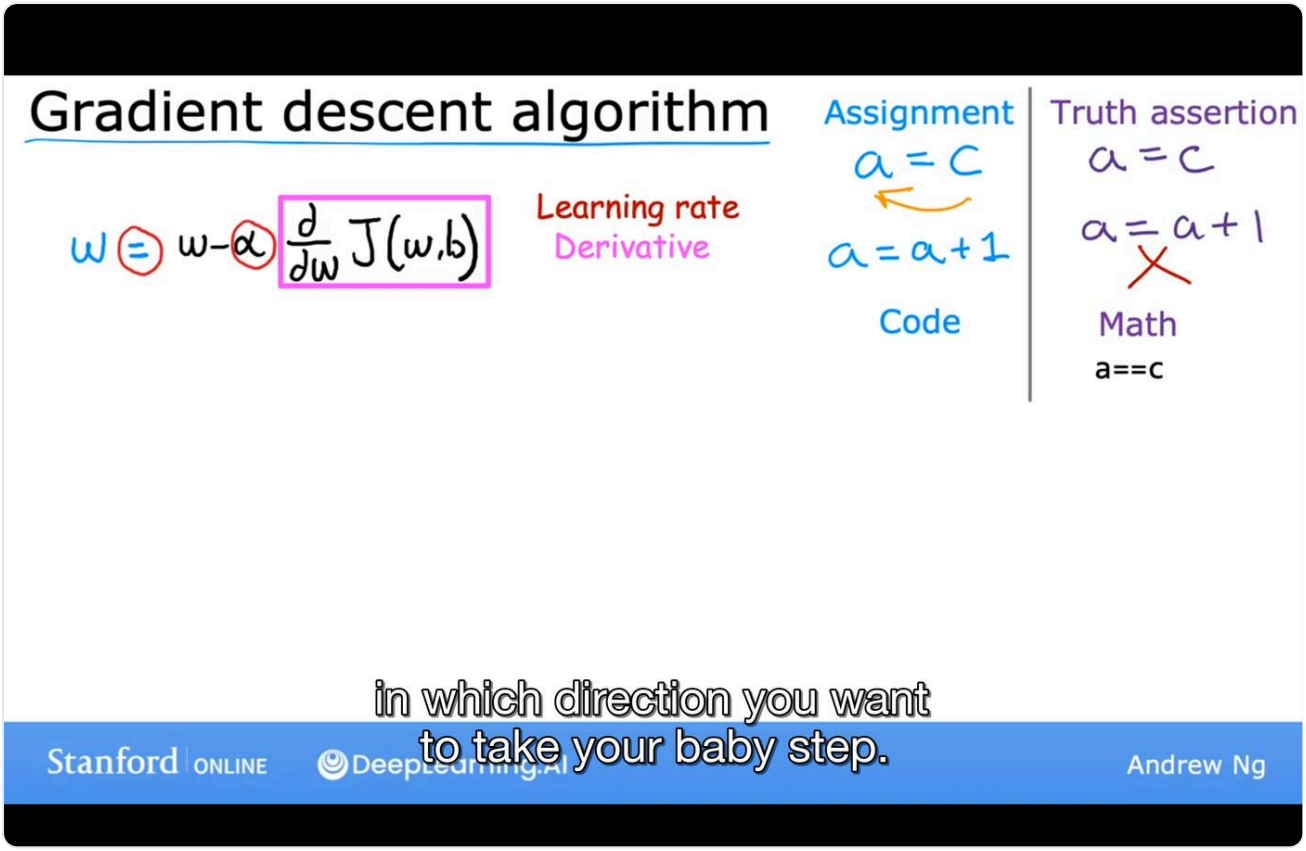


FIGURE 16.2 | Learning Rate

But for now, you can think of this derivative term that I drew a magenta box around as telling you in which direction you want to take your baby step. In combination with the learning rate Alpha, it also determines the size of the steps you want to take downhill. Now, I do want to mention that derivatives come from calculus. Even if you aren't familiar with calculus, don't worry about it. Even without knowing any calculus, you'd be able to figure out all you need to know about this derivative term in this video and the next. One more thing. Remember your model has two parameters, not just w, but also b. You also have an assignment operations update the parameter b that looks very similar. b is assigned the old value of b minus the learning rate Alpha times this slightly different derivative term, d/db of J of wb.

Remember in the graph of the surface plot where you're taking baby steps until you get to the bottom of the value, well, for the gradient descent algorithm, you're going to repeat these two update steps until the algorithm converges.

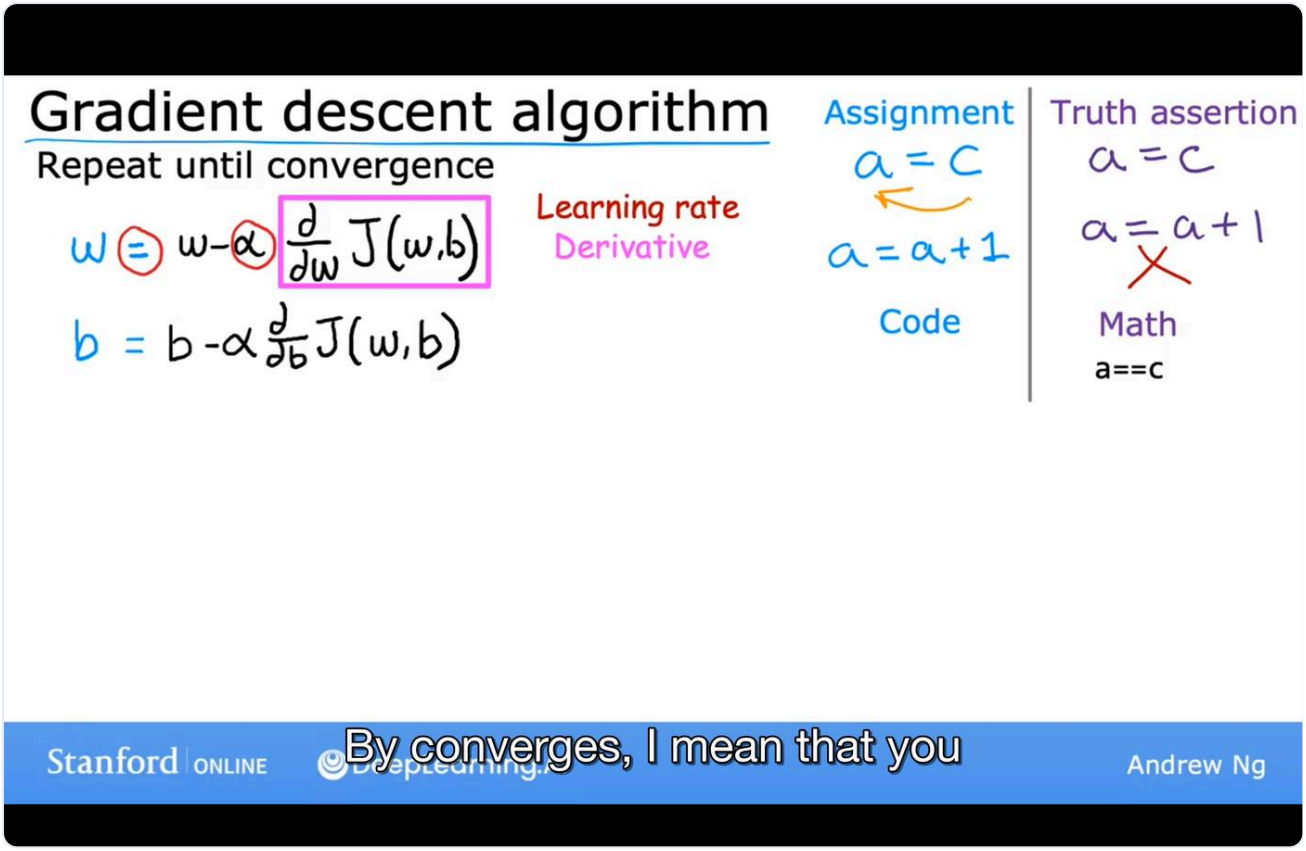


FIGURE 16.3 | Learning Rate

By converges, I mean that you reach the point at a local minimum where the parameters w and b no longer change much with each additional step that you take. Now, there's one more subtle detail about how to correctly in semantic gradient descent, you're going to update two parameters, w and b. This update takes place for both parameters, w and b. One important detail is that for gradient descent, you want to simultaneously update w and b, meaning you want to update both parameters at the same time. What I mean by that, is that in this expression, you're going to update w from the old w to a new w, and you're also updating b from its oldest value to a new value of b.

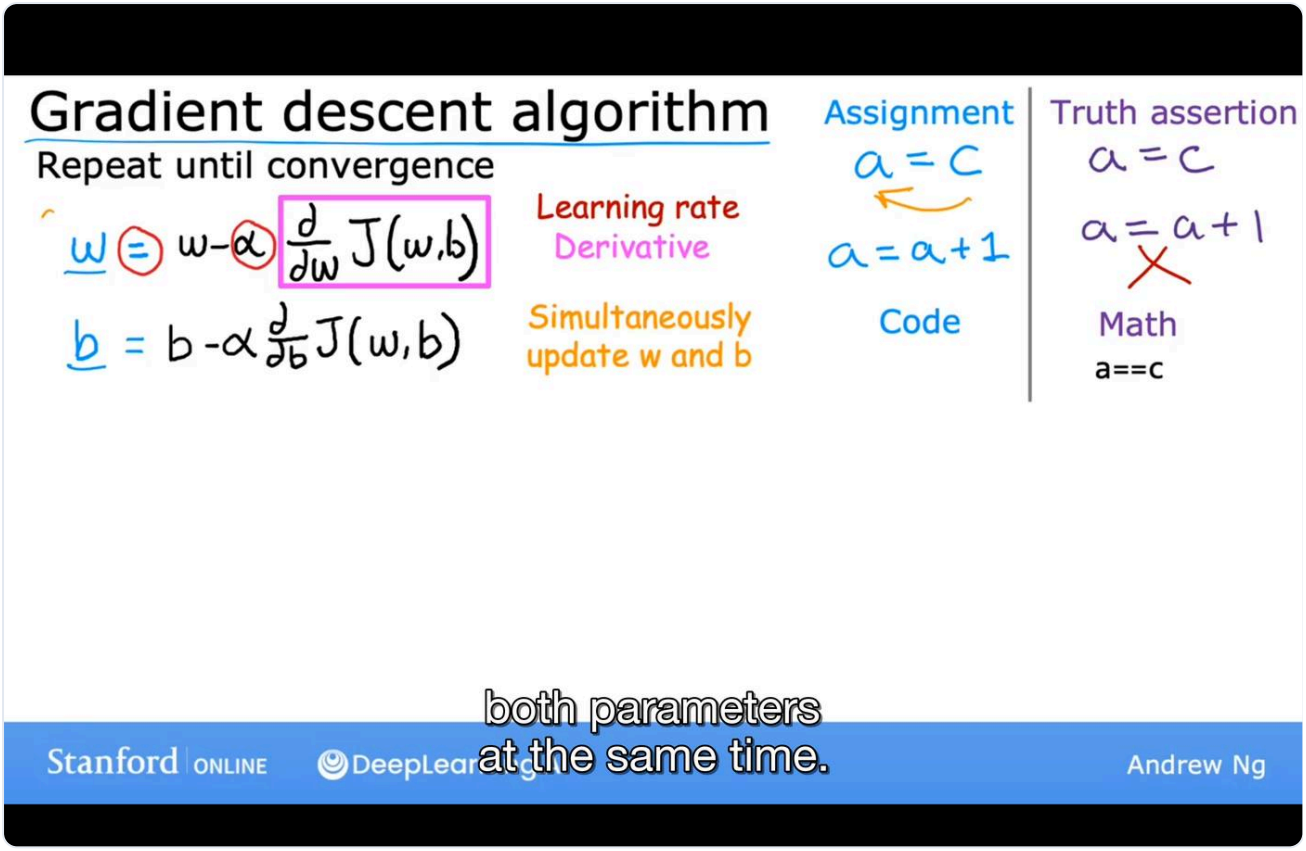


FIGURE 16.4 | Learning Rate

The way to implement this is to compute the right side, computing this thing for w and b, and simultaneously at the same time, update w and b to the new values. Let's take a look at what this means. Here's the correct way to implement gradient descent which does a simultaneous update. This sets a variable temp_w equal to that expression, which is w minus that term here. There's also a set in another variable temp_b to that, which is b minus that term.

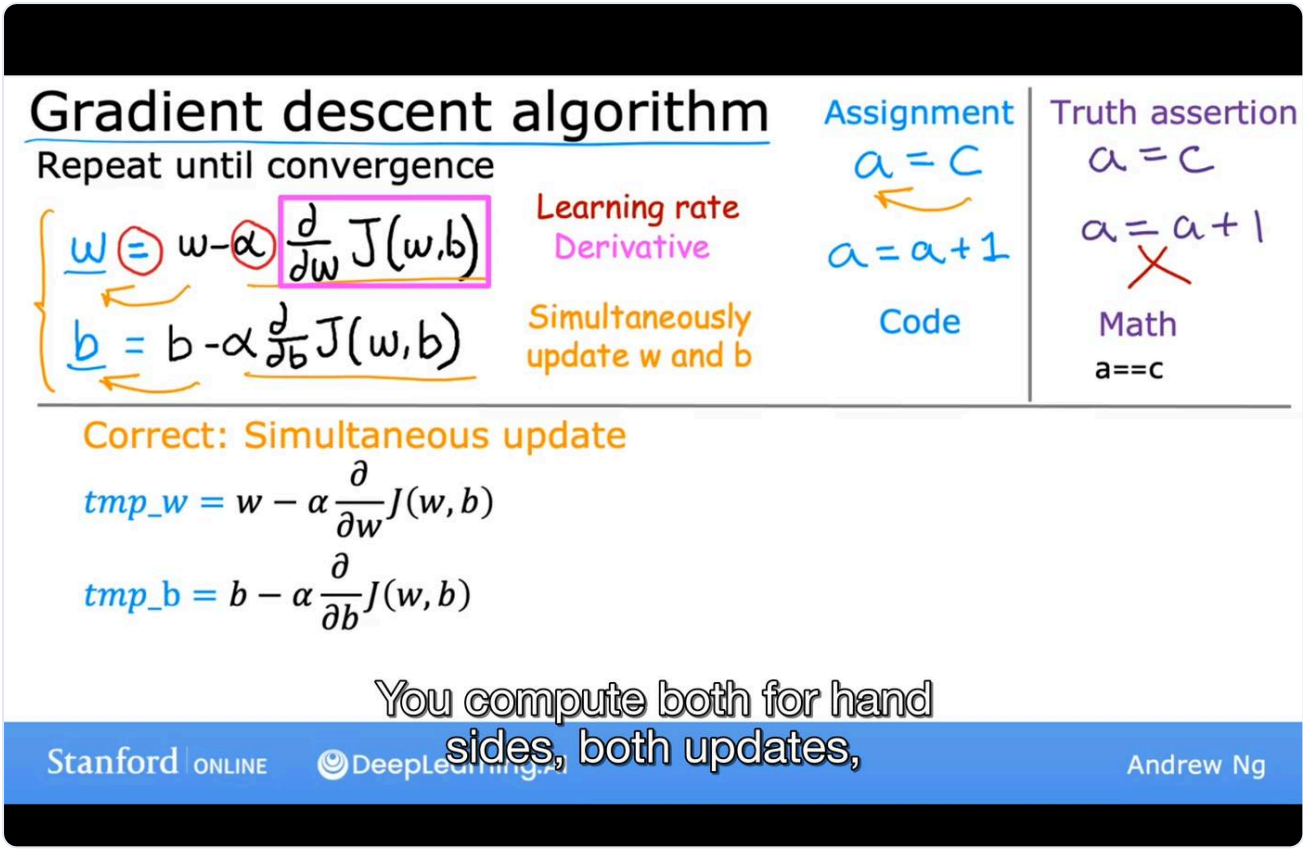


FIGURE 16.5 | Learning Rate

You compute both for hand sides, both updates, and store them into variables temp_w and temp_b. Then you copy the value of temp_w into w, and you also copy the value of temp_b into b. Now, one thing you may notice is that this value of w is from the for w gets updated. Here, I noticed that the pre-update w is where it goes into the derivative term over here.

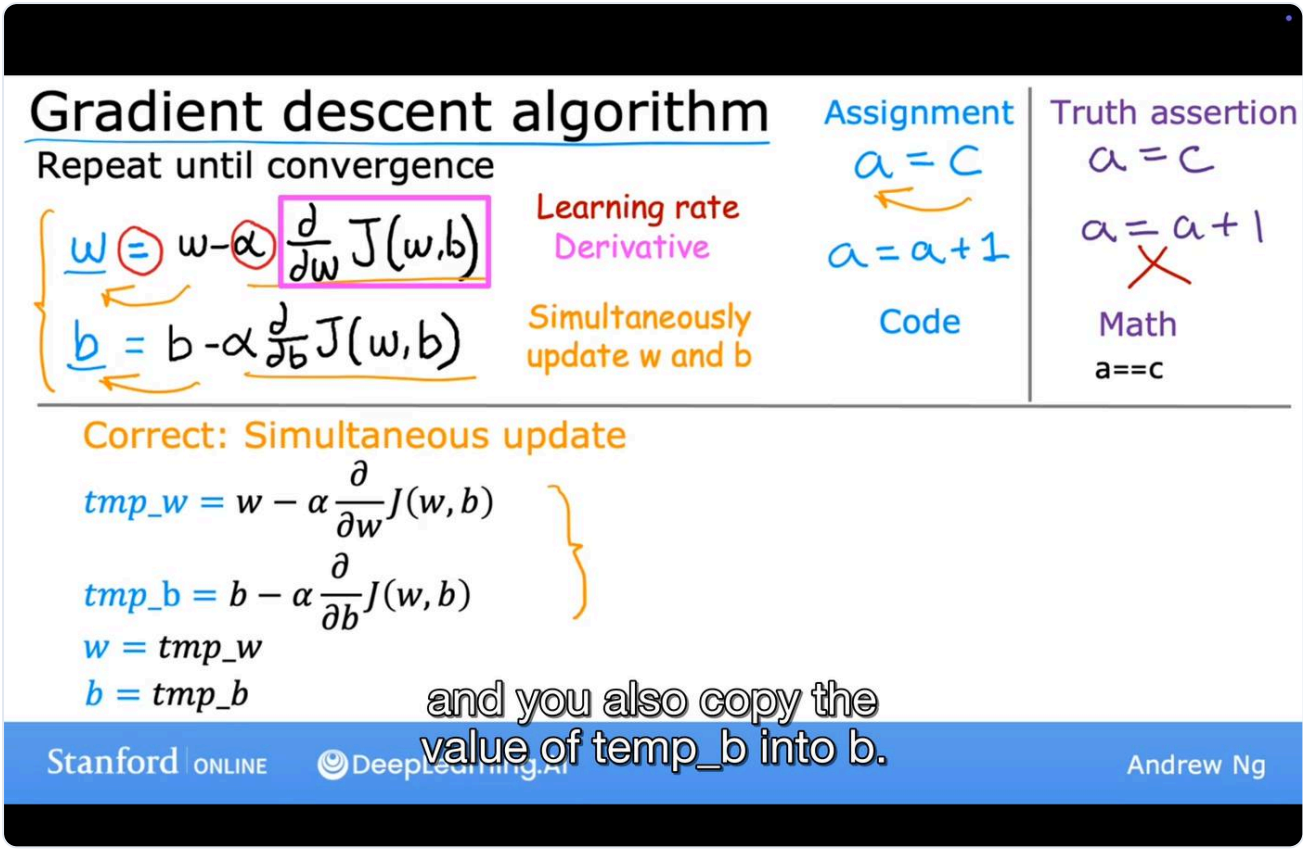


FIGURE 16.6 | Learning Rate

In contrast, here is an incorrect implementation of gradient descent that does not do a simultaneous update. In this incorrect implementation, we compute temp_w, same as before, so far that's okay. Now here's where things start to differ. We then update w with the value in temp_w before calculating the new value for the other parameter to be. Next, we calculate temp_b as b minus that term here, and finally, we update b with the value in temp_b.

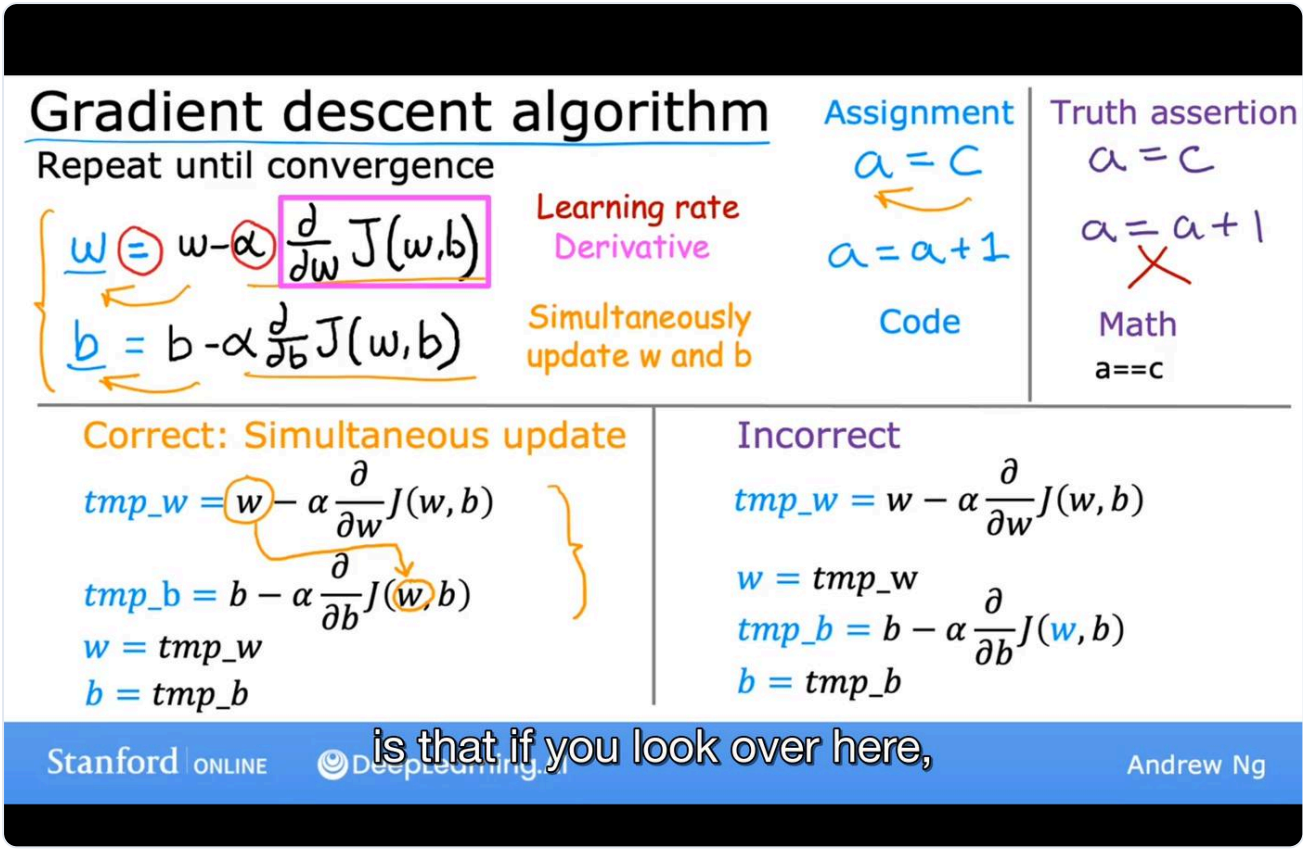


FIGURE 16.7

The difference between the right-hand side and the left-hand side implementations is that if you look over here, this w has already been updated to this new value, and this is updated w that actually goes into the cost function j of w, b. It means that this term here on the right is not the same as this term over here that you see on the left. That also means this temp_b term on the right is not quite the same as the temp b term on the left, and thus this updated value for b on the right is not the same as this updated value for variable b on the left. The way that gradient descent is implemented in code, it actually turns out to be more natural to implement it the correct way with simultaneous updates.

When you hear someone talk about gradient descent, they always mean the gradient descents where you perform a simultaneous update of the parameters. If however, you were to implement non-simultaneous update, it turns out it will probably work more or less anyway. But doing it this way isn't really the correct way to implement it, is actually some other algorithm with different properties. I would advise you to just stick to the correct simultaneous update and not use this incorrect version on the right. That's gradient descent. In the next video, we'll go into details of the derivative term which you saw in this video, but that we didn't really talk about in detail.

Derivatives are part of calculus, and again, if you're not familiar with calculus, don't worry about it. You don't need to know calculus at all in order to complete this course or this specialization, and you have all the information you need in order to implement gradient descent. Coming up in the next video, we'll go over derivatives together, and you come away with the intuition and knowledge you need to be able to implement and apply gradient descent yourself. I think that'll be an exciting thing for you to know how to implement. Let's go on to the next video to see how to do that.

LECTURE 17

Gradient Descent Algorithm

Now let's dive more deeply in gradient descent to gain better intuition about what it's doing and why it might make sense. Here's the gradient descent algorithm that you saw in the previous video. As a reminder, this variable, this Greek symbol Alpha, is the learning rate. The learning rate controls how big of a step you take when updating the model's parameters, w and b. This term here, this d over dw, this is a derivative term. By convention in math, this d is written with this funny font here. In case anyone watching this has PhD in math or is an expert in multivariate calculus, they may be wondering, that's not the derivative, that's the partial derivative. Yes, they be right.

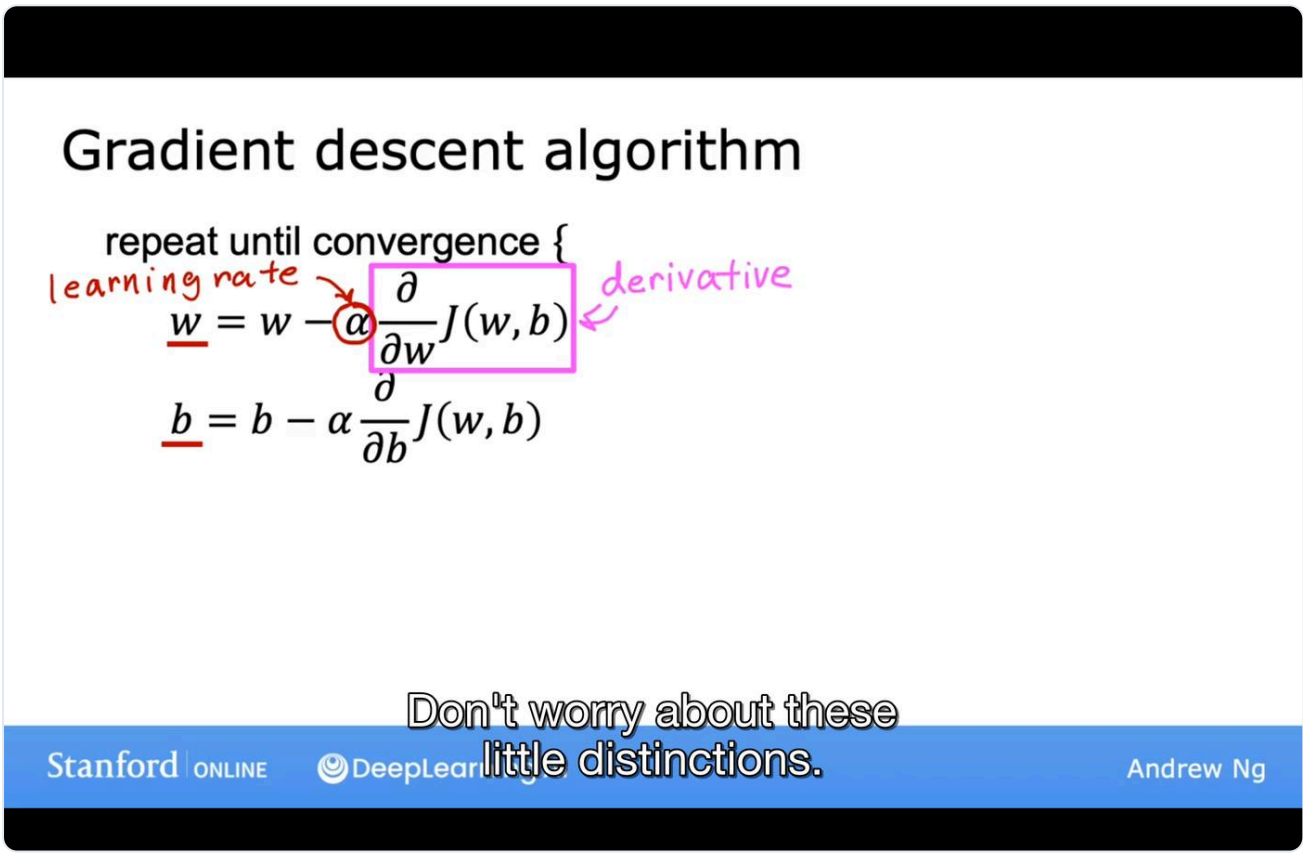


FIGURE 17.1 | Gradient Descent Algorithm

But for the purposes of implementing a machine learning algorithm, I'm just going to call it derivative. Don't worry about these little distinctions. What we're going to focus on now is get more intuition about what this learning rate and what this derivative are doing and why when multiplied together like this, it results in updates to parameters w and b. That makes sense. In order to do this let's use a slightly simpler example where we work on minimizing just one parameter. Let's say that you have a cost function J of just one parameter w with w is a number. This means the gradient descent now looks like this.

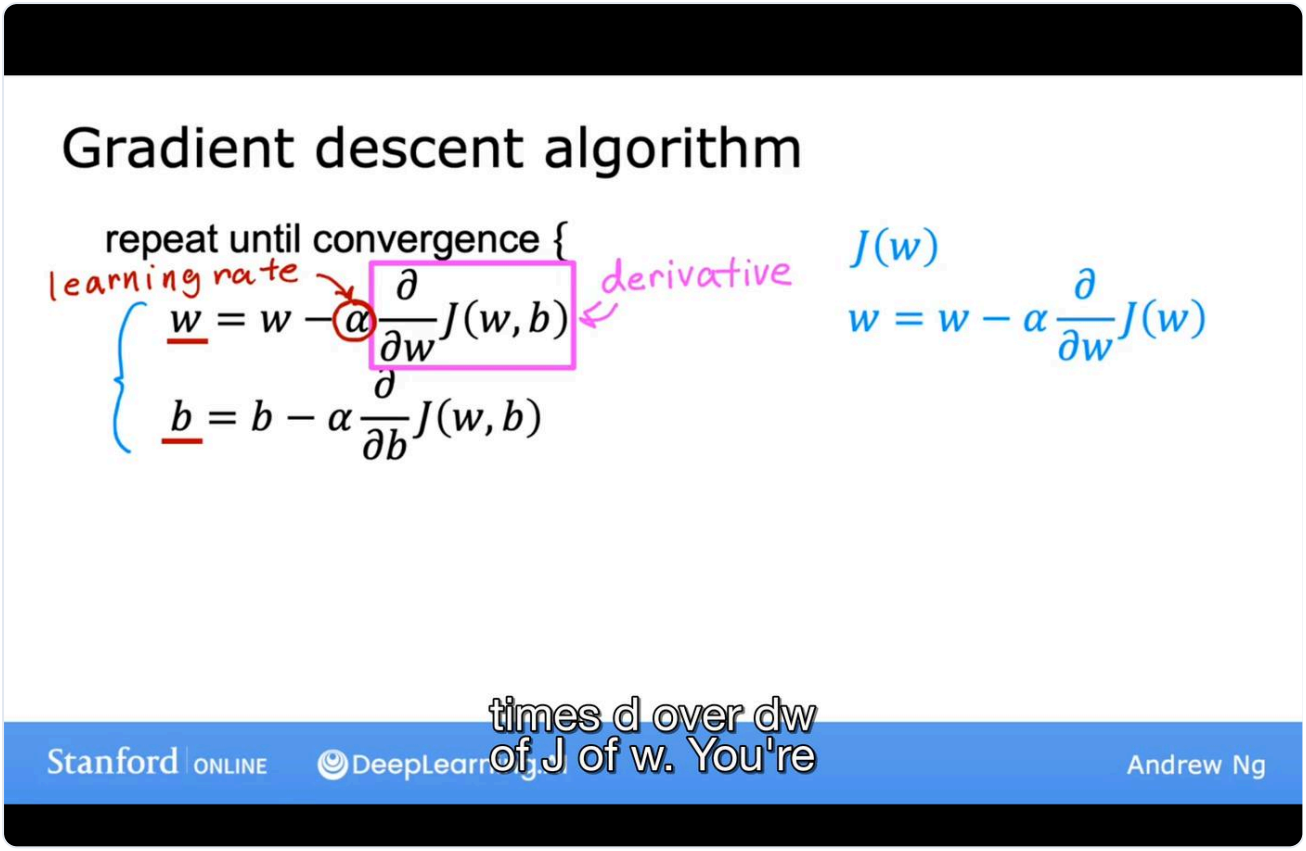


FIGURE 17.2 | Gradient Descent Algorithm

W is updated to w minus the learning rate Alpha times d over dw of J of w.

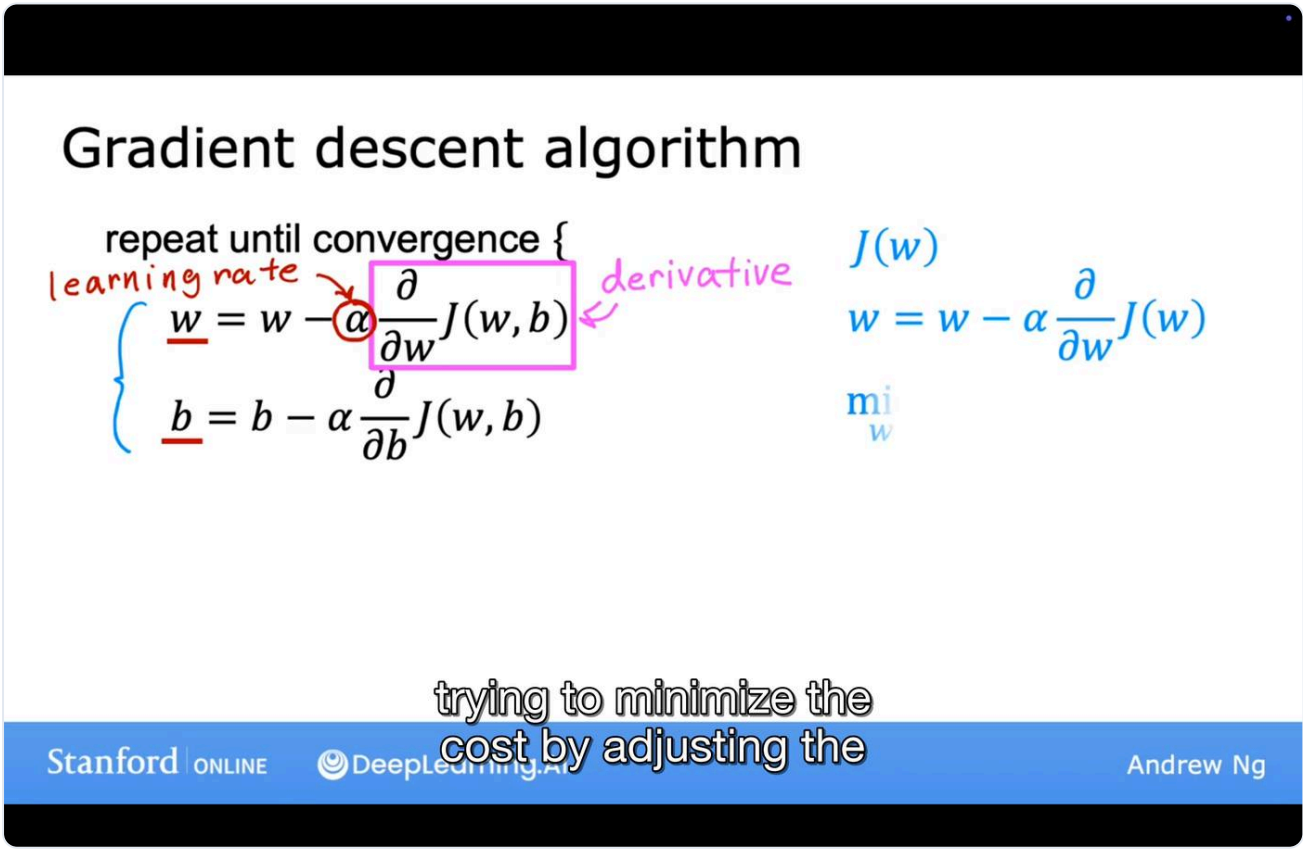


FIGURE 17.3 | Gradient Descent Algorithm

You're trying to minimize the cost by adjusting the parameter w.

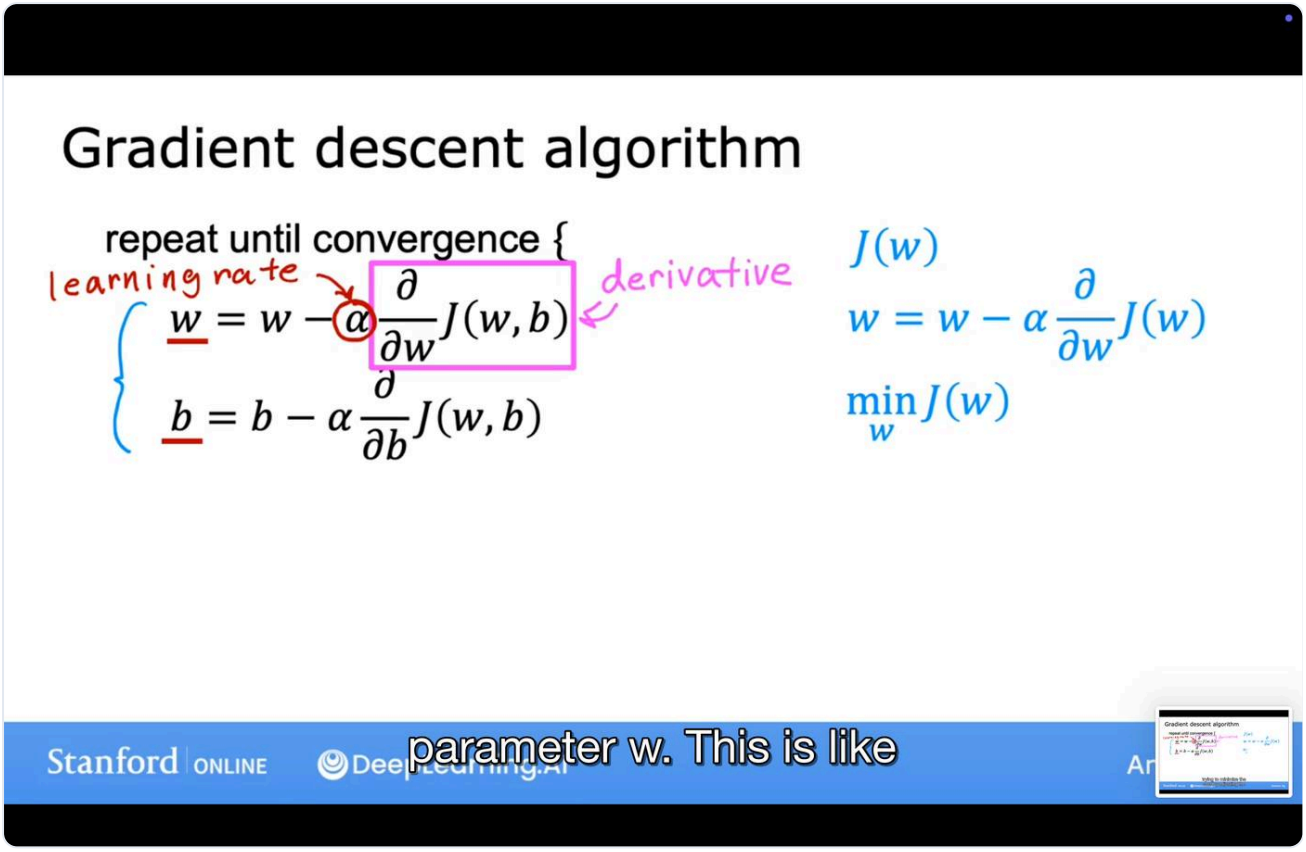


FIGURE 17.4 | Gradient Descent Algorithm

This is like our previous example where we had temporarily set b equal to 0 with one parameter w instead of two, you can look at two-dimensional graphs of the cost function j, instead of three dimensional graphs. Let's look at what gradient descent does on just function J of w. Here on the horizontal axis is parameter w, and on the vertical axis is the cost j of w. Now less initialized gradient descent with some starting value for w.

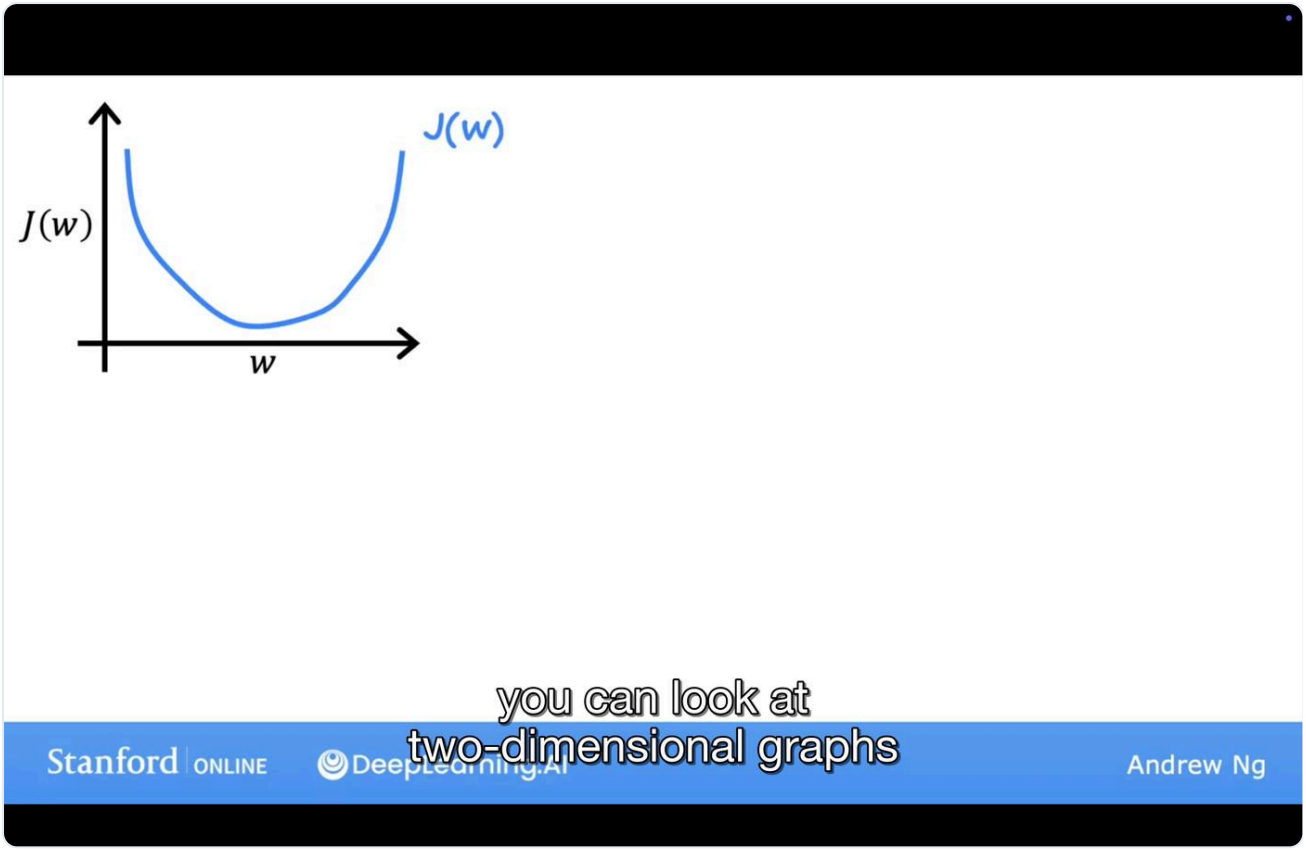


FIGURE 17.5

Let's initialize it at this location. Imagine that you start off at this point right here on the function J, what gradient descent will do is it will update w to be w minus learning rate Alpha times d over dw of J of w. Let's look at what this derivative term here means.

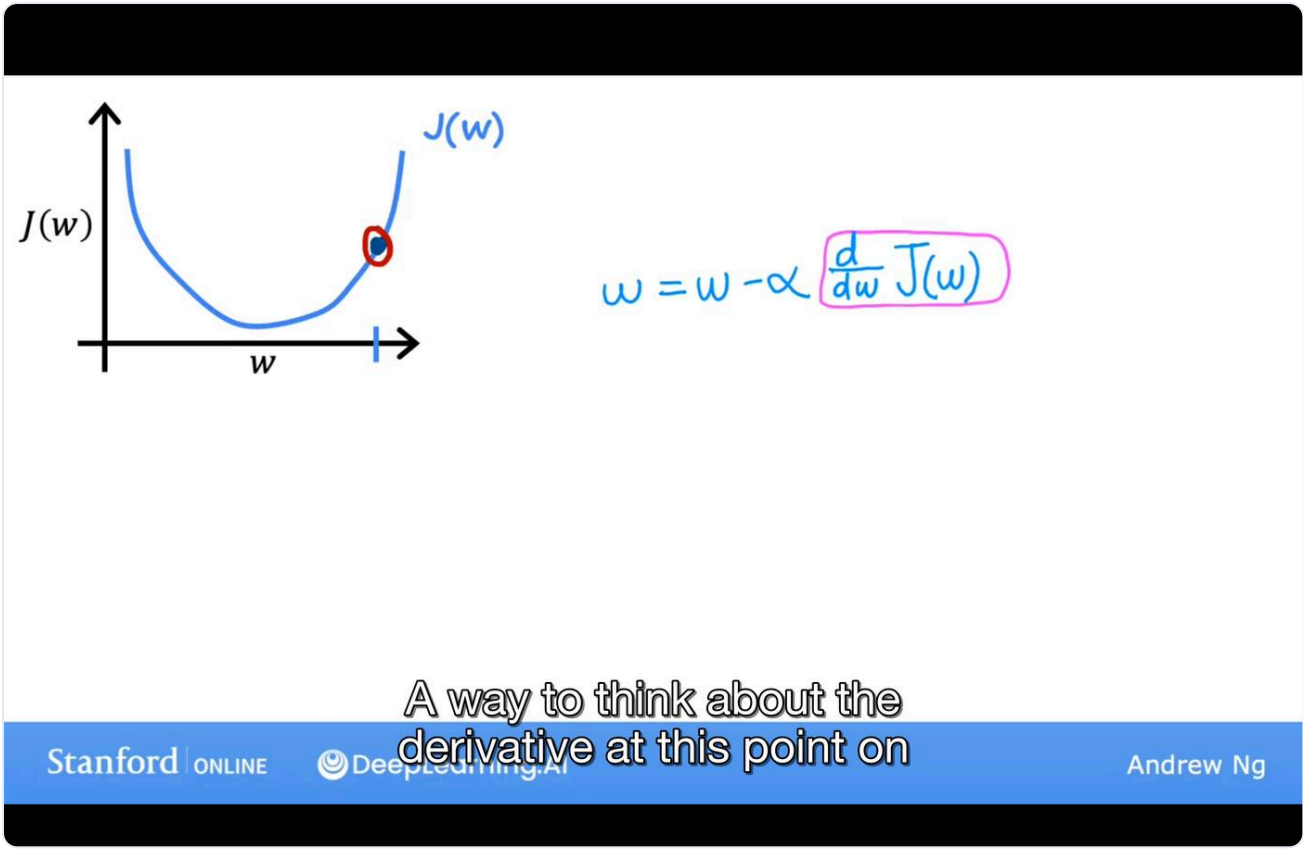


FIGURE 17.6

A way to think about the derivative at this point on the line is to draw a tangent line, which is a straight line that touches this curve at that point. Enough, the slope of this line is the derivative of the function j at this point. To get the slope, you can draw a little triangle like this. If you compute the height divided by the width of this triangle, that is the slope.

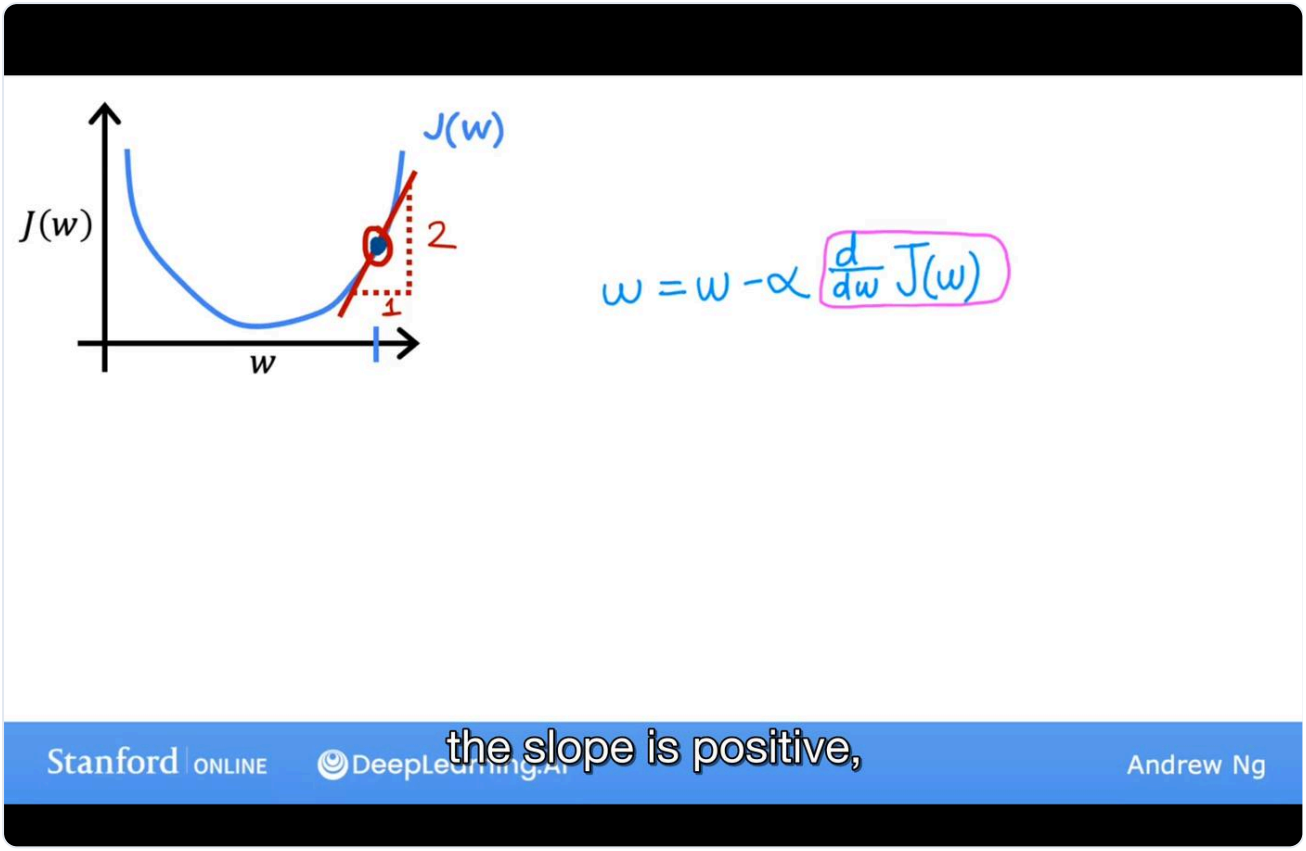


FIGURE 17.7

For example, this slope might be 2 over 1, for instance and when the tangent line is pointing up and to the right, the slope is positive, which means that this derivative is a positive number, so is greater than 0. The updated w is going to be w minus the learning rate times some positive number.

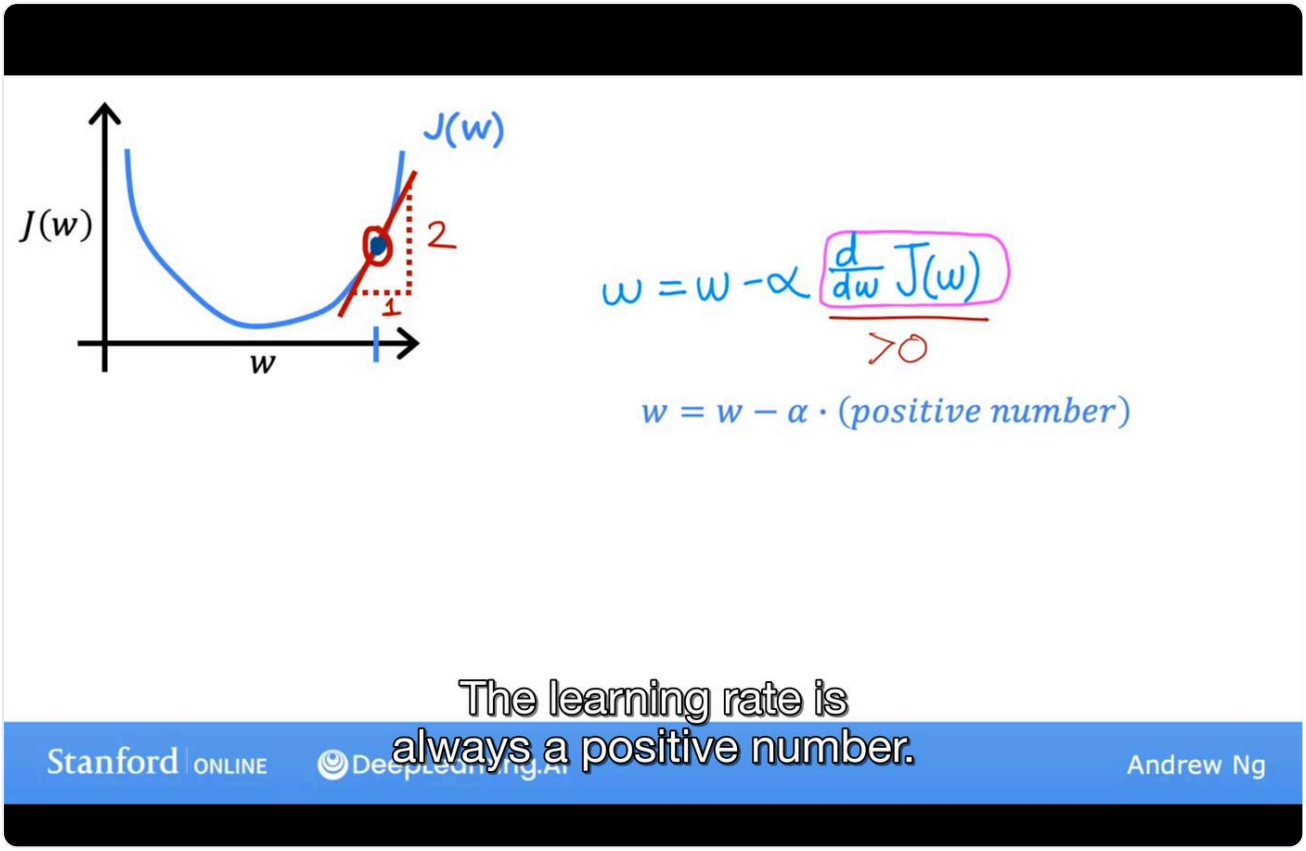


FIGURE 17.8

The learning rate is always a positive number.

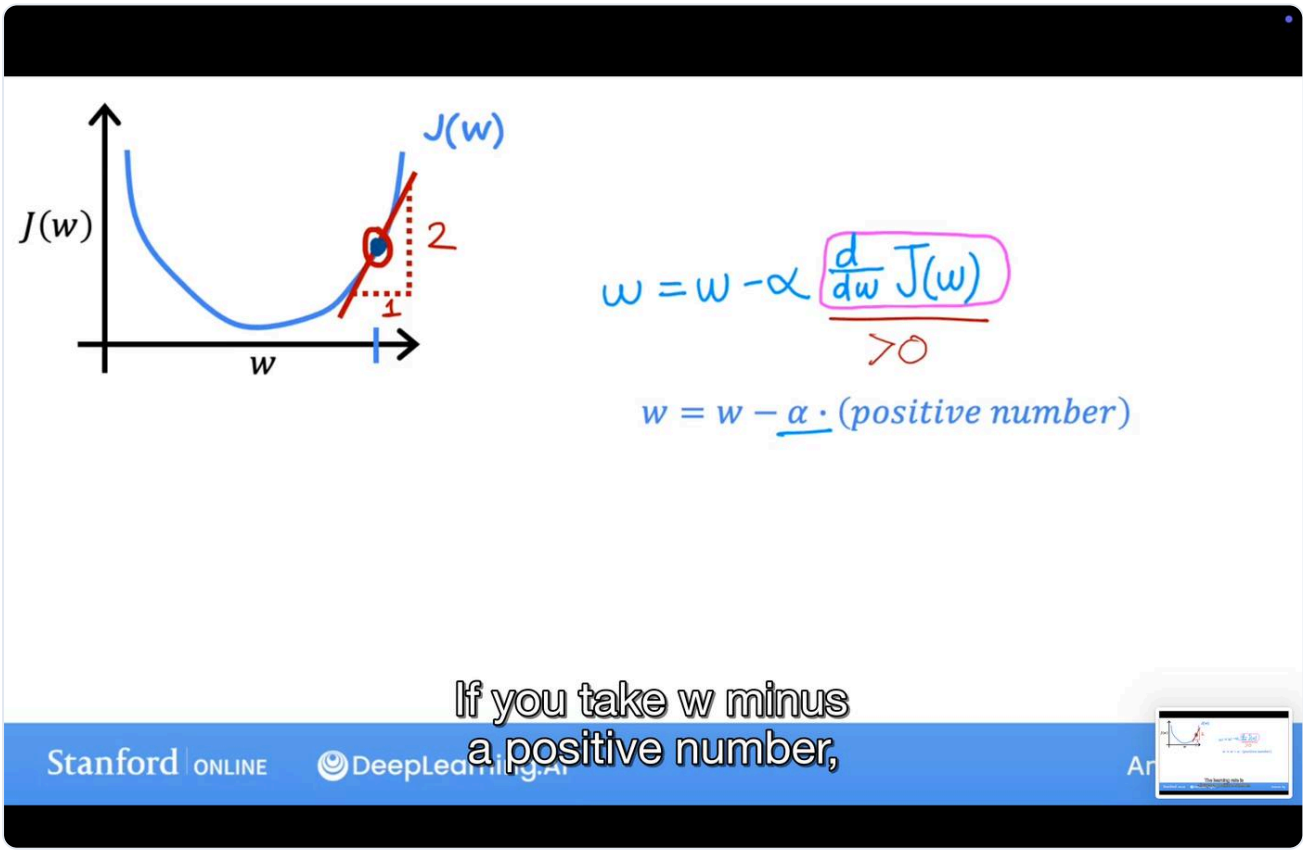


FIGURE 17.9

If you take w minus a positive number, you end up with a new value for w, that's smaller.

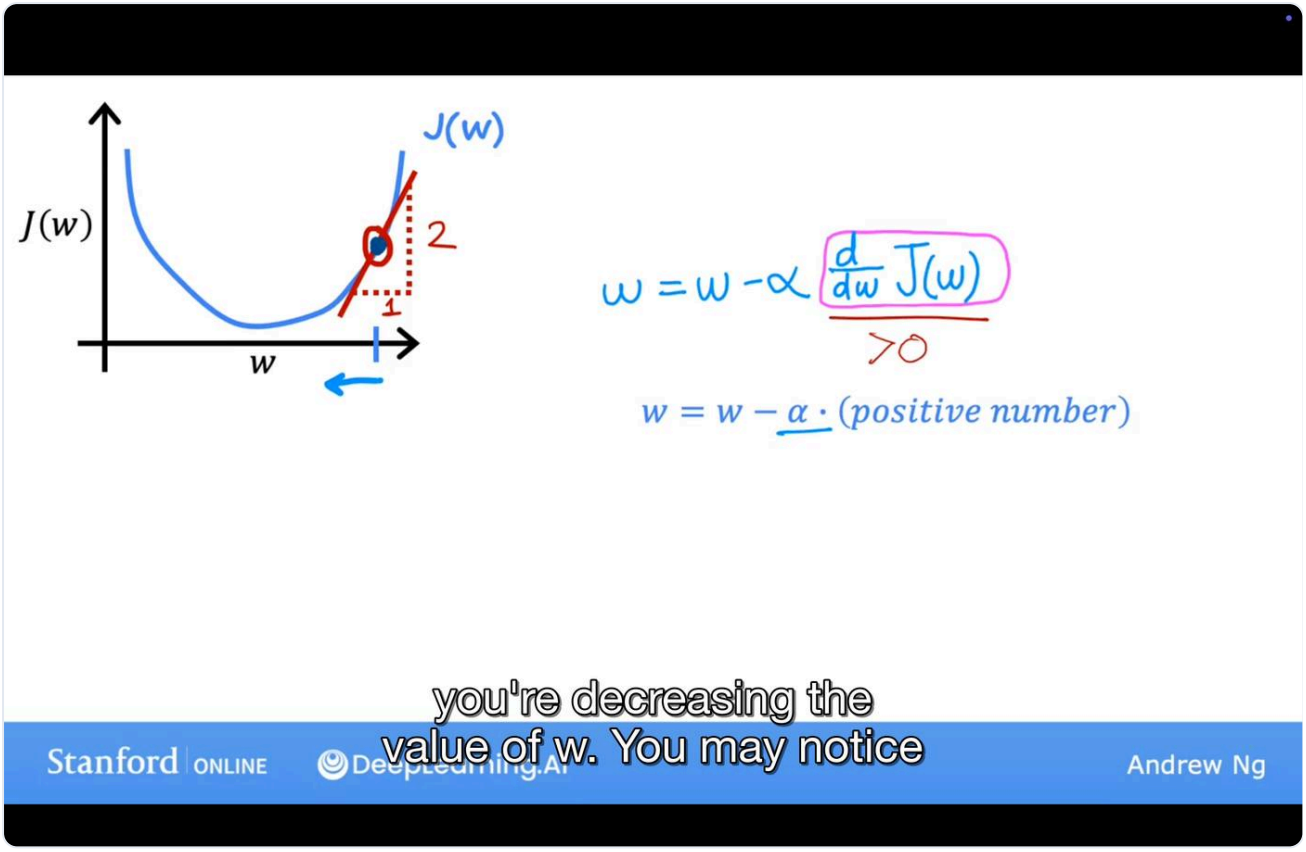


FIGURE 17.10

On the graph, you're moving to the left, you're decreasing the value of w. You may notice that this is the right thing to do if your goal is to decrease the cost J, because when we move towards the left on this curve, the cost j decreases, and you're getting closer to the minimum for J, which is over here.

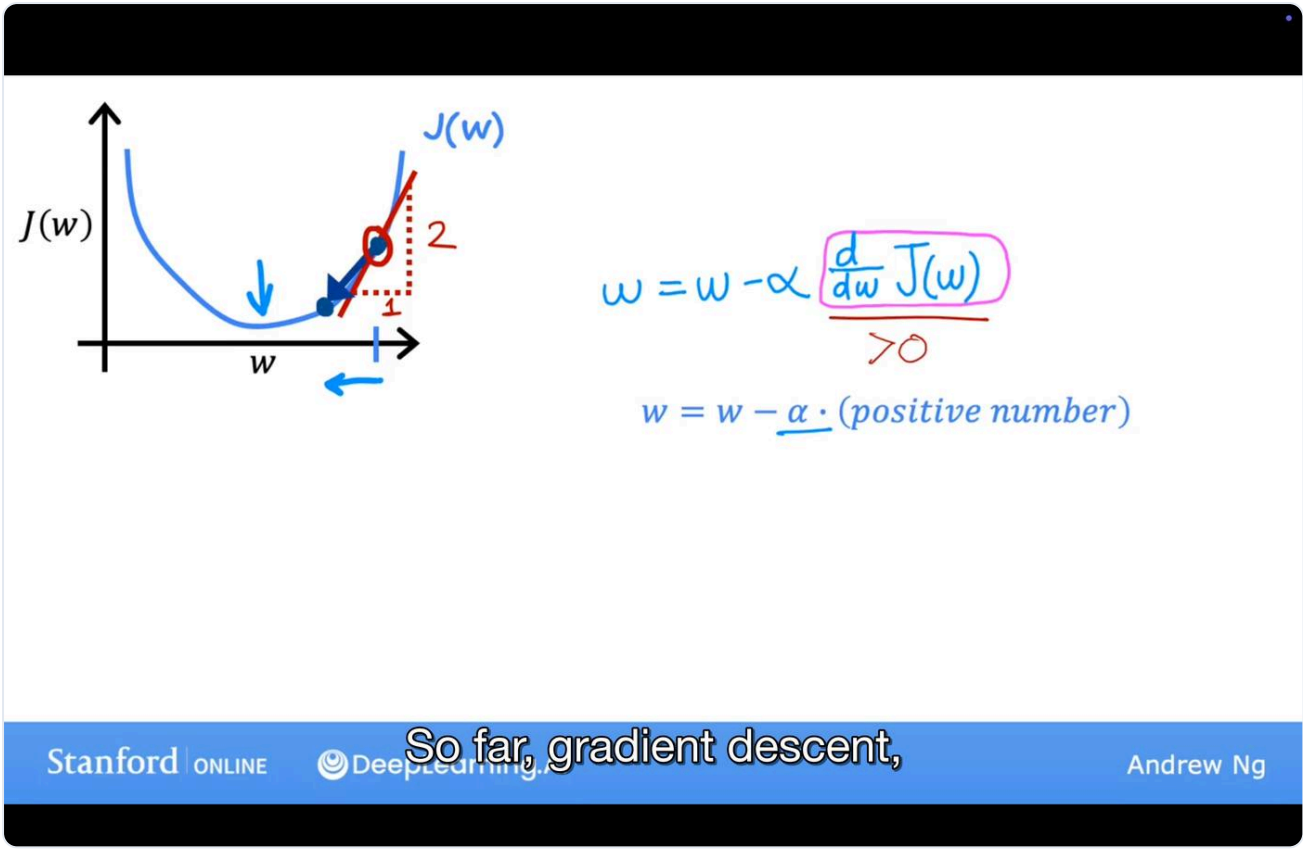


FIGURE 17.11

So far, gradient descent, seems to be doing the right thing. Now, let's look at another example. Let's take the same function j of w as above, and now let's say that you initialized gradient descent at a different location. Say by choosing a starting value for w that's over here on the left.

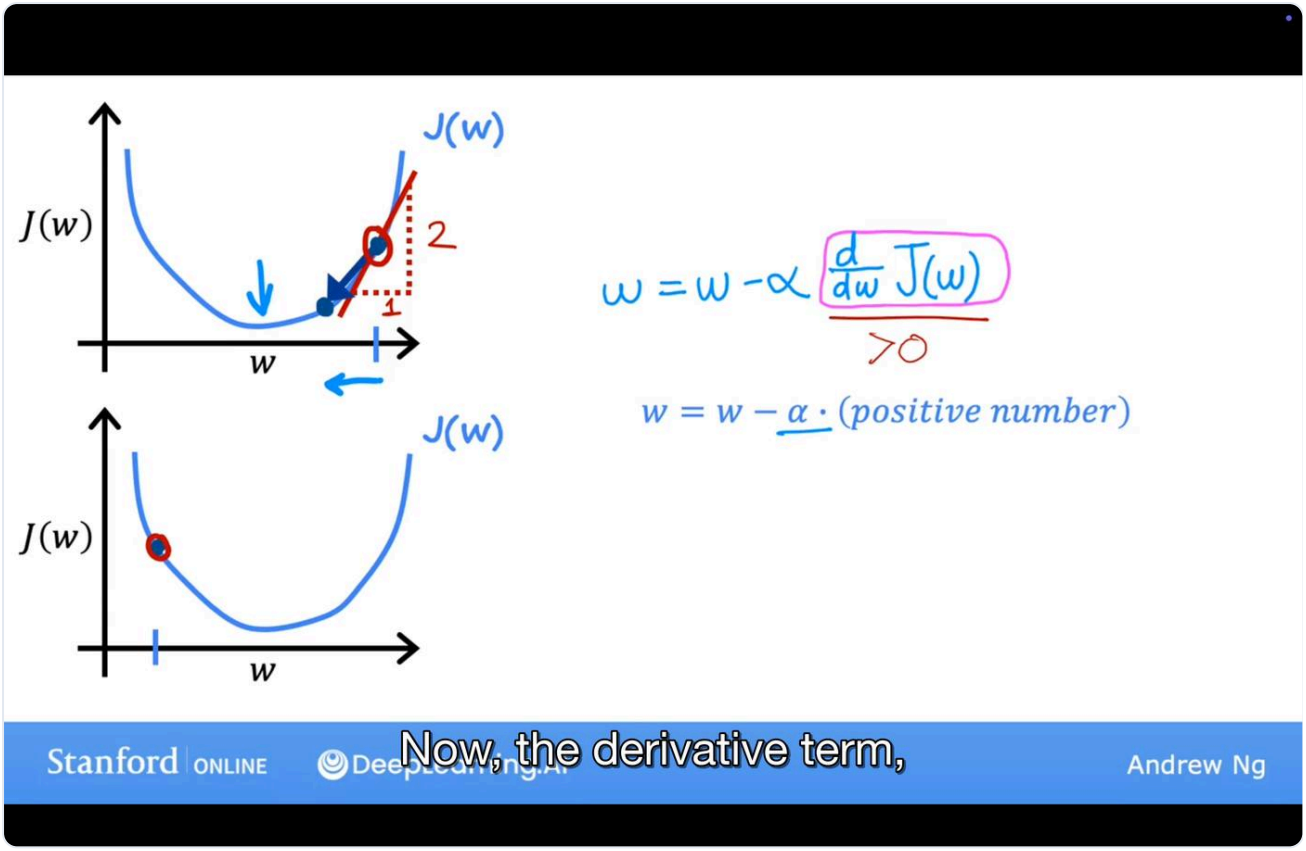


FIGURE 17.12

That's this point of the function j. Now, the derivative term, remember is d over dw of J of w, and when we look at the tangent line at this point over here, the slope of this line is a derivative of J at this point. But this tangent line is sloping down into the right.

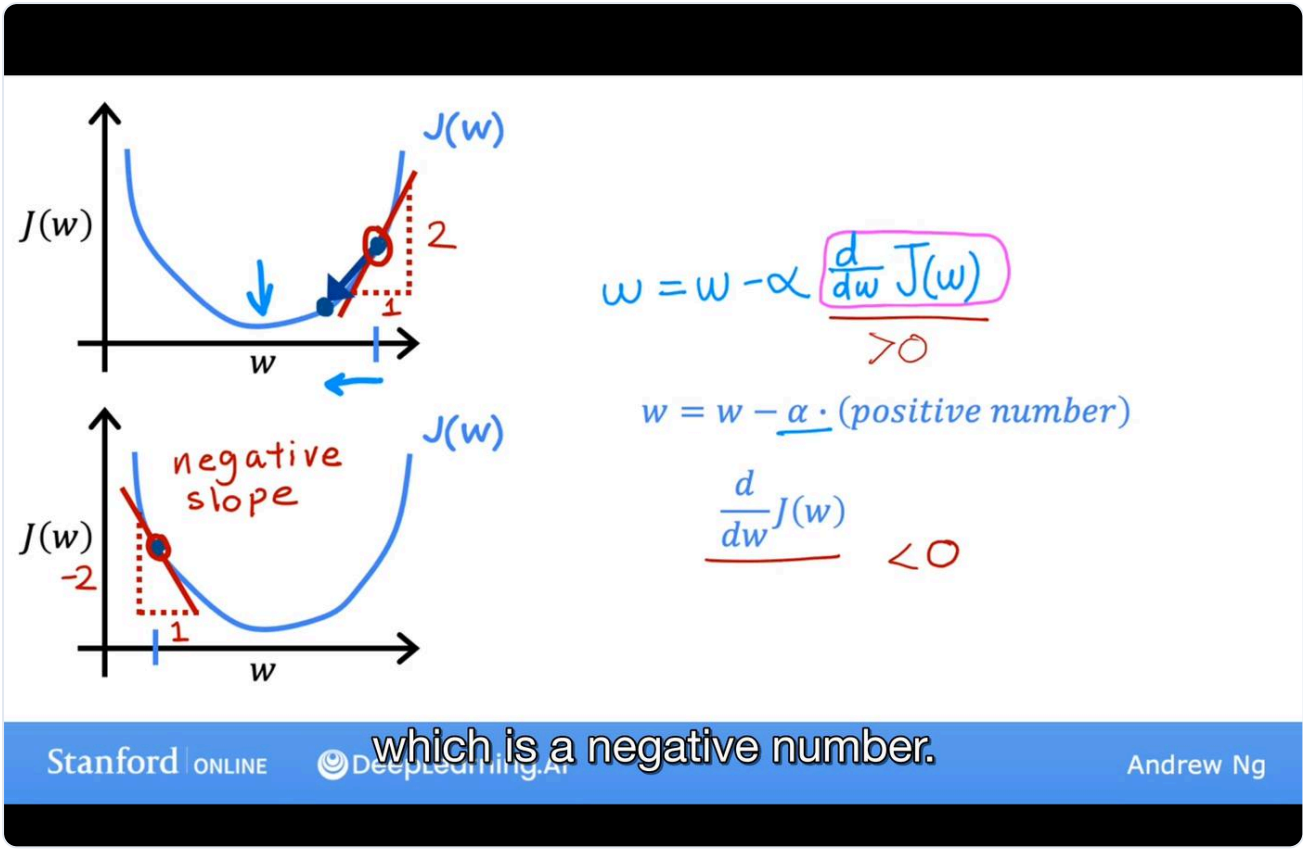


FIGURE 17.13

This lines sloping down into the right has a negative slope. In other words, the derivative of J at this point is a negative number. For instance, if you draw a triangle, then the height like this is negative 2 and the width is 1, the slope is negative 2 divided by 1, which is negative 2, which is a negative number. When you update w, you get w minus the learning rate times a negative number.

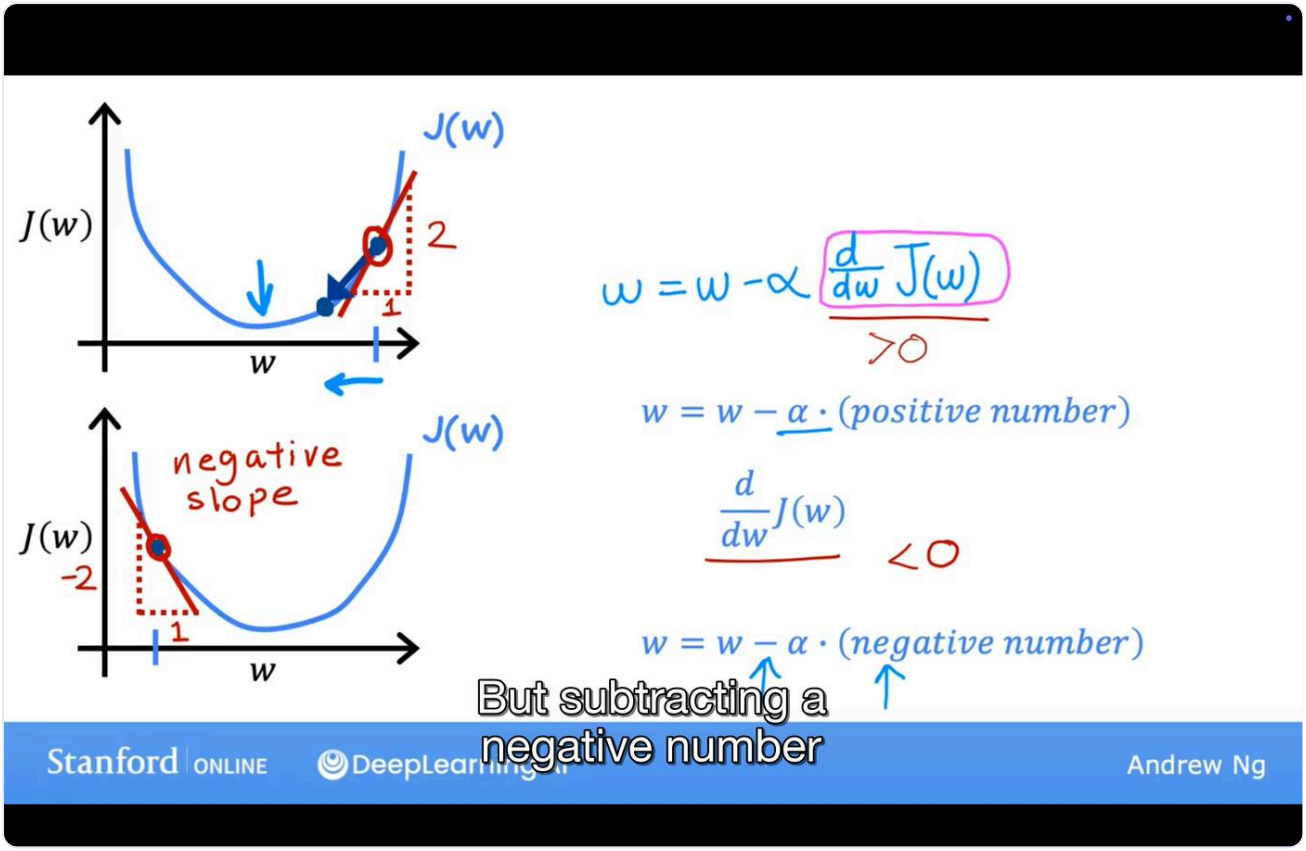


FIGURE 17.14

This means you subtract from w, a negative number. But subtracting a negative number means adding a positive number, and so you end up increasing w. Because subtracting a negative number is the same as adding a positive number to w.

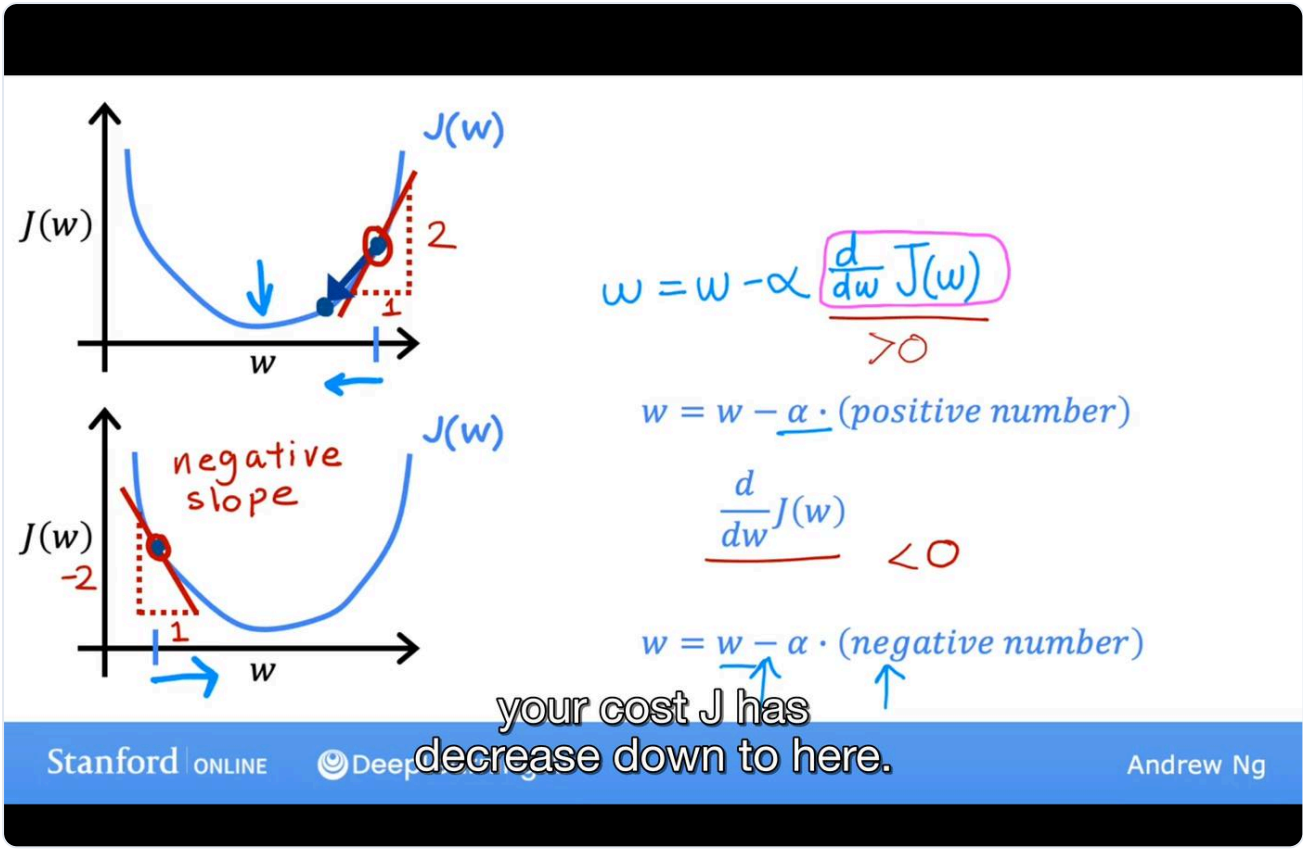


FIGURE 17.15

This step of gradient descent causes w to increase, which means you're moving to the right of the graph and your cost J has decrease down to here. Again, it looks like gradient descent is doing something reasonable, is getting you closer to the minimum. Hopefully, these last two examples show some of the intuition behind what a derivative term is doing and why this host gradient descent change w to get you closer to the minimum. I hope this video gave you some sense for why the derivative term in gradient descent makes sense. One other key quantity in the gradient descent algorithm is the learning rate Alpha. How do you choose Alpha? What happens if it's too small or what happens when it's too big?

In the next video, let's take a deeper look at the parameter Alpha to help build intuitions about what it does, as well as how to make a good choice for a good value of Alpha for your implementation of gradient descent.

LECTURE 18

Gradient Descent May Be Slow

The choice of the learning rate, alpha will have a huge impact on the efficiency of your implementation of gradient descent. And if alpha, the learning rate is chosen poorly rate of descent may not even work at all. In this video, let's take a deeper look at the learning rate. This will also help you choose better learning rates for your implementations of gradient descent. So here again, is the gradient descent rule. W is updated to be W minus the learning rate, alpha times the derivative term. To learn more about what the learning rate alpha is doing. Let's see what could happen if the learning rate alpha is either too small or if it is too large. For the case where the learning rate is too small.

Here's a graph where the horizontal axis is W and the vertical axis is the cost J. And here's the graph of the function J of W.

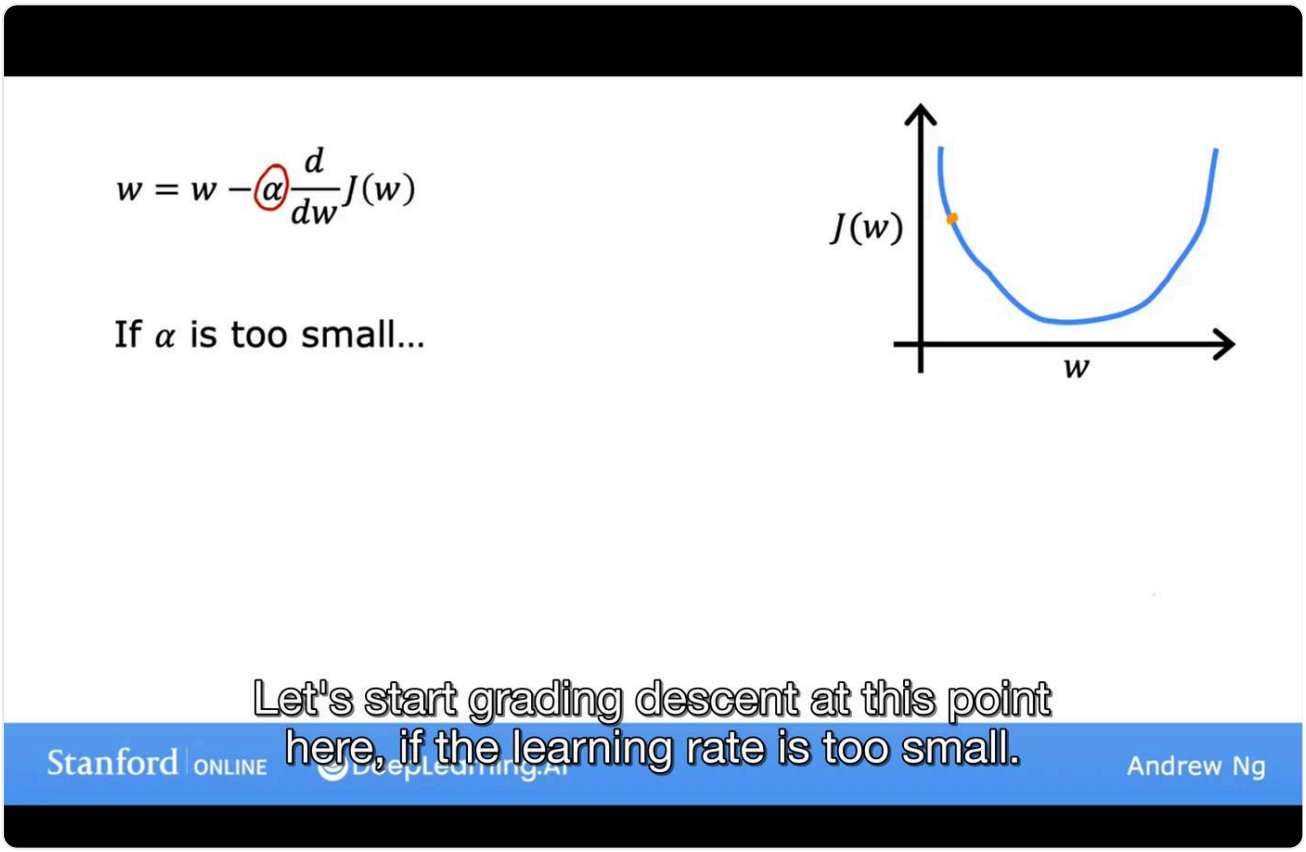


FIGURE 18.1

Let's start grading descent at this point here, if the learning rate is too small. Then what happens is that you multiply your derivative term by some really, really small number. So you're going to be multiplying by number alpha. That's really small, like 0.0000001. And so you end up taking a very small baby step like that.

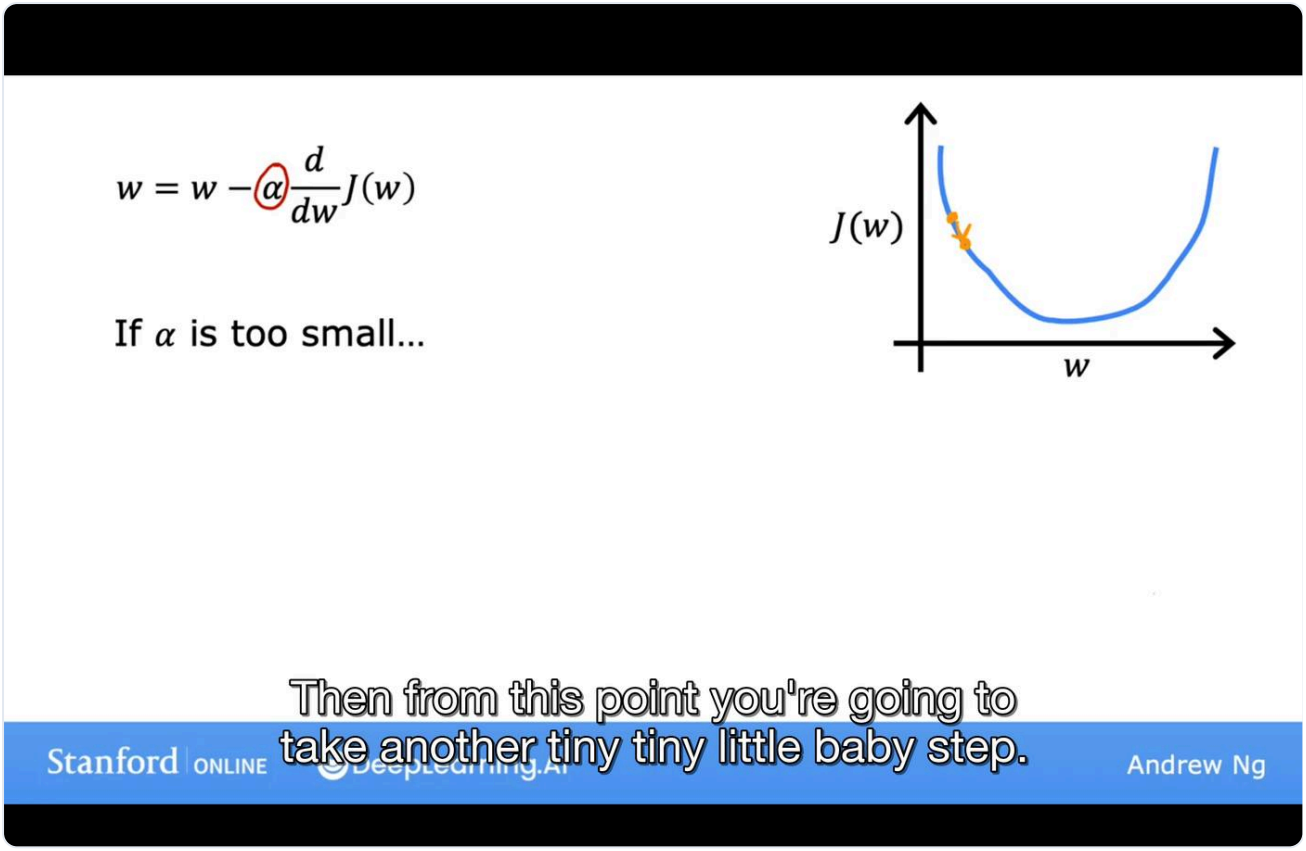


FIGURE 18.2

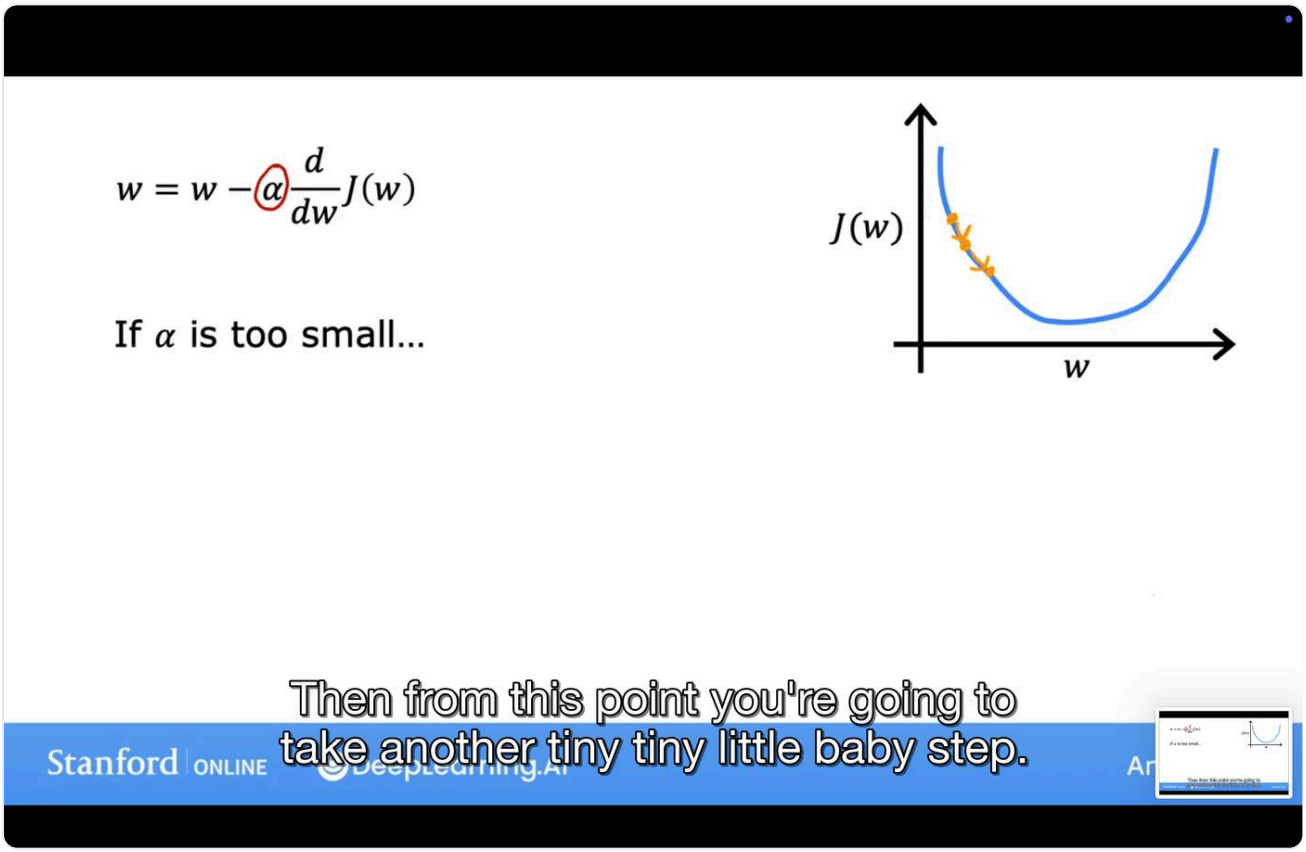


FIGURE 18.3

Then from this point you're going to take another tiny tiny little baby step.

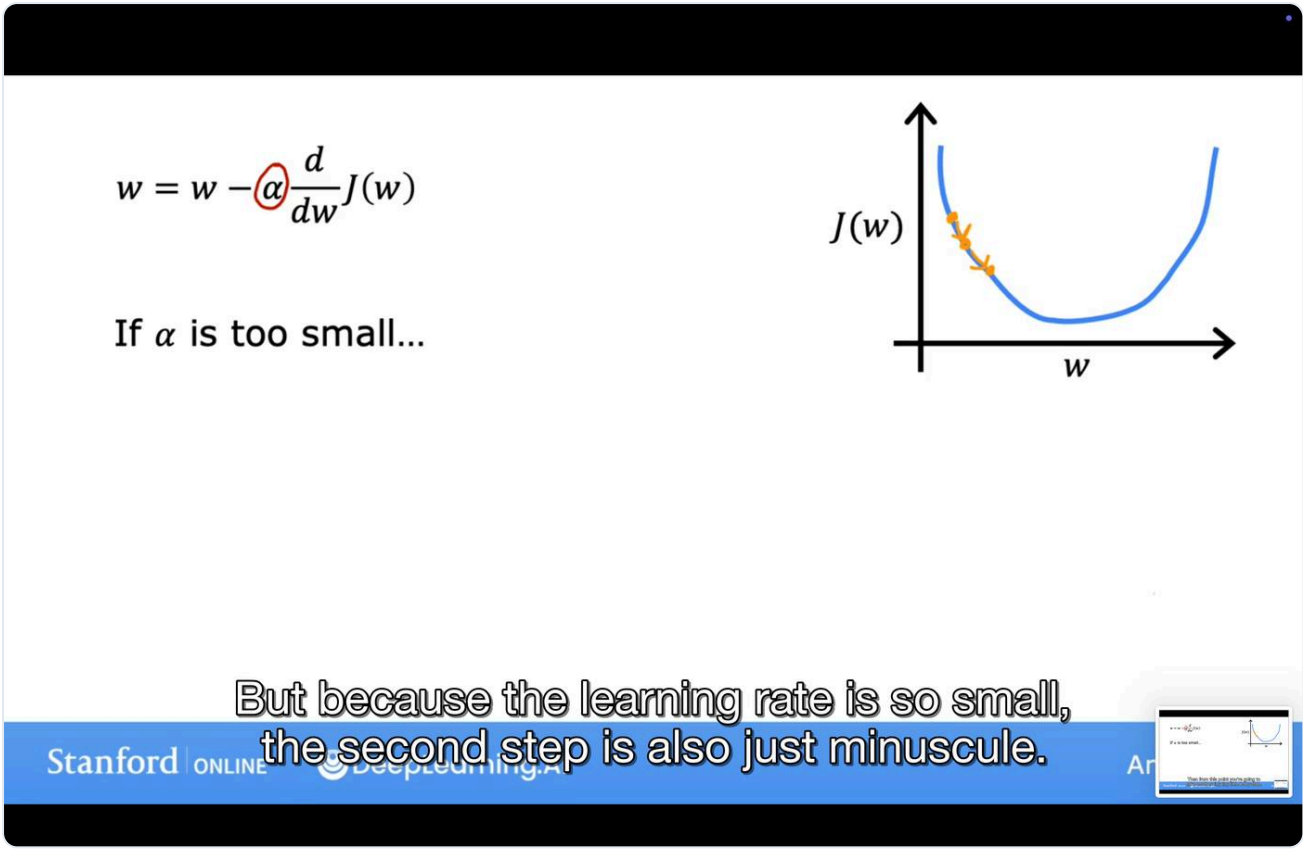


FIGURE 18.4 | If a Is Too Small

But because the learning rate is so small, the second step is also just minuscule. The outcome of this process is that you do end up decreasing the cost J but incredibly slowly.

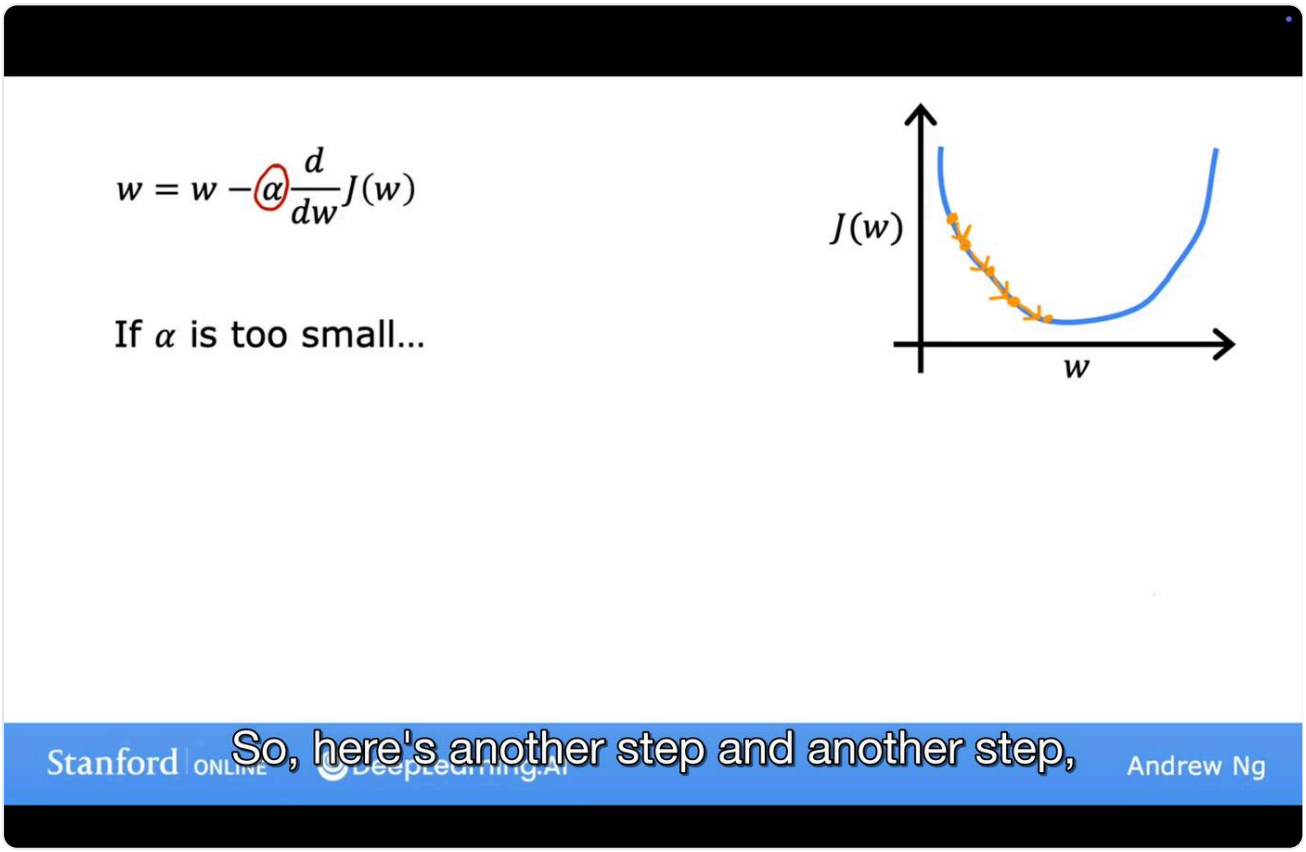


FIGURE 18.5 | If a Is Too Small

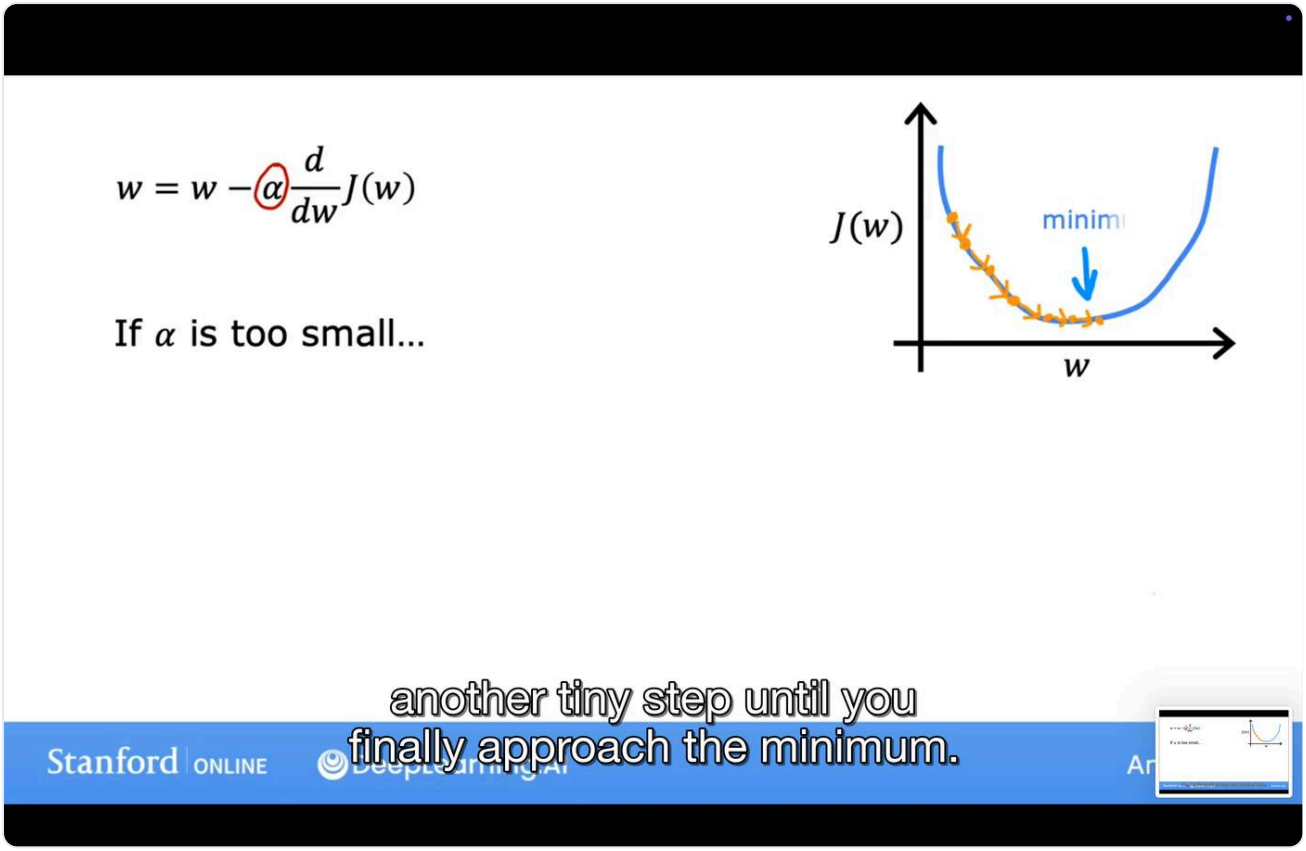


FIGURE 18.6 | If a Is Too Small

So, here's another step and another step, another tiny step until you finally approach the minimum. But as you may notice you're going to need a lot of steps to get to the minimum. So to summarize if the learning rate is too small, then gradient descents will work, but it will be slow.

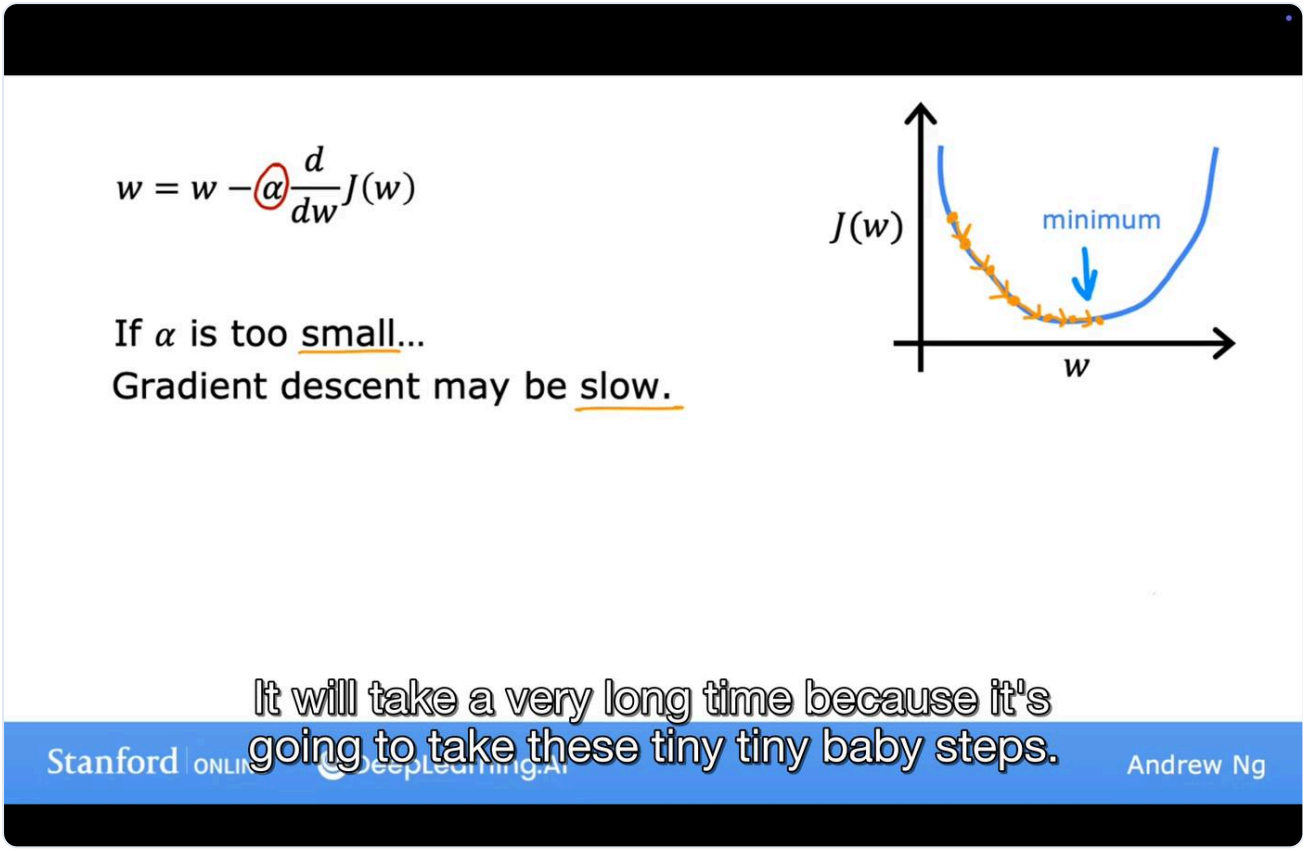


FIGURE 18.7

It will take a very long time because it's going to take these tiny tiny baby steps. And it's going to need a lot of steps before it gets anywhere close to the minimum. Now, let's look at a different case. What happens if the learning rate is too large? Here's another graph of the cost function. And let's say we start grating descent with W at this value here. So it's actually already pretty close to the minimum.

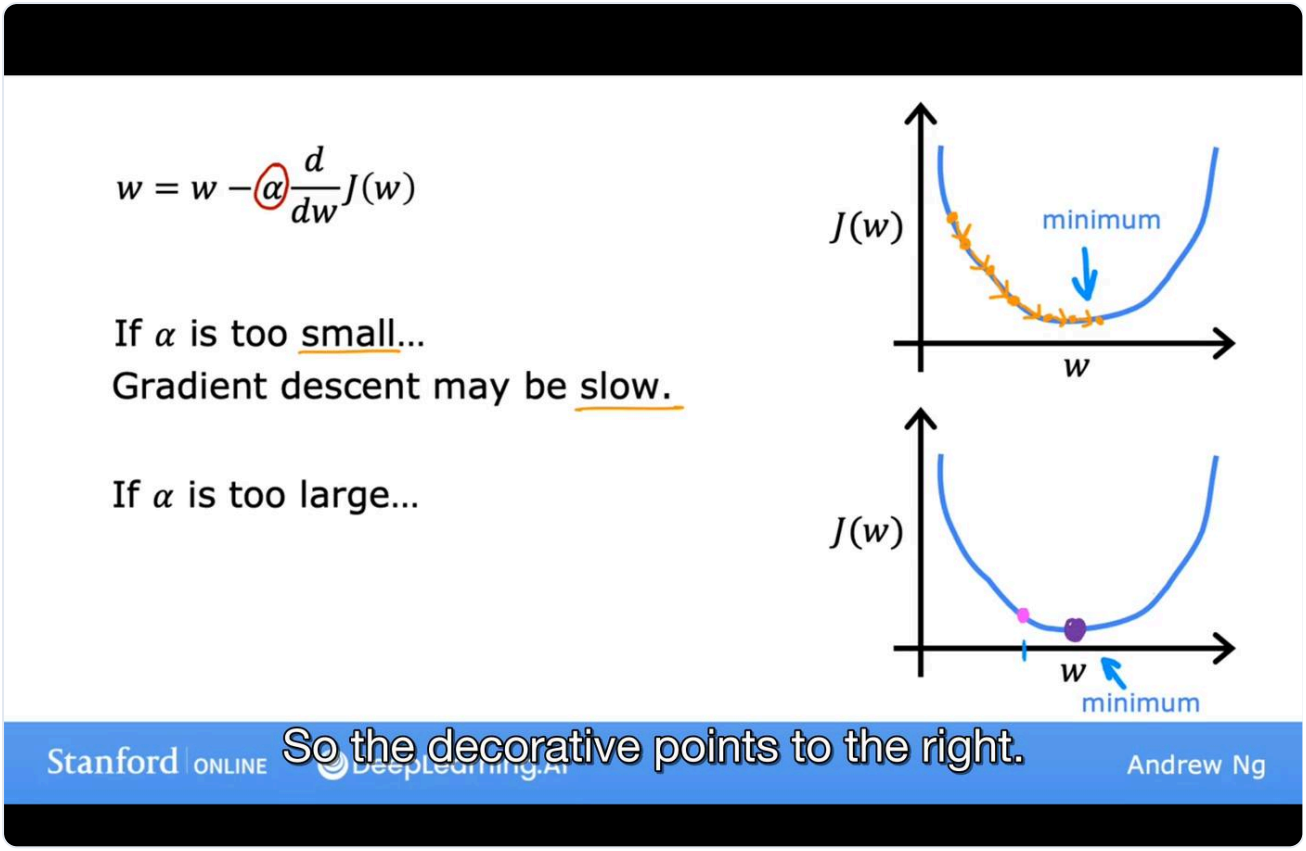


FIGURE 18.8

So the decorative points to the right.

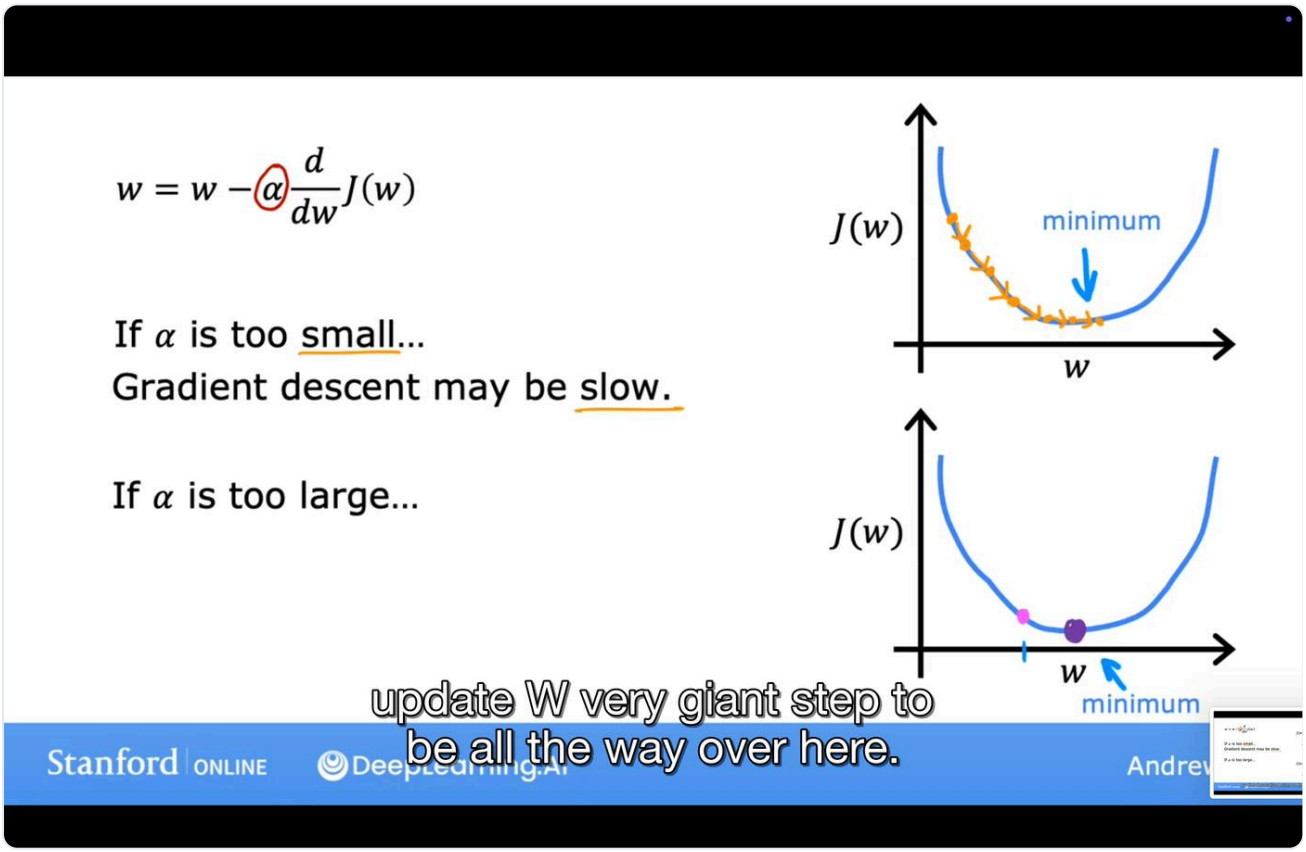


FIGURE 18.9 | If a Is Too Small

But if the learning rate is too large then you update W very giant step to be all the way over here. And that's this point here on the function J.

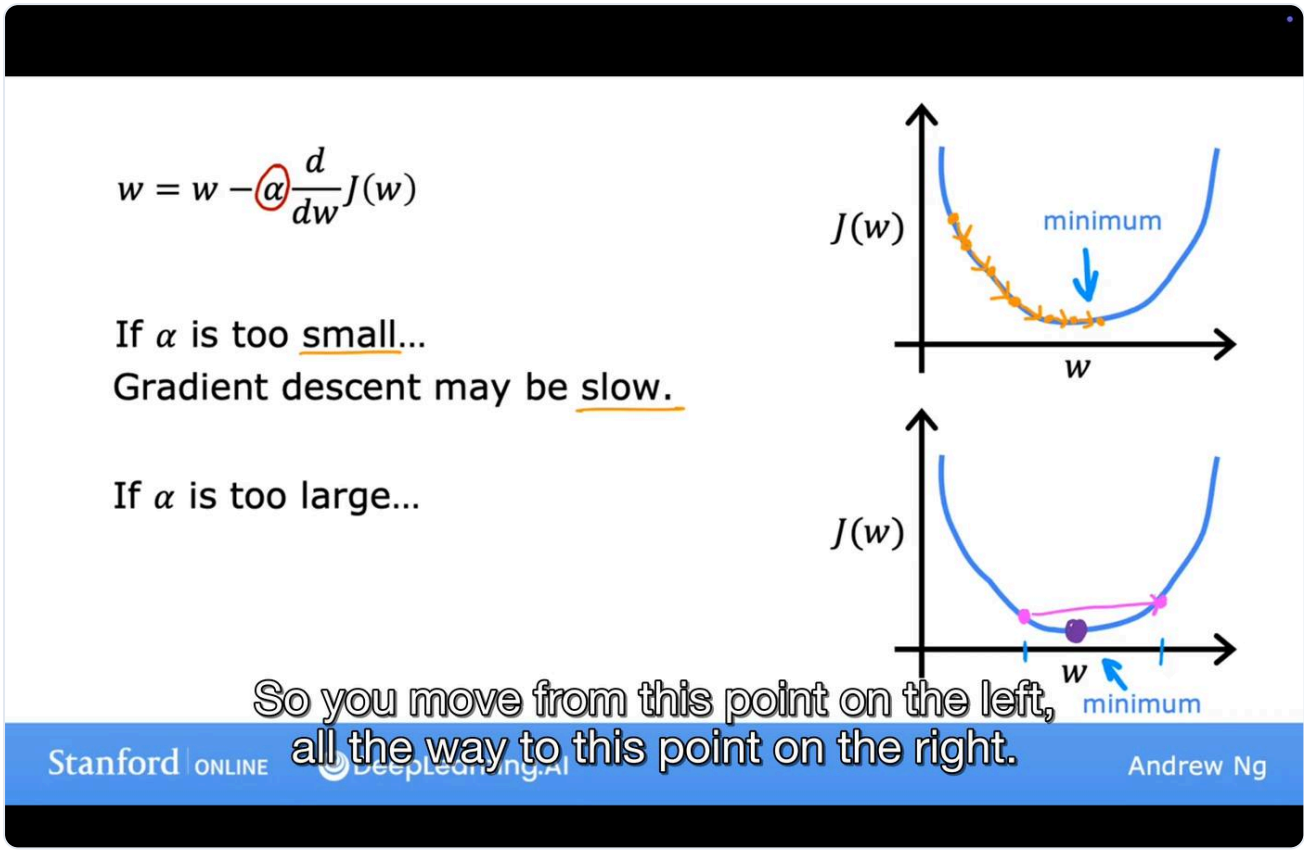


FIGURE 18.10 | Gradient Descent May Be Slow

So you move from this point on the left, all the way to this point on the right. And now the cost has actually gotten worse. It has increased because it started out at this value here and after one step, it actually increased to this value here. Now the derivative at this new point says to decrease W but when the learning rate is too big. Then you may take a huge step going from here all the way out here.

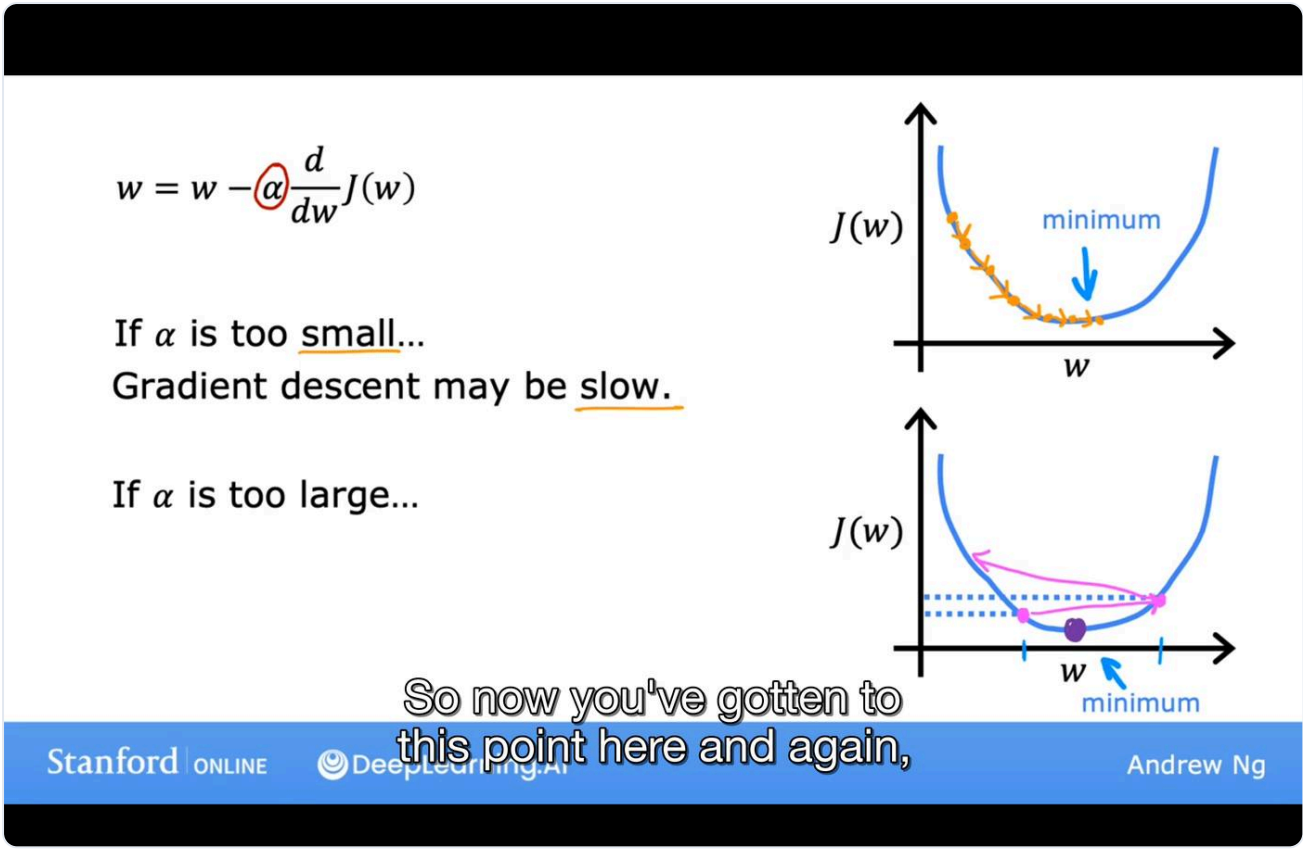


FIGURE 18.11 | If a Is Too Small

So now you've gotten to this point here and again, if the learning rate is too big.

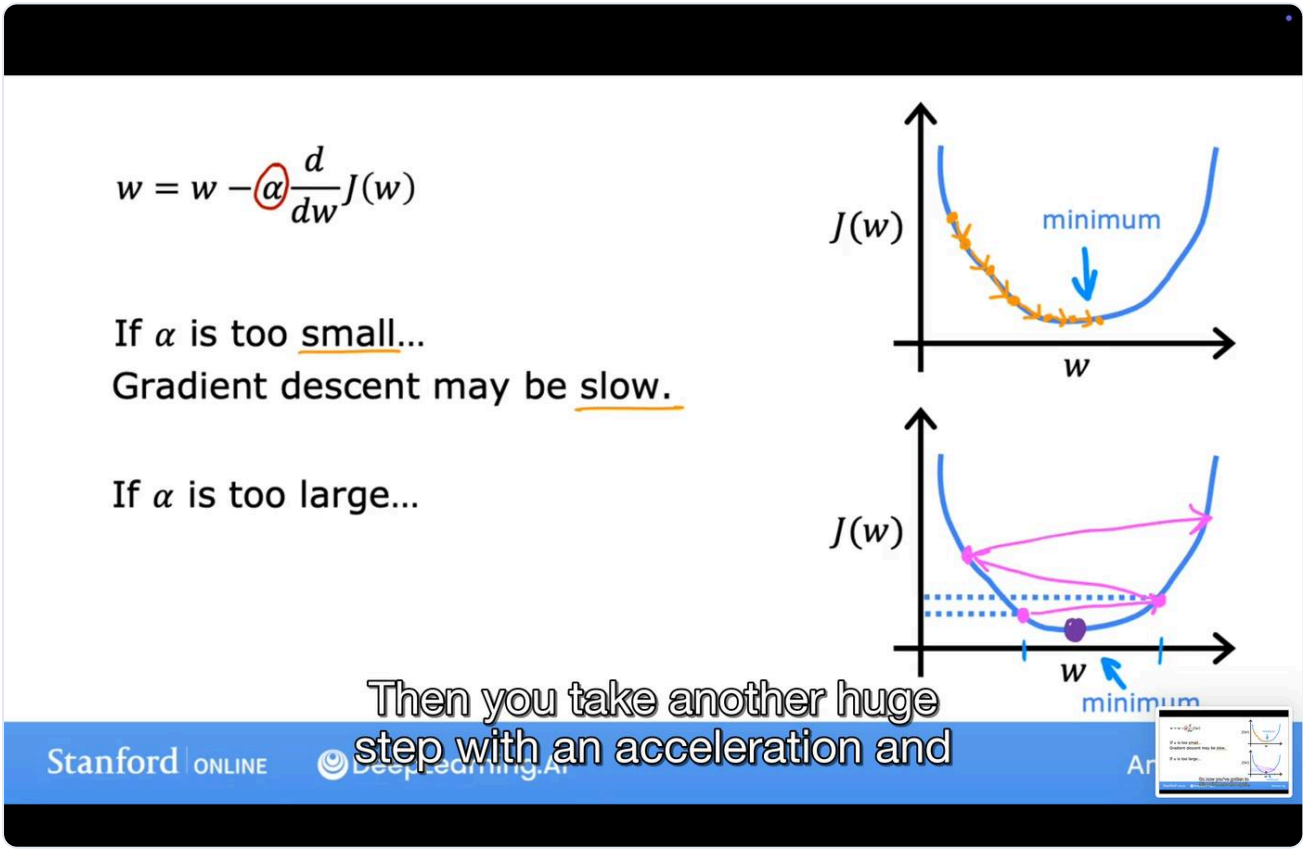


FIGURE 18.12

Then you take another huge step with an acceleration and way overshoot the minimum again. So now you're at this point on the right and one more time you do another update.

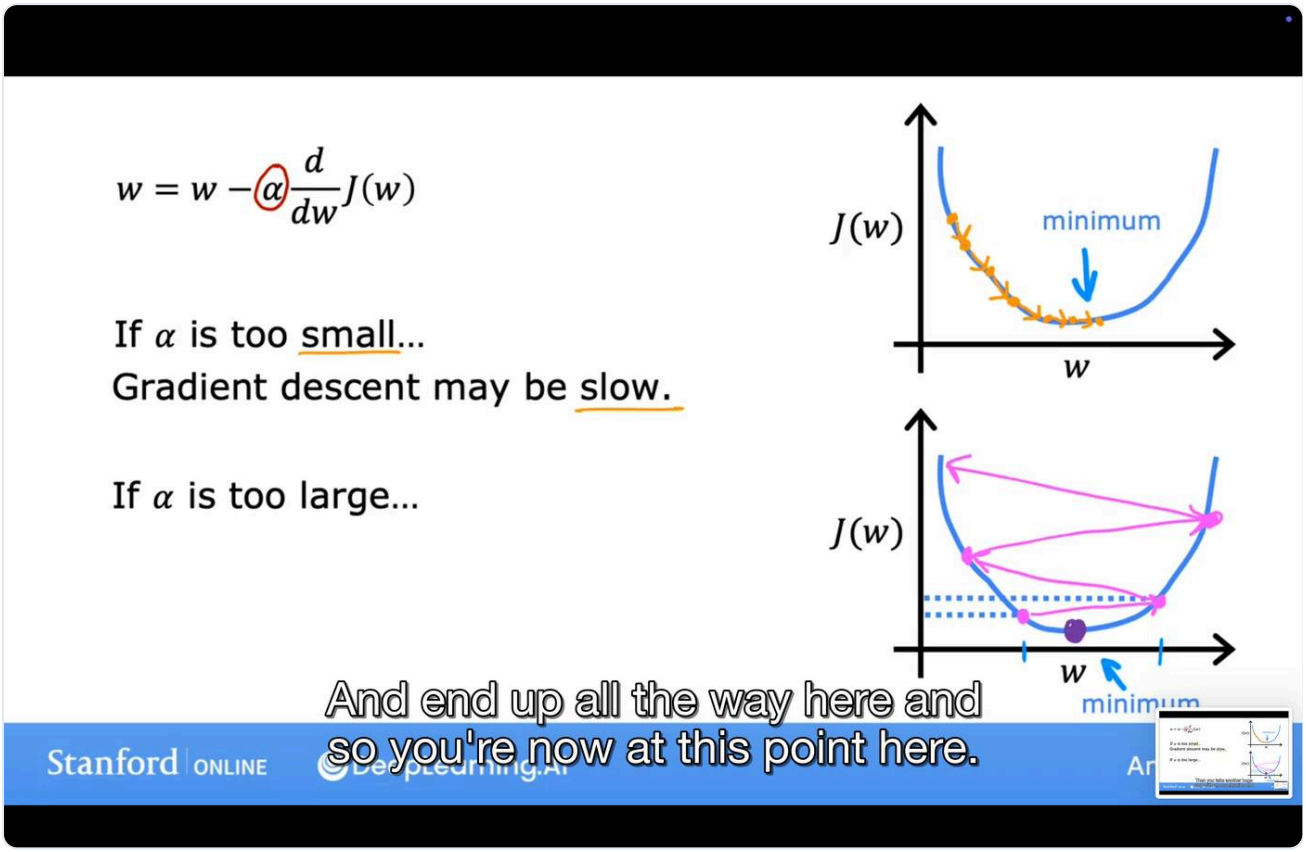


FIGURE 18.13

And end up all the way here and so you're now at this point here.

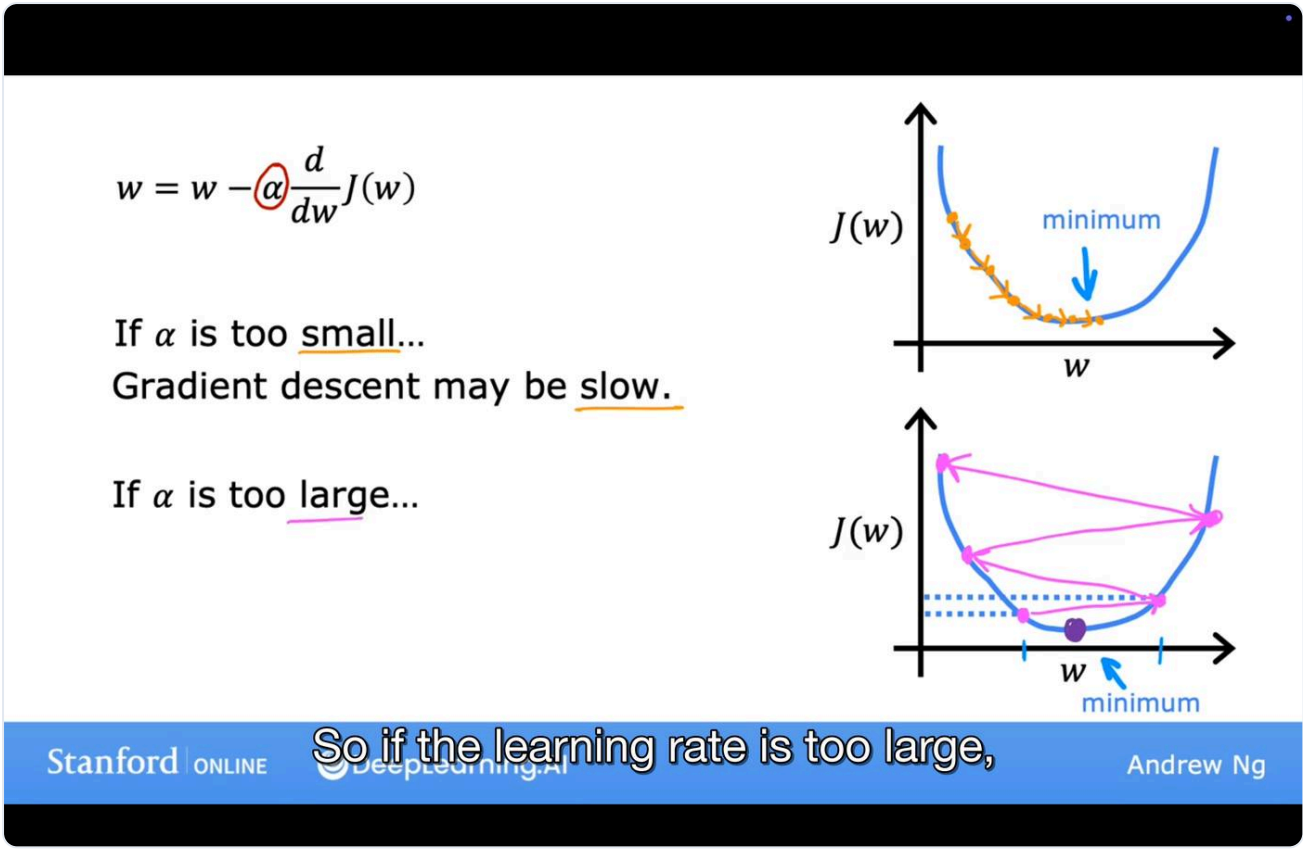


FIGURE 18.14 | If a Is Too Small

So as you may notice you're actually getting further and further away from the minimum.

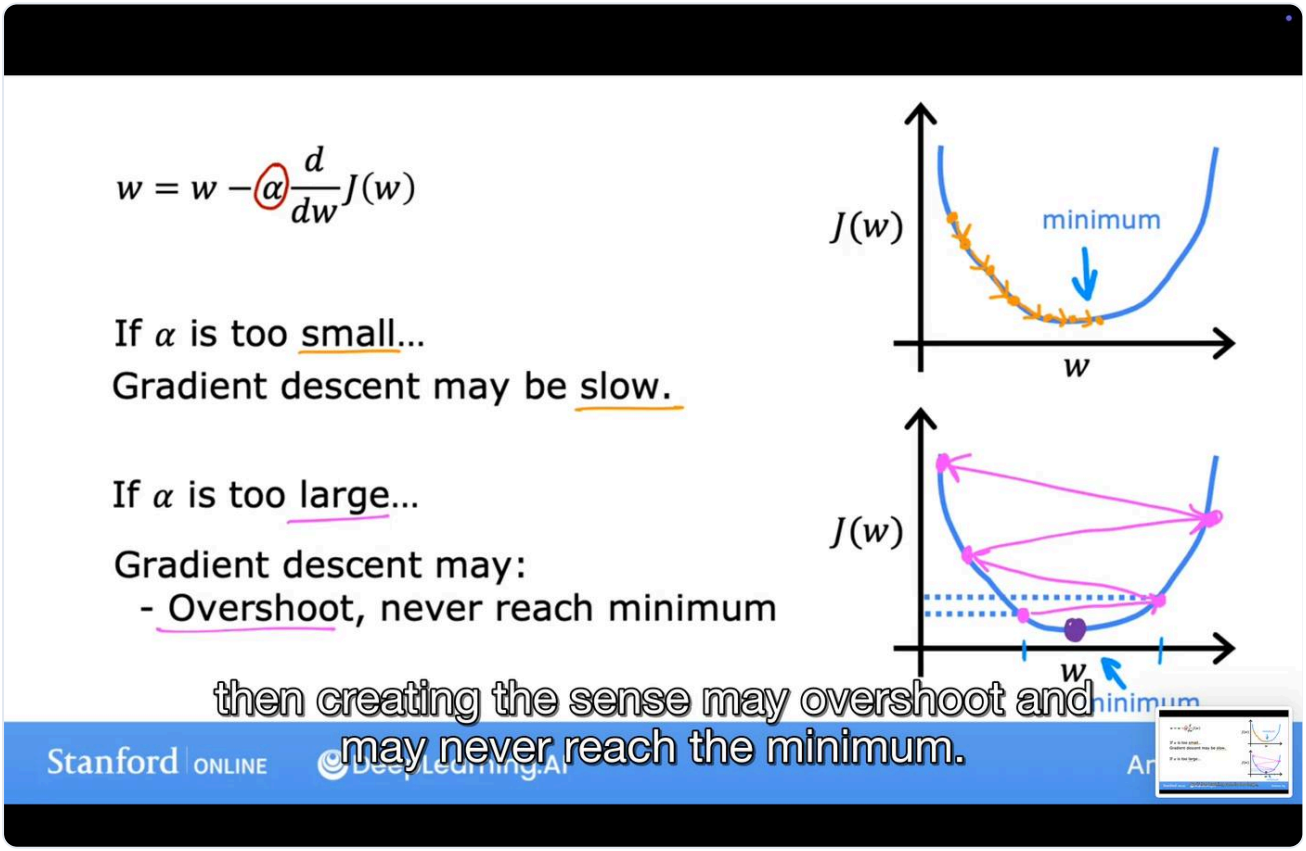


FIGURE 18.15 | If a Is Too Small

So if the learning rate is too large, then creating the sense may overshoot and may never reach the minimum.

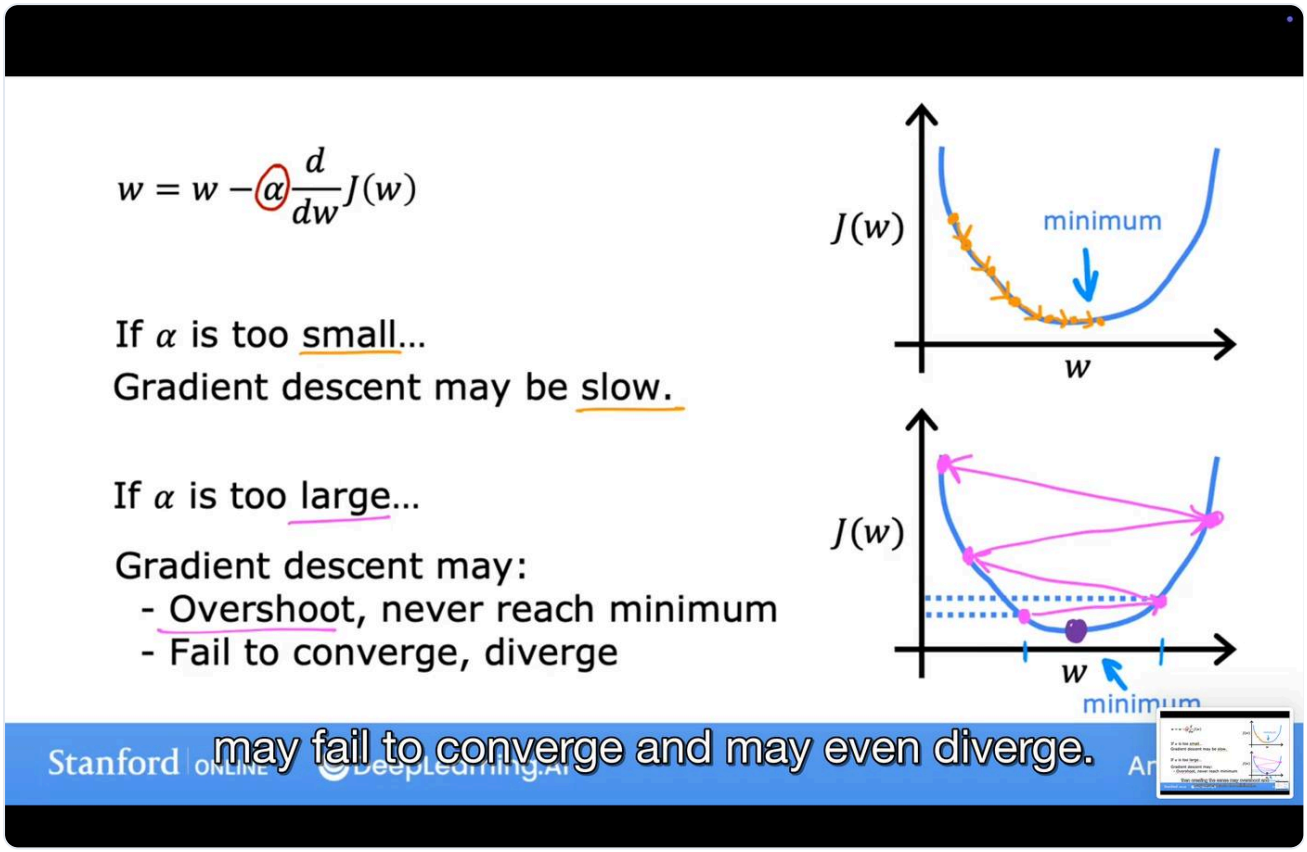


FIGURE 18.16 | If a Is Too Small

And another way to say that is that great intersect may fail to converge and may even diverge. So, here's another question, you may be wondering one of your parameter W is already at this point here.

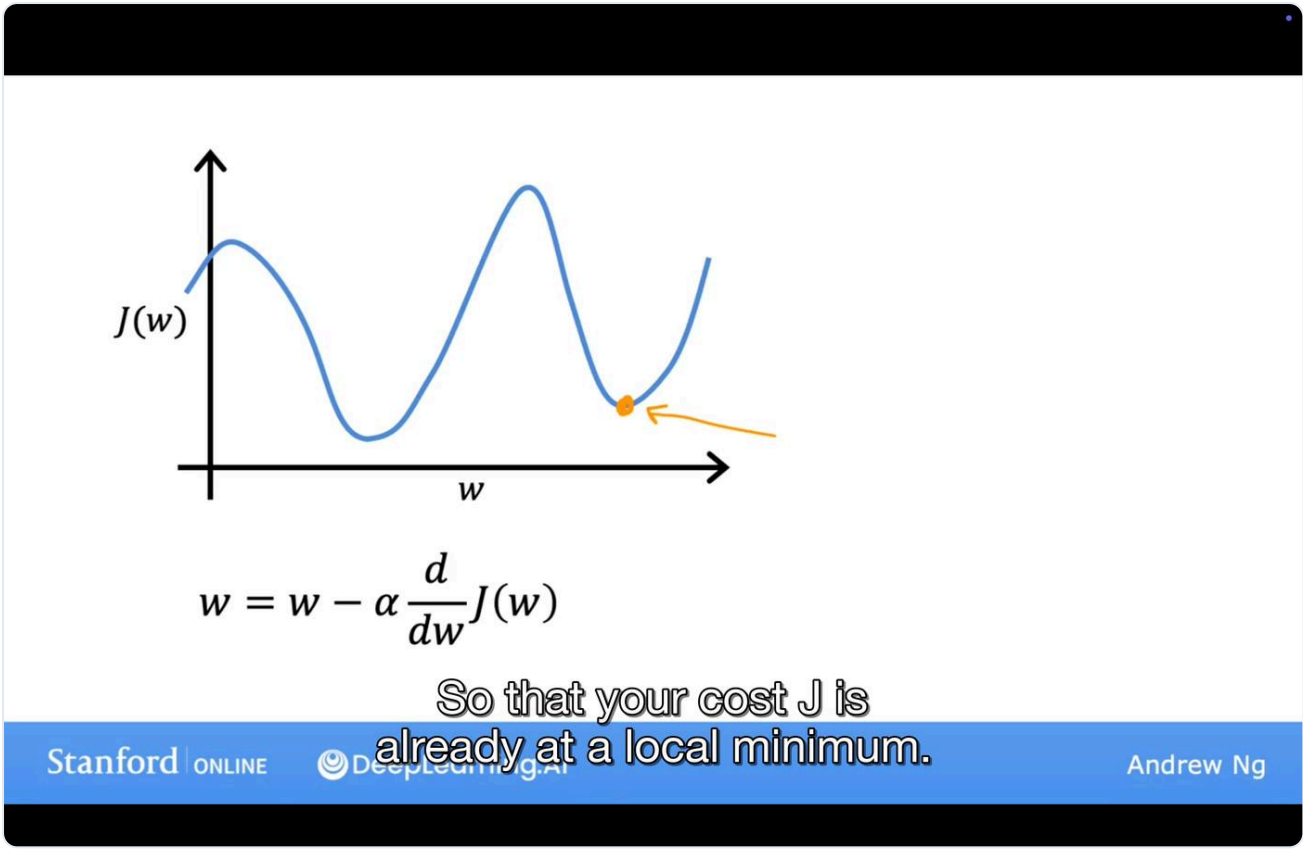


FIGURE 18.17

So that your cost J is already at a local minimum. What do you think? One step of gradient descent will do if you've already reached a minimum? So this is a tricky one. When I was first learning this stuff, it actually took me a long time to figure it out. But let's work through this together.

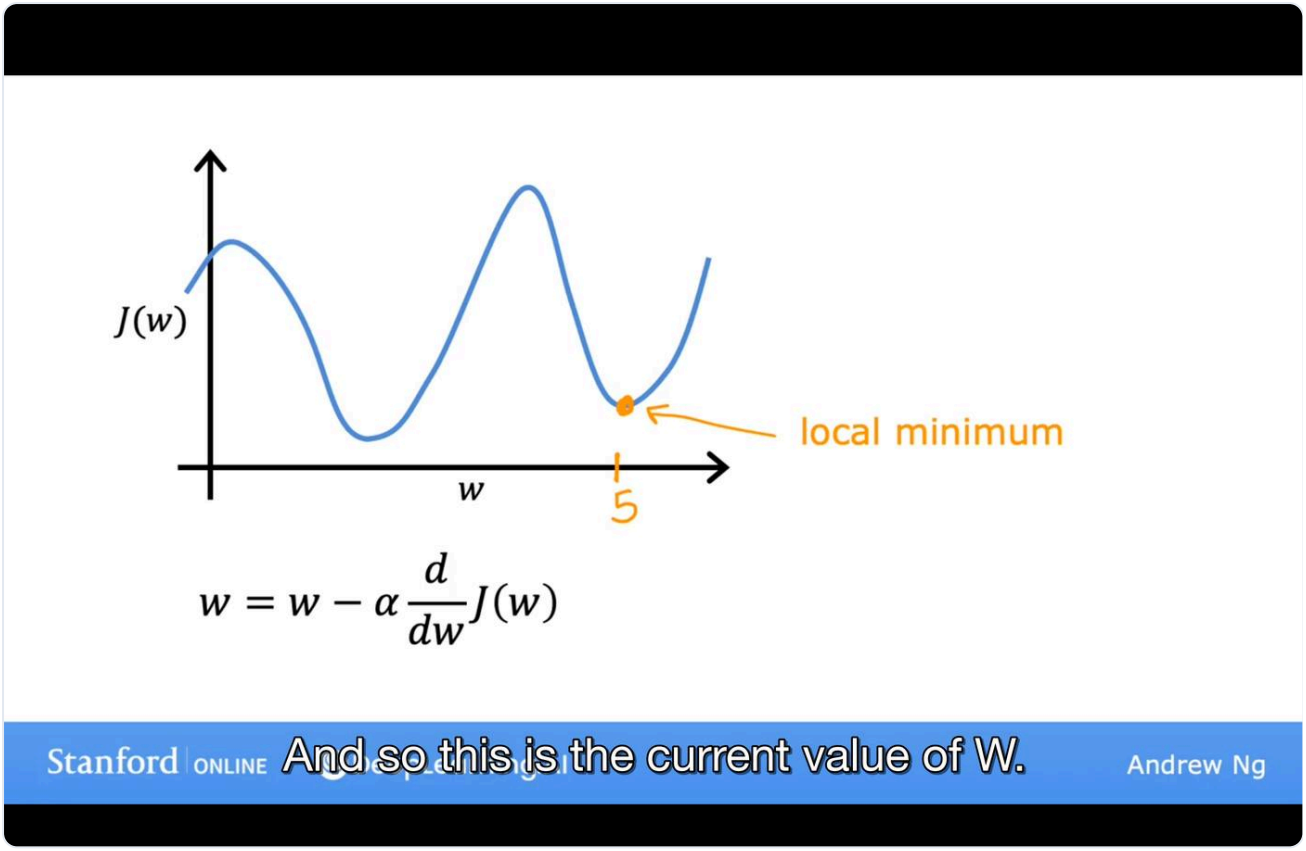


FIGURE 18.18

Let's suppose you have some cost function J. And the one you see here isn't a square error cost function and this cost function has two local minima corresponding to the two valleys that you see here. Now let's suppose that after some number of steps of gradient descent, your parameter W is over here, say equal to five. And so this is the current value of W. This means that you're at this point on the cost function J. And that happens to be a local minimum, turns out if you draw attention to the function at this point.

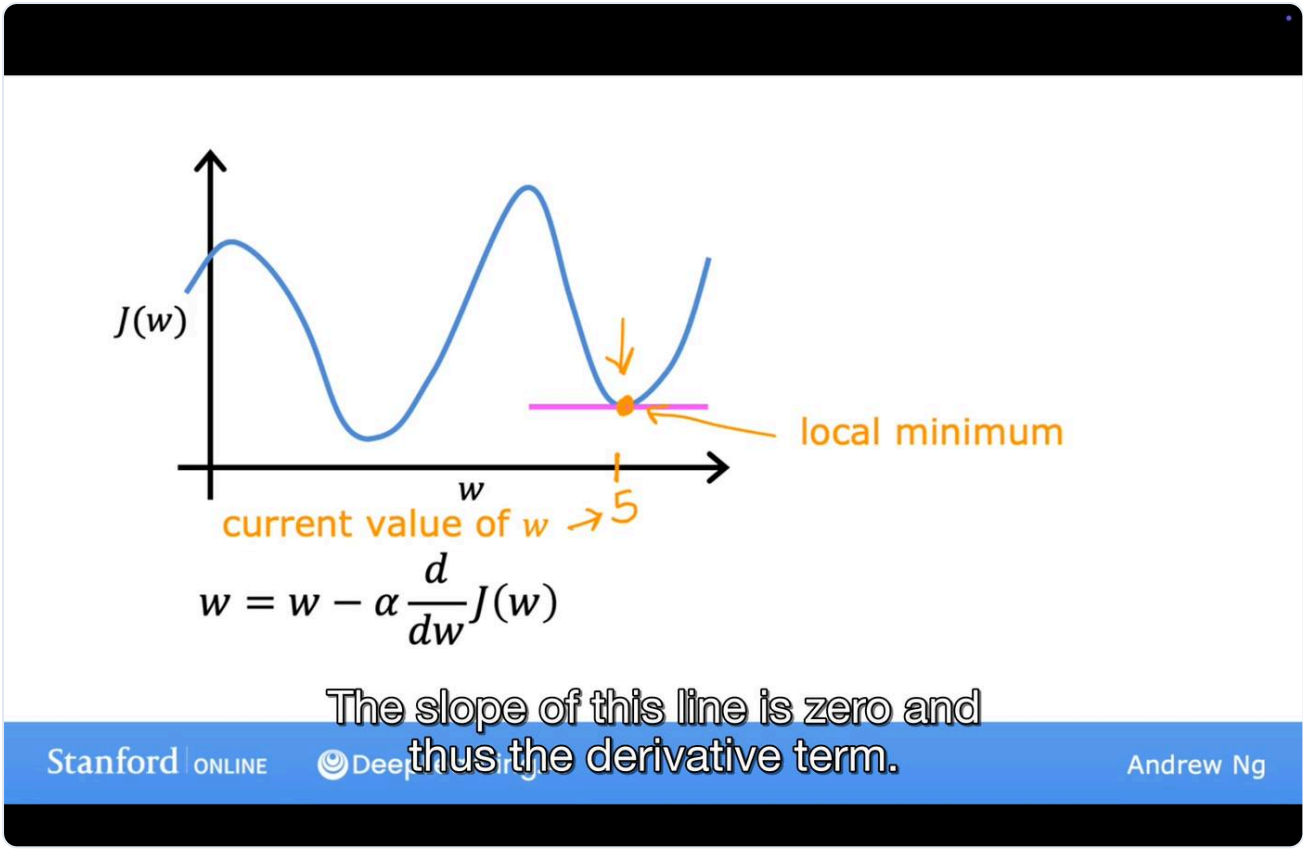


FIGURE 18.19

The slope of this line is zero and thus the derivative term.

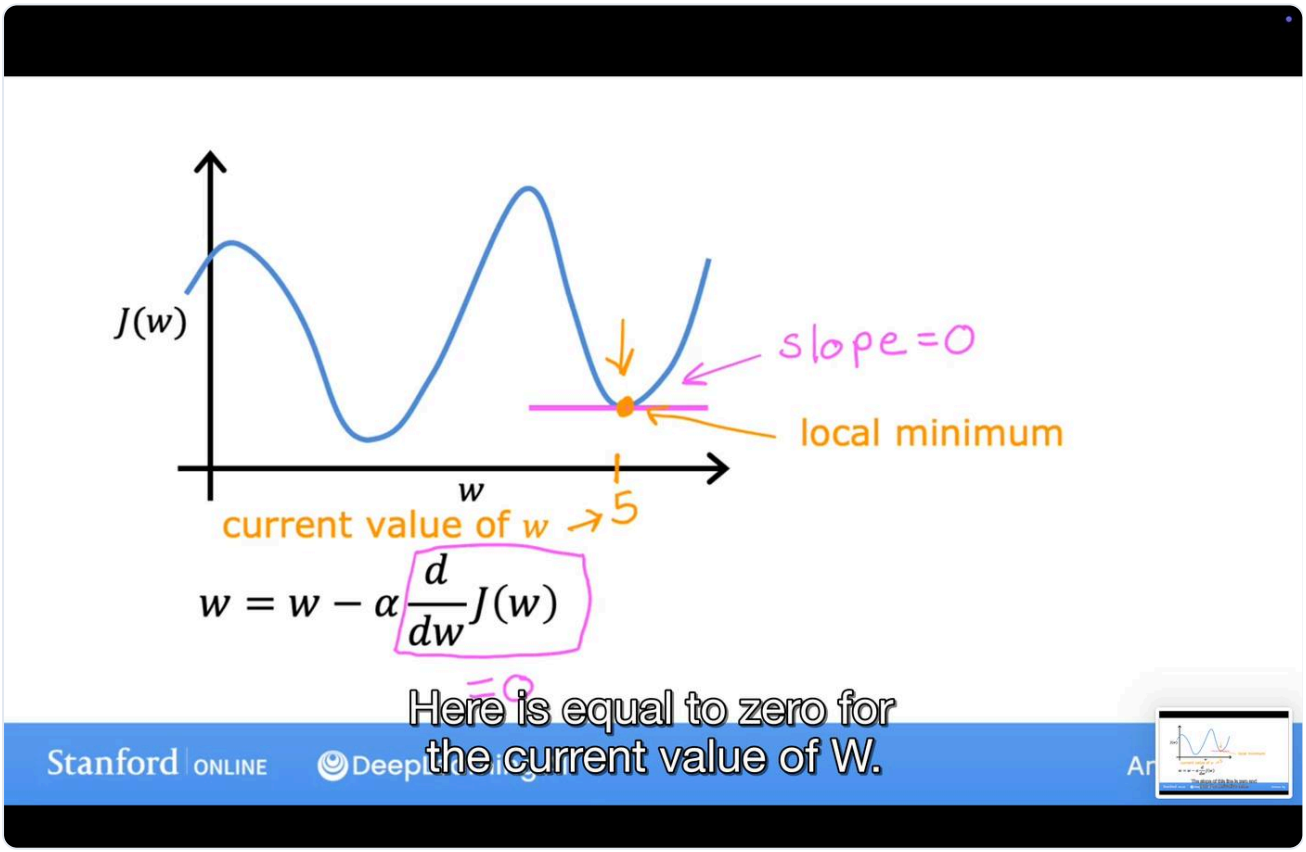


FIGURE 18.20

Here is equal to zero for the current value of W.

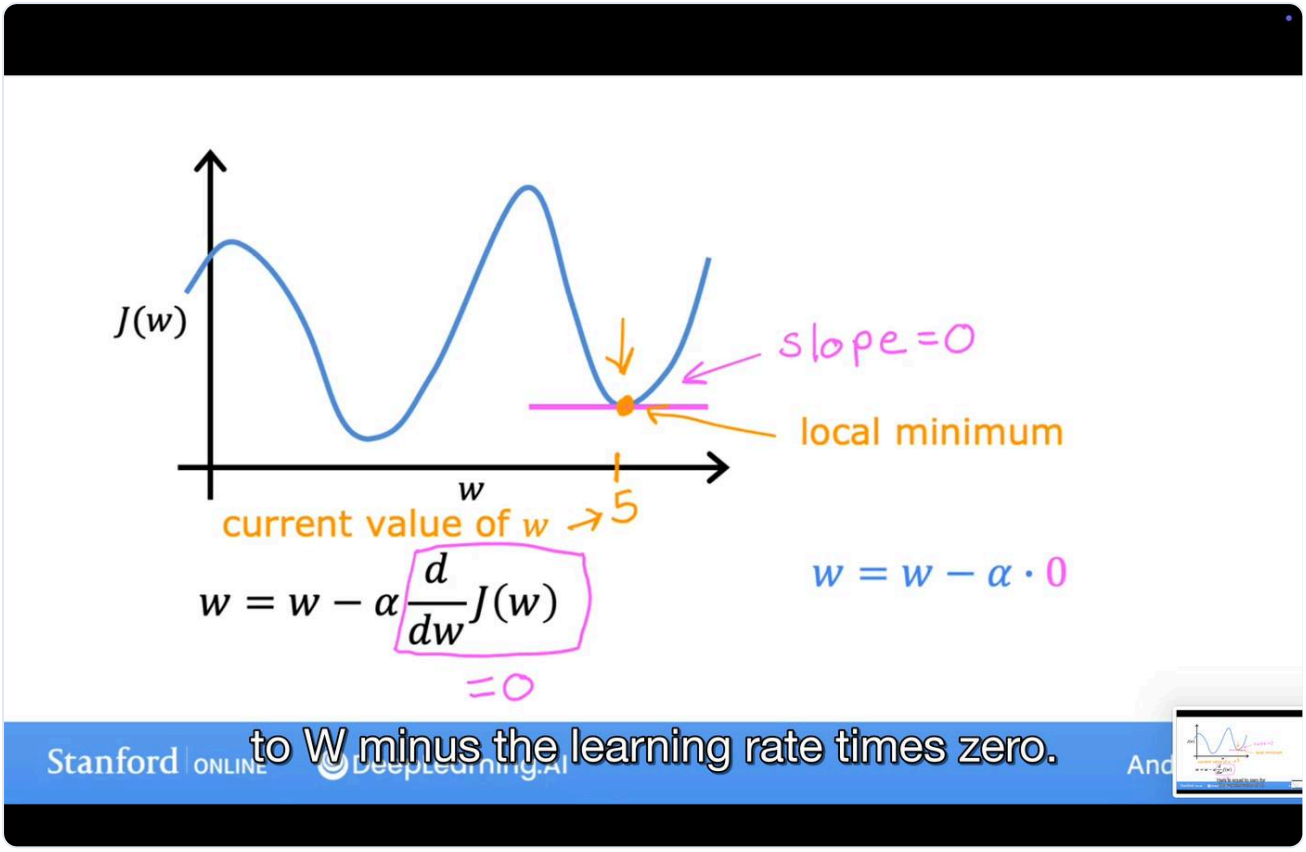


FIGURE 18.21

And so you're grading descent update becomes W is updated to W minus the learning rate times zero. We're here that's because the derivative term is equal to zero. And this is the same as saying let's set W to be equal to W.

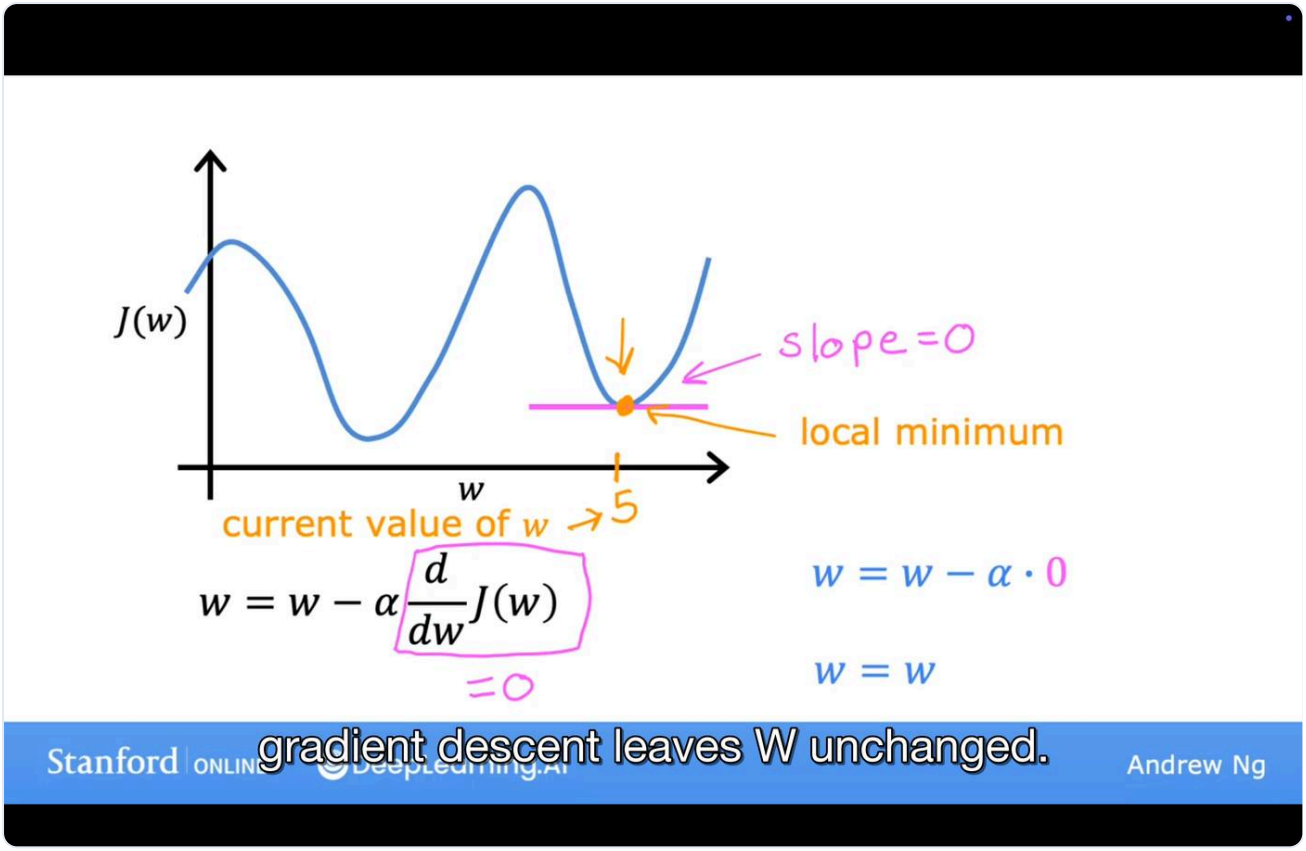


FIGURE 18.22

So this means that if you're already at a local minimum, gradient descent leaves W unchanged. Because it just updates the new value of W to be the exact same old value of W. So concretely, let's say if the current value of W is five. And alpha is 0.1 after one iteration, you update W as W minus alpha times zero and it is still equal to five. So if your parameters have already brought you to a local minimum, then further gradient descent steps to absolutely nothing. It doesn't change the parameters which is what you want because it keeps the solution at that local minimum. This also explains why gradient descent can reach a local minimum, even with a fixed learning rate alpha.

Here's what I mean, to illustrate this, let's look at another example.

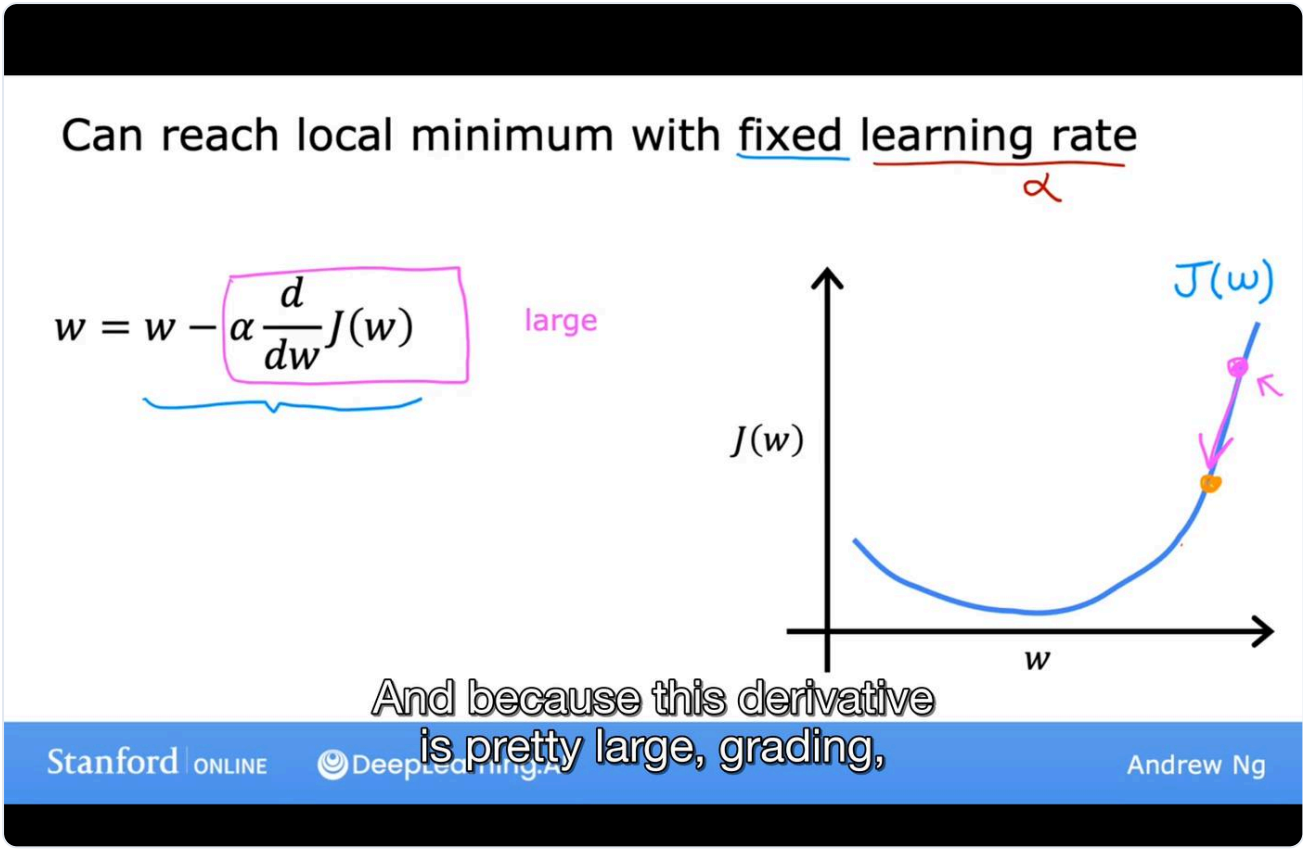


FIGURE 18.23 | Can Reach Local Minimum with Fixed Learning Rate

Here's the cost function J of W that we want to minimize. Let's initialize gradient descent up here at this point. If we take one update step, maybe it will take us to that point. And because this derivative is pretty large, grading, descent takes a relatively big step right. Now, we're at this second point where we take another step. And you may notice that the slope is not as steep as it was at the first point. So the derivative isn't as large. And so the next update step will not be as large as that first step. Now, read this third point here and the derivative is smaller than it was at the previous step.

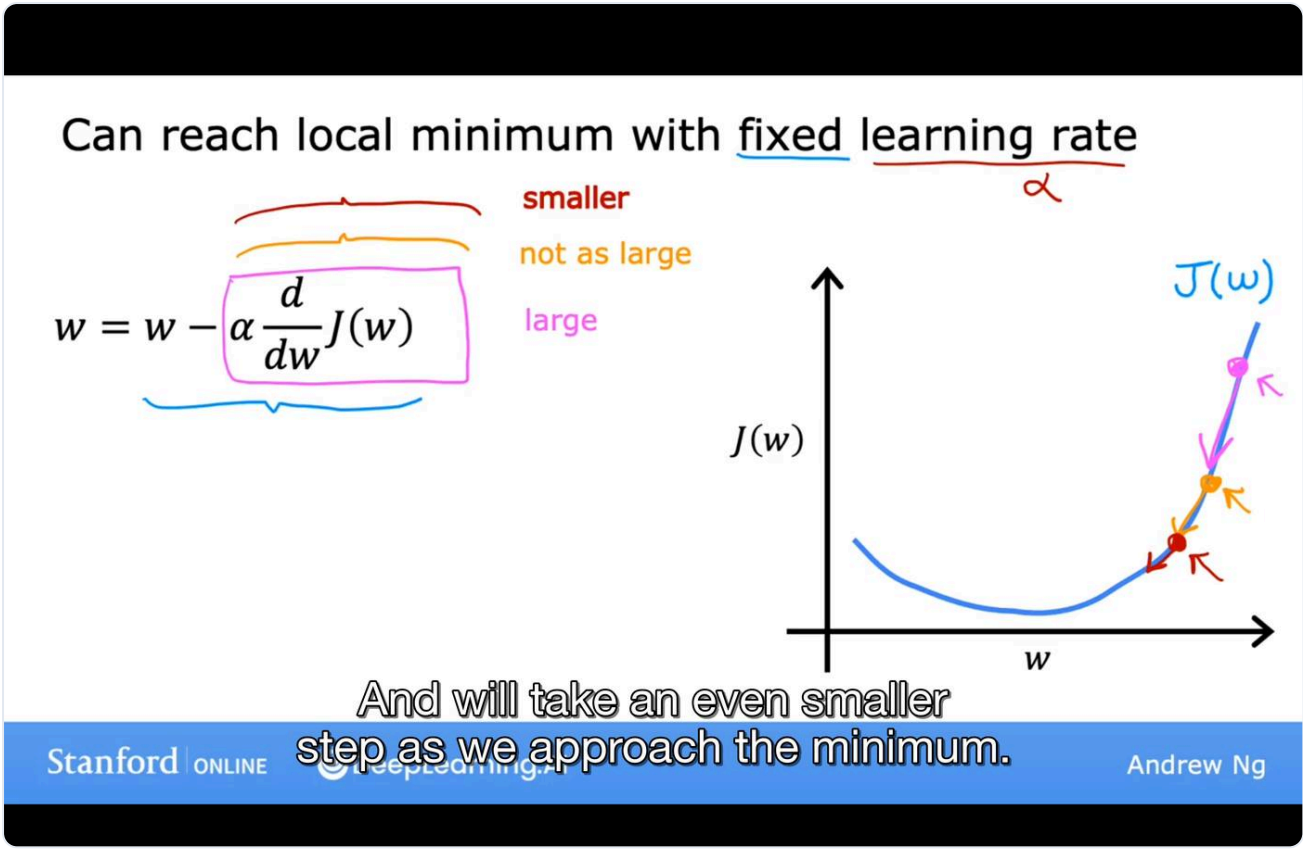


FIGURE 18.24 | Can Reach Local Minimum with Fixed Learning Rate

And will take an even smaller step as we approach the minimum. The decorative gets closer and closer to zero.

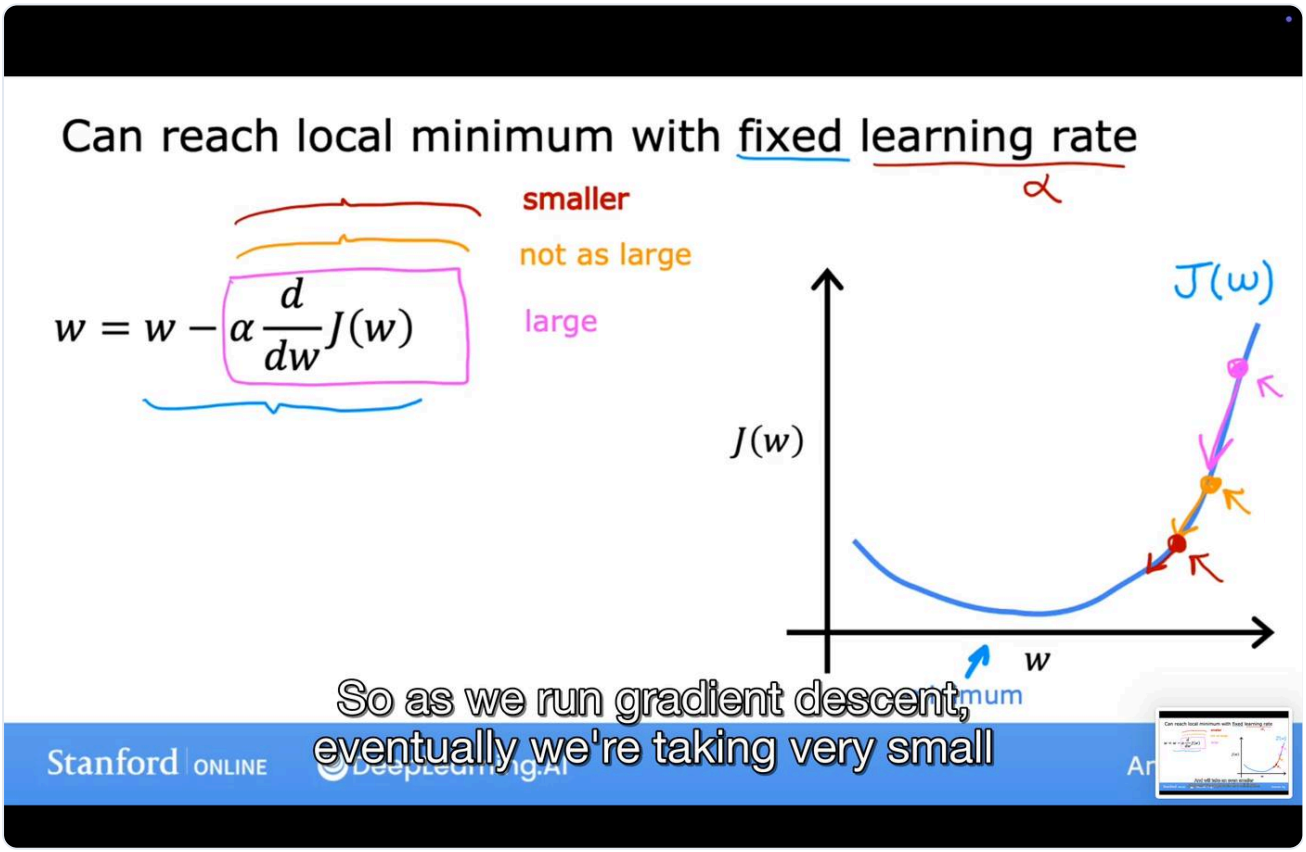


FIGURE 18.25 | Can Reach Local Minimum with Fixed Learning Rate

So as we run gradient descent, eventually we're taking very small steps until you finally reach a local minimum. So just to recap, as we get nearer a local minimum gradient descent will automatically take smaller steps.

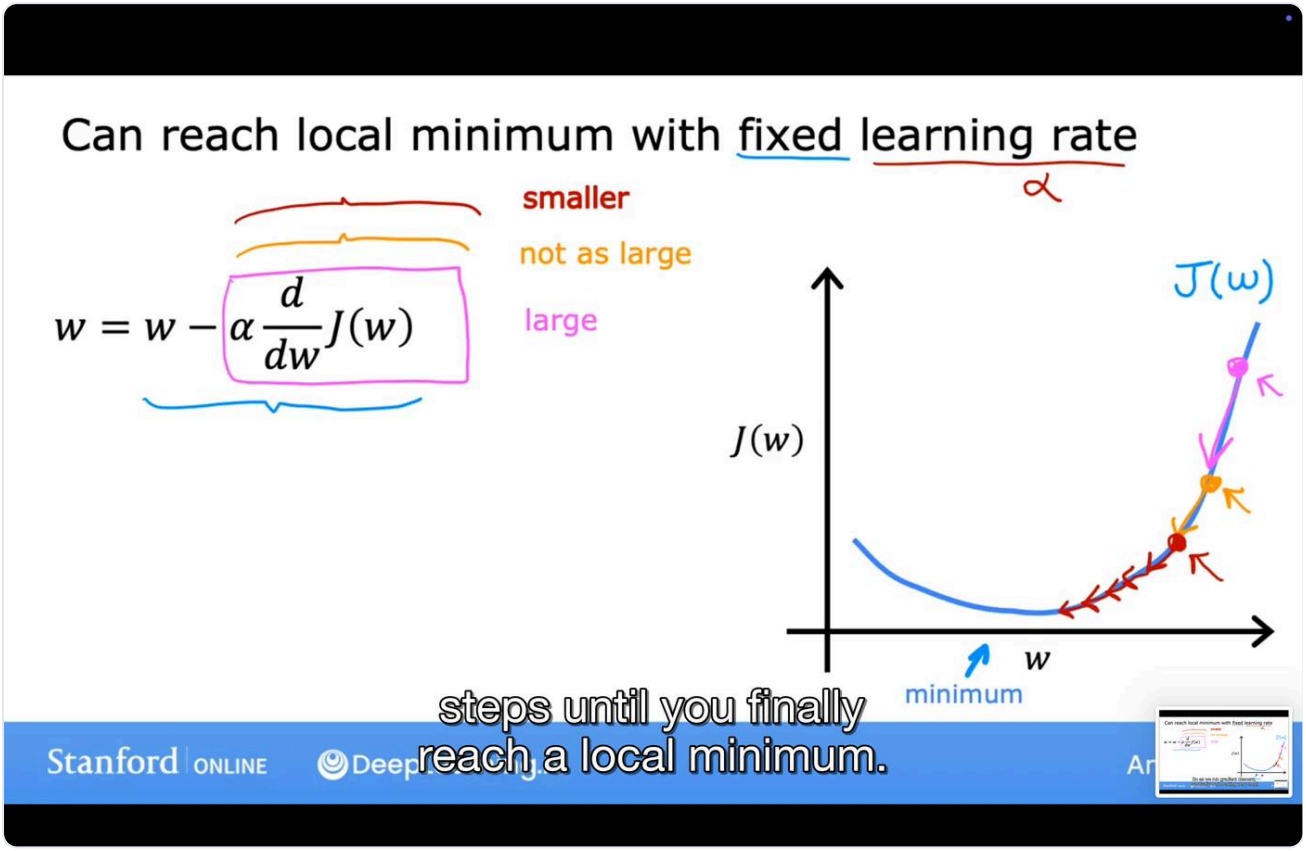


FIGURE 18.26 | Can Reach Local Minimum with Fixed Learning Rate

And that's because as we approach the local minimum, the derivative automatically gets smaller. And that means the update steps also automatically gets smaller.

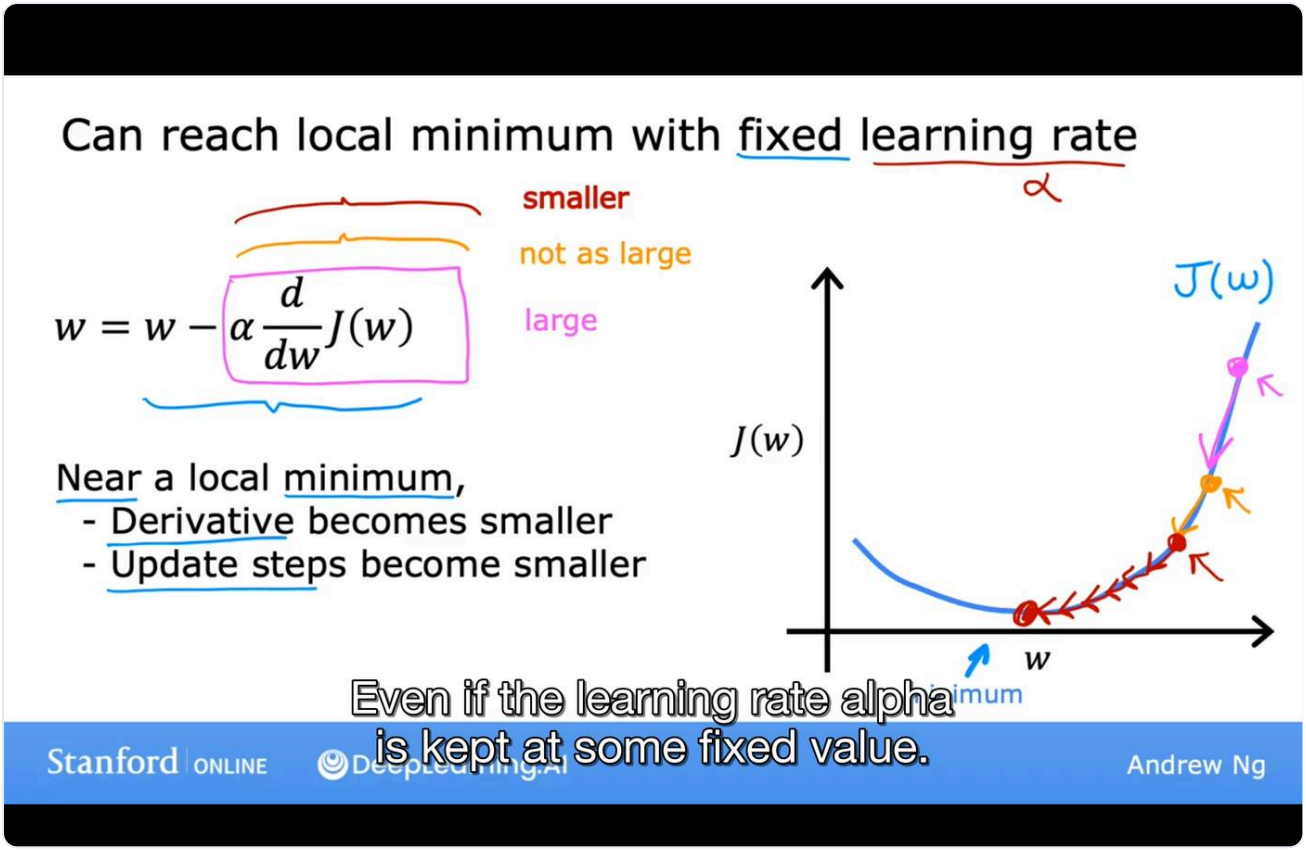


FIGURE 18.27 | Can Reach Local Minimum with Fixed Learning Rate

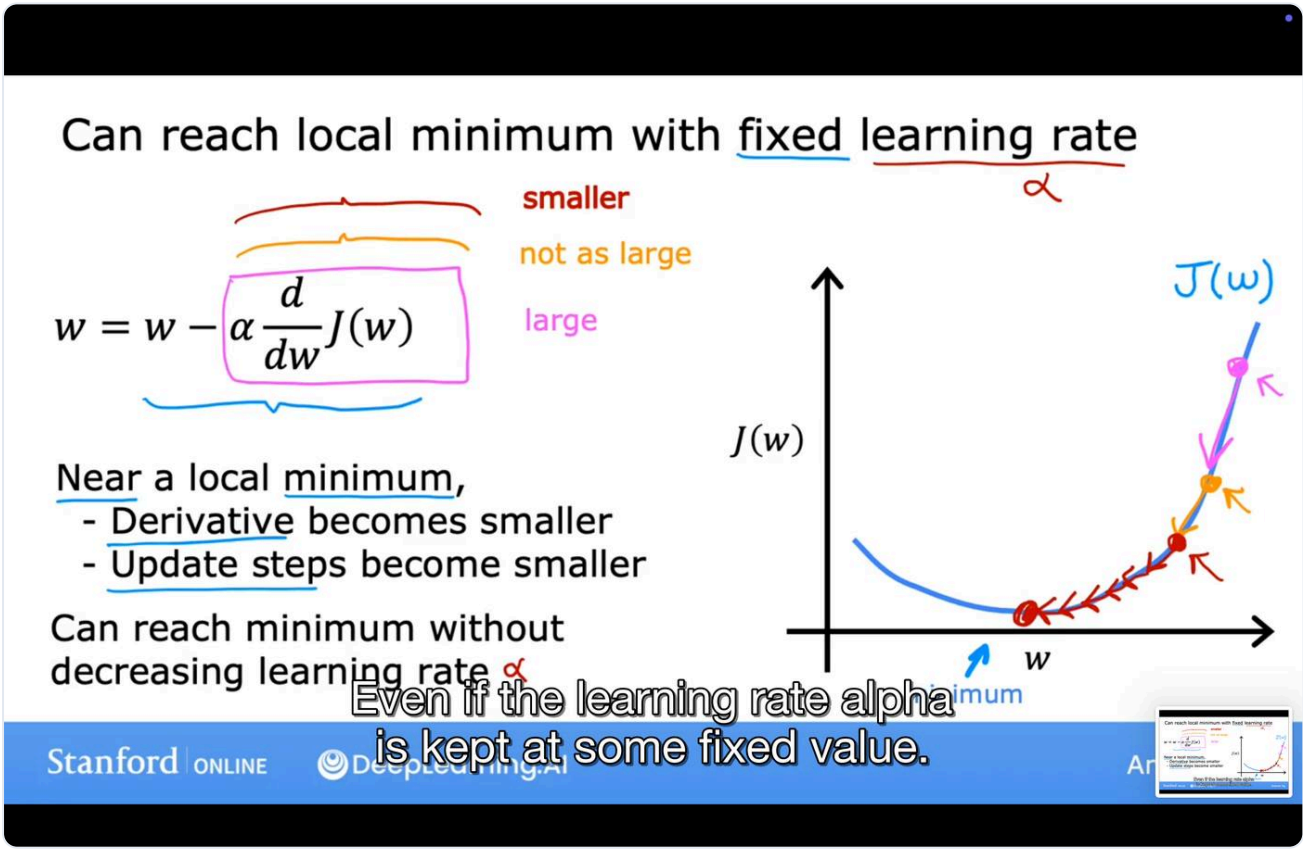


FIGURE 18.28

Even if the learning rate alpha is kept at some fixed value. So that's the gradient descent algorithm, you can use it to try to minimize any cost function J. Not just the mean squared error cost function that we're using for the new regression. In the next video, we're going to take the function J and set that back to be exactly the linear regression models cost function. The mean squared error cost function that we come up with earlier. And putting together great in dissent with this cost function that will give you your first learning algorithm, the linear regression algorithm.

LECTURE 19

Linear Regression Model Cost Function

Previously, you took a look at the linear regression model and then the cost function, and then the gradient descent algorithm. In this video, we're going to pull out together and use the squared error cost function for the linear regression model with gradient descent. This will allow us to train the linear regression model to fit a straight line to achieve the training data. Let's get to it. Here's the linear regression model. To the right is the squared error cost function. Below is the gradient descent algorithm. It turns out if you calculate these derivatives, these are the terms you would get. The derivative with respect to W is this 1 over m, sum of i equals 1 through m.

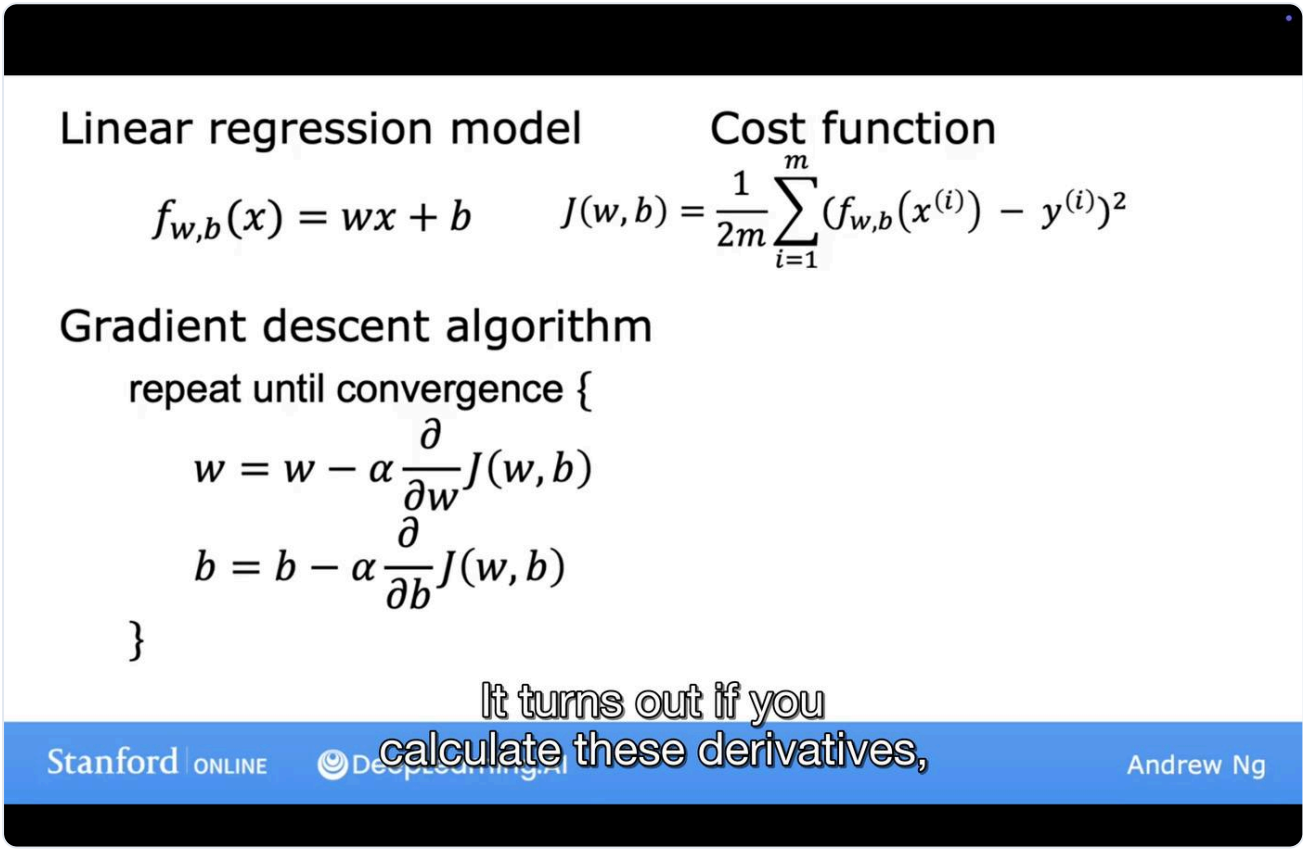


FIGURE 19.1 | Linear Regression Model Cost Function

Then the error term, that is the difference between the predicted and the actual values times the input feature xi. The derivative with respect to b is this formula over here, which looks the same as the equation above, except that it doesn't have that xi term at the end. If you use these formulas to compute these two derivatives and implements gradient descent this way, it will work. Now, you may be wondering, where did I get these formulas from? They're derived using calculus. If you want to see the full derivation, I'll quickly run through the derivation on the next slide. But if you don't remember or aren't interested in the calculus, don't worry about it.

You can skip the materials on the next slide entirely and still be able to implement gradient descent and finish this class and everything will work just fine. In this slide, which is one of the most mathematical slide of the entire specialization, and again is completely optional, we'll show

you how to calculate the derivative terms.

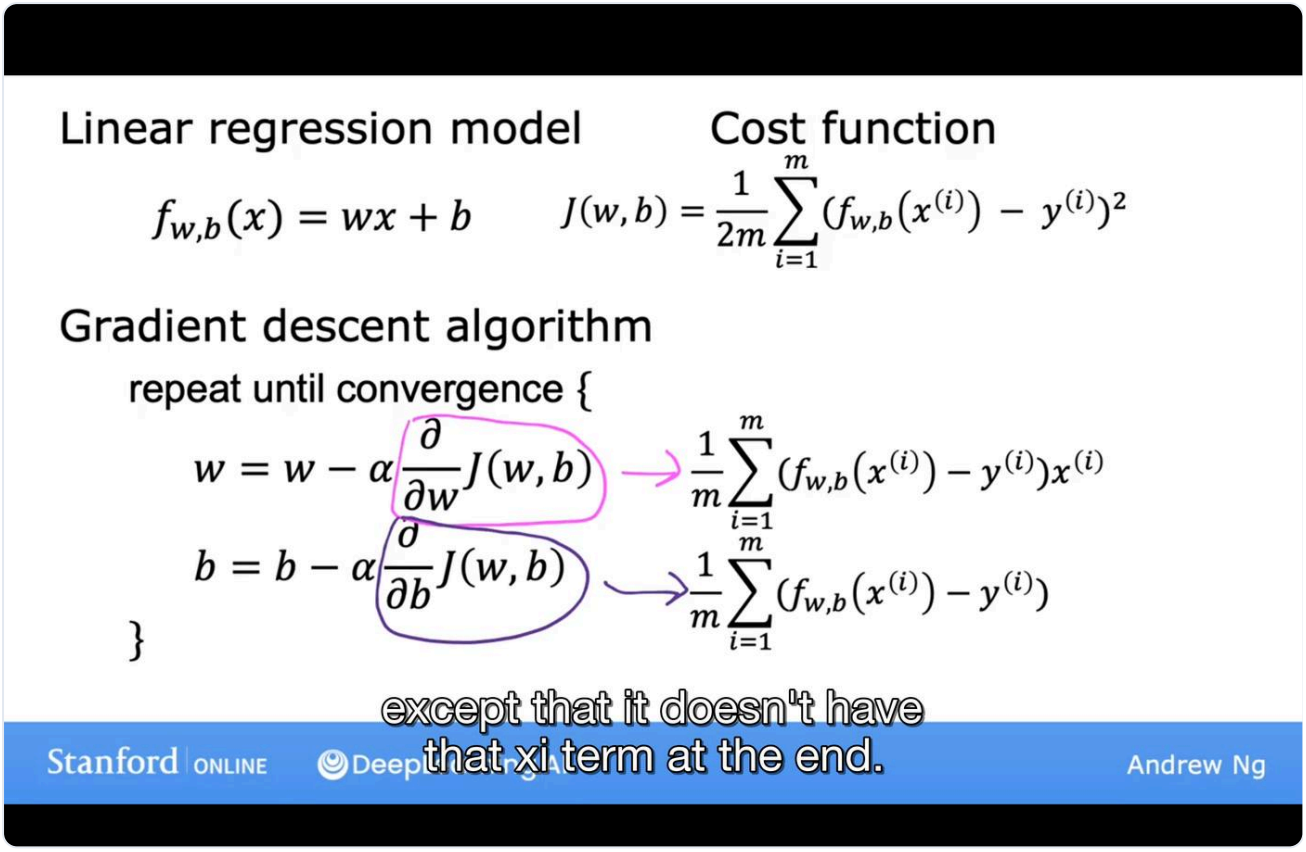


FIGURE 19.2 | Linear Regression Model Cost Function

Let's start with the first term. The derivative of the cost function J with respect to w. We'll start by plugging in the definition of the cost function J. J of WP is this. 1 over 2m times this sum of the squared error terms. Now remember also that f of wb of X^i is equal to this term over here, which is WX^i plus b. What we would like to do is compute the derivative, also called the partial derivative with respect to w of this equation right here on the right. If you taken a calculus class before, and again is totally fine if you haven't, you may know that by the rules of calculus, the derivative is equal to this term over here. Which is why the two here and two here cancel out, leaving us with this equation that you saw on the previous slide.

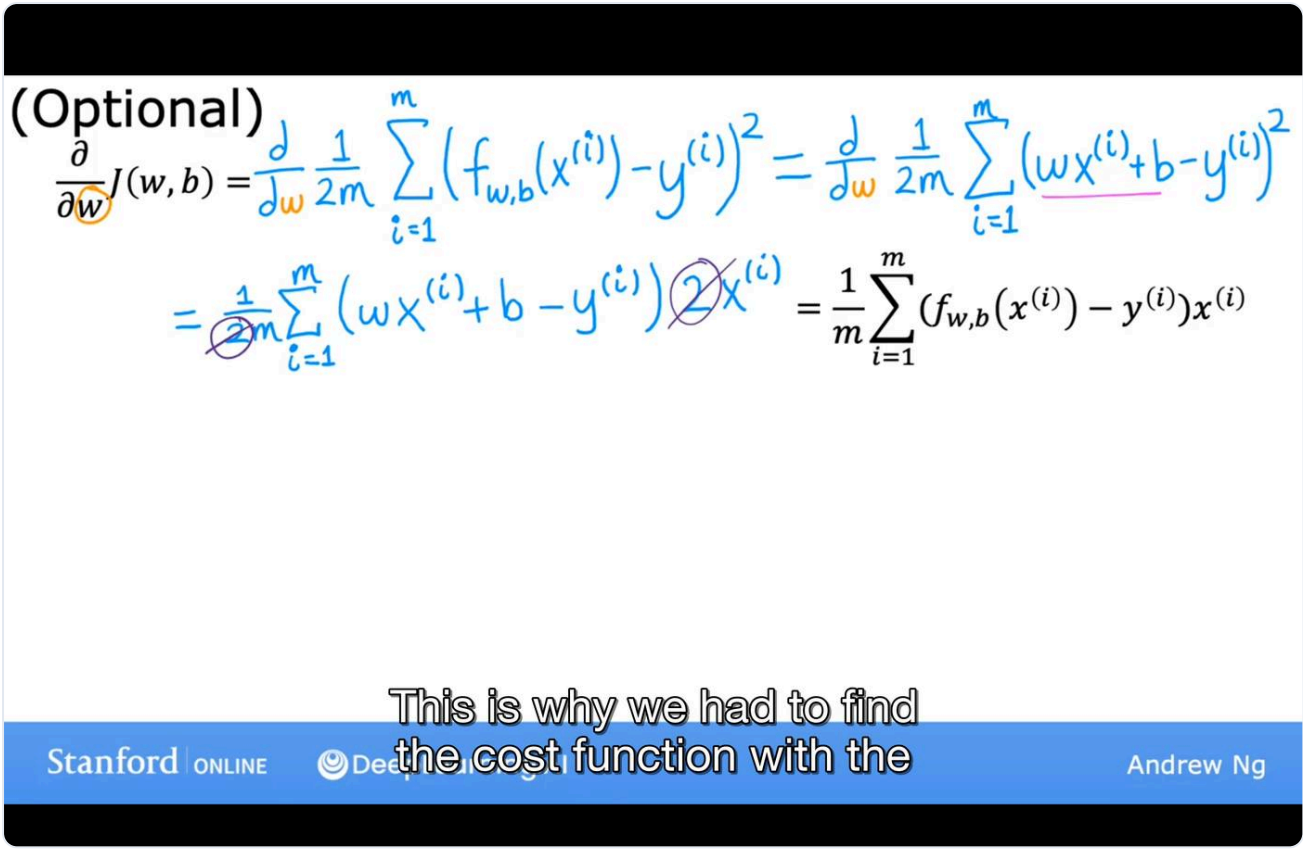


FIGURE 19.3

This is why we had to find the cost function with the 1.5 earlier this week is because it makes the partial derivative neater. It cancels out the two that appears from computing the derivative. For the other derivative with respect to b, this is quite similar. I can write it out like this, and once again, plugging the definition of f of X^i, giving this equation. By the rules of calculus, this is equal to this where there's no X^i anymore at the end. The 2's cancel one small and you end up with this expression for the derivative with respect to b.

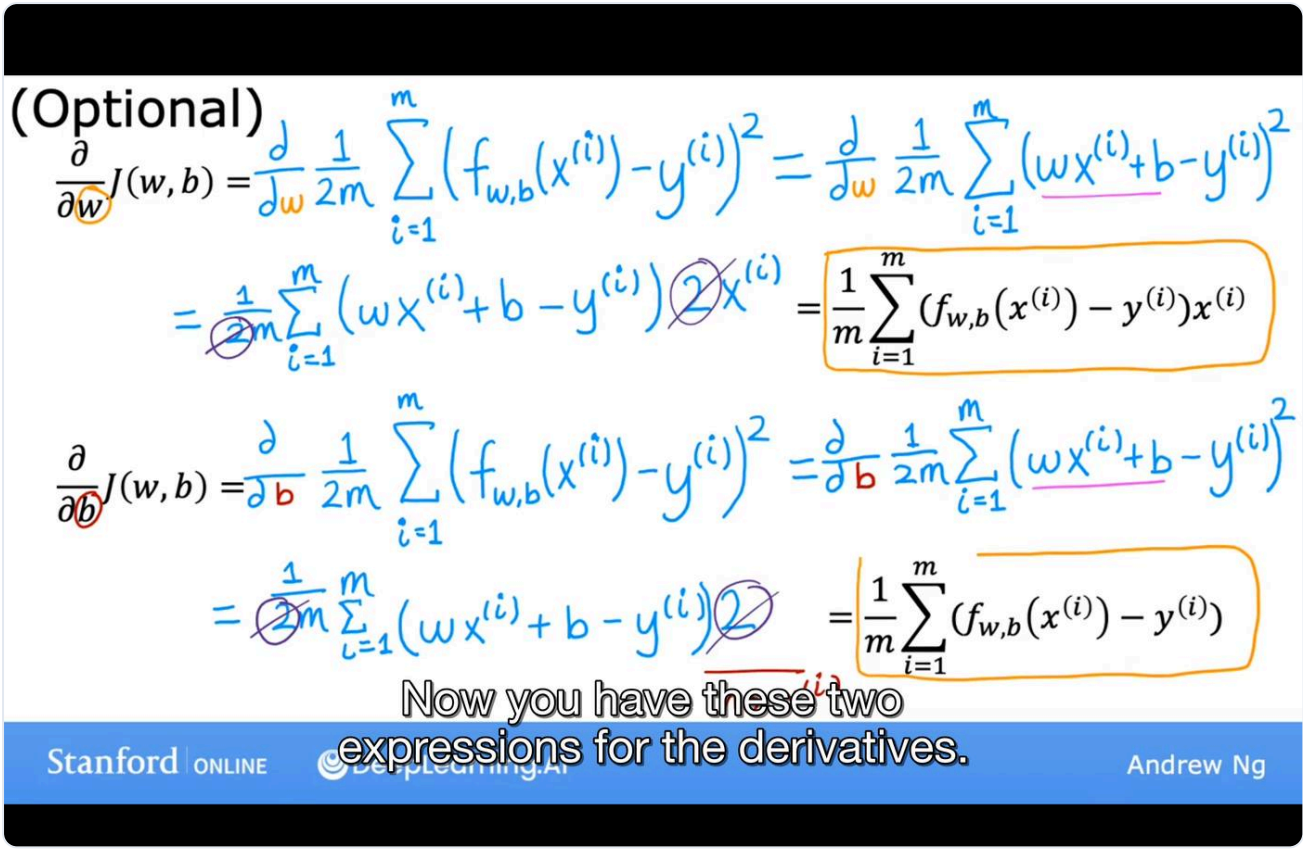


FIGURE 19.4

Now you have these two expressions for the derivatives. You can plug them into the gradient descent algorithm. Here's the gradient descent algorithm for linear regression. You repeatedly carry out these updates to w and b until convergence. Remember that this f of x is a linear regression model, so as equal to w times x plus b. This expression here is the derivative of the cost function with respect to w. This expression is the derivative of the cost function with respect to b.

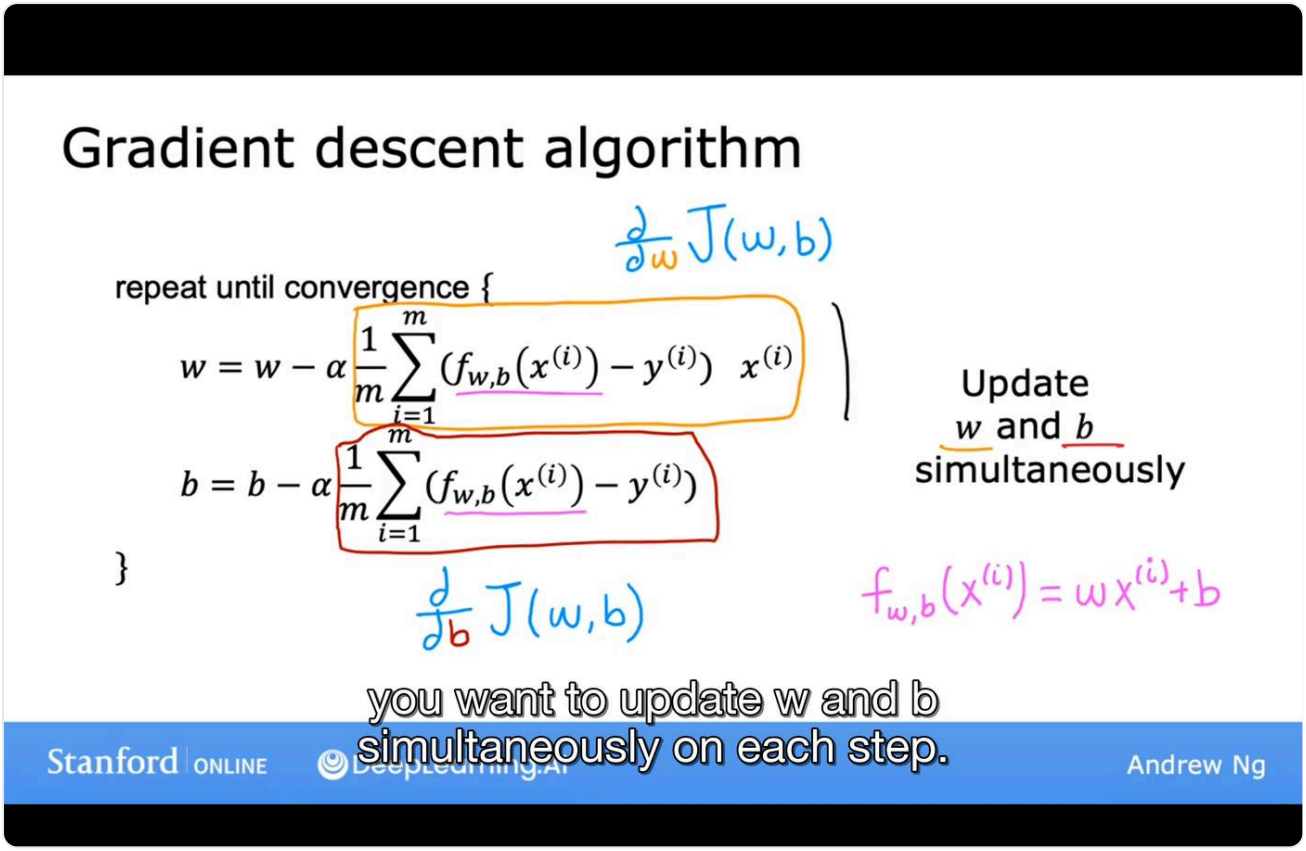


FIGURE 19.5 | Gradient Descent Algorithm

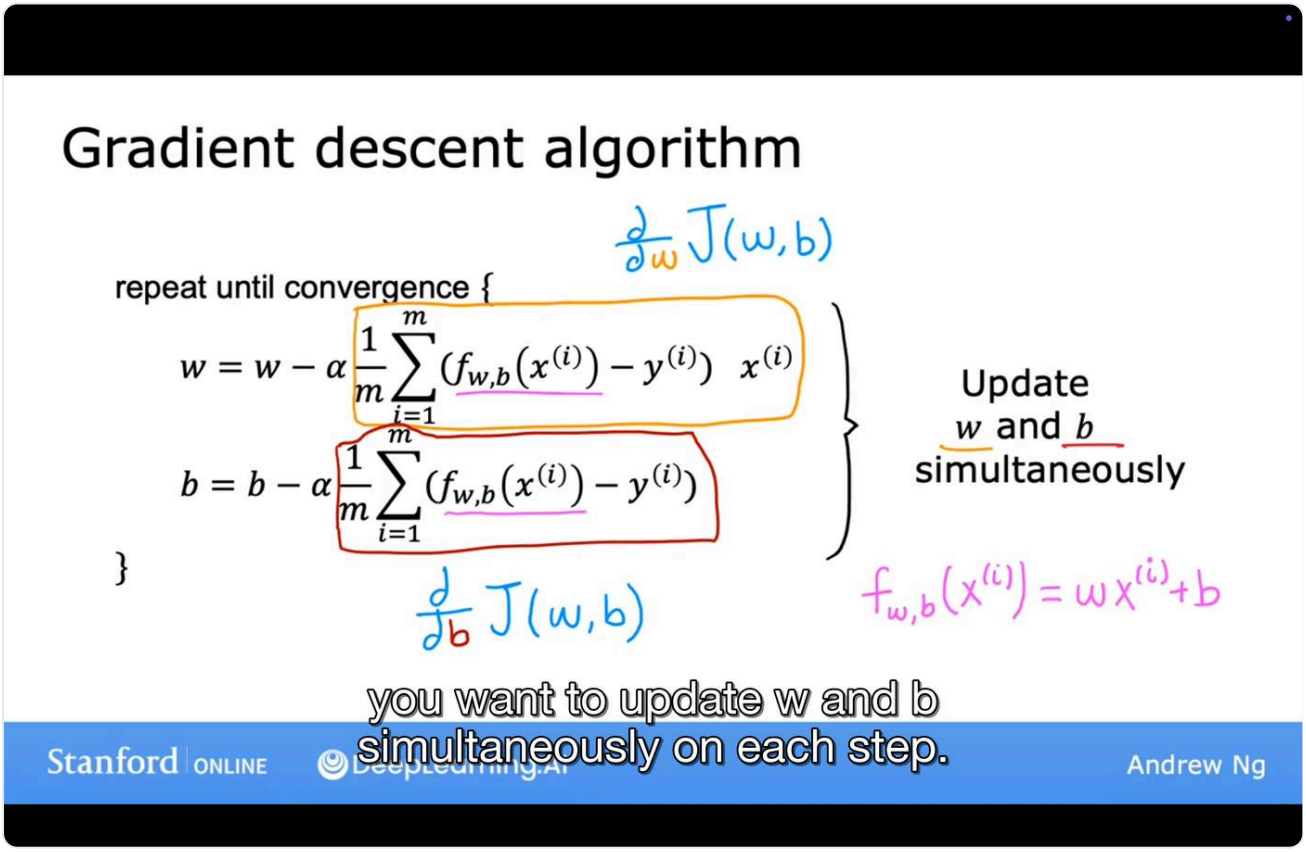


FIGURE 19.6 | Gradient Descent Algorithm

Just as a reminder, you want to update w and b simultaneously on each step. Now, let's get familiar with how gradient descent works. One the shoe we saw with gradient descent is that it can lead to a local minimum instead of a global minimum.

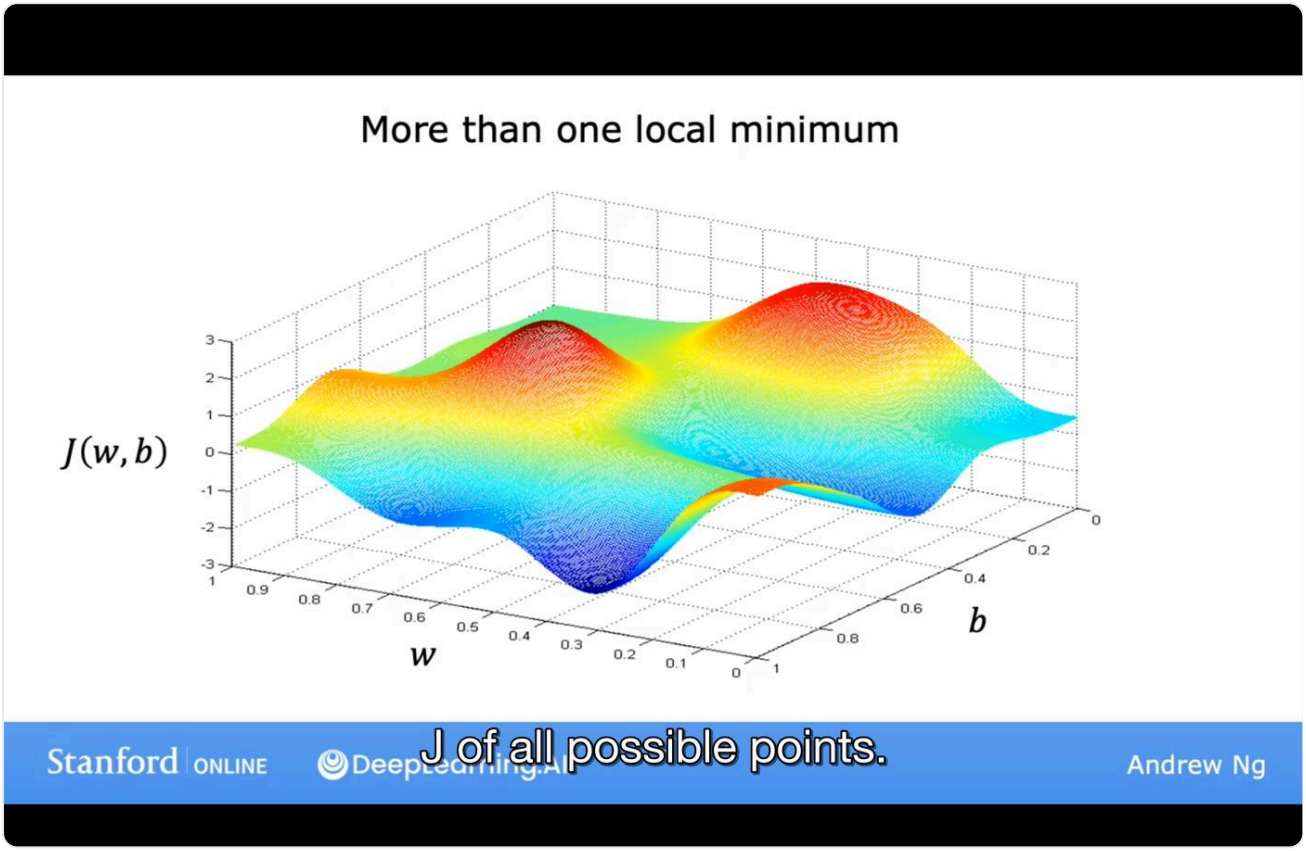


FIGURE 19.7 | More Than One Local Minimum

Whether global minimum means the point that has the lowest possible value for the cost function J of all possible points. You may recall this surface plot that looks like an outdoor park with a few hills with the process and the birds as a relaxing Hobo Hill.

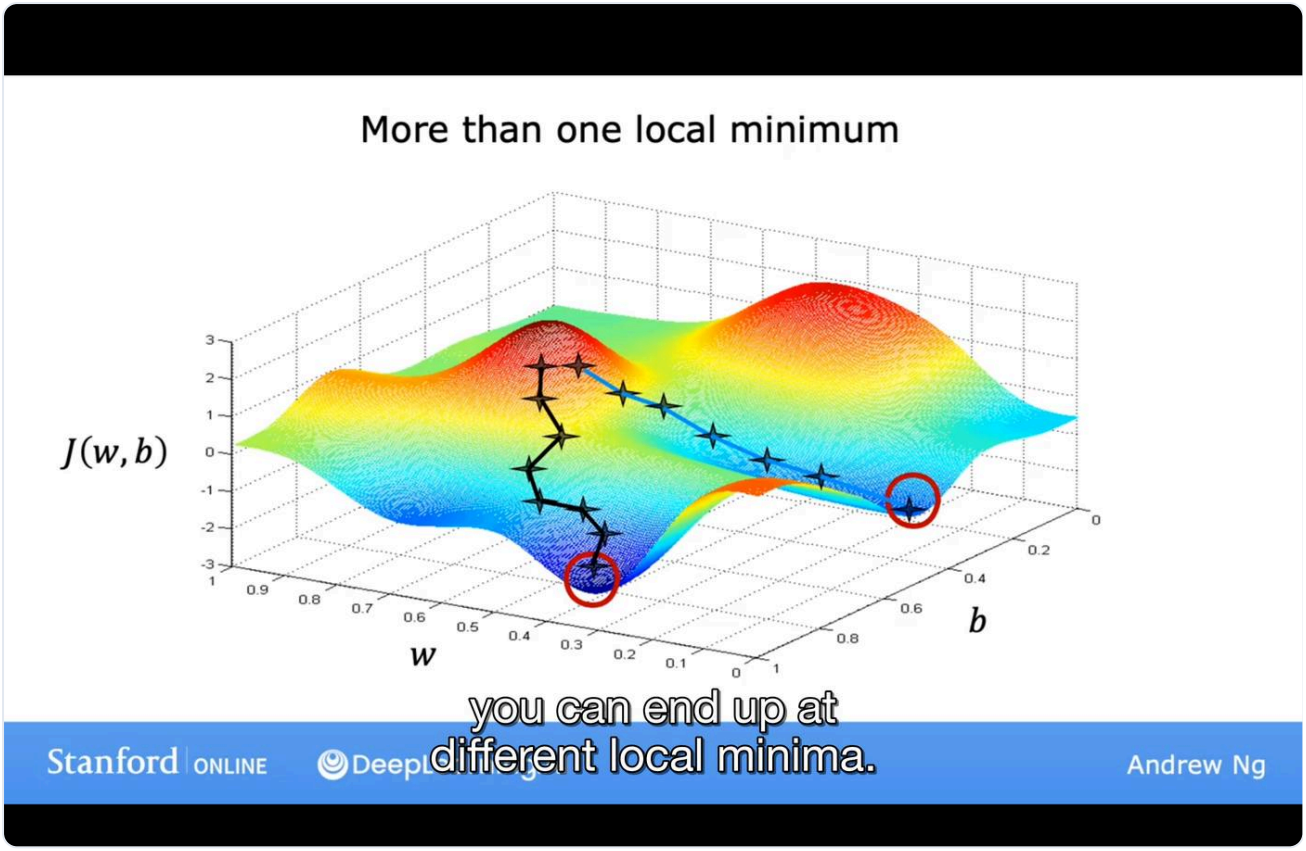


FIGURE 19.8 | More Than One Local Minimum

This function has more than one local minimum. Remember, depending on where you initialize the parameters w and b, you can end up at different local minima. You can end up here, or you can end up here.

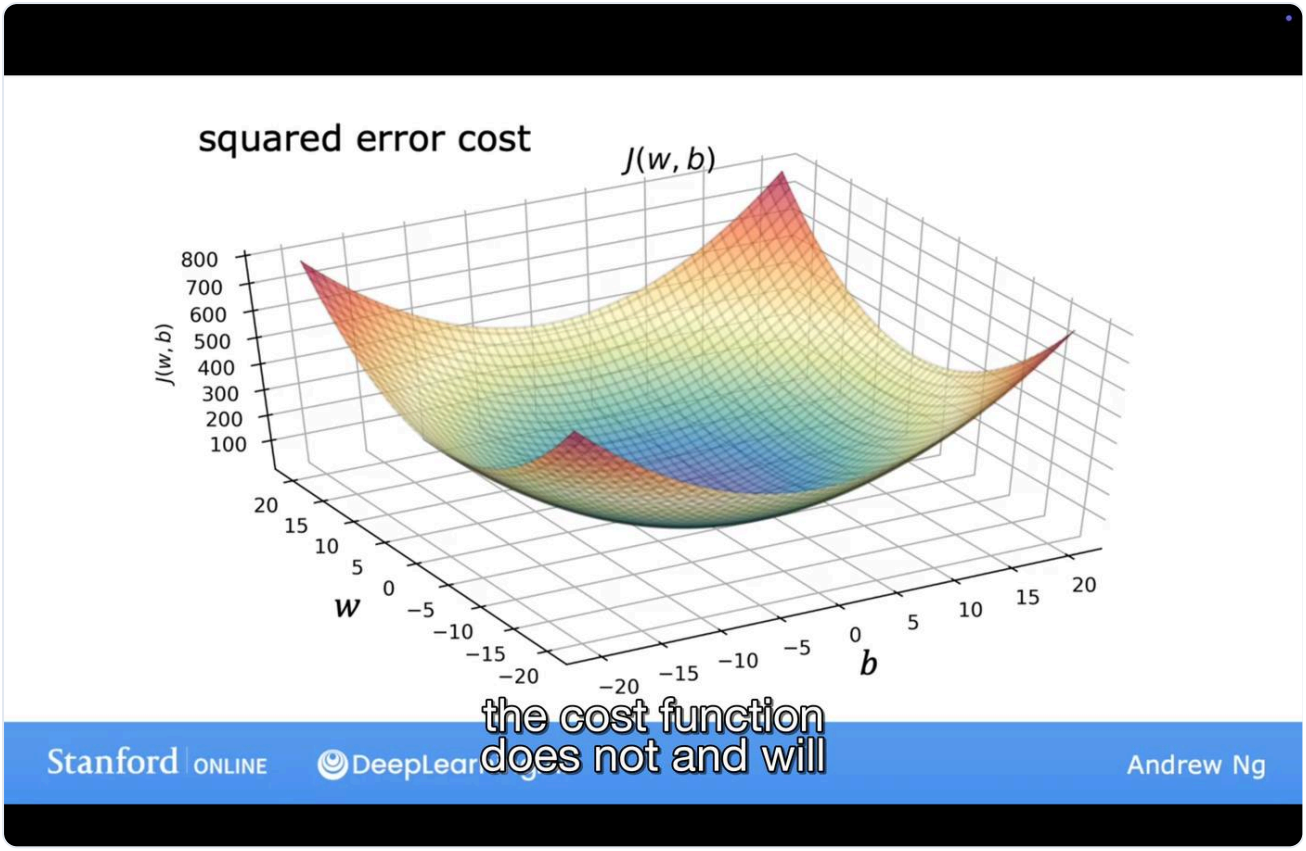


FIGURE 19.9

But it turns out when you're using a squared error cost function with linear regression, the cost function does not and will never have multiple local minima. It has a single global minimum because of this bowl-shape.

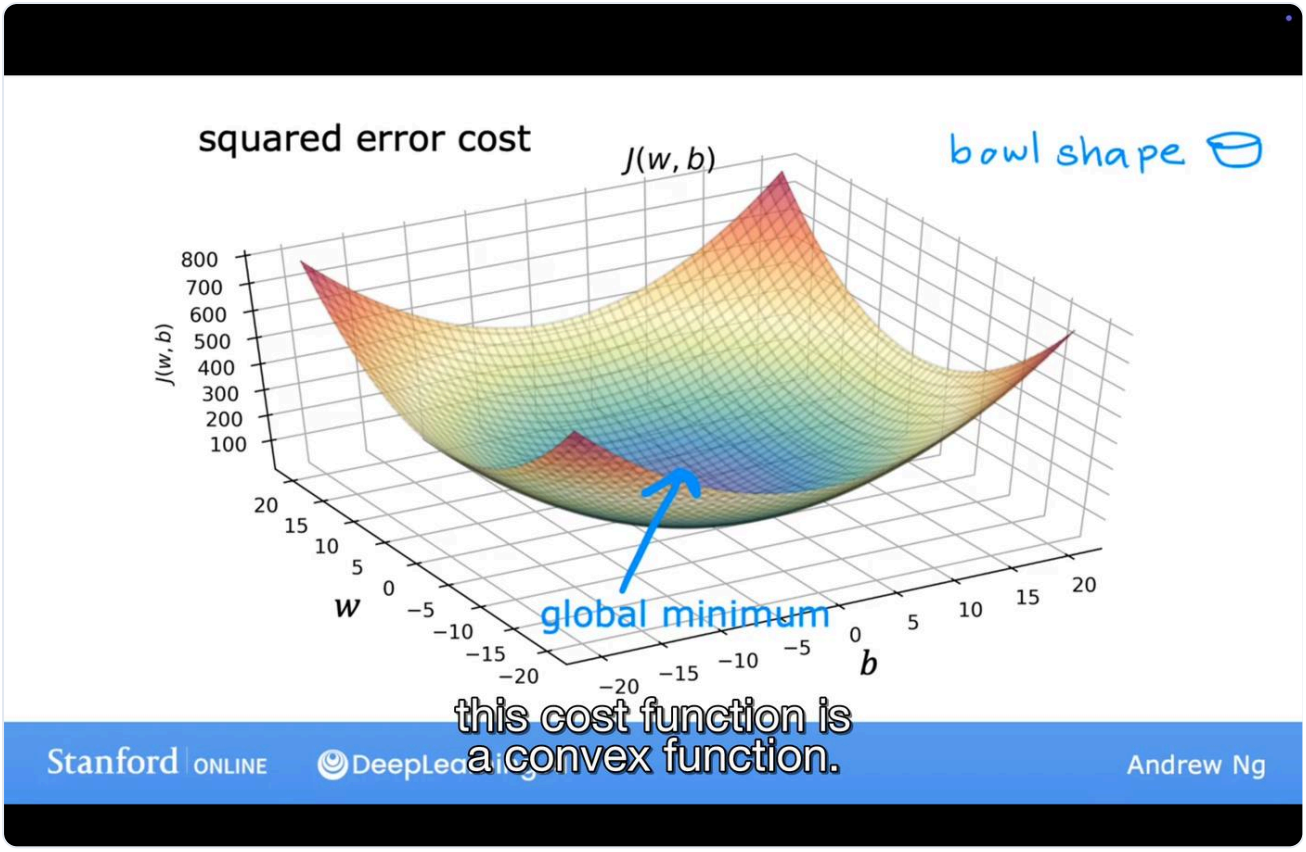


FIGURE 19.10

The technical term for this is that this cost function is a convex function.

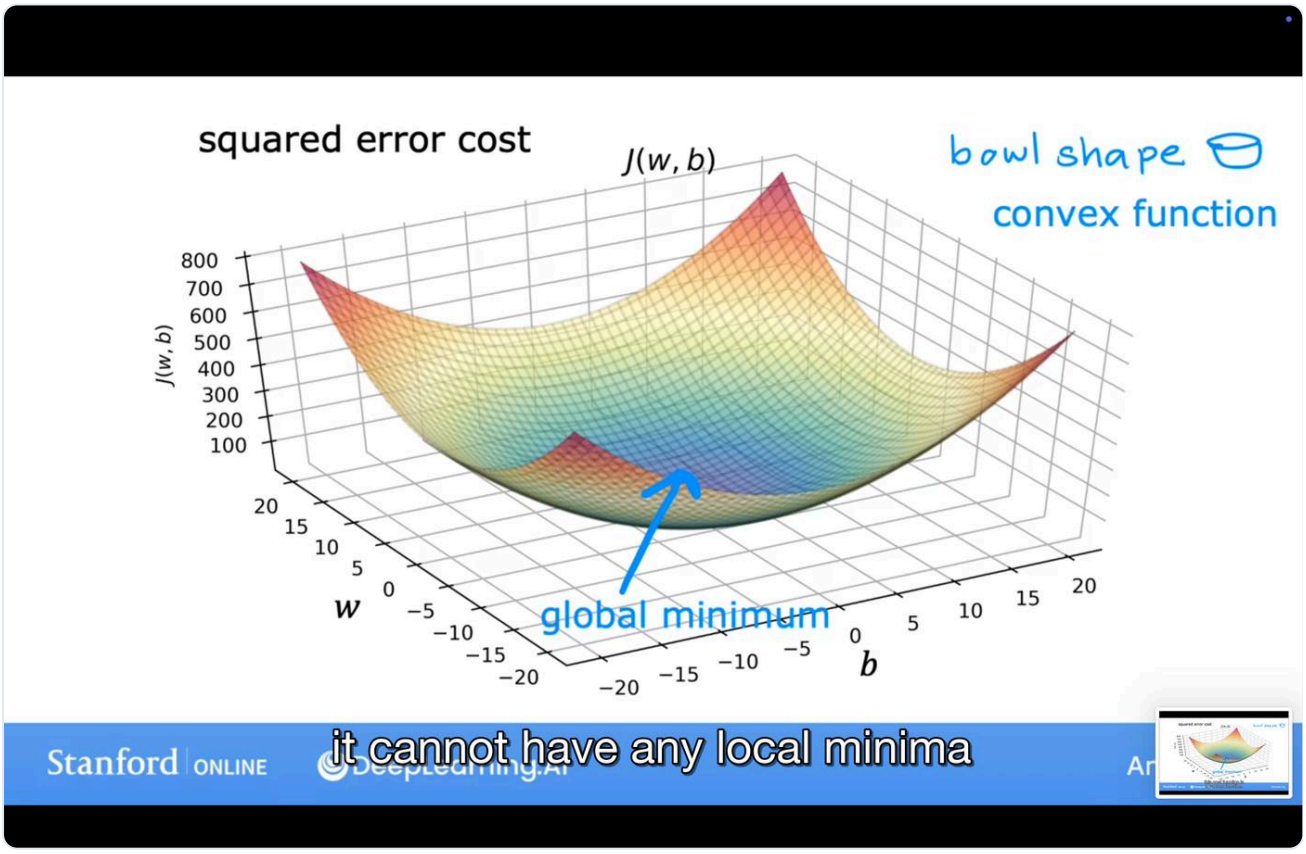


FIGURE 19.11 | A Convex Function

Informally, a convex function is of bowl-shaped function and it cannot have any local minima other than the single global minimum. When you implement gradient descent on a convex function, one nice property is that so long as you're learning rate is chosen appropriately, it will always converge to the global minimum. Congratulations, you now know how to implement gradient descent for linear regression. We have just one last video for this week. That video, we'll see this algorithm in action. Let's go to that last video.

LECTURE 20

Gradient Descent

Let's see what happens when you run gradient descent for linear regression. Let's go see the algorithm in action. Here's a plot of the model and data on the upper left and a contour plot of the cost function on the upper right and at the bottom is the surface plot of the same cost function.

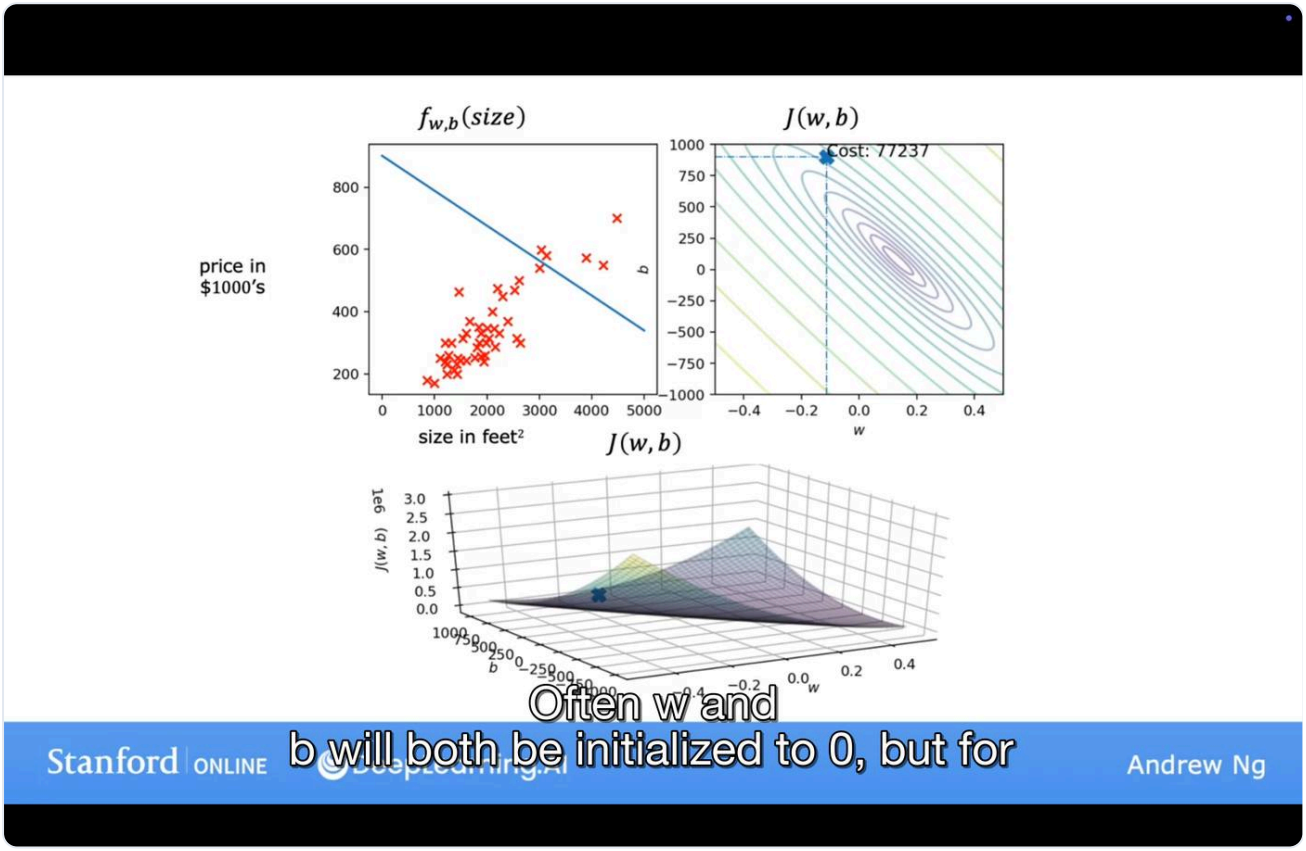


FIGURE 20.1

Often w and b will both be initialized to 0, but for this demonstration, lets initialized w = -0.1 and b = 900.

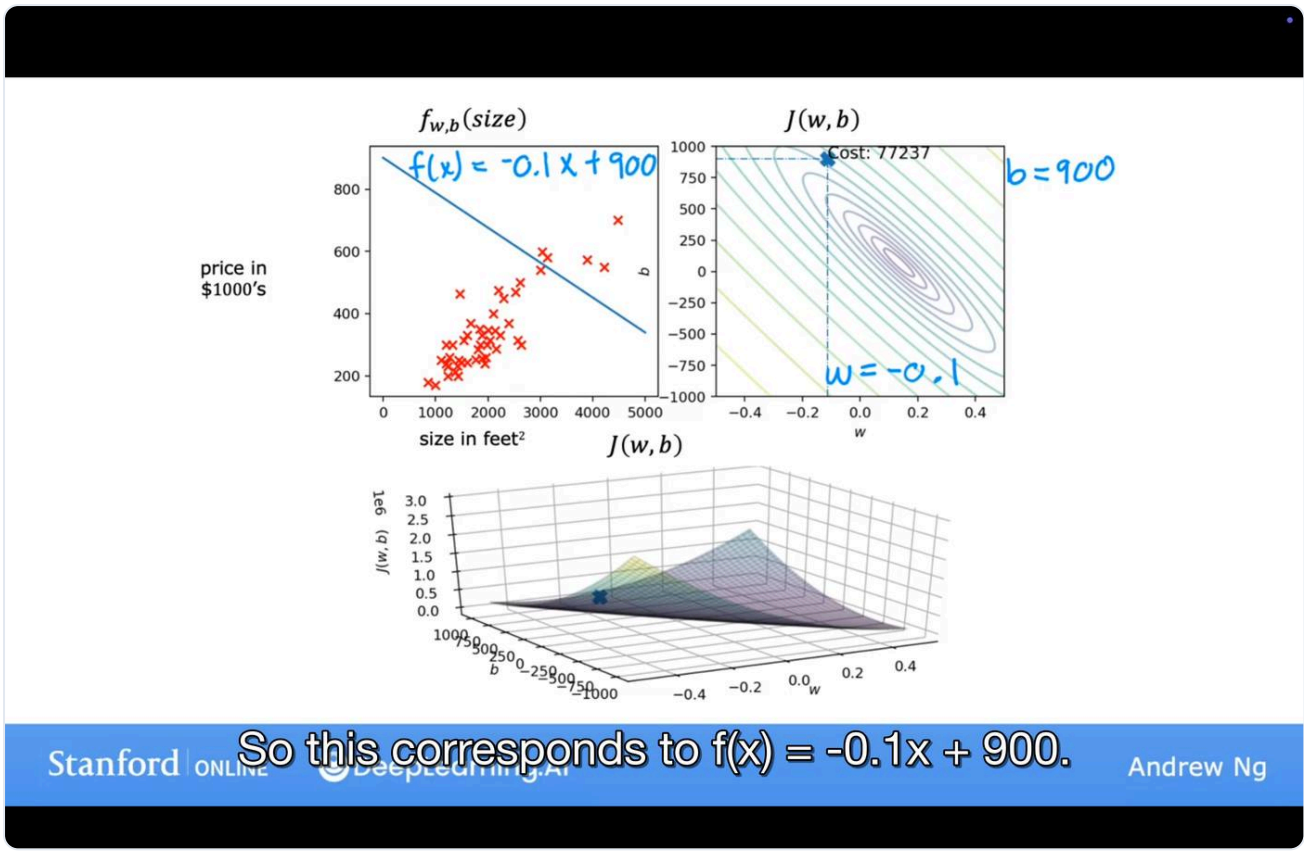


FIGURE 20.2

So this corresponds to f(x) = -0.1x + 900.

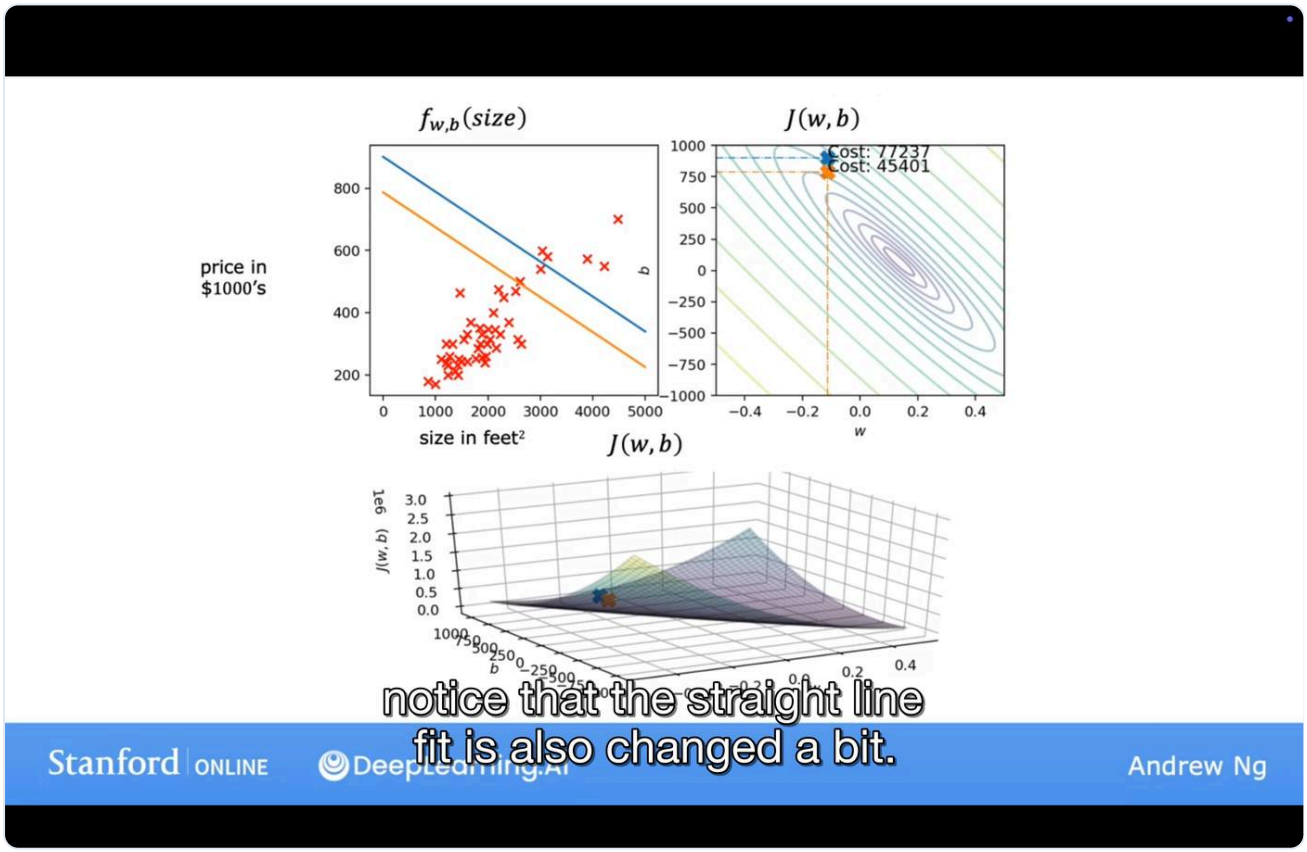


FIGURE 20.3

Now, if we take one step using gradient descent, we ended up going from this point of the cost function out here to this point just down and to the right and notice that the straight line fit is also changed a bit. Let's take another step.

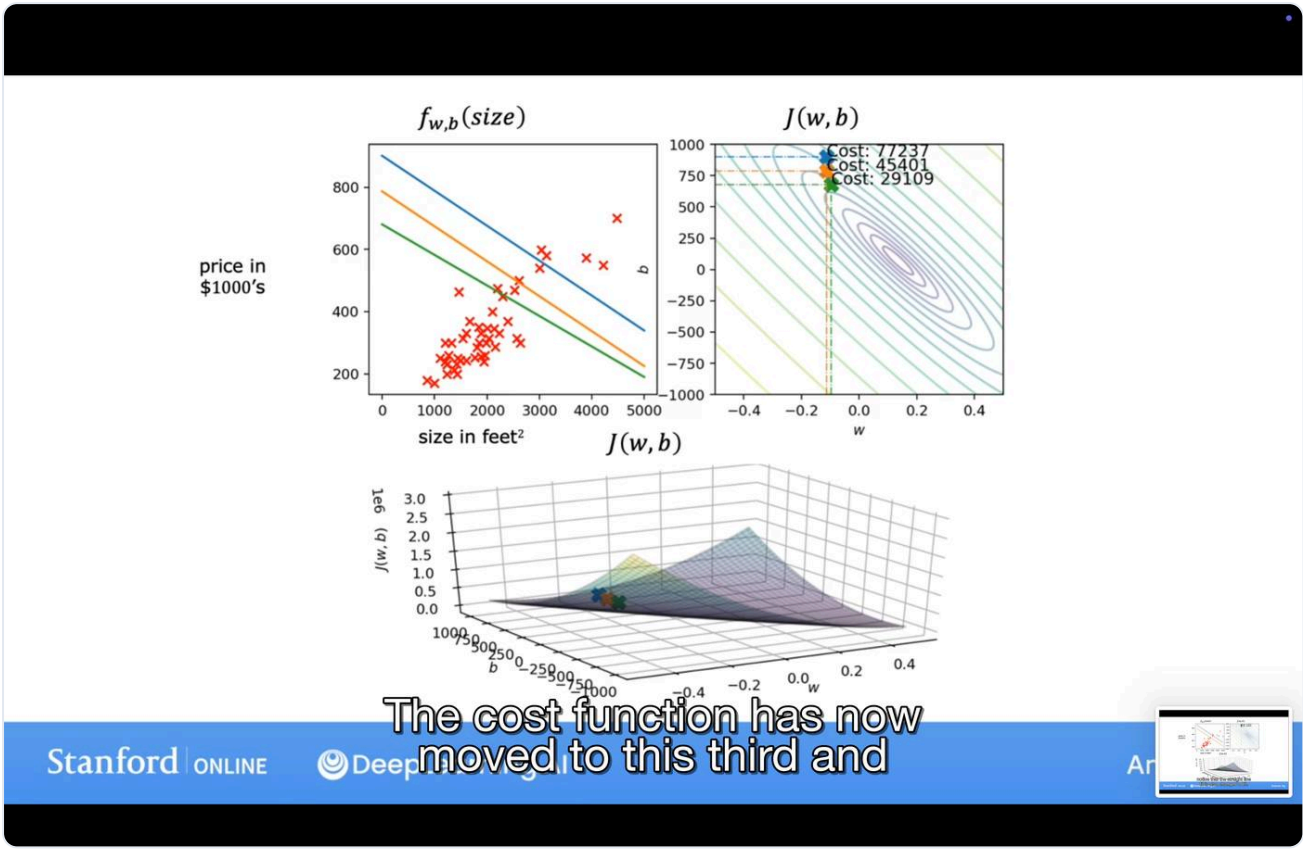


FIGURE 20.4

The cost function has now moved to this third and again the function f(x) has also changed a bit.

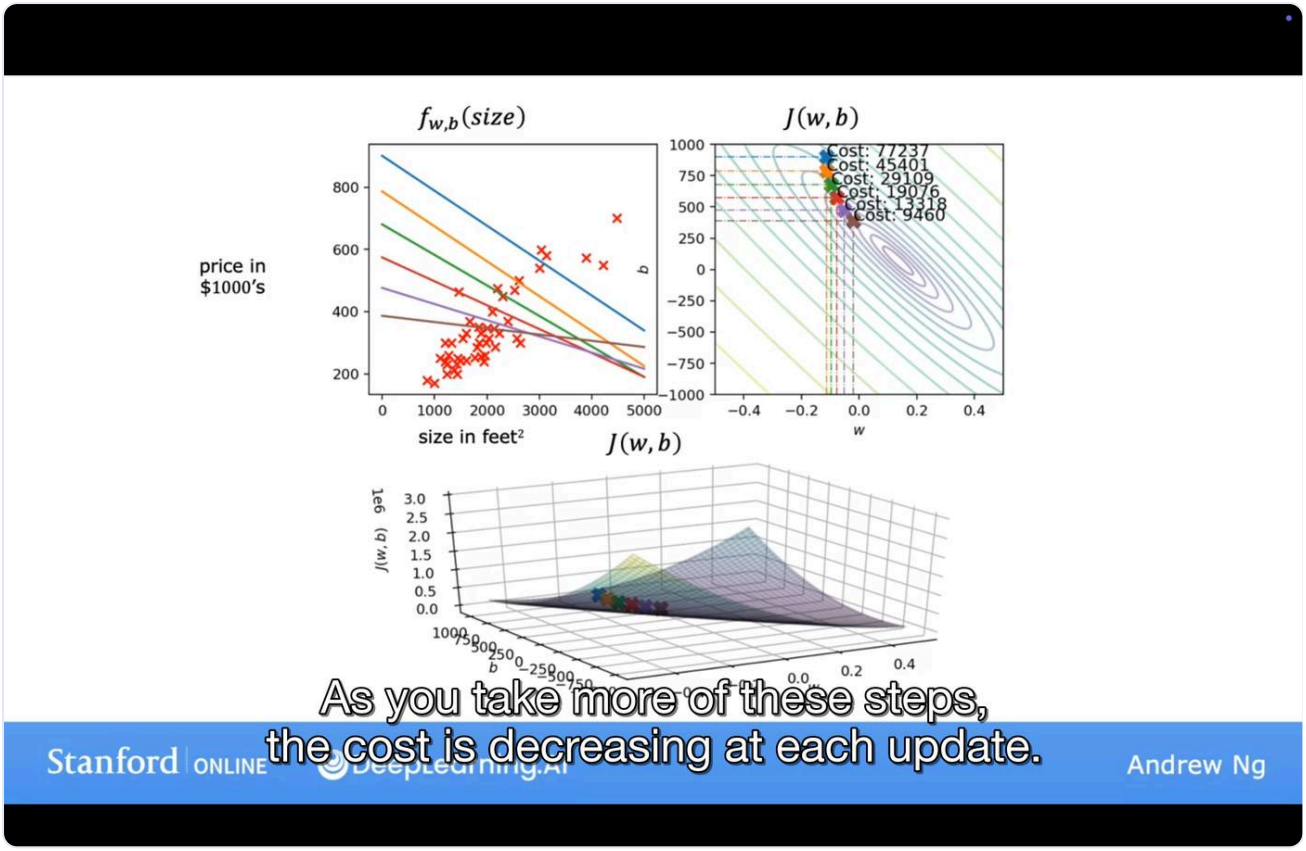


FIGURE 20.5

As you take more of these steps, the cost is decreasing at each update.

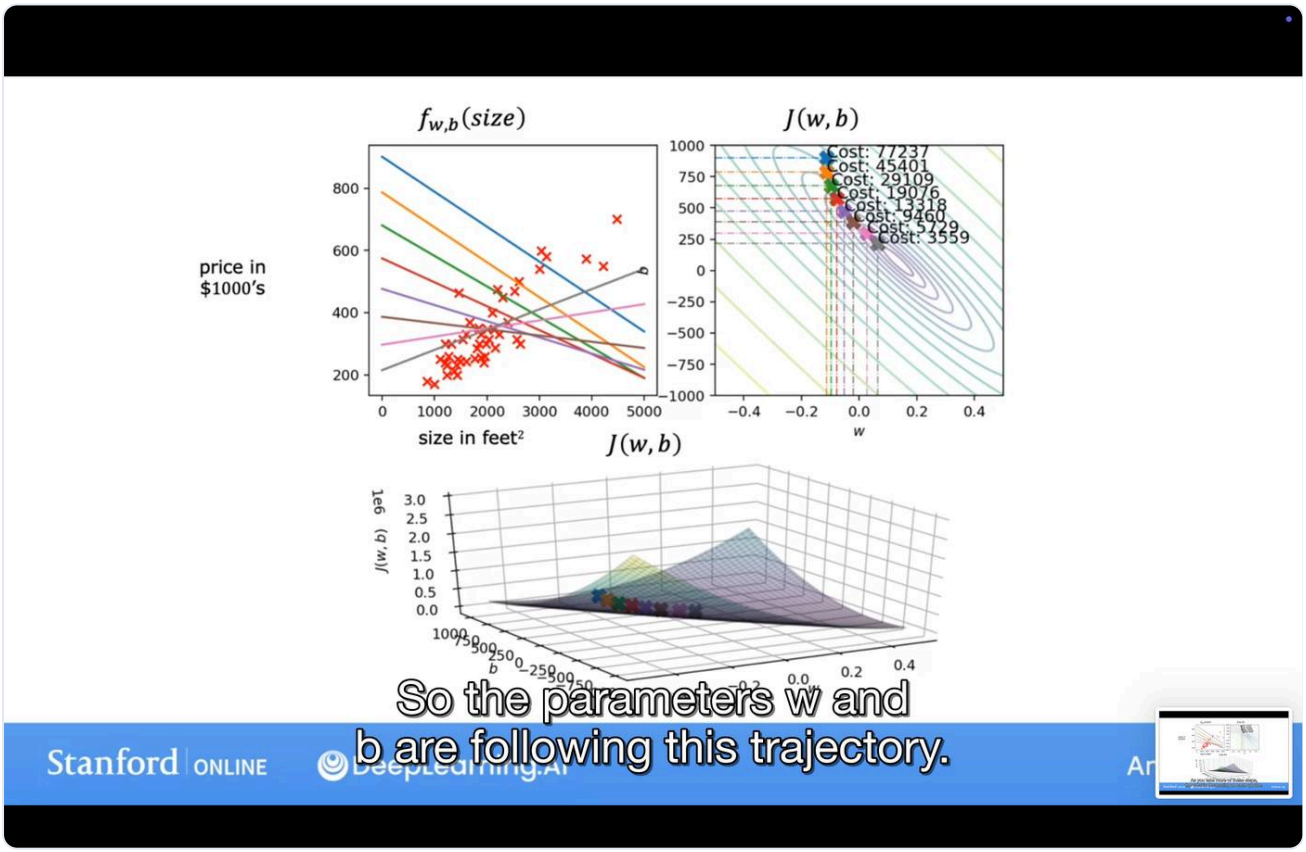


FIGURE 20.6

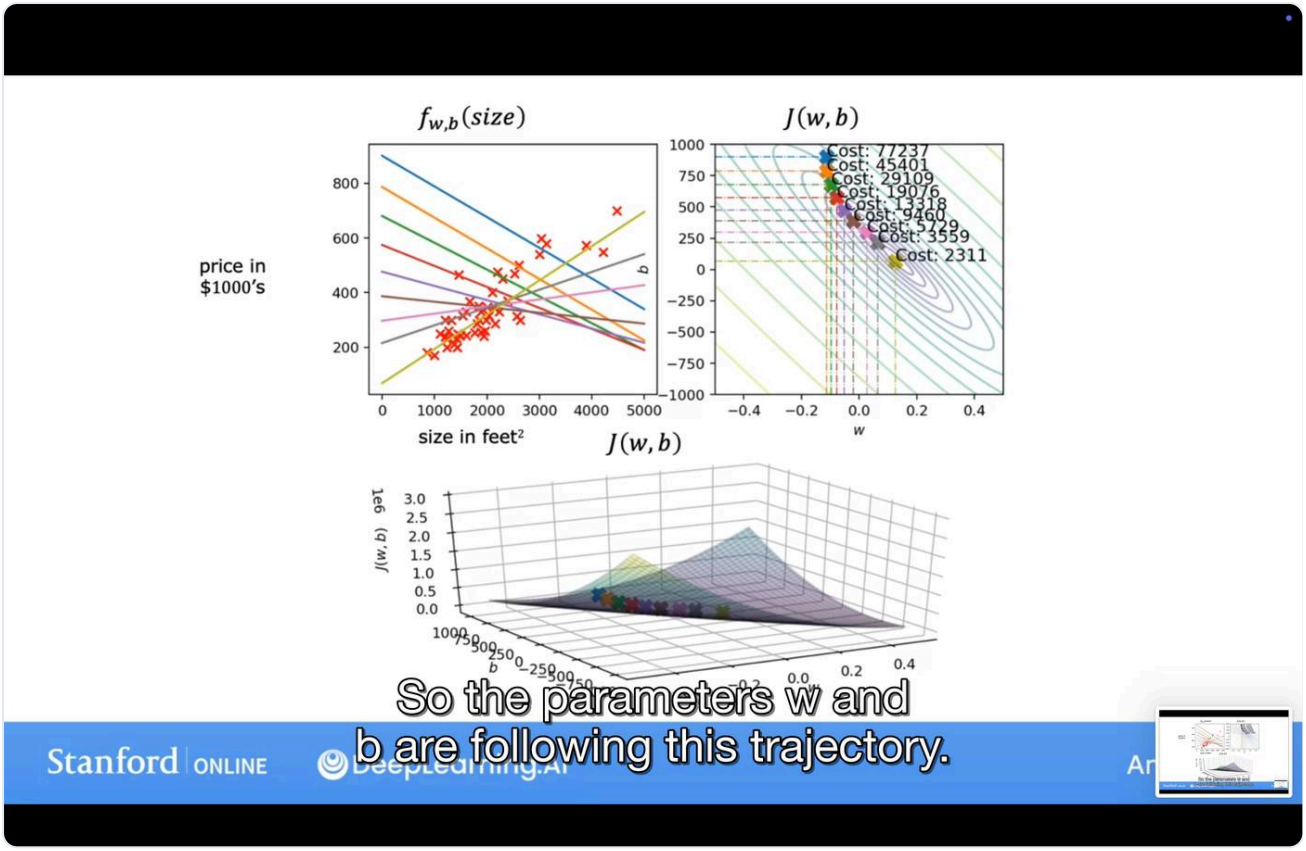


FIGURE 20.7

So the parameters w and b are following this trajectory.

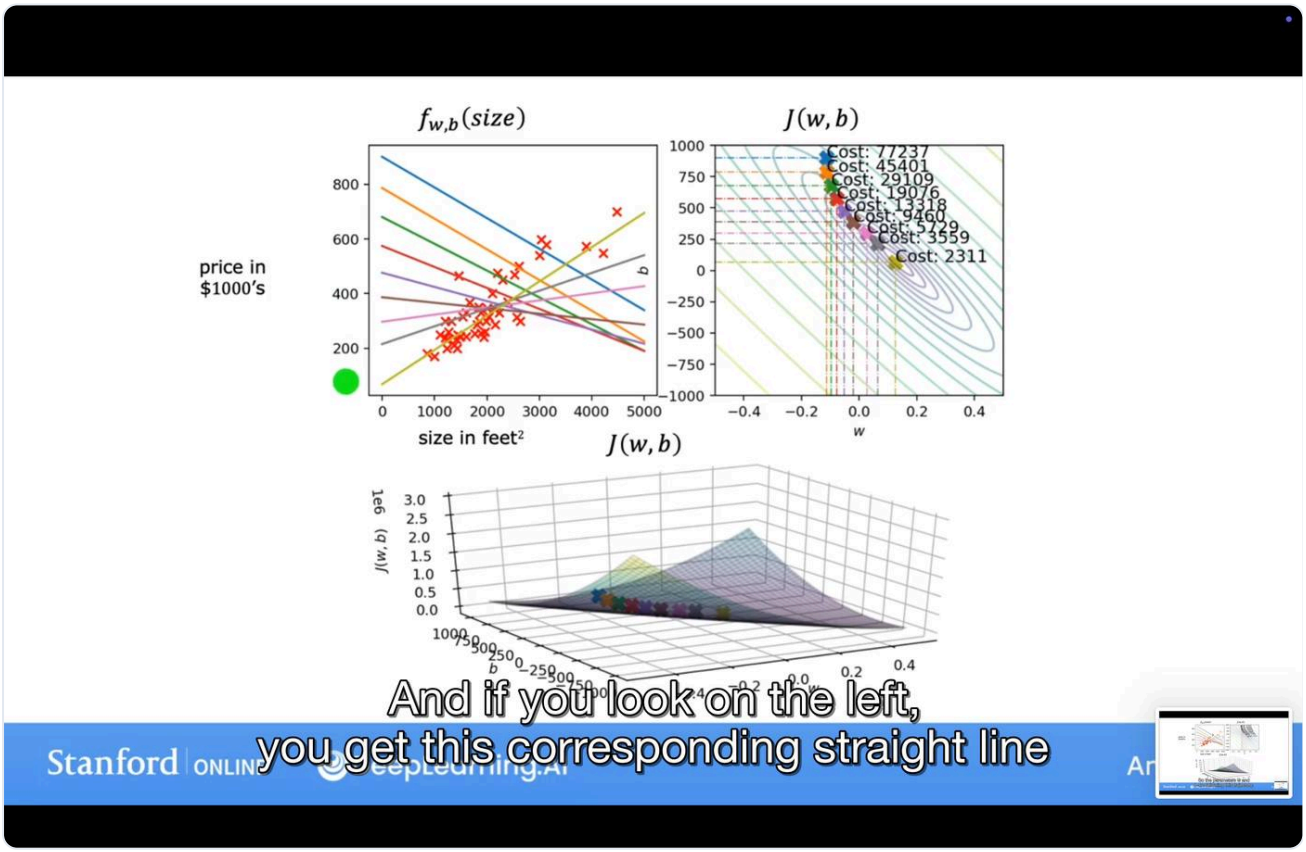


FIGURE 20.8

And if you look on the left, you get this corresponding straight line fit that fits the data better and better until we've reached the global minimum. The global minimum corresponds to this straight line fit, which is a relatively good fit to the data. I mean, isn't that cool. And so that's gradient descent and we're going to use this to fit a model to the holding data. And you can now use this f(x) model to predict the price of your clients house or anyone else's house.

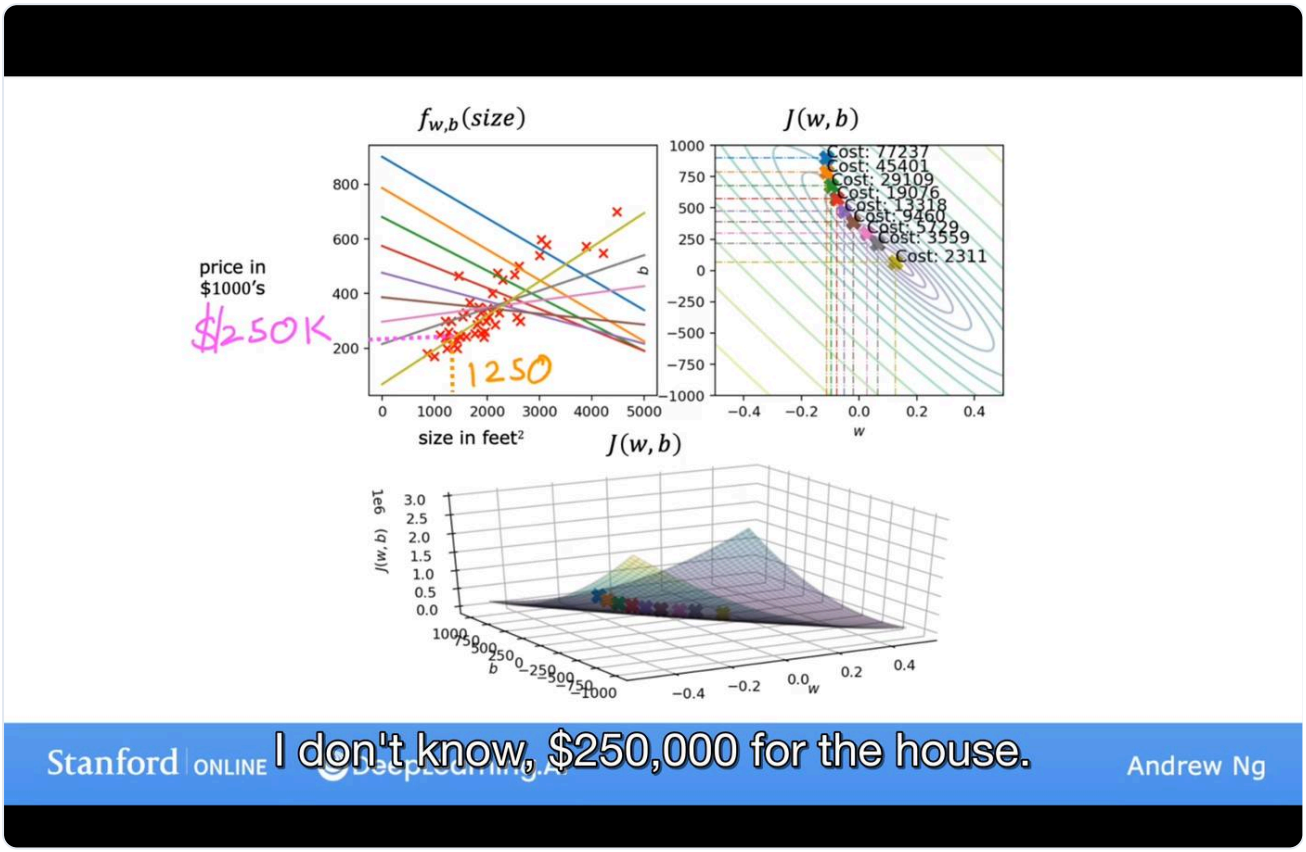


FIGURE 20.9

For instance, if your friend's house size is 1250 square feet, you can now read off the value and predict that maybe they could get, I don't know, $250,000 for the house.

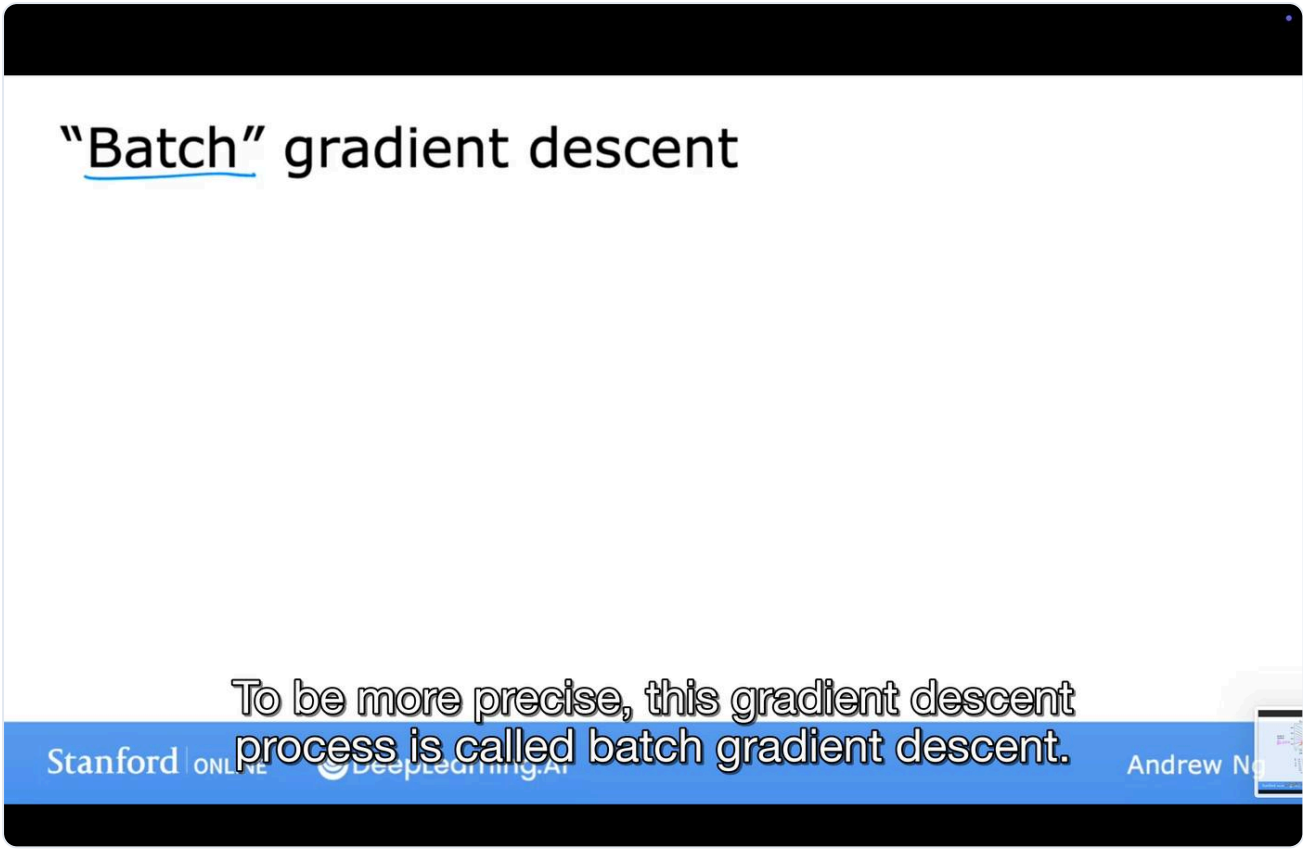


FIGURE 20.10 | To Be More Precise, This Graclient Descent

To be more precise, this gradient descent process is called batch gradient descent.

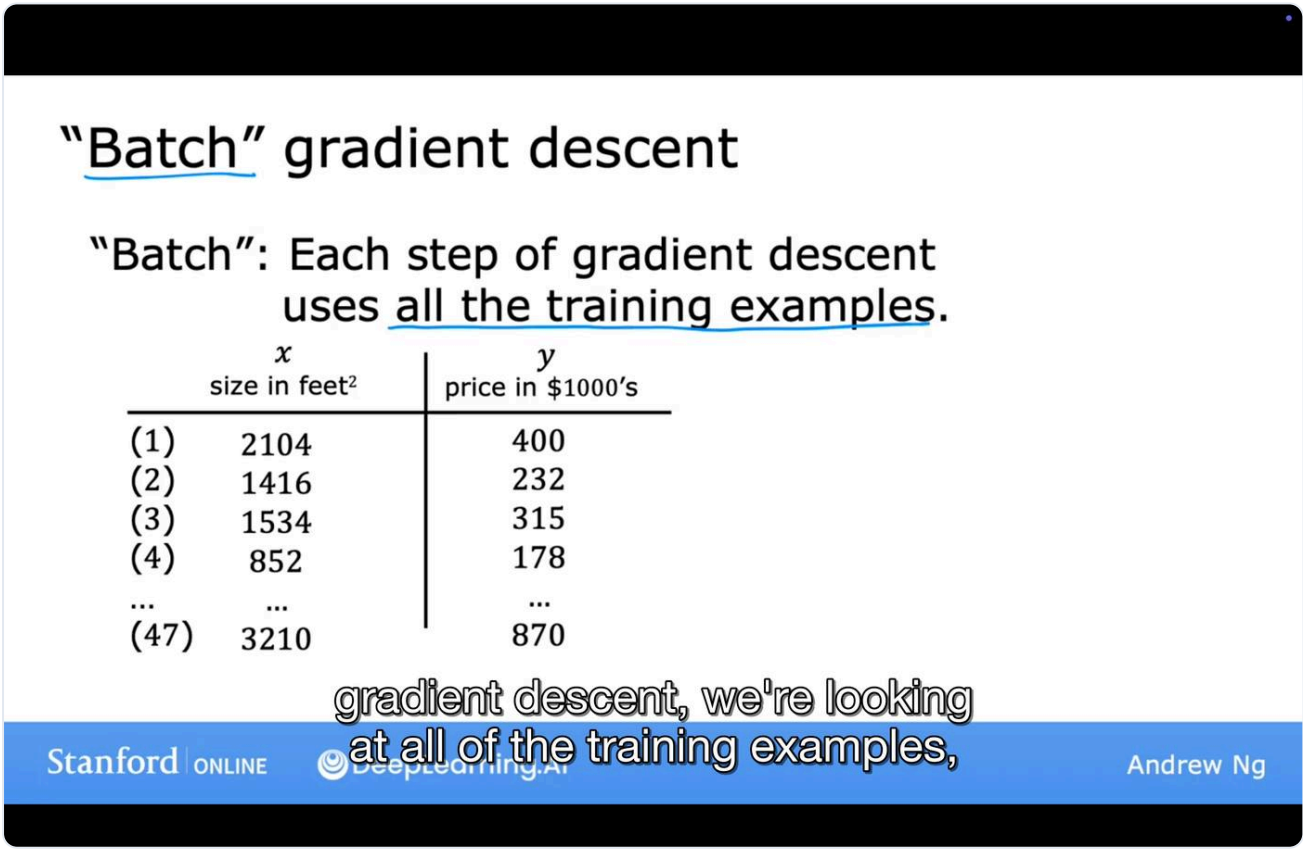


FIGURE 20.11

The term batch gradient descent refers to the fact that on every step of gradient descent, we're looking at all of the training examples, instead of just a subset of the training data. So in computing grading descent, when computing derivatives, when computing the sum from i =1 to m.

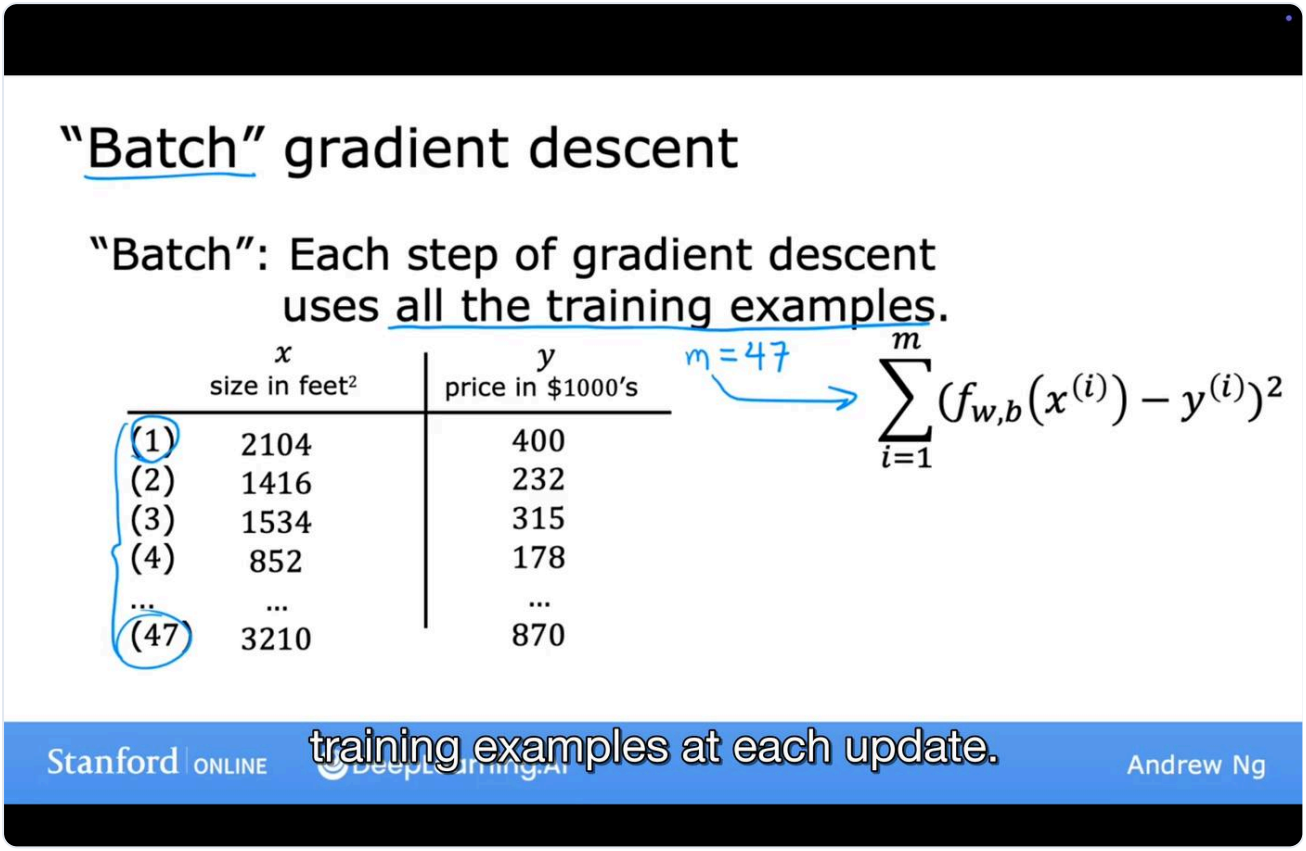


FIGURE 20.12

And bash gradient descent is looking at the entire batch of training examples at each update. I know that bash grading percent may not be the most intuitive name, but this is what people in the machine learning community call it. If you've heard of the newsletter The Batch, that's published by DeepLearning.AI. The newsletter The batch was also named for this concept in machine learning. And then it turns out that there are other versions of gradient descent that do not look at the entire training set, but instead looks at smaller subsets of the training data at each update step. But we'll use batch gradient descent for linear regression. So that's it for linear regression.

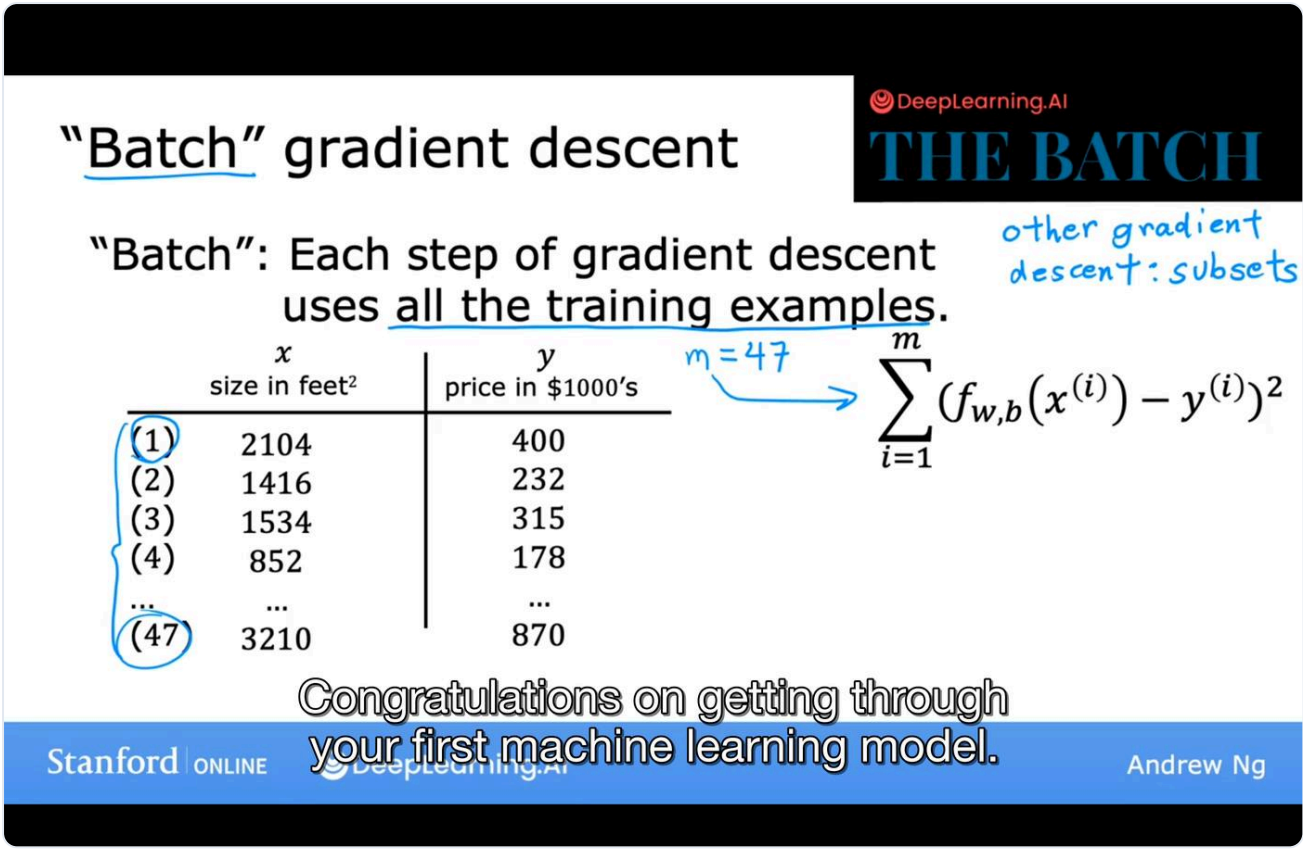


FIGURE 20.13

Congratulations on getting through your first machine learning model. I hope you go and celebrate or I don't know maybe take a nap in your hammock. In the optional lab that follows this video. You'll see a review of the gradient descent algorithm as was how to implement it in code. You'll also see a plot that shows how the cost decreases as you continue training more iterations. And you'll also see a contour plot, seeing how the cost gets closer to the global minimum as gradient descent finds better and better values for the parameters w and b. So remember that to do the optional lab. You just need to read and run this code. You will need to write any code yourself and I hope you take a few moments to do that.

And also become familiar with the gradient descent code because this will help you to implement this and similar algorithms in the future yourself. Thanks for sticking with me through the end of this last video for the first week and congratulations for making it all the way here. You're on your way to becoming a machine learning person. In addition to the optional labs, if you haven't done so yet. I hope you also check out the practice quizzes, which are a nice way that you can double check your own understanding of the concepts. It's also totally fine, if you don't get them all right the first time. And you can also take the quizzes multiple times until you get the score that you want.

You now know how to implement linear regression with one variable and that brings us to the close of this week. Next week, we'll learn to make linear regression much more powerful instead of one feature like size of a house, you learn how to get it to work with lots of features. You'll also learn how to get it to fit nonlinear curves. These improvements will make the algorithm much

more useful and valuable. Lastly, we'll also go over some practical tips that will really hope for getting linear regression to work on practical applications. I'm really happy to have you here with me in this class and I look forward to seeing you next week.

LECTURE 21

Multiple Features (Variables)

Welcome back. In this week, we'll learn to make linear regression much faster and much more powerful, and by the end of this week, you'll be two thirds of the way to finishing this first course. Let's start by looking at the version of linear regression that look at not just one feature, but a lot of different features. Let's take a look. In the original version of linear regression, you had a single feature x, the size of the house and you're able to predict y, the price of the house. The model was fwb of x equals wx plus b.

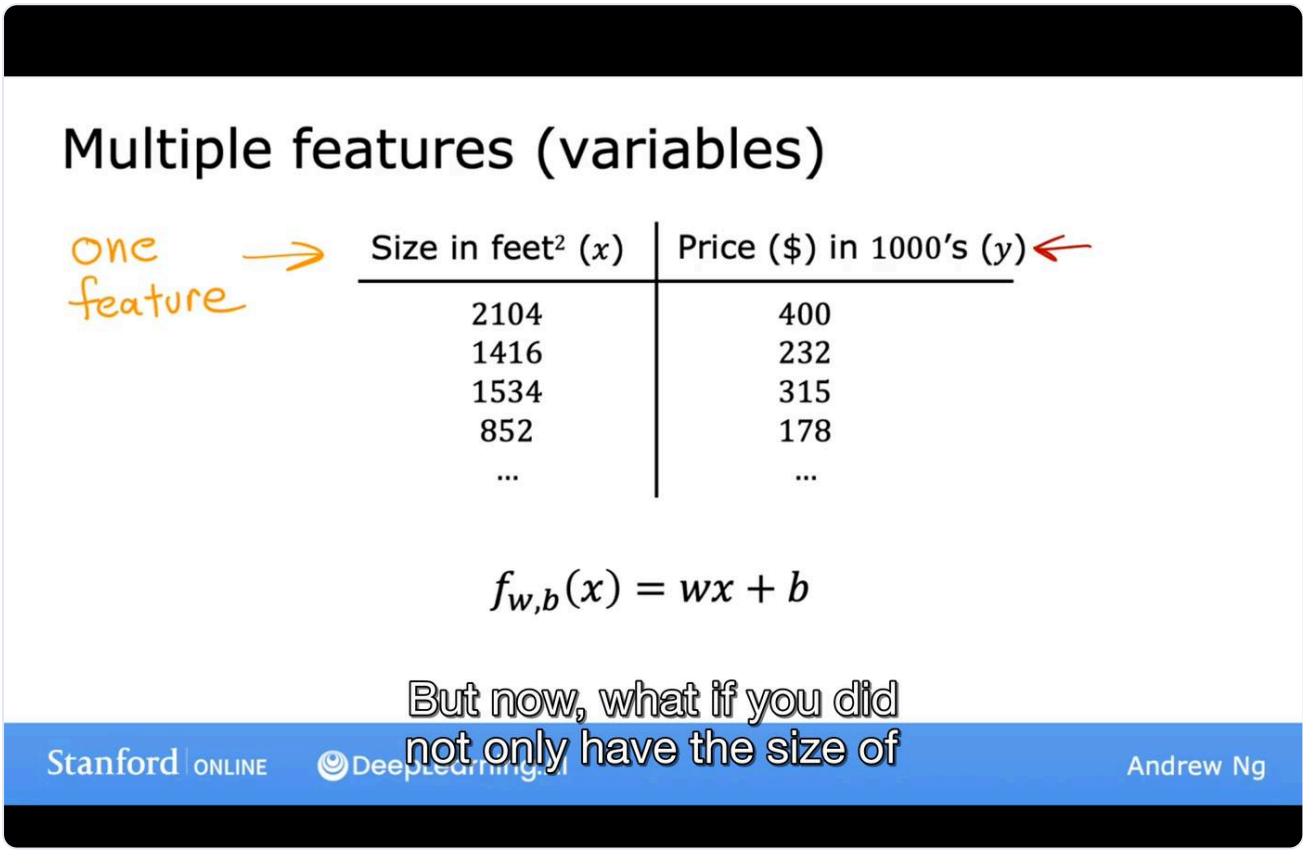


FIGURE 21.1 | Multiple Features (Variables)

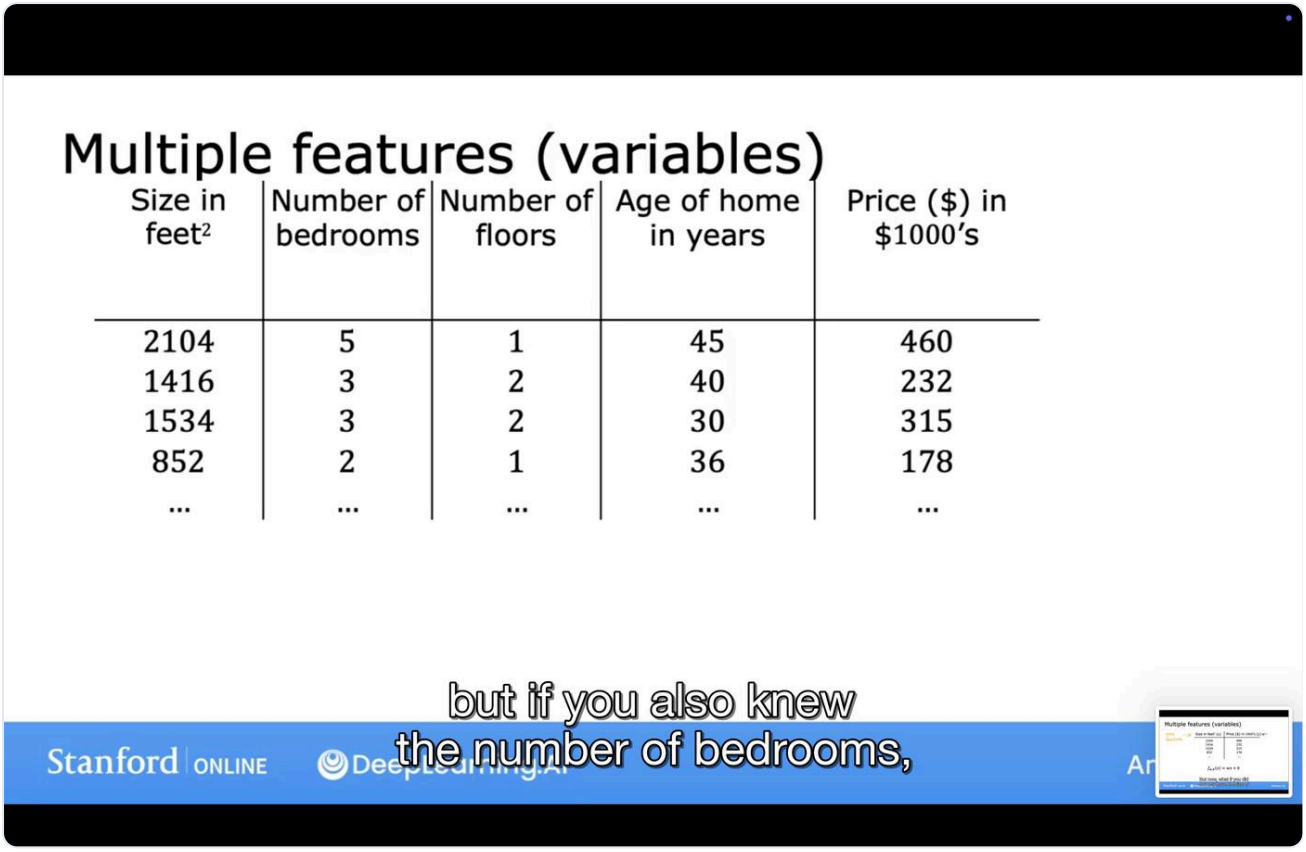


FIGURE 21.2

But now, what if you did not only have the size of the house as a feature with which to try to predict the price, but if you also knew the number of bedrooms, the number of floors and the age of the home in years. It seems like this would give you a lot more information with which to predict the price. To introduce a little bit of new notation, we're going to use the variables X_1, X_2, X_3 and X_4, to denote the four features. For simplicity, let's introduce a little bit more notation.

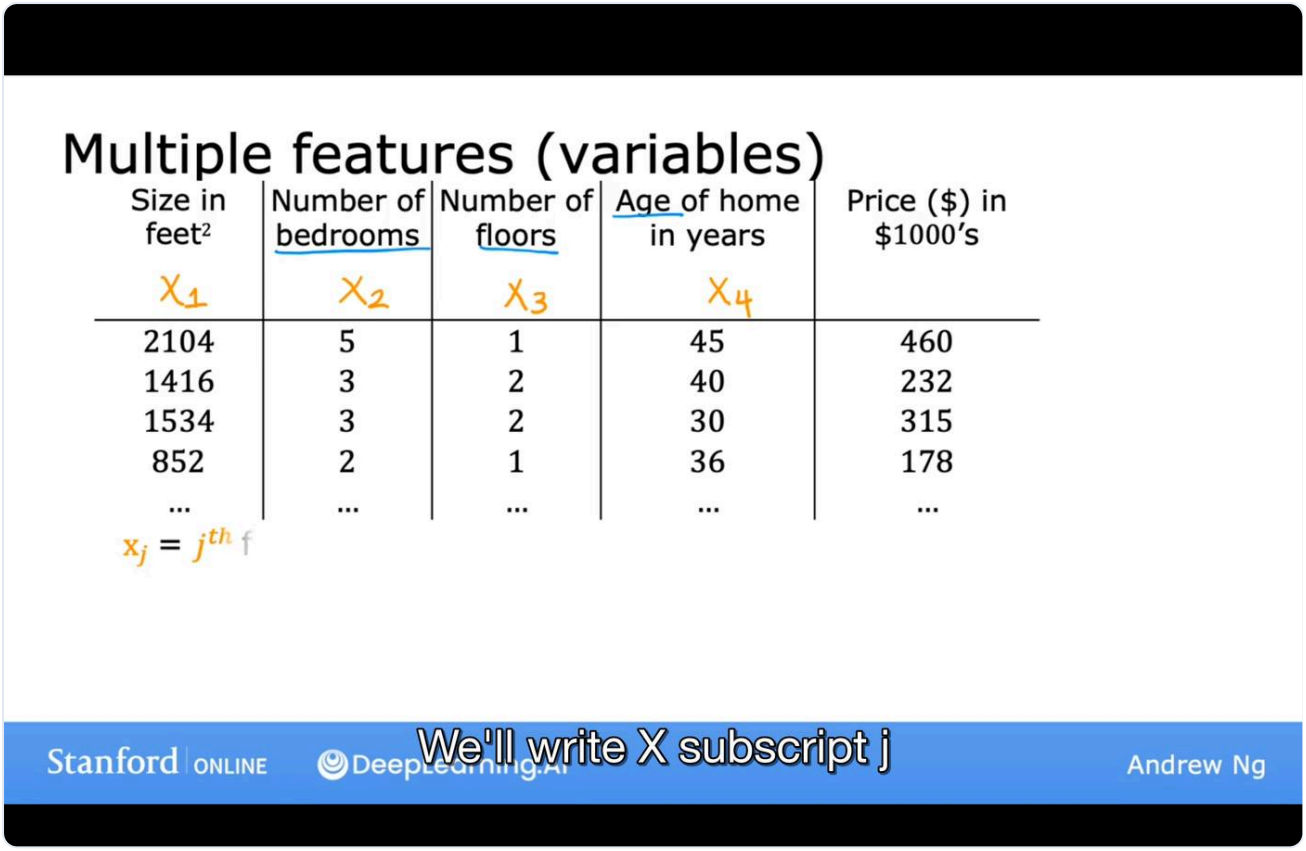


FIGURE 21.3

We'll write X subscript j or sometimes I'll just say for short, X sub j, to represent the list of features. Here, j will go from one to four, because we have four features.

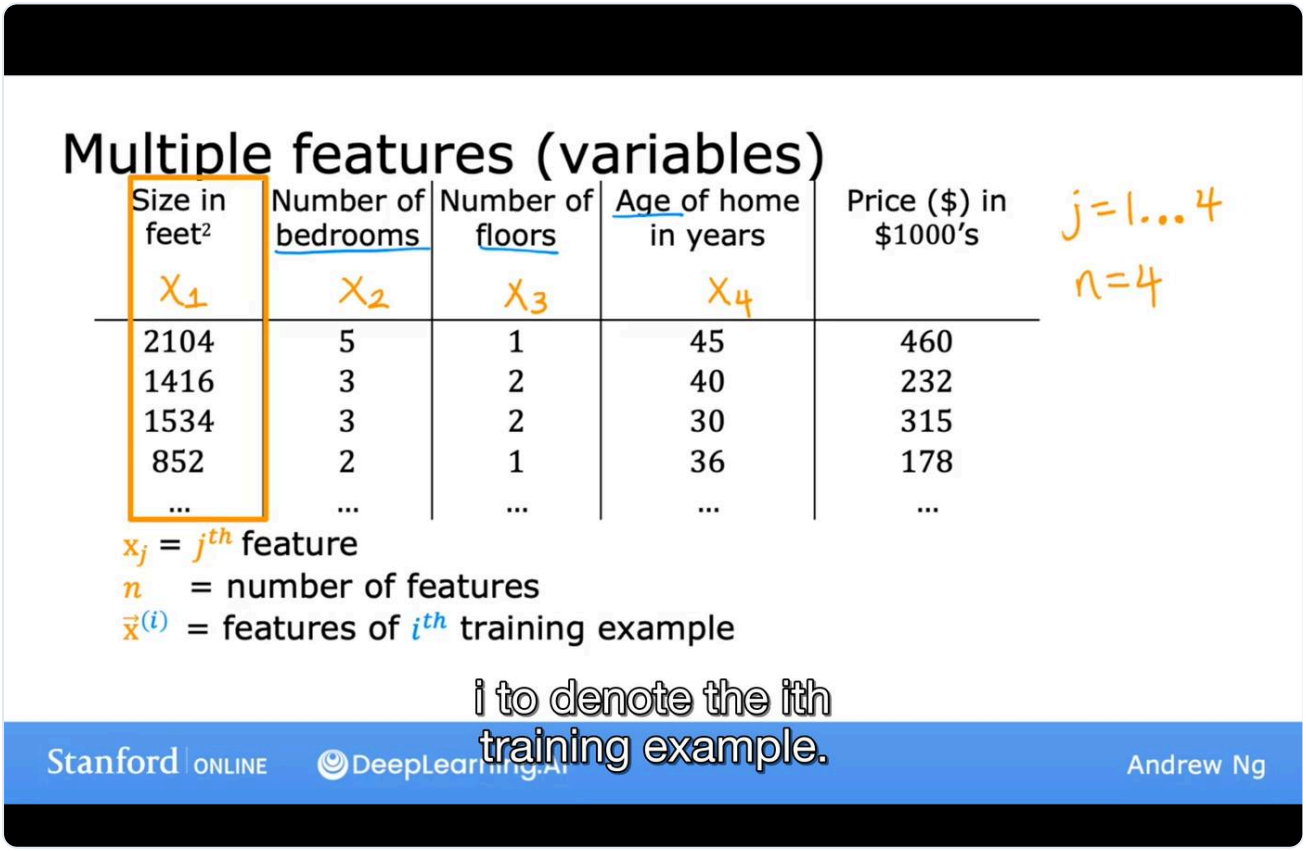


FIGURE 21.4

I'm going to use lowercase n to denote the total number of features, so in this example, n is equal to 4.

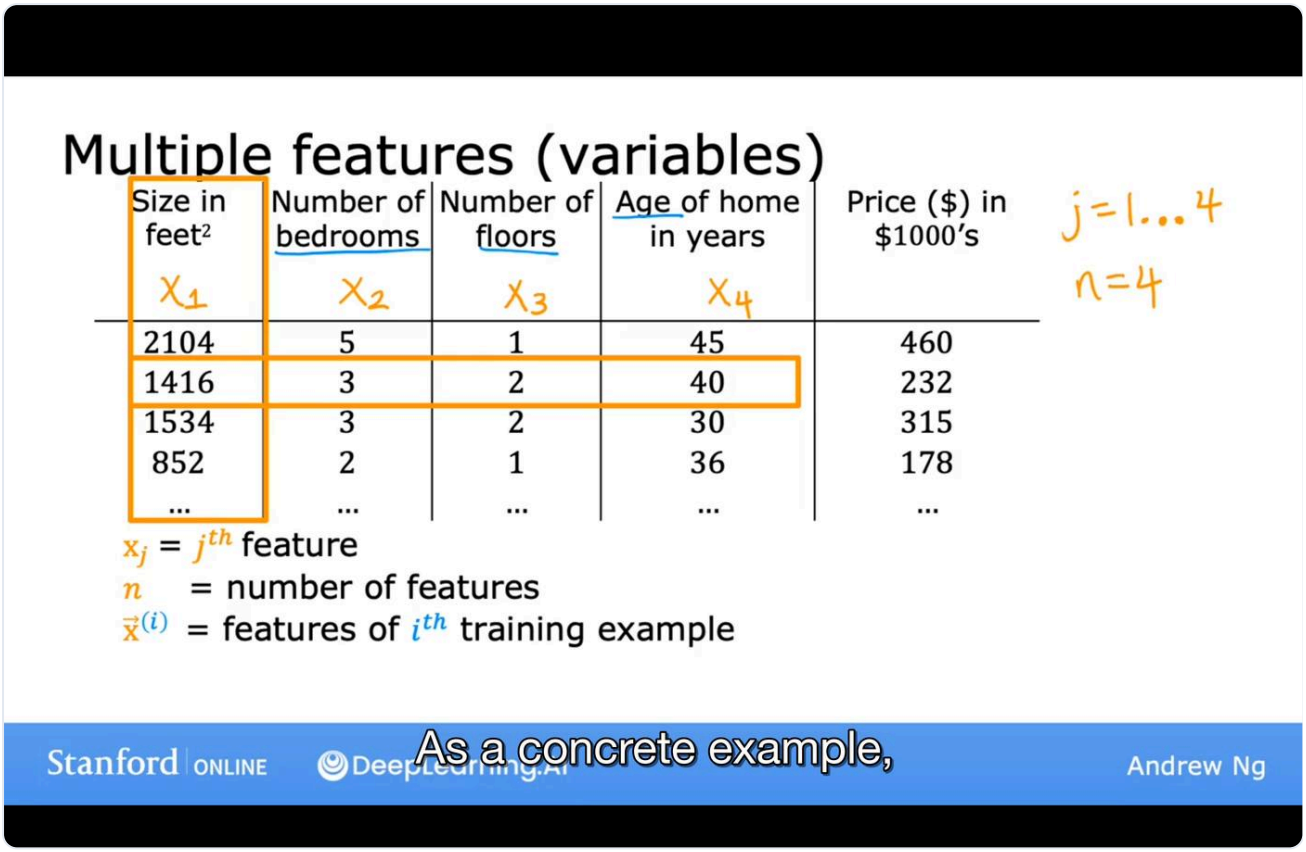


FIGURE 21.5

As before, we'll use X superscript i to denote the ith training example. Here X superscript i is actually going to be a list of four numbers, or sometimes we'll call this a vector that includes all the features of the ith training example.

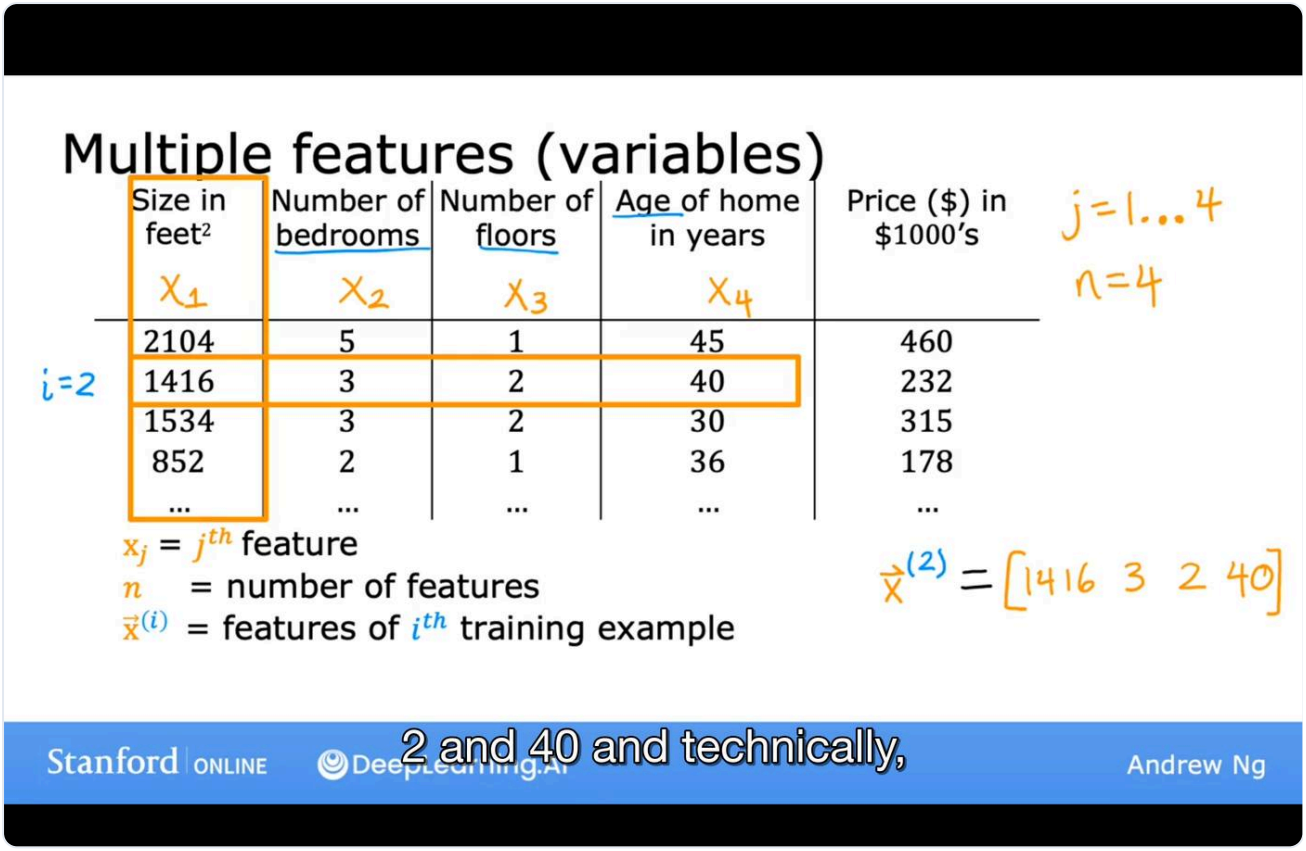


FIGURE 21.6

As a concrete example, X superscript in parentheses 2, will be a vector of the features for the second training example, so it will equal to this 1416, 3, 2 and 40 and technically, I'm writing these numbers in a row, so sometimes this is called a row vector rather than a column vector. But if you don't know what the difference is, don't worry about it, it's not that important for this purpose.

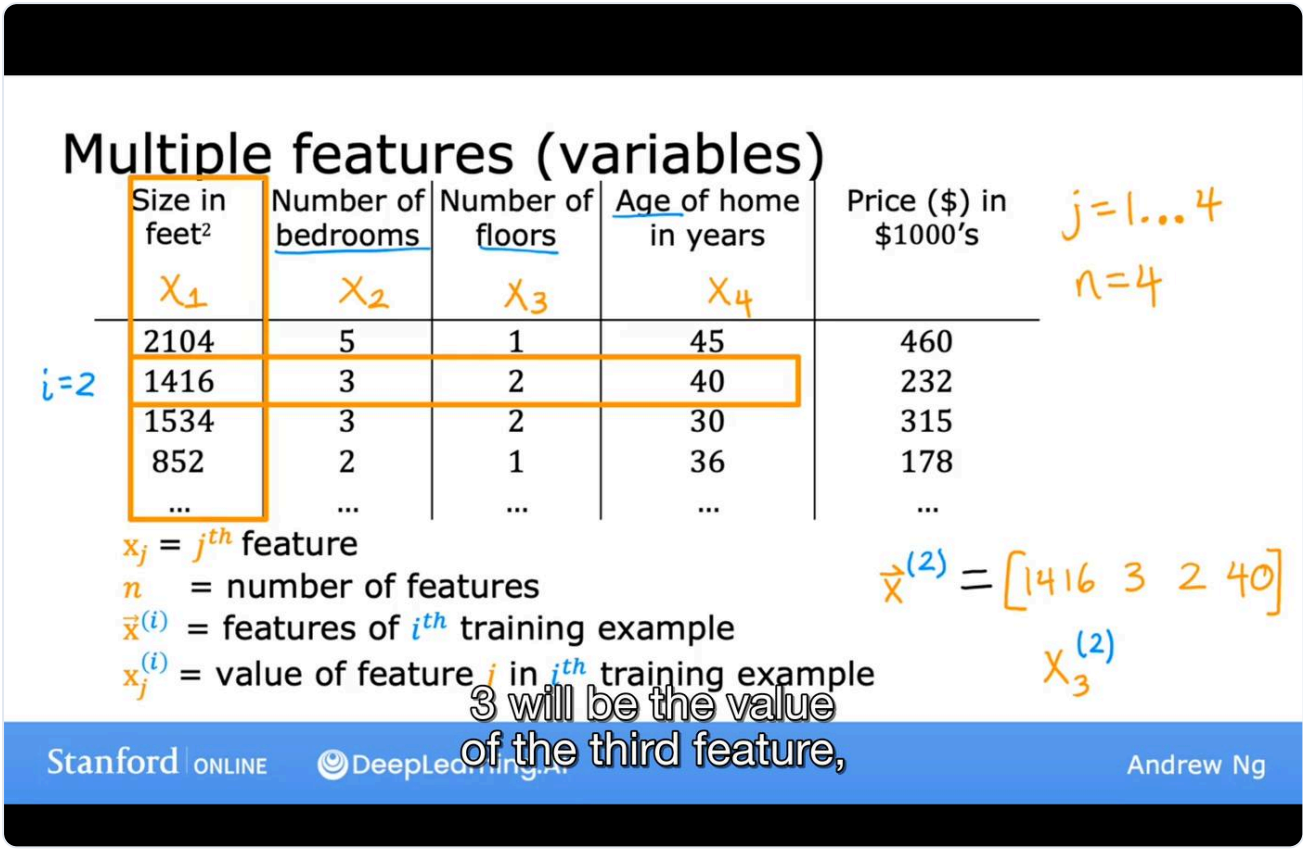


FIGURE 21.7

To refer to a specific feature in the ith training example, I will write X superscript i, subscript j, so for example, X superscript 2 subscript 3 will be the value of the third feature, that is the number of floors in the second training example and so that's going to be equal to 2. Sometimes in order to emphasize that this X^2 is not a number but is actually a list of numbers that is a vector, we'll draw an arrow on top of that just to visually show that is a vector and over here as well, but you don't have to draw this arrow in your notation. You can think of the arrow as an optional signifier. They're sometimes used just to emphasize that this is a vector and not a number.

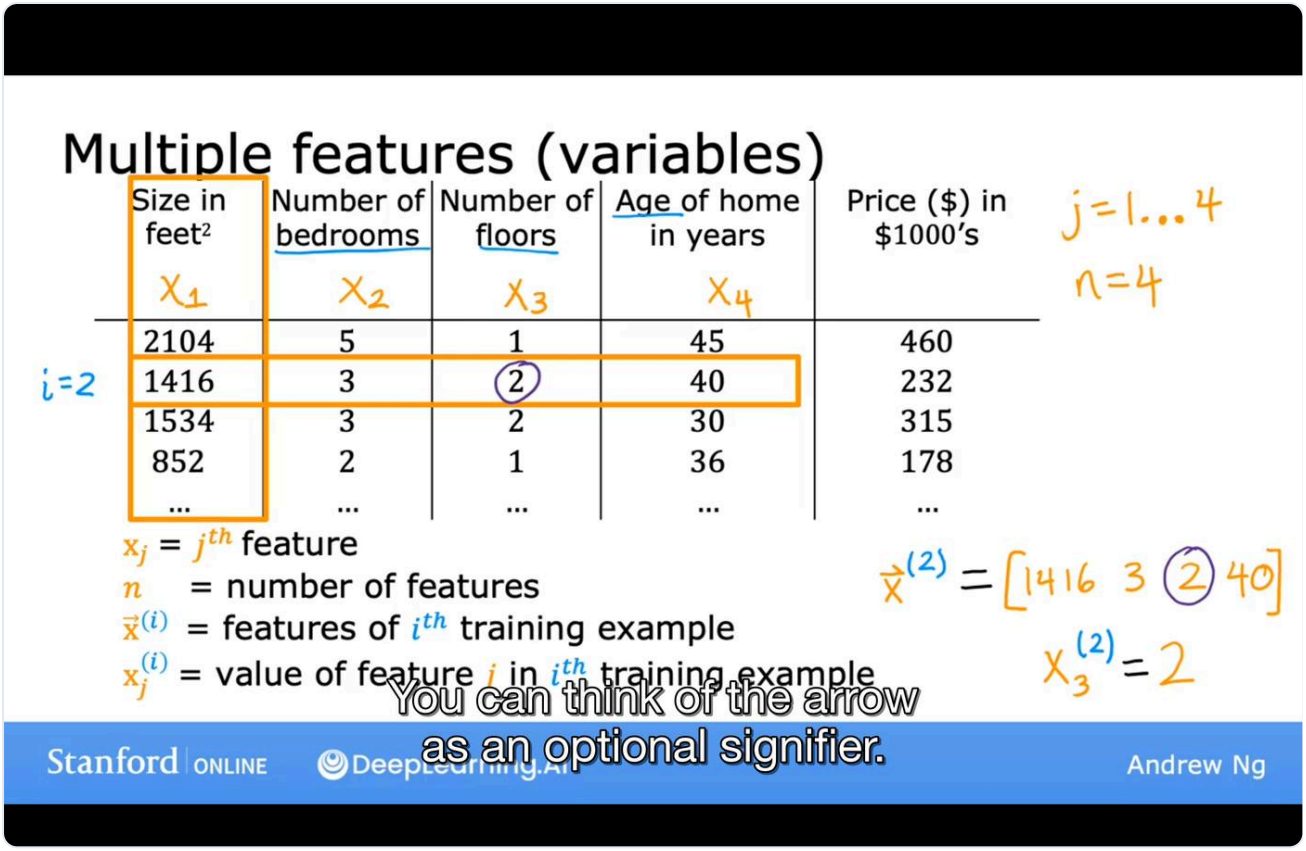


FIGURE 21.8

Now that we have multiple features, let's take a look at what a model would look like. Previously, this is how we defined the model, where X was a single feature, so a single number. But now with multiple features, we're going to define it differently. Instead, the model will be, fwb of X equals w1x1 plus w2x2 plus w3x3 plus w4x4 plus b. Concretely for housing price prediction, one possible model may be that we estimate the price of the house as 0.1 times X_1, the size of the house, plus four times X_2, the number of bedrooms, plus ten times X_3, the number of floors, minus 2 times X_4, the age of the house in years plus 80.

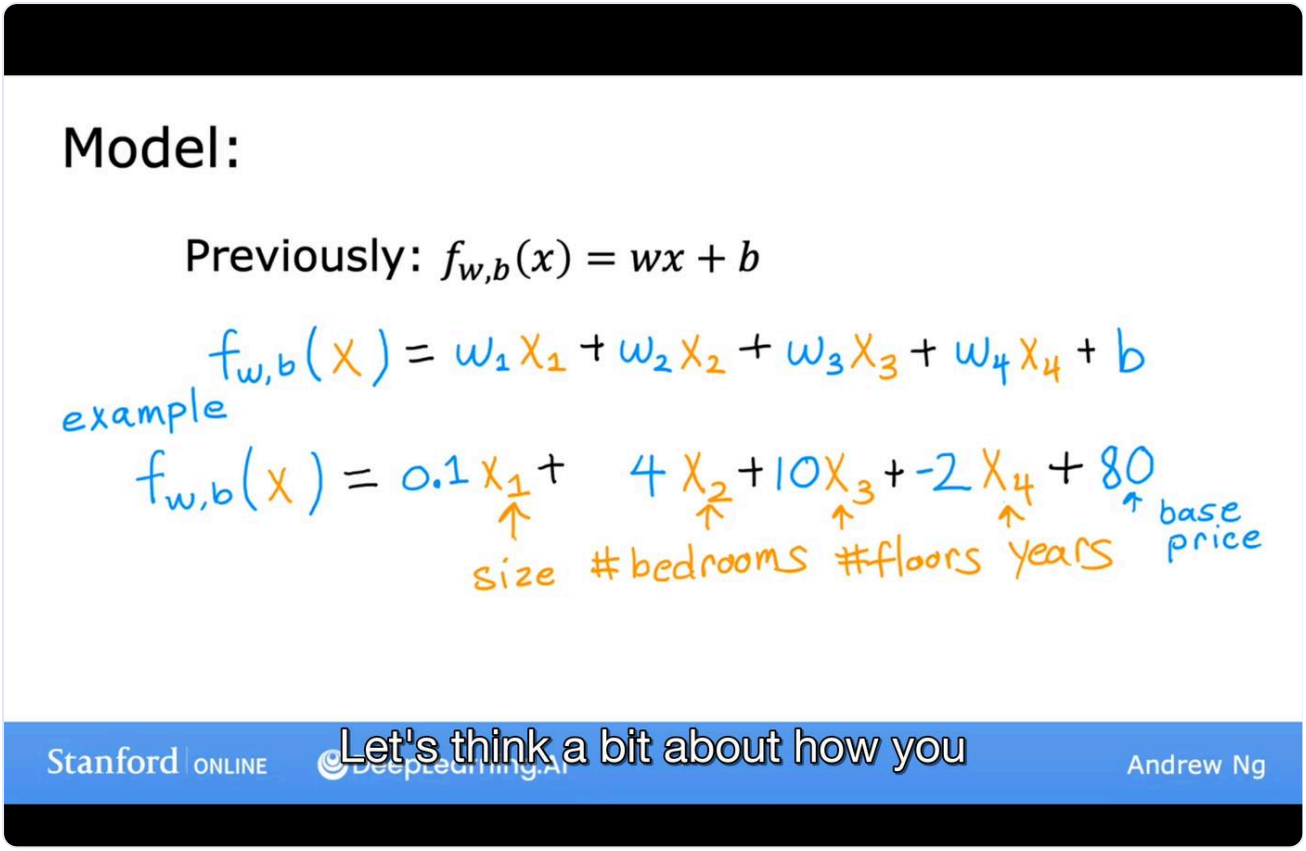


FIGURE 21.9

Let's think a bit about how you might interpret these parameters. If the model is trying to predict the price of the house in thousands of dollars, you can think of this b equals 80 as saying that the base price of a house starts off at maybe $80,000, assuming it has no size, no bedrooms, no floor and no age. You can think of this 0.1 as saying that maybe for every additional square foot, the price will increase by 0.1 $1,000 or by $100, because we're saying that for each square foot, the price increases by 0.1, times $1,000, which is $100.

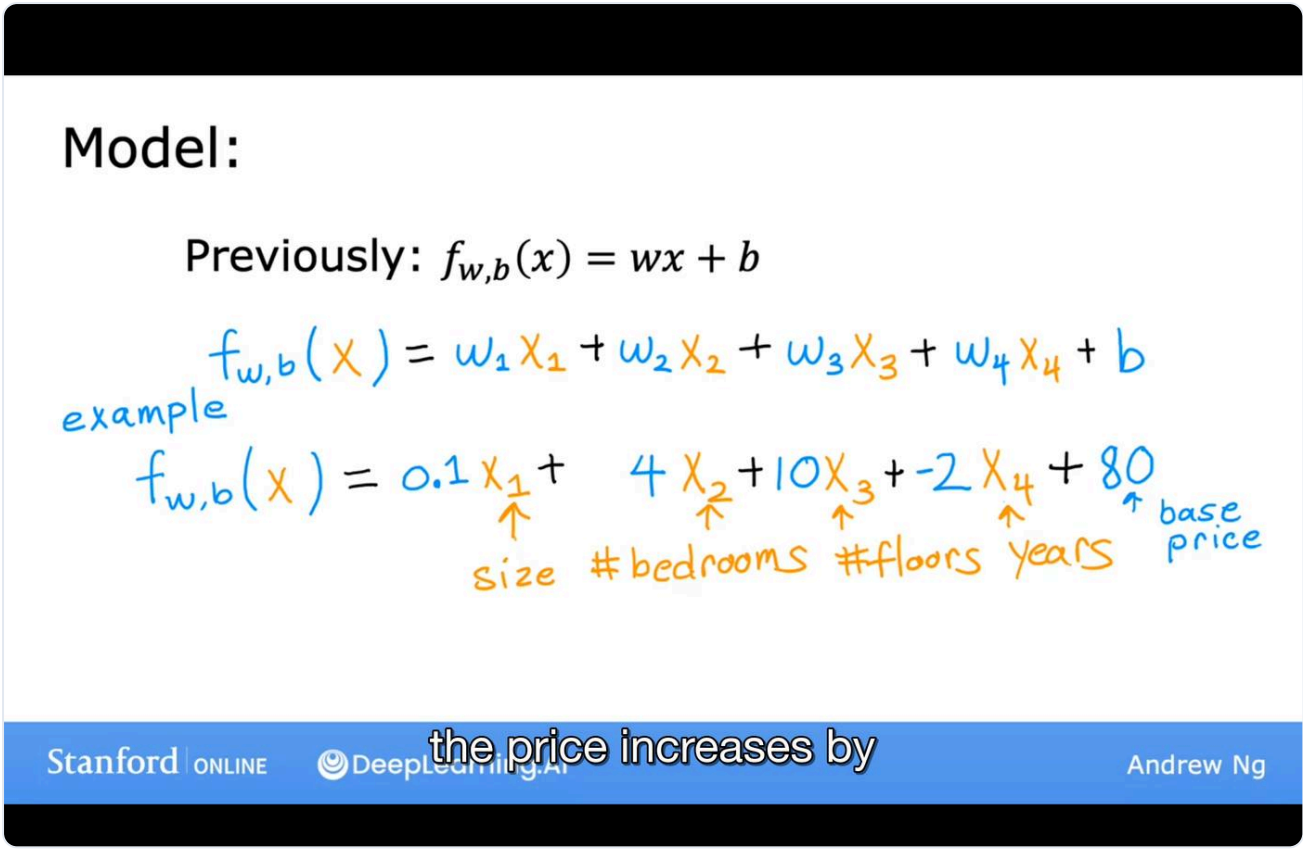


FIGURE 21.10

Maybe for each additional bathroom, the price increases by $4,000 and for each additional floor the price may increase by $10,000 and for each additional year of the house's age, the price may decrease by $2,000, because the parameter is negative 2. In general, if you have n features, then the model will look like this. Here again is the definition of the model with n features.

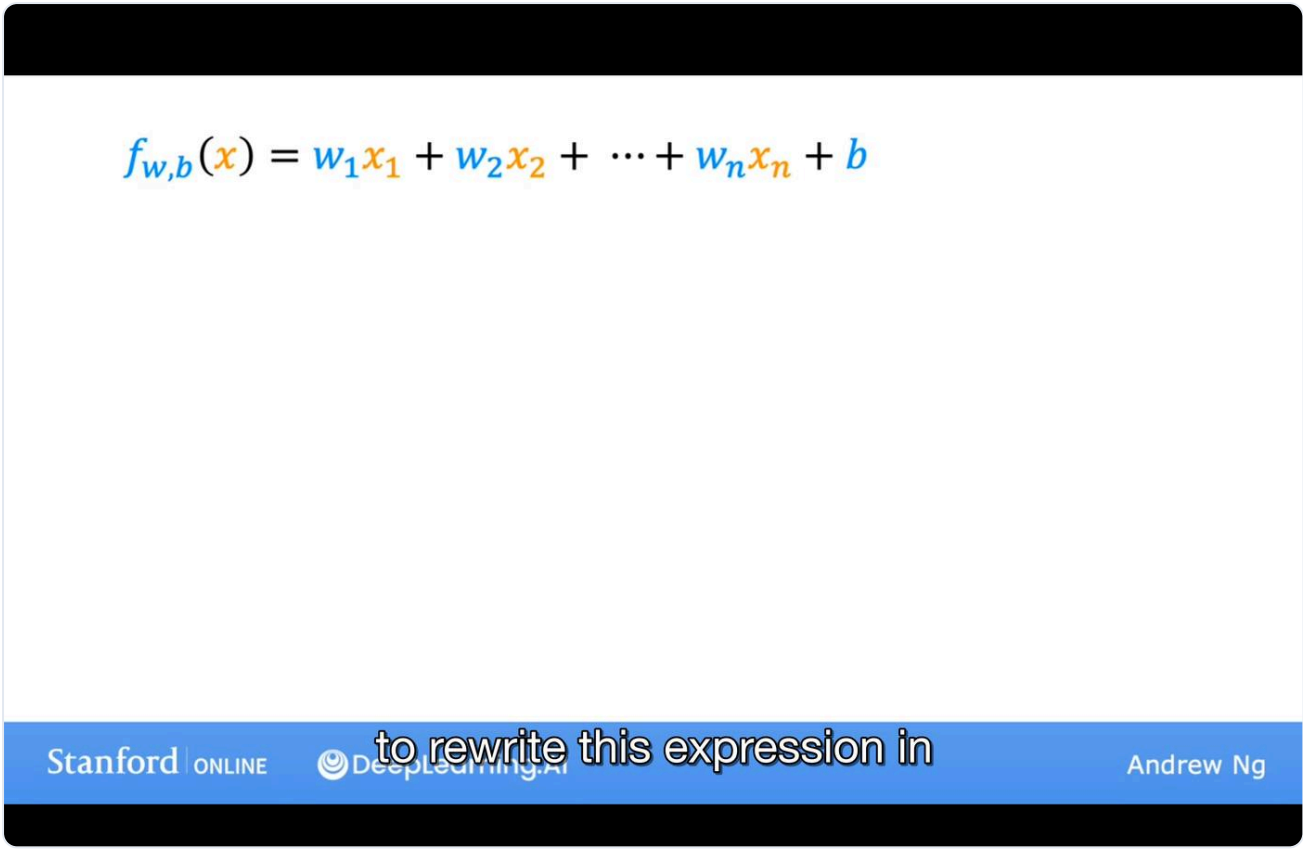


FIGURE 21.11

What we're going to do next is introduce a little bit of notation to rewrite this expression in a simpler but equivalent way. Let's define W as a list of numbers that list the parameters W_1, W_2, W_3, all the way through W_n. In mathematics, this is called a vector and sometimes to designate that this is a vector, which just means a list of numbers, I'm going to draw a little arrow on top. You don't always have to draw this arrow and you can do so or not in your own notation, so you can think of this little arrow as just an optional signifier to remind us that this is a vector. If you've taken the linear algebra class before, you might recognize that this is a row vector as opposed to a column vector.

But if you don't know what those terms means, you don't need to worry about it. Next, same as before, b is a single number and not a vector and so this vector W together with this number b are the parameters of the model. Let me also write X as a list or a vector, again a row vector that lists all of the features X_1, X_2, X_3 up to X_n, this is again a vector, so I'm going to add a little arrow up on top to signify.

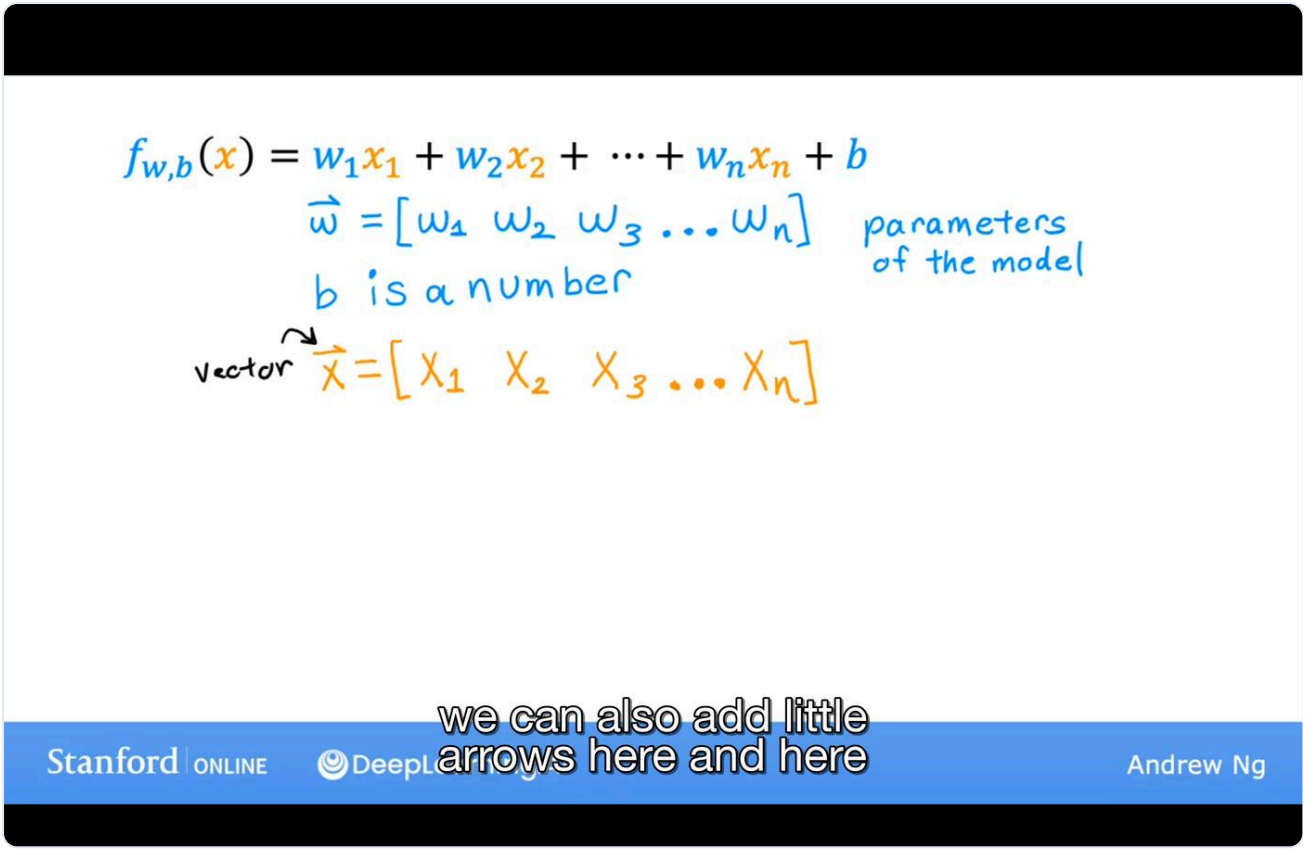


FIGURE 21.12

In the notation up on top, we can also add little arrows here and here to signify that that W and that X are actually these lists of numbers, that they're actually these vectors.

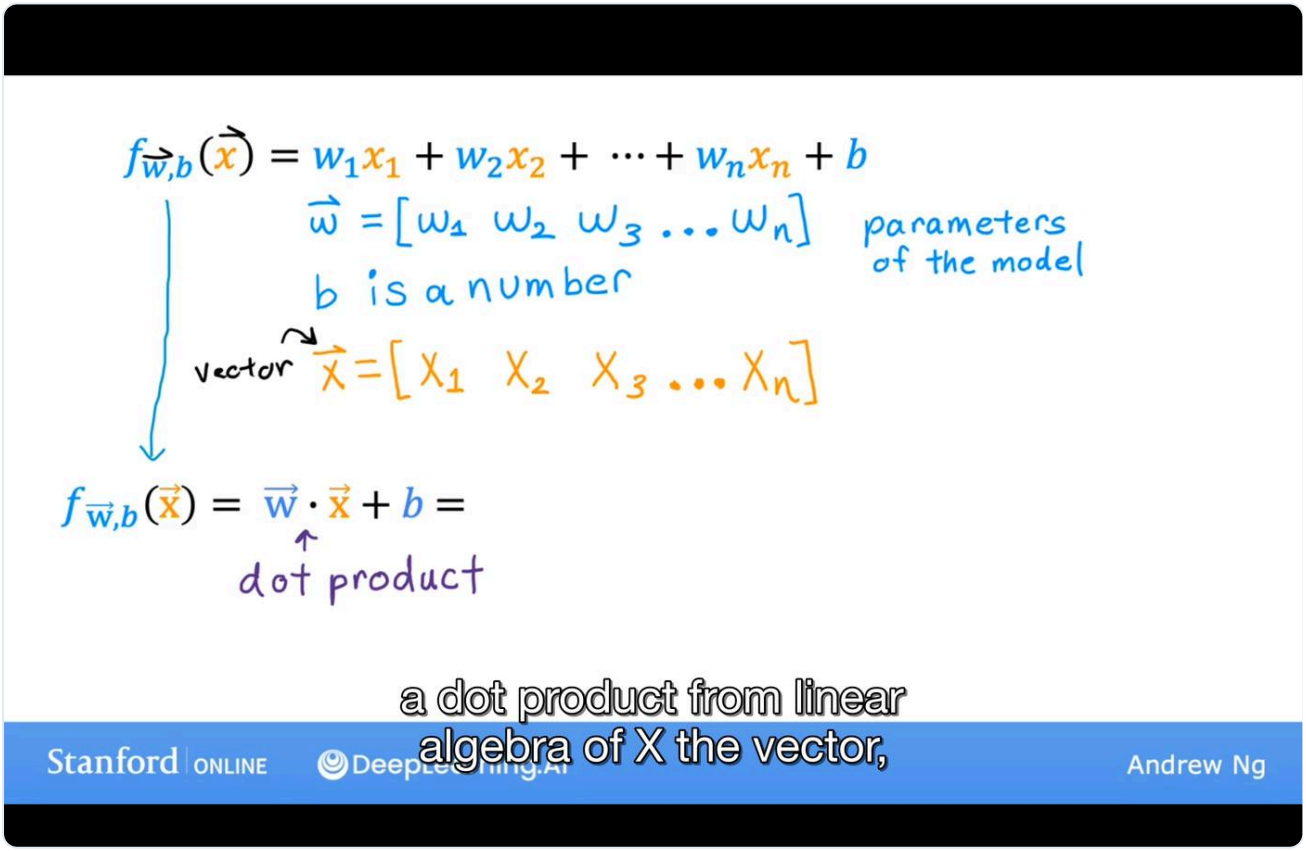


FIGURE 21.13

With this notation, the model can now be rewritten more succinctly as f of x equals, the vector w dot and this dot refers to a dot product from linear algebra of X the vector, plus the number b. What is this dot product thing? Well, the dot products of two vectors of two lists of numbers W and X, is computed by checking the corresponding pairs of numbers, W_1 and X_1 multiplying that, W_2 X_2 multiplying that, W_3 X_3 multiplying that, all the way up to W_n and X_n multiplying that and then summing up all of these products. Writing that out, this means that the dot products is equal to W_1X_1 plus W_2X_2 plus W_3X_3 plus all the way up to W_nX_n.

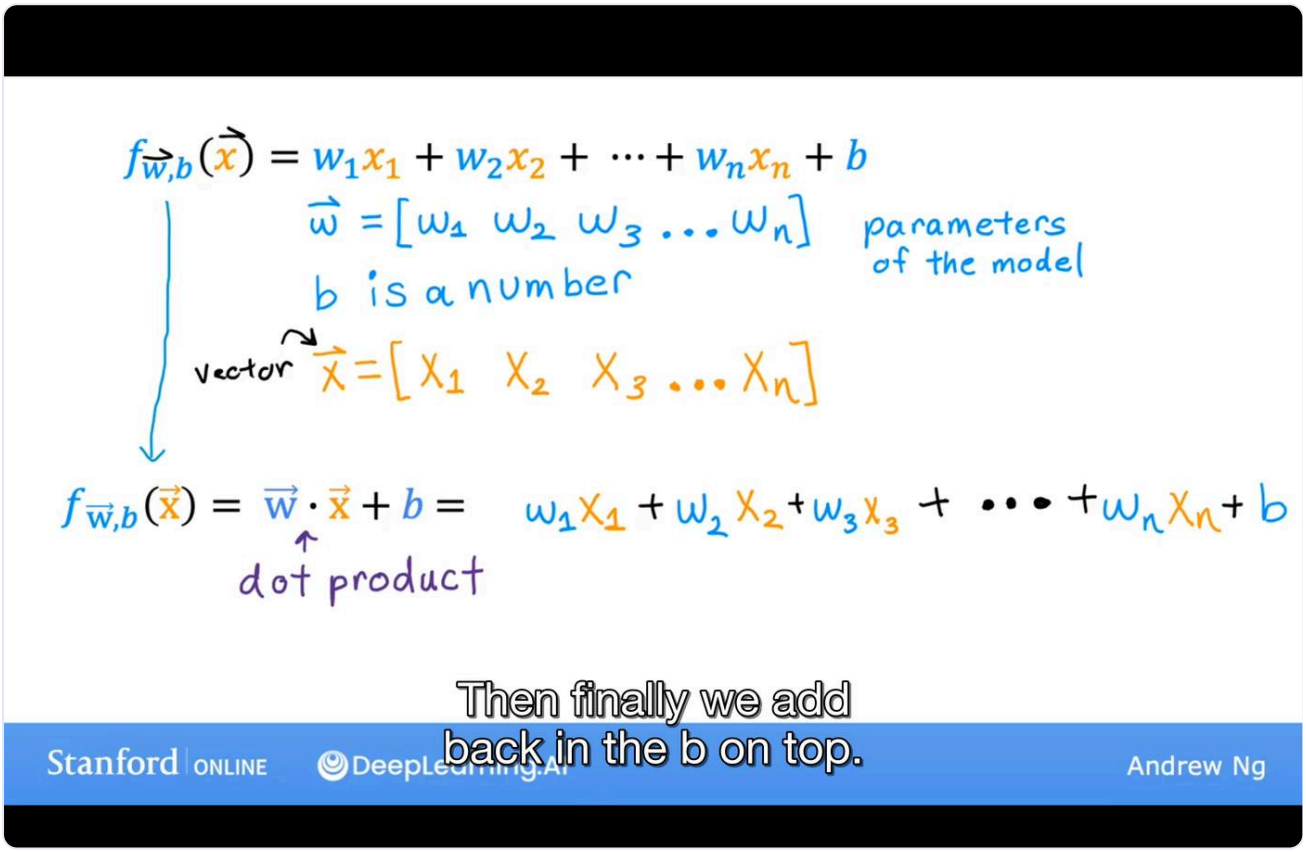


FIGURE 21.14

Then finally we add back in the b on top. You notice that this gives us exactly the same expression as we had on top. The dot traffic notation lets you write the model in a more compact form with fewer characters. The name for this type of linear regression model with multiple input features is multiple linear regression.

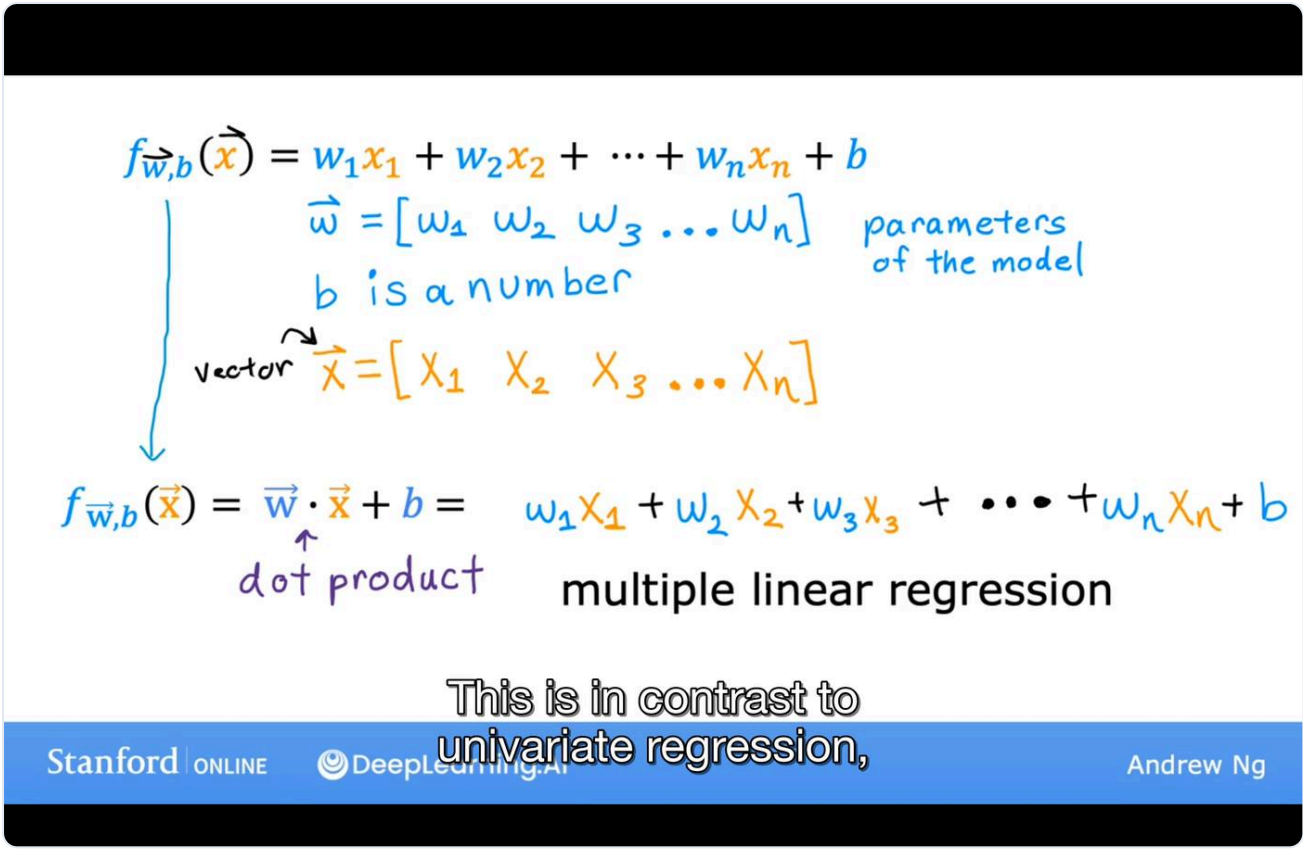


FIGURE 21.15

This is in contrast to univariate regression, which has just one feature. By the way, you might think this algorithm is called multivariate regression, but that term actually refers to something else that we won't be using here.

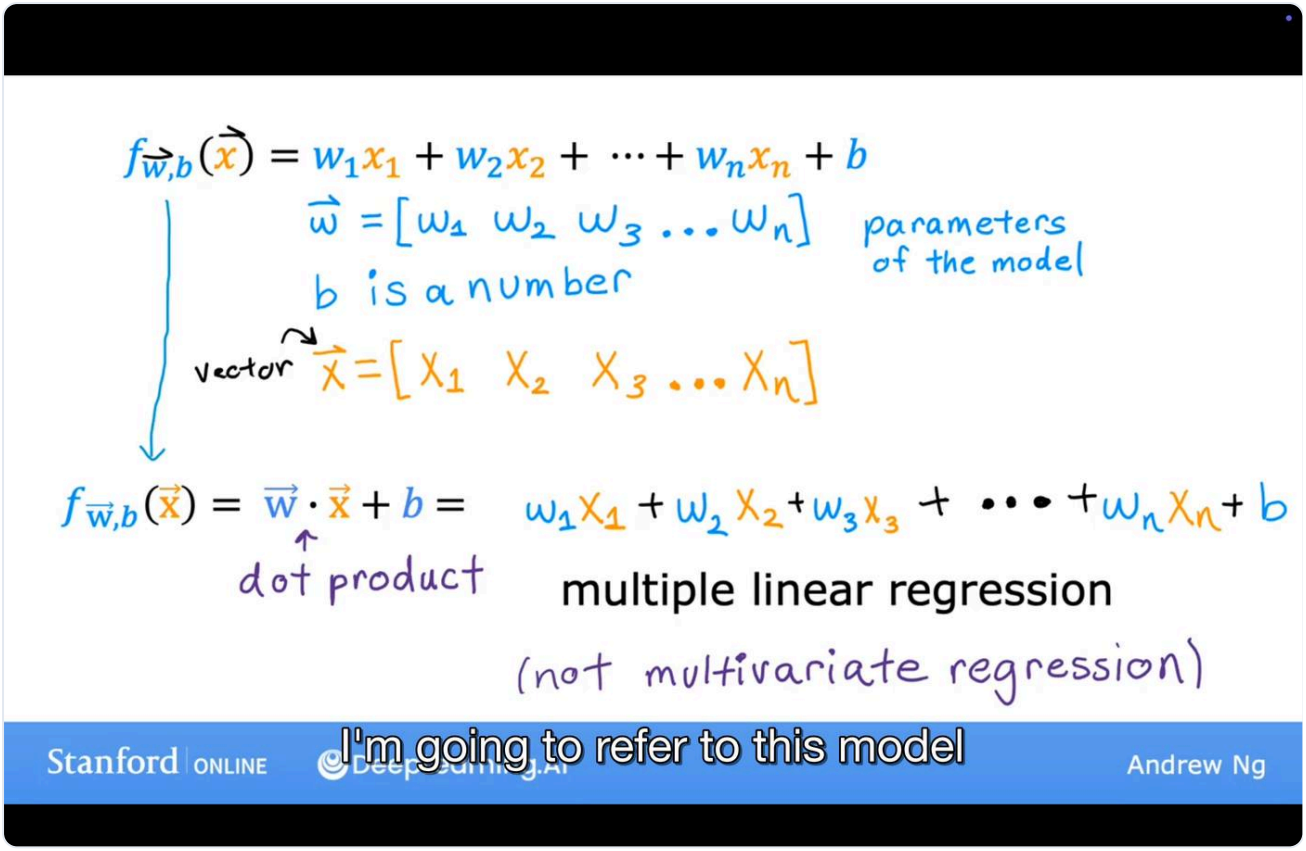


FIGURE 21.16

I'm going to refer to this model as multiple linear regression. That's it for linear regression with multiple features, which is also called multiple linear regression. In order to implement this, there's a really neat trick called vectorization, which will make it much simpler to implement this and many other learning algorithms. Let's go on to the next video to take a look at what is vectorization.

LECTURE 22

Parameters and Features Without Vectorization

In this video, you see a very useful idea called vectorization. When you're implementing a learning algorithm, using vectorization will both make your code shorter and also make it run much more efficiently. Learning how to write vectorized code will allow you to also take advantage of modern numerical linear algebra libraries, as well as maybe even GPU hardware that stands for graphics processing unit. This is hardware objectively designed to speed up computer graphics in your computer, but turns out can be used when you write vectorized code to also help you execute your code much more quickly. Let's look at a concrete example of what vectorization means.

Here's an example with parameters w and b, where w is a vector with three numbers, and you also have a vector of features x with also three numbers. Here n is equal to 3. Notice that in linear algebra, the index or the counting starts from 1 and so the first value is subscripted w1 and x1.

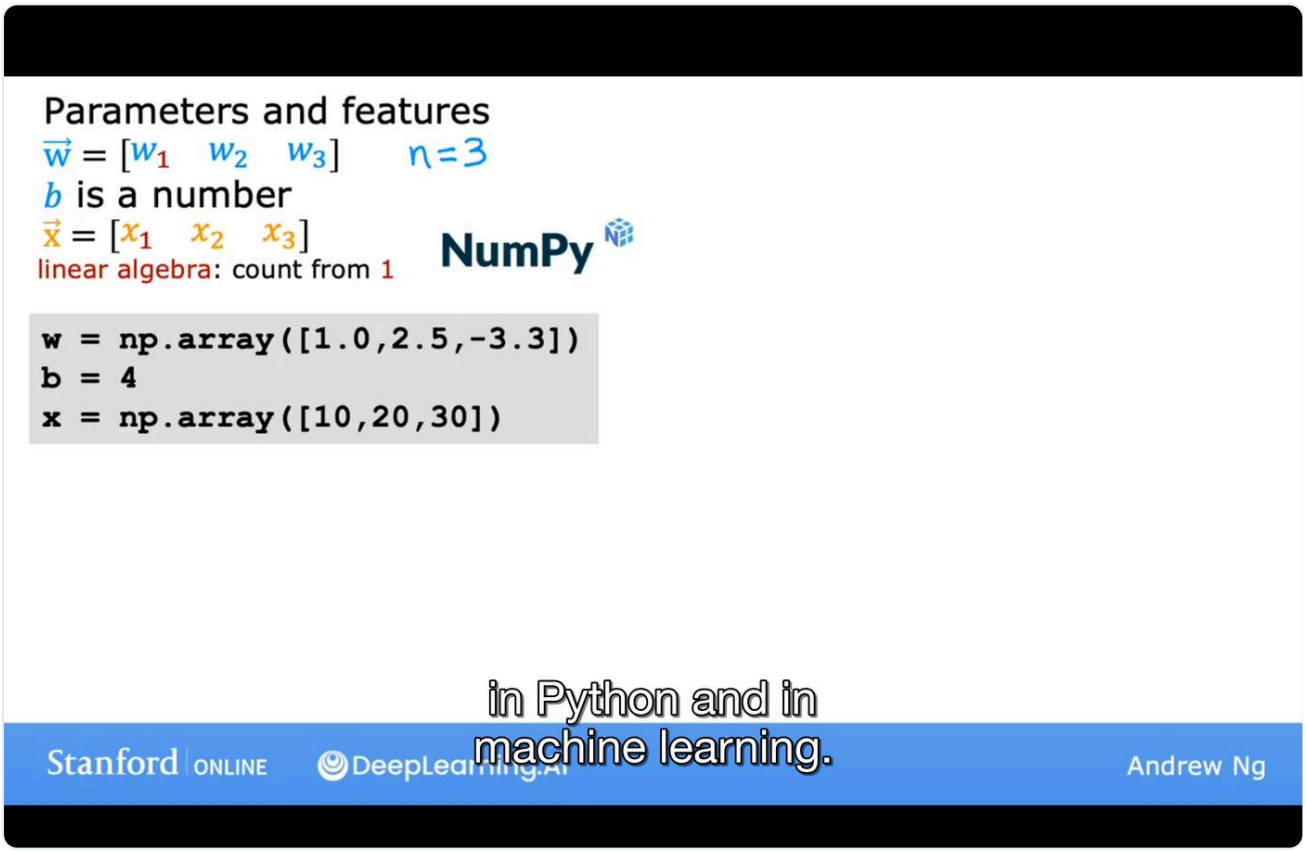


FIGURE 22.1 | Parameters and Features

In Python code, you can define these variables w, b, and x using arrays like this. Here, I'm actually using a numerical linear algebra library in Python called NumPy, which is by far the most widely used numerical linear algebra library in Python and in machine learning. Because in Python, the indexing of arrays while counting in arrays starts from 0, you would access the first value of w using w square brackets 0. The second value using w square bracket 1, and the third

and using w square bracket 2. The indexing here, it goes from 0,1 to 2 rather than 1, 2 to 3. Similarly, to access individual features of x, you will use x0, x1, and x2. Many programming languages including Python start counting from 0 rather than 1.

Now, let's look at an implementation without vectorization for computing the model's prediction.

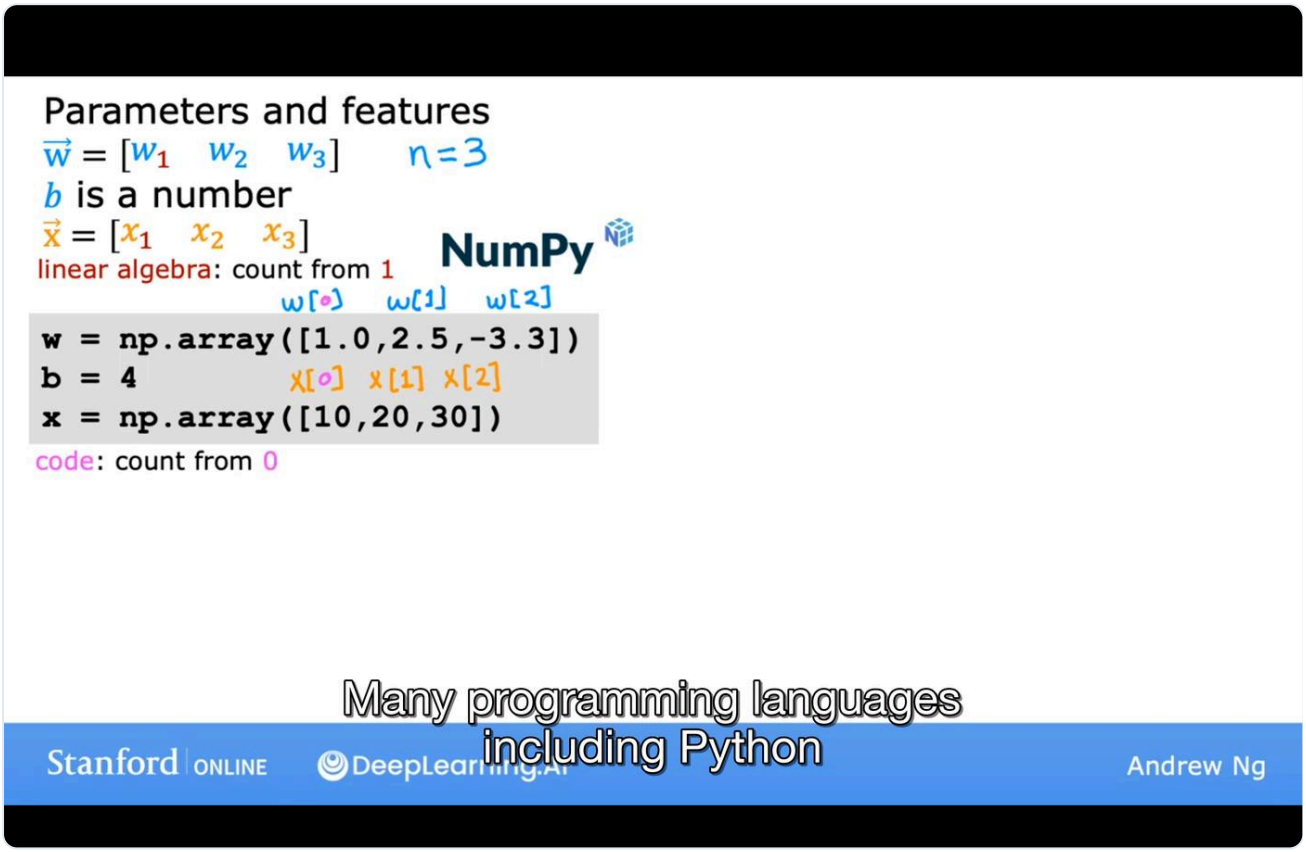


FIGURE 22.2 | Parameters and Features

In codes, it will look like this. You take each parameter w and multiply it by his associated feature. Now, you could write your code like this, but what if n isn't three but instead n is a 100 or a 100,000 is both inefficient for you the code and inefficient for your computer to compute. Here's another way. Without using vectorization but using a for loop. In math, you can use a summation operator to add all the products of w_j and x_j for j equals 1 through n. Then I'll cite the summation you add b at the end. To summation goes from j equals 1 up to and including n.

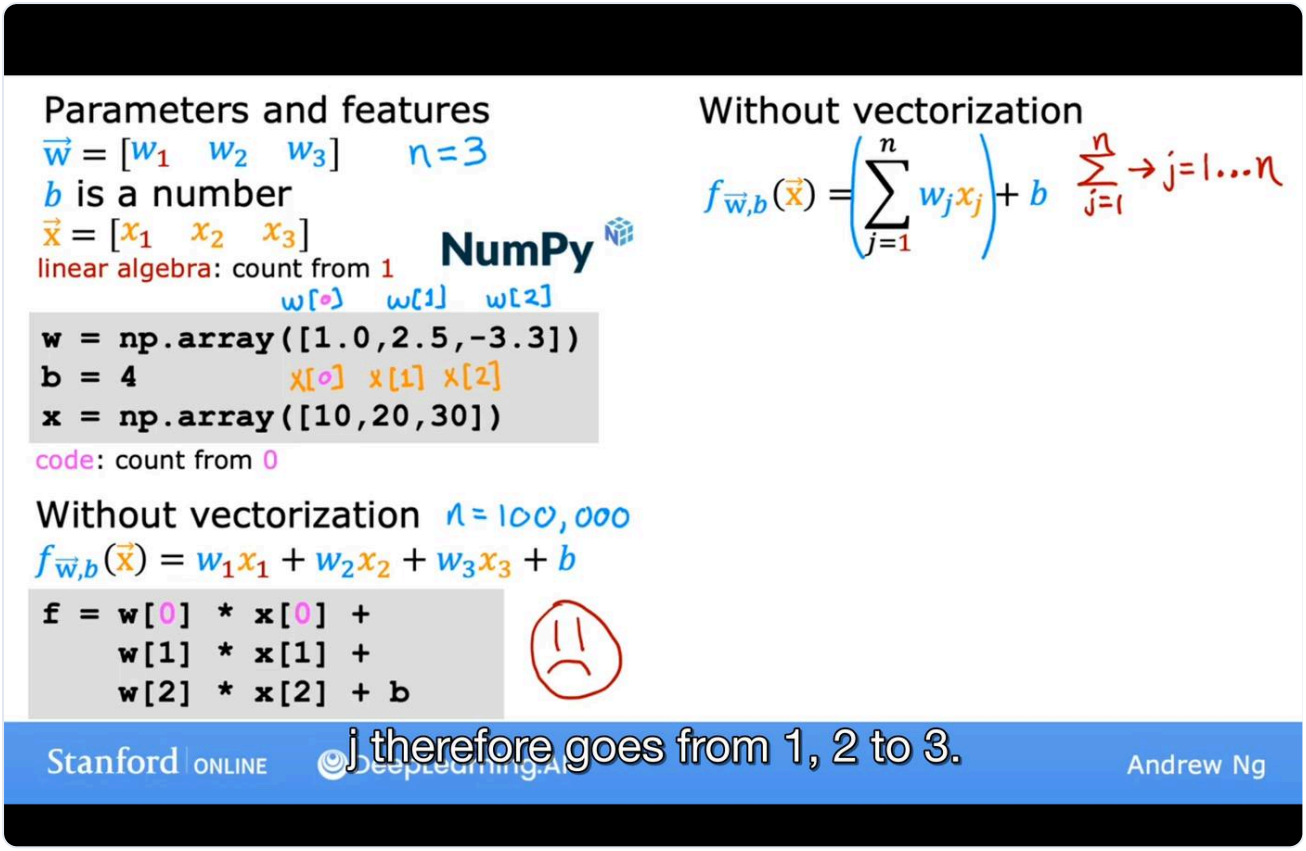


FIGURE 22.3 | Parameters and Features Without Vectorization

For n equals 3, j therefore goes from 1, 2 to 3. In code, you can initialize after 0. Then for j in range from 0 to n, this actually makes j go from 0 to n minus 1. From 0, 1 to 2, you can then add to f the product of w_j times x_j.

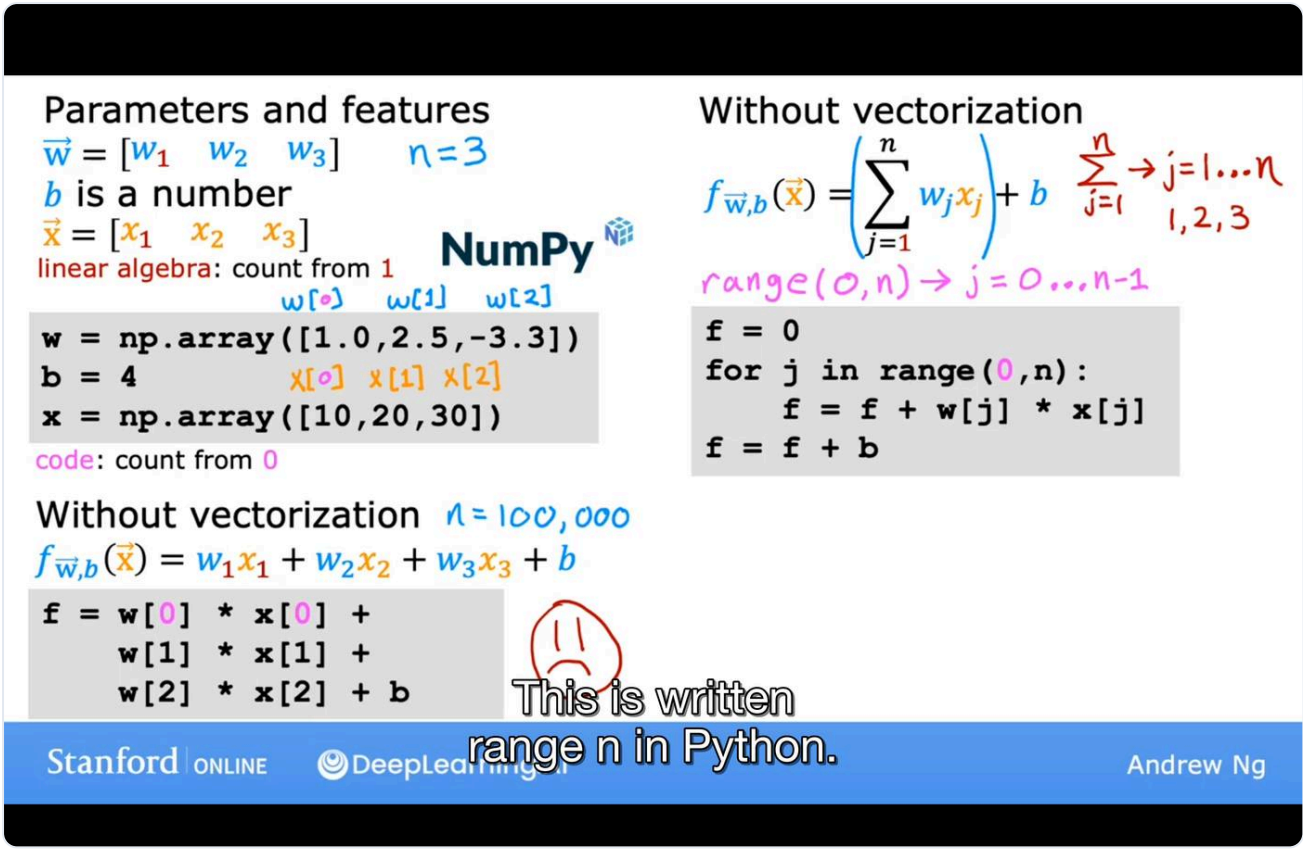


FIGURE 22.4 | Parameters and Features Without Vectorization

Finally, outside the for loop, you add b. Notice that in Python, the range 0 to n means that j goes from 0 all the way to n minus 1 and does not include n itself. This is written range n in Python. But in this video, I added a 0 here just to emphasize that it starts from 0. While this implementation is a bit better than the first one, this still doesn't use factorization, and isn't that efficient?

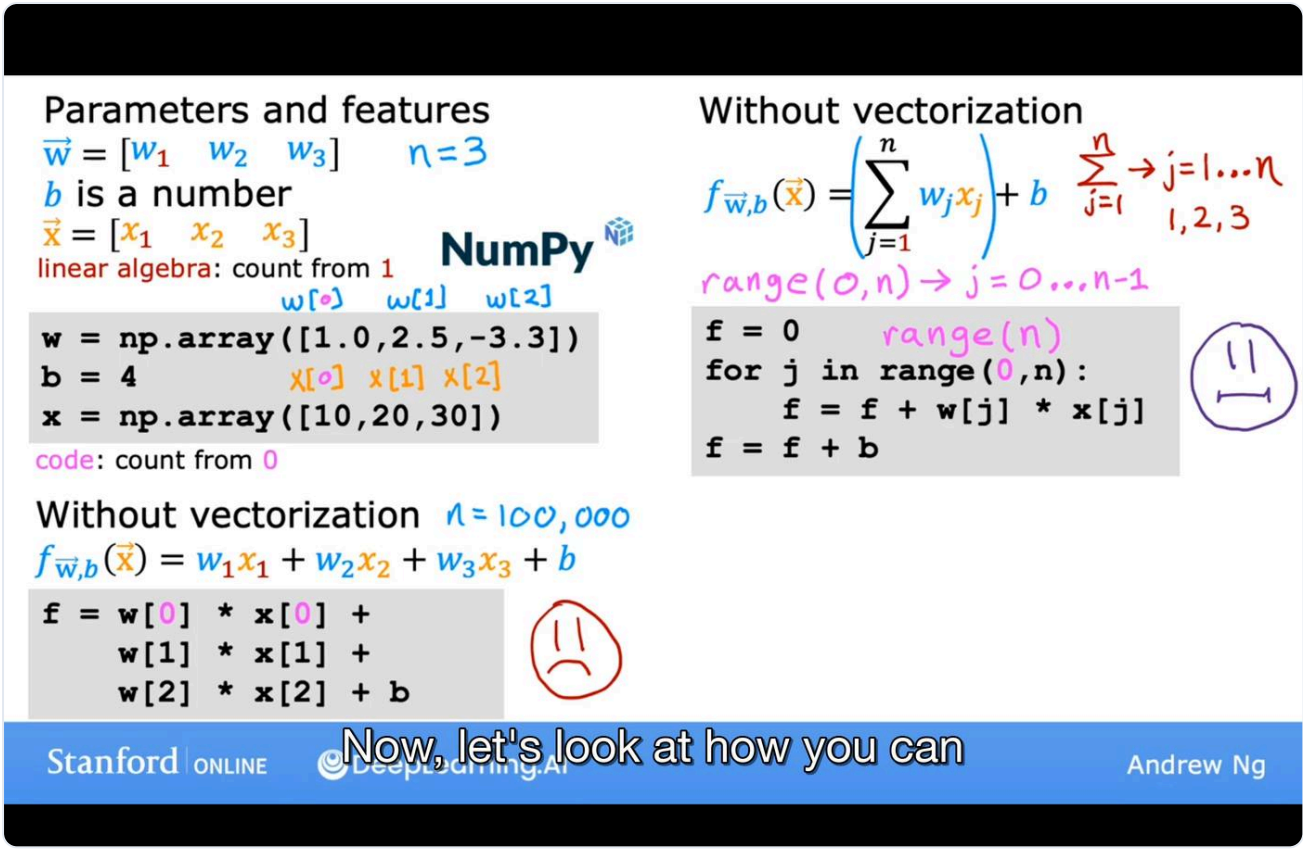


FIGURE 22.5 | Parameters and Features Without Vectorization

Now, let's look at how you can do this using vectorization. This is the math expression of the function f, which is the dot product of w and x plus b, and now you can implement this with a single line of code by computing fp equals np dot dot, I said dot dot because the first dot is the period and the second dot is the function or the method called DOT.

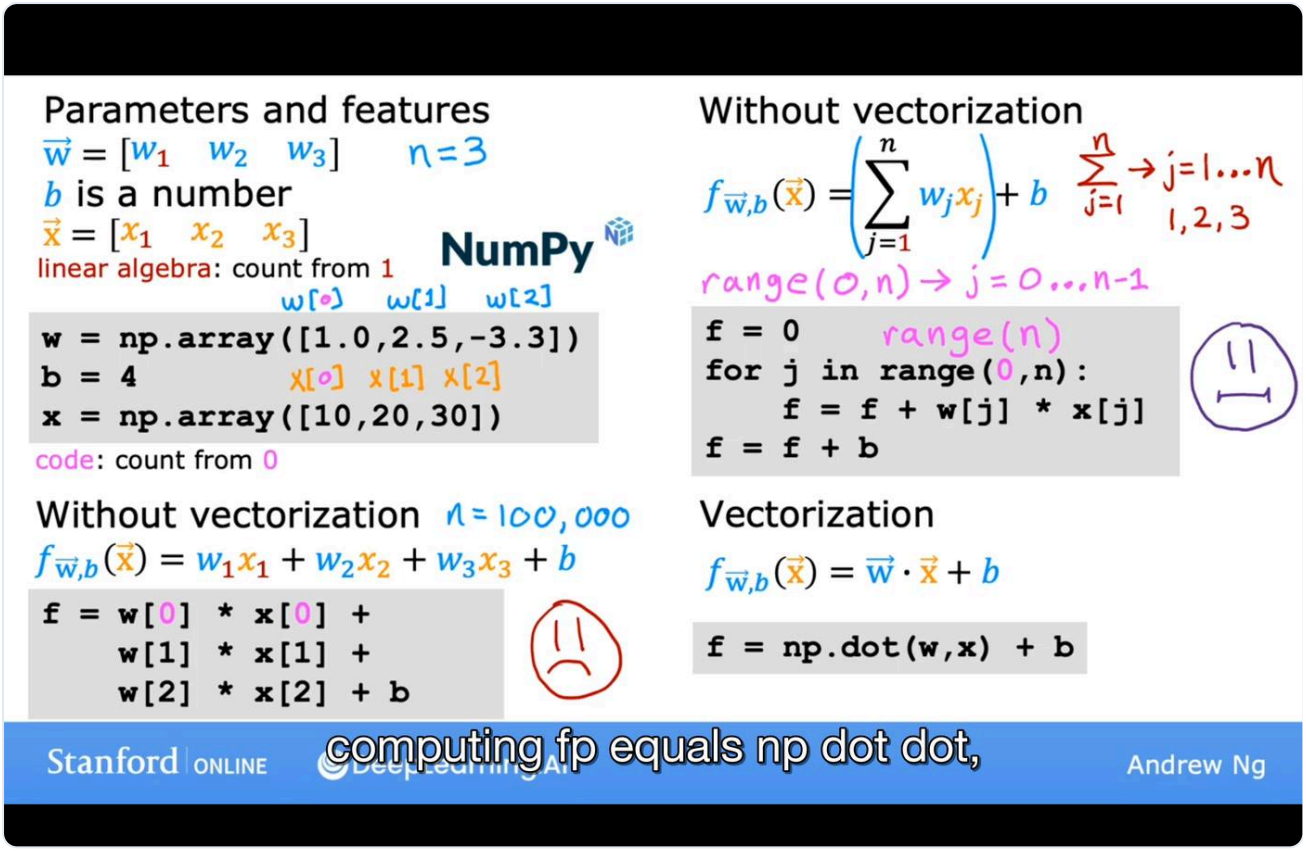


FIGURE 22.6 | Parameters and Features Without Vectorization

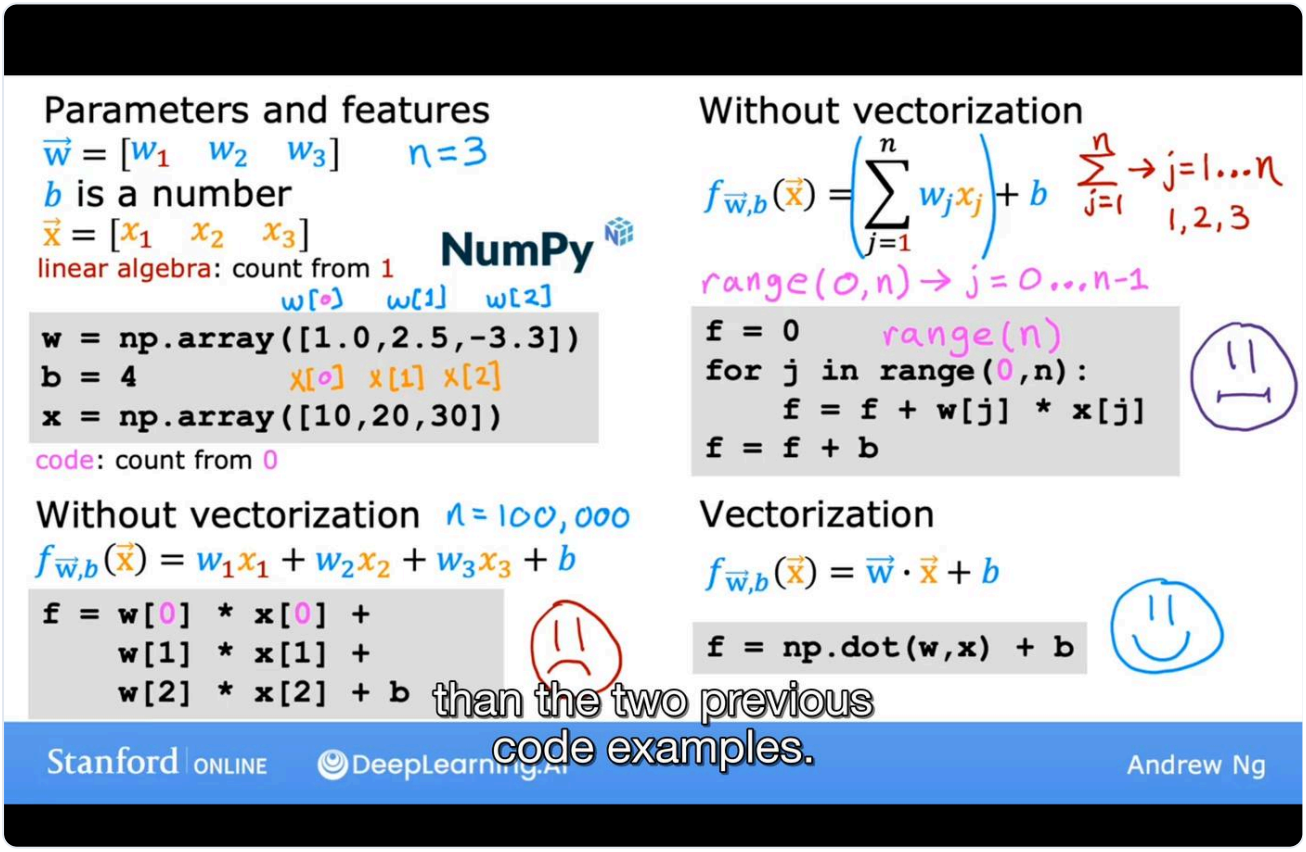


FIGURE 22.7 | Parameters and Features

But is fp equals np dot dot w comma x and this implements the mathematical dot products between the vectors w and x. Then finally, you can add b to it at the end. This NumPy dot function is a vectorized implementation of the dot product operation between two vectors and especially when n is large, this will run much faster than the two previous code examples. I want to emphasize that vectorization actually has two distinct benefits. First, it makes code shorter, is now just one line of code. Isn't that cool? Second, it also results in your code running much faster than either of the two previous implementations that did not use vectorization. The reason that the vectorized implementation is much faster is behind the scenes.

The NumPy dot function is able to use parallel hardware in your computer and this is true whether you're running this on a normal computer, that is on a normal computer CPU or if you are using a GPU, a graphics processor unit, that's often used to accelerate machine learning jobs. The ability of the NumPy dot function to use parallel hardware makes it much more efficient than the for loop or the sequential calculation that we saw previously. Now, this version is much more practical when n is large because you are not typing w0 times x0 plus w1 times x1 plus lots of additional terms like you would have had for the previous version. But while this saves a lot on the typing, is still not that computationally efficient because it still doesn't use vectorization.

To recap, vectorization makes your code shorter, so hopefully easier to write and easier for you or others to read, and it also makes it run much faster. But honest, this magic behind vectorization that makes this run so much faster. Let's take a look at what your computer is actually doing behind the scenes to make vectorized code run so much faster.

LECTURE 23

Without Vectorization

I remember when I first learned about vectorization, I spent many hours on my computer taking an unvectorized version of an algorithm running it, see how long it run, and then running a vectorized version of the code and seeing how much faster that run, and I just spent hours playing with that. And it frankly blew my mind that the same algorithm vectorized would run so much faster. It felt almost like a magic trick to me. In this video, let's figure out how this magic trick really works. Let's take a deeper look at how a vectorized implementation may work on your computer behind the scenes. Let's look at this for loop. The for loop like this runs without vectorization.

If j ranges from 0 to say 15, this piece of code performs operations one after another. On the first timestamp which I'm going to write as t0. It first operates on the values at index 0. At the next time-step, it calculates values corresponding to index 1 and so on until the 15th step, where it computes that.

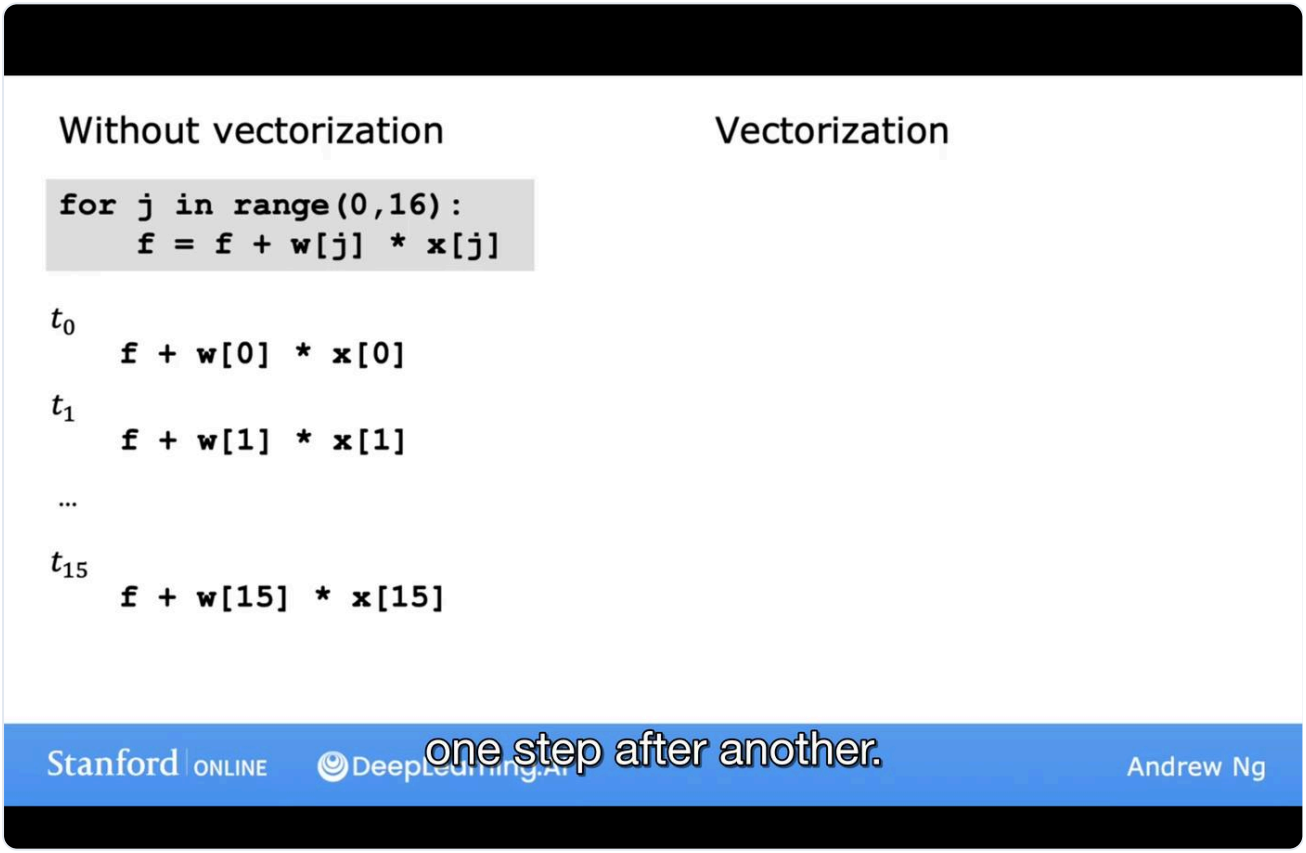


FIGURE 23.1

In other words, it calculates these computations one step at a time, one step after another. In contrast, this function in NumPy is implemented in the computer hardware with vectorization. The computer can get all values of the vectors w and x, and in a single-step, it multiplies each pair of w and x with each other all at the same time in parallel.

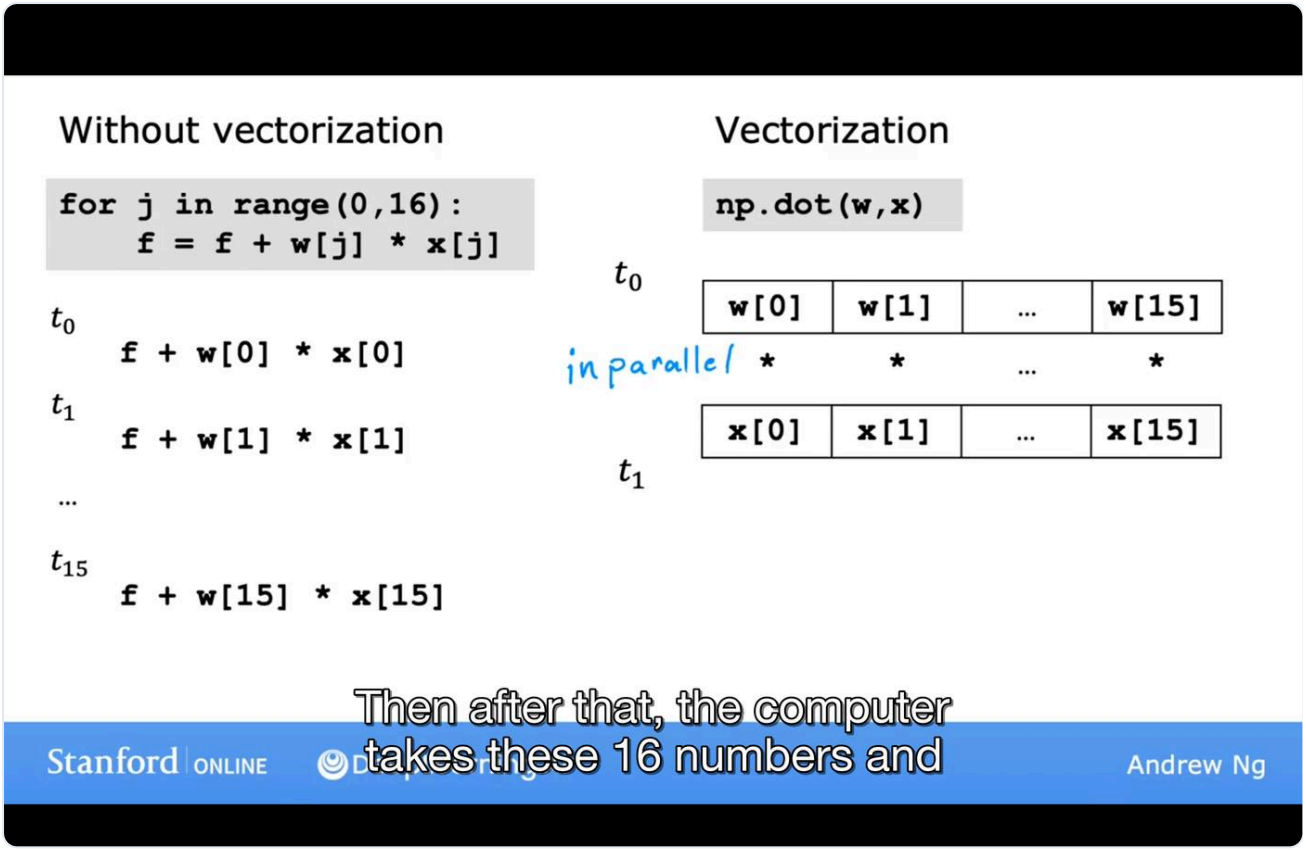


FIGURE 23.2

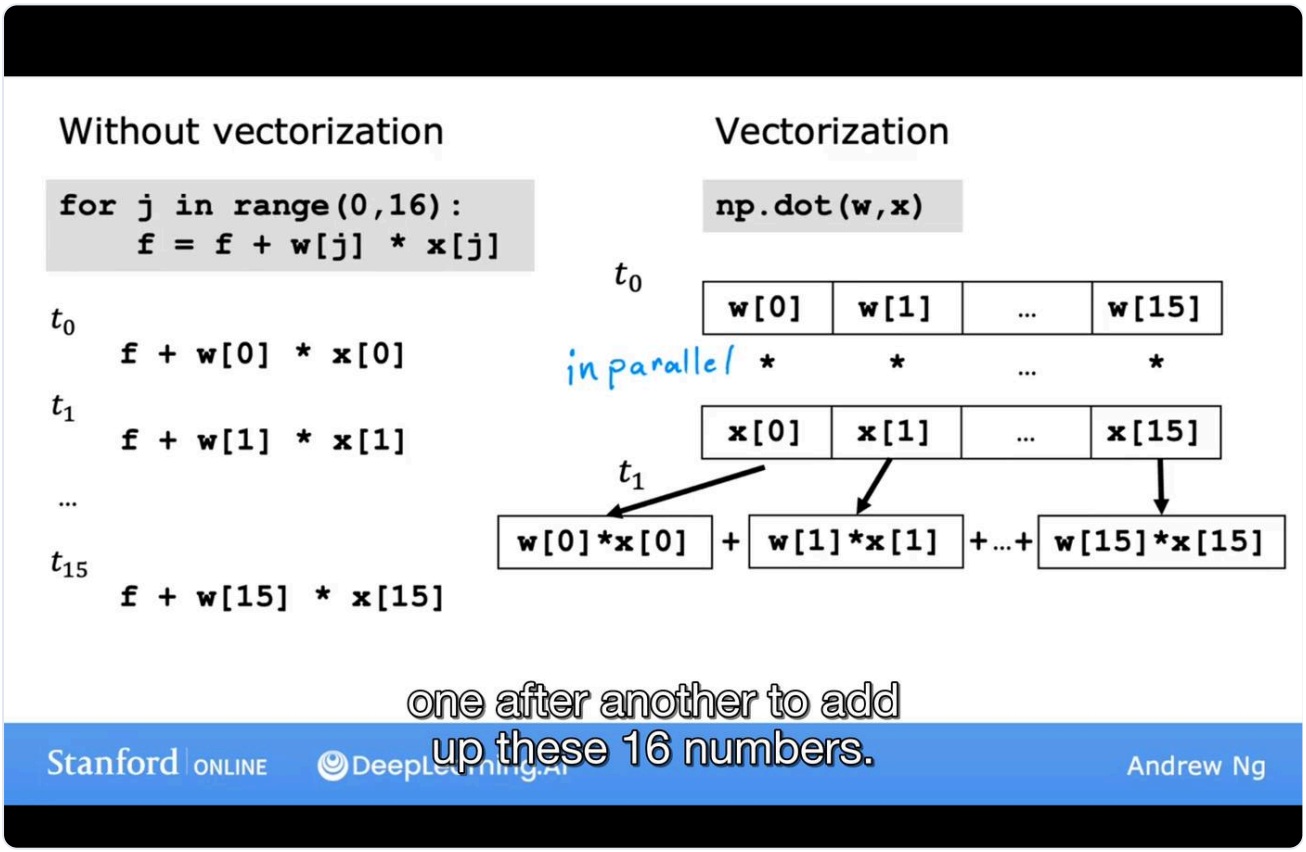


FIGURE 23.3

Then after that, the computer takes these 16 numbers and uses specialized hardware to add them altogether very efficiently, rather than needing to carry out distinct additions one after another to add up these 16 numbers. This means that codes with vectorization can perform calculations in much less time than codes without vectorization. This matters more when you're running algorithms on large data sets or trying to train large models, which is often the case with machine learning.

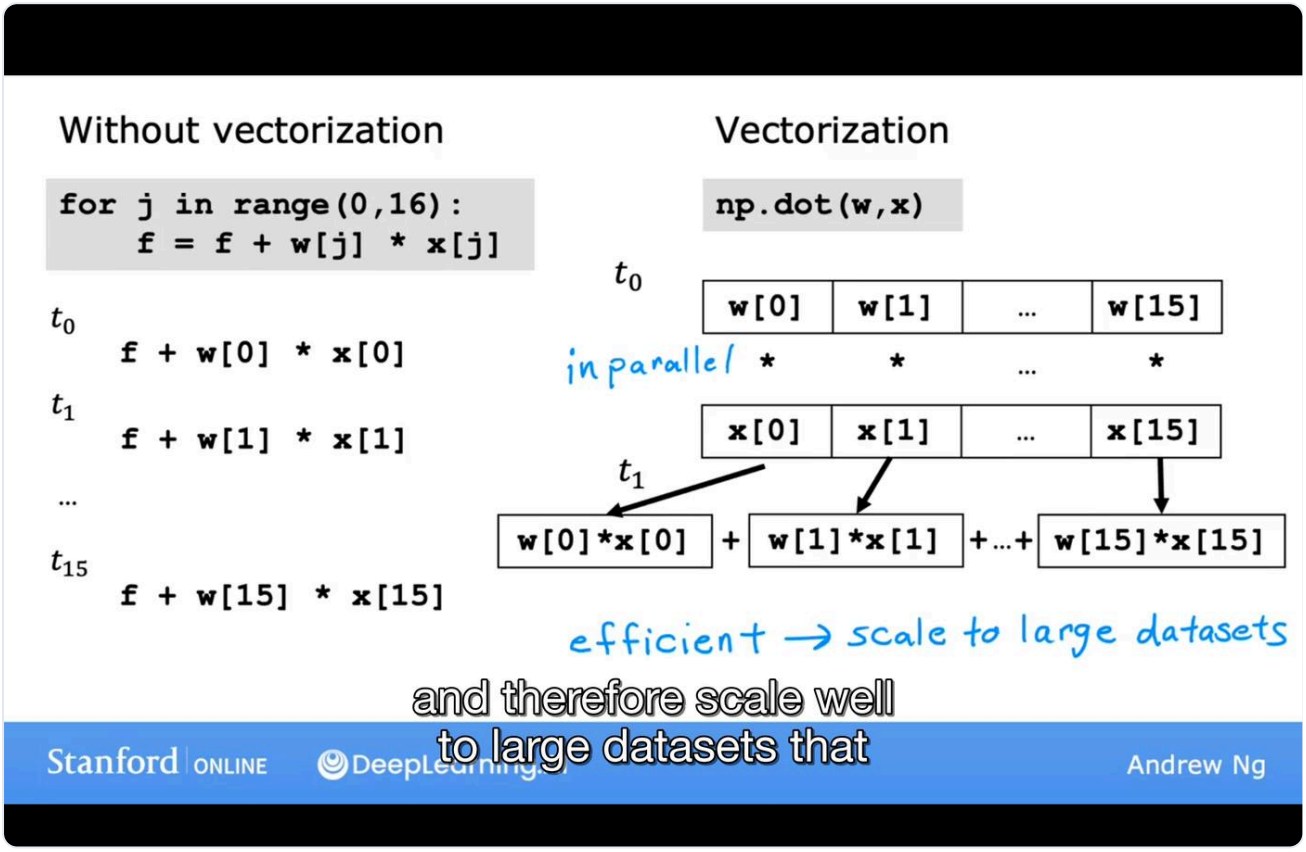


FIGURE 23.4

That's why being able to vectorize implementations of learning algorithms, has been a key step to getting learning algorithms to run efficiently, and therefore scale well to large datasets that many modern machine learning algorithms now have to operate on. Now, let's take a look at a concrete example of how this helps with implementing multiple linear regression and this linear regression with multiple input features.

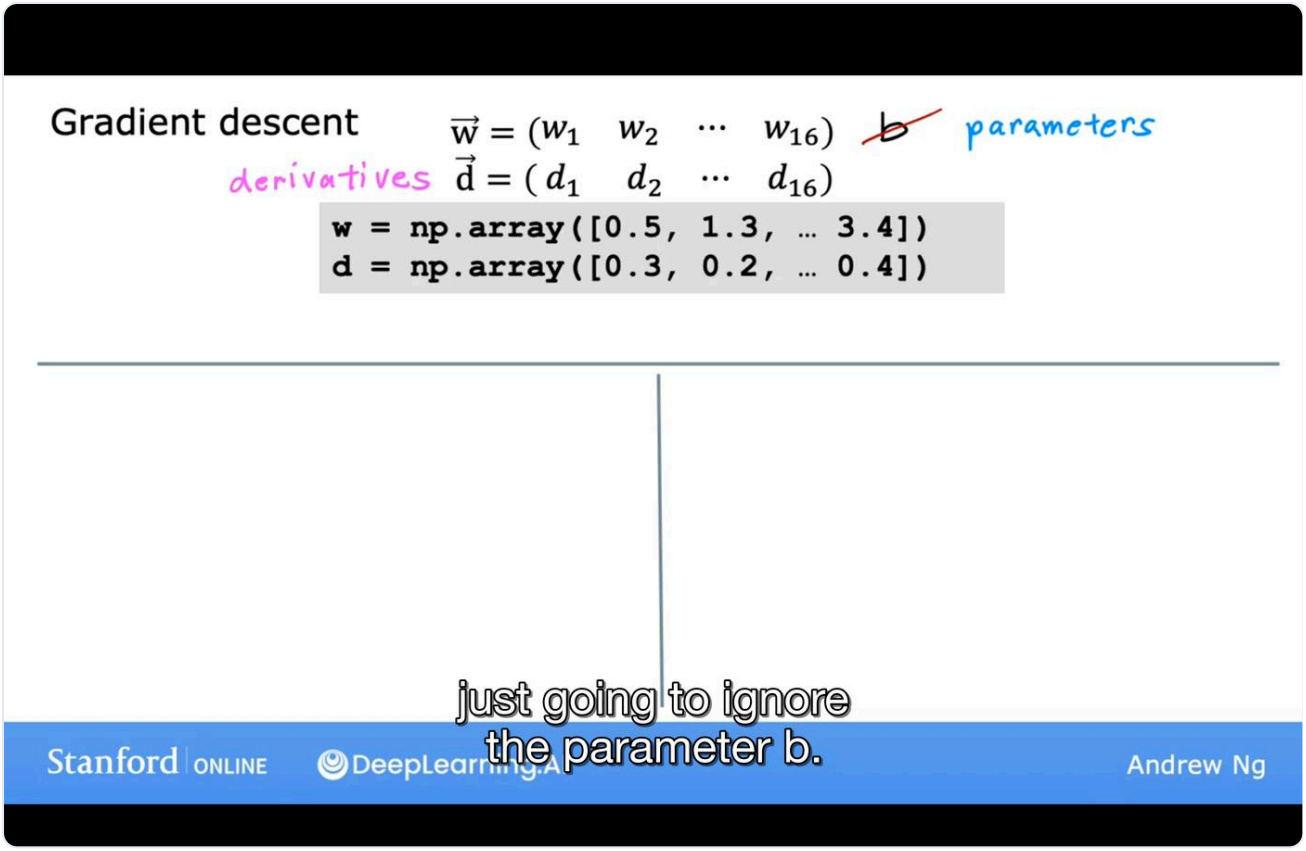


FIGURE 23.5 | Gradient Descent

Say you have a problem with 16 features and 16 parameters, w1 through w16, in addition to the parameter b. You calculate it 16 derivative terms for these 16 weights and codes, maybe you store the values of w and d in two np.arrays, with d storing the values of the derivatives.

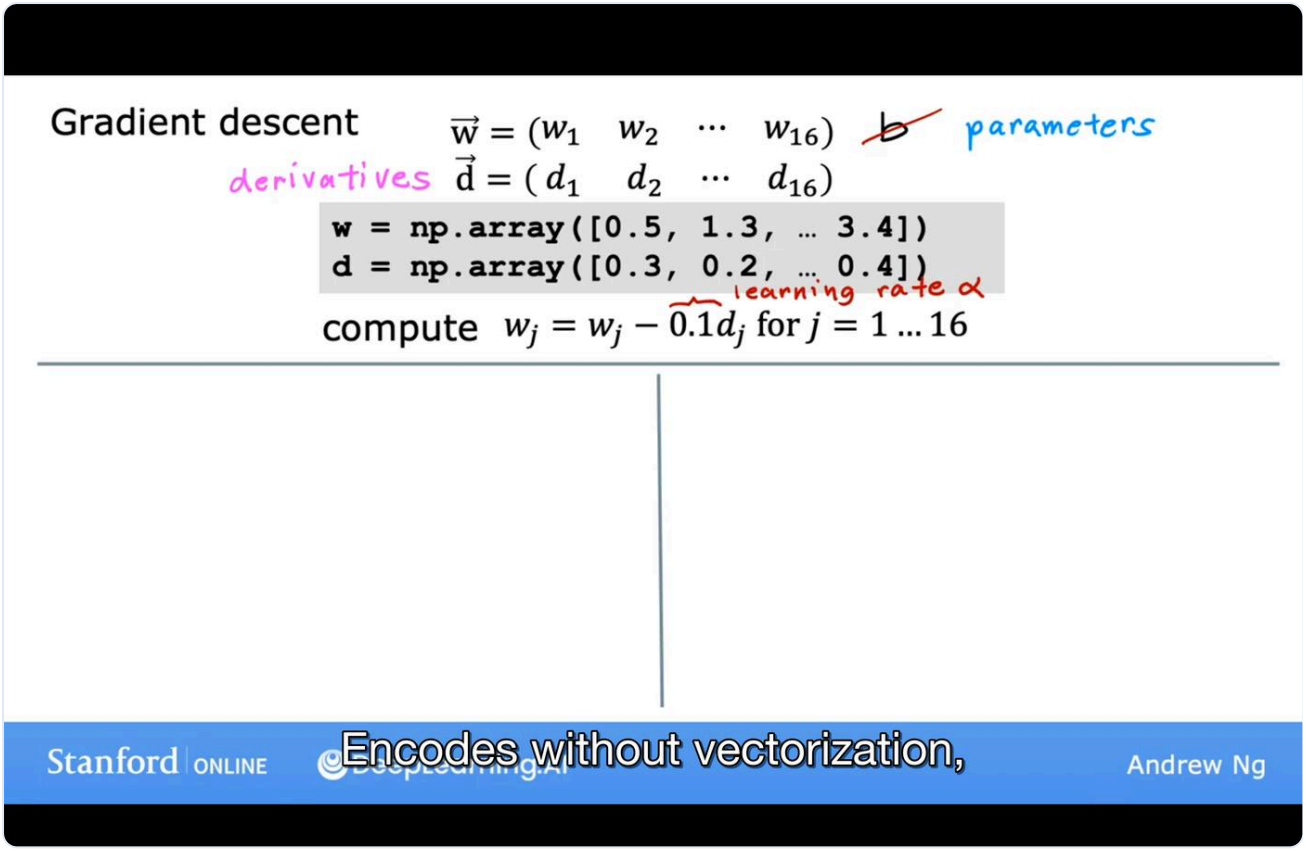


FIGURE 23.6

For this example, I'm just going to ignore the parameter b. Now, you want to compute an update for each of these 16 parameters. W_j is updated to w_j minus the learning rate, say 0.1, times d_j, for j from 1 through 16.

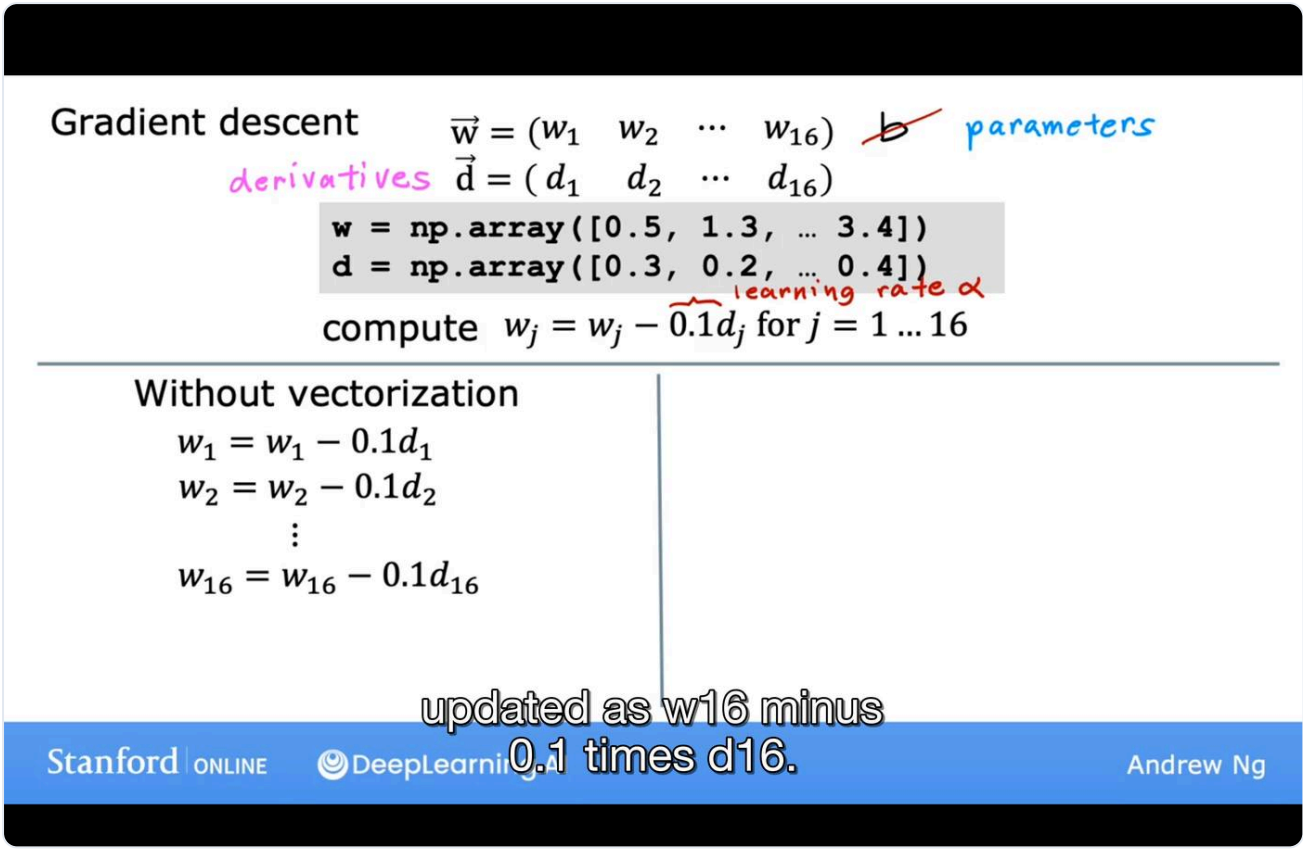


FIGURE 23.7 | Without Vectorization

Encodes without vectorization, you would be doing something like this. Update w1 to be w1 minus the learning rate 0.1 times d1, next, update w2 similarly, and so on through w16, updated as w16 minus 0.1 times d16.

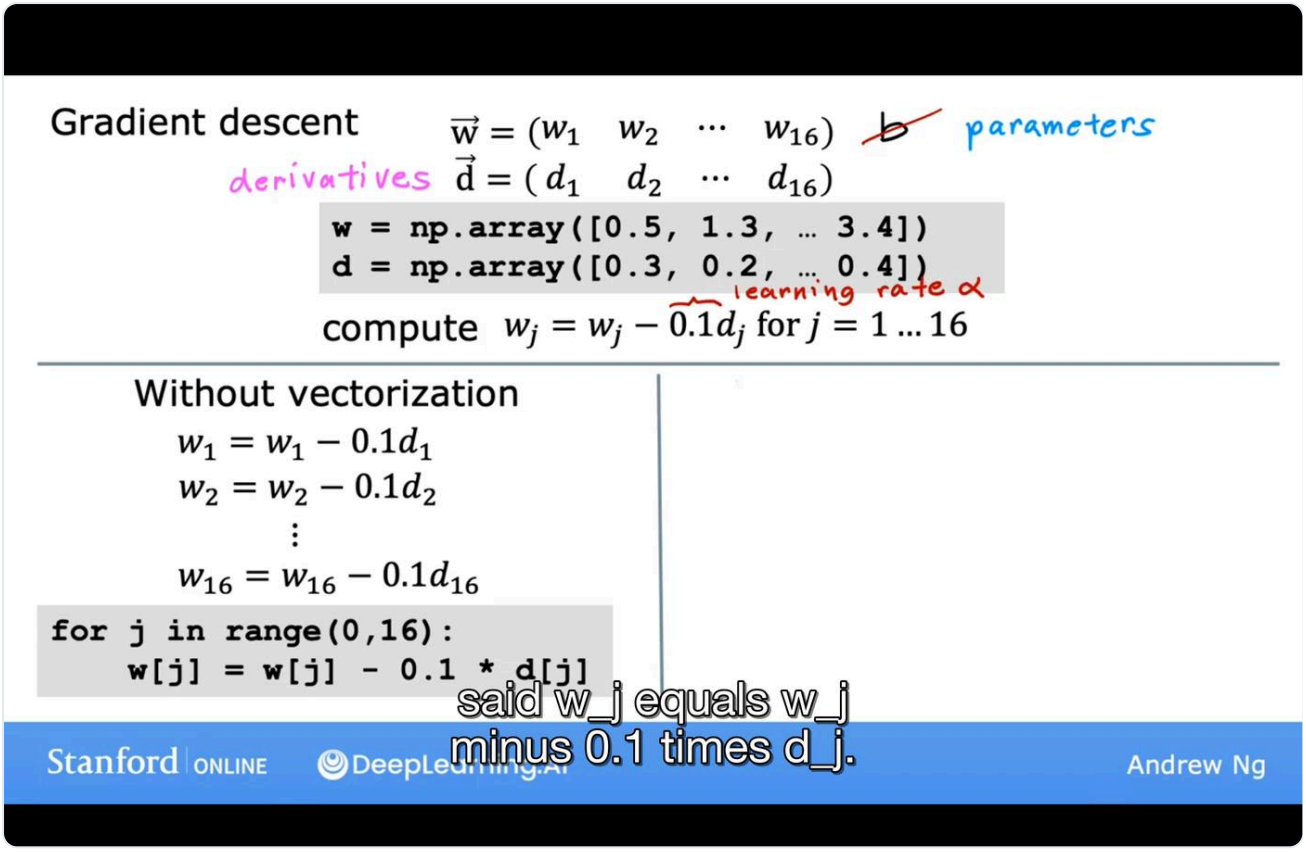


FIGURE 23.8

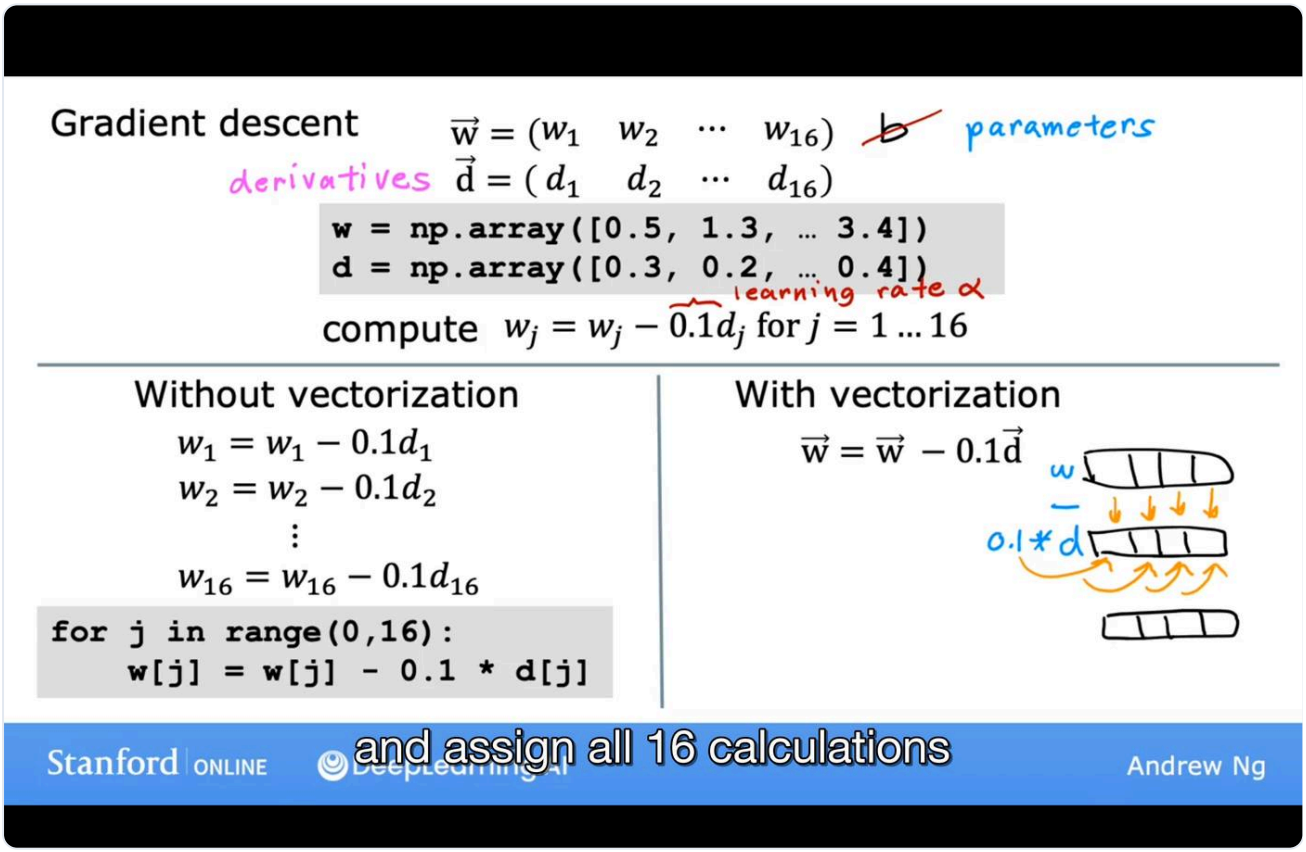


FIGURE 23.9 | Without Vectorization

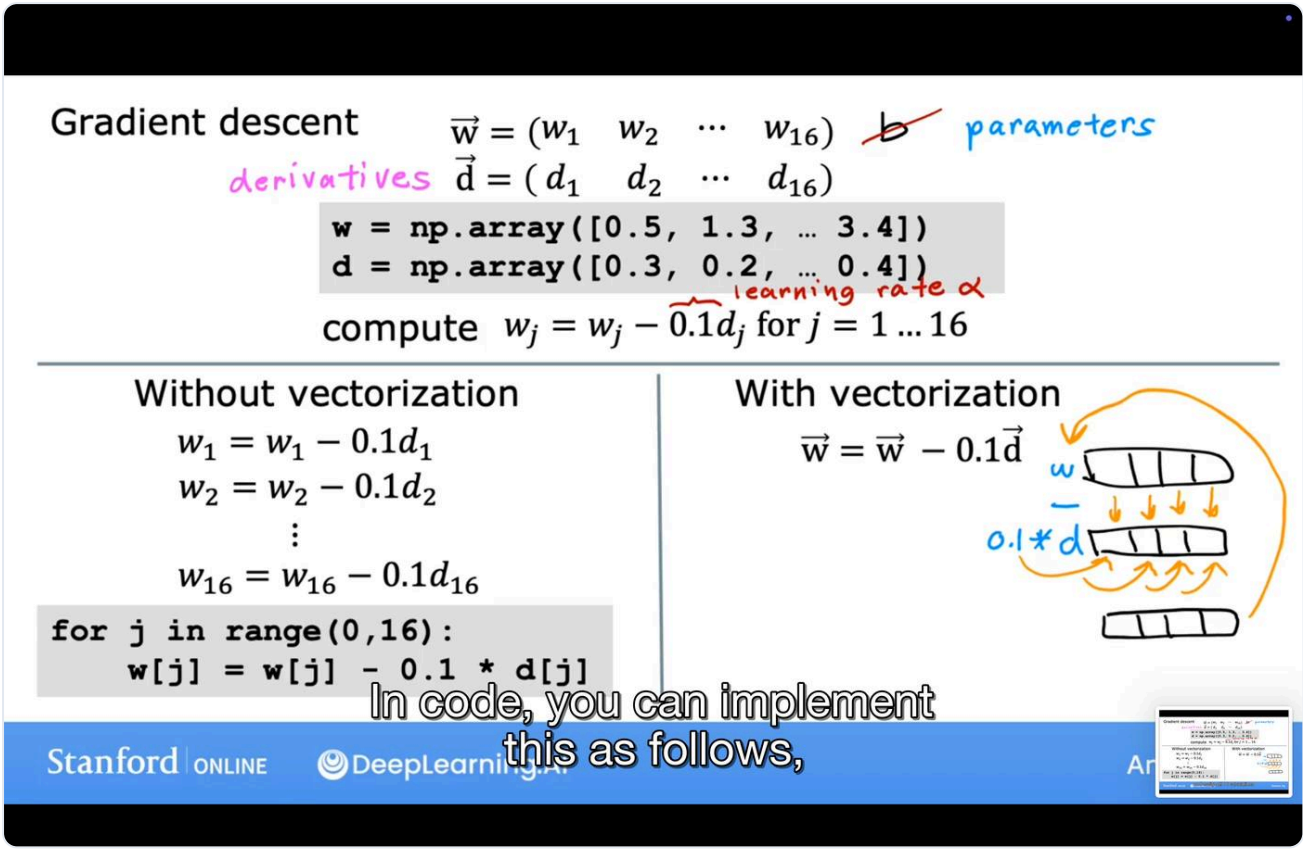


FIGURE 23.10

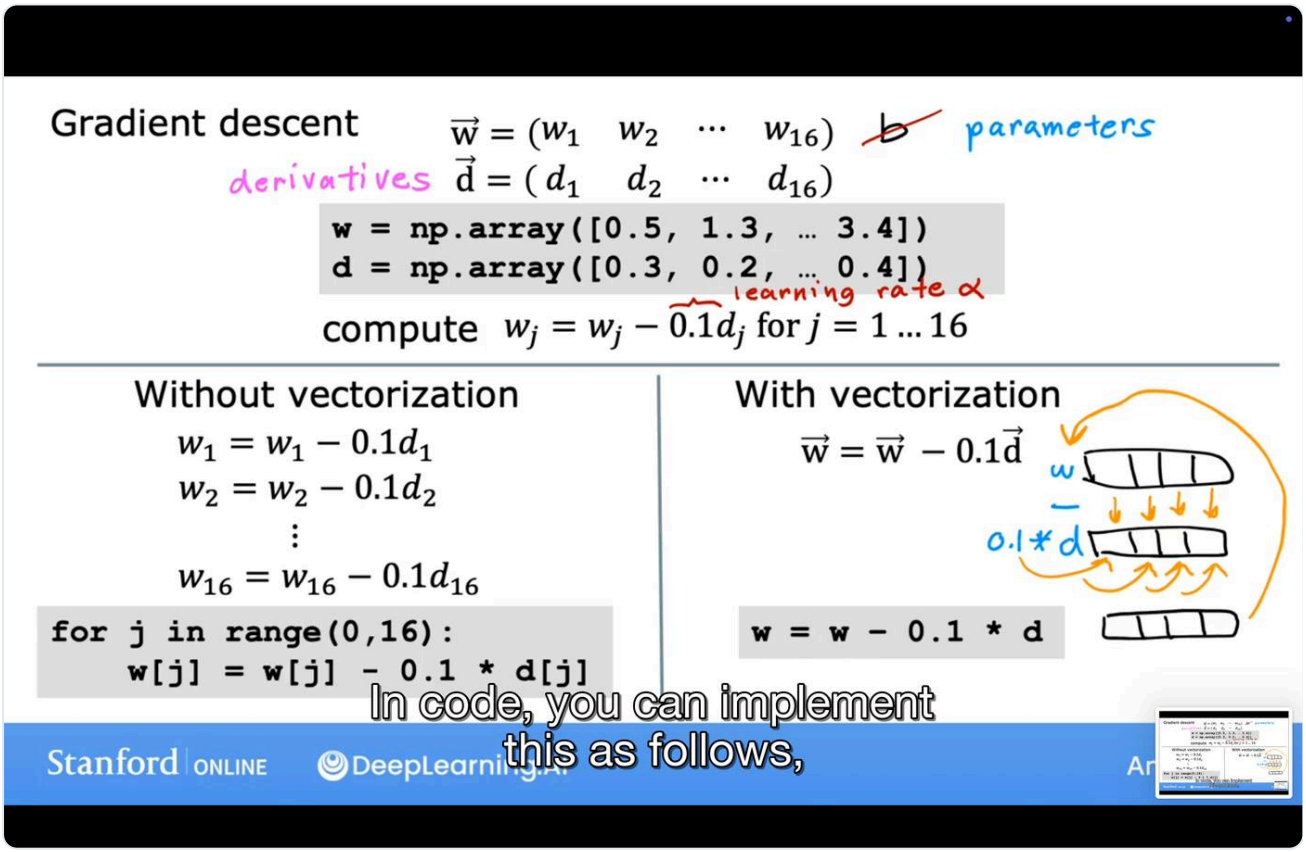


FIGURE 23.11

Encodes without vectorization, you can use a for loop like this for j in range 016, that again goes from 0-15, said w_j equals w_j minus 0.1 times d_j. In contrast, with factorization, you can imagine the computer's parallel processing hardware like this. It takes all 16 values in the vector w and subtracts in parallel, 0.1 times all 16 values in the vector d, and assign all 16 calculations back to w all at the same time and all in one step. In code, you can implement this as follows, w is assigned to w minus 0.1 times d. Behind the scenes, the computer takes these NumPy arrays, w and d, and uses parallel processing hardware to carry out all 16 computations efficiently.

Using a vectorized implementation, you should get a much more efficient implementation of linear regression. Maybe the speed difference won't be huge if you have 16 features, but if you have thousands of features and perhaps very large training sets, this type of vectorized implementation will make a huge difference in the running time of your learning algorithm. It could be the difference between codes finishing in one or two minutes, versus taking many hours to do the same thing. In the optional lab that follows this video, you see an introduction to one of the most used Python libraries and Machine Learning, which we've already touched on in this video called NumPy.

You see how they create vectors encode and these vectors or lists of numbers are called NumPy arrays, and you also see how to take the dot product of two vectors using a NumPy function called dot. You also get to see how vectorized code such as using the dot function, can run much faster than a for-loop. In fact, you'd get to time this code yourself, and hopefully see it run much faster. This optional lab introduces a fair amount of new NumPy syntax, so when you read through the optional lab, please still feel like you have to understand all the code right away, but you can save this notebook and use it as a reference to look at when you're working with data stored in NumPy arrays.

Congrats on finishing this video on vectorization. You've learned one of the most important and useful techniques in implementing machine learning algorithms. In the next video, we'll put the math of multiple linear regression together with vectorization, so that you will influence gradient descent for multiple linear regression with vectorization. Let's go on to the next video.

LECTURE 24

Gradient Descent

You've learned about gradient descents about multiple linear regression and also vectorization. Let's put it all together to implement gradient descent for multiple linear regression with vectorization. This would be cool. Let's quickly review what multiple linear regression look like.

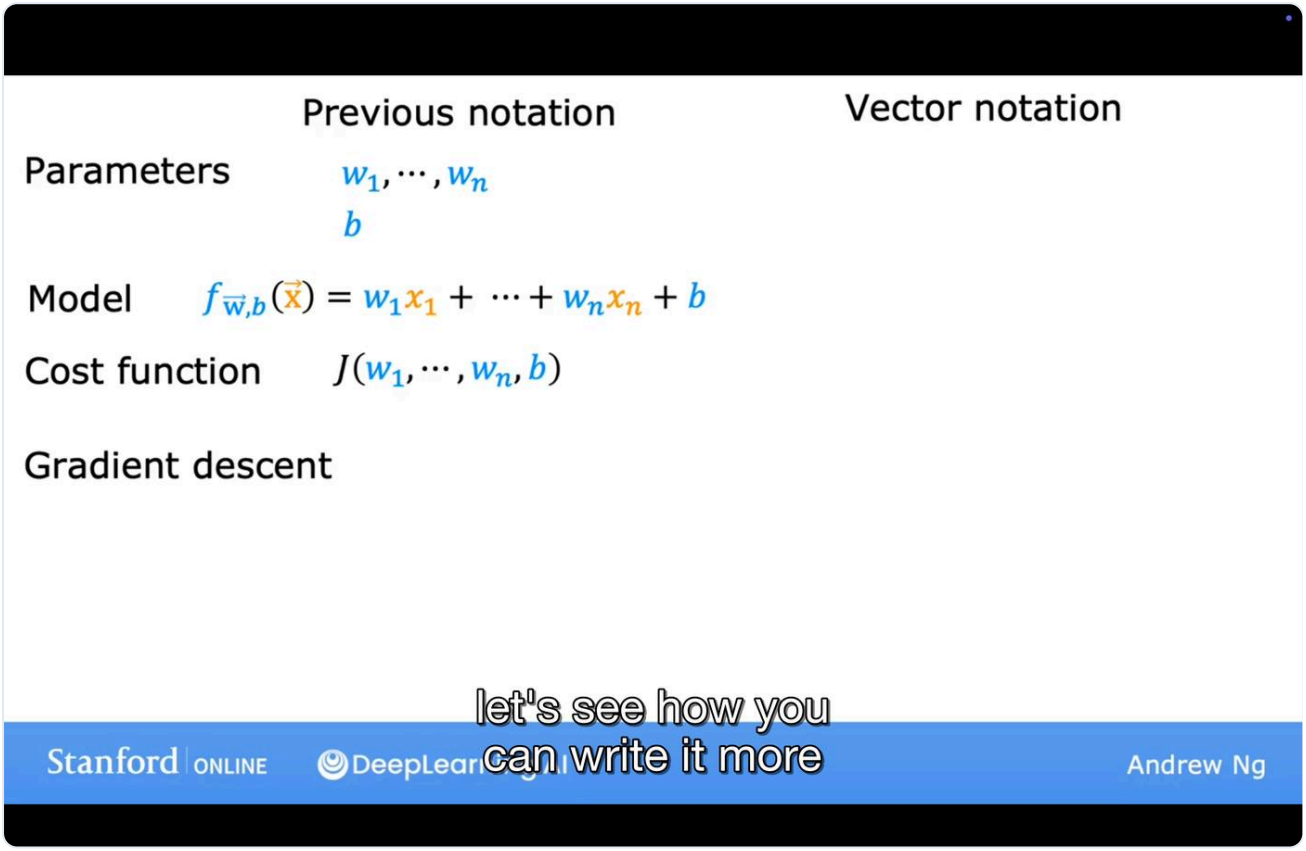


FIGURE 24.1 | Previous Notation Vector Notation

Using our previous notation, let's see how you can write it more succinctly using vector notation. We have parameters w_1 to w_n as well as b. But instead of thinking of w_1 to w_n as separate numbers, that is separate parameters, let's start to collect all of the w's into a vector w so that now w is a vector of length n. We're just going to think of the parameters of this model as a vector w, as well as b, where b is still a number same as before. Whereas before we had to find multiple linear regression like this, now using vector notation, we can write the model as f_w, b of x equals the vector w dot product with the vector x plus b. Remember that this dot here means.product.

Our cost function can be defined as J of w_1 through w_n, b. But instead of just thinking of J as a function of these and different parameters w_j as well as b, we're going to write J as a function of parameter vector w and the number b. This w_1 through w_n is replaced by this vector W and J now takes this input of vector w and a number b and returns a number. Here's what gradient descent looks like. We're going to repeatedly update each parameter w_j to be w_j minus Alpha times the derivative of the cost J, where J has parameters w_1 through w_n and b.

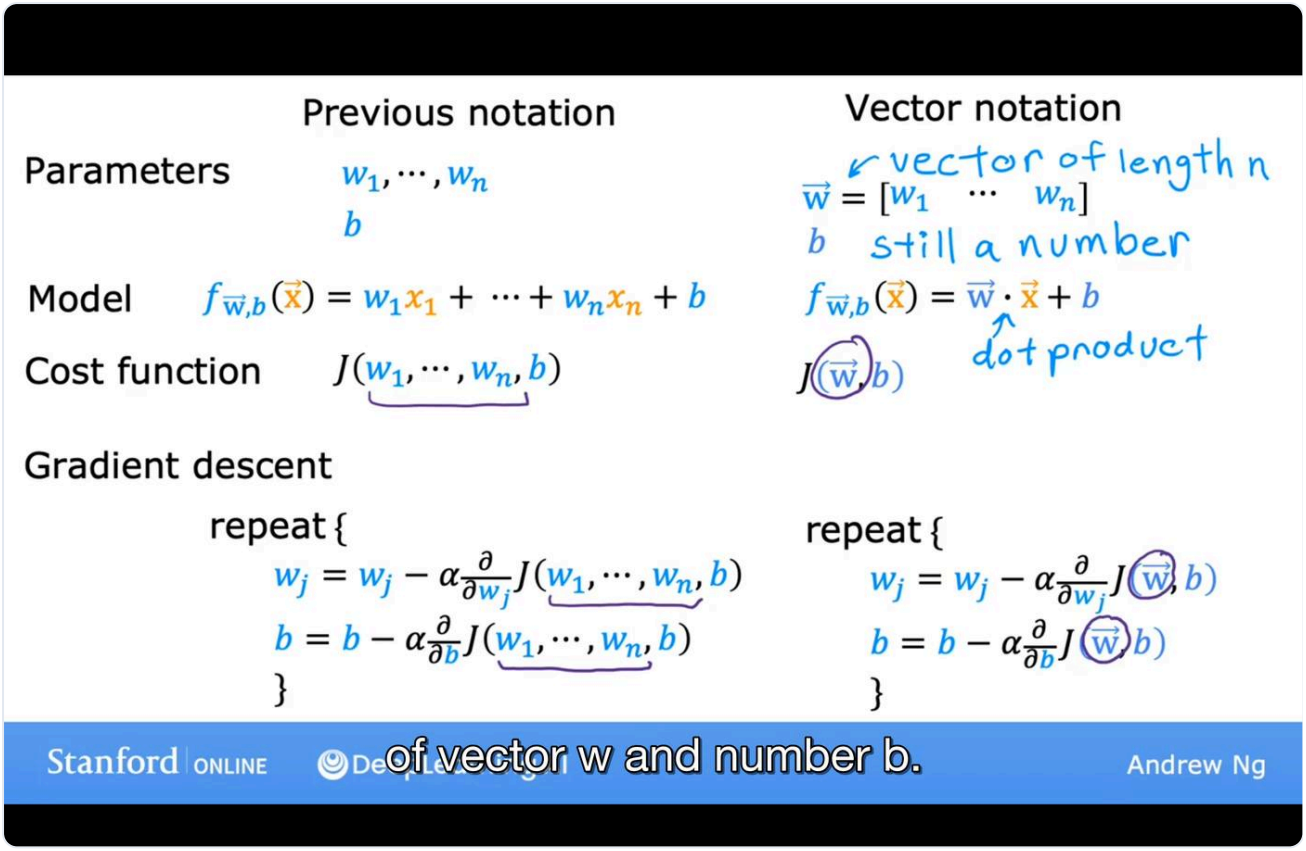


FIGURE 24.2 | Previous Notation Vector Notation

Once again, we just write this as J of vector w and number b. Let's see what this looks like when you implement gradient descent and in particular, let's take a look at the derivative term. We'll see that gradient descent becomes just a little bit different with multiple features compared to just one feature. Here's what we had when we had gradient descent with one feature. We had an update rule for w and a separate update rule for b. Hopefully, these look familiar to you.

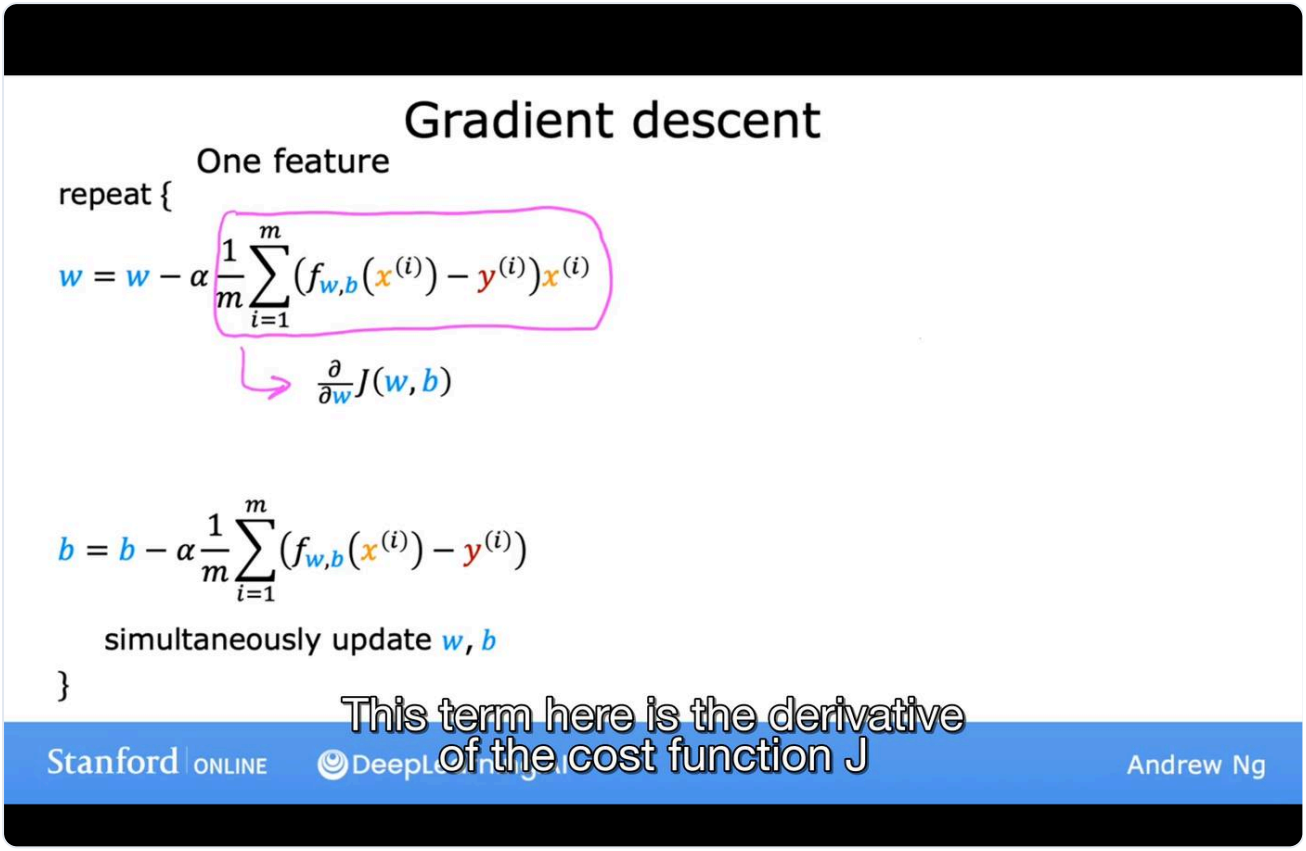


FIGURE 24.3 | Gradient Descent

This term here is the derivative of the cost function J with respect to the parameter w. Similarly, we have an update rule for parameter b, with univariate regression, we had only one feature. We call that feature xi without any subscript. Now, here's a new notation for where we have n features, where n is two or more. We get this update rule for gradient descent. Update w_1 to be w_1 minus Alpha times this expression here and this formula is actually the derivative of the cost J with respect to w_1.

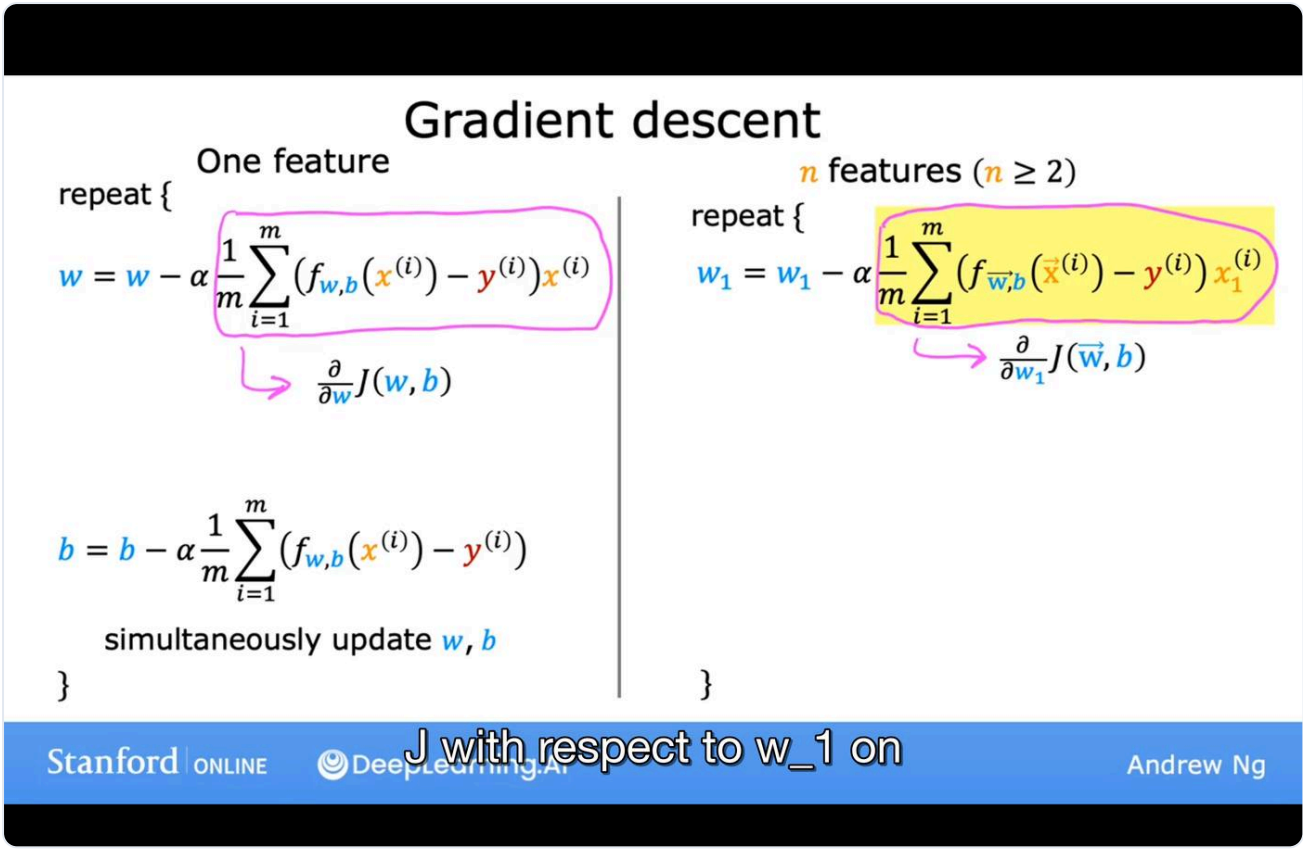


FIGURE 24.4 | Gradient Descent

The formula for the derivative of J with respect to w_1 on the right looks very similar to the case of one feature on the left. The error term still takes a prediction f of x minus the target y. One difference is that w and x are now vectors and just as w on the left has now become w_1 here on the right, xi here on the left is now instead xi _1 here on the right and this is just for J equals 1.

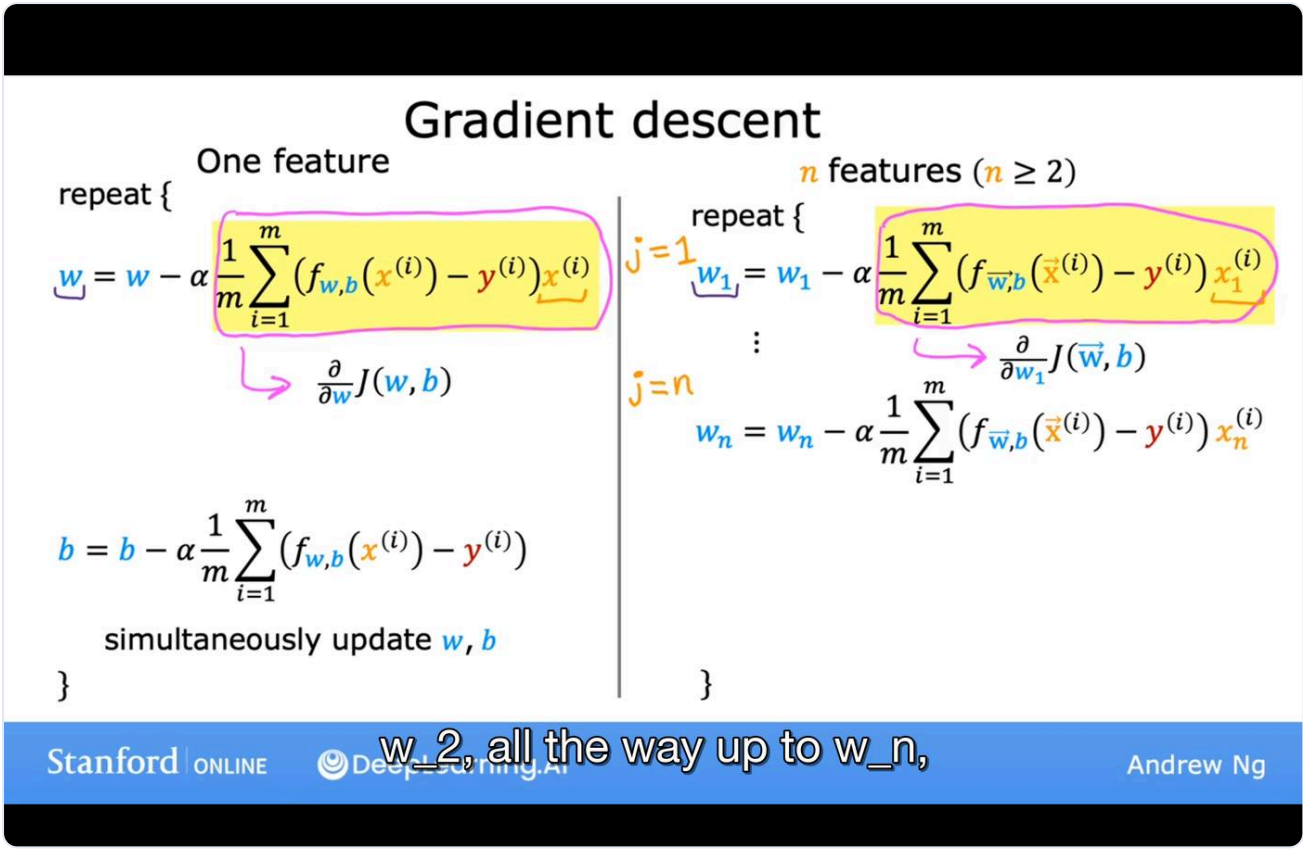


FIGURE 24.5 | Gradient Descent

For multiple linear regression, we have J ranging from 1 through n and so we'll update the parameters w_1, w_2, all the way up to w_n, and then as before, we'll update b. If you implement this, you get gradient descent for multiple regression. That's it for gradient descent for multiple regression. Before moving on from this video, I want to make a quick aside or a quick side note on an alternative way for finding w and b for linear regression. This method is called the normal equation. Whereas it turns out gradient descent is a great method for minimizing the cost function J to find w and b, there is one other algorithm that works only for linear regression and pretty much none of the other algorithms you see in this specialization for solving for w and b and this other method does not need an iterative gradient descent algorithm.

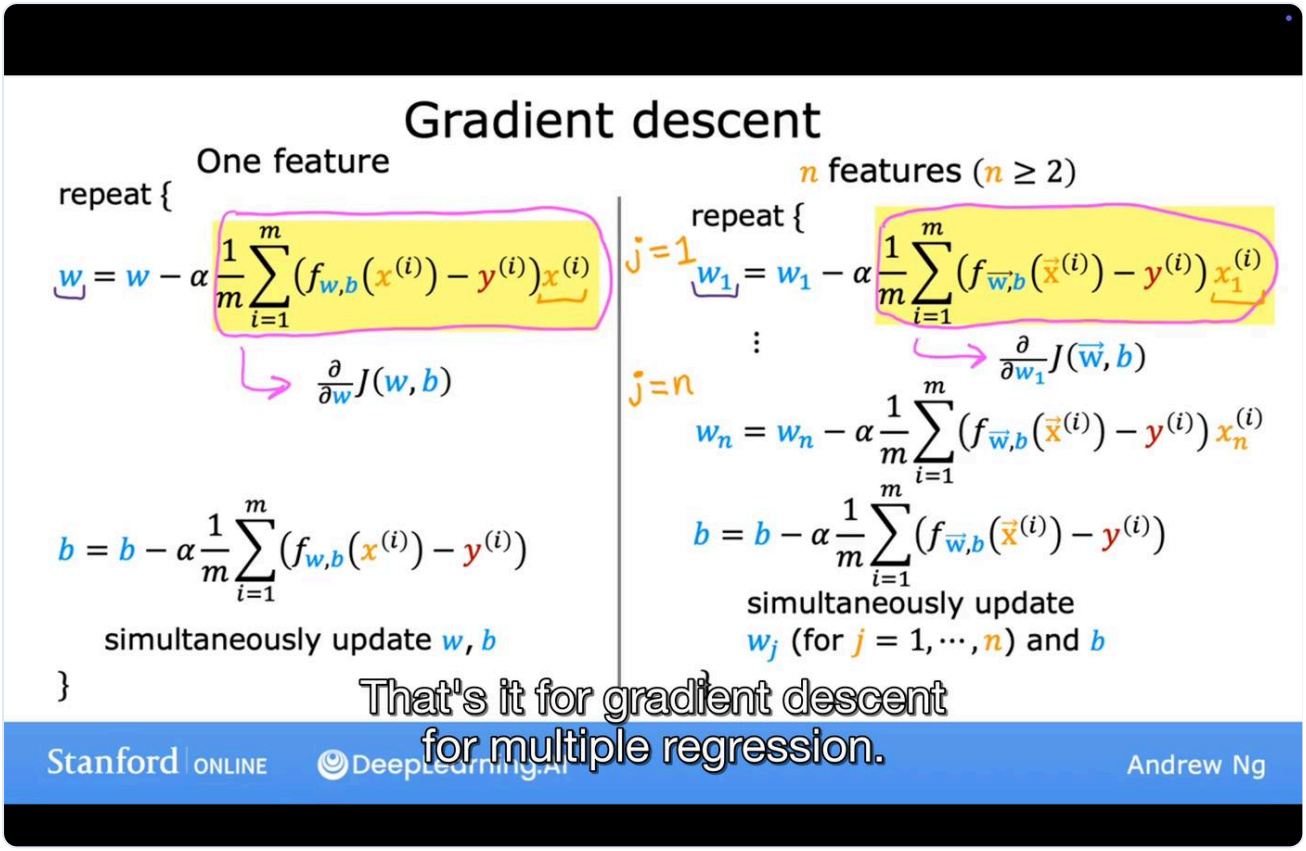


FIGURE 24.6 | Gradient Descent

Called the normal equation method, it turns out to be possible to use an advanced linear algebra library to just solve for w and b all in one goal without iterations. Some disadvantages of the normal equation method are; first unlike gradient descent, this is not generalized to other learning algorithms, such as the logistic regression algorithm that you'll learn about next week or the neural networks or other algorithms you see later in this specialization. The normal equation method is also quite slow if the number of features and this large. Almost no machine learning practitioners should implement the normal equation method themselves but if you're using a mature machine learning library and call linear regression, there is a chance that on the back-end, it'll be using this to solve for w and b.

If you're ever in the job interview and hear the term normal equation, that's what this refers to. Don't worry about the details of how the normal equation works. Just be aware that some machine learning libraries may use this complicated method in the back-end to solve for w and b.

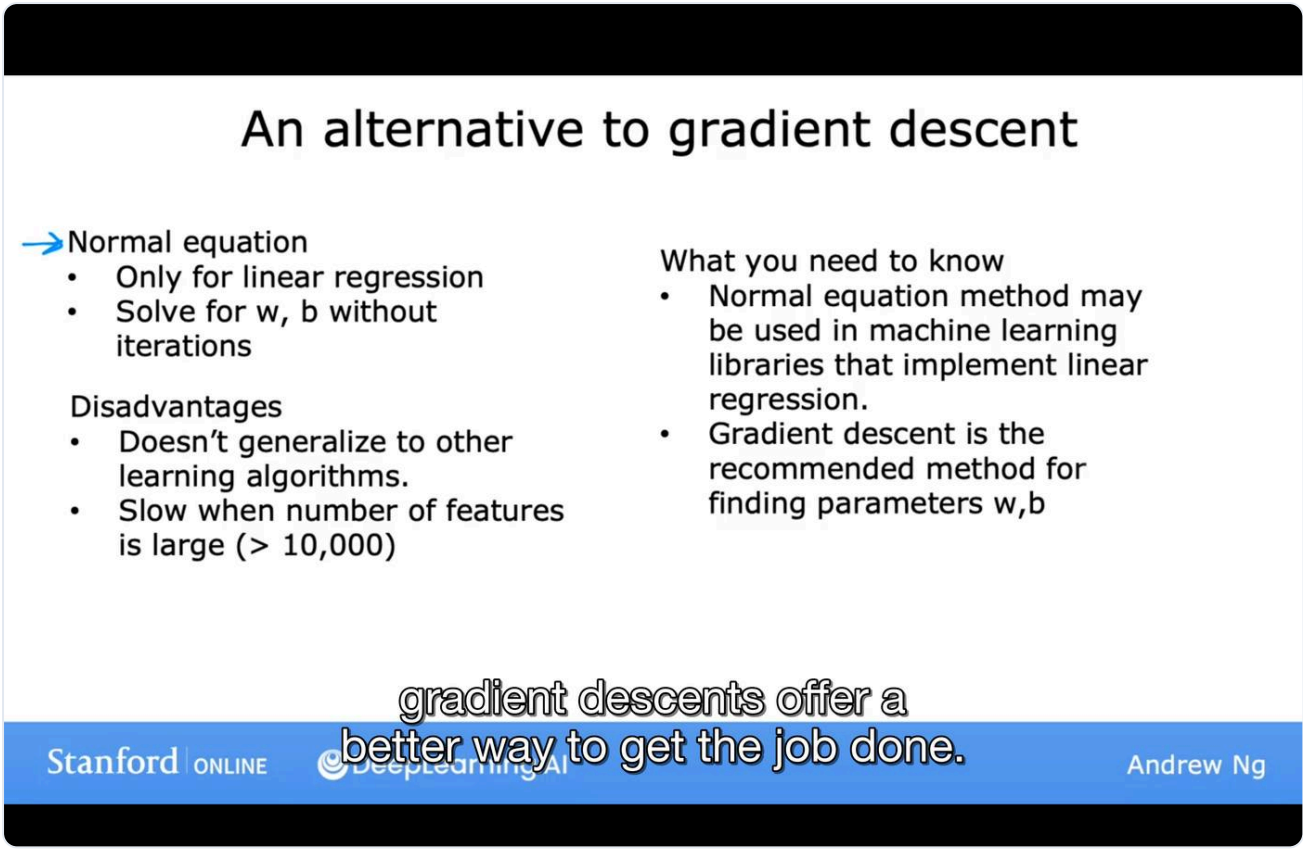


FIGURE 24.7 | An Alternative to Gradient Descent

But for most learning algorithms, including how you implement linear regression yourself, gradient descents offer a better way to get the job done. In the optional lab that follows this video, you'll see how to define a multiple regression model encode and also how to calculate the prediction f of x. You'll also see how to calculate the cost and implement gradient descent for a multiple linear regression model. This will be using Python's NumPy library. If any of the code looks very new, that's okay but you should feel free also to take a look at the previous optional lab that introduces NumPy and vectorization for a refresher of NumPy functions and how to implement those in code.

That's it. You now know multiple linear regression. This is probably the single most widely used learning algorithm in the world today. But there's more. With just a few tricks such as picking and scaling features appropriately and also choosing the learning rate alpha appropriately, you'd really make this work much better. Just a few more videos to go for this week. Let's go on to the next video to see those little tricks that will help you make multiple linear regression work much better.

LECTURE 25

Feature and Parameter Values

So welcome back. Let's take a look at some techniques that make great inter sense work much better. In this video you see a technique called feature scaling that will enable gradient descent to run much faster. Let's start by taking a look at the relationship between the size of a feature that is how big are the numbers for that feature and the size of its associated parameter. As a concrete example, let's predict the price of a house using two features x1 the size of the house and x2 the number of bedrooms. Let's say that x1 typically ranges from 300 to 2000 square feet. And x2 in the data set ranges from 0 to 5 bedrooms. So for this example, x1 takes on a relatively large range of values and x2 takes on a relatively small range of values.

Now let's take an example of a house that has a size of 2000 square feet has five bedrooms and a price of 500k or $500,000. For this one training example, what do you think are reasonable values for the size of parameters w1 and w2? Well, let's look at one possible set of parameters. Say w1 is 50 and w2 is 0.1 and b is 50 for the purposes of discussion. So in this case the estimated price in thousands of dollars is 100,000k here plus 0.5 k plus 50 k. Which is slightly over 100 million dollars.

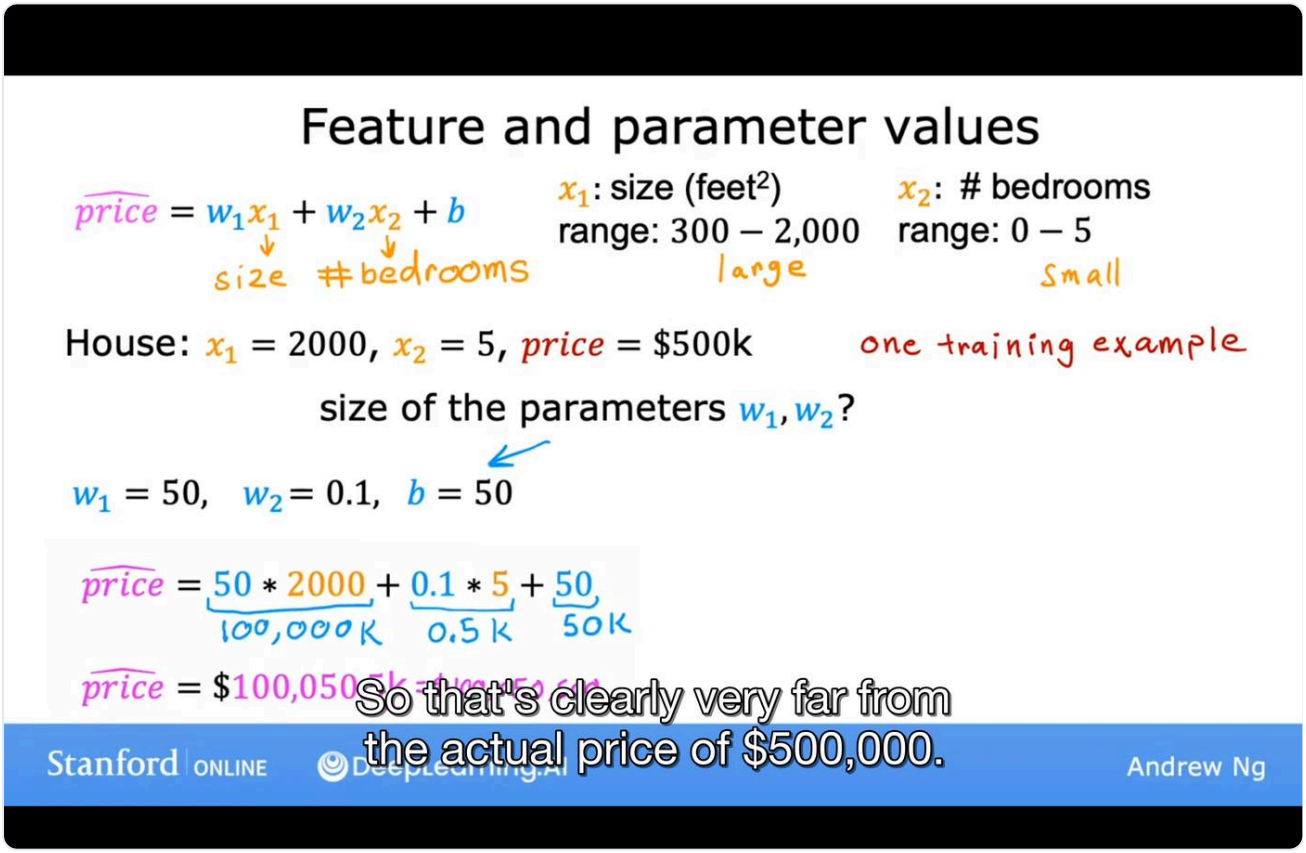


FIGURE 25.1 | Feature and Parameter Values

So that's clearly very far from the actual price of $500,000. And so this is not a very good set of parameter choices for w1 and w2. Now let's take a look at another possibility.

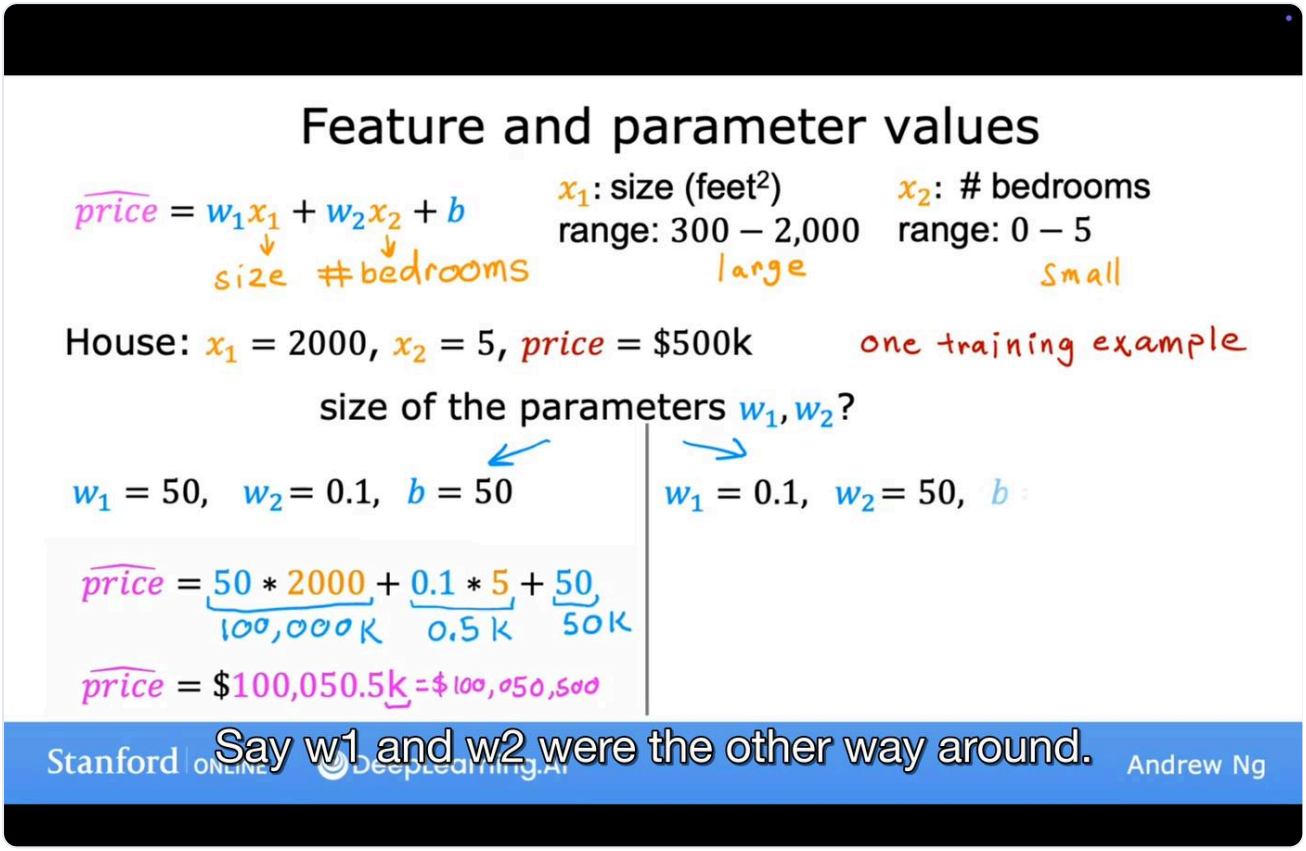


FIGURE 25.2 | Feature and Parameter Values

Say w1 and w2 were the other way around. W1 is 0.1 and w2 is 50 and b is still also 50. In this choice of w1 and w2, w1 is relatively small and w2 is relatively large, 50 is much bigger than 0.1. So here the predicted price is 0.1 times 2000 plus 50 times five plus 50. The first term becomes 200k, the second term becomes 250k, and the plus 50. So this version of the model predicts a price of $500,000 which is a much more reasonable estimate and happens to be the same price as the true price of the house.

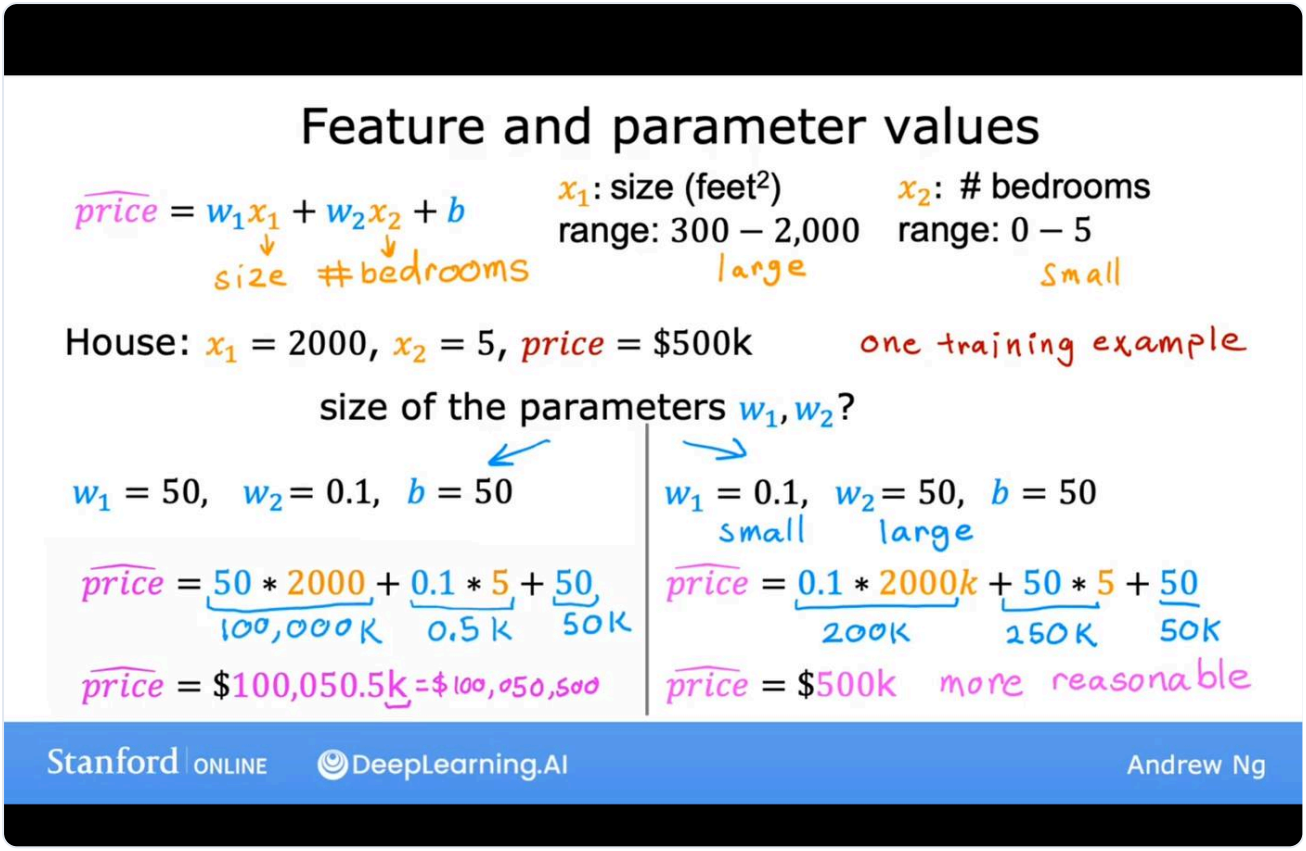


FIGURE 25.3 | Feature and Parameter Values

So hopefully you might notice that when a possible range of values of a feature is large, like the size and square feet which goes all the way up to 2000. It's more likely that a good model will learn to choose a relatively small parameter value, like 0.1. Likewise, when the possible values of the feature are small, like the number of bedrooms, then a reasonable value for its parameters will be relatively large like 50. So how does this relate to grading descent? Well, let's take a look at the scatter plot of the features where the size square feet is the horizontal axis x1 and the number of bedrooms exudes is on the vertical axis. If you plot the training data, you notice that the horizontal axis is on a much larger scale or much larger range of values compared to the vertical axis.

Next let's look at how the cost function might look in a contour plot.

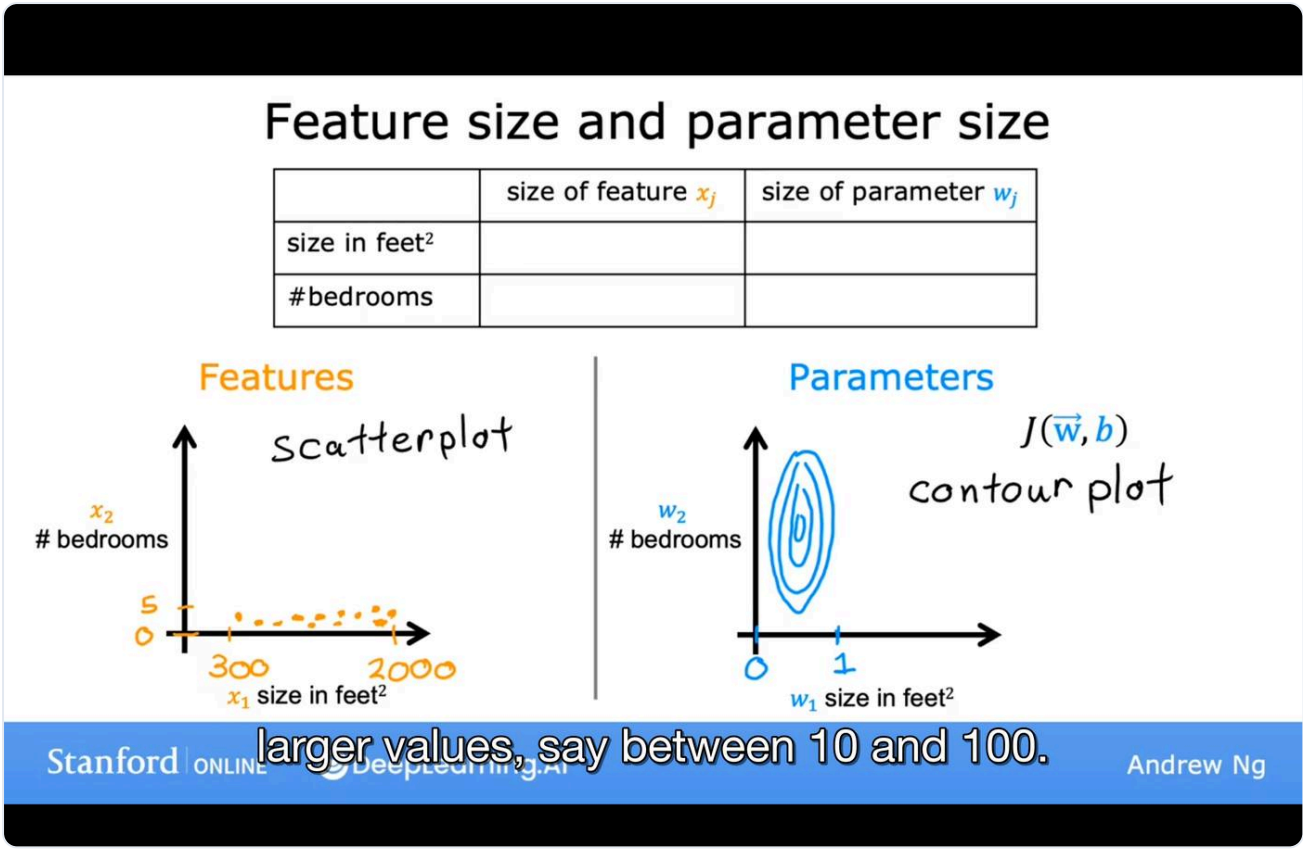


FIGURE 25.4

You might see a contour plot where the horizontal axis has a much narrower range, say between zero and one, whereas the vertical axis takes on much larger values, say between 10 and 100.

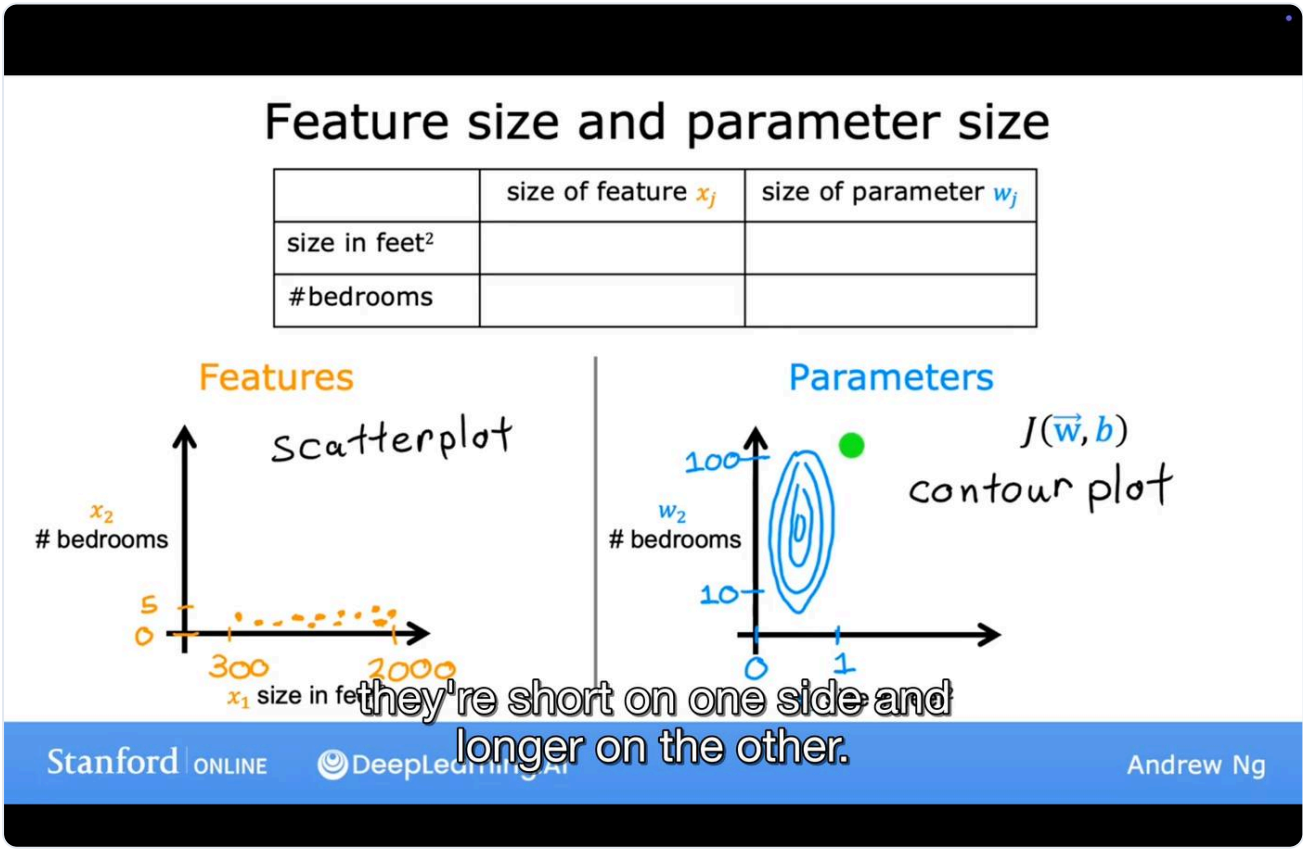


FIGURE 25.5 | Feature Size and Parameter Size

So the contours form ovals or ellipses and they're short on one side and longer on the other. And this is because a very small change to w1 can have a very large impact on the estimated price and that's a very large impact on the cost J. Because w1 tends to be multiplied by a very large number, the size and square feet. In contrast, it takes a much larger change in w2 in order to change the predictions much. And thus small changes to w2, don't change the cost function nearly as much. So where does this leave us? This is what might end up happening if you were to run great in dissent, if you were to use your training data as is. Because the contours are so tall and skinny gradient descent may end up bouncing back and forth for a long time before it can finally find its way to the global minimum.

In situations like this, a useful thing to do is to scale the features. This means performing some transformation of your training data so that x1 say might now range from 0 to 1 and x2 might also range from 0 to 1. So the data points now look more like this and you might notice that the scale of the plot on the bottom is now quite different than the one on top.

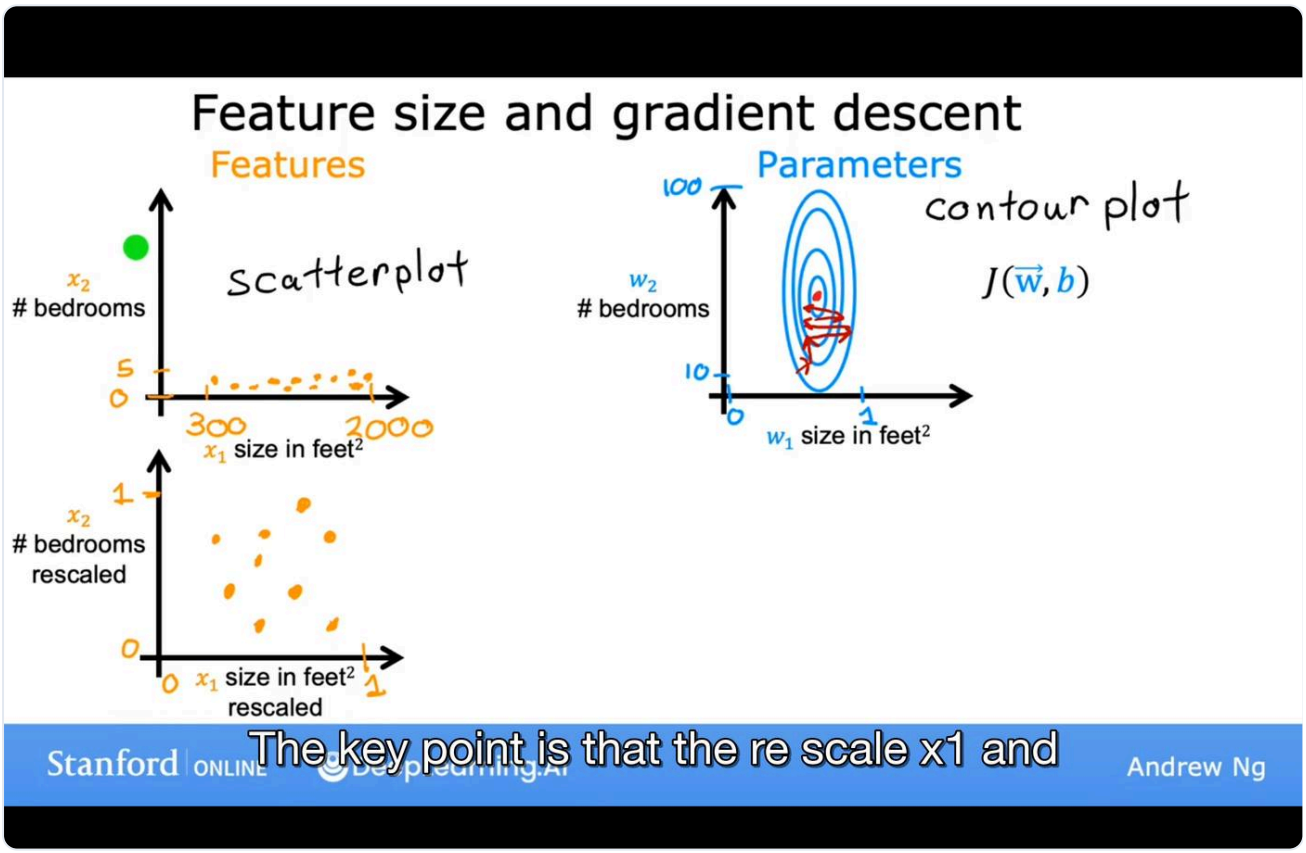


FIGURE 25.6 | Feature Size and Gradient Descent

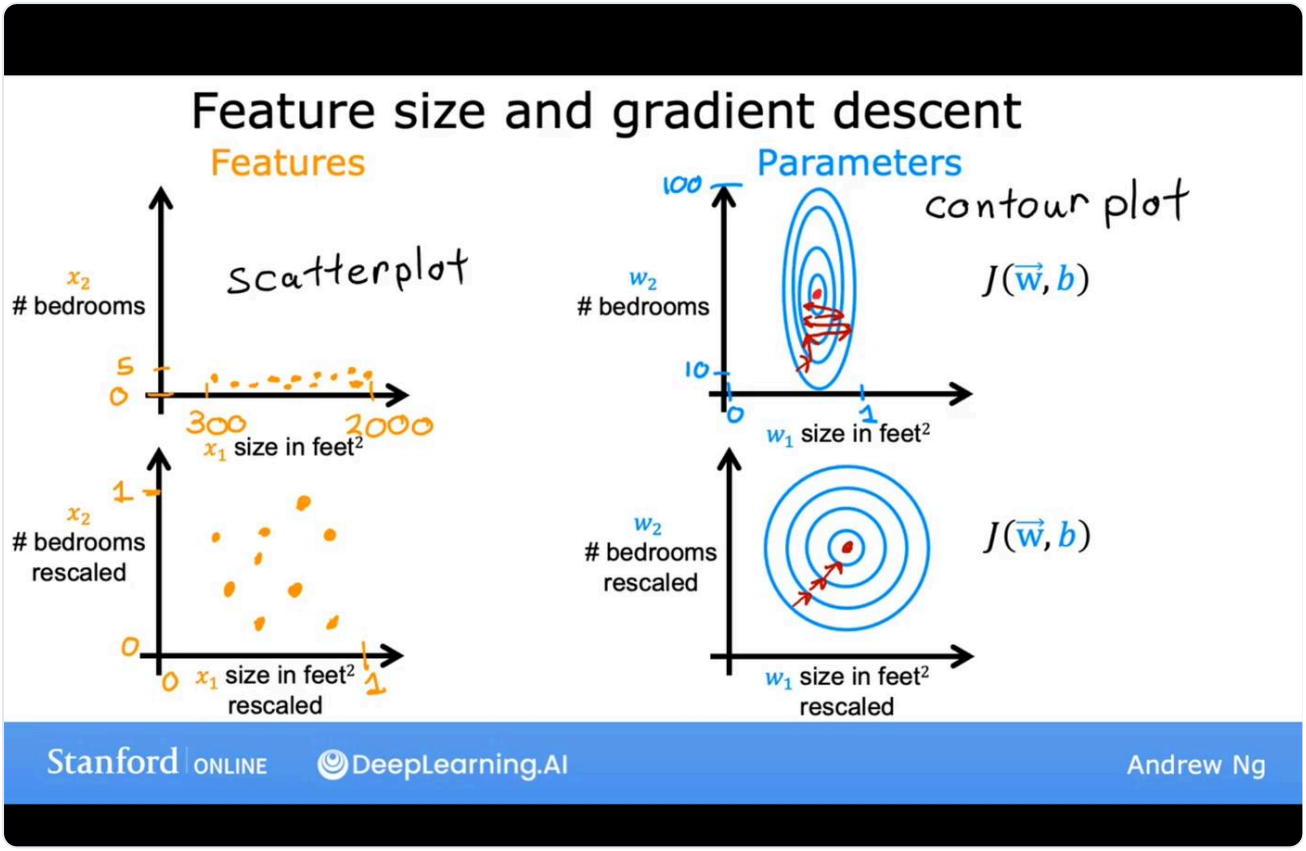


FIGURE 25.7 | Feature Size and Gradient Descent

The key point is that the re scale x1 and x2 are both now taking comparable ranges of values to each other. And if you run gradient descent on a cost function to find on this, re scaled x1 and x2 using this transformed data, then the contours will look more like this more like circles and less tall and skinny. And gradient descent can find a much more direct path to the global minimum. So to recap, when you have different features that take on very different ranges of values, it can cause gradient descent to run slowly but re scaling the different features so they all take on comparable range of values. because speed, upgrade and dissent significantly.

How do you actually do this? Let's take a look at that in the next video.

LECTURE 26

Feature Scaling

Let's look at how you can implement feature scaling, to take features that take on very different ranges of values and skill them to have comparable ranges of values to each other. How do you actually scale features? Well, if x_1 ranges from 3-2,000, one way to get a scale version of x_1 is to take each original x1_ value and divide by 2,000, the maximum of the range. The scale x_1 will range from 0.15 up to one. Similarly, since x_2 ranges from 0-5, you can calculate a scale version of x_2 by taking each original x_2 and dividing by five, which is again the maximum. So the scale is x_2 will now range from 0-1.

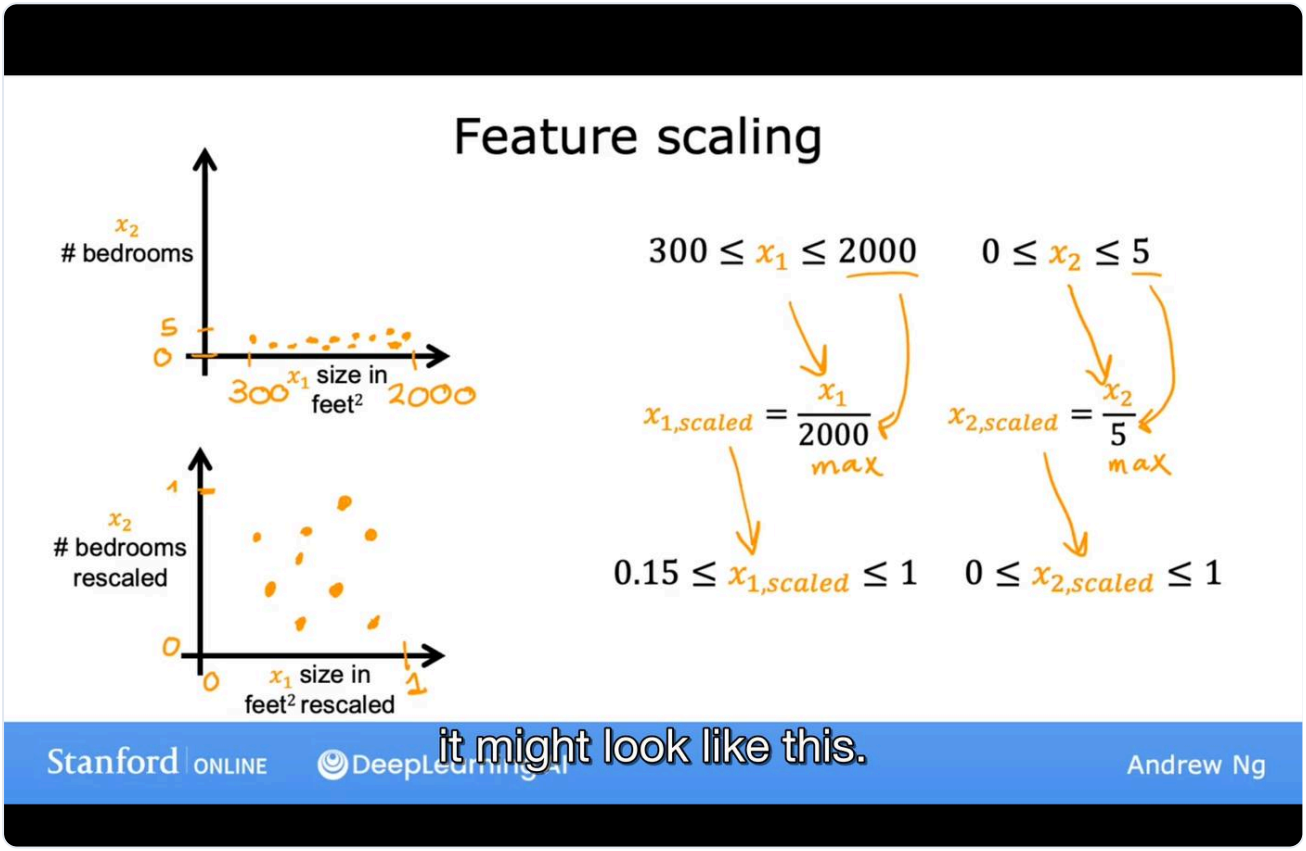


FIGURE 26.1 | Feature Scaling

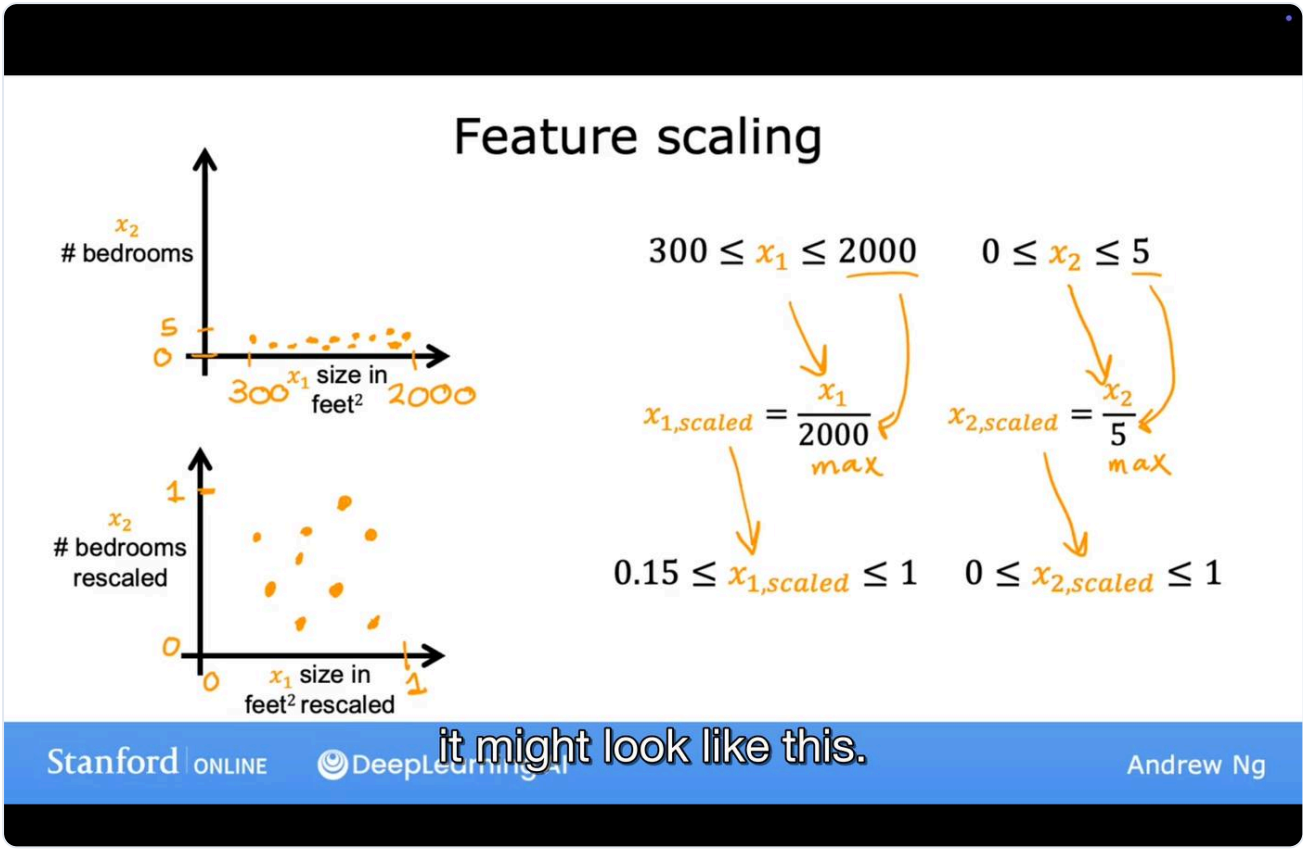


FIGURE 26.2 | Feature Scaling

If you plot the scale to x_1 and x_2 on a graph, it might look like this. In addition to dividing by the maximum, you can also do what's called mean normalization. What this looks like is, you start with the original features and then you re-scale them so that both of them are centered around zero. Whereas before they only had values greater than zero, now they have both negative and positive values that may be usually between negative one and plus one. To calculate the mean normalization of x_1, first find the average, also called the mean of x_1 on your training set, and let's call this mean Mu_1, with this being the Greek alphabets Mu. For example, you may find that the average of feature 1, Mu_1 is 600 square feet.

Let's take each x_1, subtract the mean Mu_1, and then let's divide by the difference 2,000 minus 300, where 2,000 is the maximum and 300 the minimum, and if you do this, you get the normalized x_1 to range from negative 0.18-0.82. Similarly, to mean normalized x_2, you can calculate the average of feature 2. For instance, Mu_2 may be 2.3. Then you can take each x_2, subtract Mu_2 and divide by 5 minus 0. Again, the max 5 minus the mean, which is 0. The mean normalized x_2 now ranges from negative 0.46-0 54. If you plot the training data using the mean normalized x_1 and x_2, it might look like this.

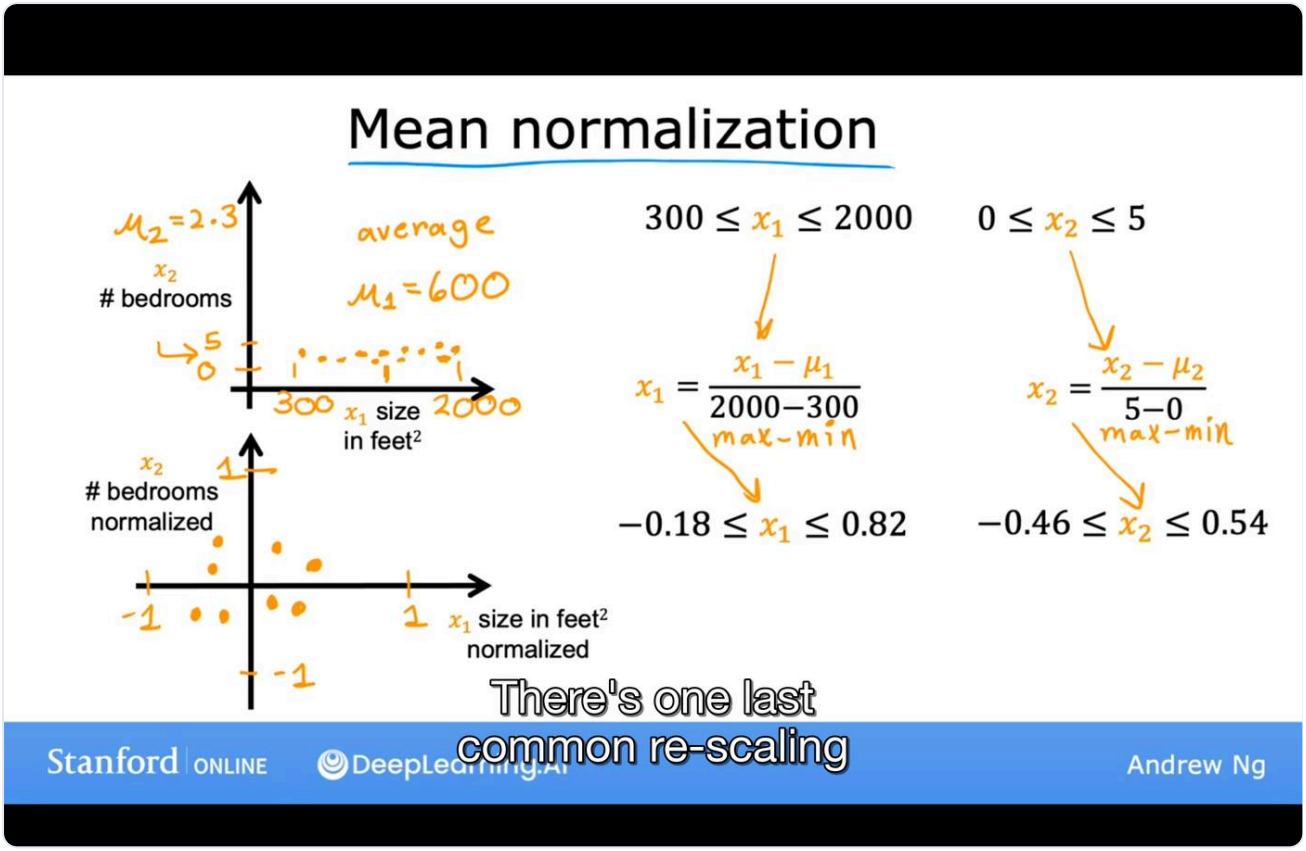


FIGURE 26.3 | Mean Normalization

There's one last common re-scaling method call Z-score normalization. To implement Z-score normalization, you need to calculate something called the standard deviation of each feature. If you don't know what the standard deviation is, don't worry about it, you won't need to know it for this course. Or if you've heard of the normal distribution or the bell-shaped curve, sometimes also called the Gaussian distribution, this is what the standard deviation for the normal distribution looks like. But if you haven't heard of this, you don't need to worry about that either. But if you do know what is the standard deviation, then to implement a Z-score normalization, you first calculate the mean Mu, as well as the standard deviation, which is often denoted by the lowercase Greek alphabet Sigma of each feature.

For instance, maybe feature 1 has a standard deviation of 450 and mean 600, then to Z-score normalize x_1, take each x_1, subtract Mu_1, and then divide by the standard deviation, which I'm going to denote as Sigma 1. What you may find is that the Z-score normalized x_1 now ranges from negative 0.67-3.1.

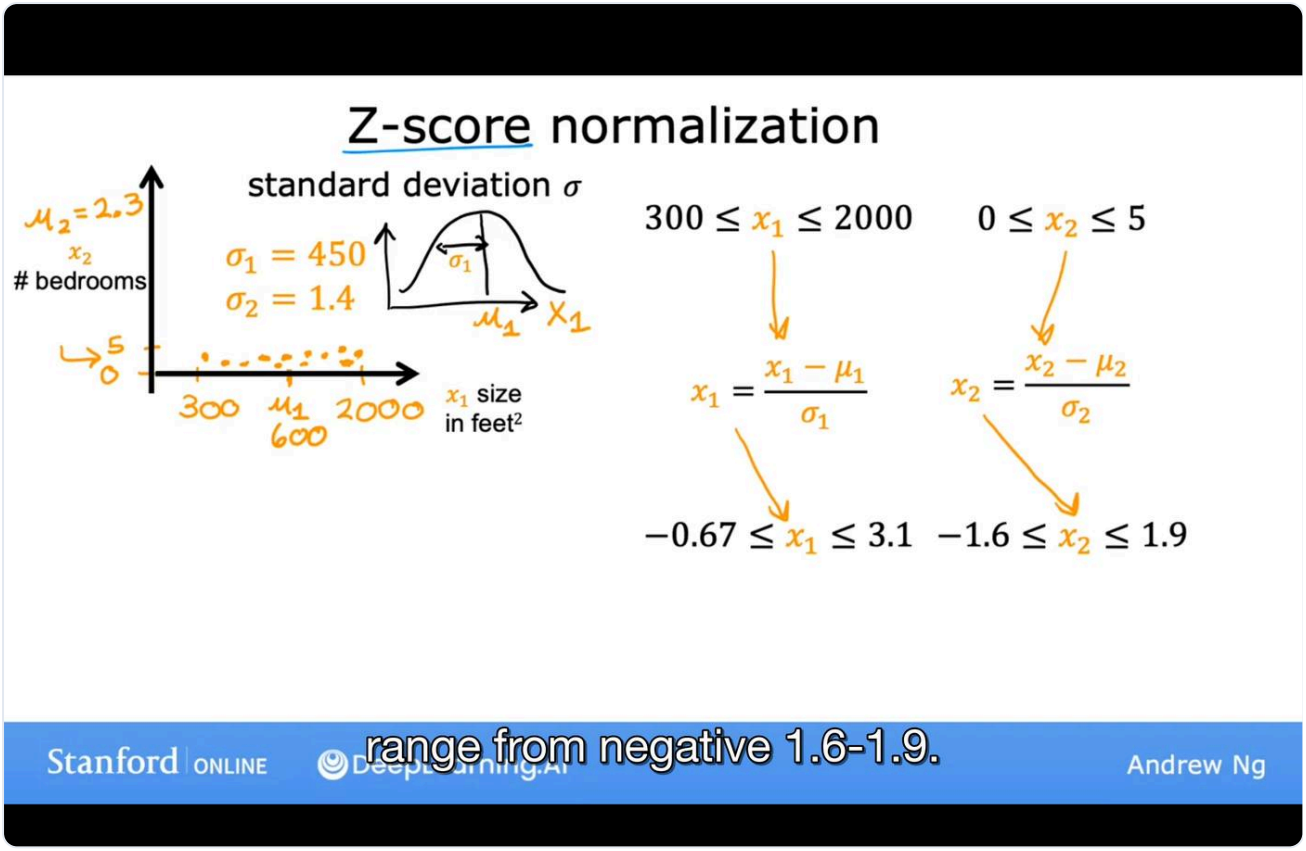


FIGURE 26.4

Similarly, if you calculate the second features standard deviation to be 1.4 and mean to be 2.3, then you can compute x_2 minus Mu_2 divided by Sigma_2, and in this case, the Z-score normalized by x_2 might now range from negative 1.6-1.9.

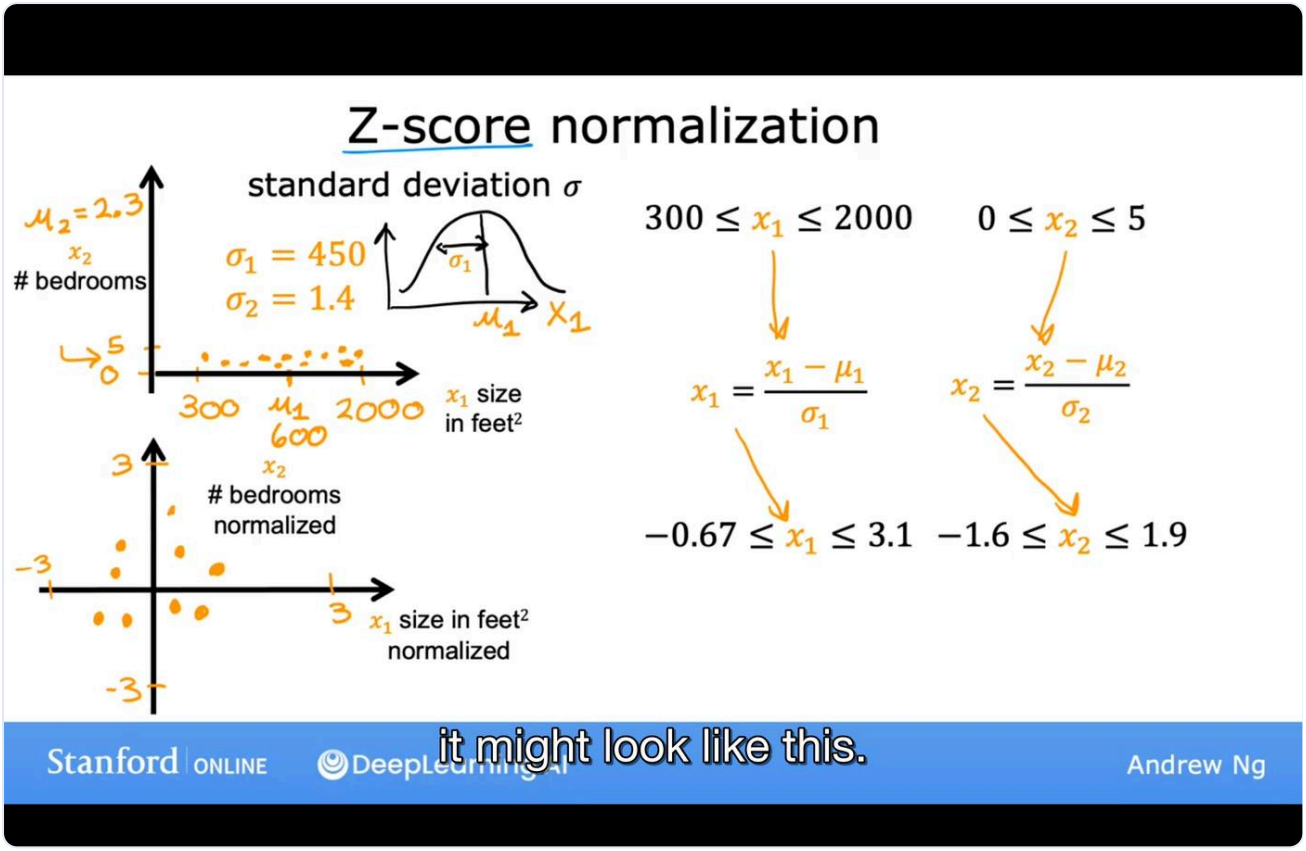


FIGURE 26.5

If you plot the training data on the normalized x_1 and x_2 on a graph, it might look like this. As a rule of thumb, when performing feature scaling, you might want to aim for getting the features to range from maybe anywhere around negative one to somewhere around plus one for each feature x. But these values, negative one and plus one can be a little bit loose. If the features range from negative three to plus three or negative 0.3 to plus 0.3, all of these are completely okay. If you have a feature x_1 that winds up being between zero and three, that's not a problem. You can re-scale it if you want, but if you don't re-scale it, it should work okay too.

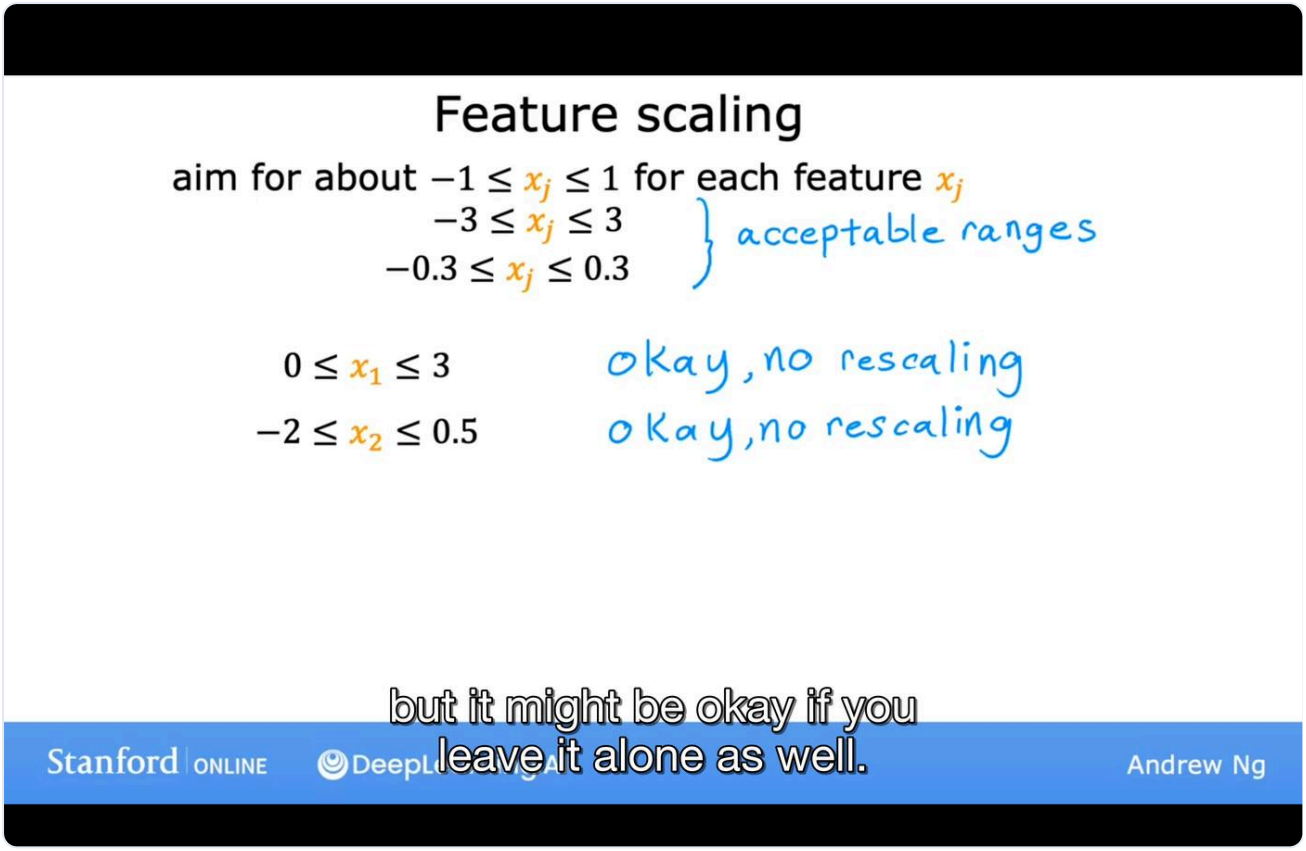


FIGURE 26.6 | Feature Scaling

Or if you have a different feature, x_2, whose values are between negative 2 and plus 0.5, again, that's okay, no harm re-scaling it, but it might be okay if you leave it alone as well.

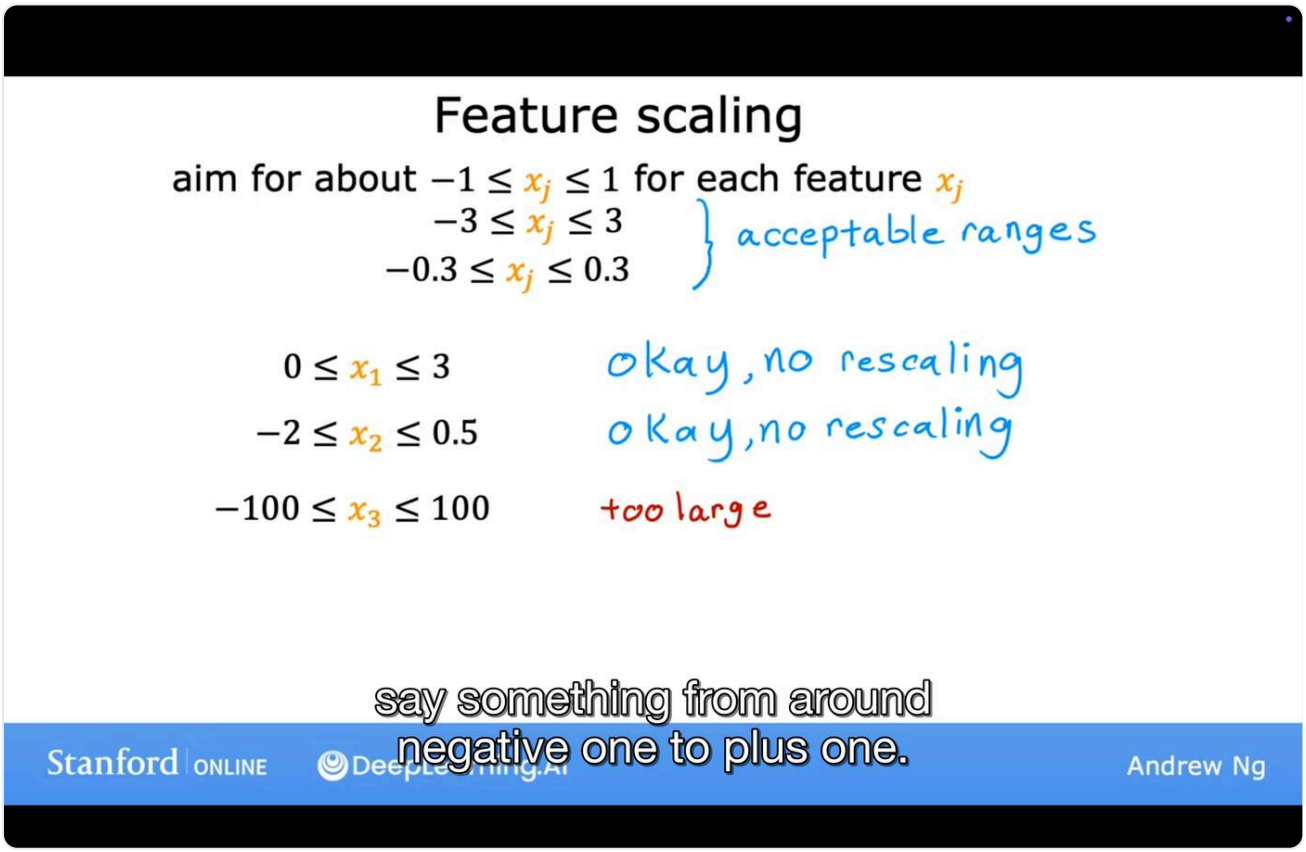


FIGURE 26.7 | Feature Scaling

But if another feature, like x_3 here, ranges from negative 100 to plus 100, then this takes on a very different range of values, say something from around negative one to plus one.

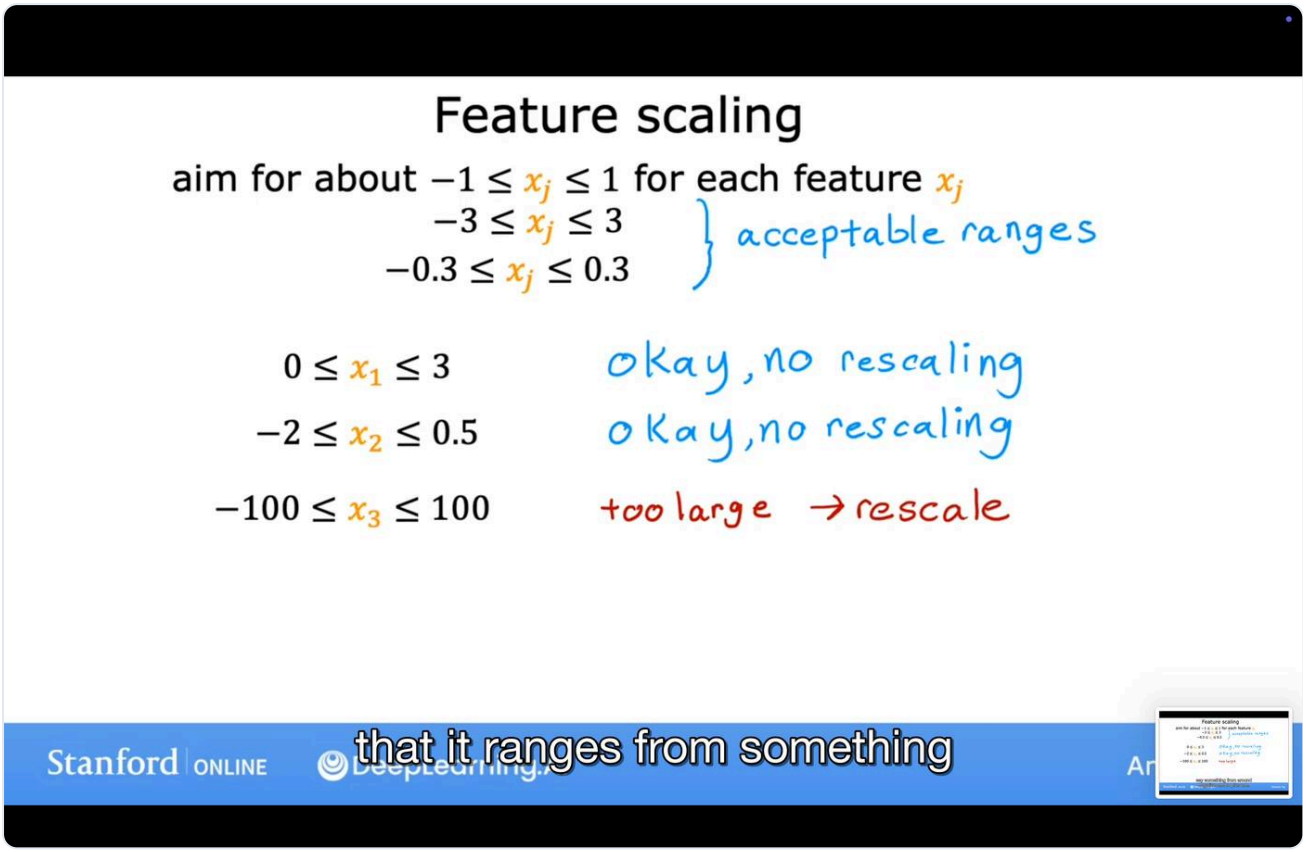


FIGURE 26.8 | Feature Scaling

You're probably better off re-scaling this feature x_3 so that it ranges from something closer to negative one to plus one. Similarly, if you have a feature x_4 that takes on really small values, say between negative 0.001 and plus 0.001, then these values are so small.

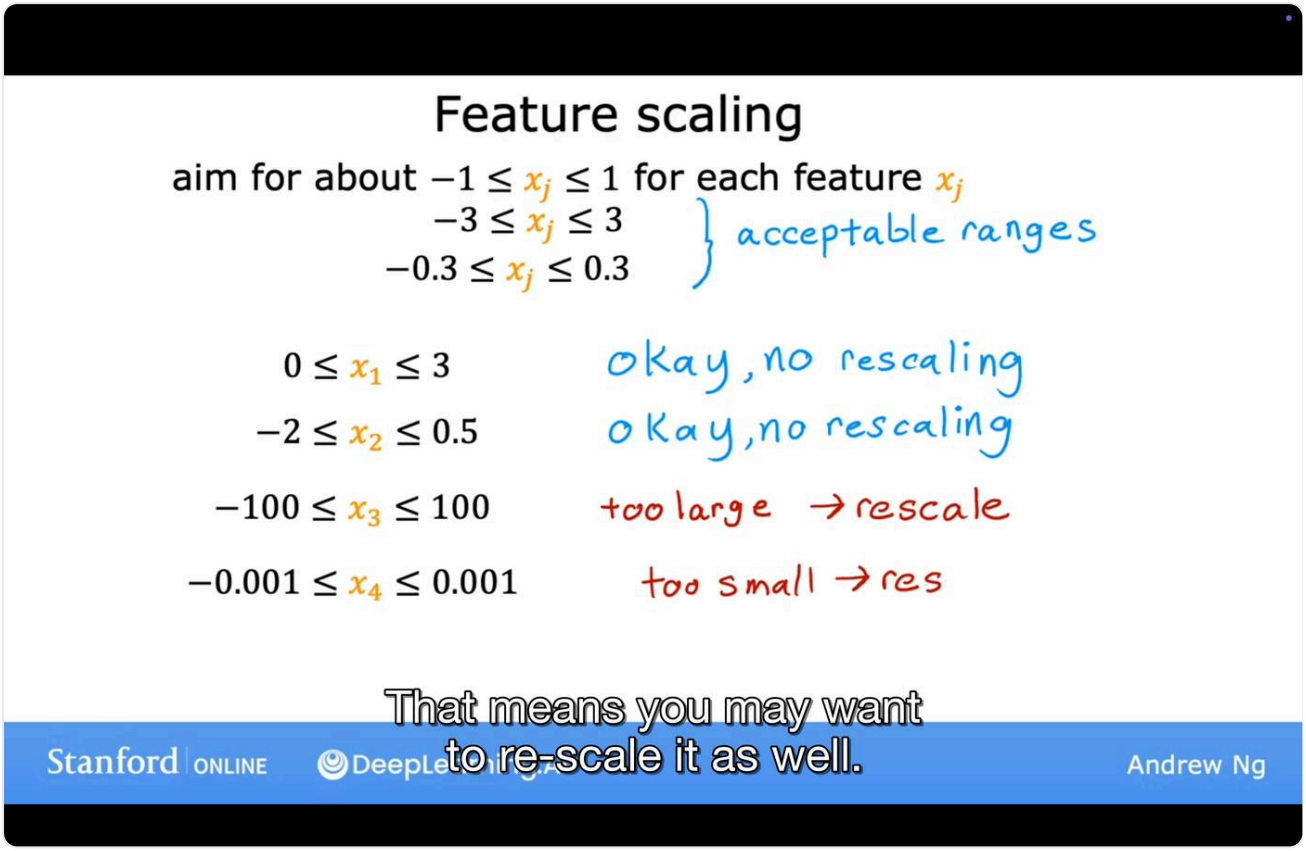


FIGURE 26.9 | Feature Scaling

That means you may want to re-scale it as well.

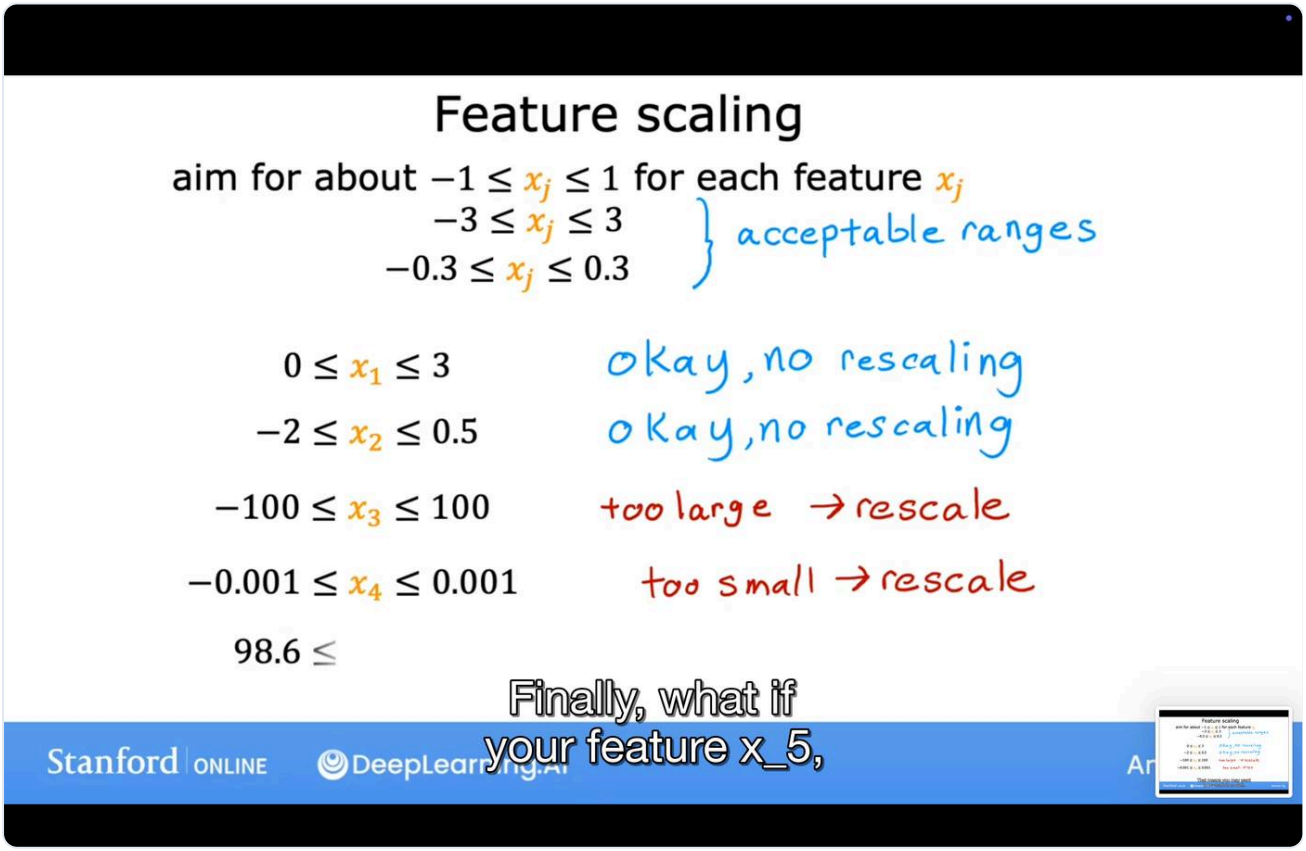


FIGURE 26.10

Finally, what if your feature x_5, such as measurements of a hospital patients by the temperature ranges from 98.6-105 degrees Fahrenheit? In this case, these values are around 100, which is actually pretty large compared to other scale features, and this will actually cause gradient descent to run more slowly.

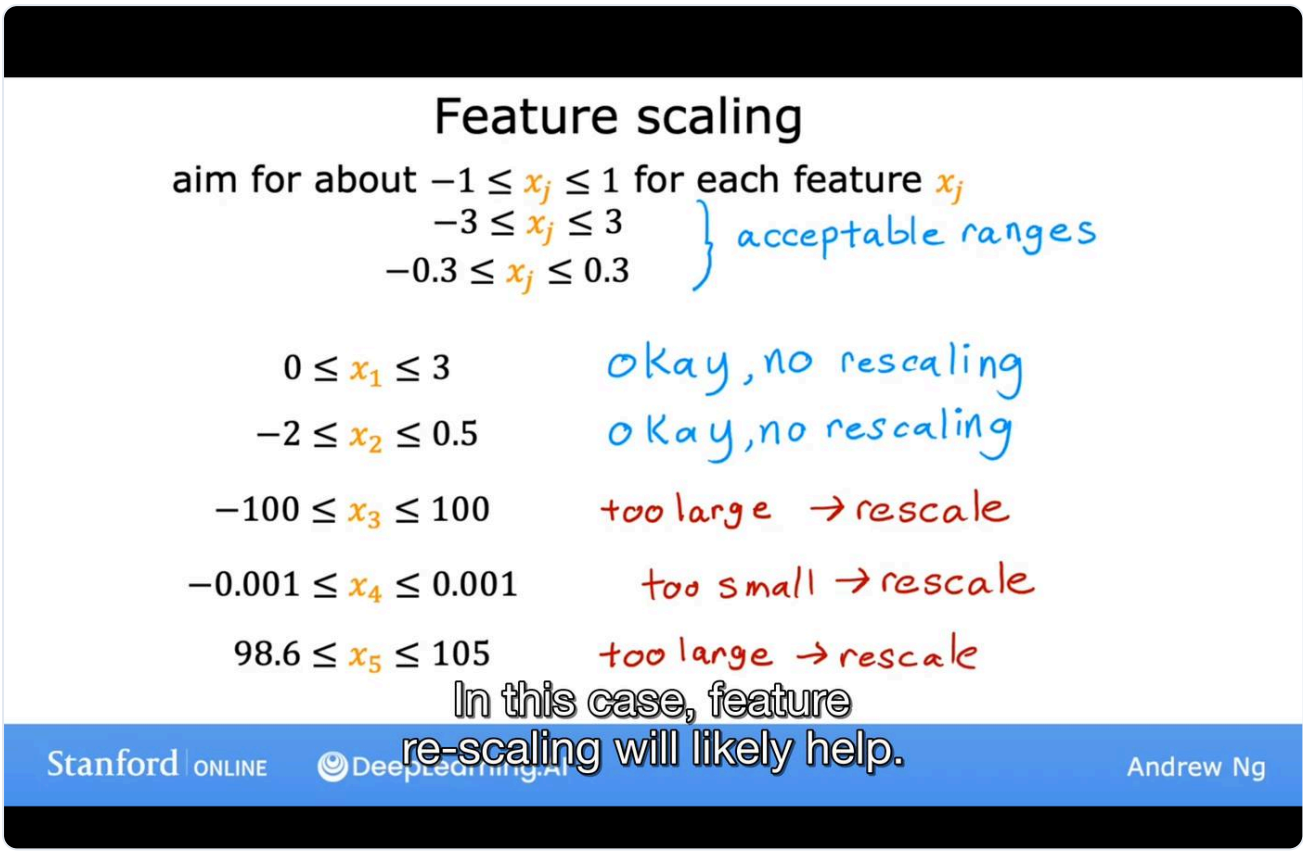


FIGURE 26.11 | Feature Scaling

In this case, feature re-scaling will likely help. There's almost never any harm to carrying out feature re-scaling. When in doubt, I encourage you to just carry it out. That's it for feature scaling. With this little technique, you'll often be able to get gradient descent to run much faster. That's features scaling. With or without feature scaling, when you run gradient descent, how can you know, how can you check if gradient descent is really working? If it is finding you the global minimum or something close to it. In the next video, let's take a look at how to recognize if gradient descent is converging, and then in the video after that, this will lead to discussion of how to choose a good learning rate for gradient descent.

LECTURE 27

Gradient Descent

When running gradient descent, how can you tell if it is converging? That is, whether it's helping you to find parameters close to the global minimum of the cost function. By learning to recognize what a well-running implementation of gradient descent looks like, we will also, in a later video, be better able to choose a good learning rate Alpha. Let's take a look. As a reminder, here's the gradient descent rule. One of the key choices is the choice of the learning rate Alpha.

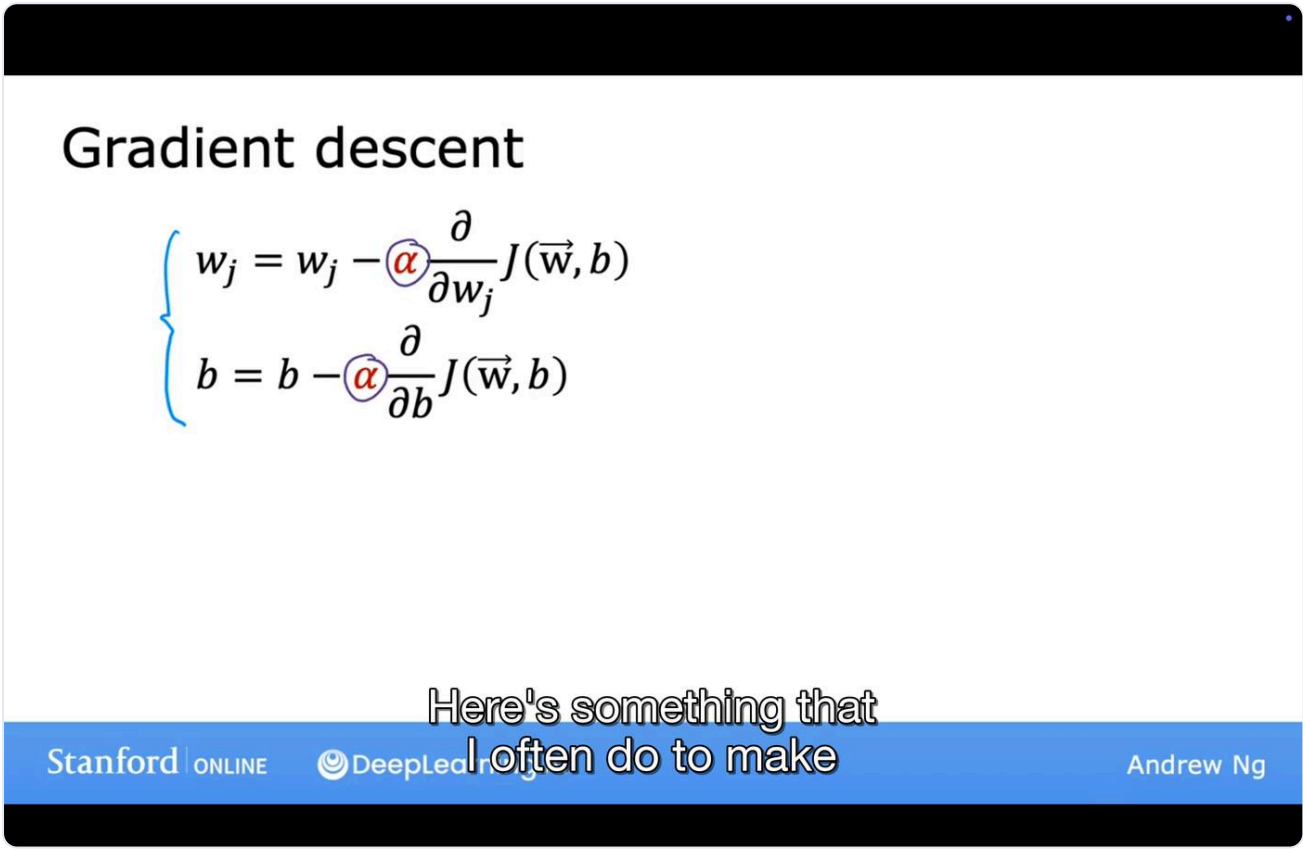


FIGURE 27.1

Here's something that I often do to make sure that gradient descent is working well. Recall that the job of gradient descent is to find parameters w and b that hopefully minimize the cost function J. What I'll often do is plot the cost function J, which is calculated on the training set, and I plot the value of J at each iteration of gradient descent. Remember that each iteration means after each simultaneous update of the parameters w and b. In this plot, the horizontal axis is the number of iterations of gradient descent that you've run so far. You may get a curve that looks like this. Notice that the horizontal axis is the number of iterations of gradient descent and not a parameter like w or b.

This differs from previous graphs you've seen where the vertical axis was cost J and the horizontal axis was a single parameter like w or b. This curve is also called a learning curve.

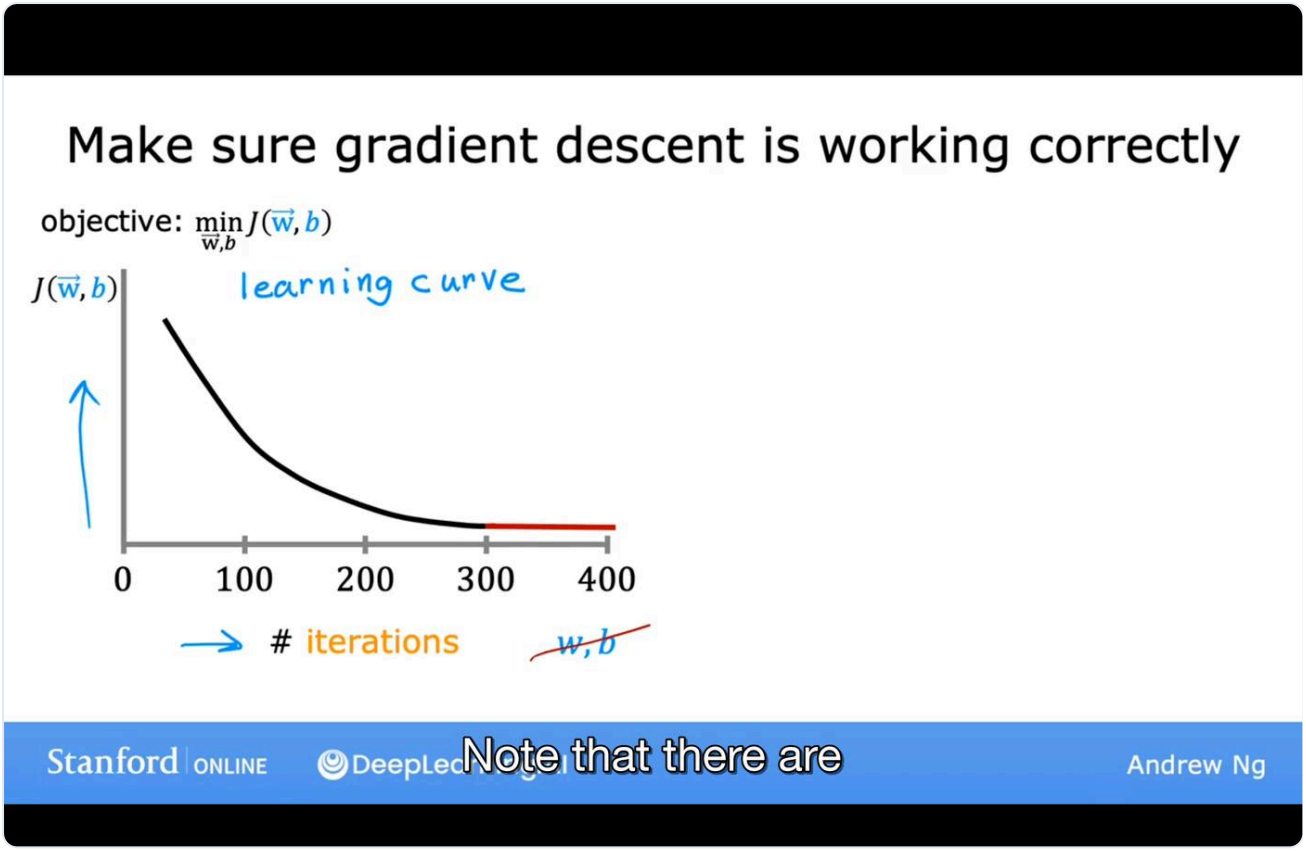


FIGURE 27.2 | Make Sure Gradient Descent Is Working Correctly

Note that there are a few different types of learning curves used in machine learning, and you see some of the types later in this course as well. Concretely, if you look here at this point on the curve, this means that after you've run gradient descent for 100 iterations, meaning 100 simultaneous updates of the parameters, you have some learned values for w and b.

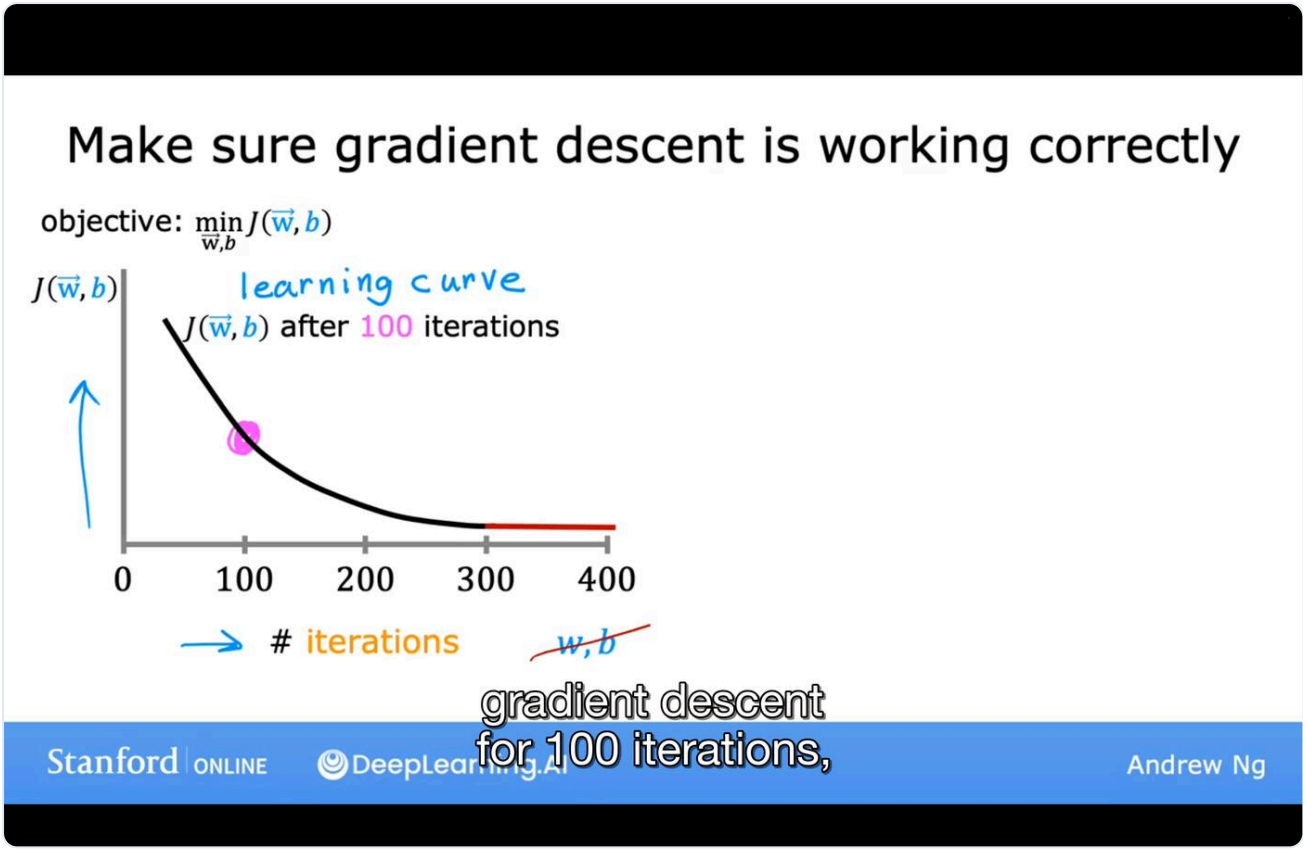


FIGURE 27.3 | Make Sure Gradient Descent Is Working Correctly

If you compute the cost J, w, b for those values of w and b, the ones you got after 100 iterations, you get this value for the cost J. That is this point on the vertical axis.

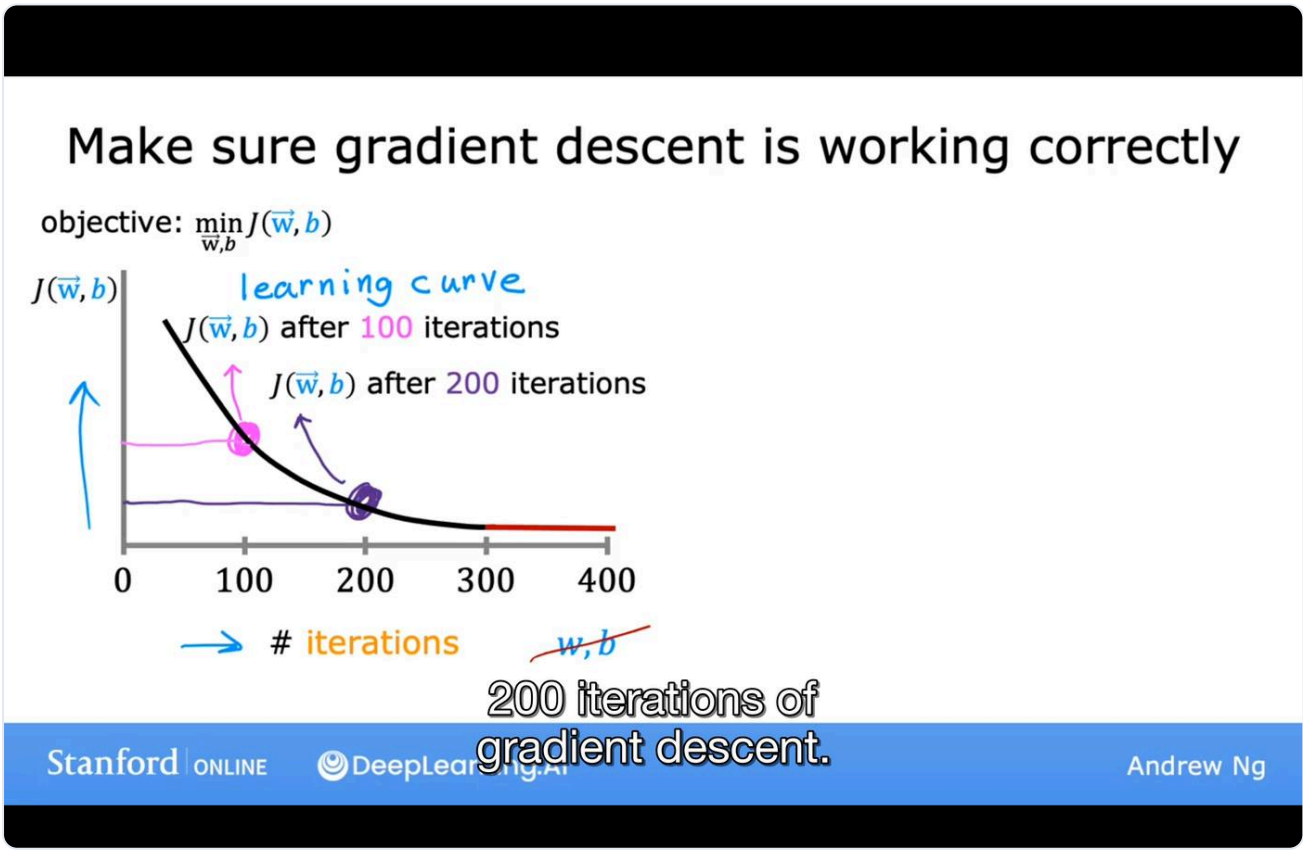


FIGURE 27.4 | Make Sure Gradient Descent Is Working Correctly

This point here corresponds to the value of J for the parameters that you got after 200 iterations of gradient descent. Looking at this graph helps you to see how your cost J changes after each iteration of gradient descent.

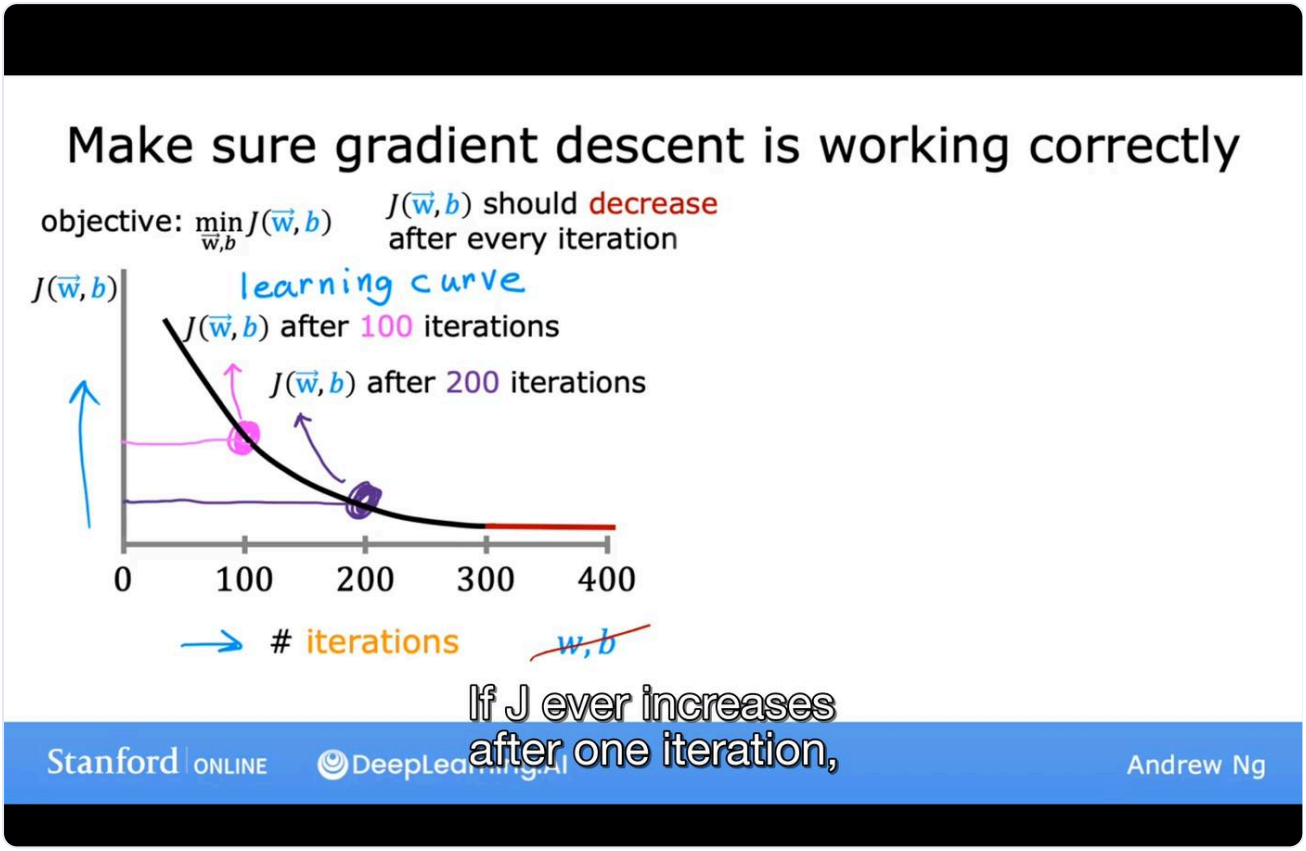


FIGURE 27.5 | Make Sure Gradient Descent Is Working Correctly

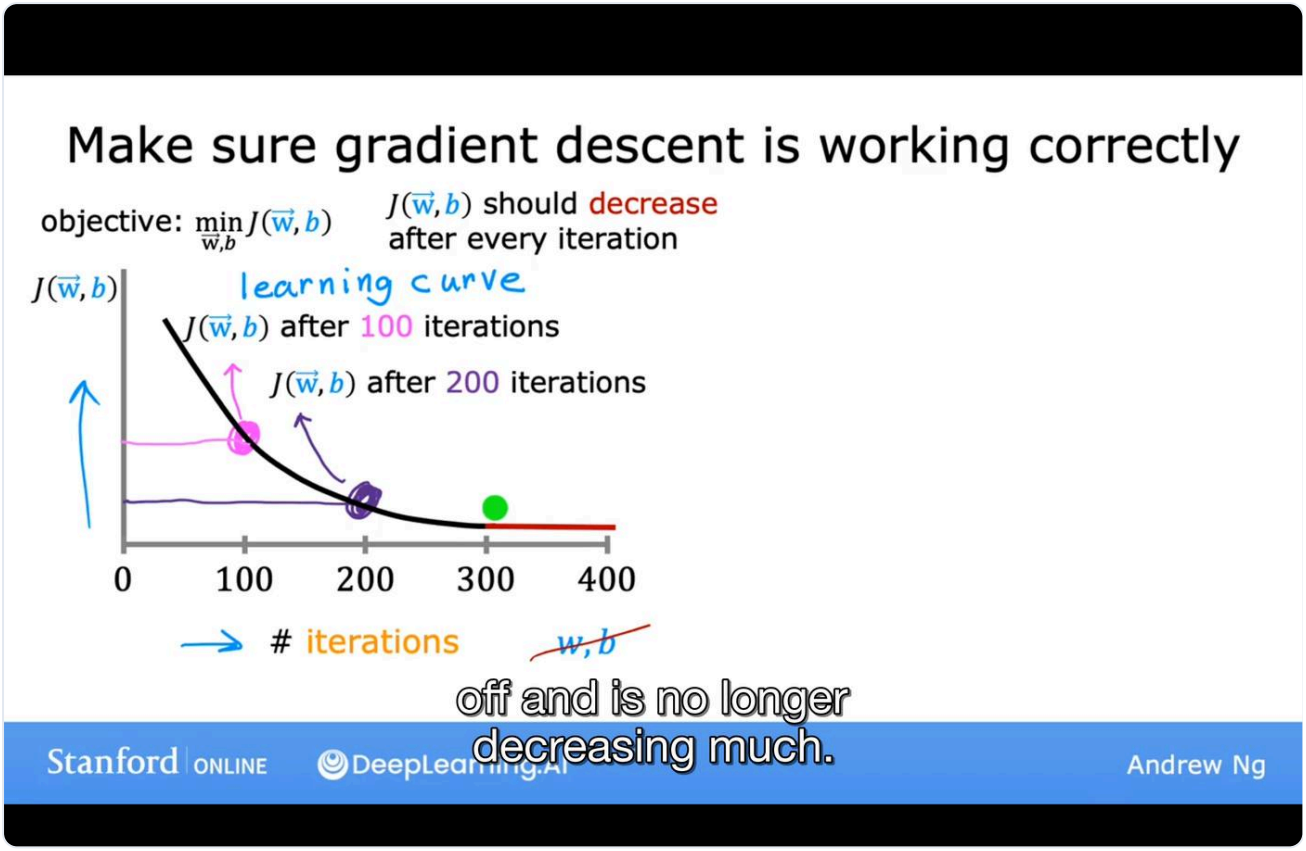


FIGURE 27.6 | Make Sure Gradient Descent Is Working Correctly

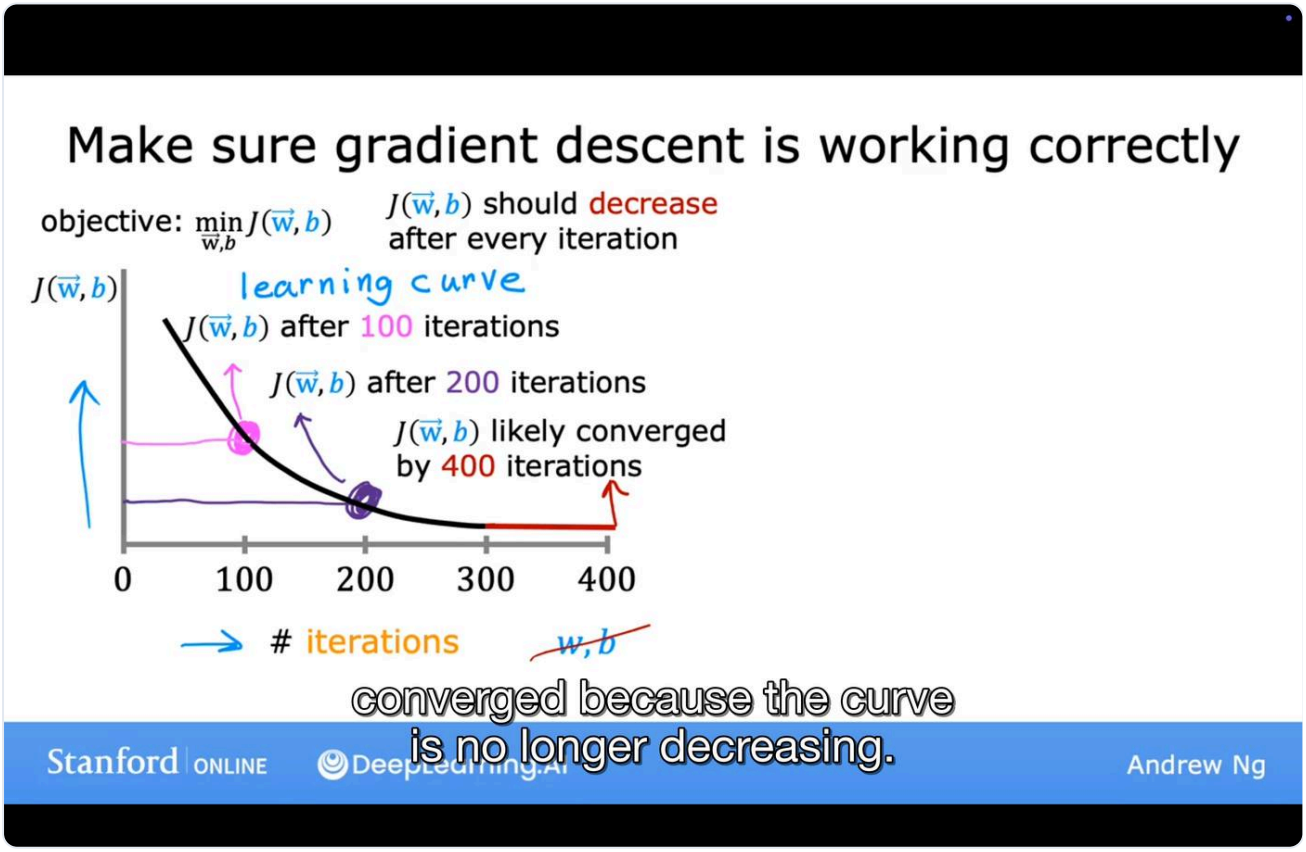


FIGURE 27.7 | Make Sure Gradient Descent Is Working Correctly

If gradient descent is working properly, then the cost J should decrease after every single iteration. If J ever increases after one iteration, that means either Alpha is chosen poorly, and it usually means Alpha is too large, or there could be a bug in the code. Another useful thing that this part can tell you is that if you look at this curve, by the time you reach maybe 300 iterations also, the cost J is leveling off and is no longer decreasing much. By 400 iterations, it looks like the curve has flattened out. This means that gradient descent has more or less converged because the curve is no longer decreasing. Looking at this learning curve, you can try to spot whether or not gradient descent is converging.

By the way, the number of iterations that gradient descent takes a conversion can vary a lot between different applications.

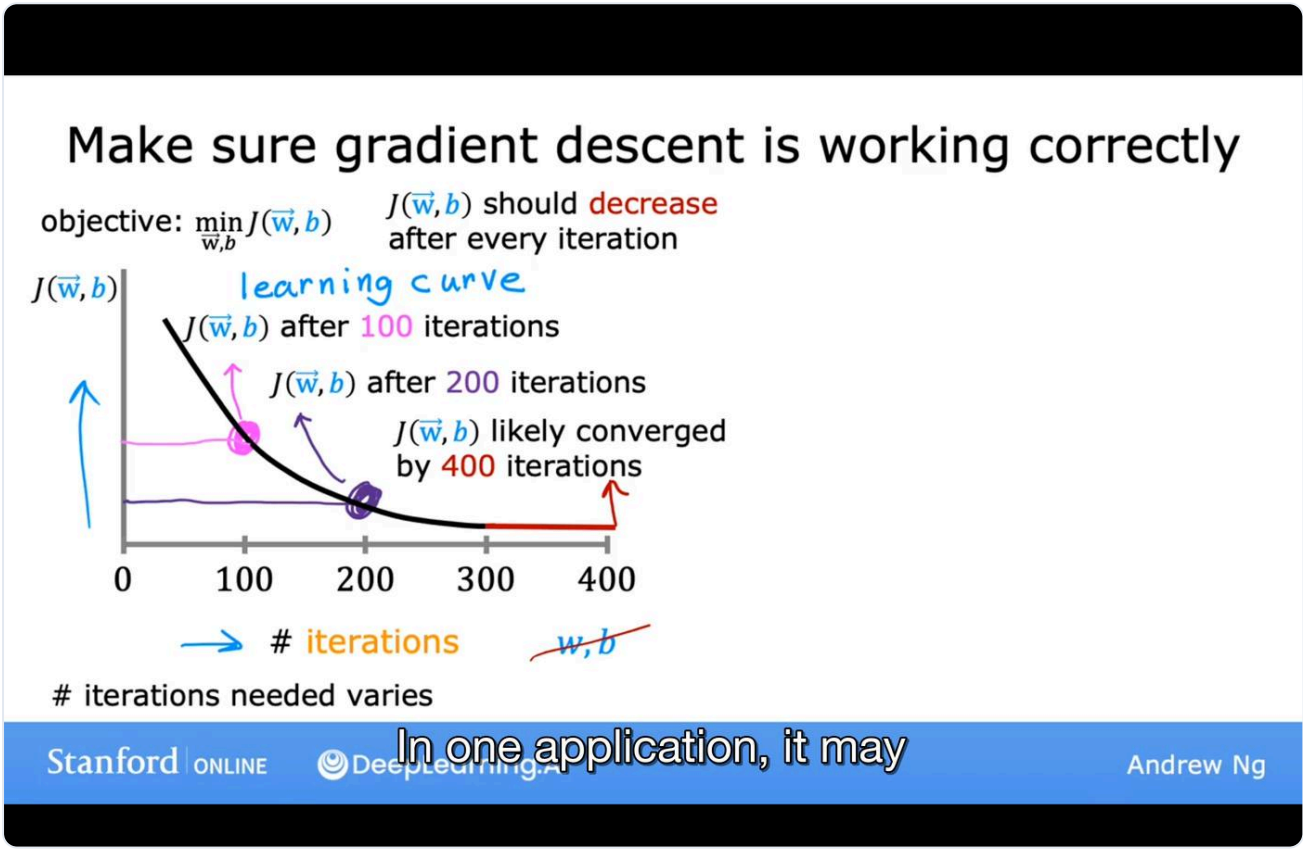


FIGURE 27.8 | Make Sure Gradient Descent Is Working Correctly

In one application, it may converge after just 30 iterations. For a different application, it could take 1,000 or 100,000 iterations. It turns out to be very difficult to tell in advance how many iterations gradient descent needs to converge, which is why you can create a graph like this, a learning curve. Try to find out when you can start training your particular model. Another way to decide when your model is done training is with an automatic convergence test. Here is the Greek alphabet epsilon.

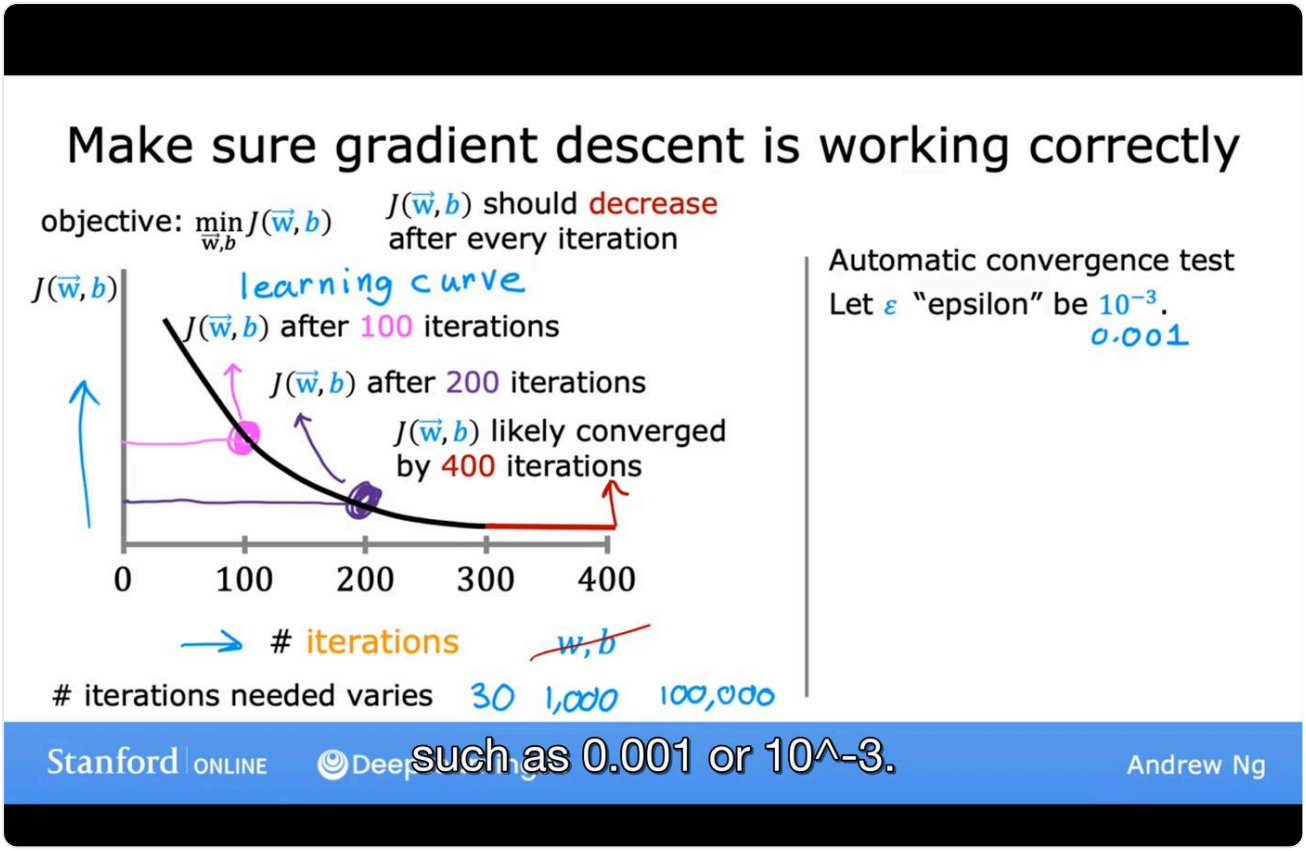


FIGURE 27.9 | Make Sure Gradient Descent Is Working Correctly

Let's let epsilon be a variable representing a small number, such as 0.001 or 10^-3. If the cost J decreases by less than this number epsilon on one iteration, then you're likely on this flattened part of the curve that you see on the left and you can declare convergence.

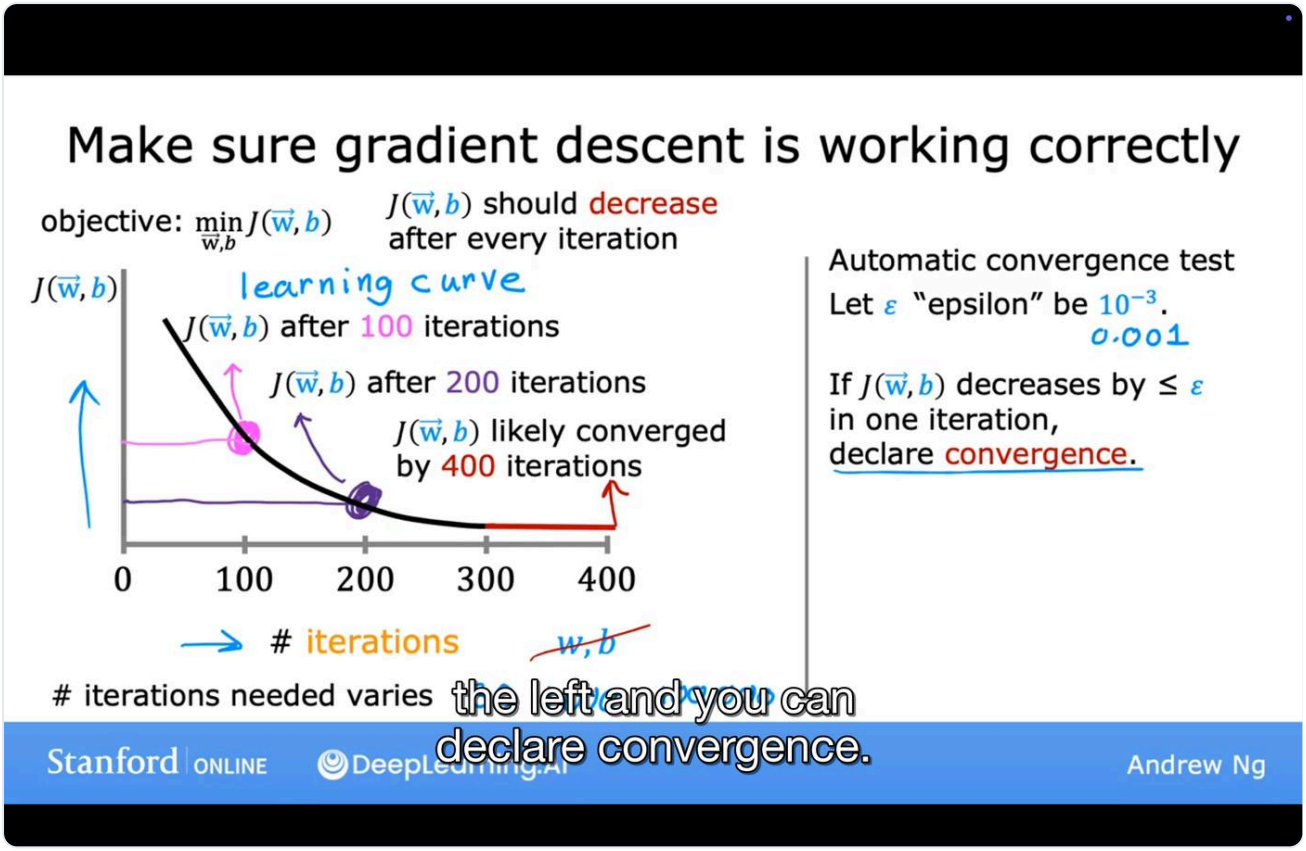


FIGURE 27.10 | Make Sure Gradient Descent Is Working Correctly

Remember, convergence, hopefully in the case that you found parameters w and b that are close to the minimum possible value of J.

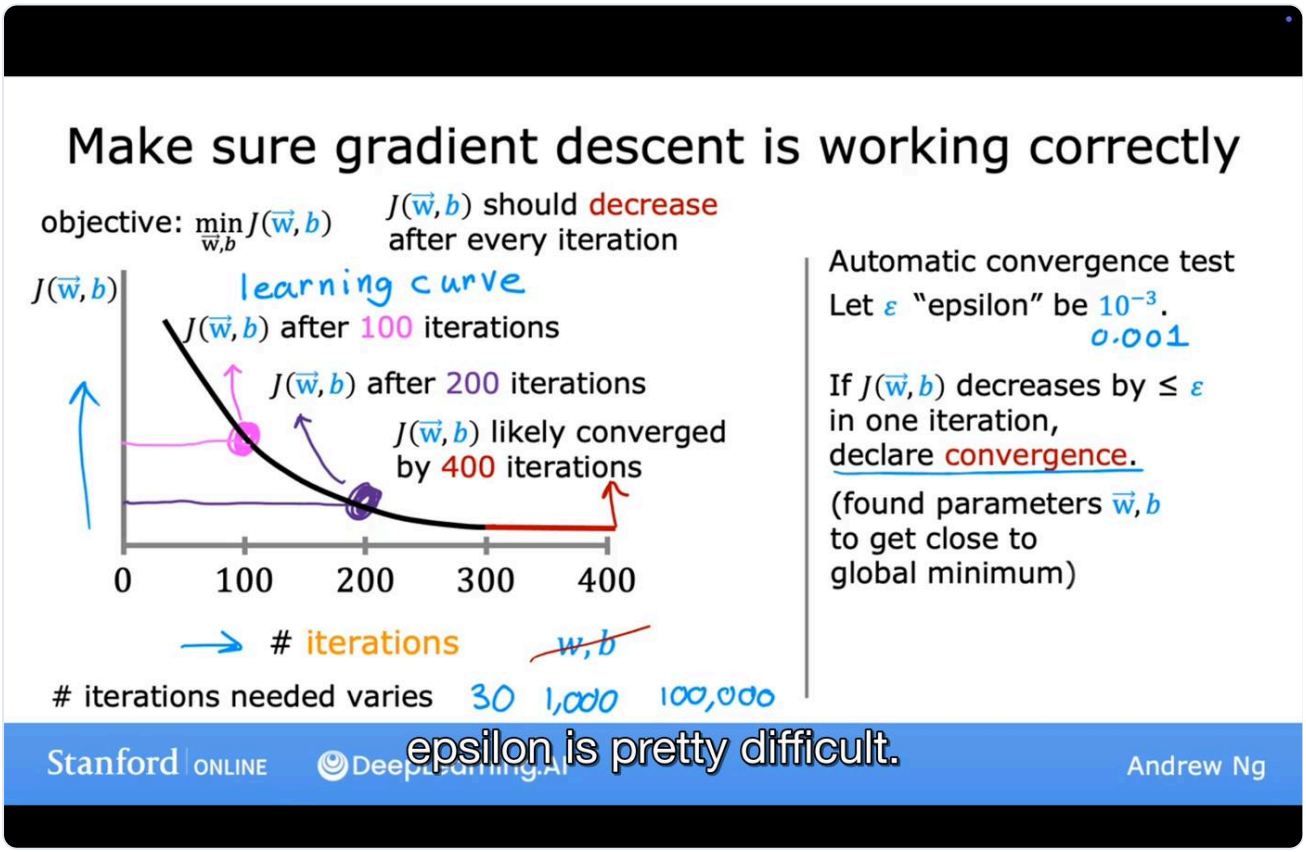


FIGURE 27.11 | Make Sure Gradient Descent Is Working Correctly

I usually find that choosing the right threshold epsilon is pretty difficult. I actually tend to look at graphs like this one on the left, rather than rely on automatic convergence tests. Looking at the solid figure can tell you, I'll give you at some advanced warning if maybe gradient descent is not working correctly as well. You've now seen what the learning curve should look like when gradient descent is running well. Let's take these insights and in the next video, take a look at how to choose an appropriate learning rate.

LECTURE 28

Learning Rate

Your learning algorithm will run much better with an appropriate choice of learning rate. If it's too small, it will run very slowly and if it is too large, it may not even converge. Let's take a look at how you can choose a good learning rate for your model. Concretely, if you plot the cost for a number of iterations and notice that the costs sometimes goes up and sometimes goes down, you should take that as a clear sign that gradient descent is not working properly. This could mean that there's a bug in the code. Or sometimes it could mean that your learning rate is too large. So here's an illustration of what might be happening.

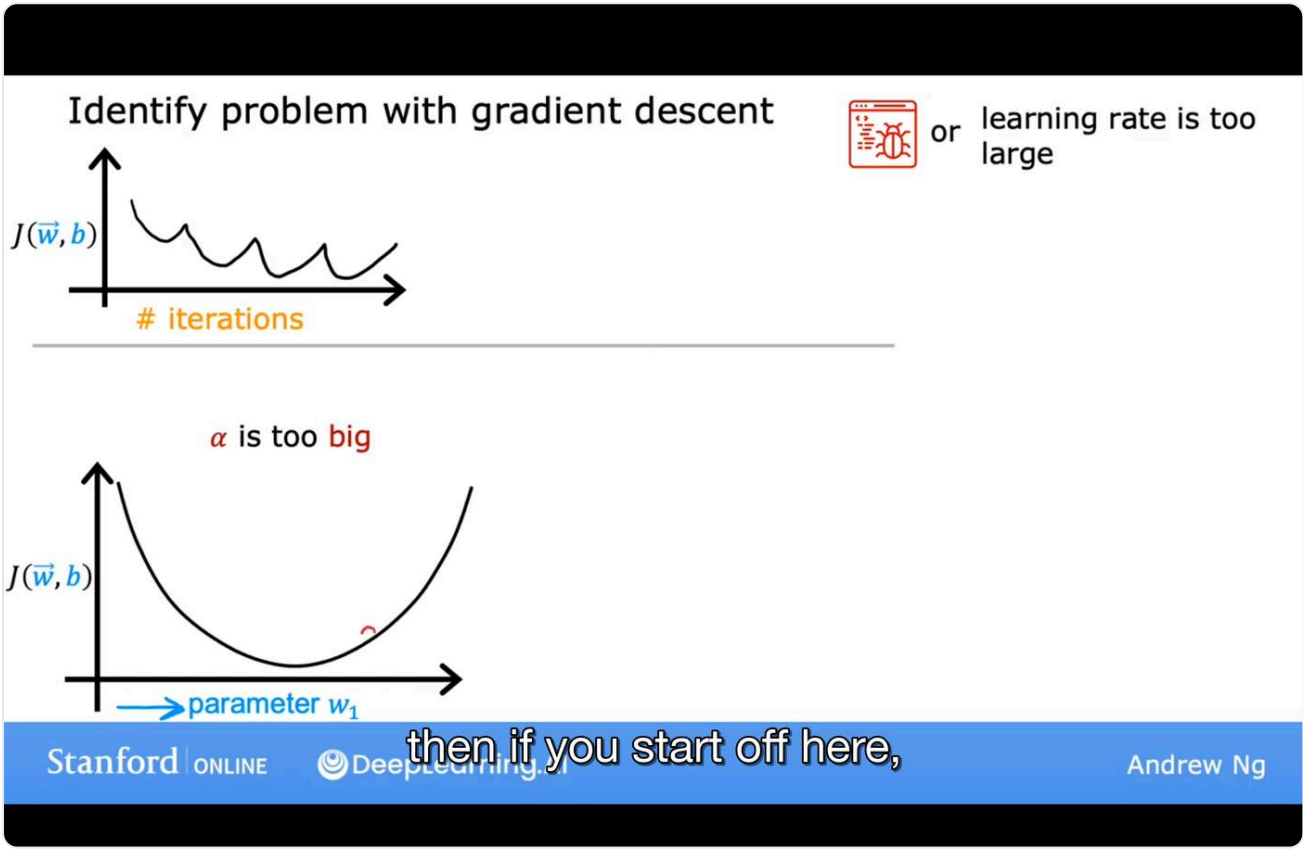


FIGURE 28.1

Here the vertical axis is a cost function J, and the horizontal axis represents a parameter like maybe w_1 and if the learning rate is too big, then if you start off here, your update step may overshoot the minimum and end up here, and in the next update step here, your gain overshooting so you end up here and so on. That's why the cost can sometimes go up instead of decreasing.

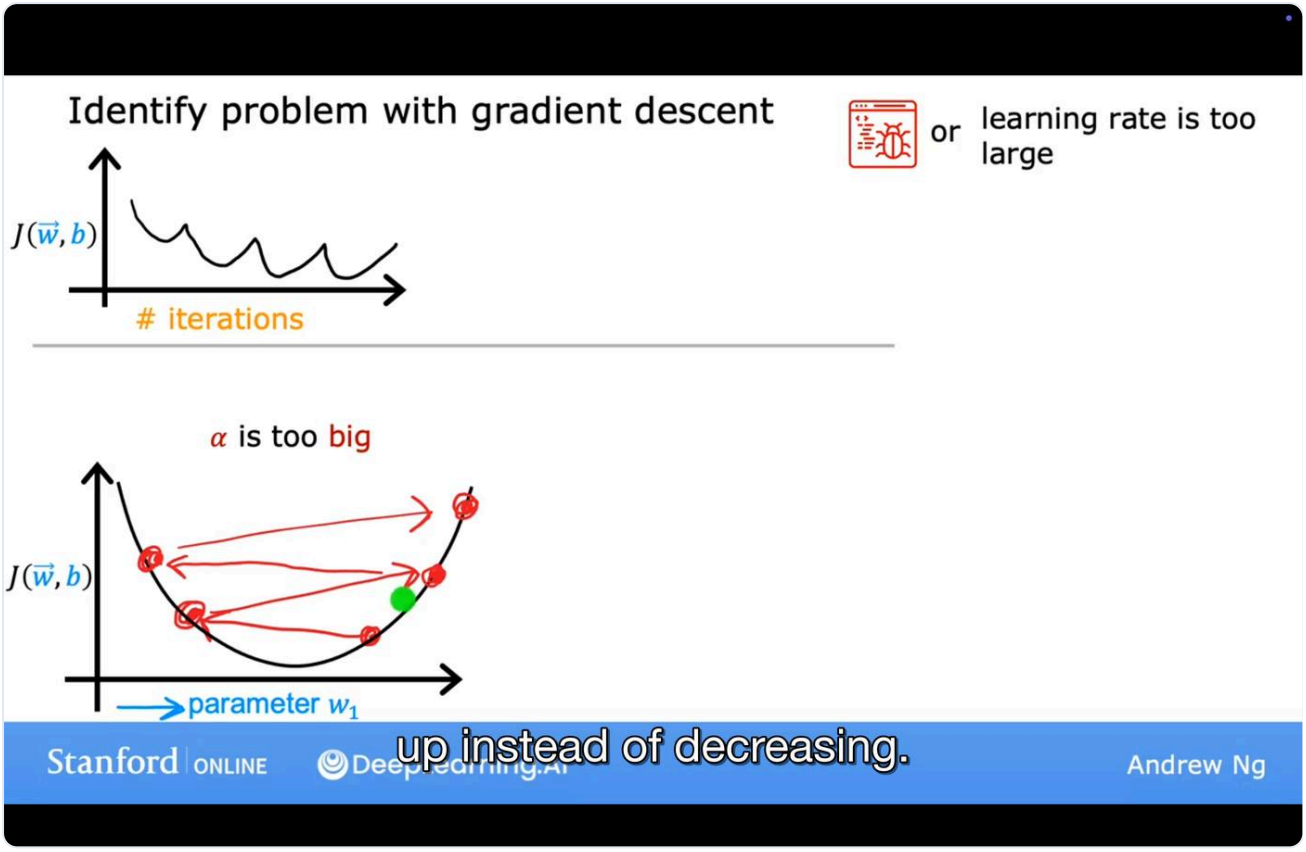


FIGURE 28.2

To fix this, you can use a smaller learning rate.

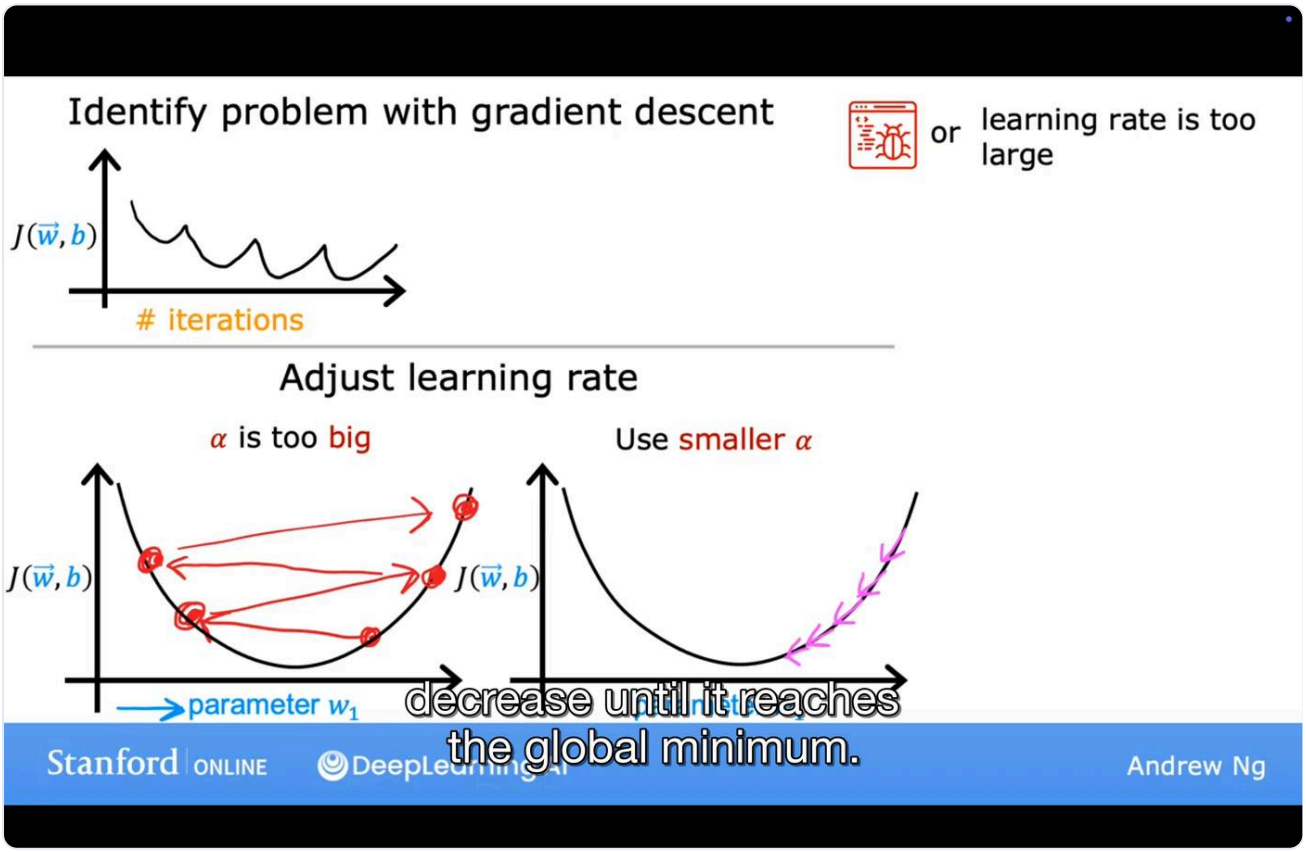


FIGURE 28.3

Then your updates may start here and go down a little bit and down a bit, and we'll hopefully consistently decrease until it reaches the global minimum. Sometimes you may see that the cost consistently increases after each iteration, like this curve here.

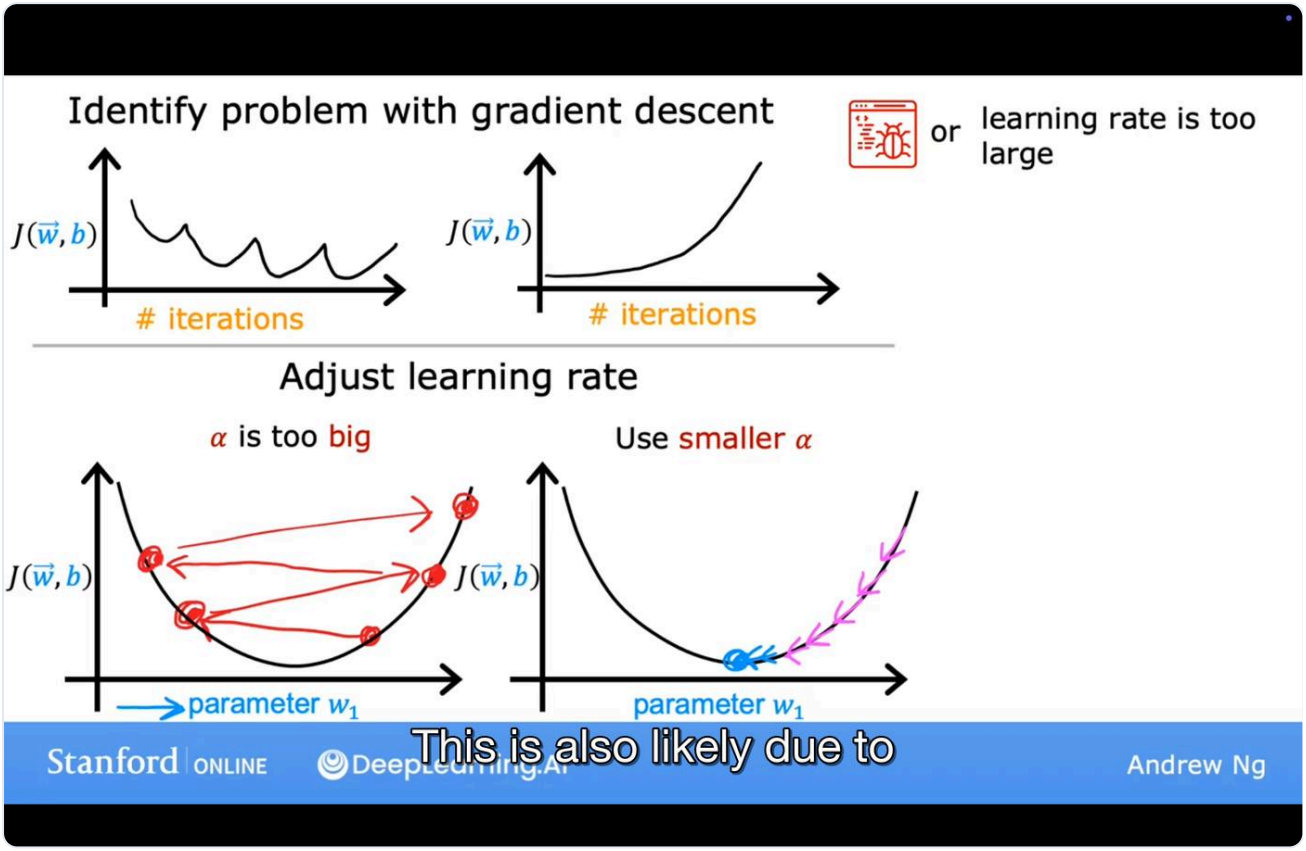


FIGURE 28.4

This is also likely due to a learning rate that is too large, and it could be addressed by choosing a smaller learning rate. But learning rates like this could also be a sign of a possible broken code. For example, if I wrote my code so that w_1 gets updated as w_1 plus Alpha times this derivative term, this could result in the cost consistently increasing at each iteration.

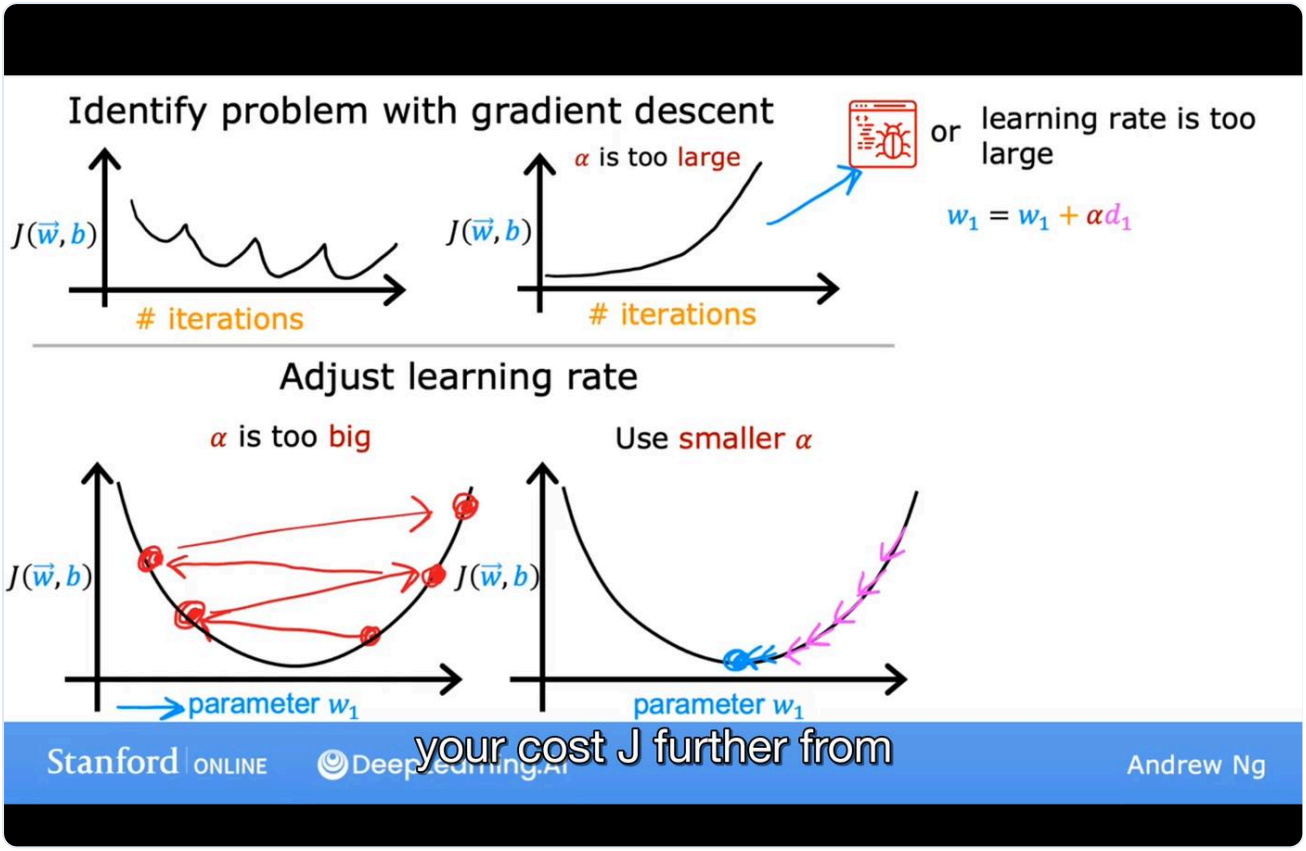


FIGURE 28.5

This is because having the derivative term moves your cost J further from the global minimum instead of closer.

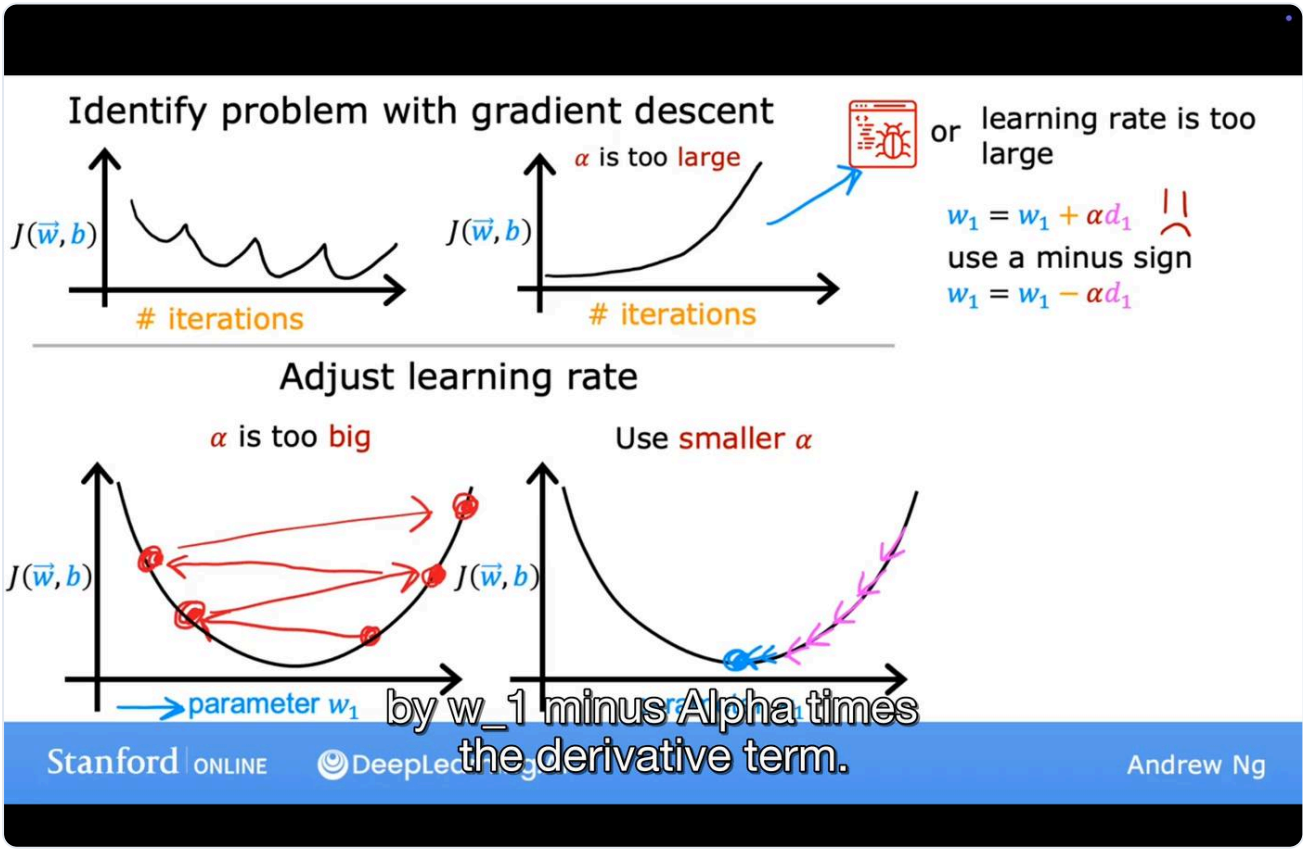


FIGURE 28.6

So remember, you want to use in minus sign, so the code should be updated w_1 updated by w_1 minus Alpha times the derivative term.

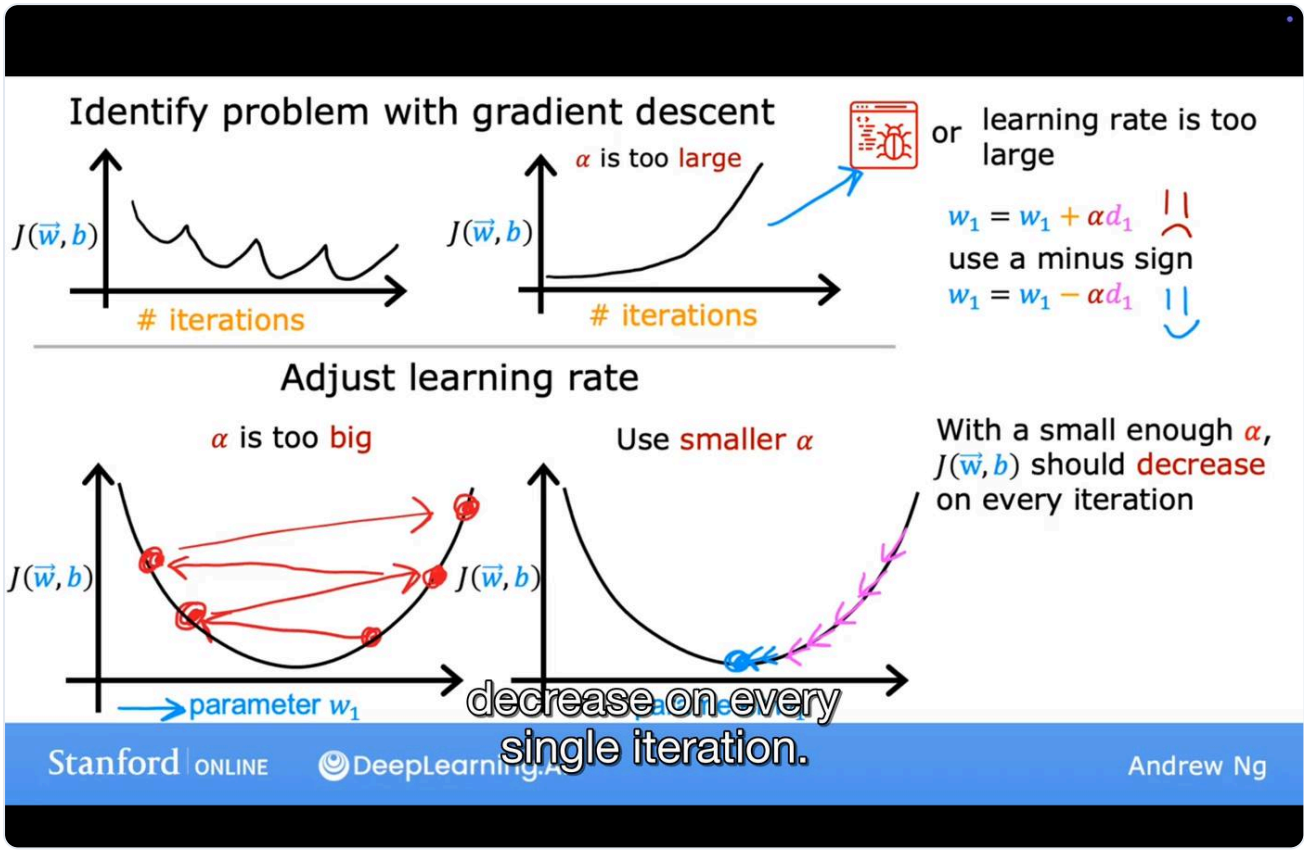


FIGURE 28.7

One debugging tip for a correct implementation of gradient descent is that with a small enough learning rate, the cost function should decrease on every single iteration. So if gradient descent isn't working, one thing I often do and I hope you find this tip useful too, one thing I'll often do is just set Alpha to be a very small number and see if that causes the cost to decrease on every iteration.

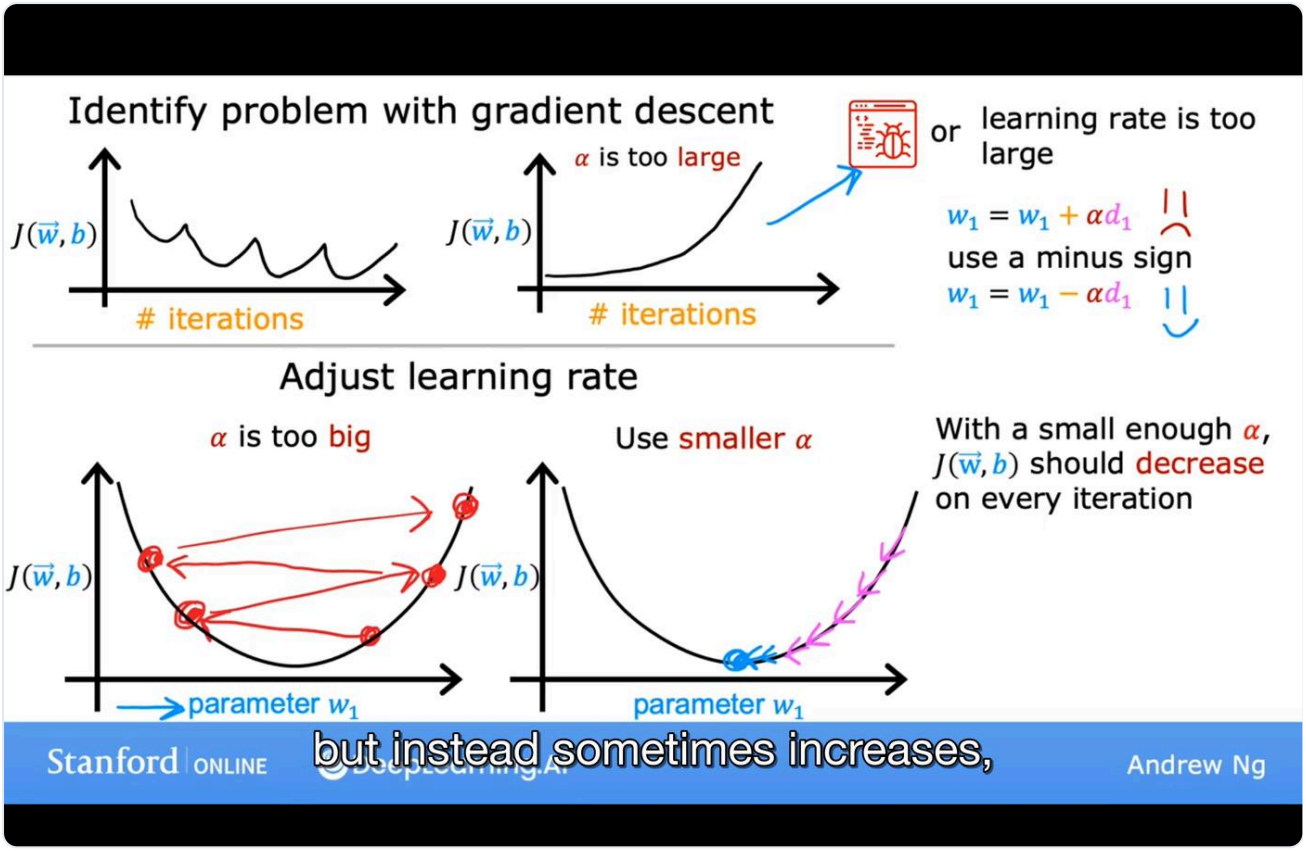


FIGURE 28.8

If even with Alpha set to a very small number, J doesn't decrease on every single iteration, but instead sometimes increases, then that usually means there's a bug somewhere in the code. Note that setting Alpha to be really small is meant here as a debugging step and a very small value of Alpha is not going to be the most efficient choice for actually training your learning algorithm.

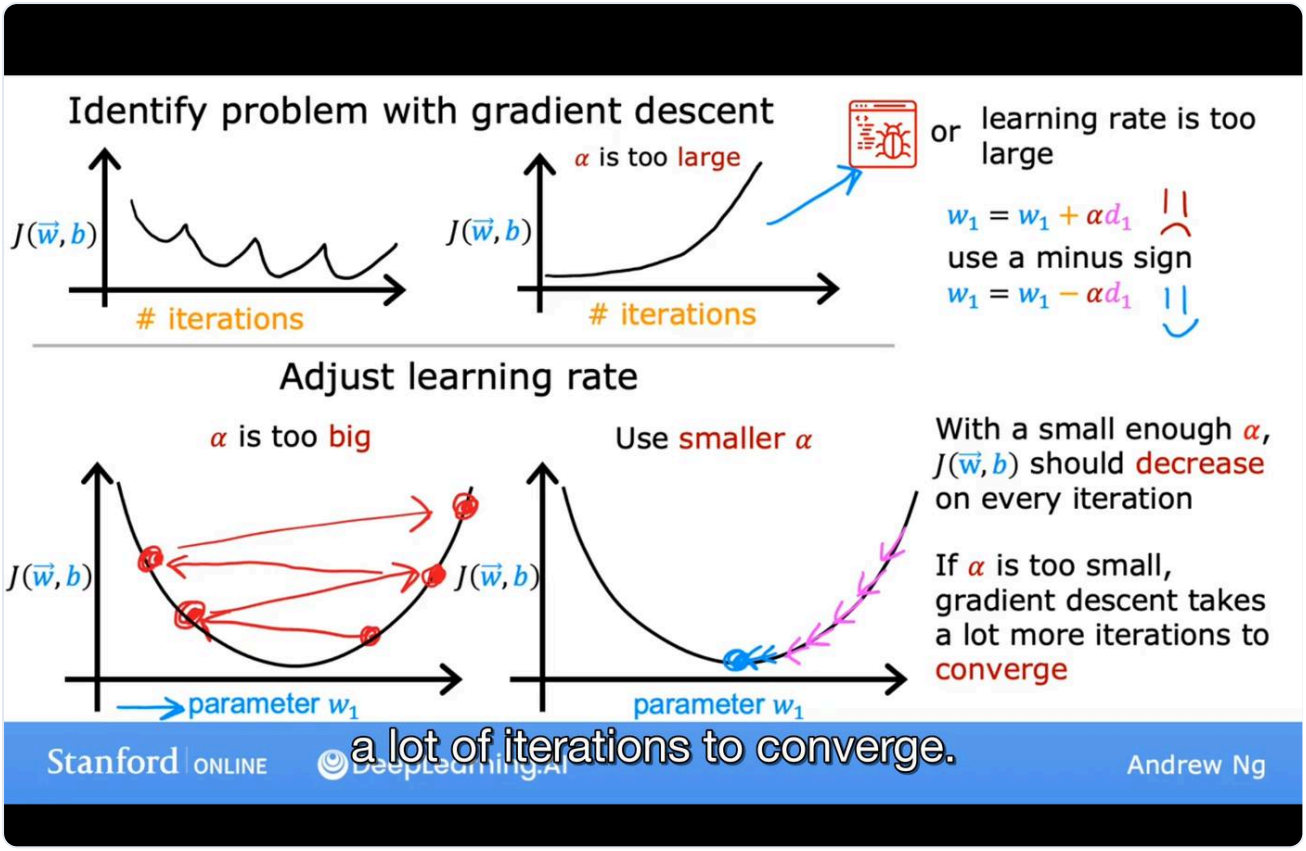


FIGURE 28.9

One important trade-off is that if your learning rate is too small, then gradient descents can take a lot of iterations to converge. So when I am running gradient descent, I will usually try a range of values for the learning rate Alpha. I may start by trying a learning rate of 0.001 and I may also try learning rate as 10 times as large say 0.01 and 0.1 and so on.

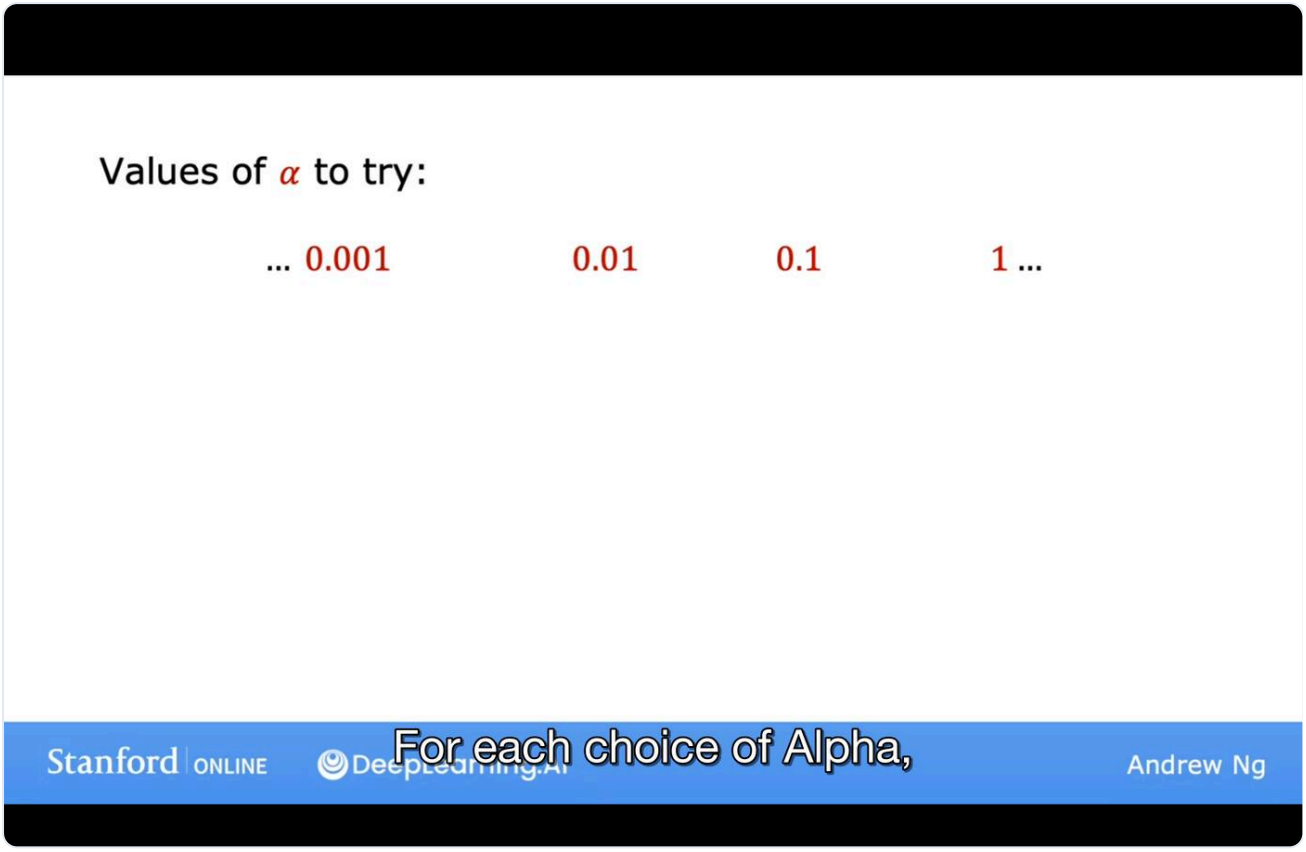


FIGURE 28.10 | Values of a to Try:

For each choice of Alpha, you might run gradient descent just for a handful of iterations and plot the cost function J as a function of the number of iterations and after trying a few different values, you might then pick the value of Alpha that seems to decrease the learning rate rapidly, but also consistently. In fact, what I actually do is try a range of values like this.

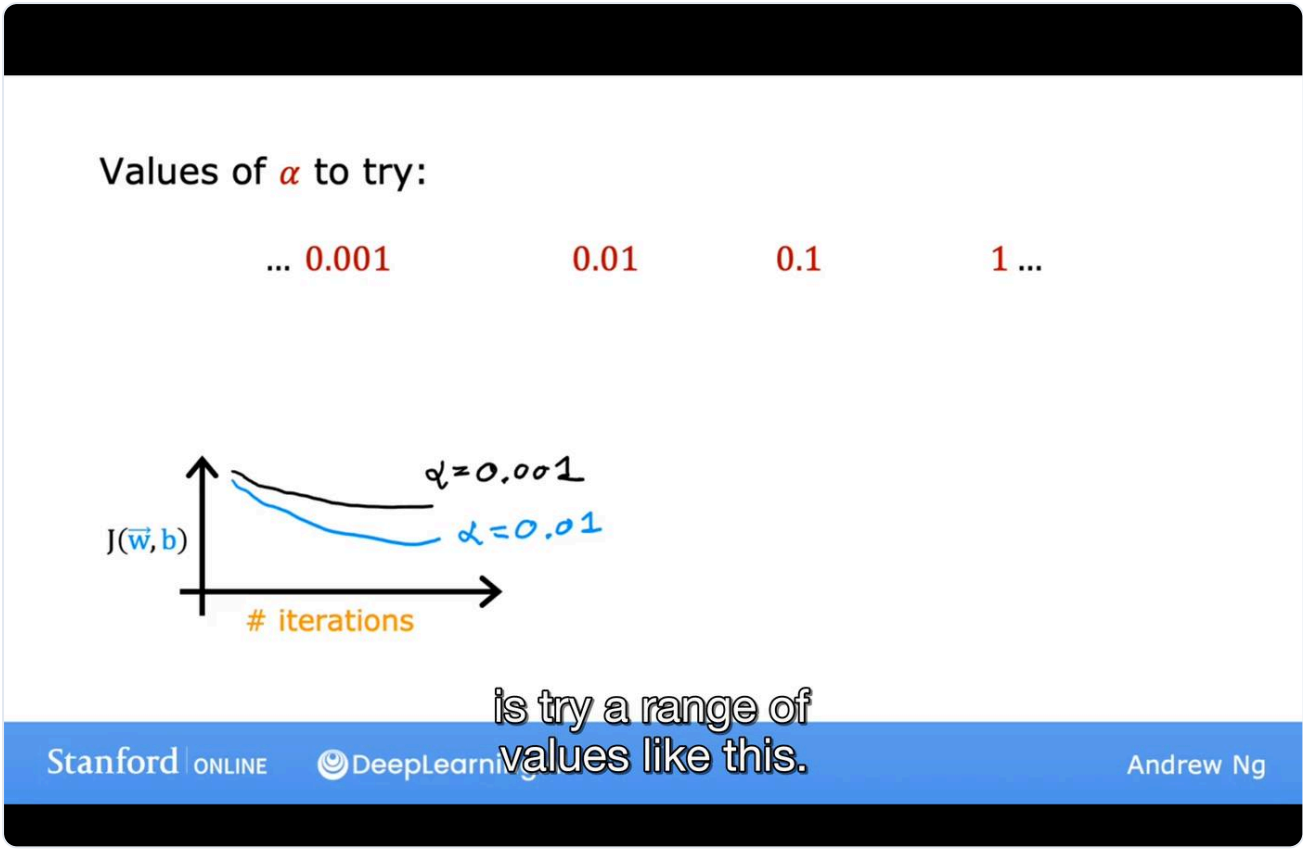


FIGURE 28.11 | Values of a to Try:

After trying 0.001, I'll then increase the learning rate threefold to 0.003.

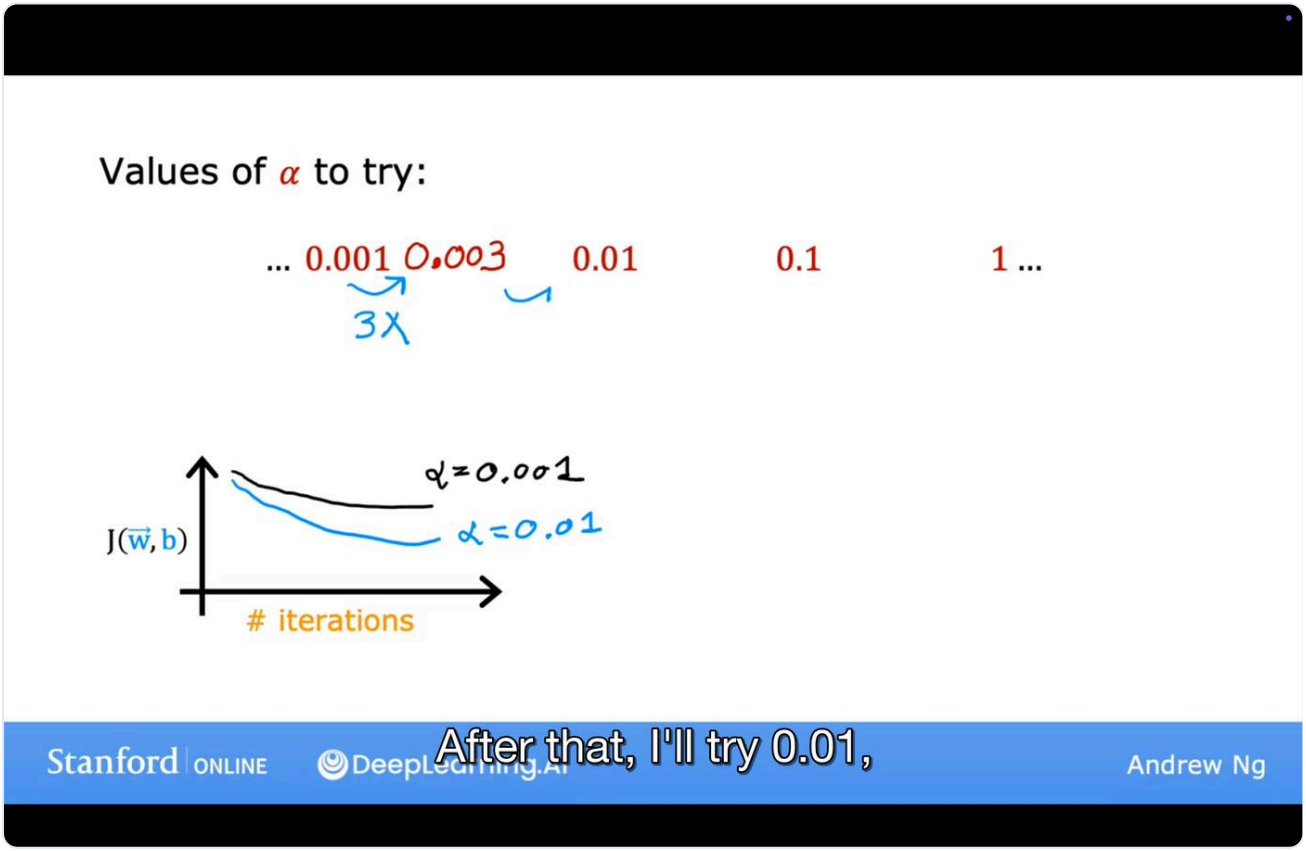


FIGURE 28.12 | Values of a to Try:

After that, I'll try 0.01, which is again about three times as large as 0.003.

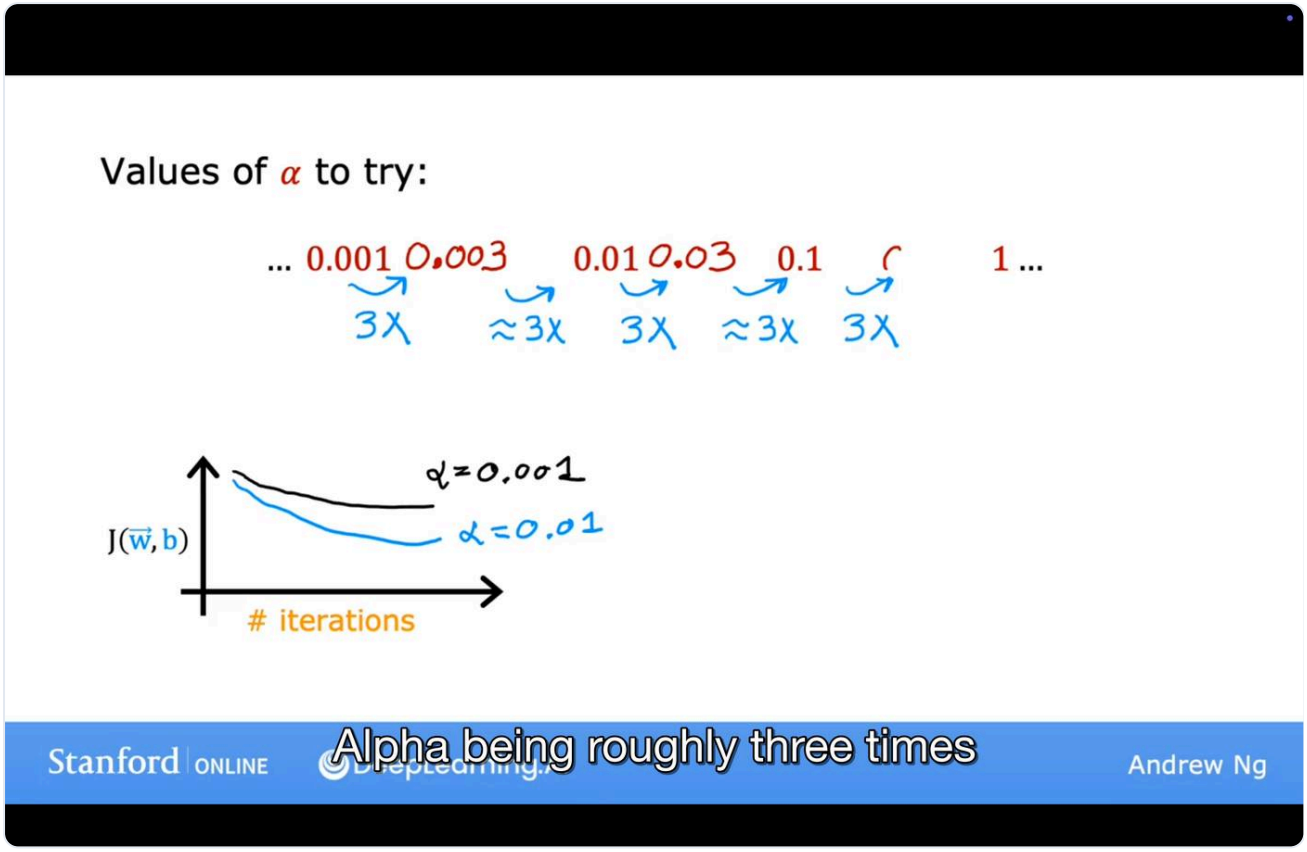


FIGURE 28.13 | Values of a to Try:

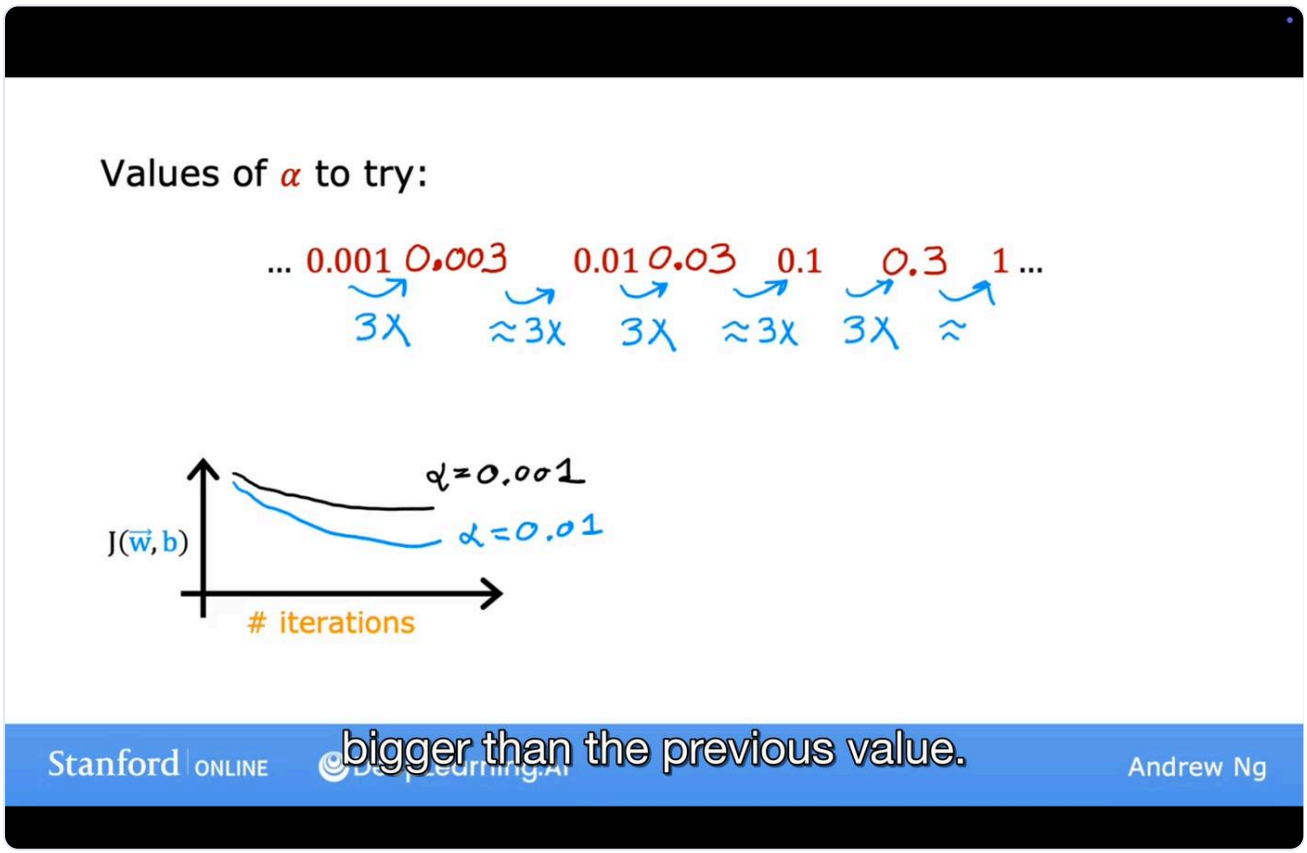


FIGURE 28.14 | Values of a to Try:

So these are roughly trying out gradient descents with each value of Alpha being roughly three times bigger than the previous value.

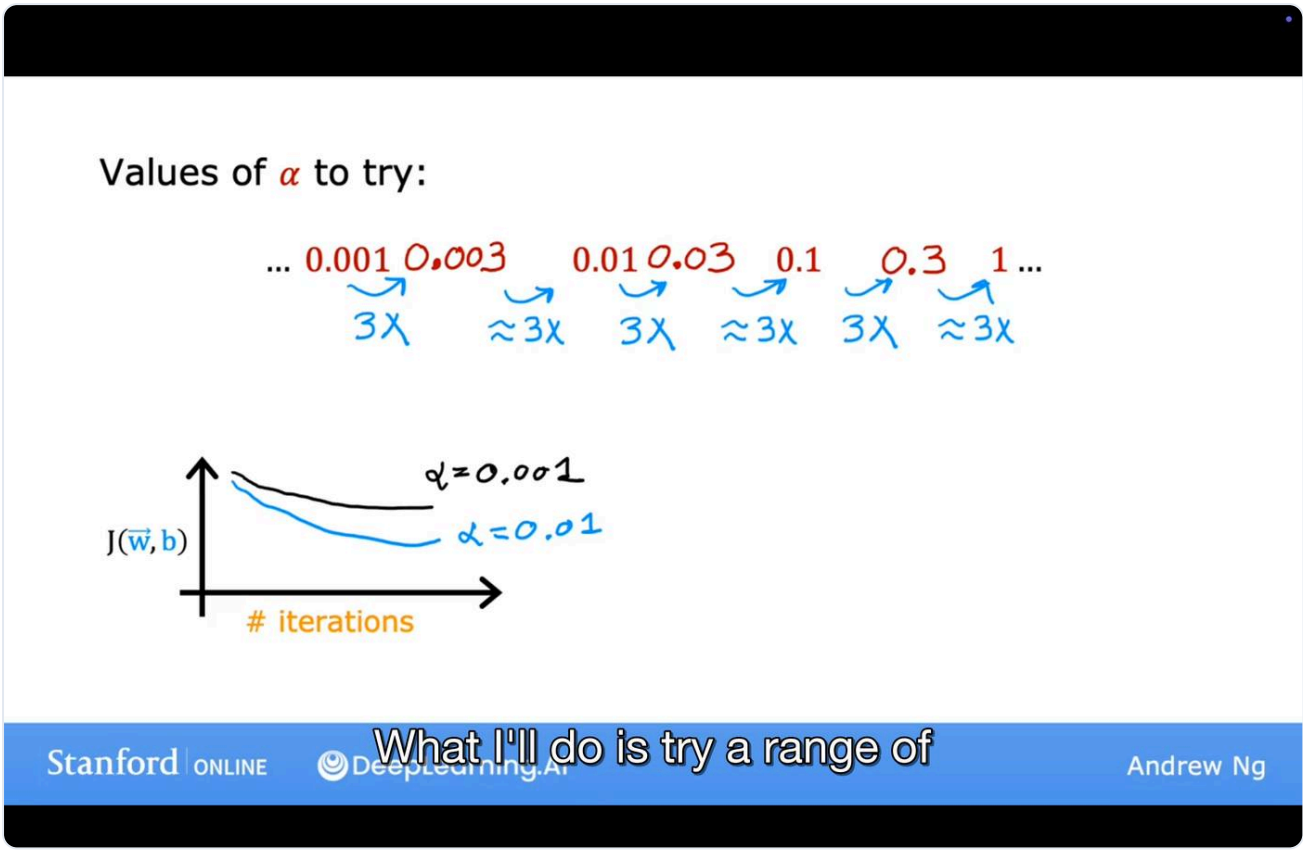


FIGURE 28.15

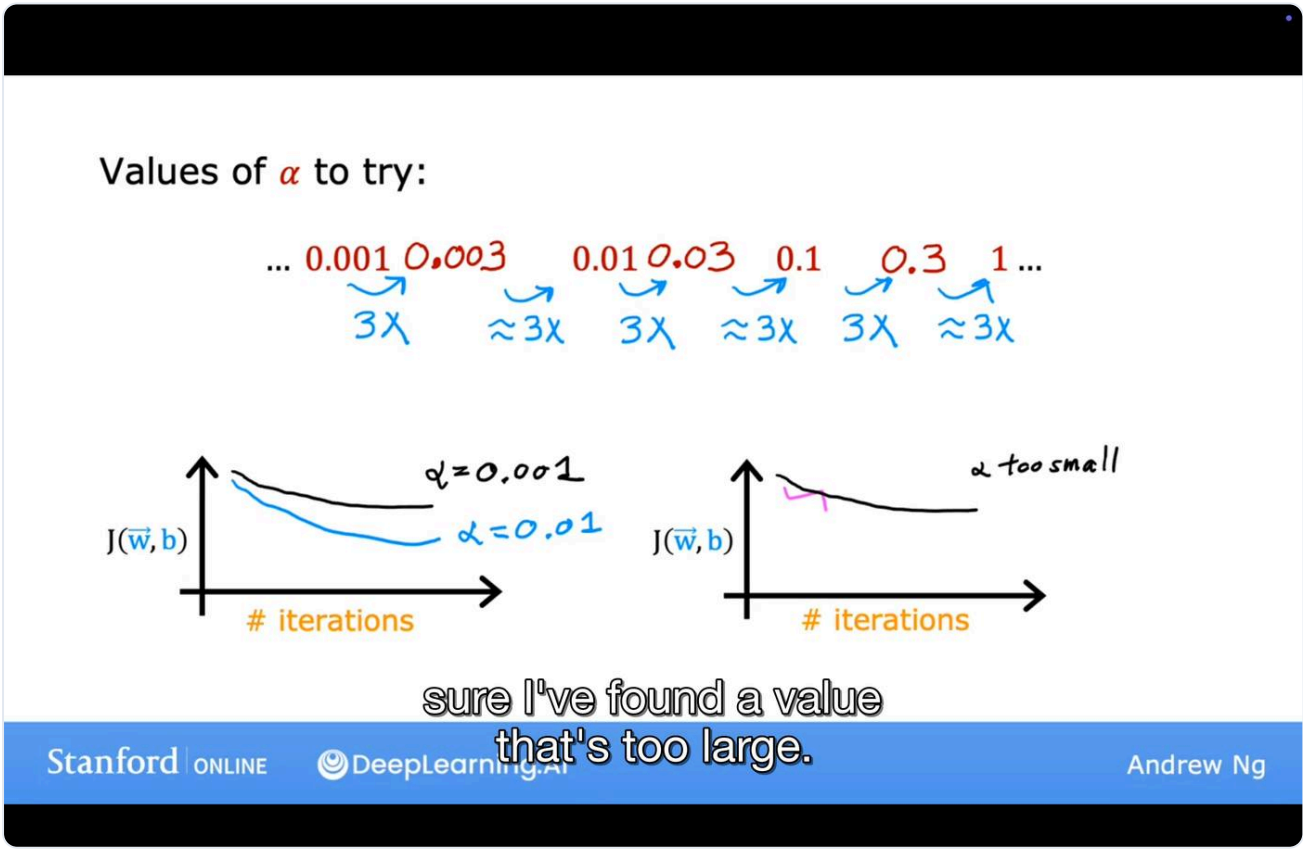


FIGURE 28.16 | Values of a to Try:

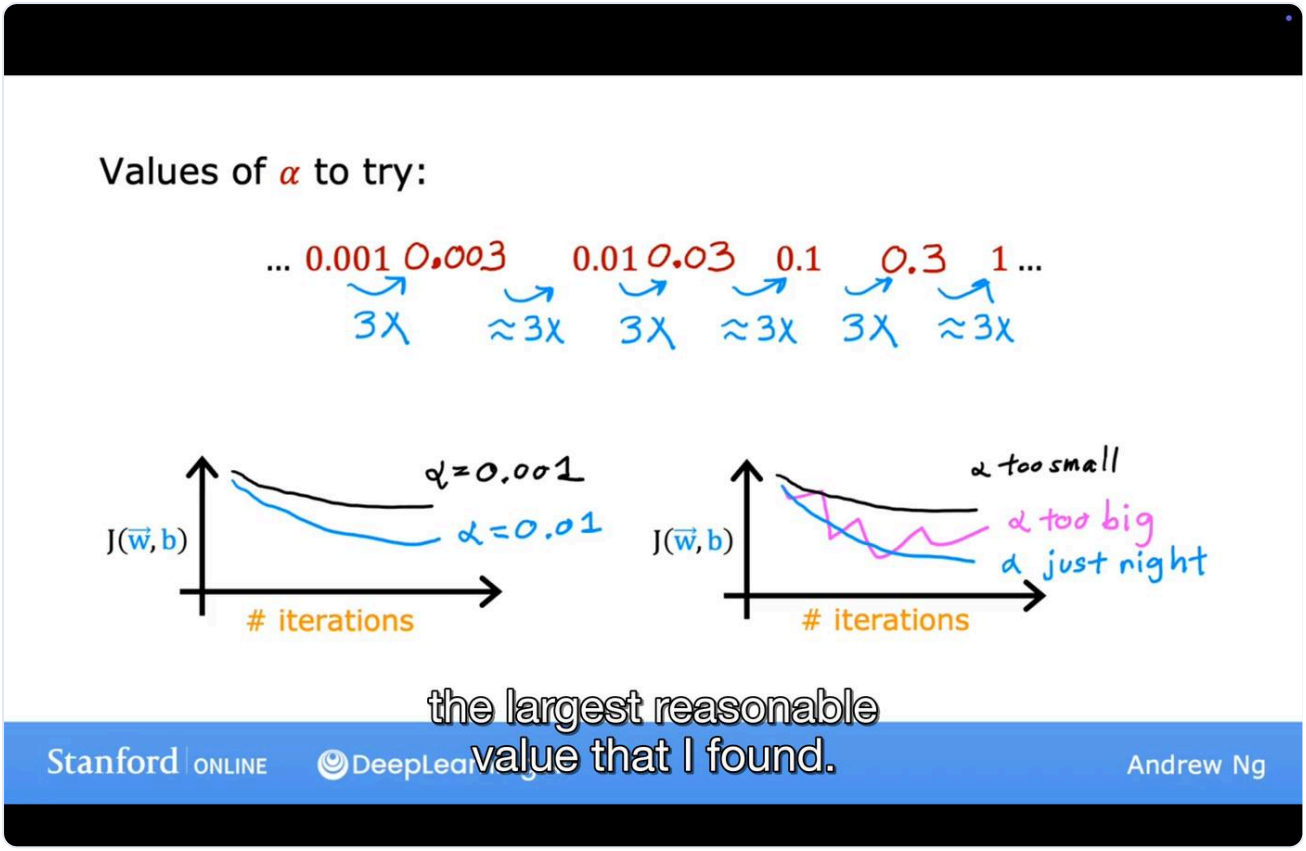


FIGURE 28.17 | Values of a to Try:

What I'll do is try a range of values until I found the value of that's too small and then also make sure I've found a value that's too large. I'll slowly try to pick the largest possible learning rate, or just something slightly smaller than the largest reasonable value that I found. When I do that, it usually gives me a good learning rate for my model. I hope this technique too will be useful for you to choose a good learning rate for your implementation of gradient descent. In the upcoming optional lab you can also take a look at how feature scaling is done in code and also see how different choices of the learning rate Alpha can lead to either better or worse training of your model.

I hope you have fun playing with the value of Alpha and seeing the outcomes of different choices of Alpha. Please take a look and run the code in the optional lab to gain a deeper intuition about feature scaling, as well as the learning rate Alpha. Choosing learning rates is an important part of training many learning algorithms and I hope that this video gives you intuition about different choices and how to pick a good value for Alpha. Now, there are couple more ideas that you can use to make multiple linear regression much more powerful. That is choosing custom features, which will also allow you to fit curves, not just a straight line to your data.

Let's take a look at that in the next video.

LECTURE 29

Feature Engineering

The choice of features can have a huge impact on your learning algorithm's performance. In fact, for many practical applications, choosing or entering the right features is a critical step to making the algorithm work well. In this video, let's take a look at how you can choose or engineer the most appropriate features for your learning algorithm. Let's take a look at feature engineering by revisiting the example of predicting the price of a house. Say you have two features for each house. X_1 is the width of the lot size of the plots of land that the house is built on. This in real state is also called the frontage of the lot, and the second feature, x_2, is the depth of the lot size of, lets assume the rectangular plot of land that the house was built on.

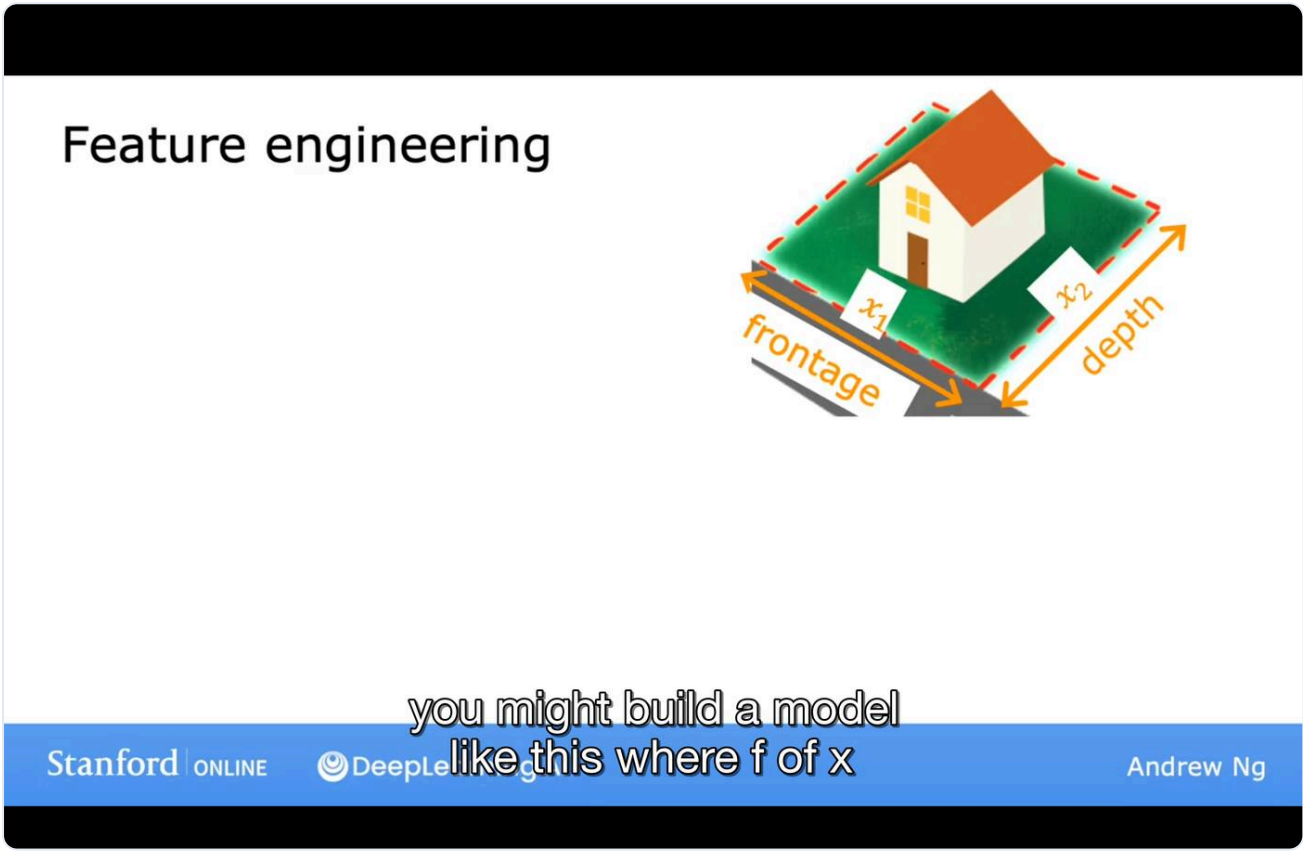


FIGURE 29.1 | Feature Engineering

Given these two features, x_1 and x_2, you might build a model like this where f of x is w_1x_1 plus w_2x_2 plus b, where x_1 is the frontage or width, and x_2 is the depth.

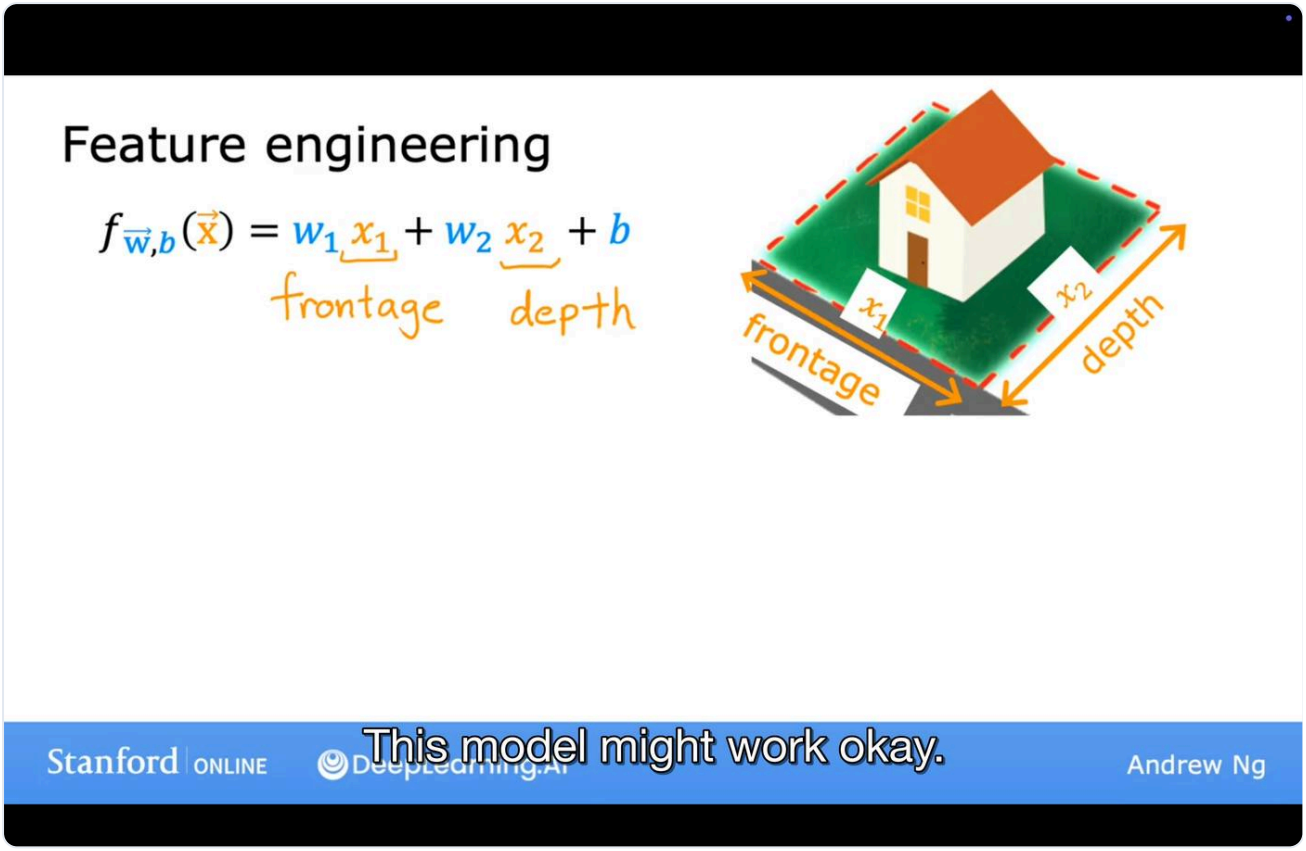


FIGURE 29.2 | Feature Engineering

This model might work okay. But here's another option for how you might choose a different way to use these features in the model that could be even more effective. You might notice that the area of the land can be calculated as the frontage or width times the depth. You may have an intuition that the area of the land is more predictive of the price, than the frontage and depth as separate features.

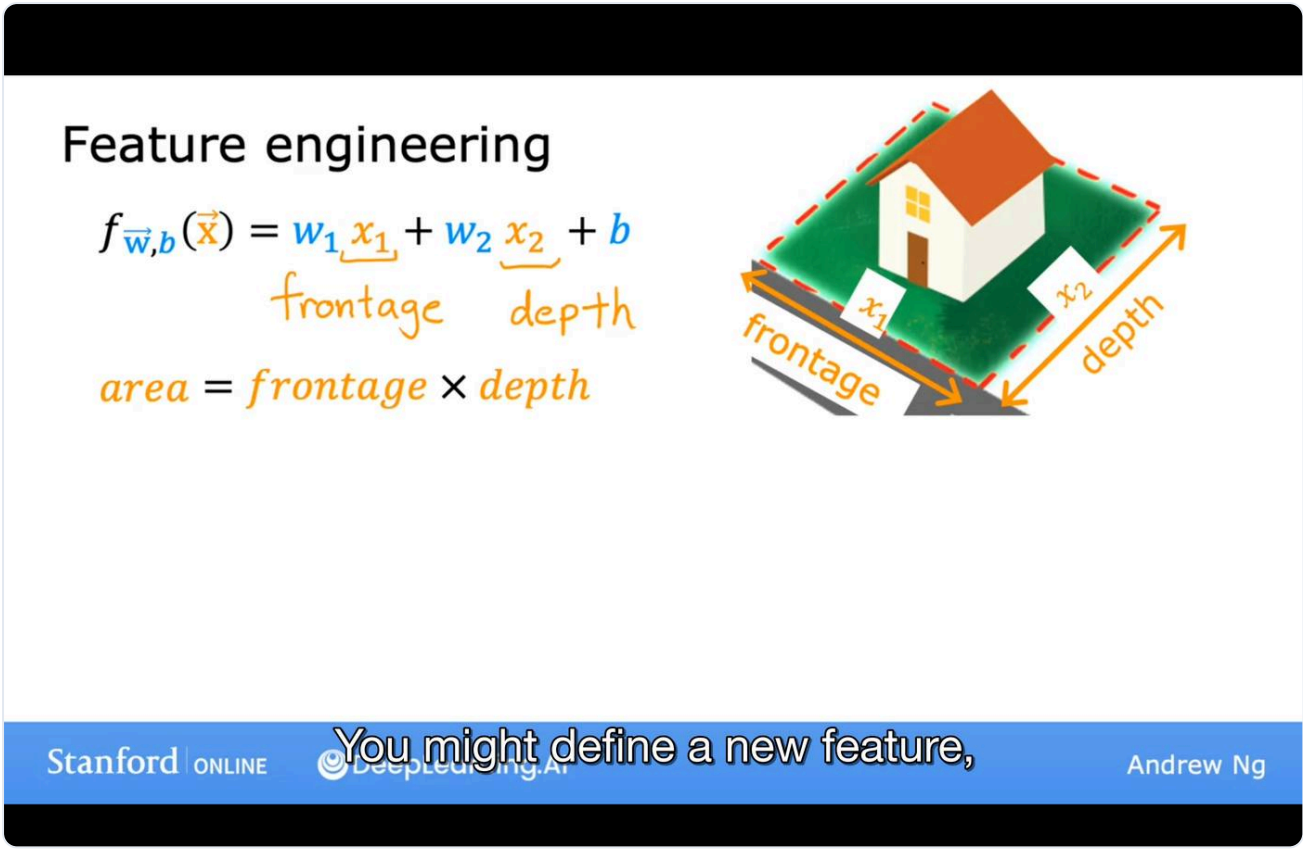


FIGURE 29.3 | Feature Engineering

You might define a new feature, x_3, as x_1 times x_2.

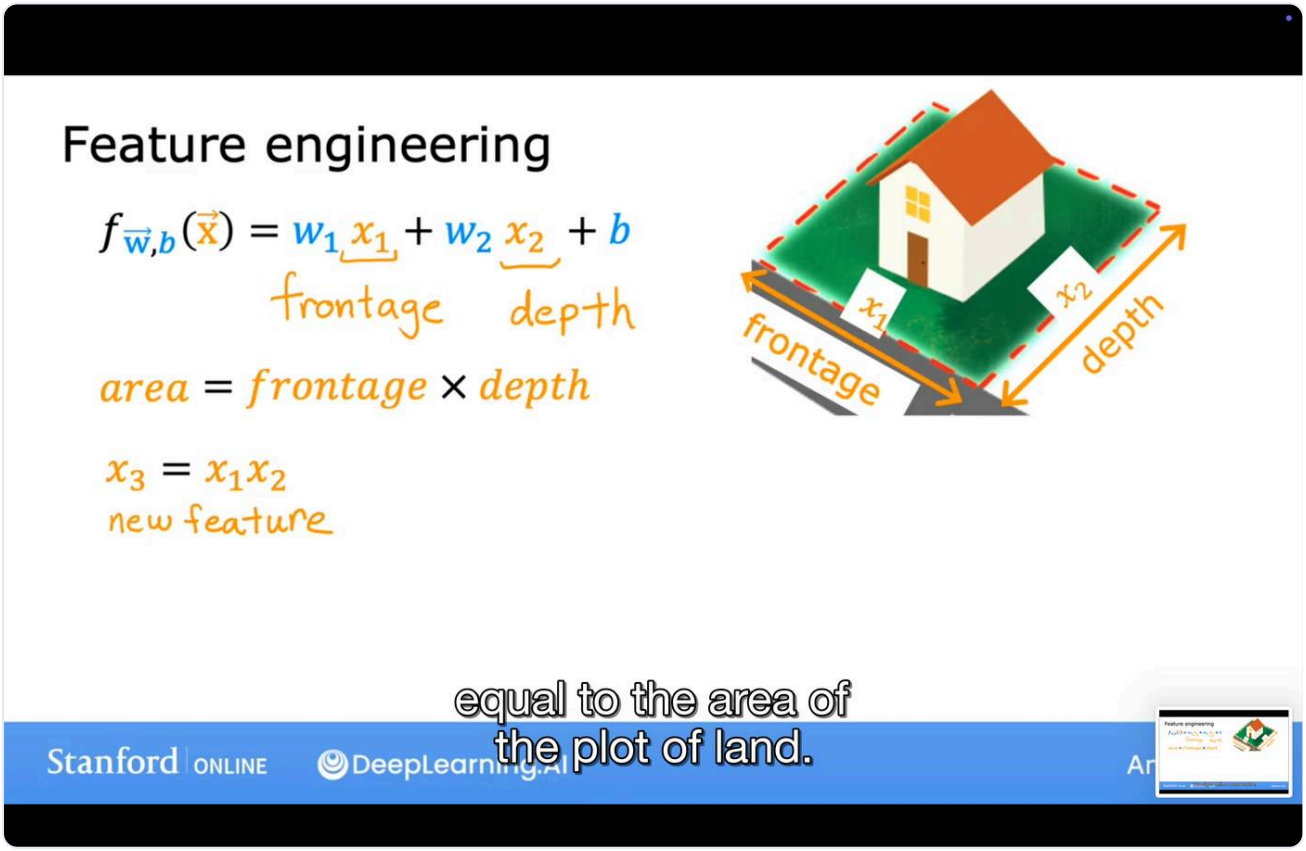


FIGURE 29.4 | Feature Engineering

This new feature x_3 is equal to the area of the plot of land.

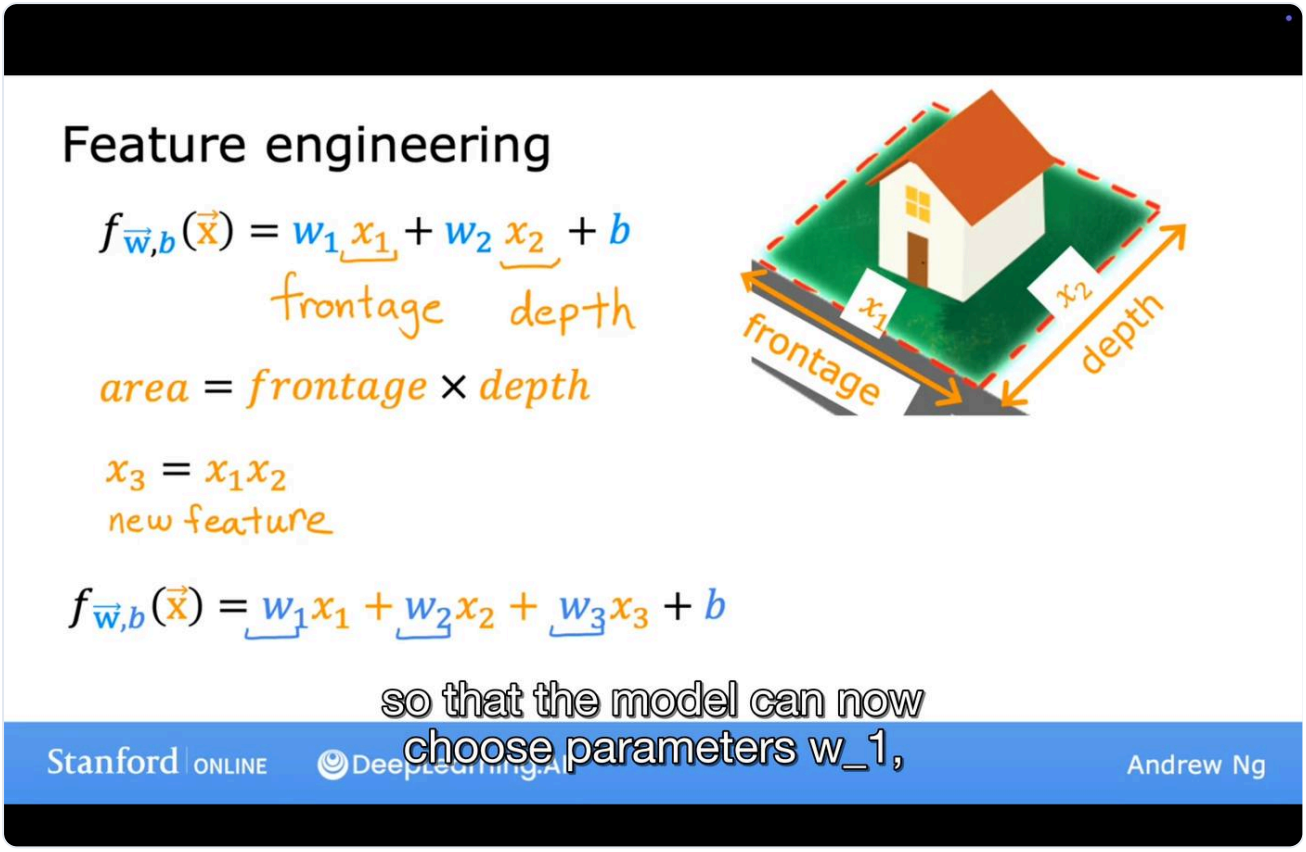


FIGURE 29.5 | Feature Engineering

With this feature, you can then have a model f_w, b of x equals w_1x_1 plus w_2x_2 plus w_3x_3 plus b so that the model can now choose parameters w_1, w_2, and w_3, depending on whether the data shows that the frontage or the depth or the area x_3 of the lot turns out to be the most important thing for predicting the price of the house.

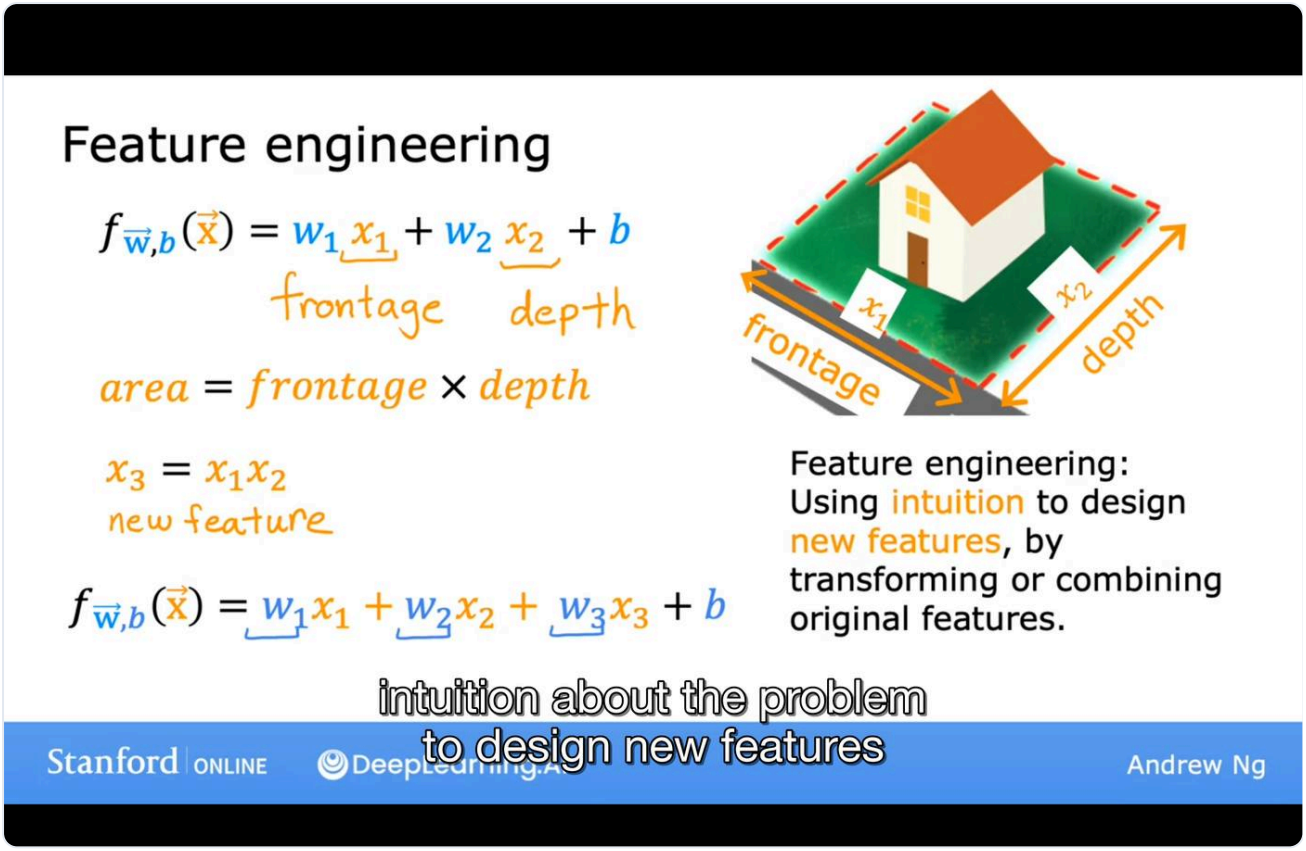


FIGURE 29.6 | Feature Engineering

What we just did, creating a new feature is an example of what's called feature engineering, in which you might use your knowledge or intuition about the problem to design new features usually by transforming or combining the original features of the problem in order to make it easier for the learning algorithm to make accurate predictions. Depending on what insights you may have into the application, rather than just taking the features that you happen to have started off with sometimes by defining new features, you might be able to get a much better model. That's feature engineering. It turns out that this one flavor of feature engineering, that allow you to fit not just straight lines, but curves, non-linear functions to your data.

Let's take a look in the next video at how you can do that.

LECTURE 30

Polynomial Regression

So far we've just been fitting straight lines to our data. Let's take the ideas of multiple linear regression and feature engineering to come up with a new algorithm called polynomial regression, which will let you fit curves, non-linear functions, to your data. Let's say you have a housing data-set that looks like this, where feature x is the size in square feet. It doesn't look like a straight line fits this data-set very well.

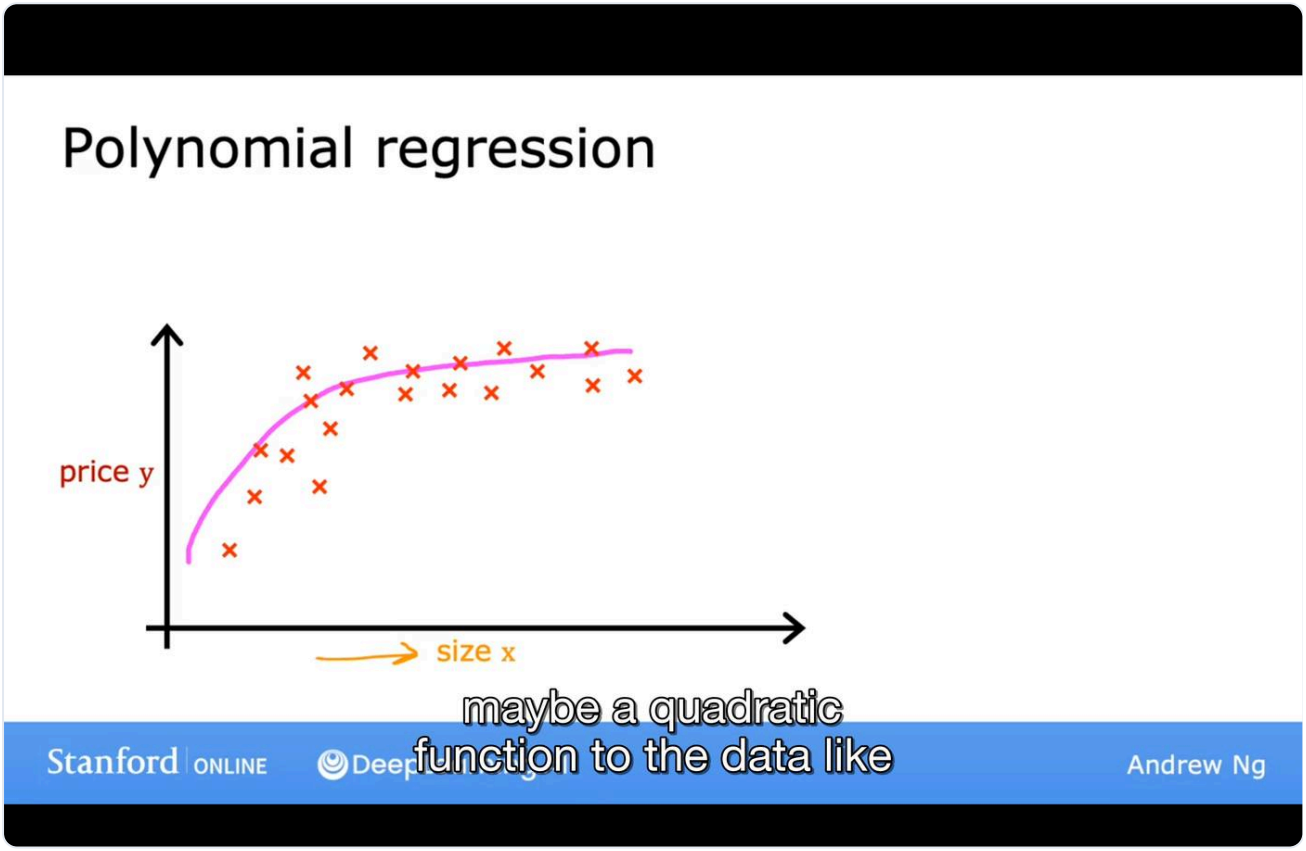


FIGURE 30.1 | Polynomial Regression

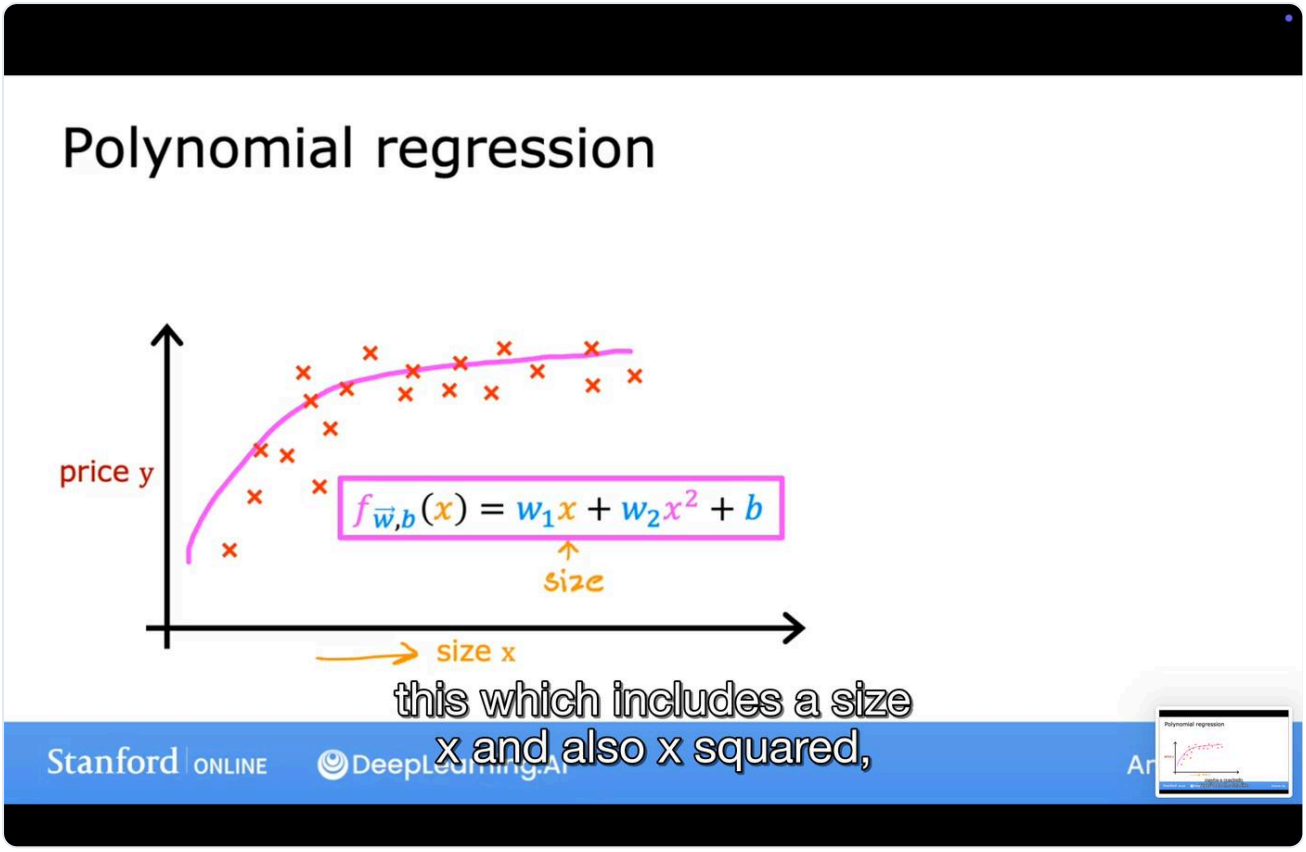


FIGURE 30.2 | Polynomial Regression

Maybe you want to fit a curve, maybe a quadratic function to the data like this which includes a size x and also x squared, which is the size raised to the power of two. Maybe that will give you a better fit to the data. But then you may decide that your quadratic model doesn't really make sense because a quadratic function eventually comes back down. Well, we wouldn't really expect housing prices to go down when the size increases.

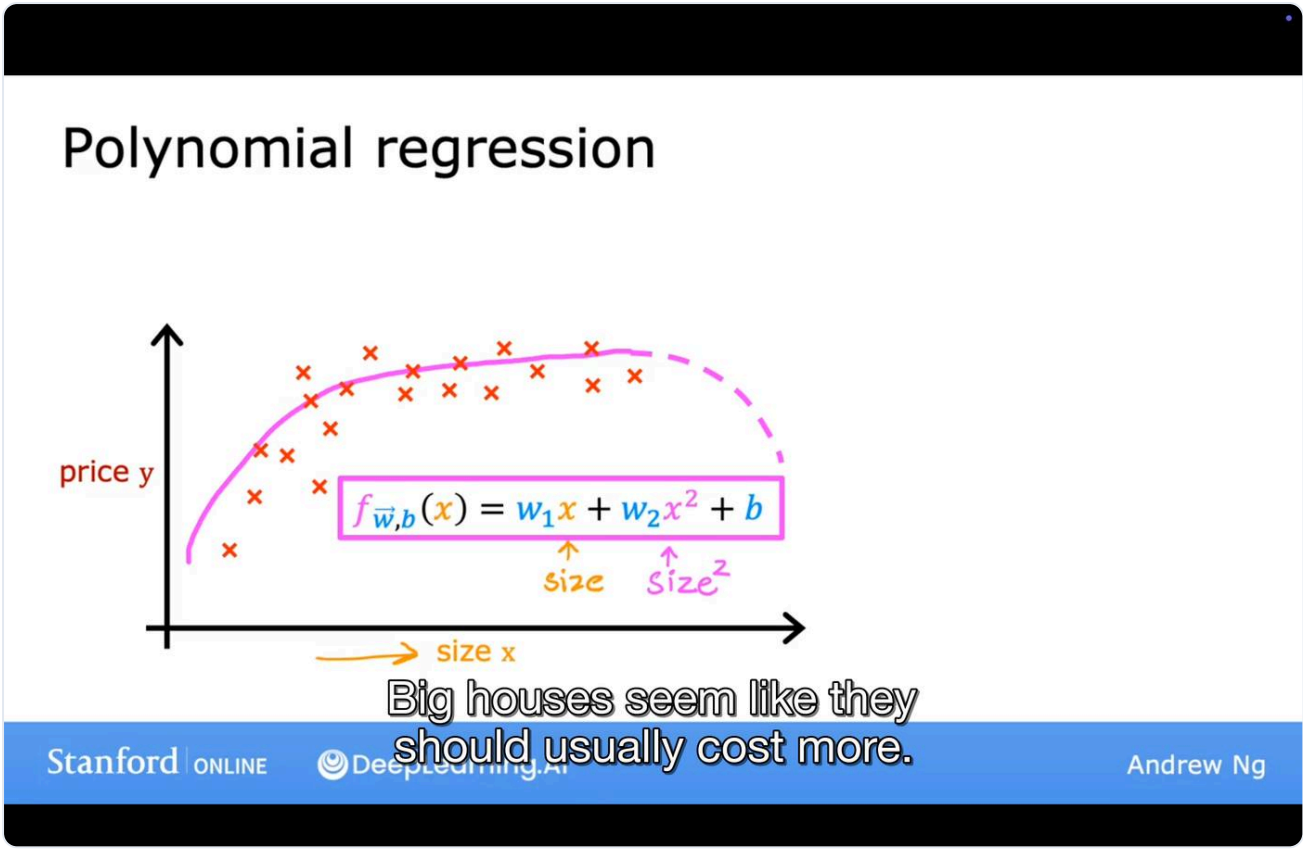


FIGURE 30.3 | Polynomial Regression

Big houses seem like they should usually cost more. Then you may choose a cubic function where we now have not only x squared, but x cubed.

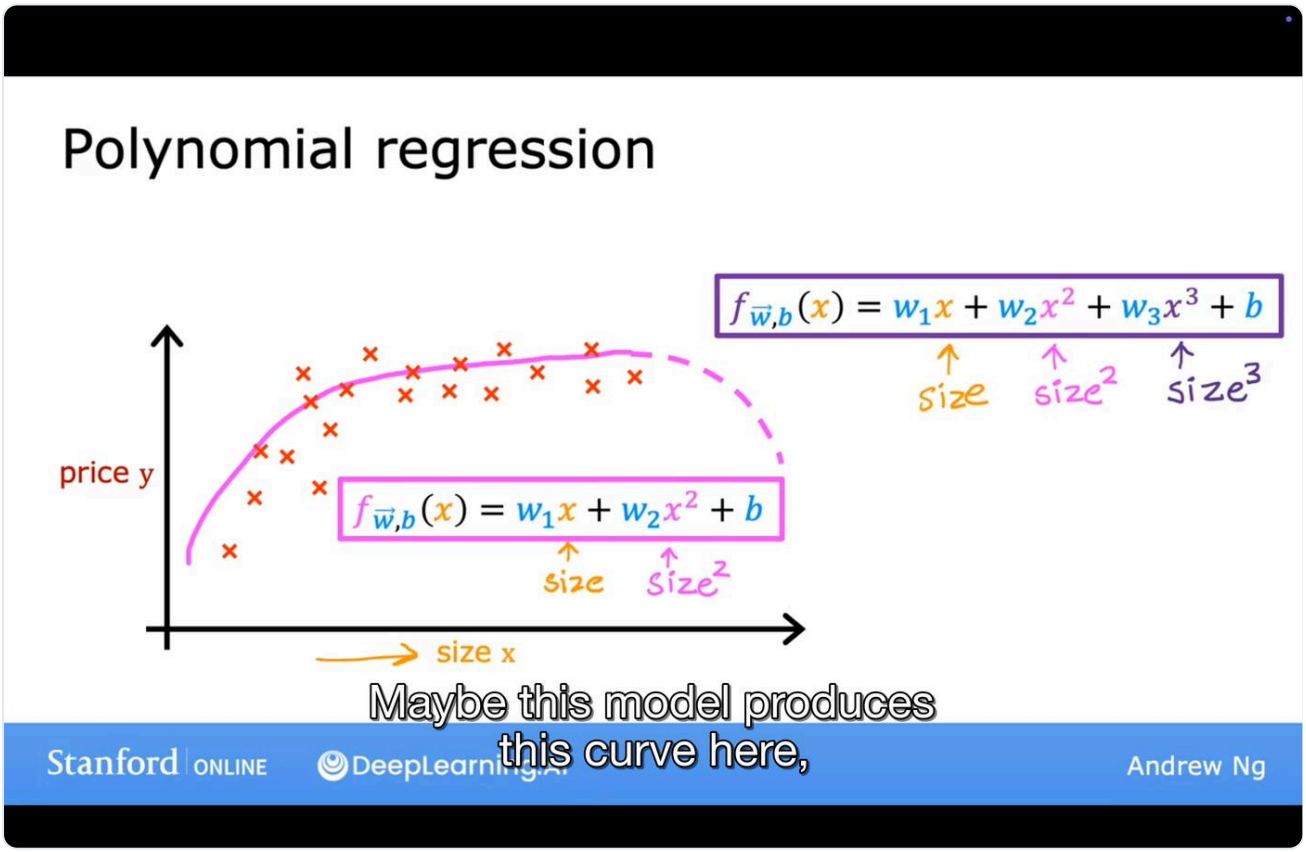


FIGURE 30.4 | Polynomial Regression

Maybe this model produces this curve here, which is a somewhat better fit to the data because the size does eventually come back up as the size increases. These are both examples of polynomial regression, because you took your optional feature x, and raised it to the power of two or three or any other power.

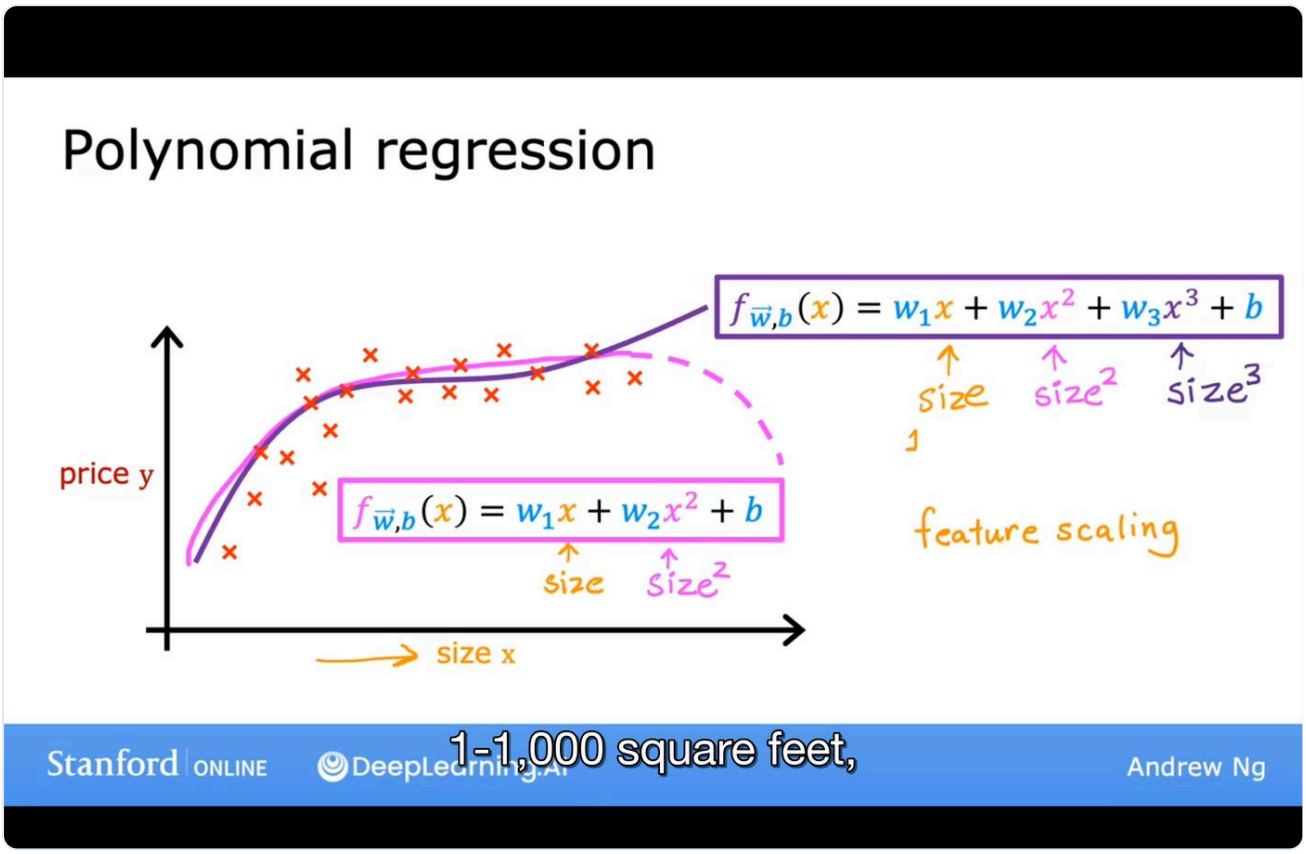


FIGURE 30.5 | Polynomial Regression

In the case of the cubic function, the first feature is the size, the second feature is the size squared, and the third feature is the size cubed. I just want to point out one more thing, which is that if you create features that are these powers like the square of the original features like this, then feature scaling becomes increasingly important.

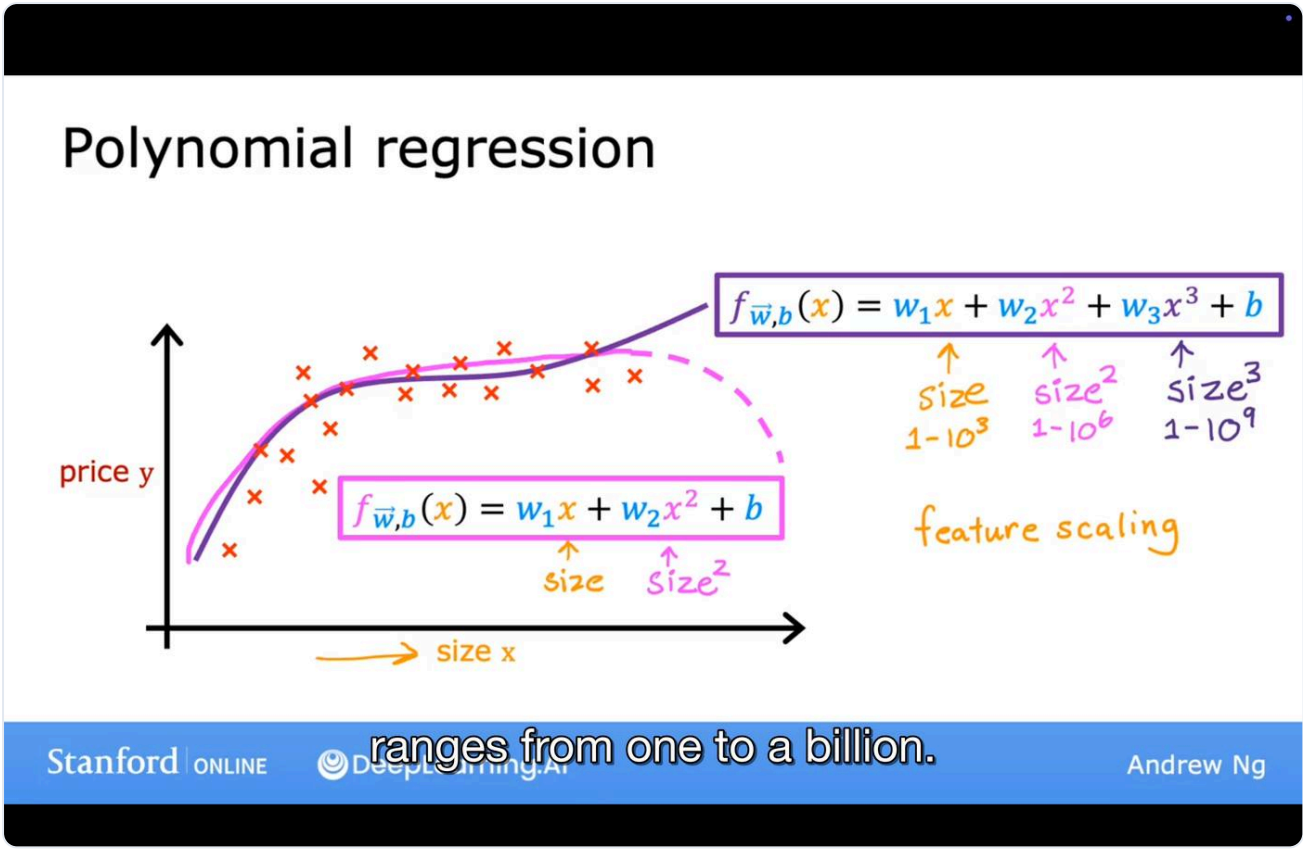


FIGURE 30.6 | Polynomial Regression

If the size of the house ranges from say, 1-1,000 square feet, then the second feature, which is a size squared, will range from one to a million, and the third feature, which is size cubed, ranges from one to a billion. These two features, x squared and x cubed, take on very different ranges of values compared to the original feature x.

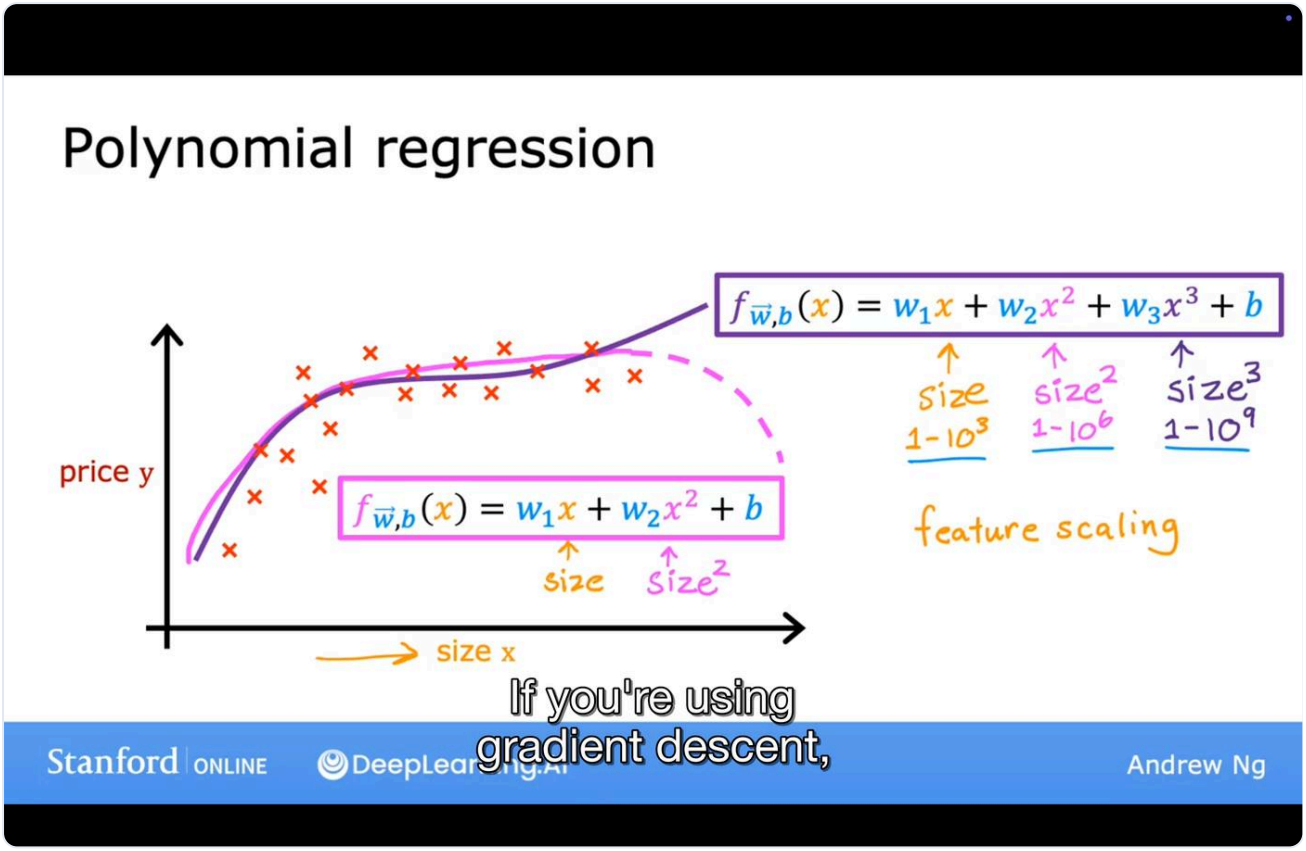


FIGURE 30.7 | Polynomial Regression

If you're using gradient descent, it's important to apply feature scaling to get your features into comparable ranges of values. Finally, just one last example of how you really have a wide range of choices of features to use. Another reasonable alternative to taking the size squared and size cubed is to say use the square root of x.

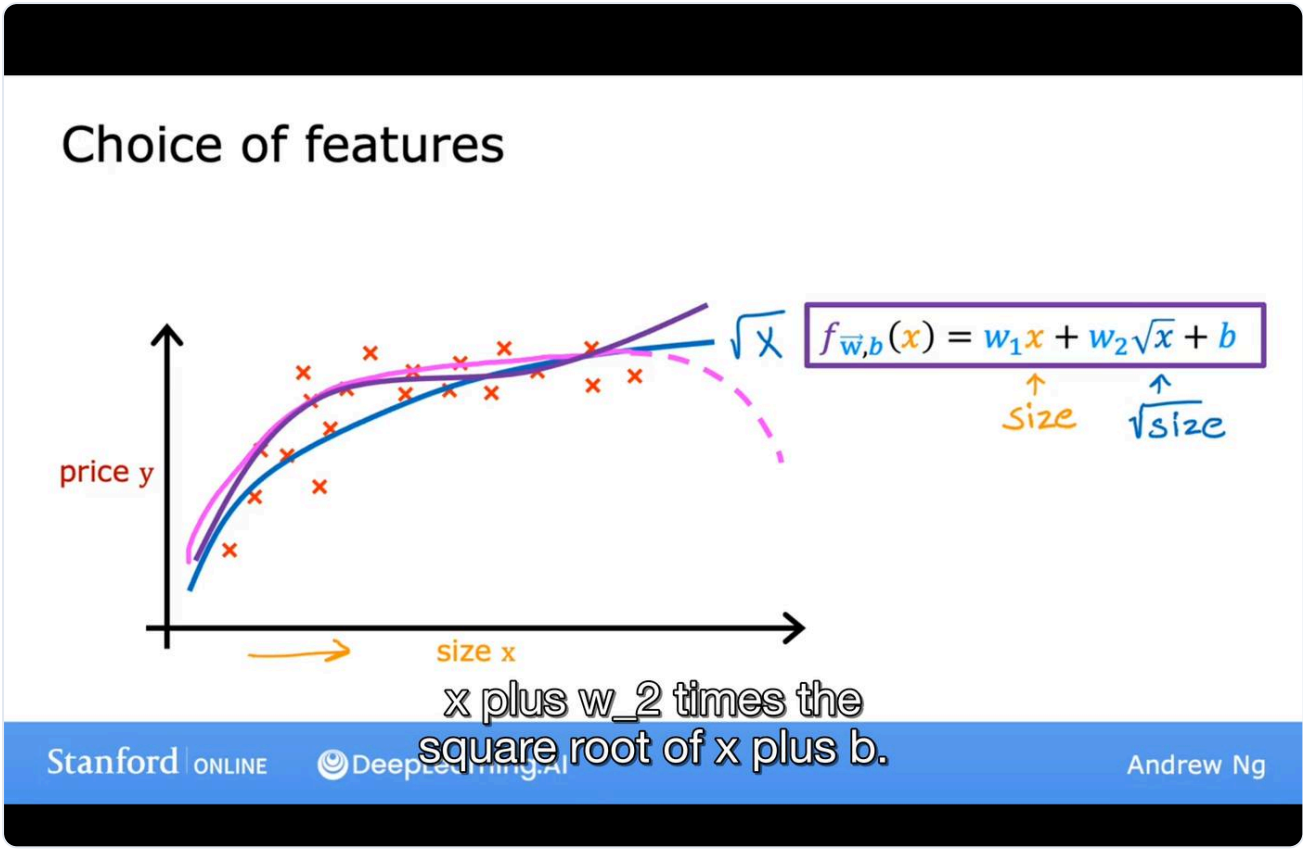


FIGURE 30.8 | Choice of Features

Your model may look like w_1 times x plus w_2 times the square root of x plus b. The square root function looks like this, and it becomes a bit less steep as x increases, but it doesn't ever completely flatten out, and it certainly never ever comes back down.

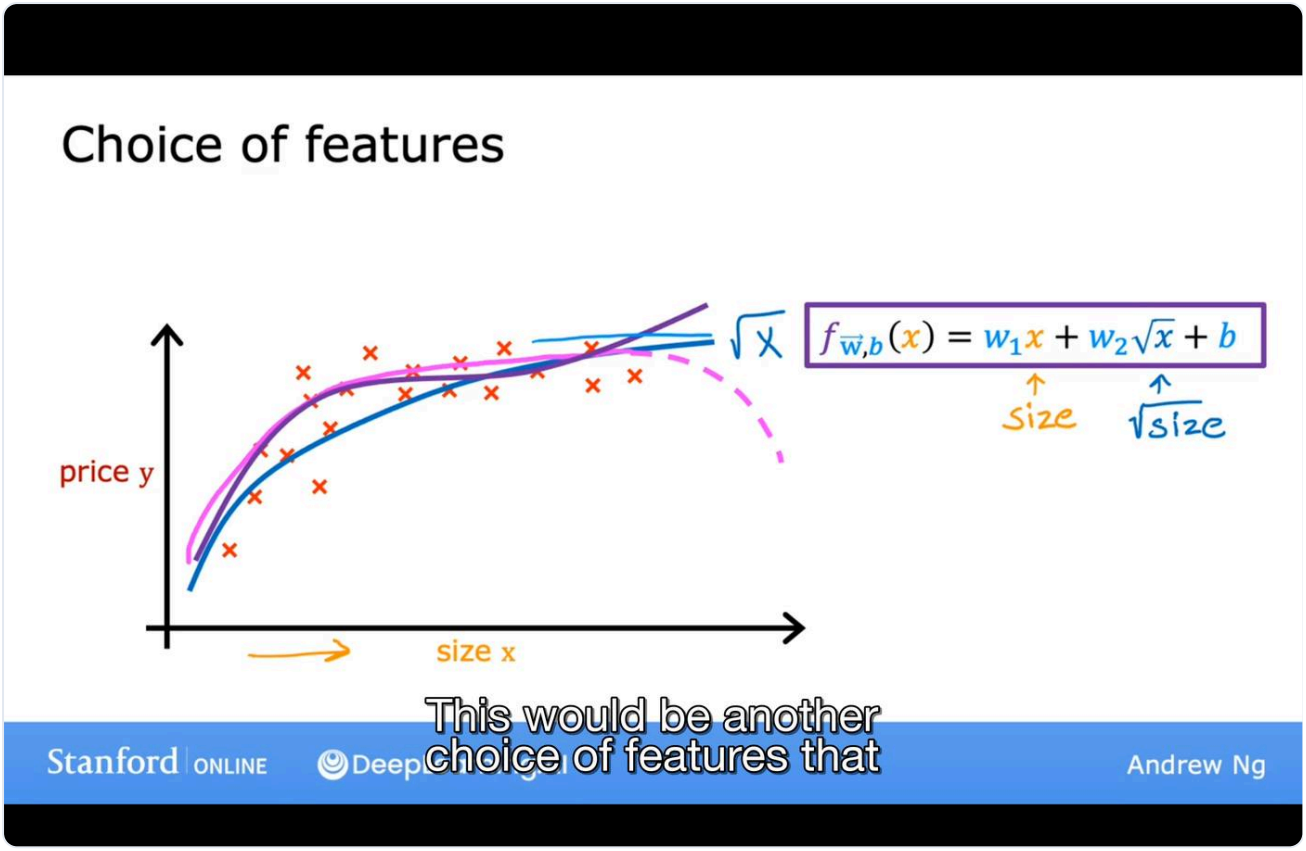


FIGURE 30.9

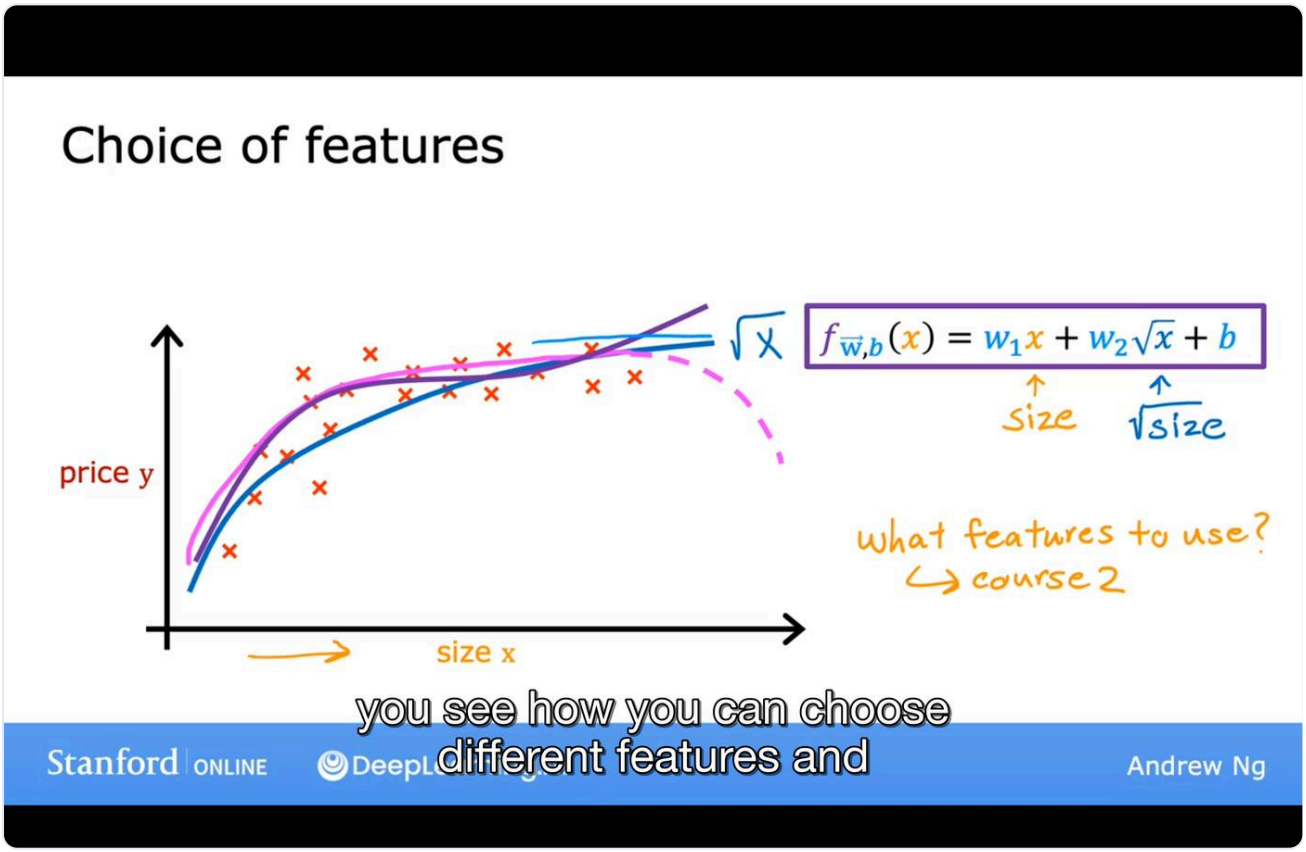


FIGURE 30.10 | Choice of Features

This would be another choice of features that might work well for this data-set as well. You may ask yourself, how do I decide what features to use? Later in the second course in this specialization, you see how you can choose different features and different models that include or don't include these features, and you have a process for measuring how well these different models perform to help you decide which features to include or not include. For now, I just want you to be aware that you have a choice in what features you use. By using feature engineering and polynomial functions, you can potentially get a much better model for your data. In the optional lab that follows this video, you will see some code that implements polynomial regression using features like x, x squared, and x cubed.

Please take a look and run the code and see how it works. There's also another optional lab after that one that shows how to use a popular open source toolkit that implements linear regression. Scikit-learn is a very widely used open source machine learning library that is used by many practitioners in many of the top AI, internet, machine learning companies in the world. If either now or in the future you're using machine learning in your job, there's a very good chance you'll be using tools like Scikit-learn to train your models. Working through that optional lab will give you a chance to not only better understand linear regression, but also see how this can be done in just a few lines of code using a library like Scikit-learn.

For you to have a solid understanding of these algorithms, and be able to apply them, I do think is important that you know how to implement linear regression yourself and not just call some scikit-learn function that is a black-box. But scikit-learn also has an important role in a way machine learning is done in practice today. We're just about at the end of this week. Congratulations on finishing all of this week's videos. Please do take a look at the practice quizzes and also the practice lab, which I hope will let you try out and practice ideas that we've discussed. In this week's practice lab, you implement linear regression. I hope you have a lot of fun getting this learning algorithm to work for yourself.

Best of luck with that. I also look forward to seeing you in next week's videos, where we'll go beyond regression, that is predicting numbers, to talk about our first classification algorithm, which can predict categories. I'll see you next week.

LECTURE 31

Linear Regression and Spam

Welcome to the third week of this course. By the end of this week, you have completed the first course of this specialization. So let's jump in. Last week you learned about linear regression, which predicts a number. This week, you learn about classification where your output variable y can take on only one of a small handful of possible values instead of any number in an infinite range of numbers. It turns out that linear regression is not a good algorithm for classification problems. Let's take a look at why and this will lead us into a different algorithm called logistic regression. Which is one of the most popular and most widely used learning algorithms today.

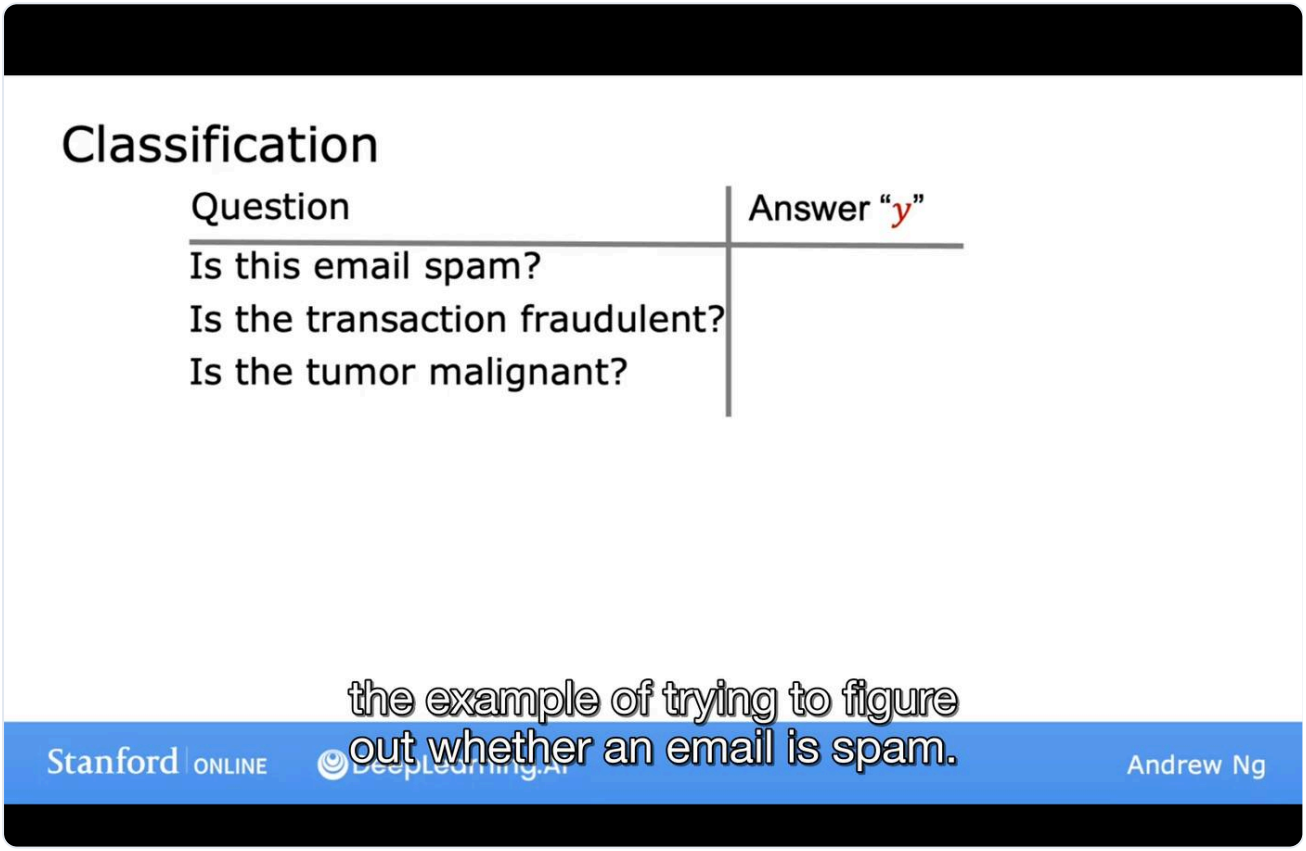


FIGURE 31.1 | Is the Transaction Fraudulent?

Here are some examples of classification problems recall the example of trying to figure out whether an email is spam. So the answer you want to output is going to be either a no or a yes. Another example would be figuring out if an online financial transaction is fraudulent. Fighting online financial fraud is something I once worked on and it was strangely exhilarating. Because I knew there were forces out there trying to steal money and my team's job was to stop them. So the problem is given a financial transaction. Can your learning algorithm figure out is this transaction fraudulent, such as what this credit card stolen? Another example we've touched on before was trying to classify a tumor as malignant versus not.

In each of these problems the variable that you want to predict can only be one of two possible values. No or yes. This type of classification problem where there are only two possible outputs is called binary classification. Where the word binary refers to there being only two possible classes or two possible categories.

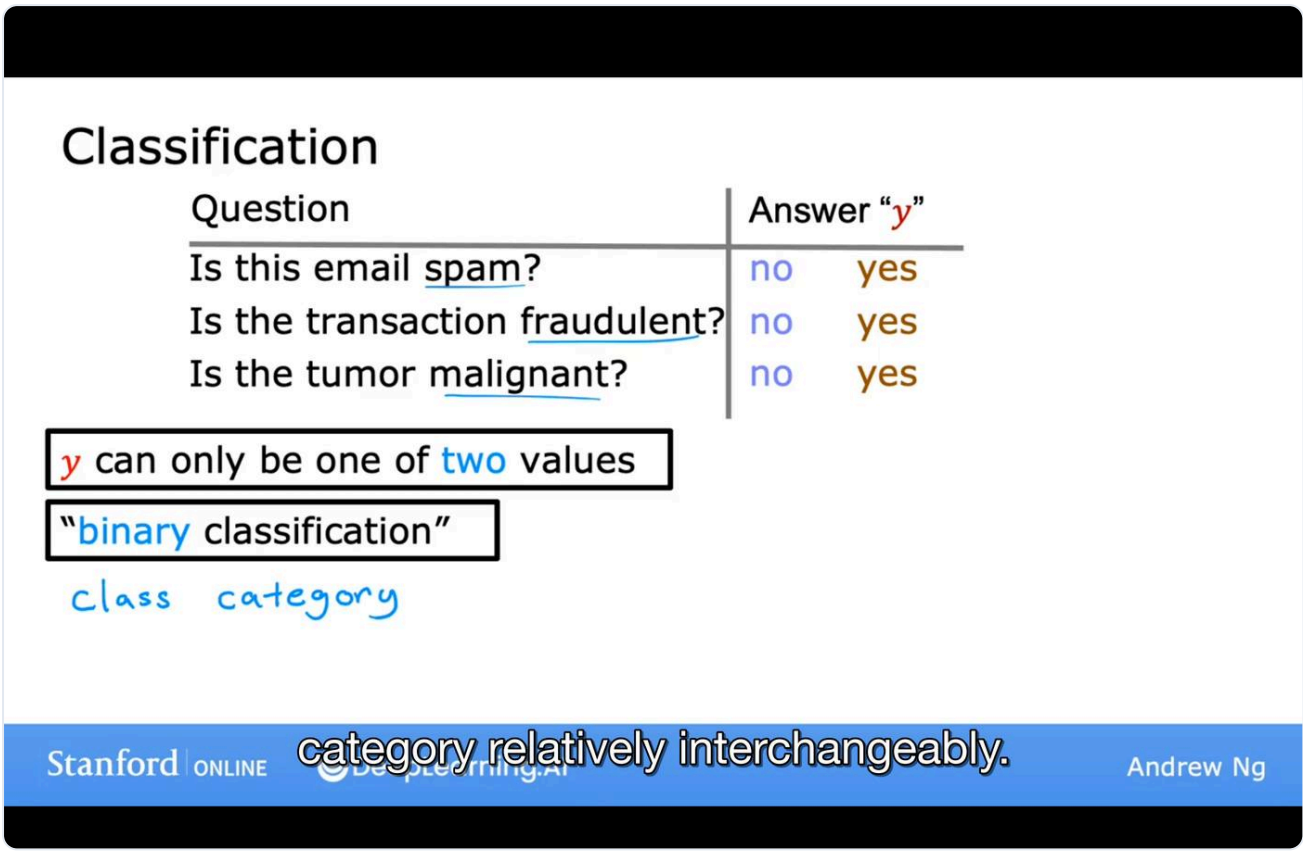


FIGURE 31.2 | Is This Email Spam? 7 Yes

In these problems I will use the terms class and category relatively interchangeably. They mean basically the same thing. By convention we can refer to these two classes or categories in a few common ways. We often designate clauses as no or yes or sometimes equivalently false or true or very commonly using the numbers zero or one.

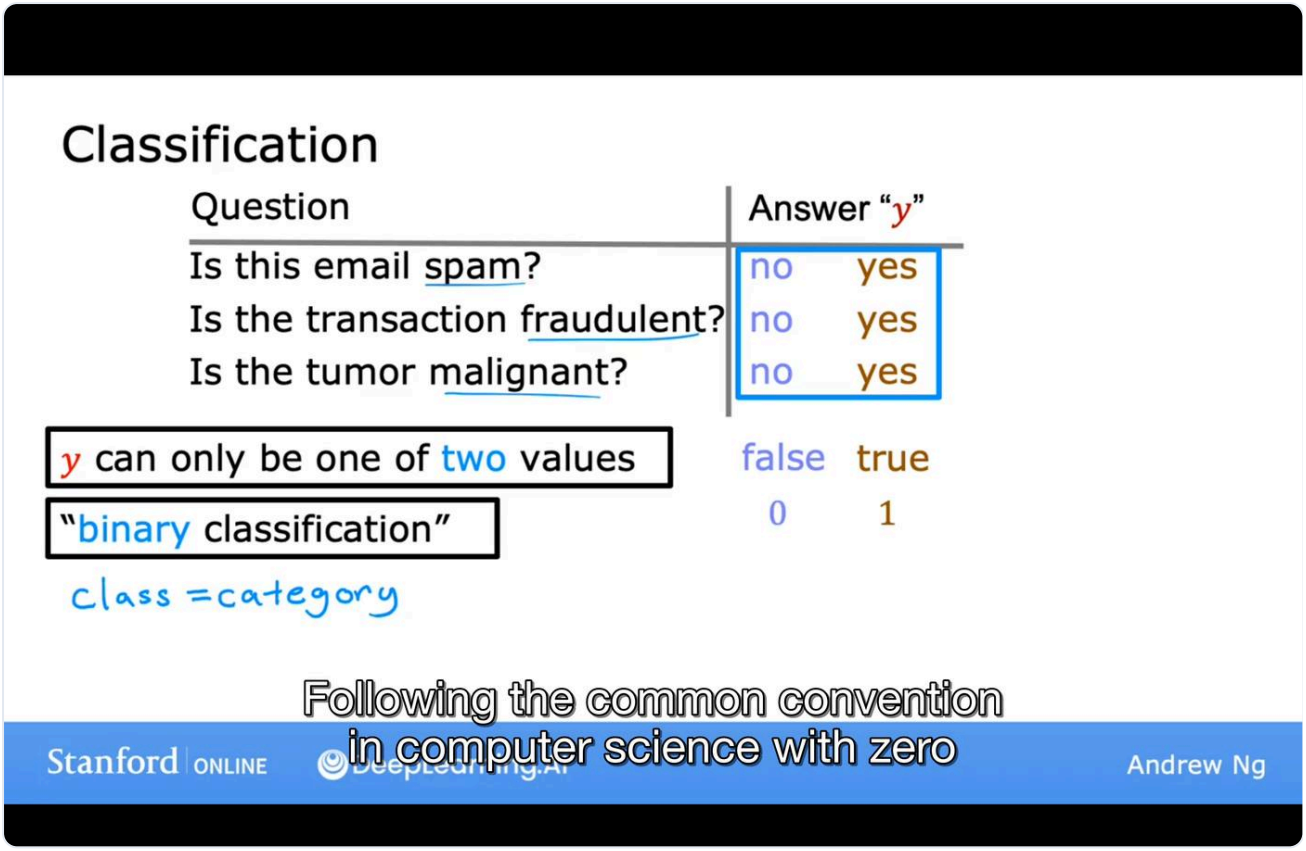


FIGURE 31.3 | Is the Tumor Malignant?

Following the common convention in computer science with zero denoting falls and one denoting true. I'm usually going to use the numbers zero and one to represent the answer y. Because that will fit in most easily with the types of learning algorithms we want to implement.

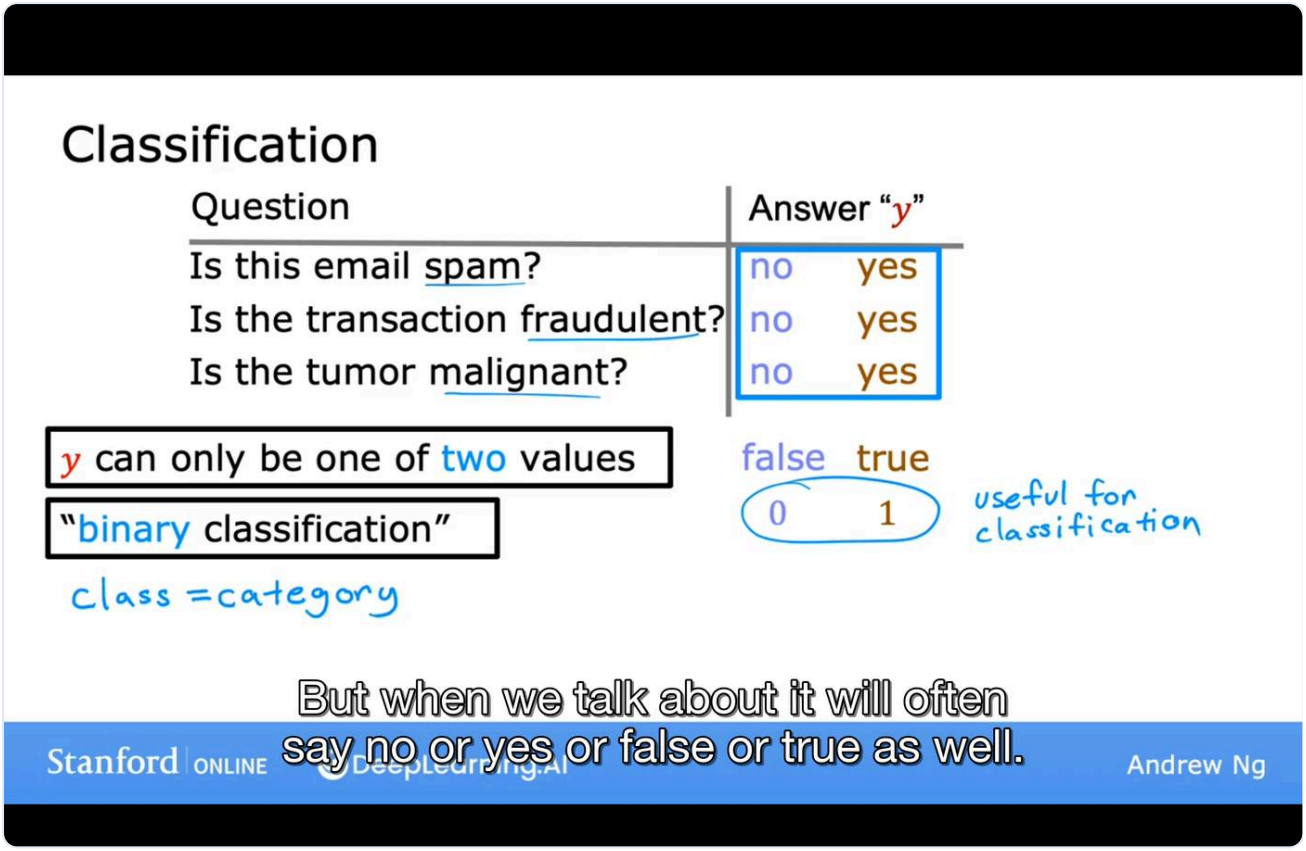


FIGURE 31.4 | Is the Tumor Malignant?

But when we talk about it will often say no or yes or false or true as well. One of the technologies commonly used is to call the false or zero class.

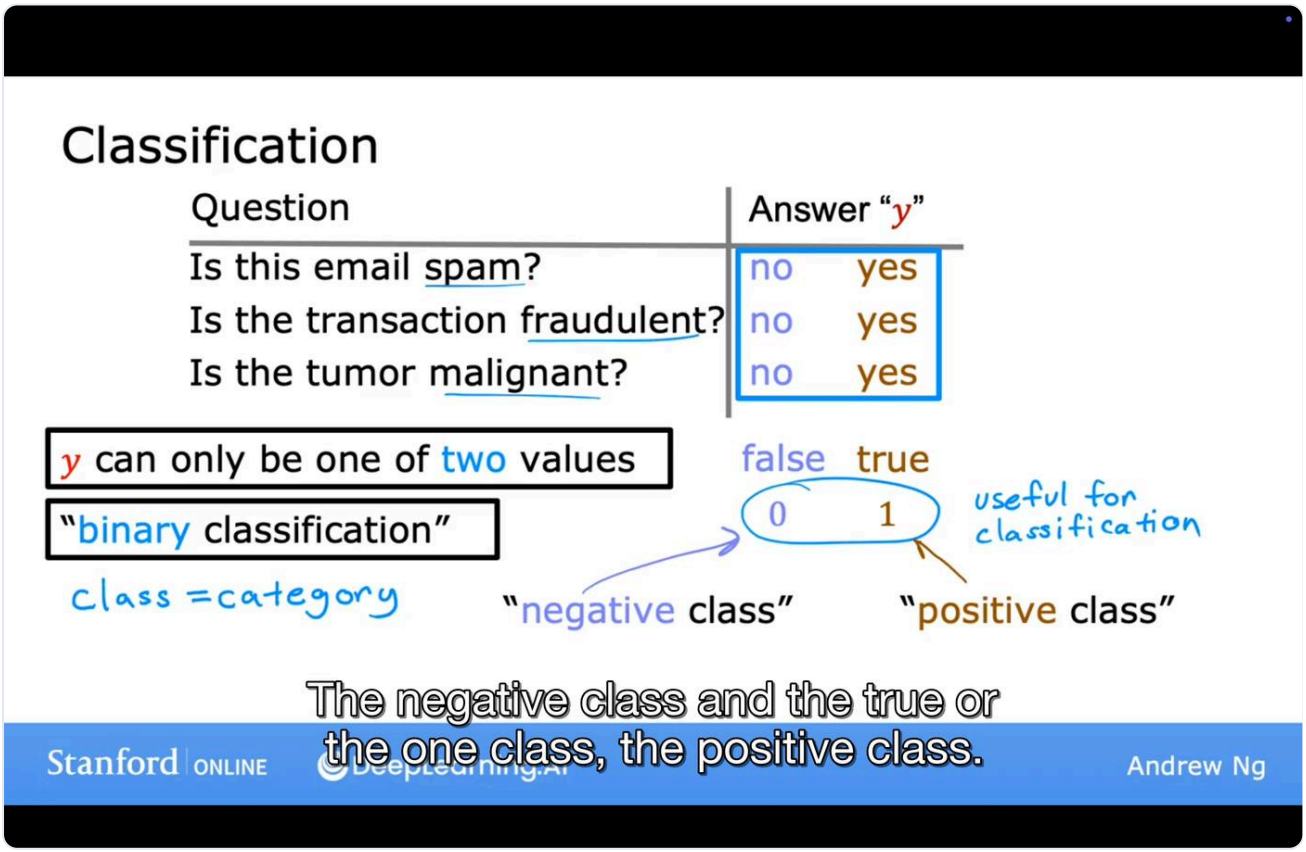


FIGURE 31.5 | Is the Tumor Malignant?

The negative class and the true or the one class, the positive class. For example, for spam classification, an email that is not spam may be referred to as a negative example. Because the output to the question of is a spam. The output is no or zero. In contrast, an email that has spam might be referred to as a positive training example. Because the answer to is it spam is yes or true or one to be clear, negative and positive. Do not necessarily mean bad versus good or evil versus good.

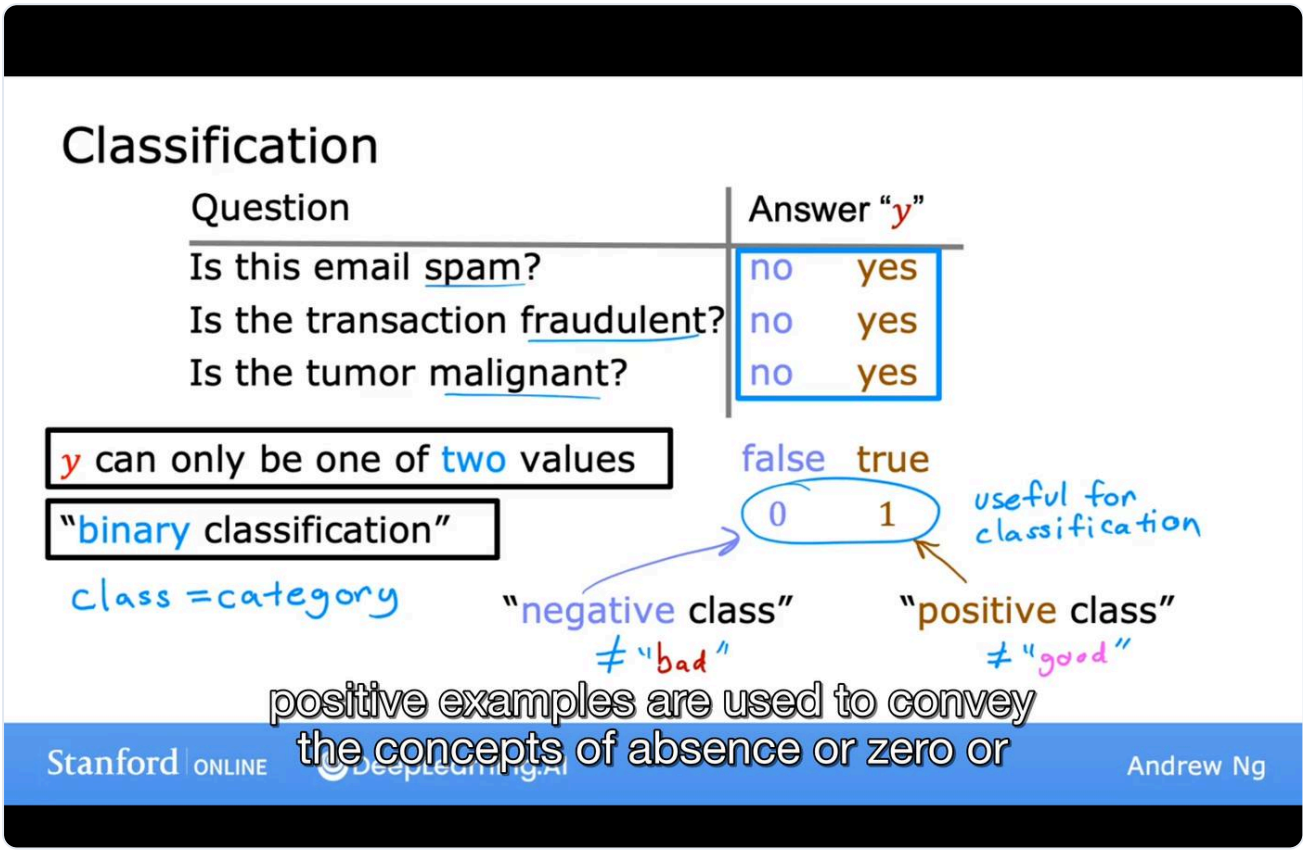


FIGURE 31.6 | Is the Tumor Malignant?

It's just that negative and positive examples are used to convey the concepts of absence or zero or false vs the presence or true or one of something you might be looking for. Such as the absence or presence of the spam illness or the spam property of an email or the absence of presence of broadening activity or absence of presence of malignancy of the tumor. Between non spam and spam emails.

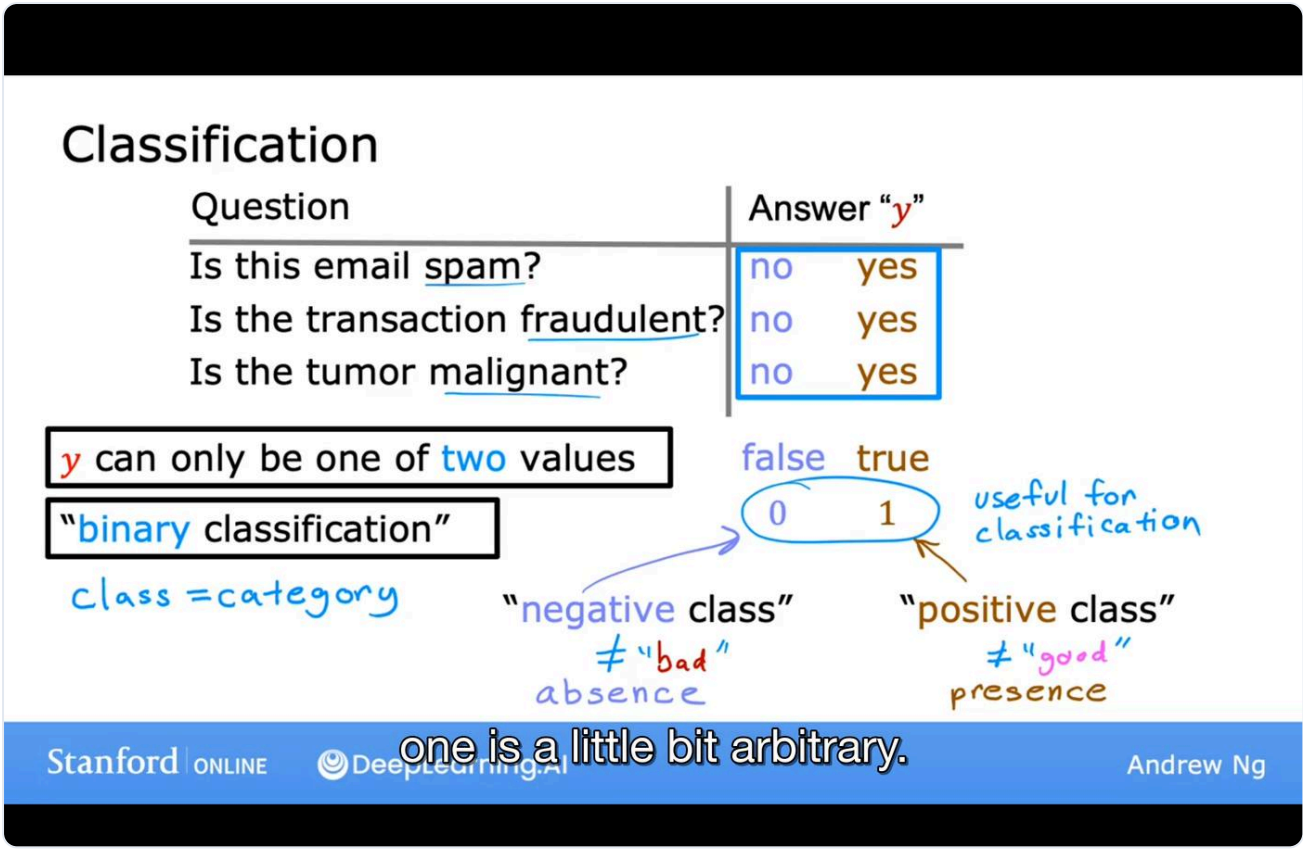


FIGURE 31.7 | Is the Tumor Malignant?

Which one you call false or zero and which one you call true or one is a little bit arbitrary. Often either choice could work. So, different engineer might actually swap it around and have the positive class B. The presence of a good email or the possible causes be the presence of a real financial transaction or a healthy patient. So how do you build a classification algorithm? Here's the example of a training set for classifying if the tumor is malignant. A class one, positive class, yes class or benign, class zero or negative class. I plotted both the tumor size on the horizontal axis as well as the label Y on the vertical axis. By the way, in week one, when we first talked about classification.

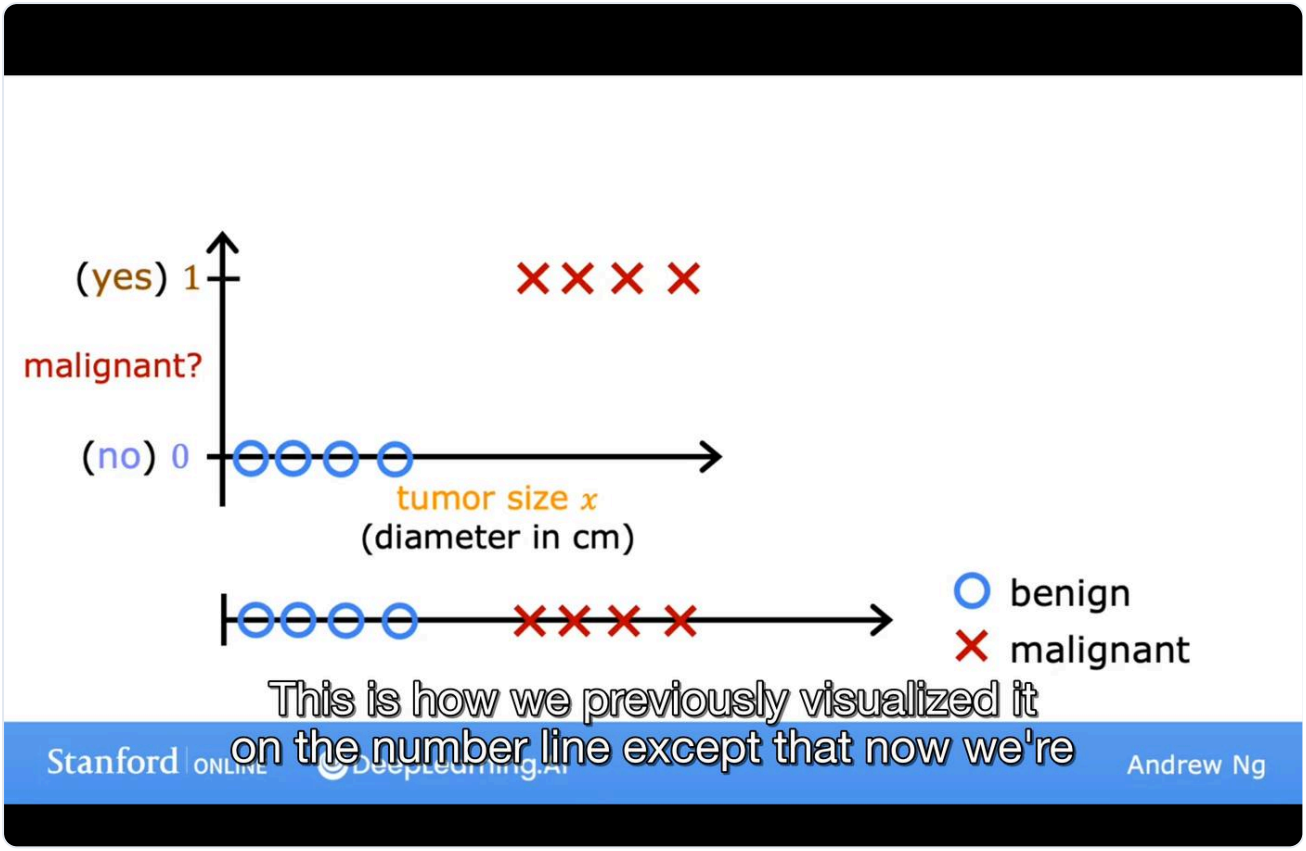


FIGURE 31.8

This is how we previously visualized it on the number line except that now we're calling the classes zero. And one and plotting them on the vertical axis. Now, one thing you could try on this training set is to apply the album you already know. Linear regression and try to fit a straight line to the data. If you do that, maybe the straight line looks like this, right? And that's your F effects. Linear regression predicts not just the values zero and one. But all numbers between zero and one or even less than zero or greater than one. But here we want to predict categories.

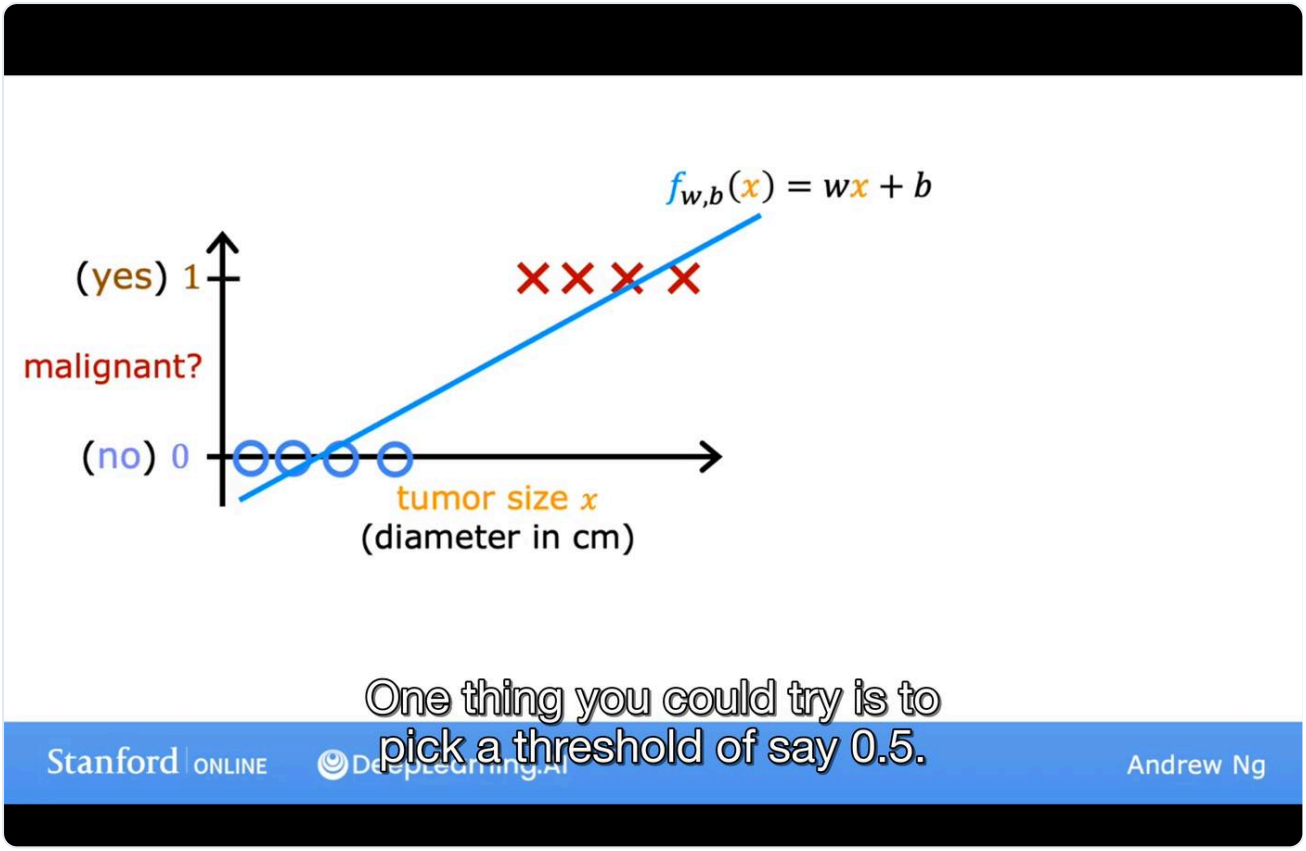


FIGURE 31.9

One thing you could try is to pick a threshold of say 0.5.

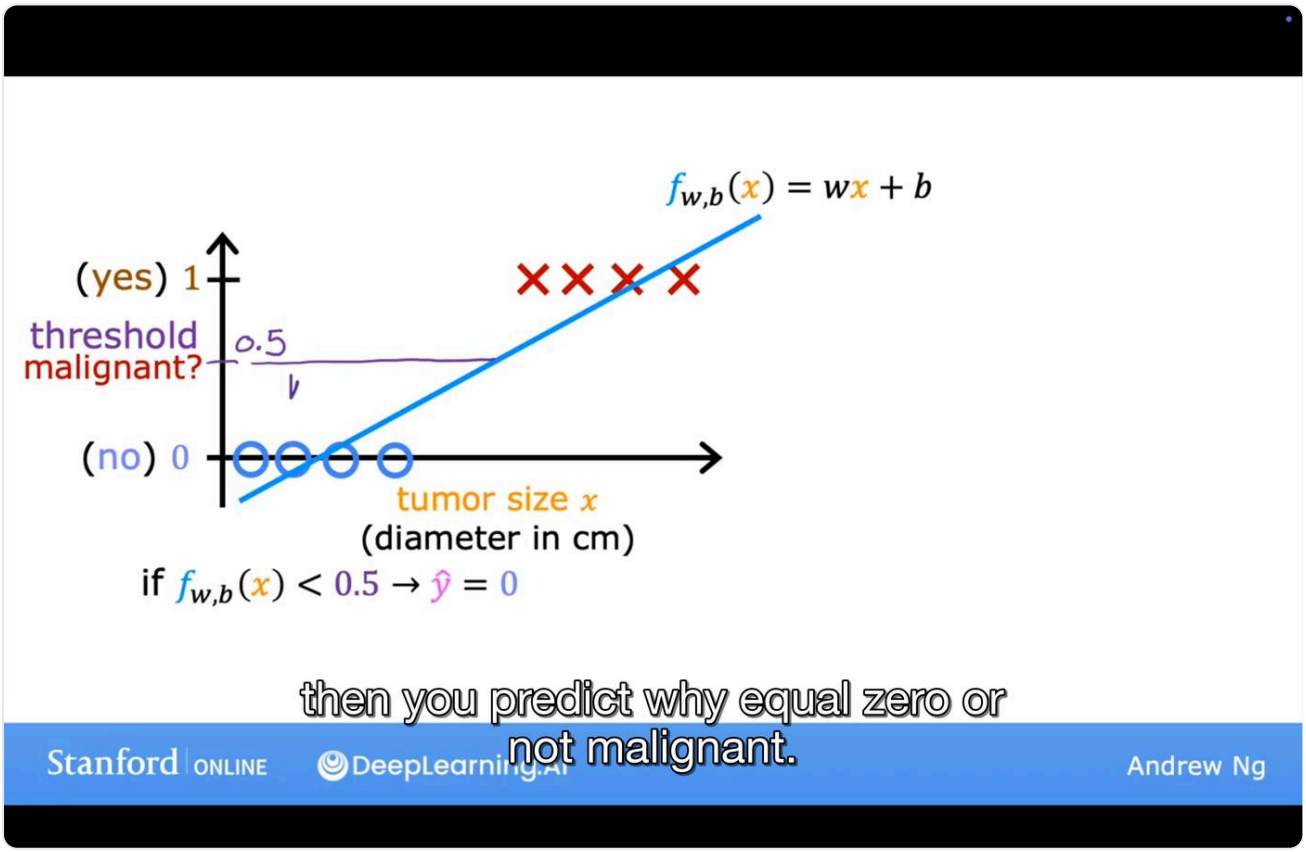


FIGURE 31.10

So that if the model outputs a value below 0.5, then you predict why equal zero or not malignant. And if the model outputs a number equal to or greater than 0.5, then predict Y equals one or malignant. Notice that this threshold value of 0.5 intersects the best fit straight line at this point.

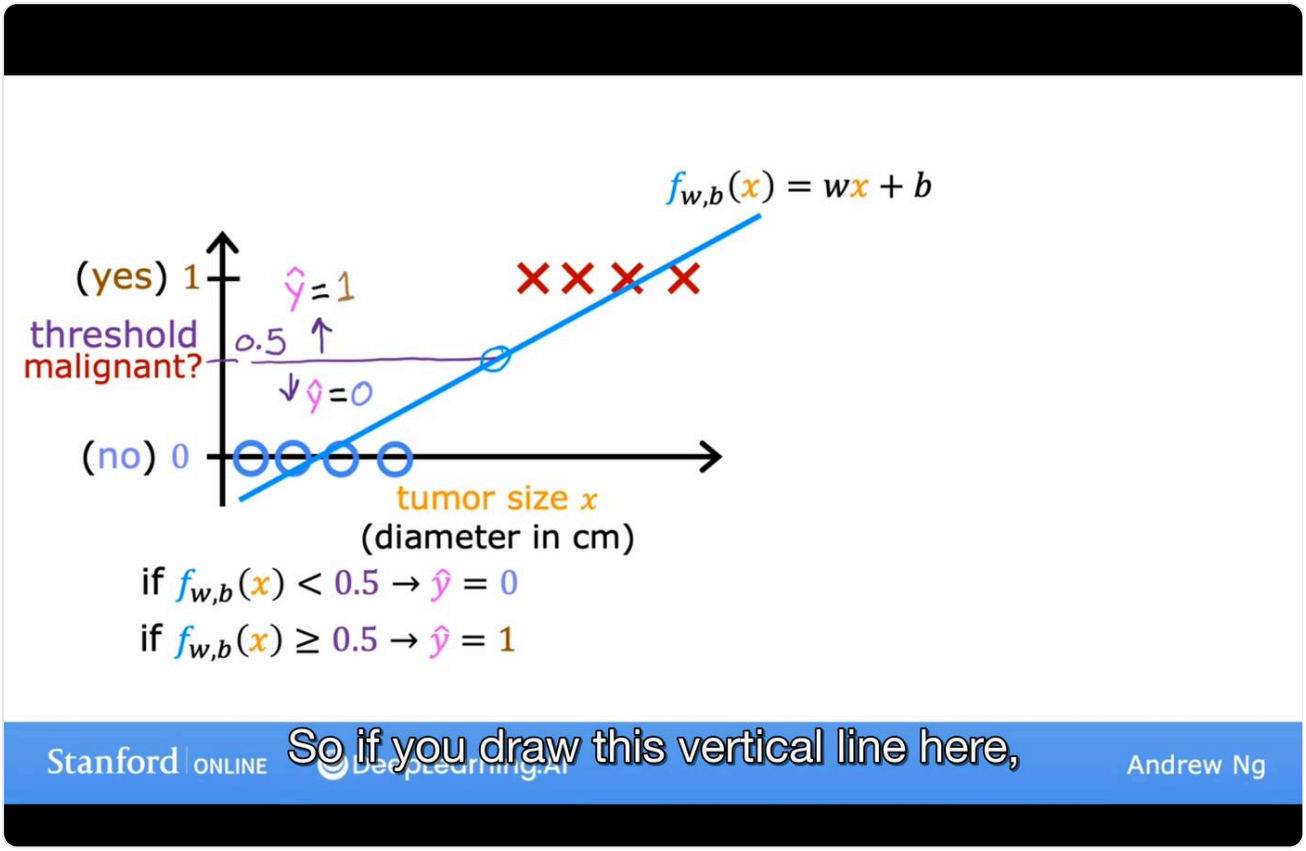


FIGURE 31.11

So if you draw this vertical line here, everything to the left ends up with a prediction of y equals zero. And everything on the right ends up with the prediction of y equals one. Now, for this particular data set it looks like linear regression could do something reasonable. But now let's see what happens if your dataset has one more training example. This one way over here on the right. Let's also extend the horizontal axis. Notice that this training example shouldn't really change how you classify the data points. This vertical dividing line that we drew just now still makes sense as the cut off where tumors smaller than this should be classified as zero.

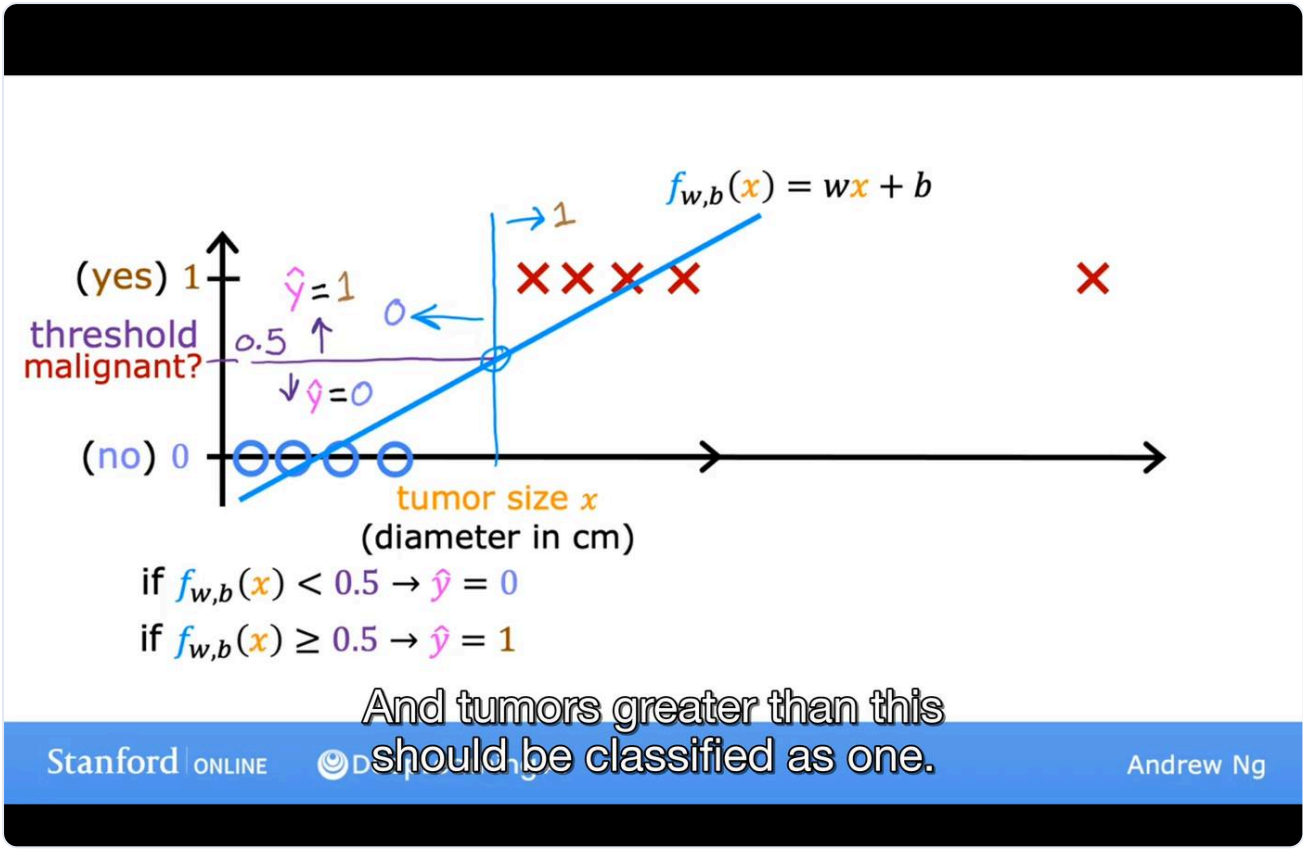


FIGURE 31.12

And tumors greater than this should be classified as one. But once you've added this extra training example on the right. The best fit line for linear regression will shift over like this. And if you continue using the threshold of 0.5, you now notice that everything to the left of this point is predicted at zero non malignant. And everything to the right of this point is predicted to be one or malignant.

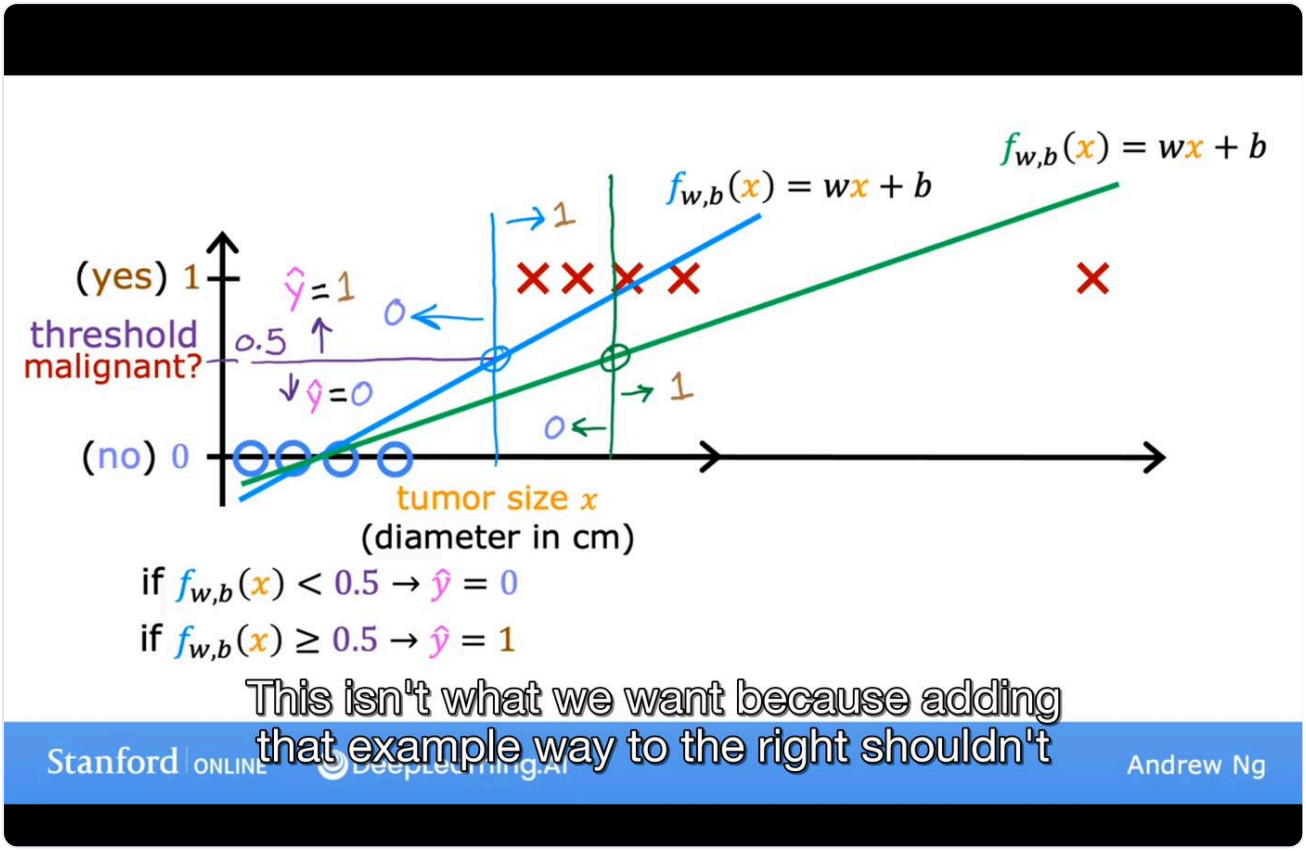


FIGURE 31.13

This isn't what we want because adding that example way to the right shouldn't change any of our conclusions about how to classify malignant versus benign tumors. But if you try to do this with linear regression, adding this one example which feels like it shouldn't be changing anything. It ends up with us learning a much worse function for this classification problem. Clearly, when the tumor is large, we want the algorithm to classify it as malignant. So what we just saw was linear regression causes the best fit line.

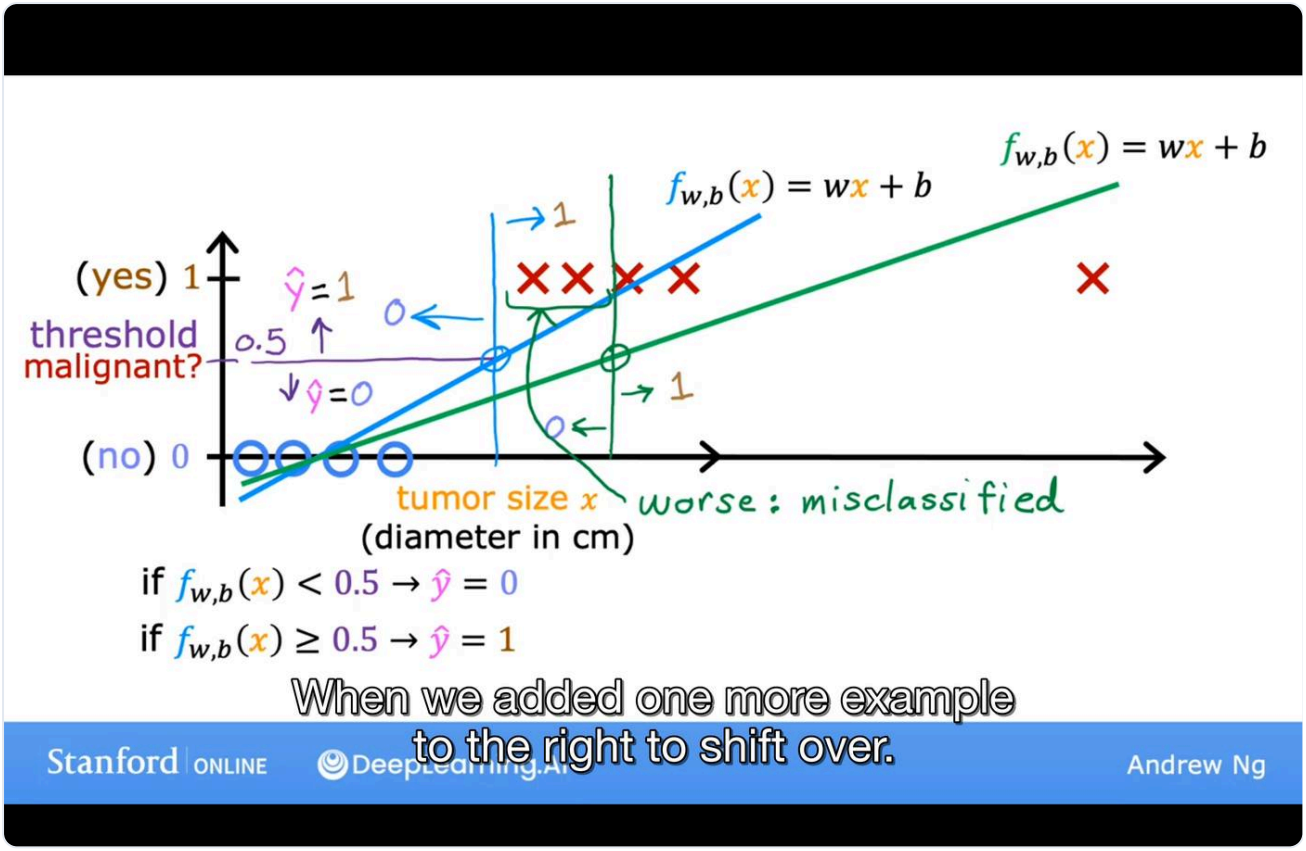


FIGURE 31.14

When we added one more example to the right to shift over.

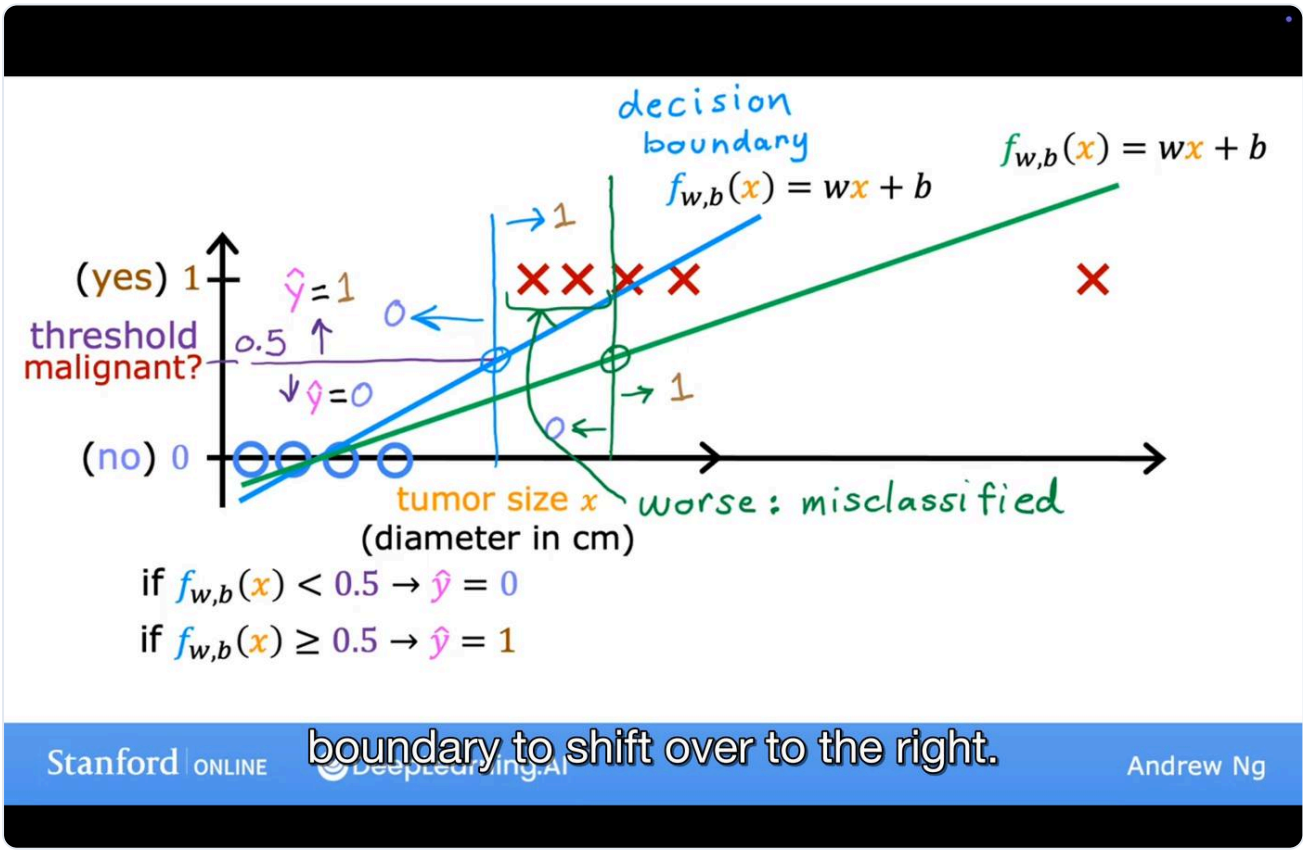


FIGURE 31.15

And does the dividing line also called the decision boundary to shift over to the right. You learn more about the decision boundary in the next video, you also learn about an algorithm called logistic regression. Where the output value of the outcome will always be between zero and one. And the average will avoid these problems that we're seeing on this slide. By the way one thing confusing about the name logistic regression is that even though it has the word of regression in it is actually used for classification.

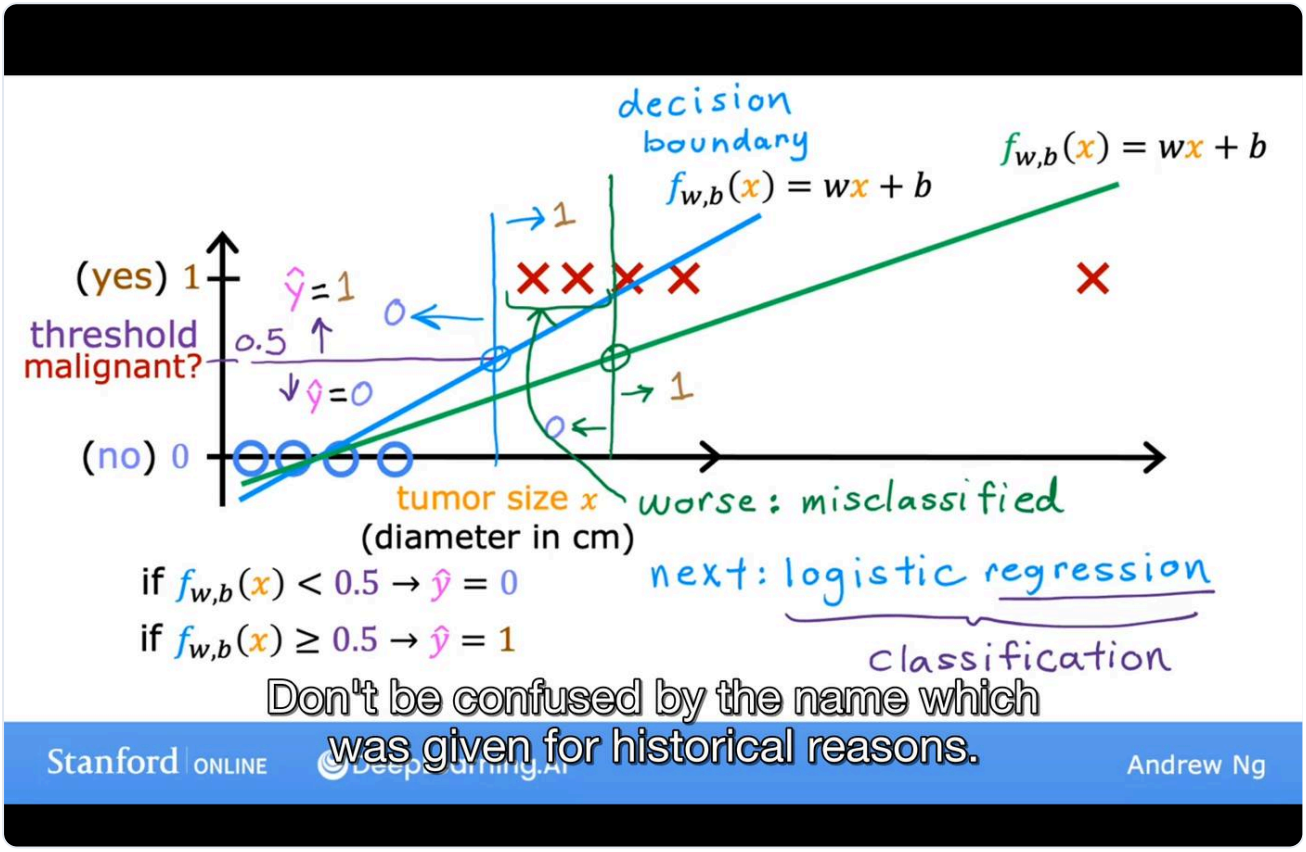


FIGURE 31.16

Don't be confused by the name which was given for historical reasons. It's actually used to solve binary classification problems with output label y is either zero or one. In the upcoming optional lab you also get to take a look at what happens when you try to use linear regression for classification. Sometimes you get lucky and it may work but often it will not work well. Which is why I don't use linear regression myself for classification. In the optional lab, you see an interactive plot that attempts to classify between two categories. And hopefully notice how this often doesn't work very well. Which is okay because that motivates the need for a different model to do classification talks.

So please check out this optional lab and after that we're going to the next video to look at logistic regression for classification.

LECTURE 32

Interpretation of Logistic Regression Output

Let's talk about logistic regression, which is probably the single most widely used classification algorithm in the world. This is something that I use all the time in my work. Let's continue with the example of classifying whether a tumor is malignant. Whereas before we're going to use the label 1 or yes to the positive class to represent malignant tumors, and zero or no and negative examples to represent benign tumors.

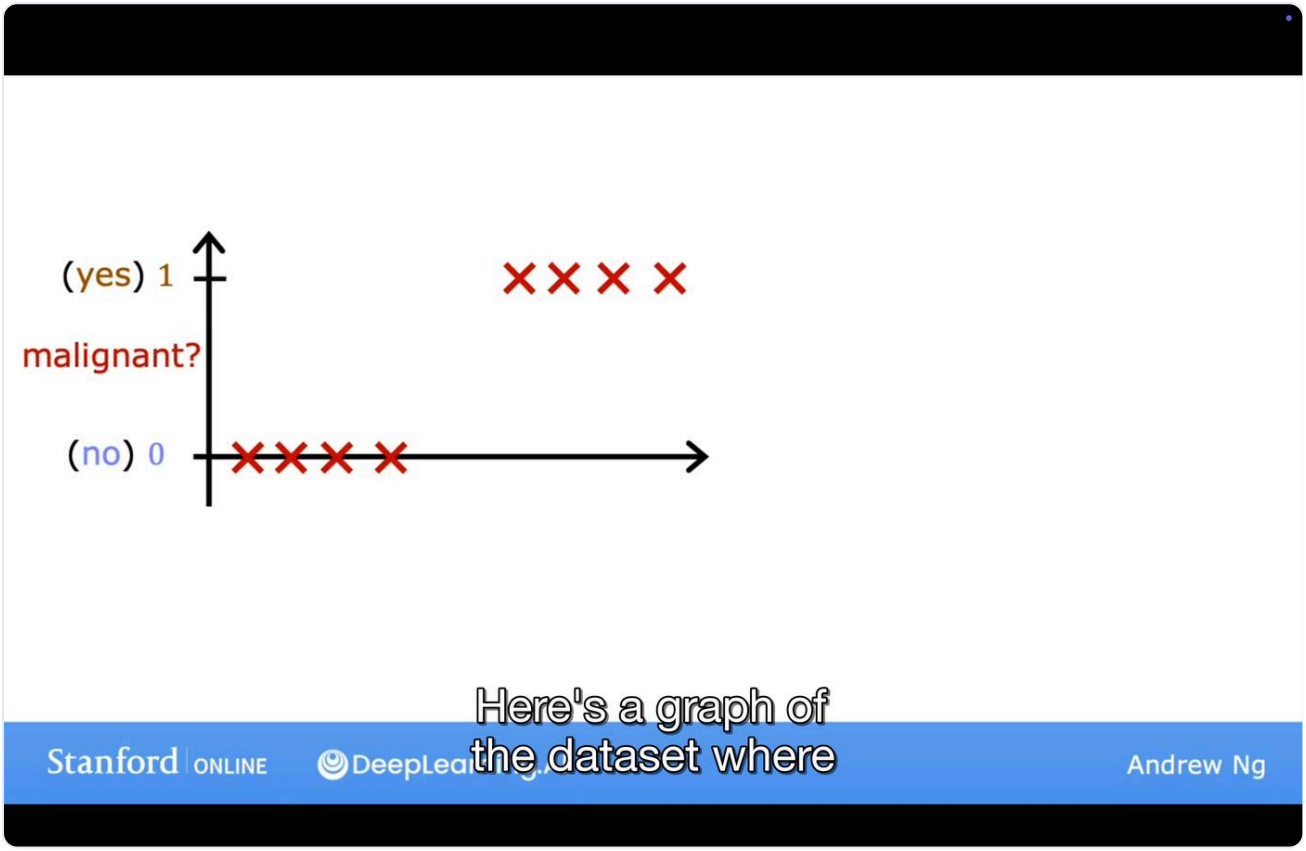


FIGURE 32.1

Here's a graph of the dataset where the horizontal axis is the tumor size and the vertical axis takes on only values of 0 and 1, because is a classification problem. You saw in the last video that linear regression is not a good algorithm for this problem. In contrast, what logistic regression we end up doing is fit a curve that looks like this, S-shaped curve to this dataset.

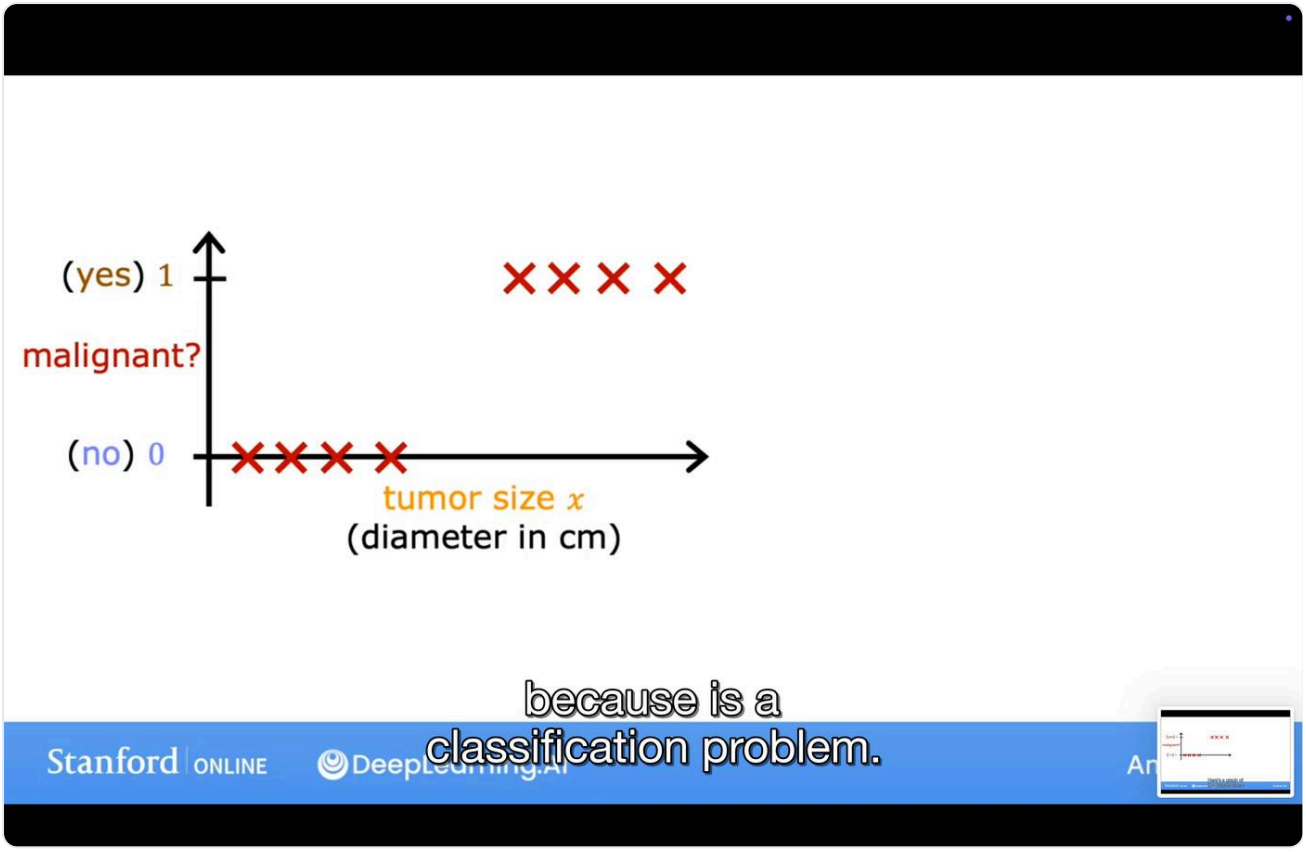


FIGURE 32.2

For this example, if a patient comes in with a tumor of this size, which I'm showing on the x-axis, then the algorithm will output 0.7 suggesting that is closer or maybe more likely to be malignant and benign. Will say more later what 0.7 actually means in this context. But the output label y is never 0.7 is only ever 0 or 1.

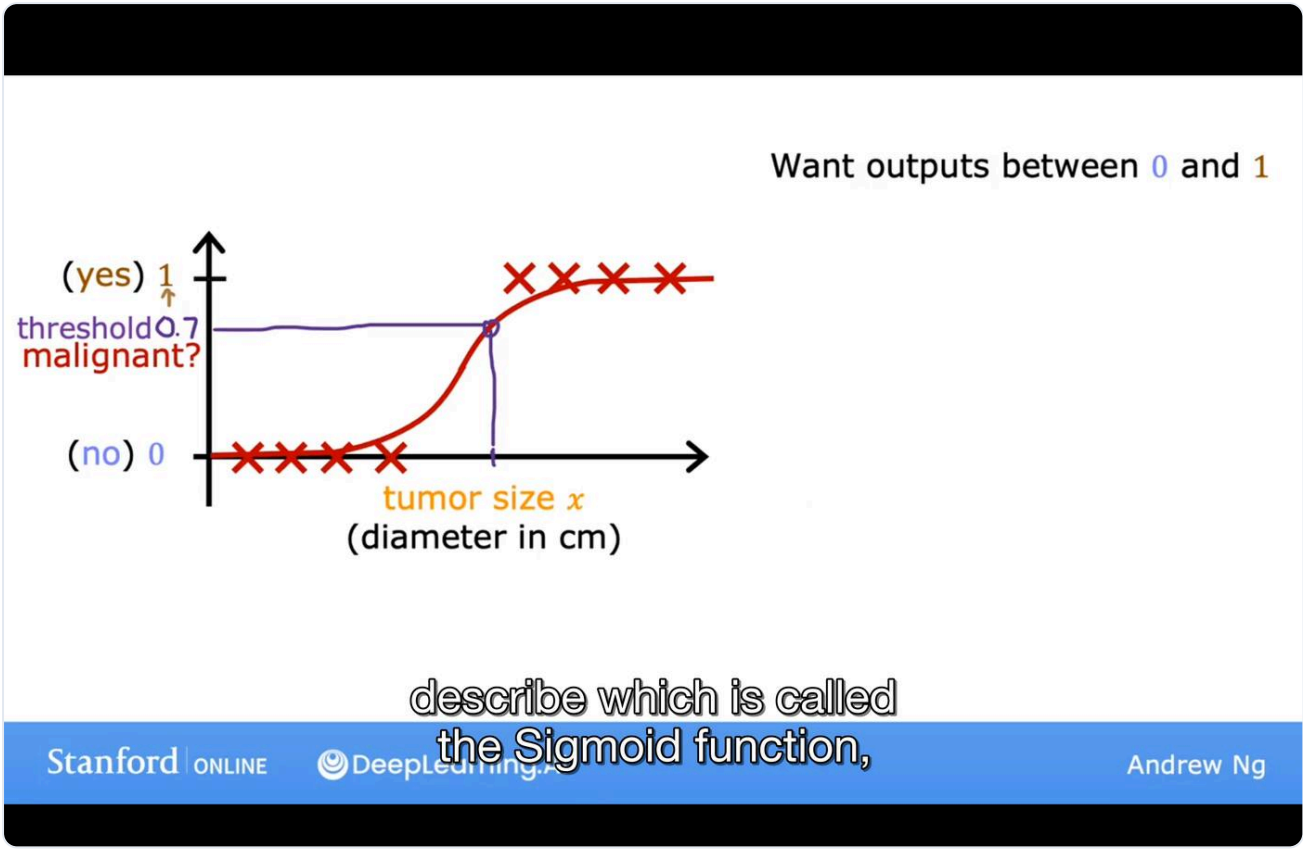


FIGURE 32.3 | Want Outputs Between 0 and 1

To build out to the logistic regression algorithm, there's an important mathematical function I like to describe which is called the Sigmoid function, sometimes also referred to as the logistic function. The Sigmoid function looks like this. Notice that the x-axis of the graph on the left and right are different. In the graph to the left on the x-axis is the tumor size, so is all positive numbers.

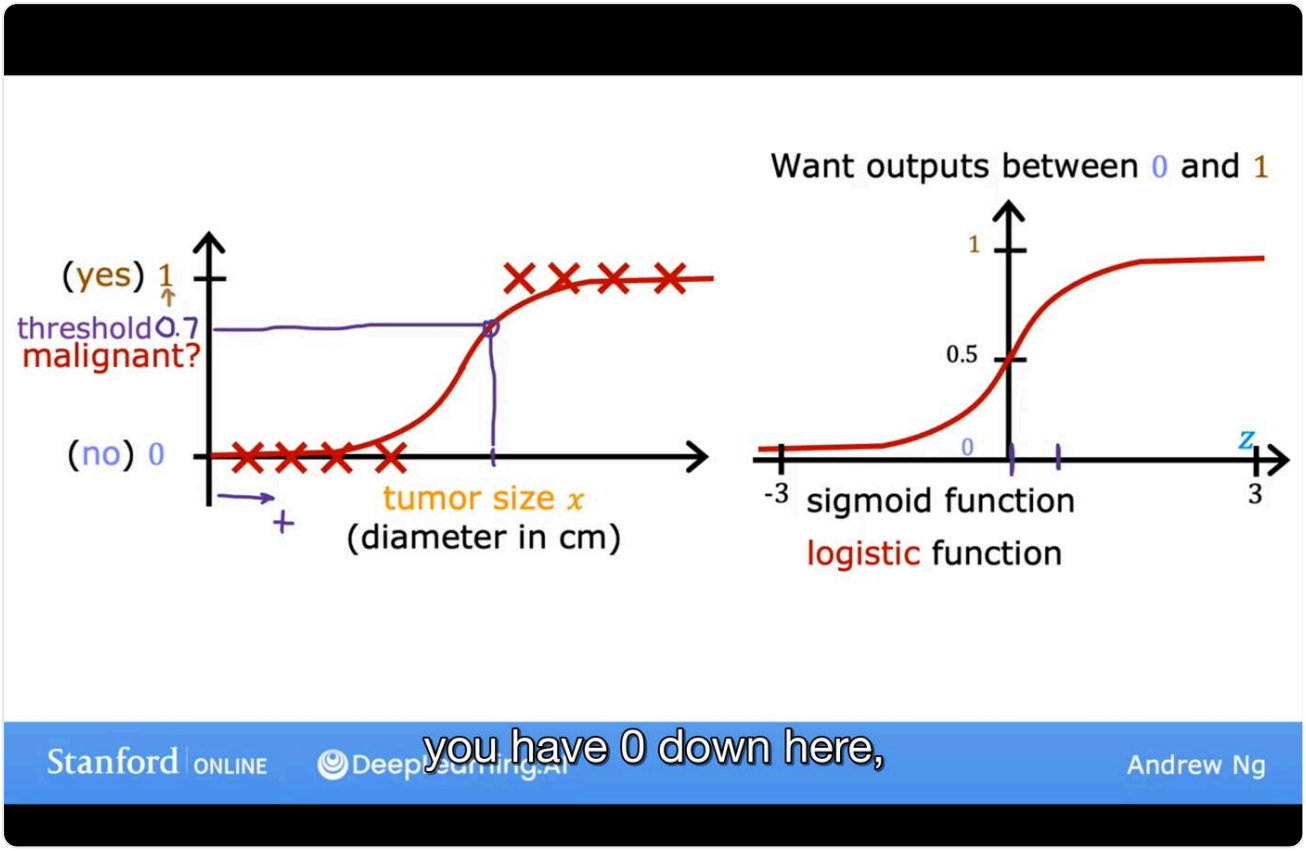


FIGURE 32.4 | Want Outputs Between 0 and 1

Whereas in the graph on the right, you have 0 down here, and the horizontal axis takes on both negative and positive values and have label the horizontal axis Z. I'm showing here just a range of negative 3 to plus 3. So the Sigmoid function outputs value is between 0 and 1.

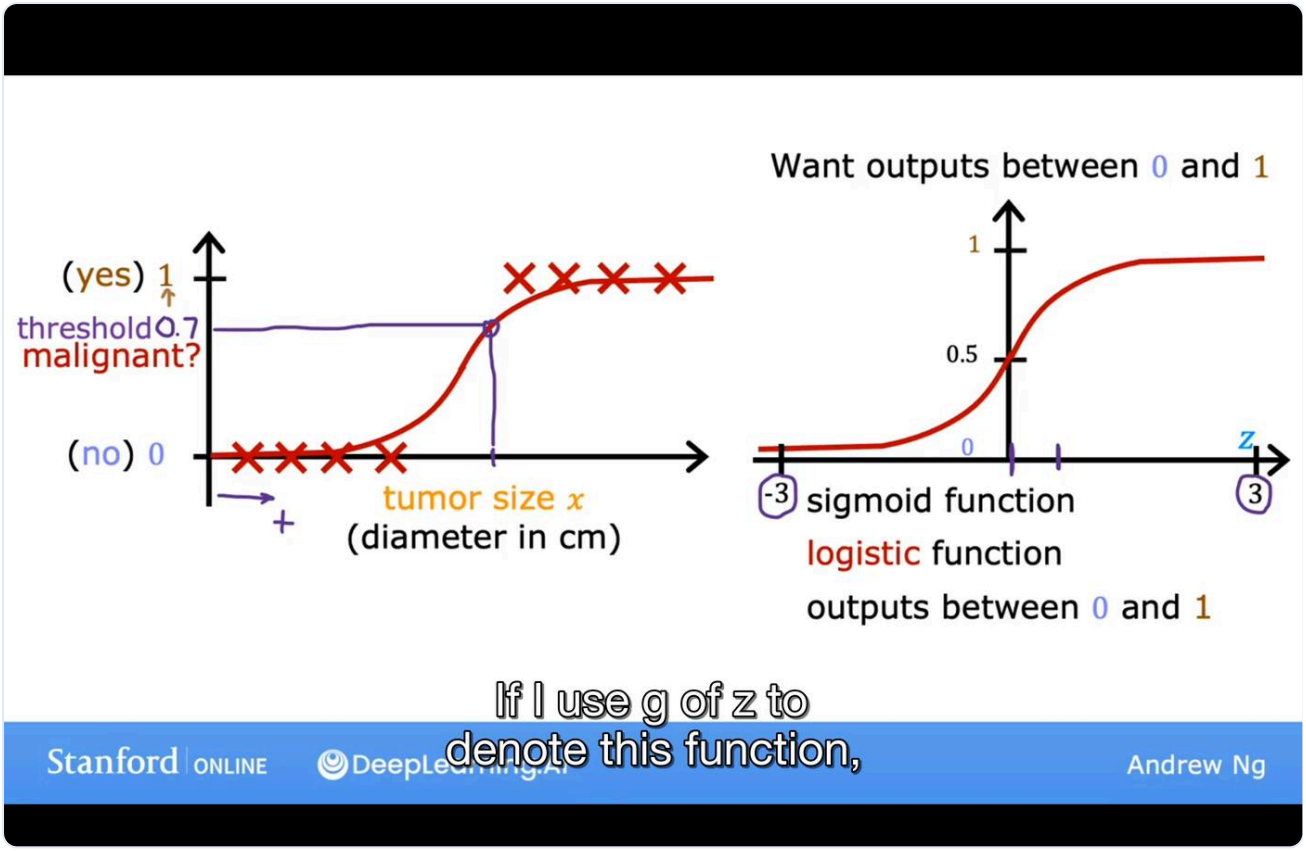


FIGURE 32.5

If I use g of z to denote this function, then the formula of g of z is equal to 1 over 1 plus e to the negative z. Where here e is a mathematical constant that takes on a value of about 2.7, and so e to the negative z is that mathematical constant to the power of negative z. Notice if z where really be, say a 100, e to the negative z is e to the negative 100 which is a tiny number. So this ends up being 1 over 1 plus a tiny little number, and so the denominator will be basically very close to 1. Which is why when z is large, g of z that is a Sigmoid function of z is going to be very close to 1. Conversely, you can also check for yourself that when z is a very large negative number, then g of z becomes 1 over a giant number, which is why g of z is very close to 0.

That's why the sigmoid function has this shape where it starts very close to zero and slowly builds up or grows to the value of one.

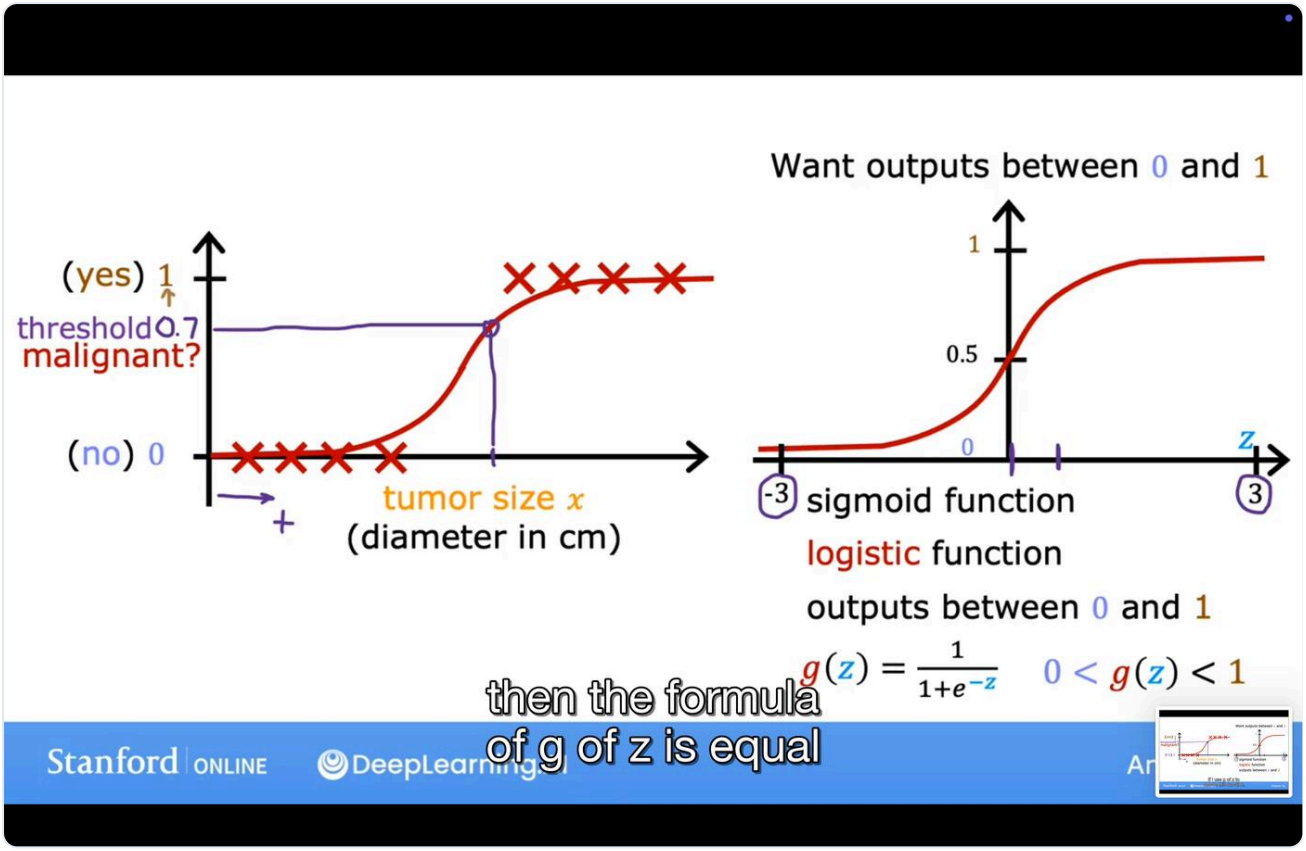


FIGURE 32.6

Also, in the Sigmoid function when z is equal to 0, then e to the negative z is e to the negative 0 which is equal to 1, and so g of z is equal to 1 over 1 plus 1 which is 0.5, so that's why it passes the vertical axis at 0.5. Now, let's use this to build up to the logistic regression algorithm. We're going to do this in two steps. In the first step, I hope you remember that a straight line function, like a linear regression function can be defined as w. product of x plus b. Let's store this value in a variable which I'm going to call z, and this will turn out to be the same z as the one you saw on the previous slide, but we'll get to that in a minute.

The next step then is to take this value of z and pass it to the Sigmoid function, also called the logistic function, g. Now, g of z then outputs a value computed by this formula, 1 over 1 plus e to the negative z. There's going to be between 0 and 1. When you take these two equations and put them together, they then give you the logistic regression model f of x, which is equal to g of wx plus b. Or equivalently g of z, which is equal to this formula over here.

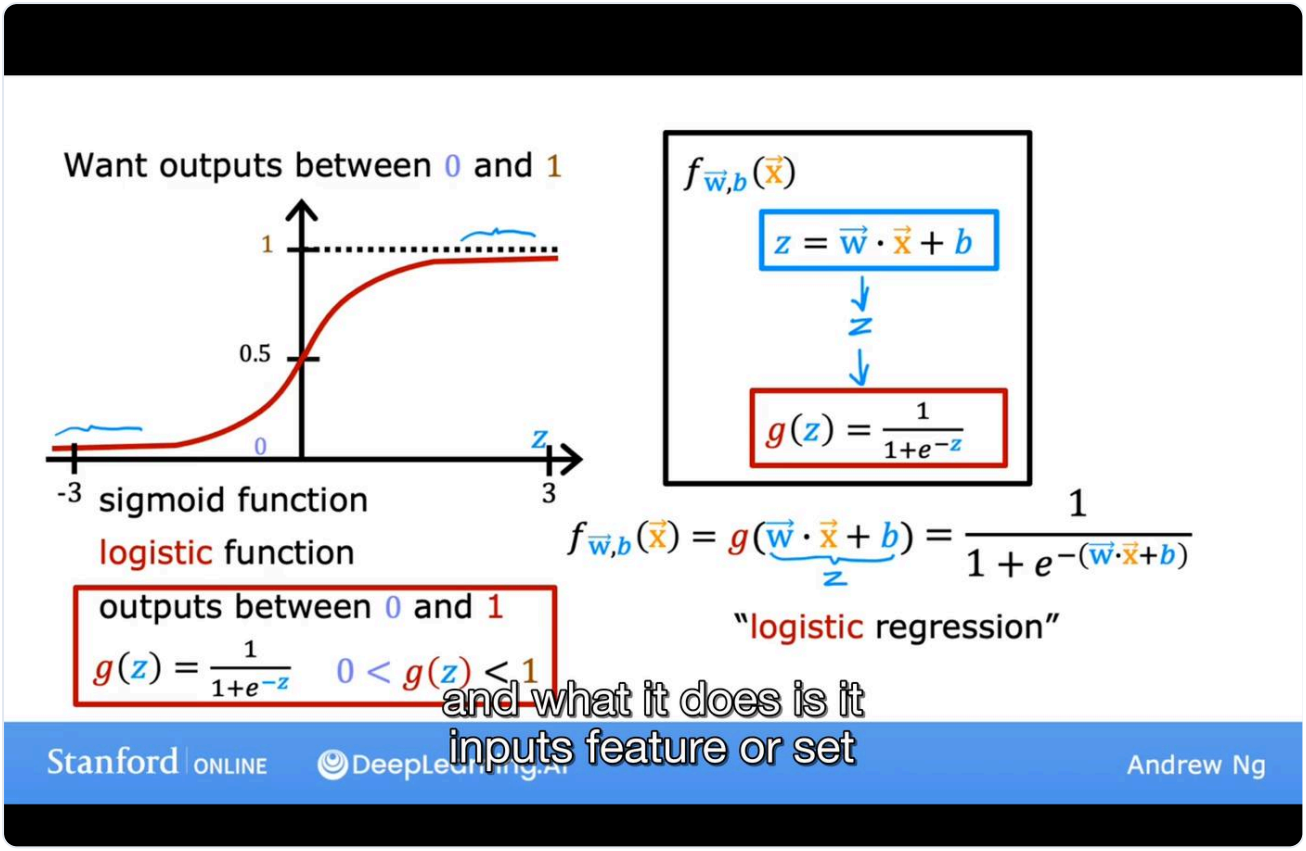


FIGURE 32.7 | 3 Sigmoid Function 3

This is the logistic regression model, and what it does is it inputs feature or set of features X and outputs a number between 0 and 1. Next, let's take a look at how to interpret the output of logistic regression. We'll return to the tumor classification example. The way I encourage you to think of logistic regressions output is to think of it as outputting the probability that the class or the label y will be equal to 1 given a certain input x.

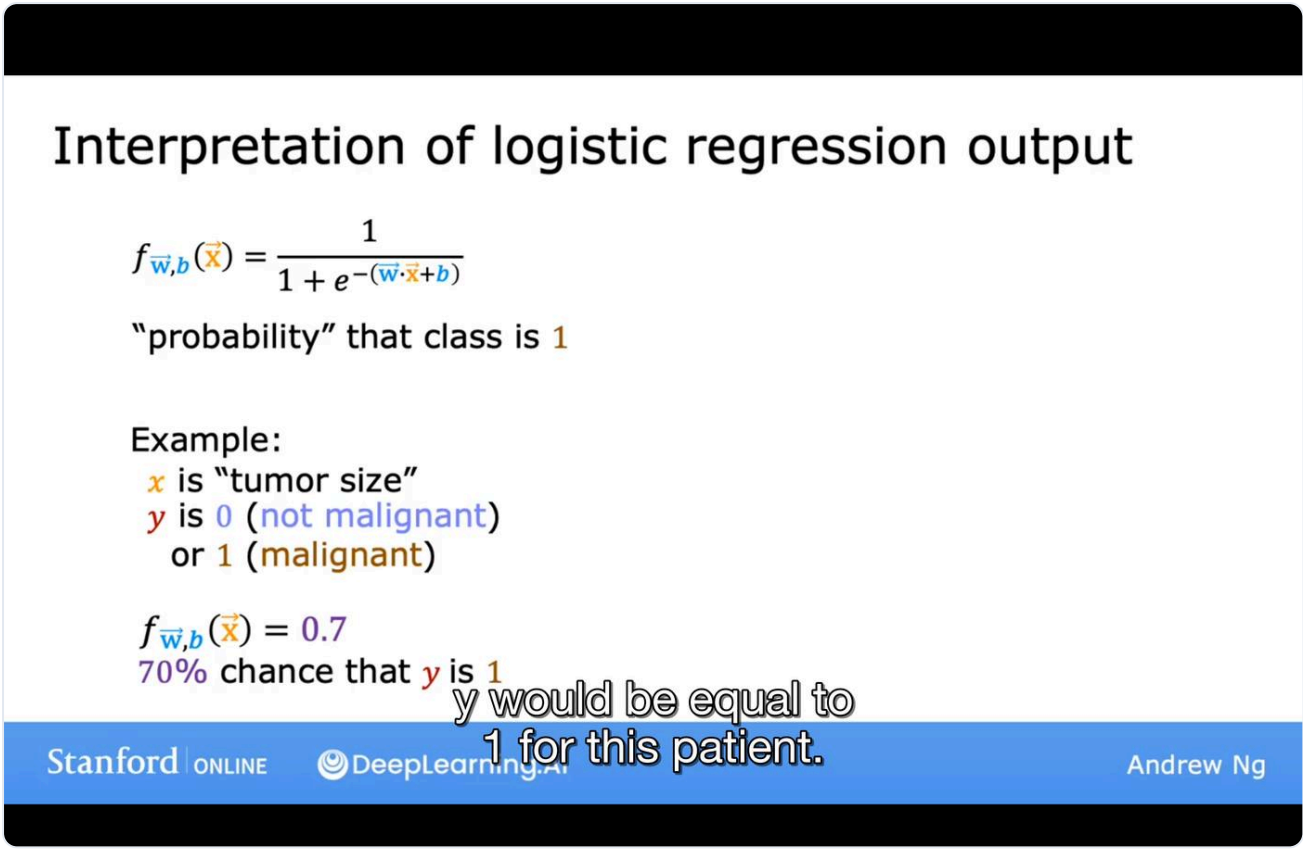


FIGURE 32.8 | Interpretation of Logistic Regression Output

For example, in this application, where x is the tumor size and y is either 0 or 1, if you have a patient come in and she has a tumor of a certain size x, and if based on this input x, the model I'll plus 0.7, then what that means is that the model is predicting or the model thinks there's a 70 percent chance that the true label y would be equal to 1 for this patient. In other words, the model is telling us that it thinks the patient has a 70 percent chance of the tumor turning out to be malignant. Now, let me ask you a question. See if you can get this right. We know that y has to be either 0 or 1, so if y has a 70 percent chance of being 1, what is the chance that it is 0?

So y has got to be either 0 or 1, and thus the probability of it being 0 or 1 these two numbers have to add up to one or to a 100 percent chance. That's why if the chance of y being 1 is 0.7 or 70 percent chance, then the chance of it being 0 has got to be 0.3 or 30 percent chance. If someday you read research papers or blog pulls of all logistic regression, sometimes you see this notation that f of x is equal to p of y equals 1 given the input features x and with parameters w and b.

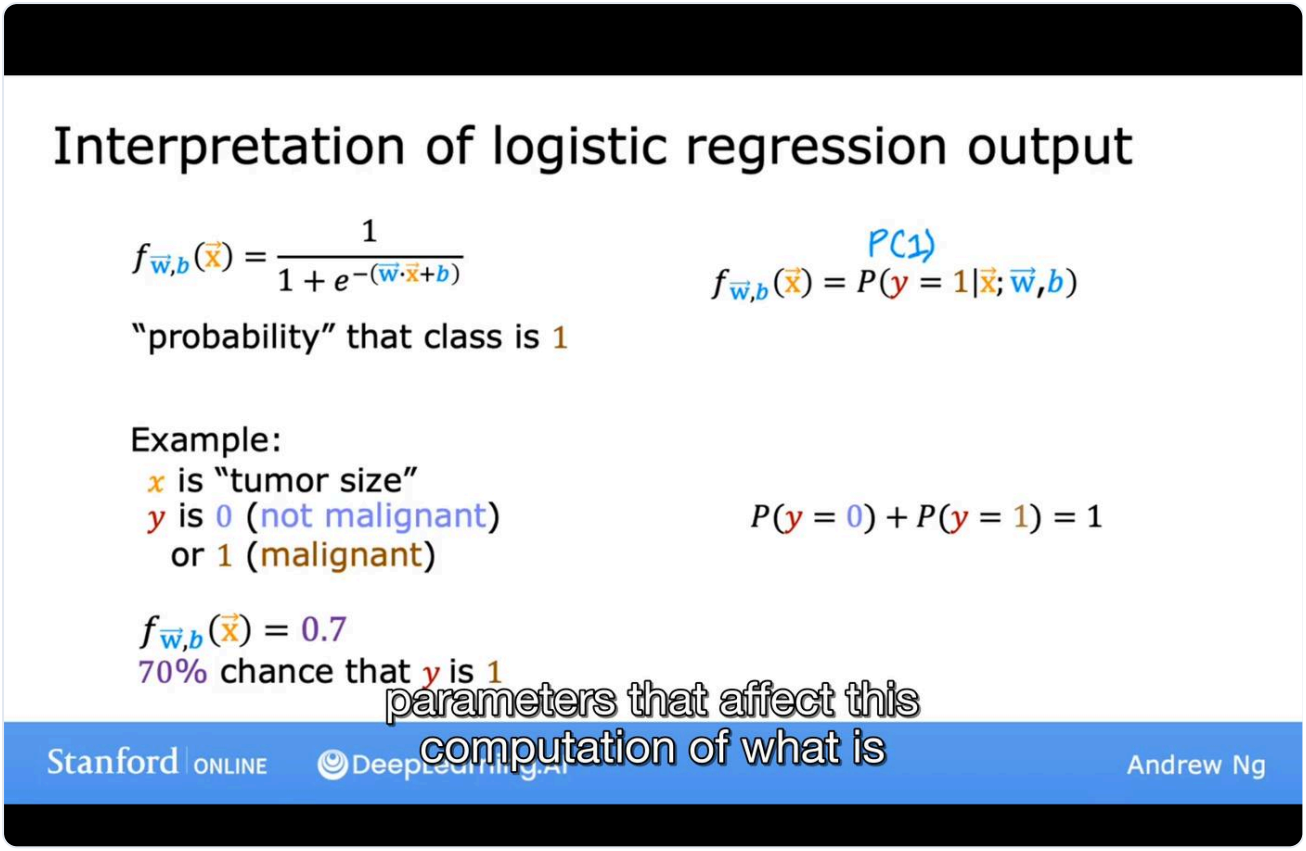


FIGURE 32.9 | Interpretation of Logistic Regression Output

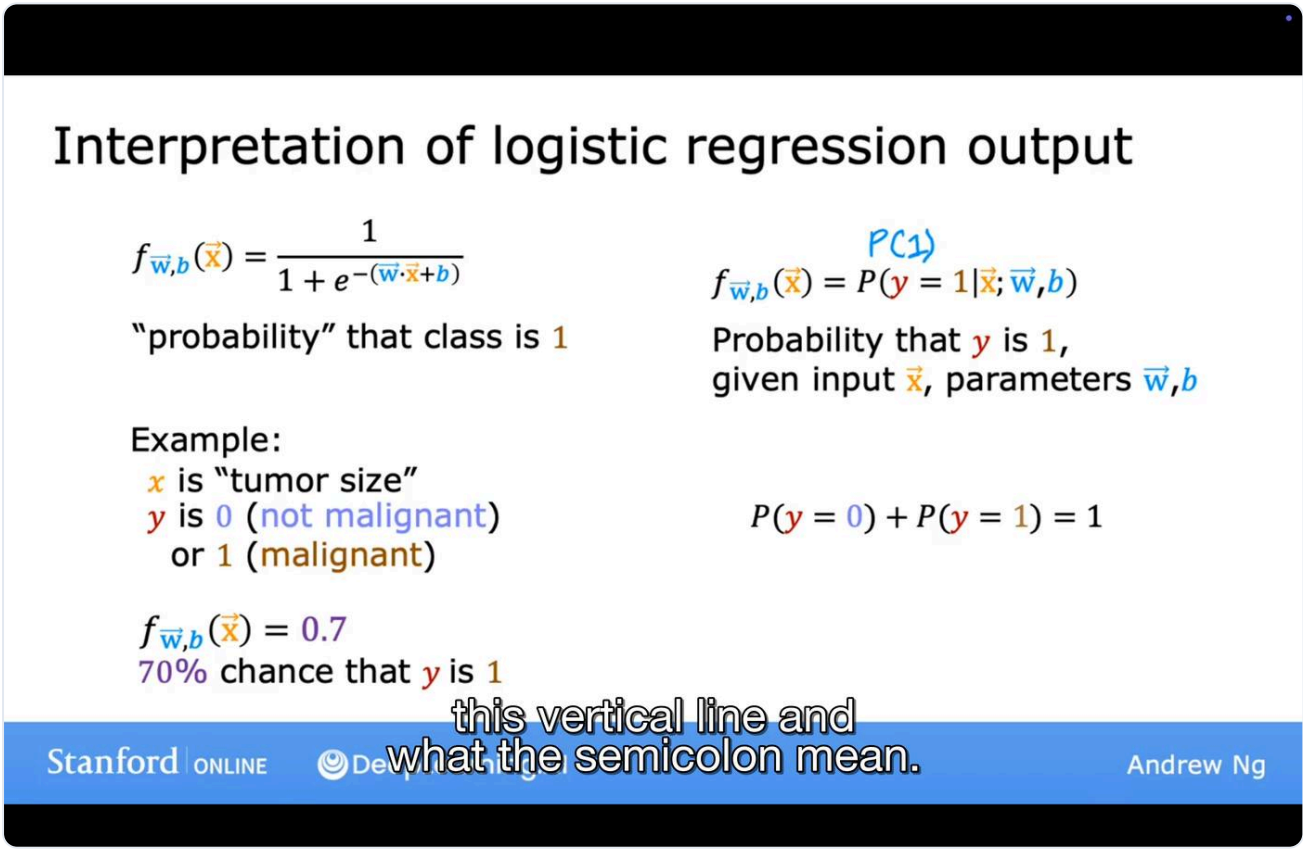


FIGURE 32.10 | Interpretation of Logistic Regression Output

What the semicolon here is used to denote is just that w and b are parameters that affect this computation of what is the probability of y being equal to 1 given the input feature x? For the purpose of this class, don't worry too much about what this vertical line and what the semicolon mean. You don't need to remember or follow any of this mathematical notation for this class. I'm mentioning this only because you may see this in other places. In the optional lab that follows this video, you also get to see how the Sigmoid function is implemented in code. You can see a plot that uses the Sigmoid function so as to do better on the classification tasks that you saw in the previous optional lab.

Remember that the code will be provided to you, so you just have to run it. I hope you take a look and get familiar with the code. Congrats on getting here. You now know what is the logistic regression model as well as the mathematical formula that defines logistic regression. For a long time, a lot of Internet advertising was actually driven by basically a slight variation of logistic regression. This was very lucrative for some large companies, and this is basically the algorithm that decided what ad was shown to you and many others on some large websites. Now, there's, even more, to learn about this algorithm. In the next video, we'll take a look at the details of logistic regression.

We'll look at some visualizations and also examines something called the decision boundary. This will give you a few different ways to map the numbers that this model outputs, such as 0.3, or 0.7, or 0.65 to a prediction of whether y is actually 0 or 1. Let's go on to the next video to learn more about logistic regression.

LECTURE 33

Decision Boundary

In the last video, you learned about the logistic regression model. Now, let's take a look at the decision boundary to get a better sense of how logistic regression is computing these predictions. To recap, here's how the logistic regression models outputs are computed in two steps. In the first step, you compute z as w.x plus b. Then you apply the Sigmoid function g to this value z. Here again, is the formula for the Sigmoid function. Another way to write this is we can say f of x is equal to g, the Sigmoid function, also called the logistic function, applied to w.x plus b, where this is of course, the value of z.

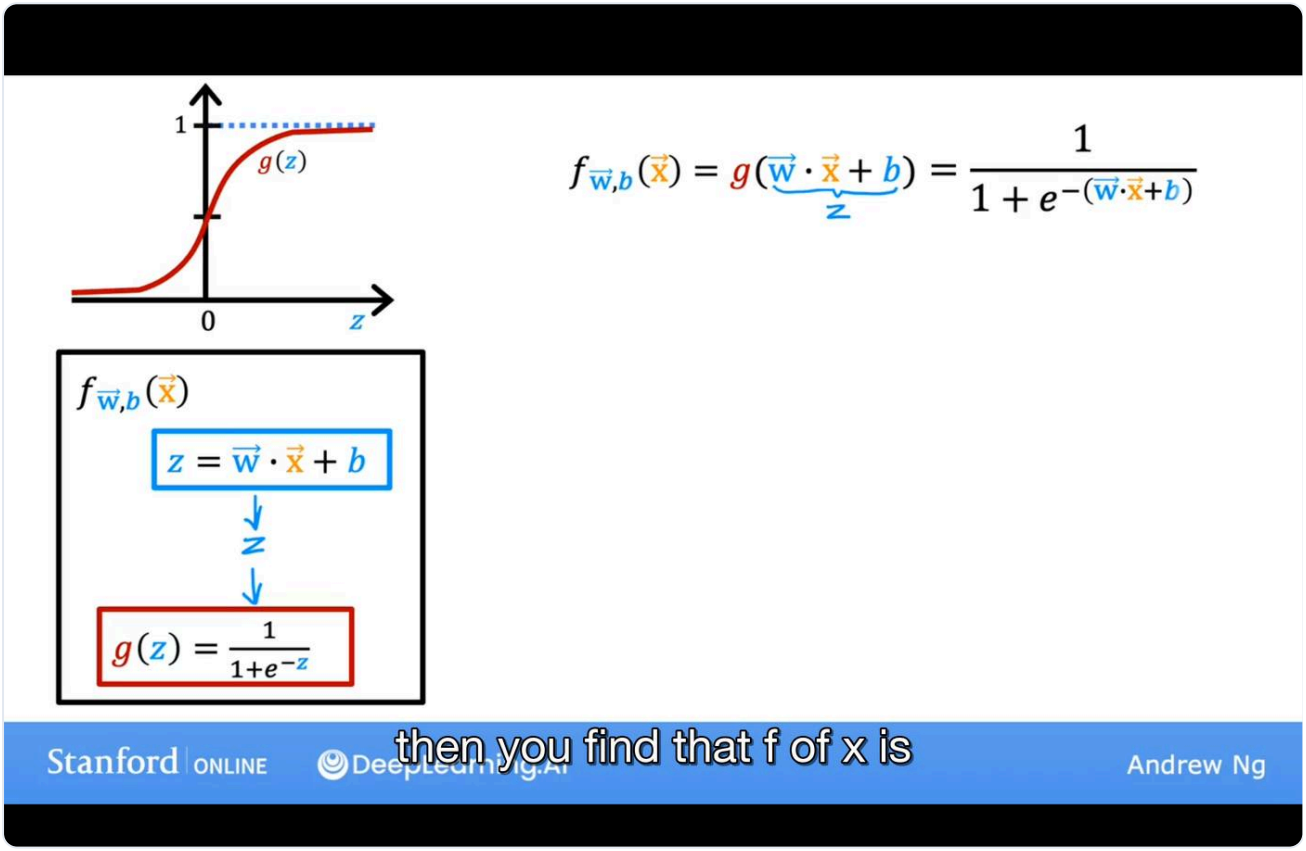


FIGURE 33.1

If you take the definition of the Sigmoid function and plug in the definition of z, then you find that f of x is equal to this formula over here, 1 over 1 plus e to the negative z, where z is wx plus b. You may remember we said in the previous video that we interpret this as the probability that y is equal to 1 given x and with parameters w and b.

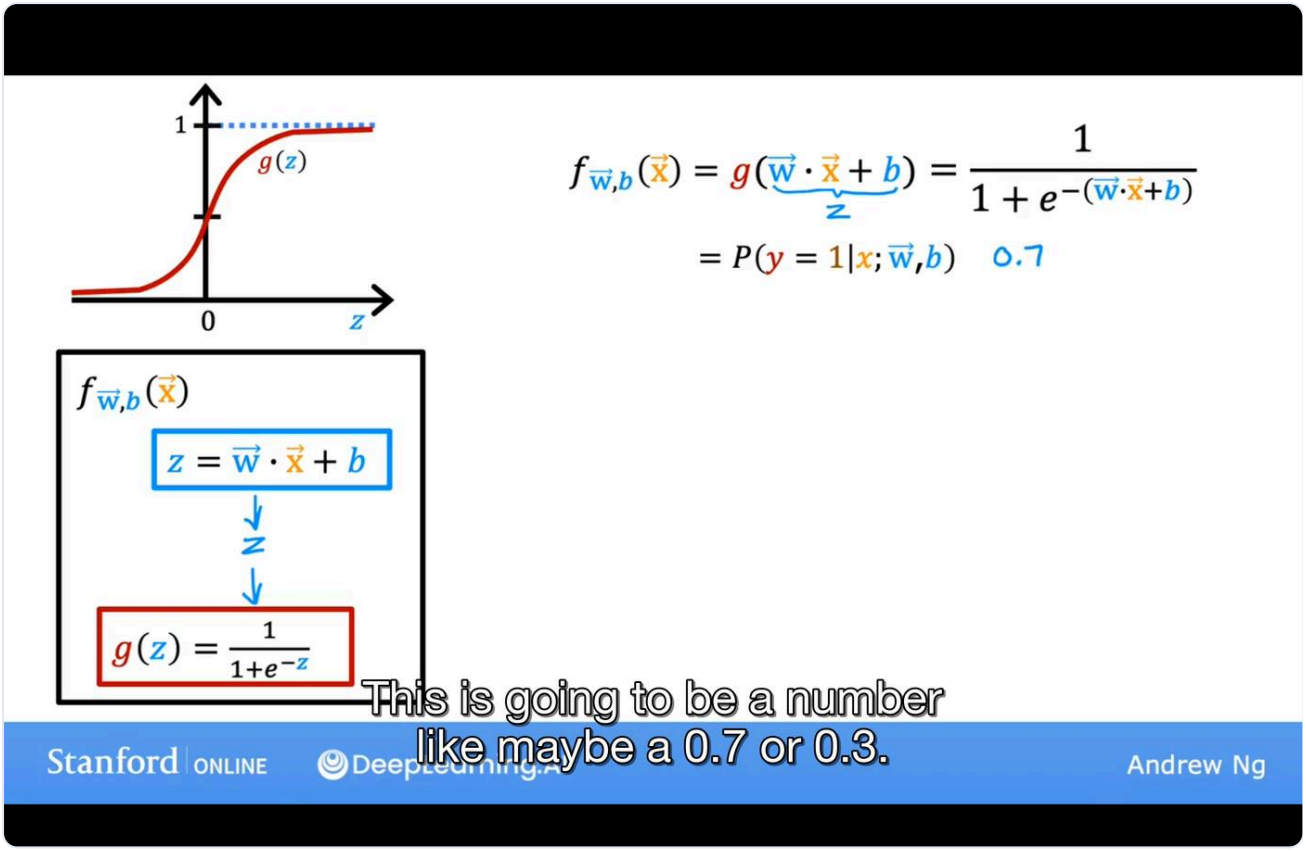


FIGURE 33.2

This is going to be a number like maybe a 0.7 or 0.3. Now, what if you want to learn the algorithm to predict. Is the value of y going to be zero or one?

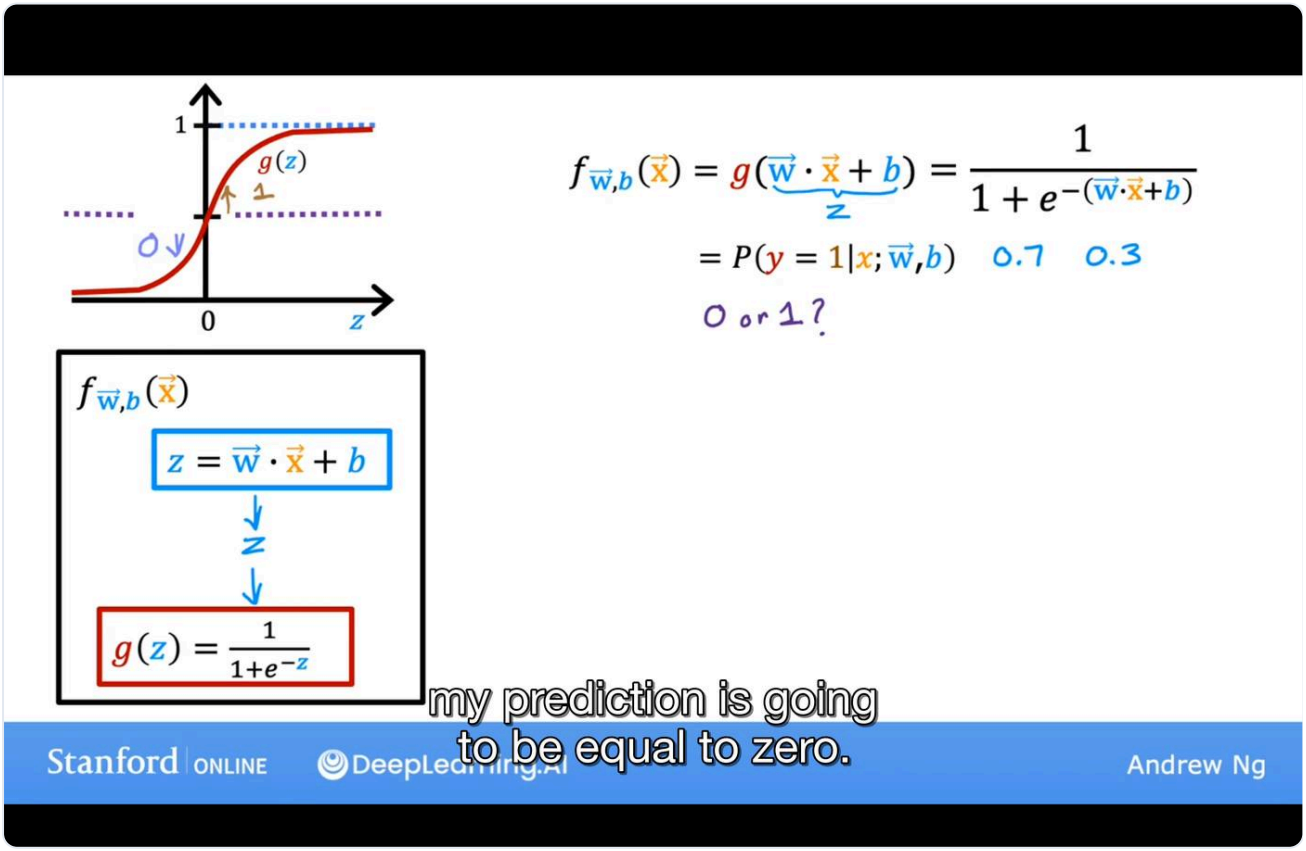


FIGURE 33.3

Well, one thing you might do is set a threshold above which you predict y is one, or you set y hat to prediction to be equal to one and below which you might say y hat, my prediction is going to be equal to zero. A common choice would be to pick a threshold of 0.5 so that if f of x is greater than or equal to 0.5, then predict y is one. We write that prediction as y hat equals 1, or if f of x is less than 0.5, then predict y is 0, or in other words, the prediction y hat is equal to 0.

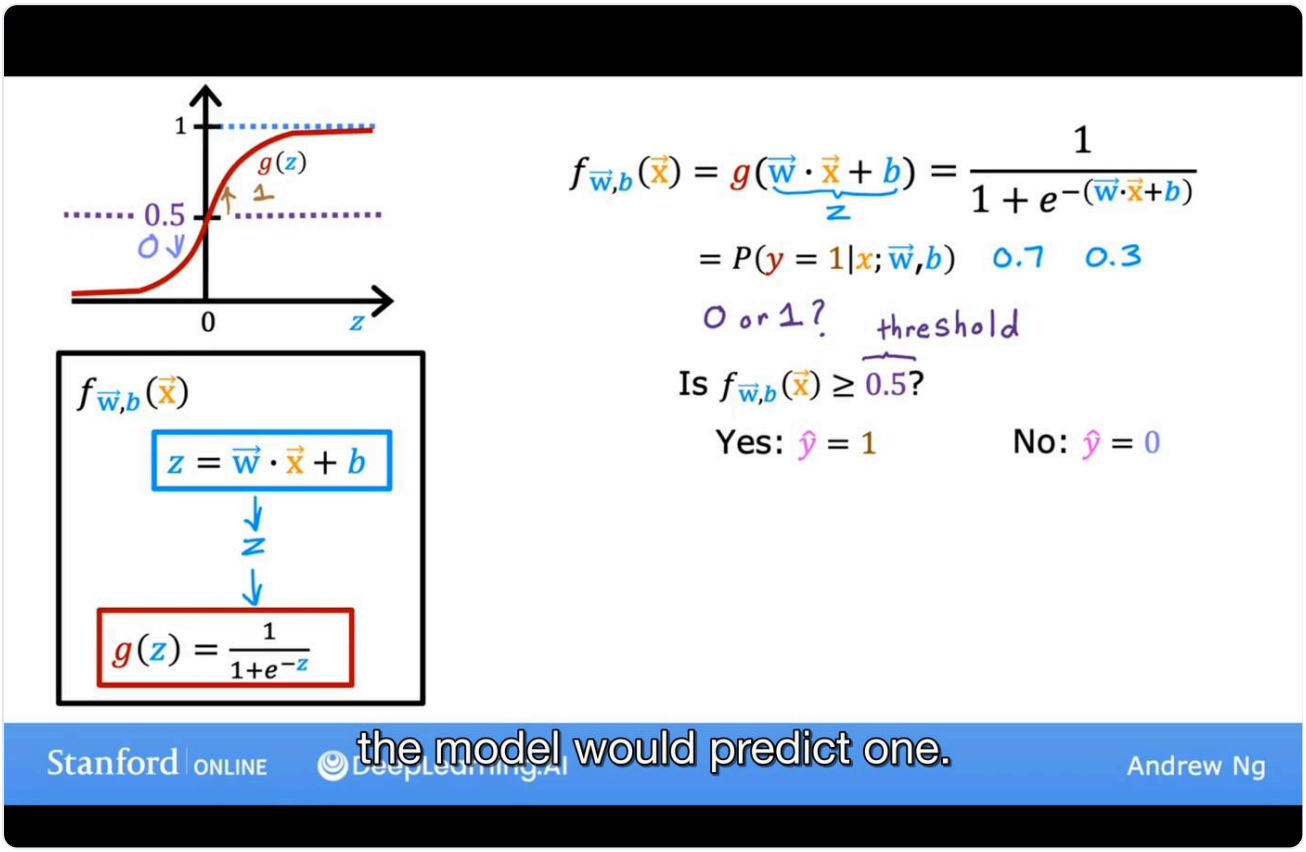


FIGURE 33.4

Now, let's dive deeper into when the model would predict one. In other words, when is f of x greater than or equal to 0.5. We'll recall that f of x is just equal to g of z.

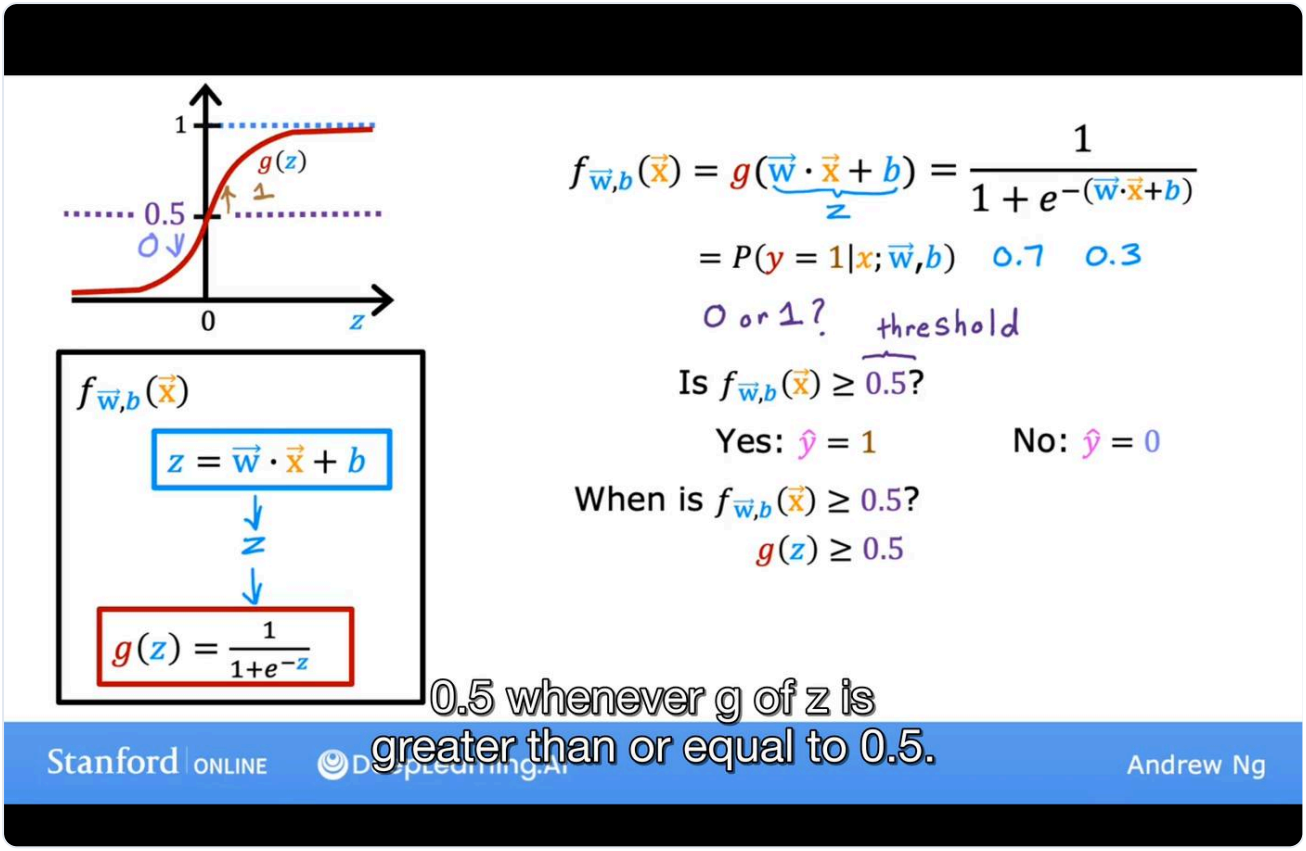


FIGURE 33.5

So f is greater than or equal to 0.5 whenever g of z is greater than or equal to 0.5. But when is g of z greater than or equal to 0.5? Well, here's a Sigmoid function over here. So g of z is greater than or equal to 0.5 whenever z is greater than or equal to 0. That is whenever z is on the right half of this axis. Finally, when is z greater than or equal to zero?

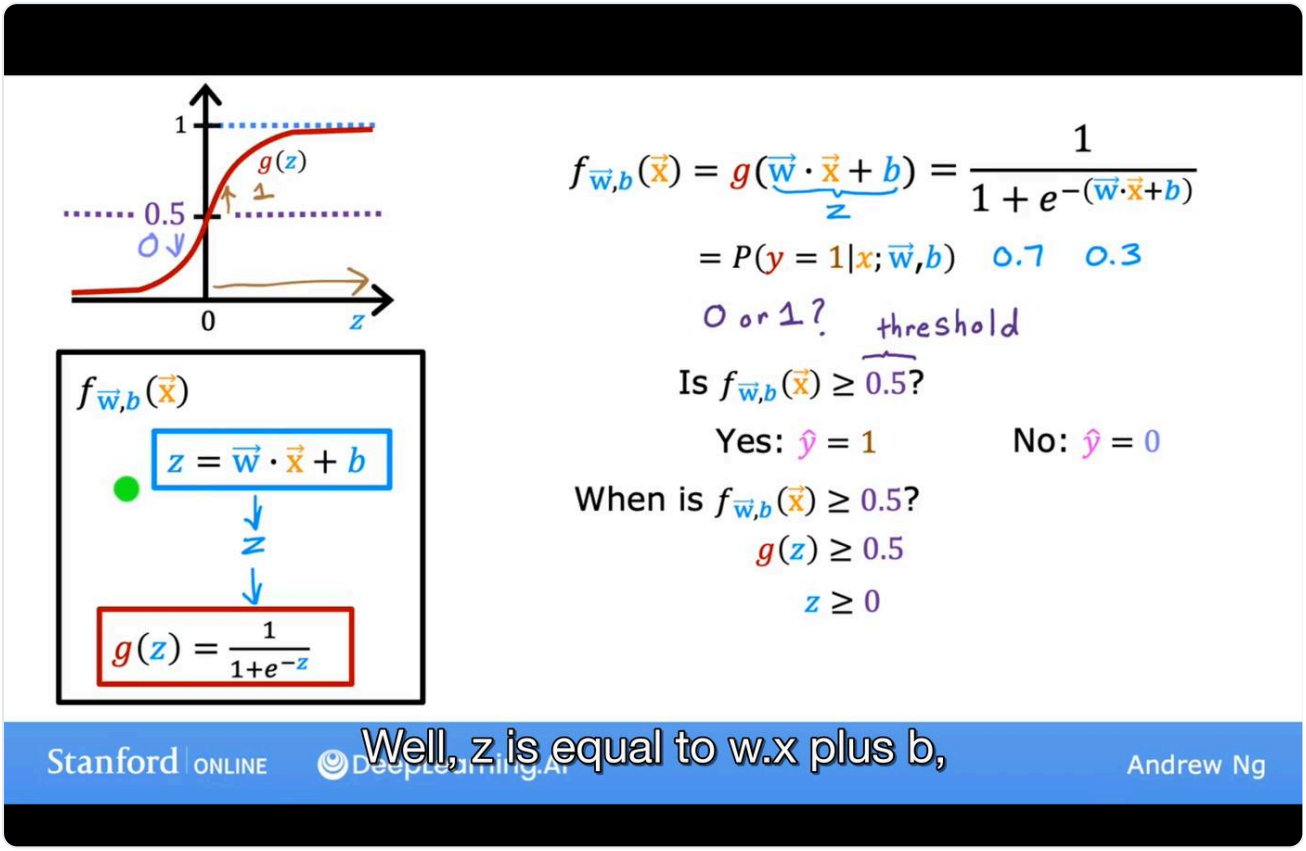


FIGURE 33.6

Well, z is equal to w.x plus b, so z is greater than or equal to zero whenever w.x plus b is greater than or equal to zero.

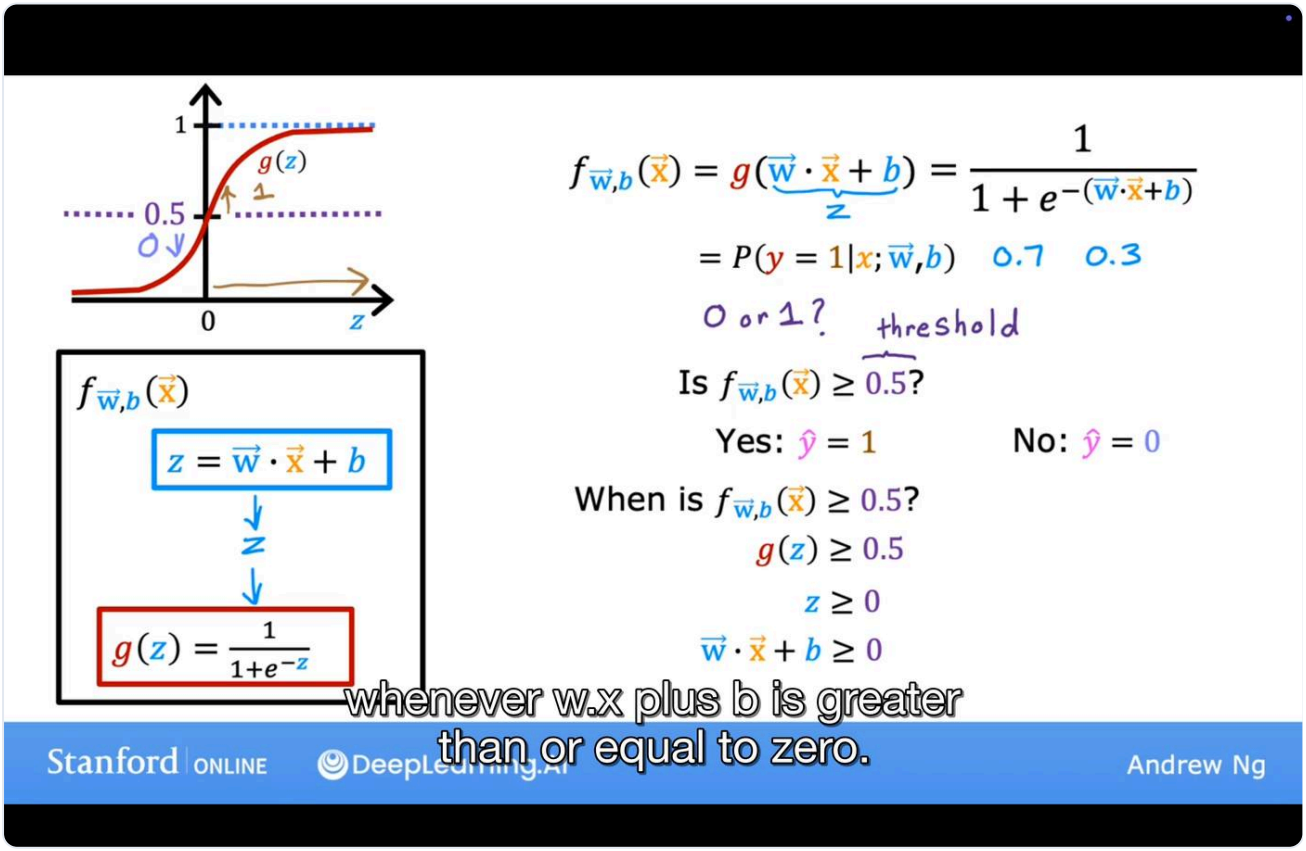


FIGURE 33.7

To recap, what you've seen here is that the model predicts 1 whenever w.x plus b is greater than or equal to 0.

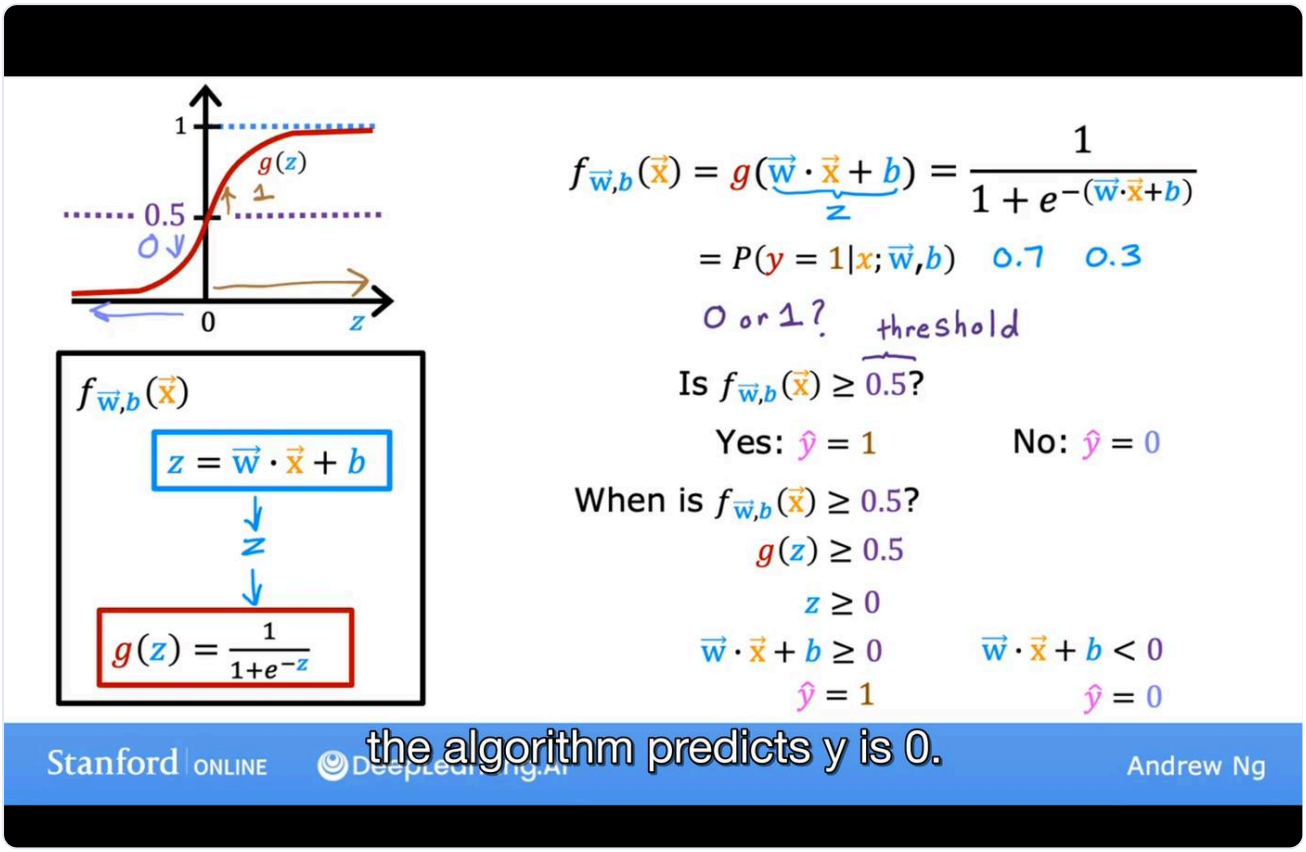


FIGURE 33.8

Conversely, when w.x plus b is less than zero, the algorithm predicts y is 0. Given this, let's now visualize how the model makes predictions. I'm going to take an example of a classification problem where you have two features, x1 and x2 instead of just one feature.

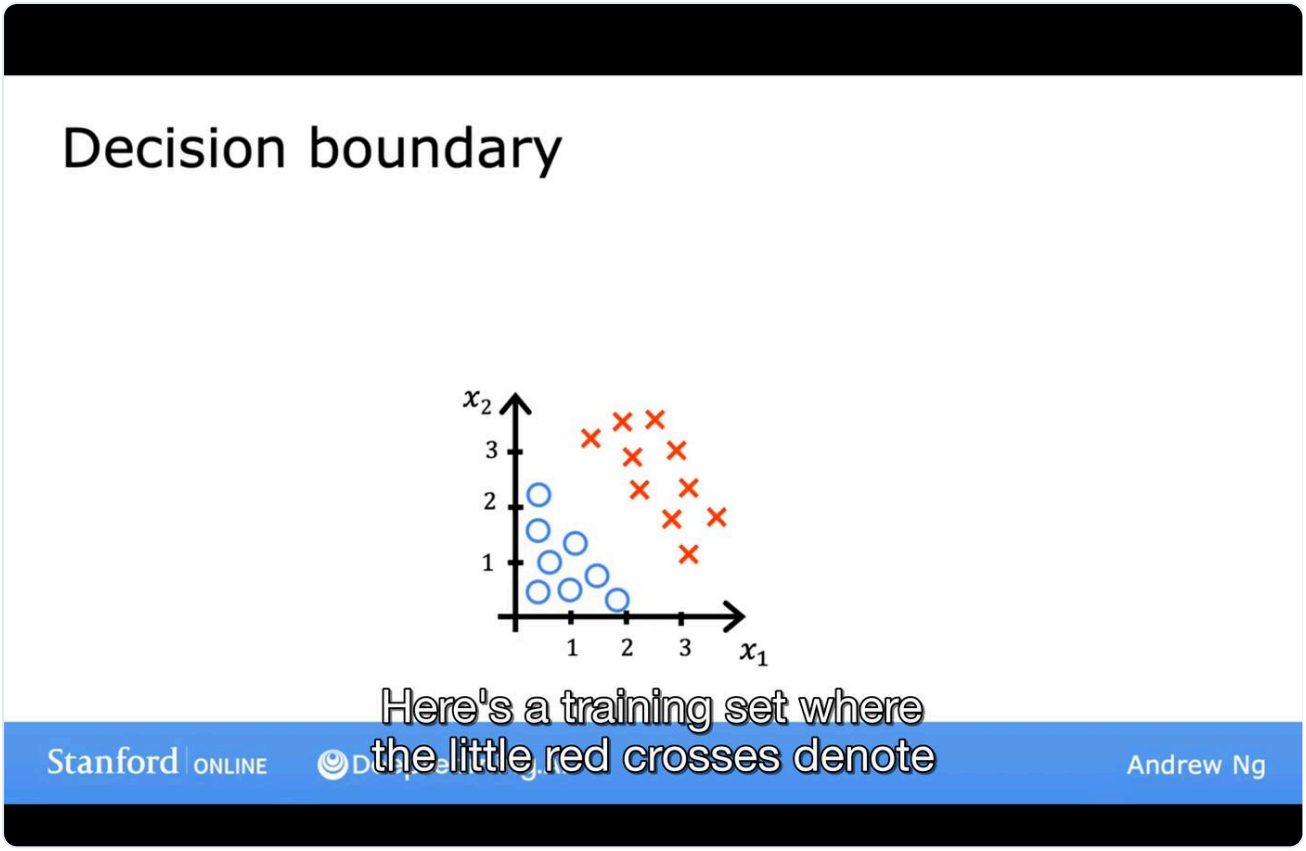


FIGURE 33.9 | Decision Boundary

Here's a training set where the little red crosses denote the positive examples and the little blue circles denote negative examples. The red crosses corresponds to y equals 1, and the blue circles correspond to y equals 0. The logistic regression model will make predictions using this function f of x equals g of z, where z is now this expression over here, w1x1 plus w2x2 plus b, because we have two features x1 and x2. Let's just say for this example that the value of the parameters are w1 equals 1, w2 equals 1, and b equals negative 3. Let's now take a look at how logistic regression makes predictions. In particular, let's figure out when wx plus b is greater than equal to 0 and when wx plus b is less than 0.

To figure that out, there's a very interesting line to look at, which is when wx plus b is exactly equal to 0.

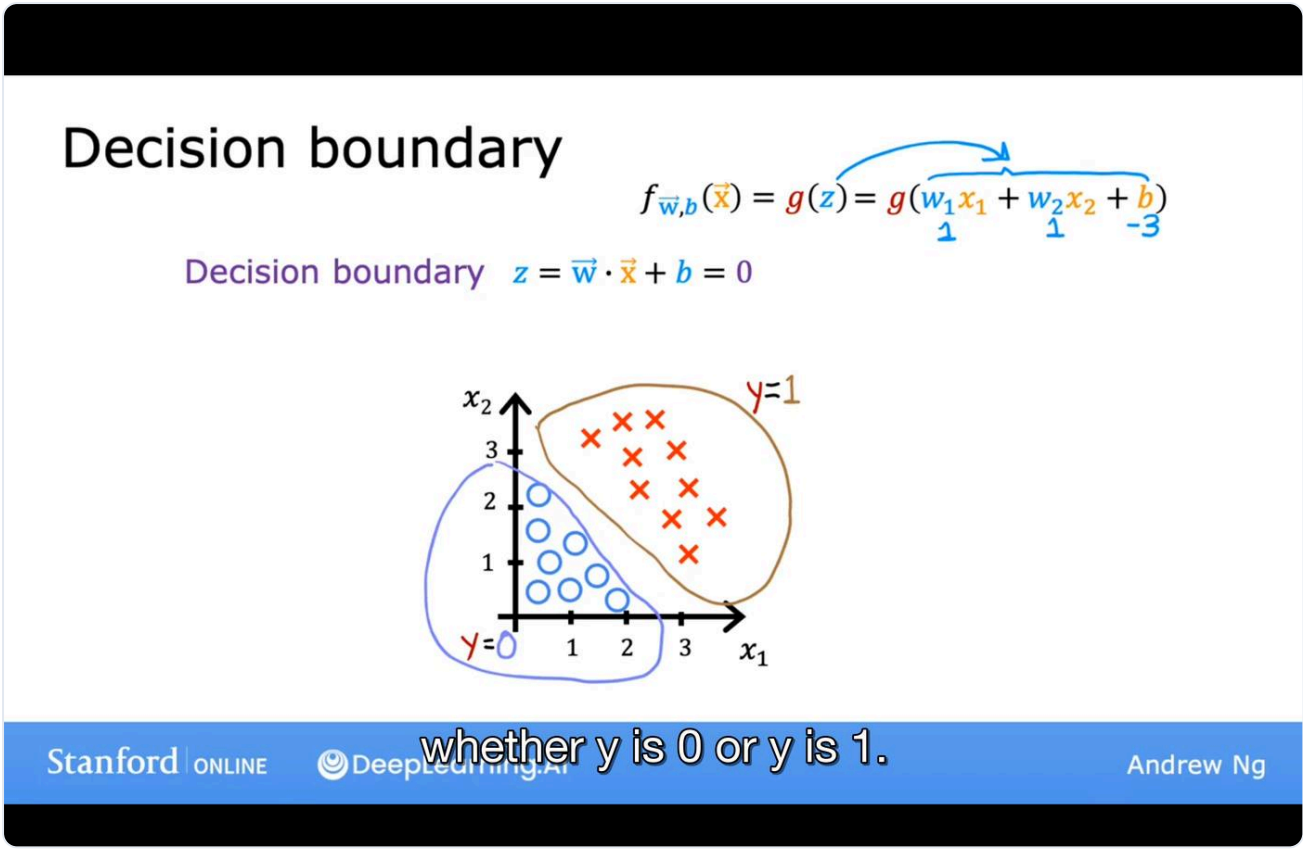


FIGURE 33.10 | Decision Boundary

It turns out that this line is also called the decision boundary because that's the line where you're just almost neutral about whether y is 0 or y is 1. Now, for the values of the parameters w_1, w_2, and b that we had written down above, this decision boundary is just x_1 plus x_2 minus 3. When is x_1 plus x_2 minus 3 equal to 0?

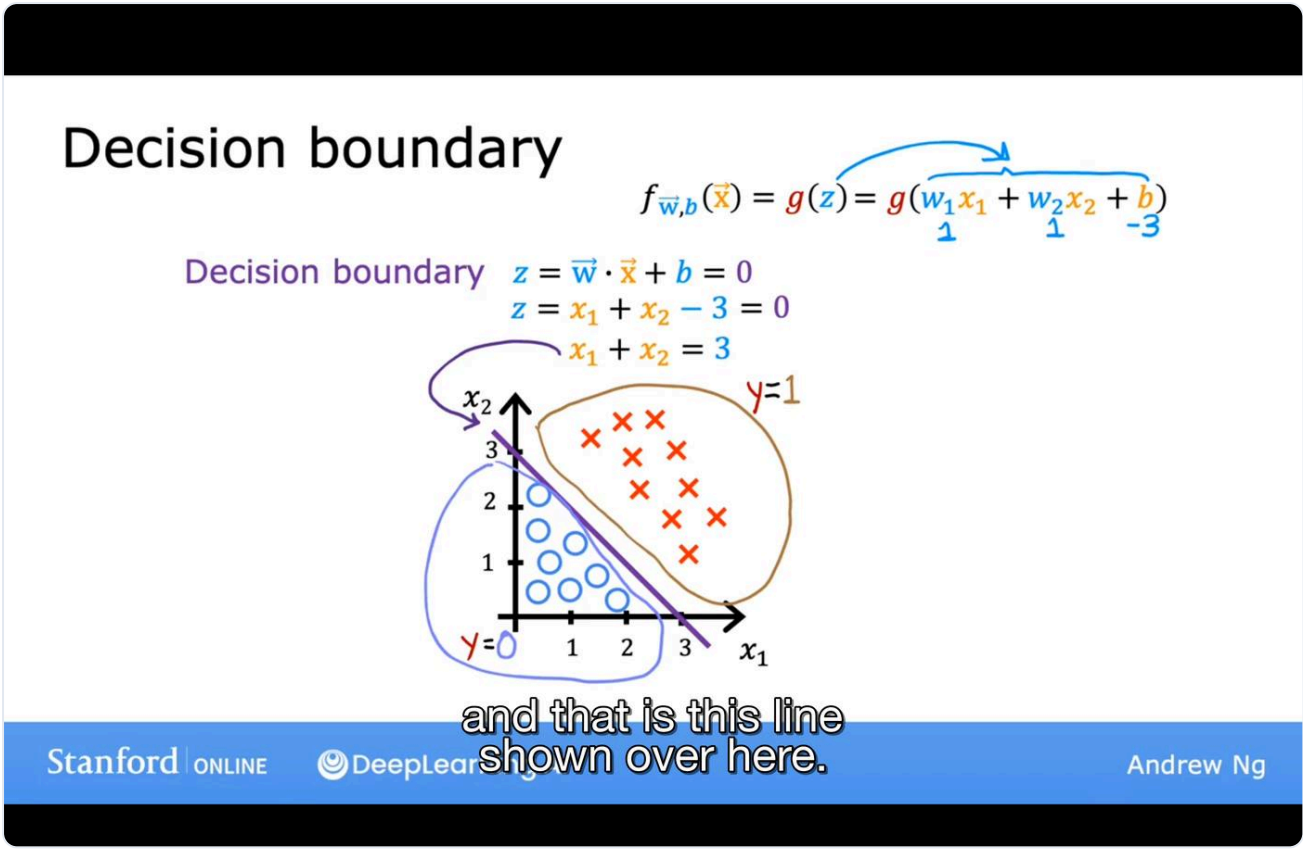


FIGURE 33.11 | Decision Boundary

Well, that will correspond to the line x_1 plus x_2 equals 3, and that is this line shown over here. This line turns out to be the decision boundary, where if the features x are to the right of this line, logistic regression would predict 1 and to the left of this line, logistic regression with predicts 0.

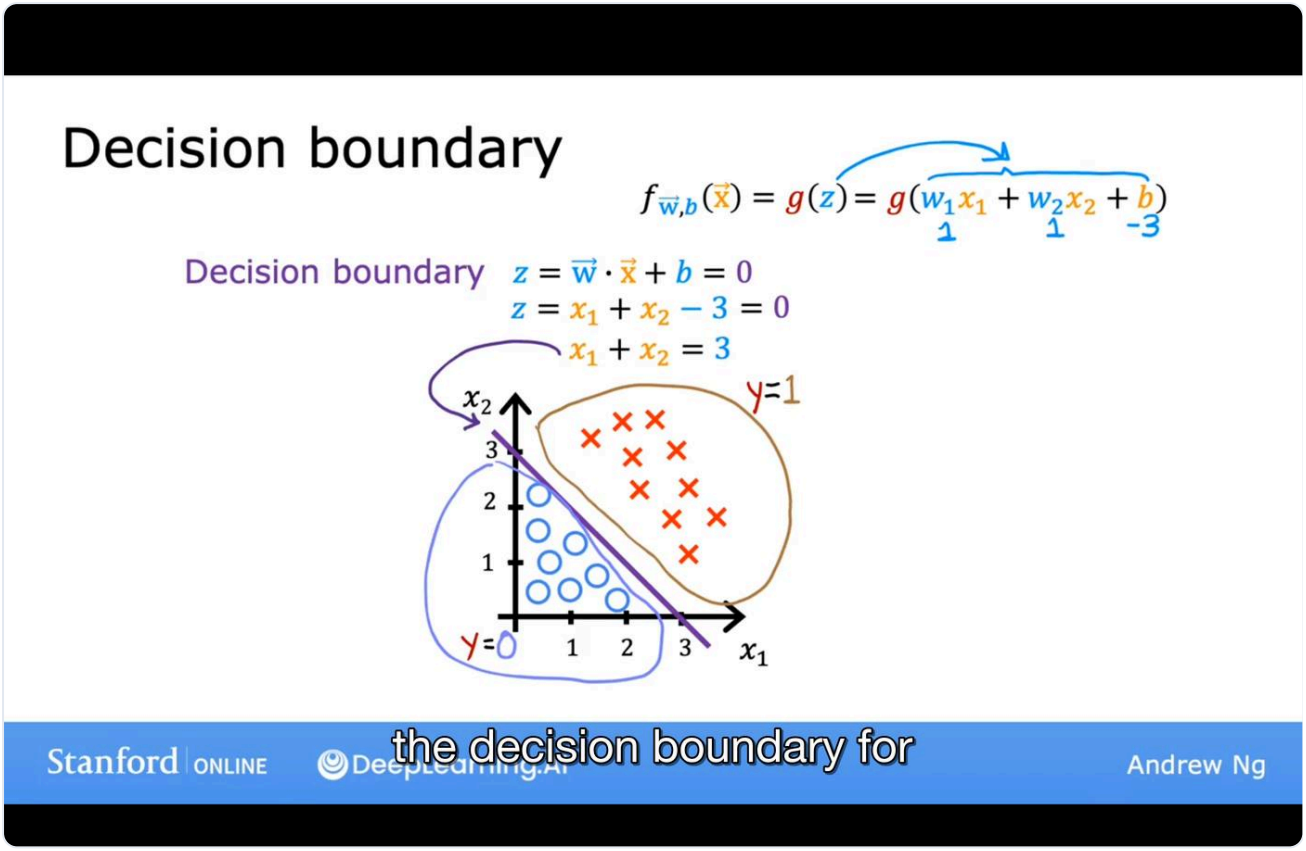


FIGURE 33.12

In other words, what we have just visualize is the decision boundary for logistic regression when the parameters w_1, w_2, and b are 1,1 and negative 3. Of course, if you had a different choice of the parameters, the decision boundary would be a different line. Now let's look at a more complex example where the decision boundary is no longer a straight line.

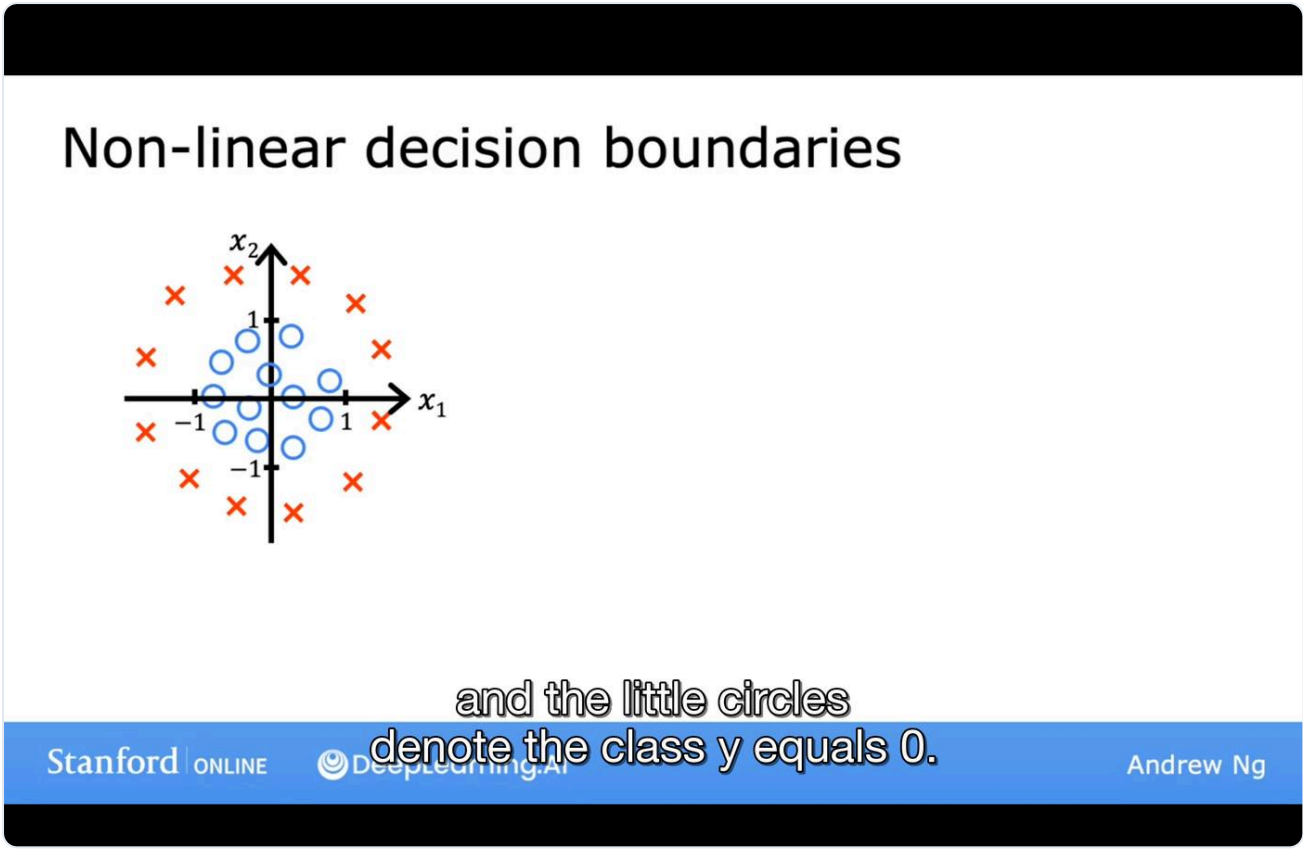


FIGURE 33.13

As before, crosses denote the class y equals 1, and the little circles denote the class y equals 0. Earlier last week, you saw how to use polynomials in linear regression, and you can do the same in logistic regression.

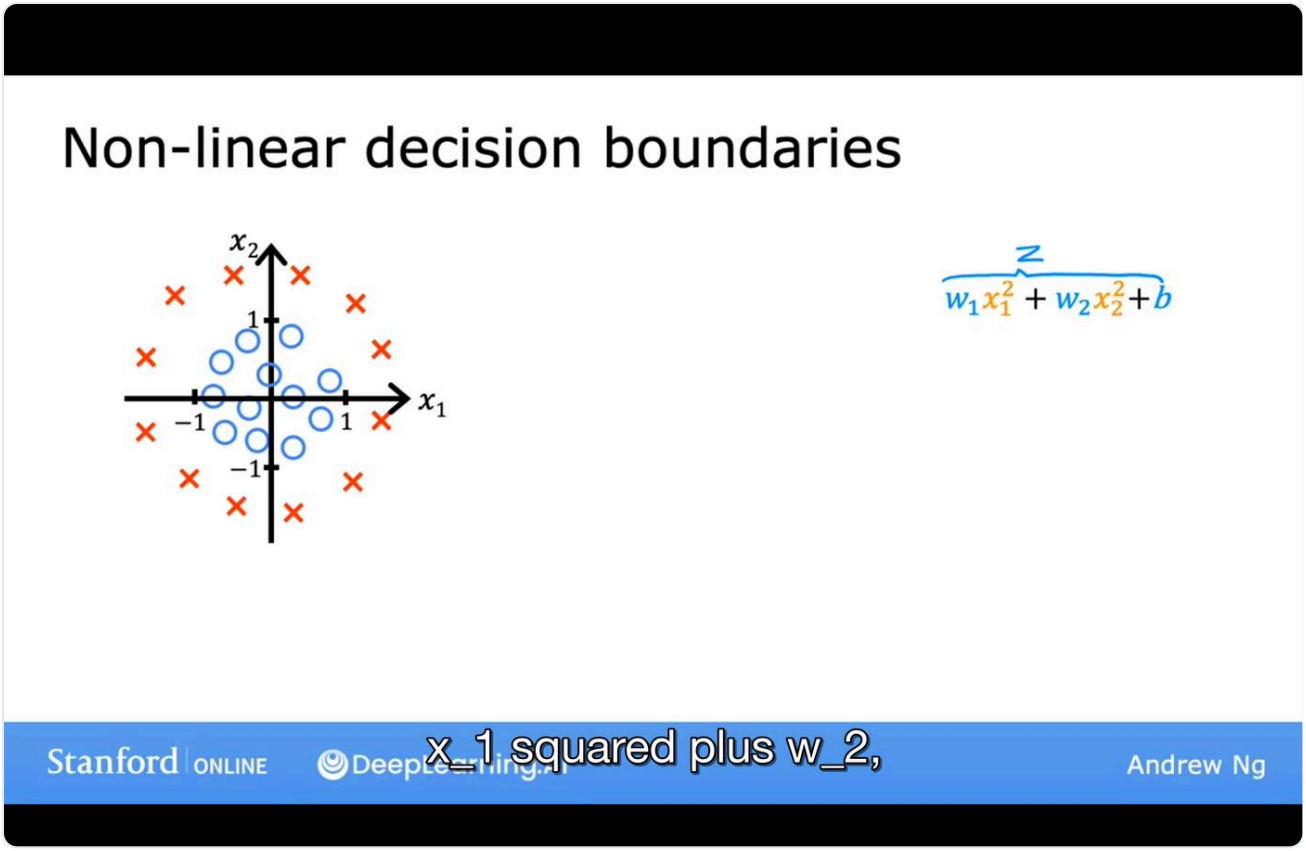


FIGURE 33.14

This set z to be w_1, x_1 squared plus w_2, x_2 squared plus b. With this choice of features, polynomial features into a logistic regression.

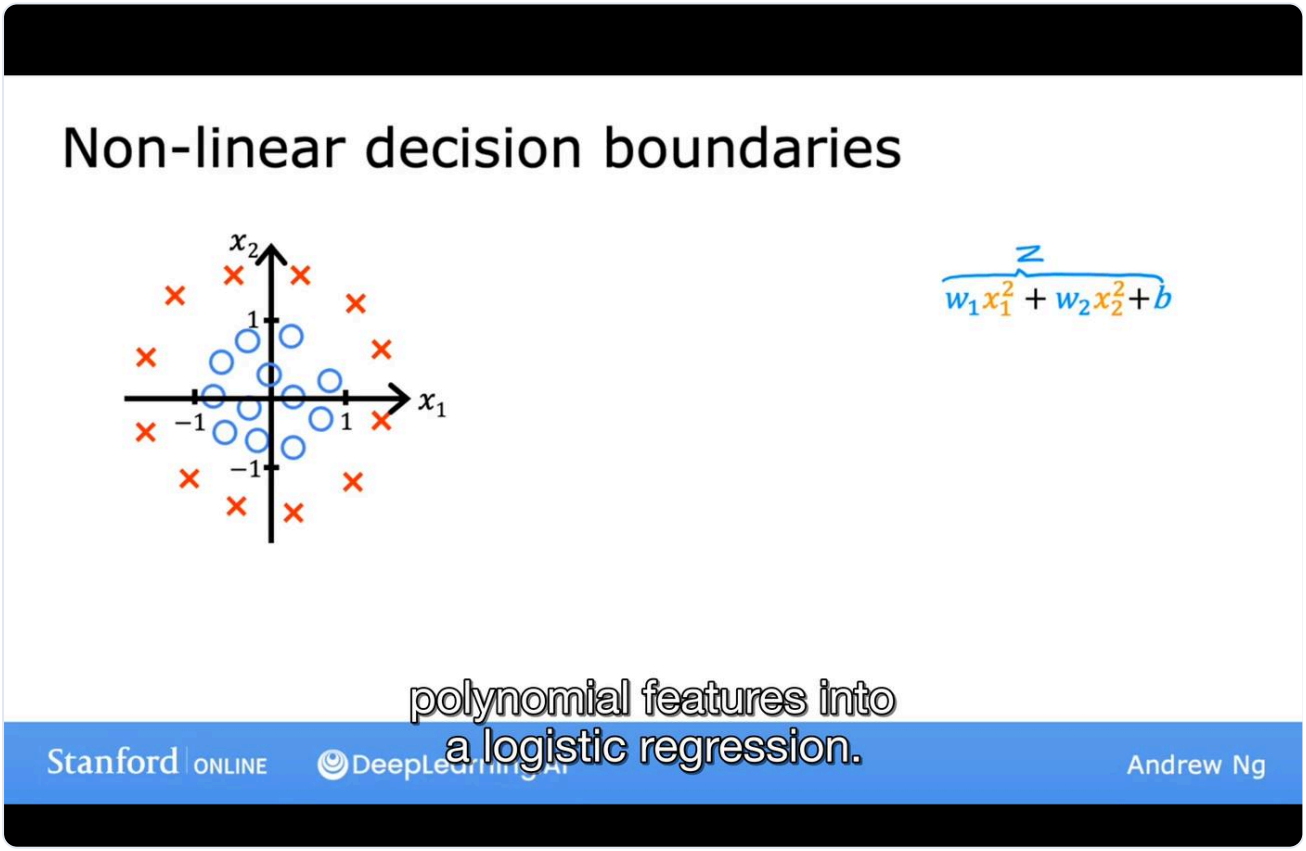


FIGURE 33.15

F of x, which equals g of z, is now g of this expression over here.

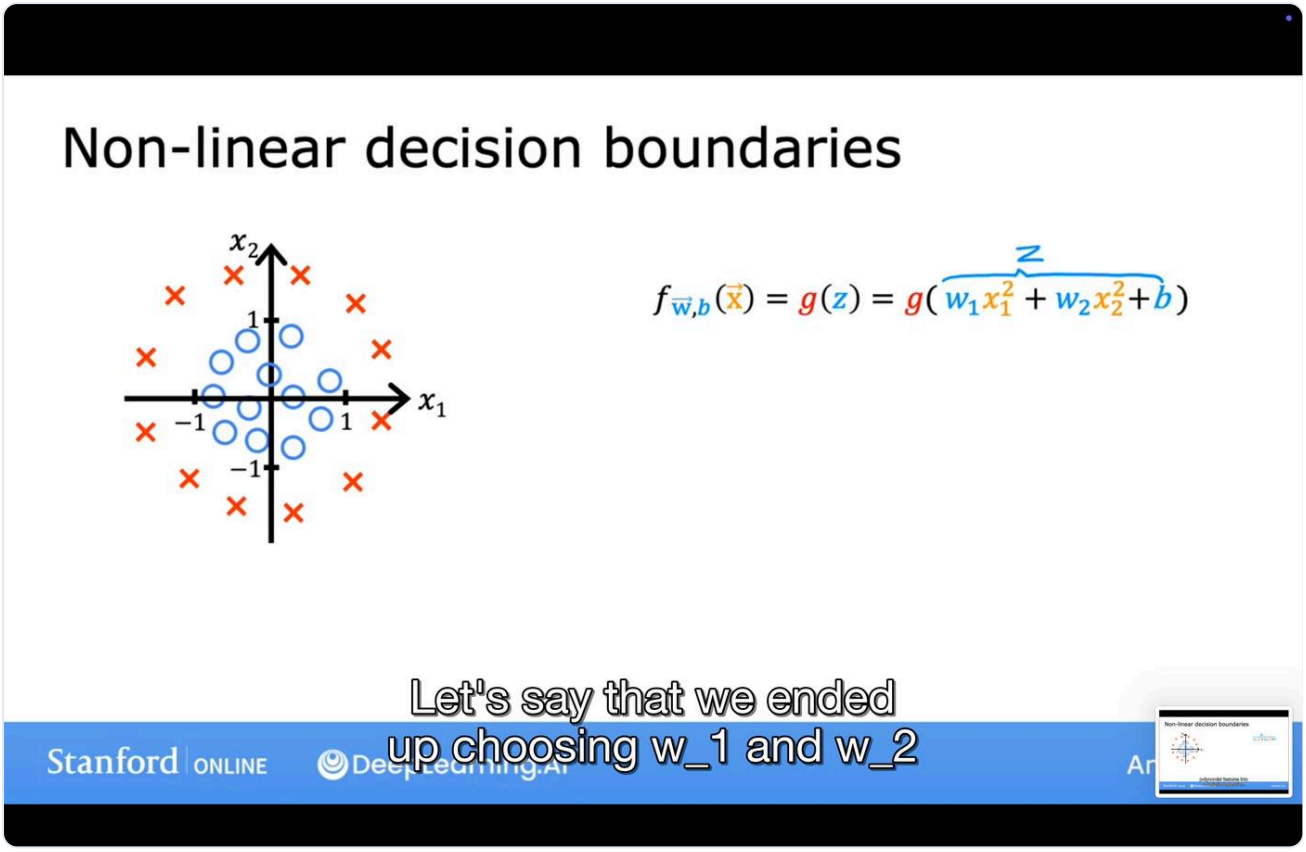


FIGURE 33.16

Let's say that we ended up choosing w_1 and w_2 to be 1 and b to be negative 1.

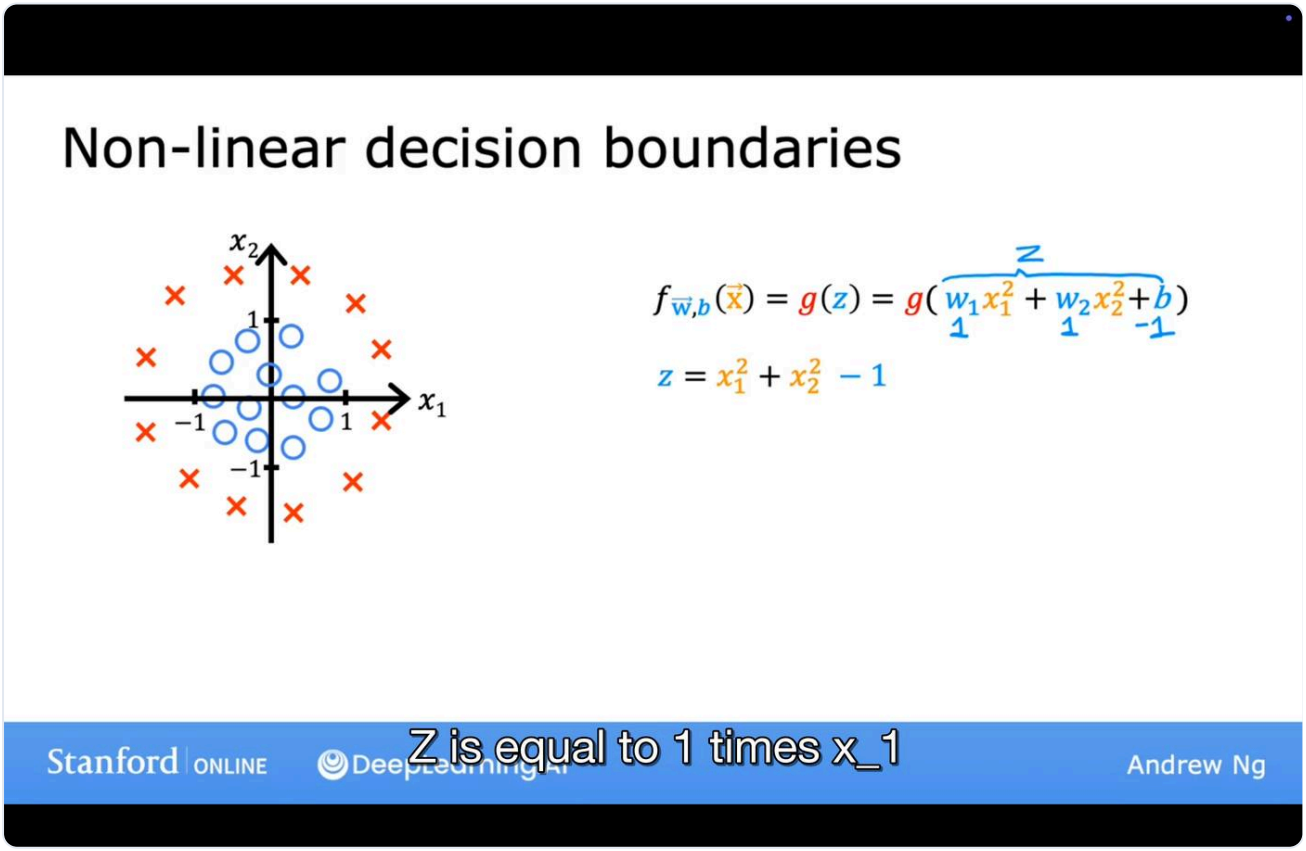


FIGURE 33.17

Z is equal to 1 times x_1 squared plus 1 times x_2 squared minus 1. The decision boundary, as before, will correspond to when z is equal to 0.

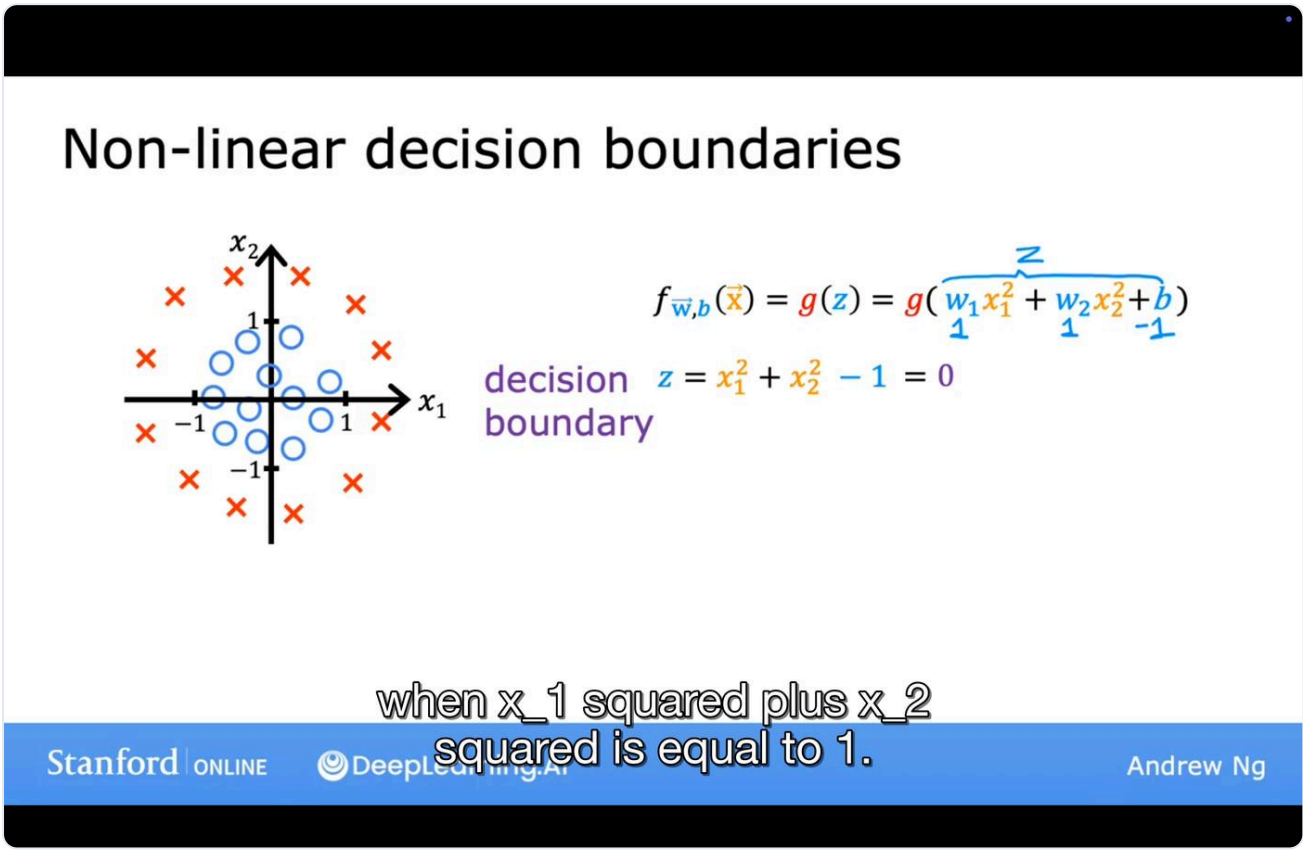


FIGURE 33.18

This expression will be equal to 0 when x_1 squared plus x_2 squared is equal to 1.

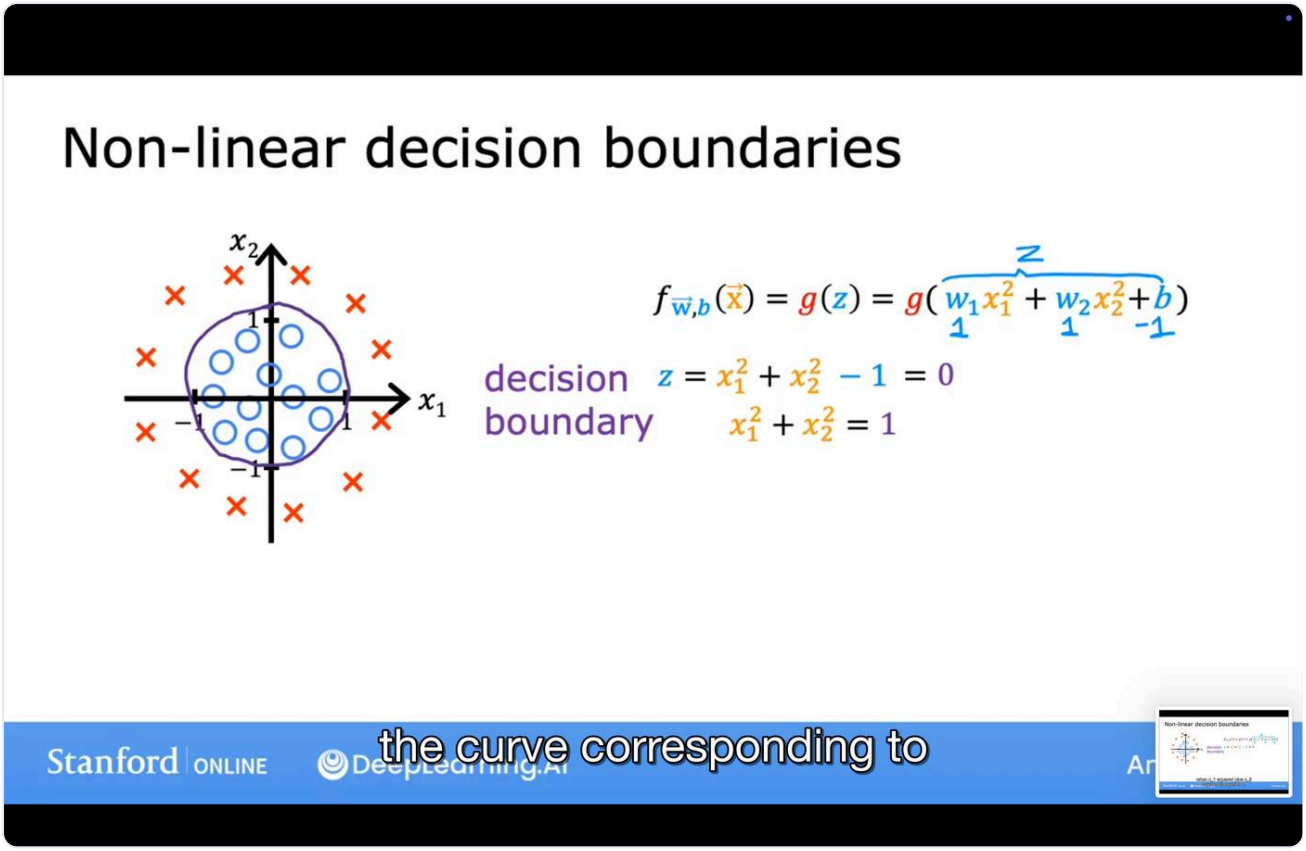


FIGURE 33.19

If you plot on the diagram on the left, the curve corresponding to x_1 squared plus x_2 squared equals 1, this turns out to be the circle. When x_1 squared plus x_2 squared is greater than or equal to 1, that's this area outside the circle and that's when you predict y to be 1.

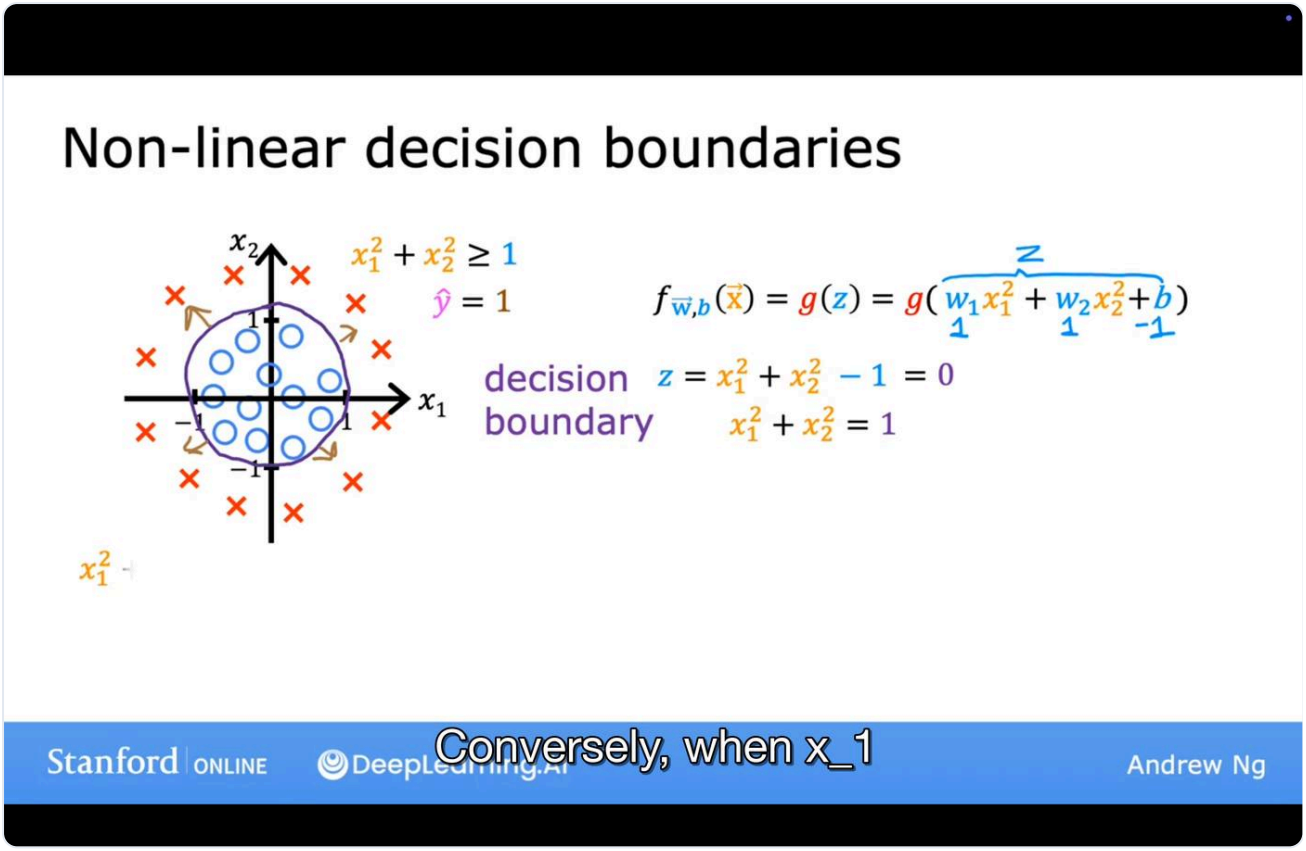


FIGURE 33.20

Conversely, when x_1 squared plus x_2 squared is less than 1, that's this area inside the circle and that's when you predict y to be 0.

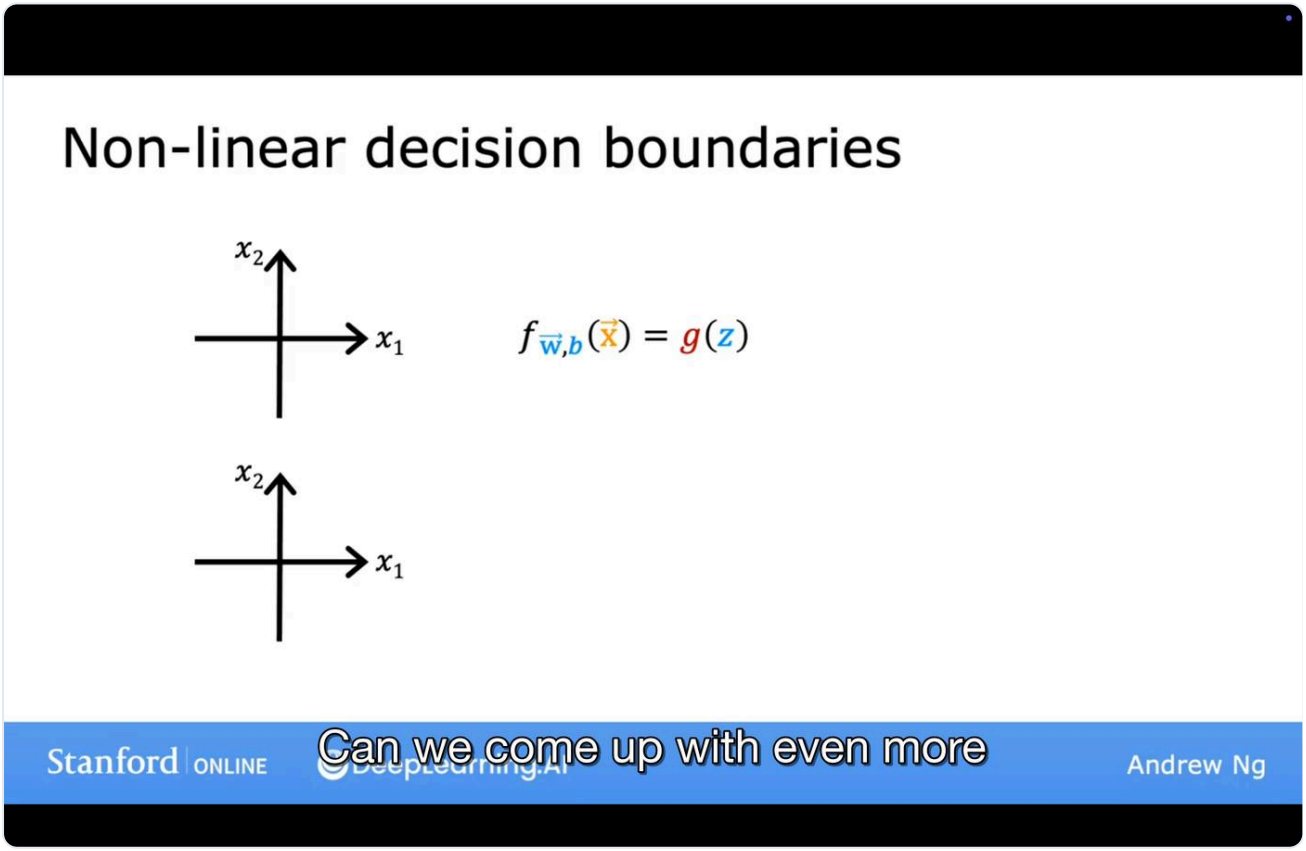


FIGURE 33.21

Can we come up with even more complex decision boundaries than these? Yes, you can. You can do so by having even higher-order polynomial terms. Say z is w_1, x_1 plus w_2, x_2 plus w_3, x_1 squared plus w_4, x_1, x_2 plus w_5, x_2 squared.

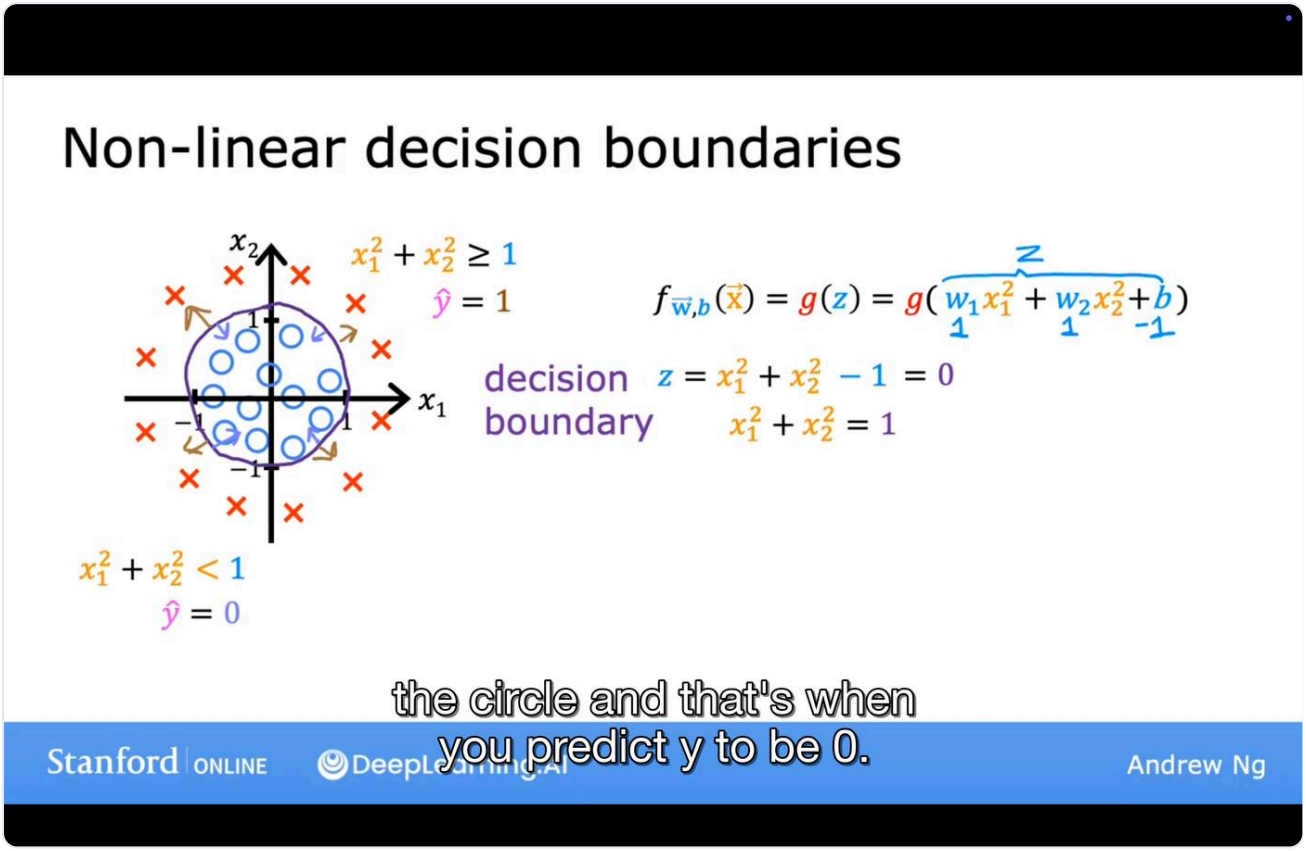


FIGURE 33.22

Then it's possible you can get even more complex decision boundaries. The model can define decision boundaries, such as this example, an ellipse just like this, or with a different choice of the parameters. You can even get more complex decision boundaries, which can look like functions that maybe looks like that.

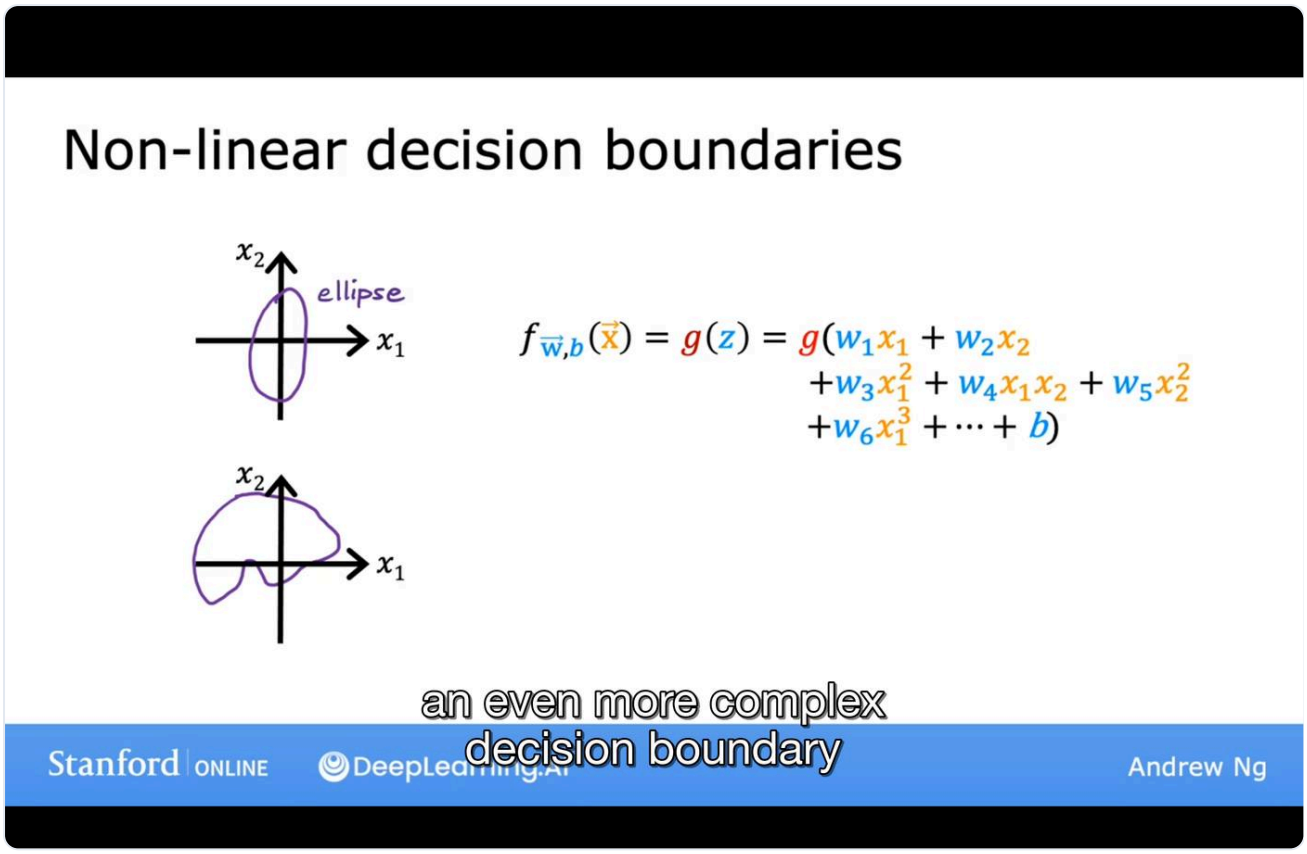


FIGURE 33.23

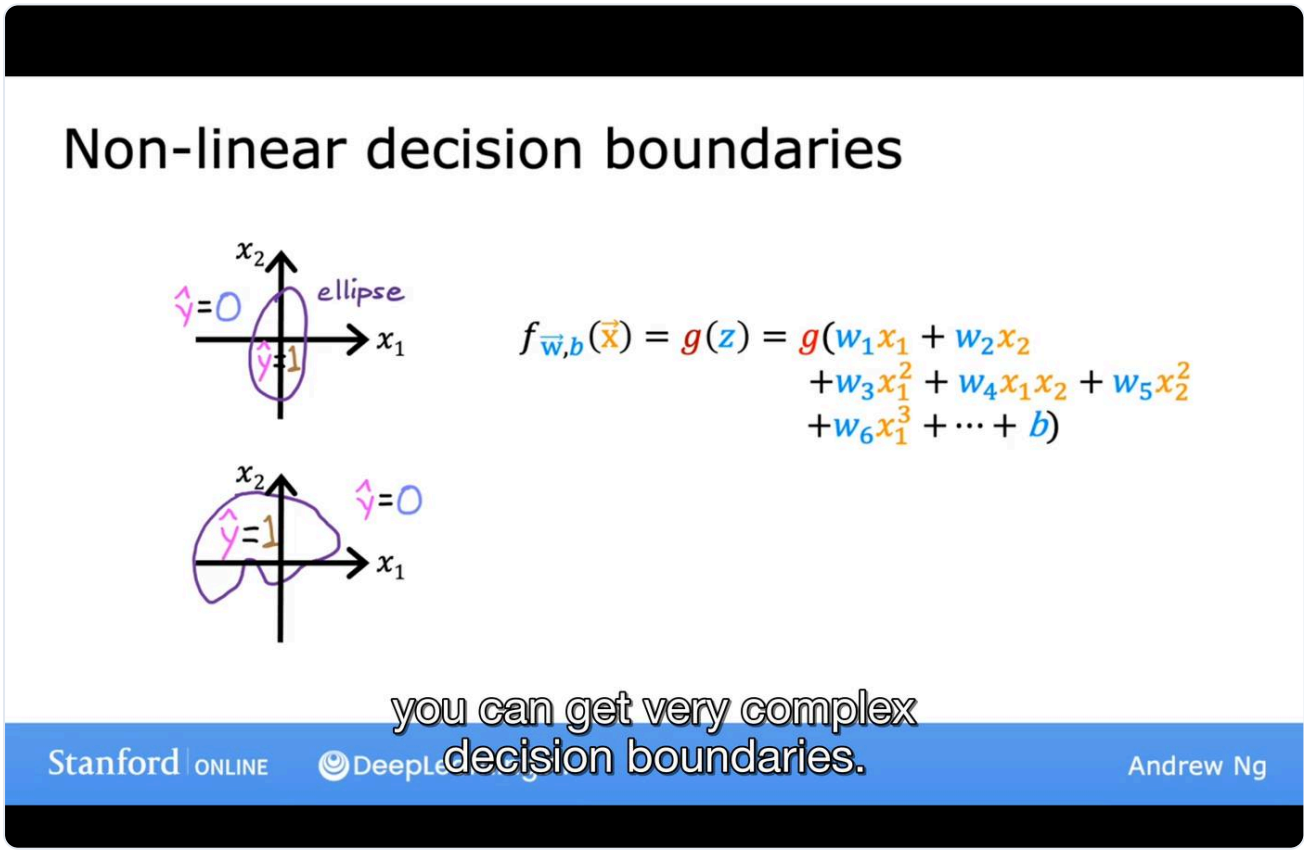


FIGURE 33.24

So this is an example of an even more complex decision boundary than the ones we've seen previously. This implementation of logistic regression will predict y equals 1 inside this shape and outside the shape will predict y equals 0. With these polynomial features, you can get very complex decision boundaries. In other words, logistic regression can learn to fit pretty complex data. Although if you were to not include any of these higher-order polynomials, so if the only features you use are x_1, x_2, x_3, and so on, then the decision boundary for logistic regression will always be linear, will always be a straight line. In the upcoming optional lab, you also get to see the code implementation of the decision boundary.

In the example in the lab, there will be two features so you can see that decision boundary as a line. With this visualization, I hope that you now have a sense of the range of possible models you can get with logistic regression. Now that you've seen what f of x can potentially compute, let's take a look at how you can actually train a logistic regression model. We'll start by looking at the cost function for logistic regression and after that, figured out how to apply gradient descent to it. Let's go on to the next video.

LECTURE 34

Squared Error Cost

Remember that the cost function gives you a way to measure how well a specific set of parameters fits the training data. Thereby gives you a way to try to choose better parameters. In this video, we'll look at how the squared error cost function is not an ideal cost function for logistic regression. We'll take a look at a different cost function that can help us choose better parameters for logistic regression. Here's what the training set for our logistic regression model might look like.

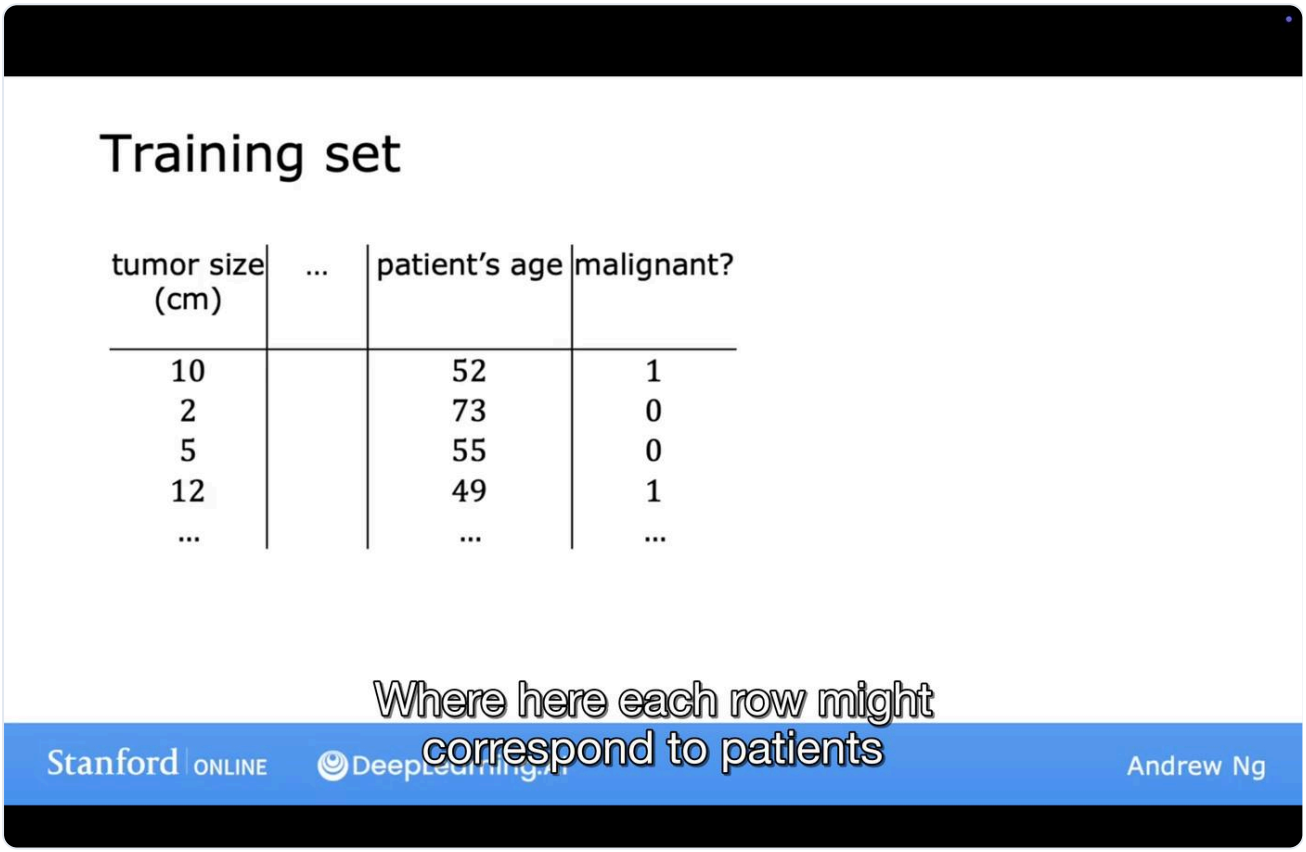


FIGURE 34.1 | Training Set

Where here each row might correspond to patients that was paying a visit to the doctor and one dealt with some diagnosis. As before, we'll use m to denote the number of training examples. Each training example has one or more features, such as the tumor size, the patient's age, and so on for a total of n features. Let's call the features X_1 through X_n.

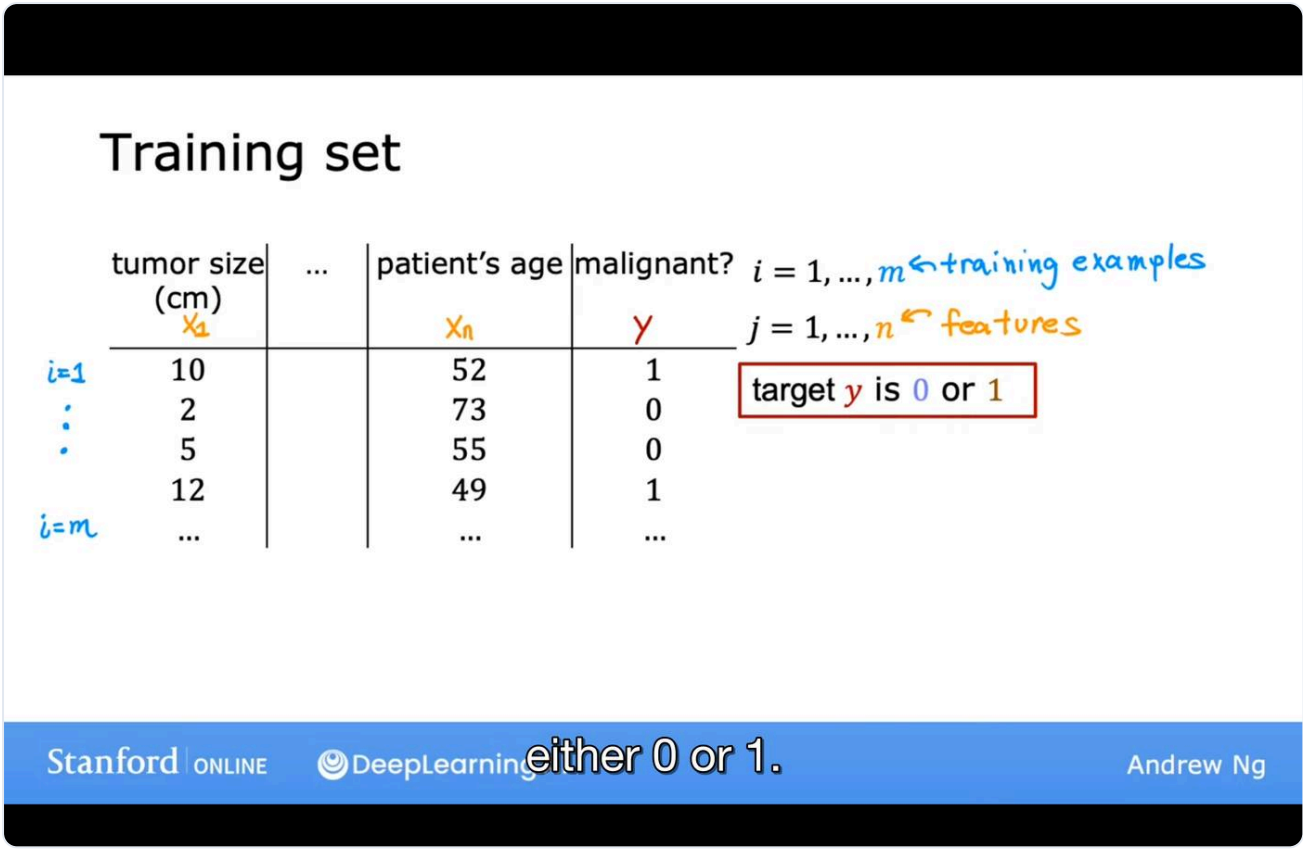


FIGURE 34.2 | Training Set

Since this is a binary classification task, the target label y takes on only two values, either 0 or 1. Finally, the logistic regression model is defined by this equation.

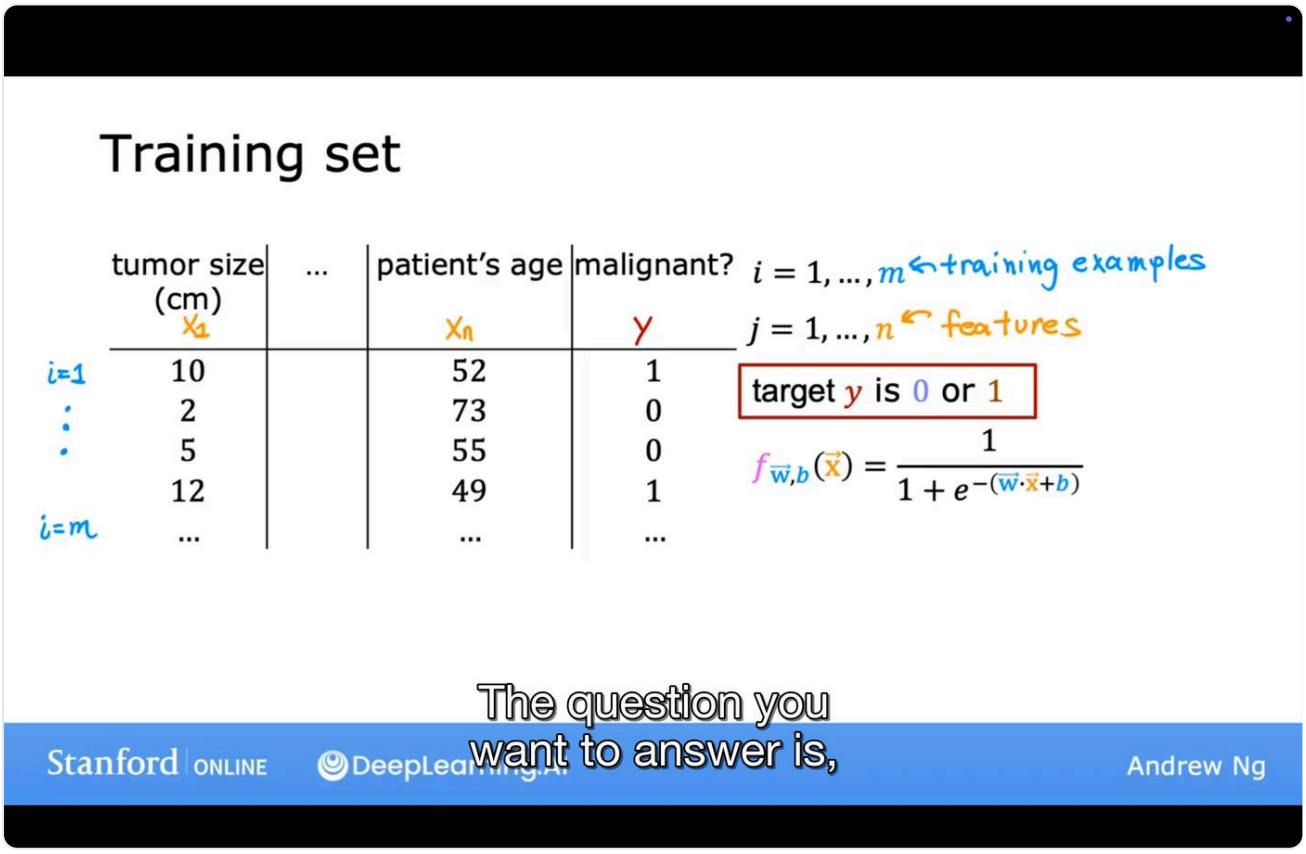


FIGURE 34.3 | Training Set

The question you want to answer is, given this training set, how can you choose parameters w and b?

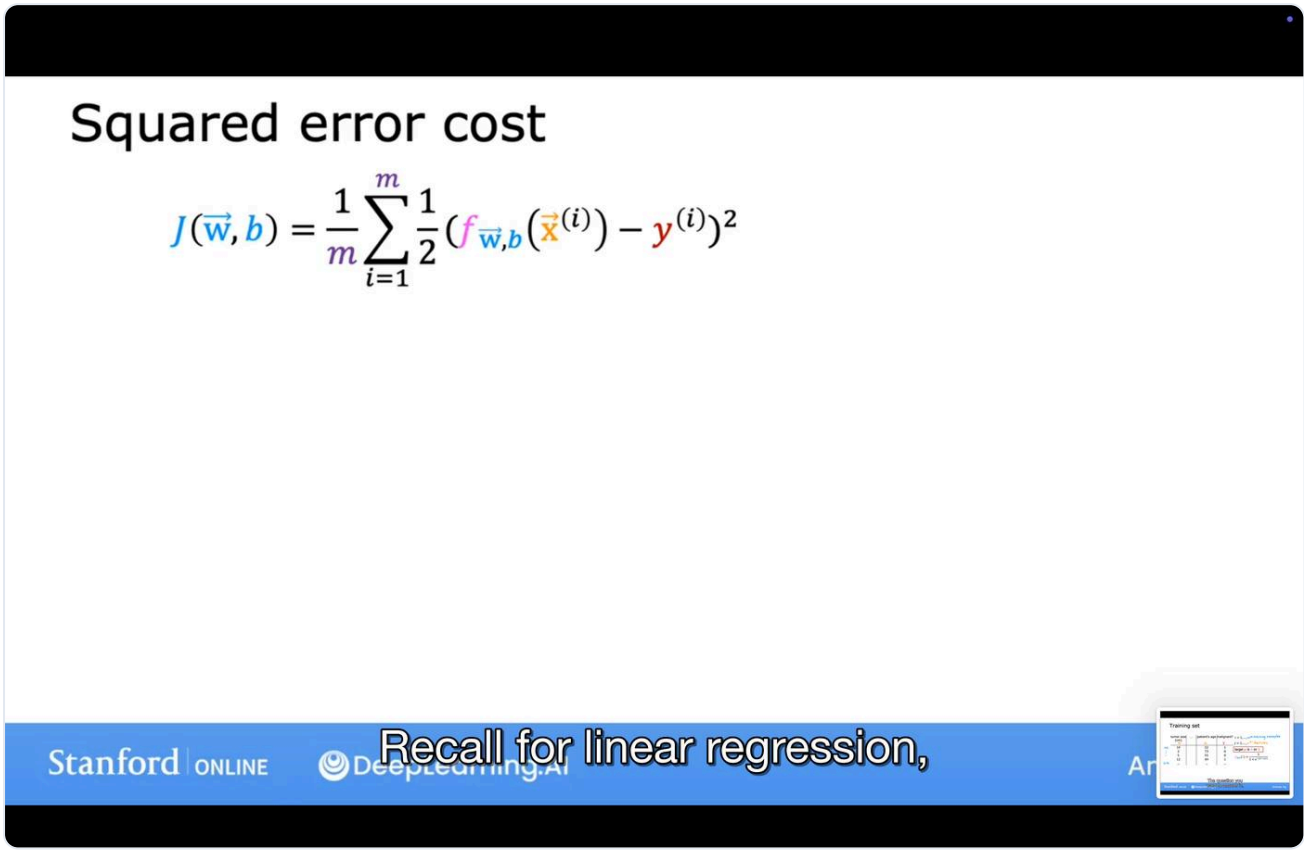


FIGURE 34.4 | Squared Error Cost

Recall for linear regression, this is the squared error cost function. The only thing I've changed is that I put the one half inside the summation instead of outside the summation.

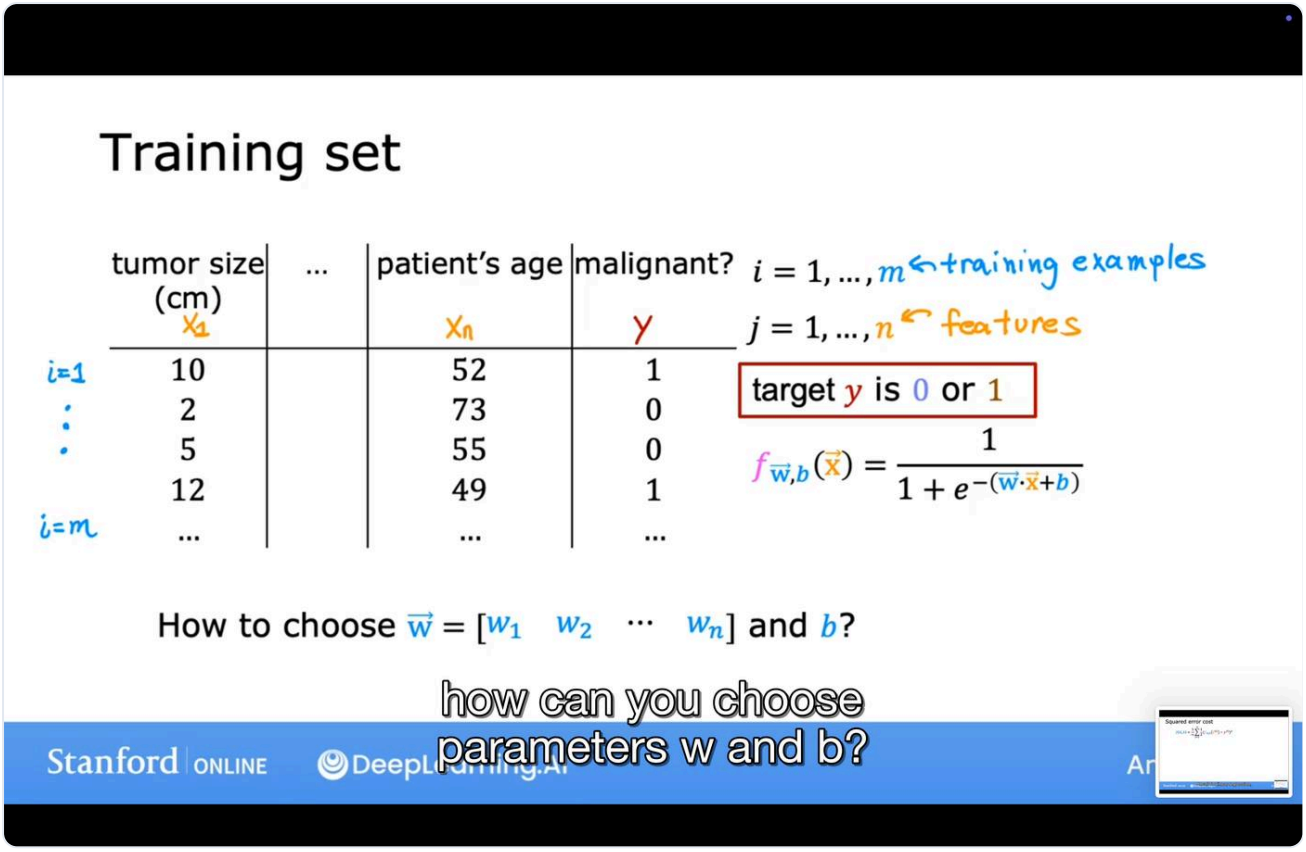


FIGURE 34.5 | Training Set

You might remember that in the case of linear regression, where f of x is the linear function, w dot x plus b. The cost function looks like this, is a convex function or a bowl shape or hammer shape.

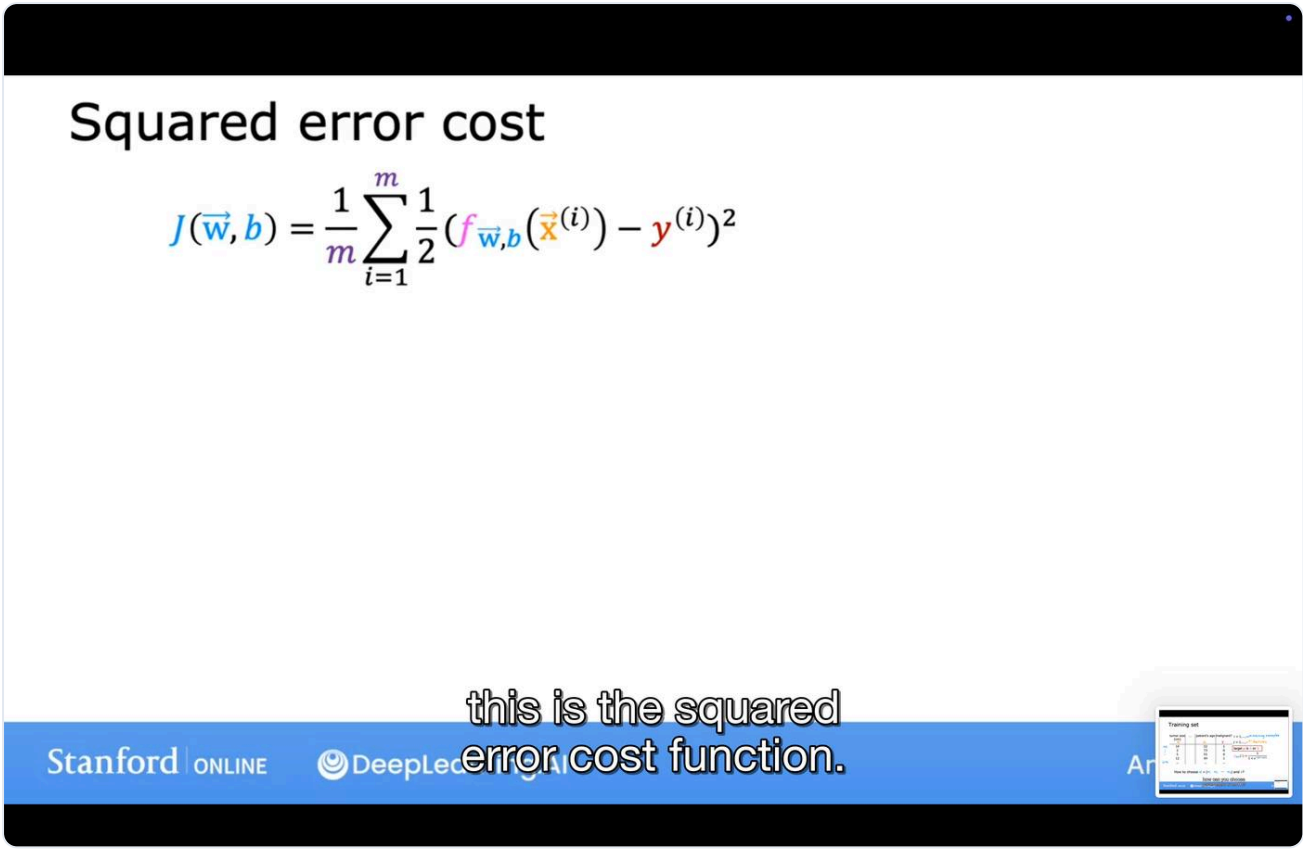


FIGURE 34.6 | Squared Error Cost

Gradient descent will look like this, where you take one step, one step, and so on to converge at the global minimum.

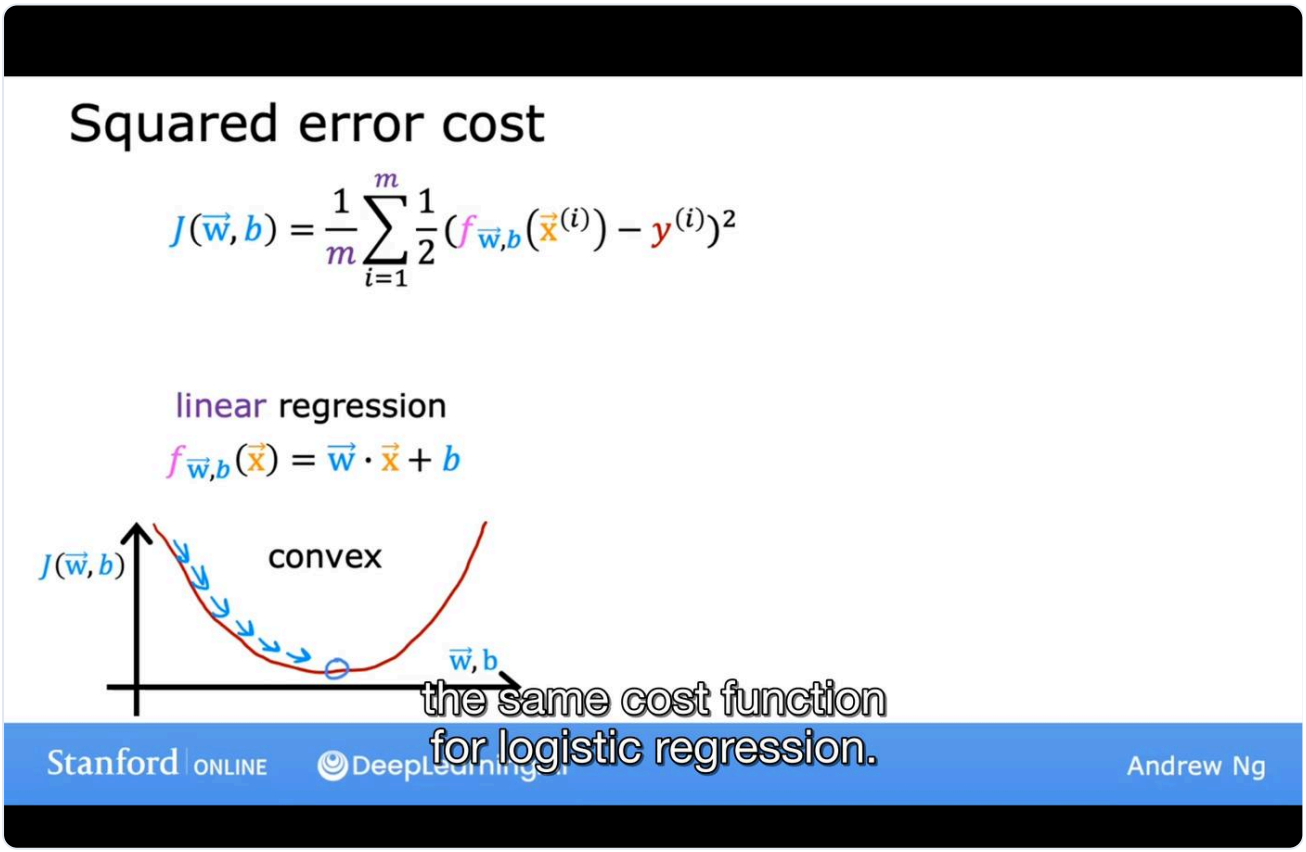


FIGURE 34.7 | Squared Error Cost

Now you could try to use the same cost function for logistic regression. But it turns out that if I were to write f of x equals 1 over 1 plus e to the negative wx plus b and plot the cost function using this value of f of x, then the cost will look like this.

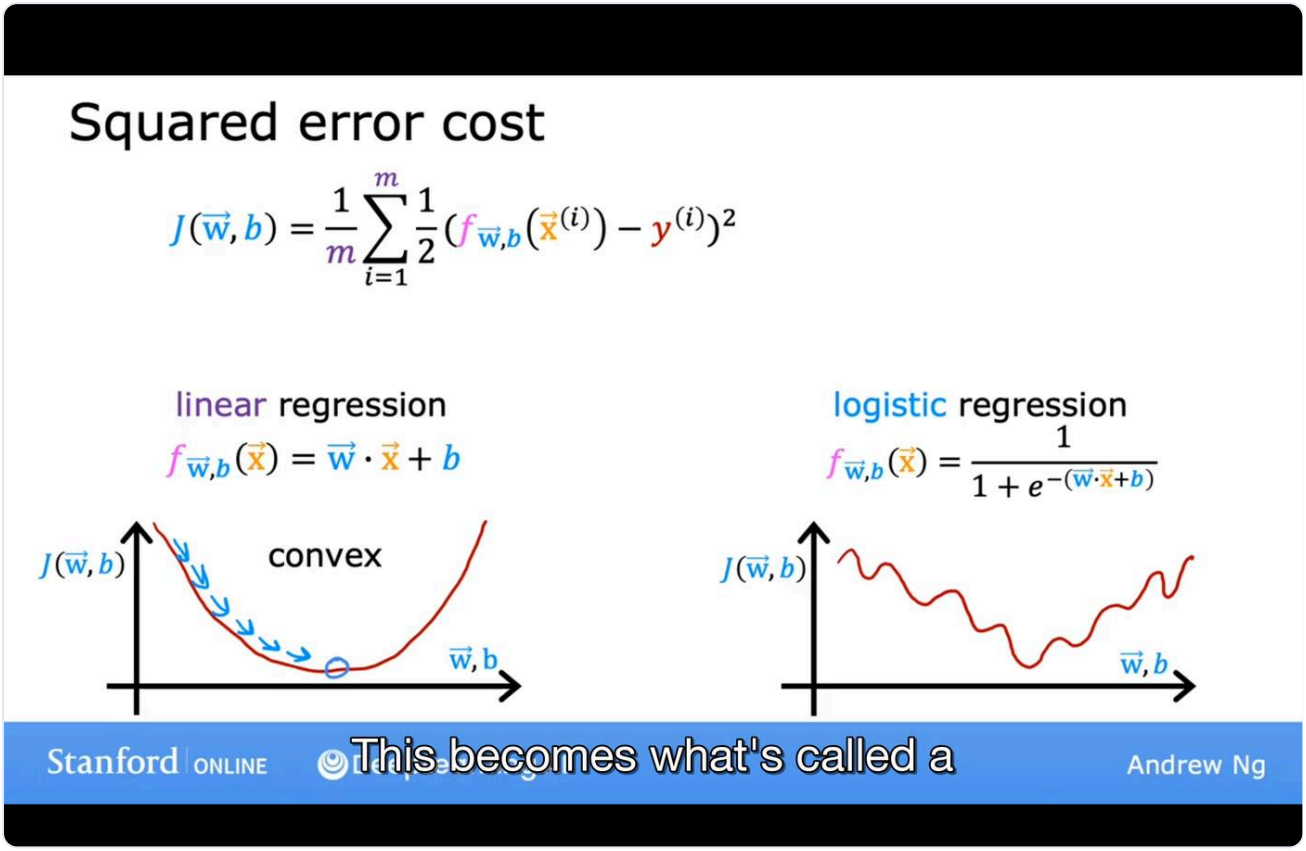


FIGURE 34.8 | Squared Error Cost

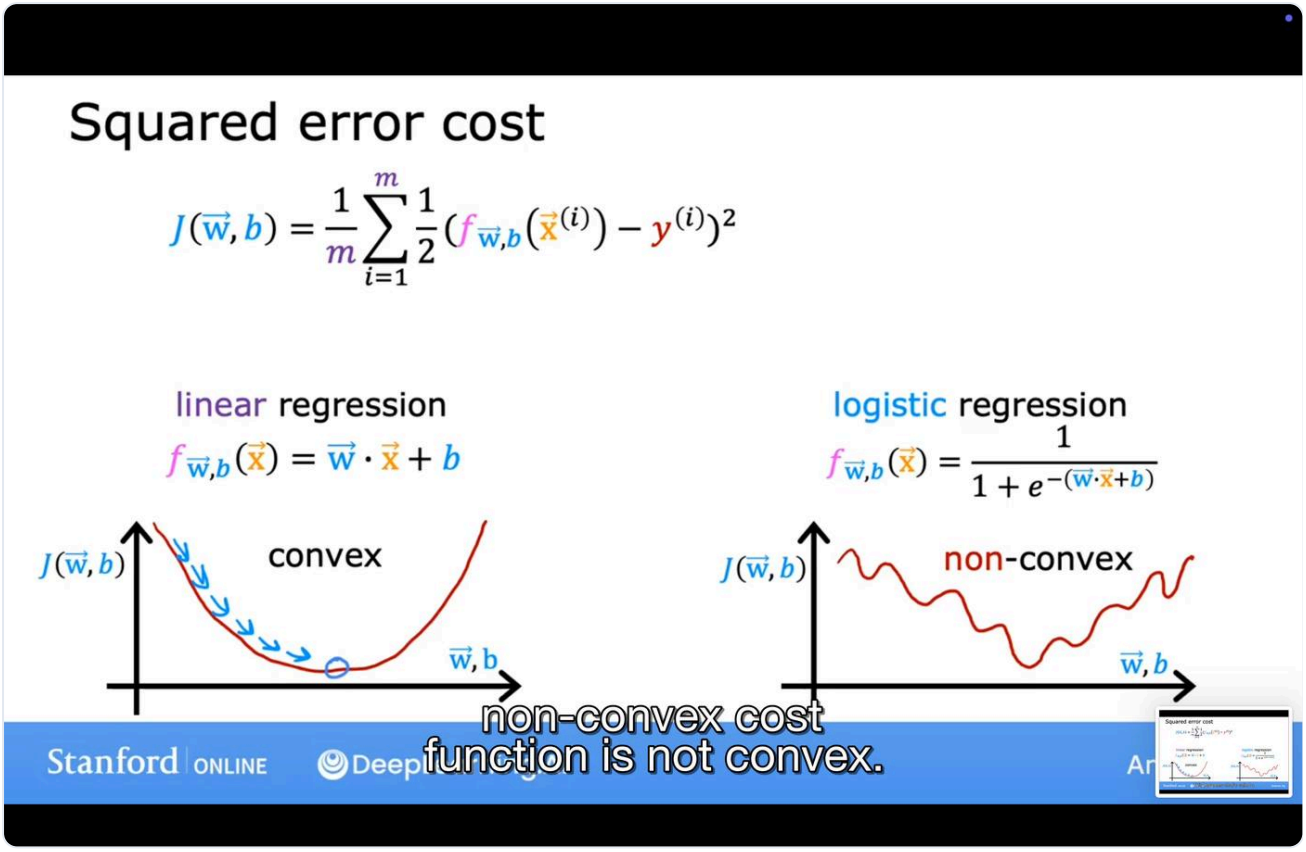


FIGURE 34.9 | Squared Error Cost

This becomes what's called a non-convex cost function is not convex. What this means is that if you were to try to use gradient descent. There are lots of local minima that you can get sucking. It turns out that for logistic regression, this squared error cost function is not a good choice.

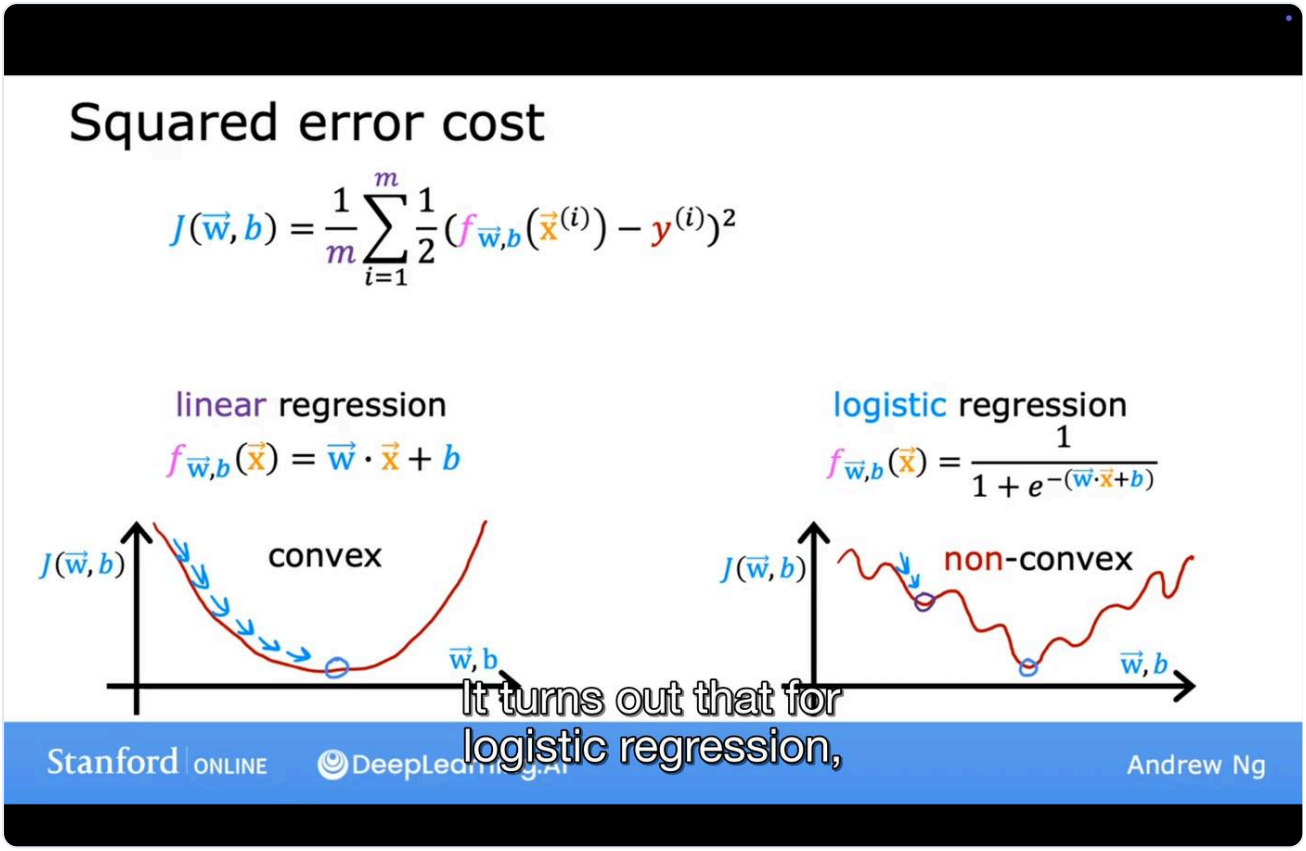


FIGURE 34.10 | Squared Error Cost

Instead, there will be a different cost function that can make the cost function convex again. The gradient descent can be guaranteed to converge to the global minimum. The only thing I've changed is that I put the one half inside the summation instead of outside the summation. This will make the math you see later on this slide a little bit simpler. In order to build a new cost function, one that we'll use for logistic regression.

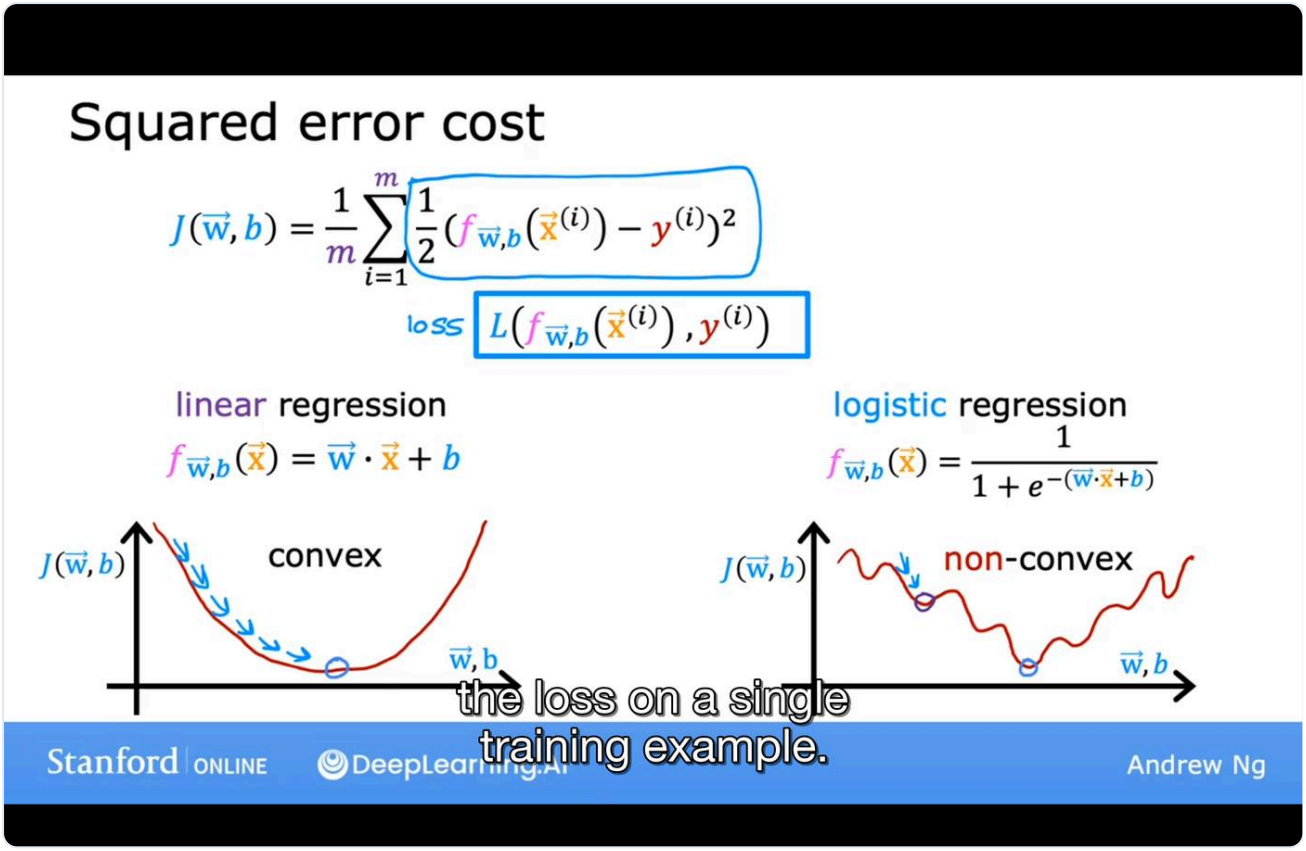


FIGURE 34.11 | Squared Error Cost

I'm going to change a little bit the definition of the cost function J of w and b. In particular, if you look inside this summation, let's call this term inside the loss on a single training example. I'm going to denote the loss via this capital L and as a function of the prediction of the learning algorithm, f of x as well as of the true label y. The loss given the predictor f of x and the true label y is equal in this case to 1.5 of the squared difference.

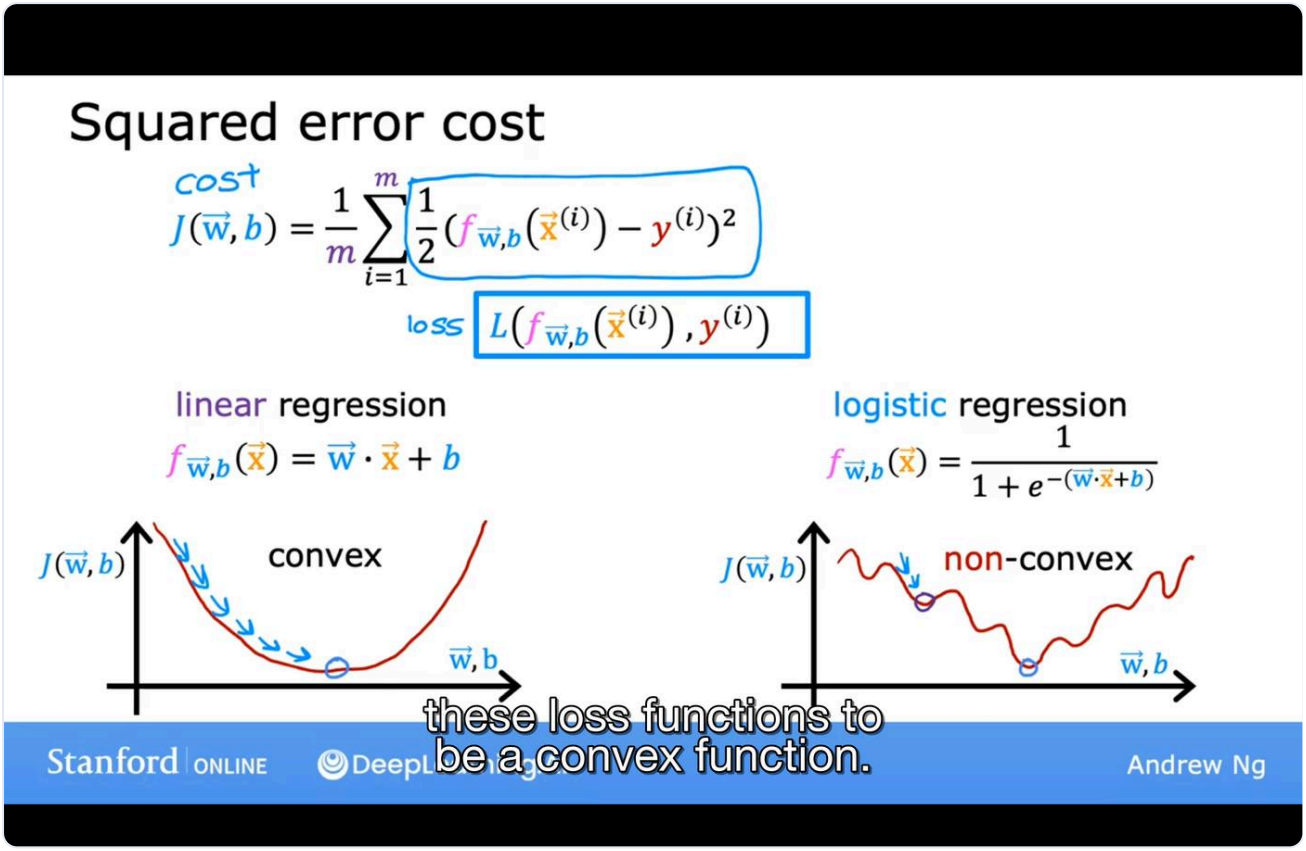


FIGURE 34.12 | Squared Error Cost

We'll see shortly that by choosing a different form for this loss function, will be able to keep the overall cost function, which is 1 over n times the sum of these loss functions to be a convex function. Now, the loss function inputs f of x and the true label y and tells us how well we're doing on that example.

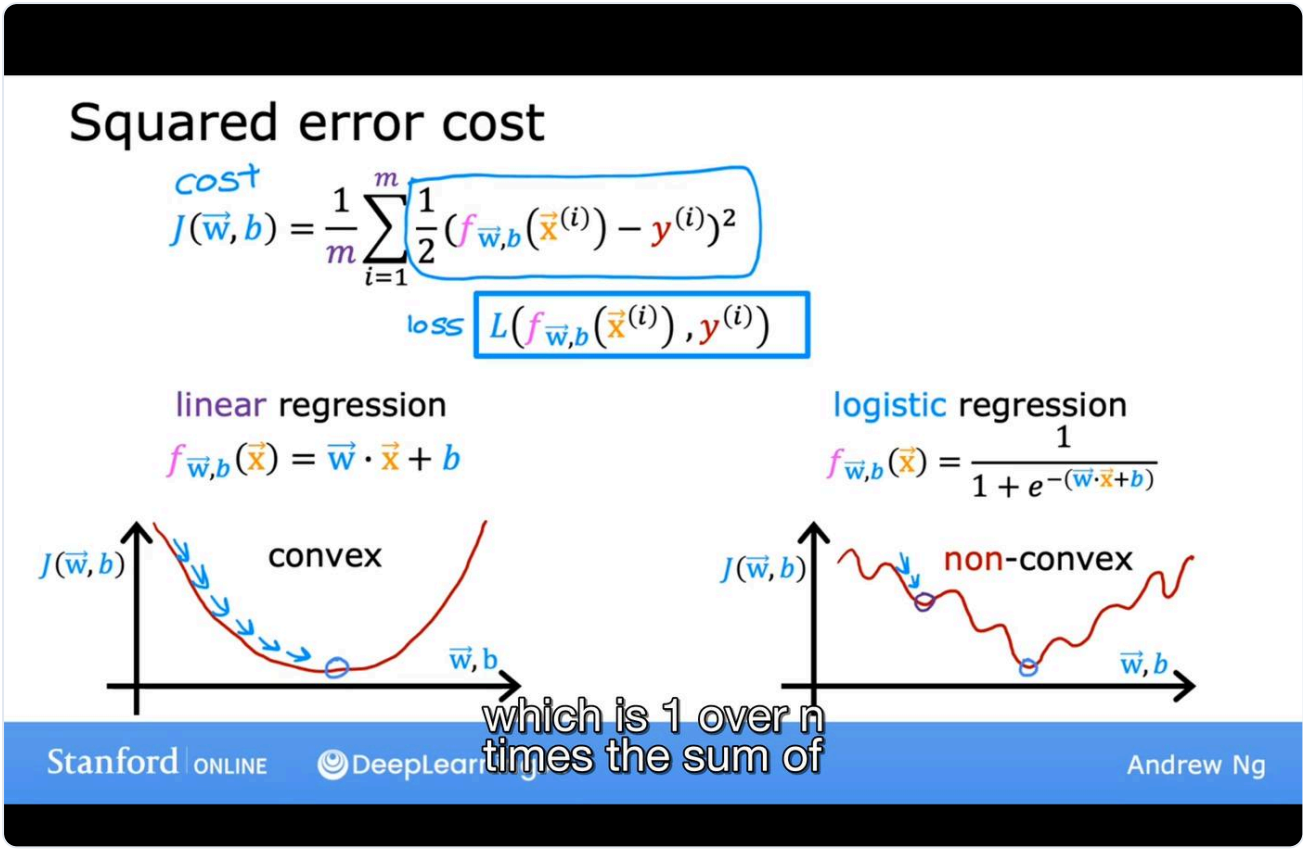


FIGURE 34.13 | Squared Error Cost

I'm going to just write down here at the definition of the loss function we'll use for logistic regression. If the label y is equal to 1, then the loss is negative log of f of x and if the label y is equal to 0, then the loss is negative log of 1 minus f of x.

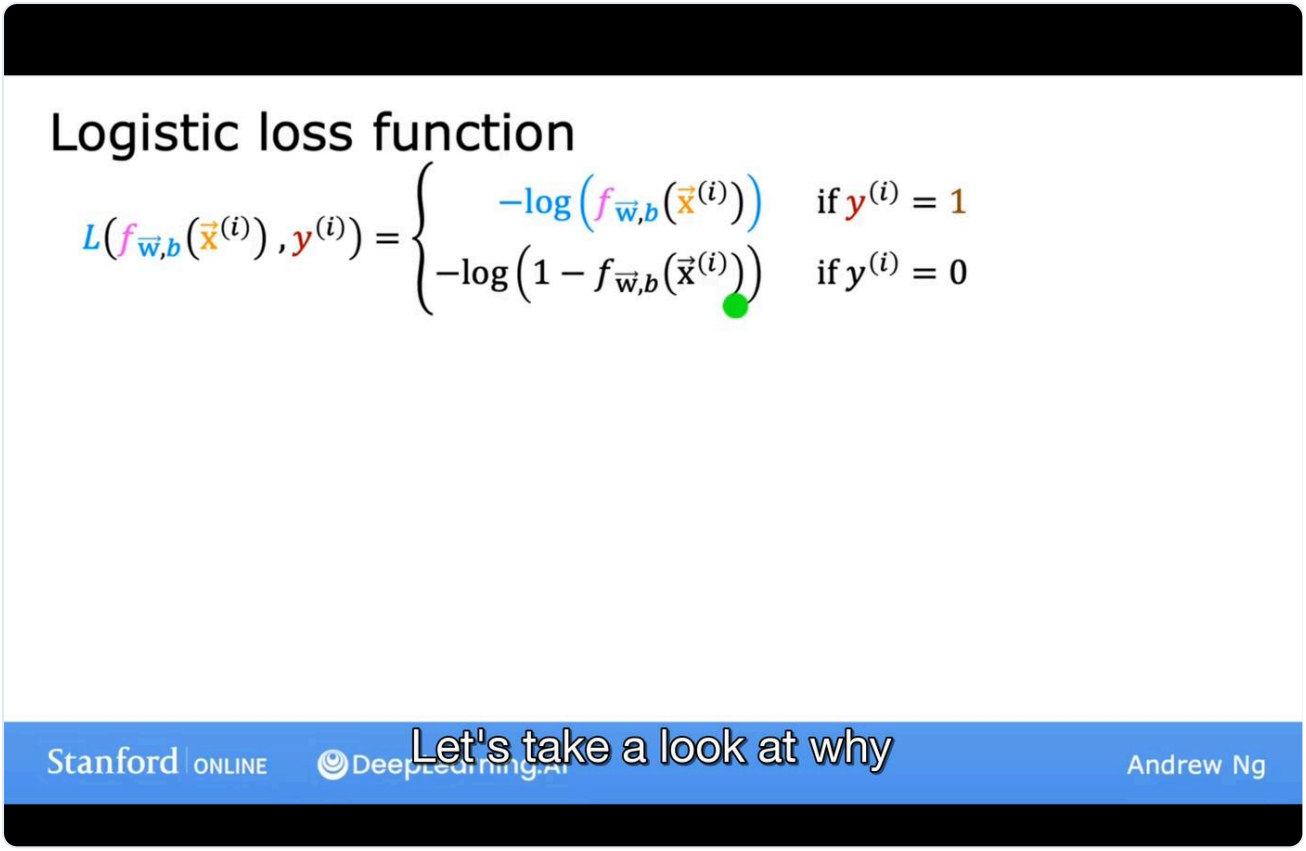


FIGURE 34.14 | Logistic Loss Function

Let's take a look at why this loss function hopefully makes sense. Let's first consider the case of y equals 1 and plot what this function looks like to gain some intuition about what this loss function is doing. Remember, the loss function measures how well you're doing on one training example and is by summing up the losses on all of the training examples that you then get, the cost function, which measures how well you're doing on the entire training set. If you plot log of f, it looks like this curve here, where f here is on the horizontal axis. A plot of a negative of the log of f looks like this, where we just flip the curve along the horizontal axis.

Notice that it intersects the horizontal axis at f equals 1 and continues downward from there. Now, f is the output of logistic regression. Thus, f is always between zero and one because the output of logistic regression is always between zero and one. The only part of the function that's relevant is therefore this part over here, corresponding to f between 0 and 1. Let's zoom in and take a closer look at this part of the graph.

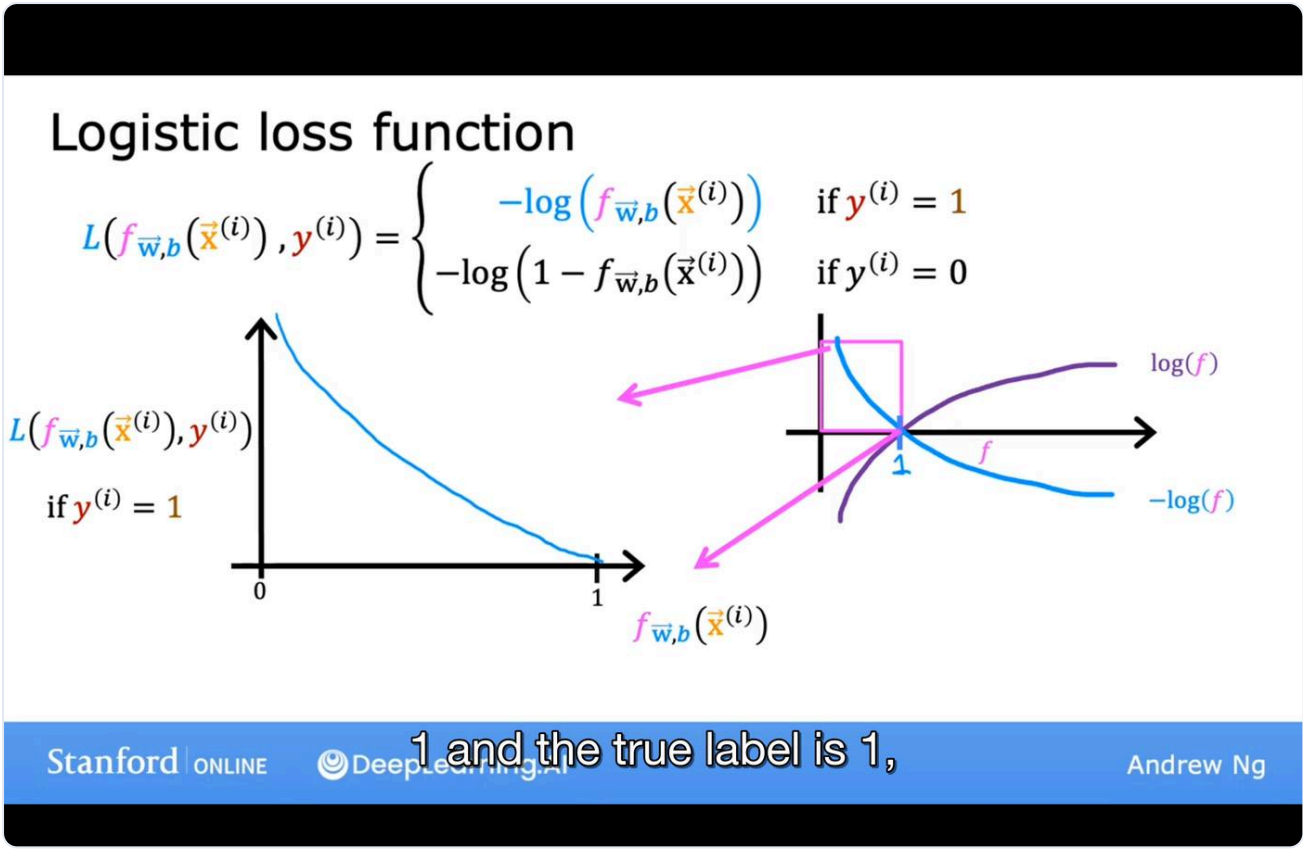


FIGURE 34.15 | Logistic Loss Function

If the algorithm predicts a probability close to 1 and the true label is 1, then the loss is very small. It's pretty much 0 because you're very close to the right answer. Now continue with the example of the true label y being 1, say everything is a malignant tumor. If the algorithm predicts 0.5, then the loss is at this point here, which is a bit higher but not that high. Whereas in contrast, if the algorithm were to have outputs at 0.1 if it thinks that there is only a 10 percent chance of the tumor being malignant but y really is 1. If really is malignant, then the loss is this much higher value over here. When y is equal to 1, the loss function incentivizes or nurtures, or helps push the algorithm to make more accurate predictions because the loss is lowest, when it predicts values close to 1.

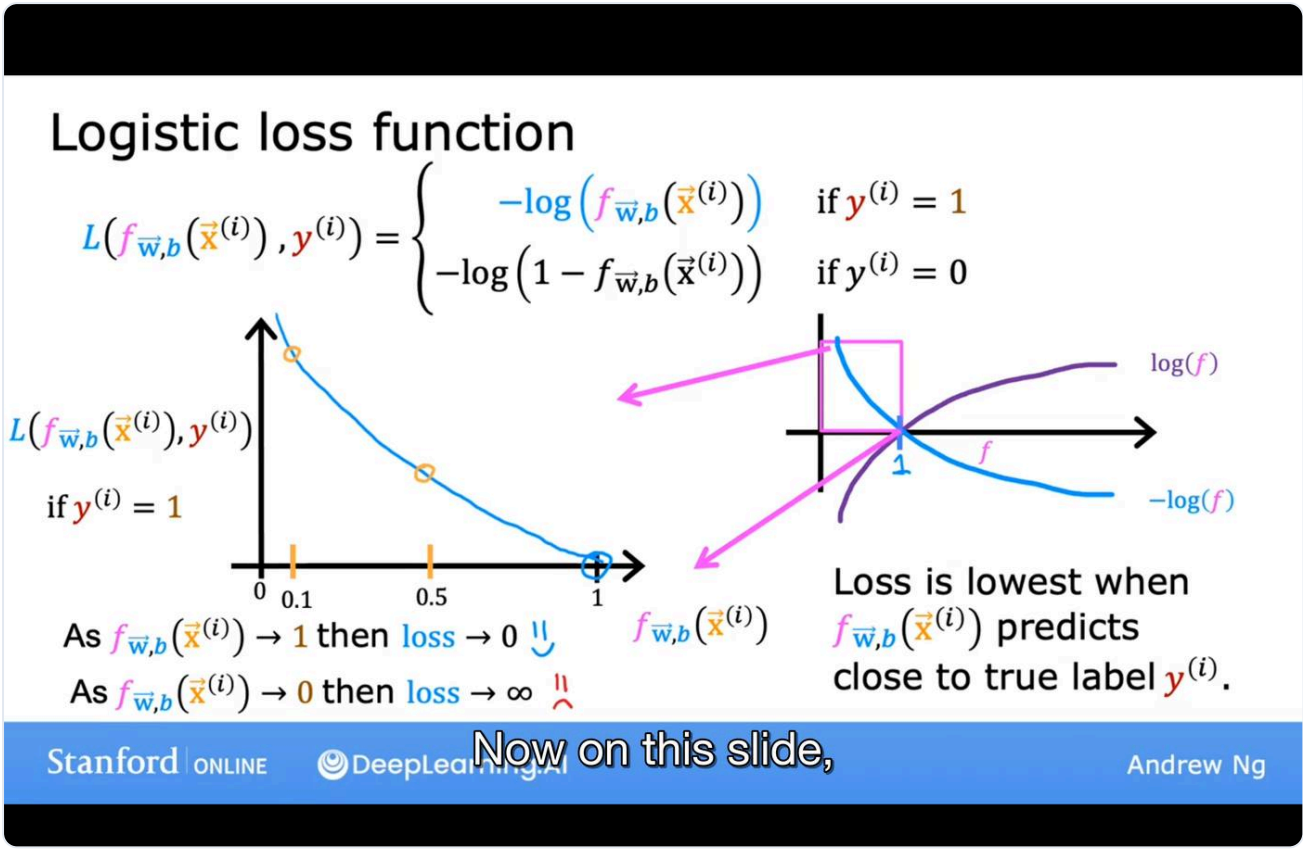


FIGURE 34.16 | Logistic Loss Function

Now on this slide, we'll be looking at what the loss is when y is equal to 1. On this slide, let's look at the second part of the loss function corresponding to when y is equal to 0. In this case, the loss is negative log of 1 minus f of x. When this function is plotted, it actually looks like this. The range of f is limited to 0 to 1 because logistic regression only outputs values between 0 and 1.

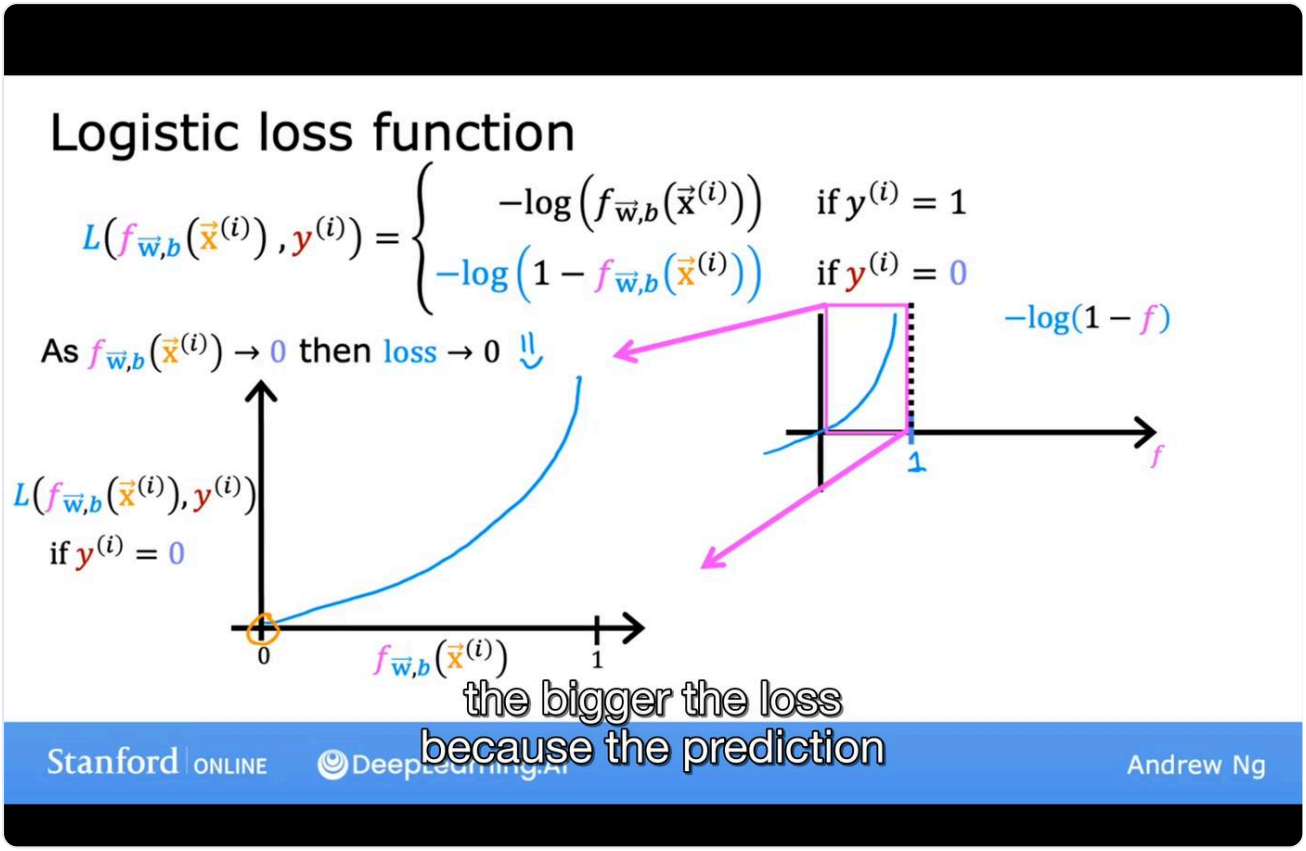


FIGURE 34.17 | Logistic Loss Function

If we zoom in, this is what it looks like. In this plot, corresponding to y equals 0, the vertical axis shows the value of the loss for different values of f of x. When f is 0 or very close to 0, the loss is also going to be very small which means that if the true label is 0 and the model's prediction is very close to 0, well, you nearly got it right so the loss is appropriately very close to 0. The larger the value of f of x gets, the bigger the loss because the prediction is further from the true label 0. In fact, as that prediction approaches 1, the loss actually approaches infinity.

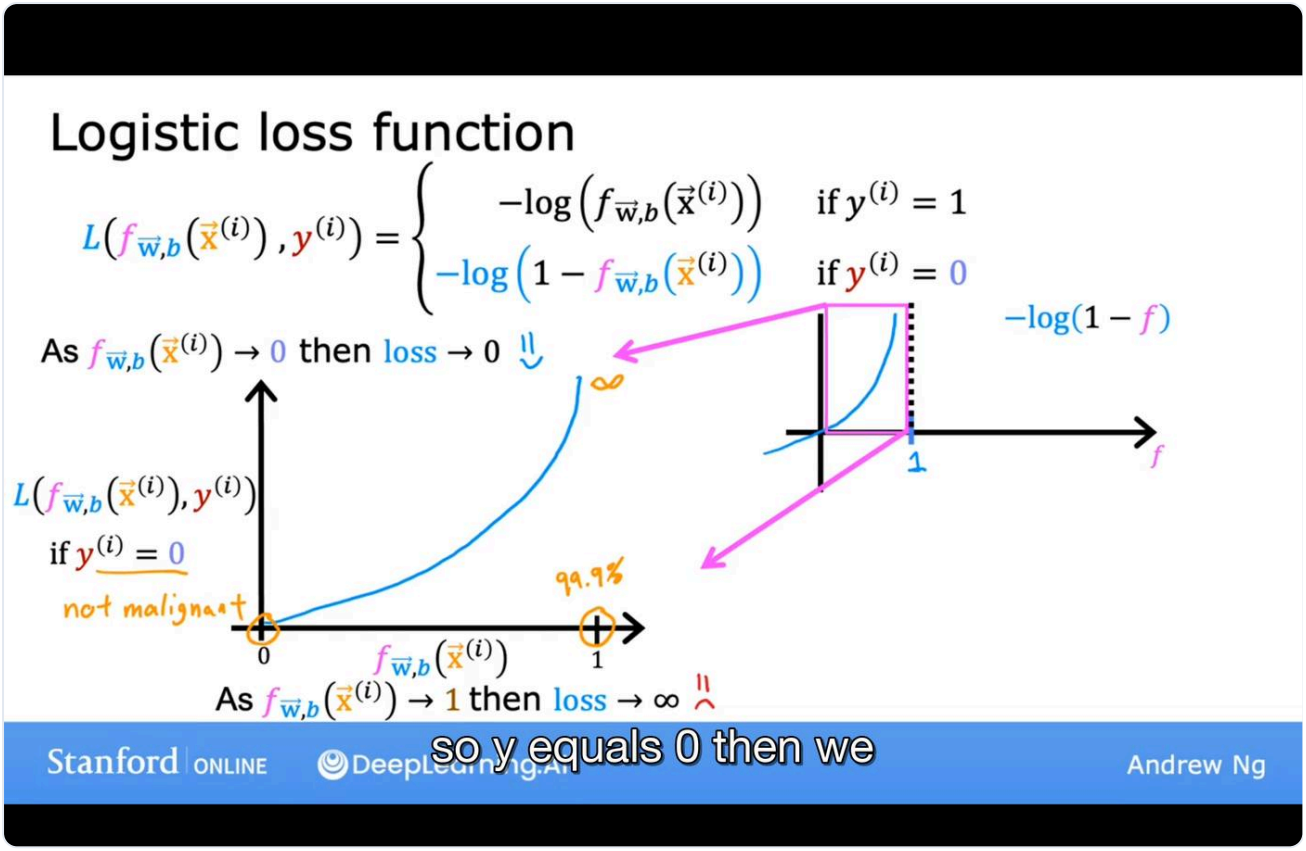


FIGURE 34.18 | Logistic Loss Function

Going back to the tumor prediction example just says if the model predicts that the patient's tumor is almost certain to be malignant, say, 99.9 percent chance of malignancy, that turns out to actually not be malignant, so y equals 0 then we penalize the model with a very high loss.

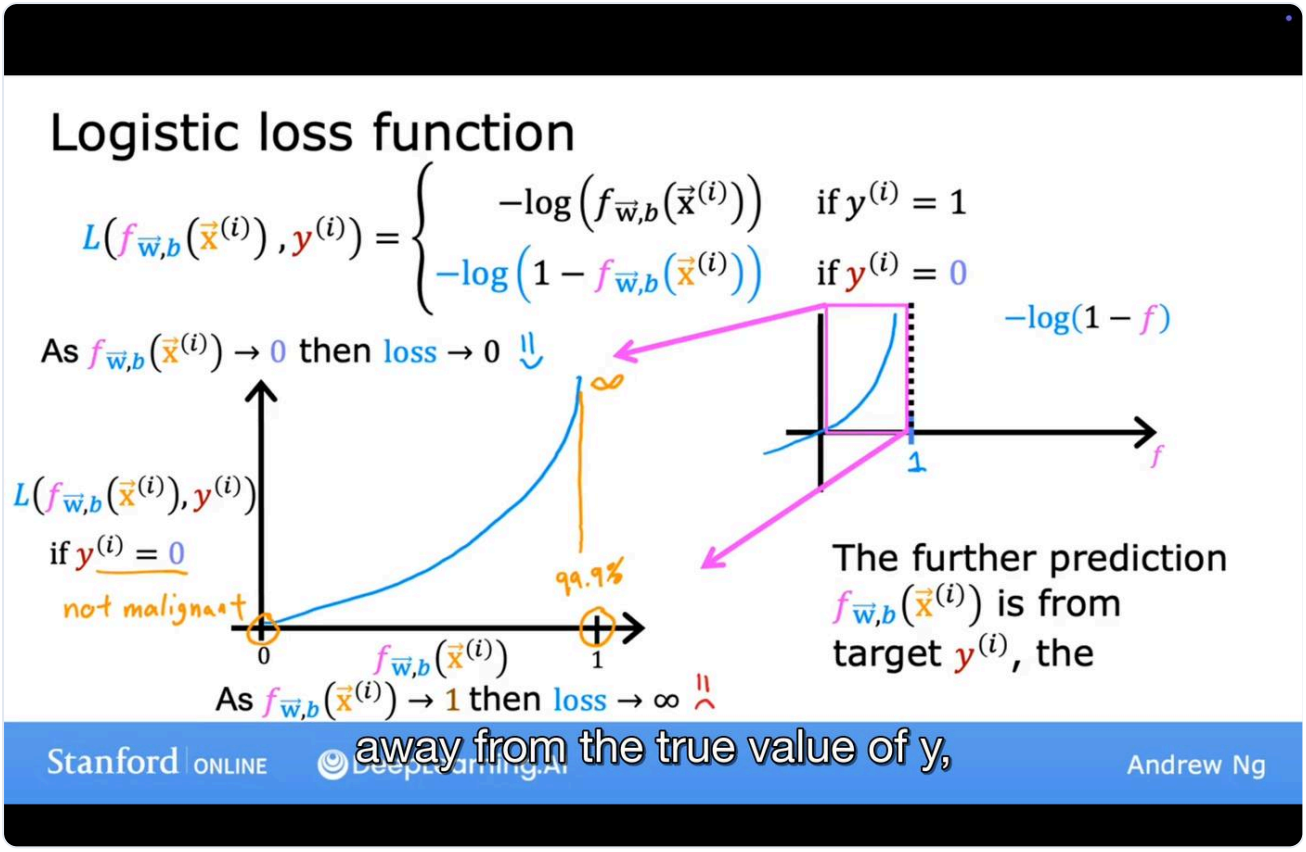


FIGURE 34.19 | Logistic Loss Function

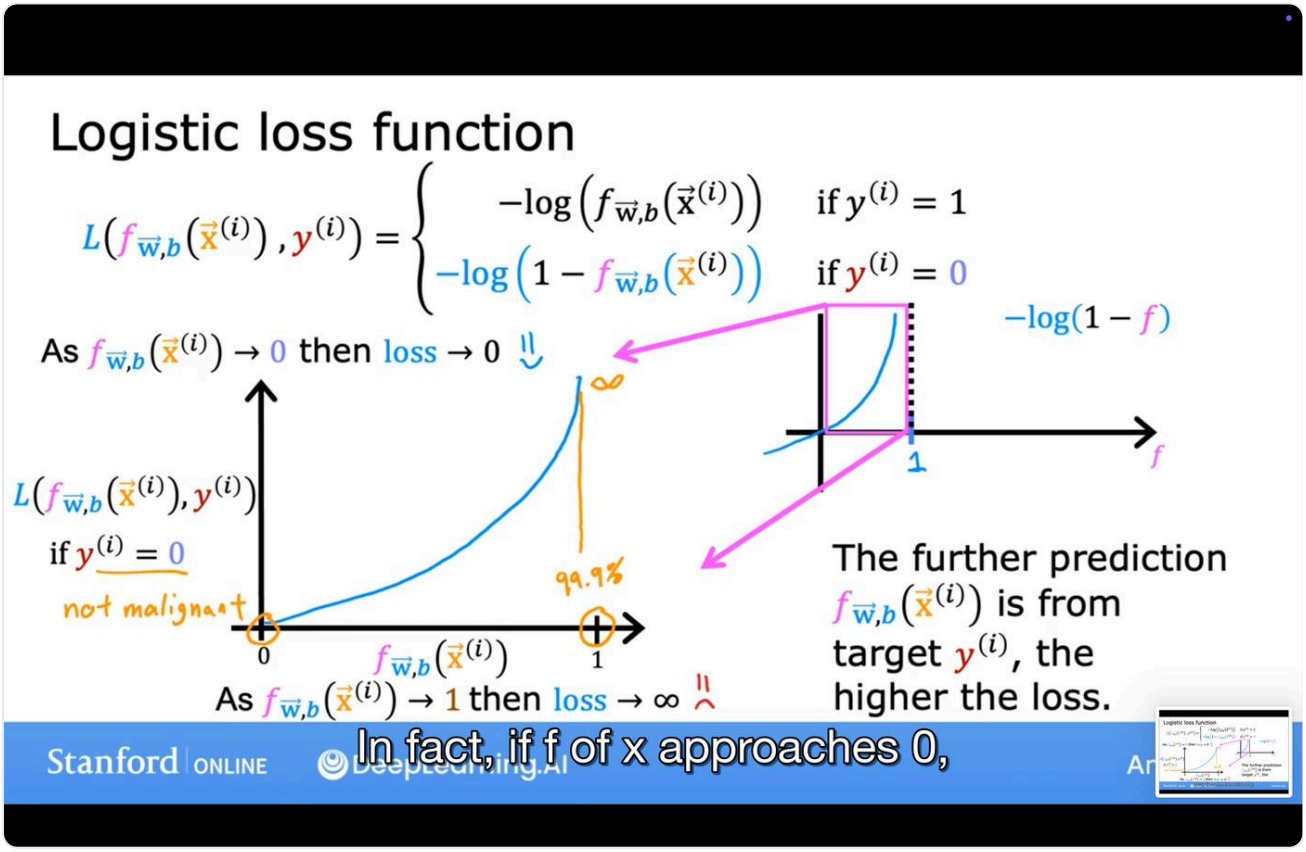


FIGURE 34.20 | Logistic Loss Function

In this case of y equals 0, so this is in the case of y equals 1 on the previous slide, the further the prediction f of x is away from the true value of y, the higher the loss. In fact, if f of x approaches 0, the loss here actually goes really large and in fact approaches infinity. When the true label is 1, the algorithm is strongly incentivized not to predict something too close to 0. In this video, you saw why the squared error cost function doesn't work well for logistic regression. We also defined the loss for a single training example and came up with a new definition for the loss function for logistic regression.

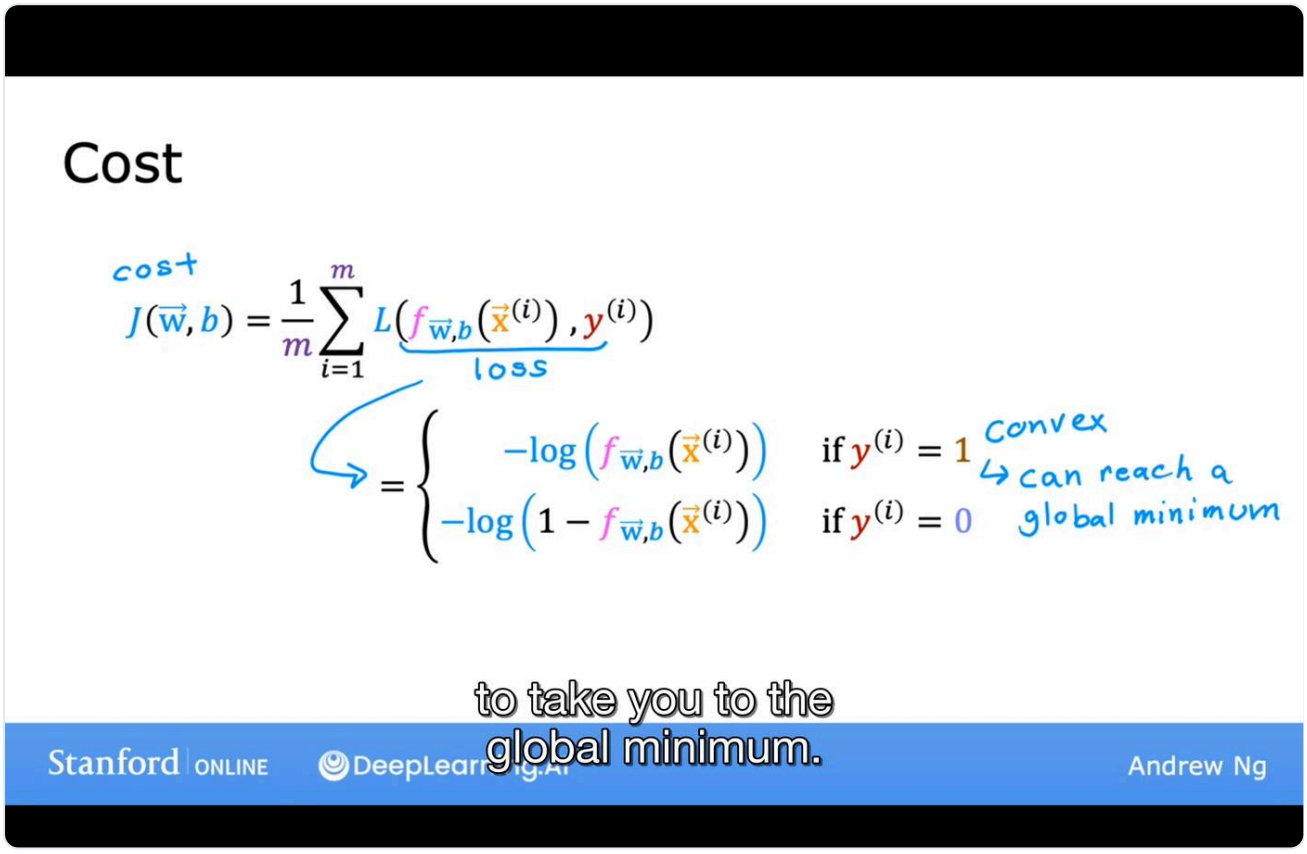


FIGURE 34.21

It turns out that with this choice of loss function, the overall cost function will be convex and thus you can reliably use gradient descent to take you to the global minimum. Proving that this function is convex, it's beyond the scope of this cost. You may remember that the cost function is a function of the entire training set and is, therefore, the average or 1 over m times the sum of the loss function on the individual training examples. The cost on a certain set of parameters, w and b, is equal to 1 over m times the sum of all the training examples of the loss on the training examples.

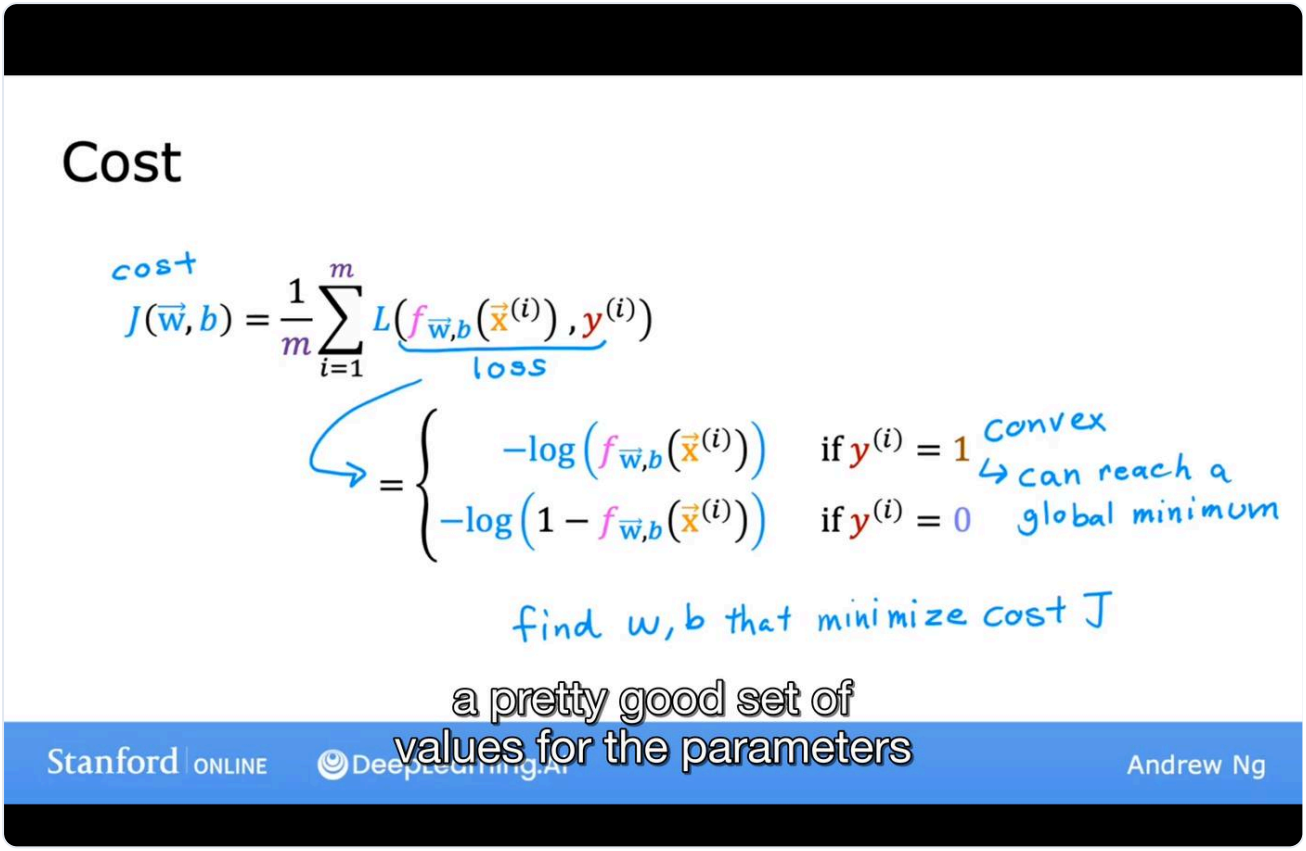


FIGURE 34.22

If you can find the value of the parameters, w and b, that minimizes this, then you'd have a pretty good set of values for the parameters w and b for logistic regression. In the upcoming optional lab, you'll get to take a look at how the squared error cost function doesn't work very well for classification, because you see that the surface plot results in a very wiggly costs surface with many local minima. Then you'll take a look at the new logistic loss function. As you can see here, this produces a nice and smooth convex surface plot that does not have all those local minima. Please take a look at the cost and the plots after this video. We've seen a lot in this video.

In the next video, let's go back and take the loss function for a single train example and use that to define the overall cost function for the entire training set. We'll also figure out a simpler way to write out the cost function, which will then later allow us to run gradient descent to find good parameters for logistic regression. Let's go on to the next video.

LECTURE 35

Simplified Cost Function

In the last video you saw the loss function and the cost function for logistic regression. In this video you'll see a slightly simpler way to write out the loss and cost functions, so that the implementation can be a bit simpler when we get to gradient descent for fitting the parameters of a logistic regression model. Let's take a look. As a reminder, here is the loss function that we had defined in the previous video for logistic regression. Because we're still working on a binary classification problem, y is either zero or one. Because y is either zero or one and cannot take on any value other than zero or one, we'll be able to come up with a simpler way to write this loss function.

You can write the loss function as follows. Given a prediction f of x and the target label y, the loss equals negative y times log of f minus 1 minus y times log of 1 minus f. It turns out this equation, which we just wrote in one line, is completely equivalent to this more complex formula up here. Let's see why this is the case.

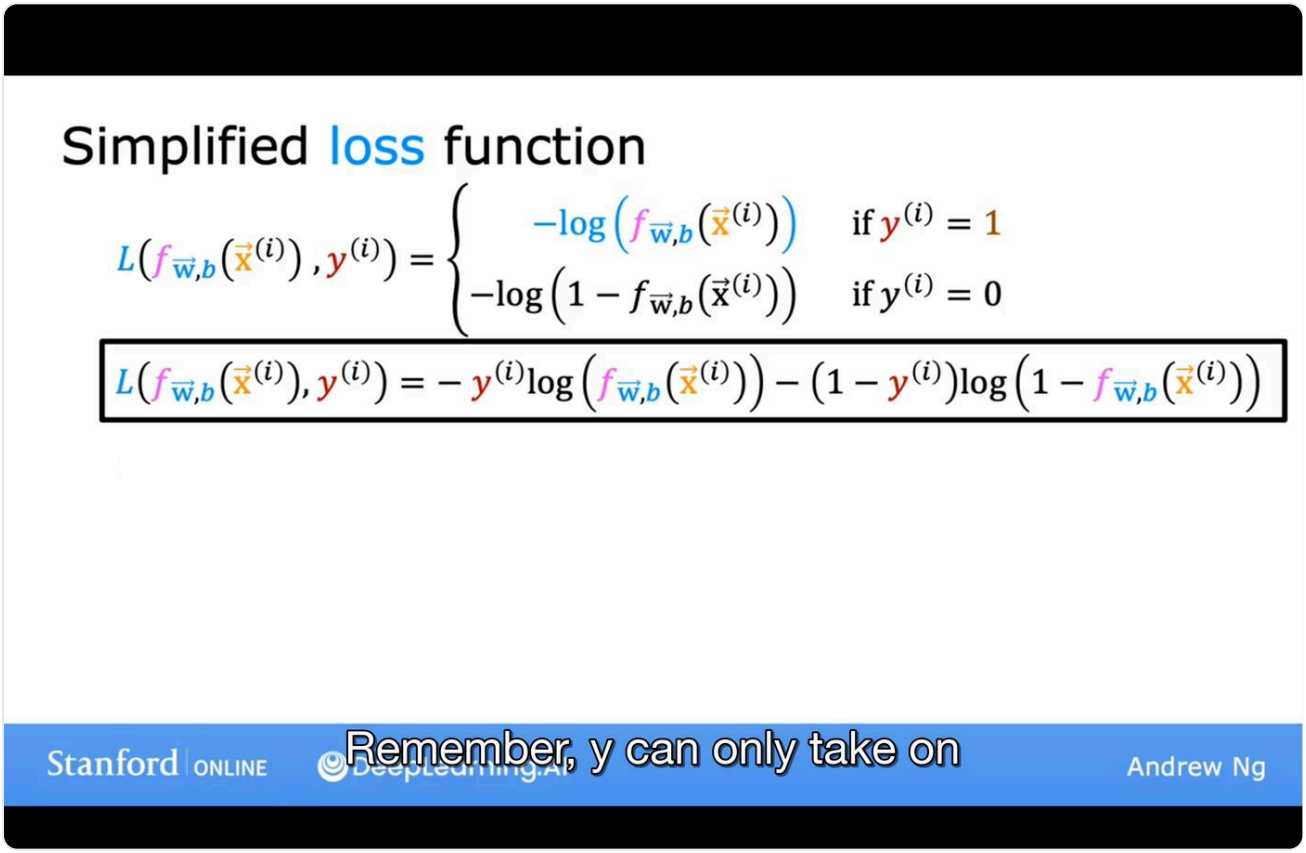


FIGURE 35.1 | Simplified Loss Function

Remember, y can only take on the values of either one or zero. In the first case, let's say y equals 1. This first y over here is one and this 1 minus y is 1 minus 1, which is therefore equal to 0. So the loss becomes negative 1 times log of f of x minus 0 times a bunch of stuff. That becomes zero and goes away. When y is equal to 1, the loss is indeed the first term on top, negative log of f of x. Let's

look at the second case, when y is equal to 0. In this case, this y here is equal to 0, so this first term goes away, and the second term is 1 minus 0 times that logarithmic term. The loss becomes this negative 1 times log of 1 minus f of x. That's just equal to this second term up here.

In the case of y equals 0, we also get back the original loss function as defined above.

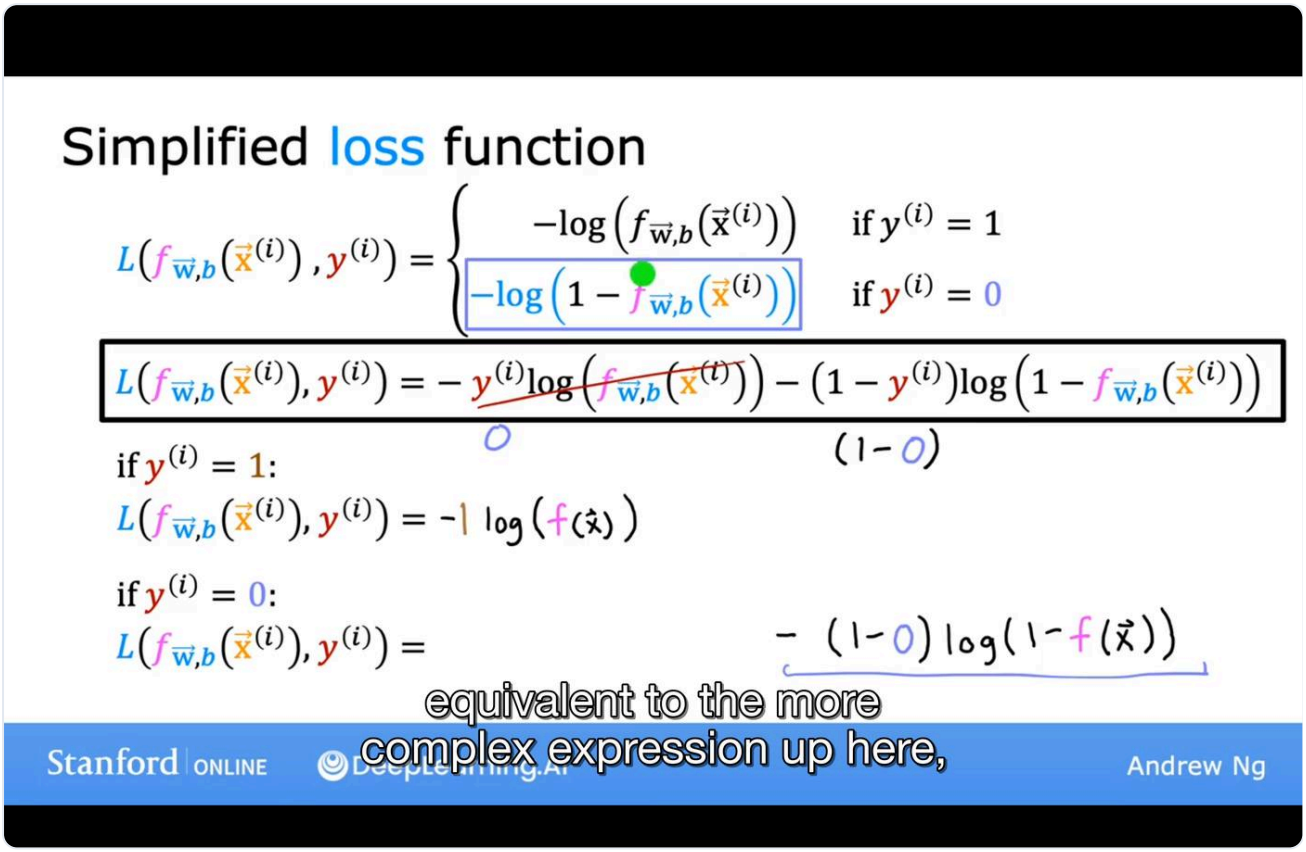


FIGURE 35.2 | Simplified Loss Function

What you see is that whether y is one or zero, this single expression here is equivalent to the more complex expression up here, which is why this gives us a simpler way to write the loss with just one equation without separating out these two cases, like we did on top. Using this simplified loss function, let's go back and write out the cost function for logistic regression. Here again is the simplified loss function.

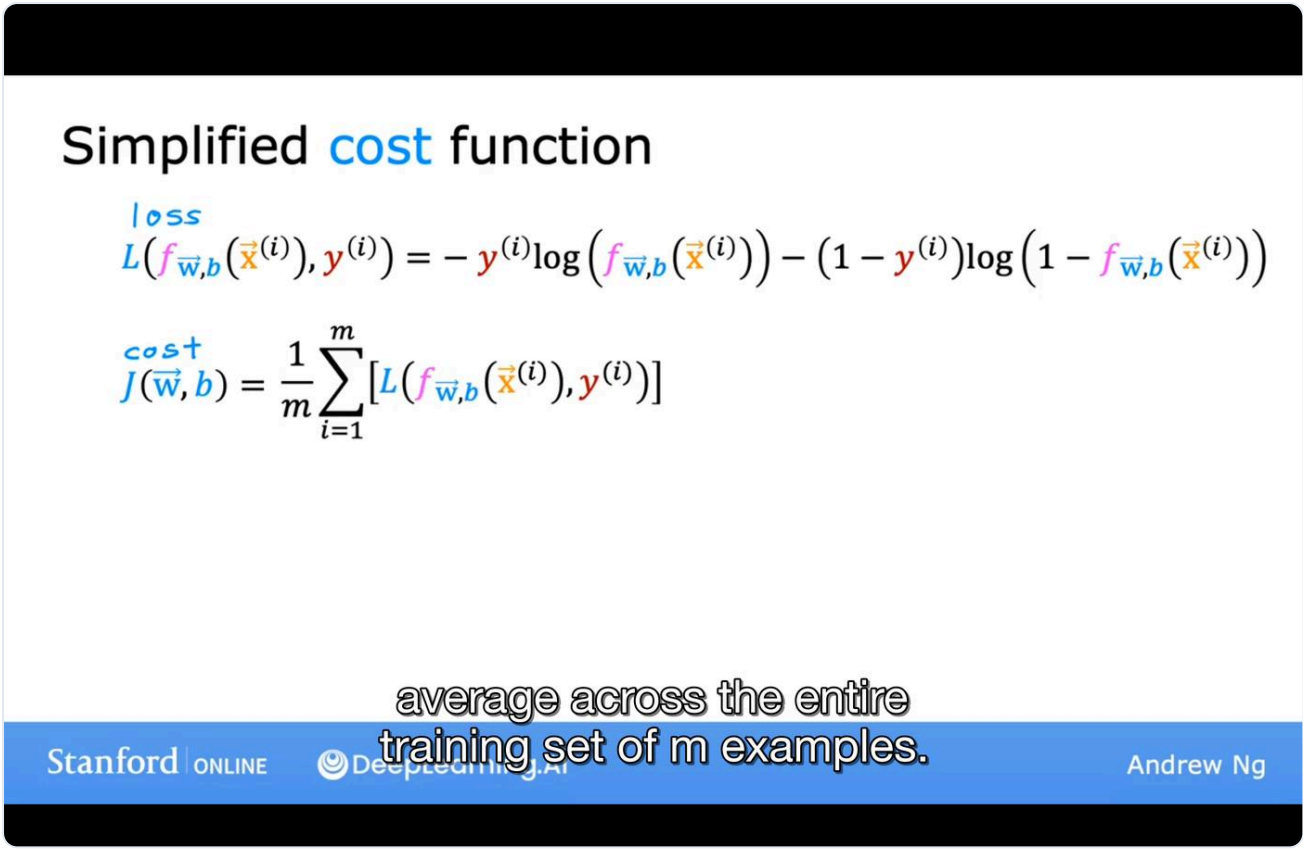


FIGURE 35.3 | Simplified Cost Function

Recall that the cost J is just the average loss, average across the entire training set of m examples. So it's 1 over n times the sum of the loss from i equals 1 to m. If you plug in the definition for the simplified loss from above, then it looks like this, 1 over m times the sum of this term above. If you bring the negative signs and move them outside, then you end up with this expression over here, and this is the cost function.

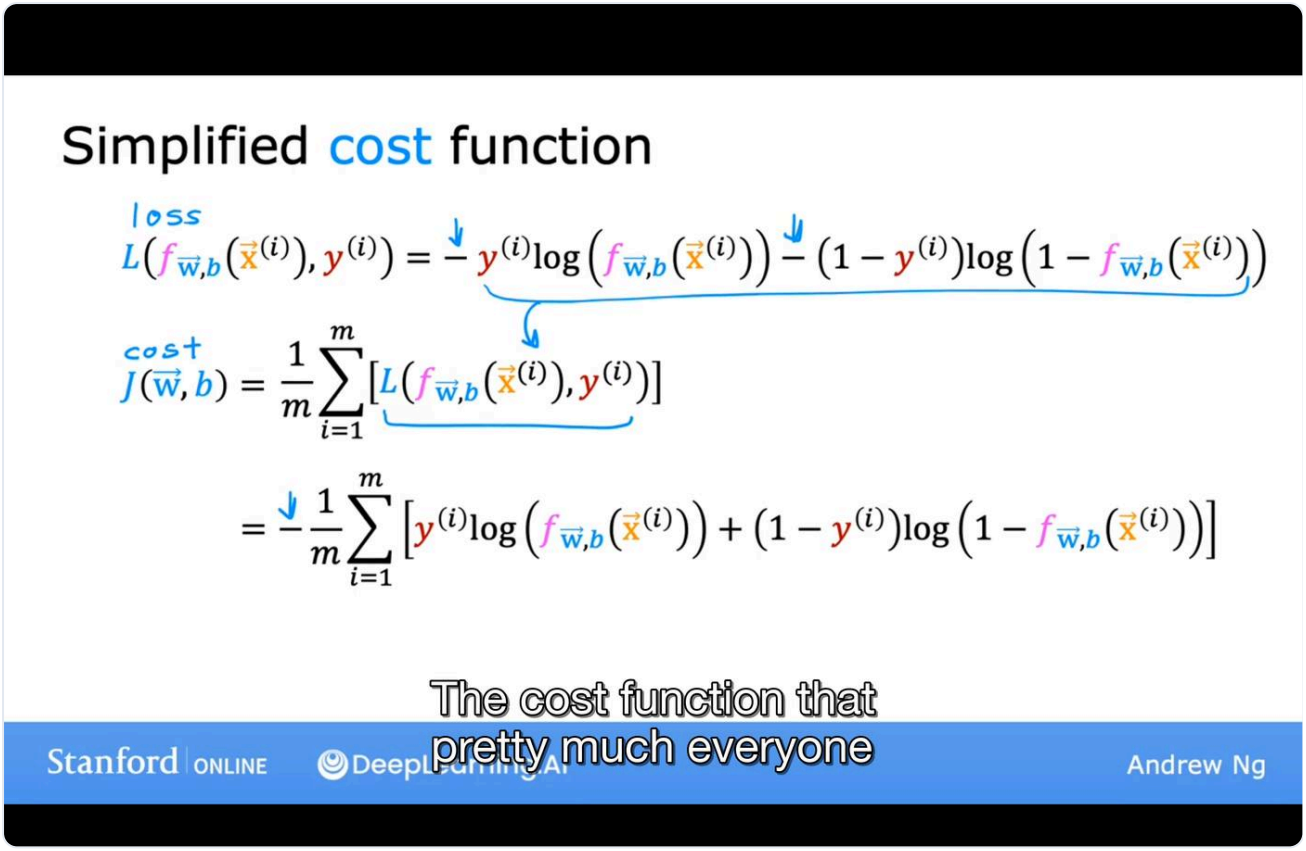


FIGURE 35.4 | Simplified Cost Function

The cost function that pretty much everyone uses to train logistic regression. You might be wondering, why do we choose this particular function when there could be tons of other costs functions we could have chosen?

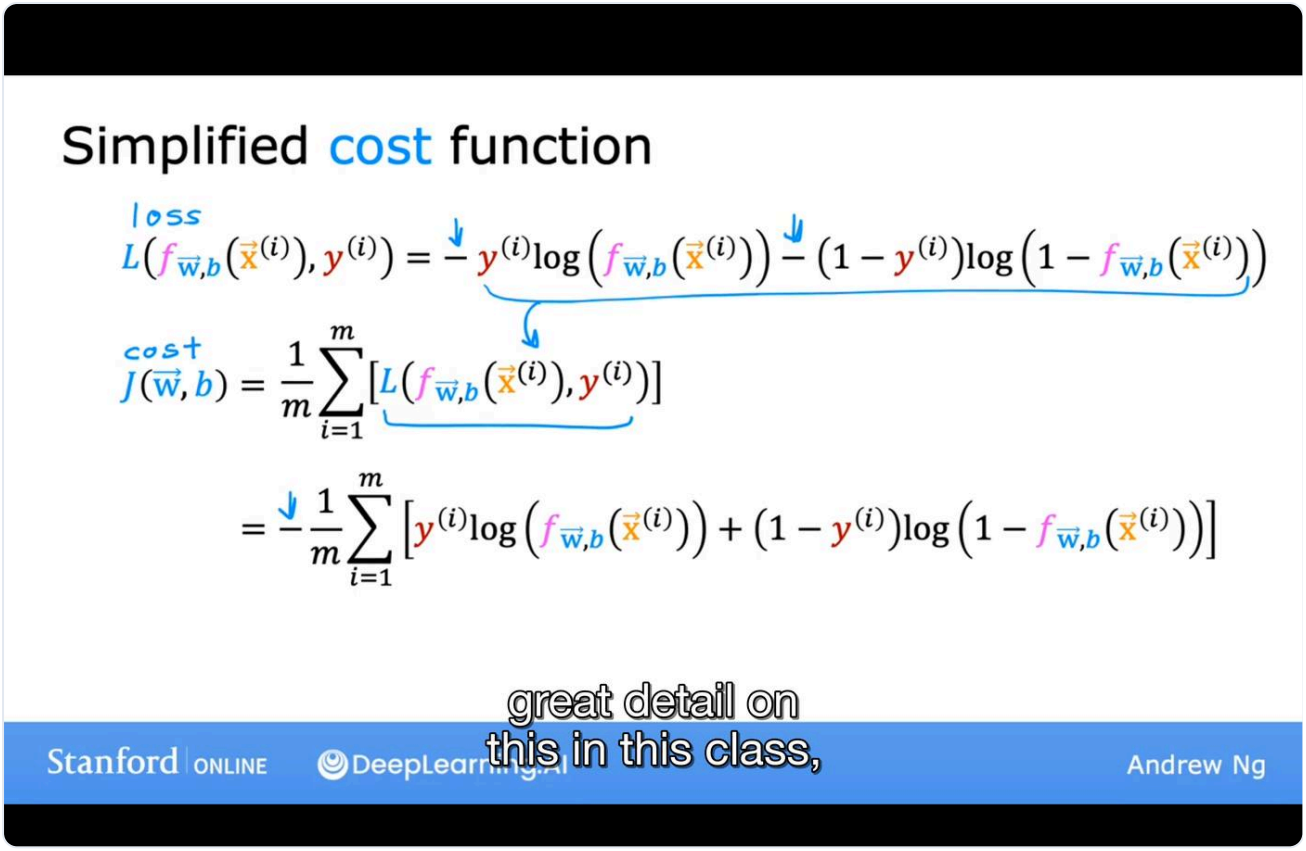


FIGURE 35.5 | Simplified Cost Function

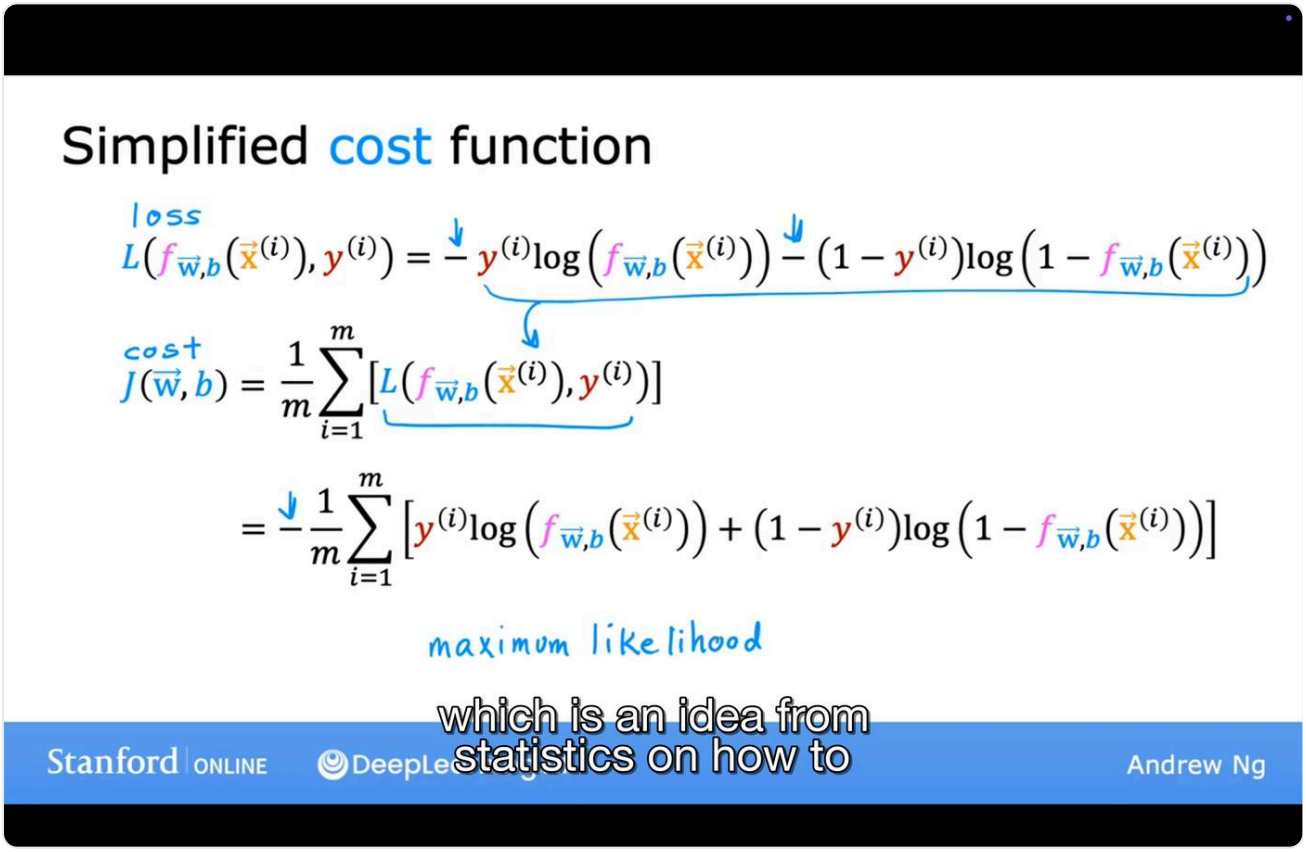


FIGURE 35.6 | Simplified Cost Function

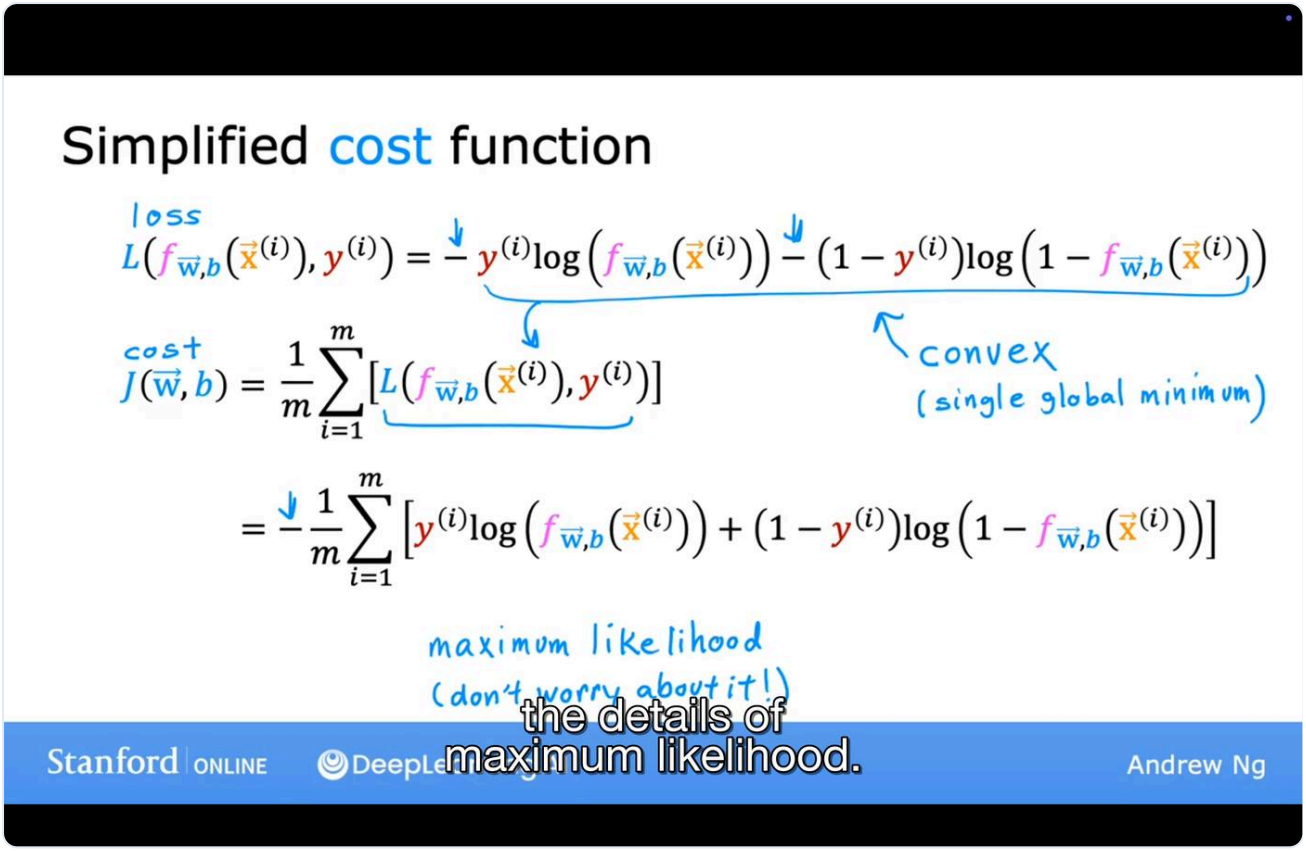


FIGURE 35.7 | Simplified Cost Function

Although we won't have time to go into great detail on this in this class, I'd just like to mention that this particular cost function is derived from statistics using a statistical principle called maximum likelihood estimation, which is an idea from statistics on how to efficiently find parameters for different models. This cost function has the nice property that it is convex. But don't worry about learning the details of maximum likelihood. It's just a deeper rationale and justification behind this particular cost function. The upcoming optional lab will show you how the logistic cost function is implemented in code. I recommend taking a look at it, because you implement this later into practice lab at the end of the week.

This upcoming optional lab also shows you how two different choices of the parameters will lead to different cost calculations. You can see in the plot that the better fitting blue decision boundary has a lower cost relative to the magenta decision boundary. So with the simplified cost function, we're now ready to jump into applying gradient descent to logistic regression. Let's go see that in the next video.

LECTURE 36

Gradient Descent

To fit the parameters of a logistic regression model, we're going to try to find the values of the parameters w and b that minimize the cost function J of w and b, and we'll again apply gradient descent to do this. Let's take a look at how. In this video we'll focus on how to find a good choice of the parameters w and b. After you've done so, if you give the model a new input, x, say a new patients at the hospital with a certain tumor size and age, then these are diagnosis.

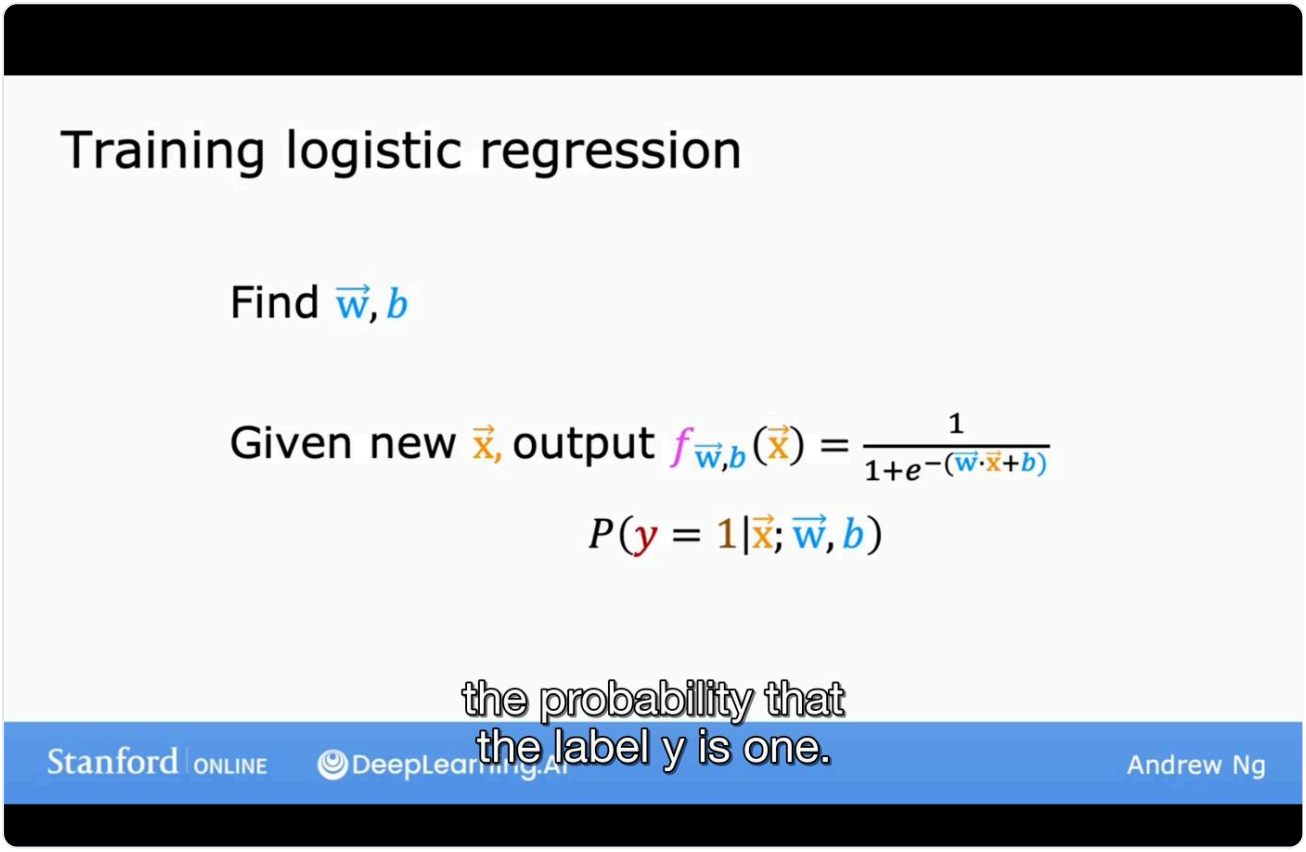


FIGURE 36.1 | Training Logistic Regression

The model can then make a prediction, or it can try to estimate the probability that the label y is one. The average you can use to minimize the cost function is gradient descent. Here again is the cost function.

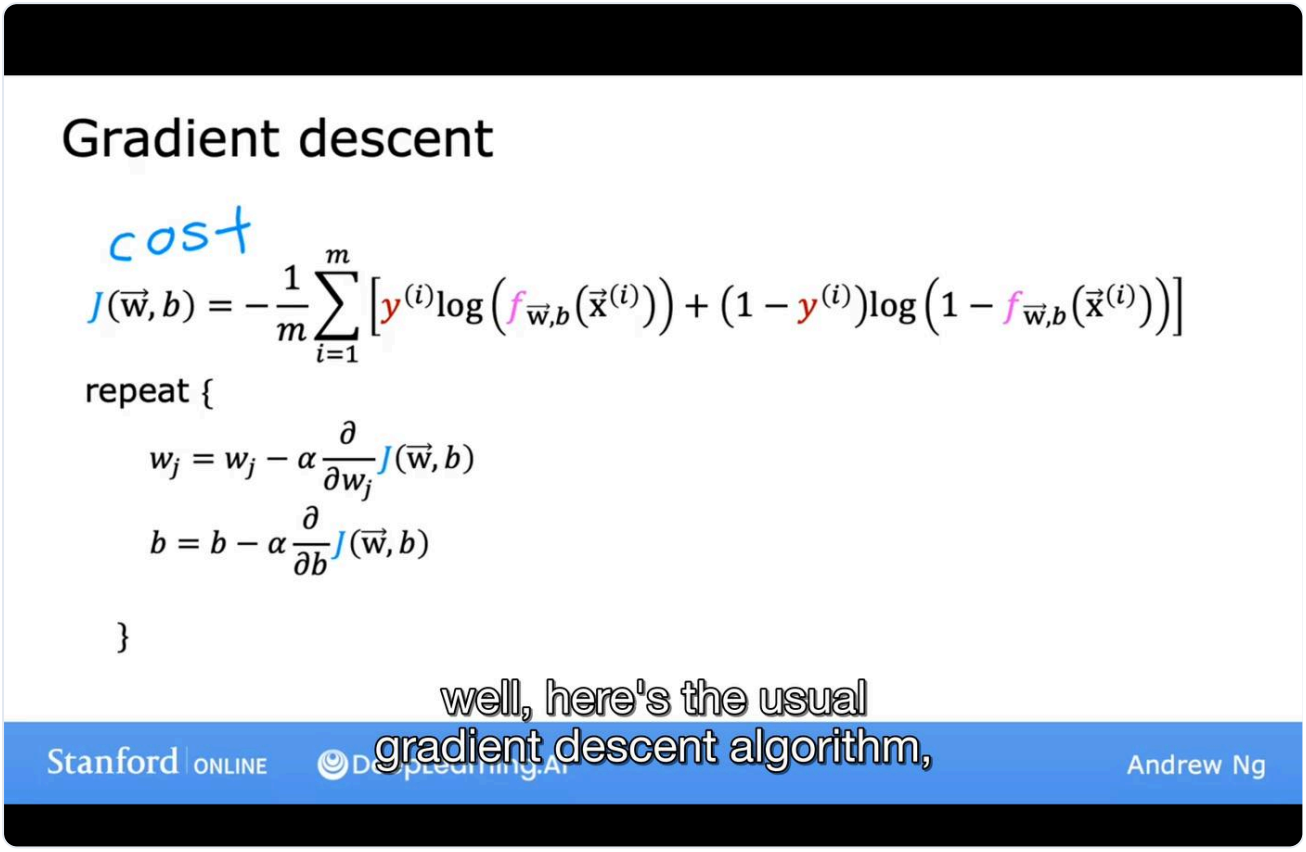


FIGURE 36.2 | Gradient Descent

If you want to minimize the cost j as a function of w and b, well, here's the usual gradient descent algorithm, where you repeatedly update each parameter as the 0 value minus Alpha, the learning rate times this derivative term. Let's take a look at the derivative of j with respect to w_j. This term up on top here, where as usual, j goes from one through n, where n is the number of features. If someone were to apply the rules of calculus, you can show that the derivative with respect to w_j of the cost function capital J is equal to this expression over here, is 1 over m times the sum from 1 through m of this error term.

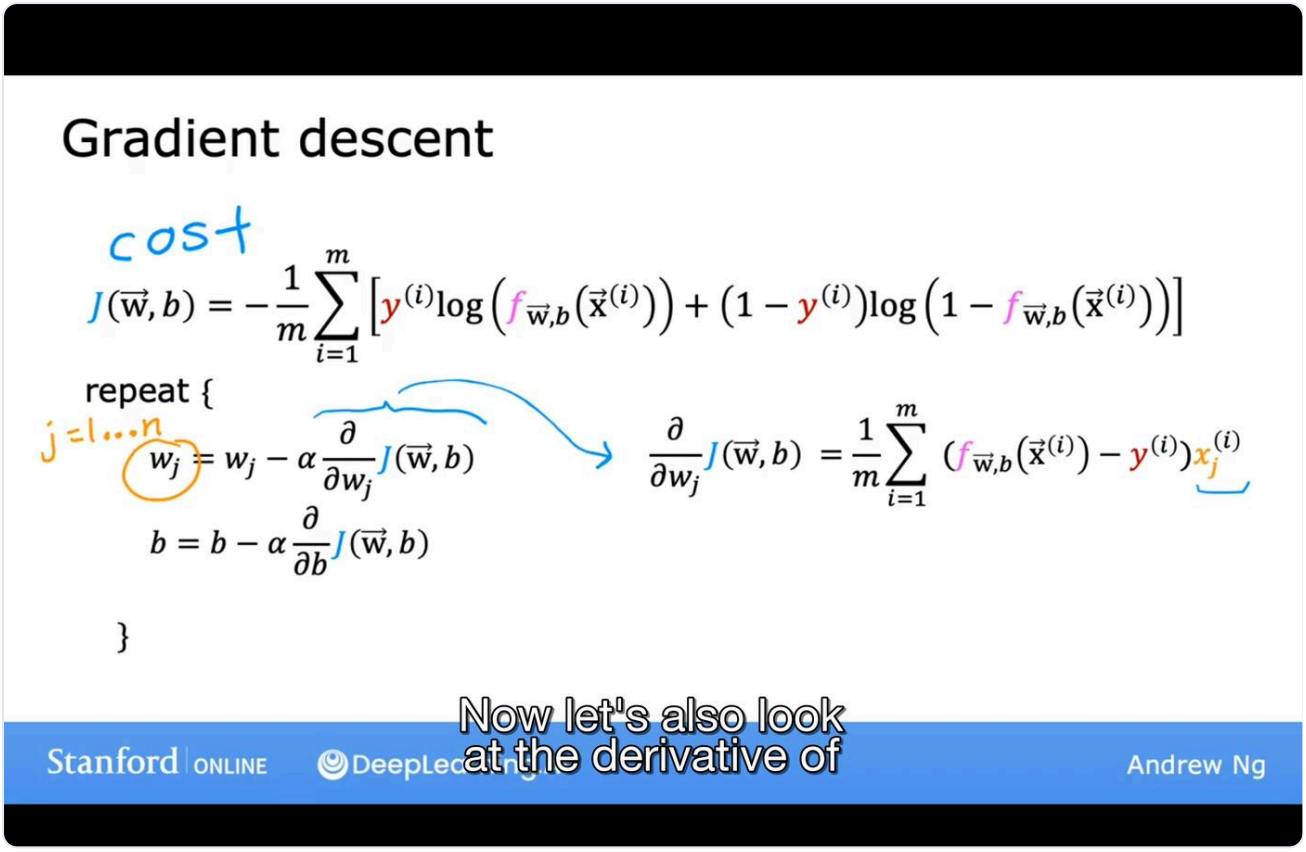


FIGURE 36.3 | Gradient Descent

That is f minus the label y times x_j. Here are just x I j is the j feature of training example i. Now let's also look at the derivative of j with respect to the parameter b. It turns out to be this expression over here.

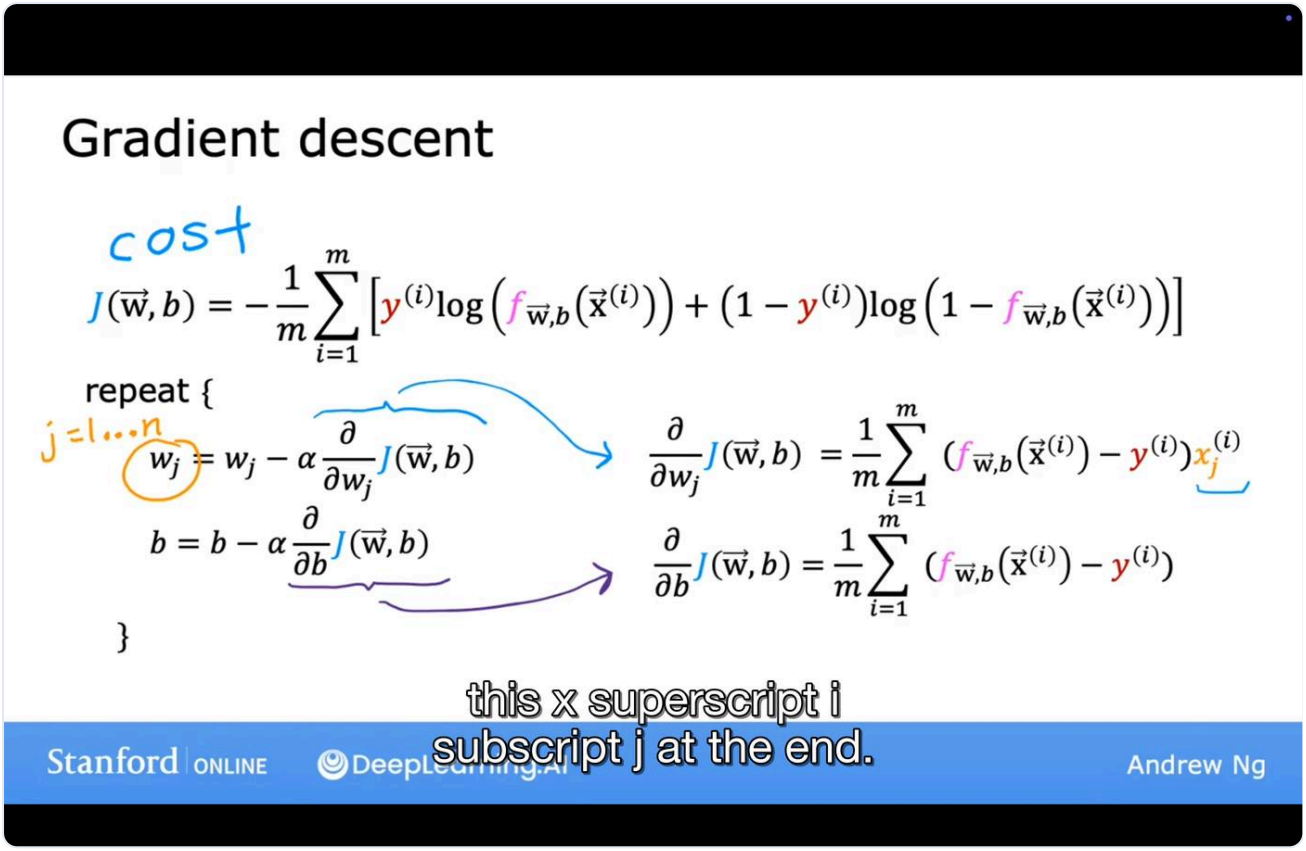


FIGURE 36.4 | Gradient Descent

It's quite similar to the expression above, except that it is not multiplied by this x superscript i subscript j at the end.

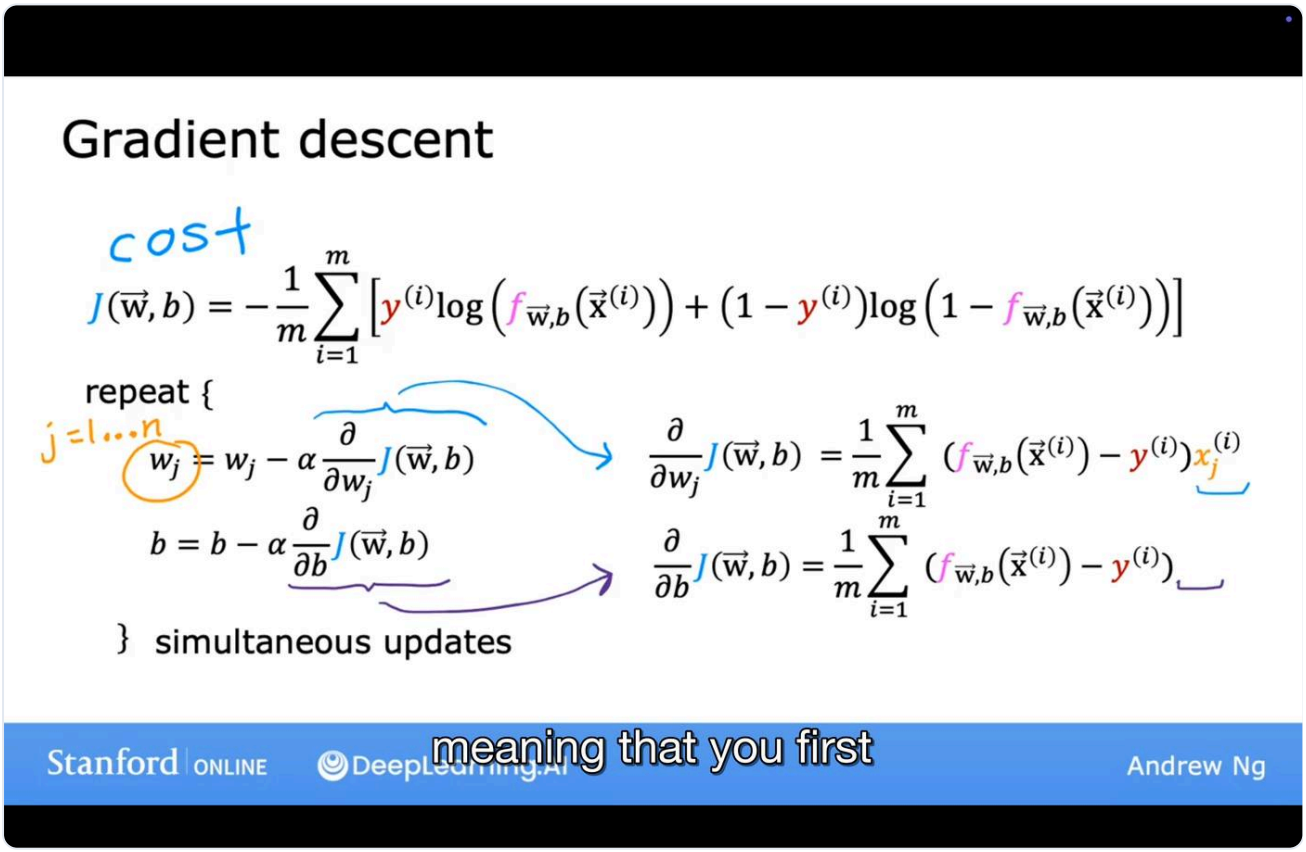


FIGURE 36.5 | Gradient Descent

Just as a reminder, similar to what you saw for linear regression, the way to carry out these updates is to use simultaneous updates, meaning that you first compute the right-hand side for all of these updates and then simultaneously overwrite all the values on the left at the same time. Let me take these derivative expressions here and plug them into these terms here. This gives you gradient descent for logistic regression. Now, one funny thing you might be wondering is, that's weird. These two equations look exactly like the average we had come up with previously for linear regression so you might be wondering, is linear regression actually secretly the same as logistic regression?

Well, even though these equations look the same, the reason that this is not linear regression is because the definition for the function f of x has changed. In linear regression, f of x is, this is wx plus b.

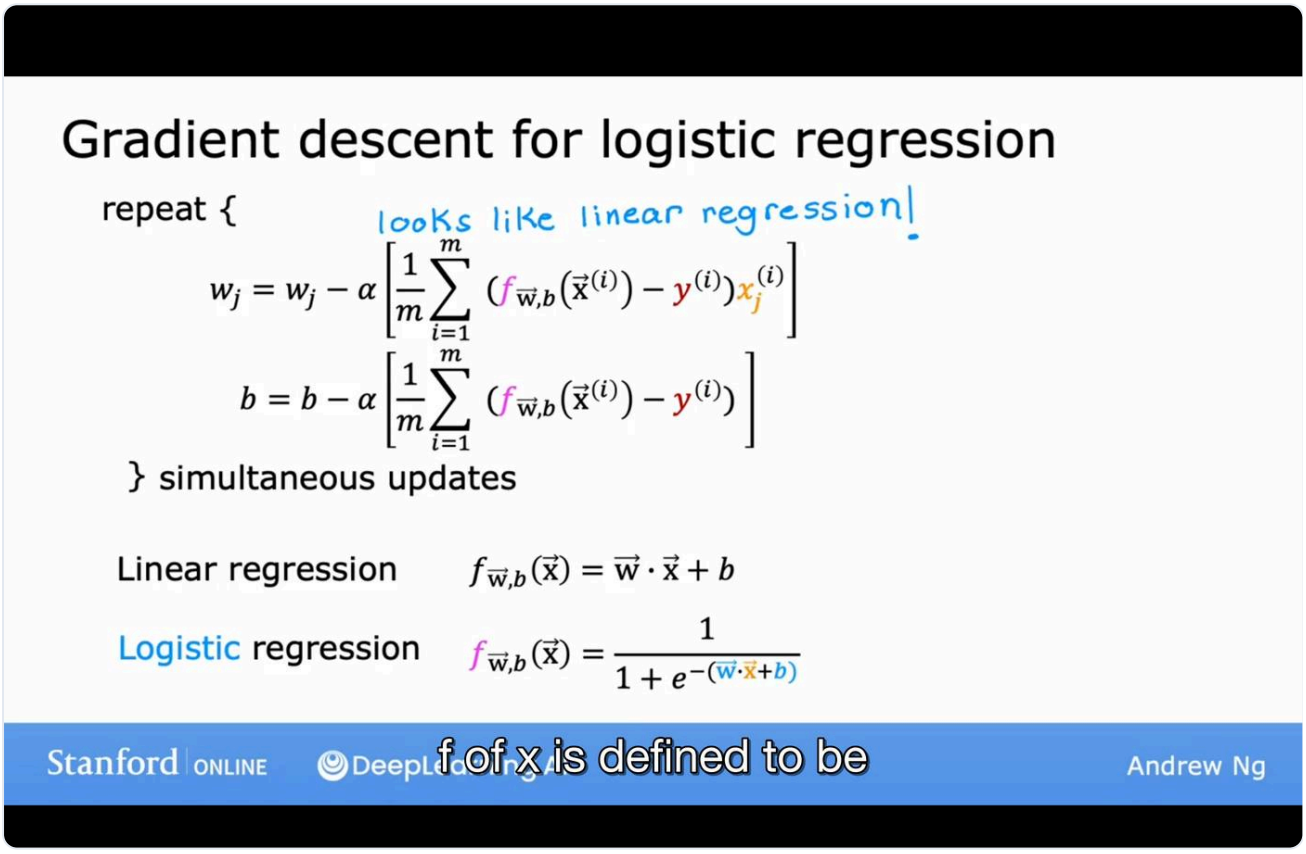


FIGURE 36.6 | Gradient Descent for Logistic Regression

But in logistic regression, f of x is defined to be the sigmoid function applied to wx plus b. Although the algorithm written looked the same for both linear regression and logistic regression, actually they're two very different algorithms because the definition for f of x is not the same. When we talked about gradient descent for linear regression previously, you saw how you can monitor a gradient descent to make sure it converges.

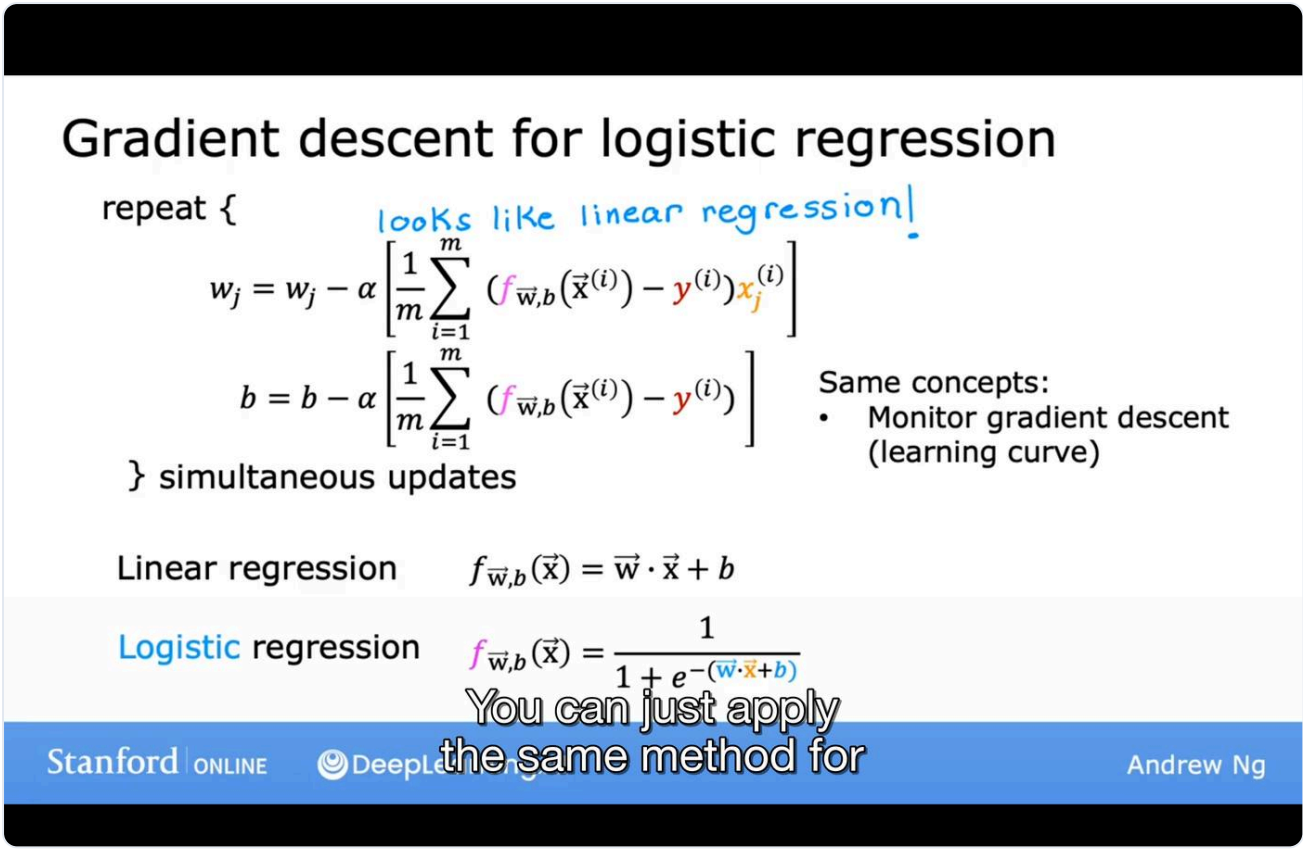


FIGURE 36.7

You can just apply the same method for logistic regression to make sure it also converges. I've written out these updates as if you're updating the parameters w_j one parameter at a time.

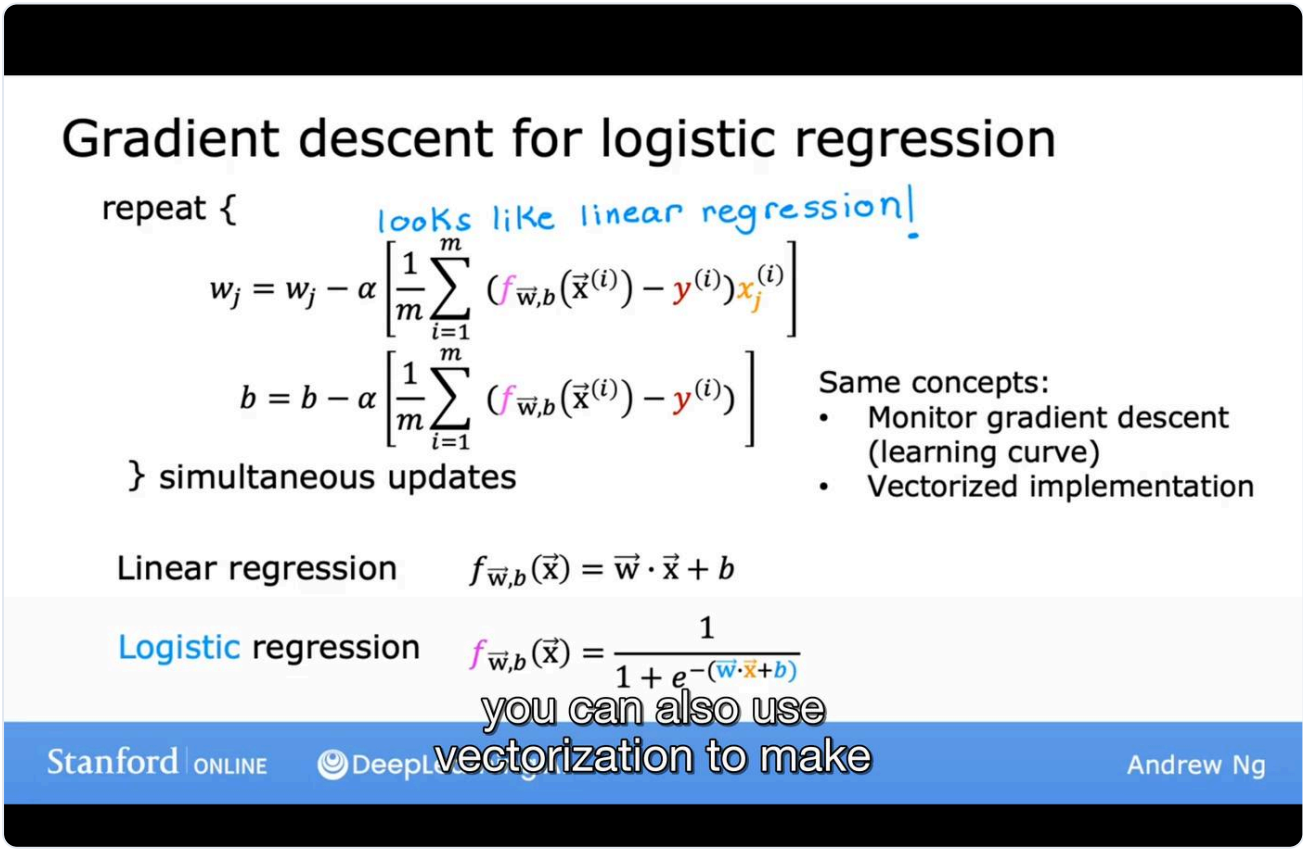


FIGURE 36.8 | Gradient Descent for Logistic Regression

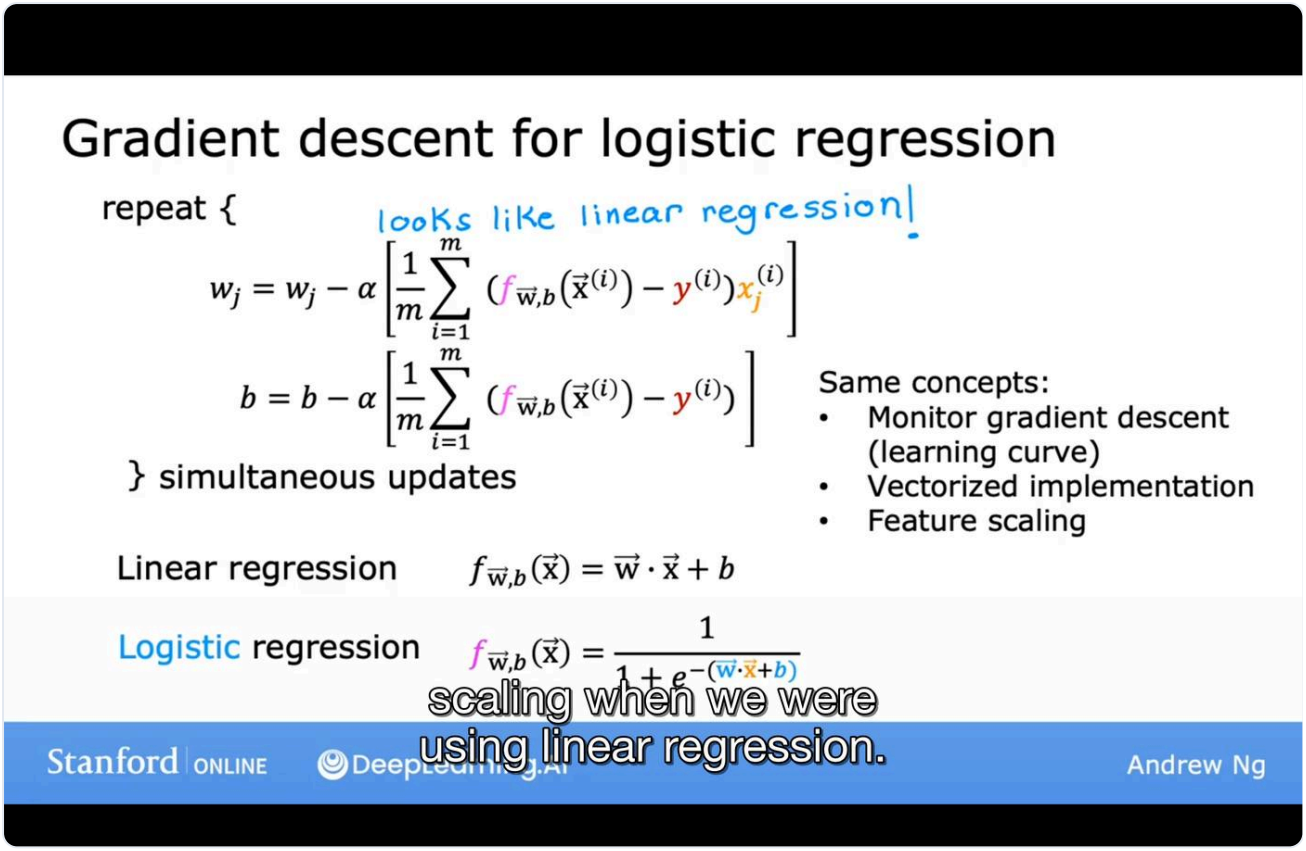


FIGURE 36.9 | Gradient Descent for Logistic Regression

Similar to the discussion on vectorized implementations of linear regression, you can also use vectorization to make gradient descent run faster for logistic regression. I won't dive into the details of the vectorized implementation in this video. But you can also learn more about it and see the code in the optional labs. Now you know how to implement gradient descent for logistic regression. You might also remember feature scaling when we were using linear regression. Where you saw how feature scaling, that is scaling all the features to take on similar ranges of values, say between negative 1 and plus 1, how they can help gradient descent to converge faster.

Feature scaling applied the same way to scale the different features to take on similar ranges of values can also speed up gradient descent for logistic regression. In the upcoming optional lab, you also see how the gradient for the logistic regression can be calculated in code. This will be useful to look at because you also implement this in the practice lab at the end of this week. After you run gradient descent in this lab, there'll be a nice set of animated plots that show gradient descent in action. You see the sigmoid function, the contour plot of the cost, the 3D surface plot of the cost, and the learning curve or evolve as gradient descent runs.

There will be another optional lab after that, which is short and sweet, but also very useful because they're showing you how to use the popular scikit-learn library to train the logistic regression model for classification. Many machine learning practitioners in many companies today use scikit-learn regularly as part of their job. I hope you check out the scikit-learn function as well and take a look at how that is used. That's it. You should now know how to implement logistic regression. This is a very powerful and very widely used learning algorithm and you now know how to get it to work yourself. Congratulations.

LECTURE 37

Regression Example

Now you've seen a couple of different learning algorithms, linear regression and logistic regression. They work well for many tasks. But sometimes in an application, the algorithm can run into a problem called overfitting, which can cause it to perform poorly. What I like to do in this video is to show you what is overfitting, as well as a closely-related, almost opposite problem called underfitting. In the next videos after this, I'll share with you some techniques for accuracy overfitting. In particular, there's a method called regularization. Very useful technique. I use it all the time. Then regularization will help you minimize this overfitting problem and get your learning algorithms to work much better.

Let's take a look at what is overfitting? To help us understand what is overfitting.

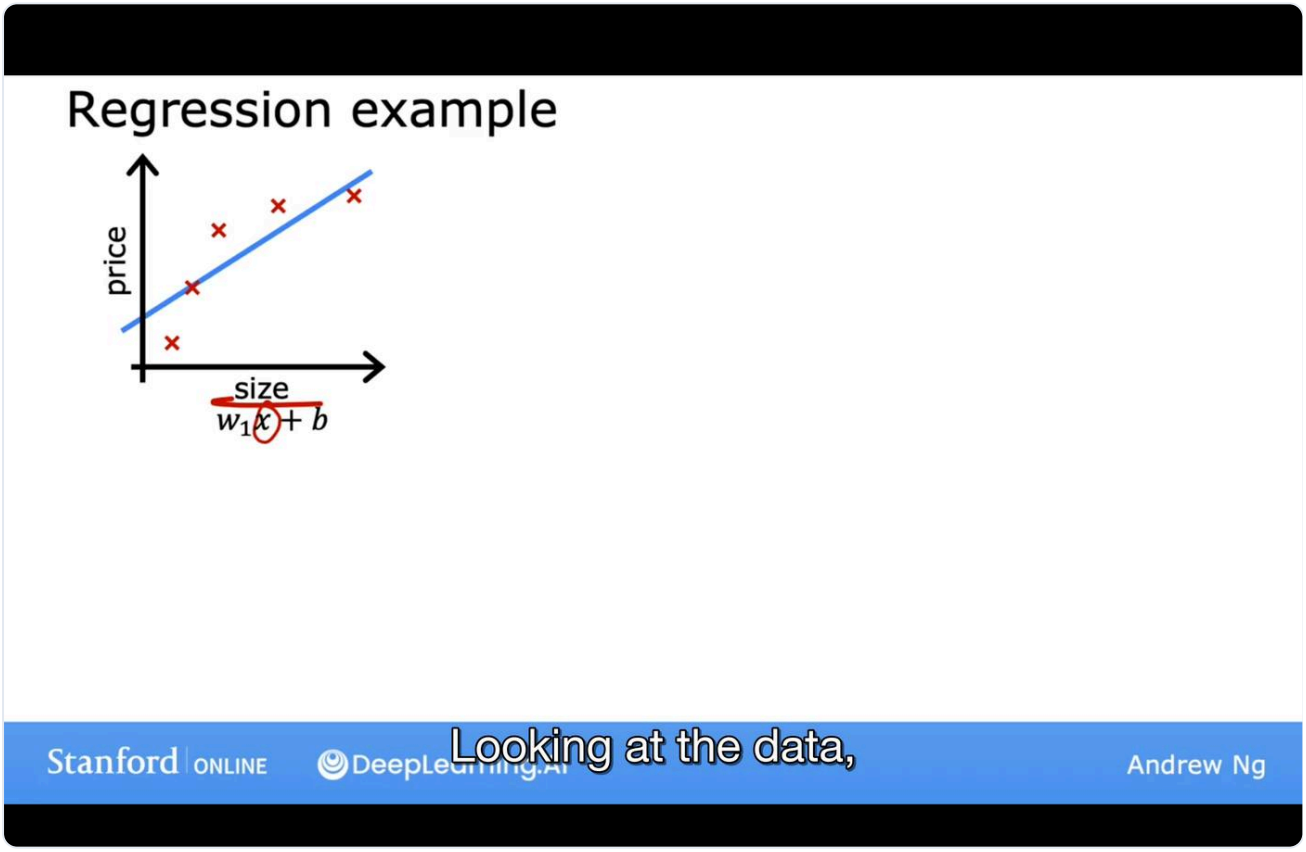


FIGURE 37.1 | Regression Example

Let's take a look at a few examples. Let's go back to our original example of predicting housing prices with linear regression. Where you want to predict the price as a function of the size of a house. To help us understand what is overfitting, let's take a look at a linear regression example. I'm going to go back to our original running example of predicting housing prices with linear regression. Suppose your data-set looks like this, with the input feature x being the size of the house, and the value, y that you're trying to predict the price of the house. One thing you could do is fit a linear function to this data. If you do that, you get a straight line fit to the data that maybe looks like this.

But this isn't a very good model. Looking at the data, it seems pretty clear that as the size of the house increases, the housing process flattened out. This algorithm does not fit the training data very well.

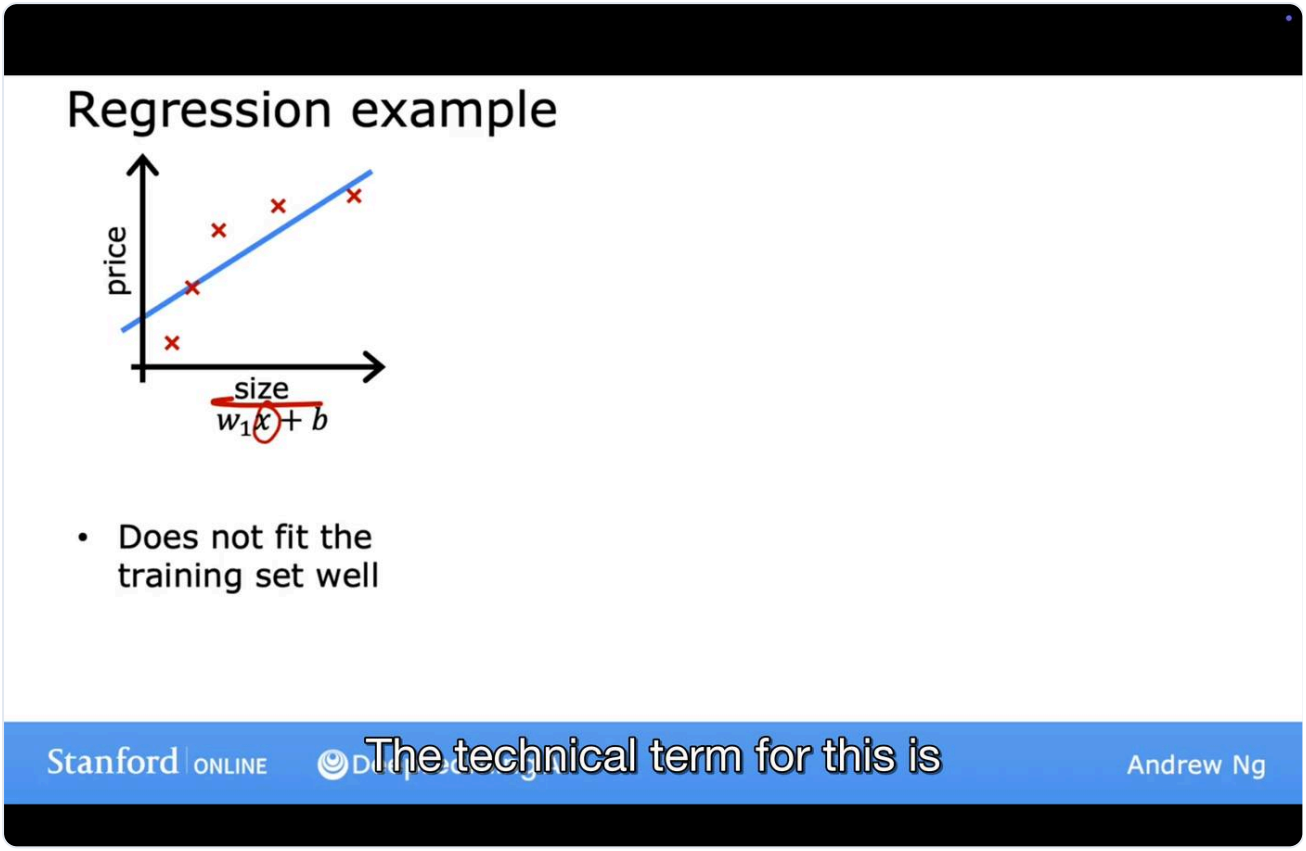


FIGURE 37.2 | Regression Example

The technical term for this is the model is underfitting the training data.

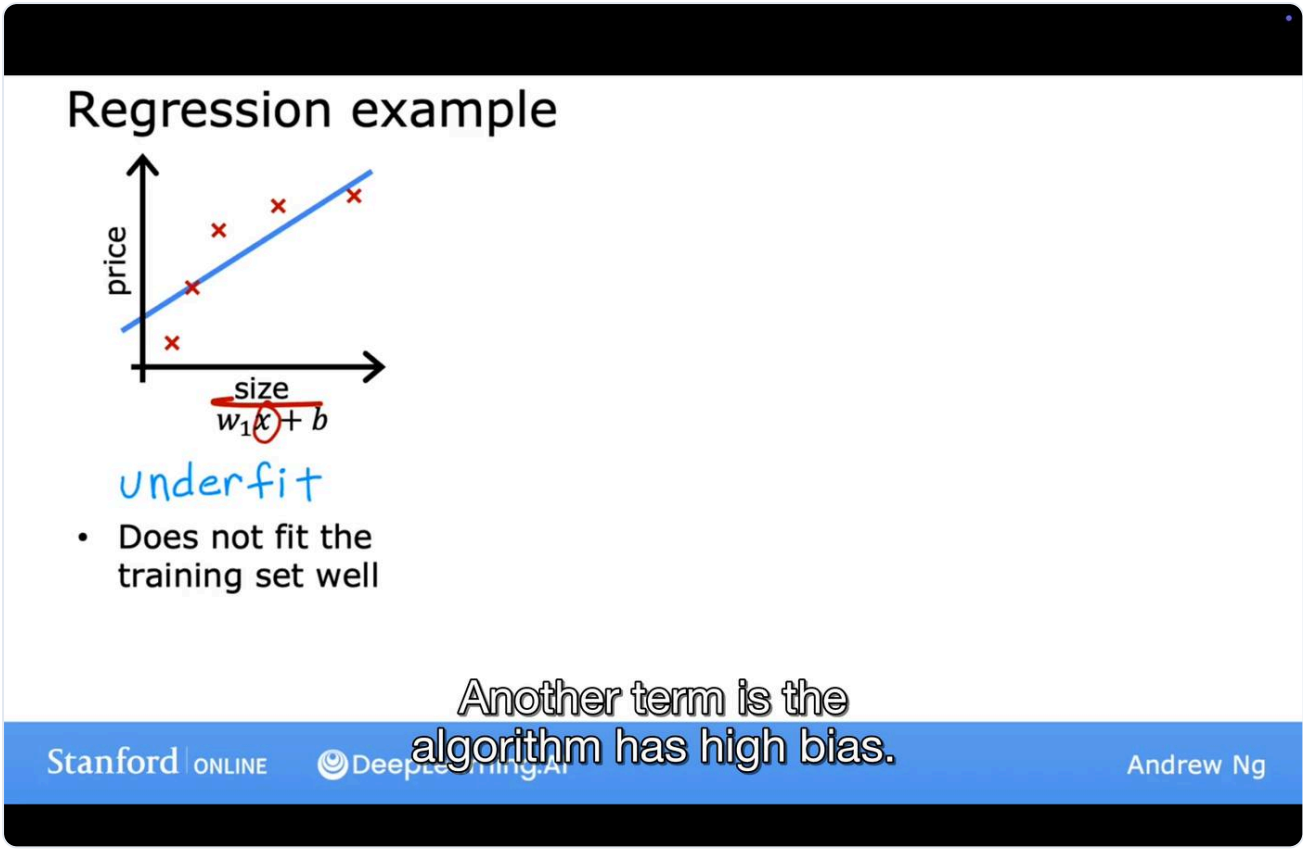


FIGURE 37.3 | Regression Example

Another term is the algorithm has high bias. You may have read in the news about some learning algorithms really, unfortunately, demonstrating bias against certain ethnicities or certain genders. In machine learning, the term bias has multiple meanings. Checking learning algorithms for bias based on characteristics such as gender or ethnicity is absolutely critical.

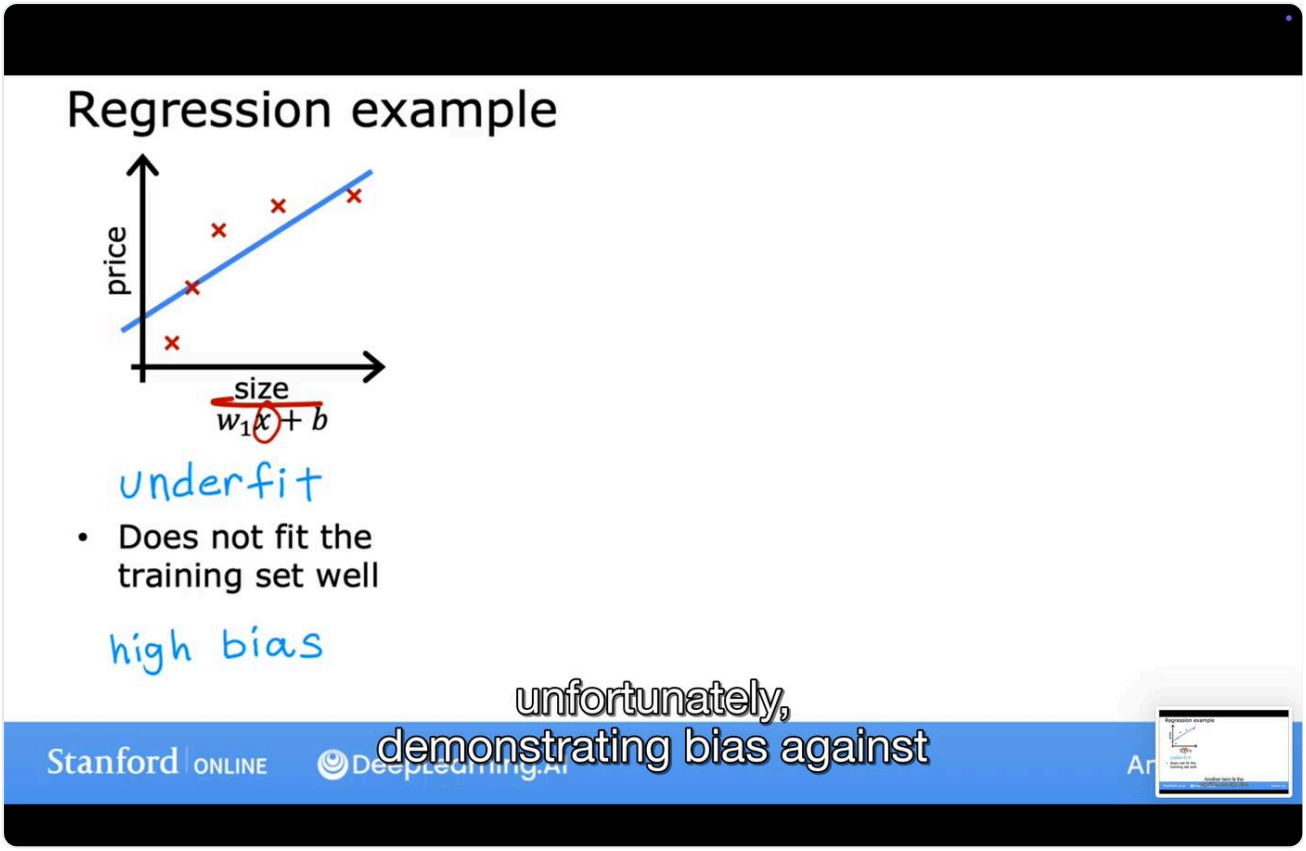


FIGURE 37.4 | Regression Example

But the term bias has a second technical meaning as well, which is the one I'm using here, which is if the algorithm has underfit the data, meaning that it's just not even able to fit the training set that well. There's a clear pattern in the training data that the algorithm is just unable to capture. Another way to think of this form of bias is as if the learning algorithm has a very strong preconception, or we say a very strong bias, that the housing prices are going to be a completely linear function of the size despite data to the contrary. This preconception that the data is linear causes it to fit a straight line that fits the data poorly, leading it to underfitted data.

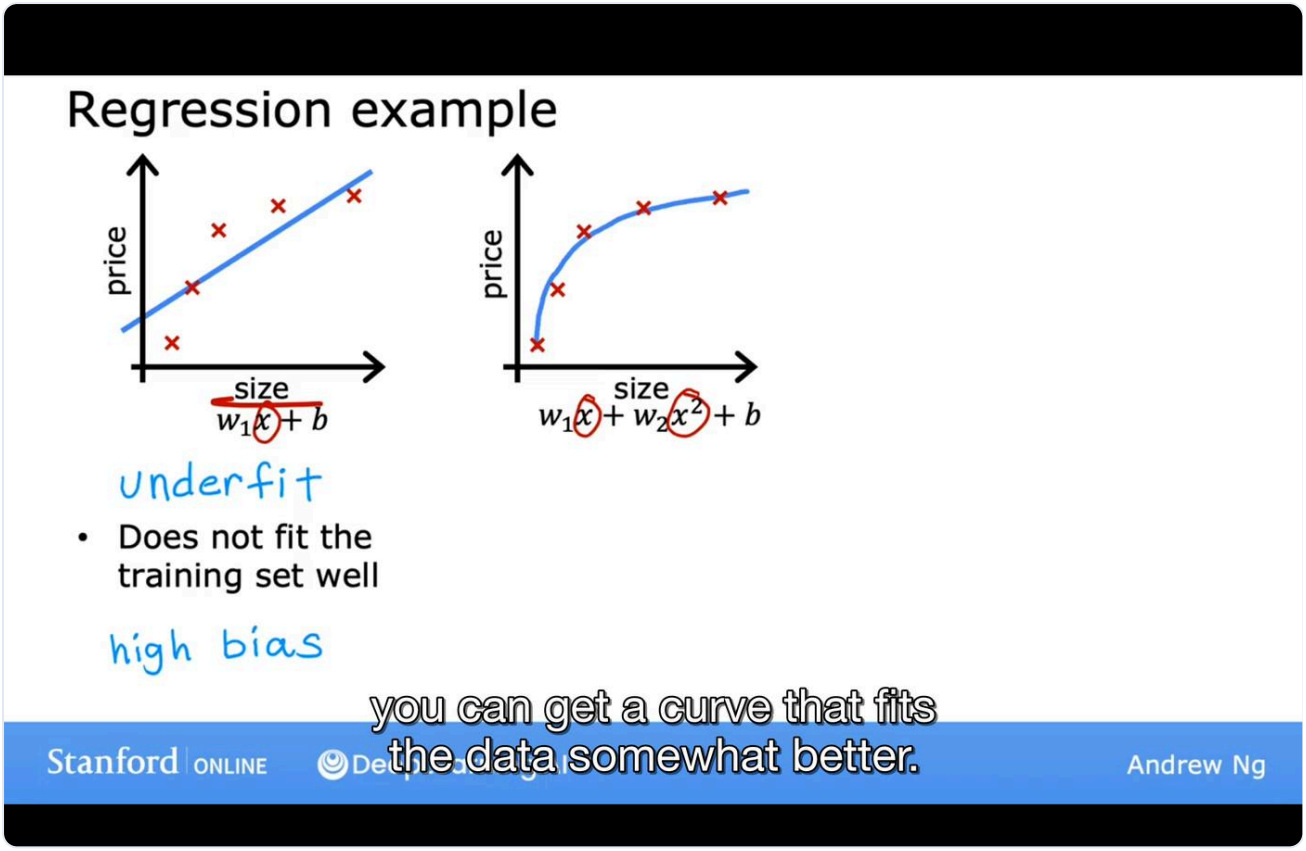


FIGURE 37.5 | Regression Example

Now, let's look at a second variation of a model, which is if you insert for a quadratic function at the data with two features, x and x^2, then when you fit the parameters W1 and W2, you can get a curve that fits the data somewhat better. Maybe it looks like this. Also, if you were to get a new house, that's not in this set of five training examples. This model would probably do quite well on that new house. If you're real estate agents, the idea that you want your learning algorithm to do well, even on examples that are not on the training set, that's called generalization. Technically we say that you want your learning algorithm to generalize well, which means to make good predictions even on brand new examples that it has never seen before.

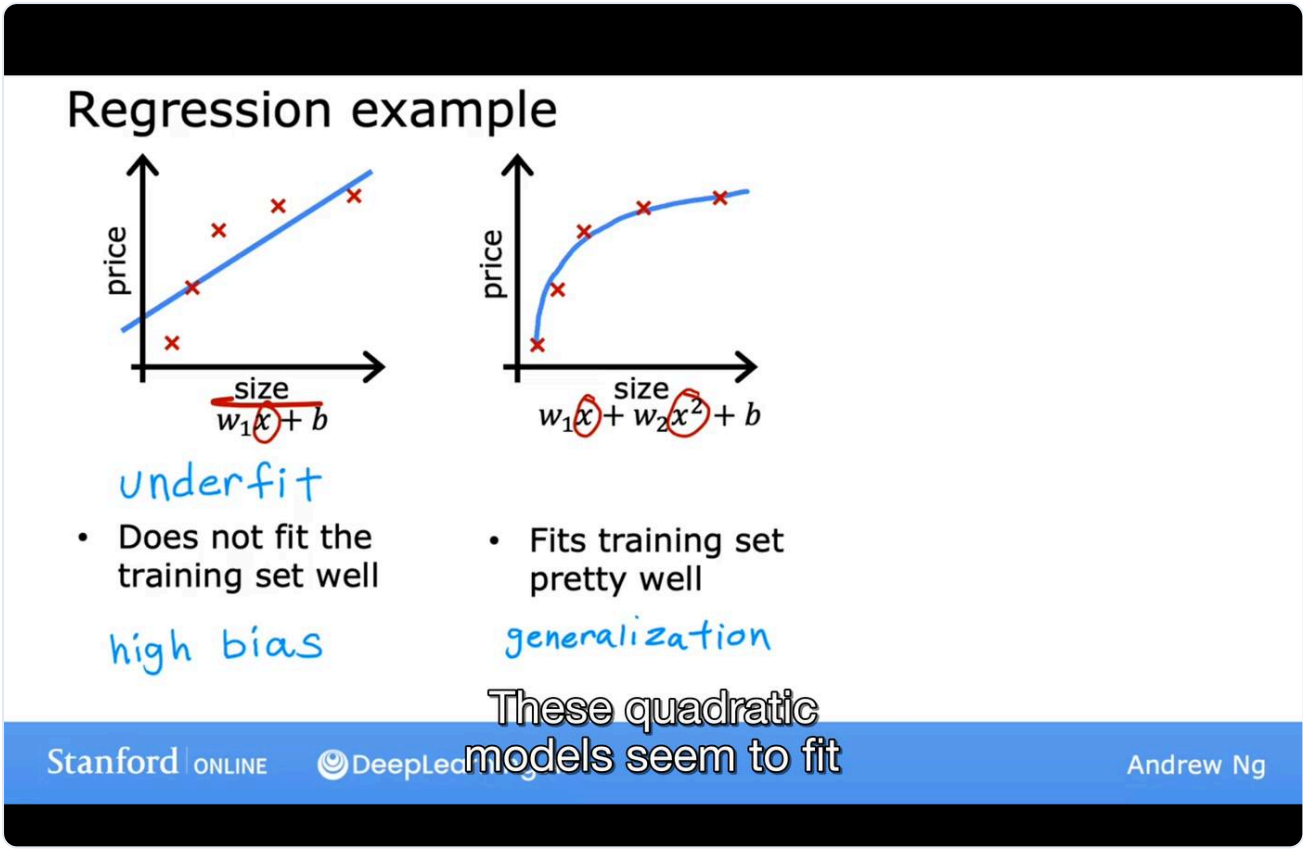


FIGURE 37.6 | Regression Example

These quadratic models seem to fit the training set not perfectly, but pretty well. I think it would generalize well to new examples. Now let's look at the other extreme. What if you were to fit a fourth-order polynomial to the data? You have x, x^2, x^3, and x^4 all as features. With this fourth for the polynomial, you can actually fit the curve that passes through all five of the training examples exactly. You might get a curve that looks like this. This, on one hand, seems to do an extremely good job fitting the training data because it passes through all of the training data perfectly.

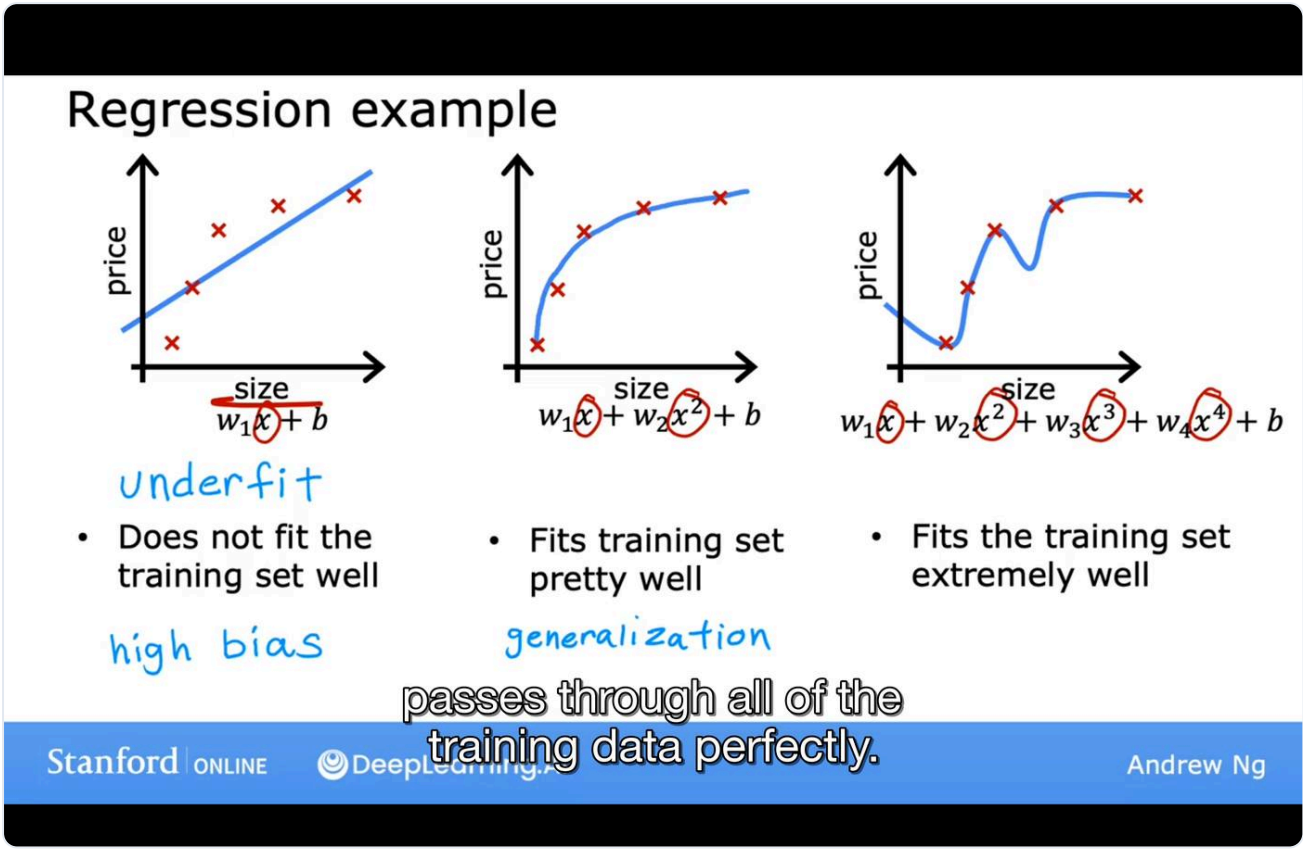


FIGURE 37.7 | Regression Example

In fact, you'd be able to choose parameters that will result in the cost function being exactly equal to zero because the errors are zero on all five training examples. But this is a very wiggly curve, its going up and down all over the place. If you have this whole size right here, the model would predict that this house is cheaper than houses that are smaller than it. We don't think that this is a particularly good model for predicting housing prices. The technical term is that we'll say this model has overfit the data, or this model has an overfitting problem. Because even though it fits the training set very well, it has fit the data almost too well, hence is overfit.

It does not look like this model will generalize to new examples that's never seen before. Another term for this is that the algorithm has high variance.

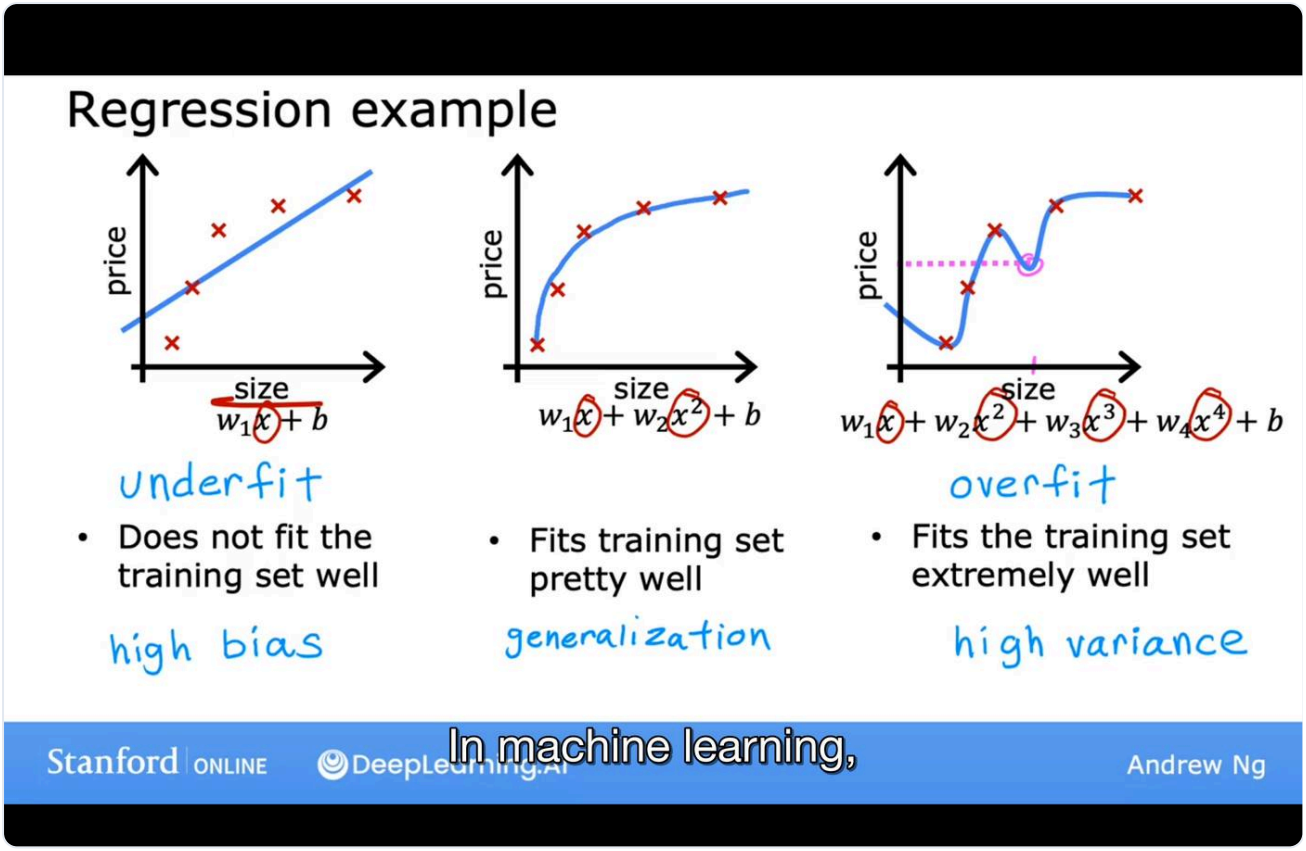


FIGURE 37.8 | Regression Example

In machine learning, many people will use the terms over-fit and high-variance almost interchangeably. We'll use the terms underfit and high bias almost interchangeably. The intuition behind overfitting or high-variance is that the algorithm is trying very hard to fit every single training example. It turns out that if your training set were just even a little bit different, say one holes was priced just a little bit more little bit less, then the function that the algorithm fits could end up being totally different. If two different machine learning engineers were to fit this fourth-order polynomial model, to just slightly different datasets, they couldn't end up with totally different predictions or highly variable predictions.

That's why we say the algorithm has high variance. Contrasting this rightmost model with the one in the middle for the same house, it seems, the middle model gives them much more reasonable prediction for price. There isn't really a name for this case in the middle, but I'm just going to call this just right, because it is neither underfit nor overfit.

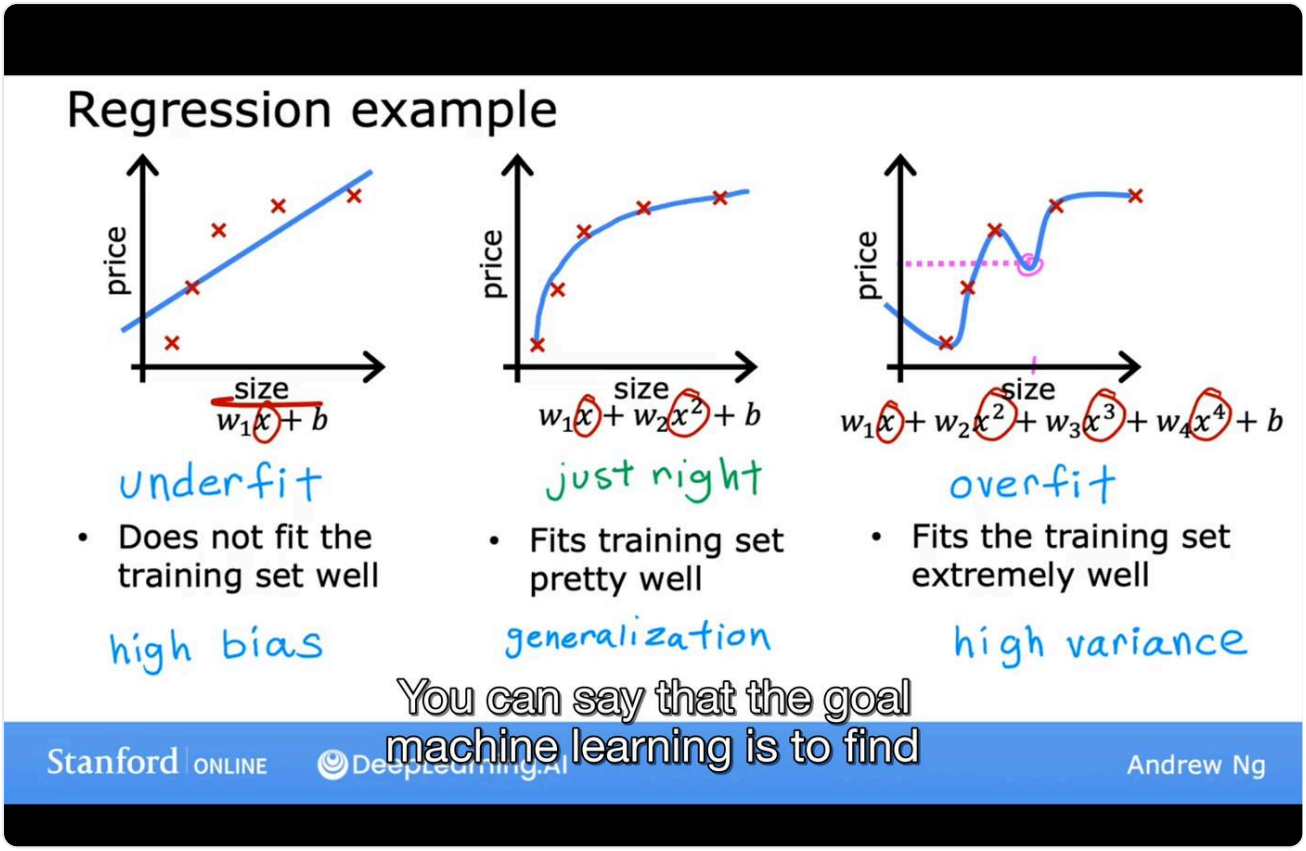


FIGURE 37.9 | Regression Example

You can say that the goal machine learning is to find a model that hopefully is neither underfitting nor overfitting. In other words, hopefully, a model that has neither high bias nor high variance. When I think about underfitting and overfitting, high bias and high variance. I'm sometimes reminded of the children's story of Goldilocks and the Three Bears in this children's tale, a girl called Goldilocks visits the home of a bear family. There's a bowl of porridge that's too cold to taste and so that's no good. There's also a bowl of porridge that's too hot to eat. That's no good either. But there's a bowl of porridge that is neither too cold nor too hot.

The temperature is in the middle, which is just right to eat. To recap, if you have too many features like the fourth-order polynomial on the right, then the model may fit the training set well, but almost too well or overfit and have high variance.

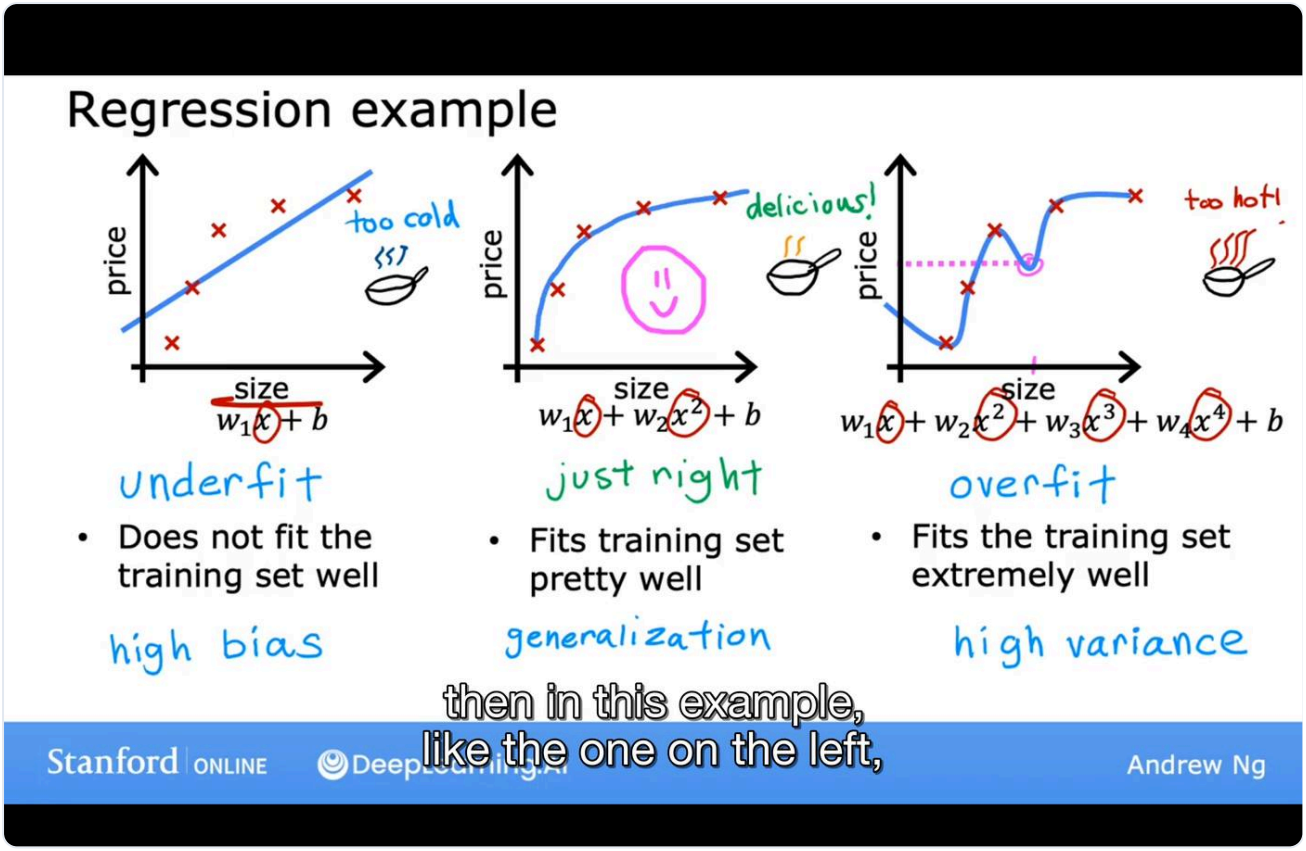


FIGURE 37.10 | Regression Example

On the flip side if you have too few features, then in this example, like the one on the left, it underfits and has high bias. In this example, using quadratic features x and x squared, that seems to be just right. So far we've looked at underfitting and overfitting for linear regression model. Similarly, overfitting applies a classification as well. Here's a classification example with two features, x_1 and x_2, where x_1 is maybe the tumor size and x_2 is the age of patient. We're trying to classify if a tumor is malignant or benign, as denoted by these crosses and circles, one thing you could do is fit a logistic regression model.

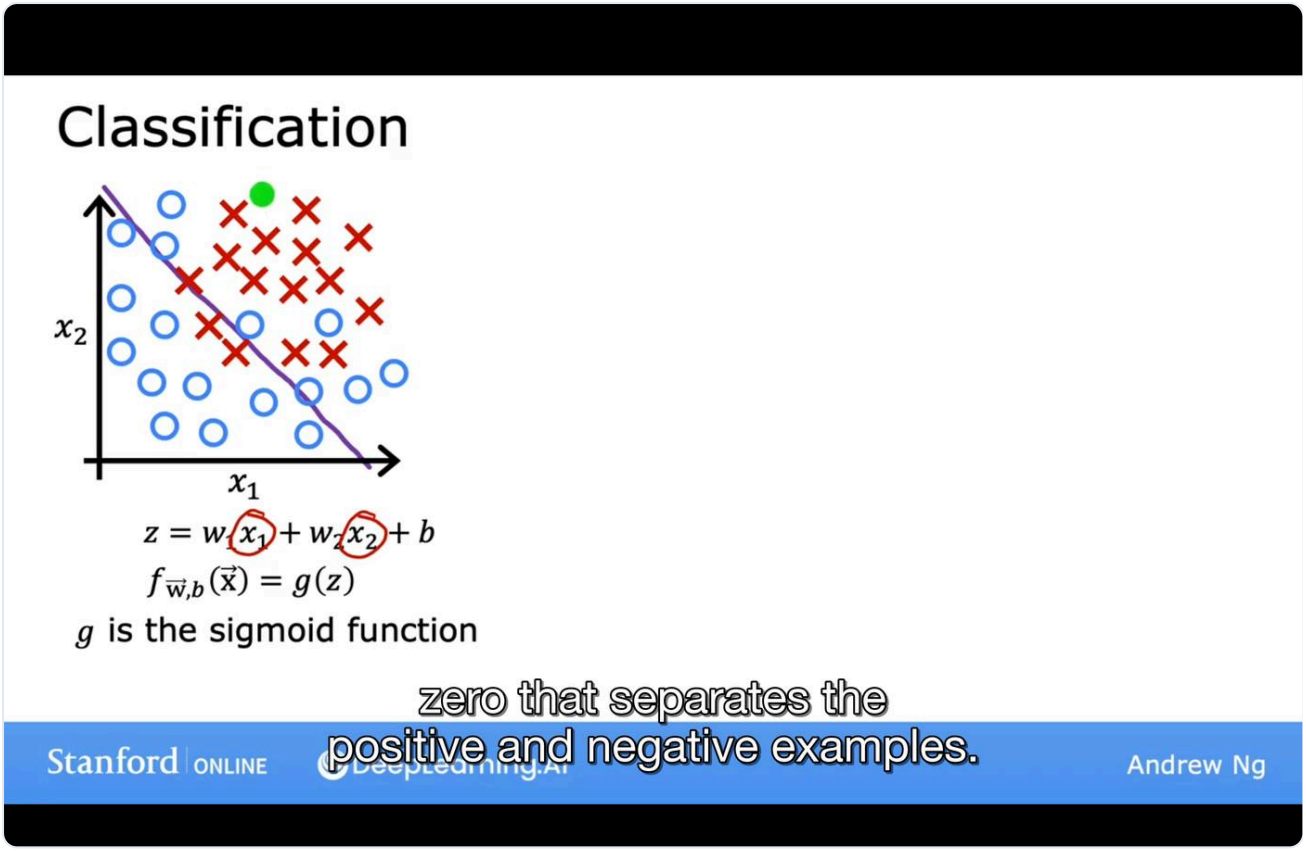


FIGURE 37.11

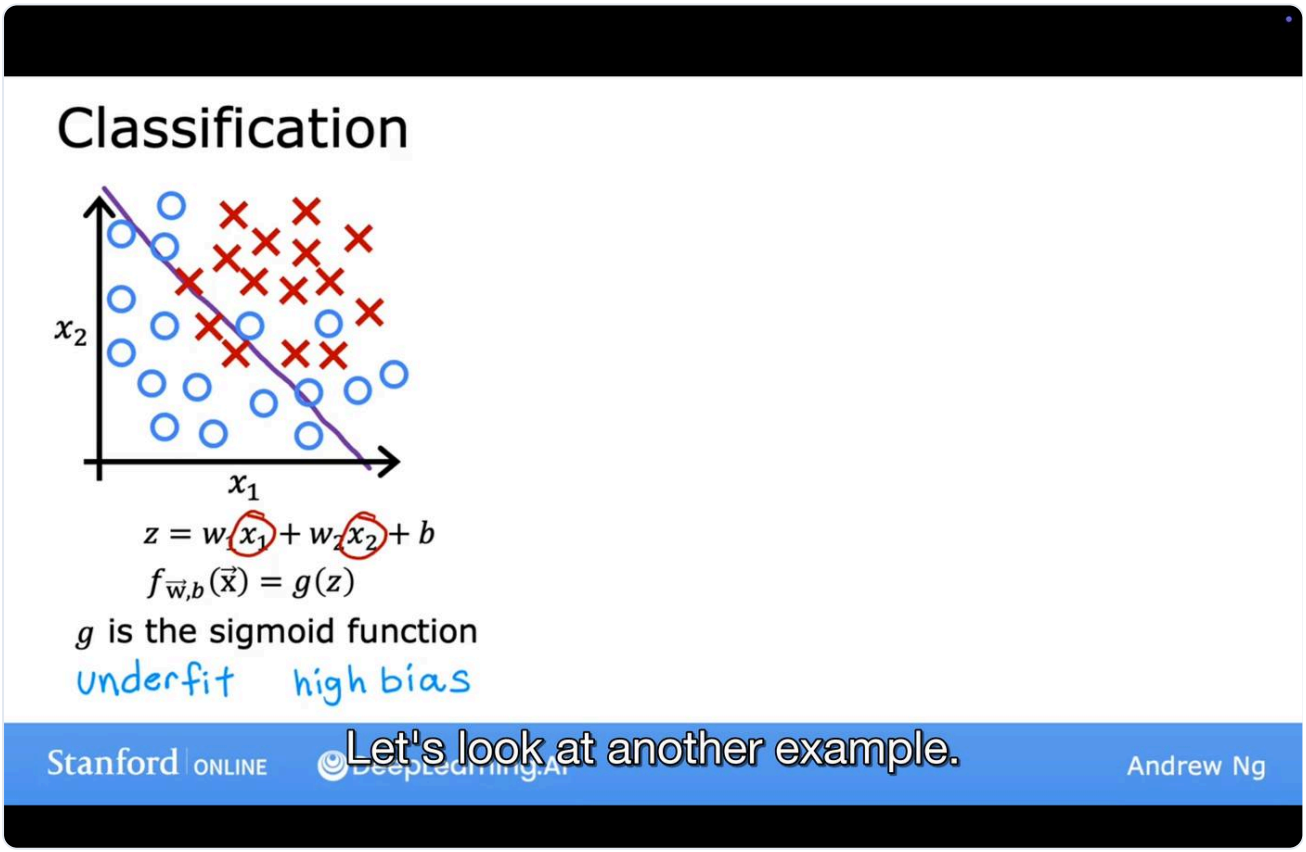


FIGURE 37.12

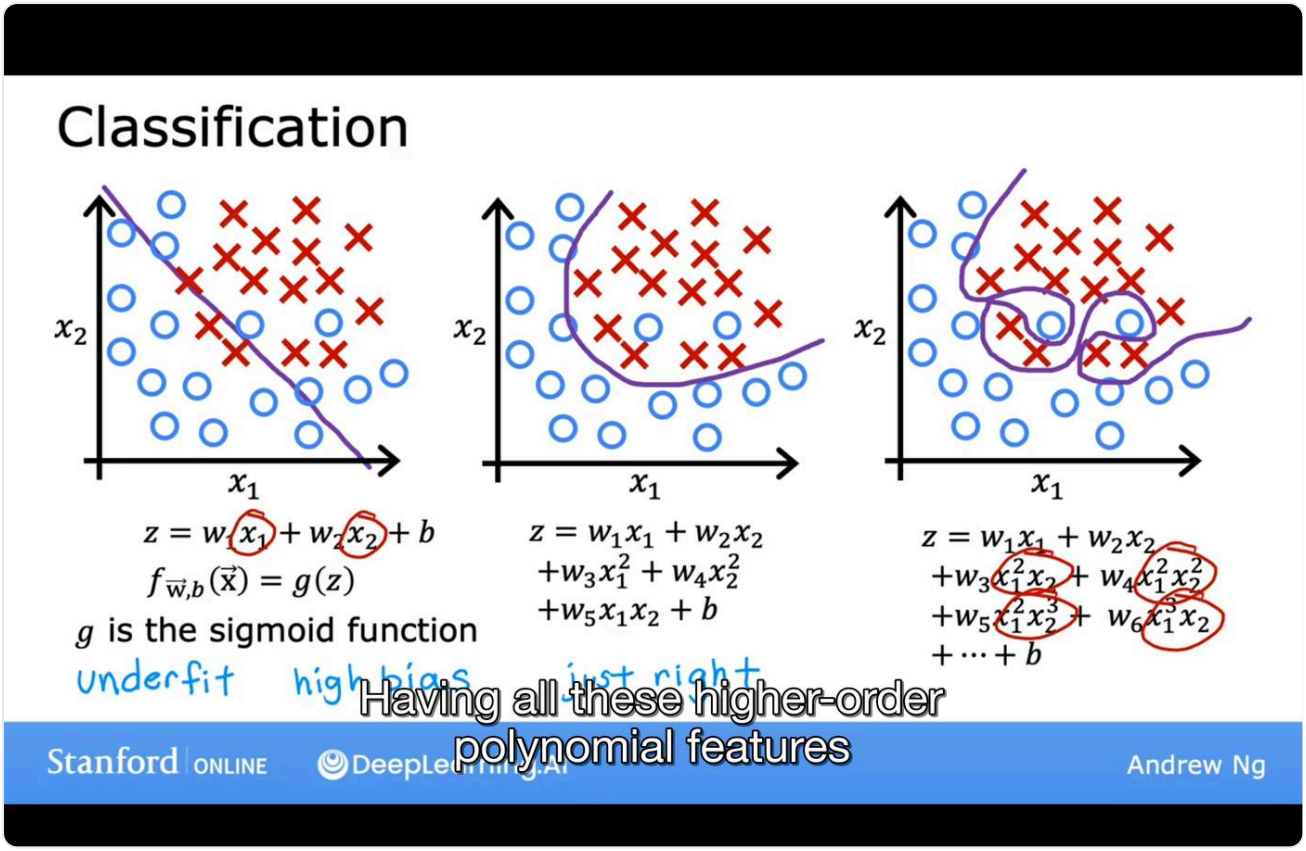


FIGURE 37.13

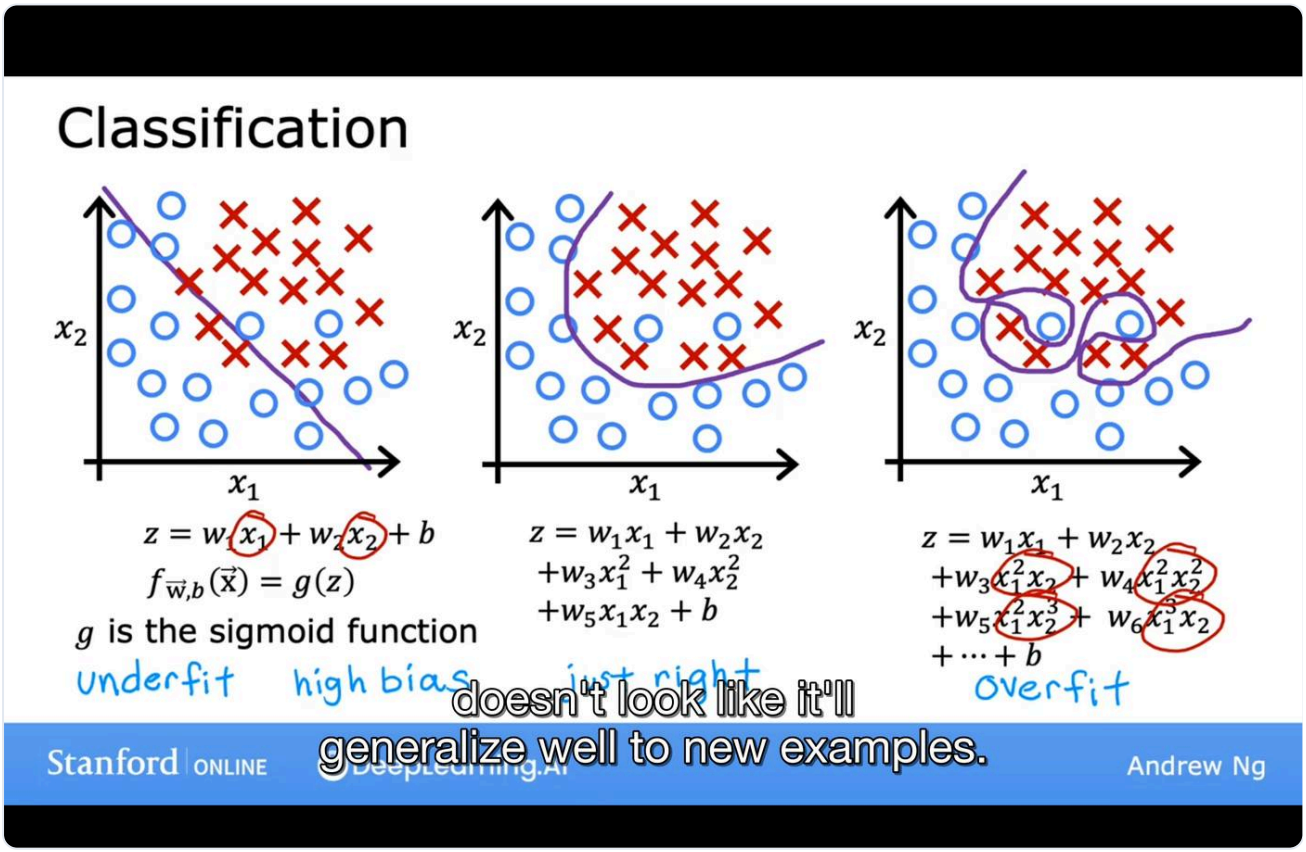


FIGURE 37.14

Just a simple model like this, where as usual, g is the sigmoid function and this term here inside is z. If you do that, you end up with a straight line as the decision boundary. This is the line where z is equal to zero that separates the positive and negative examples. This straight line doesn't look terrible. It looks okay, but it doesn't look like a very good fit to the data either. This is an example of underfitting or of high bias. Let's look at another example. If you were to add to your features these quadratic terms, then z becomes this new term in the middle and the decision boundary, that is where z equals zero can look more like this, more like an ellipse or part of an ellipse.

This is a pretty good fit to the data, even though it does not perfectly classify every single training example in the training set. Notice how some of these crosses get classified among the circles. But this model looks pretty good. I'm going to call it just right. It looks like this generalized pretty well to new patients. Finally, at the other extreme, if you were to fit a very high-order polynomial with many features like these, then the model may try really hard and contoured or twist itself to find a decision boundary that fits your training data perfectly. Having all these higher-order polynomial features allows the algorithm to choose this really over the complex decision boundary.

If the features are tumor size in age, and you're trying to classify tumors as malignant or benign, then this doesn't really look like a very good model for making predictions. Once again, this is an instance of overfitting and high variance because its model, despite doing very well on the training set, doesn't look like it'll generalize well to new examples. Now you've seen how an algorithm can underfit or have high bias or overfit and have high variance. You may want to know how you can give get a model that is just right. In the next video, we'll look at some ways you can address the issue of overfitting. We'll also touch on some ideas relevant for using underfitting.

Let's go on to the next video.

LECTURE 38

Select Features to Include/exclude

Later in this specialization, we'll talk about debugging and diagnosing things that can go wrong with learning algorithms. You'll also learn about specific tools to recognize when overfitting and underfitting may be occurring. But for now, when you think overfitting has occurred, lets talk about what you can do to address it. Let's say you fit a model and it has high variance, is overfit. Here's our overfit house price prediction model. One way to address this problem is to collect more training data, that's one option.

FIGURE 38.1 | Collect More Training Examples

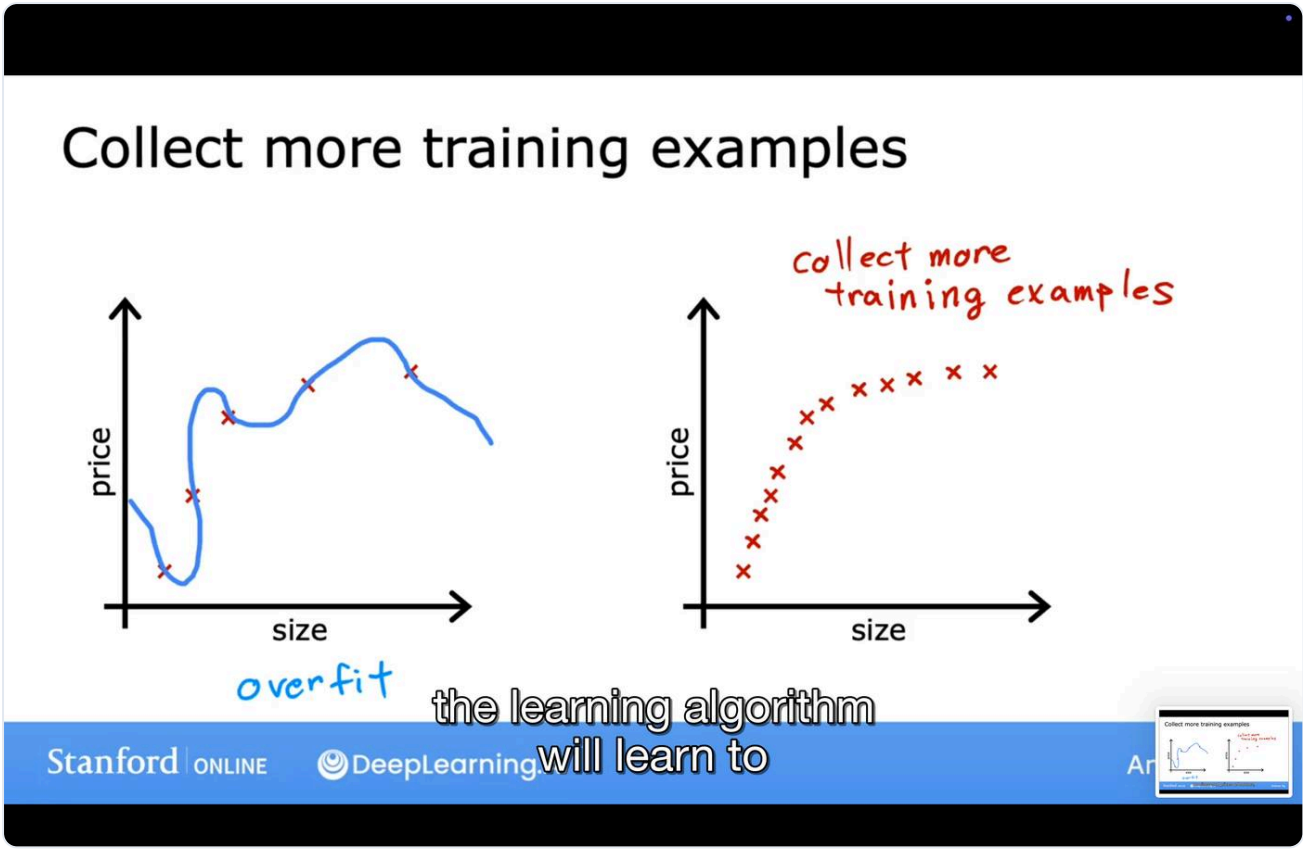


FIGURE 38.2 | Collect More Training Examples

If you're able to get more data, that is more training examples on sizes and prices of houses, then with the larger training set, the learning algorithm will learn to fit a function that is less wiggly. You can continue to fit a high order polynomial or some of the function with a lot of features, and if you have enough training examples, it will still do okay. To summarize, the number one tool you can use against overfitting is to get more training data. Now, getting more data isn't always an option. Maybe only so many houses have been sold in this location, so maybe there just isn't more data to be add. But when the data is available, this can work really well.

A second option for addressing overfitting is to see if you can use fewer features. In the previous video, our models features included the size x, as well as the size squared, and this x squared, and x cubed and x^4 and so on.

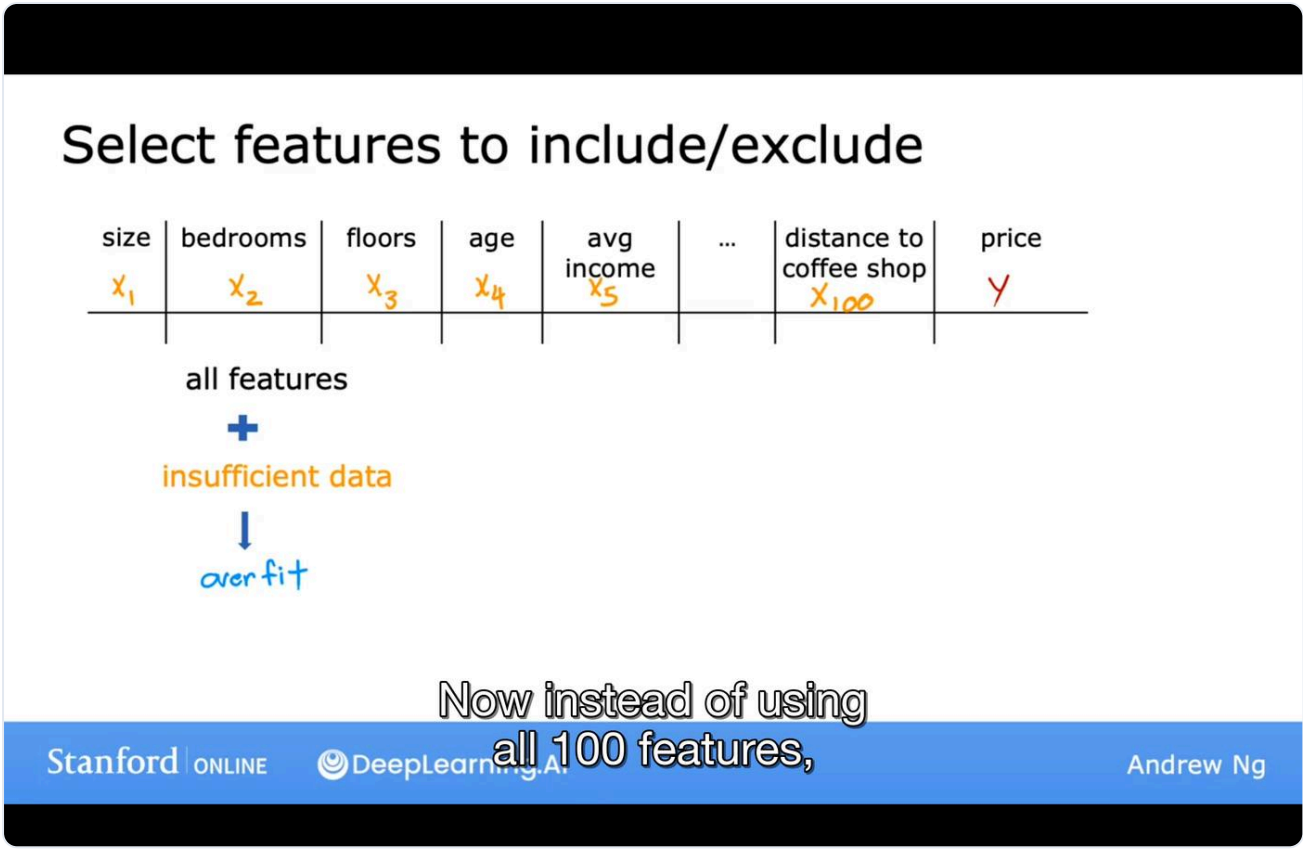


FIGURE 38.3 | Select Features to Include/exclude

These were a lot of polynomial features. In that case, one way to reduce overfitting is to just not use so many of these polynomial features. But now let's look at a different example. Maybe you have a lot of different features of a house of which to try to predict its price, ranging from the size, number of bedrooms, number of floors, the age, average income of the neighborhood, and so on and so forth, total distance to the nearest coffee shop. It turns out that if you have a lot of features like these but don't have enough training data, then your learning algorithm may also overfit to your training set. Now instead of using all 100 features, if we were to pick just a subset of the most useful ones, maybe size, bedrooms, and the age of the house.

If you think those are the most relevant features, then using just that smallest subset of features, you may find that your model no longer overfits as badly. Choosing the most appropriate set of features to use is sometimes also called feature selection.

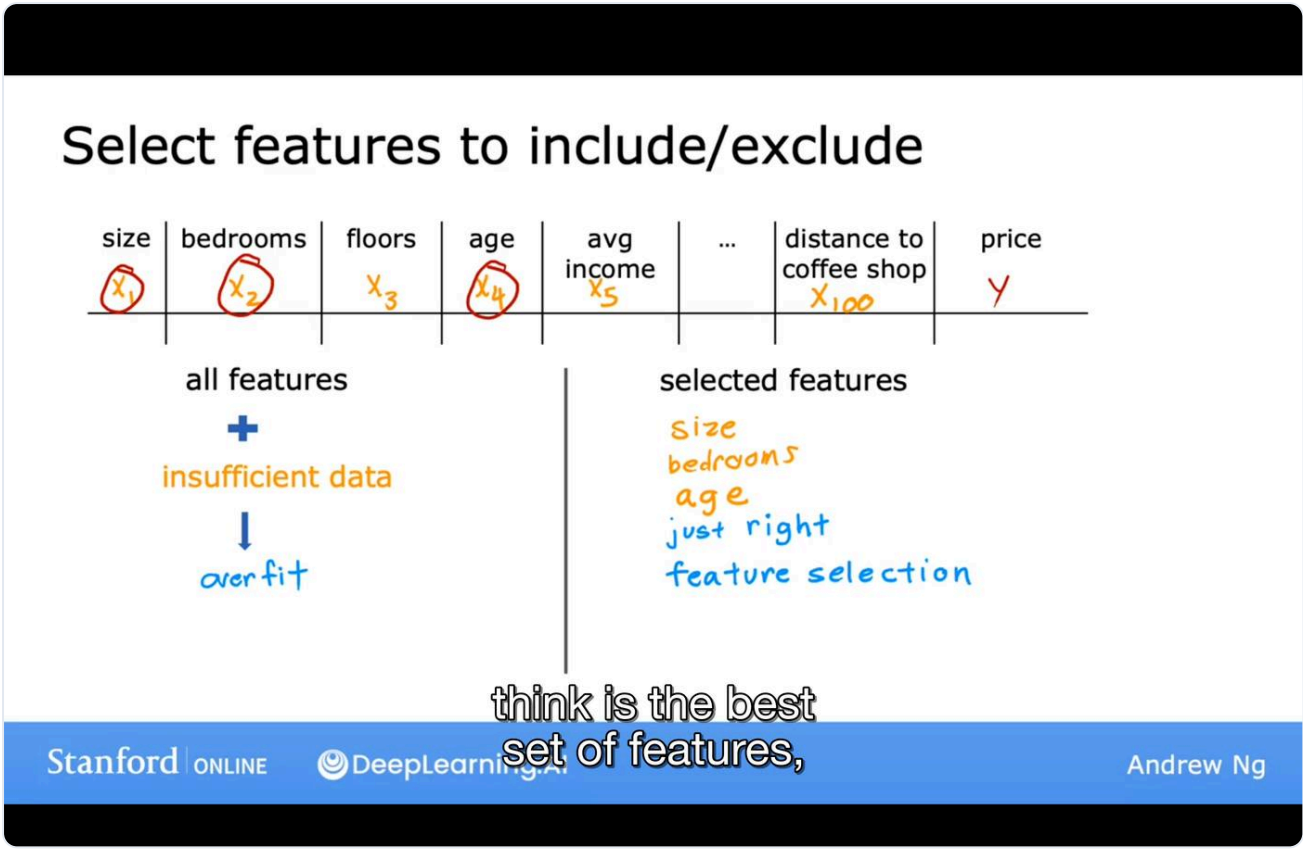


FIGURE 38.4 | Select Features to Include/exclude

One way you could do so is to use your intuition to choose what you think is the best set of features, what's most relevant for predicting the price. Now, one disadvantage of feature selection is that by using only a subset of the features, the algorithm is throwing away some of the information that you have about the houses. For example, maybe all of these features, all 100 of them are actually useful for predicting the price of a house. Maybe you don't want to throw away some of the information by throwing away some of the features. Later in Course 2, you'll also see some algorithms for automatically choosing the most appropriate set of features to use for our prediction task.

Now, this takes us to the third option for reducing overfitting. This technique, which we'll look at in even greater depth in the next video is called regularization.

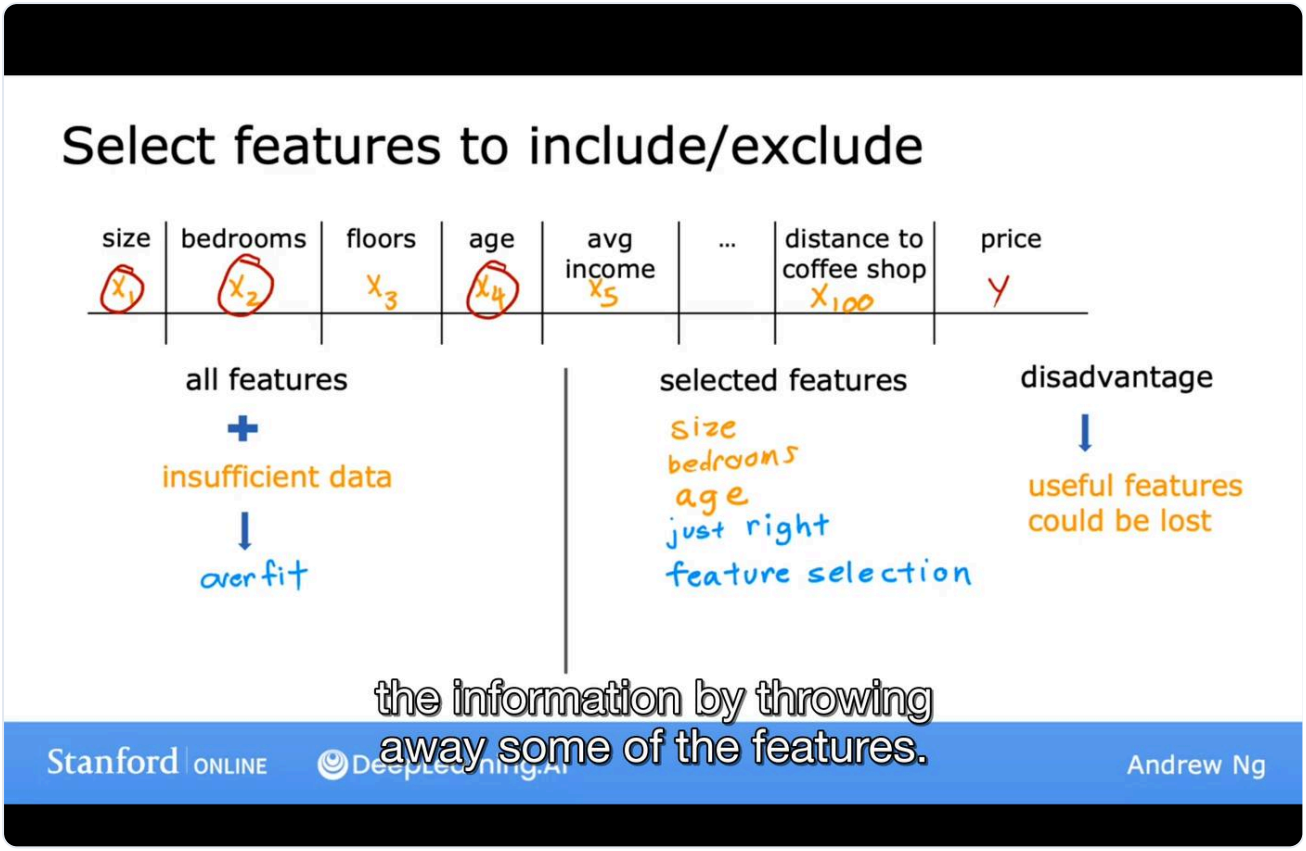


FIGURE 38.5 | Select Features to Include/exclude

If you look at an overfit model, here's a model using polynomial features: x, x squared, x cubed, and so on. You find that the parameters are often relatively large. Now if you were to eliminate some of these features, say, if you were to eliminate the feature x4, that corresponds to setting this parameter to 0. So setting a parameter to 0 is equivalent to eliminating a feature, which is what we saw on the previous slide. It turns out that regularization is a way to more gently reduce the impacts of some of the features without doing something as harsh as eliminating it outright. What regularization does is encourage the learning algorithm to shrink the values of the parameters without necessarily demanding that the parameter is set to exactly 0.

It turns out that even if you fit a higher order polynomial like this, so long as you can get the algorithm to use smaller parameter values: w1, w2, w3, w4. You end up with a curve that ends up fitting the training data much better.

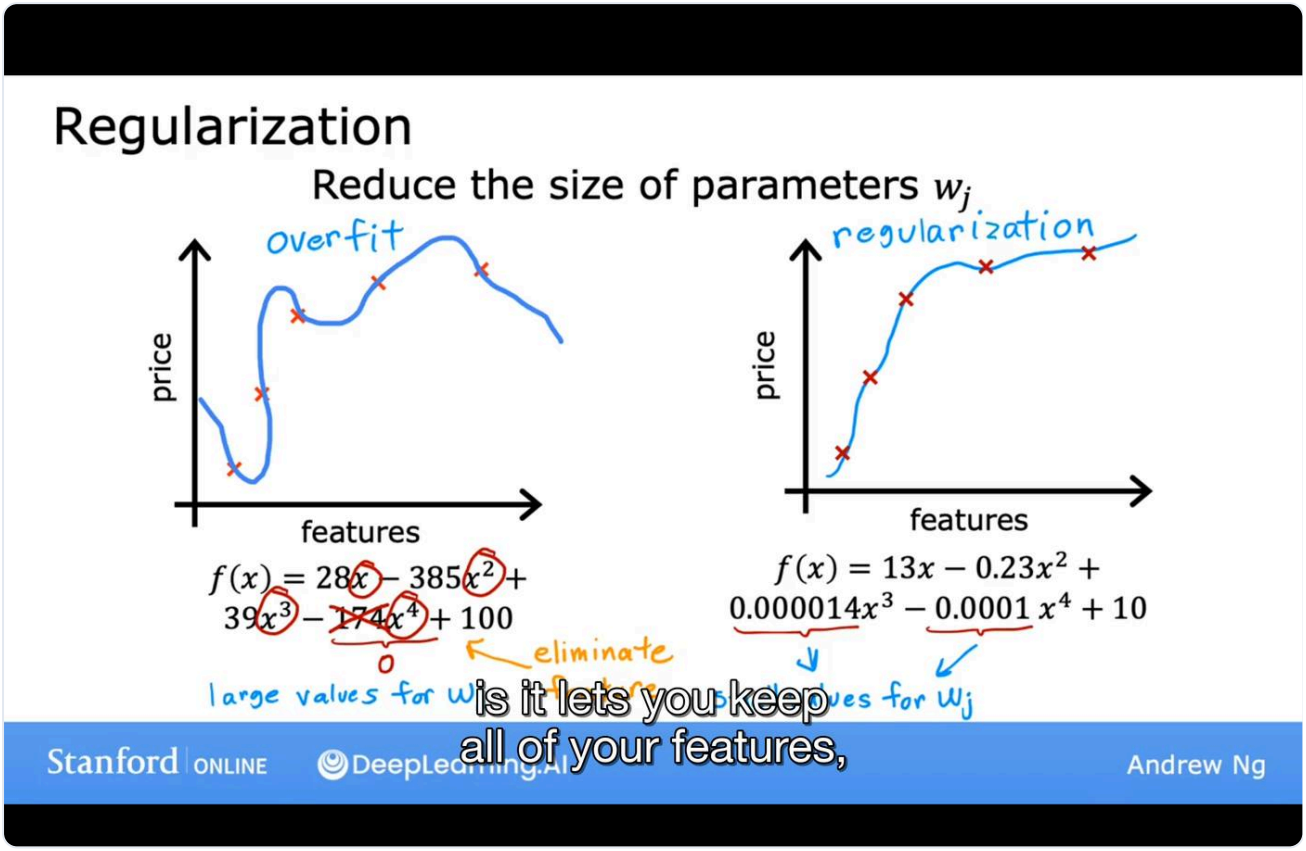


FIGURE 38.6 | Reduce the Size of Parameters W

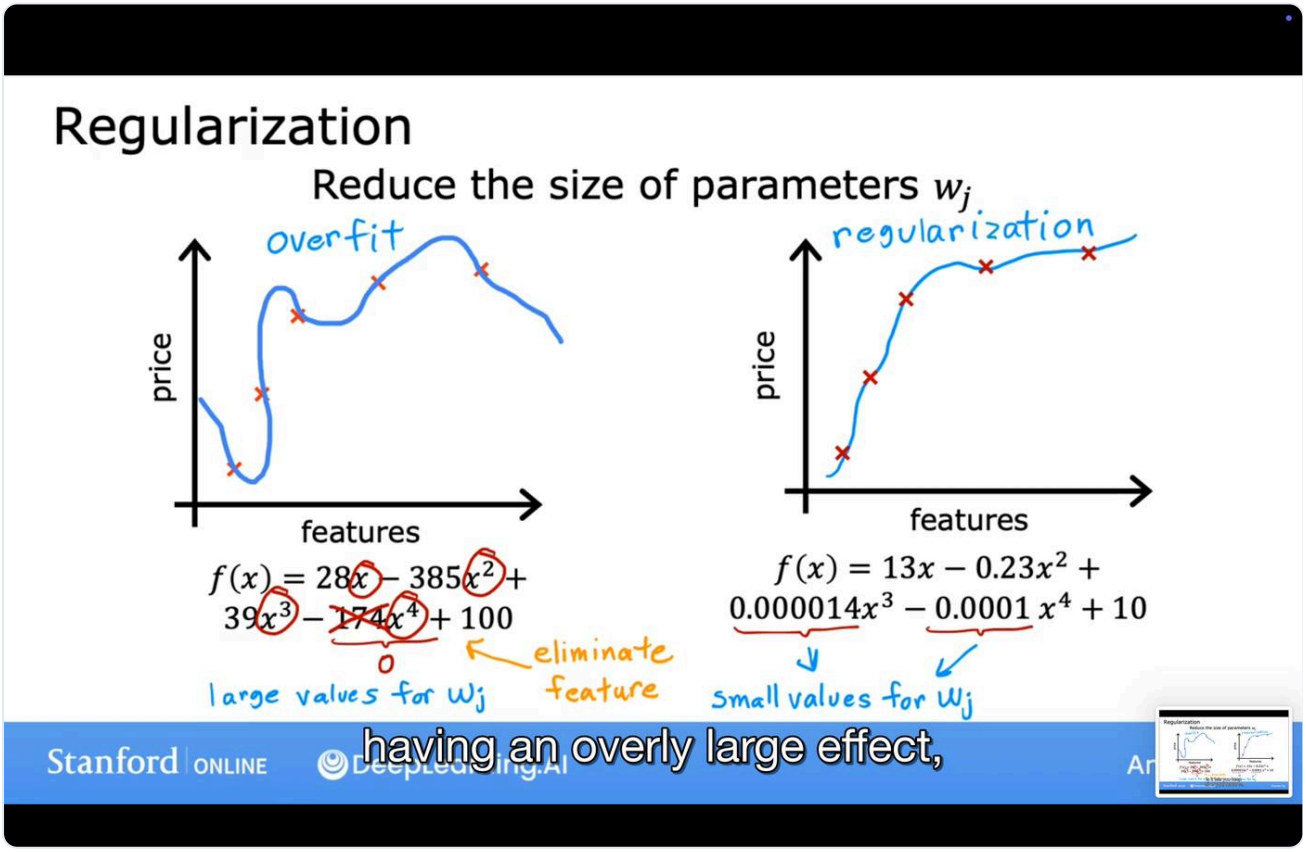


FIGURE 38.7 | Reduce the Size of Parameters W

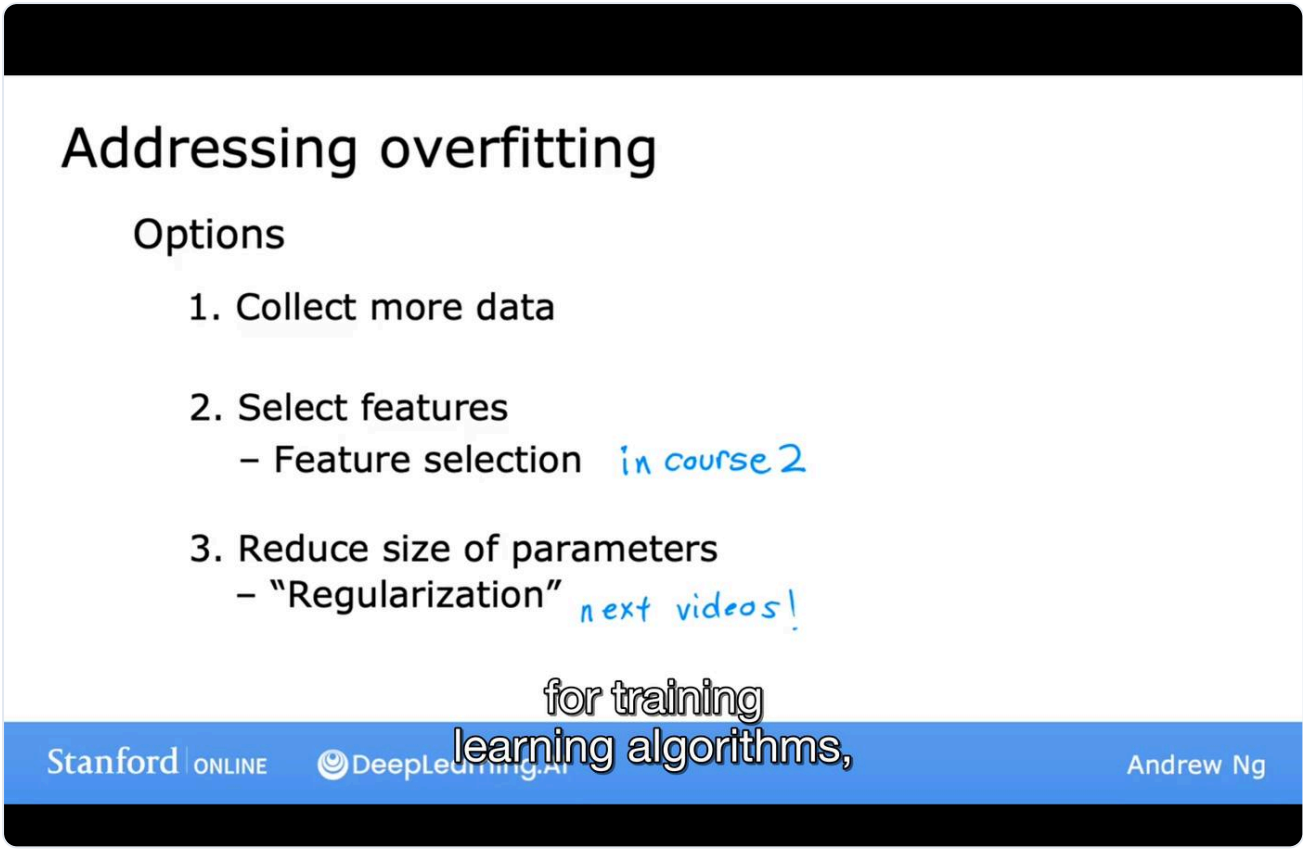


FIGURE 38.8 | Addressing Overfitting

So what regularization does, is it lets you keep all of your features, but they just prevents the features from having an overly large effect, which is what sometimes can cause overfitting. By the way, by convention, we normally just reduce the size of the wj parameters, that is w1 through wn. It doesn't make a huge difference whether you regularize the parameter b as well, you could do so if you want or not if you don't. I usually don't and it's just fine to regularize w1, w2, all the way to wn, but not really encourage b to become smaller. In practice, it should make very little difference whether you also regularize b or not. To recap, these are the three ways you saw in this video for addressing overfitting.

One, collect more data. If you can get more data, this can really help reduce overfitting. Sometimes that's not possible. In which case, some of the options are, two, try selecting and using only a subset of the features. You'll learn more about feature selection in Course 2. Three would be to reduce the size of the parameters using regularization. This will be the subject of the next video as well. Just for myself, I use regularization all the time. So this is a very useful technique for training learning algorithms, including neural networks specifically, which you'll see later in this specialization as well. I hope you'll also check out the optional lab on overfitting.

In the lab, you'll be able to see different examples of overfitting and adjust those examples by clicking on options in the plots. You'll also be able to add your own data points by clicking on the plot and see how that changes the curve that is fit. You can also try examples for both regression and classification and you will change the degree of the polynomial to be x, x squared, x cubed, and so on. The lab also lets you play with two different options for addressing overfitting. You can add additional training data to reduce overfitting and you can also select which features to

include or to exclude as another way to try to reduce overfitting. Please take a look at a lab, which I hope will help you build your intuition about overfitting as well as some methods for addressing it.

In this video, you also saw the idea of regularization at a relatively high level. I realize that all of these details on regularization may not fully make sense to you yet. But in the next video, we'll start to formulate exactly how to apply regularization and exactly what regularization means. Then we'll start to figure out how to make this work with our learning algorithms to make linear regression and logistic regression, and in the future, other algorithms as well avoid overfitting. Let's take a look at that in the next video.

LECTURE 39

Regularization Simpler Model W320

In the last video we saw that regularization tries to make the parental values W1 through WN small to reduce overfitting. In this video, we'll build on that intuition and developed a modified cost function for your learning algorithm that can use to actually apply regularization. Let's jump in, recall this example from the previous video in which we saw that if you fit a quadratic function to this data, it gives a pretty good fit. But if you fit a very high order polynomial, you end up with a curve that over fits the data. But now consider the following, suppose that you had a way to make the parameters W3 and W4 really, really small. Say close to 0. Here's what I mean.

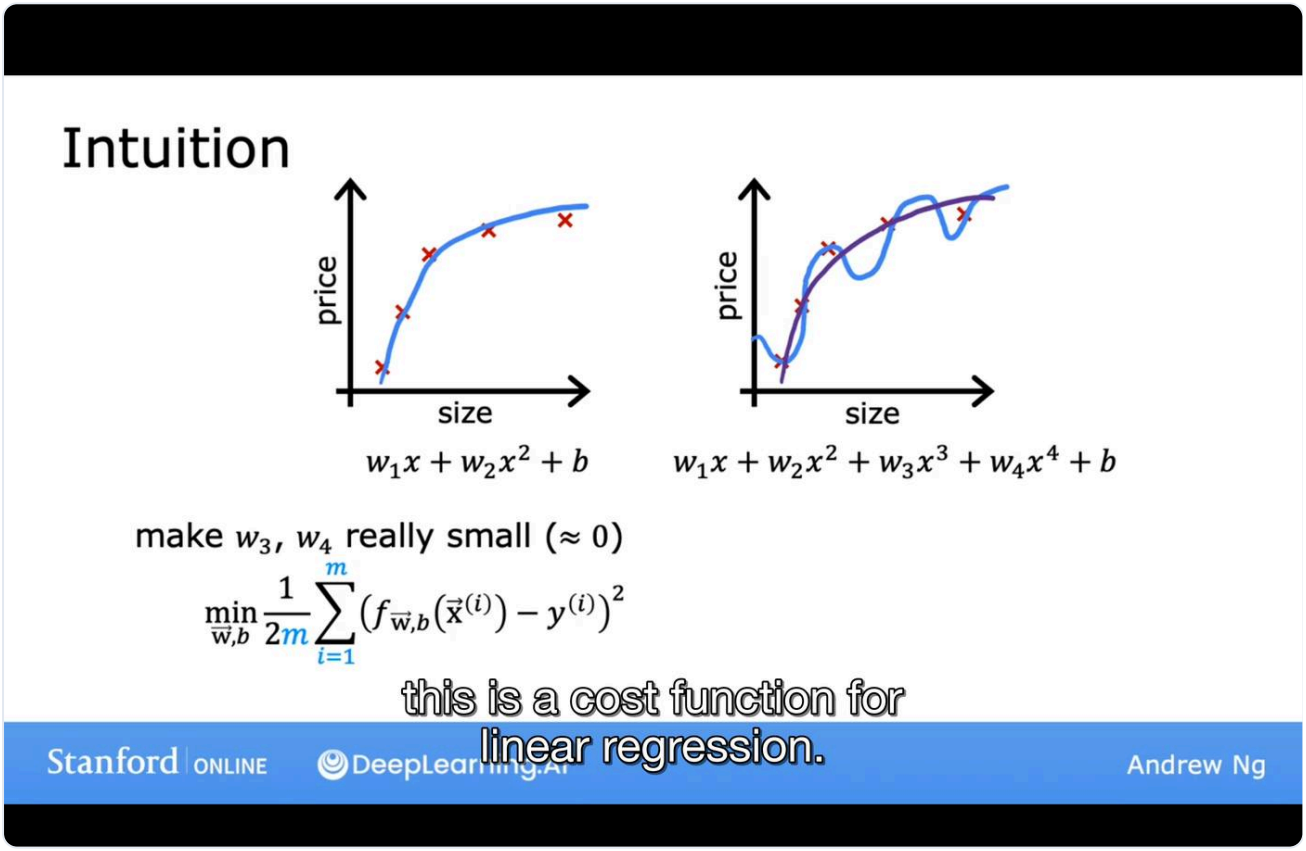


FIGURE 39.1

Let's say instead of minimizing this objective function, this is a cost function for linear regression. Let's say you were to modify the cost function and add to it 1000 times W3 squared plus 1000 times W4 squared. And here I'm just choosing 1000 because it's a big number but any other really large number would be okay. So with this modified cost function, you could in fact be penalizing the model if W3 and W4 are large. Because if you want to minimize this function, the only way to make this new cost function small is if W3 and W4 are both small, right? Because otherwise this 1000 times W3 squared and 1000 times W4 square terms are going to be really, really big.

So when you minimize this function, you're going to end up with W3 close to 0 and W4 close to 0.

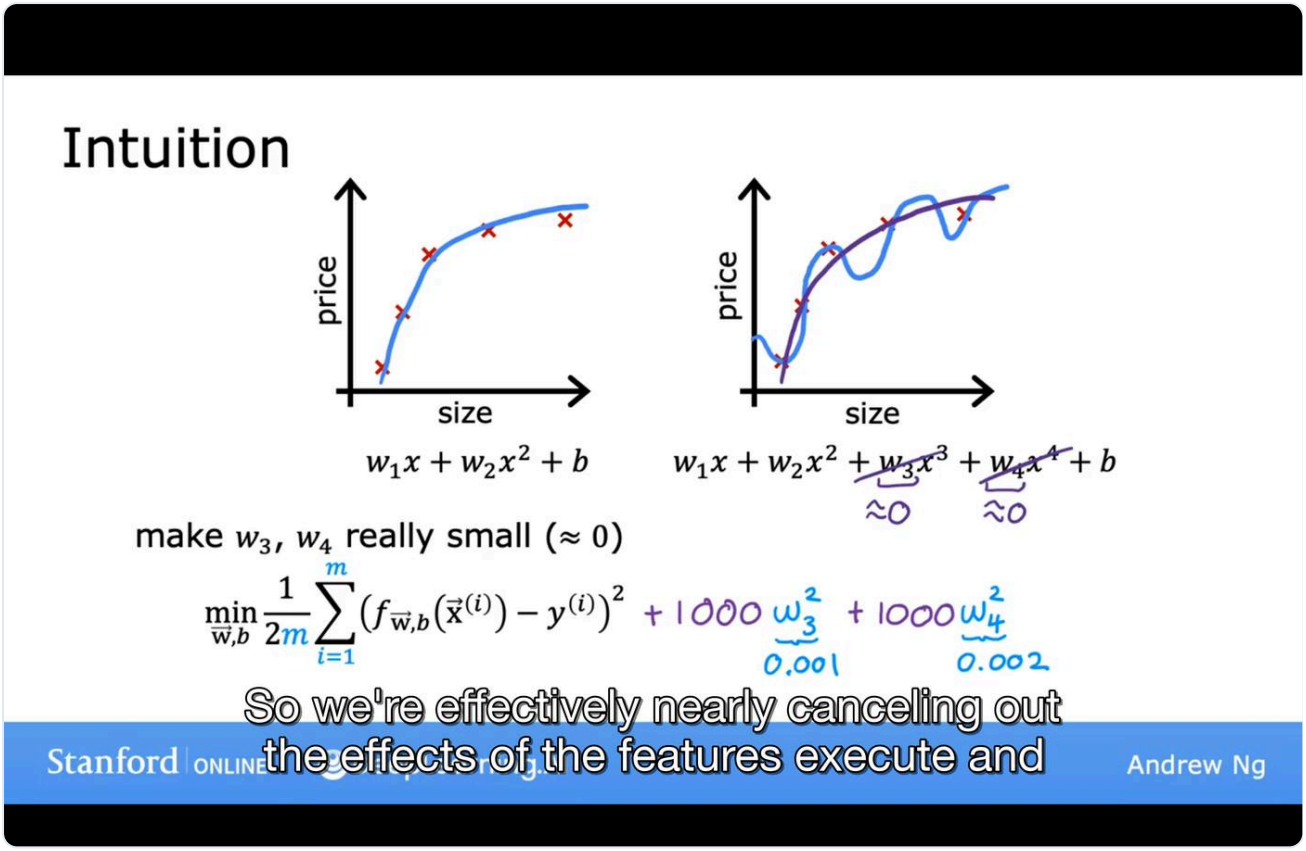


FIGURE 39.2

So we're effectively nearly canceling out the effects of the features execute and extra power of 4 and getting rid of these two terms over here. And if we do that, then we end up with a fit to the data that's much closer to the quadratic function, including maybe just tiny contributions from the features x cubed and extra 4.

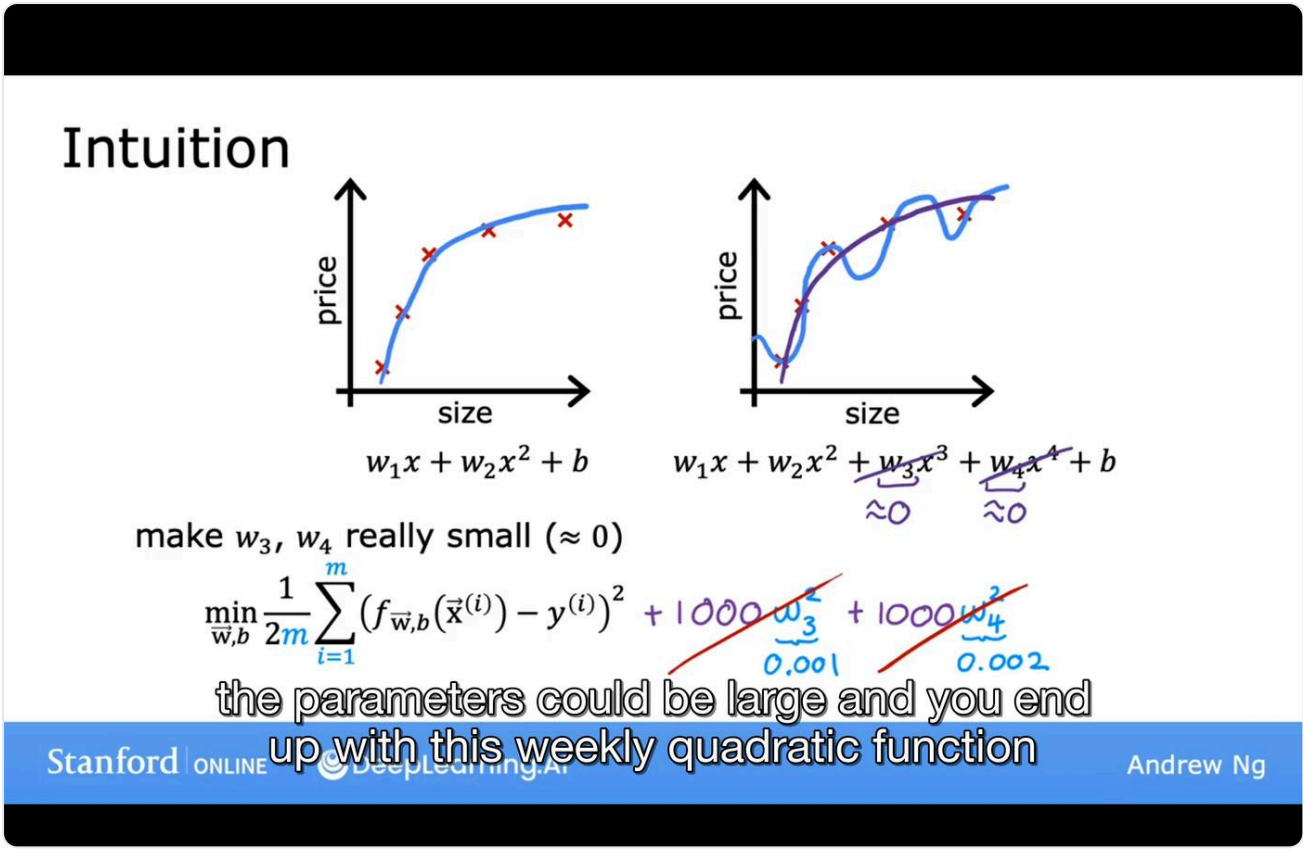


FIGURE 39.3

And this is good because it's a much better fit to the data compared to if all the parameters could be large and you end up with this weekly quadratic function more generally, here's the idea behind regularization. The idea is that if there are smaller values for the parameters, then that's a bit like having a simpler model. Maybe one with fewer features, which is therefore less prone to overfitting. On the last slide we penalize or we say we regularized only W3 and W4. But more generally, the way that regularization tends to be implemented is if you have a lot of features, say a 100 features, you may not know which are the most important features and which ones to penalize.

So the way regularization is typically implemented is to penalize all of the features or more precisely, you penalize all the WJ parameters and it's possible to show that this will usually result in fitting a smoother simpler, less weekly function that's less prone to overfitting.

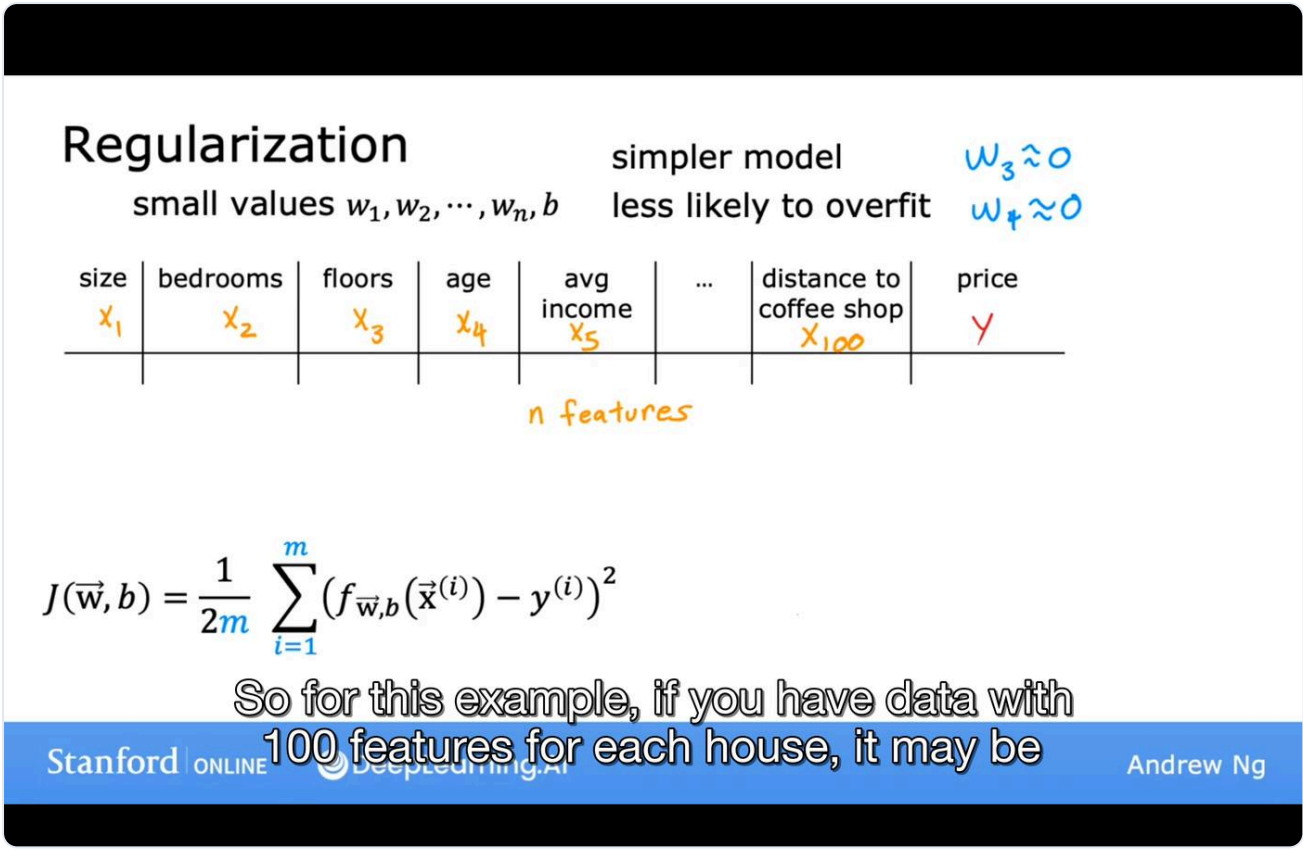


FIGURE 39.4 | Regularization Simpler Model W,20

So for this example, if you have data with 100 features for each house, it may be hard to pick an advance which features to include and which ones to exclude. So let's build a model that uses all 100 features. So you have these 100 parameters W1 through W100, as well as 100 and first parameter B.

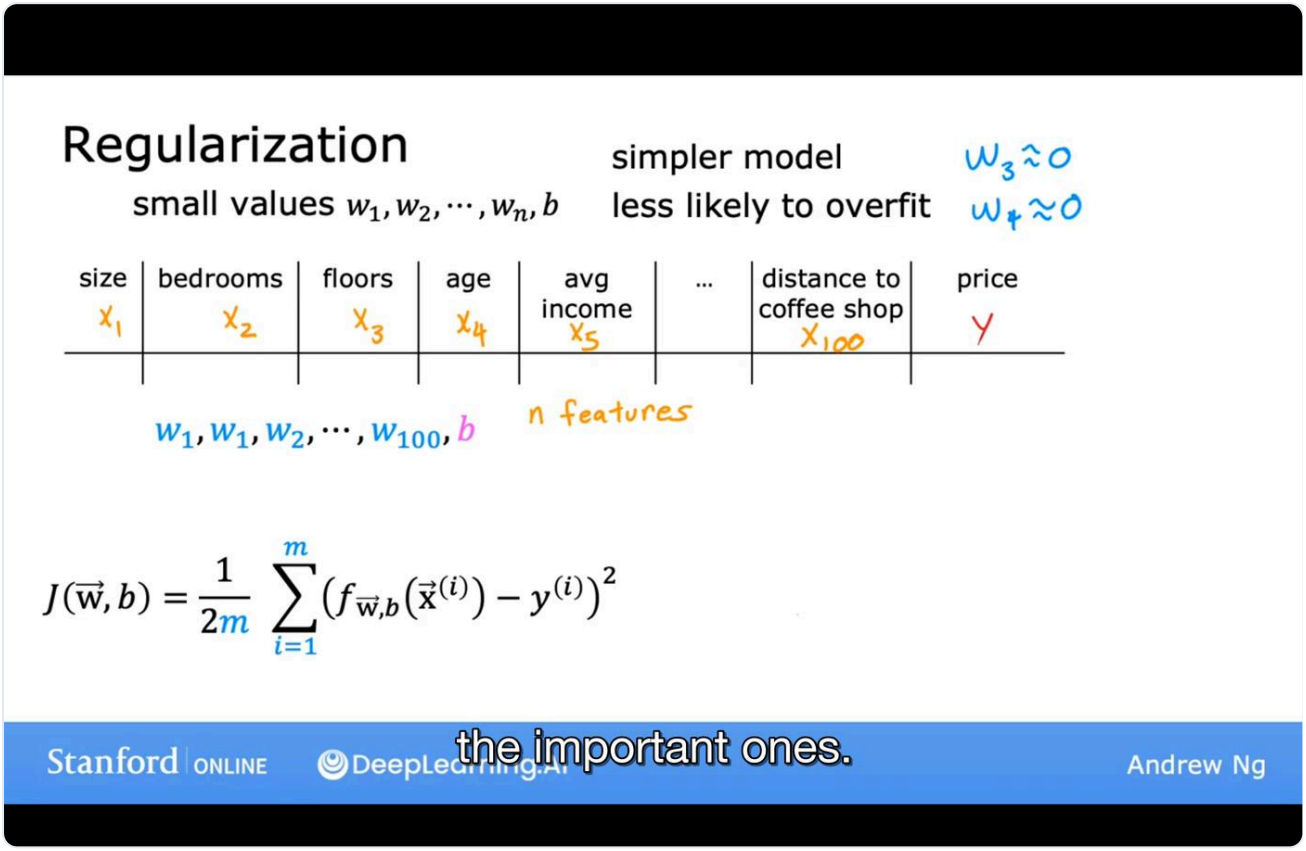


FIGURE 39.5

Because we don't know which of these parameters are going to be the important ones. Let's penalize all of them a bit and shrink all of them by adding this new term lambda times the sum from J equals 1 through n where n is 100. The number of features of wj squared.

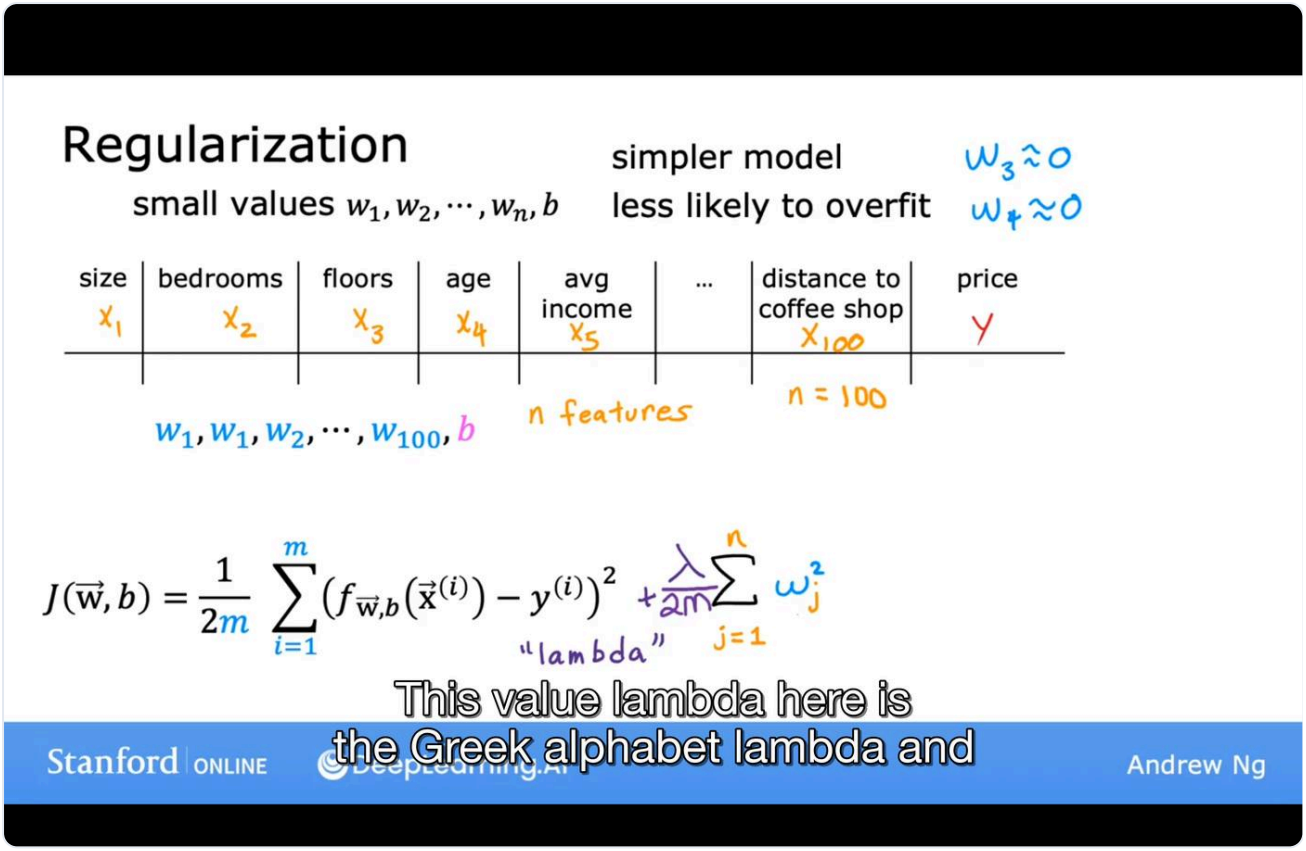


FIGURE 39.6

This value lambda here is the Greek alphabet lambda and it's also called a regularization parameter. So similar to picking a learning rate alpha, you now also have to choose a number for lambda. A couple of things I would like to point out by convention, instead of using lambda times the sum of wj squared.

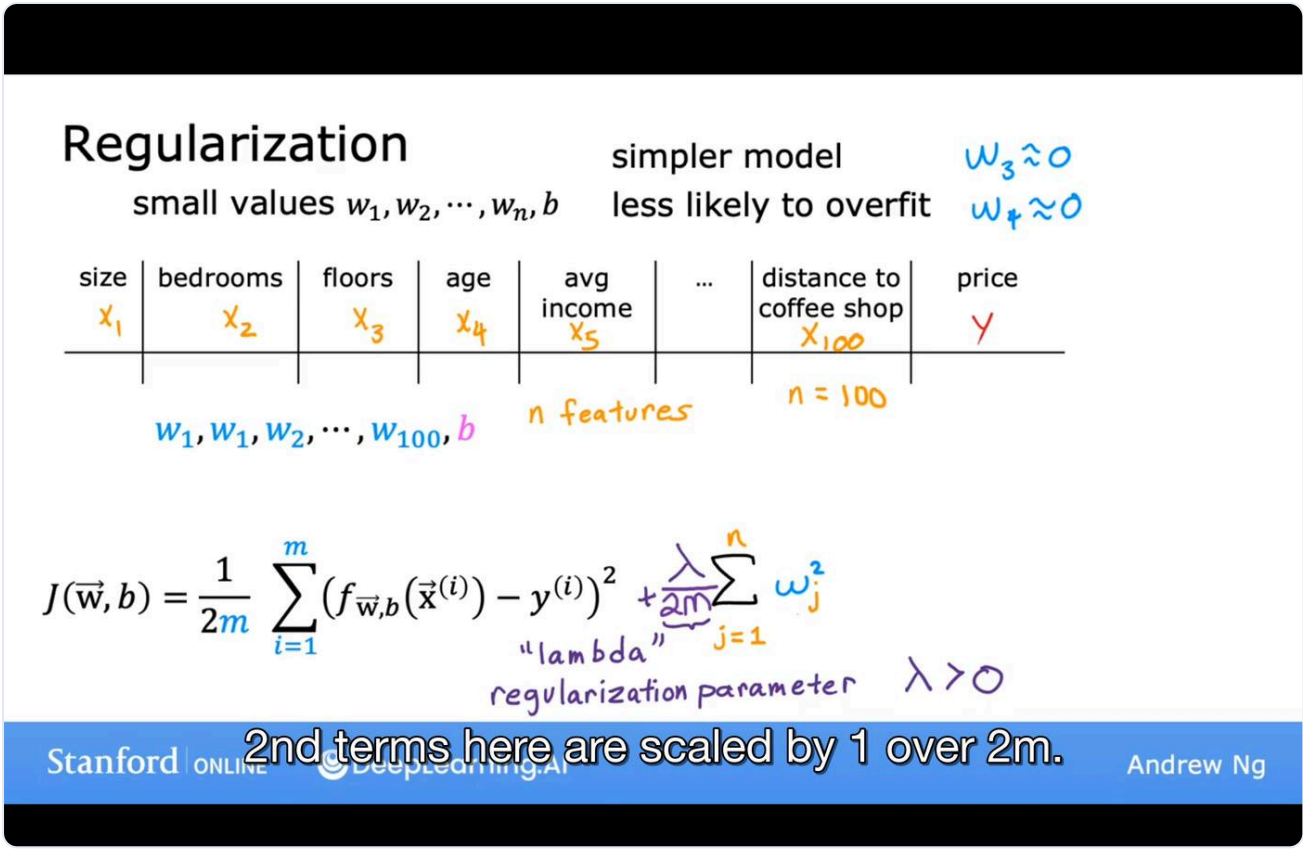


FIGURE 39.7 | Regularization Simpler Model W320

We also divide lambda by 2m so that both the 1st and 2nd terms here are scaled by 1 over 2m. It turns out that by scaling both terms the same way it becomes a little bit easier to choose a good value for lambda. And in particular you find that even if your training set size growth, say you find more training examples. So m the training set size is now bigger. The same value of lambda that you've picked previously is now also more likely to continue to work if you have this extra scaling by 2m. Also by the way, by convention we're not going to penalize the parameter b for being large. In practice, it makes very little difference whether you do or not.

And some machine learning engineers and actually some learning algorithm implementations will also include lambda over 2m times the b squared term.

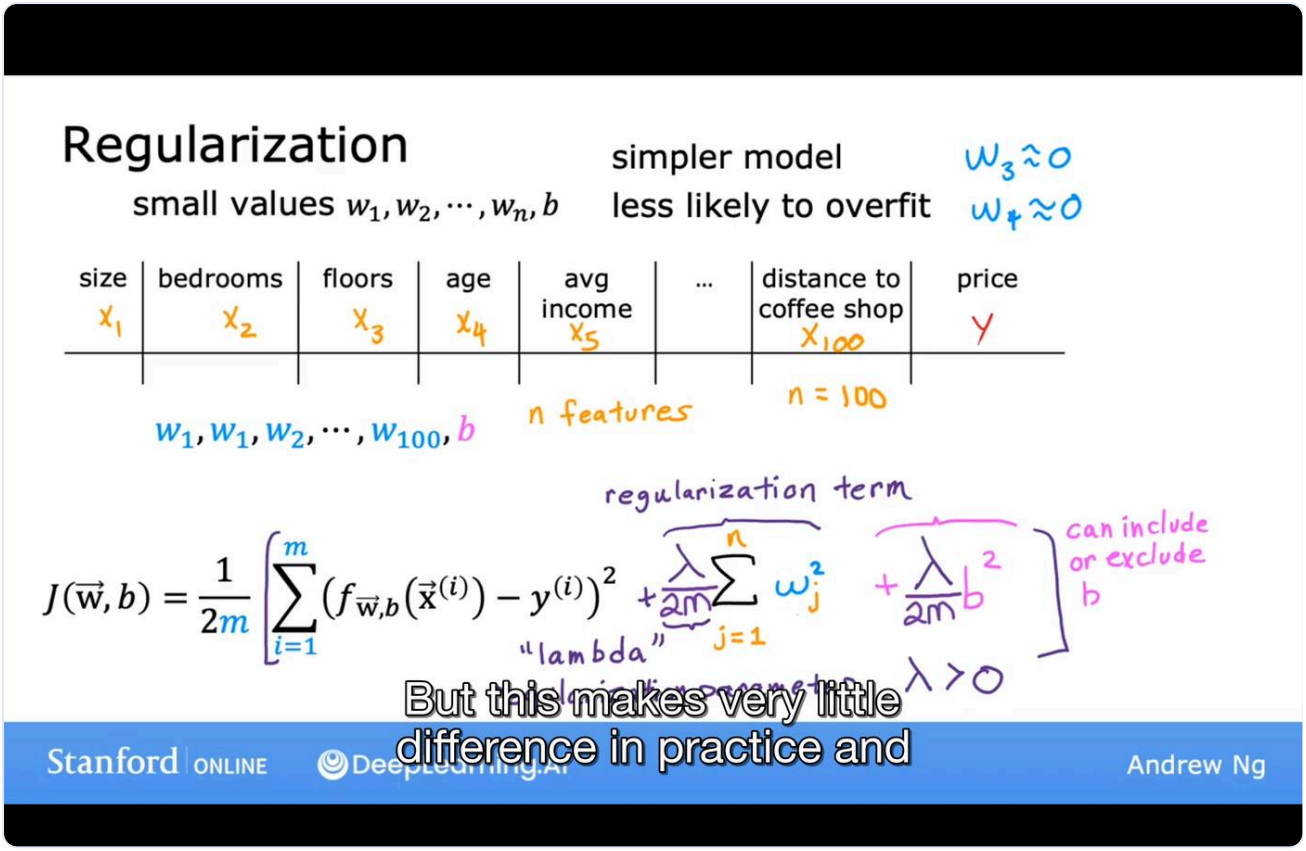


FIGURE 39.8

But this makes very little difference in practice and the more common convention which was used in this course is to regularize only the parameters w rather than the parameter b.

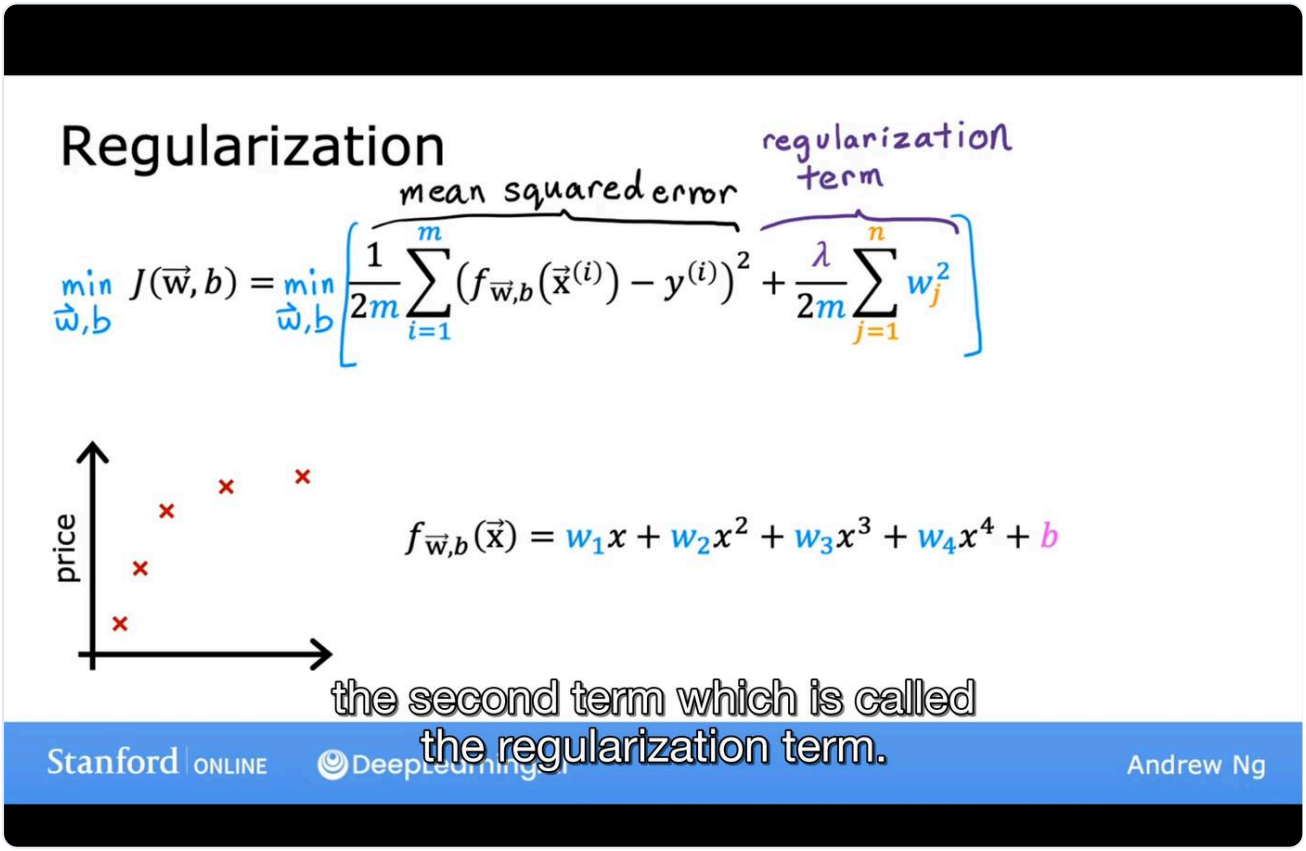


FIGURE 39.9

So to summarize in this modified cost function, we want to minimize the original cost, which is the mean squared error cost plus additionally, the second term which is called the regularization term. And so this new cost function trades off two goals that you might have. Trying to minimize this first term encourages the algorithm to fit the training data well by minimizing the squared differences of the predictions and the actual values. And try to minimize the second term. The algorithm also tries to keep the parameters wj small, which will tend to reduce overfitting. The value of lambda that you choose, specifies the relative importance or the relative trade off or how you balance between these two goals.

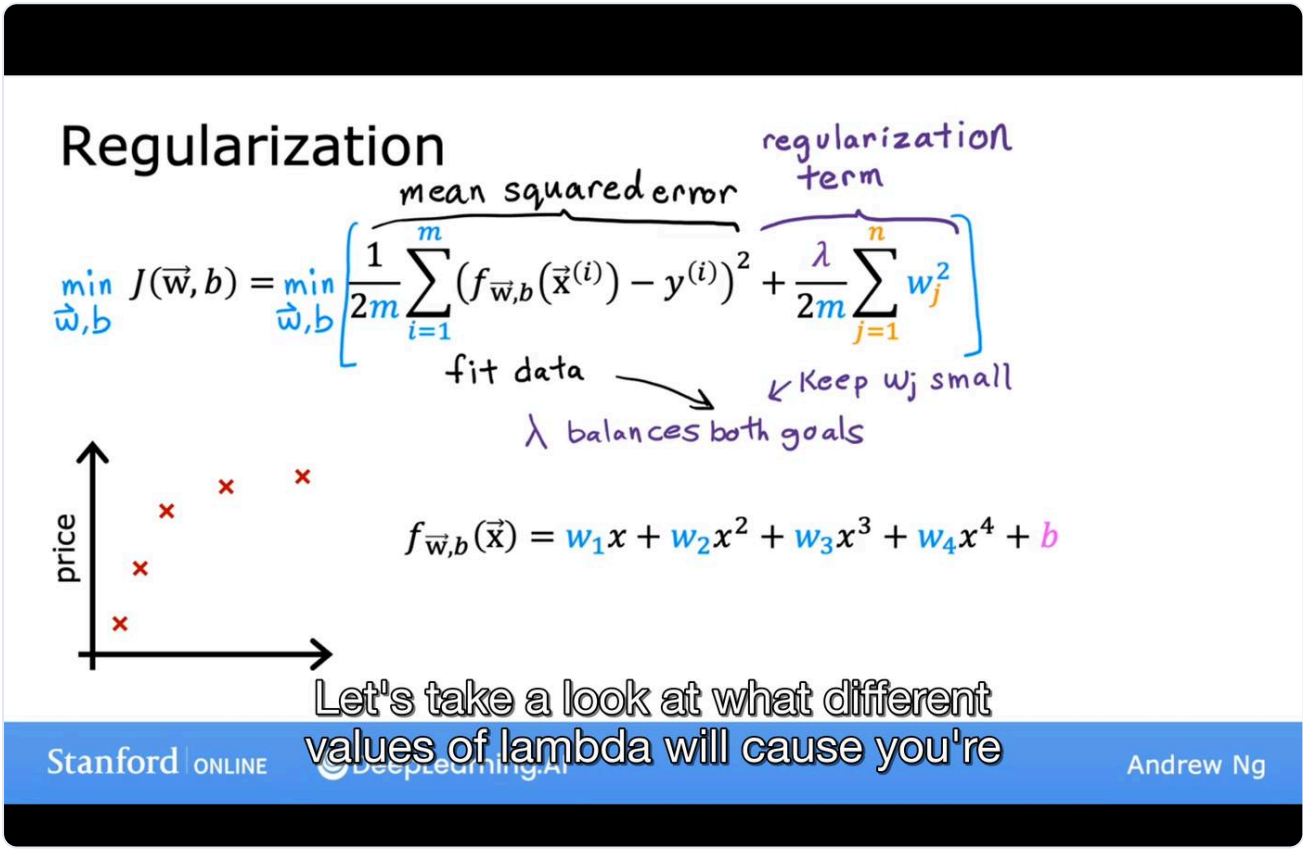


FIGURE 39.10

Let's take a look at what different values of lambda will cause you're learning algorithm to do. Let's use the housing price prediction example using linear regression. So F of X is the linear regression model. If lambda was set to be 0, then you're not using the regularization term at all because the regularization term is multiplied by 0. And so if lambda was 0, you end up fitting this overly wiggly, overly complex curve and it over fits. So that was one extreme of if lambda was 0. Let's now look at the other extreme. If you said lambda to be a really, really, really large number, say lambda equals 10 to the power of 10, then you're placing a very heavy weight on this regularization term on the right.

And the only way to minimize this is to be sure that all the values of w are pretty much very close to 0. So if lambda is very, very large, the learning algorithm will choose W1, W2, W3 and W4 to be extremely close to 0 and thus F of X is basically equal to b and so the learning algorithm fits a horizontal straight line and under fits.

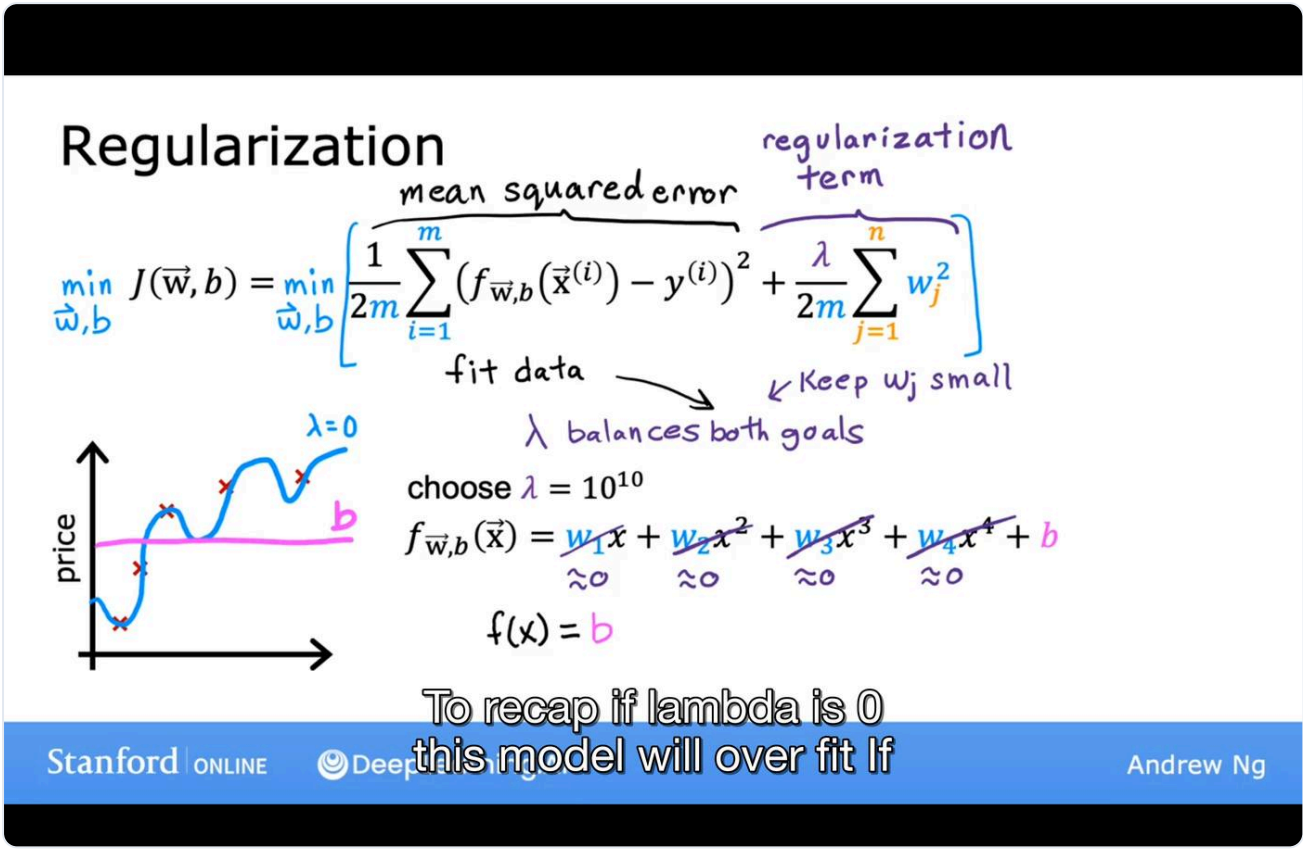


FIGURE 39.11

To recap if lambda is 0 this model will over fit If lambda is enormous like 10 to the power of 10. This model will under fit.

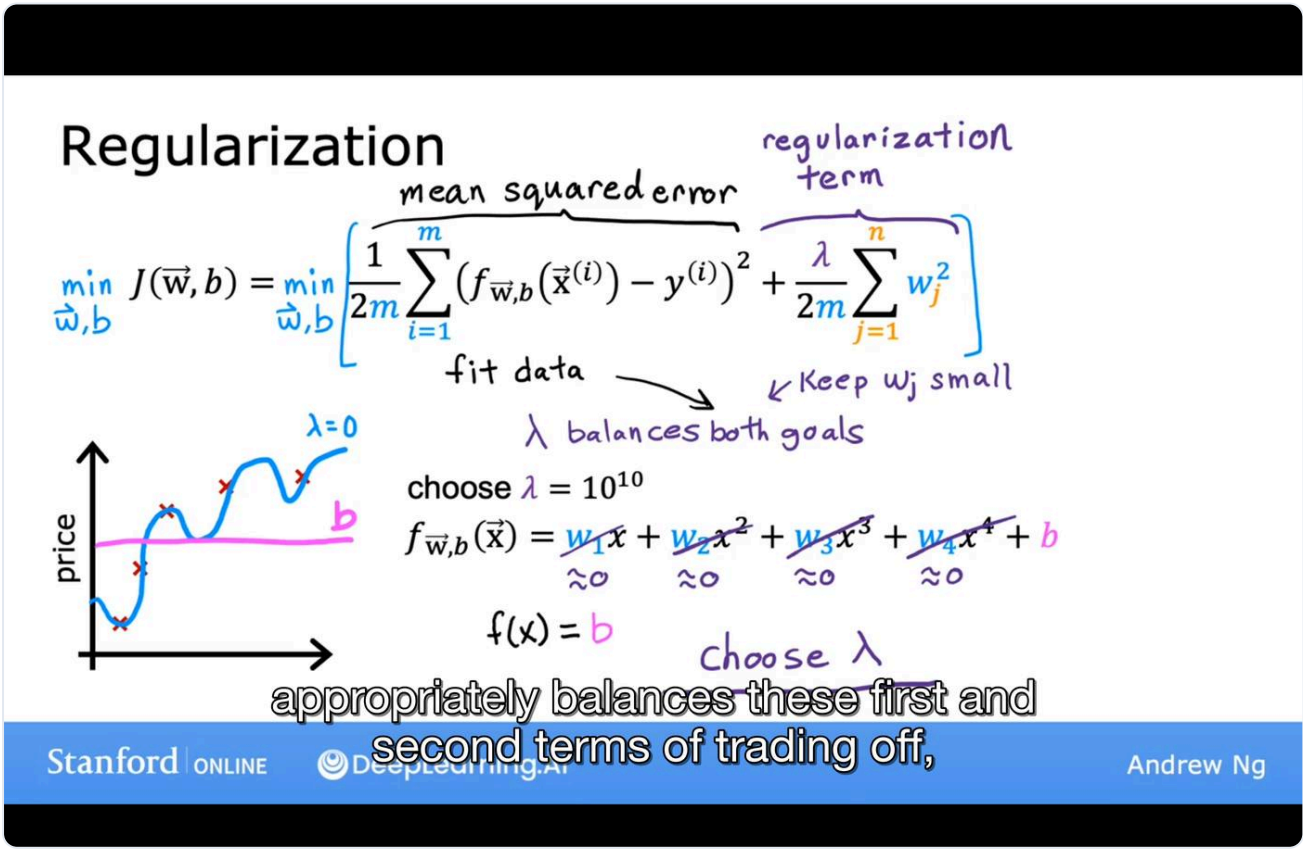


FIGURE 39.12

And so what you want is some value of lambda that is in between that more appropriately balances these first and second terms of trading off, minimizing the mean squared error and keeping the parameters small.

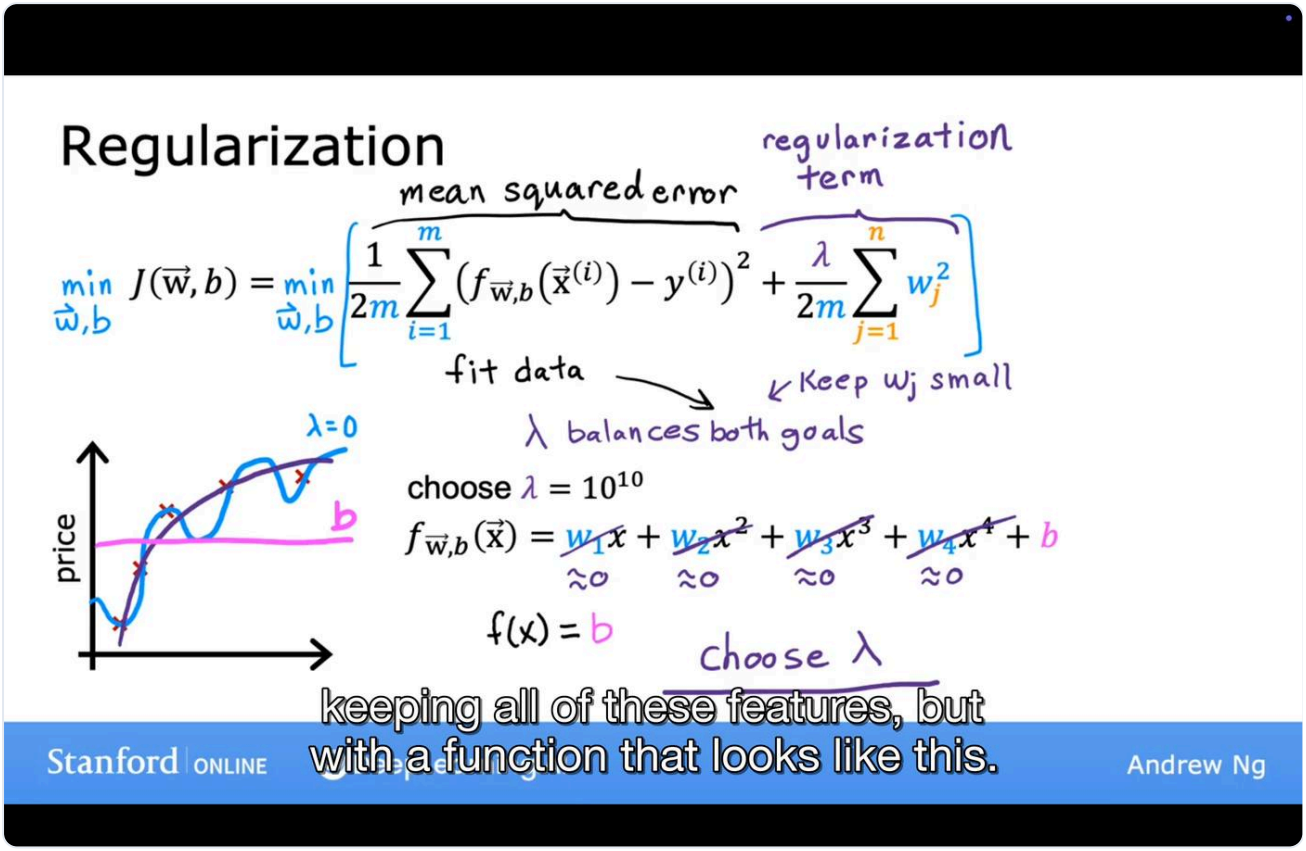


FIGURE 39.13

And when the value of lambda is not too small and not too large, but just right, then hopefully you end up able to fit a 4th order polynomial, keeping all of these features, but with a function that looks like this. So that's how regularization works. When we talk about model selection, later into specialization will also see a variety of ways to choose good values for lambda. In the next two videos will flesh out how to apply regularization to linear regression and logistic regression, and how to train these models with great in dissent with that, you'll be able to avoid overfitting with both of these algorithms.

LECTURE 40

Regularized Linear Regression

In this video, we'll figure out how to get gradient descent to work with regularized linear regression. Let's jump in.

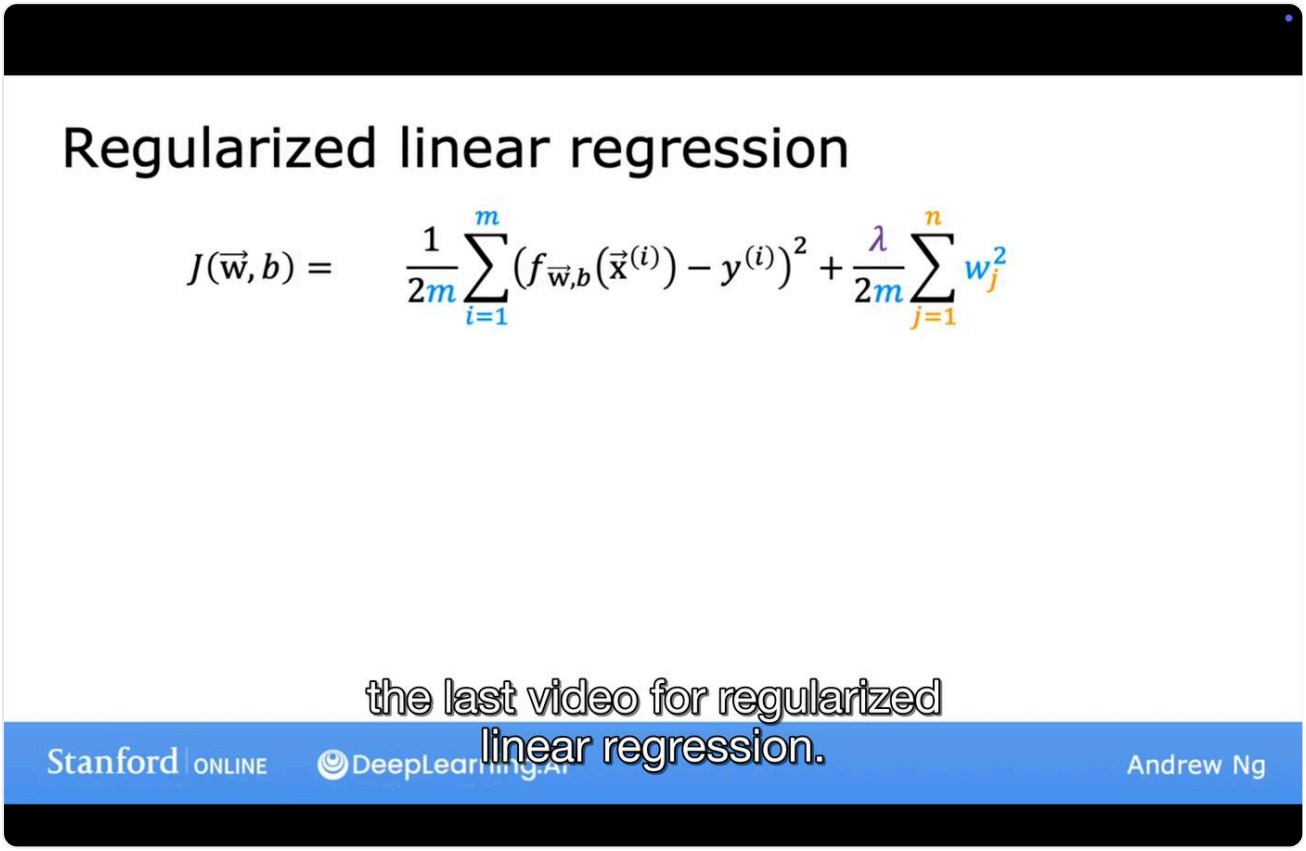


FIGURE 40.1

Here Is a cost function we've come up with in the last video for regularized linear regression. The first part is the usual squared error cost function, and now you have this additional regularization term, where Lambda is the regularization parameter, and you'd like to find parameters w and b that minimize the regularized cost function.

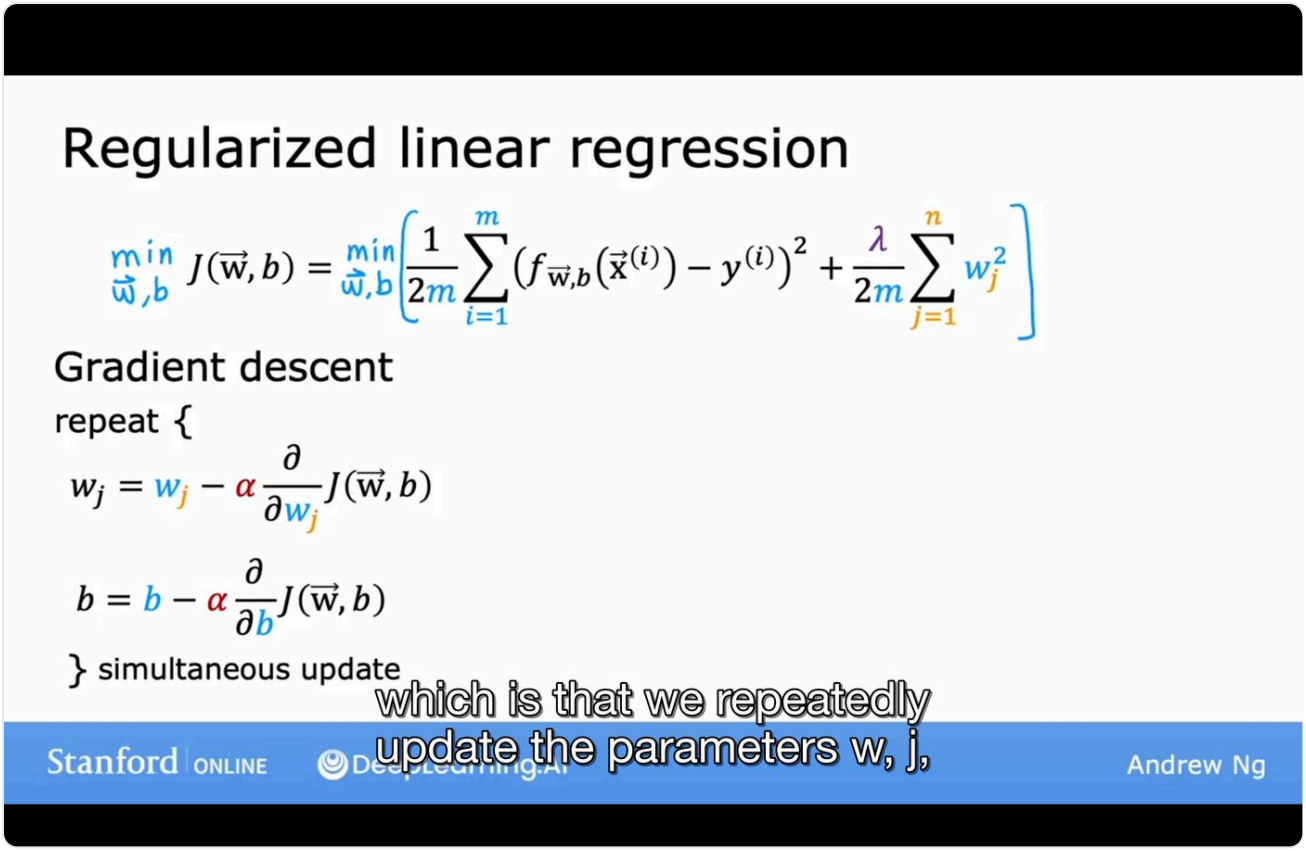


FIGURE 40.2 | Regularized Linear Regression

Previously we were using gradient descent for the original cost function, just the first term before we added that second regularization term, and previously, we had the following gradient descent algorithm, which is that we repeatedly update the parameters w, j, and b for j equals 1 through n according to this formula and b is also updated similarly. Again, Alpha is a very small positive number called the learning rate.

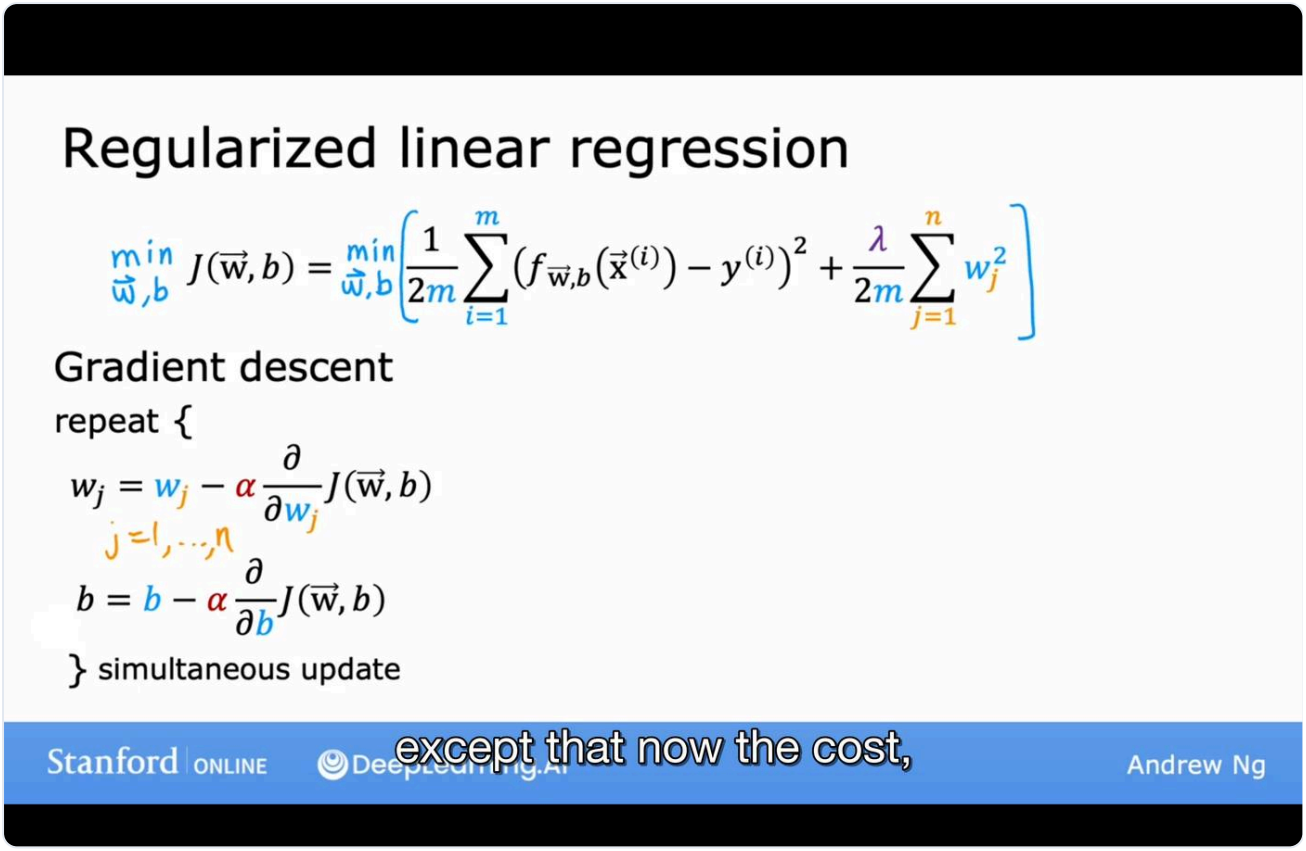


FIGURE 40.3 | Regularized Linear Regression

In fact, the updates for a regularized linear regression look exactly the same, except that now the cost, J, is defined a bit differently. Previously the derivative of J with respect to w_j was given by this expression over here, and the derivative respect to b was given by this expression over here.

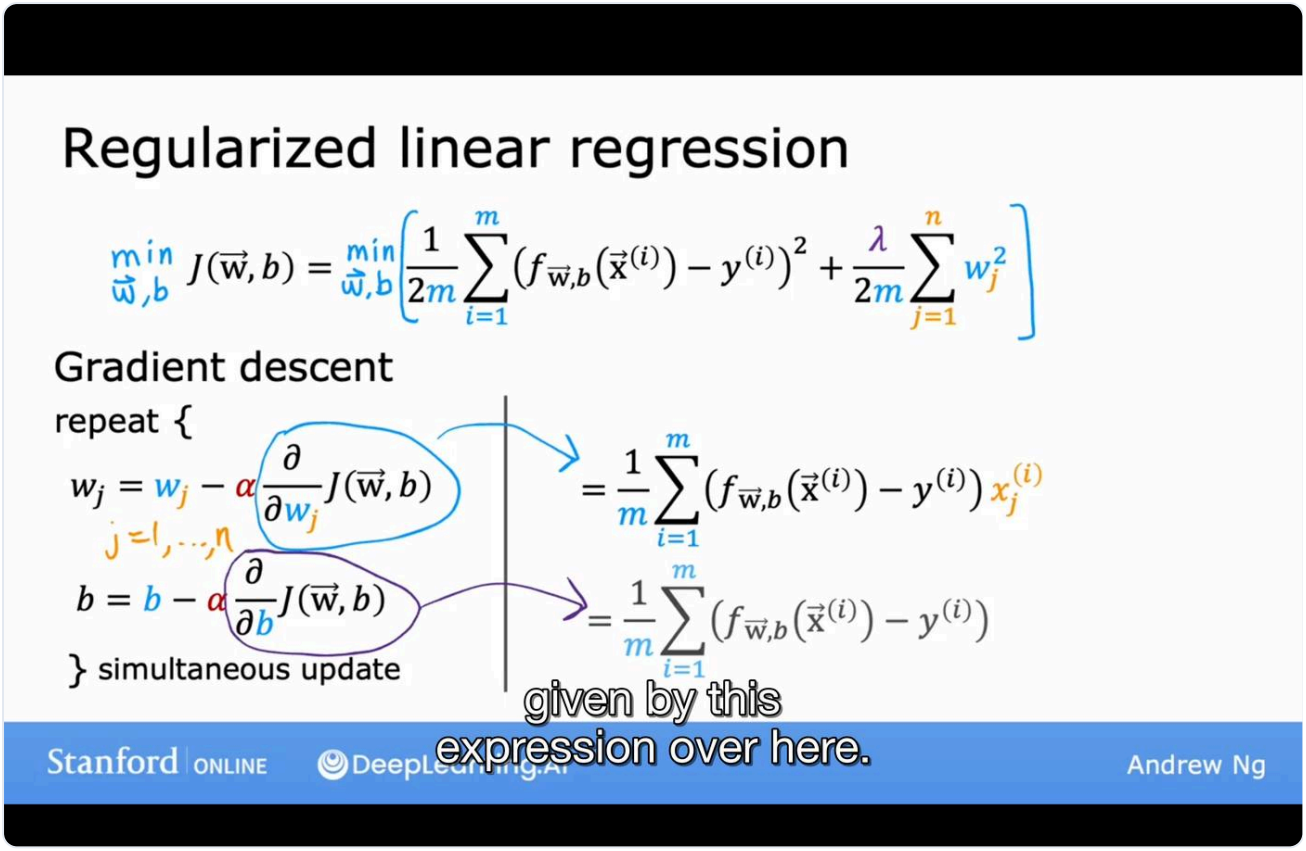


FIGURE 40.4 | Regularized Linear Regression

Now that we've added this additional regularization term, the only thing that changes is that the expression for the derivative with respect to w_j ends up with one additional term, this plus Lambda over m times w_j.

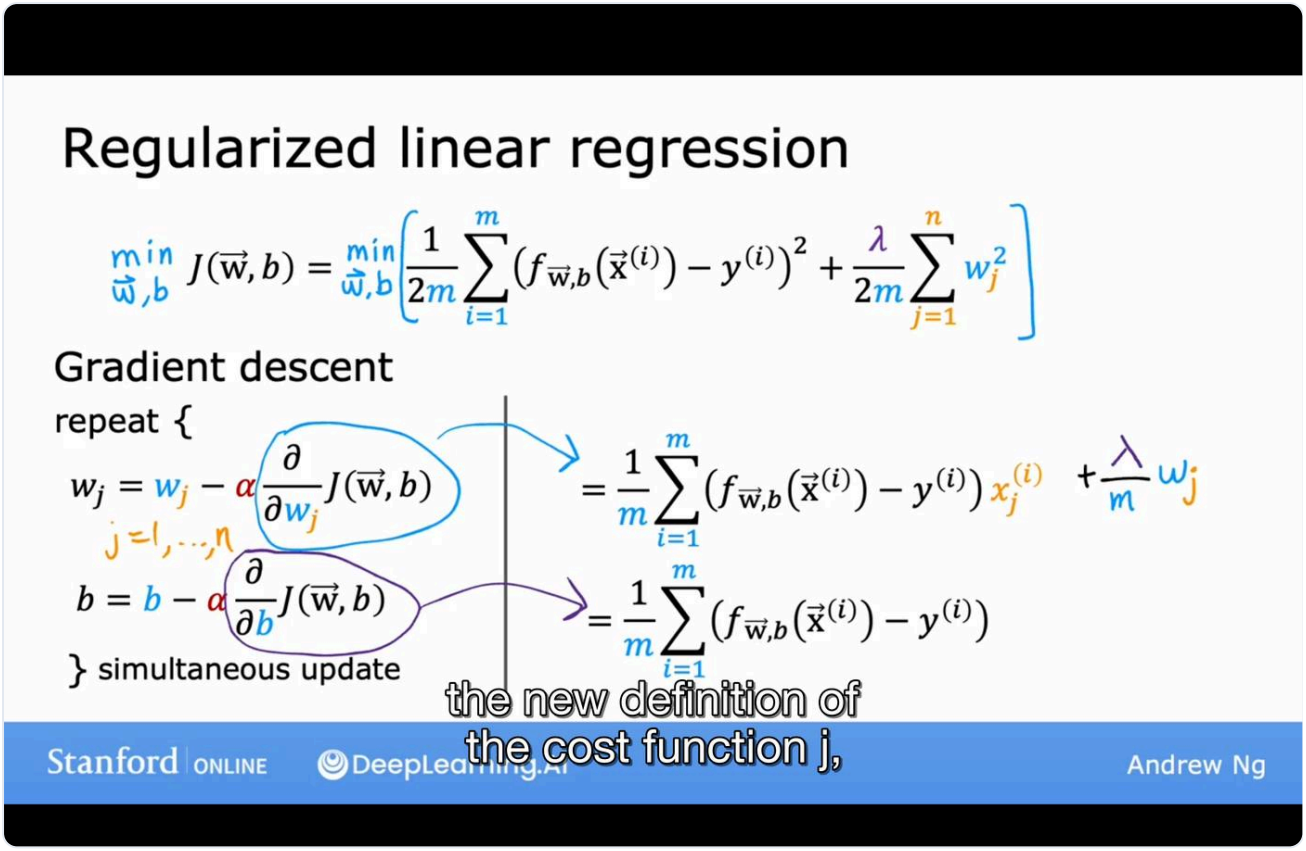


FIGURE 40.5 | Regularized Linear Regression

And in particular for the new definition of the cost function j, these two expressions over here, these are the new derivatives of J with respect to w_j and the derivative of J with respect to b.

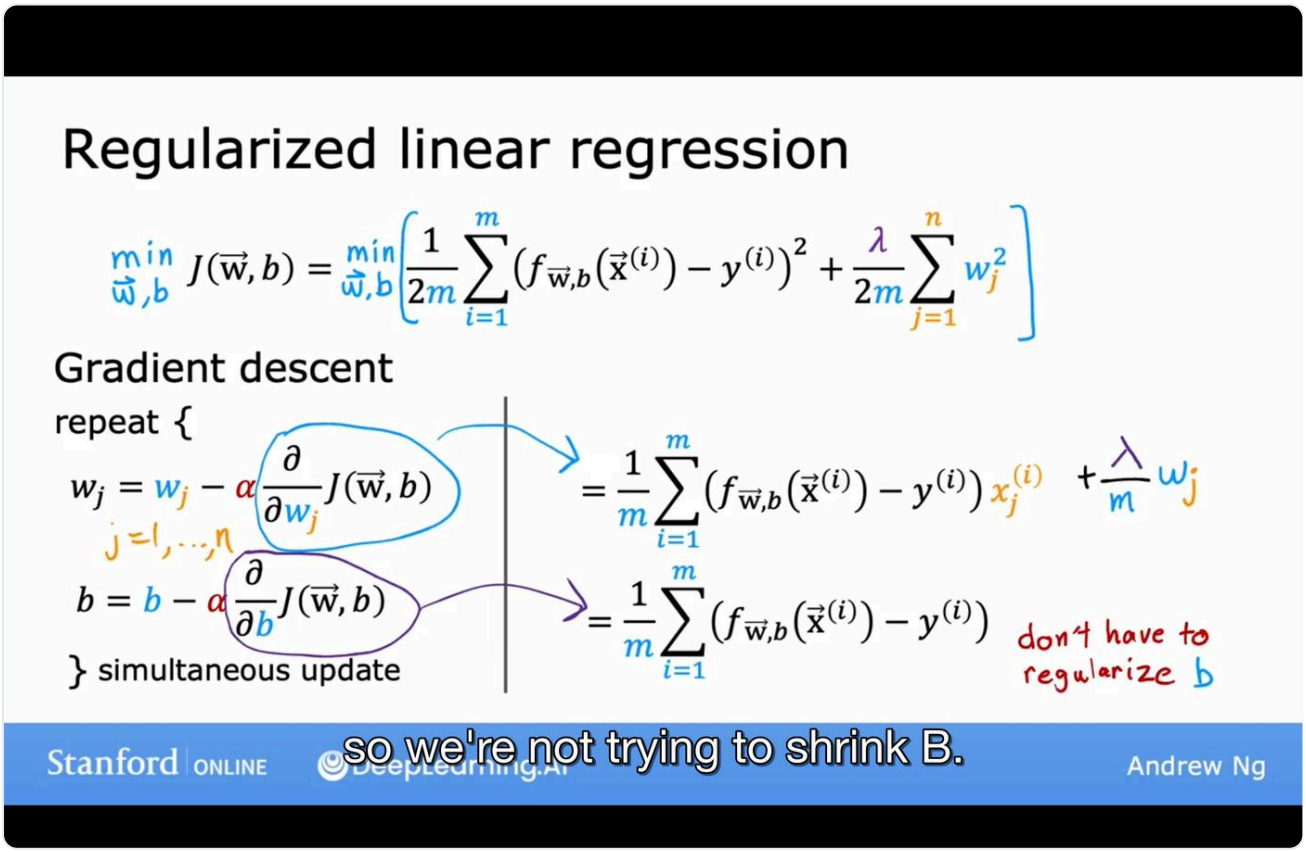


FIGURE 40.6 | Regularized Linear Regression

Recall that we don't regularize b, so we're not trying to shrink B. That's why the updated B remains the same as before, whereas the updated w changes because the regularization term causes us to try to shrink w_j. Let's take these definitions for the derivatives and put them back into the expression on the left to write out the gradient descent algorithm for regularized linear regression. To implement gradient descent for regularized linear regression, this is what you would have your code do. Here is the update for w_j, for j equals 1 through n, and here's the update for b.

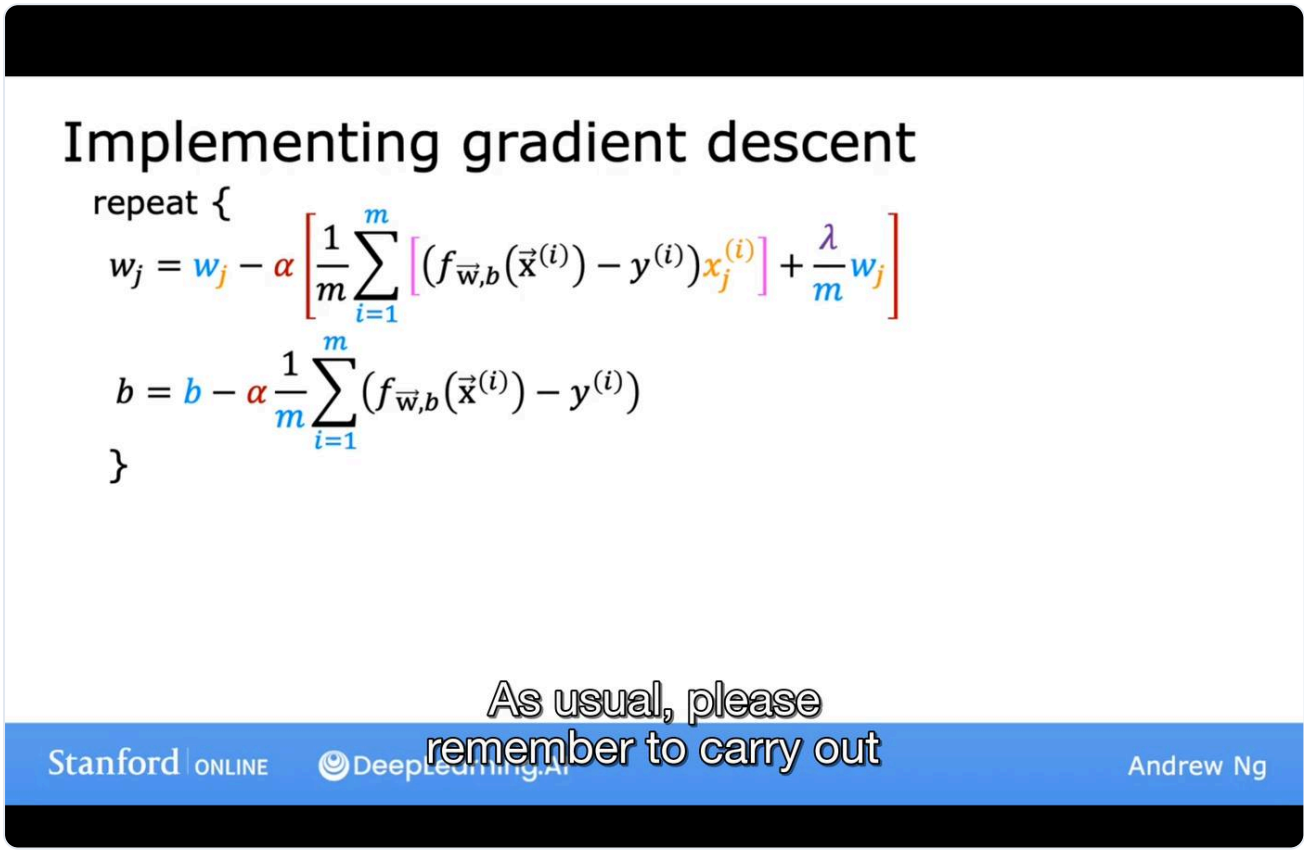


FIGURE 40.7

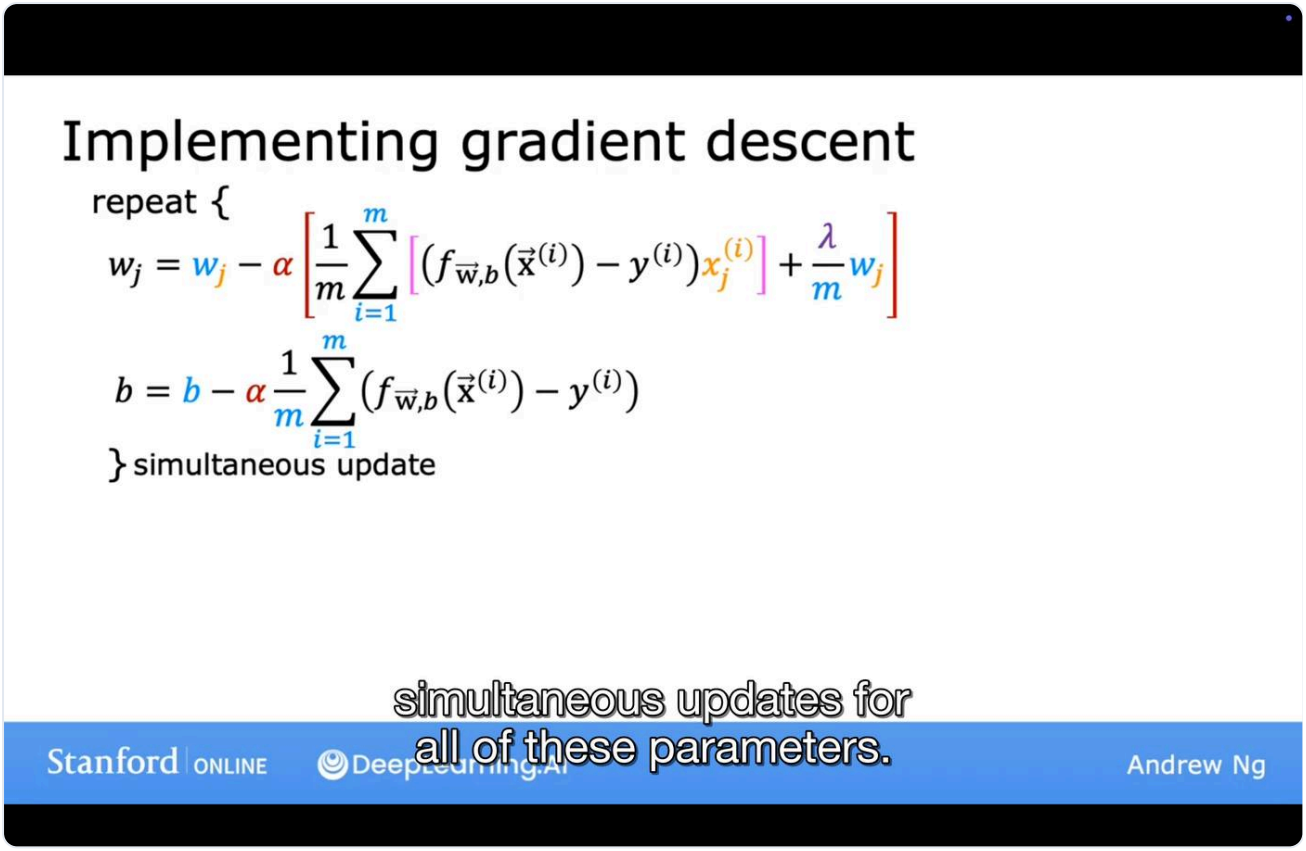


FIGURE 40.8

As usual, please remember to carry out simultaneous updates for all of these parameters. Now, in order for you to get this algorithm to work, this is all you need to know. But what I like to do in the remainder of this video is to go over some optional material to convey a slightly deeper intuition about what this formula is actually doing, as well as chat briefly about how these derivatives are derived. The rest of this video is completely optional. It's completely okay if you skip the rest of this video, but if you have a strong interests in math, then stick with me. It is always nice to hang out with you here, and through these equations, perhaps you can build a deeper intuition about what the math and what the derivatives are doing as well.

Let's take a look. Let's look at the update rule for w_j and rewrite it in another way.

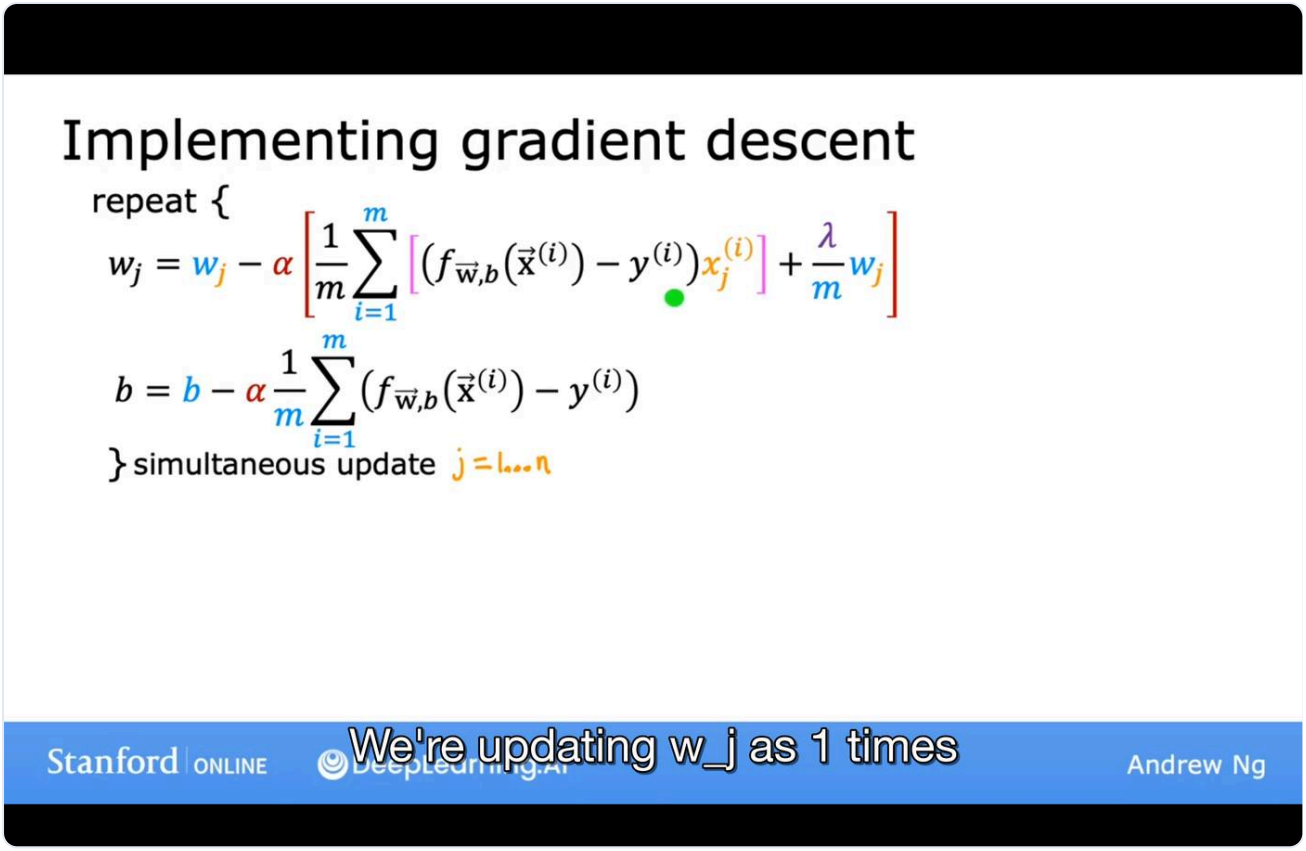


FIGURE 40.9

We're updating w_j as 1 times w_j minus Alpha times Lambda over m times w_j. I've moved the term from the end to the front here. Then minus Alpha times 1 over m, and then the rest of that term over there. We just rearranged the terms a little bit. If we simplify, then we're saying that w_j is updated as w_j times 1 minus Alpha times Lambda over m, minus Alpha times this other term over here.

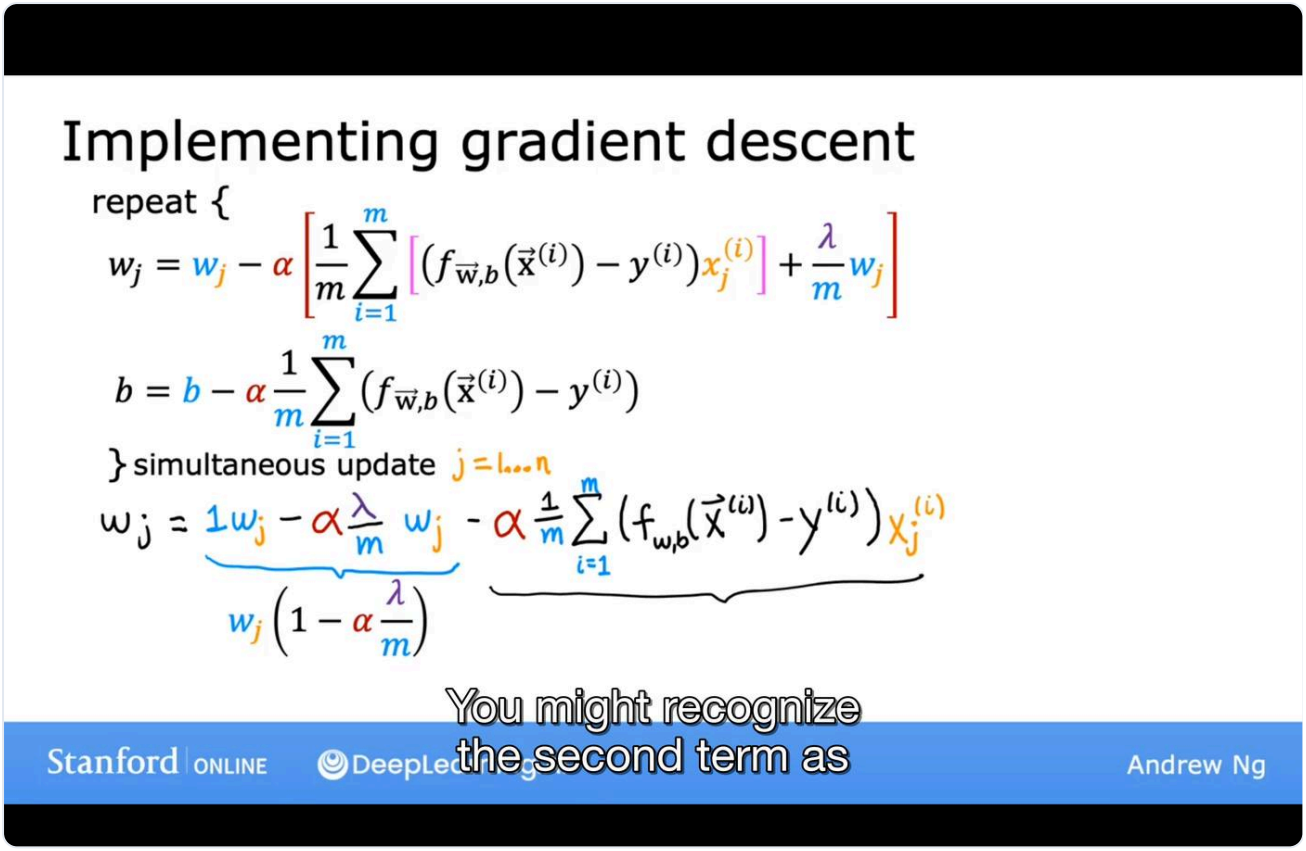


FIGURE 40.10

You might recognize the second term as the usual gradient descent update for unregularized linear regression. This is the update for linear regression before we had regularization, and this is the term we saw in Week 2 of this course. The only change we add regularization is that instead of w_j being set to be equal to w_j minus Alpha times this term is now w times this number minus the usual update. This is what we had in Week 1 of this course.

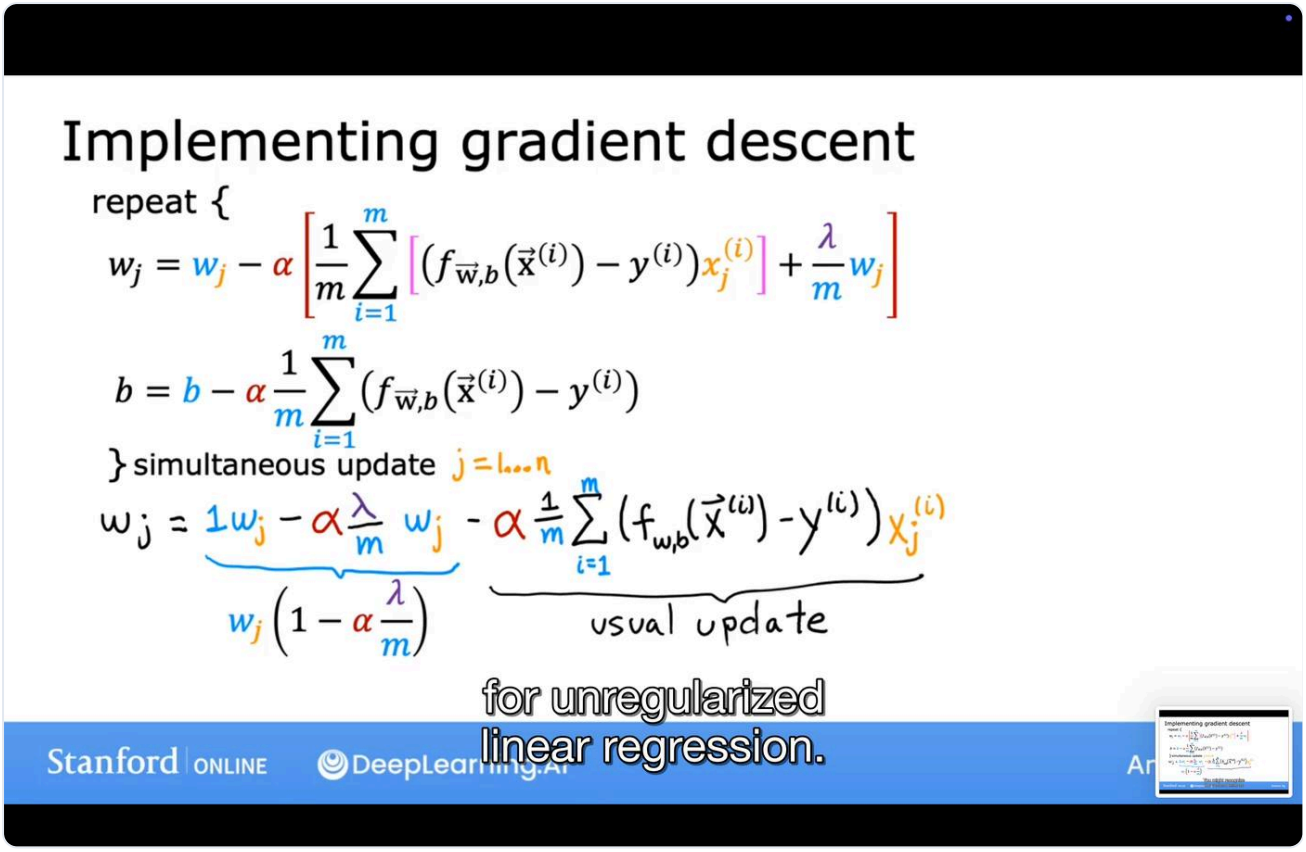
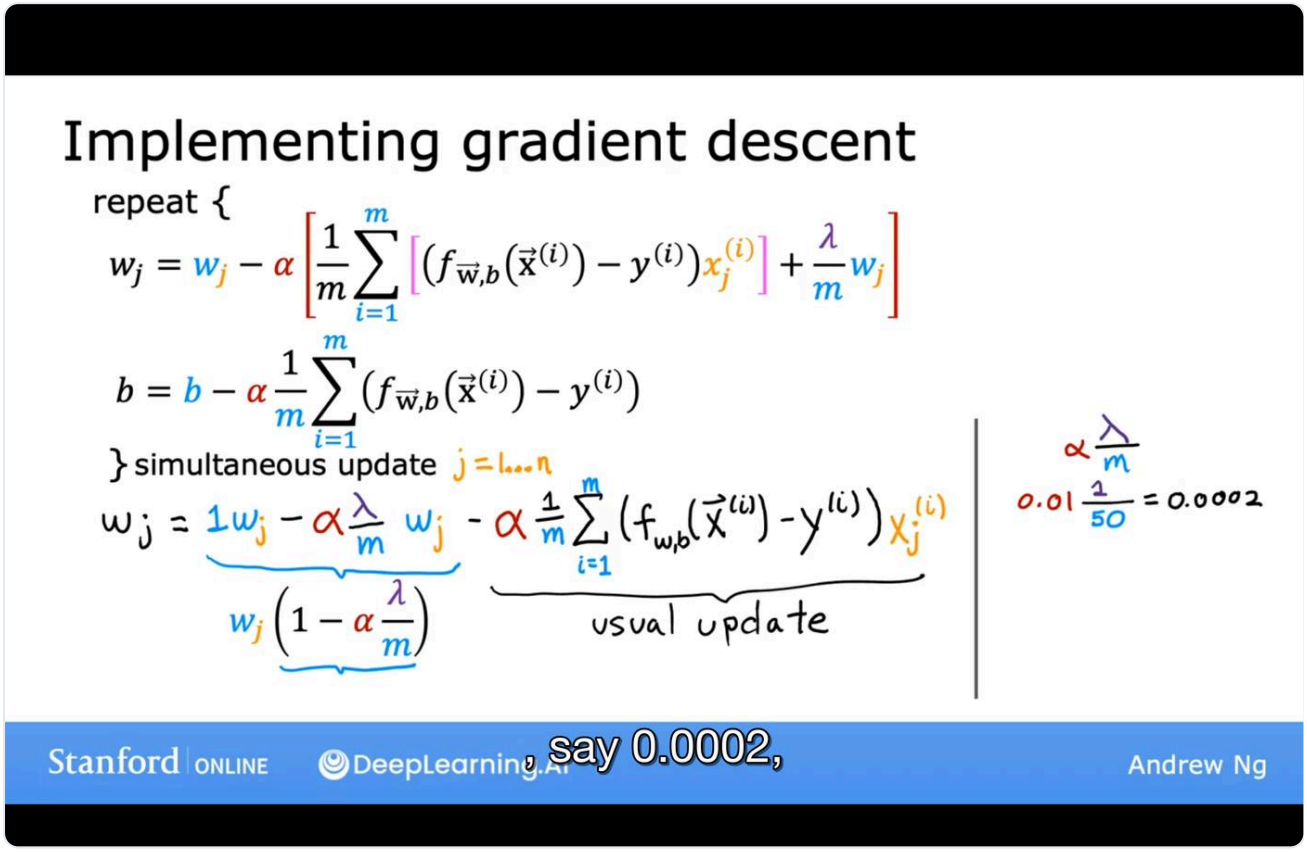


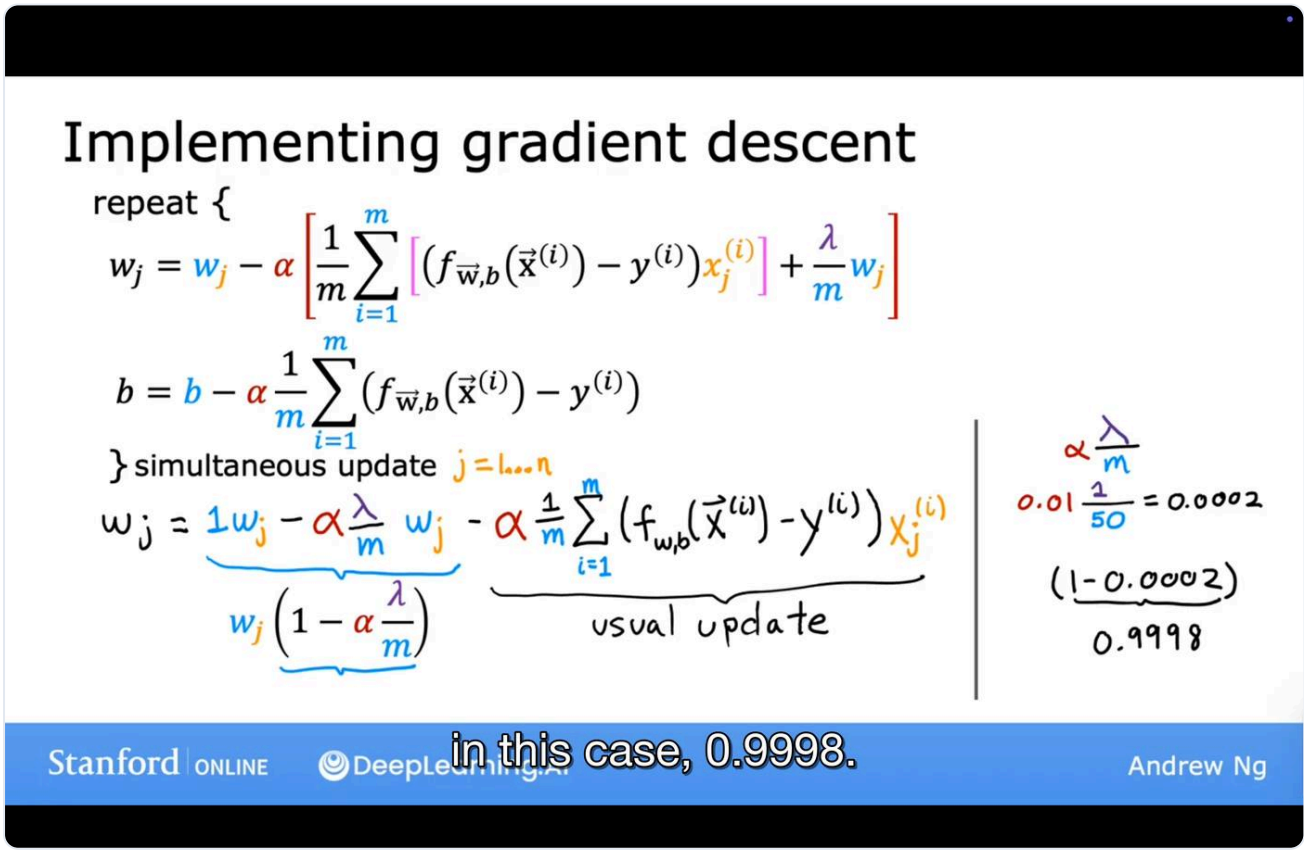
FIGURE 40.11

What is this first term over here? Well, Alpha is a very small positive number, say 0.01. Lambda is usually a small number, say 1 or maybe 10. Let's say lambda is 1 for this example and m is the training set size, say 50.



, say 0.0002,

FIGURE 40.12



in this case, 0.9998.

FIGURE 40.13

When you multiply Alpha Lambda over m, say 0.01 times 1 divided by 50, this term ends up being a small positive number , say 0.0002, and thus, 1 minus Alpha Lambda over m is going to be a number just slightly less than 1, in this case, 0.9998.

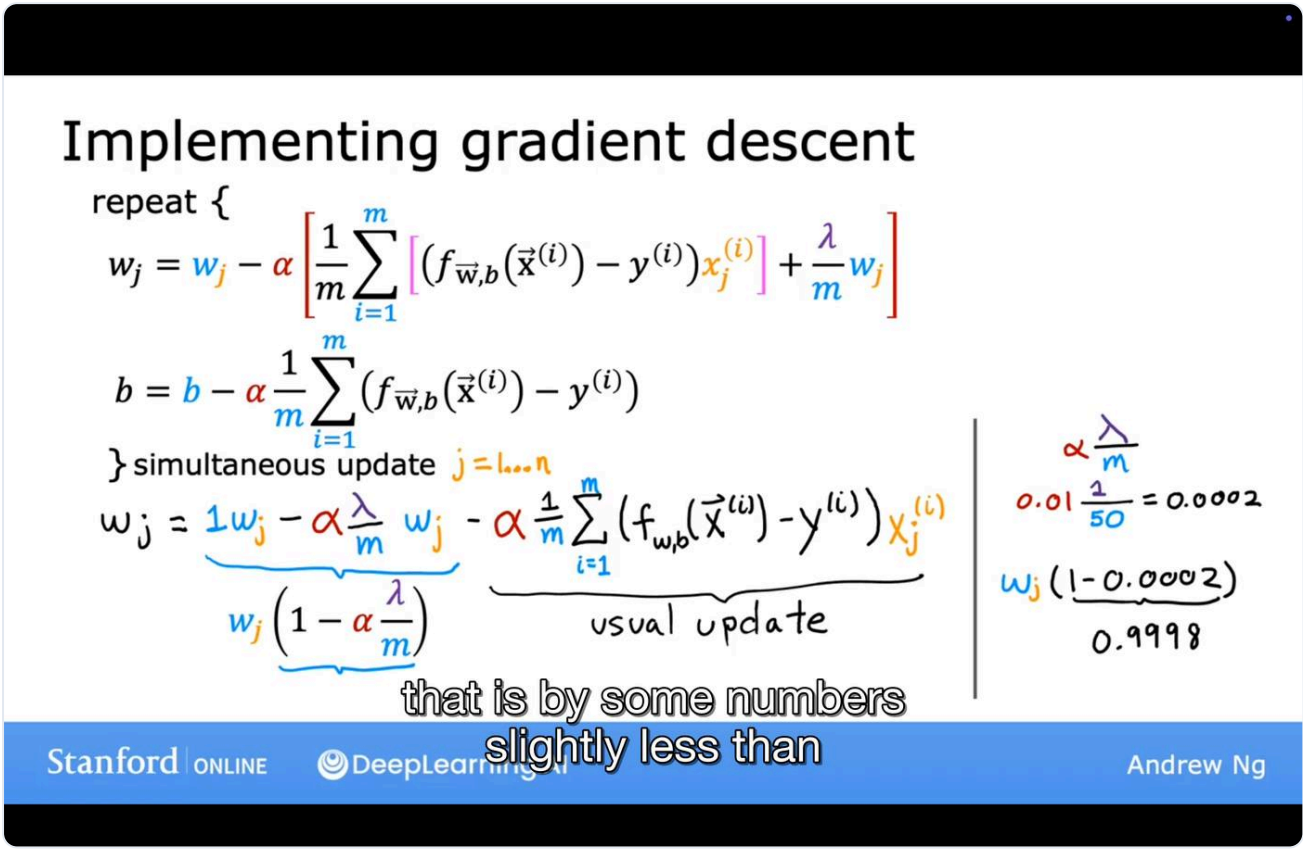


FIGURE 40.14

The effect of this term is that on every single iteration of gradient descent, you're taking w_j and multiplying it by 0.9998, that is by some numbers slightly less than one and for carrying out the usual update.

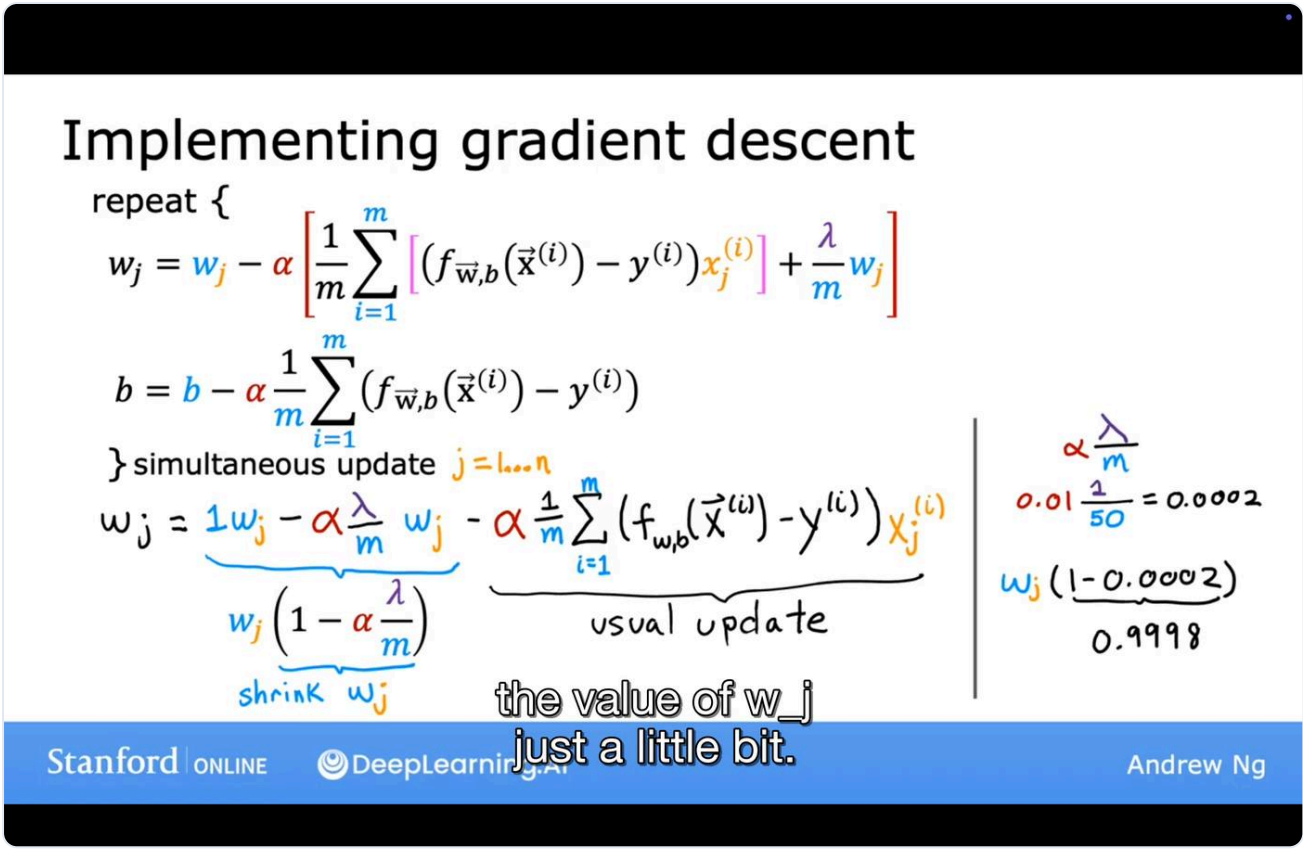


FIGURE 40.15

What regularization is doing on every single iteration is you're multiplying w by a number slightly less than 1, and that has effect of shrinking the value of w_j just a little bit. This gives us another view on why regularization has the effect of shrinking the parameters w_j a little bit on every iteration, and so that's how regularization works. If you're curious about how these derivative terms were computed, I've just one last optional slide that goes through just a little bit of a calculation of the derivative term. Again, this slide and the rest of this video are completely optional, meaning you won't need any of this to do the practice labs and the quizzes.

Let's step through quickly to derivative calculation. The derivative of J with respect to w_j looks like this.

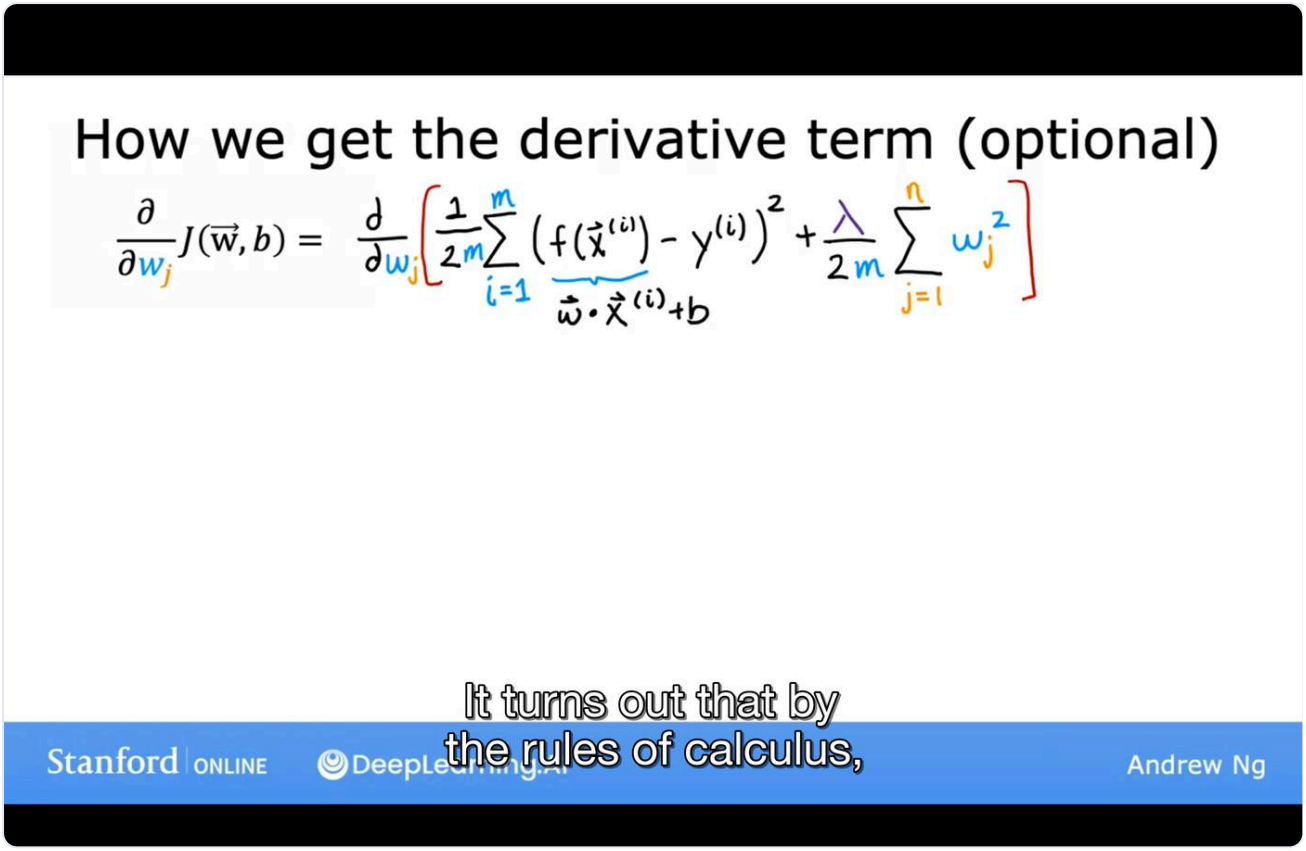


FIGURE 40.16 | How We Get the Derivative Term (Optional)

Recall that f of x for linear regression is defined as w dot x plus b or w dot product x plus b. It turns out that by the rules of calculus, the derivatives look like this, is 1 over 2m times the sum i equals 1 through m of w dot x plus b minus y times 2x_j plus the derivative of the regularization term, which is Lambda over 2m times 2 w_j. Notice that the second term does not have the summation term from j equals 1 through n anymore. The 2's cancel out here and here, and also here and here. It simplifies to this expression over here.

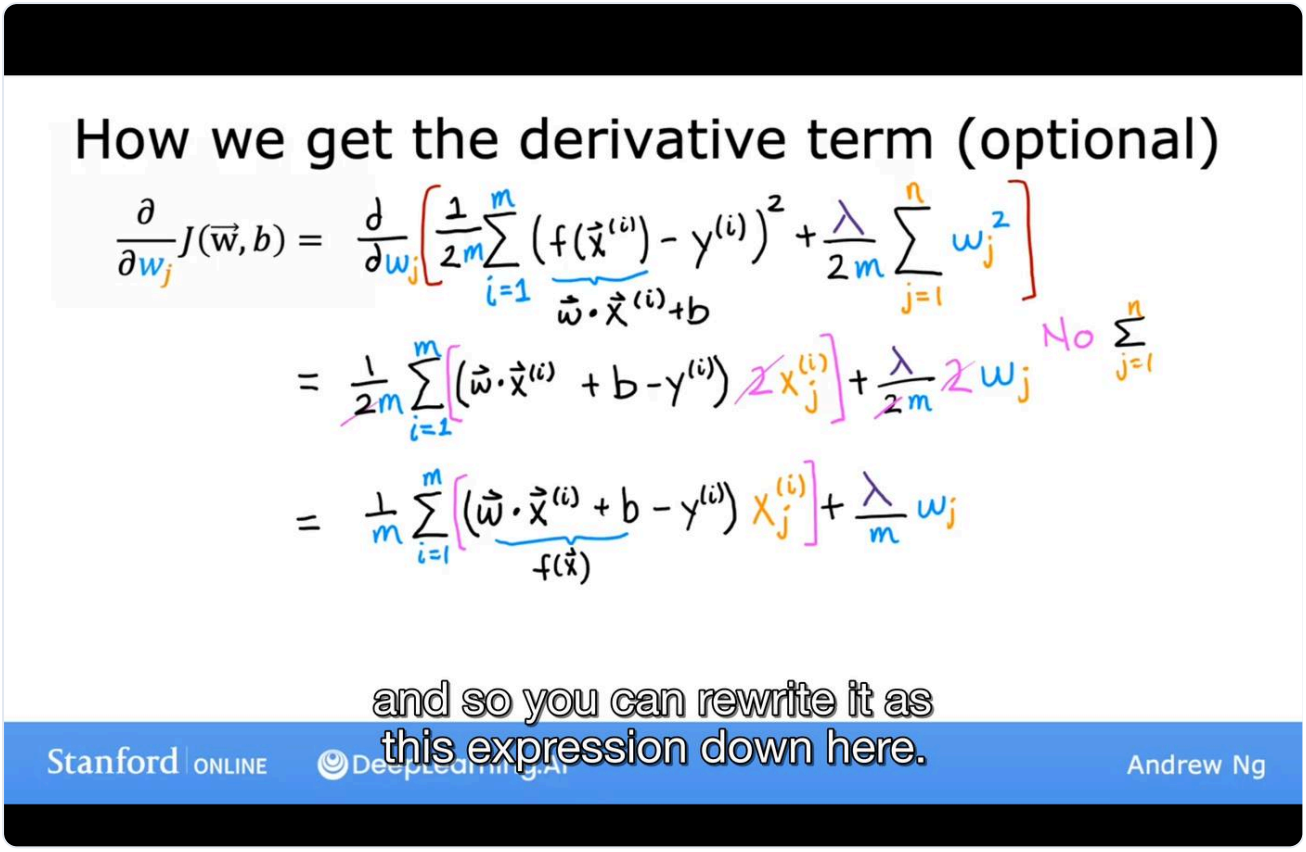


FIGURE 40.17 | How We Get the Derivative Term (Optional)

Finally, remember that wx plus b is f of x, and so you can rewrite it as this expression down here.

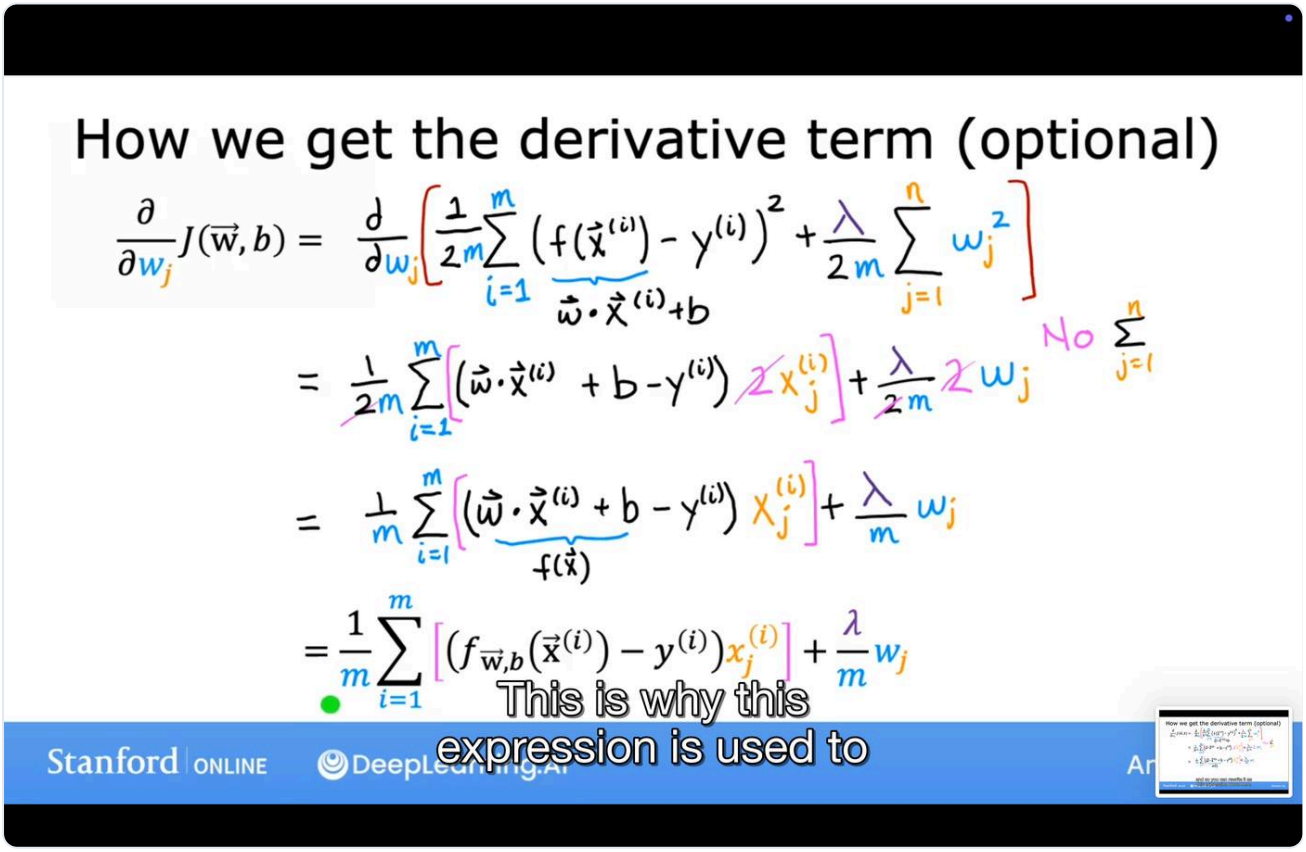


FIGURE 40.18 | How We Get the Derivative Term (Optional)

This is why this expression is used to compute the gradient in regularized linear regression. You now know how to implement regularized linear regression. Using this, you really reduce overfitting when you have a lot of features and a relatively small training set. This should let you get linear regression to work much better on many problems. In the next video, we'll take this regularization idea and apply it to logistic regression to avoid overfitting for logistic regression as well. Let's take a look at that in the next video.

LECTURE 41

Regularized Logistic Regression

In this video, you see how to implement regularized logistic regression. Just as the gradient update for logistic regression has seemed surprisingly similar to the gradient update for linear regression, you find that the gradient descent update for regularized logistic regression will also look similar to the update for regularized linear regression. Let's take a look. Here is the idea. We saw earlier that logistic regression can be prone to overfitting if you fit it with very high order polynomial features like this.

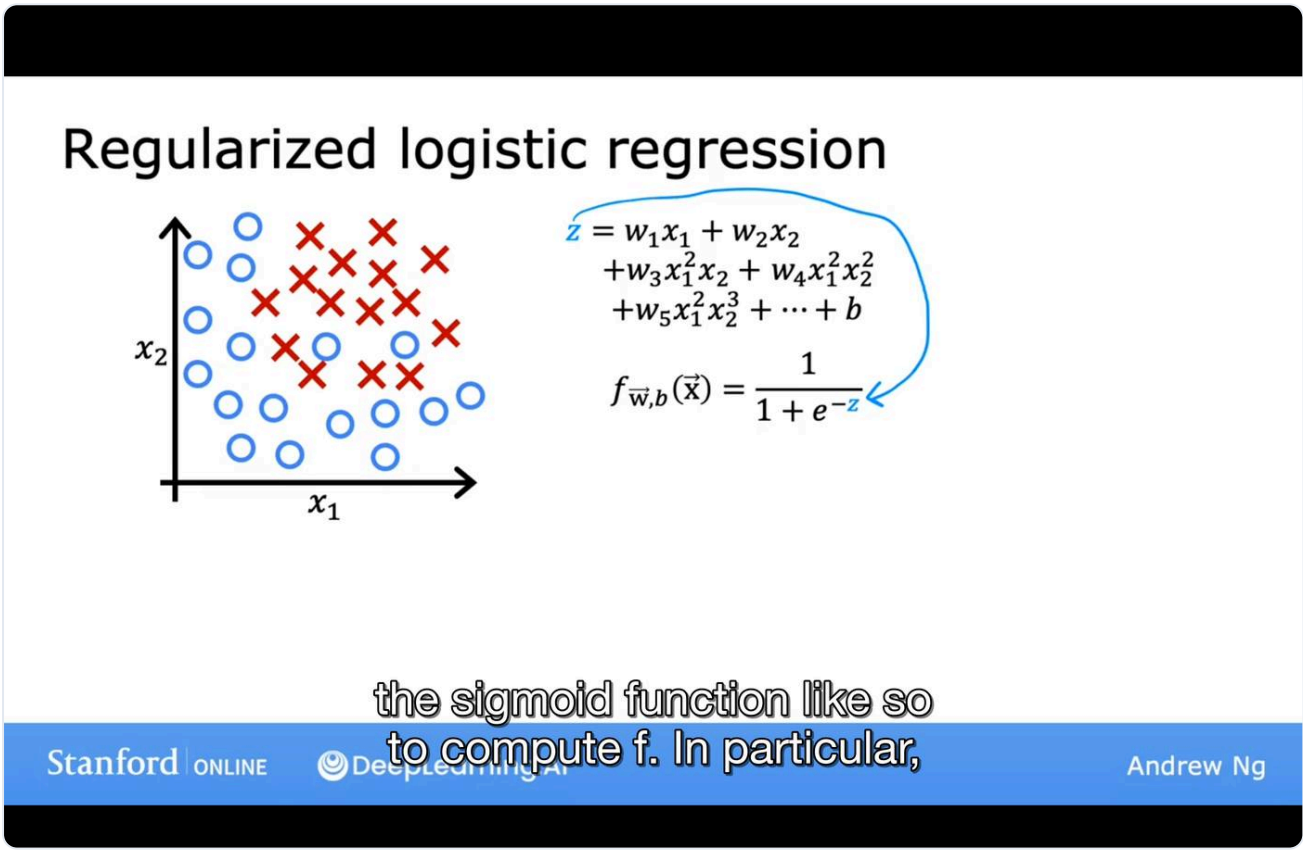


FIGURE 41.1 | Regularized Logistic Regression

Here, z is a high order polynomial that gets passed into the sigmoid function like so to compute f. In particular, you can end up with a decision boundary that is overly complex and overfits as training set. More generally, when you train logistic regression with a lot of features, whether polynomial features or some other features, there could be a higher risk of overfitting. This was the cost function for logistic regression.

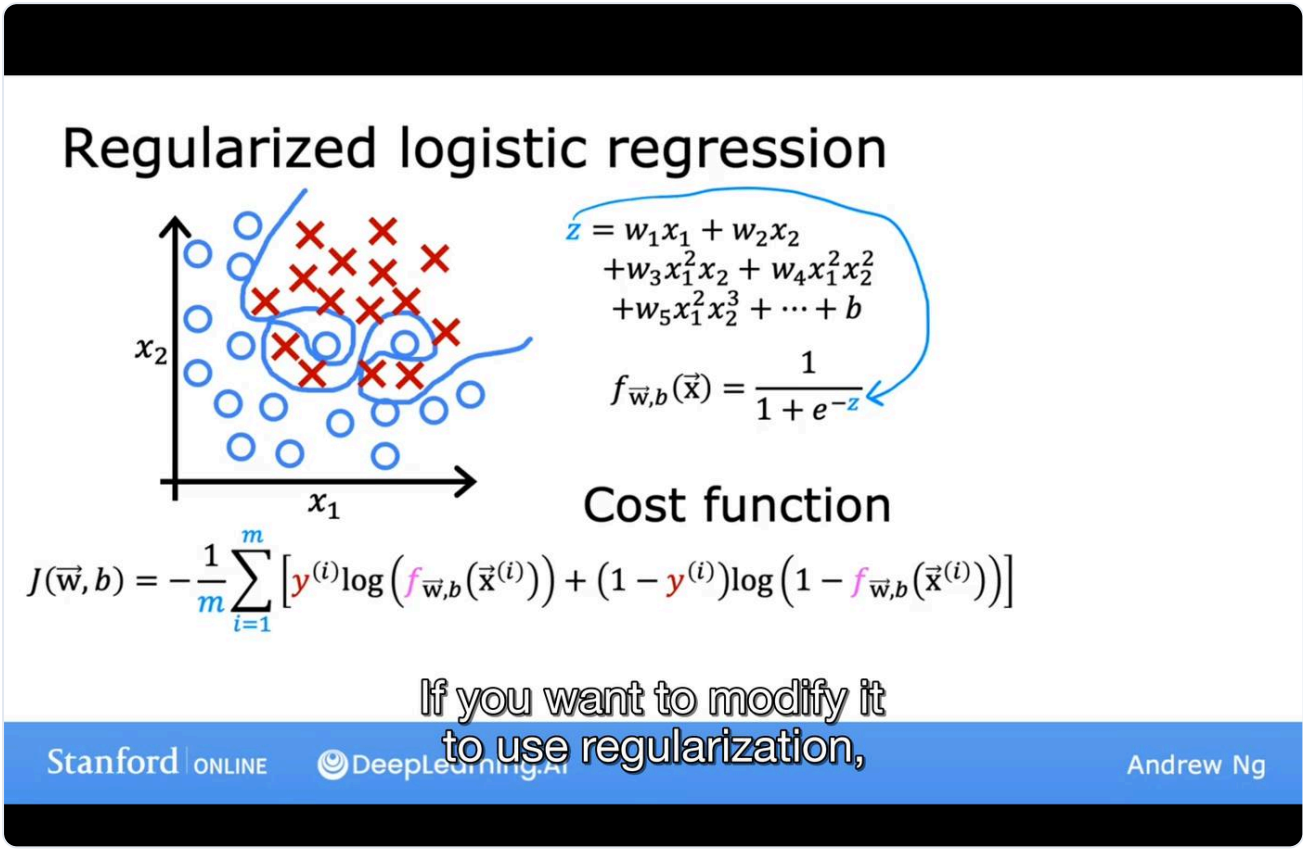


FIGURE 41.2 | Regularized Logistic Regression

If you want to modify it to use regularization, all you need to do is add to it the following term.

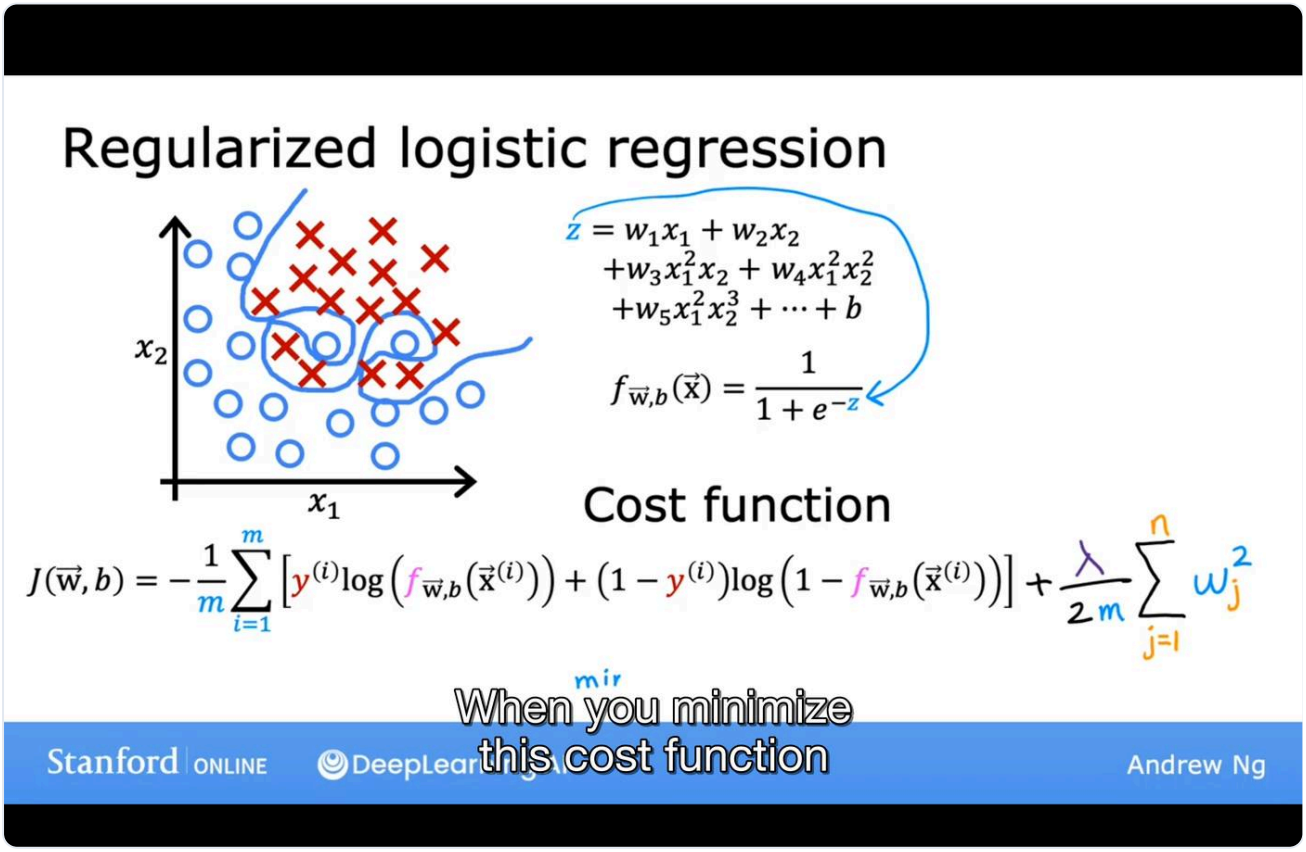


FIGURE 41.3 | Regularized Logistic Regression

Let's add lambda to regularization parameter over 2m times the sum from j equals 1 through n, where n is the number of features as usual of wj squared.

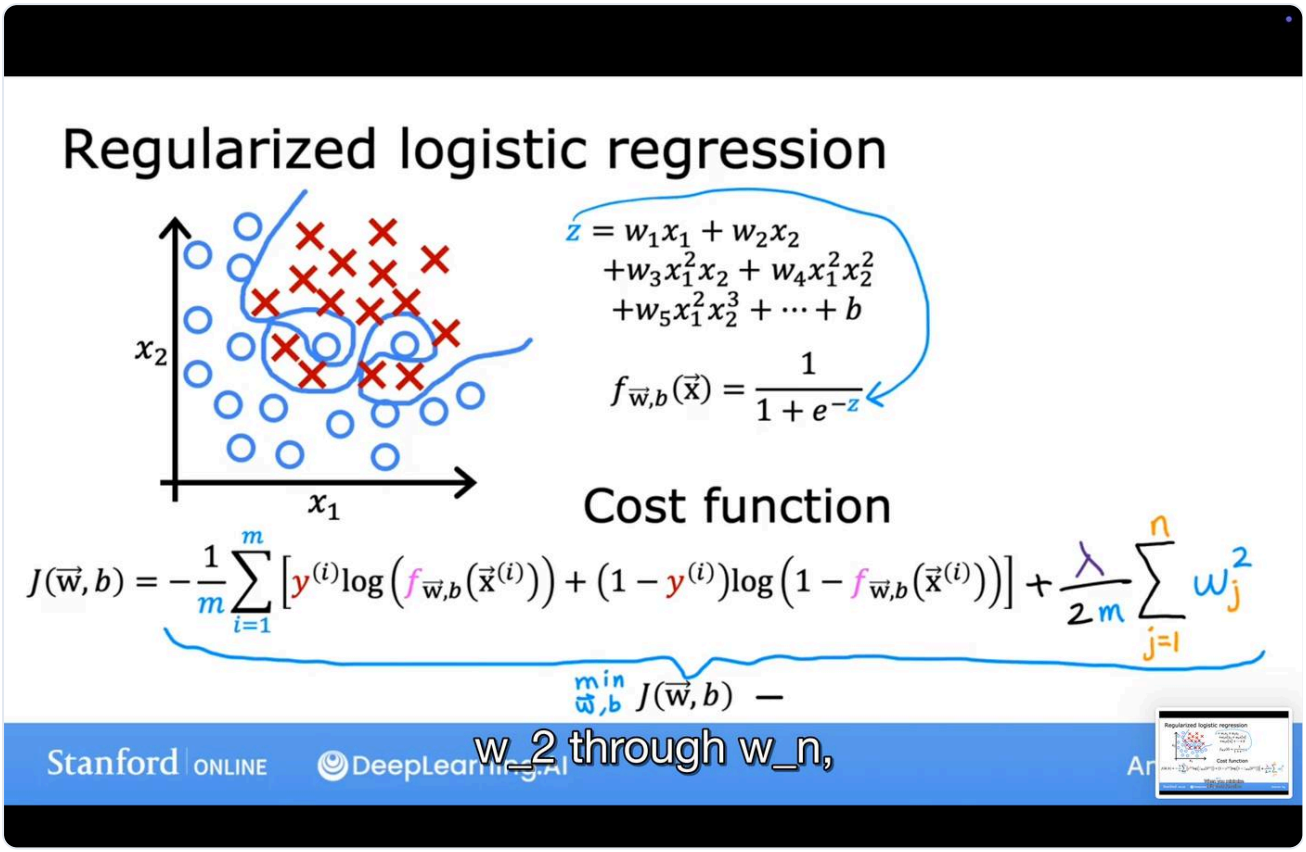


FIGURE 41.4 | Regularized Logistic Regression

When you minimize this cost function as a function of w and b, it has the effect of penalizing parameters w_1, w_2 through w_n, and preventing them from being too large. If you do this, then even though you're fitting a high order polynomial with a lot of parameters, you still get a decision boundary that looks like this. Something that looks more reasonable for separating positive and negative examples while also generalizing hopefully to new examples not in the training set. When using regularization, even when you have a lot of features.

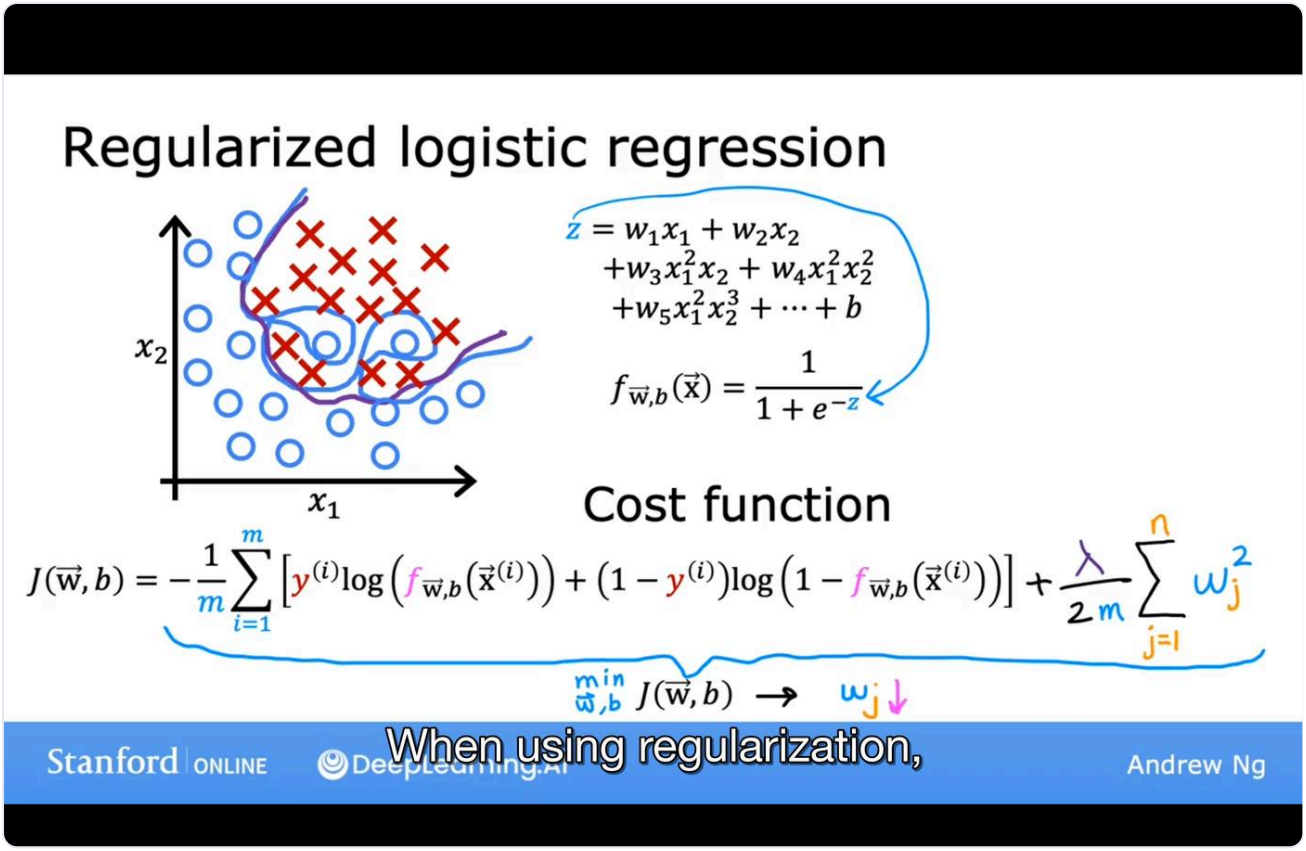


FIGURE 41.5 | Regularized Logistic Regression

How can you actually implement this? How can you actually minimize this cost function j of wb that includes the regularization term? Well, let's use gradient descent as before. Here's a cost function that you want to minimize. To implement gradient descent, as before, we'll carry out the following simultaneous updates over wj and b.

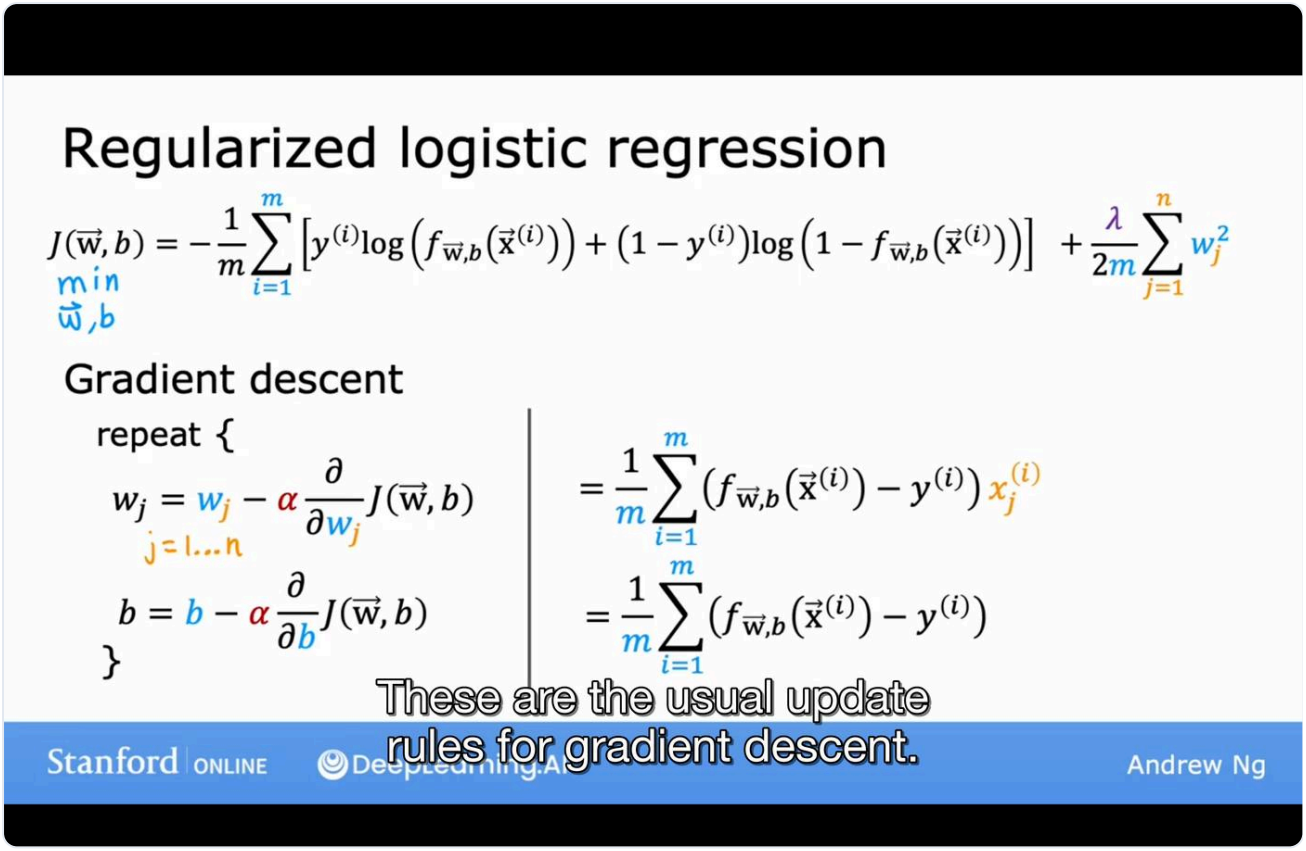


FIGURE 41.6 | Regularized Logistic Regression

These are the usual update rules for gradient descent. Just like regularized linear regression, when you compute where there are these derivative terms, the only thing that changes now is that the derivative respect to wj gets this additional term, lambda over m times wj added here at the end. Again, it looks a lot like the update for regularized linear regression.

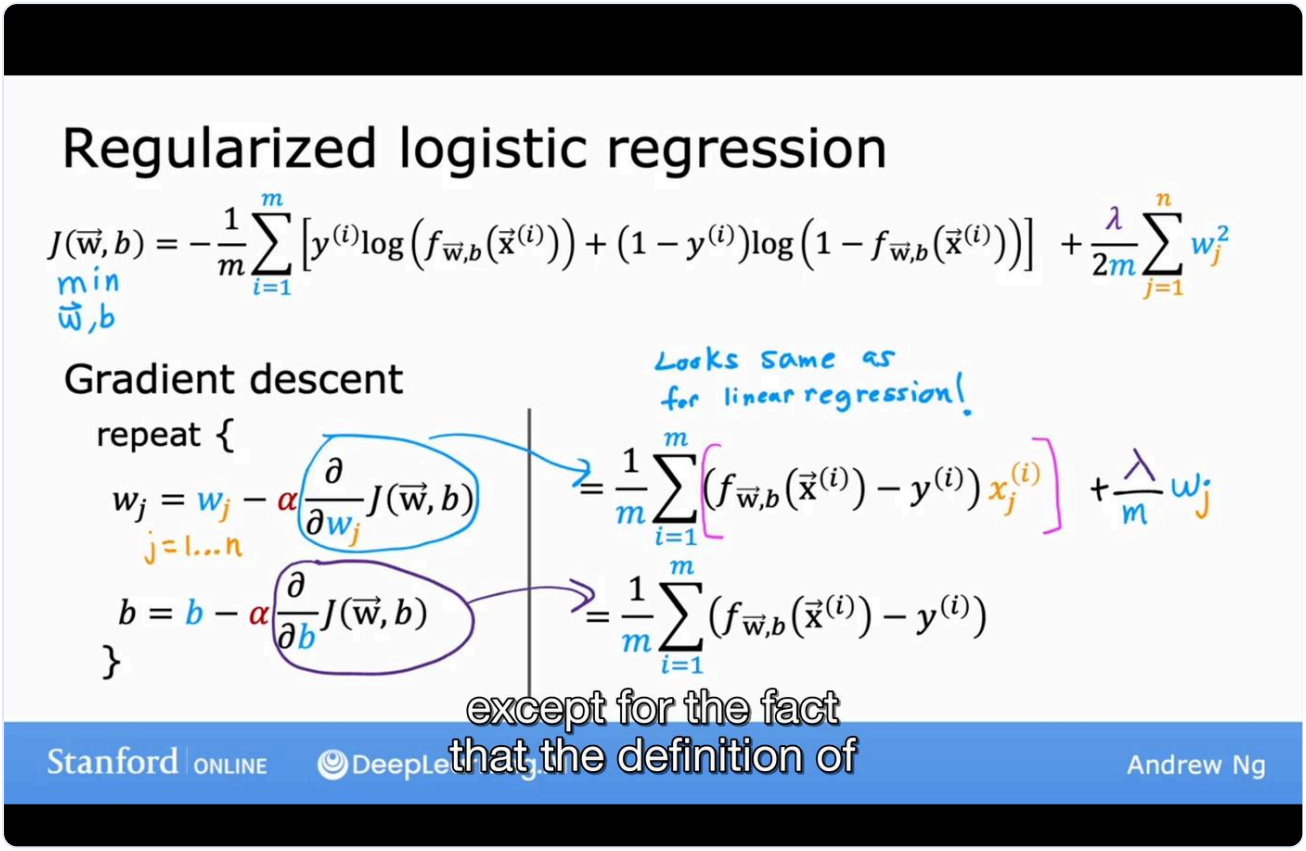


FIGURE 41.7 | Regularized Logistic Regression

In fact is the exact same equation, except for the fact that the definition of f is now no longer the linear function, it is the logistic function applied to z.

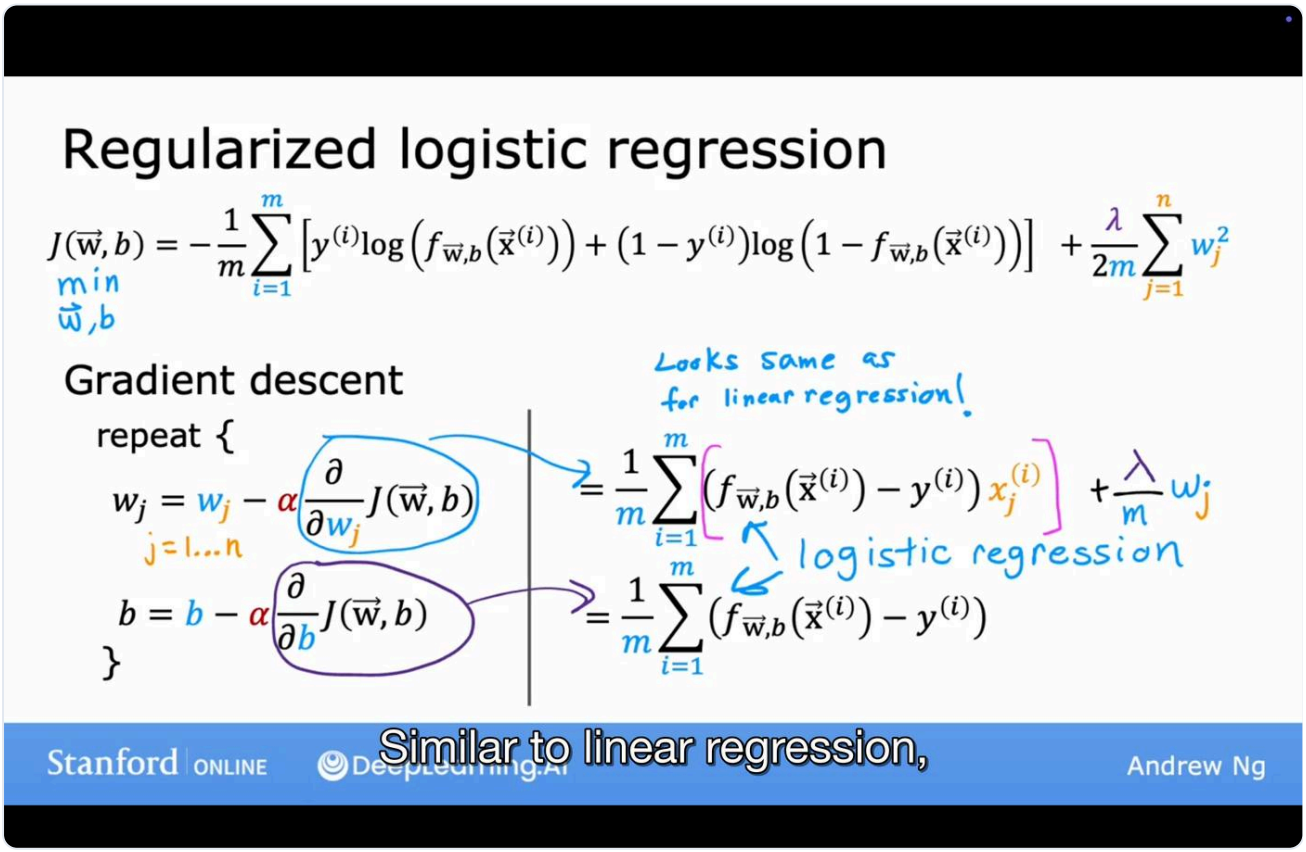


FIGURE 41.8 | Regularized Logistic Regression

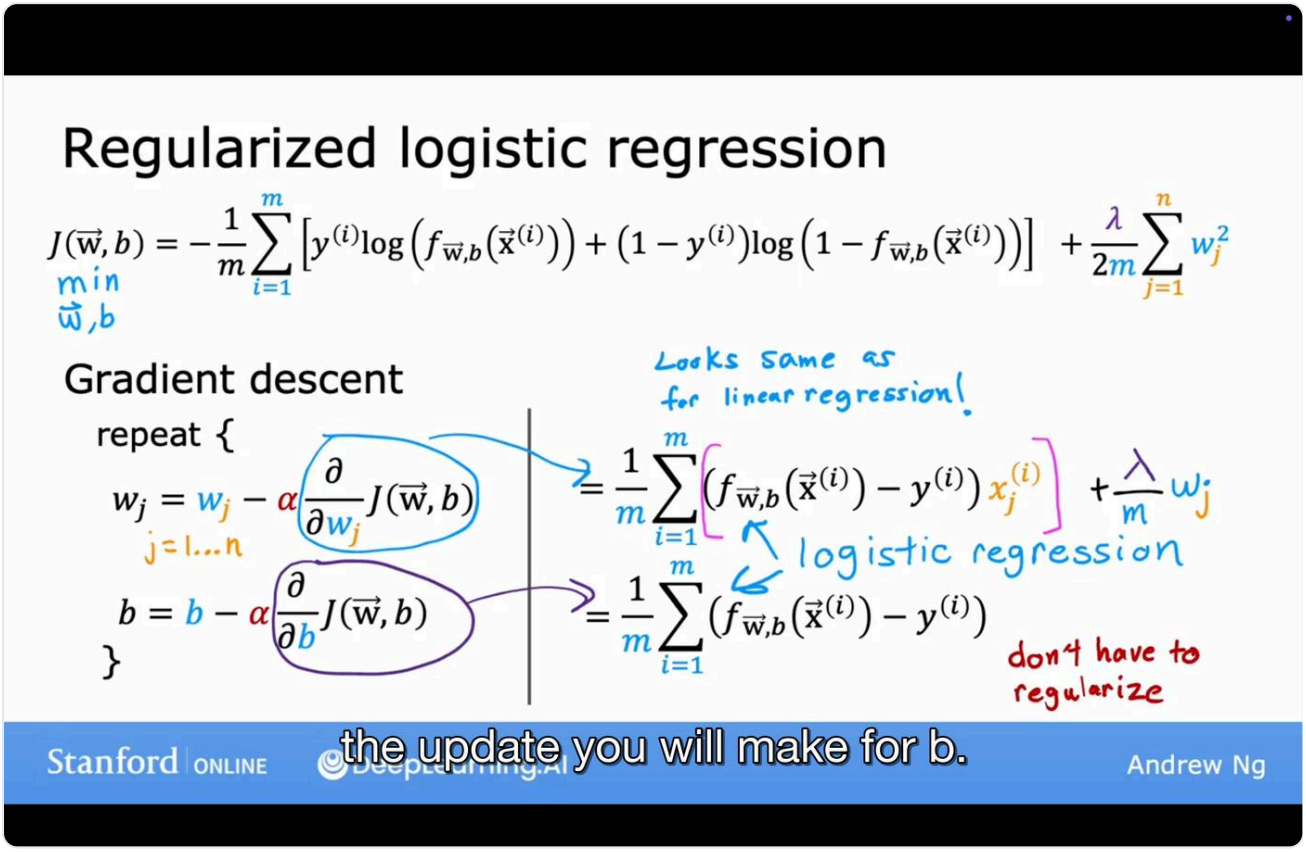


FIGURE 41.9 | Regularized Logistic Regression

Similar to linear regression, we will regularize only the parameters w, j, but not the parameter b, which is why there's no change the update you will make for b. In the final optional lab of this week, you revisit overfitting. In the interactive plot in the optional lab, you can now choose to regularize your models, both regression and classification, by enabling regularization during gradient descent by selecting a value for lambda. Please take a look at the code for implementing regularized logistic regression in particular, because you'll implement this in practice lab yourself at the end of this week. Now you know how to implement regularized logistic regression.

When I walk around Silicon Valley, there are many engineers using machine learning to create a ton of value, sometimes making a lot of money for the companies. I know you've only been studying this stuff for a few weeks but if you understand and can apply linear regression and logistic regression, that's actually all you need to create some very valuable applications. While the specific learning outcomes you use are important, knowing things like when and how to reduce overfitting turns out to be one of the very valuable skills in the real world as well. I want to say congratulations on how far you've come and I want to say great job for getting through all the way to the end of this video.

I hope you also work through the practice labs and quizzes. Having said that, there are still many more exciting things to learn. In the second course of this specialization, you'll learn about neural networks, also called deep learning algorithms. Neural networks are responsible for many of the latest breakthroughs in the eye today, from practical speech recognition to computers accurately recognizing objects and images, to self-driving cars. The way neural network gets built ac-

tually uses a lot of what you've already learned, like cost functions, and gradient descent, and sigmoid functions. Again, congratulations on reaching the end of this third and final week of Course 1.

I hope you have [inaudible] and I will see you in next week's material on neural networks.

LECTURE 42

Computer Vision and Walks of Life

Hi, I'm delighted to have with us here today my old friend, professor Fei-Fei Li. Fei-Fei is a professor of computer science at Stanford University and also co-director of HAI; the Human-Centered AI Institute. Previously, she also was responsible for AI at Google Cloud as a chief scientist for the division. It's great to have you Fei-Fei. Thank you, Andrew. Very happy to be here. How long have we known each other? I've lost track. Definitely more than a decade. I've known your work before we even met and I came to Stanford 2009 but we started talking 2007, so 15 years. I actually still have very clear memories of how stressful it was when collectively bunch of us; me, Chris Manning, bunch of us.

We tried to figure out how to recruit you to come to Stanford. It wasn't hard. I just needed to sort out my student's life, but it's hard to resist Stanford Really great to have you as a friend and colleague here. Me too. It's been a long time and we're very lucky to be the generation. Seeing AI is great progress. There was something about your background I always found inspiring, which is today people are entering AI from all walks of life and sometimes people still wonder, is AI a right path for me? So I thought one of the most interesting parts of your background was that you actually started out not studying computer science or AI, but you started out studying physics and then had this path to becoming one of the most globally recognizable AI scientists.

How did you make that switch from physics to AI? Well, that's a great question, Andrew, especially both of us are passionate about young people's future and they come into the world of AI. The truth is, if I could enter AI back then more than 20 years ago today, anybody can enter AI because AI has become such a prevailing and globally impactful technology. But myself, maybe I was an accident. I have always been a physics kid or a STEM kid. I'm sure you were too, but physics was my passion all the way through middle school, high school, college and I went to Princeton and majored in physics. One thing physics has taught me till today is really the passion for asking big questions, the passion for seeking no stars.

I was really having fun as a physics student at Princeton. One thing I did was reading up stories and just writings of great physicists of the 20th century and just hear about what they think about the world, especially people like Albert Einstein, Roger Penrose, Erwin Schrodinger, and it was really funny to notice that many of the writings towards the later half of the career of these great physicists were not about just the atomic world or the physical world, but ponderings about equally audacious questions like life, like intelligence, like human conditions. Schrodinger wrote this book, What is Life? Roger Penrose wrote this book, Emperor's New Mind.

That really got me very curious about the topic of intelligence. One thing led to another during college time, I did intern at the top of neuroscience labs and especially vision-related, and I was like, wow, this is just as audacious question to ask as the beginning of the universe or what is matter made of? That got me to switch from undergraduate degree in physics to graduate degree in AI. Even though I don't know about you during our time, AI was a dirty word. It was AI winter,

so it was more machine learning and computer vision, and computational neuroscience. Yeah, I know. Honestly, I think when I was an undergrad, I was too busy writing code. I just managed to blithely ignore the AI winter and just kept on coding.

Yeah, well, I was too busy solving PDE equations. Actually, do you have an audacious question now? Yes, my audacious question is still intelligence. I think since Alan Turing, humanity has not fully understand what is the fundamental computing principles behind intelligence. Today we use the words AI, we use the word AGI, but at the end of the day, I still dream of a set of simple equations or simple principles that can define the process of intelligence, whether it's animal intelligence or machine intelligence. This is similar to physics. For example, many people have drawn the analogy of flying. Are we replicating birds flying or are we building airplane?

A lot of people ask the question of the relationship between AI and brain. To me, whether we're building a bird or replicating a bird or building an airplane, at the end of the day, aerodynamics and physics that govern the process of flying, and I do believe one day we'll discover that. I sometimes think about this one learning algorithm hypothesis. Could a lot of intelligence, maybe not all, but a lot of it be explained by one or a very simple machine learning principle? It feels like we're still so far from cracking that nut. But in the weekends when I have spare time when I think about learning algorithms and where they could go, this is one of the things I'm excited about.

Just thinking about that. I totally agree. I still feel like we're pre-Newtonian if we're doing physics analogy before Newton. There has been great physics, great physicists, a lot of phenomenology, a lot of studies of how the astral bodies move and all that. But it was Newton who started to write very simple laws. I think we are still going through that very exciting coming of age of AI as a basic science. We're pre-Newton in my opinion. It's really nice to hear you talk about how despite machine learning and AI having come so far. It still feels like there are a lot more unanswered questions, a lot more work to be done by maybe some of the people joining the field today than work that's already been done.

Absolutely. Let's calculate, it's only 160 years about. It's a very nascent field. Modern physics and chemistry and biology are all hundreds of years. I think it is very exciting to be entering the field of science of intelligence and studying AI today. Yeah, actually that's good. I think I remember chatting with the late Professor John McCarthy who had coined the term artificial intelligence, and boy, the field has changed since when he conceived of it at a workshop and came up with the term AI. But maybe another ten years from now, maybe someone watching this will come with a new set of ideas and then we'll be saying, boy, AI show us different than what you and I thought it would be.

That's the exciting future to build towards. Yeah, I'm sure Newton would have not dreamed of Einstein. Our evolution of science sometimes takes strides, sometimes takes a while, and I think we're absolutely in exciting phase of AI right now. It's interesting hearing you paint this grand vision for AI. Going back a little bit, there was one of the piece of your background that I found inspiring, which is when you're just getting started, I've heard you speak about how your physics student, but not only that, you're also running a laundromat to pay for school. Just tell us more

about that. I came to this country, to New Jersey actually when I was 15. One thing great about being in New Jersey is it was close to Princeton so I often just take a weekend trip with my parents and to admire the place where Einstein spent most of his career in the latter half of his life.

But with typical immigrant life and it was tough. By the time I enter Princeton, my parents didn't speak English and one thing led to another. It turns out running a dry cleaner might be the best option for my family, especially for me to lead that business because it's a weekend business. If it's a weekday business, it would be hard for me to be a student. It's actually, believe or not, running a dry cleaning shop is very machine-heavy, which is good for a STEM student like me. We decided to open a dry cleaner shop in a small town in New Jersey called Parsippany in New Jersey. It turned out we were physically not too far from Bell Labs and where lots of early convolutional neural network research was happening, but I had no idea.

It's just summer intern at the ancient. That is right, with Rob Shapiro. With Michael Kearns was my mentor and Rob Shapiro, inventor of those things creator algorithms. You're encoding AI. I was trying to [inaudible] Only much later that I started interning. Yeah. Then it was seven years. I did that for the entire undergrad and most of my grad school and I hire my parents. Yeah. Yeah. Now, that's really inspiring. I know you've been bred into doing exactly where all your life. I think the story of running a laundromat to globally prominent computer scientists. I hope that inspires some people watching this that no matter where you are, there's plenty of for everyone.

I don't even know this. My high-school job was office admin and so to this day I remember doing a lot of photocopying. The exciting part was using this shredder. That was a glamorous one. I was doing so much coffee in high school I thought, boy, felony, I could build a robot to do this for the coffee, maybe I could do something. Did you succeed? Still working on it. When people think about you and the work you've done, one of the huge successes everyone thinks about this ImageNet where help establish early benchmark for computer vision. It was really completely instrumental to the modern rise of deep learning and computer vision. One thing I bet not many people know about is how you actually got started on ImageNet.

Tell us the origin story of ImageNet. Yeah. Well Andrew, that's a good question because a lot of people see ImageNet as just labeling a ton of images. But where we began was really going after a Northstar brings back my physics background. When I enter grad school, when did you enter grad year? '97. I was three years later that year, 2000. That was a very exciting period because I was in computer vision and computational neural science lab of Pietro Purana and Christoph car at Caltech. Leading up to that, there has been, first of all, two things was very exciting. One is that the world of AI at that point wasn't called AI. Computer vision or natural language processing has founded Lingua Franca.

It's machine learning, statistical modeling as a new tool has emerged. I mean, it's been around. I remember when the idea of applying machine learning to computer vision, that was a controversial thing. I was the first generation of graduate students who were embracing all the base net, all the inference algorithms and all that. That was one exciting happening. A certainly exciting happening that most people don't know and don't appreciate is the couple of decades probably one of

them two or three decades of incredible cognitive science and cognitive neuroscience work in the field of vision, in the world of vision, human vision. That has really established a couple of really critical Northstar problems.

Just understanding of human visual processing and human intelligence. One of them is the recognition of understanding of natural objects and natural things. Because a lot of the psychology and cognitive science work is pointing to us. That is an innately optimized, whatever that word is. Functionality and the ability of human intelligence is more robust, faster, and more nuanced than we had thought. We even find neural correlates, brain areas devoted to faces or places or body parts. These two things lead to my PhD study of using machine-learning methods to work on real-world objects recognition. But it becomes very painful very quickly that we are coming banging against one of the most continued to be the most important challenge in AI machine learning is the lack of generalizability.

You can design a beautiful model or you want if you're overfitting the model. I remember when it used be possible to publish a computer vision and paper showing it works on one image. Exactly. It's just the overfitting. The models are not very expressive, and we lack the data. We also as a field was betting on making the variables very rich by hand engineered features. Remember, every variable carrying a ton of semantic meaning, but with hand engineered features. Then towards the end of my PhD, my advisor, Pietro and I start to look at each other and say, well, boy, we need more data. If we believe in this North Star problem of object recognition, and we look back at the tools we have, mathematically speaking, we're overfitting every model we're encountering.

We need to take a fresh look at this. One thing led to another. He and I decided we'll just do at that point. We think it was a large-scale data project called Caltech 101. I remember the dataset. I wrote papers using your Caltech 101 dataset way back. You did, you and your early graduate student. You have benefit a lot of researchers, that Caltech 101 dataset. That was me and my mom labeling images, and a couple of undergrads, but it was the early days of Internet. Suddenly the availability of data was a new thing. I remember Pietro still have this super expensive digital camera. I think it was Canon or something like $6,000 walking around Caltech taking pictures.

But we're the Internet generation. I go to Google image search, I start to see these thousands and tens of thousands of images and I tell Pietro, "Let's just download. Of course it's not that easy to download. One thing led to another. We build this Caltech 101 dataset of 101 object categories, and about 30,000 pictures. I think us really saying that even though everyone's heard of ImageNet today, even you took a couple of iterations where you did Caltech 101 and that was a success. Lots of people used it for even the early learnings from building Caltech 101. They gave you the basis to build what turned out to be an even bigger success. Except that by the time I became an assistant professor, we started to look at the problem.

I realized it's way bigger than we think. Just mathematically speaking, Caltech 101 was not sufficient to power the algorithms. We decided to do image there. That was the time people start to think we're doing too much. It's just too crazy, the idea of downloading the entire Internet of images, mapping out all the English nouns was a little bit. I start to get a lot of push back. I remember at one of the CVPR conference when I presented the early idea of ImageNet, a couple of re-

searchers publicly questioned and said, "If you cannot recognize one category of object, let's say the chair you're sitting in, how do you imagine or what's the use of a dataset of 22,000 classes of 15 million images.

In the end, that giant dataset unlocked a lot of value for [inaudible] number of researchers around the world. I think it was the combination of betting on the right North Star problem and the data that drives it. It was a fun process. To me when I think about that story, it seems like one of those examples where sometimes people feel like they should only work on projects without the huge thing at the first outset. But I feel like for people working in machine learning, if your first project is a bit smaller, it's totally fine. Have a good win. Use the learning to build up to even bigger things, and then sometimes you get the ImageNet size win all of it.

But in the meantime, I think it's also important to be driven by an audacious goal though. You can size your problem or size your project as local milestones and so on along this journey, but I also look at some of our current students. They're so pure pressured by this current climate of publishing nonstop. It becomes more incremental papers to just get into a publication for the sake of it. I personally always push my students to ask the question, what is the North Star that's driving you? Yeah, that's true. Myself when I do research over the years, I've always pretty much done what I'm excited about, where I want to try to push the view forward. Doesn't have to listen to people.

Have to listen to people let them shape your opinion. But in the end, I think the best researchers let the world shape their opinion, but in the end, drive things for using their own opinion. Totally agree, yeah. It's your own fire. As a research program developed, you've wound up taking your, let's say, foundations in computer vision and neuroscience and apply it to all sorts of topics including your very visibly health care. Looking at neuroscience applications. We'd love to hear a bit more about that. Yeah, happy to. I think the evolution of my research in computer vision also follows the evolution of visual intelligence in animals. There are two topics that truly excites me.

One is what is a truly impactful application area that would help human lives? That's my health care work. The other one is what is vision at the end of the day about? That brings me to trying to close the loop between perception and robotic learning. On the healthcare side, one thing, Andrew, there was a number that shocked me about 10 years ago when I met my long term collaborator, Dr. Arnie Milstein at Stanford Medical School, and that number is about a quarter of a million Americans die of medical errors every year. I had never imagined a number being that high due to medical errors. There are many reasons, but we can rest assure most of the reasons are not intentional.

These are errors of unintended mistakes and solar, for example. That's a mind boggling number. It is. It's been about 40,000 deaths a year from automotive accidents. It's just completely tragic and this is even [inaudible] I was going to say that. I'm glad you brought it up. Just one example. One number within that mind-boggling number is the number of hospital acquired infection resulted fatality is more than 95,000. That's 2.5 times than the death of car accidents. In this particular case, hospital acquired infection as a result of many things. But in enlarge, lack of good hand hygiene practice. If you look at WHO, there has been a lot of protocols about clinicians hand hygiene practice.

But in real health care delivery, when things get busy and when the process is tedious and when there's a lack of feedback system, you still make a lot of mistakes. Another tragic medical fact is that more than $70 billion every year are spent in full resulted injuries and fatalities. Most of this happen to elderlies at home, but also in the hospital rooms. These are huge issues. When Arnie and I got together back in 2012, it was the height of self-driving car, let's say not hype. But what's the right word? Excitement in Silicon Valley. Then we look at the technology of smart sensing cameras, lidars, whatever, smart sensors, machine-learning algorithm, and holistic understanding of a complex environment with high-stakes for human lives.

I was looking at all that for self-driving car and realized in healthcare delivery, we have the same situation. Much of the process, the human behavior process of health care is in the dark. If we could have smart sensors be it in patient rooms or senior homes to help our clinicians and patients to stay safer, that will be amazing. Arnie and I embarked on this, what we call ambient intelligence research agenda. But one thing I learned which probably will lead to our other topics, is as soon as you're applying AI to real human conditions, there's a lot of human issues in addition to machine learning issues, for example, privacy. I remember reading some of your papers with Arnie and found it really interesting how you could build and deploy systems that were relatively privacy preserving.

Yeah, well, thank you. Well, the first iteration of that technology is we use cameras that do not capture RGB information. You've used a lot of that in self-driving car, that depth cameras, for example. There you preserve a lot of privacy information just by not seeing the faces and the identity of the people. But what's really interesting over the past decade is the changes of technology is actually giving us a bigger toolset for privacy, preserved, a computing in this condition, for example, on-device inference. As the chips getting more and more powerful, if you don't have to transmit any data through the network and to the central server, you help people better.

Federated learning, we know it's still early stage, but that's another potential tool for privacy, preserved computing. Then differential privacy and also encryption technologies. We're starting to see that human demand, privacy and other issues is driving actually a new wave of machine learning technology in ambient intelligence in health care. Yeah, I've been encouraged to see your real practical applications of differential privacy that are actually real. Federated Learning as you said, Pray the PRRs little bit ahead of the reality, but I think we'll get there. But it's interesting how consumers in the last several years have, fortunately, gotten much more knowledgeable about privacy and increasingly.

I think the public is also making us to be better scientist. Yeah, I think ultimately people understand the AI hosts everyone, including us, but holds everyone accountable for really doing the right thing. Yeah. On that note, one of the really interesting piece of work you've been doing has been leading several efforts to help educate legislators. I hope governments, especially US government, work through better laws and better regulation, especially as it relates to AI. That sounds a very important and I suspect some days they'll be, I would guess, somewhat frustrating work, but we'd love to hear more about that. Yeah, first of all, I have to credit many, many people.

About four years ago, I was actually finishing my sabbatical from Google time. I was very privileged to work with so many businesses. Enterprise developers, just a large number and variety of vertical industries are realizing AI's human impact. That was one, meaning faculty leaders at Stanford and also just our president, provost, former President and former provost all get together and realize there is a historical role that Stanford needs to play in advances of AI. We were part of the birth place of AI. A lot of work, our previous generation have done and all of work you've done and some of the work I've done lead to today's AI, what is our historical opportunity and responsibility?

With that, we believe that the next generation of AI education and research and policy needs to be human centered. Having established the humans center AI institute, what we call HAI, one of the work that really took me outside of my comfort zone were aiming expertise is really a deeper engagement with policy thinkers and makers. Because we're here in Silicon Valley and there is a culture in Silicon Valley is we just keep making things and the law will catch up by itself. But AI is impacting human lives and sometimes negatively so rapidly that it is not good for any of us. If we, the experts, are not at the table with a policy thinkers and makers to really try to make this technology better for the people.

We're talking about fairness, we're talking about privacy, we also are talking about the brain drain of AI to industry and the concentration of data and compute in a small number of technology companies. All these are really part of the changes of our time. Some are really exciting changes, some have profoundly impact that we cannot necessarily predicting yet. One of the policy work that Stanford AI has very proudly engaged in, is we were one of the leading universities that lobbied a bill called the National AI Research Cloud Task Force Bill. It changed the name from research Cloud to research resource. Now the bill's acronym is NAIR, National AI Research Resource.

This bill is calling for a task force to put together a road-map for America's public sector, especially higher education, and research sector to increase their access to resource for AI compute and AI data and really is aimed to rejuvenate America's ecosystem in AI innovation and research. I'm on the 12-person task-force under Biden administration for this bill, and we hope that's a piece of policy that is not a regulatory policy, it's more an incentive policy to build and rejuvenate ecosystems. I'm glad that you're doing this, The Help Shape US Policy, and making sure enough resources are allocated to ensure healthy development of AI. I feel like this is something that every country needs at this point.

Yeah. Just from the things that you are doing by yourself, not to speak of the things that the Global AI Community is doing, there's just so much going on in AI right now so many opportunities, so much excitement. I found that for someone getting started in machine learning for the first time, sometimes there's so much going on and they can almost feel a little bit overwhelming. Totally. What advice do you have for someone getting started in machine learning? Good question, Andrew, I'm sure you have great advice. You're one of the world-known advocate for AI machine learning education. I do get this question a lot as well, and one thing you're totally right is AI really today feels different from our time.

Just for the record, you all are still on time. That's true when we were starting in AI, I love that exactly we're still part of this. When we first started the entrance to AI and machine learning was relatively narrow. You almost have to start from computer science and go. As a physics major, I still had to wedge myself into the computer science track or electrical engineering track to get to AI. But today, I actually think that there is many aspect of AI that creates entry points for people from all walks of life. On the technical side, I think it's obvious that there's just incredible plethora of resources out there on the Internet from Coursera to YouTube, to TikTok there's just so much that students worldwide can learn about AI and machine learning compared to the time we began learning machine learning.

Also any campuses, we're not talking about just college campuses we're talking about high school campuses, or even sometimes earlier, we're starting to see more available classes and resources. I do encourage those of the young people with a technical interest and resource and opportunity to embrace these resources because it's a lot of fun. But having said that, for those of you who are not coming from a technical angle, who still are passionate about AI, whether it's the downstream application or the creativity it engenders, or the policy and social angle, or important social problems, whether it's digital economics or the governance or history, ethics, political sciences, I do invite you to join us because there is a lot of work to be done.

There's a lot of unknown questions, for example, my colleague at HAI are trying to find answers on how do you define our economy in the digital age? What does it mean when robots, or software, are participating in the workflow more and more? How do you measure our economy? That's not AI coding question that is AI impact question. We're looking at the incredible advances of generative AI and there will be more. What does that mean for creativity, and to the creators from music to art, to writing? I think there is a lot of concerns, and I think it's rightfully so, but in the meantime, it takes people together to figure this out and also to use this new tool.

In short, I just think it's a very exciting time, and anybody with any walks of life, as long as you are passionate about this, there's a role to play. That's really exciting when you talk about economics, think about my conversations with Professor Erik Brynjolfsson. Impact of AI on the economy, but from what you're saying and I agree, it seems like no matter what your current interests are, AI is such a general-purpose technology that the combination of your current interest in AI is often promising. I find that even for learners that may not yet have a specific interest, if you find your way into AI, start learning things, often the interest will evolve and then you can start to craft your own path.

Given way AI is today, there's still so much room and so much need for a lot more people to craft their own path, to do this exciting work that I think the world still needs a lot more of. Totally agree. One piece of work that you did I thought was very cool was starting a program initially called SAILORS and then later AI4ALL, which was really reaching out to high school and even younger students, to try to give them more opportunities in AI, including people of all walks of life. I'd love to hear more about that. This is in the spirit of this conversation is that was back in 2015. There was starting to be a lot of excitement of AI, but there was also starting to be this talk about killer robot coming next door, terminators coming.

At that time Andrew, I was the director of Stanford AI Lab, and I was thinking, we know how far we are from terminators coming and that seemed to be a little bit far-fetched concern. But I was living my work life with a real concern I felt no one was talking about, which was the lack of representation in AI. At that time, I guess after Daphne has left, I was the only woman faculty at Stanford AI Lab. We're having very small, around 15% of women graduate students, and we really don't see anybody from the underrepresented minority groups in Stanford AI program. This is a national or even worldwide issue, so it wasn't just Stanford. Frankly, it still needs a lot of work today.

Exactly. How do we do this? Well, I got together with my former student Ugawa Sakofski, and also a long-term educator of STEM topics Doctor Rick Summer from Stanford Pre-Collegiate Study Program, and thought about inviting high schoolers at that time women, high-school young women to participate in a summer program to inspire them to learn AI, and that was how it started in 2015, and 2017 we got a lot of encouragement and support from people like Jensen and Lori Hung and Melinda Gates, and we formed a national non-profit AI4ALL, which is really committed to training or shaping tomorrow's leaders for AI. From students of all walks of life, especially the traditionally under-served and underrepresented communities.

Till today, we've had many summer camps and summer programs across the country more than 15 universities are involved, and we have online curriculum to encourage students as well as college pathway programs to continue support these students' career by matching them with internships and mentors. It's a continuous effort of encouraging students of all walks of life. I remember back then, I think your group was printing these really cool t-shirts that asked the question. AI will change the world, who will change AI? I thought the answer of making sure everyone can come in and participate, that was a great answer. Still an important question today. That's a great thought and I think that takes us toward the end of the interview.

Any final thoughts for the people watching this? Still, that this is a very nascent field. As you said, Andrew, we're still in the middle of this. I still feel there's just so many questions that I wake up excited to work on with my students in the lab, and I think there's a lot more opportunities for the young people out there who want to learn and contribute and shape tomorrow's AI. Well said [inaudible] that's very inspiring, really great to chat with you, and thank you for attending this video. Thank you. It's fun to have these conversations.